iOS Development

Study guide: Swift, SwiftUI, design, data, device APIs, networking, testing and shipping

Companion to the interactive course page. Every code example is printed with a five-part explanation of each line: who acts, what it does, where it lives, when it runs, and why it is written that way.

Contents

00 Start Here
Welcome to iOS Development · Three Ways Swift Runs Here · Worked Examples

01 The Apple Platform and Its Ecosystem
One Swift Codebase, Many Screens · Xcode and the Developer Toolbox · Anatomy of a SwiftUI App · The App Lifecycle · The Software Development Life Cycle · Kinds of Apps and Extensions · Swift Beyond the iPhone · Worked Examples

02 Swift Fundamentals
Values, Types and Your First Program · Numbers, Operators and Strings · Optionals: Values That Might Be Missing · Arrays, Dictionaries and Sets · Decisions and Loops · Functions · Closures and Higher-Order Functions · Worked Examples

03 Types, Protocols and Programming Paradigms
Structs: Your Own Value Types · Classes and Reference Semantics · Enums: Modeling Choices and State · Protocols and Protocol-Oriented Design · Generics: Write It Once, Keep the Types · Error Handling · Concurrency: async/await and Actors · Programming Paradigms in Swift · Worked Examples

04 SwiftUI and Reactive Interfaces
Describe the Screen, Not the Steps · Views and Modifiers · Stacks, Frames, and Grids · State Drives the Screen · Observable Models · Lists and Dynamic Data · Navigation, Tabs, and Sheets · Forms and Input · Motion, Adaptivity, and Liquid Glass · Worked Examples

05 Designing Great iOS Apps
Principles of iOS Design · Liquid Glass · Typography and Dynamic Type · Color, Dark Mode, and Contrast · Layout, Spacing, and Touch Targets · SF Symbols and App Icons · Choosing a Navigation Pattern · States, Feedback, and Clear Copy · Designing for Accessibility · Process, Prototypes, and Critique · Worked Examples

06 Storing Data: Local Storage and Databases

Where App Data Lives · Settings with @AppStorage · Codable and JSON · Reading and Writing Files · SwiftData Fundamentals · Relationships and Schema Changes · Secrets in the Keychain · iCloud and CloudKit Sync · Worked Examples

07 Device APIs: Location, Maps, Cloud and More
Permission and Privacy · Core Location, the Modern Way · MapKit for SwiftUI · Cloud Storage with CloudKit · Photos, Camera and Sharing · On-Device Intelligence · Beyond the App: Widgets, Intents and Notifications · Worked Examples

08 Networking and Backend Communication
How an App Talks to a Server · Speaking HTTP · REST Resources and JSON · URLSession with async/await · Errors, Retries and Security · Live Data Beyond Request and Response · Designing the API Layer · Worked Examples

09 Architecture, Testing and Shipping
Architecture That Survives Change · Testing with Swift Testing · UI Tests, Previews and Accessibility Checks · Debugging and Instruments · Performance, Accessibility and Localization · From Archive to App Store · Capstone: Study Buddy from Idea to Release · Worked Examples

MODULE 00

Start Here

Learn how this interactive course works, run your first Swift program and your first SwiftUI screen, and meet the three Swift engines inside the page.

- Understand the essentials of Apple development and its ecosystem
- Create various types of iOS applications in Swift

Welcome to iOS Development

CONCEPT

Build real iPhone apps, one line at a time

This course teaches you to design and build apps for iPhone and the rest of Apple's platforms with **Swift**, Apple's programming language, and **SwiftUI**, its framework for building user interfaces. The phone on the right is not a picture. It is a live SwiftUI app running in this page: tap *Start focusing* or change the session length.

By the end you will be able to:

Navigate the ecosystem	Use Xcode, the Simulator, and Apple's platforms, and follow an app from idea to the App Store.
Design reactive interfaces	Build SwiftUI screens whose controls and properties update automatically when data changes.
Write Swift apps	Create different kinds of iOS apps in Swift, a modern, safe, industry-standard language.
Store data	Save data locally and in an on-device database, and sync it through the cloud.
Use device and network APIs	Add location, maps, and cloud storage, and talk to servers over HTTP and other protocols.

Interactive figure in the HTML version: phone preview.

CONCEPT

How this page works

Move through the course with the **Next** and **Previous** buttons or the ← and → keys. The outline on the left lists every module and lesson; your progress is saved in this browser.

Every code example is editable and every line has a note. Click a line (or use ↑ and ↓) and the inspector explains it in five parts:

Who	Which part of the system acts: the compiler, SwiftUI, iOS, your own type, or the person tapping the screen.
What	Exactly what that one line does.
Where	Where it lives or runs: which type or scope, which thread, the device or a server.
When	When it runs: at compile time, at launch, every time a screen redraws, or on a tap.
Why	The reason it is written that way: a design rule, a best practice, or the bug it prevents.

> **Break things on purpose**
>
> Edit any example. Programs re-run and previews redraw as you type. **Reset** restores the original, so experiments are free.

CONCEPT

Five kinds of steps

Every screen in this course is one of five kinds. Each has its own layout, so you always know what to do.

Concept	An explanation, sometimes with an interactive figure beside it, like the phone on the first screen.
Code	An annotated example. The editor sits on the left with its Try it list. The five-part notes sit beside it, with a console under them for programs, or a live phone or a simulation in a third pane.
Compare	Two annotated snippets side by side, A and B, with a verdict naming the lesson.
Widget	A full-screen interactive. Its intro says what to try and what to notice.
Quiz	A self-check that ends every lesson. Pick an answer and the explanation appears at once. A wrong answer gets a Try again button; a right one locks the question. The dots below jump between questions, and your answers stay in this browser.

> **Try it prompts**
>
> Under each code example, **Try it** lists edits to make in the editor. Make them, watch the result, then press **Reset** to restore the original.

CODE · RUNS IN THE BROWSER

Your first Swift program

This program runs right here in your browser. Read the note for each line, then press **Run** or edit a value: the output updates as you type. **Real compiler** sends the same code to Apple's open-source Swift compiler on a server and shows its answer.

```
let course = "iOS Development"
var weeksLeft = 4

weeksLeft -= 1
print("Welcome to \(course)!")
print("Weeks left: \(weeksLeft)")
```

OUTPUT

```
Welcome to iOS Development!
Weeks left: 3
```

Line by line

```
1  let course = "iOS Development"
```

WHO — You, the programmer, declare it; the Swift compiler records its type.
WHAT — **Creates a constant named course that holds the text "iOS Development".**
WHERE — At the top level of the program, so every later line can use it.
WHEN — When the program starts, before any line below it runs.
WHY — let makes a value that never changes. Swift style is to prefer let and switch to var only when a value must change.

```
2  var weeksLeft = 4
```

WHO — You declare it; the compiler infers its type, Int, from the whole number 4.
WHAT — **Creates a variable named weeksLeft with the starting value 4.**
WHERE — At the top level, next to course.
WHEN — Right after line 1 runs.
WHY — var is for values that will change. No type is written because Swift infers Int from the literal.

```
4  weeksLeft -= 1
```

WHO — The Swift runtime, following your instruction.
WHAT — **Subtracts 1 from weeksLeft and stores the result back, so it becomes 3.**
WHERE — It changes the variable declared on line 2.
WHEN — After lines 1 and 2, in order from top to bottom.
WHY — Compound operators like -= update a var in place. Trying this on a let would be a compile error, which is exactly why constants are safer.

```
5  print("Welcome to \(course)!")
```

WHO — The Swift standard library function print writes to the console.
WHAT — **Prints Welcome to iOS Development! by inserting course into the text with \( ).**
WHERE — The output appears in the panel on the right; in a real app it goes to Xcode's debug console.
WHEN — Each time this line runs: once, here.
WHY — String interpolation, \(value), builds text from values without manual conversion or + chains.

```
6  print("Weeks left: \(weeksLeft)")
```

WHO — print, called with the current value of weeksLeft.
WHAT — **Prints Weeks left: 3, because line 4 already subtracted 1.**
WHERE — The console panel beside the editor, on the line right after the welcome message.
WHEN — After line 5. Swift runs top-level statements in order.
WHY — Printing values is the quickest way to check what your program is doing while you learn and debug.

> **Try it**
>
> 1. Change 4 on line 2 to 10 and watch the output update.
> 2. Change let on line 1 to var, then try assigning a new course name on a new line.
> 3. Change var on line 2 to let. Read the error, then change it back.

CODE · LIVE SWIFTUI PREVIEW

Your first SwiftUI screen

This is a complete SwiftUI view. The phone on the right runs it live: tap the button. Then edit the text or the color and watch the screen redraw.

```swift
import SwiftUI

struct GreetingView: View {
    @State private var taps = 0

    var body: some View {
        VStack(spacing: 16) {
            Text("Hello, Swift!")
                .font(.largeTitle)
            Text("Button tapped \(taps) times")
                .foregroundStyle(.secondary)
            Button("Tap me") {
                taps += 1
            }
            .buttonStyle(.borderedProminent)
        }
    }
}

#Preview {
    GreetingView()
}
```

Line by line

1 `import SwiftUI`

WHO	The Swift compiler, while it builds your app.
WHAT	**Makes SwiftUI's types available in this file: View, Text, Button, VStack and @State.**
WHERE	The first line of the file; it applies to this whole file.
WHEN	While the compiler builds this file, before GreetingView or its preview exists.
WHY	SwiftUI is a framework, not part of the language, so each file imports what it uses.

3 `struct GreetingView: View {`

WHO	You define the type; SwiftUI will create and display it.
WHAT	**Declares a structure named GreetingView that conforms to the View protocol.**
WHERE	At the top level of the file, so other views and the preview can use it.
WHEN	The type is checked at compile time; SwiftUI creates instances whenever it needs this screen.
WHY	SwiftUI views are lightweight structs, values that describe UI. Conforming to View means "I can describe something to draw."

4 `@State private var taps = 0`

WHO	SwiftUI owns the storage; this view reads and writes it.
WHAT	**Declares a piece of state named taps that starts at 0.**
WHERE	Inside GreetingView; private means only this view can touch it.
WHEN	Created when the view first appears and kept while it stays on screen.
WHY	@State is the single source of truth for data this view owns. When it changes, SwiftUI redraws the parts of the screen that depend on it. Module 04 (SwiftUI) explains it in full; for now read it as "a value that survives redraws".

```
6  var body: some View {
```

WHO	SwiftUI calls this property; you write what it returns.
WHAT	**Declares body, the description of what this view shows.**
WHERE	Inside GreetingView; every View must provide body.
WHEN	Every time SwiftUI needs to draw the view, including after taps changes.
WHY	Describing the screen as a function of state is what makes SwiftUI reactive: you never update labels by hand.

```
7  VStack(spacing: 16) {
```

WHO	SwiftUI's layout system, which measures the stack and positions its three children.
WHAT	**Stacks its child views vertically with 16 points of space between them.**
WHERE	Inside body: it is the single view that body returns.
WHEN	Every time SwiftUI evaluates GreetingView's body, including right after taps changes.
WHY	Stacks are the basic building blocks of layout. Spacing in multiples of 4 or 8 points keeps screens visually consistent.

```
8  Text("Hello, Swift!")
```

WHO	SwiftUI draws the text with the system font.
WHAT	**Shows the words Hello, Swift!**
WHERE	First child of the VStack, so it appears at the top.
WHEN	Each time body runs, although the words never change, so there is nothing new to draw.
WHY	Text is the simplest view. It uses Dynamic Type by default, so it respects the user's preferred text size.

```
9  .font(.largeTitle)
```

WHO	A modifier, applied by SwiftUI to the Text above.
WHAT	**Uses the Large Title text style (34 points at the default size).**
WHERE	Attached to the Text on line 8.
WHEN	Each time body runs, right after the Text above it is created and before the stack lays it out.
WHY	Semantic text styles such as .largeTitle scale with accessibility settings; fixed point sizes do not.

```
10 Text("Button tapped \(taps) times")
```

WHO	SwiftUI reads taps while building this Text.
WHAT	**Shows how many times the button was tapped, using string interpolation.**
WHERE	Second child of the VStack.
WHEN	Every time body runs, so the number is always current.
WHY	Because this text depends on taps, SwiftUI updates it automatically when taps changes.

```
11 .foregroundStyle(.secondary)
```

WHO	SwiftUI applies a semantic color.
WHAT	**Draws the text in the secondary label color, a softer gray.**
WHERE	Attached to the Text on line 10.
WHEN	Each time body runs; the actual gray is resolved when the text is drawn, so it follows Light or Dark Mode.
WHY	Semantic colors adapt to Light and Dark Mode automatically, so you never pick grays by hand.

```
12 Button("Tap me") {
```

WHO	The person using the app triggers it; SwiftUI runs the closure.
WHAT	**Creates a button titled Tap me whose action is the code in the braces.**
WHERE	Third child of the VStack.
WHEN	The button is created each time body runs; its action runs only on a tap.
WHY	Buttons separate what they look like (the title) from what they do (the action closure).

```
13  taps += 1
```

WHO	Your code, run by SwiftUI when the button is tapped.
WHAT	**Adds 1 to the taps state.**
WHERE	Inside the button's action closure.
WHEN	Once per tap on the button, after the finger lifts, and never while body is being evaluated.
WHY	Changing @State is all you do. SwiftUI notices the change, runs body again, and updates the text on line 10.

```
15  .buttonStyle(.borderedProminent)
```

WHO	SwiftUI styles the button.
WHAT	**Gives the button a filled, tinted capsule, the style for a screen's main action.**
WHERE	Attached to the Button that starts on line 12.
WHEN	Each time body runs, when the Button on line 12 is rebuilt; the style itself never changes.
WHY	A prominent style signals the primary action. Use it for one button per screen so the hierarchy stays clear.

```
20  #Preview {
```

WHO	Xcode (and this page) reads the #Preview macro.
WHAT	**Declares a preview that shows GreetingView on a simulated device.**
WHERE	At the top level of the file, outside the struct.
WHEN	At design time, while you edit; it is not part of the shipped app.
WHY	Previews give instant visual feedback, so you see each change without building and launching the whole app.

```
21  GreetingView()
```

WHO	SwiftUI creates an instance of your view.
WHAT	**Makes a GreetingView to display in the preview.**
WHERE	Inside the #Preview block.
WHEN	Each time the preview refreshes.
WHY	Creating views is cheap because they are structs, so SwiftUI can rebuild them freely.

> **Try it**
>
> 1. Change "Hello, Swift!" to your own name.
> 2. Change .borderedProminent to .bordered, then to .glassProminent (the Liquid Glass style).
> 3. Change taps += 1 to taps += 10.

CHECK YOURSELF

Check your understanding

1. When does a line of SwiftUI code such as `Text("Button tapped \(taps) times")` run?
   - A. Once, when the app launches, and then never again
   - B. Each time SwiftUI runs body, such as after a tap
   - C. Only when the person taps that Text on the screen
   - D. At compile time, when Xcode checks the file
2. In *Your first Swift program*, what would happen if `var weeksLeft = 4` were changed to `let weeksLeft = 4`?
   - A. It prints Weeks left: 4, because the subtraction is skipped
   - B. It prints Weeks left: 3, exactly as before
   - C. The compiler rejects weeksLeft -= 1
   - D. It builds, then crashes at the subtraction

3. Why does the SwiftUI example store the tap count in `@State` instead of a plain `var`?

   A. @State makes adding to the number faster
   B. Changing @State makes SwiftUI redraw views that read it
   C. @State keeps the number on disk between app launches
   D. A plain var inside a View struct cannot hold an Int at all

Answers: Welcome to iOS Development

1. **B. Each time SwiftUI runs body, such as after a tap** body is re-run whenever the state it reads changes, so this Text is rebuilt with the new count. That is the heart of reactive UI. It is not a launch-only line, Text has no tap behavior of its own, and compile time is when code is checked, not run.
2. **C. The compiler rejects weeksLeft -= 1** A let constant can never be changed, and the compiler catches that before the program runs: "cannot assign to value: 'weeksLeft' is a 'let' constant". Nothing is printed and nothing crashes, because the program never builds. Catching mistakes at compile time, instead of crashing later, is one of Swift's safety features.
3. **B. Changing @State makes SwiftUI redraw views that read it** @State is SwiftUI-managed storage. When it changes, SwiftUI re-runs body and updates the screen. It is no faster than a plain var, and it does not persist across app launches (that is @AppStorage or a database, covered in the Data module). A plain var can hold an Int; it just cannot trigger a redraw.

Three Ways Swift Runs Here

CONCEPT

Engine 1: a Swift interpreter in your browser

Browsers cannot run Swift directly. This page carries its own **Swift interpreter**, a program that reads Swift source code and carries it out step by step. It supports the language features this course teaches: types, optionals, collections, functions, closures, structs, classes, enums, protocols, generics, errors, and async/await.

Every console example in the course was run on the real Swift compiler, and the interpreter produces the same output. It also reproduces the compiler errors beginners meet most, word for word.

Step through	Console examples have a Step through tab: move line by line and watch every variable change.
Real compiler	The Real compiler button sends your code to Compiler Explorer (godbolt.org), which compiles it with Swift on a Linux server.

> **Know the limits**
>
> An interpreter is slower than compiled code and covers the language, not every Apple framework. When an example needs a real iPhone (location, maps, cloud), it is marked *Read-along*, explained line by line, and paired with a simulation.

CONCEPT

Engine 2: a live SwiftUI preview

SwiftUI examples render on a simulated iPhone, much like the canvas in Xcode. The preview supports stacks, text, images and SF Symbols, buttons, lists, navigation, tabs, sheets, alerts, forms and controls, animation, and Liquid Glass, along with `@State`, `@Binding`, `@Observable` models, `@Environment`, and `@AppStorage`.

Use the buttons above each phone to switch devices, toggle Dark Mode, change the Dynamic Type text size, and turn on the accessibility inspector, which outlines every tap target and flags any smaller than Apple's 44-point minimum.

> **Test like a designer**
>
> Check every screen in Dark Mode, at a large text size, and on the small iPhone SE. Most layout bugs show up in one of those three.

Interactive figure in the HTML version: phone preview.

CONCEPT

Engine 3: real Swift compiled to WebAssembly

WebAssembly (WASM) is a compact machine-code format that every modern browser can run at near-native speed. Since Swift 6.2, the official toolchain ships a Swift SDK for WebAssembly, so it can compile Swift into WebAssembly.

This page includes a small module of real Swift, compiled ahead of time with that SDK. Its functions power the color-contrast checker in the Design module and the distance calculator in the Location lesson. Race it on the right against JavaScript and against the interpreter.

> **Swift beyond the iPhone**
>
> The same language runs on iPhone, iPad, Mac, Apple Watch, Apple TV, Vision Pro, Linux servers, Windows, and now the web. Learning Swift opens all of them.

Interactive figure in the HTML version: swift wasm race.

CODE · RUNS IN THE BROWSER

The Swift behind the race

This is the prime-counting function that runs as WebAssembly in the race. In the compiled module it also carries two attributes, `@_expose(wasm)` and `@_cdecl`, that export it to JavaScript. Here it runs in the interpreter so you can step through it.

```swift
func countPrimes(_ n: Int) -> Int {
    if n < 2 { return 0 }
    var isPrime = [Bool](repeating: true, count: n + 1)
    isPrime[0] = false
    isPrime[1] = false
    var i = 2
    while i * i <= n {
        if isPrime[i] {
            var multiple = i * i
            while multiple <= n {
                isPrime[multiple] = false
                multiple += i
            }
        }
        i += 1
    }
    return isPrime.filter { $0 }.count
}

print(countPrimes(100))
```

OUTPUT

```
25
```

Line by line

1 `func countPrimes(_ n: Int) -> Int {`

WHO	You define the function; callers (here, the last line) run it.
WHAT	**Declares countPrimes, which takes a whole number n and returns how many primes are at or below it.**
WHERE	At the top level of the program.
WHEN	Defined at compile time; its body runs each time it is called.
WHY	The underscore means callers write countPrimes(100) rather than countPrimes(n: 100), which reads naturally.

```
2  if n < 2 { return 0 }
```

WHO The function itself, guarding against small inputs.
WHAT **Returns 0 right away when n is less than 2, since there are no primes below 2.**
WHERE First line of the function body.
WHEN At the start of every call.
WHY Handling edge cases first keeps the main algorithm simple and prevents an invalid array size below.

```
3  var isPrime = [Bool](repeating: true, count: n + 1)
```

WHO The function allocates the array; Swift manages its memory.
WHAT **Creates an array of n + 1 Booleans, all true, one slot for each number from 0 to n.**
WHERE A local variable inside the function.
WHEN Once per call, after the guard.
WHY The Sieve of Eratosthenes starts by assuming every number is prime, then crosses out the ones that are not.

```
4  isPrime[0] = false
```

WHO The function updates the array.
WHAT **Marks 0 as not prime.**
WHERE Inside the function, changing slot 0 of isPrime.
WHEN Once per call to countPrimes, immediately after the array is created.
WHY By definition 0 and 1 are not prime numbers.

```
5  isPrime[1] = false
```

WHO The function updates the array.
WHAT **Marks 1 as not prime.**
WHERE Inside the function, changing slot 1.
WHEN Once per call, right after slot 0 is cleared on the line above.
WHY Same reason as the line above.

```
6  var i = 2
```

WHO The function keeps a counter.
WHAT **Starts the candidate number i at 2, the first prime.**
WHERE A local variable inside the function.
WHEN Once per call, before the loop.
WHY var because the loop increases i.

```
7  while i * i <= n {
```

WHO The Swift runtime repeats the block while the condition holds.
WHAT **Loops while i squared is at most n.**
WHERE Inside countPrimes, after the array and the counter i are set up; it encloses the crossing-out code.
WHEN Checked before every pass of the loop.
WHY Any composite number up to n has a factor no larger than its square root, so the loop can stop early.

```
8  if isPrime[i] {
```

WHO The loop checks the array.
WHAT **Continues only if i has not been crossed out, meaning i is prime.**
WHERE Inside the while loop.
WHEN Once per value of i.
WHY Only primes need to cross out their multiples; multiples of a composite were already crossed out by its factors.

```
9  var multiple = i * i
```

WHO The function starts a second counter.
WHAT **Begins crossing out at i squared.**
WHERE Inside the if block.
WHEN Each time a prime i is found.
WHY Smaller multiples of i were already crossed out by smaller primes, so starting at i * i saves work.

```
10 while multiple <= n {
```

WHO The runtime repeats the inner block.
WHAT **Walks through the multiples of i up to n.**
WHERE Nested inside the outer loop.
WHEN Repeatedly, for each prime i.
WHY This inner loop does most of the sieve's work.

```
11 isPrime[multiple] = false
```

WHO The function updates the array.
WHAT **Crosses out the current multiple, since it has i as a factor.**
WHERE Inside the inner loop.
WHEN Once for every multiple of the current prime i, from i squared up to n.
WHY A number with a factor other than 1 and itself is not prime.

```
12 multiple += i
```

WHO The function advances the counter.
WHAT **Moves to the next multiple of i.**
WHERE Inside the inner loop.
WHEN After each crossing out.
WHY Adding i each time visits exactly the multiples of i.

```
15 i += 1
```

WHO The function advances the outer counter.
WHAT **Moves on to the next candidate number.**
WHERE At the end of the outer loop body.
WHEN Once per pass of the outer loop.
WHY Without this line the loop would never end. An infinite loop is a common bug, and the interpreter stops one for you.

```
17 return isPrime.filter { $0 }.count
```

WHO The function hands its answer back to the caller.
WHAT **Keeps only the true slots and counts them: the number of primes.**
WHERE The last line of the function.
WHEN Once per call, after the sieve finishes.
WHY filter with the closure { $0 } keeps the elements that are true. Functional tools like filter express "count the trues" in one readable line.

```
20 print(countPrimes(100))
```

WHO Top-level code calls the function and print shows the result.
WHAT **Counts the primes up to 100 and prints 25.**
WHERE At the top level, after the function definition.
WHEN After countPrimes is defined, when the program reaches its only top-level statement.
WHY A small known answer (there are 25 primes below 100) is a quick way to check an algorithm before trusting it with bigger inputs.

Try it

1. Change 100 to 1000. There are 168 primes below 1,000.
2. Open the Step through tab and watch isPrime, i, and multiple change.
3. Delete the line i += 1 and run it to see how the interpreter stops an infinite loop.

CONCEPT

What you need for the rest of the course

Everything in this page runs in a web browser. To build and ship real apps you also need:

A Mac with Xcode	Xcode 26 or later, free from the Mac App Store. It includes Swift, the iOS SDK, the Simulator, Instruments, and the SwiftUI canvas.
An Apple Account	Sign in to Xcode to run apps on your own iPhone for free. Publishing on the App Store requires the paid Apple Developer Program.
Optional: an iPhone	The Simulator covers most work, but sensors, the camera, and real performance need a device.
Optional: Swift Playgrounds	A free iPad and Mac app for experimenting with Swift and building simple SwiftUI apps.

Use both together

Learn a concept here, then rebuild the example in Xcode. Typing it yourself, running it, and fixing your own mistakes is how the knowledge sticks.

CHECK YOURSELF

Check your understanding

1. Which statement about the in-browser Swift interpreter is accurate?
   A. It runs every Apple framework, including Core Location
   B. It reads the Swift source and carries it out step by step
   C. It translates the Swift into JavaScript, then runs that
   D. It runs faster than Swift compiled for a real device
2. What is WebAssembly?
   A. A JavaScript library that lets web pages call Swift code
   B. A binary format browsers run at near-native speed
   C. Apple’s framework for building iOS apps with web technologies
   D. A server-side service that compiles Swift for the web
3. A friend’s SwiftUI screen looks fine on their iPhone 17 Pro. Which checks would most likely reveal a layout bug?
   A. Check it in Light Mode at the default text size on another iPhone 17 Pro
   B. Check it in Dark Mode, at a large text size, and on an iPhone SE
   C. Check it on the newest Pro Max model at the default text size
   D. Take a screenshot and zoom in to inspect the pixel alignment

Answers: Three Ways Swift Runs Here

1. **B. It reads the Swift source and carries it out step by step** An interpreter executes source code directly, step by step, which is why it can show a Step through view. Nothing is translated to JavaScript first. Interpreting is slower than running compiled machine code, as the race shows, and it does not include device frameworks such as Core Location.
2. **B. A binary format browsers run at near-native speed** WebAssembly is a portable binary format, not a library, a framework or a service. The Swift toolchain can compile Swift into it, and the browser runs the result much faster than interpreted code.
3. **B. Check it in Dark Mode, at a large text size, and on an iPhone SE** Color assumptions break in Dark Mode, fixed sizes break at large text sizes, and tight layouts break on small screens. Repeating the same conditions on another large phone, or zooming into a screenshot of them, shows you the same healthy layout. The preview toolbar lets you test all three in seconds.

Worked Examples

WORKED EXAMPLE

Your first program, line by line

Problem.
Your very first program prints a greeting. Step through the frames first: each one adds a single line or a single idea. Then press Play and watch the finished run greet Ada by name and say what she will build.

Frame 1 of 6

The whole program is one line: print shows Hello! in the console. There is nothing else yet, no stored values and no name, which is exactly why it is a good place to start.

```
1    print("Hello!")
```

OUTPUT SO FAR

```
Hello!
```

Frame 2 of 6

A constant named visitor now stores the name Ada above the print. The greeting has not changed yet, because no line uses the new value; storing a value and using it are two separate steps.

```
1  + let visitor = "Ada"
2  +
3  + print("Hello!")
```

OUTPUT SO FAR

```
Hello!
```

Frame 3 of 6

The print line now builds its text from the stored name: interpolation places the value of visitor inside the greeting, so the console shows Hello, Ada! without the name being typed twice.

```
1    let visitor = "Ada"
2
3  + print("Hello, \(visitor)!")
```

OUTPUT SO FAR

```
Hello, Ada!
```

Frame 4 of 6

A second constant stores the course name, iOS Development, ready for the line that is coming. The output still shows only the greeting, because storing a value never prints anything by itself.

```
  let visitor = "Ada"
+ let course = "iOS Development"
+
+ print("Hello, \(visitor)!")
```

OUTPUT SO FAR

```
Hello, Ada!
```

Frame 5 of 6

A third constant stores what the course will build: your first iPhone app. All three values the finished program needs are now in place, and the greeting line still works exactly as before.

```
  let visitor = "Ada"
  let course = "iOS Development"
+ let project = "your first iPhone app"
+
+ print("Hello, \(visitor)!")
```

OUTPUT SO FAR

```
Hello, Ada!
```

Frame 6 of 6

A second print line joins the program and uses the two stored values at once. The finished run prints both lines in order: the greeting first, then what Ada will build in this course.

```
  let visitor = "Ada"
  let course = "iOS Development"
  let project = "your first iPhone app"

  print("Hello, \(visitor)!")
+ print("In \(course), you will build \(project).")
```

OUTPUT SO FAR

```
Hello, Ada!
In iOS Development, you will build your first iPhone app.
```

FINAL RESULT — CHECKED OUTPUT

```
Hello, Ada!
In iOS Development, you will build your first iPhone app.
```

Line by line: the finished program

```
1  let visitor = "Ada"
```

WHO — You, the programmer, declare it; the Swift compiler records its type as String.
WHAT — **Creates a constant named visitor that holds the text Ada, the name of the person being greeted.**
WHERE — At the top level of the program, above the print lines, so every later line can use it.
WHEN — When the program starts, before any line below it runs.
WHY — let makes a value that never changes. Storing the name in a constant means the greeting lines never repeat the name itself.

```
2  let course = "iOS Development"
```

WHO — You declare it; the compiler infers its type, String, from the quoted text.
WHAT — **Creates a constant named course that holds the text iOS Development, the name of this course.**
WHERE — At the top level, next to visitor, with the other values the program will use.
WHEN — When the program starts, after visitor and before the greeting is printed.
WHY — The second printed line will need this name, so it is stored now, before that line exists, ready to be interpolated.

```
3  let project = "your first iPhone app"
```

WHO — You declare it; the compiler infers String from the quoted text.
WHAT — **Creates a constant named project that holds the text your first iPhone app.**
WHERE — At the top level, beside visitor and course, the last of the three stored values.
WHEN — When the program starts, after course and before the greeting is printed.
WHY — Storing what the course builds in its own constant keeps the final line short and makes the promise of the course one value.

```
5  print("Hello, \(visitor)!")
```

WHO — The print function, with Swift string interpolation filling in the visitor value.
WHAT — **Prints Hello, Ada! by placing the value of visitor inside the text where the interpolation marker sits.**
WHERE — Top-level code; its output appears in the console panel beside the editor.
WHEN — After the visitor constant exists, when the program reaches this line.
WHY — Interpolation builds text from values, so changing the stored name changes the greeting without touching this line.

```
6  print("In \(course), you will build \(project).")
```

WHO — The print function, with interpolation filling in both course and project.
WHAT — **Prints In iOS Development, you will build your first iPhone app. as the second line of output.**
WHERE — Top-level code, directly under the greeting line, so its output lands on the next console line.
WHEN — Last, after the greeting, when all three constants hold their values.
WHY — A greeting alone only says hello. This line says what happens next, and it is built entirely from the stored values.

Try it

The starter stores the name Sam, then prints a plain Hello! that ignores it. Change the print line so it greets the stored name by interpolation, and the program prints Hello, Sam!

```
1  let name = "Sam"
2  print("Hello!")
```

EXPECTED OUTPUT

```
Hello, Sam!
```

WORKED EXAMPLE

Change the greeting, then make it yours

Problem.
Make a greeting screen you can change. Start with one line of text, style it, add a picture above it, then store the greeting in state so a button can swap in a brand-new greeting.

Frame 1 of 6

The screen starts as a single line of text: the greeting, drawn by a Text view inside body. A preview at the bottom shows the struct, and every later frame adds one idea to this view.

```
import SwiftUI

struct WelcomeView: View {
    var body: some View {
        Text("Hello, Ada!")
    }
}

#Preview {
    WelcomeView()
}
```

Frame 2 of 6

A font modifier goes under the text and makes the greeting a large title. Modifiers wrap the view written above them, so this one changes only the greeting and nothing else on the screen.

```
import SwiftUI

struct WelcomeView: View {
    var body: some View {
        Text("Hello, Ada!")
+           .font(.largeTitle)
+    }
+ }
+
+ #Preview {
+    WelcomeView()
+ }
```

Frame 3 of 6

A second modifier joins the chain and colors the greeting blue. The text is now big and colored, the two halves of emphasis, while the words themselves have not changed at all.

```
  import SwiftUI

  struct WelcomeView: View {
      var body: some View {
          Text("Hello, Ada!")
              .font(.largeTitle)
+             .foregroundStyle(.blue)
+     }
+ }
+
+ #Preview {
+     WelcomeView()
+ }
```

Frame 4 of 6

A VStack now wraps the screen, and an Image showing a sun from SF Symbols takes the top spot. The greeting keeps its size and color, and the stack places the picture above the words.

```
  import SwiftUI

  struct WelcomeView: View {
      var body: some View {
+         VStack(spacing: 12) {
+             Image(systemName: "sun.max.fill")
+                 .font(.largeTitle)
+             Text("Hello, Ada!")
+                 .font(.largeTitle)
+                 .foregroundStyle(.blue)
+         }
+     }
  }
+
+ #Preview {
+     WelcomeView()
+ }
```

Frame 5 of 6

The greeting text moves into @State, and the Text view now shows that state instead of fixed words. The screen looks identical, but the words now live in one stored value the screen reads.

```
  import SwiftUI

  struct WelcomeView: View {
+     @State private var greeting = "Hello, Ada!"
+     var body: some View {
+         VStack(spacing: 12) {
+             Image(systemName: "sun.max.fill")
+                 .font(.largeTitle)
+             Text(greeting)
+                 .font(.largeTitle)
+                 .foregroundStyle(.blue)
+         }
+     }
+ }
+
+ #Preview {
+     WelcomeView()
+ }
```

Frame 6 of 6

A Button joins the bottom of the stack. Its action replaces the greeting state with Welcome back, Ada!, and because the Text reads that same state, tapping Surprise me redraws the greeting on its own.

```
  import SwiftUI

  struct WelcomeView: View {
      @State private var greeting = "Hello, Ada!"
      var body: some View {
          VStack(spacing: 12) {
              Image(systemName: "sun.max.fill")
                  .font(.largeTitle)
              Text(greeting)
                  .font(.largeTitle)
                  .foregroundStyle(.blue)
+             Button("Surprise me") {
+                 greeting = "Welcome back, Ada!"
+             }
+         }
+     }
+ }
+
+ #Preview {
+     WelcomeView()
+ }
```

FINAL RESULT — CHECKED ON SCREEN

```
Tap “Surprise me” and the screen shows “Welcome back, Ada!”.
```

Line by line: the finished program

```
1  import SwiftUI
```

WHO The Swift compiler, which loads the SwiftUI framework for this file.
WHAT **Makes View, Text, VStack, Image, Button and @State available in this file.**
WHERE The first line of the file; an import covers only its own file.
WHEN At compile time, before any of the code below is checked.
WHY SwiftUI is a separate framework, so every file that describes a screen imports it.

```
3  struct WelcomeView: View {
```

WHO You declare it; SwiftUI creates values of it whenever the greeting screen is shown.
WHAT **Declares WelcomeView, a struct that conforms to View and describes the greeting screen.**
WHERE At the top level of the file, so the preview at the bottom can create it.
WHEN Compiled once; a fresh value is created whenever a parent or the preview asks for the screen.
WHY A screen in SwiftUI is a value that describes an interface, which is why the whole example is one struct.

```
4  @State private var greeting = "Hello, Ada!"
```

WHO SwiftUI, which stores the value outside the struct, tied to this screen while it is shown.
WHAT **Declares greeting, the text the screen shows, starting as Hello, Ada!; @State makes SwiftUI keep and watch it.**
WHERE Inside WelcomeView, marked private because only this screen changes it.
WHEN Created when the screen first appears and kept across every redraw.
WHY The text below is read from this one value, so when the button changes it, the screen and the data can never disagree.

```
5  var body: some View {
```

WHO SwiftUI calls this property; your code never calls body directly.
WHAT **Returns the description of the greeting screen for the current greeting text.**
WHERE A computed property inside WelcomeView.
WHEN When the screen appears, and again after every change to the greeting state.
WHY Describing the screen from the data is the whole SwiftUI idea: change the greeting and the screen follows.

```
6  VStack(spacing: 12) {
```

WHO SwiftUI layout system, which measures the children and stacks them.
WHAT **Stacks the picture and the greeting text vertically, with 12 points of space between them.**
WHERE The root view returned by body, wrapping everything the screen shows.
WHEN Measured and placed again every time body produces a new description.
WHY A stack states the arrangement once, so the picture sits above the words without any coordinates to manage.

```
7  Image(systemName: "sun.max.fill")
```

WHO SwiftUI, which looks the name up in the SF Symbols library built into the system.
WHAT **Shows the sun.max.fill symbol, a filled sun, as a picture above the greeting.**
WHERE The first child of the VStack, so it sits at the top of the screen.
WHEN Drawn every time body runs; the symbol is a template picture the system can size and recolor.
WHY A symbol gives the screen a picture with no image files to manage, matching the friendly tone of a greeting.

```
8  .font(.largeTitle)
```

WHO The font modifier, applied to the Image view directly above it.
WHAT **Sets the picture size with the largeTitle text style, so the sun renders big.**
WHERE Chained under the Image; modifiers apply to the view written before them.
WHEN Read while body builds the description, before layout measures the picture.
WHY SF Symbols are drawn like a font, so a text style sizes them, and the size follows the text-size setting of the reader.

9 `Text(greeting)`

- **WHO** SwiftUI, which reads the greeting state while building this Text view.
- **WHAT** **Shows whatever the greeting state currently holds, starting with Hello, Ada!, with no words written in this line at all.**
- **WHERE** A child of the VStack, under the picture, in the same position the fixed text occupied.
- **WHEN** Rebuilt after every change to greeting, so it always shows the current value.
- **WHY** Reading the state instead of a fixed string is what lets one button press change the words on screen.

10 `.font(.largeTitle)`

- **WHO** The font modifier, applied to the Text view directly above it.
- **WHAT** **Sets the greeting in the largeTitle text style, the biggest standard style, so it reads as the headline of the screen.**
- **WHERE** Chained under the greeting Text; it affects that text and nothing else.
- **WHEN** Applied while the description is built, before the text is measured.
- **WHY** A greeting is the point of this screen, so it gets the loudest type on it; styles also scale with accessibility settings.

11 `.foregroundStyle(.blue)`

- **WHO** The foregroundStyle modifier, applied to the Text view above it in the chain.
- **WHAT** **Colors the greeting blue, so the headline stands apart from plain body text.**
- **WHERE** Chained under the font modifier on the greeting Text.
- **WHEN** Applied while the description is built, each time body runs.
- **WHY** Color is the second half of emphasis: size said the greeting matters most, and color now makes it the first thing seen.

12 `Button("Surprise me") {`

- **WHO** You declare it; the person using the app triggers its action by tapping.
- **WHAT** **Creates the Surprise me button; the closure after the title is its action, the code that runs on a tap.**
- **WHERE** The last child of the VStack, under the greeting text.
- **WHEN** The button is created each time body runs; its action runs only at the moment of a tap.
- **WHY** The tap check for this example presses exactly this button, so its title is the promise the screen makes.

13 `greeting = "Welcome back, Ada!"`

- **WHO** Your code, run by SwiftUI when the button is tapped.
- **WHAT** **Replaces the greeting state with the new text Welcome back, Ada!, storing it in the same single place the Text reads.**
- **WHERE** Inside the button action closure, the only line in this screen that changes data.
- **WHEN** Once per tap, at the moment of the tap, and never while body is being built.
- **WHY** Changing the state is the whole action: SwiftUI notices, runs body again, and the greeting text updates by itself.

19 `#Preview {`

- **WHO** Xcode and this page, which render the screen without running a whole app.
- **WHAT** **Starts a preview block that shows WelcomeView while you work on it.**
- **WHERE** At the bottom of the file, outside the struct.
- **WHEN** Whenever the file is opened or edited; previews are not part of a shipped app.
- **WHY** Seeing the screen while you build it is how each frame of this example was checked.

20 `WelcomeView()`

- **WHO** The preview renderer, which creates the view value to display.
- **WHAT** **Creates one WelcomeView for the preview to show.**
- **WHERE** The only line inside the preview block.
- **WHEN** Each time the preview refreshes.
- **WHY** The preview needs a value to render; this is the same creation a real app screen would do.

Try it

Back in plain console Swift, the starter stores the name Ada and prints a greeting that ignores it. Change the print line so it uses the stored name and prints Hello, Ada! Ready to build?

```
let name = "Ada"
print("Hello!")
```

EXPECTED OUTPUT

```
Hello, Ada! Ready to build?
```

MODULE 01

The Apple Platform and Its Ecosystem

Know the Apple platforms and their screens, find your way around Xcode, explain how an app is built, launched and suspended, apply the development life cycle to an iOS app, and choose the right kind of app or extension for an idea.

- Understand the essentials of Apple development and its ecosystem
- Create various types of iOS applications in Swift

One Swift Codebase, Many Screens

CONCEPT

Six platforms, one family

Apple makes six software **platforms**. A platform is an operating system plus the frameworks Apple ships with it. Each one runs on its own kind of device, yet all of them share the same core: the same kernel underneath, the same Swift language, and the same SwiftUI framework for building interfaces.

When you build "an iOS app," you build for the iOS platform. Most of what you learn here carries straight over to the others.

iOS and iPadOS	iPhone and iPad. The largest audience, and the focus of this course.
macOS	Mac. Resizable windows, menus, keyboard and pointer.
watchOS	Apple Watch. Glanceable information, used a few seconds at a time.
tvOS	Apple TV. Viewed from across the room and driven by a remote.
visionOS	Apple Vision Pro. Windows and 3D content placed in the room around you.

> **One version number for everything**
>
> Since 2025 every Apple platform shares a version number named for the year ahead: iOS 26, iPadOS 26, macOS Tahoe 26, watchOS 26, tvOS 26 and visionOS 26, all built with Xcode 26. Before that, iOS was at 18 while watchOS was at 11.

CONCEPT

Write once, then adapt

With SwiftUI you describe your interface once, and the framework draws each control in the native style of the platform it runs on. A `Toggle` is a switch on iPhone and a checkbox on Mac. A list in the first column of a `NavigationSplitView` becomes a sidebar on iPad and Mac, and collapses into a list you drill into on iPhone.

Sharing code is not the same as sharing a design. Each device has a different **input method** (finger, pointer, Digital Crown, remote, eyes and hands) and sits at a different distance from the eye. Good apps keep the logic shared and adjust layout and controls for each device.

Multiplatform target	One Xcode target can build for several platforms, for example iPhone, iPad, Mac and Vision Pro, from the same Swift files.
Adaptive controls	SwiftUI chooses the native look and behavior of each control for the current platform.
Platform checks	Code between #if os(watchOS) and #endif is compiled only when building for that platform.

CONCEPT

Points, not pixels

A **pixel** is one physical dot of light on a display. Screens differ in how many pixels they pack into an inch, so a layout measured in pixels would make a button tiny on a sharp screen and huge on an older one.

Instead, Apple platforms measure layout in **points**: an abstract unit that stays roughly the same physical size on every device. The **scale factor** says how many pixels make up one point along each side.

@1x	1 point is 1 pixel. Older, non-Retina screens and Apple TV in HD.
@2x	1 point is 2 x 2 pixels. iPad, Mac Retina displays, Apple Watch, iPhone SE.
@3x	1 point is 3 x 3 pixels. Most current iPhones.

> **The 44-point rule**
>
> Apple's Human Interface Guidelines recommend tap targets of at least 44 x 44 points. That is 88 x 88 pixels at @2x and 132 x 132 pixels at @3x, yet it feels about the same size under your finger on every iPhone.

INTERACTIVE

Explore the devices

Pick a device to see its screen size in points, its scale factor and the pixels behind it, plus how people hold it and navigate it. Switch to **Compare at true scale** to line the touch devices up side by side: the orange square is 44 x 44 points on every one. Then use the converter to turn a size in points into the pixel sizes your image assets need.

This step is an interactive simulation in the HTML version.

CONCEPT

Size classes: design for space, not for models

New devices arrive every year, so code that asks "is this an iPhone 17 Pro?" breaks quickly. Instead, iOS describes the space your view has with two **size classes**, one for width and one for height. Each is either *compact* (constrained) or *regular* (roomy).

iPhone, portrait	Compact width, regular height.
iPhone, landscape	Compact height. Width is compact on most models and regular on the largest ones, such as the Pro Max.
iPad, full screen	Regular width and regular height, in both orientations.
iPad, narrow window	Compact width when the user makes your window narrow, so an iPad app may need its iPhone layout.

> **Reading it in SwiftUI**
>
> `@Environment(\.horizontalSizeClass)` gives a view the current width class. SwiftUI updates it, and redraws your view, whenever the user rotates the device or resizes a window. Module 04 (SwiftUI) explains `@Environment` in full; for now read it as a value SwiftUI hands to your view.

CODE · READ-ALONG

A layout that follows the size class

This view stacks its icon above its title when width is compact, and places them side by side when width is regular. It never mentions a device model, so it adapts to a narrow iPad window as well as to a phone. Size classes come from the environment, which the browser preview does not model, so this example is explained rather than run.

```swift
import SwiftUI

struct WelcomeView: View {
    @Environment(\.horizontalSizeClass) private var widthClass

    var body: some View {
        let layout = widthClass == .compact
            ? AnyLayout(VStackLayout(spacing: 16))
            : AnyLayout(HStackLayout(spacing: 32))

        layout {
            Image(systemName: "swift")
                .font(.system(size: 80))
                .foregroundStyle(.orange)
            Text("One codebase, every screen")
                .font(.title)
        }
        .padding()
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the app.
- **WHAT** **Makes SwiftUI available in this file, including View, AnyLayout, the stack layouts and the environment values.**
- **WHERE** At the top of the file; an import applies only to the file it appears in.
- **WHEN** While the compiler builds WelcomeView's file, before any of the views below exist.
- **WHY** Frameworks are separate modules, so each file names the ones it uses; that keeps name lookup and build times predictable.

3 `struct WelcomeView: View {`

- **WHO** Your code declares it; SwiftUI creates and re-creates instances of it.
- **WHAT** **Declares a value type named WelcomeView that conforms to the View protocol, so SwiftUI knows how to draw it.**
- **WHERE** A top-level type in this file, visible to the rest of the app.
- **WHEN** The type exists from compile time; SwiftUI makes a new instance whenever a parent view rebuilds.
- **WHY** Views are small structs that are cheap to create and discard, which lets SwiftUI rebuild them freely.

```
4  @Environment(\.horizontalSizeClass) private var widthClass
```

WHO — SwiftUI's environment, which passes values down the view tree.

WHAT — **Reads the current width class, .compact or .regular, into a property named widthClass.**

WHERE — A property of WelcomeView; the value comes from the window that contains the view.

WHEN — Read each time body runs; SwiftUI re-runs body when the class changes, such as on rotation or a window resize.

WHY — Asking how much space there is, instead of which device this is, keeps the layout right on devices that do not exist yet. Module 04 (SwiftUI) explains @Environment in full; for now read it as "ask SwiftUI for a value it shares with every view".

```
6  var body: some View {
```

WHO — SwiftUI calls this computed property; you never call it yourself.

WHAT — **Returns the content of the view; some View means one specific view type that the compiler works out for you.**

WHERE — The one requirement of the View protocol, inside WelcomeView.

WHEN — When the view first appears and again whenever a value it reads, here widthClass, changes.

WHY — An opaque some View type tells SwiftUI the exact shape of your UI at compile time, which makes updates fast.

```
7  let layout = widthClass == .compact
```

WHO — Your code, running inside body.

WHAT — **Starts a constant named layout whose value depends on whether the width class is compact.**

WHERE — A local constant inside body; it exists only while body runs.

WHEN — Each time SwiftUI evaluates body.

WHY — Making the choice in one place keeps the views below it identical in both cases.

```
8  ? AnyLayout(VStackLayout(spacing: 16))
```

WHO — The ternary operator picks this branch; SwiftUI's layout system uses the result.

WHAT — **For a compact width, chooses a vertical stack layout with 16 points between items, wrapped in AnyLayout.**

WHERE — The true branch of the expression that began on the line above.

WHEN — During body evaluation, when widthClass is .compact.

WHY — AnyLayout erases the concrete layout type so both branches of the ternary have the same type and the code compiles.

```
9  : AnyLayout(HStackLayout(spacing: 32))
```

WHO — The ternary operator, choosing its false branch.

WHAT — **For a regular width, chooses a horizontal stack layout with a wider 32-point gap.**

WHERE — The false branch of the same expression.

WHEN — During body evaluation, whenever the width class is not compact.

WHY — A full-screen iPad has room for content side by side, so the icon and title sit in a row.

```
11 layout {
```

WHO — SwiftUI's layout engine, running whichever AnyLayout the ternary chose.

WHAT — **Uses the chosen layout as a container and places the views between its braces.**

WHERE — The root of this view's content, inside body.

WHEN — Every time body runs; the actual positions are computed in SwiftUI's layout pass just after.

WHY — Only the layout changes while the children stay the same, so SwiftUI can animate from column to row and keep any state the children hold.

```
12  Image(systemName: "swift")
```

WHO — SwiftUI, drawing from Apple's SF Symbols icon library.
WHAT — **Shows the Swift bird symbol as a vector image.**
WHERE — The first child of the layout container.
WHEN — Created during body and drawn in the next frame.
WHY — SF Symbols are vector icons that scale cleanly at any size and on any scale factor, so you rarely need icon image files.

```
13  .font(.system(size: 80))
```

WHO — A SwiftUI view modifier, applied to the Swift symbol Image.
WHAT — **Sizes the symbol by giving it an 80-point system font.**
WHERE — Applies to the Image on the line above, and only to it.
WHEN — Applied each time body builds the Image.
WHY — Symbols are sized like text, so you control them with font rather than a frame.

```
14  .foregroundStyle(.orange)
```

WHO — A SwiftUI view modifier on the already-sized Image.
WHAT — **Colors the symbol with the system orange.**
WHERE — Wraps the already-sized Image.
WHEN — Each time body runs, after .font sizes the symbol; the color is resolved when the frame is drawn.
WHY — foregroundStyle is the modern replacement for foregroundColor; it also accepts gradients and semantic styles like .secondary.

```
15  Text("One codebase, every screen")
```

WHO — SwiftUI's Text view, which lays out and draws the title string.
WHAT — **Shows the title string as a label.**
WHERE — The second child of the layout container, after the icon.
WHEN — Created during body; drawn in the next frame.
WHY — Text handles fonts, wrapping, right-to-left languages and VoiceOver for you.

```
16  .font(.title)
```

WHO — A SwiftUI view modifier on the title Text.
WHAT — **Uses the built-in title text style.**
WHERE — Applies to the Text above it.
WHEN — Applied each time body runs; the actual size is resolved from the user's settings.
WHY — Text styles grow and shrink with the user's Dynamic Type setting, unlike a fixed point size.

```
18  .padding()
```

WHO — A SwiftUI view modifier wrapping the whole layout container.
WHAT — **Adds the standard amount of space around the whole layout.**
WHERE — Wraps the layout container, so it pads the icon and title together.
WHEN — Each time body runs, after the layout container and both children are built.
WHY — Default padding uses a platform-appropriate amount, so it looks right on iPhone, iPad and Mac without hard-coded numbers.

> **Try it**
>
> 1. In Xcode, preview this view on an iPhone and on an iPad. Which device gets the row, and why?
> 2. Which line would you change to make the compact layout use 24 points of spacing?

CODE · LIVE SWIFTUI PREVIEW

Let SwiftUI pick what fits

Sometimes the right layout depends on the content, not on the size class. `ViewThatFits` tries each child in order and shows the first one that fits in the space available. Edit the title in the phone preview and watch the layout switch between a row and a column.

```swift
import SwiftUI

struct DeviceBadge: View {
    var title = "Runs on iPhone, iPad and Mac"

    var body: some View {
        ViewThatFits {
            Label(title, systemImage: "swift")
            VStack(spacing: 8) {
                Image(systemName: "swift")
                Text(title)
            }
        }
        .font(.title)
        .padding()
    }
}

#Preview {
    DeviceBadge()
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds DeviceBadge's file.
- **WHAT** **Brings in SwiftUI so the file can use ViewThatFits, HStack, VStack, Image and Text.**
- **WHERE** The first line of the file; it covers this file only.
- **WHEN** At compile time, before DeviceBadge or its preview exists.
- **WHY** Every Swift file states the frameworks it depends on.

3 `struct DeviceBadge: View {`

- **WHO** Your code defines it; SwiftUI draws it.
- **WHAT** **Declares a view type named DeviceBadge.**
- **WHERE** A top-level struct in this file.
- **WHEN** Defined at compile time; instantiated by the #Preview block below.
- **WHY** Small, named views are easier to read, preview and reuse than one giant body.

4 `var title = "Runs on iPhone, iPad and Mac"`

- **WHO** Your struct, storing the text it displays.
- **WHAT** **Declares the badge text with a default value that callers may override.**
- **WHERE** A stored property of DeviceBadge, kept inside each instance.
- **WHEN** Set when SwiftUI creates the view; read every time body runs.
- **WHY** A var with a default appears as an optional argument in the generated initializer, so DeviceBadge() and DeviceBadge(title:) both work.

```
6  var body: some View {
```

WHO SwiftUI calls it; you only write what it returns.
WHAT **Describes what DeviceBadge looks like.**
WHERE The required property of the View protocol.
WHEN On first appearance and whenever the view's inputs change.
WHY Returning some View lets the compiler keep the full structure of the UI as one precise type.

```
7  ViewThatFits {
```

WHO SwiftUI's layout system measures each candidate child.
WHAT **Offers two alternative layouts and shows the first one whose ideal size fits the space available.**
WHERE The root of body; its children are the candidates, in order of preference.
WHEN During layout, and again whenever the available space or the content changes.
WHY It adapts to the real content, such as a long translation, instead of a guessed screen width.

```
8  Label(title, systemImage: "swift")
```

WHO SwiftUI's Label view, which pairs a title with an SF Symbol.
WHAT **The first choice: the Swift icon and the title side by side on one line.**
WHERE The first candidate inside ViewThatFits.
WHEN Measured first during every layout pass.
WHY A label in a row is the most compact design, so it is preferred whenever its one-line width fits.

```
9  VStack(spacing: 8) {
```

WHO SwiftUI's VStack layout, which stacks the icon above the title.
WHAT **The fallback: icon above title, 8 points apart.**
WHERE The second and last candidate inside ViewThatFits.
WHEN Measured only if the row does not fit; used as-is if nothing fits.
WHY ViewThatFits always shows its last child when no option fits, so the most flexible layout goes last.

```
10 Image(systemName: "swift")
```

WHO SwiftUI, with the SF Symbols library.
WHAT **Shows the Swift symbol on top.**
WHERE First child of the VStack.
WHEN Created each time body runs, but measured only if ViewThatFits rejects the one-line Label.
WHY Repeating the icon in each candidate keeps each layout complete on its own.

```
11 Text(title)
```

WHO SwiftUI's Text view, drawing the same title string the Label used.
WHAT **Shows the title under the icon, wrapping onto more lines if needed.**
WHERE Second child of the VStack.
WHEN Created each time body runs; drawn only when the column layout is the one shown.
WHY In a column the text gets the full width, so long titles still read well.

```
14 .font(.title)
```

WHO A SwiftUI modifier that passes a value down the environment.
WHAT **Gives every text and symbol inside the badge the title text style.**
WHERE Applied to ViewThatFits, so both candidates inherit it.
WHEN Applied each time body runs, before ViewThatFits measures.
WHY Setting the font once on a container avoids repeating it on every child.

15 `.padding()`

WHO A SwiftUI modifier that wraps the entire ViewThatFits.
WHAT **Adds standard space around the badge.**
WHERE Wraps the whole ViewThatFits.
WHEN Each time body runs, before ViewThatFits measures its candidates against the remaining space.
WHY Padding also shrinks the space ViewThatFits can use, which affects which candidate fits.

19 `#Preview {`

WHO Xcode's preview canvas, and this page's phone preview.
WHAT **Declares a preview that renders a DeviceBadge.**
WHERE A top-level macro in the file, outside any type.
WHEN Whenever the canvas refreshes after you edit the file.
WHY Previews let you see a view without launching the whole app.

20 `DeviceBadge()`

WHO The preview, creating your view.
WHAT **Makes a DeviceBadge using the default title.**
WHERE Inside the #Preview closure.
WHEN Each time the preview renders.
WHY Using the default keeps the preview identical to what the app would show.

> **Try it**
>
> 1. Change the title to "Runs everywhere". Which layout does the phone show now?
> 2. Change .font(.title) to .font(.largeTitle). Does the row still fit?

CHECK YOURSELF

Check your understanding

1. A designer gives you an icon that should appear at 30 x 30 points. Which image sizes do you add to the asset catalog for @2x and @3x screens?
   A. 30 x 30 for both @2x and @3x
   B. 60 x 60 for @2x and 90 x 90 for @3x
   C. 15 x 15 for @2x and 10 x 10 for @3x
   D. 120 x 120 for @2x and 180 x 180 for @3x
2. A button measures 132 pixels wide in a screenshot from an iPhone 17 Pro, a @3x device. How wide is it in your SwiftUI code?
   A. 132 points
   B. 66 points
   C. 44 points
   D. 396 points
3. Your iPad app shows a sidebar beside its content. A user drags your app into a narrow window next to another app. What should your layout code check to decide whether the sidebar still fits?
   A. The device model name
   B. The screen size in pixels
   C. Whether the device is an iPad
   D. The horizontal size class

4. Which statement about SwiftUI across Apple platforms is accurate?
   A. Every view must be rewritten by hand for each platform the app supports
   B. One set of views adapts per platform, but layout still needs tuning
   C. SwiftUI apps look and behave identically on every Apple platform, by design
   D. SwiftUI runs only on iPhone and iPad; the Mac and Watch need other frameworks

5. A `ViewThatFits` holds an `HStack` first and a `VStack` second, and neither fits the available width. What does SwiftUI show?
   A. Nothing; the view renders empty
   B. The HStack, clipped at the trailing edge
   C. The VStack, because the last child is the fallback
   D. A runtime error, because no candidate fits the width

Answers: One Swift Codebase, Many Screens

1. **B. 60 x 60 for @2x and 90 x 90 for @3x** Multiply the size in points by the scale factor: 30 x 2 = 60 and 30 x 3 = 90. Supplying 30 x 30 pixels for both would look blurry, because the system would have to stretch the image to fill twice or three times as many pixels, and 120 and 180 would be four and six times too large.
2. **C. 44 points** Divide pixels by the scale factor: 132 / 3 = 44 points, the minimum recommended tap target. 66 points would be the answer on a @2x screen; you always need the scale factor of the device the screenshot came from.
3. **D. The horizontal size class** The narrow window gets a compact horizontal size class even though the device is still an iPad, so checking "is this an iPad" would keep a sidebar that no longer fits. Size classes describe the space your view actually has.
4. **B. One set of views adapts per platform, but layout still needs tuning** SwiftUI gives each control its native look per platform, so much of the code is shared and nothing has to be rewritten by hand. It does not make a design right for a watch face and a TV at the same time; that part is still your job.
5. **C. The VStack, because the last child is the fallback** When no candidate fits, ViewThatFits uses its last child. That is why the most flexible layout goes last. It never shows an empty view or crashes because of size.

Xcode and the Developer Toolbox

CONCEPT

Xcode: one workshop for every Apple app

Xcode is Apple's integrated development environment (IDE): one app that holds the code editor, the Swift compiler, a live preview canvas, a debugger, the Simulator, and the tools that package an app for the App Store. It is free, and it runs only on a Mac.

Each Xcode release bundles **SDKs** (software development kits): the frameworks for one platform version. Xcode 26 ships the iOS 26 SDK, which is what lets your code call iOS 26 APIs.

Project	The .xcodeproj file that lists your source files, targets and build settings.
Target	One product to build: the app itself, its tests, or an extension such as a widget.
Scheme	Which target to build, and how to run, test, profile or archive it.
Run destination	Where the build runs: a simulated device or a real one plugged in or on the same network.

INTERACTIVE

Tour the Xcode window

This is a working model of an Xcode 26 window. Click any region, or press **Next** to tour the regions in order. Then try the real interactions: pick a different file in the navigator, change the font in the inspector and watch both the code and the canvas update, and press **Run** to see the console come alive.

This step is an interactive simulation in the HTML version.

CONCEPT

Simulator or a real device?

The **Simulator** is a Mac app that runs your iOS app as a Mac process, compiled for a simulated iPhone or iPad. It starts quickly, offers every screen size, and can fake a location, switch to dark appearance or change the text size in a click.

A **real device** runs your app on actual hardware, with the real camera, sensors, memory limits, battery and network. Everything you ship should be tried on one.

Simulator limits	No camera, and speed and memory come from your Mac, not from a phone.
Free provisioning	A free Apple Account can run apps on your own iPhone; the signing profile expires after 7 days.
Developer Mode	An iPhone needs Developer Mode turned on, in Settings > Privacy & Security, before Xcode can run apps on it.

> **Fast in the Simulator is not fast on a phone**
>
> The Simulator borrows your Mac's processor and memory, which are far larger than a phone's. Measure performance on a real device with Instruments.

INTERACTIVE

What happens when you press Run

Press **Run** and follow your code from Swift source files to a running app. Switch between Simulator and a real device, or between Debug and Release, and notice which stages change. Then choose a problem from the menu and run again: the build stops at the exact stage that catches it, with the message Xcode would show.

This step is an interactive simulation in the HTML version.

CONCEPT

The rest of the toolbox

Xcode sits at the center, but a few more tools show up in almost every iOS developer's week.

Swift Playgrounds	An app for iPad and Mac for learning Swift and sketching ideas; it can even build and publish small SwiftUI apps.
Swift Package Manager	Swift's built-in tool for adding and sharing libraries, called packages, each described by a Package.swift file.
Instruments	Xcode's profiler. It records CPU time, memory, energy, network traffic and, in Xcode 26, SwiftUI view updates.
Developer documentation	Every API with articles and sample code, in Xcode under Help > Developer Documentation and on developer.apple.com.
Apple Developer Program	The paid membership needed for TestFlight betas and App Store distribution.

> **Free account or paid program?**
>
> A free Apple Account is enough to learn, use the Simulator and run apps on your own devices. The Apple Developer Program costs 99 USD per year and is required to distribute through TestFlight and the App Store.

CODE · READ-ALONG

A Swift package manifest

Every Swift package describes itself in a `Package.swift` manifest, and the manifest is itself Swift code. This one declares a small library, TipMath, that depends on Apple's open-source swift-algorithms package. Xcode uses the same system when you choose File > Add Package Dependencies.

```swift
// swift-tools-version: 6.2
import PackageDescription

let package = Package(
    name: "TipMath",
    platforms: [.iOS(.v26), .macOS(.v26)],
    products: [.library(name: "TipMath", targets: ["TipMath"])],
    dependencies: [
        .package(
            url: "https://github.com/apple/swift-algorithms",
            from: "1.2.0"
        )
    ],
    targets: [
        .target(name: "TipMath", dependencies: [
            .product(name: "Algorithms", package: "swift-algorithms")
        ]),
        .testTarget(name: "TipMathTests", dependencies: ["TipMath"])
    ]
)
```

Line by line

1 `// swift-tools-version: 6.2`

- **WHO** Swift Package Manager reads this comment before anything else.
- **WHAT** **Declares the minimum version of the package tools, and of the PackageDescription API, this manifest needs.**
- **WHERE** The very first line of Package.swift; it must stay first.
- **WHEN** When Xcode or the swift command loads the package, before compiling the manifest.
- **WHY** Pinning the tools version lets the manifest use newer APIs, like .v26 platforms, and tells older tools to refuse it clearly.

2 `import PackageDescription`

- **WHO** The Swift compiler, compiling the manifest.
- **WHAT** **Loads the types used to describe a package: Package, Product, Target and dependency requirements.**
- **WHERE** Second line of the manifest, at file scope.
- **WHEN** When Swift Package Manager compiles and runs the manifest to read its settings.
- **WHY** A manifest is a real Swift program, so it imports its vocabulary like any other file.

4 `let package = Package(`

- **WHO** Swift Package Manager looks for this constant by name.
- **WHAT** **Creates the Package value that describes everything below.**
- **WHERE** A top-level constant; the name must be exactly package.
- **WHEN** Evaluated each time the package is resolved or built.
- **WHY** A plain Swift initializer gives you type checking and autocompletion for the whole configuration.

5 `name: "TipMath",`

- **WHO** Swift Package Manager and Xcode, for display.
- **WHAT** **Names the package.**
- **WHERE** The first argument of Package(...).
- **WHEN** Read when the package is loaded.
- **WHY** The name appears in Xcode's Package Dependencies list and in error messages.

6 `platforms: [.iOS(.v26), .macOS(.v26)],`

WHO The compiler, when building the package for each platform.
WHAT **Sets the oldest iOS and macOS versions the package supports: version 26 of each.**
WHERE An argument of Package(...), applied to every target in the package.
WHEN Checked when an app adds the package and on every build.
WHY Raising the minimum lets the code use iOS 26 APIs without availability checks; apps targeting older systems are told up front.

7 `products: [.library(name: "TipMath", targets: ["TipMath"])],`

WHO Swift Package Manager, and any app that adds this package.
WHAT **Publishes a library product, built from the TipMath target, that other code can import.**
WHERE An argument of Package(...); products are the public face of a package.
WHEN Read when an app picks which products to link from the package.
WHY Targets are internal building blocks; only products are visible to the apps that depend on the package.

8 `dependencies: [`

WHO Swift Package Manager's dependency resolver.
WHAT **Starts the list of other packages this package needs.**
WHERE An argument of Package(...).
WHEN Read when packages are resolved, before anything compiles.
WHY Declaring dependencies lets the tool download, version and cache them for you.

9 `.package(`

WHO Swift Package Manager's dependency resolver, reading this entry.
WHAT **Begins one dependency on a remote package.**
WHERE The first entry in the dependencies array.
WHEN When packages are resolved: on the first build, and again whenever you update packages.
WHY Each entry pairs a location with a version rule.

10 `url: "https://github.com/apple/swift-algorithms",`

WHO Swift Package Manager, which clones the repository with Git.
WHAT **Says where the dependency's source code lives.**
WHERE The first argument of .package(...).
WHEN The first time packages are resolved, and when you ask Xcode to update them.
WHY Packages are distributed as Git repositories, so any public or private repository URL works.

11 `from: "1.2.0"`

WHO Swift Package Manager's version resolver.
WHAT **Accepts version 1.2.0 or any newer version below 2.0.0.**
WHERE The second argument of .package(...).
WHEN At resolution time; the exact version chosen is recorded in Package.resolved.
WHY Semantic versioning promises no breaking changes within a major version, so updates inside 1.x are safe to take.

14 `targets: [`

WHO Swift Package Manager, which builds each target as a module.
WHAT **Starts the list of modules in this package.**
WHERE An argument of Package(...).
WHEN Read when the package is built.
WHY Each target maps to a folder of source files, by default Sources/TargetName.

```
15  .target(name: "TipMath", dependencies: [
```

WHO The Swift compiler, which builds this target as a module named TipMath.
WHAT **Declares the library code target and starts its list of dependencies.**
WHERE The first entry in targets; its files live in Sources/TipMath.
WHEN Compiled whenever an app or test that uses TipMath builds.
WHY Listing dependencies per target keeps each module's imports explicit and its build order correct.

```
16  .product(name: "Algorithms", package: "swift-algorithms")
```

WHO Swift Package Manager links the product; the compiler makes it importable.
WHAT **Lets TipMath's code write import Algorithms.**
WHERE Inside the TipMath target's dependency list.
WHEN At build time, before TipMath compiles.
WHY A package can offer several products, so you name the exact one you use.

```
18  .testTarget(name: "TipMathTests", dependencies: ["TipMath"])
```

WHO Swift Package Manager and the Swift Testing runner.
WHAT **Declares a test target whose files can import TipMath and test it.**
WHERE The second entry in targets; its files live in Tests/TipMathTests.
WHEN Built and run only when you run tests, for example with Command-U in Xcode.
WHY Keeping tests in their own target means test code never ships inside an app.

> **Try it**
>
> 1. Which line would you change to accept only 1.2.x versions of the dependency, and what rule might you use instead of from:?
> 2. If an app targets iOS 18, can it add this package? Which line decides?

CHECK YOURSELF

Check your understanding

1. You want to see how your layout looks on an iPad mini, but you only own an iPhone. What is the quickest way?
   A. Borrow or buy an iPad mini and run the app on it
   B. Choose an iPad mini in the Simulator
   C. Rotate your iPhone to landscape to get a wider layout
   D. Upload a TestFlight build and ask an iPad owner to try it
2. Your app runs in the Simulator, but when you choose your new iPhone as the destination, Xcode refuses to run it and mentions Developer Mode. What fixes it?
   A. Join the paid Apple Developer Program
   B. Switch the build configuration to Release
   C. Turn on Developer Mode in Settings on the iPhone
   D. Reinstall Xcode and its iOS platform support
3. Xcode reports `cannot find 'greeting' in scope`. At which stage of the build did this happen?
   A. Code signing, when the identity was checked
   B. The linker, when it joined the object files
   C. Installing the app on the device
   D. The Swift compiler, while type-checking

4. You want to use an open-source charting library in your app and keep it updated. Which tool manages it?
   A. Add it with Swift Package Manager
   B. Copy its source files into your project
   C. Open it in Swift Playgrounds
   D. Profile it with Instruments

Answers: Xcode and the Developer Toolbox

1. **B. Choose an iPad mini in the Simulator** The Simulator includes every current device size, so you can switch the run destination to an iPad mini in seconds, without a device or a TestFlight upload. Rotating an iPhone gives you a different shape, but still an iPhone's size classes and dimensions.
2. **C. Turn on Developer Mode in Settings on the iPhone** iPhones block apps installed from Xcode until Developer Mode is on, under Settings > Privacy & Security. A paid membership is not needed to run on your own device; a free Apple Account can sign apps for personal testing.
3. **D. The Swift compiler, while type-checking** Name lookup and type checking happen in the Swift compiler, the first stage after your source files. The build stops there, so the linker and code signing never run. Linker errors mention missing symbols, not names in scope.
4. **A. Add it with Swift Package Manager** Swift Package Manager downloads the library, resolves a compatible version, links it into your build and can update it later. Copied files go stale the moment the library changes. Instruments measures performance; it does not manage code.

Anatomy of a SwiftUI App

CONCEPT

What the App template gives you

In Xcode, **File > New > Project**, then **iOS > App** with the SwiftUI interface, creates a project that already builds and runs. Every item in it has a job. Click the items in the project navigator to see what each one is, who reads it and when.

> **Where the bundle identifier comes from**
>
> The new-project dialog joins your organization identifier and product name: `com.example` plus `HelloWorld` becomes `com.example.HelloWorld`.

Interactive figure in the HTML version: ecosystem project navigator.

CONCEPT

Identity, metadata and permissions

Three pieces of configuration travel with every app. iOS reads them before any of your Swift code runs.

Bundle identifier	The app's unique ID in reverse-domain form, such as com.example.HelloWorld. It cannot change after you upload a build to App Store Connect.
Info.plist	A property list (a key-value file) of metadata: display name, version, supported orientations, and the purpose strings shown in permission prompts. Xcode generates it from your target's Info tab.
Capabilities	Protected services your app opts into under Signing & Capabilities, such as iCloud, Push Notifications or App Groups.
Entitlements	The signed list of those capabilities, kept in a .entitlements file and sealed into the code signature so iOS can check what the app may do.

> **A missing purpose string ends the app**
>
> If your app asks for the camera without an `NSCameraUsageDescription` entry in Info.plist, iOS terminates the app instead of showing the permission prompt.

CODE · READ-ALONG

The app's entry point

This is the whole entry point of a new SwiftUI app. There is no `main()` function to write: the `@main` attribute tells Swift where the program starts, and the `App` protocol tells SwiftUI which scenes to create.

```swift
import SwiftUI

@main
struct HelloWorldApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

Line by line

1 `import SwiftUI`

WHO	The Swift compiler, while it builds HelloWorldApp.swift.
WHAT	**Brings in SwiftUI, which defines the App and Scene protocols and WindowGroup.**
WHERE	The top of HelloWorldApp.swift.
WHEN	At compile time, before the @main entry point is generated.
WHY	The app's structure is described with SwiftUI types, so this file needs the framework too, not just the view files.

3 `@main`

WHO	The Swift compiler at build time, then iOS at launch.
WHAT	**Marks the type below as the program's entry point; the compiler generates the real main function, which hands control to SwiftUI.**
WHERE	An attribute on the struct declared on the next line. Exactly one type in an app may carry it.
WHEN	Code is generated at build time; that generated main runs once, first, when the user launches the app.
WHY	It removes start-up boilerplate and makes a second, conflicting entry point a compile error.

4 `struct HelloWorldApp: App {`

WHO	Your code declares it; SwiftUI creates one instance.
WHAT	**Declares the app type and conforms it to the App protocol, which requires a body of scenes.**
WHERE	A top-level struct, by convention in a file named after the product.
WHEN	SwiftUI creates the single instance at launch, before any window appears.
WHY	Describing the app as a value, like a view, keeps start-up declarative: you state what exists, SwiftUI does the work.

5 `var body: some Scene {`

WHO	SwiftUI reads it at launch; you never call it yourself.
WHAT	**Lists the scenes the app offers to the system.**
WHERE	The one requirement of the App protocol.
WHEN	At launch, and again if state the app body reads changes.
WHY	Scenes, not views, are what the system shows, hides and duplicates, so the app declares scenes and each scene holds views.

6 `WindowGroup {`

WHO	SwiftUI together with the system's scene manager.
WHAT	**Declares a scene that presents windows whose content is ContentView.**
WHERE	The only scene inside body.
WHEN	At launch the system asks for one window; on iPad, Mac and Vision Pro it creates more whenever the user opens a new window.
WHY	One declaration covers a single full-screen iPhone window and multiple side-by-side windows on larger devices.

7 `ContentView()`

WHO SwiftUI creates it inside each window.
WHAT **Makes ContentView the root view of every window in the group.**
WHERE The content closure of WindowGroup.
WHEN Each time a window is created for this group.
WHY Keeping the UI in its own view file leaves the entry point tiny and lets you preview ContentView on its own.

> **Try it**
>
> 1. Suppose you add @main to a second struct as well. Predict whether the app still builds, and why.

CODE · LIVE SWIFTUI PREVIEW

The template's first view

This is the `ContentView.swift` file exactly as the App template creates it. It runs live in the phone beside the code: edit the text or the symbol name and the preview updates as you type.

```swift
import SwiftUI

struct ContentView: View {
    var body: some View {
        VStack {
            Image(systemName: "globe")
                .imageScale(.large)
                .foregroundStyle(.tint)
            Text("Hello, world!")
        }
        .padding()
    }
}

#Preview {
    ContentView()
}
```

Line by line

1 `import SwiftUI`

WHO The Swift compiler, while it builds ContentView.swift.
WHAT **Makes View, VStack, Image and Text available in this file.**
WHERE The first line of ContentView.swift.
WHEN At compile time, before ContentView is type-checked.
WHY Each file imports exactly the frameworks it uses.

3 `struct ContentView: View {`

WHO Your code declares it; SwiftUI creates and redraws it.
WHAT **Declares the first screen of the app as a struct conforming to View.**
WHERE A top-level type, used by the WindowGroup in the app's entry point.
WHEN Created when the window opens and re-created whenever its parent rebuilds.
WHY A view is a lightweight description of UI, not the pixels themselves, so creating one is cheap.

```
4  var body: some View {
```

WHO	SwiftUI calls it whenever it needs to know what ContentView shows.
WHAT	**Returns what this screen shows.**
WHERE	The required property of the View protocol.
WHEN	When the view appears and whenever its data changes.
WHY	All of a view's appearance lives in one computed property, so reading it tells you the whole screen.

```
5      VStack {
```

WHO	SwiftUI's vertical stack layout.
WHAT	**Arranges its children top to bottom, centered, with default spacing.**
WHERE	The root view inside body.
WHEN	Laid out each time body runs.
WHY	Stacks are the basic building block of SwiftUI layout; most screens are stacks inside stacks.

```
6          Image(systemName: "globe")
```

WHO	SwiftUI, using the SF Symbols library built into iOS.
WHAT	**Shows the globe symbol.**
WHERE	First child of the VStack.
WHEN	Created each time ContentView's body runs, before the Text below it.
WHY	A system symbol needs no image file and matches the weight of nearby text.

```
7              .imageScale(.large)
```

WHO	A SwiftUI modifier for symbol images.
WHAT	**Draws the symbol at its large scale, relative to the current font.**
WHERE	Applies to the globe image.
WHEN	Each time body runs, right after the globe Image is created.
WHY	Symbol scales keep icons proportional to text, so they grow with Dynamic Type.

```
8              .foregroundStyle(.tint)
```

WHO	A SwiftUI modifier, reading the app's accent color.
WHAT	**Colors the globe with the tint, which comes from AccentColor in the asset catalog.**
WHERE	Applies to the globe image.
WHEN	Applied each time body runs; the tint is resolved at draw time.
WHY	Using the tint instead of a fixed color means one change in Assets restyles the whole app.

```
9          Text("Hello, world!")
```

WHO	SwiftUI's Text view, drawing the template's greeting.
WHAT	**Shows the greeting under the globe.**
WHERE	Second child of the VStack.
WHEN	Created each time body runs, as the second child of the VStack.
WHY	A string literal passed to Text is looked up for translation automatically when the app is localized.

```
11     .padding()
```

WHO	A SwiftUI modifier applied to the whole VStack.
WHAT	**Adds standard space around the stack.**
WHERE	Wraps the VStack that holds the globe and the greeting, so it is the outermost view in body.
WHEN	Each time body runs, after the VStack and both children are built.
WHY	Default padding keeps content away from the screen edges without hard-coded numbers.

15 `#Preview {`

WHO	Xcode's canvas, and the phone preview on this page.
WHAT	**Declares a preview of ContentView. #Preview is a macro: code that generates more code at compile time.**
WHERE	At file scope, after the view.
WHEN	Each time the canvas refreshes after an edit.
WHY	You see the view as you type, without building and launching the whole app.

16 `ContentView()`

WHO	The #Preview block, creating one ContentView to display.
WHAT	**Creates the view to display.**
WHERE	Inside the #Preview closure.
WHEN	Each time the preview renders.
WHY	A preview can create a view with sample data, which is how you check edge cases quickly.

> **Try it**
>
> 1. Change "globe" to "iphone". What appears?
> 2. Change .foregroundStyle(.tint) to .foregroundStyle(.green). Which parts of the screen change color?

CONCEPT

App, scenes and views

A SwiftUI app is a tree with three layers. The **App** is the whole program, and there is exactly one. It declares one or more **scenes**: units of interface that the system can show, hide and duplicate on its own. Each scene holds a hierarchy of **views**.

On iPhone a WindowGroup shows one window filling the screen. On iPad, Mac and Vision Pro the same WindowGroup lets people open several windows of your app at once, each with its own state.

WindowGroup	The standard scene: one or more windows showing the same kind of view.
DocumentGroup	Windows tied to files the user creates, opens and saves.
Settings, MenuBarExtra	Mac scenes for a settings window and a menu bar item.
ImmersiveSpace	A visionOS scene that surrounds the user with 3D content.

CODE · LIVE SWIFTUI PREVIEW

A first screen of your own

Here is a slightly richer first screen: a large symbol, a title and a Liquid Glass button. Tapping the button flips a piece of `@State`, a value SwiftUI stores for the view, and the screen redraws to match. Module 04 (SwiftUI) explains `@State` in full; for now read it as a value that survives redraws. Tap it in the phone, then edit the code.

```swift
import SwiftUI

struct FirstScreen: View {
    @State private var isGreeting = false

    var body: some View {
        VStack(spacing: 20) {
            Image(systemName: "swift")
                .font(.system(size: 72))
                .foregroundStyle(.orange)
            Text(isGreeting ? "Welcome aboard" : "Hello, world")
                .font(.largeTitle)
            Button(isGreeting ? "Start over" : "Say hello") {
                isGreeting.toggle()
            }
            .buttonStyle(.glassProminent)
        }
        .padding()
    }
}

#Preview { FirstScreen() }
```

Line by line

1 `import SwiftUI`

WHO The Swift compiler, while it builds FirstScreen's file.

WHAT **Brings in SwiftUI, including the @State property wrapper and the glass button styles.**

WHERE The first line of the file; it covers FirstScreen and the preview below.

WHEN At compile time, before FirstScreen or its @State is checked.

WHY Without it, none of the view types or modifiers below would be found.

3 `struct FirstScreen: View {`

WHO Your code declares it; SwiftUI draws it.

WHAT **Declares a view named FirstScreen.**

WHERE A top-level struct in this file.

WHEN Created by the preview, and re-created whenever SwiftUI rebuilds its parent.

WHY A descriptive type name tells the next reader what the screen is for.

4 `@State private var isGreeting = false`

WHO SwiftUI owns the storage; this view reads and writes it.

WHAT **Creates a Boolean that remembers whether the greeting is showing, starting as false.**

WHERE Stored by SwiftUI outside the struct, so it survives when the struct is re-created.

WHEN Created once when the view first appears; changing it makes SwiftUI run body again.

WHY A view struct is rebuilt often, so values that must persist between rebuilds live in @State. It is private because only this view owns it.

6 `var body: some View {`

WHO SwiftUI calls it; your code only describes what it returns.

WHAT **Describes the screen for the current value of isGreeting.**

WHERE The View protocol's required property.

WHEN On first appearance and after every change to isGreeting.

WHY The screen is a function of its state: change the state and SwiftUI works out what to redraw.

```
7  VStack(spacing: 20) {
```

WHO SwiftUI's VStack layout, holding the symbol, title and button.
WHAT **Stacks the symbol, title and button with 20 points between each.**
WHERE The root view of body.
WHEN Laid out each time body runs.
WHY A single explicit spacing value gives the screen an even rhythm.

```
8  Image(systemName: "swift")
```

WHO SwiftUI, with SF Symbols.
WHAT **Shows the Swift bird.**
WHERE First child of the stack.
WHEN Created each time FirstScreen's body runs, including after each tap.
WHY A vector symbol looks sharp at any size and on any scale factor.

```
9  .font(.system(size: 72))
```

WHO A SwiftUI modifier on the Swift symbol Image.
WHAT **Makes the symbol 72 points tall.**
WHERE Applies to the image.
WHEN Each time body runs, right after the Image is created.
WHY A large, fixed size suits a decorative hero symbol that should not grow with text settings.

```
10 .foregroundStyle(.orange)
```

WHO A SwiftUI modifier on the already-sized Image.
WHAT **Colors the bird orange.**
WHERE Applies to the sized image.
WHEN Each time body runs, after .font sizes the symbol.
WHY Orange is the color of the Swift logo, so the screen reads as a Swift welcome.

```
11 Text(isGreeting ? "Welcome aboard" : "Hello, world")
```

WHO SwiftUI's text view, fed by a ternary expression.
WHAT **Shows "Welcome aboard" when isGreeting is true and "Hello, world" when it is false.**
WHERE Second child of the stack.
WHEN Re-evaluated every time body runs, so it always matches the state.
WHY Deriving the text from state means there is no separate step that could forget to update the label.

```
12 .font(.largeTitle)
```

WHO A SwiftUI modifier on the title Text.
WHAT **Uses the largest standard text style.**
WHERE Applies to the title text.
WHEN Each time body runs, after the ternary picks which title to show.
WHY Text styles respect Dynamic Type, so the title grows for people who use larger text.

```
13 Button(isGreeting ? "Start over" : "Say hello") {
```

WHO SwiftUI creates the button; the user triggers it.
WHAT **Creates a button whose title also depends on isGreeting, and opens its action closure.**
WHERE Third child of the stack.
WHEN Created when body runs; its closure runs later, on each tap.
WHY The title tells the user what tapping will do next, which changes with the state.

14 `isGreeting.toggle()`

WHO	Your code, running on the main thread when the user taps.
WHAT	**Flips isGreeting from false to true, or back.**
WHERE	Inside the button's action closure.
WHEN	Only when the button is tapped, never while body is being built.
WHY	Changing state is the only thing the button does; SwiftUI notices and redraws the text and the title for you.

16 `.buttonStyle(.glassProminent)`

WHO	A SwiftUI modifier using the iOS 26 Liquid Glass design.
WHAT	**Draws the button as prominent tinted glass.**
WHERE	Applies to the button.
WHEN	Each time body runs, when the Button is rebuilt with its current title.
WHY	The prominent glass style marks the screen's main action in the current system design language.

18 `.padding()`

WHO	A SwiftUI modifier applied to the outermost VStack.
WHAT	**Adds standard space around the whole stack.**
WHERE	Wraps the VStack that holds the symbol, title and button, as the last modifier in body.
WHEN	Each time body runs, after all three children are built.
WHY	It keeps the content clear of the screen edges on every device size.

22 `#Preview { FirstScreen() }`

WHO	Xcode's canvas, and the phone preview on this page.
WHAT	**Previews a FirstScreen on a single line.**
WHERE	At file scope, after the view.
WHEN	Each time the preview refreshes.
WHY	Short previews fit on one line; longer ones use braces across several lines.

> **Try it**
>
> 1. Tap the button twice in the phone. What are the two titles you see?
> 2. Change .glassProminent to .glass. How does the button change?
> 3. Make the stack spacing 40. Which line do you edit?

CHECK YOURSELF

Check your understanding

1. You delete the `@main` line from `HelloWorldApp.swift` and build. What happens?
   A. The app still launches ContentView, because Xcode finds the App struct
   B. The build fails, because the app has no entry point
   C. Xcode adds the attribute back for you the next time you build
   D. The app launches to a black screen with no window content
2. Your trail app needs the user's location. Where does the sentence "Shows trails near you" belong so iOS can display it in the permission prompt?
   A. In ContentView, as a Text above the map
   B. In the Signing & Capabilities tab of the target
   C. In the asset catalog, next to the app icon
   D. In Info.plist, as the location purpose string

3. In the template, you change `.foregroundStyle(.tint)` to `.foregroundStyle(.green)`. What changes on screen?
   A. Only the globe turns green
   B. The globe and the text turn green
   C. The whole background turns green
   D. Nothing, because the tint overrides it
4. You are building a sketching app where every drawing is a file the user can open from the Files app. Which scene type fits?
   A. WindowGroup
   B. Settings
   C. DocumentGroup
   D. ImmersiveSpace

Answers: Anatomy of a SwiftUI App

1. **B. The build fails, because the app has no entry point** Without @main, nothing tells Swift where the program starts, so the build fails and there is no app to launch at all, black screen or otherwise. Xcode does not guess an entry point for you.
2. **D. In Info.plist, as the location purpose string** iOS shows the purpose string from Info.plist (NSLocationWhenInUseUsageDescription) inside its own prompt. A Text in your view appears only in your interface, Capabilities lists services rather than wording, and the system will not ask for permission without the Info.plist entry.
3. **A. Only the globe turns green** The modifier is attached to the Image only, so only the globe changes. To color the text as well, the modifier would have to wrap the VStack.
4. **C. DocumentGroup** DocumentGroup ties each window to a file and provides open, save and Files integration for you. A WindowGroup could work, but you would have to build all the file handling yourself.

The App Lifecycle

CONCEPT

Five states every app moves through

On iOS the system, not your app, decides when your code runs. To protect battery life and memory, iOS moves every app through a small set of states and tells the app about the transitions it can observe.

Not running	Never launched, or terminated. No code runs.
Inactive	On screen but not receiving touches: briefly during launch, or while a system alert or the app switcher covers it.
Active	On screen and receiving events. The normal state while someone uses your app.
Background	Off screen but still running code for a short time, for example to finish saving.
Suspended	Kept in memory but frozen; no code runs. iOS can remove it at any moment to free memory.

> **Termination is silent**
>
> When iOS reclaims a suspended app's memory, the app gets no warning and runs no code. Anything not saved before it was suspended is gone. Save when you move to the background, not later.

INTERACTIVE

Drive an app through its lifecycle

Type a draft into the notes app, then use the system events to move the app between states. The log shows which SwiftUI callbacks fire. Turn **Save when backgrounded** off, go to the Home Screen, apply memory pressure and relaunch: is your draft still there? Then turn saving on and repeat the experiment.

This step is an interactive simulation in the HTML version.

CONCEPT

scenePhase: the part SwiftUI lets you see

SwiftUI reports the lifecycle through `scenePhase`, an environment value with three cases. You read it with `@Environment(\.scenePhase)` and react to changes with the `.onChange(of:)` modifier.

.active	In the foreground and interactive. Start timers and resume live updates.
.inactive	Visible but not interactive, or in transition. Pause anything the user should not miss.
.background	Not visible. Save data and release resources now; suspension may come next.

> **One scene, or the whole app?**
>
> Read inside a view, scenePhase describes that view's scene. Read in your App struct, it summarizes every scene: .active if any scene is active. Suspended and not running never appear, because no code runs in those states. Module 04 (SwiftUI) explains `@Environment` and `.onChange` in full; for now read them as a value SwiftUI hands you and a block that runs when it changes.

CODE · READ-ALONG

Save when the app leaves the screen

The safest moment to save is when the app goes to the background, the last point where it reliably gets to run code before it may be suspended. This App struct watches the app-wide phase and saves the draft at exactly that moment. Module 04 (SwiftUI) explains `@Environment`, `@State` and `@Observable` in full; for now read along. The simulator beside the code lets you test the idea.

```swift
import SwiftUI

@main
struct NotesApp: App {
    @Environment(\.scenePhase) private var scenePhase
    @State private var store = DraftStore()

    var body: some Scene {
        WindowGroup {
            DraftScreen(store: store)
        }
        .onChange(of: scenePhase) { oldPhase, newPhase in
            print("Phase: \(oldPhase) -> \(newPhase)")
            if newPhase == .background {
                store.save()
            }
        }
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds NotesApp's file.
- **WHAT** **Brings in SwiftUI, which defines App, scenes, @Environment, @State and the ScenePhase type.**
- **WHERE** The top of the app's entry-point file.
- **WHEN** At compile time, before the @main entry point is generated.
- **WHY** Lifecycle handling is part of SwiftUI's app model, so no extra framework is needed.

3 `@main`

- **WHO** The compiler, then iOS at launch.
- **WHAT** **Makes NotesApp the entry point of the program.**
- **WHERE** On the struct below; only one type in the app has it.
- **WHEN** Code is generated at build time and runs first at launch.
- **WHY** Putting lifecycle code on the App struct means it covers every window the app opens.

```
4  struct NotesApp: App {
```

WHO Your code declares it; SwiftUI creates it once.
WHAT **Declares the notes app.**
WHERE A top-level struct conforming to App.
WHEN Created at launch, before the first window.
WHY The App struct lives as long as the process, so it is a natural owner for app-wide data.

```
5  @Environment(\.scenePhase) private var scenePhase
```

WHO SwiftUI's environment, fed by iOS lifecycle events.
WHAT **Reads the app-wide scene phase: active, inactive or background.**
WHERE A property of the App struct, so it summarizes all of the app's scenes.
WHEN Updated by SwiftUI each time the system moves the app between states.
WHY Reading the phase at the app level saves once for the whole app instead of once per window.

```
6  @State private var store = DraftStore()
```

WHO SwiftUI keeps the storage alive; DraftStore is your @Observable model class.
WHAT **Creates the single store that holds the draft text in memory.**
WHERE Owned by the App struct and passed down to the screen.
WHEN Created once at launch and kept until the process ends.
WHY Without @State, a new, empty store could be created whenever the App struct is re-created. Module 04 (SwiftUI) explains @State and @Observable in full; for now read this line as "keep one store alive for the whole app".

```
8  var body: some Scene {
```

WHO SwiftUI reads it at launch; you never call it yourself.
WHAT **Declares the app's scenes.**
WHERE The App protocol's required property.
WHEN At launch, and when state the body reads changes.
WHY Scene-level modifiers, like the onChange below, attach here.

```
9  WindowGroup {
```

WHO SwiftUI and the system scene manager.
WHAT **Declares the window that shows the draft screen.**
WHERE Inside NotesApp's body, as its only scene.
WHEN When the system creates a window, at launch or when the user opens another.
WHY Every window gets the same shared store, so drafts stay consistent across windows.

```
10 DraftScreen(store: store)
```

WHO SwiftUI creates the view in each window.
WHAT **Shows the editing screen and hands it the shared store.**
WHERE The content of the WindowGroup.
WHEN When a window is created and whenever the scene rebuilds.
WHY Passing the model in explicitly makes the dependency visible and easy to preview with sample data.

```
12 .onChange(of: scenePhase) { oldPhase, newPhase in
```

WHO SwiftUI calls the closure.
WHAT **Watches scenePhase and runs the closure with the previous and the new value whenever it changes.**
WHERE A modifier on the WindowGroup scene.
WHEN On every lifecycle transition, such as active to inactive, or inactive to background.
WHY Reacting to transitions, not polling, means the code runs exactly when the state changes. Module 02 (Swift Fundamentals) explains closures with parameters in full; for now read oldPhase, newPhase in as "the two values SwiftUI hands to this block".

```
13 print("Phase: \(oldPhase) -> \(newPhase)")
```

WHO — Your code; the output goes to the Xcode debug console.
WHAT — **Logs each transition, for example from inactive to background.**
WHERE — The first statement in the onChange closure.
WHEN — On every phase change while the debugger is attached.
WHY — Logging transitions while you develop shows you the real order of events on a device.

```
14 if newPhase == .background {
```

WHO — You, the app author, inside the onChange closure.
WHAT — **Checks whether the app has just gone to the background.**
WHERE — Inside the onChange closure.
WHEN — On each phase change; true only when the app leaves the screen.
WHY — Background is the last reliable chance to run code before suspension, and a suspended app may be terminated silently.

```
15 store.save()
```

WHO — Your DraftStore model object, which owns the draft text.
WHAT — **Writes the in-memory draft to permanent storage.**
WHERE — Runs inside the if, on the main thread.
WHEN — Once each time the app moves to the background.
WHY — Saving here means the draft survives even if iOS later terminates the suspended app without warning.

Try it

1. In the simulator, turn saving off, go home, apply memory pressure and relaunch. Then turn it on and repeat. What differs?
2. Why would saving in .inactive alone be a poor choice? Hint: think about what makes an app inactive.

CODE · READ-ALONG

Restore where the user left off

Data is not the only thing worth keeping. If iOS terminates your app in the background, people expect to come back to the same tab and the same half-typed text. `@SceneStorage` saves small pieces of interface state for each scene and restores them automatically. Module 04 (SwiftUI) explains property wrappers and the `$` binding syntax in full; for now read `$draft` as a two-way link to the stored text.

```swift
import SwiftUI

struct ComposeView: View {
    @SceneStorage("compose.draft") private var draft = ""
    @SceneStorage("compose.tab") private var tab = 0

    var body: some View {
        TabView(selection: $tab) {
            Tab("Write", systemImage: "pencil", value: 0) {
                TextEditor(text: $draft)
                    .padding()
            }
            Tab("Preview", systemImage: "doc.text", value: 1) {
                Text(draft)
                    .padding()
            }
        }
    }
}
```

Line by line

1 `import SwiftUI`

WHO	The Swift compiler, while it builds ComposeView's file.
WHAT	**Brings in SwiftUI, which provides @SceneStorage, TabView and TextEditor.**
WHERE	The top of the file that holds ComposeView.
WHEN	At compile time, before ComposeView is type-checked.
WHY	Scene storage is a SwiftUI feature; nothing else needs importing.

3 `struct ComposeView: View {`

WHO	Your code; SwiftUI draws it.
WHAT	**Declares a two-tab compose screen.**
WHERE	A top-level view type.
WHEN	Created when its window opens.
WHY	A compact, realistic screen shows restoration of both text and navigation.

4 `@SceneStorage("compose.draft") private var draft = ""`

WHO	SwiftUI saves and restores the value for this scene.
WHAT	**Keeps the draft text under the key "compose.draft", starting empty.**
WHERE	Stored per scene, so two windows of the app keep two separate drafts.
WHEN	Saved by the system as the scene goes to the background; restored when the scene is recreated after termination.
WHY	The user's unfinished text reappears after a silent termination. Scene storage is for interface state; it can be discarded, for example when the user closes the window.

5 `@SceneStorage("compose.tab") private var tab = 0`

WHO	SwiftUI's scene storage, the same system that keeps the draft.
WHAT	**Remembers which tab is selected, starting at tab 0.**
WHERE	Stored alongside the draft, for this scene only.
WHEN	Saved and restored with the scene, like the draft.
WHY	Restoring navigation makes a terminated app feel as if it never left.

```
7  var body: some View {
```

WHO SwiftUI calls it whenever the scene needs drawing.
WHAT **Describes the tab view.**
WHERE The View protocol's requirement.
WHEN On appearance and whenever draft or tab changes.
WHY The view reads the stored values like ordinary properties.

```
8  TabView(selection: $tab) {
```

WHO SwiftUI's TabView container, which shows one tab at a time.
WHAT **Shows tabs and binds the selected tab to the stored tab value.**
WHERE The root view of body.
WHEN Built when body runs; the binding writes back each time the user switches tabs.
WHY The $ prefix passes a binding, so switching tabs updates scene storage automatically.

```
9  Tab("Write", systemImage: "pencil", value: 0) {
```

WHO SwiftUI's Tab API, which declares the first tab.
WHAT **Declares the first tab, labeled Write with a pencil icon, identified by the value 0.**
WHERE First child of the TabView.
WHEN Built when body runs.
WHY The value ties this tab to the stored selection.

```
10 TextEditor(text: $draft)
```

WHO SwiftUI's multi-line text editor; the user types into it.
WHAT **Edits the draft, writing every keystroke back to scene storage through the binding.**
WHERE The content of the Write tab.
WHEN Updates on every keystroke.
WHY Binding straight to scene storage means there is no separate save step to forget.

```
11 .padding()
```

WHO A SwiftUI modifier on the TextEditor.
WHAT **Insets the editor from the edges.**
WHERE Applies to the TextEditor.
WHEN Each time body runs, right after the TextEditor is created.
WHY Text that touches the screen edge is harder to read and select.

```
13 Tab("Preview", systemImage: "doc.text", value: 1) {
```

WHO SwiftUI's Tab API, which declares the second tab.
WHAT **Declares the second tab, identified by the value 1.**
WHERE Second child of the TabView.
WHEN Built when body runs.
WHY Two tabs make the restored selection visible when you test termination.

```
14 Text(draft)
```

WHO SwiftUI's Text view, showing the stored draft read-only.
WHAT **Shows the current draft read-only.**
WHERE The content of the Preview tab.
WHEN Re-rendered whenever draft changes.
WHY Both tabs read the same stored value, so they always agree.

```
15  .padding()
```

WHO	A SwiftUI modifier on the preview Text.
WHAT	**Adds space around the preview text.**
WHERE	Applies to the Text.
WHEN	Each time body runs, right after the preview Text is created.
WHY	Matching the editor's inset makes switching tabs feel steady.

> **Try it**
>
> 1. Scene storage keeps the draft, so why does the previous example still save to a store? What kind of data belongs in each?

CONCEPT

Every app lives in a sandbox

Each iOS app runs in a **sandbox**: a private container of folders that only that app can read and write. Your app cannot open another app's files, and other apps cannot open yours. Shared resources such as photos, contacts and location go through system frameworks that ask the user first.

Documents	Files the user creates. Backed up, and can be exposed in the Files app.
Library/Application Support	Your app's own data, such as a SwiftData store. Backed up.
Library/Caches	Data you can download again. Not backed up; iOS may delete it when storage runs low.
tmp	Short-lived scratch files that the system can delete when your app is not running.

> **Sharing with your own extensions**
>
> An app and its extensions, such as a widget, each get a separate sandbox. Adding the App Groups capability creates a shared container that both can read and write.

CHECK YOURSELF

Check your understanding

1. A user types a note, swipes up to the Home Screen and forgets about the app. An hour later iOS reclaims its memory. Which of your code ran last?
   A. A termination callback, just before iOS removed the app
   B. The onChange closure reporting .background, as the user swiped home
   C. onDisappear on the note view, when the memory was reclaimed
   D. Nothing ran after the user started typing the note

2. A system alert, such as a permission prompt, appears over your app. What is the scene phase while it is on screen?
   A. .active
   B. .background
   C. Suspended
   D. .inactive

3. Your workout app runs a stopwatch with a timer. When should it pause the timer?
   A. When scenePhase leaves .active
   B. Only when the app is terminated
   C. Never; timers keep running while suspended
   D. When the view's body runs

4. Your app shows thumbnails it downloads from a server and can download again at any time. Which sandbox folder fits them best?
   A. Documents
   B. Library/Caches
   C. Library/Application Support
   D. Another app's container

5. Your Home Screen widget cannot read the data file the main app writes. What is the cause?
   A. Widgets cannot read files from disk, only values passed in
   B. The file name is wrong, or the path differs in the widget
   C. They have separate sandboxes and need an App Group
   D. The widget runs only on the Lock Screen, outside the app

Answers: The App Lifecycle

1. **B. The onChange closure reporting .background, as the user swiped home** The app was suspended shortly after it reached the background, and suspended apps run no code, so the background transition was the last chance. iOS removes suspended apps silently; no termination callback runs.
2. **D. .inactive** Your app is still visible behind the alert but cannot receive touches, which is exactly the inactive phase. It does not go to the background, because it never left the screen.
3. **A. When scenePhase leaves .active** Leaving .active means the user can no longer see or interact with the app, so pause there and record the time. Timers do not fire while an app is suspended, and termination gives no callback at all.
4. **B. Library/Caches** Caches holds re-creatable data. It is not backed up and iOS may clear it when space is low, which is fine for thumbnails. Documents is backed up and meant for files the user creates, so caches there waste the user's backup space.
5. **C. They have separate sandboxes and need an App Group** An extension is a separate program with its own sandbox. Both targets must join the same App Group and use its shared container. A correct file name does not help if the widget cannot reach the folder, and widgets read files fine once they share one.

The Software Development Life Cycle

CONCEPT

A loop, not a line

The **Software Development Life Cycle** (SDLC) names the phases a software product passes through, from the first idea to its last update. For an iOS app the cycle never really ends: users send feedback, Apple ships a new iOS every year, and each update starts the cycle again.

Requirements	What the app must do, written so that you can test it.
Design	Screens, flows and architecture, often prototyped as SwiftUI previews.
Implementation and testing	Writing Swift with automated tests alongside it.
Deployment	TestFlight betas, App Review, then release.
Maintenance	Crash fixes, OS updates and new features, which feed the next cycle.

> **Waterfall and agile**
>
> The classic *waterfall* model runs each phase once, in order. Most app teams work in an *agile* way instead: short iterations, often one or two weeks long, that pass through every phase and end with something people can test.

INTERACTIVE

Follow one app through the cycle

Follow PlantPal, a plant-watering reminder app, through the life cycle. Select a phase to see its artifacts, the people involved, the Apple tools they use and a common failure. Press **Skip this phase** to see where the damage surfaces later. Then open **Waterfall vs agile** to compare when real user feedback first arrives.

This step is an interactive simulation in the HTML version.

CONCEPT

Requirements you can test

A requirement is useful only if you can tell whether the app meets it. Agile teams often write **user stories**, each with **acceptance criteria** that spell out exactly what "done" means.

User story	"As a plant owner, I want a reminder when a plant needs water, so that it stays healthy."
Acceptance criterion	"Given a fern watered on day 10 with a 3-day interval, when day 13 arrives, then it shows as due."
Non-functional requirement	A quality rather than a feature: launches quickly, works offline, supports Dynamic Type and VoiceOver.

> **Criteria become tests**
>
> The Given / When / Then shape maps straight onto an automated test: set up the data, perform the action, check the result. The next step turns this criterion into one.

CODE · SWIFT TESTING

An acceptance criterion as a test

Here the fern criterion becomes a Swift Testing test. If someone later breaks the watering rule, this test fails before the change ever reaches a tester. Module 09 (Architecture, Testing and Shipping) explains `@Test` and `#expect` in full; for now read `@Test` as "this function is a test" and `#expect` as "this must be true", and notice how directly the requirement maps onto code.

```swift
import Testing

struct WateringSchedule {
    let intervalDays: Int
    let lastWatered: Int

    func isDue(onDay day: Int) -> Bool {
        day - lastWatered >= intervalDays
    }
}

@Test("Fern watered on day 10 is due on day 13")
func fernIsDueAfterInterval() {
    let fern = WateringSchedule(intervalDays: 3, lastWatered: 10)
    #expect(fern.isDue(onDay: 13))
    #expect(!fern.isDue(onDay: 12))
}
```

Line by line

1 `import Testing`

- **WHO** The Swift compiler, linking Apple's Swift Testing library.
- **WHAT** **Makes the @Test macro and the #expect macro available.**
- **WHERE** The top of a file in the app's test target.
- **WHEN** At compile time; the tests run later, when you run the test target.
- **WHY** Test code lives in a separate target, so this import never ships in the app.

3 `struct WateringSchedule {`

- **WHO** Your app's model code (shown here so the test is self-contained).
- **WHAT** **Declares the rule that decides when a plant needs water.**
- **WHERE** Normally in the app target; the test target would import it.
- **WHEN** Defined at compile time.
- **WHY** Keeping rules in a plain struct, separate from views, makes them easy to test.

```
4  let intervalDays: Int
```

WHO Each WateringSchedule value, which owns this stored constant.
WHAT **Stores how many days may pass between waterings.**
WHERE A constant stored property of WateringSchedule.
WHEN Set when a schedule is created; never changes afterwards.
WHY A let property cannot be changed by accident later on.

```
5  let lastWatered: Int
```

WHO Each WateringSchedule value, alongside intervalDays.
WHAT **Stores the day number of the last watering.**
WHERE A constant stored property.
WHEN Set once, when WateringSchedule(intervalDays:lastWatered:) is called.
WHY Day numbers keep the example simple; the real app would store a Date.

```
7  func isDue(onDay day: Int) -> Bool {
```

WHO Your model; called by the app and by the test.
WHAT **Declares a method that answers whether the plant needs water on a given day.**
WHERE An instance method of WateringSchedule.
WHEN Whenever someone asks, such as when the plant list refreshes.
WHY The argument label onDay makes call sites read like a sentence: isDue(onDay: 13).

```
8  day - lastWatered >= intervalDays
```

WHO Swift's integer subtraction and comparison, inside isDue.
WHAT **Returns true when at least intervalDays have passed since the last watering.**
WHERE The single expression in the method body, returned implicitly.
WHEN Each time isDue is called.
WHY Using >= means the plant is due on the boundary day itself, which is what the criterion says.

```
12 @Test("Fern watered on day 10 is due on day 13")
```

WHO Swift Testing finds and runs functions marked with @Test.
WHAT **Marks the function below as a test and gives it a readable name.**
WHERE At file scope in the test target.
WHEN Discovered at build time; run each time you run the tests, for example with Command-U.
WHY The display name repeats the acceptance criterion, so a failure report reads like the requirement.

```
13 func fernIsDueAfterInterval() {
```

WHO Swift Testing calls it.
WHAT **Declares the test function.**
WHERE A global function in the test file.
WHEN When the test runner reaches it.
WHY A descriptive function name helps when tests are listed in Xcode's Test navigator.

```
14 let fern = WateringSchedule(intervalDays: 3, lastWatered: 10)
```

WHO You, the test author, inside fernIsDueAfterInterval.
WHAT **Creates the "Given": a fern watered on day 10 that needs water every 3 days.**
WHERE A local constant inside the test.
WHEN At the start of each run of the test.
WHY Building the exact scenario from the criterion makes the test easy to check against the requirement.

```
15 #expect(fern.isDue(onDay: 13))
```

WHO The #expect macro from Swift Testing.
WHAT **Checks the "Then": on day 13 the fern must be due. A false result is recorded as a failure.**
WHERE Inside the test function.
WHEN When the test runs.
WHY On failure, #expect shows the values involved, so you see exactly what went wrong.

```
16 #expect(!fern.isDue(onDay: 12))
```

WHO The #expect macro from Swift Testing, checking the opposite case.
WHAT **Checks the boundary: on day 12 the fern must not be due yet.**
WHERE Inside the test function, after the first expectation.
WHEN When the test runs; both checks run even if the first one fails.
WHY Testing the day just before the boundary catches off-by-one mistakes, such as writing > instead of >=.

> **Try it**
>
> 1. Change >= to > in isDue. Which expectation would fail, and what would the failure say about the requirement?

CONCEPT

Version numbers and build numbers

Every upload carries two numbers. The **version** (CFBundleShortVersionString) is what people see on the App Store, usually *major.minor.patch*, such as 2.1.0. The **build** (CFBundleVersion) identifies one specific upload, and it must go up each time you upload a build for the same version.

You set both on the target's General tab in Xcode. Use the lab to upload builds and ship versions, and try uploading a build number twice.

Interactive figure in the HTML version: ecosystem version lab.

CONCEPT

Beta testing and App Review

Before release, builds go through **TestFlight**, Apple's beta-testing service. Internal testers on your team, up to 100, get builds right away. External testers, up to 10,000 people invited by email or public link, get them after a brief Beta App Review. Each build stays testable for 90 days.

Every App Store release then passes **App Review**, which checks the app against the App Store Review Guidelines.

Safety	No objectionable content; apps with user-generated content need ways to report and block.
Performance	The build must be complete: no crashes, no placeholder text, no broken links.
Business	Digital content and features are generally sold through In-App Purchase, with some regional exceptions.
Design	An app must offer real functionality, not just a repackaged website.
Legal	A privacy policy, honest privacy labels, and consent before collecting data.

CONCEPT

Privacy labels and phased release

On every App Store product page, **privacy labels** summarize what data an app collects and whether it is linked to the user or used for tracking. You fill them in on App Store Connect, and they must match what your code and every third-party library in it actually do.

Once an update is approved, a **phased release** rolls it out to people with automatic updates over seven days: 1%, 2%, 5%, 10%, 20%, 50%, then 100%. If crash reports spike, you can pause the rollout, fix the bug and ship a new build. Anyone can still update by hand from the App Store.

Privacy manifest	A PrivacyInfo.xcprivacy file, in your app and in each third-party SDK, declaring the data used and the reasons for calling certain sensitive APIs.
App Tracking Transparency	Tracking people across other companies' apps and websites requires asking their permission first.

CHECK YOURSELF

Check your understanding

1. A user story says "Watering should be easy." What is the main problem with it?
   A. It is too short to count as a requirement
   B. It has no acceptance criteria, so it cannot be tested
   C. It does not name the SwiftUI views that will implement it
   D. User stories belong only to waterfall projects, not agile ones
2. You uploaded version 1.2.0, build 14, to TestFlight and then found a typo. What should the next upload be?
   A. 1.2.0, build 14, replacing the first upload
   B. 1.2.1, build 14
   C. 1.2.0, build 15
   D. 2.0.0, build 1
3. Your update is in a phased release, and at the 5% stage crash reports jump. What is the best move?
   A. Pause the rollout, fix the crash, submit a new build
   B. Release to 100% right away so the crash data becomes clearer
   C. Remove the app from sale until a fixed version is approved
   D. Wait the full seven days for the phased release to finish
4. Your app includes an analytics library that collects device identifiers, but your privacy label says "Data Not Collected." What is wrong?
   A. Nothing, because the library collects the data, not your code
   B. Only the privacy policy needs to mention what the library collects
   C. Privacy labels apply only to paid apps, so it can stay
   D. The label must cover all data the app collects, including through libraries

Answers: The Software Development Life Cycle

1. **B. It has no acceptance criteria, so it cannot be tested** "Easy" cannot be checked. Acceptance criteria such as "a plant can be marked watered with one tap from the list" make it testable. Length is not the issue, and a story describes what the user needs, not which views implement it.
2. **C. 1.2.0, build 15** The release users will see has not changed, so the version stays 1.2.0, but every upload needs a higher build number. Uploading build 14 again is rejected as a duplicate.
3. **A. Pause the rollout, fix the crash, submit a new build** Pausing stops automatic updates from reaching more people while you fix the bug. Releasing to everyone would spread the crash to the other 95%, and pulling the app from sale punishes the people who are not affected.
4. **D. The label must cover all data the app collects, including through libraries** You are responsible for everything your app ships, including third-party code. The label must declare that data, and the library's privacy manifest helps you find out what it collects.

Kinds of Apps and Extensions

CONCEPT

Apps and app extensions

An **app** is what people launch from the Home Screen. An **app extension** is a separate, smaller program bundled inside your app that the system runs somewhere else: on the Home Screen as a widget, in another app's share sheet, or as a custom keyboard.

Separate process	An extension runs in its own process, with its own sandbox and tight memory limits.
Launched by the system	People never open an extension directly; iOS starts it when the place that hosts it needs it.
Shared data	The app and its extensions share data through an App Group container.
Shipped together	Each extension is a target in your Xcode project and ships inside the same app bundle.

> **App Intents connect the pieces**
>
> Siri, Shortcuts, Spotlight, widgets and Control Center controls all learn what your app can do through one framework, App Intents. Describe an action once and it shows up in all of them.

CONCEPT

Common shapes of apps

Most apps fit one of a few well-known shapes, and each shape has a matching SwiftUI structure. Recognizing the shape early tells you which frameworks to reach for.

Utility	One focused task on one or two screens: a tip calculator, a unit converter, a timer.
List-detail data app	Browse a collection and drill into items: notes, recipes, contacts. NavigationStack or NavigationSplitView over SwiftData.
Document-based	Each window edits a file the user owns, with Files app integration, built on DocumentGroup.
Game	Real-time graphics and input: SpriteKit for 2D, RealityKit for 3D, Metal for custom rendering, Game Center for leaderboards.
Companion	A watchOS, iPadOS or macOS app that works alongside the iPhone app and shares its data.

CONCEPT

Experiences outside your app

Some of the most-used parts of an app appear where people already are, without opening it at all.

Widgets (WidgetKit)	Glanceable views on the Home Screen, Lock Screen and in StandBy, drawn from a timeline of snapshots rather than live code.

Live Activities (ActivityKit)	The changing status of an event in progress, such as a delivery or a game score, on the Lock Screen and in the Dynamic Island.
App Clips	A small part of your app that opens instantly from an NFC tag, a QR code, an App Clip Code or a link, without a full install.
App Intents	Actions your app offers to Siri, Shortcuts, Spotlight and the Action button.
Share extensions	Your app's entry in the share sheet of other apps.

CODE · READ-ALONG

An action Siri and Shortcuts can run

An App Intent describes one action your app can perform. The provider below publishes a phrase, so people can say "Log water in" plus your app's name, or run it from Shortcuts, without opening your app. A small `WaterLog` actor that stores the count lives elsewhere in the app. Module 03 (Types, Protocols and Programming Paradigms) explains `actor` and `await` in full; for now read `await` as "wait here for the answer".

```swift
import AppIntents

struct LogWaterIntent: AppIntent {
    static let title: LocalizedStringResource = "Log a Glass of Water"

    func perform() async throws -> some IntentResult & ProvidesDialog {
        let total = await WaterLog.shared.addGlass()
        return .result(dialog: "Logged. That is \(total) today.")
    }
}

struct WaterShortcuts: AppShortcutsProvider {
    static var appShortcuts: [AppShortcut] {
        AppShortcut(
            intent: LogWaterIntent(),
            phrases: ["Log water in \(.applicationName)"],
            shortTitle: "Log Water",
            systemImageName: "drop"
        )
    }
}
```

Line by line

1 `import AppIntents`

- **WHO** The Swift compiler, while it builds this file in the app target.
- **WHAT** **Brings in the App Intents framework: AppIntent, AppShortcut and the dialog types.**
- **WHERE** The top of the file, inside the app target.
- **WHEN** At compile time. Xcode also extracts intent metadata at build time so the system knows about the action before the app ever runs.
- **WHY** Intents are discovered from your compiled code, so there is no separate configuration file to keep in sync.

```
3  struct LogWaterIntent: AppIntent {
```

WHO Your code declares it; the system creates and runs it.
WHAT **Declares an action that logs one glass of water.**
WHERE A top-level struct in the app target.
WHEN Created by the system each time someone triggers the action through Siri, Shortcuts or Spotlight.
WHY Each intent is a small value type with one job, which makes it easy to reuse across system features.

```
4  static let title: LocalizedStringResource = "Log a Glass of Water"
```

WHO The Shortcuts app, Spotlight and Siri display it.
WHAT **Gives the action a human-readable, translatable name.**
WHERE A static property of the intent type, shared by every instance.
WHEN Read at build time for the metadata, and whenever the system shows the action.
WHY A let constant of a Sendable type is safe under Swift 6 concurrency checking; LocalizedStringResource lets the title be translated.

```
6  func perform() async throws -> some IntentResult & ProvidesDialog {
```

WHO The system calls it when the action runs.
WHAT **Declares the work of the intent; it can await, can throw, and returns a result that includes something to say.**
WHERE The required method of AppIntent.
WHEN Each time the action is triggered, possibly while your app is not on screen.
WHY async lets the action await other work, and throws lets it report failure; the result type promises a spoken or shown reply.

```
7  let total = await WaterLog.shared.addGlass()
```

WHO Your WaterLog actor, which serializes access to its data.
WHAT **Adds a glass to today's count and gets back the new total.**
WHERE Inside perform(), waiting on the actor.
WHEN Each time the intent runs.
WHY An actor keeps the count safe even if Siri and the app update it at the same moment; await marks the point where this code may wait its turn.

```
8  return .result(dialog: "Logged. That is \(total) today.")
```

WHO The system, which speaks or shows the dialog.
WHAT **Finishes the action and tells the person the new total.**
WHERE The last line of perform().
WHEN When the work is done.
WHY Confirming the result builds trust, especially when the person cannot see your app.

```
12 struct WaterShortcuts: AppShortcutsProvider {
```

WHO The system reads it when the app is installed.
WHAT **Declares the App Shortcuts the app offers with no setup by the user.**
WHERE A top-level type in the app target; an app has one provider.
WHEN Read when the app is installed or updated, so the phrases work immediately.
WHY Without a provider, people would have to build the shortcut themselves in the Shortcuts app.

```
13 static var appShortcuts: [AppShortcut] {
```

WHO The system reads this list.
WHAT **Lists every App Shortcut the app offers.**
WHERE The provider's required static property, built with a result builder.
WHEN Read at install and update time.
WHY The result builder lets you list shortcuts one after another, like views in a stack.

14 `AppShortcut(`

WHO The App Intents framework, building the shortcut value.
WHAT **Creates one shortcut.**
WHERE The only entry in the list.
WHEN Read with the list.
WHY Each shortcut pairs an intent with the phrases and look that expose it.

15 `intent: LogWaterIntent(),`

WHO The App Intents framework, pairing the shortcut with your intent.
WHAT **Says which action the shortcut runs.**
WHERE The first argument of AppShortcut.
WHEN Read at install time; a fresh intent is created on each run.
WHY Connecting phrases to an intent value keeps the action's code in one place.

16 `phrases: ["Log water in \(.applicationName)"],`

WHO Siri matches what people say against these phrases.
WHAT **Defines the spoken phrase; the placeholder becomes the app's actual name.**
WHERE An argument of AppShortcut.
WHEN Read at install time; matched every time someone speaks to Siri.
WHY Every phrase must contain the app name, so Siri knows which app should handle the request.

17 `shortTitle: "Log Water",`

WHO The Shortcuts app and Spotlight.
WHAT **Gives a short label for the shortcut tile.**
WHERE An argument of AppShortcut.
WHEN Shown whenever the shortcut is displayed.
WHY Tiles have little room, so they get their own short title.

18 `systemImageName: "drop"`

WHO The Shortcuts app and Spotlight.
WHAT **Uses the drop SF Symbol as the shortcut's icon.**
WHERE The last argument of AppShortcut.
WHEN Shown with the tile.
WHY A matching symbol makes the action recognizable at a glance.

> **Try it**
>
> 1. Write a second phrase that would also work, such as one starting with "Add a glass". What must it contain?

CODE · READ-ALONG

A Home Screen widget

A widget is a configuration plus a SwiftUI view for each moment in a timeline. The `WaterProvider` referenced here, defined in another file, supplies dated entries, and WidgetKit renders each one ahead of time. Your code is not running while the widget sits on the Home Screen.

```swift
import WidgetKit
import SwiftUI

struct WaterWidget: Widget {
    var body: some WidgetConfiguration {
        StaticConfiguration(
            kind: "WaterWidget",
            provider: WaterProvider()
        ) { entry in
            VStack {
                Image(systemName: "drop.fill")
                Text("\(entry.glasses) glasses")
            }
            .containerBackground(.blue.gradient, for: .widget)
        }
        .configurationDisplayName("Water Today")
        .description("Shows how many glasses you logged.")
        .supportedFamilies([.systemSmall, .accessoryCircular])
    }
}
```

Line by line

1 `import WidgetKit`

WHO	The Swift compiler, while it builds the widget extension target.
WHAT	**Brings in WidgetKit: Widget, StaticConfiguration and the widget families.**
WHERE	The top of a file in the widget extension target, not the app target.
WHEN	At compile time, before WaterWidget is type-checked.
WHY	Widgets are an extension, a separate program with its own target and imports.

2 `import SwiftUI`

WHO	The Swift compiler, building the same widget file.
WHAT	**Brings in SwiftUI for the widget's views.**
WHERE	The second import of the file.
WHEN	At compile time, right after WidgetKit is imported.
WHY	Widget content is always drawn with SwiftUI.

4 `struct WaterWidget: Widget {`

WHO	Your code declares it; WidgetKit reads it.
WHAT	**Declares a widget type.**
WHERE	A top-level struct in the widget extension.
WHEN	Read when someone adds the widget, and when WidgetKit renders it.
WHY	Like App and View, a widget is described declaratively with a body.

5 `var body: some WidgetConfiguration {`

WHO	WidgetKit reads it when it loads the extension.
WHAT	**Returns the widget's configuration rather than a view.**
WHERE	The Widget protocol's required property.
WHEN	When the system loads the extension.
WHY	The configuration bundles identity, data source, view and supported sizes in one place.

```
6  StaticConfiguration(
```

WHO — WidgetKit, which builds the widget from this configuration.
WHAT — **Creates a widget with no options for the user to configure.**
WHERE — The root of body.
WHEN — When the extension loads.
WHY — A static configuration is the simplest kind; an AppIntentConfiguration would let users pick options.

```
7      kind: "WaterWidget",
```

WHO — WidgetKit, and your app when it asks for a reload.
WHAT — **Gives the widget a stable identifier.**
WHERE — The first argument of StaticConfiguration.
WHEN — Used whenever the widget is reloaded or looked up.
WHY — Your app can call WidgetCenter.shared.reloadTimelines(ofKind: "WaterWidget") after logging water to refresh it right away.

```
8      provider: WaterProvider()
```

WHO — WidgetKit asks this provider for entries.
WHAT — **Supplies the timeline of dated WaterEntry values.**
WHERE — The second argument of StaticConfiguration.
WHEN — Called when the widget is added and whenever the timeline expires or is reloaded.
WHY — Separating data (the provider) from appearance (the closure below) lets WidgetKit render entries ahead of time.

```
9  ) { entry in
```

WHO — WidgetKit calls the closure for each entry.
WHAT — **Opens the trailing closure that turns one timeline entry into a view.**
WHERE — The content closure of StaticConfiguration.
WHEN — Once per entry, ahead of the moment it will be shown.
WHY — Each entry becomes a snapshot, which is why widgets cost so little battery. Module 02 (Swift Fundamentals) explains closures in full; for now read { entry in as "a block of code that receives one entry".

```
10     VStack {
```

WHO — SwiftUI's VStack layout, inside the widget's content closure.
WHAT — **Stacks the icon above the count.**
WHERE — The root view of the widget's content.
WHEN — Rendered for each entry.
WHY — Simple stacks render predictably at every widget size.

```
11         Image(systemName: "drop.fill")
```

WHO — SwiftUI, with SF Symbols.
WHAT — **Shows a filled water drop.**
WHERE — First child of the VStack.
WHEN — Rendered for each entry.
WHY — An icon identifies the widget at a glance on a busy Home Screen.

```
12         Text("\(entry.glasses) glasses")
```

WHO — SwiftUI's Text view, reading one timeline entry.
WHAT — **Shows the count from this entry, such as "3 glasses".**
WHERE — Second child of the VStack.
WHEN — Rendered once per entry, not continuously.
WHY — The view reads only the entry, so a snapshot can be drawn ahead of time.

```
14  .containerBackground(.blue.gradient, for: .widget)
```

WHO — WidgetKit and SwiftUI together, drawing the widget's background.
WHAT — **Sets the widget's background to a blue gradient.**
WHERE — Applies to the content view inside the widget.
WHEN — Rendered with each entry.
WHY — Declaring the background this way lets the system replace it where needed, such as on the Lock Screen or in StandBy.

```
16  .configurationDisplayName("Water Today")
```

WHO — The widget gallery, the picker people open when adding widgets.
WHAT — **Names the widget in the gallery people browse when adding widgets.**
WHERE — A modifier on the configuration.
WHEN — Shown while choosing a widget.
WHY — A clear name helps people find the right widget among many apps.

```
17  .description("Shows how many glasses you logged.")
```

WHO — The widget gallery, below the display name.
WHAT — **Explains in one sentence what the widget shows.**
WHERE — A modifier on the configuration.
WHEN — Shown while choosing a widget.
WHY — People decide in seconds; one sentence of purpose is enough.

```
18  .supportedFamilies([.systemSmall, .accessoryCircular])
```

WHO — WidgetKit, which offers only the sizes named here.
WHAT — **Offers a small Home Screen size and a circular Lock Screen size.**
WHERE — The last modifier on the configuration.
WHEN — Read when the gallery lists sizes.
WHY — Listing only sizes you have designed for avoids cramped or empty layouts.

> **Try it**
>
> 1. Which line would you change to also offer a medium Home Screen widget?

INTERACTIVE

Match the idea to the app type

Read each product idea and choose the kind of app or extension that fits it best. The feedback explains the match, names the framework you would use, and says why the most tempting wrong answer falls short. There are ten ideas, shuffled each time you start over.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. A widget shows "Next watering: fern, tomorrow." How does it stay up to date on the Home Screen?
   A. Your SwiftUI code keeps running on the Home Screen to redraw it
   B. Your provider supplies dated entries; WidgetKit shows each on time
   C. The person has to open the app, which refreshes every widget
   D. It reloads itself every second, like a view with a timer

2. A ride-sharing app wants to show the car's approach, updating as it gets closer, on the Lock Screen and in the Dynamic Island. Which feature fits?
   A. A Home Screen widget
   B. An App Clip
   C. A Live Activity
   D. A share extension

3. Why must every App Shortcut phrase include the `.applicationName` placeholder?
   A. So Siri knows which app handles the request
   B. So the app name is translated into the user's language
   C. It is optional decoration that makes the phrase read well
   D. So the app name is passed into perform() as a parameter

4. Your share extension loads the user's entire photo library into memory and keeps getting terminated. What is the most likely reason?
   A. Share extensions are not allowed to read photos at all
   B. The app is not signed with a valid identity
   C. The host app crashed and took the extension with it
   D. Extensions run under much tighter memory limits than apps

Answers: Kinds of Apps and Extensions

1. **B. Your provider supplies dated entries; WidgetKit shows each on time** Widgets are snapshots drawn from a timeline. Your app can ask for a reload after its data changes, but no widget code runs between entries, which is why widgets barely touch the battery.
2. **C. A Live Activity** Live Activities show an event in progress on the Lock Screen and in the Dynamic Island, and can be updated as it changes. A widget shows glanceable information but is not designed for a live, short-lived event.
3. **A. So Siri knows which app handles the request** Many apps could offer "log water." The app name tells Siri which one you mean. The placeholder does not pass any value to your code.
4. **D. Extensions run under much tighter memory limits than apps** Extensions run in their own process with tight memory limits, and the system terminates one that exceeds them. Load only what was shared and do heavy work in the main app.

Swift Beyond the iPhone

CONCEPT

Swift is open source

Apple released Swift as open source in 2015, a year after introducing it. Today the language is developed in public at swift.org, and it runs well beyond Apple devices.

Servers	Back ends on Linux, built with frameworks such as Vapor and Hummingbird, can share model code with the iOS app.
Linux and Windows	Official toolchains build command-line tools and services.
WebAssembly	Swift 6.2 compiles to WebAssembly, a portable binary format that web browsers run at close to native speed.
Embedded Swift	A compact compilation mode for microcontrollers and other very small targets.

> **From Objective-C to Swift**
>
> Apple introduced Swift in 2014 to succeed Objective-C, a language from the 1980s built on top of C. Many system frameworks still contain Objective-C, and Swift calls into it seamlessly.

CONCEPT

This page runs compiled Swift

Not everything on this page is simulated. A small Swift file, `SwiftCore.swift`, was compiled in Embedded Swift mode into a WebAssembly module, and your browser downloaded it with the page. When a widget checks a color's contrast or measures the distance between two map points, it calls those Swift functions directly.

The race beside this text times the same prime-counting job in that compiled Swift and in other ways of running code on this page. Compiled code does its analysis once, ahead of time, instead of while it runs.

Interactive figure in the HTML version: swift wasm race.

CODE · READ-ALONG

A Swift function the browser calls

These are the exact lines of `contrastRatio` from SwiftCore.swift. Widgets in the design lessons pass two colors and get back the WCAG contrast ratio, computed by Swift running as WebAssembly. Two underscored attributes, unusual in app code, export the function to JavaScript.

```swift
/// WCAG contrast ratio between two colors: 1.0 (none) to 21.0 (black on white).
@_expose(wasm, "contrastRatio")
@_cdecl("contrastRatio")
public func contrastRatio(_ foreground: Int32, _ background: Int32) -> Double {
    let l1 = relativeLuminance(foreground)
    let l2 = relativeLuminance(background)
    return (max(l1, l2) + 0.05) / (min(l1, l2) + 0.05)
}
```

Line by line

1 `/// WCAG contrast ratio between two colors: 1.0 (none) to 21.0 (black on white).`

- **WHO** Xcode's Quick Help and documentation tools read it; the compiler skips it.
- **WHAT** **Documents what the function returns and the range of possible values.**
- **WHERE** Directly above the declaration it describes.
- **WHEN** Never executed; read when someone Option-clicks the function in Xcode or generates documentation.
- **WHY** Three slashes make a documentation comment, so the 1-to-21 range appears wherever the function is used.

2 `@_expose(wasm, "contrastRatio")`

- **WHO** The Swift compiler, when it targets WebAssembly.
- **WHAT** **Exports the function from the WebAssembly module under the name contrastRatio, so JavaScript can call it.**
- **WHERE** An attribute on the declaration below.
- **WHEN** At compile time; it adds an entry to the module's export table.
- **WHY** Module functions are hidden from JavaScript unless exported. The leading underscore marks an attribute that is not yet a stable, official part of the language.

3 `@_cdecl("contrastRatio")`

- **WHO** The Swift compiler, when it emits the function's symbol.
- **WHAT** **Gives the function a plain C symbol name and C calling convention.**
- **WHERE** A second attribute on the same declaration.
- **WHEN** At compile time, as the function's name and calling convention are fixed.
- **WHY** Swift normally mangles names and uses its own calling convention; a C-style function with simple number types is what WebAssembly hosts expect.

4 `public func contrastRatio(_ foreground: Int32, _ background: Int32) -> Double {`

- **WHO** JavaScript on this page calls it through the WebAssembly export.
- **WHAT** **Declares a function that takes two colors packed as 0xRRGGBB integers and returns the ratio as a Double.**
- **WHERE** A public, top-level function in SwiftCore.swift.
- **WHEN** Each time a widget asks for a contrast ratio, such as when you drag a color slider.
- **WHY** Int32 and Double map directly onto WebAssembly's 32-bit integer and 64-bit float types, so no conversion layer sits between JavaScript and Swift.

5 `let l1 = relativeLuminance(foreground)`

- **WHO** Swift code inside the WebAssembly module.
- **WHAT** **Computes how bright the foreground color appears, from 0 for black to 1 for white.**
- **WHERE** A local constant inside the function.
- **WHEN** On every call from JavaScript, before l2 is computed.
- **WHY** Contrast depends on perceived brightness, not raw RGB numbers; the WCAG formula weights green most because human eyes are most sensitive to it.

```
6  let l2 = relativeLuminance(background)
```

WHO Swift code inside the module.
WHAT **Computes the background's relative luminance the same way.**
WHERE A second local constant.
WHEN On every call, right after the first.
WHY Computing both values first lets the next line order them without caring which color is lighter.

```
7  return (max(l1, l2) + 0.05) / (min(l1, l2) + 0.05)
```

WHO Swift's floating-point arithmetic, running inside the WebAssembly module.
WHAT **Divides the lighter luminance by the darker one, each plus 0.05, giving a ratio from 1 to 21.**
WHERE The last line of the function; the result goes back to JavaScript as a number.
WHEN On every call, after both luminance values are ready.
WHY max and min make the argument order irrelevant, and the 0.05 term, part of the WCAG definition, also prevents dividing by zero for pure black.

> **Try it**
>
> 1. Predict: does contrastRatio(0xFFFFFF, 0x000000) return the same value as contrastRatio(0x000000, 0xFFFFFF)? Which line guarantees it?

CODE · RUNS IN THE BROWSER

Run the same math yourself

This is the same contrast formula as plain Swift, with each color given as three 0-255 channels. Run it in the console. Gray #767676 on white is the darkest gray that still passes the 4.5:1 minimum for normal text.

```swift
import Foundation

func linear(_ channel: Int) -> Double {
    let c = Double(channel) / 255.0
    return c <= 0.04045 ? c / 12.92 : pow((c + 0.055) / 1.055, 2.4)
}

func luminance(_ r: Int, _ g: Int, _ b: Int) -> Double {
    0.2126 * linear(r) + 0.7152 * linear(g) + 0.0722 * linear(b)
}

func contrast(_ a: Double, _ b: Double) -> Double {
    (max(a, b) + 0.05) / (min(a, b) + 0.05)
}

let white = luminance(255, 255, 255)
let gray = luminance(118, 118, 118)
let black = luminance(0, 0, 0)
print("Gray on white:", String(format: "%.2f", contrast(gray, white)))
print("Black on white:", String(format: "%.2f", contrast(black, white)))
```

OUTPUT

```
Gray on white: 4.54
Black on white: 21.00
```

Line by line

```
1  import Foundation
```

WHO	The Swift compiler, while it builds this program.
WHAT	**Brings in Foundation, which provides pow and String(format:).**
WHERE	The first line of the program.
WHEN	At compile time, before pow or String(format:) is looked up.
WHY	pow and formatted strings come from Foundation, not from the core language.

```
3  func linear(_ channel: Int) -> Double {
```

WHO	Your program defines it; luminance calls it.
WHAT	**Declares a function that converts one 0-255 color channel to linear light.**
WHERE	A top-level function, defined before luminance so that function can call it.
WHEN	Called three times per color.
WHY	Screens store color with a gamma curve; brightness math needs linear values.

```
4  let c = Double(channel) / 255.0
```

WHO	Swift's floating-point division, inside linear.
WHAT	**Scales the channel from 0-255 down to 0-1.**
WHERE	A local constant inside linear.
WHEN	On each call to linear, before the gamma expression below.
WHY	Converting to Double first avoids integer division, which would round everything to 0 or 1.

```
5  return c <= 0.04045 ? c / 12.92 : pow((c + 0.055) / 1.055, 2.4)
```

WHO	Foundation's pow and the ternary operator.
WHAT	**Undoes the sRGB gamma curve: a straight line for very dark values, a 2.4 power curve for the rest.**
WHERE	The result of linear.
WHEN	On each call to linear, after c is computed.
WHY	These constants come straight from the sRGB standard, so the result matches the WCAG definition.

```
8  func luminance(_ r: Int, _ g: Int, _ b: Int) -> Double {
```

WHO	You, the program author, defining luminance.
WHAT	**Declares a function that turns red, green and blue channels into a single brightness value.**
WHERE	A top-level function, between linear and contrast.
WHEN	Called once per color.
WHY	Unlabeled parameters keep calls short, like luminance(255, 255, 255).

```
9  0.2126 * linear(r) + 0.7152 * linear(g) + 0.0722 * linear(b)
```

WHO	Swift arithmetic, calling linear three times.
WHAT	**Weights each channel by how sensitive the eye is to it and adds them up.**
WHERE	The single expression in the function, returned implicitly.
WHEN	On each call to luminance, calling linear three times.
WHY	Green contributes most and blue least, matching human vision.

```
12 func contrast(_ a: Double, _ b: Double) -> Double {
```

WHO	You, the program author, defining contrast.
WHAT	**Declares the contrast ratio between two luminance values.**
WHERE	A top-level function, the last one before the constants.
WHEN	Called once per comparison.
WHY	Taking luminance values keeps this function independent of how colors are stored.

```
13  (max(a, b) + 0.05) / (min(a, b) + 0.05)
```

WHO Swift's floating-point arithmetic, with max and min from the standard library.
WHAT **Divides the lighter value by the darker one, each plus 0.05.**
WHERE The implicit return of contrast.
WHEN On each call to contrast, as its single expression.
WHY Identical to the WebAssembly version, so both always agree.

```
16  let white = luminance(255, 255, 255)
```

WHO Your program's top-level code.
WHAT **Computes the luminance of pure white, which is 1.0.**
WHERE A global constant, the first of three colors.
WHEN When the program starts, in order from top to bottom.
WHY Naming each color makes the comparisons below read clearly.

```
17  let gray = luminance(118, 118, 118)
```

WHO Top-level code, calling luminance with the gray channels.
WHAT **Computes the luminance of the gray #767676 (118 is 0x76).**
WHERE A global constant, the second of three colors.
WHEN Next, as the program runs.
WHY This gray sits just above the accessibility threshold for normal text.

```
18  let black = luminance(0, 0, 0)
```

WHO Top-level code, calling luminance with zeros.
WHAT **Computes the luminance of pure black, which is 0.**
WHERE A global constant, the last of the three.
WHEN Right after gray, as the program runs from top to bottom.
WHY Black on white is the maximum possible contrast, a useful sanity check.

```
19  print("Gray on white:", String(format: "%.2f", contrast(gray, white)))
```

WHO print, writing to standard output.
WHAT **Prints the gray-on-white ratio rounded to two decimal places.**
WHERE Top-level code, after the three constants.
WHEN After the constants are computed.
WHY print separates its items with a space, and %.2f rounds to two decimals, so the output reads cleanly.

```
20  print("Black on white:", String(format: "%.2f", contrast(black, white)))
```

WHO print, writing to standard output.
WHAT **Prints the black-on-white ratio.**
WHERE The last line of the program.
WHEN Last, after the gray line has printed.
WHY It should print the maximum, 21.00, confirming the formula.

Try it

1. Try a lighter gray: change 118 to 130 everywhere it appears. Does it still reach 4.5?
2. Compare white with itself. What is the lowest ratio the formula can produce?

CHECK YOURSELF

Check your understanding

1. Why does compiled Swift in WebAssembly usually beat an interpreter running the same algorithm?
   A. WebAssembly hands the arithmetic to the graphics card instead of the CPU
   B. The code was translated ahead of time, so nothing is interpreted as it runs
   C. Interpreters cannot run loops, so they fall back to slower code
   D. The browser caches the interpreter's answers between runs
2. What does `contrastRatio(0x767676, 0xFFFFFF)` return compared with `contrastRatio(0xFFFFFF, 0x767676)`?
   A. The second is the reciprocal of the first
   B. The second is negative
   C. The same value, about 4.54
   D. The second always returns 1.0
3. Your team wants the iOS app and its server to use the same Swift model types. What makes that possible?
   A. Rewriting the models in JavaScript for the server
   B. Copying the model files by hand before each release
   C. Running the server code on a spare iPhone
   D. Swift on Linux, with the models in a package both depend on
4. Why does `contrastRatio` take `Int32` parameters instead of a Swift `Color`?
   A. Int32 maps onto a WebAssembly number type JavaScript can pass
   B. Color cannot be marked public, so it cannot be exported to JavaScript
   C. Int32 is the fastest type for every kind of arithmetic
   D. Int32 uses less memory, which keeps the module tiny

Answers: Swift Beyond the iPhone

1. **B. The code was translated ahead of time, so nothing is interpreted as it runs** An interpreter reads and dispatches each operation while the program runs. A compiler does that work once, ahead of time, producing low-level instructions the browser runs directly. Both run on the CPU; no graphics card is involved, and interpreters run loops fine, just more slowly.
2. **C. The same value, about 4.54** The function puts the larger luminance on top with max and the smaller on the bottom with min, so argument order never matters. Without that, swapping the colors would give a reciprocal below 1.
3. **D. Swift on Linux, with the models in a package both depend on** Swift runs on Linux servers, so a shared package can hold the models for both. Copying files by hand works once, then drifts out of sync.
4. **A. Int32 maps onto a WebAssembly number type JavaScript can pass** WebAssembly functions exchange plain numbers. A packed 0xRRGGBB integer crosses the boundary with no conversion, while a Swift Color has no representation JavaScript understands, whatever its access level. Speed and size are not the reason.

Worked Examples

WORKED EXAMPLE

Press Run: follow one build from plan to launch

Problem.
Pressing Run in Xcode carries one tiny app, HelloWorld, through six stages. Follow a console model of that journey: each frame grows the program by one stage line, and the diagram advances beside it until the app launches and greets.

Frame 1 of 6

Plan stage: the whole program is one print line naming the app and its destination, the iPhone 17 Pro Simulator. The diagram marks Plan as the active stage; nothing has been compiled yet.

```
  print("Plan: build HelloWorld for iPhone 17 Pro Simulator")
```

Frame 2 of 6

Swift compiler stage: a greeting function joins the program and a compile line prints. The function returns the text the app would show, and the last line prints it after the stage line.

```
+ func greeting() -> String {
+     return "Hello, world!"
+ }
+ print("Plan: build HelloWorld for iPhone 17 Pro Simulator")
+ print("Compile: ContentView.swift checked")
+ print(greeting())
```

Frame 3 of 6

LLVM stage: one more stage line prints, reporting the object file for ContentView. The greeting still prints last, unchanged, because later stages only transform what the compiler already checked.

```
  func greeting() -> String {
      return "Hello, world!"
  }
  print("Plan: build HelloWorld for iPhone 17 Pro Simulator")
  print("Compile: ContentView.swift checked")
+ print("LLVM: object file ContentView.o ready")
+ print(greeting())
```

Frame 4 of 6

Link stage: the log adds the assembled app bundle, HelloWorld.app. The diagram advances to Link; the program itself only narrates the stage, it does not really link anything.

```
    func greeting() -> String {
        return "Hello, world!"
    }
    print("Plan: build HelloWorld for iPhone 17 Pro Simulator")
    print("Compile: ContentView.swift checked")
    print("LLVM: object file ContentView.o ready")
+   print("Link: HelloWorld.app assembled")
+   print(greeting())
```

Frame 5 of 6

Install stage: the log reports the app installed on the iPhone 17 Pro Simulator, the destination named since the Plan frame. One stage remains before the app can launch.

```
    func greeting() -> String {
        return "Hello, world!"
    }
    print("Plan: build HelloWorld for iPhone 17 Pro Simulator")
    print("Compile: ContentView.swift checked")
    print("LLVM: object file ContentView.o ready")
    print("Link: HelloWorld.app assembled")
+   print("Install: HelloWorld on iPhone 17 Pro (Simulator)")
+   print(greeting())
```

Frame 6 of 6

Launch stage: the final stage line prints, then the greeting itself, exactly what the launched app would show. The diagram sits at Launch with every earlier stage marked done.

```
    func greeting() -> String {
        return "Hello, world!"
    }
    print("Plan: build HelloWorld for iPhone 17 Pro Simulator")
    print("Compile: ContentView.swift checked")
    print("LLVM: object file ContentView.o ready")
    print("Link: HelloWorld.app assembled")
    print("Install: HelloWorld on iPhone 17 Pro (Simulator)")
+   print("Launch: ContentView appeared")
+   print(greeting())
```

Line by line: the finished program

```
func greeting() -> String {
```

WHO	Your code declares it; the console model calls it below.
WHAT	**Declares greeting, a function that returns the greeting text the app would show.**
WHERE	At the top of the program, before the stage lines that use it.
WHEN	Compiled first; the function runs only when the last line calls it.
WHY	Keeping the greeting in one function means the Launch stage prints exactly what the app shows.

```
2   return "Hello, world!"
```

WHO The greeting function, when the last line calls it.
WHAT **Returns the fixed greeting text, Hello, world!, to the caller.**
WHERE The only line inside greeting.
WHEN At the moment print(greeting()) runs, at the end of the program.
WHY Returning the text, instead of printing inside the function, lets the caller decide when it appears.

```
4   print("Plan: build HelloWorld for iPhone 17 Pro Simulator")
```

WHO Your console model, narrating the first pipeline stage.
WHAT **Prints the Plan stage line, naming the app and the run destination.**
WHERE The first line of the program in frame 1; later frames keep it as the opening line of the log.
WHEN First, before any other stage line prints.
WHY Every Run starts with a plan: Xcode reads the scheme and target before compiling anything.

```
5   print("Compile: ContentView.swift checked")
```

WHO Your console model, narrating the compiler stage.
WHAT **Prints that ContentView.swift passed the Swift compiler stage.**
WHERE After the greeting function, as the first stage line of the log.
WHEN After the Plan line, in pipeline order.
WHY The Swift compiler checks names and types before any machine code exists.

```
6   print("LLVM: object file ContentView.o ready")
```

WHO Your console model, narrating the LLVM stage.
WHAT **Prints that LLVM produced the ContentView.o object file.**
WHERE The second stage line of the log, under the compile line.
WHEN After compiling, in pipeline order.
WHY LLVM turns the checked Swift code into machine code, one object file per source file.

```
7   print("Link: HelloWorld.app assembled")
```

WHO Your console model, narrating the Link stage.
WHAT **Prints that the linker assembled the object files into HelloWorld.app.**
WHERE The third stage line of the log.
WHEN After LLVM, in pipeline order.
WHY Linking joins the object files into one app bundle the system can install.

```
8   print("Install: HelloWorld on iPhone 17 Pro (Simulator)")
```

WHO Your console model, narrating the Install stage.
WHAT **Prints that the app was installed on the simulated iPhone destination.**
WHERE The fourth stage line of the log.
WHEN After linking, in pipeline order.
WHY Install copies the signed bundle into the destination, here the iPhone 17 Pro Simulator.

```
9   print("Launch: ContentView appeared")
```

WHO Your console model, narrating the Launch stage.
WHAT **Prints that the installed app launched and its first view appeared.**
WHERE The last stage line, just before the greeting itself prints.
WHEN After install, at the end of the pipeline.
WHY Launch is the payoff stage: the system starts the process and SwiftUI shows ContentView.

```
10 print(greeting())
```

WHO The print function, calling greeting() for its text.
WHAT **Prints the greeting the app itself would show on screen.**
WHERE The last line of the program.
WHEN Last, after every stage line.
WHY Ending with the app output proves the whole pipeline delivered the one thing the user sees.

Try it

The app gains a second file, SettingsView.swift. The starter plans the build and compiles ContentView, then prints the greeting. Add the compile line for SettingsView.swift between the ContentView line and the greeting, so both files are checked before the app output.

```
print("Plan: build HelloWorld for iPhone 17 Pro Simulator")
print("Compile: ContentView.swift checked")
print("Hello, world!")
```

EXPECTED OUTPUT

```
Plan: build HelloWorld for iPhone 17 Pro Simulator
Compile: ContentView.swift checked
Compile: SettingsView.swift checked
Hello, world!
```

WORKED EXAMPLE

Diagnose and repair a compiler error

Problem.

A greeting program fails to build, and the error names a function that looks almost right. Diagnose the failure from the compiler message, repair it with the smallest change, then grow the fixed program with a parameter and a loop.

Frame 1 of 6

The program starts with just the function: greet takes a name and prints a greeting for it. Nothing calls it yet, so this frame runs and prints nothing at all.

```
func greet(_ name: String) {
    print("Hello, \(name)!")
}
```

OUTPUT SO FAR

Frame 2 of 6

A call site appears, but it calls greett with an extra t. The compiler cannot find that name, so the build stops here: this frame is the failure to diagnose.

This frame is unfinished on purpose: The call site misspells greet as greett, so the compiler stops the build.

```
    func greet(_ name: String) {
        print("Hello, \(name)!")
    }
+   greett("Ana")
```

ERROR (ON PURPOSE)

```
cannot find 'greett' in scope
```

Frame 3 of 6

The repair is one letter: the call now spells greet exactly as declared. The program runs and prints the first greeting, Hello, Ana!, proving the diagnosis was right.

```
    func greet(_ name: String) {
        print("Hello, \(name)!")
    }
+   greet("Ana")
```

OUTPUT SO FAR

```
Hello, Ana!
```

Frame 4 of 6

The working program grows: greet gains a day parameter, and the call passes Friday with its label. The greeting now uses both pieces of information the caller supplies.

```
+   func greet(_ name: String, day: String) {
+       print("Hello, \(name)! Happy \(day).")
    }
+   greet("Ana", day: "Friday")
```

OUTPUT SO FAR

```
Hello, Ana! Happy Friday.
```

Frame 5 of 6

Three names move into an array and a loop calls greet for each one. The output triples without tripling the code: one call site now serves the whole list.

```
    func greet(_ name: String, day: String) {
        print("Hello, \(name)! Happy \(day).")
    }
+   let friends = ["Ana", "Ben", "Cleo"]
+   for friend in friends {
+       greet(friend, day: "Friday")
+   }
```

OUTPUT SO FAR

```
Hello, Ana! Happy Friday.
Hello, Ben! Happy Friday.
Hello, Cleo! Happy Friday.
```

Frame 6 of 6

A final line counts the greetings from the array itself and prints the total. The finished program greets three friends and reports exactly how many it greeted.

```
1    func greet(_ name: String, day: String) {
2        print("Hello, \(name)! Happy \(day).")
3    }
4    let friends = ["Ana", "Ben", "Cleo"]
5    for friend in friends {
6        greet(friend, day: "Friday")
7    }
8  + print("Greeted \(friends.count) friends")
```

OUTPUT SO FAR

```
Hello, Ana! Happy Friday.
Hello, Ben! Happy Friday.
Hello, Cleo! Happy Friday.
Greeted 3 friends
```

FINAL RESULT — CHECKED OUTPUT

```
Hello, Ana! Happy Friday.
Hello, Ben! Happy Friday.
Hello, Cleo! Happy Friday.
Greeted 3 friends
```

Line by line: the finished program

1 `func greet(_ name: String, day: String) {`

WHO	You declare it; the call sites below run it.
WHAT	**Declares greet with two parameters: a name and a day, so the greeting can wish a happy day.**
WHERE	At the top of the program, replacing the one-parameter version.
WHEN	Compiled first; the body runs once per call.
WHY	A second parameter carries the day in, instead of the function guessing or hard-coding it.

2 `print("Hello, \(name)! Happy \(day).")`

WHO	The greet function, each time a call runs it.
WHAT	**Prints the greeting with both parameters interpolated: name first, then the day.**
WHERE	The only line inside greet.
WHEN	Once per call, at the moment of the call.
WHY	Both facts come from the caller, so the same body greets anyone on any day.

```swift
4  let friends = ["Ana", "Ben", "Cleo"]
```

WHO Your top-level code; the loop below reads it.
WHAT **Stores three names in an array, in greeting order.**
WHERE Under the function, before the loop.
WHEN At run time, before the loop starts.
WHY An array turns three separate calls into one loop the compiler can check.

```swift
5  for friend in friends {
```

WHO The for-in loop, visiting each name in friends.
WHAT **Starts a loop that binds each name in friends to friend, one at a time.**
WHERE Top-level code, under the array.
WHEN At run time; the body runs three times, once per name.
WHY Looping over the array means adding a name later adds a greeting with no new code.

```swift
6  greet(friend, day: "Friday")
```

WHO The loop body, calling greet once per name.
WHAT **Calls greet for the current friend, always on Friday.**
WHERE Inside the loop, indented under it.
WHEN Three times, once for each name in friends.
WHY The day stays fixed here; only the name changes per visit, which is what the loop is for.

```swift
8  print("Greeted \(friends.count) friends")
```

WHO Your top-level code, reading the array after the loop.
WHAT **Prints how many friends were greeted, counted from the array itself.**
WHERE The last line of the program, after the loop.
WHEN After the loop finishes all three greetings.
WHY Counting from friends.count keeps the summary honest if the list grows.

Try it

The starter greets only Ana. Add two more calls so the program also greets Ben and Cleo, in that order, reusing the same greet function for each name.

```swift
func greet(_ name: String) {
    print("Hello, \(name)!")
}
greet("Ana")
```

EXPECTED OUTPUT

```
Hello, Ana!
Hello, Ben!
Hello, Cleo!
```

WORKED EXAMPLE

Follow an app through its day: launch, work, background, gone

Problem.

Model one day in an app life: launch, two edits, a trip to the background with an autosave, a deferred edit, suspension, and a return that applies what waited. The diagram tracks memory and disk beside the console log.

Frame 1 of 7

Not running: the model declares its phase enum and counters, starts at notRunning, and prints the opening line. The diagram shows the app absent from memory and disk alike.

```
    enum ScenePhase {
        case notRunning, inactive, active, background, suspended
    }
    var phase = ScenePhase.notRunning
    var edits = 0
    var pending: [String] = []
    print("Phase: not running")
```

Frame 2 of 7

Launch begins: phase moves to inactive and the log adds its line. The app is loaded and on screen, but not yet receiving touches, so no edits happen yet.

```
    enum ScenePhase {
        case notRunning, inactive, active, background, suspended
    }
    var phase = ScenePhase.notRunning
    var edits = 0
    var pending: [String] = []
    print("Phase: not running")
+   phase = .inactive
+   print("Phase: inactive")
```

Frame 3 of 7

Active: two edits accumulate in the counter and the log prints the count. The diagram shows the draft living only in memory; nothing has reached the disk yet.

```
    enum ScenePhase {
        case notRunning, inactive, active, background, suspended
    }
    var phase = ScenePhase.notRunning
    var edits = 0
    var pending: [String] = []
    print("Phase: not running")
    phase = .inactive
    print("Phase: inactive")
+   phase = .active
+   edits += 1
+   edits += 1
+   print("Active: \(edits) unsaved edits")
```

Frame 4 of 7

Background: the user leaves, and the model autosaves at exactly this moment, printing that both edits are saved. The diagram now shows the saved state on disk.

```
   enum ScenePhase {
       case notRunning, inactive, active, background, suspended
   }
   var phase = ScenePhase.notRunning
   var edits = 0
   var pending: [String] = []
   print("Phase: not running")
   phase = .inactive
   print("Phase: inactive")
   phase = .active
   edits += 1
   edits += 1
   print("Active: \(edits) unsaved edits")
+  phase = .background
+  print("Saved \(edits) edits")
```

Frame 5 of 7

Still in Background, an edit arrives: the phase check defers it into the pending list and the log says it will apply on return, instead of editing now.

```
   enum ScenePhase {
       case notRunning, inactive, active, background, suspended
   }
   var phase = ScenePhase.notRunning
   var edits = 0
   var pending: [String] = []
   print("Phase: not running")
   phase = .inactive
   print("Phase: inactive")
   phase = .active
   edits += 1
   edits += 1
   print("Active: \(edits) unsaved edits")
   phase = .background
   print("Saved \(edits) edits")
+  if phase == .background {
+      pending.append("Fix title")
+      print("Deferred: Fix title until active")
+  }
```

Frame 6 of 7

Suspended: the system freezes the app in memory. The log prints its suspended line, and after this frame no model code runs until the user returns.

```
   enum ScenePhase {
       case notRunning, inactive, active, background, suspended
   }
   var phase = ScenePhase.notRunning
   var edits = 0
   var pending: [String] = []
   print("Phase: not running")
   phase = .inactive
   print("Phase: inactive")
   phase = .active
   edits += 1
   edits += 1
   print("Active: \(edits) unsaved edits")
   phase = .background
   print("Saved \(edits) edits")
   if phase == .background {
       pending.append("Fix title")
       print("Deferred: Fix title until active")
   }
+ phase = .suspended
+ print("Phase: suspended")
```

Frame 7 of 7

Active again: the loop drains the pending list and prints the title fix being applied. The day ends with every edit either saved or applied, none lost.

```
   enum ScenePhase {
       case notRunning, inactive, active, background, suspended
   }
   var phase = ScenePhase.notRunning
   var edits = 0
   var pending: [String] = []
   print("Phase: not running")
   phase = .inactive
   print("Phase: inactive")
   phase = .active
   edits += 1
   edits += 1
   print("Active: \(edits) unsaved edits")
   phase = .background
   print("Saved \(edits) edits")
   if phase == .background {
       pending.append("Fix title")
       print("Deferred: Fix title until active")
   }
   phase = .suspended
   print("Phase: suspended")
+ phase = .active
+ for item in pending { print("Applied on return: \(item)") }
```

Line by line: the finished program

```
1  enum ScenePhase {
```

WHO You declare it; the program compares phase against its cases.
WHAT **Declares ScenePhase, an enum naming the five lifecycle states from this module.**
WHERE At the top of the program, before the variables that use it.
WHEN Compiled first; declaring it runs no code.
WHY An enum makes the phase one of five known values, never a misspelled string.

```
2  case notRunning, inactive, active, background, suspended
```

WHO The ScenePhase type, listing its possibilities.
WHAT **Names the five cases in day order: notRunning, inactive, active, background, suspended.**
WHERE The only line inside the enum.
WHEN Compile time; the list is fixed for the whole program.
WHY Grouping the cases on one line keeps this small model compact.

```
4  var phase = ScenePhase.notRunning
```

WHO Your top-level code; later lines reassign it.
WHAT **Starts phase at notRunning: the app has not launched yet.**
WHERE The first variable of the model, under the enum.
WHEN First, before any transition line runs.
WHY Every day in this model starts from not running, matching a fresh launch.

```
5  var edits = 0
```

WHO Your top-level code; the Active stage adds to it.
WHAT **Starts the unsaved-edit counter at zero.**
WHERE Under phase, as part of the model state.
WHEN At the start, before the app becomes active.
WHY Counting edits in the model, not in a view, lets the Background stage save them.

```
6  var pending: [String] = []
```

WHO Your top-level code; the Background stage appends to it.
WHAT **Starts the pending list empty: edits waiting for the app to return to Active.**
WHERE Under edits, as part of the model state.
WHEN At the start, before anything is deferred.
WHY A list, not a flag, so several deferred edits could queue in order.

```
7  print("Phase: not running")
```

WHO The print function, narrating the model.
WHAT **Prints the opening phase line: the app is not running.**
WHERE The first action line of the script.
WHEN First, while phase is still notRunning.
WHY Each stage prints as it happens, so the console reads as the day log.

```
8  phase = .inactive
```

WHO Your top-level code, moving the model.
WHAT **Moves phase to inactive: the app is launching, on screen but not yet interactive.**
WHERE After the not-running line.
WHEN Second, at the start of the launch.
WHY Launch passes through Inactive briefly before the app can receive touches.

```
9  print("Phase: inactive")
```

WHO	The print function, narrating the model.
WHAT	**Prints the inactive phase line as the app finishes launching.**
WHERE	Right after the move to inactive.
WHEN	While phase is inactive.
WHY	The log line matches the diagram phase beside it, frame by frame.

```
10 phase = .active
```

WHO	Your top-level code, moving the model.
WHAT	**Moves phase to active: the app is on screen and receiving events.**
WHERE	After the inactive line, and again at the end of the day.
WHEN	When launch completes, and again when the user returns.
WHY	Active is the only phase where editing makes sense, so the edits happen there.

```
11 edits += 1
```

WHO	Your top-level code, doing the user work of the model.
WHAT	**Adds one unsaved edit to the counter.**
WHERE	In the Active stage, twice in a row.
WHEN	While phase is active, before any save.
WHY	Each increment is one change the user made and has not saved yet.

```
12 edits += 1
```

WHO	Your top-level code, doing the user work of the model.
WHAT	**Adds one unsaved edit to the counter.**
WHERE	In the Active stage, twice in a row.
WHEN	While phase is active, before any save.
WHY	Each increment is one change the user made and has not saved yet.

```
13 print("Active: \(edits) unsaved edits")
```

WHO	The print function, reading the counter.
WHAT	**Prints how many unsaved edits the Active stage produced: two.**
WHERE	After both edit lines.
WHEN	While phase is still active.
WHY	Printing the count proves the edits landed in the model before the app backgrounds.

```
14 phase = .background
```

WHO	Your top-level code, moving the model.
WHAT	**Moves phase to background: the user left the app, but it is still resident.**
WHERE	After the Active stage lines.
WHEN	Fourth, when the user goes home.
WHY	Background is the last reliable chance to run code, which is why saving happens here.

```
15 print("Saved \(edits) edits")
```

WHO	The print function, reporting the autosave.
WHAT	**Prints that the model saved both edits as the app entered the background.**
WHERE	Right after the move to background.
WHEN	While phase is background, before suspension.
WHY	Saving on entering Background means a later silent termination loses nothing.

```
16 if phase == .background {
```

WHO Your top-level code, testing the current phase.
WHAT **Checks that the app really is in the background before deferring the edit.**
WHERE In the Background stage, after the save line.
WHEN Once, while phase is background.
WHY The same edit during Active would apply at once; the phase decides the behavior.

```
17 pending.append("Fix title")
```

WHO The if body, queuing the work.
WHAT **Appends the Fix title edit to the pending list instead of applying it.**
WHERE Inside the if, in the Background stage.
WHEN While phase is background.
WHY Background code should finish quickly; real editing waits for the user to return.

```
18 print("Deferred: Fix title until active")
```

WHO The print function, inside the if body.
WHAT **Prints that the title fix is deferred until the app is active again.**
WHERE Inside the if, after the append.
WHEN While phase is background.
WHY Naming the deferral in the log makes the model behavior visible, not silent.

```
20 phase = .suspended
```

WHO Your top-level code, moving the model.
WHAT **Moves phase to suspended: the app is frozen in memory and runs no code.**
WHERE After the Background stage.
WHEN Fifth, some time after backgrounding.
WHY Suspension is the system decision, not the app choice; the model just records it.

```
21 print("Phase: suspended")
```

WHO The print function, narrating the model.
WHAT **Prints the suspended line: the last line the model can print until it thaws.**
WHERE Right after the move to suspended.
WHEN While phase is suspended.
WHY A suspended app prints nothing more, so the log pauses here until the return.

```
22 phase = .active
```

WHO Your top-level code, moving the model.
WHAT **Moves phase to active: the app is on screen and receiving events.**
WHERE After the inactive line, and again at the end of the day.
WHEN When launch completes, and again when the user returns.
WHY Active is the only phase where editing makes sense, so the edits happen there.

```
23 for item in pending { print("Applied on return: \(item)") }
```

WHO The for-in loop, draining the pending list.
WHAT **Visits each deferred edit in pending and prints it as applied, all on one line.**
WHERE In the final Active stage, after phase returns to active.
WHEN Last stage, when the user comes back.
WHY Applying the queue on return makes the deferred title fix actually happen.

Try it

The starter counts one edit while active and stops. Finish the day: count a second edit, move the phase to background, and print the save line showing how many edits were saved and in which phase.

```
var phase = "active"
var edits = 0
edits += 1
print("Edits so far: \(edits)")
```

EXPECTED OUTPUT

```
Saved 2 edits in background
```

MODULE 02

Swift Fundamentals

Read, write and run real Swift: values and types, strings and numbers, optionals, collections, control flow, functions and closures, the language every iOS app is built from.

- Create various types of iOS applications in Swift

Values, Types and Your First Program

CONCEPT

What a Swift program is

A Swift program starts as **source code**: plain text you write in Xcode, saved in files ending in `.swift`. Before anything runs, the **compiler** (the tool that translates your code into instructions a processor understands) reads every line and checks that each value is used the way its type allows. Mistakes it finds are *compile-time errors*, and the program does not start until you fix them. Problems that only appear while the program runs, such as a crash, are *run-time errors*.

In these examples, code written outside any function is **top-level code**. Swift runs it from the first line to the last, one **statement** at a time. `print` shows a value in the console so you can watch the program work.

Compile time	When the compiler checks and translates your code, before it runs.
Run time	When the finished program executes on a Mac, an iPhone, or here in the browser.
Statement	One complete instruction, usually one line, such as a declaration or a print call.

> **Swift in context**
>
> Apple introduced Swift in 2014 and made it open source in 2015. Swift 6 added compile-time data-race safety. Swift runs on Apple platforms, Linux, Windows and WebAssembly, which is how the examples on these pages run right in your browser.

CODE · RUNS IN THE BROWSER

Your first Swift program

A Swift program is a list of **statements** (single instructions) that run from top to bottom. `print` writes text to the console, the panel under the editor. Run it, then click any line to read its note.

```swift
print("Hello, Swift")
print("Cups sold:", 3)
print("2026", "10", "20", separator: "-")
print("Loading", terminator: "...")
print("done")
```

OUTPUT

```
Hello, Swift
Cups sold: 3
2026-10-20
Loading...done
```

Line by line

```
1  print("Hello, Swift")
```

WHO — The Swift standard library, which gives every program the print function without any import.
WHAT — **Writes the text Hello, Swift to the console and then ends the line, so the next output starts on a fresh line.**
WHERE — Top-level code: statements outside any function or type form the program entry point, like main in C or Java.
WHEN — First, because Swift runs top-level statements in order starting at line 1.
WHY — Printing is the fastest way to see what a program is doing; you will use it constantly to check values.

```
2  print("Cups sold:", 3)
```

WHO — The same print function, which accepts any number of items separated by commas.
WHAT — **Prints the text Cups sold: and the number 3 on one line, with a single space placed between them.**
WHERE — Still top-level code, directly after the first statement.
WHEN — Second, as soon as line 1 has finished.
WHY — Passing several items saves you from building one string yourself, and the items may have different types: here a String and an Int.

```
3  print("2026", "10", "20", separator: "-")
```

WHO — The separator parameter of print, which controls what goes between the items.
WHAT — **Joins the three items with a hyphen instead of the default space, producing 2026-10-20.**
WHERE — The labeled argument separator: comes after the items, inside the same parentheses.
WHEN — Third; print does the joining just before the text reaches the console.
WHY — Labels such as separator: name optional settings right at the call, so the line explains itself when you read it later.

```
4  print("Loading", terminator: "...")
```

WHO — The terminator parameter of print, which controls what is written after the last item.
WHAT — **Prints Loading followed by three dots and, unlike every earlier line, does not move to a new line.**
WHERE — Top-level code; the default terminator is a newline character, and this call replaces it with three dots.
WHEN — Fourth, leaving the console cursor at the end of the same line.
WHY — Changing the terminator lets several print calls build up one line of output piece by piece.

```
5  print("done")
```

WHO — A plain print call that uses the default newline terminator.
WHAT — **Writes done right after the dots, completing the line Loading...done, then ends the line.**
WHERE — The final statement of the program's top-level code.
WHEN — Fifth and last; after it there is nothing left to run, so the program exits.
WHY — It shows that output continues wherever the previous print stopped, which makes the terminator change visible.

> **Try it**
>
> 1. Change separator: "-" to separator: "/" and run it again.
> 2. Delete terminator: "..." from line 4. Where does done appear now?
> 3. Add a third item to line 2, such as "cups", and predict the output before you run it.

CONCEPT

Constants and variables

Programs remember values by giving them names. Swift has two kinds of named storage. A **constant**, declared with `let`, gets its value once and keeps it. A **variable**, declared with `var`, can receive a new value later through **assignment** with `=`.

Start every declaration with `let` and switch to `var` only when the value genuinely needs to change. The compiler then guarantees that your constants never change, so you can read code without hunting for every line that might modify a value. If you write `var` for something that never changes, Xcode warns that the variable was never mutated and suggests `let`.

let	Declares a constant: one value, set once, never reassigned.
var	Declares a variable: its value can be replaced while the program runs.
Value	A piece of data, such as 3, 4.75, true or "Ada".
Assignment	Storing a value under a name with =. It is not an equality test; that is ==.

> **Prefer let**
>
> SwiftUI apps are full of `let`: view inputs, identifiers and settings rarely change. Treat `var` as a signal to readers that says: watch this value, it changes.

CODE · RUNS IN THE BROWSER

Mistake: changing a constant

This program tries to change a constant. Swift refuses to compile it, so nothing runs, not even the first `print`. Run it, read the error, then read each line's note to see why the compiler is right.

```
1  let maxLives = 3
2  print("Maximum lives:", maxLives)
3  maxLives = 2
4  print("Maximum lives:", maxLives)
```

COMPILER ERROR (ON PURPOSE)

```
cannot assign to value: 'maxLives' is a 'let' constant
```

Line by line

1 `let maxLives = 3`

WHO	The compiler, which records maxLives as a constant of type Int when it reads this declaration.
WHAT	**Creates a constant named maxLives and gives it the value 3 for good.**
WHERE	The program's top-level code, so maxLives is a global constant visible to every line below it.
WHEN	The compiler checks it at compile time; the value would be stored at launch, if the program ever ran.
WHY	The maximum number of lives is a rule of the game, not a running total, so let is the honest choice.

2 `print("Maximum lives:", maxLives)`

WHO	The print function from the standard library.
WHAT	**Would print Maximum lives: 3, but never gets the chance because the file fails to compile.**
WHERE	Top-level code, between the declaration and the faulty line.
WHEN	Never in this version: one compile-time error stops the whole program, not just the bad line.
WHY	It shows that Swift rejects the entire program up front instead of running the lines that happen to be correct.

3 `maxLives = 2`

- **WHO** The compiler's type checker, which remembers which names were declared with let.
- **WHAT** **Attempts to store 2 in a constant, which produces the error: cannot assign to value: 'maxLives' is a 'let' constant.**
- **WHERE** Line 3 of the top-level code; Xcode underlines this exact line and offers a fix-it that changes let to var.
- **WHEN** At compile time, before the program can start.
- **WHY** Catching this early means a value you promised would never change cannot be changed by accident, even in a large app.

4 `print("Maximum lives:", maxLives)`

- **WHO** The print function, called a second time.
- **WHAT** **Would show the updated value if the assignment were allowed; here it never runs.**
- **WHERE** Top-level code at the very end of the file, a second print of the same global constant that line 2 already printed.
- **WHEN** Never, because compilation already failed at line 3.
- **WHY** Printing before and after a change is a common way to watch a value evolve; the next step does exactly that with var.

> **Try it**
>
> 1. Change let on line 1 to var and run it again. What does the program print now?
> 2. Undo that change, then delete line 3 instead. Does the program compile?

CODE · RUNS IN THE BROWSER

The fix: use var for values that change

The fix is to decide, for each name, whether its value should ever change. The limit stays a constant. The score and the remaining lives change during play, so they become variables. `+=` and `-=` update a variable starting from its current value.

```
let maxLives = 3
var lives = maxLives
var score = 0

score = 150
score += 50
lives -= 1

print("Score:", score)
print("Lives left: \(lives) of \(maxLives)")
```

OUTPUT

```
Score: 200
Lives left: 2 of 3
```

Line by line

```
1  let maxLives = 3
```

WHO The compiler, which infers the type Int from the literal 3 and marks the name as a constant.
WHAT **Declares the game's fixed limit of 3 lives.**
WHERE Top-level code; as a global constant it can be read on every line below.
WHEN Stored once when the program starts and never touched again.
WHY The limit is a rule, so let protects it while the other values change around it.

```
2  var lives = maxLives
```

WHO Your program, copying a value from one name into another.
WHAT **Declares a variable lives that starts with a copy of maxLives, which is 3.**
WHERE Top-level code; lives holds its own independent copy of the number, so changing lives never changes maxLives.
WHEN Right after maxLives exists, which is required because a name must be declared before it is used.
WHY Starting from the constant keeps both values in step at the start without typing 3 twice.

```
3  var score = 0
```

WHO The compiler, which infers Int from the literal 0.
WHAT **Declares a variable score with a starting value of 0.**
WHERE Top-level code, alongside the rest of the game state.
WHEN At program start, before any points are earned.
WHY Swift requires a value before first use, so a clear starting value rules out reading an undefined number.

```
5  score = 150
```

WHO The assignment operator, acting on a variable.
WHAT **Replaces the old value 0 with 150; the old value is simply gone.**
WHERE Top-level code; legal only because score was declared with var.
WHEN After the declarations, as if the player had just earned points.
WHY Plain assignment is for setting a brand-new value that does not depend on the old one.

```
6  score += 50
```

WHO The compound assignment operator += from the standard library, which reads the current score and writes the new total back.
WHAT **Adds 50 to the current value, so score becomes 200; it is shorthand for score = score + 50.**
WHERE Top-level code, working on the same variable.
WHEN Immediately after the previous assignment, so it builds on 150.
WHY The shorthand avoids typing the variable name twice, which removes a chance for typos.

```
7  lives -= 1
```

WHO The compound assignment operator -=, the subtracting partner of +=, acting on the variable lives rather than on maxLives.
WHAT **Subtracts 1, leaving lives at 2 while maxLives stays at 3.**
WHERE Top-level code; this is the kind of change that failed in the previous step when it targeted a let constant.
WHEN After scoring, as if the player had just lost a life.
WHY Lives change during the game, so lives needed var; the limit does not, so maxLives kept let.

```
9  print("Score:", score)
```

WHO The print function, receiving a String and an Int.
WHAT **Writes Score: 200 to the console, with print inserting one space between the label and the number.**
WHERE Top-level code, after all the updates.
WHEN After every change has happened, so it shows the final value.
WHY Printing at the end confirms that both score updates were applied, in order.

```
10 print("Lives left: \(lives) of \(maxLives)")
```

WHO Swift string interpolation, which evaluates each backslash-parenthesis pair and inserts the result into the text.

WHAT **Builds and prints the single line Lives left: 2 of 3.**

WHERE Inside a string literal; each interpolation can hold any expression, here a variable and a constant.

WHEN The values are read at the moment this line runs, so it shows lives after the decrease.

WHY Interpolation puts values exactly where they belong in a sentence; the next lesson covers it in depth.

> **Try it**
>
> 1. Add the line score += 25 just before the prints and predict the new score.
> 2. Change lives -= 1 to lives -= 3. What does the last line print?
> 3. Add maxLives -= 1 anywhere and read the error message.

CODE · RUNS IN THE BROWSER

Type inference and type annotations

Every value has a **type**: the kind of data it is and what you can do with it. Usually the compiler **infers** (works out) the type from the starting value. You can also state it with a **type annotation**: a colon and a type name after the name.

```
1  let cupsSold = 42
2  let price = 4.75
3  let isOpen = true
4  let shopName = "Bean There"
5  let size: Character = "L"
6  let ratio: Double = 3
7
8  let dailyGoal: Int
9  dailyGoal = 50
10
11 print(cupsSold, price, isOpen)
12 print(shopName, size)
13 print(ratio)
14 print(dailyGoal)
```

OUTPUT

```
42 4.75 true
Bean There L
3.0
50
```

Line by line

```
1 let cupsSold = 42
```

WHO The compiler's type inference, which looks at the literal 42.

WHAT **Creates a constant of type Int, Swift's standard whole-number type, holding 42.**

WHERE Top-level code; on a 64-bit iPhone or Mac an Int is stored in 64 bits.

WHEN The type is decided at compile time; the value itself is stored at run time.

WHY Inference keeps code short while staying fully typed: cupsSold can never later hold text or a decimal.

```
2  let price = 4.75
```

WHO Type inference again, this time seeing a decimal point.
WHAT **Creates a constant of type Double, a 64-bit floating-point number, holding 4.75.**
WHERE Top-level code, next to the other coffee shop values.
WHEN Inferred at compile time from the decimal point in the literal.
WHY Swift picks Double rather than Float for decimal literals because it is more precise and is the default across Apple frameworks.

```
3  let isOpen = true
```

WHO Type inference, recognizing the keyword true.
WHAT **Creates a Bool constant; a Bool can only ever be true or false.**
WHERE Top-level code, so isOpen is a global constant that any later line in this program could read in a condition.
WHEN Inferred at compile time, with no run-time cost.
WHY Naming a Bool like a yes-or-no question (isOpen) makes later conditions such as if isOpen read naturally.

```
4  let shopName = "Bean There"
```

WHO Type inference, recognizing a double-quoted string literal.
WHAT **Creates a String constant holding the text Bean There.**
WHERE Top-level code; the String manages its own storage for the characters.
WHEN The compiler fixes the type as String at compile time; the text itself is stored when this line runs.
WHY Double quotes mean String by default; the next line shows how to ask for something else.

```
5  let size: Character = "L"
```

WHO You, overriding inference with an explicit type annotation.
WHAT **Stores a single Character instead of a one-letter String.**
WHERE The annotation sits between the name and the equals sign: name, colon, type.
WHEN The compiler checks at compile time that the literal really is exactly one character.
WHY Without the annotation "L" would become a String; an annotation is how you request a different type.

```
6  let ratio: Double = 3
```

WHO The compiler, using your annotation to interpret the literal 3.
WHAT **Stores 3 as a Double, so it will print as 3.0 rather than 3.**
WHERE Top-level code, sitting right beside the inferred constants above so you can compare an annotated declaration with inferred ones.
WHEN The literal is turned into a Double at compile time, so this costs nothing while running.
WHY A number literal has no fixed type of its own; the annotation decides whether 3 becomes an Int or a Double.

```
8  let dailyGoal: Int
```

WHO The compiler's definite-initialization check, which tracks whether a name has a value yet.
WHAT **Declares a constant with a type but no value so far.**
WHERE Top-level code; the annotation is required here because there is no value to infer the type from.
WHEN At compile time the compiler verifies it is assigned exactly once before anything reads it.
WHY Declaring first and assigning later is useful when the value depends on a decision made in between.

```
9  dailyGoal = 50
```

WHO The assignment operator, giving the constant its one and only value.
WHAT **Sets dailyGoal to 50; a second assignment to it would be a compile-time error.**
WHERE Top-level code, before any line reads dailyGoal.
WHEN At run time, when the program reaches this line.
WHY A let may be assigned once, later than its declaration, which keeps it constant without forcing a value too early.

11 `print(cupsSold, price, isOpen)`

- **WHO** The print function, which knows how to describe an Int, a Double and a Bool.
- **WHAT** **Prints 42 4.75 true on one line, separated by spaces.**
- **WHERE** Top-level code, after every value exists.
- **WHEN** After all the declarations, at run time.
- **WHY** Seeing the values side by side shows that each kept its own type and format.

12 `print(shopName, size)`

- **WHO** The print function, receiving a String and a Character.
- **WHAT** **Prints Bean There L; a Character looks just like a one-letter string when printed.**
- **WHERE** Top-level code; both values are global constants declared above, and their text goes to the console as one line.
- **WHEN** Second line of output, immediately after the line showing 42 4.75 true has been printed.
- **WHY** The difference between String and Character lives in the type, not in how the value looks.

13 `print(ratio)`

- **WHO** The print function, turning the Double value of ratio into text using the standard formatting for floating-point numbers.
- **WHAT** **Prints 3.0, because whole-number Doubles always show a decimal point.**
- **WHERE** Top-level code; it reads the global constant ratio, whose type the annotation on line 6 set to Double.
- **WHEN** At run time, after the earlier prints.
- **WHY** The .0 is a visible clue that the annotation changed the type of the literal.

14 `print(dailyGoal)`

- **WHO** The print function, reading the constant assigned a few lines earlier.
- **WHAT** **Writes 50 to the console, the value that line 9 assigned to the constant after it was declared without one.**
- **WHERE** Top-level code; reading dailyGoal is legal only because it was assigned first.
- **WHEN** Last, so the program ends after this line.
- **WHY** It proves the delayed assignment worked and the constant now holds its value.

> **Try it**
>
> 1. Change line 6 to let ratio = 3, without the annotation. What prints now, and why?
> 2. Add let temperature = -3.5 and print it. Which type did Swift infer?
> 3. Change the goal to 75 and run it again.

CODE · RUNS IN THE BROWSER

Mistake: mixing Int and Double

A tip in whole dollars is an `Int`; a bill with cents is a `Double`. Adding them looks harmless, but Swift never converts one number type to another on its own. Run it and read the error.

```
1  let tipDollars = 3
2  let billAmount = 18.75
3  let total = tipDollars + billAmount
4  print(total)
```

COMPILER ERROR (ON PURPOSE)

```
binary operator '+' cannot be applied to operands of type 'Int' and 'Double'
```

Line by line

1 `let tipDollars = 3`

- **WHO** Type inference, which sees a whole-number literal.
- **WHAT** **Creates an Int constant holding 3.**
- **WHERE** Top-level code at the start of the program.
- **WHEN** Its type is fixed at compile time, before any arithmetic is attempted.
- **WHY** A tip in whole dollars is naturally a whole number, so Int is a reasonable inference.

2 `let billAmount = 18.75`

- **WHO** Type inference, which sees a decimal point.
- **WHAT** **Creates a Double constant holding 18.75.**
- **WHERE** Top-level code, right after the tip.
- **WHEN** Its type is also fixed at compile time.
- **WHY** Amounts with cents need a fractional part, so Swift chooses Double.

3 `let total = tipDollars + billAmount`

- **WHO** The compiler, searching for a version of the + operator that accepts an Int on the left and a Double on the right.
- **WHAT** **Fails with: binary operator '+' cannot be applied to operands of type 'Int' and 'Double'.**
- **WHERE** Top-level code; the error points at the plus sign between the two names.
- **WHEN** At compile time, so the program never starts.
- **WHY** Silent conversion can lose data or precision without anyone noticing; Swift makes you state which conversion you mean.

4 `print(total)`

- **WHO** The print function, which never gets the chance to run because the compiler rejects the program first.
- **WHAT** **Would print the total, but it never runs because the line above does not compile.**
- **WHERE** Last line of the top-level code, the place where the sum would have been shown if the addition were legal.
- **WHEN** Never in this version of the program.
- **WHY** It is the line you would expect to see output from, which makes the missing output a clue that compilation failed.

> **Try it**
>
> 1. Change line 1 to let tipDollars = 3.0 and run it again. Why does it compile now?
> 2. Change it back, then annotate line 1 as let tipDollars: Double = 3. What happens?

CODE · RUNS IN THE BROWSER

The fix: convert on purpose

To combine different number types, convert one of them yourself. `Double(x)` makes a Double from an Int. `Int(x)` makes an Int from a Double by *dropping* the fraction. To round instead, call `rounded()` first. Because every conversion is written out, readers can see exactly where precision might change.

```swift
let tipDollars = 3
let billAmount = 18.75

let total = Double(tipDollars) + billAmount
print(total)

let wholeDollars = Int(billAmount)
print(wholeDollars)

let nearestDollar = Int(billAmount.rounded())
print(nearestDollar)
```

OUTPUT

```
21.75
18
19
```

Line by line

1 `let tipDollars = 3`

- WHO — Type inference in the compiler, which sees a whole-number literal with no decimal point and picks Int.
- WHAT — **Stores the tip of 3 whole dollars.**
- WHERE — Top-level code, unchanged from the broken version.
- WHEN — The Int type is fixed at compile time; the value 3 is stored when this first line runs.
- WHY — The fix does not change the inputs; it changes how they are combined.

2 `let billAmount = 18.75`

- WHO — Type inference again, this time seeing the decimal point in 18.75 and picking Double for the bill.
- WHAT — **Creates a global Double constant holding 18.75, the bill in dollars with its cents kept as a fraction.**
- WHERE — Top-level code, the second global constant, identical to the broken version so only the combining line differs.
- WHEN — The Double type is decided at compile time; the value is stored as the second statement runs.
- WHY — Keeping the bill as a Double preserves the cents.

4 `let total = Double(tipDollars) + billAmount`

- WHO — The Double initializer, a function that builds a new Double from another value.
- WHAT — **Converts the tip to 3.0 and adds it to 18.75, so total is the Double 21.75.**
- WHERE — Top-level code; the conversion creates a new value and leaves tipDollars untouched as an Int.
- WHEN — At run time, after both inputs exist.
- WHY — Converting the Int to a Double loses nothing, so it is the safe direction for this sum.

5 `print(total)`

- WHO — The print function, formatting the Double sum produced by Double(tipDollars) plus billAmount.
- WHAT — **Writes 21.75 to the console: 3.0 plus 18.75, with the cents intact because both operands were Double.**
- WHERE — Top-level code, reading the global constant total that the fixed addition on line 4 created.
- WHEN — Right after the sum is computed.
- WHY — It confirms the program now compiles and produces the expected total.

```
7  let wholeDollars = Int(billAmount)
```

WHO The Int initializer, converting a Double into an Int.
WHAT **Drops the .75 and keeps 18; it truncates toward zero instead of rounding.**
WHERE Top-level code; the result is a new Int constant.
WHEN When this line runs, after the sum is printed; the compiler only checks that a Double may be converted.
WHY Truncation surprises many beginners, which is one reason Swift insists the conversion be written out.

```
8  print(wholeDollars)
```

WHO The print function, describing an Int.
WHAT **Writes 18 to the console, not 19, because Int(billAmount) cut off the .75 instead of rounding it.**
WHERE Top-level code, reading the global Int constant wholeDollars from the line just above.
WHEN Second line of output, right after the truncating conversion has produced 18.
WHY Seeing 18 makes the dropped fraction impossible to miss.

```
10 let nearestDollar = Int(billAmount.rounded())
```

WHO The rounded() method of Double, followed by the Int initializer.
WHAT **Rounds 18.75 to 19.0 first, then converts that whole number to the Int 19.**
WHERE Top-level code; rounded() returns a new Double and leaves billAmount unchanged.
WHEN At run time; the inner call happens before the outer conversion.
WHY When you want the nearest whole number, round first and then convert, so the conversion has no fraction left to drop.

```
11 print(nearestDollar)
```

WHO The print function, showing the Int that came from rounding the Double first and converting it second.
WHAT **Writes 19 to the console, because 18.75 rounded to the nearest whole number is 19.0 before Int drops the .0.**
WHERE The final line of top-level code; once it prints, the program has nothing left to run and exits.
WHEN Last, after which the program ends.
WHY Comparing 18 with 19 shows the difference between converting directly and rounding first.

> **Try it**
>
> 1. Change billAmount to 18.25. Which of the three printed values change?
> 2. Compute Int(Double(tipDollars) + billAmount) and print it. Predict the result first.

CONCEPT

Naming and comments

Code is read far more often than it is written, so Swift has firm naming conventions. Values, variables and functions use **lowerCamelCase**: `cupsSold`, `maxLives`. Types use **UpperCamelCase**: `String`, `CoffeeOrder`. Prefer clarity over brevity: `isLoggedIn` beats `flag`, and `retryCount` beats `rc`. Name a Bool like a yes-or-no question.

Comments are notes for people; the compiler ignores them. `//` turns the rest of a line into a comment, and `/* ... */` can span several lines. `///` starts a **documentation comment**, which Xcode shows in Quick Help when you Option-click the name.

lowerCamelCase	Values, properties and functions: dailyGoal, fetchWeather.
UpperCamelCase	Types and protocols: WeatherReport, Identifiable.
Good comments	Explain why the code exists or a decision you made; the code already says what it does.

> **Names are design**
>
> Apple's Swift API Design Guidelines name clarity at the point of use as the most important goal. A well-chosen name often makes a comment unnecessary.

CHECK YOURSELF

Check your understanding

1. What does this program print?

```
let price: Double = 5
print(price)
```

   A. 5
   B. 5.0
   C. Double(5)
   D. Nothing; it does not compile

2. Which line stops this program from compiling?

```
let city = "Austin"      // line 1
var temperature = 31     // line 2
temperature = 29         // line 3
city = "Denver"          // line 4
```

   A. Line 2
   B. Line 3
   C. Line 4
   D. None; it compiles

3. What happens with this code?

```
let a = 7
let b = 2.5
let c = a + b
```

   A. It does not compile; convert first, for example Double(a) + b
   B. c is 9.5, because Swift converts a to a Double
   C. c is 9, because Swift rounds the result to an Int
   D. It compiles but crashes when it runs

4. You need a constant that records whether the user has turned on notifications. Which name follows Swift conventions best?

   A. NotificationsFlag
   B. notifications_on
   C. flag
   D. areNotificationsEnabled

5. What does `print(Int(9.99))` print?

   A. 10
   B. 9
   C. 9.99
   D. 9.0

Answers: Values, Types and Your First Program

1. **B. 5.0** The annotation makes `price` a `Double`, and Swift prints whole-number Doubles with a decimal point: `5.0`. It compiles fine, because a whole-number literal can become a Double when the type says so. 5 would print only if Swift had inferred `Int`, which the annotation prevents.
2. **C. Line 4** `city` is a `let` constant, so assigning to it on line 4 fails with *cannot assign to value: 'city' is a 'let' constant*. Line 3 is fine: `temperature` was declared with `var`, and changing a variable is exactly what `var` is for.
3. **A. It does not compile; convert first, for example Double(a) + b** Swift never converts between number types on its own, so adding an `Int` and a `Double` is a compile-time error. Many languages would quietly produce 9.5, which makes that the natural guess, but Swift asks you to write `Double(a)` so the conversion is visible. It is not a crash either: the compiler stops the program before it can run.
4. **D. areNotificationsEnabled** `areNotificationsEnabled` is lowerCamelCase and reads as a yes-or-no question, so `if areNotificationsEnabled` reads like English. `NotificationsFlag` uses UpperCamelCase, which Swift reserves for types. `notifications_on` uses underscores, which Swift style avoids, and `flag` tells a reader nothing.
5. **B. 9** Converting a `Double` to an `Int` drops the fractional part; it does not round, so the result is 9. The answer 10 would need rounding first: `Int(9.99.rounded())`. The result is an `Int`, so it can never print as 9.0 or 9.99.

Numbers, Operators and Strings

CODE · RUNS IN THE BROWSER

Arithmetic and integer division

Swift has the usual operators: `+`, `-`, `*` and `/`. The surprise is `/` between two `Int` values: it performs **integer division**, keeping only the whole part. The **remainder operator** `%` gives you what was left over. Predict each line of output before you run it.

```
let totalMinutes = 135
let hours = totalMinutes / 60
let minutes = totalMinutes % 60
print(hours, "h", minutes, "min")

let slices = 8
let people = 3
print(slices / people, "each,", slices % people, "left over")
print(Double(slices) / Double(people))
print(-7 / 2, -7 % 2)
print(7 * 2 + 1, 7 * (2 + 1))
```

OUTPUT

```
2 h 15 min
2 each, 2 left over
2.6666666666666665
-3 -1
15 21
```

Line by line

1 `let totalMinutes = 135`

- WHO — Your program declares the constant; the compiler infers its type from the literal 135.
- WHAT — **Creates a constant named totalMinutes holding 135, typed as Int because the literal has no decimal point.**
- WHERE — In the program top-level code, so totalMinutes is a global constant that every line below can read.
- WHEN — Its type is fixed at compile time; the value is stored as soon as the program starts running.
- WHY — A workout length in whole minutes never changes in this program, so a let constant of type Int fits exactly.

2 `let hours = totalMinutes / 60`

- WHO — The division operator for Int, defined in the Swift standard library.
- WHAT — **Divides 135 by 60 and keeps only the whole part, so hours is 2; the fraction is discarded, not rounded.**
- WHERE — Top-level code; because both operands are Int, the result is an Int stored in a new global constant.
- WHEN — Right after totalMinutes exists, as the second statement of the run.
- WHY — Integer division answers "how many full hours" directly, which is exactly the question being asked here.

```
3  let minutes = totalMinutes % 60
```

WHO The remainder operator %, which works on integer types.
WHAT **Computes what is left after taking out as many 60s as fit into 135, which is 15.**
WHERE Top-level code, a third global constant sitting beside hours.
WHEN Third statement of the run, immediately after hours, so both halves of the 135 minutes are ready before anything prints.
WHY Pairing / with % splits one quantity into whole units and leftovers, the standard recipe for minutes to hours and minutes.

```
4  print(hours, "h", minutes, "min")
```

WHO The standard library print function, receiving two Int constants and two String literals as four separate items.
WHAT **Prints four items with a single space between each, producing 2 h 15 min.**
WHERE Writes to standard output, which appears in the console under the editor.
WHEN Once both hours and minutes hold their values.
WHY print accepts any number of items of any type and separates them with spaces, so mixing numbers and text needs no conversion.

```
6  let slices = 8
```

WHO The compiler again infers Int from the literal.
WHAT **Stores 8, the number of pizza slices to share.**
WHERE A new global constant in the top-level code.
WHEN After the first line of output has been printed.
WHY Sharing slices among friends is a familiar case where whole pieces and leftovers both matter.

```
7  let people = 3
```

WHO Your program, declaring a second Int constant.
WHAT **Stores 3, the number of people sharing the slices.**
WHERE A global Int constant in top-level code, declared next to slices so the two inputs of the division sit together.
WHEN Right after slices is created and before the division line, which reads both constants to compute shares and leftovers.
WHY Naming the divisor instead of typing a bare 3 in the math below makes the expressions read like the problem itself.

```
8  print(slices / people, "each,", slices % people, "left over")
```

WHO The Int operators / and %, then print.
WHAT **Evaluates 8 / 3 as 2 and 8 % 3 as 2, then prints 2 each, 2 left over.**
WHERE Each expression is computed into a temporary value that exists only for this call.
WHEN Both expressions are evaluated, left to right, before print writes anything.
WHY You can pass whole expressions to print; this is a quick way to check arithmetic without extra constants.

```
9  print(Double(slices) / Double(people))
```

WHO The Double initializer, which converts an Int, and the division operator for Double.
WHAT **Converts 8 and 3 to 8.0 and 3.0 first, so the division keeps the fraction and prints 2.6666666666666665.**
WHERE The two converted values are temporaries used only inside this expression; slices and people stay Int.
WHEN Each conversion happens before the division, because arguments are evaluated first.
WHY Swift never turns an Int into a Double for you, so you convert on purpose; the final 5 shows a Double stores a close approximation of two thirds.

```
10 print(-7 / 2, -7 % 2)
```

WHO — The same Int operators, now with a negative left operand.

WHAT — **Prints -3 -1: integer division truncates toward zero instead of rounding down to -4, and the remainder takes the sign of the left side.**

WHERE — Pure arithmetic on literals; nothing is stored.

WHEN — Evaluated when this line runs, just before printing.

WHY — Knowing the rule for negative numbers prevents off-by-one bugs, for example when positions can fall left of zero.

```
11 print(7 * 2 + 1, 7 * (2 + 1))
```

WHO — Swift operator precedence rules, which the compiler applies while parsing.

WHAT — **Prints 15 21: multiplication happens before addition, unless parentheses group the addition first.**

WHERE — Two independent expressions in one print call.

WHEN — The grouping is decided at compile time; the values are computed when the line runs.

WHY — Swift follows ordinary math precedence; add parentheses whenever they make your intent clearer to the next reader.

Try it

1. Change totalMinutes to 200 and predict the first line before you run it.
2. Change people to 4. Which number on the second line becomes 0, and why?

CODE · RUNS IN THE BROWSER

Compound assignment and number helpers

A **compound assignment** such as `+=` updates a variable using its own current value. Swift also gives numbers handy helpers: `abs`, `min`, `max`, `isMultiple(of:)` and `rounded()`. The last line shows that a `Double` is an approximation, which every programmer must know.

```
var steps = 4000
steps += 2500
steps -= 500
steps *= 2
print(steps)
var distance = 10.0
distance /= 4
print(distance)
print(abs(-12), min(3, 9), max(3, 9))
print(steps.isMultiple(of: 1000))
let pace = 2.567
print(pace.rounded(), (pace * 100).rounded() / 100)
print(Double.pi)
print(0.1 + 0.2)
```

OUTPUT

```
12000
2.5
12 3 9
true
3.0 2.57
3.141592653589793
0.30000000000000004
```

Line by line

1 `var steps = 4000`

WHO — Your program declares a variable; the compiler infers Int.
WHAT — **Creates a variable named steps holding 4000.**
WHERE — Top-level code; var means later lines may replace the stored value.
WHEN — The first statement of the run.
WHY — A step counter changes during the day, so it must be var; with let, every update below would be a compile error.

2 `steps += 2500`

WHO — The compound assignment operator +=, which the standard library defines for every numeric type, including Int.
WHAT — **Adds 2500 to the current value and stores the result back, so steps becomes 6500.**
WHERE — Changes the same variable in place; no new variable is created.
WHEN — Second statement of the run; it reads the current 4000 before writing 6500 back into steps.
WHY — It is shorter than steps = steps + 2500 and names the variable only once, so you cannot update the wrong one by mistake.

3 `steps -= 500`

WHO — The -= operator, the subtracting member of the same family.
WHAT — **Subtracts 500 from the current 6500 and stores the result back in steps, leaving 6000.**
WHERE — Updates the global variable steps in place.
WHEN — Third statement, after the addition, so it starts from 6500 rather than the original 4000.
WHY — Every arithmetic operator has a compound form, and -= reads naturally as "decrease by".

4 `steps *= 2`

WHO — The *= operator for Int, which multiplies a variable by a value and writes the product back into it.
WHAT — **Doubles the value, so steps is now 12000.**
WHERE — Same variable, updated a third time.
WHEN — After the subtraction; order matters, because each line starts from the previous result.
WHY — Reading the four lines top to bottom tells the whole story of the value, which makes this style easy to follow.

5 `print(steps)`

WHO — The standard library print function, reading the global variable steps after its three updates.
WHAT — **Writes 12000 as the first line of output: 4000 plus 2500, minus 500, times 2.**
WHERE — Standard output, shown in the console under the editor; steps is only read here, so it stays 12000.
WHEN — After all three updates have run.
WHY — Printing the final value confirms you traced the updates correctly.

6 `var distance = 10.0`

WHO — The compiler, which sees a decimal point in the literal.
WHAT — **Creates a Double variable holding 10.0.**
WHERE — A global Double variable in top-level code; var allows the /= on the next line to change it in place.
WHEN — After the first print, so the step total appears in the console before any distance work begins.
WHY — Writing 10.0 instead of 10 is how you ask for a Double without writing a type annotation.

```
7  distance /= 4
```

WHO The /= compound assignment operator for Double, which divides the variable and stores the quotient back.

WHAT **Divides distance by 4 and stores 2.5; the literal 4 becomes a Double because the other operand is a Double.**

WHERE Changes the existing global variable distance in place; no new name is created and the old 10.0 is overwritten.

WHEN The very next statement after the declaration, so it divides the starting value 10.0.

WHY A plain number literal has no fixed type until context decides, so 4 works here; only values that already have a type need converting.

```
8  print(distance)
```

WHO The print function, describing a Double value that now holds a fractional part.

WHAT **Writes 2.5 as the second line of output, showing that the fraction from 10.0 divided by 4 was kept.**

WHERE Standard output, directly below the 12000 printed earlier; the variable itself is only read here.

WHEN After the division has updated distance, and before the abs, min and max line runs.

WHY It confirms the division kept the fraction, because the variable is a Double.

```
9  print(abs(-12), min(3, 9), max(3, 9))
```

WHO The global functions abs, min and max from the standard library.

WHAT **Prints 12 3 9: the distance of -12 from zero, the smaller value, and the larger value.**

WHERE They are free functions, available everywhere without an import.

WHEN Each call runs before print receives its three items.

WHY These helpers state your intent, such as "the larger of", more clearly than writing a comparison each time.

```
10 print(steps.isMultiple(of: 1000))
```

WHO The isMultiple(of:) method that every Int value has.

WHAT **Asks whether 12000 divides evenly by 1000 and prints true.**

WHERE Called on the variable steps with a dot, because the method belongs to the Int type.

WHEN When this line runs, using the current value of steps.

WHY It reads like English and is clearer than steps % 1000 == 0, although both give the same answer.

```
11 let pace = 2.567
```

WHO The compiler, inferring Double from the decimal literal.

WHAT **Stores 2.567, a running pace in minutes per kilometre.**

WHERE A global Double constant in top-level code, read by the rounding line that follows and never changed afterward.

WHEN Before the rounding line that uses it.

WHY A value with three decimals gives the rounding examples below something to do.

```
12 print(pace.rounded(), (pace * 100).rounded() / 100)
```

WHO The rounded() method of Double, which rounds to the nearest whole number, with halfway cases rounding away from zero.

WHAT **Prints 3.0 2.57: the first rounds to the nearest whole number; the second shifts two places, rounds, and shifts back.**

WHERE Both results are new Double values; pace itself is unchanged.

WHEN Each expression is evaluated, left to right, before printing.

WHY rounded() returns a Double, not an Int, which is why the output shows 3.0 instead of 3.

```
13 print(Double.pi)
```

WHO The Double type itself, which provides pi as a static property.

WHAT **Prints 3.141592653589793, the closest Double to pi.**

WHERE pi belongs to the type, not to any one number, so you write the type name before the dot.

WHEN When this line runs; the constant is built into the standard library.

WHY Using the built-in constant avoids typing a less precise value like 3.14 by hand.

```
14 print(0.1 + 0.2)
```

WHO The processor floating-point hardware, which stores Doubles in binary.
WHAT **Prints 0.30000000000000004, because 0.1 and 0.2 cannot be stored exactly in binary.**
WHERE This happens in every language that uses standard 64-bit floating point, not just Swift.
WHEN The tiny error appears when the sum is computed at run time.
WHY Never compare Doubles with == after arithmetic, and format numbers before showing them to people, as a later step does.

> **Try it**
>
> 1. Change steps *= 2 to steps /= 3. Is 4000 a multiple of 1000 now?
> 2. Change pace to 2.5 and run it. Which way does rounded() go?

CODE · RUNS IN THE BROWSER

Comparison and logical operators

Comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`) produce a `Bool`. **Logical operators** combine Bools: `&&` means and, `||` means or, `!` means not. The **ternary operator** `?:` picks one of two values. Here they decide whether a visitor can enter a cinema and what the ticket costs.

```swift
let age = 20
let hasTicket = true
let isMember = false
print(age >= 18)
print(age == 21, age != 21)
let canEnter = age >= 18 && hasTicket
let getsDiscount = isMember || age < 25
print(canEnter, getsDiscount)
print(!isMember)
let price = isMember ? 8 : 12
print("Ticket:", price)
```

OUTPUT

```
true
false true
true true
true
Ticket: 12
```

Line by line

```
1 let age = 20
```

WHO The compiler infers Int for the literal 20.
WHAT **Stores the visitor age as the whole number 20, which every rule in this program will test.**
WHERE A global constant in the top-level code.
WHEN First statement of the run, so the age exists before any comparison or logical rule reads it.
WHY Every rule below depends on the age, so it is declared first and given a clear name.

```
2  let hasTicket = true
```

WHO The compiler infers Bool, the type with exactly two values, true and false.
WHAT **Records that the visitor holds a ticket.**
WHERE A global Bool constant in top-level code, read later by the && rule that decides entry.
WHEN Second statement of the run, right after age, so both inputs to canEnter are ready.
WHY Names that start with has or is read like yes-or-no questions, the Swift convention for Bool values.

```
3  let isMember = false
```

WHO Your program, declaring a third input; the compiler infers Bool from the literal false.
WHAT **Records that the visitor is not a loyalty member.**
WHERE A global Bool constant beside hasTicket.
WHEN Third statement of the run; it feeds the discount rule, the not operator and the ternary price choice below.
WHY A positive name like isMember stays clear when negated; isNotMember would lead to confusing double negatives.

```
4  print(age >= 18)
```

WHO The greater-than-or-equal operator for Int, which returns a Bool rather than a number.
WHAT **Compares 20 with 18 and prints true.**
WHERE The comparison creates a temporary Bool passed straight to print.
WHEN Fourth statement, producing the first line of output before any combined rule has been computed.
WHY Every comparison produces a Bool, and only a Bool can drive an if or a while in Swift.

```
5  print(age == 21, age != 21)
```

WHO The equality operator == and the inequality operator !=.
WHAT **Prints false true, because 20 is not equal to 21.**
WHERE Two separate comparisons inside one print call.
WHEN Both comparisons are evaluated left to right as this line runs, before print writes false true.
WHY A single = assigns while == compares; Swift refuses to compile an assignment used as a condition, which prevents a classic C bug.

```
6  let canEnter = age >= 18 && hasTicket
```

WHO The logical AND operator &&, which combines two Bool values and is true only when both are true.
WHAT **Is true only when both sides are true; here both are, so canEnter is true.**
WHERE The combined result is stored in a new global Bool constant.
WHEN Swift evaluates the left side first; had it been false, the right side would be skipped entirely (short-circuit evaluation).
WHY Naming a combined rule makes later code read like plain English, for example if canEnter.

```
7  let getsDiscount = isMember || age < 25
```

WHO The logical OR operator ||, which combines two Bool values and is true when at least one is true.
WHAT **Is true when at least one side is true; isMember is false but age less than 25 is true, so the result is true.**
WHERE Another global Bool constant in top-level code, storing the finished rule so later lines can reuse it by name.
WHEN Swift checks isMember first; because it is false, it must also evaluate the age test. Had isMember been true, the right side would never run.
WHY Short-circuiting lets you put a cheap or protective check first, a pattern you will rely on often.

```
8  print(canEnter, getsDiscount)
```

WHO The print function, receiving the two Bool constants computed by the && and || rules above.
WHAT **Writes true true on one line: the visitor may enter and also qualifies for the under-25 discount.**
WHERE Standard output, the third line in the console; print separates the two Bool values with a single space.
WHEN After both rules have been computed.
WHY Printing the intermediate Bools is a simple way to debug a complicated condition.

9 `print(!isMember)`

- **WHO** The logical NOT operator, a prefix !.
- **WHAT** **Flips false to true and prints true.**
- **WHERE** The ! sits directly before the value, with no space.
- **WHEN** Evaluated as this line runs: the not operator flips the stored false into a temporary true before printing.
- **WHY** Negating a positive name with ! reads as "not a member", which stays clear to the next reader.

10 `let price = isMember ? 8 : 12`

- **WHO** The ternary conditional operator ?:, the only Swift operator with three operands.
- **WHAT** **Chooses 8 if isMember is true, otherwise 12; here price becomes 12.**
- **WHERE** The chosen value is stored in a global Int constant.
- **WHEN** The condition is checked first, then only the chosen side is evaluated.
- **WHY** It suits a short either-or choice of a value; for anything longer, an if statement is easier to read.

11 `print("Ticket:", price)`

- **WHO** The print function, given a String label and the Int that the ternary operator chose.
- **WHAT** **Writes Ticket: 12, because isMember is false and the ternary therefore picked the non-member price.**
- **WHERE** Standard output, the fifth and final line in the console, with a space between the label and the number.
- **WHEN** Last statement of the run, after the ternary has already stored 12 in price.
- **WHY** Labeling output makes the console readable once a program prints more than a few values.

> **Try it**
>
> 1. Change isMember to true. Which two lines of output change?
> 2. Change age to 16. Predict canEnter and getsDiscount, then run it.

CODE · RUNS IN THE BROWSER

String interpolation and multi-line strings

String interpolation puts a value into text: write `\(expression)` inside the quotes and Swift converts the result to text. A **multi-line string literal** starts and ends with three quotes, `"""`, and keeps your line breaks. Swift removes the indentation that matches the closing quotes, so you can indent the literal neatly.

```
let drink = "latte"
let size = 16
let price = 4.5
print("A \(size) oz \(drink) costs $\(price).")
print("Two of them cost $\(price * 2).")
let receipt = """
    Order: \(drink)
    Size: \(size) oz
      Thank you
    """
print(receipt)
```

OUTPUT

```
A 16 oz latte costs $4.5.
Two of them cost $9.0.
Order: latte
Size: 16 oz
  Thank you
```

Line by line

1 `let drink = "latte"`

- WHO — The compiler infers String from the quoted literal.
- WHAT — **Stores the drink name latte, which both the sentence and the multi-line receipt below will insert.**
- WHERE — A global String constant in top-level code, readable by every interpolation that follows, including inside the receipt literal.
- WHEN — First statement of the run; it must exist before any string that interpolates it is built.
- WHY — Keeping data in named constants means the sentences below update automatically when the data changes.

2 `let size = 16`

- WHO — The compiler infers Int from the literal 16, since it has no decimal point.
- WHAT — **Stores the cup size in ounces, 16.**
- WHERE — A global Int constant in top-level code, used twice: in the sentence and in the receipt Size line.
- WHEN — Second statement of the run, right after drink and before any printing happens.
- WHY — Interpolation below will mix this Int with text without any manual conversion.

3 `let price = 4.5`

- WHO — The compiler infers Double from the decimal point.
- WHAT — **Stores the price 4.5 as a Double, which interpolation will print exactly as 4.5 with no currency formatting.**
- WHERE — A global Double constant in top-level code, read by both price sentences that follow it.
- WHEN — Third statement of the run, the last input created before the first print builds its sentence.
- WHY — Using a Double here shows how decimals look when interpolated without formatting.

4 `print("A \(size) oz \(drink) costs $\(price).")`

- WHO — The compiler, which turns each interpolated piece into text as the string is built.
- WHAT — **Produces A 16 oz latte costs $4.5. by inserting an Int, a String and a Double into one sentence.**
- WHERE — The finished string is a temporary value handed to print.
- WHEN — The string is assembled when this line runs, using the current values.
- WHY — Interpolation works with any type and keeps the sentence readable in one piece, unlike gluing strings with +.

5 `print("Two of them cost $\(price * 2).")`

- WHO — The interpolation machinery evaluates the expression inside the parentheses first.
- WHAT — **Computes price * 2 as 9.0 and prints Two of them cost $9.0.**
- WHERE — Any expression can go inside the interpolation parentheses, not just a name.
- WHEN — The multiplication runs while the string is being built.
- WHY — Notice 9.0: a Double always prints at least one decimal place, which is not how money should look; a later step fixes that with formatting.

```
6  let receipt = """
```

WHO The compiler, which recognizes three quotes as the start of a multi-line literal.
WHAT **Begins a String constant named receipt whose text starts on the next line.**
WHERE The opening quotes must end their line; the content always begins on the line below.
WHEN Parsed at compile time; the string value is created when this statement runs.
WHY Multi-line literals keep line breaks exactly as typed, ideal for receipts, messages and sample data, with no \n escapes to manage.

```
7  Order: \(drink)
```

WHO The compiler, interpolating inside a multi-line literal exactly as in a normal string.
WHAT **Becomes the first line of the text: Order: latte.**
WHERE Its four leading spaces are removed because they match the indentation of the closing quotes.
WHEN Built at run time along with the rest of receipt.
WHY Indenting the literal to match the surrounding code keeps source tidy without changing the output.

```
8  Size: \(size) oz
```

WHO The same interpolation, now with an Int.
WHAT **Becomes the second line of the text: Size: 16 oz.**
WHERE A line break separates it from the previous line inside the one string.
WHEN Built together with the whole literal.
WHY Each source line becomes one output line, so the code looks like the result.

```
9  Thank you
```

WHO The compiler, applying the indentation rule.
WHAT **Keeps two leading spaces in the result, because this line is indented two spaces more than the closing quotes.**
WHERE Only indentation beyond the closing delimiter column survives.
WHEN Decided at compile time from the source layout.
WHY This rule gives you exact control over spacing in the output while the code stays nicely indented.

```
10 """
```

WHO The compiler, which treats three quotes on their own line as the end of the literal.
WHAT **Closes the multi-line string; its column sets how much indentation is stripped from every line above.**
WHERE It must be on its own line, and no content line may be indented less than it.
WHEN Read at compile time; the finished value is assigned to receipt at run time.
WHY Putting the closing delimiter where you want the left margin is how Swift knows which spaces are layout and which are content.

```
11 print(receipt)
```

WHO The print function, given the single multi-line String built by the triple-quoted literal above.
WHAT **Prints the three receipt lines, the last one indented by two spaces.**
WHERE The console; the string contains line breaks between its lines but none at the very end.
WHEN Last statement of the run, after receipt has been fully assembled with every interpolation filled in.
WHY print adds the final line break itself, so a multi-line literal should not end with an empty line.

Try it

1. Add a line Total: \(price * 2) inside the receipt and run it.
2. Indent Thank you by six extra spaces instead of two. What changes?

CODE · RUNS IN THE BROWSER

Mistake: asking a string for its length

Programmers who know Java or JavaScript often reach for `length` to measure a string. Swift has no such property. Run this program and read the compiler message: it names the type and the member it could not find. Nothing runs, not even line 1, because the program never finished compiling.

```
1  let username = "grace_h"
2  let length = username.length
3  print(length)
```

COMPILER ERROR (ON PURPOSE)

```
has no member
```

Line by line

1 `let username = "grace_h"`

- WHO — The compiler, which infers String from the quoted literal before it type checks the next line.
- WHAT — **Stores the username grace_h, which has seven characters.**
- WHERE — A global String constant in top-level code; its type, String, is where the compiler searches for a length member.
- WHEN — It would run first, but this program stops at compile time, so it never executes.
- WHY — A realistic value makes the mistake below look exactly like one you might type.

2 `let length = username.length`

- WHO — The compiler type checker, which looks up every name after a dot on the value type.
- WHAT — **Asks String for a property named length; none exists, so compilation fails with value of type String has no member length.**
- WHERE — The error points at this line and at the name after the dot.
- WHEN — At compile time, before a single line of the program runs.
- WHY — Swift checks every member access in advance, so a habit from another language is caught in the editor instead of failing in front of a user.

3 `print(length)`

- WHO — The print function, which would receive length if the program had compiled.
- WHAT — **Would print the length, but this line never runs.**
- WHERE — Top-level code after the broken line.
- WHEN — Never in this version: one compile error anywhere stops the whole program from building.
- WHY — Fix compile errors from the first one down; later errors are often side effects of earlier ones.

Try it

1. Replace length with count on line 2 and run it again.
2. Misspell count as cuont. Notice the message names the exact member it could not find.

CODE · RUNS IN THE BROWSER

The fix: the String API

A string measures itself with `count`. This step tours the most common **String API** (the properties and methods a type offers). Methods like `uppercased()` return a *new* string; the original never changes. One method here, `replacingOccurrences(of:with:)`, comes from the Foundation framework, so the file imports it.

```swift
import Foundation

let username = "grace_h"
print(username.count)
print(username.isEmpty)
print(username.uppercased())
let title = "Swift Playgrounds"
print(title.lowercased())
print(title.hasPrefix("Swift"), title.hasSuffix("app"))
print(title.contains("Play"))
let slug = title.replacingOccurrences(of: " ", with: "-")
print(slug)
let cafe = "café"
print(cafe.count)
```

OUTPUT

```
7
false
GRACE_H
swift playgrounds
true false
true
Swift-Playgrounds
4
```

Line by line

1 `import Foundation`

- WHO — The compiler, loading the Foundation framework that ships with every Apple platform.
- WHAT — **Adds extra String methods to this file, including replacingOccurrences(of:with:) used below.**
- WHERE — The first line of the file; an import applies to this whole file only.
- WHEN — At compile time, before any code runs.
- WHY — count, uppercased() and hasPrefix belong to the Swift standard library, but text replacement comes from Foundation, so leaving out this line breaks the build.

3 `let username = "grace_h"`

- WHO — The compiler infers String again, so every member used below is looked up on the String type.
- WHAT — **Stores the same username as the broken example.**
- WHERE — A global String constant in top-level code; this time every line that reads it compiles.
- WHEN — First statement that runs, after the import line has been handled at compile time.
- WHY — Reusing the value makes it easy to compare the fix with the mistake.

4 `print(username.count)`

- WHO — The count property of String, the Swift name for what other languages call length.
- WHAT — **Counts the characters and prints 7.**
- WHERE — Read with a dot because count belongs to the String value.
- WHEN — Computed when this line runs, by walking through the characters.
- WHY — count is the correct Swift name for a length, and it counts characters as people see them.

```
5  print(username.isEmpty)
```

WHO The isEmpty property of String, a Bool that answers whether the string has no characters at all.
WHAT **Prints false, because the string has characters.**
WHERE A property read on the same username constant, written with a dot just like count on the line above.
WHEN Second line of output, evaluated when this statement runs, immediately after count printed 7.
WHY isEmpty states the intent clearly and can answer instantly, while count == 0 must count every character first.

```
6  print(username.uppercased())
```

WHO The uppercased() method of String, which builds a new string with every letter converted to its capital form.
WHAT **Returns a new string GRACE_H and prints it; username itself still holds grace_h.**
WHERE The new string is a temporary value; the constant is untouched.
WHEN Third line of output; the uppercase copy is created at run time and discarded after printing.
WHY Strings are values in Swift, so methods hand back changed copies and the original can never be modified behind your back.

```
7  let title = "Swift Playgrounds"
```

WHO The compiler infers String from the literal, which contains two words separated by a space.
WHAT **Stores a title containing a space.**
WHERE A second global String constant in top-level code, used by every remaining line except the café example.
WHEN Declared after the three username lines, so their output appears in the console first.
WHY A title with two words gives the search and replace methods below something to work on.

```
8  print(title.lowercased())
```

WHO The lowercased() method of String, the counterpart of uppercased() that converts every letter to lowercase.
WHAT **Builds the new string swift playgrounds and prints it as the fourth line of output.**
WHERE Returns a new string; title is unchanged.
WHEN When this line runs, creating a temporary lowercase copy that disappears once print has written it.
WHY Lowercasing both sides before comparing is a simple way to make a search box ignore capital letters.

```
9  print(title.hasPrefix("Swift"), title.hasSuffix("app"))
```

WHO The hasPrefix and hasSuffix methods of String, each returning a Bool about the start or the end of the text.
WHAT **Prints true false: the title starts with Swift but does not end with app.**
WHERE Both checks run on the same title, one after the other.
WHEN Evaluated left to right before printing.
WHY These checks are case sensitive, so swift in lowercase would not match the prefix.

```
10 print(title.contains("Play"))
```

WHO The contains method of String, which here searches for a whole run of text rather than a single character.
WHAT **Searches anywhere inside the title for Play and prints true.**
WHERE Runs on title; the match can be at any position.
WHEN Sixth line of output, evaluated as this line runs, before the replacement line creates the slug.
WHY contains answers "does this text appear anywhere", the basis of simple search features.

```
11 let slug = title.replacingOccurrences(of: " ", with: "-")
```

WHO Foundation, imported on line 1, which adds the replacingOccurrences(of:with:) method to every String in this file.
WHAT **Builds a new string where every space becomes a hyphen, Swift-Playgrounds, and stores it in slug.**
WHERE A new global constant; title keeps its space.
WHEN When this line runs, after Foundation was linked in at compile time.
WHY A slug is a web-address-friendly version of a title; the argument labels of and with make the call read like a sentence.

```
12 print(slug)
```

WHO The print function, reading the new slug constant rather than the original title.
WHAT **Writes Swift-Playgrounds, showing the hyphen that replaced the space between the two words.**
WHERE Standard output, the seventh line in the console; title still holds the version with a space.
WHEN Right after slug is created, so the replacement can be checked before the program moves on to café.
WHY Printing confirms every space was replaced, not just the first one.

```
13 let cafe = "café"
```

WHO The compiler infers String; the literal contains an accented letter.
WHAT **Stores the four-letter word café, whose last letter, é, carries an accent mark.**
WHERE A global String constant in top-level code; the source file stores the accented letter as Unicode text.
WHEN Near the end of the run.
WHY Accented letters reveal how a language counts text, which matters for names in every language.

```
14 print(cafe.count)
```

WHO The count property, which follows the Unicode rules for what a reader sees as one character.
WHAT **Prints 4, because é counts as one character even if it is stored using more than one byte.**
WHERE The counting happens on the String value in memory.
WHEN Last statement of the run, counting characters at run time and writing the final line of output.
WHY Many languages report bytes or code units instead; Swift counts characters the way people do, so text limits and cursors behave correctly.

Try it

1. Change the username to an empty string "" and run it. Which two lines change?
2. Make the slug lowercase by calling lowercased() on the result.

CODE · RUNS IN THE BROWSER

Characters inside a string

A `String` is a collection of `Character` values, so you can loop over it one character at a time. Here a loop counts vowels, `first` reads the opening character, `reversed()` flips the order, and `split(separator:)` cuts text into pieces. You will meet loops and `??` in full in later lessons; the notes explain enough for now.

```swift
let word = "Playground"
var vowels = 0
for letter in word {
    if "aeiou".contains(letter) {
        vowels += 1
    }
}
print("Vowels:", vowels)
let initial: Character = word.first ?? "?"
print(initial)
print(String(word.reversed()))
let tags = "swift,ios,xcode".split(separator: ",")
print(tags)
print(tags.count)
```

OUTPUT

```
Vowels: 3
P
dnuorgyalP
["swift", "ios", "xcode"]
3
```

Line by line

1 `let word = "Playground"`

- WHO — The compiler infers String from the literal, so word is a collection of Character values.
- WHAT — **Stores the ten-character word Playground, which the loop, first and reversed() will all examine.**
- WHERE — A global String constant in top-level code, read by the loop and by every line after it.
- WHEN — First statement of the run, before the vowel counter and the loop that reads it.
- WHY — A single word keeps the character examples easy to check by eye.

2 `var vowels = 0`

- WHO — Your program declares a counter; the compiler infers Int.
- WHAT — **Creates a variable that starts at 0 and will count vowels.**
- WHERE — Declared outside the loop so it survives after the loop finishes.
- WHEN — Second statement, before the loop begins, so the counter already holds 0 when the first character is checked.
- WHY — A counter must be var because the loop increases it, and it must start at a known value.

3 `for letter in word {`

- WHO — The for-in loop, which asks the string for its characters one at a time.
- WHAT — **Repeats the body once per character, binding each to a new constant named letter of type Character.**
- WHERE — letter exists only inside this loop braces.
- WHEN — The body runs ten times: P, l, a, y, g, r, o, u, n, d.
- WHY — Because a String is a collection of Characters, you can loop over it just like an array.

4 `if "aeiou".contains(letter) {`

- WHO — The contains method, called on a small String of vowels.
- WHAT — **Checks whether the current character is one of a, e, i, o or u.**
- WHERE — Inside the loop body, so it runs once per character.
- WHEN — On every pass through the loop.
- WHY — Asking "is this character in that string" is shorter and clearer than five equality checks joined with ||.

5 `vowels += 1`

- WHO — The += operator on the counter.
- WHAT — **Adds one to vowels each time the check succeeds.**
- WHERE — Inside the if, so it runs only for vowels.
- WHEN — For a, o and u, three times in total.
- WHY — Updating a counter declared outside the loop is the standard way to tally items while looping.

8 `print("Vowels:", vowels)`

- WHO — The print function, given a label and the final value of the counter that the loop updated.
- WHAT — **Prints Vowels: 3; y is not in the vowel list.**
- WHERE — After the loop, where vowels holds its final value.
- WHEN — Once, after all ten passes have finished.
- WHY — Printing after the loop shows the total rather than every intermediate step.

```
9  let initial: Character = word.first ?? "?"
```

WHO The first property and the nil-coalescing operator ??, which the next lesson covers in depth.

WHAT **first returns the opening character only if the string is not empty; ?? supplies "?" otherwise. Here initial is P.**

WHERE The annotation Character makes the quoted "?" a single Character rather than a String.

WHEN Evaluated once, after the loop has finished; first is read and ?? is applied in a single expression.

WHY An empty string has no first character, so Swift makes you plan for that case instead of letting the program crash.

```
10 print(initial)
```

WHO The print function, which writes a Character exactly as it would write a one-letter String.

WHAT **Writes P, the opening character of Playground, as the second line of output.**

WHERE Standard output, directly below the vowel total; initial is only read here, never changed.

WHEN Right after initial is set, before the reversed() line builds its new string.

WHY It confirms that first found the opening character.

```
11 print(String(word.reversed()))
```

WHO The reversed() method and the String initializer.

WHAT **reversed() gives the characters in backward order, and String( ) builds a new string from them: dnuorgyalP.**

WHERE reversed() returns a lightweight reversed view, not a String, so it is wrapped to make a real one.

WHEN Both calls run as this line executes, inner call first.

WHY Returning a view avoids copying until you need to; wrapping in String( ) makes a value you can print or store.

```
12 let tags = "swift,ios,xcode".split(separator: ",")
```

WHO The split(separator:) method of String, which finds every comma in the literal and cuts the text around it.

WHAT **Cuts the text at every comma, producing an array of three pieces.**

WHERE The pieces are Substring values that share memory with the original text.

WHEN When this line runs, near the end of the program; the three pieces are created at run time.

WHY Splitting is how you turn comma-separated input into separate values; convert a piece with String( ) if you keep it long-term.

```
13 print(tags)
```

WHO The print function, which formats arrays in square brackets.

WHAT **Writes ["swift", "ios", "xcode"], each piece in quotes and separated by commas inside square brackets.**

WHERE Standard output, the fourth line in the console; print formats the whole array, not one piece at a time.

WHEN After the split has produced the array, and just before its count is printed.

WHY Seeing quotes around each item tells you the pieces are text, not numbers.

```
14 print(tags.count)
```

WHO The count property of the array.

WHAT **Prints 3, the number of pieces.**

WHERE Read on the tags array, a collection of Substring pieces, using the same count name that strings use.

WHEN Last statement of the run, writing 3 as the final line of output.

WHY count works on every Swift collection, strings and arrays alike, with the same meaning.

Try it

1. Change word to "Swift". Predict the vowel count and the reversed text.
2. Change word to "" and run it. What does initial print now?
3. Add a fourth tag after a comma and run it again.

CODE · RUNS IN THE BROWSER

Formatting decimals with String(format:)

Interpolation prints every digit a `Double` holds, which is wrong for prices. `String(format:)` from Foundation builds text from a **format string**: placeholders such as `%.2f` (a decimal with two places), `%d` (an integer) and `%@` (a string) are filled by the arguments, in order. Here it prints a small receipt.

```
1  import Foundation
2
3  let item = "Bagel"
4  let quantity = 3
5  let price = 2.75
6  let subtotal = price * Double(quantity)
7  let tax = subtotal * 0.07
8  print(String(format: "%d x %@", quantity, item))
9  print(String(format: "Subtotal: $%.2f", subtotal))
10 print(String(format: "Tax:      $%.2f", tax))
11 print(String(format: "Total:    $%.2f", subtotal + tax))
12 print(String(format: "Rating: %5.1f", 4.86))
13 print(String(format: "Rating: %5.1f", 10.0))
```

OUTPUT

```
3 x Bagel
Subtotal: $8.25
Tax:      $0.58
Total:    $8.83
Rating:   4.9
Rating:  10.0
```

Line by line

1 `import Foundation`

- WHO — The compiler, loading the Foundation framework.
- WHAT — **Makes String(format:) available in this file.**
- WHERE — First line of the file, covering everything below.
- WHEN — At compile time; without it, the format calls do not compile.
- WHY — String(format:) is a Foundation feature, not part of the Swift standard library.

3 `let item = "Bagel"`

- WHO — The compiler infers String from the quoted literal, the kind of value a %@ placeholder accepts.
- WHAT — **Stores the item name Bagel, which the first receipt line will print after the quantity.**
- WHERE — A global String constant in top-level code, passed later as the second argument to String(format:).
- WHEN — First statement of the run, executed after Foundation was imported at compile time.
- WHY — It will fill the %@ placeholder below.

4 `let quantity = 3`

- WHO — The compiler infers Int from the whole-number literal 3, the type that %d expects.
- WHAT — **Stores how many bagels were bought.**
- WHERE — A global Int constant in top-level code, used both in the subtotal math and in the first receipt line.
- WHEN — Second statement of the run, right after item and before any arithmetic happens.
- WHY — It will fill the %d placeholder, which expects an integer.

```
5  let price = 2.75
```

WHO The compiler infers Double from the decimal point in the literal 2.75.
WHAT **Stores the unit price of one bagel, 2.75, as a Double that the subtotal multiplies.**
WHERE A global Double constant in top-level code; it is never printed directly, only used in the math.
WHEN Third statement of the run, the last input created before the subtotal is computed.
WHY Prices have cents, so they need a Double here.

```
6  let subtotal = price * Double(quantity)
```

WHO The Double initializer and the multiplication operator for Double.
WHAT **Converts 3 to 3.0 and multiplies, so subtotal is 8.25.**
WHERE A new global Double constant in top-level code, holding the full-precision product for the lines below.
WHEN Fourth statement of the run, after price and quantity both exist, so the multiplication has its inputs.
WHY Int and Double cannot be multiplied directly, so the quantity is converted on purpose.

```
7  let tax = subtotal * 0.07
```

WHO The multiplication operator for Double, combining the subtotal with the literal tax rate 0.07.
WHAT **Computes a 7 percent tax, 0.5775.**
WHERE A global Double constant holding the full-precision value.
WHEN Fifth statement, after subtotal is known, and before any formatted line is printed.
WHY Keeping the unrounded value for math and rounding only for display keeps totals accurate.

```
8  print(String(format: "%d x %@", quantity, item))
```

WHO Foundation, which reads the format string and fills each placeholder from the arguments.
WHAT **Replaces %d with 3 and %@ with Bagel, printing 3 x Bagel.**
WHERE The arguments after the format string are matched to the placeholders from left to right.
WHEN Sixth statement, producing the first receipt line before any money amount is printed.
WHY The letter after % must match the argument type; %d with a String would produce garbage, so check each pairing.

```
9  print(String(format: "Subtotal: $%.2f", subtotal))
```

WHO Foundation formatting with the %.2f placeholder.
WHAT **Prints Subtotal: $8.25, a Double shown with exactly two digits after the decimal point.**
WHERE The dollar sign and label are ordinary text in the format string.
WHEN Seventh statement, the second receipt line; the formatting happens at run time each time the line executes.
WHY The .2 sets the number of decimal places; f stands for a floating-point number.

```
10 print(String(format: "Tax:      $%.2f", tax))
```

WHO Foundation formatting again, reusing the %.2f placeholder, this time for the tax amount.
WHAT **Rounds 0.5775 to two places and prints Tax: $0.58.**
WHERE Only the text is rounded; the constant tax still holds 0.5775.
WHEN Eighth statement, printing the third receipt line right after the subtotal line.
WHY The extra spaces after Tax: line up the dollar signs under Subtotal, so the receipt reads as a column.

```
11 print(String(format: "Total:    $%.2f", subtotal + tax))
```

WHO Double addition followed by Foundation formatting, which receives only the finished sum as its argument.
WHAT **Adds 8.25 and 0.5775 to get 8.8275 and prints Total: $8.83.**
WHERE The sum is a temporary value passed as the argument.
WHEN The addition runs first, while the argument is evaluated; then the format string rounds the sum and prints it.
WHY Rounding the final sum, not each part, avoids cents drifting from the true total.

```swift
print(String(format: "Rating: %5.1f", 4.86))
```

WHO The %5.1f placeholder: a total width of 5 characters with 1 decimal place.
WHAT **Rounds 4.86 to 4.9 and pads it on the left to width 5, printing Rating: 4.9.**
WHERE The padding spaces are added inside the placeholder output, before the number.
WHEN After the receipt is complete, starting a separate two-line example of fixed-width columns.
WHY A fixed width makes numbers of different sizes line up in a column.

```swift
print(String(format: "Rating: %5.1f", 10.0))
```

WHO The same %5.1f placeholder, this time filled by a number with two digits before the decimal point.
WHAT **Prints Rating: 10.0 with one padding space, so its decimal point lines up with the line above.**
WHERE The console, directly under the previous rating.
WHEN Last statement of the run, printing directly below the 4.9 rating so the two columns can be compared.
WHY Aligned columns help in logs and debug output; in an app interface, SwiftUI Text with a format argument handles currency and the user locale for you.

> **Try it**
>
> 1. Change the quantity to 4 and predict the total before you run it.
> 2. Change %.2f to %.1f on the Total line. How does the rounding change?

CHECK YOURSELF

Check your understanding

1. What does this print?

   ```swift
   print(17 / 5, 17 % 5)
   ```

   A. 3.4 2
   B. 3 2
   C. 3 3
   D. 3.4 0.4

2. A fitness app stores `let totalKm = 10` and `let runs = 4`. Which expression gives the average distance per run as 2.5?

   A. totalKm / runs
   B. Double(totalKm / runs)
   C. totalKm / runs * 1.0
   D. Double(totalKm) / Double(runs)

3. Given `let temperature = 72` and `let isRaining = false`, which expression is `true`?

   A. temperature > 70 && !isRaining
   B. temperature > 80 || isRaining
   C. !(temperature > 70)
   D. temperature == 70 && !isRaining

4. Which line prints exactly `3.14`? Assume the file starts with `import Foundation`.

   A. print(Double.pi)
   B. print(Double.pi.rounded())
   C. print(String(format: "%.2f", Double.pi))
   D. print(Int(Double.pi))

5. A teammate who usually writes JavaScript adds `let name = "Mia"` and then `print(name.length)`. What happens?

   A. It prints 3
   B. It does not compile: String has no length
   C. It prints 0
   D. It compiles, then crashes when that line runs

Answers: Numbers, Operators and Strings

1. **B. 3 2** Both operands are `Int`, so `/` performs integer division and drops the fraction: 17 / 5 is 3. `%` gives the remainder, 2. You would see 3.4 only after converting to `Double` first, as in `Double(17) / Double(5)`.
2. **D. Double(totalKm) / Double(runs)** Converting both values before dividing keeps the fraction: 10.0 / 4.0 is 2.5. `Double(totalKm / runs)` is the classic trap: the integer division runs first and produces 2, so converting afterward gives 2.0. Multiplying by 1.0 does not even compile, because Swift never mixes `Int` and `Double` silently.
3. **A. temperature > 70 && !isRaining** 72 > 70 is true, and `!isRaining` flips false to true, so && sees two trues. The `||` choice is tempting because `||` needs only one true side, but here both sides are false: 72 > 80 is false and `isRaining` is false.
4. **C. print(String(format: "%.2f", Double.pi))** `%.2f` formats a `Double` with exactly two digits after the decimal point, giving 3.14. `print(Double.pi)` shows every digit Swift stores, 3.141592653589793; `rounded()` rounds to the nearest whole number but stays a `Double`, printing 3.0; `Int(...)` truncates to 3.
5. **B. It does not compile: String has no length** Swift checks every member name at compile time. The String property is `count`, not `length`, so the build fails with *value of type 'String' has no member 'length'* and nothing runs. It cannot crash at run time, because a program with a compile error never starts.

Optionals: Values That Might Be Missing

CONCEPT

Why Swift has nil

Some questions have no answer. Turn the text `"abc"` into a number? There is none. Ask an empty list for its first item? Nothing is there. Swift spells "no value" as `nil`.

Many languages let any variable secretly be empty. Swift instead makes possible absence part of the **type**. An `Int?` (read "optional Int") is a box that holds either an `Int` or nothing. A plain `Int` always holds a number. Because they are different types, the compiler will not let you use the box as if it were the number. You must open it first, and say what happens when it is empty.

Optional	A type written with a trailing ? that holds a value or nil.
nil	The absence of a value. Only optionals can be nil.
Unwrap	Take the value out of the box, with a plan for when the box is empty.

> **The billion-dollar mistake**
>
> Tony Hoare added null references to a language in 1965 and later called them his "billion-dollar mistake" because of the crashes they caused. In Java or JavaScript, almost any reference can be null and you find out when the program runs. Swift moves that check to compile time.

CODE · RUNS IN THE BROWSER

Optionals in the wild

Everyday operations return an optional when they can fail or come up empty. Run the program and see how `print` shows the box: a value appears wrapped, as in `Optional(42)`, and an empty box prints `nil`. The real compiler also warns on each of these prints, nudging you to unwrap first.

```
let typed = "42"
let parsed = Int(typed)
print(parsed)

let misspelled = Int("forty-two")
print(misspelled)

var nickname: String? = nil
print(nickname)
nickname = "Ace"
print(nickname)

let waitlist = ["Mia", "Omar"]
print(waitlist.first)
let emptyList: [String] = []
print(emptyList.first)
```

OUTPUT

```
Optional(42)
nil
nil
Optional("Ace")
Optional("Mia")
nil
```

Line by line

1 `let typed = "42"`

- WHO — You, the author, standing in for text a person typed into a text field.
- WHAT — **Creates a constant holding the String 42: two characters, not a number yet.**
- WHERE — Top-level code, so typed is a global constant that every line below can read.
- WHEN — Runs first, as soon as the program starts.
- WHY — User input always arrives as text. Turning it into a number is a step that can fail, which is exactly what optionals describe.

2 `let parsed = Int(typed)`

- WHO — The Int type's failable initializer: a way to create an Int that is allowed to produce nothing.
- WHAT — **Tries to read the characters as a whole number. It succeeds, so parsed holds 42 inside an optional.**
- WHERE — Top-level code. The inferred type of parsed is Int? (optional Int), not Int.
- WHEN — Runs once, right after typed is created.
- WHY — Converting text cannot succeed for every string, so the return type is Int? and the possibility of failure is visible to you and the compiler.

3 `print(parsed)`

- WHO — The standard library print function, receiving parsed while it is still an optional Int rather than a plain number.
- WHAT — **Prints Optional(42), showing the box around the value.**
- WHERE — Writes to the console panel under the editor.
- WHEN — Immediately after the conversion on line 2.
- WHY — Printing an optional directly reveals its wrapper. The compiler warns here because you usually want the value, not the box.

5 `let misspelled = Int("forty-two")`

- WHO — The same failable Int initializer as line 2, this time given text made of letters and a hyphen instead of digits.
- WHAT — **Cannot read spelled-out words as digits, so misspelled holds nil.**
- WHERE — Another top-level constant, also inferred as Int?.
- WHEN — Runs after Optional(42) has been printed; the conversion is attempted at run time, so the failure is only discovered then.
- WHY — Instead of crashing, or returning a fake number such as 0, Swift reports the failure honestly as an empty box.

6 `print(misspelled)`

- WHO — print once more, this time handed the empty optional stored in misspelled, which it describes with the word nil.
- WHAT — **Prints nil, which means the box is empty.**
- WHERE — Writes the second line of console output, directly below the Optional(42) printed by line 3.
- WHEN — Immediately after the failed conversion on line 5, and before nickname is declared below.
- WHY — Seeing nil confirms the conversion failed while the program kept running normally.

```
8  var nickname: String? = nil
```

WHO	You, declaring storage that may legitimately be empty.
WHAT	**Creates a variable whose type is optional String and starts it with no value.**
WHERE	Top-level variable. The annotation String? is required because nil alone does not say what kind of value could go in the box.
WHEN	Runs once, at this point in the program.
WHY	Not every user has a nickname. An optional says so in the type, instead of a magic stand-in like an empty string.

```
9  print(nickname)
```

WHO	print, reading the current contents of nickname.
WHAT	**Prints nil because nothing has been stored yet.**
WHERE	Writes the third line of console output, reading the global variable nickname declared one line above.
WHEN	Before the assignment on the next line, so it sees the empty box.
WHY	Printing before and after a change shows that the same variable can move between empty and full.

```
10 nickname = "Ace"
```

WHO	Your code, acting like a user who saves a nickname.
WHAT	**Stores the String Ace in the box. Swift wraps it in an optional for you.**
WHERE	Assigns to the variable declared on line 8; this is why it was declared with var.
WHEN	After the first nickname print, so that earlier output still says nil.
WHY	You never write Optional(...) by hand. A plain value is wrapped automatically when the destination is optional.

```
11 print(nickname)
```

WHO	print again, on the same variable.
WHAT	**Prints Optional("Ace"). The quotes show a String is inside; the wrapper shows it is still boxed.**
WHERE	Writes the fourth line of console output, reading the same global variable after line 10 filled it.
WHEN	After the assignment, so the box is now full.
WHY	A full optional is still an optional. You must unwrap it before using it as a plain String.

```
13 let waitlist = ["Mia", "Omar"]
```

WHO	You, building a small array (an ordered list) of names.
WHAT	**Creates an array of two Strings; Swift infers its type as an array of String.**
WHERE	Top-level constant. Arrays get their own lesson later; here you only need their first property.
WHEN	Runs once, after the nickname prints, building the two-element array before line 14 asks for its first item.
WHY	Lists are a common source of optionals, because a list might be empty.

```
14 print(waitlist.first)
```

WHO	The array's first property, provided by the standard library.
WHAT	**Returns the first name wrapped in an optional, so this prints Optional("Mia").**
WHERE	Reads from waitlist and writes to the console.
WHEN	At run time on line 14; first is a computed property, worked out from the array contents at the moment you ask.
WHY	first must return an optional because some arrays are empty, so even a full array hands you a box.

```
15 let emptyList: [String] = []
```

WHO	You, creating a list with nothing in it.
WHAT	**Makes an array of Strings with zero elements.**
WHERE	Top-level constant. The annotation names the element type because an empty pair of brackets gives Swift nothing to infer from.
WHEN	Runs once, near the end of the program.
WHY	Sets up the case where there truly is no first item.

16 `print(emptyList.first)`

- **WHO** The same first property, on a different array.
- **WHAT** **Prints nil because there is no first element.**
- **WHERE** Writes the sixth and final line of console output, directly below Optional("Mia") from the full waitlist.
- **WHEN** Last statement, so the program ends after it.
- **WHY** One type, String?, covers both outcomes, and the compiler makes you plan for both.

> **Try it**
>
> 1. Change "42" on line 1 to "4.2" and run again. Why does Int give up?
> 2. Start nickname as "Zoe" instead of nil. What do both prints show now?

CODE · RUNS IN THE BROWSER

Mistake: using an optional as a number

This is the most common optional mistake. The program tries to add 1 to an `Int?` as if it were a plain `Int`. Run it and read the compiler message: the box might be empty, and you cannot add 1 to nothing.

```
let typedAge = "20"
let age: Int? = Int(typedAge)
let nextYear = age + 1
print("Next year you will be \(nextYear)")
```

COMPILER ERROR (ON PURPOSE)

```
value of optional type 'Int?' must be unwrapped
```

Line by line

1 `let typedAge = "20"`

- **WHO** You, simulating text from an age field on a sign-up form.
- **WHAT** **Stores the characters 20 as a String.**
- **WHERE** Top-level constant at the start of the program.
- **WHEN** Would run first, but this program never runs because it fails to compile.
- **WHY** Text input is the classic source of an optional number.

2 `let age: Int? = Int(typedAge)`

- **WHO** The failable Int initializer, plus your explicit type annotation.
- **WHAT** **Converts the text and stores the result in age, declared as Int? to make the box obvious.**
- **WHERE** Top-level constant. Writing Int? here is optional; Swift would infer it anyway.
- **WHEN** Type-checked at compile time; it is valid, so the compiler moves on to line 3.
- **WHY** Spelling out Int? shows clearly that age is a box, which is the root of the error on the next line.

3 `let nextYear = age + 1`

- **WHO** The compiler's type checker, which rejects this line before anything runs.
- **WHAT** **Tries to add the Int 1 to age, an Int?, and fails with: value of optional type 'Int?' must be unwrapped.**
- **WHERE** Line 3 of top-level code; the error points at age.
- **WHEN** At compile time, so no line runs, not even line 1.
- **WHY** The + operator for numbers needs two Ints. If age were nil there would be nothing to add to, so Swift refuses to guess.

```
4  print("Next year you will be \(nextYear)")
```

WHO	print, which never gets a chance to run.
WHAT	**Would print a sentence that includes nextYear through string interpolation.**
WHERE	The fourth and final top-level statement, which would have read the global constant nextYear declared on line 3.
WHEN	Never, because compilation stopped at line 3.
WHY	A compile error stops the whole program. That is the point: the bug is caught on your Mac instead of on someone's phone.

> **Try it**
>
> 1. Remove the ? from Int? on line 2. Which line does the error point to now, and why?
> 2. Change line 3 to let nextYear = (age ?? 0) + 1 and run it. What prints?

CODE · RUNS IN THE BROWSER

The fix: unwrap with if let

`if let` opens the box safely. If the optional holds a value, Swift binds it to a new, non-optional constant and runs the first block. If it is `nil`, the `else` block runs instead. Swift 5.7 added a shorthand, `if let age {`, that reuses the same name, and `if var` gives you a copy you can change.

```
1  let typedAge = "20"
2  let age: Int? = Int(typedAge)
3
4  if let unwrappedAge = age {
5      print("Next year you will be \(unwrappedAge + 1)")
6  } else {
7      print("That was not a number")
8  }
9
10 if let age {
11     print("Shorthand works too: \(age + 1)")
12 }
13
14 if var cups = Int("3") {
15     cups += 1
16     print("Cups after a refill: \(cups)")
17 }
```

OUTPUT

```
Next year you will be 21
Shorthand works too: 21
Cups after a refill: 4
```

Line by line

```
1  let typedAge = "20"
```

WHO	You, providing the same sample input as the mistake.
WHAT	**Stores the two characters 2 and 0 as a String; Swift does not treat them as a number until they are converted.**
WHERE	A global constant in top-level code, readable by all three unwrapping examples further down the program.
WHEN	Runs first, before the conversion on line 2 and before any of the three unwrapping examples.
WHY	Keeping the input identical to the previous step makes it clear that only the unwrapping changed.

```
2  let age: Int? = Int(typedAge)
```

WHO	The failable Int initializer from the standard library, the same call that led to the compile error in the previous step.
WHAT	**Produces Optional(20) and stores it in age.**
WHERE	A global constant whose declared type is Int?, so every later line sees a box, never a bare Int.
WHEN	Runs once, before any of the checks below.
WHY	age is still a box; the code below shows three safe ways to open it.

```
4  if let unwrappedAge = age {
```

WHO	The Swift runtime, which looks inside the optional at this moment.
WHAT	**Because age holds 20, it creates a constant unwrappedAge of type Int with that value and enters the braces.**
WHERE	unwrappedAge exists only inside this block's braces, not before or after.
WHEN	At run time, each time execution reaches this if.
WHY	Binding to a new constant gives you a plain Int you can do math with, and the compiler knows it can never be nil inside the block.

```
5  print("Next year you will be \(unwrappedAge + 1)")
```

WHO	print, receiving a String interpolated from unwrappedAge, the plain Int that if let pulled out of the box.
WHAT	**Computes 20 + 1 inside the interpolation and prints Next year you will be 21.**
WHERE	Inside the if block, where unwrappedAge is a plain Int.
WHEN	Only when age held a value.
WHY	The arithmetic that failed to compile in the mistake is legal here, because the box has been opened.

```
6  } else {
```

WHO	The else clause of the if let statement, the path Swift takes when the optional turns out to be empty.
WHAT	**Starts the block that runs when age is nil.**
WHERE	Attached directly to the if let above it.
WHEN	Only when the conversion failed; skipped in this run.
WHY	Handling the empty case on purpose is what makes optionals safe: you decide what the user sees.

```
7  print("That was not a number")
```

WHO	print, placed in the else branch as the fallback message for input that could not be converted.
WHAT	**Would print a friendly message instead of crashing.**
WHERE	Inside the else block, where unwrappedAge does not exist, because nothing was unwrapped on this path.
WHEN	Not in this run, because age has a value.
WHY	In a real app this is where you would show an error under the text field.

```
10 if let age {
```

WHO The same if let mechanism, written in the shorthand form.
WHAT **Means if let age = age: inside the braces, a new non-optional age hides (shadows) the optional one.**
WHERE The plain-Int age exists only inside these braces; outside, age is still Int?.
WHEN Checked at run time when execution reaches this line.
WHY Repeating the same name twice was so common that Swift added this shorter form. It reads as "if age has a value".

```
11 print("Shorthand works too: \(age + 1)")
```

WHO print, interpolating age, which inside these braces is the shadowing plain Int rather than the optional global.
WHAT **Adds 1 to the unwrapped 20 and prints Shorthand works too: 21, the same result as the long form above.**
WHERE Inside the shorthand block, where age is a plain Int.
WHEN Only when the optional held a value.
WHY Same result as the long form with less to read. Most modern Swift code uses this style.

```
14 if var cups = Int("3") {
```

WHO The runtime, checking the result of a conversion directly.
WHAT **Converts the text 3, unwraps it, and binds it to a variable cups that you may change.**
WHERE cups is a local variable that lives only inside this block.
WHEN At run time, right after the shorthand example.
WHY if var is the choice when you need to modify the unwrapped value; the original optional is never touched.

```
15 cups += 1
```

WHO Your code, changing the local copy.
WHAT **Adds 1, so cups becomes 4.**
WHERE Inside the if var block, changing only the local variable cups; nothing outside these braces is affected.
WHEN Runs once, right after if var converts the text 3; it would be skipped entirely if the conversion failed.
WHY This line would not compile with if let, because a let constant cannot change.

```
16 print("Cups after a refill: \(cups)")
```

WHO print, reading the local variable cups after the compound assignment changed it from 3 to 4.
WHAT **Prints Cups after a refill: 4.**
WHERE The last line of the if var block; after the closing brace, cups no longer exists.
WHEN After the increment on the previous line.
WHY Shows that the unwrapped copy really did change.

> **Try it**
>
> 1. Change "20" on line 1 to "twenty". Which lines print now?
> 2. Add an else block to the shorthand if let that prints No age.

CODE · RUNS IN THE BROWSER

guard let: exit early

A **function** is a named block of code you run by calling its name; functions get their own lesson later. When a function cannot do its job without a value, `guard let` checks at the top. If the optional is `nil`, the `else` block must leave the function. After the guard, the unwrapped value is available for the rest of the function.

```swift
func logSteps(_ text: String) {
    guard let steps = Int(text) else {
        print("Not a step count: \(text)")
        return
    }
    let remaining = max(10000 - steps, 0)
    print("\(steps) steps, \(remaining) to go")
}

logSteps("6500")
logSteps("lots")
logSteps("12000")
```

OUTPUT

```
6500 steps, 3500 to go
Not a step count: lots
12000 steps, 0 to go
```

Line by line

1 `func logSteps(_ text: String) {`

- **WHO** You, declaring a function the program can call by name.
- **WHAT** **Defines logSteps, which takes one String called text. The underscore lets callers pass it without a label.**
- **WHERE** Declared at top level. Its body, between the braces, runs only when it is called.
- **WHEN** Compiled ahead of time. The body runs once per call, on lines 10 to 12.
- **WHY** Wrapping the check in a function lets the same logic handle three different inputs without repeating code.

2 `guard let steps = Int(text) else {`

- **WHO** The guard statement, which enforces a requirement at the start of the function.
- **WHAT** **Tries to convert text to an Int. On success, steps is a plain Int; on failure, the else block runs.**
- **WHERE** Inside logSteps. Unlike if let, steps stays in scope for the rest of the function body below the guard.
- **WHEN** On every call, before any other work.
- **WHY** Checking requirements first and leaving early keeps the main logic unindented, on the left edge, where it is easy to read.

3 `print("Not a step count: \(text)")`

- **WHO** print, inside the guard's else block.
- **WHAT** **Explains which input was rejected, for example Not a step count: lots.**
- **WHERE** The else block, which can still see text but not steps.
- **WHEN** Only on a call whose text cannot be converted, which in this program is the second call, the one passing lots.
- **WHY** Saying why the input was refused is friendlier than silently doing nothing.

4 `return`

- **WHO** The compiler insists on this: a guard's else block must exit the current scope.
- **WHAT** **Leaves logSteps immediately, so lines 6 and 7 do not run for bad input.**
- **WHERE** Last statement of the else block.
- **WHEN** Right after the message, only for invalid text.
- **WHY** If the else block could fall through, steps would be used with no value. Swift rejects a guard else that does not return, break, continue or throw.

```
6  let remaining = max(10000 - steps, 0)
```

WHO — The standard library max function, working with your arithmetic.
WHAT — **Subtracts steps from a 10000-step daily goal and keeps the larger of that and 0, so it never goes negative.**
WHERE — Inside logSteps, after the guard. remaining is local to this call.
WHEN — On each call that gets past the guard: the first and third calls here, but never the call that passes lots.
WHY — steps is a plain Int here with no unwrapping, because the guard already proved it exists.

```
7  print("\(steps) steps, \(remaining) to go")
```

WHO — print, building one line from two locals of this call: steps from the guard and remaining from the line above.
WHAT — **Prints a progress line such as 6500 steps, 3500 to go.**
WHERE — The last statement of the function body.
WHEN — At the end of each successful call.
WHY — This is the happy path, the code that runs when input is valid, and it reads straight down without nesting.

```
10 logSteps("6500")
```

WHO — Top-level code, making the first call to logSteps and handing it the text 6500 as its text parameter.
WHAT — **Passes valid text, so it prints 6500 steps, 3500 to go.**
WHERE — Outside the function, in the program's main flow.
WHEN — First call; this is when the function body actually runs.
WHY — A normal input shows the happy path first.

```
11 logSteps("lots")
```

WHO — Top-level code, calling the function again.
WHAT — **Passes text that is not a number, so the guard prints the error message and returns.**
WHERE — Top-level code after the function declaration; each call starts the function body fresh with a new text parameter.
WHEN — Second call, after the first one has completely finished.
WHY — Exercises the else branch so you can see the early exit.

```
12 logSteps("12000")
```

WHO — Top-level code, making a third call.
WHAT — **Passes a value above the goal, so max keeps remaining at 0 and it prints 12000 steps, 0 to go.**
WHERE — Top-level code, the last line of the file; this call gets its own steps and remaining, unrelated to earlier calls.
WHEN — Last call, so the program ends after it.
WHY — Tests an edge case: going past the goal should never show a negative number.

> **Try it**
>
> 1. Add a fourth call with your own step count and run it.
> 2. Change the goal from 10000 to 8000. Predict the first line before you run.
> 3. Call logSteps(" 500") with a leading space. Is that a number to Int?

CODE · RUNS IN THE BROWSER

Defaults with ?? and chaining with ?.

Two operators handle optionals inside a single expression. The **nil-coalescing operator** `??` unwraps a value or falls back to a default you supply. **Optional chaining** `?.` calls a method or reads a property only if the box holds a value; otherwise the whole expression becomes `nil`. The result of a chain is always optional.

```swift
let savedName: String? = nil
let displayName = savedName ?? "Guest"
print("Welcome, \(displayName)")

let typedCups = "three"
let cups = Int(typedCups) ?? 1
print("Brewing \(cups) cup")

let city: String? = "Lisbon"
let shout = city?.uppercased()
print(shout ?? "NO CITY")

let missingCity: String? = nil
print(missingCity?.count ?? 0)
let letters = city?.count
print(letters)
```

OUTPUT

```
Welcome, Guest
Brewing 1 cup
LISBON
0
Optional(6)
```

Line by line

1 `let savedName: String? = nil`

- **WHO** You, simulating a profile where the user never entered a name.
- **WHAT** **Creates an optional String that holds no value; the annotation is required because nil alone does not reveal a type.**
- **WHERE** A global constant in top-level code, typed String? so it can represent a profile field that was left blank.
- **WHEN** Runs first, before any default or optional chain is tried on the lines below.
- **WHY** An empty profile field is a realistic case your interface must still handle gracefully.

2 `let displayName = savedName ?? "Guest"`

- **WHO** The nil-coalescing operator ??, which looks inside savedName and falls back to the String on its right when the box is empty.
- **WHAT** **Because savedName is nil, it uses the default Guest. displayName is a plain String.**
- **WHERE** Top-level constant; its type is String, not String?.
- **WHEN** Evaluated once, right after savedName exists.
- **WHY** ?? turns an optional into a guaranteed value in one readable expression, ideal for text that must always show something.

3 `print("Welcome, \(displayName)")`

- **WHO** print, interpolating displayName, a guaranteed plain String that needs no further unwrapping.
- **WHAT** **Prints Welcome, Guest, because the default supplied by ?? stood in for the missing saved name.**
- **WHERE** Writes the first line of console output, reading displayName from the global constant above.
- **WHEN** Right after the default was chosen.
- **WHY** No unwrapping is needed here because ?? already produced a non-optional value.

```
5  let typedCups = "three"
```

WHO — You, simulating messy input in a coffee-order field.
WHAT — **Stores the word three, which Int cannot read.**
WHERE — A global constant holding a String, standing in for whatever arrived from a text field on an order form.
WHEN — Runs after Welcome, Guest has been printed, as the start of the second example.
WHY — Real users type words, spaces and typos, so conversions fail often.

```
6  let cups = Int(typedCups) ?? 1
```

WHO — The failable Int initializer followed by ??.
WHAT — **The conversion returns nil, so cups becomes the default 1.**
WHERE — A global constant of plain type Int, because ?? always produces a value even when the conversion fails.
WHEN — Evaluated once on this line: the conversion runs first and returns nil, and only then does ?? supply the 1.
WHY — A sensible default keeps the app working. Choose defaults that make sense to the user, not automatically 0.

```
7  print("Brewing \(cups) cup")
```

WHO — print, reading the unwrapped cup count.
WHAT — **Prints Brewing 1 cup, using the fallback value because the word three could not be converted.**
WHERE — Writes the second line of console output, reading the global constant cups.
WHEN — After ?? has already settled on the fallback, so cups is a plain 1 by the time print reads it.
WHY — Shows that the program carried on with the default instead of failing.

```
9  let city: String? = "Lisbon"
```

WHO — You, providing an optional that does hold a value.
WHAT — **Stores Lisbon inside an optional String.**
WHERE — A global constant of type String?, holding a value now but typed to allow nil, like a city field a user may skip.
WHEN — Runs in the middle of the program.
WHY — Optional chaining behaves differently for full and empty boxes, so this example uses one of each.

```
10 let shout = city?.uppercased()
```

WHO — Optional chaining, the ?. operator, deciding whether the String method uppercased() gets called at all.
WHAT — **Because city holds Lisbon, uppercased() runs and shout becomes Optional("LISBON").**
WHERE — Top-level constant. Its type is String?, because a chain can always come up empty.
WHEN — Evaluated once, right after city is set.
WHY — The ?. means "only if there is a value". If city were nil, uppercased() would be skipped and shout would be nil instead of crashing.

```
11 print(shout ?? "NO CITY")
```

WHO — print, with ?? supplying a fallback.
WHAT — **Unwraps shout, which holds a value, and prints LISBON; the fallback text NO CITY is not needed this time.**
WHERE — Writes the third line of console output, directly below Brewing 1 cup.
WHEN — After the chain on the previous line.
WHY — Using ?? at the point of display keeps the warning away and decides what an empty value looks like.

```
13 let missingCity: String? = nil
```

WHO — You, providing an empty optional for contrast.
WHAT — **Creates an optional String with no value.**
WHERE — Another global String? constant, the empty twin of city, declared so both cases sit side by side.
WHEN — Runs after LISBON is printed, as the setup for the empty-box chain on the next line.
WHY — Lets you see a chain stop early.

```
14 print(missingCity?.count ?? 0)
```

WHO — Optional chaining and nil-coalescing working together.
WHAT — **missingCity is nil, so count is never read, the chain yields nil, and ?? substitutes 0. Prints 0.**
WHERE — Writes the fourth line of console output; the whole chain and its fallback are evaluated inside the print call.
WHEN — Evaluated left to right when the line runs.
WHY — Combining ?. with ?? is the standard pattern: try to reach inside, and supply a fallback if anything is missing.

```
15 let letters = city?.count
```

WHO — Optional chaining on the full box.
WHAT — **Reads the count of Lisbon, so letters is Optional(6).**
WHERE — Top-level constant of type Int?, even though count itself is a plain Int.
WHEN — Runs after 0 is printed for missingCity, chaining this time through the full box city.
WHY — Any value reached through ?. comes back wrapped, because Swift cannot know in advance whether the chain will succeed.

```
16 print(letters)
```

WHO — print, handed letters without unwrapping, so it receives the whole optional box rather than the Int inside.
WHAT — **Prints Optional(6). The real compiler warns that an optional is being printed.**
WHERE — Writes the fifth and final line of console output, below the 0 printed for the empty chain.
WHEN — Runs last; after it the program ends, so letters is never unwrapped anywhere.
WHY — The wrapper in the output proves that chaining made the result optional.

Try it

1. Change savedName to "Priya" and run it. Which line changes?
2. Set city to nil on line 9. Predict the last three lines before you run.

CODE · RUNS IN THE BROWSER

Force unwrapping and why it crashes

Writing `!` after an optional **force unwraps** it: you promise Swift the box is not empty. If you are wrong, the app stops immediately with `Fatal error: Unexpectedly found nil while unwrapping an Optional value`. On a phone, that looks like the app vanishing. Use `!` only when `nil` is truly impossible, and say why in a comment.

```swift
// Safe: this text is fixed and is always a number.
let maxPlayers = Int("4")!
print("Up to \(maxPlayers) players")

let typedPin = "12a4"
let parsedPin = Int(typedPin)
print(parsedPin == nil)

// Uncomment the next line to see the crash:
// print(parsedPin!)

if let pin = parsedPin {
    print("PIN accepted: \(pin)")
} else {
    print("Please type digits only")
}
```

OUTPUT

```
Up to 4 players
true
Please type digits only
```

Line by line

1 `// Safe: this text is fixed and is always a number.`

- WHO — You, writing for future readers; the compiler ignores comments.
- WHAT — **Records why the force unwrap on the next line cannot fail.**
- WHERE — A comment line in top-level code. It produces no instructions.
- WHEN — Never executes. It matters when a teammate reads or reviews the code.
- WHY — Every ! is a claim. A comment documents the reasoning so reviewers can check it, and many teams require one.

2 `let maxPlayers = Int("4")!`

- WHO — The failable Int initializer, followed by the force-unwrap operator !.
- WHAT — **Converts the fixed text 4 and unwraps it at once, so maxPlayers is a plain Int holding 4.**
- WHERE — Top-level constant of type Int, not Int?.
- WHEN — Runs first; the unwrap check happens at run time.
- WHY — The input is a literal the program controls, so it can never be nil. This is one of the few defensible uses of !.

3 `print("Up to \(maxPlayers) players")`

- WHO — print, interpolating maxPlayers, the plain Int that the force unwrap on line 2 produced.
- WHAT — **Prints Up to 4 players with no Optional wrapper, because the ! already removed the box.**
- WHERE — Writes the first line of console output, reading maxPlayers from the global constant above.
- WHEN — Right after the force unwrap on line 2 succeeded; had it found nil, the program would have stopped before this line.
- WHY — Confirms that a successful force unwrap gives you a plain value.

5 `let typedPin = "12a4"`

- WHO — You, simulating a PIN that a user mistyped.
- WHAT — **Stores text that contains a letter.**
- WHERE — A global String constant, standing in for a PIN read from a text field on a sign-in screen.
- WHEN — Runs after Up to 4 players is printed, starting the risky half of the example.
- WHY — User input is exactly the kind of value you must never force unwrap.

6 `let parsedPin = Int(typedPin)`

- WHO — The failable Int initializer, given text where the letter a sits between the digits.
- WHAT — **Fails on the letter a, so parsedPin is nil.**
- WHERE — A global constant of type Int?, kept optional on purpose instead of being force unwrapped.
- WHEN — Runs once, right after typedPin is set and before the nil comparison on line 7.
- WHY — Keeping the result optional leaves the decision about the empty case to the code below.

7 `print(parsedPin == nil)`

- WHO — The == operator, comparing an optional with nil.
- WHAT — **Prints true, because the conversion failed and the box is empty.**
- WHERE — Writes the second line of console output; the comparison produces a Bool, which print shows as true.
- WHEN — Runs before the disabled crash line and the safe if let below, so the box state is reported first.
- WHY — Comparing to nil is a quick way to ask whether a box is empty when you do not need the value itself.

9 `// Uncomment the next line to see the crash:`

- WHO: You, leaving instructions for whoever experiments with this code.
- WHAT: **Explains that the following line is disabled on purpose.**
- WHERE: A comment in top-level code placed directly above the disabled line, so the warning and the line stay together.
- WHEN: Never runs; it is read, not executed.
- WHY: Disabled code without an explanation confuses readers, so say why it is there.

10 `// print(parsedPin!)`

- WHO: Nobody yet: the line is commented out, so the compiler skips it.
- WHAT: **If enabled, it would force unwrap nil and stop the program with a fatal error.**
- WHERE: Top-level code, between the nil check and the safe version.
- WHEN: If enabled, it would run after line 7 and end the program on the spot; lines 12 to 16 would never run.
- WHY: Kept as a comment so you can switch the crash on, see it for yourself, and switch it off again.

12 `if let pin = parsedPin {`

- WHO: The runtime, checking the optional safely.
- WHAT: **Finds the box empty, so it skips this block and jumps to the else branch.**
- WHERE: pin would exist only inside these braces.
- WHEN: At run time, after the nil check.
- WHY: This is the safe replacement for the crashing line: same input, no crash.

13 `print("PIN accepted: \(pin)")`

- WHO: print, sitting in the success branch where pin is a plain Int taken out of parsedPin.
- WHAT: **Would print PIN accepted followed by the digits, but this run never reaches it.**
- WHERE: Inside the if let block, the only place where the unwrapped constant pin exists.
- WHEN: Skipped in this run, because parsedPin is nil.
- WHY: The success path only runs with a real number.

14 `} else {`

- WHO: The else branch of the if let.
- WHAT: **Starts the block for the empty case.**
- WHERE: Attached to the if let above.
- WHEN: Runs in this program, because the PIN did not parse.
- WHY: Handling the empty case is what turns a crash into a helpful message.

15 `print("Please type digits only")`

- WHO: print, sitting in the else branch as the friendly response to a PIN that cannot be converted.
- WHAT: **Prints Please type digits only, the third and last line of output in this run.**
- WHERE: Inside the else block, so no unwrapped pin is available here; only the failure message runs.
- WHEN: Only when parsedPin is nil, which is the case in this run because typedPin contains a letter.
- WHY: In a real app, this message would appear next to the PIN field instead of the app closing.

> **Try it**
>
> 1. Remove the // at the start of line 10 and run. Read the crash, then put the // back.
> 2. Change typedPin to "1234". What do lines 7 and 12 do now?

INTERACTIVE

The optional box

Put something in the `nickname` box: a name, an empty string, or `nil`. Pick a technique to open it (`!`, `??`, `if let`, `guard let` or `?.`), predict what prints, then press **Open the box**. Watch which lines run,

what the console shows, and the type of the result. Notice that an empty string is still a value, that only `!` can crash, and that `?.` hands you another optional.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. What does this program print?

```
let input = "12"
let count = Int(input) ?? 0
print(count + 1)
```

   A. 13
   B. Optional(13)
   C. 1
   D. Nothing; it does not compile

2. Which line stops this program from compiling?

```
let scores = [8, 5, 9]
let top = scores.max()
print(top * 2)
```

   A. Line 1: arrays need a type annotation
   B. Line 2: max() does not exist on arrays
   C. Line 3: top is an Int? and must be unwrapped first
   D. None of them; it prints 18

3. A function loads a profile and needs a user ID to do anything useful. The ID is optional. Which approach is the best design?
   A. Force unwrap the ID with !, since it is usually present
   B. Use ?? to substitute the ID 0 when it is missing
   C. Wrap the whole body in if let and leave the else empty
   D. Use guard let at the top and return early when the ID is nil

4. What does this program print?

```
let title: String? = nil
let length = title?.count
print(length ?? -1)
```

   A. 0
   B. -1
   C. nil
   D. It crashes

Answers: Optionals: Values That Might Be Missing

1. **A. 13** `Int("12")` succeeds, and `??` unwraps it, so `count` is a plain `Int` holding 12, and 12 + 1 is 13. `Optional(13)` looks plausible, but `??` always produces a non-optional value. You would only see 1 if the conversion had failed and the default 0 had been used.
2. **C. Line 3: top is an Int? and must be unwrapped first** `max()` returns an optional, because an empty array has no largest element. So `top` is `Int?`, and multiplying a box by 2 fails with "value of optional type 'Int?' must be unwrapped". The array literal on line 1 infers its type fine. Fixes include `if let top` or `(top ?? 0) * 2`.
3. **D. Use guard let at the top and return early when the ID is nil** `guard let` states the requirement up front, exits cleanly, and leaves the main logic unindented with a plain, non-optional ID. "Usually present" is not "always present", so `!` will eventually crash. A fake ID of 0 hides the bug and may load the wrong profile. `if let` works, but it buries the main code inside braces and makes the empty case easy to forget.
4. **B. -1** `title` is `nil`, so the chain stops before reading `count` and `length` is `nil`. Then `??` supplies -1. The answer 0 is tempting ("an empty title has 0 letters"), but `nil` is not an empty string: there is no string at all. Optional chaining never crashes; only `!` can.

Arrays, Dictionaries and Sets

CONCEPT

Three ways to hold many values

Real apps juggle groups of values: the songs in a playlist, the stock of each drink in a coffee shop, the tags on a photo. Swift gives you three **collection types** (types that hold many values of one type). Picking the right one makes your code shorter and faster.

Every collection is **typed**: `[String]` is an array of strings, `[String: Int]` is a dictionary from strings to integers, and `Set<String>` is a set of strings. Put a number into a string array and the compiler stops you before the app runs. Collections are also **values**: assigning one to a new variable makes an independent copy.

Array	An ordered list. Items keep their position and are read by index, counting from 0. Duplicates are allowed.
Dictionary	Pairs each unique key with one value, like a word and its definition. Lookups are by key, and there is no fixed order.
Set	Unique items with no order, built for fast questions such as "is ios one of the tags?"

> **Which one should I pick?**
>
> Need order, or the same item more than once? Use an array. Need to find a value by a name or ID? Use a dictionary. Need uniqueness or very fast "contains" checks? Use a set.

CODE · RUNS IN THE BROWSER

Arrays: an ordered list

An **array** keeps items in order. Each item has an **index**, its position counted from 0, so the last index is always `count - 1`. The properties `first` and `last` return optionals, because an empty array has neither.

```
var playlist: [String] = ["Intro", "Sunrise", "Drive"]
print(playlist.count)
print(playlist.isEmpty)
print(playlist[0])
print(playlist[playlist.count - 1])
playlist.append("Night Bus")
print(playlist)
if let opener = playlist.first {
    print("Opens with \(opener)")
}
print(playlist.last ?? "No songs")
```

OUTPUT

```
3
false
Intro
Drive
["Intro", "Sunrise", "Drive", "Night Bus"]
Opens with Intro
Night Bus
```

Line by line

1 `var playlist: [String] = ["Intro", "Sunrise", "Drive"]`

- WHO — The compiler, which records that playlist may only ever hold String values.
- WHAT — **Creates a variable array of three song titles, stored at indexes 0, 1 and 2.**
- WHERE — A variable in the program's top-level code, visible to every line below it.
- WHEN — Runs first, before any line reads the playlist.
- WHY — It is var, not let, because line 6 adds a song; the [String] annotation documents the element type even though Swift could infer it.

2 `print(playlist.count)`

- WHO — The Array type, which always knows how many elements it holds.
- WHAT — **Prints 3, the number of songs currently in the array.**
- WHERE — count is a property stored with the array itself, so reading it does not walk over the items.
- WHEN — Right after the array is created, before the append on line 6.
- WHY — count is the safe upper limit for indexes: valid positions run from 0 up to count minus 1.

3 `print(playlist.isEmpty)`

- WHO — The Array type, answering a yes-or-no question about itself.
- WHAT — **Prints false, because the playlist has three items.**
- WHERE — A Bool property found on every Swift collection: arrays, dictionaries, sets and strings.
- WHEN — Evaluated once, at this line, against the array as it is right now.
- WHY — isEmpty states the intent more clearly than comparing count to 0, and it is the style Swift developers expect.

4 `print(playlist[0])`

- WHO — The array subscript, the square-bracket syntax for reading an element by its index.
- WHAT — **Reads the element at index 0 and prints Intro, the first song.**
- WHERE — Index 0 is the first slot in the array's storage; Swift arrays always start counting at zero.
- WHEN — At run time; the compiler cannot know whether an index exists, so the check happens as the line runs.
- WHY — Zero-based indexing matches C, Java and JavaScript, so habits from other languages carry over.

5 `print(playlist[playlist.count - 1])`

- WHO — Your code computing an index, and the subscript that uses it.
- WHAT — **Computes 3 minus 1 to get index 2 and prints Drive, the last song.**
- WHERE — The inner expression is evaluated first, then its result is used as the position in the array.
- WHEN — Runs before the append, so the last index is still 2.
- WHY — This is the off-by-one rule in action: playlist[playlist.count] is one past the end and would stop the program with Index out of range.

```
6  playlist.append("Night Bus")
```

WHO The array's append method, which changes the array in place.
WHAT **Adds Night Bus to the end, growing the playlist to four songs at indexes 0 through 3.**
WHERE Modifies the variable playlist itself; append is a mutating method, so it only works on a var.
WHEN After line 5, so every later line sees four songs.
WHY Adding to the end is the cheapest way to grow an array, because no existing element has to move.

```
7  print(playlist)
```

WHO The print function, which knows how to format a whole array.
WHAT **Prints all four titles in order, inside square brackets, with quotes around each string.**
WHERE Writes to the console; the bracket format matches the way you write an array literal in code.
WHEN After the append, so Night Bus appears at the end.
WHY Printing the whole array is a quick way to check its contents and order while you debug.

```
8  if let opener = playlist.first {
```

WHO The Array type, which returns first as an optional String, and if let, which unwraps it.
WHAT **Unwraps the first element into the constant opener when the array is not empty.**
WHERE opener exists only inside the braces of this if statement.
WHEN Checked at run time: if the playlist were empty, first would be nil and the body would be skipped.
WHY first never crashes, unlike playlist[0] on an empty array, so it is the safer way to read the first item.

```
9  print("Opens with \(opener)")
```

WHO The body of the if let, which only runs when a first song exists.
WHAT **Prints Opens with Intro, using string interpolation to insert the title.**
WHERE Inside the if block, where opener is a plain String rather than an optional.
WHEN Only when line 8 found a value, which it does here.
WHY Inside the braces you use the unwrapped value directly, with no ? or ! needed.

```
11 print(playlist.last ?? "No songs")
```

WHO The nil-coalescing operator ??, which supplies a fallback for an optional.
WHAT **Prints Night Bus, the last element; it would print No songs if the array were empty.**
WHERE A single expression that produces a plain String, so print never sees an optional.
WHEN The final line of the program, after every change to the playlist.
WHY last is optional for the same reason as first, and ?? handles the empty case in one line.

> **Try it**
>
> 1. Change the 0 on line 4 to 2 and run it again. Which song prints?
> 2. Append a fifth song after line 6. What do the last three lines print now?
> 3. Change line 5 to read playlist[playlist.count] and run it. Read the error: that index is one past the end.

CODE · RUNS IN THE BROWSER

Looping and searching an array

Most array work is walking through the items or finding one. `enumerated()` hands you each index and element together. `contains` answers yes or no, and `firstIndex(of:)` returns an optional index. Reading an index that does not exist stops the program with *Index out of range*, so check before you read.

```swift
let songs = ["Intro", "Sunrise", "Drive", "Night Bus"]
for (index, song) in songs.enumerated() {
    print("\(index + 1). \(song)")
}
print(songs.contains("Drive"))
print(songs.contains("Encore"))
if let position = songs.firstIndex(of: "Drive") {
    print("Drive is at index \(position)")
}
let wanted = 10
if songs.indices.contains(wanted) {
    print(songs[wanted])
} else {
    print("No song at index \(wanted)")
}
```

OUTPUT

```
1. Intro
2. Sunrise
3. Drive
4. Night Bus
true
false
Drive is at index 2
No song at index 10
```

Line by line

1 `let songs = ["Intro", "Sunrise", "Drive", "Night Bus"]`

- WHO — Swift type inference, which reads the string literals and gives songs the type [String].
- WHAT — **Creates a constant array of four song titles.**
- WHERE — A top-level constant, available to the loop and the searches below.
- WHEN — Created once at the start; nothing in this program changes it.
- WHY — It is a let because this example only reads the array, and let tells readers and the compiler that it never changes.

2 `for (index, song) in songs.enumerated() {`

- WHO — The enumerated() method, which pairs every element with its position.
- WHAT — **Starts a loop that runs once per song, handing over the pair (index, song) on each pass.**
- WHERE — index and song are new constants that exist only inside this loop's braces.
- WHEN — Runs four times, in array order, from index 0 to index 3.
- WHY — Splitting the pair into two names is clearer than keeping a separate counter variable by hand.

3 `print("\(index + 1). \(song)")`

- WHO — The loop body, which runs once for every element.
- WHAT — **Prints a numbered line such as 1. Intro, adding 1 so the list starts at 1 for people.**
- WHERE — Inside the loop, where index and song hold the current pass's values.
- WHEN — On each of the four passes, so it prints four lines.
- WHY — Indexes start at 0 for the computer, but people count from 1; convert only when you display the number.

```
5  print(songs.contains("Drive"))
```

WHO The contains method, which compares each element with the value you pass.
WHAT **Prints true, because Drive is one of the four titles.**
WHERE Available on any array whose elements can be compared with ==, such as String.
WHEN Scans from the start and stops as soon as it finds a match.
WHY Asking contains reads better than writing your own loop with a found flag.

```
6  print(songs.contains("Encore"))
```

WHO The same contains method, given a title that is not in the array.
WHAT **Prints false after checking every element without finding a match.**
WHERE The comparison is exact and case-sensitive, so encore in lowercase would not match Encore either.
WHEN Has to look at all four songs before it can answer no.
WHY A miss checks every element, which is why a Set is faster for large lookups.

```
7  if let position = songs.firstIndex(of: "Drive") {
```

WHO The firstIndex(of:) method, which returns an optional Int, and if let, which unwraps it.
WHAT **Finds the index of the first Drive and binds it to position when it is found.**
WHERE position exists only inside the if block that follows.
WHEN Runs once; the search stops at index 2, where Drive sits.
WHY The result is optional because the value might be missing, and Swift makes you handle that case instead of returning a magic number like -1.

```
8  print("Drive is at index \(position)")
```

WHO The if let body, which runs only when a match was found.
WHAT **Prints Drive is at index 2.**
WHERE Inside the braces, where position is a plain Int.
WHEN Only after firstIndex(of:) returned a value.
WHY Once you know the index you can read, replace or remove that element safely.

```
10 let wanted = 10
```

WHO Your code, choosing a position to read.
WHAT **Stores the index 10, far past the last valid index of 3.**
WHERE A top-level constant used by the safety check below.
WHEN Set just before the check, the way a value might arrive from user input.
WHY In real apps an index often comes from outside, such as a tapped row, so you cannot assume it is valid.

```
11 if songs.indices.contains(wanted) {
```

WHO The indices property, a range of the valid positions, here 0 up to but not including 4.
WHAT **Checks whether 10 is a valid position before anything tries to use it.**
WHERE The condition of an if statement in top-level code.
WHEN Evaluated at run time, before any subscript is attempted.
WHY songs[10] would crash with Index out of range; checking indices first turns a crash into an ordinary else branch.

```
12 print(songs[wanted])
```

WHO The array subscript, reached only through the safety check.
WHAT **Would print the song at index wanted, but it is skipped on this run.**
WHERE Inside the if branch, which only runs for valid indexes.
WHEN Never in this run, because 10 is not a valid index.
WHY Inside the check the subscript is guaranteed to be in range, so it can never crash here.

13 `} else {`

WHO The else clause of the if statement, the path Swift takes whenever the condition on line 11 comes out false.
WHAT **Marks where the code for an invalid index begins.**
WHERE Pairs with the if on line 11; exactly one of the two branches runs.
WHEN Entered when indices.contains(wanted) is false, as it is here.
WHY Handling the missing case on purpose is what keeps the program running instead of crashing.

14 `print("No song at index \(wanted)")`

WHO The else branch, your fallback path.
WHAT **Prints No song at index 10.**
WHERE Inside the else braces, the only branch that runs for an invalid index; wanted is still readable here because it is a top-level constant.
WHEN Runs because the index check failed.
WHY A clear message or a default value serves users far better than a crash that closes the app.

Try it

1. Change wanted to 3 and run it. Which branch runs now?
2. Search for "Sunrise" instead of "Drive" with firstIndex(of:). Which index prints?
3. Remove the + 1 on line 3. How does the numbering change?

CODE · RUNS IN THE BROWSER

Mistake: changing a let array

A `let` array is frozen: you cannot add, remove or replace items. Methods that change a value in place are called **mutating** methods, and the compiler refuses to call them on a constant. Run this and read the message.

```
1 let groceries = ["milk", "bread"]
2 groceries.append("eggs")
3 print(groceries)
```

COMPILER ERROR (ON PURPOSE)

```
cannot use mutating member on immutable value
```

Line by line

1 `let groceries = ["milk", "bread"]`

WHO The compiler, which marks groceries as a constant array of String.
WHAT **Creates a two-item grocery list that can never change after this line.**
WHERE A top-level constant; because arrays are values, let locks the whole contents, not just the name.
WHEN The compiler records it as immutable while it checks the file, before anything runs.
WHY let is the right default, but only when the list really is fixed, and this one is not.

2 `groceries.append("eggs")`

WHO The compiler, which rejects this line before the program can start.
WHAT **Tries to add eggs to a constant, so you get: cannot use mutating member on immutable value.**
WHERE append is marked mutating in the Array type, meaning it rewrites the array it is called on.
WHEN Caught at compile time, so no line of the program runs.
WHY This is the protection let promises: code that relies on the list staying the same cannot be surprised by an edit.

3 `print(groceries)`

- **WHO** The print function, which would show the list.
- **WHAT** **Would print the array, but the program never runs because of the error on line 2.**
- **WHERE** Top-level code after the failing line.
- **WHEN** Never reached; a compile error stops the whole program, not just one line.
- **WHY** Swift checks the entire file first, so one error blocks everything, unlike a script that fails halfway through.

> **Try it**
>
> 1. Read the full message. Which constant does it name?
> 2. Predict the one-word change that fixes it, then check the next step.

CODE · RUNS IN THE BROWSER

The fix: var, then add, insert and remove

Declaring the list with `var` makes it mutable, and the full toolkit opens up. `append` adds to the end, `insert(_:at:)` adds at a position, `remove(at:)` takes one item out and returns it, and `popLast()` safely removes the last item. `sorted()` returns a sorted copy and leaves the original alone.

```
var groceries = ["milk", "bread"]
groceries.append("eggs")
groceries.insert("coffee", at: 0)
print(groceries)
let removed = groceries.remove(at: 2)
print("Removed \(removed)")
if let lastItem = groceries.popLast() {
    print("Took off \(lastItem)")
}
groceries.append("apples")
groceries.swapAt(0, 1)
print(groceries)
print(groceries.sorted())
```

OUTPUT

```
["coffee", "milk", "bread", "eggs"]
Removed bread
Took off eggs
["milk", "coffee", "apples"]
["apples", "coffee", "milk"]
```

Line by line

1 `var groceries = ["milk", "bread"]`

- **WHO** The compiler, which now treats groceries as a variable.
- **WHAT** **Creates the same two-item list, this time allowed to change.**
- **WHERE** A top-level variable; every mutating call below edits this one array.
- **WHEN** Created first, before any of the edits.
- **WHY** Changing let to var is the entire fix the compiler suggested, because this list is meant to change.

```
2  groceries.append("eggs")
```

WHO The mutating append method, now permitted.
WHAT **Adds eggs at the end, making the list milk, bread, eggs.**
WHERE Writes into groceries in place; no new array is assigned.
WHEN The second line, so the list has three items from here on.
WHY This is exactly the line that failed before; only the declaration changed.

```
3  groceries.insert("coffee", at: 0)
```

WHO The insert(_:at:) method, which places an element at a chosen index.
WHAT **Puts coffee at index 0 and shifts every other item one place to the right.**
WHERE Valid positions run from 0 through count; inserting at count is the same as appending.
WHEN After the append, so the list becomes coffee, milk, bread, eggs.
WHY Inserting at the front moves every element, so prefer append whenever the order allows it.

```
4  print(groceries)
```

WHO The print function, receiving the whole groceries array and writing it in Swift's bracketed list format with each String in quotes.
WHAT **Prints the four items in their current order.**
WHERE Writes to the console from top-level code.
WHEN After the insert, before anything is removed.
WHY Printing between edits lets you confirm each step did what you expected.

```
5  let removed = groceries.remove(at: 2)
```

WHO The remove(at:) method, which deletes one element and hands it back.
WHAT **Removes bread from index 2 and stores it in the constant removed.**
WHERE Items after index 2 slide one place left to close the gap.
WHEN Runs once, leaving coffee, milk, eggs.
WHY Returning the removed item lets you use it, for example to offer an Undo button.

```
6  print("Removed \(removed)")
```

WHO Your code, reporting what was taken out.
WHAT **Interpolates the String constant removed into the text and writes Removed bread as the second line of output.**
WHERE Top-level code; removed is an ordinary String constant.
WHEN Immediately after remove(at: 2) runs, while groceries holds coffee, milk and eggs and removed still holds bread.
WHY It confirms that remove(at:) returns the element itself, not its index.

```
7  if let lastItem = groceries.popLast() {
```

WHO The popLast() method, which removes the last element and returns it as an optional.
WHAT **Removes eggs and unwraps it into lastItem.**
WHERE lastItem lives only inside the if block.
WHEN Runs once, leaving coffee and milk in the list.
WHY popLast returns nil on an empty array instead of crashing, unlike removeLast(), so it is the safe choice.

```
8  print("Took off \(lastItem)")
```

WHO The body of the if let block, which Swift enters only because popLast() returned a real value rather than nil.
WHAT **Interpolates the unwrapped String lastItem and prints Took off eggs, the third line of output.**
WHERE Inside the braces, where lastItem is a plain String.
WHEN Only because popLast found an item to remove.
WHY Removing and unwrapping in one condition is a common pattern when you work through a queue of tasks.

```
10 groceries.append("apples")
```

WHO The mutating append method again, called on the same var array that has just shrunk to coffee and milk.
WHAT **Adds apples, so the list is coffee, milk, apples.**
WHERE Edits the same variable in place.
WHEN After popLast, at the end of the list.
WHY A var array can grow and shrink as many times as the program needs.

```
11 groceries.swapAt(0, 1)
```

WHO The swapAt method, which exchanges two elements.
WHAT **Swaps the items at indexes 0 and 1, giving milk, coffee, apples.**
WHERE Both indexes must be valid, or the program stops with Index out of range.
WHEN Runs once, after apples is appended and right before the print that shows the new order.
WHY Swapping in place is how reordering works, for example when someone drags a row to a new spot in a list.

```
12 print(groceries)
```

WHO The print function, now showing the array after the append and the swap so you can compare it with the first printout.
WHAT **Prints milk, coffee, apples in their stored order.**
WHERE Top-level code writing to the console; it reads groceries directly, so it shows the array's stored order.
WHEN After swapAt has run, as the fourth line of output and just before the sorted copy is printed.
WHY It shows the array keeps the order your code created, not alphabetical order.

```
13 print(groceries.sorted())
```

WHO The sorted() method, which builds a new, sorted array.
WHAT **Prints apples, coffee, milk in alphabetical order.**
WHERE The sorted copy is a temporary value; groceries itself keeps its order.
WHEN The last line, so nothing reads the sorted copy afterward.
WHY sorted() never changes the original, so it even works on let arrays; sort() is the mutating version.

> **Try it**
>
> 1. Insert "tea" at index 1 instead of coffee at index 0. Predict the first printed line, then run it.
> 2. Change remove(at: 2) to remove(at: 0). What gets removed now?
> 3. Change var back to let on line 1. Which line does the compiler stop at?

CODE · RUNS IN THE BROWSER

Dictionaries: look things up by key

A **dictionary** stores **key-value pairs**: each unique key, such as a drink name, maps to one value, such as how many are in stock. Looking up a key returns an *optional*, because the key may not exist. Dictionaries have no order, so sort the keys before you print them.

```swift
var stock: [String: Int] = ["latte": 12, "mocha": 5]
if let lattes = stock["latte"] {
    print("Lattes left: \(lattes)")
}
let teas = stock["tea"] ?? 0
print("Teas left: \(teas)")
print(stock["mocha", default: 0])
stock["mocha"] = 4
stock["tea"] = 20
stock.removeValue(forKey: "latte")
print(stock.count)
for item in stock.keys.sorted() {
    print("\(item): \(stock[item, default: 0])")
}
```

OUTPUT

```
Lattes left: 12
Teas left: 0
5
2
mocha: 4
tea: 20
```

Line by line

1 `var stock: [String: Int] = ["latte": 12, "mocha": 5]`

- **WHO** The compiler, which fixes the key type as String and the value type as Int.
- **WHAT** **Creates a coffee-shop inventory in which latte maps to 12 and mocha maps to 5.**
- **WHERE** A top-level variable; the pairs are stored by the hash of each key, not in the order written.
- **WHEN** Created once at program start, before any lookups, so every read and edit below works on these two pairs.
- **WHY** It is var because the stock levels change below; read the annotation as a dictionary from String to Int.

2 `if let lattes = stock["latte"] {`

- **WHO** The dictionary subscript, which returns an optional Int, and if let, which unwraps it.
- **WHAT** **Looks up latte, finds 12, and binds it to lattes.**
- **WHERE** lattes is a plain Int that exists only inside the braces.
- **WHEN** Checked at run time; a missing key would skip the block.
- **WHY** Unlike an array index, a missing key never crashes; it gives nil, which you must handle.

3 `print("Lattes left: \(lattes)")`

- **WHO** The body of the if let block, reached only because the subscript lookup for latte returned a value rather than nil.
- **WHAT** **Interpolates the unwrapped Int lattes and writes Lattes left: 12 as the first line of output.**
- **WHERE** Inside the block, with lattes already unwrapped.
- **WHEN** Only when the key exists, which it does.
- **WHY** Interpolating the unwrapped Int avoids printing Optional(12).

5 `let teas = stock["tea"] ?? 0`

- **WHO** The ?? operator, which replaces nil with a default.
- **WHAT** **Looks up tea, finds nothing, and stores 0 in teas.**
- **WHERE** teas is a plain Int constant in top-level code.
- **WHEN** Evaluated once; reading a missing key does not add it to the dictionary.
- **WHY** For stock counts, a missing key really does mean zero, so ?? 0 says that in one line.

```
6  print("Teas left: \(teas)")
```

WHO The print function, receiving a String with the plain Int teas interpolated into it.
WHAT **Writes Teas left: 0, showing the default value that ?? supplied for the missing tea key.**
WHERE Top-level code; teas is a non-optional Int, so the output contains no Optional wrapper around the number.
WHEN After the fallback value was applied.
WHY A missing key plus a sensible default becomes normal output with no special case.

```
7  print(stock["mocha", default: 0])
```

WHO The default: subscript, a Dictionary feature that returns a plain value, never an optional.
WHAT **Prints 5, the stock of mocha; it would print 0 if mocha were missing.**
WHERE Built into Dictionary; the default is used only for this read and is not stored.
WHEN Evaluated at this line against the current contents.
WHY It is the dictionary version of ?? and becomes especially handy when you also update the value, as the next step shows.

```
8  stock["mocha"] = 4
```

WHO The dictionary subscript used as the target of an assignment.
WHAT **Replaces the value for mocha, changing 5 to 4.**
WHERE Edits the existing pair in place; the key stays the same.
WHEN After the reads above, so they saw the old value.
WHY Assigning to a key updates it when it exists, so one syntax covers both updating and inserting.

```
9  stock["tea"] = 20
```

WHO The same subscript assignment, with a key that is not there yet.
WHAT **Adds a new pair in which tea maps to 20.**
WHERE The dictionary grows by one entry.
WHEN Runs once; from here on, looking up tea finds 20.
WHY No separate add method is needed, because assigning to a new key inserts it.

```
10 stock.removeValue(forKey: "latte")
```

WHO The mutating removeValue(forKey:) method of Dictionary, which finds a pair by its key and deletes it.
WHAT **Deletes the latte pair entirely, leaving mocha and tea.**
WHERE Edits stock in place, which is why stock had to be a var.
WHEN After the updates, just before counting.
WHY It also returns the removed value as an optional, which Swift lets you ignore when you only want the key gone.

```
11 print(stock.count)
```

WHO The Dictionary type, which tracks how many pairs it holds.
WHAT **Prints 2, one pair each for mocha and tea.**
WHERE count counts key-value pairs, not the values added together.
WHEN After one key was added and one removed.
WHY A quick count confirms that both the insert and the removal happened.

```
12 for item in stock.keys.sorted() {
```

WHO The keys property, which lists every key, and sorted(), which puts them in order.
WHAT **Loops over mocha and then tea, in alphabetical order.**
WHERE item is a String constant local to this loop.
WHEN Runs twice, once per remaining key.
WHY A dictionary has no reliable order, so sorting the keys makes the output the same on every run.

```
13 print("\(item): \(stock[item, default: 0])")
```

WHO The loop body, run once per key.
WHAT **Prints each drink and its count, such as mocha: 4.**
WHERE Inside the loop; item is the current key.
WHEN On each of the two passes.
WHY The default: subscript returns a plain Int, so the output never shows the word Optional.

> **Try it**
>
> 1. Add stock["chai"] = 7 just before the loop. Where does chai appear in the output?
> 2. Look up the misspelled key "Latte" with ?? 0. What prints, and why?

CODE · RUNS IN THE BROWSER

Counting with a dictionary

A classic dictionary job is counting: how often does each word appear? The trick is `counts[word, default: 0] += 1`. It reads the current count (0 for a word seen for the first time), adds one, and writes the result back, all in one line.

```
let sentence = "the cat saw the dog and the cat ran"
let words = sentence.split(separator: " ")
var counts: [String: Int] = [:]
for word in words {
    counts[String(word), default: 0] += 1
}
print("\(counts.count) different words")
for word in counts.keys.sorted() {
    print("\(word): \(counts[word] ?? 0)")
}
```

OUTPUT

```
6 different words
and: 1
cat: 2
dog: 1
ran: 1
saw: 1
the: 3
```

Line by line

```
1 let sentence = "the cat saw the dog and the cat ran"
```

WHO Your program, holding the input text.
WHAT **Stores a sentence of nine words, several of them repeated.**
WHERE A top-level String constant; only the split call on the next line reads it, so let is enough.
WHEN Set once as the program starts, before anything splits it into words.
WHY Repeated words such as the and cat make the counts easy to check by hand.

```
2  let words = sentence.split(separator: " ")
```

WHO The split(separator:) method, which String has because it is a collection of Characters; it needs no Foundation import.

WHAT **Cuts the sentence at every space, producing an array of nine word pieces.**

WHERE The pieces are Substrings: lightweight views into the original sentence rather than new strings.

WHEN Runs once, before the counting loop, so the loop has a finished array of nine pieces to walk through.

WHY Splitting first turns a text problem into a collection problem you can loop over.

```
3  var counts: [String: Int] = [:]
```

WHO The compiler, which needs the annotation because an empty literal has no values to infer types from.

WHAT **Creates an empty dictionary that will map each word to its count.**

WHERE A top-level variable, declared outside the loop so it still exists after the loop ends.

WHEN Before the loop, so every pass adds to the same dictionary.

WHY The literal [:] means an empty dictionary; without [String: Int] the compiler could not know the key and value types.

```
4  for word in words {
```

WHO A for-in loop over the array of word pieces.

WHAT **Visits each of the nine words in order, binding the current one to word.**

WHERE word is local to the loop body.

WHEN Runs nine times, once per word piece, in the order the words appear in the sentence.

WHY One pass over the data is enough to count everything.

```
5  counts[String(word), default: 0] += 1
```

WHO The default: subscript working together with the += operator.

WHAT **Reads the word's current count, using 0 for a new word, adds 1, and stores the result.**

WHERE Writes into counts; String(word) converts the Substring into a String so it matches the key type.

WHEN On every pass, so the third the raises its count to 3.

WHY Without default: you would need an if let to handle the first time each word appears.

```
7  print("\(counts.count) different words")
```

WHO The dictionary's count property, interpolated into the text and then written out by print.

WHAT **Prints 6 different words, because repeated words share one key.**

WHERE Top-level code after the loop's closing brace, where counts still exists because it was declared outside the loop.

WHEN After all nine words were counted.

WHY Keys are unique, so count tells you how many distinct words the sentence has.

```
8  for word in counts.keys.sorted() {
```

WHO The keys property together with sorted().

WHAT **Loops over the six distinct words in alphabetical order.**

WHERE This word is a new loop constant, separate from the word in the first loop.

WHEN Runs six times, once per distinct word.

WHY Sorting gives predictable output; the dictionary's own order can change from one run to the next.

```
9  print("\(word): \(counts[word] ?? 0)")
```

WHO The loop body, reading each count back out.

WHAT **Prints lines such as cat: 2 and the: 3.**

WHERE Inside the second loop; the plain subscript returns an optional, so ?? 0 unwraps it.

WHEN Once on each of the six passes of the second loop, producing the count lines in alphabetical order.

WHY The key came from the dictionary itself, so the lookup always succeeds, but ?? keeps the code free of force unwraps.

Try it

1. Add " the dog" to the end of the sentence. Predict the new counts for the and dog.
2. Call sentence.uppercased() before splitting. What happens to the keys and their order?

CODE · RUNS IN THE BROWSER

Sets: unique members

A **set** holds each value at most once and has no order. Inserting a duplicate does nothing. Its strength is speed: `contains` on a set answers almost instantly even with a million items, and two sets combine like the circles of a Venn diagram. Print a set through `sorted()` so the order is predictable.

```
var tags: Set<String> = ["swift", "ios", "xcode"]
tags.insert("swift")
tags.insert("design")
print(tags.count)
print(tags.contains("ios"))
let otherTags: Set<String> = ["design", "android", "swift"]
print(tags.union(otherTags).sorted())
print(tags.intersection(otherTags).sorted())
print(tags.subtracting(otherTags).sorted())
```

OUTPUT

```
4
true
["android", "design", "ios", "swift", "xcode"]
["design", "swift"]
["ios", "xcode"]
```

Line by line

1 `var tags: Set<String> = ["swift", "ios", "xcode"]`

- WHO — The compiler, which needs the Set annotation because a bracket literal on its own would become an Array.
- WHAT — **Creates a set of three photo tags.**
- WHERE — A top-level variable; set members are stored by hash, so they have no position or index.
- WHEN — Created at program start, before any insert, so the set begins with exactly three members.
- WHY — Tags are a natural set: a tag either applies or it does not, and a duplicate tag means nothing.

2 `tags.insert("swift")`

- WHO — The mutating insert method of Set, which first checks by hash whether the value is already a member.
- WHAT — **Tries to add swift, finds it already there, and leaves the set unchanged.**
- WHERE — Edits tags in place, which is why tags is a var.
- WHEN — Runs once; the set still has three members afterward.
- WHY — Duplicates are ignored automatically, so you never write a check before inserting.

3 `tags.insert("design")`

- WHO — The same insert method, given a new value.
- WHAT — **Adds design, growing the set to four members.**
- WHERE — The new member has no position; it is simply in the set.
- WHEN — Right after the ignored duplicate insert, so this is the call that actually changes the set.
- WHY — insert also reports whether anything was added, but Swift lets you ignore that result when you do not need it.

```
4  print(tags.count)
```

WHO The count property of the Set type, read once and handed to print.
WHAT **Prints 4, not 5, because the second swift was ignored.**
WHERE Top-level code; count reports unique members only, so the ignored duplicate never adds to it.
WHEN After both inserts have run, once the set has settled at four members.
WHY It proves the uniqueness rule with a number you can check.

```
5  print(tags.contains("ios"))
```

WHO The contains method, which uses the hash of ios to jump straight to where it would be stored.
WHAT **Prints true, because ios was one of the three original tags and nothing has removed it.**
WHERE A set lookup does not scan the members one by one the way an array does.
WHEN Evaluated once, at this line, against the four members the set holds after both inserts.
WHY Fast membership checks are the main reason to choose a Set over an Array.

```
6  let otherTags: Set<String> = ["design", "android", "swift"]
```

WHO Your code, creating a second set to compare against.
WHAT **Stores the three tags from another photo.**
WHERE A top-level constant; it is only read, so let is enough.
WHEN Created before the three set operations.
WHY Set operations need two sets, like the two circles of a Venn diagram.

```
7  print(tags.union(otherTags).sorted())
```

WHO The union method, which builds a new set from the members of both.
WHAT **Prints all five distinct tags found in either set, in alphabetical order.**
WHERE The result is a new temporary set; neither tags nor otherTags changes.
WHEN Evaluated as one expression: union first, then sorted, then print.
WHY sorted() turns the unordered set into an ordered array, so the output is the same on every run.

```
8  print(tags.intersection(otherTags).sorted())
```

WHO The intersection method of Set, which keeps only the members that appear in both sets.
WHAT **Prints design and swift, the tags found in both sets.**
WHERE Another new temporary set, built and thrown away within this line.
WHEN Right after the union line, comparing the same two sets, which the union left unchanged.
WHY Intersection answers questions such as which interests two people share.

```
9  print(tags.subtracting(otherTags).sorted())
```

WHO The subtracting method of Set, which starts from tags and drops every member also found in otherTags.
WHAT **Prints ios and xcode, the tags in tags that are not in otherTags.**
WHERE A new set; here the order of the two sets matters, unlike union and intersection.
WHEN The last line of the program, after union and intersection have left both sets untouched.
WHY Subtracting answers what is missing or new, such as which tags still need to be added.

> **Try it**
>
> 1. Insert "ios" again before the count. Predict the count, then run it.
> 2. Swap the order to otherTags.subtracting(tags). What prints now?

CODE · RUNS IN THE BROWSER

Collections are values

Arrays, dictionaries and sets are **value types**: assigning one to another variable gives an independent copy, and changing the copy never touches the original. Swift keeps this cheap with **copy-on-write**:

both names share the same storage until one of them changes.

```
let weekday = ["warm-up", "run"]
var weekend = weekday
weekend.append("stretch")
print(weekday)
print(weekend)
let dailyGoal = ["steps": 8000, "minutes": 30]
var restDay = dailyGoal
restDay["steps"] = 4000
print(dailyGoal["steps"] ?? 0)
print(restDay["steps"] ?? 0)
```

OUTPUT

```
["warm-up", "run"]
["warm-up", "run", "stretch"]
8000
4000
```

Line by line

1 `let weekday = ["warm-up", "run"]`

- WHO — Your fitness plan, stored as a constant array.
- WHAT — **Creates the weekday workout with two exercises.**
- WHERE — A top-level constant that owns its data.
- WHEN — Created first, at program start, so its two exercises exist before weekend is copied from it.
- WHY — It is let because the weekday plan itself never changes in this program.

2 `var weekend = weekday`

- WHO — Swift assignment, which copies value types.
- WHAT — **Makes weekend a separate array that starts with the same two exercises.**
- WHERE — Both names share one block of storage for now, thanks to copy-on-write.
- WHEN — The real copy is postponed until one of them is changed.
- WHY — A var copy of a let array is allowed, because the copy is a new value with its own rules.

3 `weekend.append("stretch")`

- WHO — The append method, called on the copy.
- WHAT — **Adds stretch to the end of weekend only, giving it three exercises while weekday keeps two.**
- WHERE — This is the moment Swift actually copies the storage, so weekend gets its own buffer.
- WHEN — At run time, the first time either array is changed.
- WHY — Copy-on-write means a copy costs almost nothing until a change makes it necessary.

4 `print(weekday)`

- WHO — The print function, reading the original weekday array after its copy was changed.
- WHAT — **Prints only warm-up and run: the original is unchanged.**
- WHERE — Reads the weekday array, which was never edited.
- WHEN — After the append on the copy.
- WHY — This is value semantics: no other variable can change your array behind your back.

```
5  print(weekend)
```

WHO The print function again, this time reading the weekend copy that received the append.
WHAT **Prints warm-up, run and stretch in brackets: three exercises, in the order they were added.**
WHERE Reads the weekend copy, which now has its own storage.
WHEN Right after printing the original, for comparison.
WHY Seeing the two lines one after the other makes their independence obvious.

```
6  let dailyGoal = ["steps": 8000, "minutes": 30]
```

WHO Type inference, which gives dailyGoal the type [String: Int].
WHAT **Creates a dictionary of daily fitness targets.**
WHERE A top-level constant dictionary; as a let it can never be changed directly, which keeps the test below honest.
WHEN After the array demonstration, to repeat the same experiment with a second collection type.
WHY Dictionaries follow the same value rules as arrays, and the next lines test that claim.

```
7  var restDay = dailyGoal
```

WHO Assignment again, this time copying a dictionary.
WHAT **Makes restDay an independent dictionary with the same two targets.**
WHERE Shares storage with dailyGoal until one of them changes.
WHEN Runs once, after dailyGoal exists; at this moment both names still share the same storage.
WHY Starting from a copy is an easy way to build a variation of existing data.

```
8  restDay["steps"] = 4000
```

WHO The subscript assignment, applied to the copy.
WHAT **Lowers the step target to 4000 in restDay only.**
WHERE Triggers the real copy, so dailyGoal keeps its own storage.
WHEN The first change to either dictionary.
WHY Editing the copy is safe precisely because it is a value.

```
9  print(dailyGoal["steps"] ?? 0)
```

WHO A dictionary lookup with a ?? fallback.
WHAT **Prints 8000: the original target is untouched.**
WHERE Reads the original dailyGoal dictionary; ?? 0 unwraps the optional that every key lookup returns.
WHEN After the copy restDay was changed, exactly when a shared object would have shown the change.
WHY Classes, which you will meet later, behave differently: two names can share one object, so a change shows through both.

```
10 print(restDay["steps"] ?? 0)
```

WHO The same lookup, on the copy.
WHAT **Prints 4000, the lowered target stored only in the copy, as the last line of output.**
WHERE Reads the restDay copy, which got its own storage the moment its steps value was changed.
WHEN The last line of the program, directly after the original was printed, so the two numbers sit side by side.
WHY It confirms the change landed only where it was made.

Try it

1. Change line 3 to append to weekday instead. What does the compiler say, and why?
2. Add restDay["minutes"] = 15, then print the minutes value from both dictionaries.

CHECK YOURSELF

Check your understanding

1. What does this program print?

```
var a = [1, 2, 3]
var b = a
b.append(4)
print(a.count)
```

   A. 4
   B. 3
   C. Nothing: it does not compile
   D. 7

2. A social app must remember which user IDs have liked a post and check very quickly whether the current user is one of them. A user can like a post only once. Which collection fits best?

   A. An Array of IDs
   B. A Dictionary from ID to Bool
   C. A Set of IDs
   D. One String with the IDs separated by commas

3. What happens when this runs?

```
let scores = [12, 40, 7]
print(scores[3])
```

   A. It prints 7
   B. It prints nil
   C. It does not compile, because scores is a let
   D. It stops at run time: Index out of range

4. After these lines run, what is `count`?

```
let stock = ["latte": 12]
let count = stock["mocha"]
```

   A. nil
   B. 0
   C. The program stops with a key-not-found error
   D. An empty string

Answers: Arrays, Dictionaries and Sets

1. **B. 3 3.** Arrays are values, so `var b = a` makes an independent copy and the append changes only `b`. Answering 4 is a habit from languages such as JavaScript or Java, where two variables can point at the same array object.
2. **C. A Set of IDs** A **Set** stores each ID once and answers `contains` almost instantly. An array works, but its `contains` checks every element and it happily stores the same ID twice. A dictionary to Bool works too, but the Bool value carries no real information.
3. **D. It stops at run time: Index out of range** The valid indexes are 0, 1 and 2, so index 3 is one past the end. An array subscript does not return an optional, so you never get `nil`; the program stops with *Index out of range*. Reading a `let` array is perfectly legal; only changing it is not.
4. **A. nil** A dictionary lookup returns `Int?`, and a missing key gives `nil`. You only get 0 if you ask for it with `?? 0` or `stock["mocha", default: 0]`. Unlike an array index, a missing key never crashes.

Decisions and Loops

CODE · RUNS IN THE BROWSER

if, else if and else

An `if` statement runs a block of code only when its condition is `true`. Chain more tests with `else if` and finish with `else` to catch everything left over. Swift checks the conditions from top to bottom and runs **only the first branch** whose condition is true. Every condition must be a `Bool`.

```
let batteryLevel = 18
let isCharging = false

if batteryLevel <= 5 {
    print("Shutting down")
} else if batteryLevel <= 20 && !isCharging {
    print("Low battery: \(batteryLevel)%")
} else if batteryLevel <= 50 {
    print("Battery below half")
} else {
    print("Battery fine")
}
print("Check finished")
```

OUTPUT

```
Low battery: 18%
Check finished
```

Line by line

1 `let batteryLevel = 18`

- WHO — Your program declares a constant, and the compiler infers its type, Int, from the literal 18.
- WHAT — **Stores the battery percentage 18 under the name batteryLevel; it cannot change for the rest of the program.**
- WHERE — Top-level code, so batteryLevel is a global constant that every line below can read.
- WHEN — Runs first, before any decision is made, because every condition below reads it.
- WHY — An input that never changes belongs in a let, and a descriptive name makes each condition read like a sentence.

2 `let isCharging = false`

- WHO — The compiler, which infers the type Bool from the literal false.
- WHAT — **Records that the phone is not plugged in right now.**
- WHERE — A second global constant, next to batteryLevel at the top of the program.
- WHEN — Set once at the start and read later by the second condition.
- WHY — Bool names that start with is read naturally inside a condition: low battery and not isCharging.

```
4  if batteryLevel <= 5 {
```

WHO The Swift runtime evaluates the condition; the compiler has already confirmed that it produces a Bool.
WHAT **Compares 18 with 5. The result is false, so the block below is skipped and Swift moves on to the next else if.**
WHERE Starts an if statement at top level; its braces surround the code that runs only when the condition is true.
WHEN Tested first, because an if chain checks its conditions strictly from top to bottom.
WHY The most urgent case goes first. Swift has no truthy numbers: writing if batteryLevel on its own is a compile error, because a condition must be a Bool.

```
5      print("Shutting down")
```

WHO The print function from the Swift standard library.
WHAT **Would print Shutting down, but this battery level never reaches it.**
WHERE Inside the first branch, indented four spaces to show that it belongs to that branch.
WHEN Only when batteryLevel is 5 or less; with 18 it never runs.
WHY Each branch holds the response to exactly one situation, which keeps a decision easy to scan.

```
6  } else if batteryLevel <= 20 && !isCharging {
```

WHO The runtime, evaluating two Bool expressions joined by the and operator, &&.
WHAT **Checks whether 18 is at most 20, which is true, and !isCharging, which flips false to true. Both sides are true, so this branch runs.**
WHERE The second link of the same chain; else if means consider this only if every condition above was false.
WHEN Evaluated only because the first condition was false. If the left side of && were false, Swift would skip the right side entirely.
WHY Combining both rules in one condition avoids nesting an if inside another if, which is harder to read.

```
7      print("Low battery: \(batteryLevel)%")
```

WHO print, with string interpolation supplying the number.
WHAT **Prints Low battery: 18% to the console.**
WHERE Inside the second branch, the one whose condition matched.
WHEN Right after its condition evaluates to true.
WHY Interpolation inserts the real value, so the message always matches the data instead of a hard-coded number.

```
8  } else if batteryLevel <= 50 {
```

WHO The runtime would evaluate this comparison, but it never gets the chance in this run.
WHAT **18 is also at most 50, yet this branch is skipped because an earlier branch has already run.**
WHERE The third link of the chain, after the more specific low-battery test.
WHEN Never evaluated here; once one branch runs, Swift jumps past the rest of the chain.
WHY This is why order matters: put narrow, specific conditions before broad ones, or the broad one will catch everything first.

```
9      print("Battery below half")
```

WHO print from the standard library, sitting in the third branch of the if chain and waiting for its condition to win.
WHAT **Would print Battery below half for a level from 21 to 50, or for a low level while charging.**
WHERE The body of the third branch.
WHEN Not in this run, because the second branch already handled the value 18.
WHY A middle tier gives the user useful information without the urgency of the low-battery warning.

```
10 } else {
```

WHO The compiler attaches this else to the if chain above it.
WHAT **Catches every case that none of the three conditions matched; else has no condition of its own.**
WHERE The final link of the chain.
WHEN Runs only when all three conditions are false, for example at a level of 80.
WHY An else guarantees that the chain always does something, so no battery level slips through silently.

```
11  print("Battery fine")
```

WHO — print, placed in the else branch, the fallback that runs only when no condition in the chain matched.
WHAT — **Would print Battery fine for any level above 50.**
WHERE — The body of the else branch.
WHEN — Only when every condition above was false; skipped in this run.
WHY — Even the normal case gets a message, which makes it obvious during testing which branch ran.

```
13  print("Check finished")
```

WHO — print again, this time back in top-level code, outside every branch of the if chain.
WHAT — **Writes Check finished to the console as the second and final line of output, after Low battery: 18%.**
WHERE — After the whole if chain, outside every branch, so it is not part of any decision.
WHEN — Always runs, whichever branch was taken, because the program simply continues after the chain ends.
WHY — It shows that an if statement controls only the code inside its braces; execution then carries on normally.

> **Try it**
>
> 1. Change batteryLevel to 3, then 35, then 90. Predict each message before you run.
> 2. Set isCharging to true with the level still at 18. Which branch runs now, and why?
> 3. Change the first condition to batteryLevel <= 50. Why can Low battery never appear now?

CONCEPT

switch: every case covered

A `switch` statement takes one value and compares it with a list of **patterns** (shapes a value can match), from top to bottom. The first case that matches runs, and then the whole switch ends. Swift's switch looks like the one in C, Java or JavaScript, but it is stricter and far more expressive.

Exhaustive — Every possible value must be handled. For an `Int` or a `String` that means ending with `default`; the compiler rejects a switch that could miss a value.

No fallthrough — A case never runs on into the next one, so you never write `break` at the end of a case. Forgetting `break` is a classic C and Java bug that Swift removes.

Rich patterns — A case can match a list (`200, 201`), a range (`400..<500`), or a tuple (`(0, _)`), and it can copy parts into constants with `let`.

where — Adds an extra condition to a pattern: `case let (x, y) where x == y` matches only points on the diagonal.

> **Why strictness helps**
>
> Soon you will switch over enums, types with a fixed list of cases. If you handle every case without `default`, adding a new case later makes the compiler point at every switch you still need to update. The bug is found while you build, not by a user.

CODE · RUNS IN THE BROWSER

switch with values and ranges

When an app talks to a web server, every reply carries an **HTTP status code**, a number that says how the request went. A `switch` sorts that number into a response. Each `case` lists the values or ranges it matches, and Swift runs the first case that fits.

```swift
let statusCode = 404

switch statusCode {
case 200, 201, 204:
    print("Success")
case 301, 302:
    print("Redirect: follow the new address")
case 404:
    print("Not found")
case 400..<500:
    print("Client error: check the request")
case 500...599:
    print("Server error: try again later")
default:
    print("Unexpected status \(statusCode)")
}
```

OUTPUT

```
Not found
```

Line by line

1 `let statusCode = 404`

- **WHO** Your program; in a real app this number would come from URLSession after the server replies.
- **WHAT** **Stores 404, the code a server sends when the requested address does not exist.**
- **WHERE** A top-level constant that the switch below reads.
- **WHEN** Set once at the very start of the run, before the switch on the next lines reads it.
- **WHY** Keeping the input in one constant lets you change a single number and watch which case matches.

3 `switch statusCode {`

- **WHO** The compiler checks every case against the type of statusCode, Int, and confirms that together they cover every possible Int.
- **WHAT** **Starts comparing the value 404 with each case, in order from the top.**
- **WHERE** Opens the switch body; all of the cases live inside these braces.
- **WHEN** At run time, right after statusCode is set. The exhaustiveness check already happened at compile time.
- **WHY** One switch replaces a long chain of statusCode == comparisons and lays every outcome out side by side.

4 `case 200, 201, 204:`

- **WHO** The runtime compares 404 with each value in the comma-separated list.
- **WHAT** **Matches if statusCode equals 200 (OK), 201 (Created) or 204 (No Content). 404 equals none of them.**
- **WHERE** The first case of the switch; the commas mean any one of these values.
- **WHEN** Tested first, before any other case.
- **WHY** Listing related values in one case avoids writing the same response three times.

5 `print("Success")`

- **WHO** print, in the body of the first case.
- **WHAT** **Would print Success for any of the three success codes.**
- **WHERE** A case body runs from its label down to the next case label; no braces are needed.
- **WHEN** Skipped in this run, because 404 did not match.
- **WHY** Every case must contain at least one statement; Swift rejects an empty case so nothing can fall through by accident.

```
6   case 301, 302:
```

WHO The runtime, still looking for a match.
WHAT **Matches the two common redirect codes, Moved Permanently and Found. 404 is not one of them.**
WHERE The second case, below the success codes.
WHEN Tested only because the first case failed.
WHY Redirects need different handling from errors, so they get their own case.

```
7       print("Redirect: follow the new address")
```

WHO print, belonging to the redirect case, so it speaks only for the codes 301 and 302.
WHAT **Would print a message telling the app to request the new address.**
WHERE The body of the second case.
WHEN Skipped in this run, because 404 is neither 301 nor 302, so Swift never enters this case body.
WHY A clear message per category makes it easy to see which case handled a given code while testing.

```
8   case 404:
```

WHO The runtime compares 404 with the single value 404.
WHAT **Matches exactly. This is the first case that fits, so its body runs.**
WHERE Deliberately placed before the range of 400s below, which also contains 404.
WHEN Tested third, after two cases that failed.
WHY A specific value placed ahead of a broader range gets its own, more helpful message.

```
9       print("Not found")
```

WHO print, the only statement in the case 404 body, which is the case that actually matched.
WHAT **Prints Not found. The switch then ends; Swift does not continue into the next case.**
WHERE The body of case 404, which runs from its case label down to the next case label, the range of 400s.
WHEN Right after case 404 matches as the third case tested; it is the only print that runs in this program.
WHY No break is needed: Swift cases never fall through, which removes a whole family of bugs.

```
10  case 400..<500:
```

WHO The runtime would check whether the value lies in the half-open range 400 to 499.
WHAT **404 is in that range, but this case is never tested in this run because case 404 already matched.**
WHERE After the specific 404 case, on purpose.
WHEN Skipped here; it would catch codes such as 400, 401, 403 and 429.
WHY Order cases from most specific to most general, exactly as you would order an if chain.

```
11      print("Client error: check the request")
```

WHO print, belonging to the client-error case that covers the whole range from 400 up to 499.
WHAT **Would print a message blaming the request, for any 4xx code other than 404.**
WHERE The body of the range case.
WHEN Not in this run: the earlier case 404 already matched, and a switch never runs a second case.
WHY Codes in the 400s mean the app sent something wrong, so the message points at the request.

```
12  case 500...599:
```

WHO The runtime would test membership in the closed range 500 through 599.
WHAT **Matches any server error code; the closed range operator includes 599 itself.**
WHERE The fifth case, after the client errors.
WHEN Never tested in this run, because Swift stopped looking for a match as soon as case 404 succeeded.
WHY Both range operators, the half-open one in the case above and the closed one here, work as patterns, so you can describe a whole family of values in one case.

```
13  print("Server error: try again later")
```

WHO — print, belonging to the server-error case for the codes 500 through 599.
WHAT — **Would print advice to retry, because the problem is on the server side.**
WHERE — The body of the server-error case.
WHEN — Not in this run, because 404 is far below 500 and an earlier case had already matched anyway.
WHY — A 5xx code means the request was fine but the server failed, so retrying later can succeed.

```
14  default:
```

WHO — The compiler requires this case, because an Int has billions of possible values that the cases above do not cover.
WHAT — **Catches every value that no earlier case matched, such as 99 or 700.**
WHERE — Must be the last case in the switch.
WHEN — Runs only when every case above failed to match.
WHY — Without default the program does not compile, with the message switch must be exhaustive; no value can slip through unhandled.

```
15  print("Unexpected status \(statusCode)")
```

WHO — print, with interpolation, in the default case.
WHAT — **Would print Unexpected status followed by the actual number.**
WHERE — The body of default, the last case, so it is the fallback for any code that no earlier case describes.
WHEN — Only for codes no other case handles.
WHY — Including the real value in the message makes unexpected situations easy to diagnose.

> **Try it**
>
> 1. Change statusCode to 201, then 418, then 503. Predict the message each time.
> 2. Move the two case 404 lines below the client-error range case. Which message prints now, and why?
> 3. Add 202 to the first case list and run it with statusCode set to 202.

CODE · RUNS IN THE BROWSER

switch with tuples and where

A **tuple** groups a few values into one, like the point `(x: 3, y: 3)`. A switch can match every part at once. An underscore `_` means any value here, `case let (x, y)` copies the parts into new constants, and `where` adds an extra condition. Picture a tap on a game board grid.

```swift
let tap = (x: 3, y: 3)

switch tap {
case (0, 0):
    print("Tapped the center")
case (_, 0):
    print("On the horizontal axis")
case (0, _):
    print("On the vertical axis")
case let (x, y) where x == y:
    print("On the diagonal at \(x), \(y)")
case let (x, y) where x == -y:
    print("On the other diagonal at \(x), \(y)")
default:
    print("Somewhere else: \(tap.x), \(tap.y)")
}
```

OUTPUT

```
On the diagonal at 3, 3
```

Line by line

1 `let tap = (x: 3, y: 3)`

- WHO — Your program builds a labeled tuple; the compiler infers its type as (x: Int, y: Int).
- WHAT — **Stores a grid position with two named parts, x and y, both equal to 3.**
- WHERE — A top-level constant; the labels let later code read tap.x and tap.y.
- WHEN — Created before the switch examines it.
- WHY — A tuple is the lightest way to treat two related values as one, which is exactly what a switch over both needs.

3 `switch tap {`

- WHO — The compiler checks that each pattern below has the same tuple shape, two Ints.
- WHAT — **Begins matching the whole point (3, 3) against each case in order.**
- WHERE — Opens the switch over the tuple.
- WHEN — At run time, after tap exists.
- WHY — Matching both coordinates together avoids nested if statements that test x and then y.

4 `case (0, 0):`

- WHO — The runtime compares both parts of the tuple with 0.
- WHAT — **Matches only when x and y are both 0; (3, 3) does not match.**
- WHERE — The first and most specific case.
- WHEN — Tested first, as soon as the switch starts, because Swift always tries cases from the top down.
- WHY — The single exact point must come before the axis patterns, which would also match (0, 0).

5 `print("Tapped the center")`

- WHO — print, belonging to the center case, so it speaks only for the single point (0, 0).
- WHAT — **Would write Tapped the center to the console; with the tap at (3, 3) it prints nothing.**
- WHERE — The body of case (0, 0).
- WHEN — Skipped, because the tap is not at the center.
- WHY — The center of the board deserves its own response in the game.

```
6  case (_, 0):
```

WHO The runtime ignores the first part and checks only the second.
WHAT **Matches any point whose y is 0. Here y is 3, so it fails.**
WHERE The second case; the underscore is a wildcard pattern.
WHEN Tested after the center case failed.
WHY The underscore says this part does not matter and avoids creating a name you would never use.

```
7  print("On the horizontal axis")
```

WHO print, belonging to the case (_, 0), which covers every point lying on the horizontal axis.
WHAT **Would print On the horizontal axis.**
WHERE The body of case (_, 0).
WHEN Skipped in this run, because the tap has a y of 3, not 0, so this case pattern failed.
WHY Each recognisable region of the grid gets a clear label.

```
8  case (0, _):
```

WHO The runtime checks only the first part.
WHAT **Matches any point whose x is 0. Here x is 3, so it fails.**
WHERE The third case, the mirror image of the one above.
WHEN Tested after the two earlier cases failed.
WHY Two small wildcard patterns describe both axes more clearly than one complicated condition.

```
9  print("On the vertical axis")
```

WHO print, belonging to the case (0, _), which covers every point lying on the vertical axis.
WHAT **Would print On the vertical axis.**
WHERE The body of case (0, _).
WHEN Skipped in this run, because the tap has an x of 3, not 0, so this pattern failed too.
WHY Keeps each pattern paired with its own response.

```
10 case let (x, y) where x == y:
```

WHO The runtime binds the two parts to new constants and then evaluates the where condition.
WHAT **Copies 3 into x and 3 into y; x == y is true, so this case matches.**
WHERE The new constants x and y exist only inside this case body.
WHEN Tested fourth; the binding always succeeds, and the where clause decides the match.
WHY Binding gives the parts short names for both the condition and the message, and where keeps the extra test inside the pattern.

```
11 print("On the diagonal at \(x), \(y)")
```

WHO print, using the constants bound by the case.
WHAT **Prints On the diagonal at 3, 3, and then the switch ends.**
WHERE The body of the matching case, the only place x and y are visible.
WHEN Runs right after the where condition evaluates to true.
WHY Using the bound names instead of tap.x and tap.y keeps the line short and readable.

```
12 case let (x, y) where x == -y:
```

WHO The runtime would bind x and y again and test a different condition.
WHAT **Would match points such as (2, -2) on the other diagonal; never tested here because the previous case matched.**
WHERE The fifth case; each case gets its own fresh x and y.
WHEN Never tested in this run; the diagonal case above already matched, and the switch ended right after its body.
WHY Several cases can bind the same names, because each binding is local to its case.

13 `print("On the other diagonal at \(x), \(y)")`

WHO print, in the second diagonal case.
WHAT **Would print the position of a point on the other diagonal.**
WHERE The body of the fifth case.
WHEN Not in this run; it would run for a tap such as (x: 5, y: -5), as the try challenge suggests.
WHY Shows that where clauses let one tuple pattern express many different geometric rules.

14 `default:`

WHO The compiler requires it, because the cases above do not cover every pair of Ints.
WHAT **Catches every point that is not at the center, on an axis or on a diagonal.**
WHERE The last case in the switch, after every specific pattern; Swift allows no case to follow default.
WHEN Only when every pattern above failed, for a tap such as (4, 1); in this run it is never reached.
WHY Guarantees the switch is exhaustive so it compiles and every tap gets a response.

15 `print("Somewhere else: \(tap.x), \(tap.y)")`

WHO print, reading the tuple parts by their labels.
WHAT **Would print the coordinates of any other point, such as Somewhere else: 4, 1.**
WHERE The body of default, where no names were bound, so it uses tap.x and tap.y.
WHEN Not in this run, because the diagonal case matched (3, 3) before the switch reached default.
WHY Labeled tuple parts read clearly even when no pattern has given them short names.

> **Try it**
>
> 1. Change tap to (x: 0, y: 7), then (x: 5, y: -5). Predict the output first.
> 2. Change tap to (x: 0, y: 0). Why does the center case win over both axis cases?

CODE · RUNS IN THE BROWSER

for-in with ranges and stride

A `for-in` loop runs its body once for every item in a sequence. A **range** is a sequence of whole numbers: `1...3` includes 3, while `0..<2` stops before 2. `stride` counts in steps of any size, even backward, and `where` filters which items reach the body.

```
for lap in 1...3 {
    print("Lap \(lap)")
}

for _ in 0..<2 {
    print("Beep")
}

for minute in stride(from: 0, to: 60, by: 20) {
    print("Stretch at minute \(minute)")
}

for count in stride(from: 3, through: 1, by: -1) {
    print("\(count)...")
}

for n in 1...10 where n.isMultiple(of: 4) {
    print("\(n) is a multiple of 4")
}
```

OUTPUT

```
Lap 1
Lap 2
Lap 3
Beep
Beep
Stretch at minute 0
Stretch at minute 20
Stretch at minute 40
3...
2...
1...
4 is a multiple of 4
8 is a multiple of 4
```

Line by line

1 `for lap in 1...3 {`

WHO The for-in loop, which asks the range 1...3 for its next value on each pass.
WHAT **Runs the body three times, with lap set to 1, then 2, then 3.**
WHERE lap is a new constant that exists only inside this loop's braces.
WHEN The body runs once per value, and the loop ends after the last value, 3.
WHY The closed range operator includes both ends, which matches how people count laps.

2 `print("Lap \(lap)")`

WHO print, inside the body of the first loop, reading the loop constant lap on each pass.
WHAT **Prints Lap 1, Lap 2 and Lap 3 on three separate lines.**
WHERE The loop body, where lap holds the current value.
WHEN Once per pass, three passes in total.
WHY Printing the loop constant is the simplest way to see exactly which values a loop visits.

5 `for _ in 0..<2 {`

WHO The for-in loop over a half-open range that starts at 0 and stops before 2.
WHAT **Runs the body twice, for 0 and 1, without naming the current number.**
WHERE The underscore means no constant is created for the loop at all.
WHEN Starts as soon as the first loop has finished.
WHY When you only need to repeat something, the underscore tells readers and the compiler that the value is deliberately unused.

6 `print("Beep")`

WHO print, inside the body of the second loop, which never gives its current number a name.
WHAT **Writes Beep to the console on each pass, so two Beep lines appear one after the other.**
WHERE The body of the repeat-twice loop.
WHEN Twice: the half-open range excludes its upper bound, 2.
WHY A half-open range from 0 up to but not including n always runs exactly n times, which is why it is the usual way to repeat n times.

```
9  for minute in stride(from: 0, to: 60, by: 20) {
```

WHO The standard library's stride function, which builds a sequence of evenly spaced numbers.
WHAT **Produces 0, 20 and 40. The to: label means 60 itself is excluded.**
WHERE minute is local to this loop body, just like lap above.
WHEN The sequence is produced lazily, one value per pass.
WHY stride replaces the C-style for loop with an index, a condition and an increment, which Swift removed because it was easy to get subtly wrong.

```
10 print("Stretch at minute \(minute)")
```

WHO print, inside the stride loop, reading minute through string interpolation on every pass.
WHAT **Prints Stretch at minute 0, then 20, then 40.**
WHERE The body of the stride loop.
WHEN Three times, once per stride value.
WHY A fitness timer reminder is a natural use of fixed-size steps.

```
13 for count in stride(from: 3, through: 1, by: -1) {
```

WHO stride again, this time with a negative step.
WHAT **Counts down 3, 2, 1. The through: label includes the end value 1.**
WHERE count is local to this loop.
WHEN Three passes, from the largest value to the smallest.
WHY through: and to: mirror the closed and half-open range operators, so you choose whether the end value is included.

```
14 print("\(count)...")
```

WHO print, inside the countdown loop, interpolating count and adding three dots after it.
WHAT **Prints 3..., 2... and 1... on separate lines.**
WHERE The body of the countdown loop.
WHEN Once per value of the countdown.
WHY A countdown is the classic case for a backward stride, because a range like 3...1 is not allowed.

```
17 for n in 1...10 where n.isMultiple(of: 4) {
```

WHO The for-in loop together with a where clause that filters the sequence.
WHAT **Visits 1 through 10 but lets only numbers divisible by 4, which are 4 and 8, into the body.**
WHERE The where clause belongs to the for statement itself, not to the body.
WHEN The condition is checked for every value; the body runs only when it is true.
WHY where filters in place, saving an if statement and an extra level of indentation inside the loop.

```
18 print("\(n) is a multiple of 4")
```

WHO print, inside the filtered loop, which only ever sees the values that passed the where clause.
WHAT **Prints 4 is a multiple of 4 and 8 is a multiple of 4.**
WHERE The body of the last loop.
WHEN Only on the two passes that passed the where filter.
WHY Shows that the body never even sees the values the filter rejected.

Try it

1. Change 1...3 to 1..<3. How many laps print now?
2. Make the countdown start at 5 instead of 3.
3. Change the stride step from 20 to 15 and predict the output before you run.

CODE · RUNS IN THE BROWSER

Skipping and stopping: continue and break

Two keywords change the flow of a loop. `continue` skips the rest of the current pass and moves to the next item. `break` leaves the loop completely. Here a weather sensor reports temperatures, and `-999` marks a failed reading. `enumerated()` hands you each item together with its index.

```
let readings = [21, 23, -999, 25, 31, 22]
var total = 0
var validCount = 0

for (index, reading) in readings.enumerated() {
    if reading == -999 {
        print("Skipping bad reading at index \(index)")
        continue
    }
    if reading > 30 {
        print("Heat alert at index \(index): \(reading)")
        break
    }
    total += reading
    validCount += 1
}
print("Averaged \(validCount) readings: \(total / validCount)")
```

OUTPUT

```
Skipping bad reading at index 2
Heat alert at index 4: 31
Averaged 3 readings: 23
```

Line by line

1 `let readings = [21, 23, -999, 25, 31, 22]`

- WHO — Your program creates an array; the compiler infers its type as [Int].
- WHAT — **Stores six temperature readings in order, including one failed reading, -999.**
- WHERE — A top-level constant; the loop below reads it but never changes it.
- WHEN — Created at the start of the run, before the loop, and it stays the same through every pass.
- WHY — A sentinel value such as -999 is a common way for hardware to report a failed measurement.

2 `var total = 0`

- WHO — Your program, declaring a variable the loop will update.
- WHAT — **Starts the running sum of valid readings at 0.**
- WHERE — Top-level scope, outside the loop, so it keeps its value from one pass to the next.
- WHEN — Before the loop, so it exists before the first pass adds to it.
- WHY — A value declared inside the loop would be recreated on every pass and could never accumulate.

3 `var validCount = 0`

- WHO — Your program, declaring a second counter.
- WHAT — **Counts how many readings were actually added to total.**
- WHERE — Also at top level, next to total.
- WHEN — Before the loop starts, right after total, so both counters are ready for the first pass.
- WHY — The average needs to divide by the number of good readings, not by the length of the array.

```
5  for (index, reading) in readings.enumerated() {
```

WHO The loop and enumerated(), which pairs each element with its position.
WHAT **Each pass receives a pair and splits it into index (0, 1, 2 and so on) and reading (21, 23, -999 and so on).**
WHERE index and reading are constants that exist for one pass of the loop body.
WHEN Up to six passes, one per reading, unless something stops the loop early.
WHY Splitting the pair right in the for statement gives both values clear names without extra lines.

```
6      if reading == -999 {
```

WHO The runtime compares the current reading with the sentinel value.
WHAT **Is true only on the third pass, where the reading is -999.**
WHERE An if statement nested inside the loop body.
WHEN Checked at the start of every pass.
WHY Rejecting bad data first keeps the calculation lines below free of special cases.

```
7          print("Skipping bad reading at index \(index)")
```

WHO print, inside the bad-reading branch, interpolating the index that enumerated() supplied for this pass.
WHAT **Prints Skipping bad reading at index 2.**
WHERE Inside the if, inside the loop, eight spaces deep.
WHEN Only on the pass that found -999.
WHY Logging what was skipped makes the program honest about the data it ignored.

```
8          continue
```

WHO Loop control: continue acts on the innermost loop that contains it.
WHAT **Jumps straight to the next pass, skipping the lines that add to total and validCount.**
WHERE Inside the if; it affects the for loop around it, not the if itself.
WHEN Only on the pass with the bad reading.
WHY continue says skip this item but keep going, which is exactly what bad data needs.

```
10     if reading > 30 {
```

WHO The runtime compares the reading with the heat threshold.
WHAT **Is true on the fifth pass, where the reading is 31.**
WHERE A second if statement inside the loop body.
WHEN Checked on every pass that was not skipped by continue.
WHY This turns the loop into a search for the first dangerous value.

```
11         print("Heat alert at index \(index): \(reading)")
```

WHO print, inside the heat-alert branch, interpolating both the position and the temperature from this pass.
WHAT **Prints Heat alert at index 4: 31.**
WHERE Inside the second if, inside the loop.
WHEN Only on the pass that found a reading above 30.
WHY Reporting both the index and the value tells you exactly where in the data the problem appeared.

```
12         break
```

WHO Loop control: break ends the innermost loop immediately.
WHAT **Leaves the for loop entirely; the last reading, 22, is never examined or added.**
WHERE Inside the if; execution resumes at the first line after the loop's closing brace.
WHEN On the fifth pass, right after the alert prints.
WHY break suits searches: once you have found what you were looking for, continuing would waste work.

14 `total += reading`

WHO	The += operator in the loop body, writing to total, a variable that lives outside the loop.
WHAT	**Adds the current reading to the running total.**
WHERE	Inside the loop, but it changes total, which lives outside.
WHEN	Only on passes that survived both checks: the readings 21, 23 and 25.
WHY	The compound operator += reads the old total, adds, and stores the result in one step.

15 `validCount += 1`

WHO	The loop body, updating the second outer variable.
WHAT	**Adds 1 to the count of good readings.**
WHERE	Inside the loop, after total is updated.
WHEN	Three times in this run, on the passes for 21, 23 and 25; the -999 pass skips it and break stops the loop at 31.
WHY	Counting as you go is simpler and safer than trying to work it out afterwards.

17 `print("Averaged \(validCount) readings: \(total / validCount)")`

WHO	print, back at top level after the loop has finished.
WHAT	**Prints Averaged 3 readings: 23, because 69 divided by 3 is exactly 23.**
WHERE	After the loop; total and validCount are still visible because they were declared before it.
WHEN	Once, after break has ended the loop.
WHY	Both values are Int, so the division is integer division; a total that does not divide evenly would be truncated.

> **Try it**
>
> 1. Change 31 to 29 and run. How many readings are averaged now?
> 2. Replace break with continue. What changes in the output, and why?

CODE · RUNS IN THE BROWSER

Mistake: using a loop's value after the loop

A name declared inside braces exists only inside those braces. That region is the name's **scope**. This program creates `doubled` inside the loop body and then tries to use it after the loop has ended. Run it and read the compiler error.

```
let distances = [3, 5, 2]

for distance in distances {
    let doubled = distance * 2
    print("Round trip: \(doubled) km")
}
print("Last round trip: \(doubled) km")
```

COMPILER ERROR (ON PURPOSE)

```
cannot find 'doubled' in scope
```

Line by line

```
1  let distances = [3, 5, 2]
```

WHO — Your program creates an array of Int values.
WHAT — **Stores three one-way trip distances in kilometers.**
WHERE — A top-level constant, visible everywhere below, including inside the loop.
WHEN — Would be created first, before the loop; in this program nothing runs, because the compile error stops the build.
WHY — Names declared in an outer scope are visible in every scope nested inside it.

```
3  for distance in distances {
```

WHO — The for-in loop, visiting each element of the array.
WHAT — **Runs the body three times, with distance set to 3, then 5, then 2.**
WHERE — Opens the loop body, a new, inner scope.
WHEN — Would run after the array is created, if the program compiled.
WHY — Each pair of braces creates a scope, which is how Swift decides where a name is visible.

```
4      let doubled = distance * 2
```

WHO — The loop body, which creates a fresh constant on every pass.
WHAT — **Computes a round trip (6, then 10, then 4) and names it doubled.**
WHERE — Declared inside the loop's braces, so its scope is a single pass of the body; it vanishes at the closing brace.
WHEN — Would be created three times, once per pass, and discarded at the end of each.
WHY — Keeping temporary values in the narrowest scope is good practice, as long as nothing outside needs them.

```
5      print("Round trip: \(doubled) km")
```

WHO — print, inside the loop body, in the same pair of braces that declared doubled.
WHAT — **Would print each round-trip distance; this line itself is perfectly legal.**
WHERE — Inside the same scope as doubled, so the name is visible here.
WHEN — Never runs, because the program fails to compile.
WHY — Shows that doubled works fine within its scope; the problem is only on the last line.

```
7  print("Last round trip: \(doubled) km")
```

WHO — The compiler, which looks up the name doubled in the current scope and every scope that encloses it.
WHAT — **Fails to compile with cannot find 'doubled' in scope, because no doubled exists at top level.**
WHERE — After the loop's closing brace, at top level, where the loop's local names are no longer visible.
WHEN — At compile time; the program never starts, so not even the Round trip lines print.
WHY — Swift catches this before running, instead of letting a variable leak out of a loop with a stale or undefined value.

> **Try it**
>
> 1. Move the last print inside the loop, just above the closing brace. What does the program print now?
> 2. Before the next step, decide where you would declare a variable so the last line can see it.

CODE · RUNS IN THE BROWSER

The fix: declare it before the loop

To use a value after a loop, declare it **before** the loop, in the outer scope, and update it inside. A `while` loop depends on this: its condition reads variables declared outside it. A `repeat-while` loop checks its condition at the end, so its body always runs at least once.

```swift
let distances = [3, 5, 2]
var lastRoundTrip = 0
for distance in distances {
    lastRoundTrip = distance * 2
}
print("Last round trip: \(lastRoundTrip) km")

var followers = 100
var weeks = 0
while followers < 1000 {
    followers *= 2
    weeks += 1
}
print("\(weeks) weeks to pass 1000: \(followers)")

var volume = 8
repeat {
    volume /= 2
    print("Volume \(volume)")
} while volume > 10
```

OUTPUT

```
Last round trip: 4 km
4 weeks to pass 1000: 1600
Volume 4
```

Line by line

1 `let distances = [3, 5, 2]`

- WHO — Your program creates the same array as in the mistake.
- WHAT — **Stores the same three one-way distances, 3, 5 and 2 kilometers, as an array of Int.**
- WHERE — A top-level constant, so the loop below and every later line of the program can read it.
- WHEN — Created first, at the very start of the run, before lastRoundTrip and the loop.
- WHY — Keeping the data identical makes it clear that only the declaration changed.

2 `var lastRoundTrip = 0`

- WHO — Your program declares a variable in the outer, top-level scope.
- WHAT — **Creates lastRoundTrip with a starting value of 0 that the loop can overwrite.**
- WHERE — Outside the loop, so it remains visible after the loop's closing brace.
- WHEN — Before the loop starts; it must exist before the first pass assigns to it.
- WHY — Swift requires a variable to hold a value before it is read; 0 is a safe starting value even if the array were empty.

3 `for distance in distances {`

- WHO — The same for-in loop as in the mistake, unchanged, handing each array element to the body.
- WHAT — **Visits 3, then 5, then 2, binding each one to the loop constant distance for one pass.**
- WHERE — Opens the inner scope of the loop body.
- WHEN — Runs three passes, right after lastRoundTrip has been created with its starting value of 0.
- WHY — The loop itself did not need to change; only the place where the result lives did.

```
4  lastRoundTrip = distance * 2
```

WHO The loop body, assigning to the outer variable.
WHAT **Overwrites lastRoundTrip with 6, then 10, then 4. There is no let, so no new constant is created.**
WHERE Inside the loop, writing to a name that belongs to the outer scope.
WHEN Once per pass; each pass replaces the previous value.
WHY Assigning instead of declaring is the whole fix: the loop updates a name that outlives it.

```
6  print("Last round trip: \(lastRoundTrip) km")
```

WHO print, at top level after the loop.
WHAT **Prints Last round trip: 4 km, the value written by the final pass.**
WHERE Outside the loop, in the same scope where lastRoundTrip was declared.
WHEN Once, after the loop has finished all three passes, as the first line of output in this program.
WHY The compiler can now find the name, because it was declared in this scope.

```
8  var followers = 100
```

WHO Your program declares the variable the next loop will test and change.
WHAT **Starts a follower count at 100.**
WHERE Top-level scope, outside the loop that follows.
WHEN Before the while loop, which reads it immediately.
WHY A while condition is checked before the body ever runs, so its variables must already exist outside the loop.

```
9  var weeks = 0
```

WHO Your program, declaring a second variable that counts how many passes the while loop makes.
WHAT **Will record how many weeks it takes to pass 1000 followers.**
WHERE Top-level scope, next to followers, so the print after the loop can still read it.
WHEN Before the while loop, right after followers, so it already holds 0 when the first pass adds to it.
WHY The answer is needed after the loop, so the counter must live outside it.

```
10 while followers < 1000 {
```

WHO The while loop, which re-checks its condition before every pass.
WHAT **Tests 100, 200, 400 and 800, all less than 1000, so the body runs each time; then 1600, which ends the loop.**
WHERE Its condition reads followers, declared above, which is why while loops keep their state outside.
WHEN The condition is evaluated five times and the body runs four times. If followers started at 1000, the body would never run.
WHY Use while when you cannot know in advance how many passes you need; for-in needs a known sequence.

```
11 followers *= 2
```

WHO The *= operator in the while body, writing to followers, a variable declared outside the loop.
WHAT **Doubles the follower count on each pass.**
WHERE Inside the while body, the braces that run only after the condition has been checked and found true.
WHEN Once per pass, before the condition is checked again.
WHY Something in a while body must move the condition toward false, or the loop runs forever and the app freezes.

```
12 weeks += 1
```

WHO The loop body, updating the counter.
WHAT **Adds 1 to weeks, recording one more week each time the follower count has doubled.**
WHERE Inside the while body, after the doubling.
WHEN Four times in this run, once after each doubling: at 200, 400, 800 and 1600 followers.
WHY Counting passes is a common way to answer how long or how many times.

```swift
14 print("\(weeks) weeks to pass 1000: \(followers)")
```

WHO print, after the while loop ends.
WHAT **Prints 4 weeks to pass 1000: 1600.**
WHERE At top level, where both outer variables are visible.
WHEN Once, when the condition finally became false.
WHY Printing the final state shows the loop overshot to 1600, because it stops only after a check fails.

```swift
16 var volume = 8
```

WHO Your program declares the variable for the last loop.
WHAT **Starts the volume at 8, already 10 or less.**
WHERE Top-level scope, outside the repeat loop.
WHEN Before the repeat loop begins, as the last declaration in the program, after the while loop print.
WHY Starting below the limit shows the special behavior of repeat-while clearly.

```swift
17 repeat {
```

WHO The repeat-while loop, the Swift loop that checks its condition at the bottom instead of the top.
WHAT **Starts the body without checking anything first.**
WHERE Opens the loop body; the condition comes after the closing brace.
WHEN Immediately; the check happens only at the end of each pass.
WHY Use repeat-while when the body must run once before a check makes sense, such as asking for input before validating it.

```swift
18 volume /= 2
```

WHO The repeat body, updating the outer variable.
WHAT **Halves volume using integer division, so 8 becomes 4.**
WHERE Inside the repeat body, writing to volume, which was declared in the outer scope above the loop.
WHEN On the first and, in this run, only pass.
WHY Like a while body, a repeat body must move its condition toward false.

```swift
19 print("Volume \(volume)")
```

WHO print, inside the repeat body, interpolating the value volume holds right after the halving.
WHAT **Writes Volume 4 to the console, the third and final line of output.**
WHERE Inside the repeat body, after the halving.
WHEN Once in this run: the body runs a single time, and then the bottom check finds 4 is not greater than 10.
WHY Printing inside the body shows how many passes actually happened.

```swift
20 } while volume > 10
```

WHO The runtime evaluates the condition after the body has finished.
WHAT **4 is not greater than 10, so the loop stops after one pass.**
WHERE Trails the closing brace of the repeat body.
WHEN At the end of every pass.
WHY The body ran even though volume started below the limit; a while loop with the same condition would have run zero times.

Try it

1. Change followers to 1000. How many weeks print now, and why?
2. Change volume to 80 and predict every line the repeat loop prints.

INTERACTIVE

Trace a loop step by step

Watch a loop run one step at a time. Pick a loop, then press **Step** (or the right arrow key) to advance and **Back** to rewind. Each step highlights the line Swift is running, shows every variable's current value, and adds any printed output. Notice that a `while` checks its condition before every pass, that a variable changes only on the line that assigns it, and that `continue` and `break` jump to different places.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. What does this print?

```
let speed = 15
switch speed {
case 10...20: print("Moderate")
case 15: print("Exactly fifteen")
default: print("Fast")
}
```

   A. Fast
   B. Moderate
   C. Moderate, then Exactly fifteen
   D. Exactly fifteen

2. This program does not compile. Why?

```
let level = 3
switch level {
case 1:
    print("Easy")
case 2, 3:
    print("Hard")
}
```

   A. Case 2, 3 needs a break statement at the end
   B. A case cannot list two values separated by a comma
   C. switch only works with String values
   D. The cases do not cover every possible Int, so it needs default

3. What does this print?

```
for n in 1...6 {
    if n == 2 { continue }
    if n == 5 { break }
    print(n, terminator: " ")
}
```

A. 1 3 4
B. 1 3 4 6
C. 1 2 3 4
D. 3 4 5 6

4. Both loops start with the value 5. How many times does each body run?

```
var a = 5
while a < 5 { a += 1 }

var b = 5
repeat { b += 1 } while b < 5
```

A. Both run zero times
B. Both run once
C. The while body runs zero times; the repeat-while body runs once
D. The while body runs once; the repeat-while body runs zero times

5. Which change makes this program compile and print the last label?

```
let temps = [18, 24, 31]
for t in temps {
    let label = t > 25 ? "warm" : "cool"
}
print(label)
```

A. Change let label to var label inside the loop
B. Declare var label = "" before the loop, then assign label inside it
C. Add break on the line after let label
D. Move print(label) above the for line

Answers: Decisions and Loops

1. **B. Moderate** Swift tests the cases from top to bottom and runs only the **first** match. 15 lies in `10...20`, so Moderate prints and the switch ends. Cases never fall through, so `case 15` is never reached. Put specific cases before broad ranges.
2. **D. The cases do not cover every possible Int, so it needs default** A Swift switch must be **exhaustive**: every possible value of the type must match some case. An `Int` could be 0, 4 or -50, so the compiler reports "switch must be exhaustive" until you add `default`. A `break` is never required, because cases do not fall through, and listing values with commas is legal.
3. **A. 1 3 4** `continue` skips the rest of the pass where `n` is 2, so 2 never prints. `break` ends the whole loop when `n` reaches 5, so neither 5 nor 6 prints. The near miss `1 3 4 6` treats `break` like `continue`, but `break` leaves the loop completely.
4. **C. The while body runs zero times; the repeat-while body runs once** A `while` loop checks its condition **before** each pass: 5 is not less than 5, so its body never runs. A `repeat-while` loop checks **after** each pass, so its body always runs once before the check fails. That difference is the only reason to choose one over the other.
5. **B. Declare var label = "" before the loop, then assign label inside it** `label` is declared inside the loop body, so its scope ends at the loop's closing brace and the final `print` cannot find it. Switching `let` to `var` changes whether the value can be modified, not where the name is visible. Declaring it before the loop puts it in the outer scope; assigning inside updates it, and the program prints warm.

Functions

CONCEPT

What a function is

A **function** is a named block of code you run by *calling* it, as often as you like. It takes inputs called **parameters** and can hand back one result, its **return value**. Swift gives each parameter two names: an **argument label** that the caller writes, and a **parameter name** that the body uses.

`func greeting(for name: String) -> String` is called as `greeting(for: "Ada")`. The label `for` makes the call read like a phrase, while `name` reads well inside the body.

Parameter	A named, typed input the caller must supply, unless it has a default value.
Argument label	The word written before a value at the call, such as at in insert(_:at:).
Return type	The type written after the arrow. With no return type, the function returns nothing.
Call site	Any place in code where the function is used. Swift names are designed to read well there.

> **Clarity at the point of use**
>
> Apple's Swift API Design Guidelines ask that calls read like English phrases: `list.remove(at: 2)`, `names.insert("Zoe", at: 0)`, `player.move(to: spot)`. Name a function for what it does, and use labels to explain each argument's role.

CODE · RUNS IN THE BROWSER

Parameters and argument labels

Two functions. `greeting(for:at:)` takes two labeled parameters and returns a `String`. `seconds(in:)` has a body that is a single expression, so Swift returns that expression without the word `return`. Notice how the calls at the bottom read almost like sentences.

```
func greeting(for name: String, at hour: Int) -> String {
    let part = hour < 12 ? "morning" : "afternoon"
    return "Good \(part), \(name)."
}

func seconds(in minutes: Int) -> Int {
    minutes * 60
}

let message = greeting(for: "Ada", at: 9)
print(message)
print(greeting(for: "Omar", at: 15))
print(seconds(in: 2))
```

OUTPUT

```
Good morning, Ada.
Good afternoon, Omar.
120
```

Line by line

1 `func greeting(for name: String, at hour: Int) -> String {`

- WHO — The Swift compiler, which records this function's name, parameter types and return type before any code runs.
- WHAT — **Declares a function named greeting that takes a String and an Int and promises to give back a String.**
- WHERE — At the top level of the program, so every line below it in this file can call it.
- WHEN — Checked at compile time; the body runs later, once for every call.
- WHY — Each parameter has two names: callers write the labels for and at, while the body uses name and hour, so both sides read naturally.

2 `let part = hour < 12 ? "morning" : "afternoon"`

- WHO — The function body, working with the hour value the caller passed in.
- WHAT — **Picks the word morning when hour is less than 12 and afternoon otherwise, and stores it in the constant part.**
- WHERE — Inside the body of greeting; part is a local constant that exists only during one call.
- WHEN — Every time greeting is called, right after the arguments are bound to name and hour.
- WHY — The ternary operator chooses between two values in one expression, which keeps a simple either-or decision on one readable line.

3 `return "Good \(part), \(name)."`

- WHO — The function body, handing its result back to whoever called it.
- WHAT — **Builds a sentence such as Good morning, Ada. with string interpolation and returns it.**
- WHERE — The last line of greeting; return ends the call on the spot.
- WHEN — Once per call, after part has been chosen.
- WHY — The declaration promised a String, and the compiler checks that every path through the body returns one.

6 `func seconds(in minutes: Int) -> Int {`

- WHO — The compiler, registering a second top-level function.
- WHAT — **Declares seconds, which takes one Int labeled in and returns an Int.**
- WHERE — Top level of the file, beside greeting; inside the body the parameter is called minutes.
- WHEN — Checked at compile time; the body runs only when something calls it.
- WHY — The call seconds(in: 2) reads as seconds in 2 minutes, which is exactly what reading like a phrase means.

7 `minutes * 60`

- WHO — The body of seconds, which has no return keyword.
- WHAT — **Multiplies the minutes argument by 60 and makes that product the result of the call.**
- WHERE — The only statement in the body of seconds; minutes is a local constant holding whatever value the caller passed after the in: label.
- WHEN — Each time seconds is called; in this program that happens once, on the last line, with minutes set to 2.
- WHY — When a body is one expression, Swift returns it implicitly, which keeps tiny functions short and clear.

10 `let message = greeting(for: "Ada", at: 9)`

- WHO — The program's top-level code, calling the function.
- WHAT — **Calls greeting with Ada and 9, then stores the returned text in the constant message.**
- WHERE — Top-level code; message is a global constant whose type, String, is inferred from the return type.
- WHEN — When execution reaches this line; the declarations above it never run on their own.
- WHY — Naming a result is worthwhile when you use it more than once or when the name explains its meaning.

11 `print(message)`

- **WHO** The standard library print function, receiving the String that greeting returned and the constant message stored.
- **WHAT** **Prints Good morning, Ada. because 9 is less than 12.**
- **WHERE** Top-level code, writing to the console.
- **WHEN** Right after message has been computed.
- **WHY** This output checks the morning branch of the ternary inside the function.

12 `print(greeting(for: "Omar", at: 15))`

- **WHO** print and greeting working together; the inner call finishes first.
- **WHAT** **Calls greeting with Omar and 15 and prints the result directly: Good afternoon, Omar.**
- **WHERE** Top-level code; the returned String is a temporary value that is never given a name.
- **WHEN** The inner call runs first, then print receives its result.
- **WHY** A call is an expression, so its result can go straight into another call when you need it only once.

13 `print(seconds(in: 2))`

- **WHO** The program, calling seconds and then print.
- **WHAT** **Computes 2 times 60 and prints 120.**
- **WHERE** The last line of top-level code; the Int 120 returned by seconds is a temporary that goes straight into print.
- **WHEN** Third and last, after both greeting lines are on the console, so 120 becomes the final line of output.
- **WHY** It shows that a function with an implicit return behaves exactly like one written with return.

> **Try it**
>
> 1. Print greeting(for: "Mia", at: 12). Which word does she get, and why?
> 2. Write func hours(in days: Int) -> Int with an implicit return, then print hours(in: 3).

CODE · RUNS IN THE BROWSER

Mistake: leaving out a label

Argument labels are part of a function's name. Programmers coming from other languages often call a function with bare values. Swift refuses: the call below does not match the declaration, so the program does not compile. Run it and read the message.

```
1  func brew(drink: String, size: String) -> String {
2      "One \(size) \(drink), coming up."
3  }
4
5  print(brew("latte", size: "large"))
```

COMPILER ERROR (ON PURPOSE)

```
missing argument label
```

Line by line

1 `func brew(drink: String, size: String) -> String {`

- **WHO** The compiler, which records the full name of this function as brew(drink:size:).
- **WHAT** **Declares brew with two String parameters; with only one name written, each name is both the label and the parameter name.**
- **WHERE** Top level of the coffee-shop program.
- **WHEN** Compile time; the body never gets a chance to run because the build fails.
- **WHY** Writing a single name makes the label and the parameter name the same word, the most common case in Swift.

2 `"One \(size) \(drink), coming up."`

- **WHO** The body of brew, which would build the order text if the program ever compiled.
- **WHAT** **Builds and implicitly returns a sentence such as One large latte, coming up.**
- **WHERE** The only expression in the body of brew, where drink and size would be local constants holding the arguments of one call.
- **WHEN** It would run on each call, but never does in this broken program.
- **WHY** A body made of one expression returns its value without the return keyword.

5 `print(brew("latte", size: "large"))`

- **WHO** The compiler, matching this call against every function named brew.
- **WHAT** **Tries to call brew without the drink: label, so the compiler reports missing argument label drink: in call.**
- **WHERE** Top-level code; the error points at the first argument.
- **WHEN** At compile time, before anything prints.
- **WHY** Labels are part of the function's identity, so the compiler will not guess which function or argument you meant.

> **Try it**
>
> 1. Add drink: before "latte" and run it again.
> 2. Now remove size: instead. Which label does the compiler ask for this time?

CODE · RUNS IN THE BROWSER

The fix: write the label, or opt out with _

Two fixes. Write the label the declaration asks for, or change the declaration: an underscore `_` before a parameter name removes its label when the first argument is obvious. A **default value** such as `= "medium"` lets callers leave an argument out entirely.

```
func brew(drink: String, size: String) -> String {
    "One \(size) \(drink), coming up."
}

func order(_ drink: String, size: String = "medium") -> String {
    "One \(size) \(drink), coming up."
}

print(brew(drink: "latte", size: "large"))
print(order("mocha"))
print(order("tea", size: "small"))
```

OUTPUT

```
One large latte, coming up.
One medium mocha, coming up.
One small tea, coming up.
```

Line by line

1 `func brew(drink: String, size: String) -> String {`

- WHO The compiler, registering brew(drink:size:) exactly as before.
- WHAT **Declares the same two-parameter function as in the broken example.**
- WHERE At the top level of the fixed program, unchanged from the broken version, so the calls at the bottom can reach it.
- WHEN Compiled before the program starts running.
- WHY The declaration was never the problem; the call was, so this version keeps it unchanged.

2 `"One \(size) \(drink), coming up."`

- WHO The body of brew, unchanged from the broken version.
- WHAT **Returns the order sentence built from both arguments.**
- WHERE Inside brew, as its single expression.
- WHEN Each time brew is called; here that is once, from the first print line at the bottom.
- WHY Same implicit return as before; nothing in the body needed fixing.

5 `func order(_ drink: String, size: String = "medium") -> String {`

- WHO The compiler, recording a function whose full name is order(_:size:).
- WHAT **Declares order with an unlabeled first parameter and a size parameter that defaults to medium.**
- WHERE At the top level, declared right after brew, so both functions are visible to the three print calls below.
- WHEN Compile time; the default is filled in at every call that leaves size out.
- WHY The underscore drops the label because order("mocha") already reads clearly, and the default covers the common size so callers type less.

6 `"One \(size) \(drink), coming up."`

- WHO The body of order, run each time the function is called.
- WHAT **Returns the same kind of sentence as brew, using the size the call supplied or the default.**
- WHERE Inside order, where drink and size are local constants holding this call's arguments.
- WHEN Once per call to order; this program calls it twice, first with the default size and then with small.
- WHY Inside the body the parameter is still called drink; the underscore only changes what callers write.

9 `print(brew(drink: "latte", size: "large"))`

- WHO The program, now calling brew correctly.
- WHAT **Passes both labeled arguments and prints One large latte, coming up.**
- WHERE Top-level code, after both functions have been declared.
- WHEN First, producing the first line of output.
- WHY Labels must appear, and in the declared order; the compiler would also reject size: written before drink:.

10 `print(order("mocha"))`

- WHO The program, relying on the default value.
- WHAT **Calls order with only the drink, so size becomes medium, and prints One medium mocha, coming up.**
- WHERE Top-level code, one line below the brew call.
- WHEN Second, after the latte line has printed.
- WHY A default keeps calls short for the common case without writing a second function.

```
11 print(order("tea", size: "small"))
```

WHO — Top-level code calling order with an explicit size, which replaces the default value medium for this one call only.

WHAT — **Supplies size small, so the default is ignored, and prints One small tea, coming up.**

WHERE — The last line of top-level code; the returned sentence is a temporary String handed straight to print.

WHEN — Last, after the latte and mocha lines, so One small tea, coming up. is the third line of output.

WHY — A parameter with a default keeps its label when you do pass it, so an override is easy to spot.

Try it

1. Change the default size to "large" and run it. Which lines change?
2. Give brew a default too, size: String = "small", then print brew(drink: "cocoa").

CODE · RUNS IN THE BROWSER

Many inputs, several outputs

A **variadic** parameter, written `Int...`, accepts any number of values; inside the function they arrive as an array. A **tuple** return type bundles several values into one result, and labeling its parts lets callers write `week.average` instead of remembering positions. This fitness tracker summarizes daily step counts.

```swift
func summary(of steps: Int...)
    -> (lowest: Int, highest: Int, average: Double) {
    var total = 0
    for count in steps {
        total += count
    }
    let average = Double(total) / Double(steps.count)
    return (steps.min() ?? 0, steps.max() ?? 0, average)
}

let week = summary(of: 4200, 8100, 6500, 9000)
print(week.lowest, week.highest)
print(week.average)

let (low, high, _) = summary(of: 3000, 5000)
print("\(low) to \(high)")
```

OUTPUT

```
4200 9000
6950.0
3000 to 5000
```

Line by line

1 `func summary(of steps: Int...)`

- **WHO** The compiler, registering a function named summary(of:).
- **WHAT** **Declares a function whose one parameter accepts zero or more Int values, separated by commas at the call.**
- **WHERE** Top level of the program; inside the body, steps has the array type [Int].
- **WHEN** Compile time; each call packs its arguments into a fresh array.
- **WHY** A variadic parameter keeps calls natural, summary(of: 4200, 8100), without making the caller build an array first.

2 `-> (lowest: Int, highest: Int, average: Double) {`

- **WHO** The compiler, reading the return type that continues the declaration.
- **WHAT** **Promises a tuple of three labeled values: two Ints and a Double.**
- **WHERE** Still part of the function signature, wrapped onto a second line so it stays readable.
- **WHEN** Checked at compile time against the return statement below.
- **WHY** A labeled tuple returns several related values without declaring a whole new type, which suits small, local results.

3 `var total = 0`

- **WHO** The body of summary, preparing to add up the arguments.
- **WHAT** **Creates a running total that starts at zero.**
- **WHERE** A local variable inside summary; every call gets its own fresh copy.
- **WHEN** At the start of each call.
- **WHY** It must be var because the loop below keeps changing it.

4 `for count in steps {`

- **WHO** A for-in loop walking through the array built from the arguments.
- **WHAT** **Visits each step count in order and binds it to the constant count.**
- **WHERE** Inside summary; count exists only within the loop braces.
- **WHEN** Once per argument, in the order the caller wrote them.
- **WHY** A loop is the plain way to add up values; the next lesson shows a shorter way with reduce.

5 `total += count`

- **WHO** The body of the for loop, run once for each value.
- **WHAT** **Adds the current count to the running total.**
- **WHERE** Inside the for loop, inside summary.
- **WHEN** Once per pass through the loop.
- **WHY** Compound assignment updates total in place instead of writing total = total + count.

7 `let average = Double(total) / Double(steps.count)`

- **WHO** The function body, after the loop has finished.
- **WHAT** **Converts the total and the number of values to Double and divides them, keeping the fraction.**
- **WHERE** A local constant inside summary, created after the loop ends and discarded as soon as the function returns its tuple.
- **WHEN** Once, after every count has been added.
- **WHY** Dividing two Ints would throw away the fraction; with no arguments at all this divides 0 by 0 and gives nan rather than crashing.

```
8  return (steps.min() ?? 0, steps.max() ?? 0, average)
```

WHO — The function body, building its result.
WHAT — **Returns a tuple of the smallest value, the largest value and the average, in the declared order.**
WHERE — The final statement of summary; return ends the call and hands the three-part tuple back to the caller.
WHEN — Once per call, after the average is known.
WHY — min() and max() return optionals because an empty array has neither, so ?? supplies 0 as a safe fallback.

```
11 let week = summary(of: 4200, 8100, 6500, 9000)
```

WHO — Top-level code, calling summary with four arguments.
WHAT — **Packs four step counts into the variadic parameter and stores the returned tuple in week.**
WHERE — A global constant whose type is the labeled tuple from the declaration.
WHEN — At run time, the first time top-level execution reaches a call to summary; the declaration above does nothing until now.
WHY — Four days of steps are passed as plain comma-separated values, with no array brackets needed.

```
12 print(week.lowest, week.highest)
```

WHO — print, reading two parts of the tuple by their labels.
WHAT — **Prints 4200 9000, separated by the default single space.**
WHERE — Top-level code, reading from the global tuple week.
WHEN — First line of output, right after summary returns.
WHY — Labels make tuple access self-explanatory; week.0 would also work but hides the meaning.

```
13 print(week.average)
```

WHO — print, reading the third part of the tuple.
WHAT — **Prints 6950.0, the average of the four counts.**
WHERE — Top-level code, still reading the same tuple.
WHEN — Second line of output, after lowest and highest.
WHY — The value is a Double, so Swift prints a decimal point even though the number is whole.

```
15 let (low, high, _) = summary(of: 3000, 5000)
```

WHO — Top-level code, unpacking the returned tuple.
WHAT — **Calls summary with two values and splits the result into two new constants, low and high, ignoring the average.**
WHERE — Two global constants created by one statement.
WHEN — After the first summary has been printed.
WHY — This is called destructuring; the underscore discards a part you do not need, so nothing sits unused.

```
16 print("\(low) to \(high)")
```

WHO — print, receiving one String that interpolation built from the destructured constants low and high.
WHAT — **Prints 3000 to 5000 by placing both constants into one string.**
WHERE — The last line of top-level code, reading the two global constants created by the destructuring line above.
WHEN — Last, after the second summary call has returned.
WHY — After destructuring, low and high are ordinary Ints you can use anywhere.

Try it

1. Add a fifth day, 12000, to the first call. Which printed values change?
2. Print week.0 and week.2. Do the positions match the labels?

CODE · RUNS IN THE BROWSER

inout: letting a function change your variable

Parameters are constants inside a function, so normally a function cannot change the caller's variables. Marking a parameter `inout` changes that: the value is copied in, the function may modify it, and the new value is copied back out when the call ends. The caller must write `&` to agree.

```
1  func applyDiscount(_ percent: Double, to price: inout Double) {
2      price -= price * percent / 100
3  }
4
5  var jacket = 80.0
6  print("Before: \(jacket)")
7  applyDiscount(25, to: &jacket)
8  print("After: \(jacket)")
9  applyDiscount(10, to: &jacket)
10 print("After again: \(jacket)")
```

OUTPUT

```
Before: 80.0
After: 60.0
After again: 54.0
```

Line by line

1 `func applyDiscount(_ percent: Double, to price: inout Double) {`

- WHO — The compiler, recording applyDiscount(_:to:) with one ordinary and one inout parameter.
- WHAT — **Declares a function that takes a percentage and a price it is allowed to modify; it returns nothing.**
- WHERE — Top level of the store program.
- WHEN — Compile time; with no return type, the function returns Void, an empty value.
- WHY — inout makes the side effect visible in the signature, so readers know a call changes one of its arguments.

2 `price -= price * percent / 100`

- WHO — The function body, writing to the inout parameter.
- WHAT — **Subtracts percent percent of the price from price; 25 percent of 80.0 removes 20.0.**
- WHERE — Inside applyDiscount, working on a local copy of the caller's variable.
- WHEN — Once per call; the new value is written back to the caller as the function returns.
- WHY — Without inout this line would not compile, because ordinary parameters are let constants.

5 `var jacket = 80.0`

- WHO — Top-level code, creating the variable the function will change.
- WHAT — **Stores 80.0 in a Double variable named jacket.**
- WHERE — A global variable; it has to be var so a function may modify it.
- WHEN — At the start of the program.
- WHY — An inout argument must be mutable; passing a let constant is a compile error.

6 `print("Before: \(jacket)")`

- WHO — print, receiving a String built by interpolating the current value of the global variable jacket.
- WHAT — **Prints Before: 80.0; a Double always shows its decimal point.**
- WHERE — Top-level code, reading the global variable jacket.
- WHEN — Right after jacket is created and before the first applyDiscount call, so it shows the original price of 80.0.
- WHY — Showing the starting value makes the change easy to see.

```
7  applyDiscount(25, to: &jacket)
```

WHO The caller, handing jacket over to be modified.
WHAT **Applies a 25 percent discount; jacket holds 60.0 when the call returns.**
WHERE Top-level code; the ampersand marks jacket as passed in-out.
WHEN After the starting price has been printed.
WHY The ampersand is required at the call, so anyone reading this line knows jacket may change.

```
8  print("After: \(jacket)")
```

WHO print, reading the caller's variable again.
WHAT **Prints After: 60.0, proving the variable outside the function changed.**
WHERE Top-level code, outside the function that changed jacket.
WHEN Right after the first call returns.
WHY It confirms the copy-back step happened when the call ended.

```
9  applyDiscount(10, to: &jacket)
```

WHO The caller, applying a second discount.
WHAT **Takes 10 percent off the current 60.0, leaving 54.0.**
WHERE Top-level code; the ampersand is needed on every inout call, not just the first.
WHEN After the first discount has been written back.
WHY Each call starts from the latest value, so discounts stack.

```
10 print("After again: \(jacket)")
```

WHO print, reading jacket one last time.
WHAT **Prints After again: 54.0, the price left after 10 percent was taken off the already discounted 60.0.**
WHERE The last line of top-level code, reading jacket after the second inout call has written its new value back.
WHEN Last, after both discounts have been written back.
WHY It shows the two discounts compounding instead of both applying to the original price.

> **Try it**
>
> 1. Add var shoes = 120.0, discount it by 50 percent, and print it.
> 2. Swap the order of the two discounts. Is the final price the same?

CODE · RUNS IN THE BROWSER

Functions are values

Every Swift function has a **type**, written like `(Int, Int) -> Int`: two Ints in, one Int out. So you can store a function in a constant and pass it to another function. You can also declare a **nested function**, a helper that lives inside another function and is invisible outside it.

```swift
func add(_ a: Int, _ b: Int) -> Int {
    a + b
}

func multiply(_ a: Int, _ b: Int) -> Int {
    a * b
}

func report(_ a: Int, _ b: Int,
            using operation: (Int, Int) -> Int) {
    func describe(_ value: Int) -> String {
        let kind = value.isMultiple(of: 2) ? "even" : "odd"
        return "\(value) is \(kind)"
    }
    print(describe(operation(a, b)))
}

let combine: (Int, Int) -> Int = add
print(combine(2, 3))
report(4, 5, using: multiply)
report(4, 5, using: add)
```

OUTPUT

```
5
20 is even
9 is odd
```

Line by line

1 `func add(_ a: Int, _ b: Int) -> Int {`

- **WHO** The compiler, recording add with a function type of two Ints in and one Int out.
- **WHAT** **Declares a function that takes two unlabeled Ints and returns their sum.**
- **WHERE** At the top level of the program, so add can be called by name or stored in a constant further down.
- **WHEN** At compile time; the body runs only when add is called.
- **WHY** Its parameter and return types match multiply exactly, so either one fits wherever that function type is expected.

2 `a + b`

- **WHO** The body of add, its only statement.
- **WHAT** **Returns the sum of the two arguments through an implicit return.**
- **WHERE** Inside add, where a and b are local constants.
- **WHEN** Each time add runs, whether called by name or through a stored reference.
- **WHY** The body is one expression, so no return keyword is needed.

5 `func multiply(_ a: Int, _ b: Int) -> Int {`

- **WHO** The compiler, recording a second function.
- **WHAT** **Declares multiply with exactly the same shape as add.**
- **WHERE** Top level, right below add, sharing its function type.
- **WHEN** Compiled with the rest of the file; it runs only when report asks it to.
- **WHY** Two functions of one type let the program choose a behavior while it runs.

```
6   a * b
```

WHO	The body of multiply, its only statement.
WHAT	**Returns the product of the two arguments.**
WHERE	Inside multiply, using its own local a and b.
WHEN	Whenever multiply is called; in this program only once, through the operation parameter inside report, for the 4 and 5 call.
WHY	Like add, the body is a single expression, so Swift returns the product implicitly without the return keyword.

```
9   func report(_ a: Int, _ b: Int,
```

WHO	The compiler, reading a signature split across two lines.
WHAT	**Starts declaring report, whose first two parameters are unlabeled Ints.**
WHERE	At the top level of the program, below add and multiply, so its two calls at the bottom can reach it.
WHEN	At compile time; report runs twice later, once per call at the bottom.
WHY	Long signatures are wrapped and aligned so each parameter stays readable.

```
10  using operation: (Int, Int) -> Int) {
```

WHO	The compiler, finishing the parameter list of report.
WHAT	**Adds a third parameter, operation, whose type is a function that takes two Ints and returns an Int.**
WHERE	Second line of the signature; callers write using: before the function they pass.
WHEN	Compile time; the actual function is chosen by each caller at run time.
WHY	Taking a function as a parameter lets report work with any calculation, which makes it a higher-order function.

```
11  func describe(_ value: Int) -> String {
```

WHO	report, which owns this nested helper.
WHAT	**Declares describe, a function that only the body of report can see and call.**
WHERE	Nested inside report; code outside report cannot reach it.
WHEN	It is compiled once along with report, but it can only be called while a call to report is running.
WHY	A nested function keeps a small helper next to its only user and out of the rest of the program.

```
12  let kind = value.isMultiple(of: 2) ? "even" : "odd"
```

WHO	The body of describe, the nested helper.
WHAT	**Chooses the word even or odd for the value it was given.**
WHERE	A local constant inside describe, which is itself inside report.
WHEN	Each time describe is called, which here is once per report call, with value 20 and then 9.
WHY	isMultiple(of:) states the intent more plainly than checking whether value % 2 equals 0.

```
13  return "\(value) is \(kind)"
```

WHO	describe, handing its result back to report.
WHAT	**Returns text such as 20 is even.**
WHERE	The last line of describe; return is required because the body has two statements.
WHEN	Once per call to describe, right after kind is chosen, so it returns 20 is even and later 9 is odd.
WHY	Implicit return only applies when the whole body is a single expression.

```
15  print(describe(operation(a, b)))
```

WHO	report, calling whichever function it was handed.
WHAT	**Calls operation with a and b, describes the result, and prints it.**
WHERE	Inside report, after the nested declaration.
WHEN	Each time report is called: operation runs first, then describe, then print, innermost call first.
WHY	report never knows whether it is adding or multiplying; the caller decides, which separates what to compute from how to present it.

```
18 let combine: (Int, Int) -> Int = add
```

WHO Top-level code, storing a function in a constant.
WHAT **Puts the add function itself, not a result, into the constant combine.**
WHERE A global constant with an explicit function type annotation.
WHEN When execution reaches this line; nothing is added yet.
WHY Writing add without parentheses refers to the function instead of calling it.

```
19 print(combine(2, 3))
```

WHO The program, calling add through the constant.
WHAT **Calls the stored function with 2 and 3 and prints 5.**
WHERE Top-level code, using the global constant combine.
WHEN First, before either report call, so 5 is the opening line of output.
WHY A stored function is called with parentheses, exactly like a function called by name.

```
20 report(4, 5, using: multiply)
```

WHO The program, passing multiply into report.
WHAT **report multiplies 4 and 5 and prints 20 is even.**
WHERE Top-level code; multiply is named, not called, so it has no parentheses.
WHEN Second, after combine has printed 5.
WHY Passing a function name is how you hand a behavior to another function.

```
21 report(4, 5, using: add)
```

WHO The same call with a different function.
WHAT **report adds 4 and 5 and prints 9 is odd.**
WHERE The last line of top-level code; add is named without parentheses, so the function itself is passed, not a result.
WHEN Last, after the multiply call, so 9 is odd becomes the third and final line of output.
WHY Same report, different behavior: exactly how sorted(by:) and SwiftUI buttons accept your code, starting in the next lesson.

> **Try it**
>
> 1. Write func subtract(_ a: Int, _ b: Int) -> Int and pass it to report.
> 2. Make combine hold multiply instead of add. What does print(combine(2, 3)) show now?

CODE · RUNS IN THE BROWSER

Recursion: a function that calls itself

A **recursive** function solves a problem by calling itself on a smaller version of the same problem. It needs a **base case**, a situation it answers directly without calling itself again. Without one, the calls never stop, and the program crashes once it runs out of stack memory.

```swift
func countdown(from number: Int) {
    if number == 0 {
        print("Liftoff")
        return
    }
    print(number)
    countdown(from: number - 1)
}

func factorial(of n: Int) -> Int {
    n <= 1 ? 1 : n * factorial(of: n - 1)
}

countdown(from: 3)
print(factorial(of: 5))
```

OUTPUT

```
3
2
1
Liftoff
120
```

Line by line

1 `func countdown(from number: Int) {`

WHO	The compiler, recording a function whose full name is countdown(from:) and which returns nothing.
WHAT	**Declares a function that prints a countdown starting at number and returns nothing.**
WHERE	At the top level of the program, so the bottom line can call it and its own body can call it too.
WHEN	Checked at compile time; the body runs once per call, including the calls it makes to itself.
WHY	The label from makes the call countdown(from: 3) read naturally.

2 `if number == 0 {`

WHO	countdown, checking for its base case.
WHAT	**Tests whether the countdown has reached zero.**
WHERE	The first statement of the body, before any recursive call.
WHEN	At the start of every call, including each recursive one.
WHY	Checking the stopping condition first guarantees the recursion ends.

3 `print("Liftoff")`

WHO	The base case, the branch that stops the recursion.
WHAT	**Prints Liftoff when number is 0.**
WHERE	Inside the if block, so it runs only when number is 0.
WHEN	Only in the deepest call, after 3, 2 and 1 have printed.
WHY	The base case does real work too: it finishes the job.

4 `return`

WHO	The base case, ending this call.
WHAT	**Leaves the function immediately, skipping the lines below.**
WHERE	Inside the if block, right after the print.
WHEN	Only in the call where number is 0, immediately after Liftoff prints, which ends the deepest call first.
WHY	Without this return, the function would go on to call countdown(from: -1) and never stop.

```
6  print(number)
```

WHO The recursive case, the path taken whenever number is not 0.
WHAT **Prints the current number, such as 3.**
WHERE In the body, after the base-case check.
WHEN In every call where number is not 0.
WHY Printing before the recursive call makes the numbers appear in descending order.

```
7  countdown(from: number - 1)
```

WHO countdown itself: the function calls its own name, which is what makes it recursive.
WHAT **Starts a new call with a number one smaller.**
WHERE The last line of the body; each call gets its own frame on the call stack holding its own number.
WHEN After the current number has printed.
WHY Every call moves one step closer to the base case, which is what makes the recursion finish.

```
10 func factorial(of n: Int) -> Int {
```

WHO The compiler, recording factorial(of:), a function that takes one Int and returns an Int.
WHAT **Declares a function that returns the product of every whole number from 1 up to n.**
WHERE At the top level, below countdown, so the print on the last line can call it.
WHEN Checked at compile time; it runs when the last line calls it.
WHY Factorial is defined in terms of itself: n factorial equals n times the factorial of n minus 1.

```
11 n <= 1 ? 1 : n * factorial(of: n - 1)
```

WHO The single-expression body of factorial, which Swift returns implicitly because it is the only statement.
WHAT **Returns 1 when n is at most 1, and otherwise n times the factorial of n minus 1.**
WHERE The implicitly returned expression inside factorial.
WHEN For 5 it calls itself with 4, 3, 2 and 1, then the multiplications finish on the way back out.
WHY The ternary holds the base case and the recursive case on one line, with the base case checked first.

```
14 countdown(from: 3)
```

WHO Top-level code, making the first call.
WHAT **Prints 3, 2, 1 and Liftoff on four lines.**
WHERE Top-level code, after both function declarations.
WHEN The first output of the program.
WHY Four calls happen in total: from 3, 2 and 1, and finally from 0, the base case.

```
15 print(factorial(of: 5))
```

WHO Top-level code, calling factorial and handing the returned Int to print.
WHAT **Prints 120, which is 5 times 4 times 3 times 2 times 1.**
WHERE The last line of top-level code; the Int 120 is a temporary value passed directly to print.
WHEN After countdown has finished printing Liftoff, so 120 is the fifth and final line of output.
WHY Recursion fits problems that shrink naturally; for very long inputs a loop is usually safer, because every call uses stack memory.

> **Try it**
>
> 1. Move print(number) below the recursive call. In what order do the numbers print now, and why?
> 2. Print factorial(of: 10).

COMPARE

Naming that reads like English

Both programs compute the same things and print the same output. Only the names differ. Read each call at the bottom and ask: could a teammate tell what it does without opening the function?

A: Hard to read

```swift
func calc(a: Int, b: Int) -> Int {
    abs(b - a)
}

func proc(_ x: [Int], _ y: Bool) -> [Int] {
    y ? x.sorted() : x.sorted().reversed()
}

print(calc(a: 3, b: 10))
print(proc([42, 7, 19], false))
```

OUTPUT

```
7
[42, 19, 7]
```

1 `func calc(a: Int, b: Int) -> Int {`

- **WHO** The compiler, which accepts this name without complaint.
- **WHAT** **Declares a function that returns the distance between two numbers.**
- **WHERE** At the top level of the left program, so the print calls at the bottom can use it.
- **WHEN** Compiled like any other function; the compiler never judges names.
- **WHY** calc says nothing about what is calculated, and a and b say nothing about their roles, so every caller must open the function to understand it.

2 `abs(b - a)`

- **WHO** The body of calc, its single expression.
- **WHAT** **Subtracts a from b and returns the absolute value, so the result is never negative.**
- **WHERE** Inside calc, where b and a are its parameters.
- **WHEN** Each time calc is called; here only once, with a set to 3 and b set to 10.
- **WHY** The logic is fine; the problem is entirely in the naming.

5 `func proc(_ x: [Int], _ y: Bool) -> [Int] {`

- **WHO** The compiler, recording a function whose full name is proc(_:_:), two unlabeled parameters in a row.
- **WHAT** **Declares proc, which returns the array sorted up or down depending on a Bool.**
- **WHERE** At the top level of the left program, right below calc, sharing the same global scope.
- **WHEN** Type-checked at compile time; its body runs only when the second print line calls it.
- **WHY** An unlabeled Bool is a classic readability trap: the call ends with a bare true or false whose meaning is hidden.

6 `y ? x.sorted() : x.sorted().reversed()`

- **WHO** The body of proc, a single ternary expression.
- **WHAT** **Returns the values sorted lowest first when y is true, and highest first when it is false.**
- **WHERE** The only expression in the body of proc, so Swift returns its array implicitly.
- **WHEN** Each time proc is called; here once, with y false, so the reversed branch runs.
- **WHY** x and y force the reader to remember which is the list and which is the direction.

9 `print(calc(a: 3, b: 10))`

- **WHO** Top-level code calling calc, then print writing the returned Int to the console.
- **WHAT** **Prints 7, the gap between 3 and 10.**
- **WHERE** Top-level code on the left side, after both function declarations have been compiled.
- **WHEN** First on this side, so 7 is the first line of the left program output.
- **WHY** Reading calc(a: 3, b: 10) tells you almost nothing about the 7 that comes out.

10 `print(proc([42, 7, 19], false))`

- WHO: Top-level code calling proc with an array literal and a bare false, then printing the result.
- WHAT: **Prints [42, 19, 7], the values from highest to lowest.**
- WHERE: The last line of the left program; the array literal is a temporary built just for this call.
- WHEN: Second, after 7 has printed, so the reversed array is the final line of the left program output.
- WHY: Nothing at this call says what false means.

B: Reads like a sentence

```swift
func distance(from start: Int, to end: Int) -> Int {
    abs(end - start)
}

func sortedScores(_ scores: [Int], ascending: Bool) -> [Int] {
    ascending ? scores.sorted() : scores.sorted().reversed()
}

print(distance(from: 3, to: 10))
print(sortedScores([42, 7, 19], ascending: false))
```

OUTPUT

```
7
[42, 19, 7]
```

1 `func distance(from start: Int, to end: Int) -> Int {`

- WHO: The compiler, recording distance(from:to:), a name built from the base name plus both argument labels.
- WHAT: **Declares the same calculation as calc, with descriptive names.**
- WHERE: Top level, in place of calc.
- WHEN: Type-checked at compile time like calc; its body runs only when the first print line calls it.
- WHY: The base name says what comes back, the labels from and to give each argument a role, and start and end read well inside the body.

2 `abs(end - start)`

- WHO: The body of distance, its single expression.
- WHAT: **Returns how far apart start and end are, never negative.**
- WHERE: The only expression in the body of distance, so Swift returns its Int value implicitly.
- WHEN: Each time distance is called; here once, with start 3 and end 10, giving 7.
- WHY: Good parameter names make the body explain itself too.

5 `func sortedScores(_ scores: [Int], ascending: Bool) -> [Int] {`

- WHO: The compiler, recording sortedScores(_:ascending:), whose full name shows one unlabeled and one labeled parameter.
- WHAT: **Declares sortedScores, which takes a list with no label and a labeled Bool.**
- WHERE: Top level, in place of proc.
- WHEN: Type-checked at compile time; its body runs only when the last print line calls it.
- WHY: The base name already explains the first argument, so it drops its label; the Bool keeps one so true or false carries meaning at the call.

```
6  ascending ? scores.sorted() : scores.sorted().reversed()
```

WHO	The body of sortedScores, a single ternary expression choosing between two sorted arrays.
WHAT	**Returns the scores lowest first when ascending is true, highest first otherwise.**
WHERE	The only expression inside sortedScores, so Swift returns the chosen array without a return keyword.
WHEN	Each time sortedScores is called; here once, with ascending false, so the reversed branch runs.
WHY	A Bool named like an adjective, ascending, turns the ternary into something close to a sentence.

```
9  print(distance(from: 3, to: 10))
```

WHO	Top-level code calling distance with labeled arguments, then print writing the returned Int.
WHAT	**Prints 7, the same result as calc.**
WHERE	Top-level code on the right side, after both well-named functions have been declared.
WHEN	First on this side, so 7 opens the right program output exactly as it did on the left.
WHY	The call reads as distance from 3 to 10, so nobody needs to open the function.

```
10 print(sortedScores([42, 7, 19], ascending: false))
```

WHO	Top-level code calling sortedScores with an array literal and a labeled Bool, then printing the result.
WHAT	**Prints [42, 19, 7], highest score first.**
WHERE	The last line of the right program; the array literal is a temporary built just for this call.
WHEN	Second, after 7 has printed, matching the final line of the left program output exactly.
WHY	ascending: false documents itself, which is exactly the clarity the API Design Guidelines ask for.

> **Takeaway**
>
> The compiler treats both versions the same; people do not. Name a function for what it returns or does, label arguments by their role, drop a label only when the base name already explains the first argument, and never leave a bare `Bool` at a call site. A good name is documentation the reader sees at every call.

CHECK YOURSELF

Check your understanding

1. What does this program print?

```
func pour(_ drink: String, cups: Int = 1) -> String {
    "\(drink) x\(cups)"
}
print(pour("tea"))
```

   A. tea x0
   B. Nothing: a compile error says cups is missing
   C. tea x1
   D. tea x

2. Given `func move(from start: Int, to end: Int)`, which call compiles?
   A. move(from: 1, to: 5)
   B. move(start: 1, end: 5)
   C. move(1, 5)
   D. move(to: 5, from: 1)

3. What does this print?

```
func double(_ value: inout Int) {
    value *= 2
}
var points = 5
double(&points)
double(&points)
print(points)
```

   A. 5
   B. 10
   C. Nothing: parameters are constants, so it does not compile
   D. 20

4. You are adding a method to a list type that deletes the item at a position. Which declaration best follows the Swift API Design Guidelines?
   A. func removeItemAtIndex(index: Int)
   B. func remove(at index: Int)
   C. func rm(_ i: Int)
   D. func remove(index: Int, now: Bool)

5. A recursive function has no base case. What happens when a program calls it?
   A. The compiler always refuses to build it
   B. It quietly returns 0
   C. It crashes when the call stack overflows
   D. Swift inserts a stopping condition automatically

Answers: Functions

1. **C. tea x1** `cups` has a default value of 1, so a call may leave it out and Swift fills in 1. It is not a compile error: defaults exist precisely so callers can skip common arguments. Swift never invents 0 for a missing Int.
2. **A. move(from: 1, to: 5)** Callers write the *argument labels*, `from` and `to`, in the declared order. `start` and `end` are parameter names used only inside the body. Leaving the labels out is a missing-argument-label error, and swapping their order is rejected too.
3. **D. 20** An `inout` parameter copies its new value back to the caller when each call ends: 5 becomes 10, then 20. The answer 5 describes an ordinary parameter, but then `value *= 2` would not compile at all, because ordinary parameters are constants.
4. **B. func remove(at index: Int)** `remove(at:)` reads as a phrase at the call site, `list.remove(at: 2)`, and matches the standard library. `removeItemAtIndex(index:)` repeats information the types already give, so its calls stutter: `removeItemAtIndex(index: 2)`. Abbreviations like `rm` and unexplained Bools hide meaning.
5. **C. It crashes when the call stack overflows** Each call adds a frame to the call stack, and with no base case the frames pile up until the stack overflows and the program crashes. The compiler can warn about the most obvious infinite recursion, but in general it cannot prove whether recursion ends, so it does not refuse to build. Swift never adds a base case for you.

Closures and Higher-Order Functions

CONCEPT

Closures: functions without a name

A **closure** is a block of code you can store in a constant, pass to a function, or return from one. Think of it as a function without a name, written inline between braces. Inside the braces you list the parameters and the return type, then the keyword `in`, then the body: `{ (a: Int) -> Int in a * 2 }`.

A function that takes or returns another function is a **higher-order function**. Swift arrays come with many of them. `sorted(by:)`, `map`, `filter` and `reduce` each take a closure that says what to do with every element, so you describe the rule and the library does the looping.

Closure expression	An unnamed function written inline: parameters, return type, the keyword in, then the body.
in	The keyword that separates the closure header (its parameters and return type) from its body.
Higher-order function	A function that accepts a function as a parameter or returns one, such as map or sorted(by:).
Capture	A closure remembers the variables around the place it was written, even after that place has finished running.

> **Closures are everywhere in iOS code**
>
> In SwiftUI, `Button("Save") { save() }` runs a closure when the user taps, and `.task { }` runs one when a view appears. Reading closures fluently is a daily iOS skill.

CODE · RUNS IN THE BROWSER

A closure, written out in full

This closure answers one question: is the first name longer than the second? It is written in its most complete form, with every type spelled out, so you can see each part. Then it is handed to `sorted(by:)`, which calls it as often as it needs to put the array in order.

```
let names = ["Mia", "Priya", "Benjamin", "Omar"]

let isLonger: (String, String) -> Bool = {
    (a: String, b: String) -> Bool in
    return a.count > b.count
}

print(isLonger("Priya", "Mia"))
print(isLonger("Omar", "Benjamin"))

let longestFirst = names.sorted(by: isLonger)
print(longestFirst)
print(names)
```

OUTPUT

```
true
false
["Benjamin", "Priya", "Omar", "Mia"]
["Mia", "Priya", "Benjamin", "Omar"]
```

Line by line

1 `let names = ["Mia", "Priya", "Benjamin", "Omar"]`

- WHO — Your program creates the array, and the compiler infers its type, an array of String, from the literal.
- WHAT — **Stores four names in a deliberately unsorted order, each with a different number of characters.**
- WHERE — Top-level code, so names is a global constant that every line below can read.
- WHEN — Once, as the very first statement the program runs.
- WHY — Distinct lengths make the sorted result unambiguous, so you can check the closure just by reading the output.

3 `let isLonger: (String, String) -> Bool = {`

- WHO — The compiler, which checks that the closure on the right matches the function type written on the left.
- WHAT — **Declares a constant whose type is a function from two Strings to a Bool, and opens the closure that becomes its value.**
- WHERE — At top level; the closure is stored in the constant isLonger like any other value.
- WHEN — The closure is created here, but its body does not run yet. It runs only when something calls isLonger.
- WHY — Storing a closure in a constant shows that functions are values in Swift: you can name them, pass them and call them later.

4 `(a: String, b: String) -> Bool in`

- WHO — The closure itself, declaring what it accepts and what it hands back.
- WHAT — **Names two parameters, a and b, both of type String, states that the result is a Bool, and ends the header with in.**
- WHERE — Inside the closure braces, before in. The names a and b exist only inside this closure.
- WHEN — Checked at compile time; a and b receive fresh values every time the closure is called.
- WHY — This is the most explicit form. The ladder in the next step shows which of these parts Swift can infer for you.

5 `return a.count > b.count`

- WHO — The closure body, running with whichever two names the caller passes in.
- WHAT — **Compares the character counts of the two names and returns true when the first name has more characters.**
- WHERE — The body of the closure, after in, where a and b are in scope.
- WHEN — Each time the closure is called: twice by the print lines below and several more times inside sorted(by:).
- WHY — sorted(by:) wants a rule that answers whether the first item belongs before the second; true for longer means the longest names come first.

8 `print(isLonger("Priya", "Mia"))`

- WHO — Your code, calling the stored closure through its constant, exactly as you would call a function.
- WHAT — **Passes Priya and Mia and prints the answer, true, because 5 characters is more than 3.**
- WHERE — Top-level code. The arguments have no labels because calls to a closure never use argument labels.
- WHEN — Right after the closure exists, as the next statement in order.
- WHY — Calling the rule directly proves what it does before you trust another function to use it.

```
9  print(isLonger("Omar", "Benjamin"))
```

WHO The same closure, reused with different input.
WHAT **Prints false, because Omar has 4 characters and Benjamin has 8.**
WHERE Top-level code, just after the first call.
WHEN The second call; the body runs fresh, with new values for a and b.
WHY Checking both outcomes, true and false, is a quick habit that catches a comparison written backwards.

```
11 let longestFirst = names.sorted(by: isLonger)
```

WHO The Array method sorted(by:), a higher-order function from the standard library that calls your closure to compare pairs.
WHAT **Builds a new array with the names ordered from longest to shortest, using isLonger for every comparison.**
WHERE At top level; the result is a separate array stored in longestFirst.
WHEN Now. During this single line, sorted(by:) calls the closure as many times as its algorithm needs.
WHY Passing the rule as a value lets one sorting algorithm serve every ordering you can describe.

```
12 print(longestFirst)
```

WHO print, formatting the array the way Swift always prints arrays.
WHAT **Prints Benjamin, Priya, Omar and Mia inside square brackets, each name in quotes.**
WHERE Top-level code, reading the constant created on the line above.
WHEN Third line of output, once sorted(by:) has finished every comparison and handed back the new array.
WHY Seeing the order confirms the closure answered each comparison the way you intended.

```
13 print(names)
```

WHO print again, this time given the original array.
WHAT **Prints the original order, unchanged, showing that sorted(by:) did not touch names.**
WHERE Top-level code, reading the global constant from line 1.
WHEN Fourth and final line of output, after both the sorted copy and the original array exist side by side.
WHY sorted(by:) returns a new array. Its in-place sibling, sort(by:), needs a var; knowing which one you call prevents surprises.

Try it

1. Change the greater-than sign in the closure to a less-than sign and run it. Which name comes first now?
2. Add a third print line that calls isLonger with two names of your own.

INTERACTIVE

The closure syntax ladder

Climb down the ladder with the slider or the Previous and Next buttons. Each rung removes something from the closure: the types, the word `return`, the parameter names, the parentheses. Watch what disappears, read why the shorter form is still legal, and notice that the output never changes. Then switch to the `map` ladder and see the same rules at work.

This step is an interactive simulation in the HTML version.

CODE · RUNS IN THE BROWSER

map and filter

`map` transforms every element and returns a new array of the results. `filter` keeps only the elements for which the closure returns `true`. Neither one changes the original array. Here they add tax to drink

prices and pick names from a coffee shop menu.

```swift
import Foundation

let prices = [3.5, 4.25, 2.0]
let labels = prices.map { price in
    String(format: "$%.2f", price * 1.08)
}
print(labels)

let menu = ["Latte", "Mocha", "Tea", "Lemonade", "Matcha"]
let quick = menu.filter { item in item.count <= 5 }
print(quick)

let mDrinks = menu.filter { $0.hasPrefix("M") }
print(mDrinks)
print(menu)
```

OUTPUT

```
["$3.78", "$4.59", "$2.16"]
["Latte", "Mocha", "Tea"]
["Mocha", "Matcha"]
["Latte", "Mocha", "Tea", "Lemonade", "Matcha"]
```

Line by line

1 `import Foundation`

- **WHO** The compiler, which loads Apple's Foundation framework into this program.
- **WHAT** **Makes String(format:) available; the Swift standard library on its own does not include it.**
- **WHERE** The first line of the file; an import applies to the whole file.
- **WHEN** At compile time, before any statement runs.
- **WHY** Without this line the formatting call below would not compile, because String(format:) lives in Foundation.

3 `let prices = [3.5, 4.25, 2.0]`

- **WHO** The compiler, which infers an array of Double because the literals have decimal points.
- **WHAT** **Stores three drink prices in dollars.**
- **WHERE** Top-level code; prices is a global constant.
- **WHEN** Once, at the start of the program.
- **WHY** Writing 2.0 instead of 2 makes the intent obvious: every element is a Double, so the array has one consistent type.

4 `let labels = prices.map { price in`

- **WHO** The Array method map, which calls the closure once per element and collects what each call returns.
- **WHAT** **Opens a trailing closure that receives each price under the name price; the collected results become the new array labels.**
- **WHERE** At top level. The parameter price exists only inside these braces.
- **WHEN** map runs immediately and calls the closure three times, once per price, in order.
- **WHY** Trailing closure syntax drops the parentheses, so the call reads like a sentence: map each price to a label.

```
5  String(format: "$%.2f", price * 1.08)
```

WHO Foundation, which builds a string from a format pattern and a number.
WHAT **Adds 8 percent tax and formats the result with a dollar sign and exactly two decimals, such as $3.78.**
WHERE The only line in the closure body, so its value is returned automatically, without the word return.
WHEN Once per element, each time map calls the closure.
WHY map can change the element type: Doubles go in and Strings come out, so labels is an array of String.

```
7  print(labels)
```

WHO print, formatting the new array of strings.
WHAT **Prints the three taxed, formatted prices in their original order.**
WHERE Top-level code, after map has returned.
WHEN Once all three closure calls are done.
WHY map always returns exactly one result per element, in the same order, so labels lines up with prices.

```
9  let menu = ["Latte", "Mocha", "Tea", "Lemonade", "Matcha"]
```

WHO Your program creates a second array, and the compiler infers an array of String.
WHAT **Lists five drinks whose names differ in length and in their first letter.**
WHERE Top-level code; menu is a global constant.
WHEN After the price example finishes, in program order.
WHY Varied names give the two filters below something to keep and something to drop.

```
10 let quick = menu.filter { item in item.count <= 5 }
```

WHO The Array method filter, which asks the closure a yes-or-no question about each element.
WHAT **Keeps the drinks whose names have five characters or fewer: Latte, Mocha and Tea.**
WHERE At top level, with the whole trailing closure written on one line.
WHEN filter runs now and calls the closure five times.
WHY filter requires a closure that returns a Bool, and a comparison is exactly that, so a one-line body is enough.

```
11 print(quick)
```

WHO The print function, receiving the new array of short drink names that filter built from menu.
WHAT **Prints the three short names in the order they appear in menu.**
WHERE Top-level code, right after the filter.
WHEN Second line of output, right after filter has run the closure once for each of the five drinks.
WHY filter preserves order, so its result is a predictable subset of the original.

```
13 let mDrinks = menu.filter { $0.hasPrefix("M") }
```

WHO filter again, this time given a closure in the shortest shorthand form.
WHAT **Keeps only the drinks whose names start with a capital M: Mocha and Matcha.**
WHERE At top level; $0 stands for the closure parameter, the current menu item.
WHEN Runs the closure once per menu item, right now.
WHY For a short, obvious rule, $0 reads cleanly; for longer logic, a named parameter such as item is clearer.

```
14 print(mDrinks)
```

WHO The print function, receiving the array that the second filter, the one using $0, produced.
WHAT **Prints Mocha and Matcha; hasPrefix is case sensitive, so a lowercase m would not have matched.**
WHERE Top-level code, reading the global constant mDrinks; menu itself is untouched by this line.
WHEN Third line of output, after hasPrefix has been checked for every drink in menu.
WHY Printing each intermediate result is the easiest way to confirm a closure does what you think.

```
15 print(menu)
```

WHO print, applied to the original array.
WHAT **Prints all five drinks, proving that neither filter removed anything from menu.**
WHERE Top-level code, reading the constant declared earlier.
WHEN Last line of output, after both filters have run, so any change to menu would show up here.
WHY map and filter return new arrays and leave the source alone, which is also why they work on let constants.

> **Try it**
>
> 1. Change the tax rate from 1.08 to 1.10. Predict the first label, then run it.
> 2. Write a filter that keeps only drinks whose names contain the letter "e".
> 3. Use map to turn menu into an array of name lengths.

CODE · RUNS IN THE BROWSER

reduce and compactMap

`reduce` combines every element into one value, such as a total. You give it a starting value and a closure that merges the running result with the next element. `compactMap` transforms each element like `map` but throws away every `nil`, which makes it ideal for parsing text that might not be a number.

```
let steps = [4200, 8000, 6500, 10200]

let total = steps.reduce(0, +)
print(total)

let peak = steps.reduce(0) { most, day in max(most, day) }
print(peak)

let entries = ["3", "x", "5", "", "12"]
let parsed = entries.map { Int($0) }
print(parsed)

let numbers = entries.compactMap { Int($0) }
print(numbers)
```

OUTPUT

```
28900
10200
[Optional(3), nil, Optional(5), nil, Optional(12)]
[3, 5, 12]
```

Line by line

```
1 let steps = [4200, 8000, 6500, 10200]
```

WHO The compiler, which infers an array of Int from the whole-number literals.
WHAT **Stores four days of step counts from a fitness tracker.**
WHERE Top-level code; steps is a global constant.
WHEN First statement of the program; every reduce below reads this same array of four step counts.
WHY Whole numbers keep the sums exact, which makes the output easy to check by hand.

```
3  let total = steps.reduce(0, +)
```

WHO reduce, a standard-library method that walks the array once while carrying a running result.
WHAT **Starts at 0 and adds each day in turn: 0 plus 4200, then plus 8000, and so on, ending at 28900.**
WHERE At top level; the plus operator itself is passed in the closure position.
WHEN Runs immediately: four combining steps for four elements.
WHY The operator + is a function that takes two Ints and returns an Int, exactly the shape reduce needs, so no braces are required.

```
4  print(total)
```

WHO The print function, receiving the single Int that reduce(0, +) produced by adding every element.
WHAT **Prints 28900, the sum of all four days.**
WHERE Top-level code, reading the global constant total that reduce created on the line above.
WHEN After reduce has visited every element.
WHY reduce turns a whole array into a single value, so one number comes out.

```
6  let peak = steps.reduce(0) { most, day in max(most, day) }
```

WHO reduce again, this time with a trailing closure you write yourself.
WHAT **Keeps whichever is larger, the running result most or the current day, so the final value is the highest day: 10200.**
WHERE At top level; most and day exist only inside the closure braces.
WHEN The closure runs once per element, four times in all.
WHY Any rule that merges two values into one fits reduce. Starting at 0 is safe here because step counts are never negative.

```
7  print(peak)
```

WHO print, reporting the result of the custom reduce.
WHAT **Writes 10200 to the console, the largest step count, because the closure kept whichever value was bigger.**
WHERE Top-level code, reading the global constant peak computed by the custom reduce on line 6.
WHEN Second line of output, after the closure has compared the running maximum with each of the four days.
WHY Comparing this line with the total shows that the closure, not reduce, decides how values combine.

```
9  let entries = ["3", "x", "5", "", "12"]
```

WHO Your program, simulating text a person typed into a form.
WHAT **Creates five strings; only three of them look like whole numbers.**
WHERE Top-level constant, an array of String.
WHEN After both step-count results are printed; it starts the second half of the program about parsing text.
WHY Real input is messy, so the array includes a letter and an empty string to show what happens when parsing fails.

```
10 let parsed = entries.map { Int($0) }
```

WHO map, calling the Int initializer with each string.
WHAT **Converts each entry with Int, which returns an optional Int, so the result is an array of Int? holding three numbers and two nils.**
WHERE At top level; $0 is each entry in turn.
WHEN Five closure calls, one per entry, in order.
WHY This shows the problem compactMap solves: map keeps the nils because it always returns one result per element.

```
11 print(parsed)
```

WHO print, formatting an array of optionals.
WHAT **Prints Optional(3), nil, Optional(5), nil and Optional(12), which is how Swift shows optionals inside an array.**
WHERE Top-level code, reading parsed, a global array of type [Int?] that keeps one slot per entry.
WHEN Third line of output, right after map has tried Int($0) on each of the five strings.
WHY Optional wrappers in your output are a hint that you probably want compactMap or an unwrap instead.

```
13 let numbers = entries.compactMap { Int($0) }
```

WHO compactMap, a standard-library method that maps each element and then discards every nil result.
WHAT **Runs Int on each string and keeps only the successful conversions, producing a plain array of Int: 3, 5 and 12.**
WHERE At top level; numbers has the non-optional type array of Int.
WHEN Runs immediately, with five closure calls.
WHY One call replaces a map, a filter for nil and an unwrap, and the result never needs unwrapping again.

```
14 print(numbers)
```

WHO The print function, receiving the plain [Int] that compactMap built by keeping only successful conversions.
WHAT **Prints 3, 5 and 12 in square brackets, with no Optional wrappers and no nils.**
WHERE Top-level code, the last line of the program.
WHEN Final line of output, after compactMap has converted each entry and dropped the two that came back nil.
WHY Placing this output under the map output makes the difference between the two methods obvious.

> **Try it**
>
> 1. Change the starting value of the first reduce from 0 to 1000. What does the total become?
> 2. Add "7" and "seven" to entries. Which output lines change, and how?
> 3. Find the lowest day: use reduce with a starting value of Int.max and the min function.

CODE · RUNS IN THE BROWSER

Chaining and sorting tuples

Because `filter`, `sorted` and `map` each return a new array, you can chain them: the output of one becomes the input of the next. Each workout here is a **labeled tuple**, a small group of named values, so a closure can read `$0.minutes`. Read the chain from top to bottom like a recipe.

```swift
let workouts = [
    (name: "Run", minutes: 30),
    (name: "Yoga", minutes: 45),
    (name: "Swim", minutes: 20),
    (name: "Bike", minutes: 60),
]

let summary = workouts
    .filter { $0.minutes >= 30 }
    .sorted { $0.minutes > $1.minutes }
    .map { "\($0.name) \($0.minutes) min" }
    .joined(separator: ", ")
print(summary)

let total = workouts.map { $0.minutes }.reduce(0, +)
print("Total: \(total) min")
```

OUTPUT

```
Bike 60 min, Yoga 45 min, Run 30 min
Total: 155 min
```

Line by line

1 `let workouts = [`

- WHO — The compiler, which infers the element type from the tuples inside the brackets.
- WHAT — **Opens an array literal whose elements are labeled tuples with a name and a number of minutes.**
- WHERE — Top-level code; workouts becomes a global constant.
- WHEN — Built once, as the first statement runs; the filter, sort and map below all read this same array.
- WHY — Spreading a long literal over several lines, one element per line, keeps it readable and easy to edit.

2 `(name: "Run", minutes: 30),`

- WHO — Your data: the first element of the array.
- WHAT — **A tuple with two labeled parts, the String name and the Int minutes, describing a 30-minute run.**
- WHERE — Inside the array literal; the labels become part of the element type.
- WHEN — Evaluated as the array is built.
- WHY — Labels let code say $0.minutes instead of the cryptic $0.1, which documents what each value means.

3 `(name: "Yoga", minutes: 45),`

- WHO — Your data: the second labeled tuple in the array, describing one yoga workout.
- WHAT — **A 45-minute yoga session, using the same labels as the first tuple.**
- WHERE — Inside the array literal, between the run and the swim, so it is element index 1.
- WHEN — Created while the array literal is evaluated, before workouts holds any value.
- WHY — Every element must have the same labels and types, because an array holds a single element type.

4 `(name: "Swim", minutes: 20),`

- WHO — Your data: the third labeled tuple, the swim that the filter below is designed to drop.
- WHAT — **A 20-minute swim, the only workout shorter than half an hour.**
- WHERE — Inside the array literal as element index 2, still present in workouts even after filtering.
- WHEN — Created while the array literal is evaluated, in order, right after the yoga tuple.
- WHY — Including one short workout gives the filter below something to remove.

5 `(name: "Bike", minutes: 60),`

- WHO — Your data: the fourth and last labeled tuple, a bike ride that ends up first after sorting.
- WHAT — **A 60-minute bike ride, the longest workout, followed by a trailing comma.**
- WHERE — Inside the array literal, just before the closing bracket.
- WHEN — Created last while the array literal is evaluated; then workouts is complete and ready to use.
- WHY — Swift allows a trailing comma, so adding another workout later changes only one line.

8 `let summary = workouts`

- WHO — Your code, starting a chain of method calls.
- WHAT — **Names the result summary and begins with the workouts array as the first input.**
- WHERE — At top level. The chain continues on the following lines, each starting with a dot.
- WHEN — The whole chain runs as one statement, top to bottom, once.
- WHY — Putting each step on its own line makes a long chain readable and lets you comment out a single step while experimenting.

```
9   .filter { $0.minutes >= 30 }
```

WHO filter, receiving the full workouts array.
WHAT **Keeps only workouts lasting 30 minutes or more, dropping the swim.**
WHERE First link in the chain; $0 is one workout tuple at a time.
WHEN Runs first, calling the closure four times.
WHY Filtering early means later steps do less work and never see data you do not want.

```
10  .sorted { $0.minutes > $1.minutes }
```

WHO sorted(by:), written with a trailing closure that compares two workouts.
WHAT **Orders the remaining workouts so the one with more minutes comes first: Bike, Yoga, Run.**
WHERE Second link; $0 and $1 are the two workouts being compared.
WHEN Runs after filter, on the three workouts that survived it.
WHY A comparison closure takes two parameters, so $1 appears here for the first time; returning true puts $0 before $1.

```
11  .map { "\($0.name) \($0.minutes) min" }
```

WHO map, turning each tuple into display text.
WHAT **Builds a string such as Bike 60 min from each workout using string interpolation.**
WHERE Third link; the element type changes here from a tuple to a String.
WHEN Runs after sorting, once per remaining workout.
WHY Formatting comes last, after filtering and ordering, so the logic works on real data rather than on text.

```
12  .joined(separator: ", ")
```

WHO The array method joined(separator:), available because the elements are now Strings.
WHAT **Glues the three strings together with a comma and a space between them.**
WHERE Last link; it turns an array of String into one String.
WHEN Runs after map, finishing the chain.
WHY joined only exists for arrays of strings, which is why the map step had to come before it.

```
13  print(summary)
```

WHO print, showing the final product of the chain.
WHAT **Prints Bike 60 min, Yoga 45 min, Run 30 min on one line.**
WHERE Top-level code, reading summary, the single String that the four chained calls produced together.
WHEN After every link in the chain has run.
WHY One readable result from four small, single-purpose steps is the payoff of chaining.

```
15  let total = workouts.map { $0.minutes }.reduce(0, +)
```

WHO map and reduce working together on one line.
WHAT **Extracts the minutes from every workout, then adds them up: 30 plus 45 plus 20 plus 60 is 155.**
WHERE At top level; it starts again from the original workouts, including the swim.
WHEN After the summary line is printed; this second pipeline starts fresh from workouts rather than from summary.
WHY map pulls out the numbers so reduce can use the plus operator, a common two-step pattern.

```
16  print("Total: \(total) min")
```

WHO The print function, receiving a String built by interpolating the Int total into the text.
WHAT **Writes Total: 155 min to the console, the sum of all four workouts including the short swim.**
WHERE The final line of top-level code; after it prints, the program ends.
WHEN After the total has been computed.
WHY The total includes the swim, which shows that the earlier filter never changed the original array.

Try it

1. Change the sorted closure so the shortest workout comes first.
2. Change the filter threshold from 30 to 45 and predict the summary before running it.
3. Add a fifth workout, such as a 50-minute hike, and see where it lands.

CODE · RUNS IN THE BROWSER

Capturing values

A closure can use variables from the place where it was written, and it keeps them alive after that place has finished. This is called **capturing**. Here `makeCounter()` returns a closure that captured its own private `count`. Every counter you make gets a separate `count`.

```
func makeCounter() -> () -> Int {
    var count = 0
    return {
        count += 1
        return count
    }
}

let tapCounter = makeCounter()
let likeCounter = makeCounter()

print(tapCounter())
print(tapCounter())
print(tapCounter())
print(likeCounter())
```

OUTPUT

```
1
2
3
1
```

Line by line

1 `func makeCounter() -> () -> Int {`

- **WHO** The compiler, reading a function whose return type is itself a function type.
- **WHAT** **Declares makeCounter, which takes no arguments and returns a closure that takes nothing and returns an Int.**
- **WHERE** A top-level function declaration, available to the whole program.
- **WHEN** Declared at compile time; its body runs only when someone calls makeCounter().
- **WHY** Returning a function makes makeCounter a higher-order function: it manufactures new behavior on demand.

```
2  var count = 0
```

WHO — The body of makeCounter, which declares the state that the closure it returns will keep using.
WHAT — **Creates a variable named count that starts at zero.**
WHERE — A local variable inside makeCounter. Normally a local variable disappears when its function returns.
WHEN — On every call to makeCounter, so each call creates a brand-new count.
WHY — This is the state the returned closure will capture and keep alive after makeCounter ends.

```
3  return {
```

WHO — makeCounter, handing a closure back to its caller.
WHAT — **Returns a closure written right here; it has no header and no in, because it takes no parameters.**
WHERE — Inside makeCounter, where count is in scope, so the closure captures it.
WHEN — At the end of each call to makeCounter.
WHY — The declared return type already says the closure takes nothing and returns an Int, so Swift needs no header inside the braces.

```
4  count += 1
```

WHO — The returned closure, changing the variable it captured.
WHAT — **Adds one to the captured count, so the value goes 0 to 1 on the first call, 1 to 2 on the next.**
WHERE — Inside the closure; count refers to the captured variable itself, not to a copy.
WHEN — Every time the returned closure is called, not when makeCounter runs.
WHY — Because the closure holds on to the variable, each change survives until the next call.

```
5  return count
```

WHO — The returned closure, reporting its state.
WHAT — **Hands back the updated count. The body has two statements, so return must be written out.**
WHERE — The last line of the closure body.
WHEN — At the end of every call to the closure.
WHY — The implicit return only applies to single-expression bodies; with two statements Swift needs to be told which value to return.

```
9  let tapCounter = makeCounter()
```

WHO — Your top-level code, calling makeCounter once to get a brand-new closure with its own count.
WHAT — **Stores the returned closure in tapCounter. That closure owns its own count, currently 0.**
WHERE — A top-level constant whose value is a function.
WHEN — Now; this call creates a count and the closure captures it.
WHY — The name makes the closure read like a tool: call tapCounter() each time the user taps.

```
10 let likeCounter = makeCounter()
```

WHO — Your code, calling makeCounter a second time.
WHAT — **Produces a second, separate closure with its own separate count.**
WHERE — Another top-level constant holding a function value.
WHEN — Right after the first counter is made.
WHY — Two calls mean two independent captured variables, which is why closures make good little bundles of state.

```
12 print(tapCounter())
```

WHO — The tapCounter closure, called through its constant.
WHAT — **Raises its captured count from 0 to 1 and prints 1.**
WHERE — Top-level code; the empty parentheses call the closure with no arguments.
WHEN — The first call to tapCounter, so its captured count moves from 0 to 1.
WHY — Calling a stored closure uses the same parentheses as calling a function.

```
13 print(tapCounter())
```

WHO	The same tapCounter closure, called again.
WHAT	**Raises the same captured count to 2 and prints 2.**
WHERE	Top-level code; it calls the closure stored in the global constant tapCounter, the same one as the line above.
WHEN	The second call; the count from the first call is still there.
WHY	The output proves the captured variable outlived makeCounter and keeps its value between calls.

```
14 print(tapCounter())
```

WHO	The tapCounter closure again, still holding the same captured count it was given by makeCounter.
WHAT	**Raises the count to 3 and prints 3.**
WHERE	Top-level code, the third identical call; identical code gives a different result because the closure keeps state.
WHEN	The third call to tapCounter, after the count has already reached 2.
WHY	Each call continues from where the previous one stopped, like a tally that remembers.

```
15 print(likeCounter())
```

WHO	The likeCounter closure, called for the first time.
WHAT	**Raises its own count from 0 to 1 and prints 1.**
WHERE	The final line of top-level code, calling the closure stored in likeCounter rather than tapCounter.
WHEN	After three taps have been counted elsewhere.
WHY	The three tapCounter calls never touched this count: each closure captured a different variable.

> **Try it**
>
> 1. Add a fourth print(tapCounter()) at the end. What does it print, and why is likeCounter unaffected?
> 2. Change count += 1 to count += 10 and predict all four lines before running.
> 3. Give makeCounter a parameter step: Int and add step instead of 1 each time.

CONCEPT

Closures that outlive the call: @escaping

Every closure you have passed so far, to `sorted(by:)`, `map` or `filter`, was used up before the function returned. Such a parameter is **non-escaping**, the default in Swift: the function may call it during the call and nowhere else, and may not store it.

Some functions need to keep a closure for later: append it to an array, store it in a property, or hand it to a network request that replies seconds from now. Then the closure *escapes* the call, and Swift makes you say so by writing `@escaping` before its type. Forget it and the compiler refuses, naming the non-escaping parameter.

The mark matters because of capture. A non-escaping closure cannot outlive the call, so Swift never has to keep its captured variables alive afterward. An escaping closure may run at any later moment, so everything it captures stays alive, on the heap, as long as the closure does.

Non-escaping	The default. Called only during the call, never stored.
@escaping	Needed when a closure is stored or called after the function returns.
Explicit self	In a class, an escaping closure that uses a property must write self., admitting that it now keeps the object alive.
Retain cycle	If that object also stores the closure, each keeps the other alive forever. The Architecture, Testing and Shipping module breaks such loops with [weak self].

CODE · RUNS IN THE BROWSER

Storing closures and calling them later

`onTap` does not call the closure it receives. It stores it in an array, so the parameter must be `@escaping`. `simulateTap()` runs every stored handler later, twice, and both handlers share the captured variable `taps`. Completion handlers such as `completion: @escaping (Entry) -> Void` in the Device APIs module work the same way: the closure outlives the call that received it.

```
var handlers: [() -> Void] = []

func onTap(_ handler: @escaping () -> Void) {
    handlers.append(handler)
}

func simulateTap() {
    for handler in handlers {
        handler()
    }
}

var taps = 0
onTap { taps += 1 }
onTap { print("Tap count: \(taps)") }
print("Registered \(handlers.count) handlers")
simulateTap()
simulateTap()
```

OUTPUT

```
Registered 2 handlers
Tap count: 1
Tap count: 2
```

Line by line

1 `var handlers: [() -> Void] = []`

- WHO — Your program, declaring a global variable; the compiler reads the annotation as an array whose elements are closures.
- WHAT — **Creates an empty array that can hold closures taking no parameters and returning nothing.**
- WHERE — Top-level code, so handlers is a global that both onTap and simulateTap below can reach.
- WHEN — Once, as the first statement runs, before any handler exists.
- WHY — This array is the place the closures escape to. Without the annotation Swift could not infer an element type from an empty literal.

3 `func onTap(_ handler: @escaping () -> Void) {`

- WHO — The compiler, which reads @escaping as permission for this parameter to outlive the call.
- WHAT — **Declares onTap, which takes one closure with no argument label; @escaping says the closure may be kept after onTap returns.**
- WHERE — A top-level function; handler is its only parameter and exists only inside the body.
- WHEN — Declared at compile time; the body runs each time a handler is registered below.
- WHY — Without @escaping the next line fails to compile, because a non-escaping closure may not be placed in an array.

```
4  handlers.append(handler)
```

WHO The global handlers array, growing by one element.
WHAT **Stores the closure in the array instead of calling it.**
WHERE Inside onTap, the only statement in its body.
WHEN Immediately, on each call to onTap.
WHY This is the escape: the closure now lives on after onTap returns, so Swift keeps its captured variables alive as well.

```
7  func simulateTap() {
```

WHO Your program, declaring the function that stands in for a real tap on the screen.
WHAT **Declares a function with no parameters that will run every stored handler.**
WHERE A second top-level function, beside onTap, with access to the same global array.
WHEN Declared at compile time; it runs when the last two lines of the program call it.
WHY In a real app iOS calls this kind of function when the user touches a button; calling it yourself keeps the output predictable.

```
8  for handler in handlers {
```

WHO The for-in loop, reading the global array of stored closures.
WHAT **Walks the handlers array, binding each stored closure in turn to the name handler.**
WHERE Inside simulateTap; the loop variable handler is a closure, not a number or a string.
WHEN Each time simulateTap runs, visiting the handlers in registration order.
WHY The handlers were stored in order, so the loop runs them in that same order: counting first, printing second.

```
9  handler()
```

WHO The stored closure, called long after onTap returned.
WHAT **Runs one stored closure. The empty parentheses pass no arguments, matching the () -> Void type.**
WHERE Inside the loop body, once per element of handlers.
WHEN Twice per tap, because two handlers are registered; four times over the whole program.
WHY Calling a closure out of storage is exactly what @escaping promised the compiler would happen.

```
13 var taps = 0
```

WHO Your program, creating the variable that both closures below will capture.
WHAT **Declares a global Int that starts at zero.**
WHERE Top-level code, declared after the functions but before either handler is written.
WHEN Once, before any handler is registered.
WHY The closures capture the variable itself, not a copy, so one closure can change it and the other sees the change.

```
14 onTap { taps += 1 }
```

WHO onTap, receiving a trailing closure as its handler argument.
WHAT **Registers a closure that adds one to taps each time it runs.**
WHERE Top-level code; the closure is written here but stored as the first element of handlers.
WHEN Stored now; its body runs only when simulateTap calls it.
WHY A closure that does its work later is the normal shape of a button action in SwiftUI.

```
15 onTap { print("Tap count: \(taps)") }
```

WHO onTap again, storing a second closure in the same array.
WHAT **Registers a closure that prints the current value of taps.**
WHERE Top-level code; this closure becomes the second element of handlers.
WHEN Stored now; it runs after the counting closure on every simulated tap.
WHY It reads taps when it runs, not when it was registered, so it reports 1 and then 2: the capture is shared and live.

```
16 print("Registered \(handlers.count) handlers")
```

WHO The print function, reading the count of the global array.
WHAT **Prints Registered 2 handlers.**
WHERE Top-level code; this is the first line of output.
WHEN After both registrations and before any handler has run.
WHY Confirms that onTap stored the closures without running them: taps is still 0 at this point.

```
17 simulateTap()
```

WHO Your code, standing in for the first tap by the user.
WHAT **Runs both stored handlers in order: taps becomes 1, then Tap count: 1 prints.**
WHERE Top-level code, after the registrations.
WHEN The second line of output appears during this call.
WHY The closures run here, long after onTap returned, which is what @escaping made possible.

```
18 simulateTap()
```

WHO Your code again, standing in for a second tap.
WHAT **Runs both handlers once more: taps becomes 2 and Tap count: 2 prints.**
WHERE Top-level code, the last statement of the program.
WHEN Third line of output, immediately after the first simulated tap finishes.
WHY Stored closures can run any number of times, and each run sees the latest value of the captured variable.

> **Try it**
>
> 1. Register a third handler that prints "Third" and predict the output before you run it.
> 2. Delete @escaping from onTap and read the compiler error.
> 3. Swap the two onTap calls so the printing closure is registered first. What prints now?

CHECK YOURSELF

Check your understanding

1. What does this print?

```
let nums = [1, 2, 3, 4]
let result = nums.filter { $0 % 2 == 0 }.map { $0 * 10 }
print(result)
```

   A. [10, 20, 30, 40]
   B. [20, 40]
   C. [2, 4]
   D. 60

2. Given `let names = ["Zoe", "Ada", "Benjamin"]`, which line does **not** compile?

   A. names.map { $0.count }
   B. names.map(\.count)
   C. names.map { name in name.count }
   D. names.map { $1.count }

3. What does this print?

```
func makeAdder(_ amount: Int) -> (Int) -> Int {
    return { value in value + amount }
}
let addFive = makeAdder(5)
print(addFive(10))
```

   A. 15
   B. 5
   C. 10
   D. Nothing: it does not compile, because amount is gone once makeAdder returns

4. You have `let inputs = ["4", "four", "8"]` and need a plain array of `Int` holding only the valid numbers. Which call fits best?
   A. inputs.map { Int($0) }
   B. inputs.filter { Int($0) != nil }
   C. inputs.compactMap { Int($0) }
   D. inputs.reduce(0, +)

5. A teammate expects this to print `[9, 5, 3]`. What actually prints?

```
var scores = [3, 9, 5]
let ordered = scores.sorted { $0 > $1 }
print(scores)
```

   A. [9, 5, 3], because sorted reorders the array
   B. [3, 9, 5], because sorted returns a new array
   C. [3, 5, 9], because sorting is always ascending
   D. Nothing: sorted cannot be called on a var

Answers: Closures and Higher-Order Functions

1. **B. [20, 40]** `filter` keeps the even numbers, 2 and 4, and `map` then multiplies each by 10, giving `[20, 40]`. `[2, 4]` forgets that the chain continues after the filter, and `60` would need a `reduce` to add the values together.
2. **D. names.map { $1.count }** The closure for `map` receives exactly one parameter, so only `$0` exists. Using `$1` tells Swift the closure takes two parameters, and the compiler rejects it. The other three are rungs of the same ladder: a shorthand argument, a key path and a named parameter, all producing `[3, 3, 8]`.
3. **A. 15** The returned closure **captures** `amount`, so it still knows the value 5 after `makeAdder` has returned. Calling it with 10 gives 15. The last choice describes how local variables behave without capture; closures exist precisely to keep such values alive.
4. **C. inputs.compactMap { Int($0) }** `compactMap` converts each string and drops the `nil`, producing `[4, 8]` as plain Ints. `map` keeps the nil and returns optionals. The `filter` version removes "four" but still returns Strings, not Ints. `reduce(0, +)` does not compile, because you cannot add a String to an Int.
5. **B. [3, 9, 5], because sorted returns a new array** `sorted` returns a *new* array, stored here in `ordered`, and leaves `scores` exactly as it was. The fix is to print `ordered`, or to call `scores.sort { $0 > $1 }`, which reorders the var in place. The closure decides the order, so the result is not always ascending.

Worked Examples

WORKED EXAMPLE

A grade report with filter, map and reduce

Problem.
A class list mixes names and scores. Build a report that keeps only passing scores of 60 or more, turns each score into a letter grade, and finishes with the class average, computed exactly in tenths.

Frame 1 of 6

The raw class data comes first: five students with their scores, printed one per line. Nothing is filtered or changed yet, so this is the baseline every later frame is compared with.

```
let students = [
    (name: "Ada", score: 92), (name: "Ben", score: 74),
    (name: "Cleo", score: 58), (name: "Dan", score: 85),
    (name: "Eve", score: 67)
]

for student in students {
    print("\(student.name): \(student.score)")
}
```

OUTPUT SO FAR

```
Ada: 92
Ben: 74
Cleo: 58
Dan: 85
Eve: 67
```

Frame 2 of 6

filter keeps only scores of 60 or more, building a new passing array. The same print loop now walks passing, so Cleo, the one score below the line, simply never appears.

```
let students = [
    (name: "Ada", score: 92), (name: "Ben", score: 74),
    (name: "Cleo", score: 58), (name: "Dan", score: 85),
    (name: "Eve", score: 67)
]

+ let passing = students.filter { $0.score >= 60 }
+
+ for student in passing {
+     print("\(student.name): \(student.score)")
+ }
```

OUTPUT SO FAR

```
Ada: 92
Ben: 74
Dan: 85
Eve: 67
```

Frame 3 of 6

map turns each passing score into a letter, testing the bands from the top down: 90 or more is an A, then B, then C, and a passing score below 70 is a D.

```
   let students = [
       (name: "Ada", score: 92), (name: "Ben", score: 74),
       (name: "Cleo", score: 58), (name: "Dan", score: 85),
       (name: "Eve", score: 67)
   ]

   let passing = students.filter { $0.score >= 60 }

+ let letters = passing.map { student in
+     student.score >= 90 ? "A" :
+     student.score >= 80 ? "B" :
+     student.score >= 70 ? "C" : "D"
+ }
+
+ for (index, student) in passing.enumerated() {
+     print("\(student.name): \(letters[index])")
+ }
```

OUTPUT SO FAR

```
Ada: A
Ben: C
Dan: B
Eve: D
```

Frame 4 of 6

reduce folds every score in the class, passing or not, into one total of 376. Multiplying by ten before dividing gives the average in whole tenths, ready to print as 75.2.

```
   let students = [
       (name: "Ada", score: 92), (name: "Ben", score: 74),
       (name: "Cleo", score: 58), (name: "Dan", score: 85),
       (name: "Eve", score: 67)
   ]

+ let total = students.reduce(0) { sum, student in
+     sum + student.score
+ }
+ let averageTenths = total * 10 / students.count
+ print("Class average: \(averageTenths / 10).\(averageTenths % 10)")
```

OUTPUT SO FAR

```
Class average: 75.2
```

Frame 5 of 6

The report lines assemble name, score and letter for each passing student on one line. enumerated() supplies the index, so each student picks up the letter map stored at that same position.

```
    let students = [
        (name: "Ada", score: 92), (name: "Ben", score: 74),
        (name: "Cleo", score: 58), (name: "Dan", score: 85),
        (name: "Eve", score: 67)
    ]

+ let passing = students.filter { $0.score >= 60 }
+
+ let letters = passing.map { student in
+     student.score >= 90 ? "A" :
+     student.score >= 80 ? "B" :
+     student.score >= 70 ? "C" : "D"
+ }
+
+ for (index, student) in passing.enumerated() {
+     print("\(student.name): \(student.score) (\(letters[index]))")
+ }
```

OUTPUT SO FAR

```
Ada: 92 (A)
Ben: 74 (C)
Dan: 85 (B)
Eve: 67 (D)
```

Frame 6 of 6

The finished report puts the pieces together: filter chose the students, map made their letters, the loop prints one line each, and the reduce average closes the report as its last line.

```
    let students = [
        (name: "Ada", score: 92), (name: "Ben", score: 74),
        (name: "Cleo", score: 58), (name: "Dan", score: 85),
        (name: "Eve", score: 67)
    ]

    let passing = students.filter { $0.score >= 60 }

    let letters = passing.map { student in
        student.score >= 90 ? "A" :
        student.score >= 80 ? "B" :
        student.score >= 70 ? "C" : "D"
    }

+ let total = students.reduce(0) { sum, student in
+     sum + student.score
    }
+ let averageTenths = total * 10 / students.count
+ for (index, student) in passing.enumerated() {
+     print("\(student.name): \(student.score) (\(letters[index]))")
+ }
+ print("Class average: \(averageTenths / 10).\(averageTenths % 10)")
```

OUTPUT SO FAR

```
Ada: 92 (A)
Ben: 74 (C)
Dan: 85 (B)
Eve: 67 (D)
Class average: 75.2
```

FINAL RESULT — CHECKED OUTPUT

```
Ada: 92 (A)
Ben: 74 (C)
Dan: 85 (B)
Eve: 67 (D)
Class average: 75.2
```

Line by line: the finished program

1 `let students = [`

- **WHO** You declare the class data; the compiler infers the element type from the tuples.
- **WHAT** **Opens an array literal of labeled tuples, each holding a student name and a score.**
- **WHERE** Top-level code; students becomes a global constant for the whole program.
- **WHEN** Built once, as the first statement runs, before any step below reads it.
- **WHY** Fixed data keeps every frame predictable, so the printed output can be checked exactly.

2 `(name: "Ada", score: 92), (name: "Ben", score: 74),`

- **WHO** Your data: the first two labeled tuples in the array literal.
- **WHAT** **Ada with a score of 92 and Ben with a score of 74, written side by side on one line.**
- **WHERE** Inside the students array literal, as elements at index 0 and index 1.
- **WHEN** Created while the array literal is evaluated, before students holds its value.
- **WHY** Two short tuples share one line to keep the data block compact and easy to scan.

```
3  (name: "Cleo", score: 58), (name: "Dan", score: 85),
```

WHO Your data: the next two labeled tuples in the array literal.
WHAT **Cleo with a score of 58 and Dan with a score of 85, using the same labels as every other element.**
WHERE Inside the students array literal, as elements at index 2 and index 3.
WHEN Created while the array literal is evaluated, in order after Ada and Ben.
WHY Cleo is the one score below the passing line, which gives the filter below something to remove.

```
4  (name: "Eve", score: 67)
```

WHO Your data: the fifth and last labeled tuple in the array literal.
WHAT **Eve with a score of 67, the lowest passing score in the class data.**
WHERE The last element of the students array, at index 4, just before the closing bracket.
WHEN Created last while the array literal is evaluated; then students is complete.
WHY A score just above the passing line checks that the filter keeps borderline passes.

```
7  let passing = students.filter { $0.score >= 60 }
```

WHO The filter method, which asks the closure a yes-or-no question about each student tuple.
WHAT **Builds a new array passing holding only the tuples whose score is 60 or more; students itself is unchanged.**
WHERE Top-level code; $0 is one student tuple at a time inside the filter closure.
WHEN Runs once here, calling the closure five times, before the frames below read passing.
WHY Filtering first means every later step, grades and report alike, only ever sees passing students.

```
9  let letters = passing.map { student in
```

WHO The map method, which transforms each passing tuple into a new value of a different type.
WHAT **Opens a map over passing and names the closure parameter student, so the lines below can read each score.**
WHERE Top-level code; the result will be stored in the new constant letters.
WHEN Runs once here, calling the closure once per passing student.
WHY map produces exactly one letter per passing student, in the same order, so letters lines up with passing by index.

```
10  student.score >= 90 ? "A" :
```

WHO The ternary operator inside the map closure, testing the highest grade boundary first.
WHAT **Returns the letter A when this student score is 90 or more; otherwise the test continues on the next line.**
WHERE The first line of the map closure body, reading the score of the student parameter.
WHEN Evaluated once per passing student, before the lower boundaries below are tried.
WHY Testing from the top grade down means the first matching boundary is the right one, with no overlap.

```
11  student.score >= 80 ? "B" :
```

WHO The same ternary chain, reached only when the score was below 90.
WHAT **Returns the letter B when this student score is 80 or more; otherwise the test continues on the next line.**
WHERE The second line of the map closure body, continuing the expression started above.
WHEN Evaluated only for passing students whose score is under 90.
WHY Each line of the chain handles exactly one grade band, so the rule reads like the grade table it models.

```
12  student.score >= 70 ? "C" : "D"
```

WHO The last line of the ternary chain inside the map closure.
WHAT **Returns C when the score is 70 or more, and D otherwise; passing students cannot score below 60, so no F can appear.**
WHERE The final line of the map closure body; its value becomes this element of letters.
WHEN Evaluated only for passing students whose score is under 80.
WHY The final alternative needs no test, because every earlier band has already been ruled out.

```
15 let total = students.reduce(0) { sum, student in
```

WHO The reduce method, which walks the students array carrying one running value.

WHAT **Opens a reduce starting at 0; the closure names the running total sum and the tuple being visited student.**

WHERE Top-level code; the result will be stored in the new constant total.

WHEN Runs once here, calling the closure once per student in the class.

WHY The average needs the sum of every score in the class, passing or not, and reduce is the tool that folds an array into one value.

```
16 sum + student.score
```

WHO The reduce closure, combining the running total with the next score.

WHAT **Adds this student score to sum and hands the new total back as the running value for the next pass.**

WHERE The only line of the reduce closure body; sum and student exist only inside the closure.

WHEN Once per student, five times, so the running value ends as the full class total of 376.

WHY A single-expression closure returns its value automatically, so no return keyword is written.

```
18 let averageTenths = total * 10 / students.count
```

WHO Integer arithmetic, working in tenths so the average keeps one decimal place without fractions.

WHAT **Multiplies the total by 10 and divides by the number of students, giving the average in whole tenths: 752.**

WHERE Top-level code, reading the total that reduce produced and the count of the students array.

WHEN Runs once, after reduce has finished and before anything prints the average.

WHY Doing the math in tenths first is what lets the print below show 75.2 exactly, with no long decimal tail.

```
19 for (index, student) in passing.enumerated() {
```

WHO The enumerated() method, which pairs every passing tuple with its position in the array.

WHAT **Starts a loop binding index and student together, so the body can read the letter stored at the same index.**

WHERE Top-level code; index and student exist only inside these braces.

WHEN Runs once per passing student, four passes in all, in array order.

WHY letters and passing are separate arrays that line up by position, and enumerated() is how the loop visits both at once.

```
20 print("\(student.name): \(student.score) (\(letters[index]))")
```

WHO The print function, joining the name, score and letter for one passing student into a single report line.

WHAT **Prints one finished line such as Ada: 92 (A), reading the letter from letters at the same index.**

WHERE Inside the enumerated loop over passing, so it runs once per passing student.

WHEN Once per loop pass, after both letters and, in the final frame, the average have been computed.

WHY This is the finished report format: every fact about one student on one line, ready to read down the page.

```
22 print("Class average: \(averageTenths / 10).\(averageTenths % 10)")
```

WHO The print function, splitting the tenths value back into a whole part and one decimal digit.

WHAT **Prints Class average: 75.2 by dividing tenths by 10 for the whole part and taking the remainder for the digit.**

WHERE Top-level code, after the average has been computed in tenths.

WHEN In the frame that computes the average it is the only output; in the final frame it runs after the report loop.

WHY Splitting a scaled integer with division and remainder formats the decimal exactly, the same trick used for dollars and cents.

Try it

Three quiz scores are stored in an array. The starter adds them with reduce and prints the total. Finish the job: compute the class average in tenths, the same way the example does, and print it on one line as Average: 86.0.

```
let scores = [88, 91, 79]
let total = scores.reduce(0, +)
print("Total: \(total)")
```

EXPECTED OUTPUT

```
Average: 86.0
```

WORKED EXAMPLE

Optional input: the success path and the failure path

Problem.

Text typed into a field arrives as a String, and some of it is not a number. Parse each input safely: double the numbers, print a clear message for the rest, and never crash on bad text.

Frame 1 of 6

One good input first: the String 42 converts to the number 42, if let binds it to a plain Int, and the block prints the doubled value, 84. This is the success path.

```
    let text = "42"
    if let number = Int(text) {
        print("Doubled: \(number * 2)")
    } else {
        print("\(text) is not a number")
    }
```

OUTPUT SO FAR

```
Doubled: 84
```

Frame 2 of 6

Now the failure path, with exactly the same code and only the input changed. Int finds no number in abc, returns nil, and the else block prints a clear message instead of crashing.

```
+ let text = "abc"
    if let number = Int(text) {
        print("Doubled: \(number * 2)")
    } else {
        print("\(text) is not a number")
    }
```

OUTPUT SO FAR

```
abc is not a number
```

Frame 3 of 6

A second good input confirms the success path is a rule and not a fluke of 42: the String 7 converts, binds, and doubles to 14 through exactly the same lines of code.

```
+ let text = "7"
  if let number = Int(text) {
      print("Doubled: \(number * 2)")
  } else {
      print("\(text) is not a number")
  }
```

OUTPUT SO FAR

```
Doubled: 14
```

Frame 4 of 6

The parsing moves into a function named report, and a guard replaces the if let: bad input leaves early, good input flows straight down. One call with 42 prints the same doubled answer.

```
+ func report(_ text: String) {
+     guard let number = Int(text) else {
+         print("\(text) is not a number")
+         return
+     }
+     print("Doubled: \(number * 2)")
+ }
+
+ report("42")
```

OUTPUT SO FAR

```
Doubled: 84
```

Frame 5 of 6

A second call passes the bad input through the same function. The guard prints the not-a-number message and returns, so both paths now live in one place that any caller can reuse.

```
  func report(_ text: String) {
      guard let number = Int(text) else {
          print("\(text) is not a number")
          return
      }
      print("Doubled: \(number * 2)")
  }

  report("42")
+ report("abc")
```

OUTPUT SO FAR

```
Doubled: 84
abc is not a number
```

Frame 6 of 6

Finally a loop feeds all three canned inputs through report in order. The output tells the whole story in three lines: a doubled number, a handled failure, and another doubled number.

```
   func report(_ text: String) {
       guard let number = Int(text) else {
           print("\(text) is not a number")
           return
       }
       print("Doubled: \(number * 2)")
   }

+ let inputs = ["42", "abc", "7"]
+ for input in inputs {
+     report(input)
+ }
```

OUTPUT SO FAR

```
Doubled: 84
abc is not a number
Doubled: 14
```

FINAL RESULT — CHECKED OUTPUT

```
Doubled: 84
abc is not a number
Doubled: 14
```

Line by line: the finished program

1 `func report(_ text: String) {`

WHO	You declare the function; top-level code below calls it by name.
WHAT	**Declares report, which takes one String called text; the underscore lets callers pass it without a label.**
WHERE	At top level, so every call below and the loop at the end can use the same parsing code.
WHEN	Compiled ahead of time; its body runs only when a call reaches it.
WHY	Wrapping the parse in a function means the guard logic is written once and reused for every input.

2 `guard let number = Int(text) else {`

WHO	The guard statement, which enforces the requirement this function cannot work without.
WHAT	**Tries to convert text to an Int; on success number is a plain Int for the rest of the function, on failure the else block runs.**
WHERE	The first statement inside report; unlike if let, the unwrapped number stays in scope below the guard.
WHEN	On every call, before any other work in the function happens.
WHY	Checking the requirement first and leaving early keeps the success path on the left edge, unindented and easy to read.

```
3  print("\(text) is not a number")
```

WHO The print function, inside the guard else block, reading the text parameter.
WHAT **Prints the same clear message as before, naming the input that could not be converted.**
WHERE Inside the else block, which can still see text but has no number to read.
WHEN Only on calls whose text cannot be converted, such as the call passing abc.
WHY The failure message moved into the function with the rest of the parsing, so every caller gets it for free.

```
4  return
```

WHO The return statement, which the compiler requires in a guard else block.
WHAT **Leaves report immediately, so the success line below the guard never runs for bad input.**
WHERE The last statement of the guard else block inside report.
WHEN Right after the failure message prints, only for inputs that failed to convert.
WHY If the else block could fall through, the code below would use a number that does not exist, so Swift insists on an exit.

```
6  print("Doubled: \(number * 2)")
```

WHO The print function, reading number, the plain Int the guard unwrapped at the top of the function.
WHAT **Doubles the value inside the interpolation and prints it, for example Doubled: 84.**
WHERE The last statement of report, after the guard, on the left edge of the function body.
WHEN At the end of every call whose input converted successfully.
WHY This is the happy path, and it needs no further unwrapping because the guard already proved the value exists.

```
9  let inputs = ["42", "abc", "7"]
```

WHO You declare the canned data, standing in for three things a person typed, one after another.
WHAT **Stores an array of three Strings: 42, abc and 7, mixing good and bad input in a fixed order.**
WHERE Top-level code, after the report function, where the loop below can read it.
WHEN Runs once, before the loop starts; the values never change afterward.
WHY A fixed list makes the final output predictable: success, failure, success, in that order, every run.

```
10 for input in inputs {
```

WHO The for-in loop, which walks the inputs array from first element to last.
WHAT **Starts a loop binding each String in turn to the name input, so the body handles one typed value per pass.**
WHERE Top-level code; input exists only inside these braces.
WHEN Runs three times, once per element, in array order.
WHY The loop replaces one call per input, so the same code would handle ten inputs or a hundred unchanged.

```
11 report(input)
```

WHO The report function, called from inside the loop with the current input.
WHAT **Hands this pass input to report, which doubles it or prints the not-a-number message, depending on the value.**
WHERE The only statement of the loop body, inside report is where the output for this pass is printed.
WHEN Once per loop pass: three calls, producing the three output lines in order.
WHY The loop supplies the data and the function supplies the logic; neither repeats the other work.

Try it

The starter stores the typed text 21 and prints it back with a Tripled label, which only echoes the String. Fix it: convert the text with if let, multiply the unwrapped number by 3, and print Tripled: 63, with a not-a-number message in the else block.

```
let text = "21"
print("Tripled: \(text)")
```

EXPECTED OUTPUT

```
Tripled: 63
```

WORKED EXAMPLE

makeCounter: the closure that remembers

Problem.

Write a function that makes counters. Each counter starts at zero, adds one every time it is called, and remembers its own total between calls, without any other code being able to reach in and change it.

Frame 1 of 6

makeCounter is defined: it declares a local count and returns a closure written where count lives, so the closure captures that variable. Nothing has been counted yet; the ready line is the only output.

```
func makeCounter() -> () -> Int {
    var count = 0
    return {
        count += 1
        return count
    }
}

print("Counter ready")
```

OUTPUT SO FAR

```
Counter ready
```

Frame 2 of 6

Calling makeCounter once manufactures counter A: a closure whose private count starts at 0. Creating it counts nothing by itself, which the created line reports; the first real count comes next.

```
   func makeCounter() -> () -> Int {
       var count = 0
       return {
           count += 1
           return count
       }
   }

+ let counterA = makeCounter()
+ print("Counter A created")
```

OUTPUT SO FAR

```
Counter A created
```

Frame 3 of 6

The first call to counter A runs the captured closure: its count, kept alive after makeCounter returned, rises from 0 to 1, and the closure hands that 1 back to print.

```
   func makeCounter() -> () -> Int {
       var count = 0
       return {
           count += 1
           return count
       }
   }

   let counterA = makeCounter()
+ print(counterA())
```

OUTPUT SO FAR

```
1
```

Frame 4 of 6

The second call uses exactly the same line of code, yet prints 2: the closure found its captured count still holding 1 from the first call, added one, and remembered the new total.

```
   func makeCounter() -> () -> Int {
       var count = 0
       return {
           count += 1
           return count
       }
   }

   let counterA = makeCounter()
   print(counterA())
+ print(counterA())
```

OUTPUT SO FAR

```
1
2
```

Frame 5 of 6

A second call to makeCounter manufactures counter B with its own fresh count. B prints 1 on its first call, even though A already reached 2: the two captured variables are independent.

```
   func makeCounter() -> () -> Int {
       var count = 0
       return {
           count += 1
           return count
       }
   }

   let counterA = makeCounter()
   print(counterA())
   print(counterA())
+
+ let counterB = makeCounter()
+ print(counterB())
```

OUTPUT SO FAR

```
1
2
1
```

Frame 6 of 6

Calling A once more resumes its own count exactly where it left off, printing 3. The trip through B changed nothing for A, because each closure captured and guards its own private variable.

```
   func makeCounter() -> () -> Int {
       var count = 0
       return {
           count += 1
           return count
       }
   }

   let counterA = makeCounter()
   print(counterA())
   print(counterA())

   let counterB = makeCounter()
   print(counterB())
+ print(counterA())
```

OUTPUT SO FAR

```
1
2
1
3
```

FINAL RESULT — CHECKED OUTPUT

```
1
2
1
3
```

Line by line: the finished program

1 `func makeCounter() -> () -> Int {`

- WHO — You declare the function; its callers receive a closure from it.
- WHAT — **Declares makeCounter, which takes no arguments and returns a closure that takes nothing and returns an Int.**
- WHERE — At top level; the code inside runs only when makeCounter is called, not when it is declared.
- WHEN — Declared at compile time; the body runs once per counter created below.
- WHY — Returning a function makes makeCounter a higher-order function: it manufactures a fresh counter on every call.

2 `var count = 0`

- WHO — The body of makeCounter, declaring the state its returned closure will keep using.
- WHAT — **Creates a local variable count starting at 0, fresh on every call to makeCounter.**
- WHERE — A local variable inside makeCounter; normally it would disappear when the function returns.
- WHEN — On every call to makeCounter, before the closure below is created.
- WHY — This is the private state: nothing outside the returned closure can read or change this count.

3 `return {`

- WHO — makeCounter, handing a closure back to its caller.
- WHAT — **Returns a closure written right here; it has no header, because the return type already says it takes nothing and returns an Int.**
- WHERE — Inside makeCounter, where count is in scope, so the closure captures that very variable.
- WHEN — At the end of each call to makeCounter, as the function hands its result back.
- WHY — Writing the closure where count lives is what lets it capture count instead of receiving it as an argument.

4 `count += 1`

- WHO — The returned closure itself, changing the variable it captured.
- WHAT — **Adds one to the captured count, so the value moves 0 to 1 on the first call, 1 to 2 on the second, and so on.**
- WHERE — Inside the closure body; count refers to the captured variable itself, not to a copy of it.
- WHEN — Every time the returned closure is called, never while makeCounter itself is running.
- WHY — Because the closure holds on to the variable, each change survives until the next call to the same counter.

5 `return count`

- WHO — The returned closure, reporting its own state to whoever called it.
- WHAT — **Hands back the updated count; the body has two statements, so the return must be written out.**
- WHERE — The last line of the closure body, inside makeCounter.
- WHEN — At the end of every call to the closure, right after the count was increased.
- WHY — The implicit return only applies to single-expression bodies; with two statements Swift needs to be told which value to return.

```
9  let counterA = makeCounter()
```

WHO Top-level code, calling makeCounter once to manufacture the first counter.
WHAT **Stores the returned closure in the constant counterA; that closure owns its own captured count, currently 0.**
WHERE Top-level code, after the definition; counterA is a constant whose value is a function.
WHEN Now, at this statement; this call creates the count variable the closure captures.
WHY The name reads like the tool it is: calling counterA() is how this program counts one event.

```
10 print(counterA())
```

WHO The counterA closure, called through its constant for the first time.
WHAT **Raises its captured count from 0 to 1 and prints 1, the value the closure returns.**
WHERE Top-level code; the empty parentheses call the stored closure with no arguments.
WHEN The first call to counterA in this program, so its count moves from 0 to 1.
WHY Calling a stored closure uses the same parentheses as calling a function, because a closure is a function value.

```
11 print(counterA())
```

WHO The same counterA closure, called a second time.
WHAT **Raises the same captured count from 1 to 2 and prints 2.**
WHERE Top-level code, directly after the first call, using the same stored closure.
WHEN The second call to counterA; it starts from the count the first call left behind.
WHY The different answer from identical code is the proof that the captured variable lives between calls.

```
13 let counterB = makeCounter()
```

WHO Top-level code, calling makeCounter a second time to manufacture another counter.
WHAT **Stores a second, separate closure in counterB; this call created a brand-new count of its own, starting at 0.**
WHERE Top-level code, after counterA has already been called twice in this frame.
WHEN Now, between the counterA calls above and the counterB call below.
WHY Two calls to makeCounter mean two independent captured variables, which is what the next lines demonstrate.

```
14 print(counterB())
```

WHO The counterB closure, called through its constant for the first time.
WHAT **Raises its own captured count from 0 to 1 and prints 1, even though counterA already reached 2.**
WHERE Top-level code, after counterB was created; it calls a different closure from every counterA line.
WHEN The first call to counterB, made after counterA had already counted to 2.
WHY Starting again at 1 proves the two counters share nothing: each makeCounter call captured a different count.

```
15 print(counterA())
```

WHO The counterA closure again, still holding the count it built up earlier.
WHAT **Raises its captured count from 2 to 3 and prints 3, resuming exactly where counterA left off.**
WHERE The last line of top-level code, after the counterB call, calling the first closure once more.
WHEN The third call to counterA, made after counterB printed its own 1.
WHY The detour through counterB changed nothing for counterA: its private count waited, untouched, between calls.

Try it

The starter counts to 1 with a plain top-level variable and stops. Replace that approach with the makeCounter pattern: define the counter maker, store its closure in a constant named clicks, and call it twice so the program prints 1 and then 2.

```
var count = 0
count += 1
print(count)
```

EXPECTED OUTPUT

```
1
2
```

MODULE 03

Types, Protocols and Programming Paradigms

Model app data with structs, classes and enums, share behavior with protocols and generics, handle failure with Swift errors, keep apps responsive with async/await and actors, and choose the right programming paradigm for each job.

- Create various types of iOS applications in Swift

Structs: Your Own Value Types

CONCEPT

Why define your own types

So far you have used the types Swift provides: `Int`, `String`, `Array`. Real apps deal in *to-do items*, *coffee orders* and *weather readings*. A **type** is a blueprint: it names a kind of data, says what it holds and what it can do. Once you define a type, the compiler checks every use of it for you.

Swift has three main kinds of custom type: **structs**, **classes** and **enums**. This lesson starts with structs, the kind Apple uses most. `String`, `Array` and every SwiftUI view are structs.

Instance	One concrete value built from a type, such as one particular to-do item.
Property	A named piece of data stored in, or computed by, an instance.
Method	A function that belongs to a type and works with its properties.
Initializer	The code (init) that gives every stored property its first value.

> **Model the domain**
>
> Good apps start with good types. When your type names match the words users say (order, playlist, reading), the code reads like the problem it solves.

CODE · RUNS IN THE BROWSER

Defining a struct

A to-do item has a title, a done flag and an estimate in minutes. The struct below groups those three values into one new type. Swift writes a **memberwise initializer** for you: a ready-made `init` with one labeled parameter per stored property.

```swift
struct TodoItem {
    var title: String
    var isDone: Bool
    let minutes: Int
}

var chore = TodoItem(title: "Buy oat milk", isDone: false,
                     minutes: 15)
print(chore.title)
chore.isDone = true
print(chore.isDone)
print(chore)
```

OUTPUT

```
Buy oat milk
true
TodoItem(title: "Buy oat milk", isDone: true, minutes: 15)
```

Line by line

```
1  struct TodoItem {
```

WHO You, declaring a brand-new type; the compiler records its name and shape.

WHAT **Starts the definition of a struct named TodoItem. Everything up to the matching closing brace describes what a to-do item holds.**

WHERE At the top level of the file, so code anywhere in this program can create TodoItem values.

WHEN Compile time. Declaring a type runs no code; it teaches the compiler a new shape of data.

WHY Grouping related values into one named type lets you pass one to-do item around instead of three loose variables. Type names use UpperCamelCase.

```
2      var title: String
```

WHO Every TodoItem instance, which owns its own copy of this property.

WHAT **Declares a stored property named title that holds a String. var means it can change after the item is created.**

WHERE Inside each TodoItem value, stored right alongside the other two properties.

WHEN Receives its first value when an instance is initialized and can be reassigned after that.

WHY Users rename tasks, so the title must be mutable. Property names use lowerCamelCase.

```
3      var isDone: Bool
```

WHO Every TodoItem instance, each holding its own isDone flag.

WHAT **A stored Bool property that records whether the task is finished.**

WHERE Stored inside each TodoItem value.

WHEN Set at creation, then flipped when the user checks the task off.

WHY Bool names that read as assertions (isDone, hasOatMilk) make conditions such as if item.isDone read like plain English.

```
4      let minutes: Int
```

WHO Each TodoItem instance, which fixes this value for its whole life.

WHAT **A stored constant holding the estimated minutes. let means it can never change after initialization.**

WHERE Stored inside each TodoItem value.

WHEN Assigned exactly once, by the initializer.

WHY Using let wherever possible documents intent and lets the compiler reject accidental changes.

```
7  var chore = TodoItem(title: "Buy oat milk", isDone: false,
```

WHO The memberwise initializer that Swift synthesized for TodoItem.

WHAT **Creates a TodoItem from one labeled argument per stored property and stores it in a variable named chore.**

WHERE chore holds the whole struct value directly, not a pointer to an object somewhere else.

WHEN At run time, when execution reaches this statement.

WHY chore is a var because the program changes it later. With let, no property of the value could change, not even a var property.

```
8      minutes: 15)
```

WHO The same initializer call, continued.

WHAT **Supplies the last argument and closes the call. Labels must appear in the order the properties were declared.**

WHERE A continuation of the statement above, aligned under its first argument.

WHEN Part of the same call; the initializer runs once the closing parenthesis completes it.

WHY Breaking a long call across lines keeps each line readable without scrolling sideways.

```
9  print(chore.title)
```

WHO The standard library print function, writing to the console.
WHAT **Reads the title property with dot syntax and prints Buy oat milk.**
WHERE Top-level code; the output appears in the console.
WHEN Right after chore is created.
WHY Dot syntax (value.property) is how you reach inside any instance in Swift.

```
10 chore.isDone = true
```

WHO Your code, changing one property of the value stored in chore.
WHAT **Assigns true to isDone. The other two properties keep their values.**
WHERE Modifies the struct stored in the variable chore, in place.
WHEN Once, after the first print.
WHY This compiles only because chore is a var and isDone is a var. Changing a struct property needs both to be mutable.

```
11 print(chore.isDone)
```

WHO print, writing a Bool to the console.
WHAT **Prints the updated flag: true.**
WHERE Top-level code, writing to the console right after the assignment on the line above.
WHEN After the assignment, so it sees the new value.
WHY Printing after a change is the quickest way to confirm the change happened.

```
12 print(chore)
```

WHO print, using the default description Swift gives every struct.
WHAT **Prints the type name followed by every stored property with its label and value.**
WHERE Top-level code; the console shows the whole struct, property names included, because print uses Swift's default description.
WHEN Last line of the program.
WHY The automatic description is useful while debugging. Later in this lesson you replace it with your own text.

> **Try it**
>
> 1. Add chore.minutes = 20 at the end. Read the error: why does the compiler refuse?
> 2. Change var chore to let chore. Which line fails now, and why?

CODE · RUNS IN THE BROWSER

Computed properties and methods

Some values should not be stored because they can be worked out from others. A **computed property** runs code every time you read it. A **method** is a function that belongs to the type. Here the price is computed from the order, so it can never fall out of date.

```swift
import Foundation

struct CoffeeOrder {
    var drink: String
    var shots: Int
    var hasOatMilk: Bool

    var price: Double {
        let base = 3.50
        let extraShots = Double(shots - 1) * 0.75
        return base + extraShots + (hasOatMilk ? 0.60 : 0)
    }

    func receipt() -> String {
        let total = String(format: "%.2f", price)
        return "\(drink), \(shots) shots: $\(total)"
    }
}

let order = CoffeeOrder(drink: "Latte", shots: 2, hasOatMilk: true)
print(order.price)
print(order.receipt())
```

OUTPUT

```
4.85
Latte, 2 shots: $4.85
```

Line by line

1 `import Foundation`

WHO	The compiler, while it builds the program.
WHAT	**Loads Apple's Foundation framework, which supplies String(format:) used in receipt().**
WHERE	First line of the file; the import applies to this file only.
WHEN	Compile time, before any code runs.
WHY	String(format:) is a Foundation feature, not part of the core Swift standard library, so the file must import it.

3 `struct CoffeeOrder {`

WHO	You, defining a new value type.
WHAT	**Starts a struct that describes one coffee order.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler learns the shape of CoffeeOrder here and no code runs.
WHY	An order is plain data that gets copied around (cart, receipt, history), which is exactly what structs are for.

4 `var drink: String`

WHO	Every CoffeeOrder instance, each with its own drink name.
WHAT	**Stored property for the drink name, such as Latte.**
WHERE	Inside each CoffeeOrder value.
WHEN	Set when the order is created.
WHY	A var lets the customer change the drink before checkout.

```
5  var shots: Int
```

WHO Every CoffeeOrder instance, which stores its own shot count for the price getter to read.
WHAT **Stored property for the number of espresso shots.**
WHERE Inside each CoffeeOrder value.
WHEN Set at creation and changeable later.
WHY Storing the raw choice (2 shots), not the price it causes, keeps one source of truth.

```
6  var hasOatMilk: Bool
```

WHO Every CoffeeOrder instance; the price getter reads this flag to decide whether to add the oat-milk charge.
WHAT **Stored Bool for the oat milk option.**
WHERE Inside each CoffeeOrder value.
WHEN Set at creation and changeable later.
WHY Named as an assertion so code like order.hasOatMilk reads naturally.

```
8  var price: Double {
```

WHO The CoffeeOrder type, which provides this property to every instance.
WHAT **Declares a computed property: a getter body that calculates a Double instead of storing one.**
WHERE Belongs to the struct but takes up no storage inside the value.
WHEN The body runs every time some code reads order.price.
WHY A stored price could disagree with shots and hasOatMilk after an edit. A computed one is always correct.

```
9  let base = 3.50
```

WHO The getter, for the duration of one read.
WHAT **Creates a local constant holding the base price.**
WHERE A local inside the getter; it disappears when the getter returns.
WHEN Each time price is read.
WHY Naming the number explains what 3.50 means instead of leaving a mystery value in the formula.

```
10 let extraShots = Double(shots - 1) * 0.75
```

WHO The price getter, running inside the instance whose price was asked for.
WHAT **Charges 0.75 for every shot after the first. Double(...) converts the Int result to a Double.**
WHERE Local to the getter.
WHEN Each time price is read.
WHY Swift never mixes Int and Double silently. The explicit conversion makes you decide where precision changes.

```
11 return base + extraShots + (hasOatMilk ? 0.60 : 0)
```

WHO The getter, handing its result back to whoever read price.
WHAT **Adds the parts together. The ternary ?: adds 0.60 only when hasOatMilk is true.**
WHERE Last statement of the getter.
WHEN Each time price is read.
WHY The ternary keeps a two-way choice on one line. Use if/else when the choice grows bigger.

```
14 func receipt() -> String {
```

WHO The CoffeeOrder type, which gives every instance this method.
WHAT **Declares a method named receipt that takes no arguments and returns a String.**
WHERE Inside the struct, so its body can read drink, shots and price directly.
WHEN Runs only when called, as order.receipt().
WHY A method keeps formatting logic next to the data it formats.

```
15 let total = String(format: "%.2f", price)
```

WHO	Foundation's String(format:) initializer, called by the receipt() method.
WHAT	**Reads price (running the getter) and formats it with exactly two decimal places: 4.85.**
WHERE	Local constant inside receipt().
WHEN	Each time receipt() is called.
WHY	Money is shown with two decimals. The format specifier %.2f guarantees that.

```
16 return "\(drink), \(shots) shots: $\(total)"
```

WHO	String interpolation, building the final text.
WHAT	**Inserts drink, shots and total into the string. The $ is an ordinary character here.**
WHERE	Last statement of receipt().
WHEN	Each time receipt() is called.
WHY	Interpolation with \( ) is the idiomatic way to mix values into text in Swift.

```
20 let order = CoffeeOrder(drink: "Latte", shots: 2, hasOatMilk: true)
```

WHO	The memberwise initializer that Swift synthesized from the three stored properties.
WHAT	**Creates an order for a two-shot oat milk latte.**
WHERE	A top-level constant holding the struct value.
WHEN	At run time, when execution reaches this statement; the price getter has not run yet.
WHY	Notice there is no price argument: computed properties are not part of the memberwise initializer.

```
21 print(order.price)
```

WHO	print, which reads the computed property.
WHAT	**Runs the price getter and prints 4.85.**
WHERE	Top-level code; the console shows the Double that the price getter computed on demand.
WHEN	After the order exists.
WHY	Reading a computed property looks exactly like reading a stored one. Callers never need to know the difference.

```
22 print(order.receipt())
```

WHO	print, calling the method.
WHAT	**Calls receipt() and prints Latte, 2 shots: $4.85.**
WHERE	Top-level code; the console shows the String that receipt() returned.
WHEN	Last line of the program.
WHY	Methods use parentheses even with no arguments; properties do not.

> **Try it**
>
> 1. Set shots to 3 and hasOatMilk to false. Predict the price before you run it.
> 2. Add a computed property isLarge that returns true when shots is 3 or more, and print it.

CODE · RUNS IN THE BROWSER

Mutating methods

A struct method may not change the struct's own properties unless you mark it `mutating`. The keyword tells every reader, and the compiler, that calling the method changes the value. Notice the default value on `songs`: the memberwise initializer lets you leave that argument out.

```swift
struct Playlist {
    var name: String
    var songs: [String] = []

    mutating func add(_ song: String) {
        songs.append(song)
    }

    var summary: String {
        "\(name): \(songs.count) songs"
    }
}

var roadTrip = Playlist(name: "Road Trip")
roadTrip.add("Golden Hour")
roadTrip.add("Ocean Drive")
print(roadTrip.summary)
print(roadTrip.songs)
```

OUTPUT

```
Road Trip: 2 songs
["Golden Hour", "Ocean Drive"]
```

Line by line

1 `struct Playlist {`

WHO	You, defining a value type.
WHAT	**Starts a struct that models one playlist.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler records the Playlist shape and runs nothing.
WHY	A playlist is data the app copies, saves and shows, so a struct fits.

2 `var name: String`

WHO	Every Playlist instance, which stores its own name for summary to read.
WHAT	**Stored property for the playlist name.**
WHERE	Inside each Playlist value.
WHEN	Set by the initializer.
WHY	It has no default, so every caller must supply a name.

3 `var songs: [String] = []`

WHO	Every Playlist instance; each starts with its own empty songs array.
WHAT	**Stored array of song titles with a default value of an empty array.**
WHERE	Inside each Playlist value.
WHEN	Starts empty unless the caller passes songs to the initializer.
WHY	A default value makes the songs argument optional in the memberwise initializer, so Playlist(name:) works.

```
5  mutating func add(_ song: String) {
```

WHO The Playlist type, offering a method that changes the value it is called on.
WHAT **Declares a method that appends one song. The underscore means callers write add("x"), not add(song: "x").**
WHERE Inside the struct; self is the playlist value it was called on.
WHEN Each time someone calls add on a variable playlist.
WHY mutating is required because the body changes a stored property. It also means the method is unavailable on let constants.

```
6      songs.append(song)
```

WHO The songs array inside this playlist.
WHAT **Adds the new title to the end of the array.**
WHERE Changes the songs property of the playlist the method was called on.
WHEN During each add call.
WHY This line is the reason the method must be mutating: it modifies a stored property.

```
9  var summary: String {
```

WHO The Playlist type, declaring a read-only computed property.
WHAT **A computed property describing the playlist in one line.**
WHERE Inside the struct; takes no storage.
WHEN Each time summary is read.
WHY Read-only computed properties never need mutating because they only read.

```
10     "\(name): \(songs.count) songs"
```

WHO The summary getter, reading name and songs from the instance it was asked about.
WHAT **Builds text such as Road Trip: 2 songs. A single-expression body returns its value without writing return.**
WHERE Body of the summary getter.
WHEN Each time summary is read.
WHY Implicit return keeps short getters short.

```
14 var roadTrip = Playlist(name: "Road Trip")
```

WHO The memberwise initializer, using the default for songs.
WHAT **Creates an empty playlist named Road Trip in a variable.**
WHERE A top-level var holding the struct value.
WHEN At run time, when execution reaches this line; songs starts as the empty default.
WHY It must be var: the next lines call a mutating method on it.

```
15 roadTrip.add("Golden Hour")
```

WHO Your code, calling the mutating method.
WHAT **Appends Golden Hour to roadTrip.songs.**
WHERE Changes the value stored in roadTrip, in place.
WHEN Runs once, in order.
WHY Reads like a sentence: road trip, add Golden Hour.

```
16 roadTrip.add("Ocean Drive")
```

WHO The same mutating method.
WHAT **Appends a second song.**
WHERE Top-level code, calling the mutating method on roadTrip a second time.
WHEN After the first add.
WHY Each call builds on the value left by the previous one.

17 `print(roadTrip.summary)`

- WHO: print, reading the computed property.
- WHAT: **Prints Road Trip: 2 songs.**
- WHERE: Top-level code; the console shows the String that the summary getter built.
- WHEN: After both songs were added.
- WHY: The summary is always current because it is computed on demand.

18 `print(roadTrip.songs)`

- WHO: print, showing an array.
- WHAT: **Prints the array in Swift's array format, with quotes around each String.**
- WHERE: Top-level code; the console shows the songs array in Swift's array format.
- WHEN: The last statement of the program, after both add calls have appended their songs.
- WHY: Printing the raw array confirms the order in which songs were added.

> **Try it**
>
> 1. Add a mutating method clear() that empties the playlist, then call it and print summary.
> 2. Create the playlist with Playlist(name: "Focus", songs: ["Rain"]) instead. What changes?

CODE · RUNS IN THE BROWSER

When mutating meets let

This program does not compile, on purpose. `favorites` is declared with `let`, so the whole struct value is constant, and a `mutating` method may not run on it. Run it, read the diagnostic, then fix it.

```
struct Playlist {
    var songs: [String] = []

    mutating func add(_ song: String) {
        songs.append(song)
    }
}

let favorites = Playlist()
favorites.add("Golden Hour")
print(favorites.songs)
```

COMPILER ERROR (ON PURPOSE)

```
cannot use mutating member on immutable value
```

Line by line

1 `struct Playlist {`

- WHO: You, defining a value type.
- WHAT: **Starts a small playlist struct.**
- WHERE: Top level of the file.
- WHEN: Compile time; this is the same Playlist shape as before, reduced to the parts the error needs.
- WHY: The type itself is fine; the bug is in how it is used below.

```
2  var songs: [String] = []
```

WHO — Every Playlist instance, which owns its own songs array.
WHAT — **A stored array with an empty default.**
WHERE — Inside each Playlist value.
WHEN — Initialized to an empty array the moment a Playlist is created.
WHY — The default gives the struct a no-argument initializer, Playlist().

```
4  mutating func add(_ song: String) {
```

WHO — The Playlist type, declaring a method that is allowed to change the instance.
WHAT — **A method that changes the playlist, so it is marked mutating.**
WHERE — Inside the Playlist struct, below the songs property it modifies.
WHEN — Only when called on a variable playlist.
WHY — The compiler uses the mutating keyword to decide where this method may be called.

```
5      songs.append(song)
```

WHO — The songs array of the instance the method was called on.
WHAT — **Appends the new song title to the end of the songs array.**
WHERE — Changes the playlist the method was called on.
WHEN — During each call to add, if the call is ever allowed to compile.
WHY — Modifying a stored property is what makes the method mutating.

```
9  let favorites = Playlist()
```

WHO — Your code, creating a constant.
WHAT — **Stores an empty playlist in a let constant.**
WHERE — A top-level constant; favorites holds the whole struct value, locked by let.
WHEN — At run time, if the program compiled.
WHY — For a struct, let freezes the entire value, including every var property inside it.

```
10 favorites.add("Golden Hour")
```

WHO — The compiler, which rejects this line.
WHAT — **Tries to call a mutating method on a constant. This is the line the error points at.**
WHERE — Top-level code, calling the mutating method on a constant; this is the line the compiler rejects.
WHEN — Caught at compile time, so the program never runs.
WHY — Catching the mistake before the app ships is the whole point of let and mutating. Change let to var on the line above to fix it.

```
11 print(favorites.songs)
```

WHO — print, once the program compiles.
WHAT — **Would print the songs array once the program compiles.**
WHERE — Top-level code; this line never runs because the program does not compile.
WHEN — Never reached until the error is fixed.
WHY — After the fix it prints ["Golden Hour"].

> **Try it**
>
> 1. Change let favorites to var favorites and run it again.
> 2. Instead, delete the word mutating. Which new error appears, and why does it mention self?

CODE · RUNS IN THE BROWSER

Custom init, static members and didSet

Three more struct tools. A **custom initializer** lets callers pass Fahrenheit while the struct stores Celsius. A **static** property belongs to the type itself, not to any one reading. A **property observer** (`didSet`) runs

code every time a property is assigned, which is handy for logging or saving.

```
struct WeatherReading {
    static let freezingPoint = 0.0
    let city: String
    var celsius: Double {
        didSet {
            print("\(city): \(oldValue) -> \(celsius)")
        }
    }

    init(city: String, fahrenheit: Double) {
        self.city = city
        celsius = (fahrenheit - 32) * 5 / 9
    }

    var isFreezing: Bool {
        celsius <= WeatherReading.freezingPoint
    }
}

var denver = WeatherReading(city: "Denver", fahrenheit: 41)
denver.celsius = -2.5
print(denver.isFreezing)
```

OUTPUT

```
Denver: 5.0 -> -2.5
true
```

Line by line

1 `struct WeatherReading {`

- WHO — You, defining a value type.
- WHAT — **Starts a struct for one temperature reading in one city.**
- WHERE — Top level of the file.
- WHEN — Compile time; the compiler records the WeatherReading shape, including its observer and custom init.
- WHY — A reading is a small piece of data, a natural struct.

2 `static let freezingPoint = 0.0`

- WHO — The WeatherReading type itself, not any single instance.
- WHAT — **Declares a type-level constant, written WeatherReading.freezingPoint.**
- WHERE — Stored once for the whole type, shared by every reading.
- WHEN — Initialized the first time it is used.
- WHY — Constants that describe the type as a whole belong to the type, not copied into every instance.

3 `let city: String`

- WHO — Every WeatherReading instance, which keeps its own city name for the didSet message.
- WHAT — **Stored constant for the city name.**
- WHERE — Stored inside each WeatherReading value, next to celsius.
- WHEN — Assigned once, in init.
- WHY — A reading never moves to a different city, so let.

```
4  var celsius: Double {
```

WHO Every WeatherReading instance, which owns its own celsius value and the observer attached to it.
WHAT **A stored property with an observer block attached. It still stores a Double.**
WHERE Stored inside each WeatherReading value; the didSet block below belongs to this property.
WHEN Assigned in init and possibly changed later.
WHY Storing one unit (Celsius) and converting at the edges avoids mixing units inside the app.

```
5  didSet {
```

WHO Swift, which calls this block automatically.
WHAT **Opens a property observer that runs right after celsius receives a new value.**
WHERE Attached to the celsius property.
WHEN After every assignment to celsius, but not during the type's own initializer.
WHY Observers let you react to changes (log, save, refresh) without every caller remembering to do it.

```
6  print("\(city): \(oldValue) -> \(celsius)")
```

WHO The didSet block, run by Swift after a new celsius value has been stored.
WHAT **Prints the change. oldValue is provided automatically and holds the value before the assignment.**
WHERE Inside the didSet observer, where oldValue is available alongside the new celsius.
WHEN Once per assignment to celsius.
WHY Logging old and new values together is a classic debugging use of didSet.

```
10 init(city: String, fahrenheit: Double) {
```

WHO You, replacing the memberwise initializer with your own.
WHAT **Declares an initializer that takes Fahrenheit, the unit a US weather service might send.**
WHERE Inside the WeatherReading struct, after the stored properties it must fill in.
WHEN Each time someone writes WeatherReading(city:fahrenheit:).
WHY Writing a custom init in the struct body removes the free memberwise init, so all creation goes through your conversion.

```
11 self.city = city
```

WHO The custom initializer, writing to the stored property through self.
WHAT **Copies the city parameter into the city property. self. tells the two same-named things apart.**
WHERE Inside init, where self.city refers to the stored property and city to the parameter.
WHEN During initialization, before the new value is handed back to the caller.
WHY Every stored property must have a value before init finishes; the compiler checks this.

```
12 celsius = (fahrenheit - 32) * 5 / 9
```

WHO The custom initializer, converting the Fahrenheit parameter before storing it.
WHAT **Converts Fahrenheit to Celsius and stores it: 41 becomes 5.0.**
WHERE Inside init; the assignment goes straight to the stored property without running didSet.
WHEN During initialization, so didSet does not run and nothing is printed for it.
WHY Converting once at creation means the rest of the app only ever sees Celsius.

```
15 var isFreezing: Bool {
```

WHO The WeatherReading type, declaring a computed Bool property.
WHAT **A computed property that answers a yes-or-no question about the reading.**
WHERE Inside the struct; no storage.
WHEN Each time it is read.
WHY A named question (isFreezing) is clearer at the call site than repeating a comparison everywhere.

```
16 celsius <= WeatherReading.freezingPoint
```

WHO	The isFreezing getter, comparing the instance's celsius with the type-level constant.
WHAT	**Compares the reading with the static constant, reached through the type name.**
WHERE	Body of the getter.
WHEN	Each time isFreezing is read.
WHY	Static members are accessed on the type, not on an instance.

```
20 var denver = WeatherReading(city: "Denver", fahrenheit: 41)
```

WHO	Your custom init(city:fahrenheit:), the only initializer this struct has now.
WHAT	**Creates a reading of 5.0 degrees Celsius for Denver.**
WHERE	A top-level var, so the celsius assignment two lines down is allowed.
WHEN	At run time; the observer stays silent during this initialization.
WHY	A var, because the next line changes the temperature.

```
21 denver.celsius = -2.5
```

WHO	Your code, assigning a new value.
WHAT	**Sets the temperature, which triggers didSet and prints Denver: 5.0 -> -2.5.**
WHERE	Changes the value in denver.
WHEN	After creation, so the observer runs.
WHY	The observer fires without the caller asking for it.

```
22 print(denver.isFreezing)
```

WHO	print, reading the computed property.
WHAT	**Prints true, because -2.5 is at or below 0.0.**
WHERE	Top-level code; the console shows the Bool that the isFreezing getter computed.
WHEN	The last statement of the program, after the observer has printed its message.
WHY	The answer reflects the new temperature because it is computed on every read.

> **Try it**
>
> 1. Assign denver.celsius = -2.5 a second time. Does didSet run even though the value is the same?
> 2. Add static let boilingPoint = 100.0 and a computed property isBoiling.

CODE · RUNS IN THE BROWSER

Choosing how a value prints

Printing a struct shows every property, which gets noisy. Conforming to `CustomStringConvertible` lets you choose the text. That name is a **protocol**: a list of requirements a type promises to meet. This one asks for a single property, `description`. Protocols get their own lesson soon.

```swift
struct TodoItem: CustomStringConvertible {
    var title: String
    var isDone = false

    var description: String {
        let box = isDone ? "[x]" : "[ ]"
        return "\(box) \(title)"
    }
}

var items = [
    TodoItem(title: "Book flights"),
    TodoItem(title: "Pack charger", isDone: true),
]
items[0].isDone = true
items.append(TodoItem(title: "Water plants"))
for item in items {
    print(item)
}
print("Open: \(items.filter { !$0.isDone }.count)")
```

OUTPUT

```
[x] Book flights
[x] Pack charger
[ ] Water plants
Open: 1
```

Line by line

1 `struct TodoItem: CustomStringConvertible {`

WHO	You, declaring a struct that adopts a protocol.
WHAT	**Starts TodoItem and promises it provides everything CustomStringConvertible requires.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler checks that the promise is kept.
WHY	Adopting this protocol is how print and string interpolation learn your custom text.

2 `var title: String`

WHO	Every TodoItem instance, which stores its own title for description to read.
WHAT	**Declares the stored title property that description interpolates after the checkbox.**
WHERE	Stored inside each TodoItem value, first of its two properties.
WHEN	Set by the initializer.
WHY	No default, so the title is always required.

3 `var isDone = false`

WHO	Every TodoItem instance, which starts unfinished unless the initializer says otherwise.
WHAT	**Stored Bool with a default of false. Its type is inferred from the value.**
WHERE	Stored inside each TodoItem value; the default lets the memberwise init omit it.
WHEN	false unless the caller passes isDone.
WHY	New tasks start unfinished, so the default matches reality and shortens most initializer calls.

```
5  var description: String {
```

WHO The struct, fulfilling the protocol requirement.
WHAT **Declares the description property that CustomStringConvertible requires.**
WHERE Inside the struct; computed, so it takes no storage.
WHEN Whenever print or interpolation needs text for a TodoItem.
WHY The exact name and type matter: the protocol requires a property named description of type String.

```
6      let box = isDone ? "[x]" : "[ ]"
```

WHO The description getter, choosing a checkbox for the instance being printed.
WHAT **Chooses a checked or empty box for the task.**
WHERE Local constant inside the getter.
WHEN Each time description is read.
WHY A text checkbox makes console output scannable at a glance.

```
7      return "\(box) \(title)"
```

WHO The description getter, returning the text print will show.
WHAT **Returns text such as [x] Book flights.**
WHERE Last line of the getter.
WHEN Each time description is read.
WHY Keep descriptions short: they appear in logs and debugger output.

```
11 var items = [
```

WHO Your code, building an array literal.
WHAT **Starts an array of TodoItem values. Swift infers the type [TodoItem].**
WHERE A top-level var holding an array of TodoItem; var so the elements can change below.
WHEN At run time, once, as the array literal is evaluated.
WHY A var array, because the program changes an element and appends a new one.

```
12     TodoItem(title: "Book flights"),
```

WHO The memberwise initializer, using the default for isDone.
WHAT **Creates an unfinished task.**
WHERE First element of the array.
WHEN While the array literal is built.
WHY Leaving out isDone uses its default value, false.

```
13     TodoItem(title: "Pack charger", isDone: true),
```

WHO The memberwise initializer, this time given both arguments.
WHAT **Creates a finished task.**
WHERE The second element of the items array, the only one created already done.
WHEN While the array literal is built.
WHY A trailing comma after the last element is allowed and makes adding items later a one-line change.

```
15 items[0].isDone = true
```

WHO Your code, changing an element in place.
WHAT **Marks the first task done through the array subscript.**
WHERE Modifies the struct stored inside the array.
WHEN After the array is built.
WHY This works because items is a var. Struct elements in a let array cannot change.

```
16 items.append(TodoItem(title: "Water plants"))
```

WHO The array's append method.
WHAT **Adds a third, unfinished task at the end.**
WHERE Changes the items array.
WHEN After the first edit.
WHY Arrays of structs are the most common shape of app data.

```
17 for item in items {
```

WHO A for-in loop over the items array, binding each struct to item in turn.
WHAT **Visits each task in order, binding it to item.**
WHERE Top-level code; item is a copy of each element, valid only inside the loop body.
WHEN Three times, once per element.
WHY Iterating and printing each item uses your description, one per line.

```
18 print(item)
```

WHO print, now using your description.
WHAT **Prints the checkbox and title for this task.**
WHERE Inside the loop body.
WHEN Once per task, three times in total, each run printing one description.
WHY Without the protocol this would print TodoItem(title: ..., isDone: ...).

```
20 print("Open: \(items.filter { !$0.isDone }.count)")
```

WHO filter and count, inside an interpolation.
WHAT **Keeps tasks that are not done and prints how many: Open: 1.**
WHERE Top-level code; the console shows the count that filter produced.
WHEN The last statement, after the loop has printed all three tasks.
WHY filter takes a closure; $0 is the current element. Closures get a closer look in the paradigms lesson.

> **Try it**
>
> 1. Change the description so finished tasks show "done:" before the title instead of a box.
> 2. Print the whole array with print(items). Does Swift use your description for each element?

CHECK YOURSELF

Check your understanding

1. What does this print?

```
struct Counter { var value = 0 }
var a = Counter()
var b = a
b.value = 5
print(a.value)
```

A. 5
B. 0
C. A compile error, because b is a copy
D. nil

2. A `Playlist` stores `songs: [Song]`, and each song has a duration. Which design is best for the playlist's total duration?
   A. A stored var totalDuration that callers update after every change
   B. A global variable shared by all playlists
   C. A static property on Playlist
   D. A computed property that adds up the song durations when read

3. Why does this fail to compile?

```
struct Cart {
    var items: [String] = []
    func add(_ item: String) { items.append(item) }
}
```

   A. add mutates a stored property, so it needs mutating
   B. Arrays cannot be stored properties of a struct
   C. items must be declared with let instead
   D. Struct methods cannot take parameters

4. A property has a `didSet` observer. When does the observer run?
   A. Only when you call it like a method
   B. Only when the new value differs from the old one
   C. After any assignment outside init
   D. Once, when the instance is first created

Answers: Structs: Your Own Value Types

1. **B. 0** `var b = a` copies the struct, so `b` has its own `value`. Changing the copy leaves `a` at 0. Answering 5 assumes both names share one instance, which is how classes behave, not structs.
2. **D. A computed property that adds up the song durations when read** A computed property derives the total from the songs every time, so it can never disagree with them. A stored total is easy to forget to update, and a static or global value would be shared by every playlist instead of belonging to one.
3. **A. add mutates a stored property, so it needs mutating** Inside a non-mutating struct method, `self` is read-only, so `items.append` is rejected. Adding `mutating` fixes it. Making `items` a `let` would make the problem worse, not better.
4. **C. After any assignment outside init** `didSet` runs after every assignment, even one that stores the same value again. It does not run for assignments inside the type's own initializer, which is why creating the Denver reading printed nothing.

Classes and Reference Semantics

CONCEPT

Classes: shared objects with identity

A **class** looks a lot like a struct: properties, methods, initializers. The big difference is what assignment does. Assigning a struct copies the value. Assigning a class copies a **reference**, an arrow pointing at one shared object on the **heap** (memory that outlives any single function call). Change the object through any arrow and every holder sees the change.

Classes also offer **inheritance**: a subclass reuses and customizes a superclass. Swift frees an object automatically with **ARC** (Automatic Reference Counting) once no references to it remain.

Value type	Struct, enum, tuple. Each variable holds its own independent copy.
Reference type	Class, actor. Variables hold references to one shared instance.
Identity	Two references are identical (===) when they point at the very same object.

> **Why Swift prefers values**
>
> Objective-C, Apple's earlier language, built almost everything from classes. Swift flips the default: the standard library and SwiftUI are mostly structs, and classes are kept for things that need shared identity.

CODE · RUNS IN THE BROWSER

A class with an initializer

A joint bank account suits a class: two people share one account, and a deposit by either should show up for both. Classes get no memberwise initializer, so you write `init` yourself. Watch what happens when a second constant refers to the same account.

```swift
class BankAccount {
    let owner: String
    var balance: Double

    init(owner: String, balance: Double) {
        self.owner = owner
        self.balance = balance
    }

    func deposit(_ amount: Double) {
        balance += amount
    }
}

let joint = BankAccount(owner: "Ana and Ben", balance: 100)
let sameAccount = joint
sameAccount.deposit(50)
print(joint.balance)
print(joint === sameAccount)
```

OUTPUT

```
150.0
true
```

Line by line

1 `class BankAccount {`

WHO You, defining a reference type.
WHAT **Starts a class named BankAccount.**
WHERE Top level of the file. Instances will live on the heap.
WHEN Compile time; the compiler records the BankAccount class and its members.
WHY A shared account has identity: there is one account, no matter how many people hold a reference to it.

2 `let owner: String`

WHO Every BankAccount object, which keeps its own owner name for the whole life of the object.
WHAT **Stored constant for the owner names.**
WHERE Inside the object on the heap.
WHEN Assigned once, in init.
WHY The owners never change for this account, so let.

3 `var balance: Double`

WHO Every BankAccount object, which keeps its own balance on the heap.
WHAT **Declares a stored Double property that deposit changes later; var makes that change legal.**
WHERE Inside the object on the heap.
WHEN Assigned in init, then changed by deposits.
WHY No default value here, so the initializer must set it.

5 `init(owner: String, balance: Double) {`

WHO You, writing the initializer the class needs.
WHAT **Declares how to create an account from an owner and an opening balance.**
WHERE Inside the BankAccount class, after the two stored properties it must fill.
WHEN Each time BankAccount(owner:balance:) is called.
WHY Classes never get a free memberwise initializer, because subclasses make the right parameter list ambiguous.

6 `self.owner = owner`

WHO The initializer, storing the owner parameter into the property of the same name.
WHAT **Stores the owner parameter in the owner property.**
WHERE Inside init; self.owner is the stored property, owner alone is the parameter.
WHEN During object creation, before the reference is handed back to the caller.
WHY self. separates the property from the parameter with the same name.

7 `self.balance = balance`

WHO The initializer, storing the opening balance.
WHAT **Stores the opening balance.**
WHERE Inside init, the second of its two assignments.
WHEN During object creation, immediately after owner is stored.
WHY Every stored property must be set before init ends; the compiler enforces it.

```
10 func deposit(_ amount: Double) {
```

WHO The BankAccount class, declaring a method that changes the object's balance.
WHAT **Declares a method that adds money. Note there is no mutating keyword.**
WHERE Inside the class, after init; no mutating keyword, because classes are reference types.
WHEN Each time deposit is called.
WHY Class methods may change properties freely: they change the shared object, not the variable that refers to it.

```
11 balance += amount
```

WHO The method, updating the object.
WHAT **Adds amount to the stored balance.**
WHERE Changes the single object on the heap.
WHEN During each deposit call.
WHY Every reference to this object will see the new balance.

```
15 let joint = BankAccount(owner: "Ana and Ben", balance: 100)
```

WHO Your initializer, plus ARC, which starts counting references.
WHAT **Creates one account object on the heap and stores a reference to it in joint.**
WHERE joint is a top-level constant holding a reference, not the account itself.
WHEN At run time, when execution reaches this statement; the object is allocated on the heap here.
WHY let freezes the reference (joint always points at this account), not the object's properties.

```
16 let sameAccount = joint
```

WHO Assignment, copying a reference.
WHAT **Makes sameAccount point at the very same object. No new account is created.**
WHERE A second top-level constant; the heap still holds one object, now with two references.
WHEN At run time, immediately after the object exists; no second object is created.
WHY This single line is the core difference from structs, where assignment would have copied the whole value.

```
17 sameAccount.deposit(50)
```

WHO The deposit method, called through the second reference.
WHAT **Adds 50 to the shared object's balance.**
WHERE Changes the one object on the heap.
WHEN After both references exist.
WHY It compiles even though sameAccount is a let: the reference stays fixed, the object changes.

```
18 print(joint.balance)
```

WHO print, reading through the first reference.
WHAT **Prints 150.0: the deposit made through sameAccount is visible through joint.**
WHERE Top-level code; the console shows the balance read through joint.
WHEN After the deposit made through sameAccount has already changed the shared object.
WHY Shared changes are the point of a class. With a struct this would still be 100.0.

```
19 print(joint === sameAccount)
```

WHO The identity operator ===.
WHAT **Prints true, because both constants refer to the same object.**
WHERE Top-level code; the console shows the Bool that === produced.
WHEN The last statement of the program, after the balance has been printed.
WHY === asks "same object?", while == asks "equal values?". Only reference types have identity.

Try it

1. Create a third account with the same owner and balance and compare it with === to joint. Why is it false?
2. Change class to struct and add mutating where the compiler asks. What does the program print now?

CODE · RUNS IN THE BROWSER

Inheritance, override, super and final

A **subclass** inherits every property and method of its **superclass** and can `override` methods to specialize them. `super` calls the superclass version. `final` forbids further subclassing, which documents intent and lets the compiler call methods directly.

```
class Account {
    var balance: Double
    init(balance: Double) { self.balance = balance }

    func monthEnd() -> String {
        "Balance: \(balance)"
    }
}

final class SavingsAccount: Account {
    override func monthEnd() -> String {
        balance += balance * 0.01
        return "Interest added. " + super.monthEnd()
    }
}

let accounts: [Account] = [
    Account(balance: 200),
    SavingsAccount(balance: 1000),
]
for account in accounts {
    print(account.monthEnd())
}
```

OUTPUT

```
Balance: 200.0
Interest added. Balance: 1010.0
```

Line by line

1 `class Account {`

WHO — You, defining a base class.

WHAT — **Starts the superclass that other account kinds will build on.**

WHERE — Top level of the file.

WHEN — Compile time; Account is the base class that SavingsAccount extends below.

WHY — Not marked final, so subclasses are allowed.

```
2  var balance: Double
```

WHO Each Account object, and objects of every subclass.
WHAT **Stored balance, inherited by all subclasses.**
WHERE Inside each object on the heap.
WHEN Assigned once in init, then changed by SavingsAccount's override at every month end.
WHY Declaring shared state once in the superclass avoids repeating it in every account kind.

```
3  init(balance: Double) { self.balance = balance }
```

WHO The base class initializer.
WHAT **Creates an account with an opening balance, written on one line.**
WHERE Inside Account; the subclass inherits this initializer because it adds no stored properties.
WHEN Each time any account is created.
WHY A subclass that adds no stored properties inherits this initializer automatically.

```
5  func monthEnd() -> String {
```

WHO The Account class, declaring the method the subclass will override.
WHAT **Declares the month-end report method that subclasses may override.**
WHERE Inside Account, below the initializer.
WHEN Each time monthEnd() is called on a plain Account.
WHY A method meant for customizing is the hook that makes inheritance useful.

```
6  "Balance: \(balance)"
```

WHO The body of Account's monthEnd, reporting the balance without changing it.
WHAT **Returns a line such as Balance: 200.0.**
WHERE Inside monthEnd() in Account; the subclass reaches this line only through super.
WHEN Each call to monthEnd on a plain Account, and each time the override calls super.
WHY Single-expression bodies return implicitly.

```
10 final class SavingsAccount: Account {
```

WHO You, defining a subclass.
WHAT **Declares SavingsAccount as a kind of Account. final means nothing may subclass it.**
WHERE Top level of the file.
WHEN Compile time; the compiler checks that SavingsAccount overrides only what Account declares.
WHY Mark classes final unless you design them for subclassing. It prevents surprises and helps performance.

```
11 override func monthEnd() -> String {
```

WHO The subclass, replacing an inherited method.
WHAT **Provides a new monthEnd() for savings accounts. override is mandatory.**
WHERE Inside SavingsAccount, replacing the inherited monthEnd.
WHEN Whenever monthEnd() is called on a SavingsAccount, even through an Account-typed variable.
WHY The required override keyword stops you from replacing a superclass method by accident.

```
12 balance += balance * 0.01
```

WHO The overriding method, changing the balance inherited from Account.
WHAT **Adds 1 percent interest to the inherited balance property.**
WHERE Changes the object on the heap.
WHEN During each call to monthEnd on a SavingsAccount, before super runs.
WHY The subclass uses the inherited property as if it had declared it itself.

```
13  return "Interest added. " + super.monthEnd()
```

WHO super, which reaches the superclass implementation.
WHAT **Calls Account's monthEnd() and puts a prefix in front of its text.**
WHERE Inside the override, after the interest has been added, so super reports the new balance.
WHEN After the interest is added, so the report shows the new balance.
WHY Reusing super keeps the base formatting in one place instead of copying it.

```
17  let accounts: [Account] = [
```

WHO Your code, building an array typed as the superclass.
WHAT **Starts an array whose elements are all treated as Account.**
WHERE A top-level constant holding an array whose declared element type is Account.
WHEN At run time, as the array literal is evaluated.
WHY A subclass instance can be stored wherever its superclass is expected. This is polymorphism.

```
18      Account(balance: 200),
```

WHO The Account initializer, creating a plain base-class object.
WHAT **Creates a plain Account with a balance of 200 as the first element.**
WHERE The first element of the accounts array.
WHEN While the array is built.
WHY It will use the base monthEnd().

```
19      SavingsAccount(balance: 1000),
```

WHO The initializer inherited from Account, called on the subclass.
WHAT **A savings account, created with the init it inherited from Account.**
WHERE The second element of the accounts array, stored as an Account even though it is a SavingsAccount.
WHEN While the array is built.
WHY No init was written in SavingsAccount, yet this works because it adds no stored properties.

```
21  for account in accounts {
```

WHO A for-in loop over the accounts array, binding each reference to account.
WHAT **Visits each account, typed as Account.**
WHERE Top-level code; account has the static type Account inside the loop.
WHEN Twice, once for each account in the array.
WHY The loop does not know or care which subclass each element is.

```
22      print(account.monthEnd())
```

WHO Dynamic dispatch, choosing the method at run time.
WHAT **Calls the right monthEnd() for each object: Balance: 200.0, then Interest added. Balance: 1010.0.**
WHERE Inside the loop body, called through a reference typed as Account.
WHEN Once per account, two times in total; the override runs only on the second pass.
WHY The actual object decides which version runs, not the declared type of the variable.

> **Try it**
>
> 1. Remove the word override. Read the compiler's message.
> 2. Try class PremiumSavings: SavingsAccount {}. What does final prevent?

COMPARE

Copy a struct, share a class

The same tiny settings type, written once as a struct and once as a class. Both programs copy `phone` into `tablet` and then change the tablet's volume. Run both sides and compare the first number.

A: struct: tablet gets its own copy

```swift
struct Settings {
    var volume = 5
}

var phone = Settings()
var tablet = phone
tablet.volume = 9
print(phone.volume, tablet.volume)
```

OUTPUT

```
5 9
```

1 `struct Settings {`

WHO	You, defining a value type.
WHAT	**Declares Settings as a struct.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler records Settings as a value type.
WHY	This one keyword decides whether assignment copies or shares.

2 `var volume = 5`

WHO	Every Settings value, each carrying its own copy of volume.
WHAT	**A stored volume with a default of 5.**
WHERE	Stored inside each Settings value, with a default so no init is needed.
WHEN	Set to 5 when created.
WHY	The default gives the struct a no-argument initializer.

5 `var phone = Settings()`

WHO	The default initializer Swift synthesized, because every property has a default.
WHAT	**Creates settings with volume 5 in phone.**
WHERE	phone holds the value directly.
WHEN	At run time, when execution reaches this statement.
WHY	var so it could be changed; here only tablet changes.

6 `var tablet = phone`

WHO	Assignment of a value type.
WHAT	**Copies every stored property of phone into a brand-new value in tablet.**
WHERE	tablet holds its own independent value.
WHEN	At run time, right after phone exists; the copy happens here.
WHY	After this line the two are unrelated, like two printed copies of one page.

7 `tablet.volume = 9`

WHO	Your code, changing the copy stored in tablet.
WHAT	**Changes the volume in tablet's copy only.**
WHERE	Modifies the value inside tablet.
WHEN	After the copy on the line above, so phone is unaffected.
WHY	tablet must be a var to change a property of a struct.

8 `print(phone.volume, tablet.volume)`

- WHO: print, with two items separated by a space.
- WHAT: **Prints 5 9: the original kept its volume.**
- WHERE: Top-level code; the console shows two different numbers.
- WHEN: The last statement of the left program.
- WHY: Value semantics: no other code can change your copy behind your back.

B: class: tablet shares the object

```
class Settings {
    var volume = 5
}

let phone = Settings()
let tablet = phone
tablet.volume = 9
print(phone.volume, tablet.volume)
```

OUTPUT

```
9 9
```

1 `class Settings {`

- WHO: You, defining a reference type.
- WHAT: **Declares Settings as a class.**
- WHERE: Top level of the file; instances will live on the heap.
- WHEN: Compile time; the compiler records Settings as a reference type this time.
- WHY: Only the keyword changed from the left side.

2 `var volume = 5`

- WHO: Every Settings object, which keeps its volume on the heap.
- WHAT: **A stored volume with a default of 5.**
- WHERE: Inside the object on the heap.
- WHEN: Set to 5 when created.
- WHY: With every property defaulted, the class gets a free init().

5 `let phone = Settings()`

- WHO: The initializer, plus ARC.
- WHAT: **Creates one object and stores a reference to it in phone.**
- WHERE: phone holds a reference; the object is on the heap.
- WHEN: At run time; the object is allocated on the heap here.
- WHY: let is enough here, because the reference itself never changes.

6 `let tablet = phone`

- WHO: Assignment of a reference type.
- WHAT: **Copies the reference, so tablet points at the same object as phone.**
- WHERE: Two references, one heap object.
- WHEN: At run time, right after phone exists; only the reference is copied.
- WHY: No new Settings object is created by this line.

```
7  tablet.volume = 9
```

WHO	Your code, changing the shared object.
WHAT	**Sets volume to 9 on the one object both constants point to.**
WHERE	Changes the object on the heap.
WHEN	After the assignment on the line above, through the second reference.
WHY	Allowed even though tablet is a let: let fixes the reference, not the object.

```
8  print(phone.volume, tablet.volume)
```

WHO	print, with two values separated by a space.
WHAT	**Prints 9 9: phone sees the change made through tablet.**
WHERE	Top-level code; the console shows the same number twice.
WHEN	The last statement of the right program.
WHY	Reference semantics: every holder of the reference sees every change.

> **Takeaway**
>
> With a struct, `var tablet = phone` copies the value, so the two settings are independent. With a class, it copies the reference, so both names point at one object. Also notice the class side uses `let` and still changes `volume`: `let` freezes the reference, not the object behind it.

INTERACTIVE

See it in memory: values and references

Both worlds start with `var a = Account(balance: 100)`. Each button runs the same line in the struct world and in the class world. Copy `a` into `b`, change `b`, and watch which balances move. Then point `b` at a brand-new account and watch ARC count the references to each object.

This step is an interactive simulation in the HTML version.

CONCEPT

Struct or class? A decision guide

Apple's guidance is short: **start with a struct**. Values are easier to reason about, because no other code can change your copy behind your back. They are also safe to hand between concurrent tasks, which matters later in this module.

Reach for a **class** when an instance has a real *identity* that several parts of the app must share and see change together, or when an Apple framework requires one.

Struct	Data: a to-do item, a coffee order, a weather reading, a SwiftUI view.
Class	A shared thing with identity: the one cart that several screens update, a download in progress, an @Observable model that views watch.
Enum	A fixed set of possibilities, such as a coffee size or a loading state. Next lesson.
Actor	A reference type that protects its state from data races. Covered in the concurrency lesson.

> **A quick test**
>
> Ask: if I copy this and change the copy, should the original change too? If not, use a struct. If it should, you need shared identity, which means a class.

CHECK YOURSELF

Check your understanding

1. What does this print?

```
class Light { var isOn = false }
let a = Light()
let b = a
b.isOn = true
print(a.isOn)
```

   A. false
   B. true
   C. A compile error, because b is a let constant
   D. nil

2. Which of these is the best fit for a class rather than a struct?
   A. A GPS coordinate with a latitude and a longitude
   B. A color with red, green, blue and alpha values
   C. One shopping cart shared by every screen
   D. A date range with a start date and an end date

3. What does `===` check?
   A. Whether two references point to the same instance
   B. Whether two values have equal properties
   C. Whether two structs were copied from each other
   D. Whether a type conforms to Equatable

4. `SavingsAccount` overrides `monthEnd()` and calls `super.monthEnd()` inside it. What does that call run?
   A. The SavingsAccount version again, causing endless recursion
   B. Nothing: super only works inside initializers
   C. Whichever monthEnd() appears first in the file
   D. The original implementation in the superclass, Account

Answers: Classes and Reference Semantics

1. **B. true** a and b refer to the same `Light` object, so the change made through b is visible through a. It compiles because `let` on a class reference only stops you from pointing b at a different object.
2. **C. One shopping cart shared by every screen** The cart has identity: every screen must see the same cart change. Coordinates, colors and date ranges are plain values; copying them is exactly what you want, so they should be structs.
3. **A. Whether two references point to the same instance** === is the identity operator for reference types: same object or not. Comparing property values is the job of ==, which requires `Equatable`. Structs have no identity, so === does not apply to them.
4. **D. The original implementation in the superclass, Account** `super` skips the current class and calls the superclass implementation. That is how the override reuses the base text and adds a prefix. Calling `self.monthEnd()` instead would recurse forever.

Enums: Modeling Choices and State

CONCEPT

A fixed set of possibilities

An **enum** (enumeration) defines a type with a fixed list of **cases**. A coffee size is small, medium or large, never "medium-ish" and never a typo like "lrage". When you `switch` over an enum, the compiler insists that you handle every case. Add a case later and Swift points to every switch that needs updating.

Swift enums go further than in most languages. A case can carry a **raw value** (one fixed Int or String per case) or **associated values** (extra data attached to one particular value). Enums can also have methods and computed properties.

Case	One of the named possibilities, written with a leading dot when the type is known: .medium.
Exhaustive	A switch that covers every case. Swift requires it, so no possibility is forgotten.
Raw value	A fixed Int or String tied to each case, useful for saving and loading.
Associated value	Data stored with one specific value, such as the last four digits of a card.

> **Enums are values**
>
> Like structs, enums are value types: assigning one copies it. They are cheap, safe to share between tasks, and perfect for describing state.

CODE · RUNS IN THE BROWSER

Cases and an exhaustive switch

A coffee size has exactly three possibilities, so it is an enum. The function maps each size to fluid ounces with a `switch`. There is no `default` branch: the compiler can see that all three cases are handled, and it will complain if a fourth case is ever added.

```swift
enum CoffeeSize {
    case small
    case medium
    case large
}

func ounces(for size: CoffeeSize) -> Int {
    switch size {
    case .small:
        return 8
    case .medium:
        return 12
    case .large:
        return 16
    }
}

var size = CoffeeSize.medium
print(size, ounces(for: size))
size = .large
print(size, ounces(for: size))
```

OUTPUT

```
medium 12
large 16
```

Line by line

1 `enum CoffeeSize {`

WHO	You, defining a new enum type.
WHAT	**Starts an enum named CoffeeSize.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler records CoffeeSize and its three cases before any code runs.
WHY	A fixed menu of sizes is the textbook case for an enum: the compiler rejects any size that is not on the list.

2 `case small`

WHO	The CoffeeSize type, listing its first possible value.
WHAT	**Declares the first possible value, small.**
WHERE	Inside the enum body, first of the three cases.
WHEN	Compile time; a case declaration runs no code.
WHY	Case names use lowerCamelCase, like properties.

3 `case medium`

WHO	The CoffeeSize type, listing its second value.
WHAT	**Declares a second possible value.**
WHERE	Inside the enum body, between small and large.
WHEN	Compile time, alongside the other two cases.
WHY	Each case is a complete value on its own; no number or string is needed.

```
4  case large
```

WHO	The CoffeeSize type, listing its third and last value.
WHAT	**Declares the third and last possible value.**
WHERE	Inside the enum body, the last case before the closing brace.
WHEN	Compile time; the compiler now knows the full list that every switch must cover.
WHY	With three cases, a CoffeeSize variable can hold exactly one of these three, nothing else.

```
7  func ounces(for size: CoffeeSize) -> Int {
```

WHO	You, defining a function that takes the enum.
WHAT	**Declares a function that converts a size into fluid ounces.**
WHERE	Top level of the file.
WHEN	Runs each time it is called.
WHY	The argument label for reads naturally at the call site: ounces(for: size).

```
8  switch size {
```

WHO	The switch statement, checked by the compiler.
WHAT	**Chooses a branch based on which case size holds.**
WHERE	Inside the ounces(for:) function, switching on its only parameter.
WHEN	Each call to ounces(for:), two times in this program.
WHY	switch on an enum must be exhaustive, so a missing size is a compile error, not a runtime surprise.

```
9  case .small:
```

WHO	The switch statement, testing whether size is .small.
WHAT	**Matches when size is .small. The type is known, so the dot shorthand is enough.**
WHERE	The first branch of the switch, inside ounces(for:).
WHEN	Only when size is small.
WHY	Leading-dot syntax keeps case names short without losing type safety.

```
10 return 8
```

WHO	The ounces(for:) function, handing back its result for a small coffee.
WHAT	**Returns 8 ounces for a small.**
WHERE	Inside the .small branch.
WHEN	When the branch matches.
WHY	Swift branches never fall through to the next case, so no break is needed.

```
11 case .medium:
```

WHO	The switch statement, testing whether size is .medium.
WHAT	**Matches the .medium case; the leading dot works because the compiler knows size is a CoffeeSize.**
WHERE	The second branch of the switch, inside ounces(for:).
WHEN	Only when size is medium.
WHY	One branch per case keeps the mapping easy to scan.

```
12 return 12
```

WHO	The ounces(for:) function, returning from the medium branch.
WHAT	**Returns 12, the ounce count for a medium coffee; this is the branch the first print triggers.**
WHERE	Inside the .medium branch.
WHEN	When the branch matches.
WHY	Each branch returns, so the function always produces a value.

```
13  case .large:
```

WHO The switch statement, testing the final case so the switch is exhaustive.
WHAT **Matches the .large case, the last of the three.**
WHERE The third and final branch of the switch; no default is needed.
WHEN Only when size is large.
WHY With this branch, every case is covered, so no default is required.

```
14  return 16
```

WHO The ounces(for:) function, returning from the large branch.
WHAT **Returns 16, the ounce count for a large coffee; the second print reaches this line.**
WHERE Inside the .large branch.
WHEN When the branch matches.
WHY Leaving out default means a future case forces you to come back here.

```
18  var size = CoffeeSize.medium
```

WHO Your code, creating the first CoffeeSize value in the program.
WHAT **Creates a variable holding the .medium case. Its type is inferred as CoffeeSize.**
WHERE A top-level var, so the later assignment to .large is allowed.
WHEN At run time, when execution reaches this line; the enum type was known at compile time.
WHY The full name CoffeeSize.medium is needed here, because nothing else tells Swift the type yet.

```
19  print(size, ounces(for: size))
```

WHO print and your function.
WHAT **Prints medium 12. Printing an enum case shows its name.**
WHERE Top-level code; the console shows the case name and the ounces on one line.
WHEN After size is set.
WHY Case names print automatically, which makes enums easy to debug.

```
20  size = .large
```

WHO Your code, reassigning the variable.
WHAT **Changes size to .large. The type is already known, so the dot shorthand works.**
WHERE The same top-level var.
WHEN After the first print.
WHY Once the type is fixed, the short form reads cleanly.

```
21  print(size, ounces(for: size))
```

WHO print and your function.
WHAT **Prints large 16, because size now holds .large and the switch took its third branch.**
WHERE Top-level code; the console shows the second line of output.
WHEN The last statement of the program, after the assignment on the line above.
WHY Same call, different case, different branch.

> **Try it**
>
> 1. Add case extraLarge to the enum and run. Read where the compiler says the switch is incomplete.
> 2. Turn ounces into a computed property inside the enum: var ounces: Int { switch self { … } }.

CODE · RUNS IN THE BROWSER

Raw values and CaseIterable

A **raw value** gives each case a fixed Int or String, which is handy when you save a choice to disk or receive it from a server. Swift fills in raw values for you: Int cases count up, and String cases default to

their own names. `CaseIterable` adds an `allCases` array, perfect for a picker.

```swift
enum Priority: Int, CaseIterable {
    case low = 1
    case medium
    case high
}

enum Category: String {
    case work
    case home
    case errands = "Errands and shopping"
}

print(Priority.high.rawValue, Priority.allCases.count)
print(Category.home.rawValue)
print(Category.errands.rawValue)
let saved = 2
if let priority = Priority(rawValue: saved) {
    print("Loaded \(priority)")
}
if Priority(rawValue: 7) == nil {
    print("7 is not a priority")
}
```

OUTPUT

```
3 3
home
Errands and shopping
Loaded medium
7 is not a priority
```

Line by line

1 `enum Priority: Int, CaseIterable {`

- WHO — You, declaring an enum with a raw type and a protocol.
- WHAT — **Each case gets an Int raw value, and the type gains an allCases list.**
- WHERE — Top level of the file.
- WHEN — Compile time; the compiler generates rawValue, init?(rawValue:) and allCases.
- WHY — Int raw values are compact to store; CaseIterable lets a settings screen list every option.

2 `case low = 1`

- WHO — The Priority type, declaring its first case with an explicit raw value.
- WHAT — **Declares low with an explicit raw value of 1.**
- WHERE — Inside the enum body, first of the three cases.
- WHEN — Compile time; the raw value 1 is fixed here, not computed later.
- WHY — Starting at 1 keeps raw values human-friendly; the next cases count up from here.

```
3   case medium
```

WHO The compiler, filling in the raw value.
WHAT **Declares medium; its raw value becomes 2 automatically.**
WHERE Inside the enum body, right after low, so it continues the count.
WHEN Compile time; the compiler assigns the raw value 2 by counting on from low.
WHY Implicit raw values avoid typing a sequence by hand.

```
4   case high
```

WHO The compiler, assigning this case the next raw value.
WHAT **Declares high with raw value 3.**
WHERE Inside the enum body, the last case; its raw value becomes 3.
WHEN Compile time, when the enum is declared.
WHY Order matters: cases are numbered in the order they are written.

```
7   enum Category: String {
```

WHO You, declaring an enum with String raw values.
WHAT **Each case gets a String raw value.**
WHERE Top level of the file.
WHEN Compile time; the compiler records Category as an enum backed by String raw values.
WHY String raw values read well in saved files and JSON.

```
8   case work
```

WHO The compiler, which uses the case name, work, as the implicit raw value.
WHAT **Declares work; its raw value defaults to the text work.**
WHERE Inside the Category enum, first case.
WHEN Compile time; the implicit raw value is fixed when the case is declared.
WHY For String raw values, the case name is the default, so most cases need nothing extra.

```
9   case home
```

WHO The compiler, which again uses the case name as the raw value.
WHAT **Declares home with raw value home.**
WHERE Inside the Category enum, second case.
WHEN Compile time, like every case declaration.
WHY Same default rule as work.

```
10  case errands = "Errands and shopping"
```

WHO You, overriding the default.
WHAT **Declares errands with a custom raw value.**
WHERE Inside the Category enum, the last case, the only one with an explicit raw value.
WHEN Compile time; the string is attached to the case when it is declared.
WHY Override the default when the stored text must differ from the case name.

```
13  print(Priority.high.rawValue, Priority.allCases.count)
```

WHO print, rawValue and allCases.
WHAT **Prints 3 3: high has raw value 3, and there are three cases.**
WHERE Top-level code; the console shows the raw value 3 and the count 3 on one line.
WHEN At run time, the first statement after the two enums are declared.
WHY allCases is an array, so count, map and for-in all work on it.

```
14 print(Category.home.rawValue)
```

WHO The print function, given the String that rawValue produced for home.
WHAT **Prints home, the default raw value.**
WHERE Top-level code; the console shows the word home.
WHEN At run time, as the second line of output.
WHY Confirms that String raw values default to the case name.

```
15 print(Category.errands.rawValue)
```

WHO The print function, given the custom raw value of errands.
WHAT **Prints Errands and shopping.**
WHERE Top-level code; the console shows Errands and shopping.
WHEN At run time, as the third line of output.
WHY The custom raw value replaces the default.

```
16 let saved = 2
```

WHO Your code, standing in for a value loaded from storage.
WHAT **Holds the raw number a previous launch might have saved.**
WHERE A top-level constant standing in for a number read from storage.
WHEN At run time, before the two lookups below use it.
WHY Apps often store the raw value and rebuild the enum when they launch.

```
17 if let priority = Priority(rawValue: saved) {
```

WHO The failable initializer init?(rawValue:).
WHAT **Tries to turn 2 into a Priority. It succeeds with .medium and binds it to priority.**
WHERE A top-level if statement whose binding, priority, exists only inside its braces.
WHEN At run time; the failable initializer runs here and succeeds for 2.
WHY The initializer returns an optional, because not every Int matches a case. if let handles both outcomes.

```
18 print("Loaded \(priority)")
```

WHO print, inside the success branch.
WHAT **Prints Loaded medium, because the raw value 2 maps to the medium case.**
WHERE Inside the if let block, where priority is a plain, unwrapped Priority.
WHEN Only if the conversion succeeded.
WHY Inside the block there is no optional left to unwrap.

```
20 if Priority(rawValue: 7) == nil {
```

WHO The same failable initializer.
WHAT **Tries 7, which matches no case, so the result is nil.**
WHERE A second top-level if statement, this one with no binding.
WHEN At run time, after the first if has finished.
WHY Bad input from storage or a server produces nil instead of a crash.

```
21 print("7 is not a priority")
```

WHO print, inside the failure branch.
WHAT **Prints the warning text.**
WHERE Inside the if block.
WHEN Only when the conversion failed.
WHY Always plan for the nil case when building enums from outside data.

> **Try it**
>
> 1. Print Priority.allCases. What does Swift show for an array of enum cases?
> 2. Change case low = 1 to case low = 10. What are the raw values of medium and high now?

CODE · RUNS IN THE BROWSER

Associated values

Some cases need extra data. A card payment needs the last four digits; a gift card needs its balance; cash needs nothing. **Associated values** attach that data to an individual value. A `switch` pulls the data back out with `let` bindings, and a `where` clause can add a condition.

```
enum Payment {
    case cash
    case card(last4: String)
    case giftCard(balance: Double)
}

func describe(_ payment: Payment) -> String {
    switch payment {
    case .cash: return "Cash at pickup"
    case .card(let last4): return "Card ending in \(last4)"
    case .giftCard(let balance) where balance < 5:
        return "Gift card (low balance)"
    case .giftCard(let balance): return "Gift card with $\(balance)"
    }
}

let payments: [Payment] = [.cash, .card(last4: "4242"),
                           .giftCard(balance: 2.5),
                           .giftCard(balance: 40)]
for payment in payments {
    print(describe(payment))
}
```

OUTPUT

```
Cash at pickup
Card ending in 4242
Gift card (low balance)
Gift card with $40.0
```

Line by line

1 `enum Payment {`

- WHO — You, defining an enum with associated values.
- WHAT — **Starts an enum describing how a customer pays.**
- WHERE — Top level of the file.
- WHEN — Compile time; the compiler records Payment and the data each case carries.
- WHY — Each payment kind needs different data, which a struct with optional fields would model badly.

2 `case cash`

- WHO — The Payment type, declaring a case that carries no data.
- WHAT — **A case with no extra data.**
- WHERE — Inside the enum body, first of the three cases.
- WHEN — Compile time; a case declaration runs no code.
- WHY — Cases with and without associated values can live in one enum.

```
3   case card(last4: String)
```

WHO The Payment type, declaring a case that carries a String.
WHAT **A case that carries a labeled String.**
WHERE Inside the enum body, second case.
WHEN Compile time; each card value stores its own digits.
WHY Labels document what the data means at every call site.

```
4   case giftCard(balance: Double)
```

WHO The Payment type, declaring a case that carries a Double.
WHAT **A case that carries the remaining balance.**
WHERE Inside the enum body, the last case.
WHEN Compile time; the associated value is supplied only when a value is created later.
WHY The data exists only when the case is giftCard, so there is no meaningless balance for cash.

```
7   func describe(_ payment: Payment) -> String {
```

WHO You, defining a function over the enum.
WHAT **Declares a function that turns a payment into checkout text.**
WHERE Top level of the file.
WHEN Each call to describe, four times in this program, once per payment in the loop.
WHY Keeping presentation logic in one function makes all payment text consistent.

```
8   switch payment {
```

WHO The switch, matching patterns.
WHAT **Picks the branch that matches the case inside payment.**
WHERE Inside describe, switching on its only parameter.
WHEN Each call to describe, before any return runs.
WHY Pattern matching is how Swift reads associated values safely.

```
9   case .cash: return "Cash at pickup"
```

WHO The switch statement, testing for the case with no associated value.
WHAT **Matches cash and returns its text, all on one line.**
WHERE The first branch of the switch, inside describe.
WHEN When payment is .cash.
WHY Short branches can sit on one line when that keeps the switch easy to scan.

```
10  case .card(let last4): return "Card ending in \(last4)"
```

WHO Pattern matching with a binding.
WHAT **Matches card and binds its associated String to a new constant last4.**
WHERE Second branch; last4 exists only inside it.
WHEN When payment is a card.
WHY The binding gives you the data without any force-unwrapping.

```
11  case .giftCard(let balance) where balance < 5:
```

WHO Pattern matching with a where clause.
WHAT **Matches gift cards whose balance is under 5.**
WHERE Third branch, checked before the next one.
WHEN When payment is a gift card with a low balance.
WHY Branches are tried top to bottom, so the specific case must come before the general one.

```
12  return "Gift card (low balance)"
```

WHO The describe function, returning from the low-balance branch.
WHAT **Returns the warning text.**
WHERE Inside the low-balance branch.
WHEN When that branch matches.
WHY A low balance might prompt the app to ask for a second payment method.

```
13  case .giftCard(let balance): return "Gift card with $\(balance)"
```

WHO Pattern matching, binding the balance for any gift card the where clause rejected.
WHAT **Matches every remaining gift card and shows its balance.**
WHERE The last branch of the switch; it catches every gift card with 5 dollars or more.
WHEN When the earlier where clause did not match.
WHY Without this branch the switch would not be exhaustive, because gift cards of 5 or more would be unhandled.

```
17  let payments: [Payment] = [.cash, .card(last4: "4242"),
```

WHO Your code, building sample data.
WHAT **Starts an array of four payments. The type annotation lets the dot shorthand work.**
WHERE A top-level constant holding an array of Payment values.
WHEN At run time, as the array literal is evaluated.
WHY Sample data that hits every branch is a quick way to test a switch.

```
18  .giftCard(balance: 2.5),
```

WHO The Payment enum, creating a giftCard case with its associated value.
WHAT **A gift card with a low balance.**
WHERE The third element of the payments array.
WHEN While the array is built.
WHY This one exercises the where branch.

```
19  .giftCard(balance: 40)]
```

WHO The Payment enum, creating a second giftCard value with a larger balance.
WHAT **A gift card with plenty of balance, closing the array.**
WHERE The fourth and last element of the payments array.
WHEN While the array is built.
WHY This one falls through to the general gift card branch.

```
20  for payment in payments {
```

WHO A for-in loop over the payments array, binding each value to payment.
WHAT **Visits each of the four payments in order, binding it to payment for the body.**
WHERE Top-level code; payment is valid only inside the loop body.
WHEN Four passes, one per element of payments.
WHY Each iteration may take a different branch of the switch.

```
21  print(describe(payment))
```

WHO The print function, given the String that describe returned.
WHAT **Prints one line per payment.**
WHERE Inside the loop body, once per pass.
WHEN Once per payment, so four lines of output appear.
WHY The output shows every branch, including the where clause.

Try it

1. Add case applePay to Payment. Which line does the compiler point to?
2. Swap the two gift card branches. What prints for the 2.5 gift card now, and why?

CONCEPT

Model screen state with one enum

A screen that loads songs can be idle, loading, showing songs, or showing an error. A common first attempt uses three Bools: `isLoading`, `hasError` and `hasData`. Three Bools allow 8 combinations, but only 4 make sense. The other 4 are bugs waiting to happen, such as a spinner drawn on top of an error message.

One enum with four cases allows exactly the four valid states. Associated values carry the data each state needs: the songs, or the error text. Try both models in the figure.

Interactive figure in the HTML version: types state machine.

CODE · RUNS IN THE BROWSER

Screen state as an enum

Here is that loading state as code. A **computed property** on the enum turns each state into the message the screen should show. SwiftUI screens use exactly this pattern: the view switches over the state and draws one thing for each case.

```
enum LoadState {
    case idle
    case loading
    case loaded([String])
    case failed(String)

    var message: String {
        switch self {
        case .idle: return "Pull to refresh"
        case .loading: return "Loading..."
        case .loaded(let songs): return "\(songs.count) songs"
        case .failed(let reason): return "Error: \(reason)"
        }
    }
}

let timeline: [LoadState] = [.idle, .loading,
    .loaded(["Golden Hour", "Night Owl"]), .failed("Offline")]
for state in timeline {
    print(state.message)
}
```

OUTPUT

```
Pull to refresh
Loading...
2 songs
Error: Offline
```

Line by line

1 `enum LoadState {`

- WHO You, defining the screen’s state type.
- WHAT **Starts an enum describing every state the song screen can be in.**
- WHERE Top level of the file.
- WHEN Compile time; the compiler records LoadState and the data two of its cases carry.
- WHY One enum is the single source of truth for what the screen shows.

2 `case idle`

- WHO The LoadState type, declaring the state a screen starts in.
- WHAT **The state before any request has been made; it carries no data.**
- WHERE Inside the enum body, first of the four cases.
- WHEN Compile time; a case declaration runs no code.
- WHY Idle needs no data.

3 `case loading`

- WHO The LoadState type, declaring the state while a request is in flight.
- WHAT **A request is in flight.**
- WHERE Inside the enum body, second case.
- WHEN Compile time, like every case.
- WHY Being in this case rules out showing an error or a list at the same moment.

4 `case loaded([String])`

- WHO The LoadState type, declaring the success state together with its data.
- WHAT **Loaded, carrying the song titles as an associated array.**
- WHERE Inside the enum body, third case; the array travels with the case.
- WHEN Compile time; the songs array is attached only when a value is created.
- WHY The songs exist only in the loaded state, so no code can read stale songs while loading.

5 `case failed(String)`

- WHO The LoadState type, declaring the failure state with its reason.
- WHAT **Failed, carrying a message for the user.**
- WHERE Inside the enum body, the last case.
- WHEN Compile time; the reason string arrives when a failed value is created.
- WHY The error text travels with the failure, so it cannot be shown at the wrong time.

7 `var message: String {`

- WHO The enum, providing a computed property.
- WHAT **Declares a property that every LoadState value can answer.**
- WHERE Inside the enum; takes no storage.
- WHEN Each time message is read.
- WHY Enums can hold computed properties and methods, keeping state logic next to the states.

```
8  switch self {
```

WHO The switch, inspecting the current value.
WHAT **self is the LoadState value message was read on.**
WHERE Inside the message getter, switching on the value itself.
WHEN Each read of message, four times in this program.
WHY Switching over self is the standard way to give each case its own behavior.

```
9  case .idle: return "Pull to refresh"
```

WHO The switch statement, testing for idle.
WHAT **Text for the idle state.**
WHERE The first branch of the switch, inside the message getter.
WHEN When the state is idle.
WHY Each case maps to exactly one message.

```
10 case .loading: return "Loading..."
```

WHO The switch statement, testing for loading.
WHAT **Text for the loading state.**
WHERE The second branch of the switch, inside the message getter.
WHEN When the state is loading.
WHY A real screen would show a ProgressView here.

```
11 case .loaded(let songs): return "\(songs.count) songs"
```

WHO Pattern matching with a binding.
WHAT **Pulls out the songs array and reports how many there are.**
WHERE Third branch; songs exists only here.
WHEN When the state is loaded.
WHY The data is unwrapped by the pattern, with no optionals involved.

```
12 case .failed(let reason): return "Error: \(reason)"
```

WHO Pattern matching with a binding.
WHAT **Pulls out the error text and prefixes it.**
WHERE The last branch of the switch; after it, every case is covered.
WHEN When the state is failed.
WHY All four cases are covered, so the switch is exhaustive.

```
17 let timeline: [LoadState] = [.idle, .loading,
```

WHO Your code, simulating a screen's life.
WHAT **Starts an array of states in the order a real load would pass through them.**
WHERE A top-level constant holding an array of LoadState values.
WHEN At run time, as the array literal is evaluated.
WHY Walking through states in order is a cheap way to check every branch.

```
18 .loaded(["Golden Hour", "Night Owl"]), .failed("Offline")]
```

WHO The enum cases with associated values.
WHAT **Adds a loaded state with two songs and a failed state.**
WHERE Continuation of the array literal.
WHEN While the array is built.
WHY Associated values are supplied right where the value is created.

```
19 for state in timeline {
```

WHO — A for-in loop over the timeline array, binding each value to state.
WHAT — **Visits the four states in order, so the output reads like a screen's history.**
WHERE — Top-level code; state is valid only inside the loop body.
WHEN — Four passes, one per element of timeline.
WHY — In an app, each state would replace the previous one in a single @State property.

```
20 print(state.message)
```

WHO — print and the computed property.
WHAT — **Prints the message for each state.**
WHERE — Inside the loop body, reading the computed property of the current state.
WHEN — Once per state, so four lines of output appear.
WHY — The screen never has to combine flags to decide what to show.

> **Try it**
>
> 1. Add a computed property isBusy that is true only for .loading, and print it for each state.
> 2. Add a branch so .loaded with an empty array shows "No songs yet". Where must it go?

CHECK YOURSELF

Check your understanding

1. You add `case extraLarge` to `CoffeeSize`. The `ounces(for:)` switch has no `default`. What happens?
   A. The function returns 0 for the new case
   B. A compile error: the switch is no longer exhaustive
   C. The app crashes the first time extraLarge is used
   D. Nothing; Swift ignores cases it has not seen

2. A download either succeeds with the file's bytes or fails with a message. Which enum feature models this best?
   A. Associated values, because each result carries its own data
   B. Raw values, because each case needs a fixed number
   C. CaseIterable, so you can list every result
   D. Two optional properties, data and errorMessage, on a struct

3. What does this print?

```
enum Level: Int { case low = 1, mid, high }
print(Level(rawValue: 3) ?? .low, Level.mid.rawValue)
```

   A. low 2
   B. Optional(high) 2
   C. high 2
   D. high 1

4. Why model a screen with `enum LoadState` instead of `isLoading`, `hasError` and `hasData` Bools?
    A. Enums use less memory than three Bools
    B. SwiftUI cannot observe Bool properties
    C. Enums make the network request itself faster
    D. Contradictory states cannot exist

Answers: Enums: Modeling Choices and State

1. **B. A compile error: the switch is no longer exhaustive** Swift checks switch exhaustiveness at compile time, so the build fails and points at the switch. That is a feature: the compiler finds every place the new size needs handling. A `default` branch would have hidden the problem.
2. **A. Associated values, because each result carries its own data** Associated values attach different data to each case: bytes for success, text for failure. Raw values are fixed per case, so they cannot hold per-download data. Two optionals allow impossible states, such as both being set at once.
3. **C. high 2** Raw values count up from 1, so `mid` is 2 and `high` is 3. `Level(rawValue: 3)` returns an optional `.high`, and `??` unwraps it, so no `Optional(...)` appears in the output.
4. **D. Contradictory states cannot exist** Three Bools allow 8 combinations and only 4 are meaningful. The enum allows exactly the meaningful ones, so the compiler rules out contradictory screens. Memory and speed are not the reason; correctness is.

Protocols and Protocol-Oriented Design

CONCEPT

Protocols are contracts

A **protocol** lists requirements: properties and methods that a type must provide. A type **conforms** to a protocol by declaring it and meeting every requirement, and the compiler checks the promise. Code can then work with *any* conforming type without knowing which one it has.

You have already relied on protocols. `CustomStringConvertible` told `print` how to show your type, and every SwiftUI screen conforms to the `View` protocol by providing a `body`.

Requirement	A property or method signature the protocol demands, with no body.
Conformance	A type's promise, written after a colon, to meet every requirement.
any Protocol	A box that can hold a value of any conforming type, such as [any Priced].
Protocol extension	Shared code, including default implementations, for every conforming type.

> **Protocols everywhere**
>
> SwiftUI (View, App), JSON (Codable), lists (Identifiable) and Swift Testing all rest on protocols. Learning to read a protocol's requirements is learning to read Apple's documentation.

CODE · RUNS IN THE BROWSER

Defining and adopting a protocol

A café cart holds coffees and pastries, two different types. The `Priced` protocol says what the cart needs from anything it holds: a name and a price. Notice that one type computes its price and the other stores it; both satisfy the same `{ get }` requirement.

```
protocol Priced {
    var name: String { get }
    var price: Double { get }
}

struct Coffee: Priced {
    let name: String
    let shots: Int
    var price: Double { 3.0 + Double(shots) * 0.75 }
}

struct Pastry: Priced {
    let name: String
    let price: Double
}

let cart: [any Priced] = [Coffee(name: "Flat white", shots: 2),
                          Pastry(name: "Croissant", price: 3.25)]
for item in cart {
    print(item.name, item.price)
}
```

OUTPUT

```
Flat white 4.5
Croissant 3.25
```

Line by line

1 `protocol Priced {`

- WHO You, declaring a protocol.
- WHAT **Starts a contract named Priced.**
- WHERE Top level of the file.
- WHEN Compile time; a protocol has no runtime code of its own.
- WHY Protocols named for a capability often end in -able or describe a quality, like Priced.

2 `var name: String { get }`

- WHO The Priced protocol, stating its first requirement.
- WHAT **Requires a readable String property called name.**
- WHERE Inside the protocol body, the first of two requirements.
- WHEN Compile time; checked for every conforming type.
- WHY { get } means readable. A let, a var, or a computed property all satisfy it.

3 `var price: Double { get }`

- WHO The Priced protocol, stating its second requirement.
- WHAT **Requires a readable Double called price.**
- WHERE Inside the protocol body, the last requirement before the closing brace.
- WHEN Compile time; the compiler checks every adopting type against this line.
- WHY The protocol says what is needed, not how it is produced.

6 `struct Coffee: Priced {`

- WHO You, adopting the protocol.
- WHAT **Declares Coffee and promises it meets every Priced requirement.**
- WHERE Top level of the file.
- WHEN Compile time; a missing requirement is an error here.
- WHY Structs, classes and enums can all conform to protocols.

7 `let name: String`

- WHO Every Coffee value, which stores its own name to satisfy the first requirement.
- WHAT **A stored constant that satisfies the name requirement.**
- WHERE Stored inside each Coffee value, first of its two properties.
- WHEN Assigned once by the memberwise initializer and never changed.
- WHY A stored let fully satisfies a { get } requirement.

8 `let shots: Int`

- WHO Every Coffee value, which stores its own shot count for the price getter.
- WHAT **A property the protocol does not mention.**
- WHERE Stored inside each Coffee value, below name.
- WHEN Assigned once by the memberwise initializer, then read every time price is computed.
- WHY Conforming types may add anything else they need.

```
9  var price: Double { 3.0 + Double(shots) * 0.75 }
```

WHO Every Coffee value, answering the price requirement with a computed property.
WHAT **A computed property that satisfies the price requirement.**
WHERE Inside the struct; no storage.
WHEN Each time price is read.
WHY A computed property satisfies { get } just as well as a stored one.

```
12 struct Pastry: Priced {
```

WHO You, adopting the same protocol in a different type.
WHAT **Declares Pastry as Priced.**
WHERE Top level of the file.
WHEN Compile time; the compiler checks Pastry against both Priced requirements.
WHY Unrelated types can share a protocol without sharing a superclass.

```
13 let name: String
```

WHO Every Pastry value, which stores its own name.
WHAT **A stored constant that satisfies the name requirement, just as Coffee's did.**
WHERE Stored inside each Pastry value.
WHEN Assigned once by the memberwise initializer.
WHY Satisfies the name requirement.

```
14 let price: Double
```

WHO Every Pastry value, which stores its price directly instead of computing it.
WHAT **A stored constant that satisfies the price requirement; a stored let is enough for { get }.**
WHERE Stored inside each Pastry value, below name.
WHEN Assigned once by the memberwise initializer, the only way a Pastry is created here.
WHY Pastries have a fixed price, so storing it is simplest.

```
17 let cart: [any Priced] = [Coffee(name: "Flat white", shots: 2),
```

WHO Your code, using an existential type.
WHAT **Creates an array whose elements can be any Priced type. Starts with a coffee.**
WHERE A top-level constant holding an array of any Priced, a mixed array.
WHEN At run time, as the array literal is evaluated.
WHY any Priced is a box that can hold different concrete types side by side.

```
18 Pastry(name: "Croissant", price: 3.25)]
```

WHO The Pastry memberwise initializer.
WHAT **Adds a pastry to the same array.**
WHERE The second element of the cart array, a Pastry stored beside a Coffee.
WHEN While the array is built.
WHY Without the protocol, one array could not hold both types safely.

```
19 for item in cart {
```

WHO A for-in loop over the cart array, binding each element to item.
WHAT **Visits each item, typed as any Priced.**
WHERE Top-level code; inside the loop item has the type any Priced.
WHEN Twice, once per item in the cart.
WHY Inside the loop only the protocol's requirements are available, not shots.

```
20 print(item.name, item.price)
```

WHO	Each item, answering through the protocol.
WHAT	**Prints Flat white 4.5, then Croissant 3.25.**
WHERE	Inside the loop body, reaching name and price only through the protocol.
WHEN	Once per item, two times in total.
WHY	The loop works for any future Priced type, such as a sandwich, without changes.

Try it

1. Add struct Sandwich: Priced with a name and a fixed price, and put one in the cart.
2. Inside the loop, try printing item.shots. Why does the compiler refuse?

CODE · RUNS IN THE BROWSER

Default behavior with protocol extensions

A **protocol extension** adds code to every conforming type at once. When it implements a requirement, that becomes a **default implementation**: types get it for free but may provide their own. This is the heart of protocol-oriented programming: small capabilities, shared behavior, no superclass.

```
protocol Shareable {
    var title: String { get }
    var shareText: String { get }
}

extension Shareable {
    var shareText: String { "Check out \(title)" }
}

struct Song: Shareable { let title: String }

struct Recipe: Shareable {
    let title: String
    let minutes: Int
    var shareText: String { "\(title), ready in \(minutes) min" }
}

let items: [any Shareable] = [Song(title: "Night Owl"),
                              Recipe(title: "Pad thai", minutes: 25)]
for item in items {
    print(item.shareText)
}
```

OUTPUT

```
Check out Night Owl
Pad thai, ready in 25 min
```

Line by line

```
1  protocol Shareable {
```

WHO You, declaring a protocol.
WHAT **Starts a capability for anything the app can share.**
WHERE Top level of the file.
WHEN Compile time; the compiler records Shareable and its two requirements.
WHY Small, focused protocols are easier to adopt than one giant one.

```
2      var title: String { get }
```

WHO The Shareable protocol, stating its first requirement.
WHAT **Requires a readable title.**
WHERE Inside the protocol body, first requirement.
WHEN Compile time; every adopting type must supply a readable title.
WHY The default share text below needs a title to work with.

```
3      var shareText: String { get }
```

WHO The Shareable protocol, stating the requirement the extension below will default.
WHAT **Requires a readable shareText property, so every Shareable has something to share.**
WHERE Inside the protocol body, last requirement.
WHEN Compile time; the compiler checks each adopting type against it.
WHY Declaring it as a requirement lets conforming types replace the default and still be called correctly through any Shareable.

```
6  extension Shareable {
```

WHO You, extending the protocol itself.
WHAT **Opens a block of code that every Shareable type receives.**
WHERE Top level of the file; applies to all conformers, present and future.
WHEN Compile time; the extension attaches the default to the protocol itself.
WHY Extensions let you share behavior without inheritance.

```
7      var shareText: String { "Check out \(title)" }
```

WHO The extension, providing a default implementation.
WHAT **Implements shareText using the required title.**
WHERE Inside the protocol extension.
WHEN Each time shareText is read on a type that has no version of its own.
WHY Default implementations remove boilerplate from every conforming type.

```
10 struct Song: Shareable { let title: String }
```

WHO You, adopting the protocol with the minimum.
WHAT **Declares Song with only a title, on one line.**
WHERE Top level of the file.
WHEN Compile time; the compiler sees that Song supplies title and accepts the extension's shareText.
WHY Song satisfies shareText through the default, so it writes nothing more.

```
12 struct Recipe: Shareable {
```

WHO You, adopting the protocol and customizing it.
WHAT **Declares Recipe as Shareable.**
WHERE Top level of the file.
WHEN Compile time; the compiler checks Recipe against both requirements.
WHY Recipes deserve richer share text, so this type will replace the default.

```
13 let title: String
```

WHO Every Recipe value, which stores its own title.
WHAT **Stored title, satisfying the requirement.**
WHERE Stored inside each Recipe value, first of two properties.
WHEN Assigned once by the memberwise initializer.
WHY Same requirement, different type.

```
14 let minutes: Int
```

WHO Every Recipe value, which stores the cooking time its own shareText mentions.
WHAT **Extra stored data for cooking time.**
WHERE Stored inside each Recipe value, below title.
WHEN Assigned once by the memberwise initializer.
WHY Conforming types can carry any additional data.

```
15 var shareText: String { "\(title), ready in \(minutes) min" }
```

WHO The Recipe type, supplying its own shareText instead of the default.
WHAT **Provides its own shareText, which replaces the default for recipes.**
WHERE Inside the Recipe struct, below the two stored properties it reads.
WHEN Each time shareText is read on a recipe.
WHY A type's own implementation always wins over the protocol extension's default.

```
18 let items: [any Shareable] = [Song(title: "Night Owl"),
```

WHO Your code, building a mixed array of two Shareable values.
WHAT **Starts an array of mixed Shareable values with a song.**
WHERE A top-level constant holding an array of any Shareable.
WHEN At run time, as the array literal is evaluated.
WHY One list, two types, one shared capability.

```
19 Recipe(title: "Pad thai", minutes: 25)]
```

WHO The memberwise initializer of Recipe, given a title and a time.
WHAT **Adds a recipe and closes the array.**
WHERE The second element of the items array, stored beside the Song.
WHEN While the array is built.
WHY The array element type is any Shareable, so either kind fits.

```
20 for item in items {
```

WHO A for-in loop over the items array, binding each element to item.
WHAT **Visits the Song, then the Recipe, binding each to item in turn.**
WHERE Top-level code; inside the loop item has the type any Shareable.
WHEN Twice, once per shareable item.
WHY The loop asks each item for shareText without knowing its type.

```
21 print(item.shareText)
```

WHO Dynamic dispatch through the protocol requirement.
WHAT **Prints the default text for the song and the custom text for the recipe.**
WHERE Inside the loop body, reading shareText through the protocol.
WHEN Once per item, two times in total; the default runs first, the override second.
WHY Because shareText is a requirement, each item's own version is used, even through any Shareable.

Try it

1. Give Song its own shareText that says "Listen to" instead. Run it again.
2. Add a method func share() { print(shareText) } to the extension and call item.share() in the loop.

CONCEPT

Standard protocols you will use every day

The standard library defines a few protocols that nearly every app type adopts. For many of them the compiler writes the code for you, called **synthesis**, as long as every stored property already conforms.

Equatable	== compares two values. Needed by contains and firstIndex(of:), and by SwiftUI to spot changes.
Hashable	Produces a hash so values can go in a Set or be Dictionary keys. Includes Equatable.
Comparable	< orders values, so sorted() and max() work. You usually write < yourself.
Identifiable	A stable id property, so SwiftUI lists can track rows as they move.
Codable	Converts to and from JSON and other formats. It is Encodable plus Decodable.

> **Protocol-oriented programming**
>
> Apple calls this style protocol-oriented programming: describe capabilities as small protocols, give them default behavior in extensions, and let value types adopt exactly the capabilities they need. It replaces deep class hierarchies with flat, composable pieces.

CODE · RUNS IN THE BROWSER

Equatable, Hashable and Comparable

Declaring `Hashable` gives `TodoItem` both `==` and hashing, written by the compiler. `Comparable` needs one function from you: `<`, which defines what "comes first" means. Here, earlier due dates come first.

```swift
struct TodoItem: Hashable, Comparable {
    let title: String
    let dueDay: Int

    static func < (lhs: TodoItem, rhs: TodoItem) -> Bool {
        lhs.dueDay < rhs.dueDay
    }
}

let rent = TodoItem(title: "Pay rent", dueDay: 1)
let copy = TodoItem(title: "Pay rent", dueDay: 1)
let call = TodoItem(title: "Call the bank", dueDay: 5)
let gym = TodoItem(title: "Renew gym", dueDay: 3)
print(rent == copy)

let unique: Set<TodoItem> = [rent, copy, call, gym]
print(unique.count)

let ordered = [call, rent, gym].sorted()
print(ordered.map { $0.title })
```

OUTPUT

```
true
3
["Pay rent", "Renew gym", "Call the bank"]
```

Line by line

1 `struct TodoItem: Hashable, Comparable {`

- WHO — You, adopting two standard protocols.
- WHAT — **Declares TodoItem as Hashable (which includes Equatable) and Comparable.**
- WHERE — Top level of the file.
- WHEN — Compile time; the compiler synthesizes == and hash(into:).
- WHY — Synthesis works because String and Int are already Hashable.

2 `let title: String`

- WHO — Every TodoItem value, which stores its own title; Hashable compares and hashes it.
- WHAT — **A stored constant holding the task name, one of the two fields that == and hash(into:) use.**
- WHERE — Stored inside each TodoItem value, first of its two properties.
- WHEN — Assigned once by the memberwise initializer and never changed.
- WHY — Synthesized == compares every stored property, including this one.

3 `let dueDay: Int`

- WHO — Every TodoItem value, which stores the day the < operator compares.
- WHAT — **Stored day of the month the task is due.**
- WHERE — Stored inside each TodoItem value, below title.
- WHEN — Assigned once by the memberwise initializer; read by every comparison and hash.
- WHY — The < function below sorts by this property.

5 `static func < (lhs: TodoItem, rhs: TodoItem) -> Bool {`

- WHO — You, implementing Comparable's requirement.
- WHAT — **Defines the < operator for two TodoItems: left-hand side and right-hand side.**
- WHERE — A static function on the type, because operators belong to types.
- WHEN — Each time two items are compared, for example while sorting.
- WHY — Comparable cannot guess your ordering, so this one requirement is yours to write.

6 `lhs.dueDay < rhs.dueDay`

- WHO — The body of the < operator, comparing the two items it was given.
- WHAT — **Says one item comes before another when it is due earlier.**
- WHERE — Inside the static function.
- WHEN — Each comparison that sorted() makes, several times for three items.
- WHY — With < defined, Swift derives >, <= and >= automatically.

10 `let rent = TodoItem(title: "Pay rent", dueDay: 1)`

- WHO — The memberwise initializer, creating the first of four items.
- WHAT — **Creates a task due on day 1.**
- WHERE — A top-level constant; this is the item that copy duplicates and that unique keeps.
- WHEN — At run time, the first statement after the struct is declared.
- WHY — Sample data for the comparisons below.

```
11 let copy = TodoItem(title: "Pay rent", dueDay: 1)
```

WHO The memberwise initializer, given exactly the same arguments as rent.
WHAT **Creates a second, separate value with identical properties.**
WHERE A top-level constant, a separate value with identical contents to rent.
WHEN At run time, right after rent; the two values are equal but independent.
WHY Equal values, not the same object: structs have no identity.

```
12 let call = TodoItem(title: "Call the bank", dueDay: 5)
```

WHO The memberwise initializer, creating an item due on day 5.
WHAT **A task due on day 5.**
WHERE A top-level constant; this item sorts last because it has the largest dueDay.
WHEN At run time, the third item created.
WHY The latest due date in the sample.

```
13 let gym = TodoItem(title: "Renew gym", dueDay: 3)
```

WHO The memberwise initializer, creating an item due on day 3.
WHAT **A task due on day 3.**
WHERE A top-level constant; this item lands in the middle when sorted.
WHEN At run time, the last of the four items created.
WHY Falls between the other two when sorted.

```
14 print(rent == copy)
```

WHO The synthesized == from Equatable.
WHAT **Prints true because every stored property matches.**
WHERE Top-level code; the console shows true, the result of the synthesized ==.
WHEN At run time, the first line of output.
WHY Equality of values is exactly what you want for data like tasks.

```
16 let unique: Set<TodoItem> = [rent, copy, call, gym]
```

WHO Set, using the synthesized hashing.
WHAT **Builds a set from four values; the duplicate is stored only once.**
WHERE A top-level constant holding a Set, which calls hash(into:) on each item as it is built.
WHEN At run time, as the set literal is evaluated; the duplicate is dropped here.
WHY Sets require Hashable so they can find duplicates quickly.

```
17 print(unique.count)
```

WHO The Set's count property, reporting how many distinct items survived.
WHAT **Prints 3, because rent and copy are equal.**
WHERE Top-level code; the console shows 3, not 4.
WHEN At run time, the second line of output, after the set has removed copy.
WHY Print the count, not the set: set order is not guaranteed.

```
19 let ordered = [call, rent, gym].sorted()
```

WHO sorted(), using your < function.
WHAT **Returns a new array ordered by due day.**
WHERE A top-level constant holding the sorted array; sorted() needs Comparable.
WHEN At run time; sorted() calls your < operator here, several times.
WHY sorted() with no arguments works only because TodoItem is Comparable.

```
20 print(ordered.map { $0.title })
```

WHO	map, transforming each item to its title.
WHAT	**Prints the titles in due order.**
WHERE	Top-level code; the console shows the three titles in due-day order.
WHEN	The last statement of the program, after the sort has finished.
WHY	Printing just the titles keeps the output readable.

> **Try it**
>
> 1. Change the < function so later due days come first. Predict the new order.
> 2. Print [call, rent, gym].max()?.title. Which task is the maximum, and why?

CODE · RUNS IN THE BROWSER

Identifiable and Codable

Apps receive data as **JSON** (JavaScript Object Notation, the text format most web APIs use). Declaring `Codable` lets `JSONDecoder` and `JSONEncoder` convert between JSON and your struct with no hand-written parsing. `Identifiable` just requires an `id`, which SwiftUI lists use to track rows.

```
1  import Foundation
2
3  struct Song: Codable, Identifiable {
4      let id: Int
5      let title: String
6      let artist: String
7  }
8
9  let json = """
10 {"id": 7, "title": "Night Owl", "artist": "The Late Trains"}
11 """
12 let decoder = JSONDecoder()
13 let song = try! decoder.decode(Song.self, from: Data(json.utf8))
14 print(song.title, "by", song.artist)
15
16 let encoder = JSONEncoder()
17 encoder.outputFormatting = [.prettyPrinted, .sortedKeys]
18 let data = try! encoder.encode(song)
19 print(String(decoding: data, as: UTF8.self))
```

OUTPUT

```
Night Owl by The Late Trains
{
  "artist" : "The Late Trains",
  "id" : 7,
  "title" : "Night Owl"
}
```

Line by line

```
1  import Foundation
```

WHO The compiler, loading Foundation so JSONDecoder, JSONEncoder and Data are available.
WHAT **Loads Foundation, home of JSONDecoder, JSONEncoder and Data.**
WHERE First line of the file.
WHEN Compile time, before any line of the program runs.
WHY JSON support is part of Foundation, not the core standard library.

```
3  struct Song: Codable, Identifiable {
```

WHO You, adopting two protocols.
WHAT **Declares a Song that can be encoded, decoded and identified.**
WHERE Top level of the file.
WHEN Compile time; the compiler synthesizes the encoding and decoding code.
WHY Synthesis works because Int and String are already Codable.

```
4  let id: Int
```

WHO Every Song value, which carries the id that Identifiable requires.
WHAT **Stored id, which also satisfies Identifiable.**
WHERE Stored inside each Song value, first property, matched to the JSON key id.
WHEN Filled in from the JSON key id.
WHY Property names must match the JSON keys for the synthesized decoder to find them.

```
5  let title: String
```

WHO Every Song value, which stores the title decoded from JSON.
WHAT **Stored title, read from the key title.**
WHERE Stored inside each Song value, matched to the JSON key title.
WHEN Filled in during decoding.
WHY A missing key would make decoding throw an error.

```
6  let artist: String
```

WHO Every Song value, which stores the artist decoded from JSON.
WHAT **Stored artist, read from the key artist.**
WHERE Stored inside each Song value, matched to the JSON key artist.
WHEN Filled in during decoding.
WHY Using let keeps decoded data from being changed by accident.

```
9  let json = """
```

WHO A multi-line string literal.
WHAT **Starts a string that spans lines, standing in for a server response.**
WHERE A top-level constant holding a multi-line String of JSON text.
WHEN At run time, when the literal is evaluated; the text is fixed in the source.
WHY Triple quotes let JSON keep its own double quotes without escaping.

```
10 {"id": 7, "title": "Night Owl", "artist": "The Late Trains"}
```

WHO The string literal, supplying the one JSON object the decoder will read.
WHAT **The JSON text itself: one object with three keys.**
WHERE Content of the multi-line string.
WHEN Part of the same literal.
WHY Real apps receive text like this from URLSession, as the networking module shows.

```
11 """
```

WHO The string literal, which ends at these closing triple quotes.
WHAT **Closes the multi-line string.**
WHERE End of the literal.
WHEN Part of the same literal.
WHY The closing quotes on their own line keep the JSON exact.

```
12 let decoder = JSONDecoder()
```

WHO Foundation's JSONDecoder class, created with its default settings.
WHAT **Creates a JSON decoder.**
WHERE A top-level constant holding the decoder object.
WHEN At run time, right before the decode call that uses it.
WHY One decoder can decode many values.

```
13 let song = try! decoder.decode(Song.self, from: Data(json.utf8))
```

WHO JSONDecoder, using the synthesized Decodable code.
WHAT **Converts the text to bytes, then decodes a Song. Song.self names the type to build.**
WHERE A top-level constant holding the decoded value.
WHEN At run time; the decoder reads the JSON bytes here and builds the Song.
WHY try! means "crash if this fails". It is acceptable only for known-good sample data; the errors lesson shows safer options.

```
14 print(song.title, "by", song.artist)
```

WHO print, with three items joined by spaces.
WHAT **Prints Night Owl by The Late Trains.**
WHERE Top-level code; the console shows the title and artist that were decoded.
WHEN Right after decoding, the first line of output.
WHY Proves the JSON landed in real, typed properties.

```
16 let encoder = JSONEncoder()
```

WHO Foundation's JSONEncoder class, created with its default settings.
WHAT **Creates a JSON encoder.**
WHERE A top-level constant holding the encoder object.
WHEN At run time, after the decoded song has been printed.
WHY Encoding is the reverse trip: struct to JSON.

```
17 encoder.outputFormatting = [.prettyPrinted, .sortedKeys]
```

WHO The encoder's outputFormatting setting, changed before any encoding happens.
WHAT **Asks for indented output with keys in alphabetical order.**
WHERE Configures this encoder only.
WHEN Before encode runs on the next line, so the setting applies to this output.
WHY Sorted keys make the output identical on every run, which helps tests and diffs.

```
18 let data = try! encoder.encode(song)
```

WHO JSONEncoder, using the synthesized Encodable code.
WHAT **Turns the Song back into JSON bytes.**
WHERE A top-level constant of type Data.
WHEN At run time; the encoder turns the Song back into JSON bytes here.
WHY Encoding a Codable struct of strings and numbers cannot fail, so try! is safe here.

```
19 print(String(decoding: data, as: UTF8.self))
```

WHO String, decoding bytes as UTF-8 text.
WHAT **Prints the pretty JSON.**
WHERE Top-level code; the console shows the pretty-printed JSON with sorted keys.
WHEN The last statement of the program, after the Data has been converted to text.
WHY Data holds raw bytes; converting to String lets you read it.

> **Try it**
>
> 1. Add a property let year: Int to Song without changing the JSON. What happens, and why?
> 2. Remove .sortedKeys from outputFormatting and run it a few times.

COMPARE

some versus any

Both keywords put a protocol where a type is expected, but they promise different things. `some Shape` means "one specific type that conforms, which I am not naming". `any Shape` means "a box that may hold any conforming type, and can change". SwiftUI's `some View` is the first kind.

A: some: one hidden concrete type

```swift
protocol Shape {
    func area() -> Double
}

struct Square: Shape {
    let side: Double
    func area() -> Double { side * side }
}

func makeBadge() -> some Shape {
    Square(side: 3)
}

let badge = makeBadge()
print(badge.area())
```

OUTPUT

```
9.0
```

```
1 protocol Shape {
```

WHO You, declaring a protocol.
WHAT **Starts a protocol for things with an area.**
WHERE Top level of the file.
WHEN Compile time; the compiler records Shape and its single requirement.
WHY Both sides use the same protocol so only some and any differ.

```
2  func area() -> Double
```

WHO The Shape protocol, stating its one requirement.
WHAT **Requires a method that returns an area.**
WHERE Inside the protocol body, the only requirement.
WHEN Compile time; every adopting type must supply this method.
WHY Method requirements have no body in the protocol.

```
5  struct Square: Shape {
```

WHO You, conforming a struct.
WHAT **Declares Square as a Shape.**
WHERE Top level of the file.
WHEN Compile time; the compiler checks that Square supplies area().
WHY The concrete type that will hide behind some Shape.

```
6  let side: Double
```

WHO Every Square value, which stores the side length area() reads.
WHAT **A stored constant holding the side length of this square.**
WHERE Stored inside each Square value, its only property.
WHEN Assigned once by the memberwise initializer.
WHY All the data a square needs.

```
7  func area() -> Double { side * side }
```

WHO Every Square value, answering the area() requirement.
WHAT **Implements area() by squaring the side, satisfying the Shape protocol.**
WHERE Inside the Square struct, below the side property it reads.
WHEN Each time area() is called.
WHY Meets the contract, so Square is a Shape.

```
10 func makeBadge() -> some Shape {
```

WHO You, declaring an opaque return type.
WHAT **The function returns one particular Shape type without revealing which.**
WHERE Top level of the file.
WHEN Each call to makeBadge, once in this program.
WHY Callers get Shape abilities; you stay free to change the concrete type later.

```
11 Square(side: 3)
```

WHO The body of makeBadge, building the one concrete Shape it ever returns.
WHAT **Returns a Square. The compiler knows the exact type even though callers do not.**
WHERE Inside makeBadge, its single-expression body.
WHEN Each call to makeBadge; the compiler already knows the result is a Square.
WHY Every return path must produce the same concrete type, the defining rule of some.

```
14 let badge = makeBadge()
```

WHO Your code, calling makeBadge and keeping its result.
WHAT **Stores the returned shape.**
WHERE A top-level constant of an opaque type.
WHEN At run time, after the protocol and struct are declared.
WHY No box is involved, so calls are as fast as on a Square.

15 `print(badge.area())`

WHO print, calling area() on the opaque value.
WHAT **Prints 9.0, the area of a square with side 3, via a direct call with no boxing.**
WHERE Top-level code; the console shows a Double.
WHEN The last statement of the left program.
WHY Only Shape members are visible on badge, not side.

B: any: a box for any conforming type

```swift
protocol Shape {
    func area() -> Double
}

struct Square: Shape {
    let side: Double
    func area() -> Double { side * side }
}
struct Circle: Shape {
    let radius: Double
    func area() -> Double { Double.pi * radius * radius }
}

var shapes: [any Shape] = [Square(side: 3)]
shapes.append(Circle(radius: 1))
print(shapes.map { $0.area() })
```

OUTPUT

```
[9.0, 3.141592653589793]
```

1 `protocol Shape {`

WHO You, declaring a protocol.
WHAT **The same Shape protocol.**
WHERE Top level of the file.
WHEN Compile time; the compiler records the same Shape protocol as on the left.
WHY The protocol is unchanged on purpose: the difference between the two sides is how values are stored, not what Shape requires.

2 `func area() -> Double`

WHO The Shape protocol, stating its one requirement.
WHAT **Requires an area method.**
WHERE Inside the protocol body, the only requirement.
WHEN Compile time; both Square and Circle must supply this method.
WHY Any conforming type must implement it.

5 `struct Square: Shape {`

WHO You, conforming a struct to the protocol.
WHAT **Declares Square as a Shape.**
WHERE Top level of the file.
WHEN Compile time; the compiler checks that Square supplies area().
WHY First of two conforming types.

```
6   let side: Double
```

WHO Every Square value, which stores the side length area() reads.
WHAT **A stored constant holding the side length of this square.**
WHERE Stored inside each Square value, its only property.
WHEN Assigned once by the memberwise initializer.
WHY Square keeps the same shape as on the left so the only new thing to read is Circle and the any Shape array.

```
7   func area() -> Double { side * side }
```

WHO Every Square value, answering the area() requirement.
WHAT **Implements area() by squaring the side, just as on the left.**
WHERE Inside the Square struct, below the side property.
WHEN Each call to area(), once for the Square when map runs.
WHY The any Shape box calls this method through the protocol, so Square must still provide it.

```
9   struct Circle: Shape {
```

WHO You, conforming a second struct.
WHAT **Declares a second, different Shape type.**
WHERE Top level of the file.
WHEN Compile time; a second conforming type, which is what makes any Shape necessary.
WHY Mixing types is what any is for.

```
10  let radius: Double
```

WHO Every Circle value, which stores the radius area() reads.
WHAT **A stored constant holding the radius of this circle.**
WHERE Stored inside each Circle value, its only property.
WHEN Assigned once by the memberwise initializer.
WHY Different data from Square, same protocol.

```
11  func area() -> Double { Double.pi * radius * radius }
```

WHO Every Circle value, answering the area() requirement with a different formula.
WHAT **Implements area with pi.**
WHERE Inside the Circle struct, below the radius property.
WHEN Each call to area(), once for the Circle when map runs.
WHY Double.pi is the standard library's value of pi.

```
14  var shapes: [any Shape] = [Square(side: 3)]
```

WHO Your code, using an existential array.
WHAT **Creates a variable array of boxes, starting with a square.**
WHERE A top-level var holding an array of any Shape, so the element type is a box, not Square.
WHEN At run time, as the array literal is evaluated; var so append is allowed below.
WHY Each element is boxed, so elements may be different concrete types.

```
15  shapes.append(Circle(radius: 1))
```

WHO The shapes array, accepting a Circle because its element type is any Shape.
WHAT **Adds a circle next to the square.**
WHERE Top-level code, changing the array declared on the line above.
WHEN At run time, after the array exists; the Circle is boxed as it is stored.
WHY This is impossible with some: a some type is one fixed type.

```
16 print(shapes.map { $0.area() })
```

WHO	map, calling area through each box.
WHAT	**Prints both areas in one array.**
WHERE	Top-level code; the console shows an array of two Doubles.
WHEN	The last statement of the right program, after map has called area() on both boxes.
WHY	Each call is dispatched to the right type at run time.

> **Takeaway**
>
> Prefer `some` when one concrete type is involved: it is faster and keeps full type information, which is why SwiftUI uses `some View`. Use `any` when you truly need a mix of types in one variable or array, such as a cart of different priced items.

CHECK YOURSELF

Check your understanding

1. A protocol requires `var price: Double { get }`. Which of these satisfies it?
   A. Only a stored constant, let price
   B. Only a computed property, var price
   C. Any readable price: Double
   D. A method, func price() -> Double
2. In the Shareable example, you give `Song` its own `var shareText: String { "Listen to \(title)" }`. What prints for the song?
   A. Listen to Night Owl
   B. Check out Night Owl
   C. Both lines, one after the other
   D. A compile error: shareText is defined twice
3. A function always returns a `Square`, but you want callers to see only `Shape` abilities. Which return type fits best?
   A. any Shape
   B. Square?
   C. [any Shape]
   D. some Shape
4. Which protocol must a struct adopt so its values can be stored in a `Set`?
   A. Comparable
   B. Hashable
   C. Codable
   D. Identifiable

Answers: Protocols and Protocol-Oriented Design

1. **C. Any readable price: Double** `{ get }` only demands that the property can be read, so a `let`, a `var` or a computed property all qualify, as Coffee and Pastry showed. A method is a different kind of member and does not satisfy a property requirement.
2. **A. Listen to Night Owl** A conforming type's own implementation replaces the default from the protocol extension. Because `shareText` is a requirement, the replacement is used even through `any Shareable`. Defining it in the type and in an extension is legal; that is how defaults work.
3. **D. some Shape** `some Shape` hides the concrete type while keeping it fixed and fully known to the compiler, with no boxing cost. `any Shape` would also compile, but it boxes the value and allows different types that this function never returns.
4. **B. Hashable** A set uses hashes to find duplicates quickly, so its elements must be `Hashable`. `Comparable` is about ordering, which sets do not have, and `Identifiable` is about stable ids for lists.

Generics: Write It Once, Keep the Types

CONCEPT

Why generics

Suppose you write a function that returns the most recent searches, then need the same thing for song IDs. Copying the function for each type is tedious and error-prone. Using `[Any]` instead throws away type safety: callers get back values the compiler knows nothing about.

Generics solve both problems. A generic function or type has a **type parameter**, a placeholder written in angle brackets such as `<Item>`, that the compiler fills in with a real type at each use. You write the logic once, and every use stays fully type-checked.

Type parameter	A placeholder type name, such as Item or Element, used inside the generic code.
Constraint	A rule such as T: Comparable that limits which types may fill the placeholder.
Inference	Swift usually works out the real type from the arguments, so you rarely write it.

> **You already use generics**
>
> `Array<Element>`, `Dictionary<Key, Value>`, `Optional<Wrapped>` and `Result<Success, Failure>` are all generic standard library types. `[String]` is shorthand for `Array<String>`.

CODE · RUNS IN THE BROWSER

A generic function

One function serves a search history of Strings and a play history of Int song IDs. `Item` is the type parameter. At each call Swift infers what `Item` is from the argument, and the return type follows: Strings in, Strings out.

```
1  func recent<Item>(_ history: [Item], limit: Int) -> [Item] {
2      history.suffix(limit).reversed()
3  }
4
5  let searches = ["tacos", "sushi", "pho", "ramen"]
6  let songIDs = [101, 205, 309, 412, 515]
7
8  print(recent(searches, limit: 2))
9  print(recent(songIDs, limit: 3))
```

OUTPUT

```
["ramen", "pho"]
[515, 412, 309]
```

Line by line

```
1  func recent<Item>(_ history: [Item], limit: Int) -> [Item] {
```

WHO You, declaring a generic function.
WHAT **Declares recent with a type parameter Item. It takes an array of Item and returns an array of Item.**
WHERE Top level of the file.
WHEN The body runs each time recent is called, with Item fixed to a concrete type for that call.
WHY The same placeholder in the parameter and the return type ties them together: whatever goes in comes back out.

```
2  history.suffix(limit).reversed()
```

WHO Standard library collection methods.
WHAT **Takes the last limit items and reverses them, newest first.**
WHERE The single-expression body, returned implicitly.
WHEN Each call to recent, twice in this program, once with Strings and once with Ints.
WHY The body never inspects an Item, so it works for every type. That is what makes it safely generic.

```
5  let searches = ["tacos", "sushi", "pho", "ramen"]
```

WHO Your code, creating the first array the generic function will receive.
WHAT **An array of Strings, oldest first.**
WHERE A top-level constant holding an array of String.
WHEN At run time, after recent is declared.
WHY Sample data of one type.

```
6  let songIDs = [101, 205, 309, 412, 515]
```

WHO Your code, creating a second array with a different element type.
WHAT **An array of Ints, oldest first.**
WHERE A top-level constant holding an array of Int.
WHEN At run time, right after searches.
WHY Sample data of a completely different type.

```
8  print(recent(searches, limit: 2))
```

WHO The compiler, inferring Item as String.
WHAT **Prints the two newest searches: ramen, then pho.**
WHERE Top-level code; the console shows the last two searches, newest first.
WHEN At run time; the compiler fixed Item to String for this call.
WHY The result is a real [String], so String methods are available on it without casting.

```
9  print(recent(songIDs, limit: 3))
```

WHO The compiler, inferring Item as Int.
WHAT **Prints the three newest IDs, newest first.**
WHERE Top-level code; the console shows the last three IDs, newest first.
WHEN The last statement of the program; Item is Int for this call.
WHY Same function, a different type, no duplicated code.

Try it

1. Call recent with an array of Bools. Does it still compile?
2. Assign let top = recent(searches, limit: 1) and print top.first?.uppercased(). Why is uppercased() allowed?

CODE · RUNS IN THE BROWSER

A generic type: Stack

A **stack** is a last-in, first-out list, like screens in a navigation stack or edits in an undo history. `Stack<Element>` is written once; `Stack<String>` and `Stack<Int>` are two different, fully type-checked types made from it.

```
struct Stack<Element> {
    private var items: [Element] = []

    var top: Element? { items.last }
    var isEmpty: Bool { items.isEmpty }

    mutating func push(_ item: Element) {
        items.append(item)
    }

    mutating func pop() -> Element? {
        items.popLast()
    }
}

var screens = Stack<String>()
screens.push("Inbox")
screens.push("Message")
screens.push("Attachment")
print(screens.pop() ?? "none")
print(screens.top ?? "none")
var edits = Stack<Int>()
print(edits.isEmpty, edits.pop() == nil)
```

OUTPUT

```
Attachment
Message
true true
```

Line by line

1 `struct Stack<Element> {`

- WHO — You, declaring a generic struct.
- WHAT — **Declares Stack with a type parameter named Element.**
- WHERE — Top level of the file.
- WHEN — Compile time; each use such as Stack of String becomes its own concrete type.
- WHY — Element is the conventional name for the item type of a collection.

2 `private var items: [Element] = []`

- WHO — Each Stack value, hiding its storage.
- WHAT — **Stores the elements in an array. private means only code inside Stack can touch it.**
- WHERE — Inside each stack value.
- WHEN — Initialized to an empty array the moment a Stack is created.
- WHY — Hiding the array forces callers to use push and pop, so the stack rules cannot be broken.

```
4  var top: Element? { items.last }
```

WHO The Stack type, declaring a read-only computed property.
WHAT **A computed property returning the newest element, or nil when empty.**
WHERE Inside the Stack struct, below the private array it reads.
WHEN Each time top is read.
WHY An optional return is honest: an empty stack has no top.

```
5  var isEmpty: Bool { items.isEmpty }
```

WHO The Stack type, declaring a second computed property.
WHAT **Reports whether the stack holds nothing.**
WHERE Inside the Stack struct, next to top.
WHEN Each time it is read.
WHY Exposing a question, not the array, keeps the storage private.

```
7  mutating func push(_ item: Element) {
```

WHO The Stack type, declaring the method that adds an element.
WHAT **Adds an element on top. The parameter type is the placeholder Element.**
WHERE Inside the Stack struct, below the computed properties.
WHEN Each push call, three times in this program.
WHY mutating, because it changes the stored array.

```
8  items.append(item)
```

WHO The private items array of the stack push was called on.
WHAT **Appends the new element at the end, which is the top.**
WHERE Inside push, its only statement.
WHEN Each push call, immediately.
WHY The end of an array is the cheapest place to add and remove.

```
11 mutating func pop() -> Element? {
```

WHO The Stack type, declaring the method that removes the top element.
WHAT **Removes and returns the top element, or nil if there is none.**
WHERE Inside the Stack struct, below push.
WHEN Each pop call, twice in this program.
WHY Returning an optional avoids a crash when popping an empty stack.

```
12 items.popLast()
```

WHO The array's popLast method.
WHAT **Removes and returns the last element, or nil when empty.**
WHERE Inside pop, its single-expression body.
WHEN Each pop call; the array shrinks by one when it is not empty.
WHY popLast already has exactly the behavior a stack needs.

```
16 var screens = Stack<String>()
```

WHO Your code, specializing the generic type.
WHAT **Creates an empty stack of Strings.**
WHERE A top-level var holding a Stack whose Element is String.
WHEN At run time, when the empty stack is created.
WHY Writing the type in angle brackets fixes Element to String for this stack.

```
17 screens.push("Inbox")
```

WHO The mutating push method.
WHAT **Puts the first screen on the stack.**
WHERE Top-level code, changing the screens stack for the first time.
WHEN At run time; Inbox becomes the bottom of the stack.
WHY Like opening the first screen of a mail app.

```
18 screens.push("Message")
```

WHO The mutating push method.
WHAT **A second screen on top.**
WHERE Top-level code, pushing onto screens a second time.
WHEN At run time; Message now sits above Inbox.
WHY Like tapping into one message.

```
19 screens.push("Attachment")
```

WHO The mutating push method.
WHAT **A third screen on top.**
WHERE Top-level code, pushing onto screens a third time.
WHEN At run time; Attachment is now the top of the stack.
WHY The newest screen is now on top.

```
20 print(screens.pop() ?? "none")
```

WHO pop and the nil-coalescing operator ??.
WHAT **Removes and prints Attachment, the last screen pushed.**
WHERE Top-level code; the console shows Attachment, the element pop removed.
WHEN At run time, the first line of output.
WHY ?? supplies a fallback in case pop returns nil.

```
21 print(screens.top ?? "none")
```

WHO The top computed property, reading the last element without removing it.
WHAT **Prints Message, which is now on top.**
WHERE Top-level code; the console shows Message, now the top of the stack.
WHEN At run time, the second line of output, after one pop.
WHY Like tapping Back: the previous screen reappears.

```
22 var edits = Stack<Int>()
```

WHO Your code, specializing again.
WHAT **Creates an empty stack of Ints, a completely separate type from the String stack.**
WHERE A top-level var holding a second Stack, this time with Element fixed to Int.
WHEN At run time, after the String stack has been used.
WHY One definition, many safe concrete types.

```
23 print(edits.isEmpty, edits.pop() == nil)
```

WHO The isEmpty property and the pop method, both called on the empty Int stack.
WHAT **Prints true true: the stack is empty and popping returns nil.**
WHERE Top-level code; the console shows true true on one line.
WHEN The last statement of the program.
WHY The optional return makes the empty case safe instead of a crash.

> **Try it**
>
> 1. Try screens.push(42). Read the error: why can a String stack not hold an Int?
> 2. Add a computed property count that returns items.count.

CODE · RUNS IN THE BROWSER

Constraints and where clauses

Some generic code needs more than "any type". Finding the largest value needs `>`, which only `Comparable` types have. A **constraint** such as `T: Comparable` says so, and the compiler rejects types that do not qualify. A `where` clause expresses the same idea after the signature.

```
func largest<T: Comparable>(_ values: [T]) -> T? {
    guard var best = values.first else { return nil }
    for value in values where value > best {
        best = value
    }
    return best
}

func countMatches<T>(of target: T, in items: [T]) -> Int
    where T: Equatable {
    items.filter { $0 == target }.count
}

print(largest([72.5, 88.0, 64.25]) ?? 0)
print(largest(["kiwi", "apple", "mango"]) ?? "none")
let ratings = [5, 4, 5, 3, 5]
print(countMatches(of: 5, in: ratings))
```

OUTPUT

```
88.0
mango
3
```

Line by line

1 `func largest<T: Comparable>(_ values: [T]) -> T? {`

- WHO — You, declaring a constrained generic function.
- WHAT — **T may be any type, as long as it conforms to Comparable. Returns an optional T.**
- WHERE — Top level of the file.
- WHEN — Each call to largest, twice in this program, once with Doubles and once with Strings.
- WHY — The constraint is what allows the body to use > on two T values.

2 `guard var best = values.first else { return nil }`

- WHO — The guard statement, reading the first element and exiting if there is none.
- WHAT — **Starts best at the first value, or returns nil right away if the array is empty.**
- WHERE — First line of the function body.
- WHEN — Each call to largest, before the loop starts.
- WHY — guard handles the early exit up front, so the rest of the body can assume a value exists.

3 `for value in values where value > best {`

- WHO — A for-in loop with a where filter.
- WHAT — **Visits only the values bigger than the current best.**
- WHERE — Inside largest, looping over the whole array with a where filter.
- WHEN — Once per qualifying element.
- WHY — The > comparison compiles only because of the T: Comparable constraint.

```
4   best = value
```

WHO The loop body, updating the running maximum.
WHAT **Records the new largest value.**
WHERE Inside the loop, reached only when the where clause is true.
WHEN Each time a bigger value appears.
WHY best is a var for exactly this reason.

```
6   return best
```

WHO The largest function, handing back the winner.
WHAT **Returns the largest value, wrapped in an optional.**
WHERE End of the function.
WHEN After the loop has visited every element.
WHY The optional return type covers the empty-array case.

```
9   func countMatches<T>(of target: T, in items: [T]) -> Int
```

WHO You, declaring another generic function.
WHAT **Counts how many items equal target. The constraint follows on the next line.**
WHERE Top level of the file.
WHEN Each call to countMatches, once in this program.
WHY Long signatures read better with constraints moved into a where clause.

```
10  where T: Equatable {
```

WHO The where clause, adding the Equatable requirement to T.
WHAT **Requires T to be Equatable, so == is available.**
WHERE Part of the function signature.
WHEN Compile time, at each call site.
WHY where clauses can express several constraints, one after another, without crowding the angle brackets.

```
11  items.filter { $0 == target }.count
```

WHO The filter and count methods of the items array.
WHAT **Keeps the matching items and counts them.**
WHERE The single-expression body of countMatches, so the result is returned implicitly.
WHEN Each call to countMatches; filter runs the closure once per element.
WHY The == compiles because of the Equatable constraint.

```
14  print(largest([72.5, 88.0, 64.25]) ?? 0)
```

WHO The compiler, inferring T as Double.
WHAT **Prints 88.0, the warmest temperature.**
WHERE Top-level code; the console shows 88.0, the largest Double.
WHEN At run time, the first line of output.
WHY The fallback 0 is inferred as a Double too, so the whole expression is a Double.

```
15  print(largest(["kiwi", "apple", "mango"]) ?? "none")
```

WHO The compiler, inferring T as String.
WHAT **Prints mango: Strings compare alphabetically.**
WHERE Top-level code; the console shows mango, the largest String alphabetically.
WHEN At run time, the second line of output.
WHY The same function works for any Comparable type.

```
16 let ratings = [5, 4, 5, 3, 5]
```

WHO	Your code, creating the array that countMatches will search.
WHAT	**A list of star ratings.**
WHERE	A top-level constant holding an array of Int.
WHEN	At run time, before the final print.
WHY	Sample data for countMatches.

```
17 print(countMatches(of: 5, in: ratings))
```

WHO	The compiler, inferring T as Int.
WHAT	**Prints 3, the number of five-star ratings.**
WHERE	Top-level code; the console shows 3, the number of fives.
WHEN	The last statement of the program.
WHY	Int is Equatable, so the where clause is satisfied.

> **Try it**
>
> 1. Remove : Comparable from largest. Read the error about the > operator.
> 2. Call largest([true, false]). Why does the compiler refuse Bool?

CODE · RUNS IN THE BROWSER

Optional is a generic enum

Generics and enums combine beautifully. Swift's `Optional` is simply a generic enum with two cases: `none` and `some(Wrapped)`. Below, `Maybe` rebuilds it in a few lines, then the real `Optional` and `Array` are spelled out with their generic names.

```
1  enum Maybe<Wrapped> {
2      case none
3      case some(Wrapped)
4
5      func or(_ fallback: Wrapped) -> Wrapped {
6          switch self {
7          case .none: return fallback
8          case .some(let value): return value
9          }
10     }
11 }
12
13 let nickname: Maybe<String> = .none
14 let rating: Maybe<Int> = .some(5)
15 print(nickname.or("Guest"), rating.or(0))
16
17 let realNickname: Optional<String> = nil
18 let ratings: Array<Int> = [5, 4]
19 print(realNickname ?? "Guest", ratings)
```

OUTPUT

```
Guest 5
Guest [5, 4]
```

Line by line

1 `enum Maybe<Wrapped> {`

WHO	You, declaring a generic enum.
WHAT	**Declares Maybe with a type parameter Wrapped, mirroring the standard library's Optional.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler records Maybe as a generic enum with one placeholder.
WHY	Seeing Optional rebuilt shows there is no magic behind it, just an enum and a generic.

2 `case none`

WHO	The Maybe type, declaring its empty case.
WHAT	**The case meaning "no value".**
WHERE	Inside the enum body, first of two cases.
WHEN	Compile time; a case declaration runs no code.
WHY	Optional has exactly this case; nil is a shorthand for it.

3 `case some(Wrapped)`

WHO	The Maybe type, declaring the case that carries a value of type Wrapped.
WHAT	**The case meaning "a value is present", carrying it as an associated value.**
WHERE	Inside the enum body, the second case.
WHEN	Compile time; the payload arrives only when a some value is created.
WHY	The associated value has the generic type, so a Maybe of Int holds an Int.

5 `func or(_ fallback: Wrapped) -> Wrapped {`

WHO	The Maybe type, declaring a method every Maybe value can call.
WHAT	**A method that returns the wrapped value, or the fallback when there is none.**
WHERE	Inside the enum body, below the two cases.
WHEN	Each call to or, twice in this program.
WHY	This is the ?? operator, written as a method.

6 `switch self {`

WHO	The switch statement, inspecting the value the method was called on.
WHAT	**Checks which case this Maybe holds.**
WHERE	Inside the or method, as its first statement.
WHEN	Each call to or; the switch runs once per call.
WHY	Unwrapping an optional is pattern matching underneath.

7 `case .none: return fallback`

WHO	The switch statement, matching the empty case.
WHAT	**No value, so return the fallback.**
WHERE	The first branch of the switch, inside or.
WHEN	When the value is none.
WHY	Mirrors what ?? does with nil.

8 `case .some(let value): return value`

WHO	Pattern matching with a binding.
WHAT	**Extracts the wrapped value and returns it.**
WHERE	The second branch of the switch; with both cases covered, no default is needed.
WHEN	When a value is present.
WHY	The same pattern if let performs for you on real optionals.

```
13 let nickname: Maybe<String> = .none
```

WHO Your code, specializing the generic enum.
WHAT **A Maybe of String holding no value.**
WHERE A top-level constant whose type is Maybe with Wrapped fixed to String.
WHEN At run time, after the enum is declared.
WHY The annotation fixes Wrapped to String.

```
14 let rating: Maybe<Int> = .some(5)
```

WHO Your code, creating a some value that carries an Int.
WHAT **A Maybe of Int holding 5.**
WHERE A top-level constant whose type is Maybe with Wrapped fixed to Int.
WHEN At run time, right after nickname.
WHY Same enum, different Wrapped type.

```
15 print(nickname.or("Guest"), rating.or(0))
```

WHO Your or method, called once on each Maybe value.
WHAT **Prints Guest 5: the fallback for the empty one, the value for the full one.**
WHERE Top-level code; the console shows Guest 5 on one line.
WHEN At run time, the first line of output.
WHY Exactly how ?? behaves with real optionals.

```
17 let realNickname: Optional<String> = nil
```

WHO The standard library's Optional, written with its generic name.
WHAT **The same as writing String?. nil means the none case.**
WHERE A top-level constant declared with the long name Optional<String> instead of String?.
WHEN At run time; nil is the real Optional's equivalent of Maybe.none.
WHY String? is just shorthand for Optional of String.

```
18 let ratings: Array<Int> = [5, 4]
```

WHO The standard library's Array, written with its generic name.
WHAT **The same as writing [Int].**
WHERE A top-level constant declared with the long name Array<Int> instead of [Int].
WHEN At run time, right after realNickname.
WHY Square brackets are shorthand for the generic Array type.

```
19 print(realNickname ?? "Guest", ratings)
```

WHO The ?? operator and print.
WHAT **Prints Guest [5, 4].**
WHERE Top-level code; the console shows Guest [5, 4].
WHEN The last statement of the program.
WHY The real types behave exactly like their shorthand forms, because they are the same types.

> **Try it**
>
> 1. Add a computed property isEmpty to Maybe that returns true for .none.
> 2. Write let scores: Dictionary = ["Ana": 3] and print scores["Ana", default: 0].

CHECK YOURSELF

Check your understanding

1. In `recent(songIDs, limit: 3)`, where `songIDs` is `[Int]`, what does the compiler use for `Item`?
   A. Any
   B. Int
   C. [Int]
   D. Nothing: Item stays a placeholder while the app runs
2. Why does `largest` declare `T: Comparable`?
   A. Its body uses >, which needs Comparable
   B. Every generic function needs at least one constraint
   C. Constraints make generic code run faster
   D. So the function is allowed to return nil
3. What happens with `var tags = Stack<String>()` followed by `tags.push(42)`?
   A. 42 is converted and stored as the String "42"
   B. The app crashes when push runs
   C. A compile error: push expects a String, not an Int
   D. The stack silently becomes a stack of Any
4. Which statement about `[String]` and `Int?` is true?
   A. They are special built-in types that are not generic
   B. They are generic, but you cannot write their full names
   C. They are protocols, not types
   D. Both are shorthand for generic types

Answers: Generics: Write It Once, Keep the Types

1. **B. Int** Swift infers `Item` from the argument: an array of Int means `Item` is `Int`, so the result is `[Int]`. `[Int]` is the type of the whole array, not of one item, and generics never fall back to `Any`.
2. **A. Its body uses >, which needs Comparable** The body uses >, and the compiler only allows that on types it knows are `Comparable`. Generic functions without constraints are common, such as `recent`, and the optional return has nothing to do with the constraint.
3. **C. A compile error: push expects a String, not an Int** Once `Element` is fixed to `String`, `push` accepts only Strings, and the mistake is caught at compile time. Swift never converts numbers to text silently, and generic types never widen to `Any` on their own.
4. **D. Both are shorthand for generic types** Both are ordinary generic types from the standard library with friendly shorthand. You can write the long forms, as the Maybe example did with `Optional<String>` and `Array<Int>`; they mean exactly the same thing.

Error Handling

CONCEPT

Failure is part of the design

Networks drop, people type "15%" where a number belongs, cards get declined. Swift treats these as expected outcomes, not surprises. A function marked `throws` can stop early and hand back an **error**: a value of any type that conforms to the `Error` protocol. Callers must acknowledge the risk by writing `try`, then either handle the error in `do` / `catch` or pass it on.

Because errors are values, an enum is the natural way to list what can go wrong, with associated values for the details.

throw	Stops the current function and sends an error to the caller.
try	Marks a call that might throw, so readers can see every risky line.
do / catch	Runs code and routes any thrown error to the first matching catch.
try?	Turns a thrown error into nil when you only care whether it worked.

> **Errors are not crashes**
>
> A thrown error is normal control flow that your code handles. A crash (a failed `try!`, force-unwrapping nil, `fatalError`) stops the whole app. Throw for expected problems; crash only on true programmer mistakes.

CODE · RUNS IN THE BROWSER

Throwing and catching

A withdrawal fails when the balance is too low. The function `throws` an enum error that says how much is missing. The caller wraps the risky calls in `do`: the first call succeeds, the second throws, and control jumps straight to `catch`, skipping the rest of the `do` block.

```swift
enum WithdrawalError: Error {
    case insufficientFunds(needed: Double)
}

func withdraw(_ amount: Double,
              from balance: Double) throws -> Double {
    guard amount <= balance else {
        let short = amount - balance
        throw WithdrawalError.insufficientFunds(needed: short)
    }
    return balance - amount
}

do {
    let left = try withdraw(20, from: 50)
    print("Left:", left)
    let after = try withdraw(45, from: left)
    print("Never printed:", after)
} catch {
    print("Failed:", error)
}
print("Done")
```

OUTPUT

```
Left: 30.0
Failed: insufficientFunds(needed: 15.0)
Done
```

Line by line

1 `enum WithdrawalError: Error {`

WHO	You, defining an error type.
WHAT	**Declares an enum that conforms to Error, so its values can be thrown.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler records the error type before withdraw can throw it.
WHY	Error has no requirements; conforming simply marks the type as throwable.

2 `case insufficientFunds(needed: Double)`

WHO	The WithdrawalError type, declaring its only case and the amount it carries.
WHAT	**A case carrying how much money was missing.**
WHERE	Inside the enum body, the single case.
WHEN	Compile time; the needed amount is attached only when withdraw throws.
WHY	Carrying details lets the screen say exactly what went wrong.

5 `func withdraw(_ amount: Double,`

WHO	You, declaring a throwing function.
WHAT	**Starts the signature: the amount to take out comes first.**
WHERE	Top level of the file.
WHEN	Each call to withdraw, twice inside the do block.
WHY	The signature continues on the next line to keep lines short.

```
6  from balance: Double) throws -> Double {
```

WHO The same withdraw declaration, continued onto a second line.
WHAT **Takes the current balance, may throw, and returns the new balance.**
WHERE The function signature, before the opening brace; throws sits between the parameters and the return type.
WHEN Each call; the compiler checks at compile time that every call site writes try.
WHY throws goes before the return arrow and warns every caller that this can fail.

```
7  guard amount <= balance else {
```

WHO The guard statement, checking the request against the balance.
WHAT **Checks the withdrawal is affordable; if not, runs the else block.**
WHERE Start of the function body.
WHEN Each call to withdraw, as its first statement.
WHY Validating first keeps the success path unindented and easy to read.

```
8  let short = amount - balance
```

WHO The else block of the guard, computing how much is missing.
WHAT **Works out how much money is missing.**
WHERE Inside the guard's else.
WHEN Only when the balance is too low.
WHY Naming the value keeps the throw line readable.

```
9  throw WithdrawalError.insufficientFunds(needed: short)
```

WHO The throw statement, leaving withdraw at once with an error value.
WHAT **Ends the function immediately and sends this error to the caller.**
WHERE Inside the guard's else.
WHEN Only when the balance is too low.
WHY guard's else must leave the scope, and throw does exactly that.

```
11 return balance - amount
```

WHO The function, on its success path.
WHAT **Returns the new balance.**
WHERE End of the function.
WHEN Only when nothing was thrown.
WHY A throwing function returns normally when it succeeds.

```
14 do {
```

WHO The do statement, opening the block whose errors the catch below handles.
WHAT **Opens a block whose thrown errors will be handled by the catch below.**
WHERE Top-level code; the do and catch form one statement.
WHEN At run time, after withdraw is declared.
WHY Grouping related calls means one catch handles them all.

```
15 let left = try withdraw(20, from: 50)
```

WHO Your call, marked with try.
WHAT **Withdraws 20 from 50, which succeeds, leaving 30.0.**
WHERE Inside the do block.
WHEN First, as the do block starts.
WHY try is required on every call that can throw, even ones that end up succeeding.

```
16 print("Left:", left)
```

WHO print, on the success path.
WHAT **Prints Left: 30.0, the balance that survived the first withdrawal.**
WHERE Inside the do block.
WHEN After the successful withdrawal.
WHY Code after a successful try continues normally.

```
17 let after = try withdraw(45, from: left)
```

WHO Your call, which throws.
WHAT **Tries to take 45 from 30.0. The guard fails and an error is thrown.**
WHERE Inside the do block.
WHEN Second, right after the first withdrawal.
WHY From this point, the rest of the do block is skipped.

```
18 print("Never printed:", after)
```

WHO print, which does not run.
WHAT **Would print the new balance if the withdrawal had worked.**
WHERE Inside the do block.
WHEN Skipped, because the line above threw.
WHY Values from a failed try never exist, so they cannot be used by mistake.

```
19 } catch {
```

WHO The catch clause, receiving any error thrown inside the do block.
WHAT **Receives the thrown error in an implicit constant named error.**
WHERE Directly after the do block.
WHEN Only when something in the do block throws.
WHY A catch with no pattern catches every error.

```
20 print("Failed:", error)
```

WHO print, reporting the caught error.
WHAT **Prints Failed: insufficientFunds(needed: 15.0).**
WHERE Inside the catch block, where error is bound automatically.
WHEN After the error arrives.
WHY Printing the case and its associated value is helpful while developing.

```
22 print("Done")
```

WHO print, after the do/catch statement.
WHAT **Prints Done, showing that the program continues after the error was handled.**
WHERE Top-level code after the do/catch.
WHEN Last, whether or not an error was thrown.
WHY After an error is handled, the program carries on normally.

> **Try it**
>
> 1. Change 45 to 25. Which lines print now?
> 2. Remove the word try from the first call. Read the compiler's message.

CODE · RUNS IN THE BROWSER

Catching specific errors

A `catch` can match a pattern, just like a `switch` case. Clauses are tried top to bottom: match an enum case and bind its data, match a whole error type with `catch let error as Type`, and finish with a plain

`catch` for everything else. The loop throws three different errors to exercise each clause.

```
enum CheckoutError: Error {
    case outOfStock(item: String)
    case cardDeclined
}

struct NetworkError: Error { let code: Int }

let problems: [any Error] = [CheckoutError.cardDeclined,
    CheckoutError.outOfStock(item: "Oat milk"),
    NetworkError(code: 503)]

for problem in problems {
    do {
        throw problem
    } catch CheckoutError.outOfStock(let item) {
        print("Sorry, \(item) is sold out")
    } catch let error as NetworkError {
        print("Server error \(error.code). Try again.")
    } catch {
        print("Other:", error)
    }
}
```

OUTPUT

```
Other: cardDeclined
Sorry, Oat milk is sold out
Server error 503. Try again.
```

Line by line

1 `enum CheckoutError: Error {`

WHO	You, defining an error enum.
WHAT	**Declares the errors checkout can produce.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler records CheckoutError and its two cases.
WHY	One enum per feature keeps error lists small and meaningful.

2 `case outOfStock(item: String)`

WHO	The CheckoutError type, declaring a case that names the missing item.
WHAT	**Out of stock, carrying the item name.**
WHERE	Inside the enum body, first of two cases.
WHEN	Compile time; the item name is attached when the error value is created.
WHY	The item name lets the message be specific.

3 `case cardDeclined`

WHO	The CheckoutError type, declaring a case with no data.
WHAT	**A declined card, with no extra data.**
WHERE	Inside the enum body, the second case.
WHEN	Compile time, like every case declaration.
WHY	Not every error needs details.

```
6  struct NetworkError: Error { let code: Int }
```

WHO You, defining an error struct.
WHAT **Declares a second error type carrying an HTTP status code.**
WHERE Top level of the file.
WHEN Compile time; a second, unrelated error type declared on one line.
WHY Structs can be errors too, which suits errors with data but no variations.

```
8  let problems: [any Error] = [CheckoutError.cardDeclined,
```

WHO Your code, building a mixed array of error values.
WHAT **Starts an array holding errors of different types.**
WHERE A top-level constant holding an array of any Error.
WHEN At run time, as the array literal is evaluated.
WHY any Error is the box type that can hold any error value.

```
9  CheckoutError.outOfStock(item: "Oat milk"),
```

WHO The CheckoutError enum, creating an outOfStock value with its item name.
WHAT **An out-of-stock error for oat milk.**
WHERE The second element of the problems array.
WHEN While the array is built.
WHY This one should match the first catch.

```
10 NetworkError(code: 503)]
```

WHO The NetworkError struct's memberwise initializer.
WHAT **A server error with status 503, closing the array.**
WHERE The third and last element of the problems array.
WHEN While the array is built.
WHY This one should match the type-based catch.

```
12 for problem in problems {
```

WHO A for-in loop over the problems array, binding each error to problem.
WHAT **Visits the three errors in order, so each catch clause gets exercised.**
WHERE Top-level code; problem has the type any Error inside the loop.
WHEN Three passes, one per element of problems.
WHY One loop shows every catch clause in action.

```
13 do {
```

WHO A do block, opened fresh on every pass of the loop.
WHAT **Opens the code whose errors the catches below handle.**
WHERE Inside the loop body, wrapping the single throw.
WHEN Each pass of the loop, before the throw.
WHY Each iteration gets a fresh do/catch.

```
14 throw problem
```

WHO The throw statement, rethrowing the stored error so the catches can sort it.
WHAT **Throws the current error so the catches can sort it.**
WHERE Inside the do block.
WHEN Each pass of the loop; control jumps to the catch clauses immediately.
WHY In a real app this would be a try call to checkout code.

```
15 } catch CheckoutError.outOfStock(let item) {
```

WHO A pattern catch, matching one specific case and binding its item.
WHAT **Matches only out-of-stock errors and binds the item name.**
WHERE The first catch clause; it is tried before the other two.
WHEN When the thrown error is that case.
WHY Specific clauses go first, because the first matching clause wins.

```
16 print("Sorry, \(item) is sold out")
```

WHO print, in the out-of-stock handler.
WHAT **Prints a message that names the item.**
WHERE Inside the first catch.
WHEN For the oat milk error.
WHY Specific messages are friendlier than a generic failure.

```
17 } catch let error as NetworkError {
```

WHO A type-cast catch, matching any NetworkError and binding it as error.
WHAT **Matches any NetworkError and binds it as that type, so its properties are available.**
WHERE The second catch clause, tried only when the first did not match.
WHEN When the error is a NetworkError.
WHY as gives you the concrete type, not just any Error.

```
18 print("Server error \(error.code). Try again.")
```

WHO print, in the network error handler.
WHAT **Prints the status code with advice.**
WHERE Inside the second catch.
WHEN For the 503 error.
WHY error.code compiles only because of the as NetworkError cast.

```
19 } catch {
```

WHO The catch-all clause, with no pattern.
WHAT **Handles every error not matched above.**
WHERE The last catch clause; without it the loop body would not be exhaustive.
WHEN Only on the pass for cardDeclined, which neither pattern above matched.
WHY Swift requires the errors to be handled fully, and a final plain catch guarantees that.

```
20 print("Other:", error)
```

WHO print, in the catch-all handler.
WHAT **Prints Other: cardDeclined, using the automatic error binding.**
WHERE Inside the catch-all block.
WHEN For any unmatched error.
WHY The implicit error constant is available in every catch.

> **Try it**
>
> 1. Add a catch CheckoutError.cardDeclined clause with a friendlier message.
> 2. Move the plain catch to the top. What happens to the specific clauses?

CODE · RUNS IN THE BROWSER

try? and try!

Sometimes the reason for a failure does not matter. `try?` converts any thrown error into `nil`, giving you an optional, which pairs well with `??` for a default. `try!` asserts that a call cannot fail and crashes if it does; keep it for values you control completely.

```swift
enum ParseError: Error {
    case notANumber(String)
}

func parseTip(_ text: String) throws -> Int {
    guard let value = Int(text) else {
        throw ParseError.notANumber(text)
    }
    return value
}

let typed = try? parseTip("15")
let typo = try? parseTip("15%")
print(typed ?? 0, typo ?? 0)

let tip = (try? parseTip("abc")) ?? 18
print("Tip percent:", tip)

let preset = try! parseTip("20")
print("Preset tip:", preset)
```

OUTPUT

```
15 0
Tip percent: 18
Preset tip: 20
```

Line by line

1 `enum ParseError: Error {`

WHO	You, defining an error type.
WHAT	**Declares the error for unreadable input.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler records ParseError before parseTip can throw it.
WHY	Even simple parsers deserve a named error.

2 `case notANumber(String)`

WHO	The ParseError type, declaring a case that carries the offending text.
WHAT	**A case carrying the text that could not be parsed.**
WHERE	Inside the enum body, the single case.
WHEN	Compile time; the text is attached when parseTip throws.
WHY	Keeping the bad input helps when logging the problem.

5 `func parseTip(_ text: String) throws -> Int {`

WHO	You, declaring a throwing function.
WHAT	**Turns typed text into a tip percentage, or throws.**
WHERE	Top level of the file.
WHEN	Each call to parseTip, four times in this program.
WHY	Input from a text field is exactly the kind of thing that fails.

```
6  guard let value = Int(text) else {
```

WHO The failable Int initializer, inside a guard.
WHAT **Tries to read the text as an Int.**
WHERE The first statement of the parseTip body.
WHEN Each call; Int(text) runs here and succeeds or fails.
WHY Int(text) returns nil for text like 15%, and guard routes that to the else block.

```
7      throw ParseError.notANumber(text)
```

WHO The throw statement, leaving parseTip with the text that failed to parse.
WHAT **Reports which text failed.**
WHERE Inside the guard's else.
WHEN Only for non-numeric input.
WHY Converting nil into a named error tells callers why it failed.

```
9  return value
```

WHO The parseTip function, returning the parsed number.
WHAT **Returns the parsed number.**
WHERE End of the body.
WHEN Only when Int(text) succeeded and the guard let it through.
WHY value was unwrapped by the guard, so it is a plain Int here.

```
12 let typed = try? parseTip("15")
```

WHO try?, wrapping the result in an optional.
WHAT **Parses 15 successfully, so typed is an Int? holding 15.**
WHERE A top-level constant of type Int?, because try? wraps the result.
WHEN At run time; parseTip succeeds here, so typed holds 15.
WHY The type is Int?, because try? always produces an optional.

```
13 let typo = try? parseTip("15%")
```

WHO try?, swallowing the error.
WHAT **Parsing fails, the error is discarded, and typo is nil.**
WHERE A top-level constant of type Int?, holding nil this time.
WHEN At run time; parseTip throws here and try? swallows the error.
WHY Use try? when the only question is "did it work?".

```
14 print(typed ?? 0, typo ?? 0)
```

WHO The ?? operator, unwrapping each optional with 0 as the fallback.
WHAT **Prints 15 0: the value when present, the fallback for nil.**
WHERE Top-level code; the console shows 15 0 on one line.
WHEN At run time, the first line of output.
WHY Unwrapping with ?? avoids printing Optional(15).

```
16 let tip = (try? parseTip("abc")) ?? 18
```

WHO try? combined with ??.
WHAT **Falls back to a default tip of 18 when parsing fails.**
WHERE A top-level constant of type Int.
WHEN At run time; the parse fails, try? gives nil, and ?? supplies 18.
WHY The parentheses make it clear that ?? applies to the whole try? expression.

```
17 print("Tip percent:", tip)
```

WHO print, showing the fallback result.
WHAT **Prints Tip percent: 18.**
WHERE Top-level code; the console shows Tip percent: 18.
WHEN At run time, the second line of output.
WHY A sensible default keeps the screen usable after bad input.

```
19 let preset = try! parseTip("20")
```

WHO try!, which crashes on failure.
WHAT **Parses a hard-coded 20 and returns a plain Int.**
WHERE A top-level constant of type Int, not Int?, because try! removes the optional.
WHEN At run time; the text 20 parses, so no crash happens.
WHY Acceptable only because the input is a constant you wrote. Never use try! on user input.

```
20 print("Preset tip:", preset)
```

WHO print, showing the forced result.
WHAT **Prints Preset tip: 20.**
WHERE Top-level code; the console shows Preset tip: 20.
WHEN The last statement of the program.
WHY No unwrapping needed: try! returns the non-optional value.

> **Try it**
>
> 1. Change the preset to try! parseTip("twenty") and run. What happens to the program?
> 2. Print typed without ?? 0. How does Swift show an optional value?

CODE · RUNS IN THE BROWSER

Cleaning up with defer

A `defer` block runs when the current scope ends, however it ends: normal return, early `return`, or a thrown error. It is the right place for cleanup such as hiding a spinner, closing a file, or releasing a lock, because no exit path can skip it.

```swift
enum SyncError: Error {
    case offline
}

func syncPhotos(online: Bool) throws {
    print("Show spinner")
    defer {
        print("Hide spinner")
    }
    guard online else {
        throw SyncError.offline
    }
    print("Uploaded 12 photos")
}

for online in [true, false] {
    do {
        try syncPhotos(online: online)
    } catch {
        print("Error:", error)
    }
}
```

OUTPUT

```
Show spinner
Uploaded 12 photos
Hide spinner
Show spinner
Hide spinner
Error: offline
```

Line by line

1 `enum SyncError: Error {`

WHO	You, defining an error type.
WHAT	**Declares the error for a failed photo sync.**
WHERE	Top level of the file.
WHEN	Compile time; the compiler records SyncError before syncPhotos can throw it.
WHY	A named error documents what can fail.

2 `case offline`

WHO	The SyncError type, declaring its only case.
WHAT	**The device has no connection.**
WHERE	Inside the enum body, the single case.
WHEN	Compile time; a case declaration runs no code.
WHY	One case is enough for this example.

5 `func syncPhotos(online: Bool) throws {`

WHO	You, declaring a throwing function.
WHAT	**Simulates uploading photos, failing when offline.**
WHERE	Top level of the file.
WHEN	Each call to syncPhotos, twice in this program, once online and once offline.
WHY	Real sync code has several exit paths, which is where defer shines.

```
6  print("Show spinner")
```

WHO print, standing in for UI code.
WHAT **Shows a progress spinner.**
WHERE The first statement of the syncPhotos body, before defer is registered.
WHEN At the start of every call.
WHY Anything shown at the start must be hidden at the end, on every path.

```
7  defer {
```

WHO The defer statement, registering a block to run when the function exits.
WHAT **Registers a block to run when the function exits.**
WHERE Function body, right after the setup it undoes.
WHEN Registered now; runs at exit.
WHY Placing cleanup next to setup makes it impossible to forget.

```
8  print("Hide spinner")
```

WHO The deferred block, run by Swift as syncPhotos exits by any path.
WHAT **Prints Hide spinner, the cleanup that must happen whether the sync worked or not.**
WHERE Inside the defer block, in the body of syncPhotos.
WHEN As the function exits, after a success or after a throw.
WHY Even the error path hides the spinner, so the screen never gets stuck.

```
10 guard online else {
```

WHO The guard statement, checking the online flag.
WHAT **Checks for a connection.**
WHERE Inside syncPhotos, after the defer has been registered.
WHEN Each call to syncPhotos, right after the spinner is shown.
WHY An early exit that would skip cleanup code written at the bottom, but not a defer.

```
11 throw SyncError.offline
```

WHO The throw statement, leaving syncPhotos early with the offline error.
WHAT **Exits the function with an error.**
WHERE Inside the guard's else.
WHEN Only on the second call, when online is false.
WHY The defer block still runs on this path.

```
13 print("Uploaded 12 photos")
```

WHO print, standing in for the upload.
WHAT **Prints Uploaded 12 photos, standing in for the real upload.**
WHERE End of the function body.
WHEN Only on the first call, when online is true; the throw skips it otherwise.
WHY On this path the defer runs right after this line.

```
16 for online in [true, false] {
```

WHO A for-in loop over two Bools, binding each to online.
WHAT **Calls the function once online and once offline.**
WHERE Top-level code; online is true on the first pass and false on the second.
WHEN Twice: once online, once offline.
WHY Two runs show both exit paths.

```
17 do {
```

WHO A do block, opened fresh on each pass of the loop.
WHAT **Opens a scope where errors are handled.**
WHERE Inside the loop body, wrapping the throwing call.
WHEN Each pass of the loop, twice in total.
WHY Handles the offline error so the loop continues.

```
18 try syncPhotos(online: online)
```

WHO Your call, marked try.
WHAT **Calls syncPhotos with the current flag; try marks that it may throw.**
WHERE Inside the do block.
WHEN Each pass of the loop; the second pass throws.
WHY try is required because the function throws.

```
19 } catch {
```

WHO The catch clause, receiving the offline error on the second pass.
WHAT **Receives the offline error.**
WHERE After the do block.
WHEN Only on the second iteration.
WHY By the time catch runs, the defer inside the function has already run.

```
20 print("Error:", error)
```

WHO print, in the catch block.
WHAT **Prints Error: offline, after the deferred block has already hidden the spinner.**
WHERE Inside the catch block, where error is bound automatically.
WHEN After the spinner was hidden.
WHY The output order proves defer ran before the error reached the caller.

> **Try it**
>
> 1. Add a second defer that prints "Log sync". Which defer runs first?
> 2. Move the defer block below the guard. What changes in the offline run?

CODE · RUNS IN THE BROWSER

Result: success or failure as a value

`Result<Success, Failure>` is a generic enum with two cases, `.success` and `.failure`. It stores an outcome as a value you can keep, pass around and switch over later. You will meet it in callback-based APIs. `get()` converts it back into a throwing call.

```swift
enum WeatherError: Error {
    case cityNotFound(String)
}

func forecast(for city: String) -> Result<Int, WeatherError> {
    let highs = ["Lisbon": 24, "Oslo": 9]
    guard let high = highs[city] else {
        return .failure(.cityNotFound(city))
    }
    return .success(high)
}

for city in ["Oslo", "Atlantis"] {
    switch forecast(for: city) {
    case .success(let high):
        print("\(city): high of \(high) degrees")
    case .failure(.cityNotFound(let name)):
        print("No forecast for \(name)")
    }
}

let lisbon = try? forecast(for: "Lisbon").get()
print(lisbon ?? 0)
```

OUTPUT

```
Oslo: high of 9 degrees
No forecast for Atlantis
24
```

Line by line

1 `enum WeatherError: Error {`

- WHO — You, defining an error type.
- WHAT — **Declares the weather lookup error.**
- WHERE — Top level of the file.
- WHEN — Compile time; the compiler records WeatherError before forecast uses it in Result.
- WHY — Result needs a concrete Error type for its Failure parameter.

2 `case cityNotFound(String)`

- WHO — The WeatherError type, declaring a case that carries the unknown city name.
- WHAT — **A case carrying the unknown city.**
- WHERE — Inside the enum body, the single case.
- WHEN — Compile time; the city name is attached when a failure is created.
- WHY — The city name makes the message specific.

5 `func forecast(for city: String) -> Result<Int, WeatherError> {`

- WHO — You, declaring a function that returns a Result.
- WHAT — **Returns either a high temperature or a WeatherError, wrapped in one value.**
- WHERE — Top level of the file.
- WHEN — Each call to forecast, three times in this program.
- WHY — No throws here: the failure travels inside the return value instead.

```
6  let highs = ["Lisbon": 24, "Oslo": 9]
```

WHO Your code, standing in for a weather service.
WHAT **A dictionary of known cities and their highs.**
WHERE Local to the function.
WHEN Each call to forecast; the dictionary is rebuilt locally every time.
WHY A tiny lookup table keeps the example offline and deterministic.

```
7  guard let high = highs[city] else {
```

WHO The dictionary subscript, inside a guard.
WHAT **Looks up the city; the subscript returns nil for unknown cities.**
WHERE Inside forecast, after the dictionary it looks in.
WHEN Each call; the lookup returns nil for Atlantis and a number for the others.
WHY guard turns the missing city into an early failure.

```
8  return .failure(.cityNotFound(city))
```

WHO The Result enum, creating its failure case around a WeatherError.
WHAT **Returns the failure case carrying a WeatherError.**
WHERE Inside the guard's else.
WHEN Only for unknown cities.
WHY The return type is known, so both dots resolve without type names.

```
10 return .success(high)
```

WHO The Result enum, creating its success case around the Int high.
WHAT **Returns the success case carrying the temperature.**
WHERE End of the function.
WHEN Only for cities in the dictionary, Lisbon and Oslo.
WHY Success and failure share one return type, so callers must consider both.

```
13 for city in ["Oslo", "Atlantis"] {
```

WHO A for-in loop over two city names, binding each to city.
WHAT **Looks up one real and one imaginary city.**
WHERE Top-level code; the loop body calls forecast once per city.
WHEN Twice, once per city.
WHY Exercises both cases of the Result.

```
14 switch forecast(for: city) {
```

WHO A switch over the Result.
WHAT **Branches on success or failure.**
WHERE Inside the loop body, switching directly on the returned Result.
WHEN Each pass of the loop, twice in total.
WHY Result is an enum, so the switch must cover both cases.

```
15 case .success(let high):
```

WHO Pattern matching, binding the Int inside the success case to high.
WHAT **Binds the temperature on success.**
WHERE The first branch of the switch.
WHEN On the first pass, for Oslo, whose forecast succeeds.
WHY The value comes out already typed as Int.

```
16  print("\(city): high of \(high) degrees")
```

WHO print, on the success path.
WHAT **Prints Oslo: high of 9 degrees.**
WHERE Inside the success branch.
WHEN On the first pass only, printing the Oslo high.
WHY Interpolation mixes the city and its high.

```
17  case .failure(.cityNotFound(let name)):
```

WHO Nested pattern matching, reaching through failure to the cityNotFound case and binding name.
WHAT **Matches the failure case and the specific error inside it, binding the city name.**
WHERE The second branch of the switch; both Result cases are now covered.
WHEN On the second pass, for Atlantis, which is not in the dictionary.
WHY Patterns nest, so one line reaches two levels of enum.

```
18  print("No forecast for \(name)")
```

WHO print, on the failure path.
WHAT **Prints No forecast for Atlantis.**
WHERE Inside the failure branch.
WHEN On the second pass only, printing the missing city.
WHY A precise message beats a generic error.

```
22  let lisbon = try? forecast(for: "Lisbon").get()
```

WHO Result.get(), combined with try?.
WHAT **get() returns the success value or throws the failure; try? turns that into an optional.**
WHERE A top-level constant of type Int?.
WHEN At run time, after the loop; get() returns 24 so try? wraps it as Optional(24).
WHY get() bridges Result back into normal throwing code.

```
23  print(lisbon ?? 0)
```

WHO The ?? operator, unwrapping lisbon with 0 as the fallback.
WHAT **Prints 24, the Lisbon high unwrapped from the optional.**
WHERE Top-level code; the console shows a plain Int.
WHEN The last statement of the program.
WHY Unwrapping with ?? keeps Optional(...) out of the output.

> **Try it**
>
> 1. Add "Tokyo": 19 to the dictionary and look it up in the loop.
> 2. Replace the switch with if case .success(let high) = forecast(for: city) { … } for Oslo only.

CODE · READ-ALONG

Errors people can read

Printing `insufficientFunds(needed: 15.0)` helps you, not your users. Foundation's `LocalizedError` protocol lets an error supply a human sentence through `errorDescription`, which `localizedDescription` then returns. Good errors are specific, carry their details, and say what to do next.

```swift
import Foundation

enum SignUpError: LocalizedError {
    case emailTaken(String)
    case weakPassword(minimum: Int)

    var errorDescription: String? {
        switch self {
        case .emailTaken(let email):
            return "\(email) already has an account."
        case .weakPassword(let minimum):
            return "Use at least \(minimum) characters."
        }
    }
}

let error: any Error = SignUpError.weakPassword(minimum: 12)
let message = error.localizedDescription
```

Line by line

1 `import Foundation`

- **WHO** The compiler, loading Foundation so LocalizedError and localizedDescription are available.
- **WHAT** **Loads Foundation, which defines LocalizedError and localizedDescription.**
- **WHERE** First line of the file.
- **WHEN** Compile time, before any line of the program runs.
- **WHY** LocalizedError is a Foundation protocol, not part of the core language.

3 `enum SignUpError: LocalizedError {`

- **WHO** You, defining a user-facing error type.
- **WHAT** **Declares the sign-up errors and adopts LocalizedError, which also makes them Errors.**
- **WHERE** Top level of the file.
- **WHEN** Compile time; the compiler records SignUpError and checks its LocalizedError conformance.
- **WHY** One enum per screen or feature keeps errors focused.

4 `case emailTaken(String)`

- **WHO** The SignUpError type, declaring a case that carries the taken email.
- **WHAT** **The email already belongs to an account; carries the email.**
- **WHERE** Inside the enum body, first of two cases.
- **WHEN** Compile time; the email is attached when the error is created.
- **WHY** The address lets the message confirm what the user typed.

5 `case weakPassword(minimum: Int)`

- **WHO** The SignUpError type, declaring a case that carries the password minimum.
- **WHAT** **The password is too short; carries the required length.**
- **WHERE** Inside the enum body, the second case.
- **WHEN** Compile time; the minimum is attached when the error is created.
- **WHY** Carrying the rule means the message can never disagree with it.

```
7  var errorDescription: String? {
```

WHO The LocalizedError protocol, which reads this property.
WHAT **Provides the human-readable sentence for each case.**
WHERE Inside the enum, below the two cases, where it can switch on self.
WHEN Whenever someone reads localizedDescription.
WHY The optional type comes from the protocol; return nil to fall back to a generic message.

```
8      switch self {
```

WHO The switch statement, inspecting which case this error is.
WHAT **Chooses the sentence for the current case.**
WHERE Inside the errorDescription getter, as its only statement.
WHEN Each read of errorDescription, once in this program.
WHY Exhaustive: a new case cannot ship without a message.

```
9          case .emailTaken(let email):
```

WHO Pattern matching, binding the email carried by the emailTaken case.
WHAT **Binds the email address.**
WHERE The first branch of the switch.
WHEN Only for an emailTaken error, which this program never creates.
WHY The data is available right where the sentence is built.

```
10             return "\(email) already has an account."
```

WHO The errorDescription getter, returning the sentence for a taken email.
WHAT **Returns a sentence naming the email.**
WHERE Inside the first branch.
WHEN Only for an emailTaken error; this line does not run here.
WHY Plain, specific language helps the user fix the problem.

```
11         case .weakPassword(let minimum):
```

WHO Pattern matching, binding the minimum carried by the weakPassword case.
WHAT **Binds the minimum length.**
WHERE The second branch of the switch; both cases are now covered.
WHEN For the weakPassword error that the program creates below.
WHY Covers the last case, so the switch is complete.

```
12             return "Use at least \(minimum) characters."
```

WHO The errorDescription getter, returning the sentence for a weak password.
WHAT **Returns an instruction, not just a complaint.**
WHERE Inside the second branch.
WHEN For the weakPassword error, when localizedDescription is read.
WHY Messages that say what to do next reduce frustration and support requests.

```
17 let error: any Error = SignUpError.weakPassword(minimum: 12)
```

WHO Your code, storing the error as any Error.
WHAT **Creates an error the way a catch clause would receive it.**
WHERE A top-level constant typed as any Error, hiding the concrete SignUpError.
WHEN At run time, after the enum is declared.
WHY Catch blocks see errors as any Error, so this mirrors real use.

```
18  let message = error.localizedDescription
```

WHO	Foundation, bridging to your errorDescription.
WHAT	**Produces the text Use at least 12 characters.**
WHERE	A top-level constant holding the String that LocalizedError produced.
WHEN	At run time; this read is what runs the errorDescription getter.
WHY	Views can show localizedDescription for any error without knowing its type, for example in an alert.

Try it

1. Add var recoverySuggestion: String? that returns "Try signing in instead." for emailTaken.
2. What should errorDescription return for an error users should never see?

CHECK YOURSELF

Check your understanding

1. What does this print?

```
func save() {
    defer { print("A") }
    print("B")
}
save()
```

   A. A, then B
   B. B, then A
   C. Only B
   D. Only A

2. Given `parseTip` from this lesson, what is `let tip = try? parseTip("abc")`?

   A. An Int? holding nil
   B. An Int holding 0
   C. A crash at run time
   D. A ParseError value

3. When is `try!` a reasonable choice?

   A. For user input, to keep the code short
   B. Whenever the call is inside a do block
   C. Never: it does not compile in Swift 6
   D. When failure could only mean a bug in the app itself

4. Which design gives callers the most useful errors?

   A. Return -1 whenever something goes wrong
   B. Print a message and return normally
   C. An error enum with one case per failure
   D. Throw one generic error for every problem

Answers: Error Handling

1. **B. B, then A** A `defer` block is registered where it appears but runs when the function exits, after `print("B")`. That is why it suits cleanup: it always runs last, on every exit path.
2. **A. An Int? holding nil** `try?` discards the thrown error and produces `nil`, and its result is always optional, so the type is `Int?`. Nothing crashes; that would be `try!`. To keep the error itself, use `do/catch`.
3. **D. When failure could only mean a bug in the app itself** `try!` crashes on failure, so use it only where failure means the code itself is wrong, such as a resource you ship with the app. User input and network data fail for ordinary reasons and must be handled.
4. **C. An error enum with one case per failure** An enum lists every failure the caller must consider, and associated values carry the details needed for a precise message. Magic numbers and printing hide failures from the caller, and a single generic error cannot be handled differently per cause.

Concurrency: async/await and Actors

CONCEPT

Why apps need concurrency

iOS draws your interface on the **main thread**, the one thread that handles touches and updates the screen, up to 120 times a second. If code on that thread waits two seconds for a download, nothing redraws and taps go unanswered: the app freezes.

Concurrency lets slow work wait without blocking. You mark a function `async` when it may pause, and write `await` where it might. At each `await` the function **suspends**: it steps aside, the thread goes back to drawing frames and handling taps, and the function resumes when its result is ready.

Thread	A path of execution the system schedules on a processor core.
Suspension point	An await, where a function may pause and let other work run.
Task	A unit of asynchronous work that Swift schedules and can cancel.
Data race	Two threads touching the same memory at once, at least one of them writing.

> **Time in these examples**
>
> `Task.sleep` stands in for slow network calls. In this page a virtual clock makes a one-second sleep finish instantly but in the correct order. On a real device it truly waits.

CODE · RUNS IN THE BROWSER

async and await

An `async` function can suspend. Calling it requires `await`, which marks the exact spot where the caller may pause. This one can also throw, because `Task.sleep` throws if the work is cancelled, so the call needs `try await` inside `do` / `catch`.

```swift
func fetchForecast(for city: String) async throws -> String {
    try await Task.sleep(for: .seconds(1))
    return "\(city): 18 degrees, light rain"
}

print("Requesting forecast")
do {
    let report = try await fetchForecast(for: "Seattle")
    print(report)
} catch {
    print("Request failed:", error)
}
print("Done")
```

OUTPUT

```
Requesting forecast
Seattle: 18 degrees, light rain
Done
```

Line by line

```
1  func fetchForecast(for city: String) async throws -> String {
```

WHO	You, declaring an asynchronous function.
WHAT	**Declares a function that may suspend (async) and may fail (throws) before returning a String.**
WHERE	Top level of the file.
WHEN	Its body runs when awaited and may pause partway through.
WHY	async and throws are both part of the type, so every caller can see both risks.

```
2  try await Task.sleep(for: .seconds(1))
```

WHO	Swift's concurrency runtime, which suspends fetchForecast and resumes it after the delay.
WHAT	**Suspends this function for one second, standing in for a network request.**
WHERE	Inside fetchForecast, standing in for a real network request.
WHEN	When the function starts; it resumes about a second later.
WHY	While suspended, the thread is free for other work. Sleeping does not block anything.

```
3  return "\(city): 18 degrees, light rain"
```

WHO	The function, after it resumes.
WHAT	**Returns the forecast text.**
WHERE	End of the function.
WHEN	After the sleep finishes.
WHY	The code after an await reads as if nothing happened, even though time passed.

```
6  print("Requesting forecast")
```

WHO	print, before any waiting happens.
WHAT	**Prints the first line.**
WHERE	Top-level code, the first statement that runs.
WHEN	Immediately, before the request is made; it is the first line of output.
WHY	Marks the moment before the asynchronous call.

```
7  do {
```

WHO	A do block at top level, wrapping the throwing async call.
WHAT	**Opens a scope for the throwing call.**
WHERE	Top-level code; the do and the catch below form one statement.
WHEN	Immediately after the first print.
WHY	An async throwing call still needs its errors handled.

```
8  let report = try await fetchForecast(for: "Seattle")
```

WHO	Your call, marked try await.
WHAT	**Calls the function and suspends here until it returns.**
WHERE	Inside the do block.
WHEN	Pauses for about one second.
WHY	try comes before await. The order is required and always the same.

```
9  print(report)
```

WHO	print, after the await resumes.
WHAT	**Prints Seattle: 18 degrees, light rain.**
WHERE	Inside the do block.
WHEN	After the await resumes.
WHY	report holds a plain String; the waiting is already over.

10 `} catch {`

- WHO — The catch clause, ready for any error that fetchForecast throws.
- WHAT — **Handles a failure, such as cancellation.**
- WHERE — After the do block.
- WHEN — Only if the call throws.
- WHY — Cancelled network work usually needs no message, but it must be handled.

11 `print("Request failed:", error)`

- WHO — print, in the catch block.
- WHAT — **Reports the error with a label; it does not run in this program because the request succeeds.**
- WHERE — Inside the catch block, where error is bound automatically.
- WHEN — Only if fetchForecast throws, which the simulated request never does.
- WHY — In an app you would set an error state instead of printing.

13 `print("Done")`

- WHO — print, after the do/catch statement.
- WHAT — **Prints Done, after the awaited request has finished.**
- WHERE — Top-level code, after the do-catch statement.
- WHEN — After the do/catch finishes.
- WHY — Lines run in order: async code still reads top to bottom.

> **Try it**
>
> 1. Remove await from the call. What does the compiler say?
> 2. Call fetchForecast twice in a row for two cities. In what order do the lines print?

COMPARE

One after another, or all at once

A profile screen needs three independent downloads. On the left, each `await` finishes before the next starts: 1 + 2 + 1 = 4 seconds. On the right, `async let` starts all three immediately and waits once for all of them: about 2 seconds, the length of the slowest. The output is identical; only the waiting differs.

A: Sequential await: about 4 s

```swift
func download(_ name: String, seconds: Int) async -> String {
    try? await Task.sleep(for: .seconds(seconds))
    return name
}

func loadScreen() async -> [String] {
    let profile = await download("profile", seconds: 1)
    let photos = await download("photos", seconds: 2)
    let news = await download("news", seconds: 1)
    return [profile, photos, news]
}

let parts = await loadScreen()
print(parts)
```

OUTPUT

```
["profile", "photos", "news"]
```

```
1  func download(_ name: String, seconds: Int) async -> String {
```

WHO You, declaring a pretend download.
WHAT **An async function that waits a number of seconds and returns a name.**
WHERE Top level of the file.
WHEN Each time it is awaited.
WHY A stand-in for URLSession so the timing is easy to see.

```
2  try? await Task.sleep(for: .seconds(seconds))
```

WHO The concurrency runtime, suspending download for the requested number of seconds.
WHAT **Suspends for the given seconds. try? ignores the cancellation error.**
WHERE Inside download, standing in for a real transfer.
WHEN At the start of each download.
WHY Ignoring cancellation keeps this demo function non-throwing.

```
3  return name
```

WHO The download function, resuming and returning once the sleep ends.
WHAT **Returns the name as the "downloaded" result.**
WHERE The last line of download.
WHEN After the wait; for photos that is two seconds in.
WHY Returning the name makes the final order easy to check.

```
6  func loadScreen() async -> [String] {
```

WHO You, declaring the screen loader.
WHAT **An async function that gathers three results.**
WHERE Top level of the file.
WHEN When loadScreen is awaited at the bottom of the program.
WHY Only the body differs between the two sides.

```
7  let profile = await download("profile", seconds: 1)
```

WHO An await, suspending loadScreen until the profile download returns.
WHAT **Starts the profile download and suspends until it finishes.**
WHERE Inside loadScreen, the first of three sequential awaits.
WHEN From 0 to 1 second.
WHY Nothing else in this function starts while it waits.

```
8  let photos = await download("photos", seconds: 2)
```

WHO A second await, which cannot start until the first one has finished.
WHAT **Starts photos only after profile is done.**
WHERE Inside loadScreen, after profile has arrived.
WHEN From 1 to 3 seconds.
WHY Correct but slow: photos never needed the profile.

```
9  let news = await download("news", seconds: 1)
```

WHO A third await, started only after photos has arrived.
WHAT **Starts news only after photos is done.**
WHERE Inside loadScreen, the last of the three sequential awaits.
WHEN From 3 to 4 seconds.
WHY Total waiting is the sum of all three.

```
10 return [profile, photos, news]
```

WHO The loadScreen function, returning once all three downloads are done.
WHAT **Returns the three results in order.**
WHERE The last line of loadScreen, about four seconds after it started.
WHEN At about 4 seconds.
WHY Sequential awaits are right when each step needs the previous result.

```
13 let parts = await loadScreen()
```

WHO Top-level code, awaiting the loader.
WHAT **Runs the whole load and stores the array.**
WHERE A top-level constant holding the array loadScreen returned.
WHEN Finishes after about 4 seconds.
WHY Top-level code in a script may use await directly.

```
14 print(parts)
```

WHO print, showing the collected array.
WHAT **Prints the three names.**
WHERE Top-level code; the console shows the three names in download order.
WHEN The last statement of the left program, after about four seconds.
WHY Same output as the right side, slower.

B: Parallel async let: about 2 s

```swift
func download(_ name: String, seconds: Int) async -> String {
    try? await Task.sleep(for: .seconds(seconds))
    return name
}

func loadScreen() async -> [String] {
    async let profile = download("profile", seconds: 1)
    async let photos = download("photos", seconds: 2)
    async let news = download("news", seconds: 1)
    return await [profile, photos, news]
}

let parts = await loadScreen()
print(parts)
```

OUTPUT

```
["profile", "photos", "news"]
```

```
1 func download(_ name: String, seconds: Int) async -> String {
```

WHO You, declaring the same pretend download.
WHAT **Waits a number of seconds and returns a name.**
WHERE Top level of the file.
WHEN Each time it runs.
WHY The download function is kept identical so that the only change between the two sides is how loadScreen calls it.

```
2   try? await Task.sleep(for: .seconds(seconds))
```

WHO The concurrency runtime, suspending this download for its own number of seconds.
WHAT **Suspends for the given seconds.**
WHERE Inside download, the same simulated transfer as on the left.
WHEN At the start of each download.
WHY Three of these will now overlap in time.

```
3   return name
```

WHO The download function, resuming after its sleep.
WHAT **Returns the name it was given, so the caller can see which download finished.**
WHERE The last line of download.
WHEN After the wait; on this side all three waits overlap.
WHY Returning the name keeps the output readable when three downloads finish close together.

```
6   func loadScreen() async -> [String] {
```

WHO You, declaring the loader again.
WHAT **The same signature as the left side.**
WHERE Top level of the file.
WHEN When loadScreen is awaited at the bottom of the program.
WHY Callers cannot tell the difference, except in speed.

```
7   async let profile = download("profile", seconds: 1)
```

WHO async let, creating a child task.
WHAT **Starts the profile download in the background and moves on immediately.**
WHERE Inside loadScreen; the child task belongs to this function.
WHEN Starts at 0 seconds.
WHY No await here: starting work and waiting for it are now separate steps.

```
8   async let photos = download("photos", seconds: 2)
```

WHO A second async let, starting the photos download while profile is still running.
WHAT **Starts photos right away, alongside profile.**
WHERE Inside loadScreen, right after the first async let.
WHEN Also starts at 0 seconds.
WHY Independent work should start as early as possible.

```
9   async let news = download("news", seconds: 1)
```

WHO A third async let, so all three downloads are now in flight at once.
WHAT **Starts news alongside the other two.**
WHERE Inside loadScreen, the last child task started.
WHEN Also starts at 0 seconds.
WHY Three downloads are now in flight at once.

```
10  return await [profile, photos, news]
```

WHO One await covering all three.
WHAT **Waits until every child task has finished, then returns the array.**
WHERE The last line of loadScreen, reached after about two seconds.
WHEN Resumes at about 2 seconds, when the slowest finishes.
WHY Total time is the longest download, not the sum. Child tasks never outlive the function.

```
13 let parts = await loadScreen()
```

WHO Top-level code, awaiting the parallel version of loadScreen.
WHAT **Runs the parallel load.**
WHERE A top-level constant holding the array loadScreen returned.
WHEN Finishes after about 2 seconds.
WHY The call site is unchanged.

```
14 print(parts)
```

WHO print, showing the collected array.
WHAT **Prints the same three names in the same order.**
WHERE Top-level code; the console shows the same three names as the left side.
WHEN The last statement of the right program, after about two seconds.
WHY Order follows the array literal, not which download finished first.

> **Takeaway**
>
> Use sequential `await` when each step needs the previous result, such as logging in before fetching the inbox. Use `async let` when the work is independent. The parallel version is about twice as fast here, with no locks, callbacks or thread management.

INTERACTIVE

Watch the waiting

The same screen load, three ways. Press Play, or drag the time slider, and watch the swim lanes. While a load runs, tap Like in the phone. In the blocking version, taps pile up because the main thread is stuck. With `await` the main thread stays free, and with `async let` the whole load finishes twice as fast.

This step is an interactive simulation in the HTML version.

CODE · RUNS IN THE BROWSER

Starting work with Task

A button's action is ordinary synchronous code; it cannot `await`. `Task { }` creates a new unit of asynchronous work and returns immediately. The tap handler finishes first, then the task runs. `await task.value` waits for the task's result when you need it.

```swift
func countUnread() async -> Int {
    try? await Task.sleep(for: .milliseconds(300))
    return 4
}

print("Button tapped")
let badgeTask = Task {
    let unread = await countUnread()
    print("Badge shows \(unread)")
    return unread
}
print("Tap handler finished")
let total = await badgeTask.value
print("Task returned \(total)")
```

OUTPUT

```
Button tapped
Tap handler finished
Badge shows 4
Task returned 4
```

Line by line

1 `func countUnread() async -> Int {`

- WHO You, declaring an async function.
- WHAT **Pretends to ask a server how many messages are unread.**
- WHERE Top level of the file.
- WHEN When the Task body awaits it, after the tap handler has already finished.
- WHY A typical small network call behind a badge.

2 `try? await Task.sleep(for: .milliseconds(300))`

- WHO The concurrency runtime, suspending countUnread for 300 milliseconds.
- WHAT **Suspends for 300 milliseconds.**
- WHERE Inside countUnread, standing in for a mailbox check.
- WHEN When the function starts.
- WHY Shorter than a second, like a quick API call.

3 `return 4`

- WHO The countUnread function, returning once the sleep ends.
- WHAT **Returns four unread messages.**
- WHERE End of the function.
- WHEN After the wait, inside the task; this value becomes unread.
- WHY A fixed value keeps the output predictable.

6 `print("Button tapped")`

- WHO print, standing in for a button action.
- WHAT **Prints the first line.**
- WHERE Top-level code, which runs on the main actor here.
- WHEN Immediately, the first line of output, before any task exists.
- WHY Marks the start of the tap handler.

7 `let badgeTask = Task {`

- WHO The Task initializer, creating a new task that is scheduled at once but runs only after the current code pauses.
- WHAT **Creates a task to run the closure and returns a handle to it right away.**
- WHERE A top-level constant holding the task handle.
- WHEN Created now; its body runs once the current code gives up the main actor.
- WHY Task { } is the bridge from synchronous code, such as a button action, into async code.

8 `let unread = await countUnread()`

- WHO The task body, awaiting countUnread on its own schedule.
- WHAT **Awaits the unread count.**
- WHERE Inside the task, which inherits the main actor from where it was created.
- WHEN When the task gets to run.
- WHY Inside a task, await is allowed.

```
9  print("Badge shows \(unread)")
```

WHO print, standing in for updating the badge.
WHAT **Prints Badge shows 4.**
WHERE Inside the task body, after the await has resumed.
WHEN After the count arrives.
WHY UI updates belong on the main actor, where this task runs.

```
10 return unread
```

WHO The task body, handing its result to whoever awaits badgeTask.value.
WHAT **Makes the count the task's result.**
WHERE End of the task closure.
WHEN After the Badge shows print, as the task finishes.
WHY A task can produce a value that others await.

```
12 print("Tap handler finished")
```

WHO print, in the synchronous tap handler.
WHAT **Prints the second line, before the task has run.**
WHERE Top-level code, right after the Task was created.
WHEN Immediately after creating the task.
WHY Task { } never blocks: the handler finishes and the screen stays responsive.

```
13 let total = await badgeTask.value
```

WHO The task handle's value property.
WHAT **Suspends until the task finishes, then takes its returned count.**
WHERE A top-level constant holding the Int the task returned.
WHEN After the task has printed its line.
WHY Awaiting value is how you collect a task's result.

```
14 print("Task returned \(total)")
```

WHO print, after collecting the result.
WHAT **Prints Task returned 4.**
WHERE Top-level code; the console shows Task returned 4.
WHEN The last statement of the program, after the task has finished.
WHY The output order shows exactly when each piece of code ran.

> **Try it**
>
> 1. Add badgeTask.cancel() right after creating the task. What does the badge show now, and why?
> 2. Create a second Task that prints "Second" and await both values.

CODE · RUNS IN THE BROWSER

Actors protect shared state

An **actor** is a reference type that lets only one piece of code touch its state at a time. Callers outside the actor must `await`, because they may have to wait their turn. Here a thousand tasks tap like at the same moment, and the count still comes out exact.

```swift
actor LikeCounter {
    var likes = 0

    func like() {
        likes += 1
    }
}

let counter = LikeCounter()
await withTaskGroup(of: Void.self) { group in
    for _ in 1...1000 {
        group.addTask {
            await counter.like()
        }
    }
}
print(await counter.likes)
```

OUTPUT

```
1000
```

Line by line

1 `actor LikeCounter {`

- WHO: You, declaring an actor.
- WHAT: **Declares LikeCounter, a reference type with built-in protection for its state.**
- WHERE: Top level of the file; instances live on the heap.
- WHEN: Compile time; the compiler enforces the protection rules.
- WHY: Shared mutable state is exactly what actors are for.

2 `var likes = 0`

- WHO: The actor, which owns this property.
- WHAT: **Stored count of likes.**
- WHERE: Inside the actor, isolated from outside code.
- WHEN: Initialized to 0 when the actor is created, then changed only through like().
- WHY: Outside code can read it only with await.

4 `func like() {`

- WHO: The LikeCounter actor, declaring the only method that changes likes.
- WHAT: **A method that adds one like. Inside the actor, no await is needed.**
- WHERE: Inside the actor body, below the likes property.
- WHEN: Each time some task calls it.
- WHY: The actor runs one call at a time, so this body never overlaps with another call.

5 `likes += 1`

- WHO: The actor's isolated code.
- WHAT: **Reads the count, adds one and writes it back as one uninterrupted step.**
- WHERE: Inside like(), its single statement.
- WHEN: Once per call to like(), a thousand times in this program.
- WHY: Without the actor, two threads could read the same old value and lose an update.

```
9  let counter = LikeCounter()
```

WHO The actor's default initializer, called with no arguments.
WHAT **Creates one shared counter.**
WHERE A top-level constant holding a reference.
WHEN At run time, before the task group starts.
WHY Actors are Sendable, so many tasks may safely share this one reference.

```
10 await withTaskGroup(of: Void.self) { group in
```

WHO A task group, created and awaited at top level.
WHAT **Creates a group of child tasks that return nothing, and waits for all of them.**
WHERE Top-level code; the group lives only inside this closure.
WHEN Finishes once every child task has finished.
WHY Task groups suit a variable number of parallel jobs.

```
11 for _ in 1...1000 {
```

WHO A for-in loop over 1…1000, ignoring the counter value.
WHAT **Repeats 1,000 times. The underscore ignores the loop number.**
WHERE Inside the group closure.
WHEN Immediately, adding tasks quickly.
WHY Simulates a thousand people tapping like at once.

```
12 group.addTask {
```

WHO The task group, given one child task per loop pass.
WHAT **Adds a child task that may run in parallel with the others.**
WHERE Inside the loop body, inside the group closure.
WHEN The child runs as soon as the system schedules it.
WHY These tasks really can run on different threads at the same time.

```
13 await counter.like()
```

WHO Each child task, calling into the actor.
WHAT **Waits for its turn, then adds one like.**
WHERE Inside each child task.
WHEN Whenever the actor is free.
WHY The await marks a possible wait: the actor may be busy with another caller.

```
17 print(await counter.likes)
```

WHO print, reading isolated state from outside.
WHAT **Prints 1000: no update was lost.**
WHERE Top-level code; the console shows 1000.
WHEN After the whole group finishes.
WHY Even reading needs await from outside the actor.

> **Try it**
>
> 1. Change actor to final class and add a print. In the Swift 6 language mode the compiler rejects it. Why?
> 2. Add a method reset() to the actor and call it from outside. Do you need await?

INTERACTIVE

Race the counter

Two tasks each add one to a shared counter five times, so the total should be 10. Each increment is three tiny steps: read, add, write. Step through the unprotected class and watch for a lost update, when

both tasks read the same value. Then switch to the actor, and run each version a thousand times.

This step is an interactive simulation in the HTML version.

CONCEPT

The main actor and Swift 6 data-race safety

The **main actor** is a special actor tied to the main thread. Mark a type or function `@MainActor` and Swift guarantees it runs there, which is exactly where UI state belongs. SwiftUI views already run on the main actor.

The **Swift 6 language mode** checks for data races at compile time. A value may cross between tasks or actors only if it is `Sendable`, meaning safe to share. A struct or enum whose parts are all Sendable gets the conformance implicitly, but only when the type is not public; a public type must declare `: Sendable` itself. Actors always qualify. A class with mutable properties does not.

Default isolation	New Xcode 26 app targets set Default Actor Isolation to MainActor, so your code runs on the main actor unless you opt out.
Approachable Concurrency	Also on in new projects: async functions stay on the caller's actor, and you mark work that should leave it with @concurrent.
Language mode	New projects still use the Swift 5 mode. Set Swift Language Version to 6 for full compile-time race checking.

CODE · READ-ALONG

A main-actor model with background work

This is the shape of a real screen model in iOS 26. The model lives on the main actor, so SwiftUI can read its properties safely. The slow work is marked `@concurrent`, so it runs on the background thread pool, and its `Sendable` result travels back to the main actor. This needs the iOS SDK, so it is explained here rather than run.

```swift
import SwiftUI

struct Forecast: Sendable {
    let city: String
    let high: Int
}

@MainActor
@Observable
final class ForecastModel {
    var forecast: Forecast?
    var isLoading = false
    func refresh() async {
        isLoading = true
        defer { isLoading = false }
        forecast = await Self.download(city: "Lisbon")
    }
    @concurrent
    static func download(city: String) async -> Forecast {
        try? await Task.sleep(for: .seconds(1))
        return Forecast(city: city, high: 24)
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The compiler, loading SwiftUI, which also brings in the Observation framework.
- **WHAT** **Loads SwiftUI, which also brings in the Observation framework behind @Observable.**
- **WHERE** First line of the file.
- **WHEN** Compile time, before any code in the file is checked.
- **WHY** One import gives the model everything its views will need.

3 `struct Forecast: Sendable {`

- **WHO** You, declaring the data that crosses between threads.
- **WHAT** **Declares a struct and states that it is safe to send between concurrency domains.**
- **WHERE** Top level of the file.
- **WHEN** Compile time; the compiler verifies every stored property is Sendable too.
- **WHY** Values returned from background work must be Sendable to reach the main actor.

4 `let city: String`

- **WHO** Every Forecast value, which stores the city it describes.
- **WHAT** **A stored constant holding the city name, one of the two Sendable parts.**
- **WHERE** Stored inside each Forecast value, first property.
- **WHEN** Assigned once when download builds the Forecast.
- **WHY** String is Sendable, so it does not break the struct's promise.

5 `let high: Int`

- **WHO** Every Forecast value, which stores the high temperature.
- **WHAT** **A stored constant holding the high temperature, an Int and therefore Sendable.**
- **WHERE** Stored inside each Forecast value, second property.
- **WHEN** Assigned once when download builds the Forecast on the background thread pool.
- **WHY** Immutable value types are the easiest things to share safely.

8 `@MainActor`

WHO The compiler, applying actor isolation.
WHAT **Pins every property and method of the class below to the main actor.**
WHERE Attribute on the class declaration.
WHEN Compile time; calls from elsewhere must await.
WHY UI state must only change on the main thread. With default MainActor isolation this line is implied, but writing it makes the rule explicit.

9 `@Observable`

WHO The Observation framework macro.
WHAT **Makes SwiftUI views that read this model update automatically when its properties change.**
WHERE Attribute on the class declaration.
WHEN Expanded at compile time.
WHY @Observable replaces the older ObservableObject and @Published pattern.

10 `final class ForecastModel {`

WHO You, declaring the screen's model.
WHAT **A final class, because several views share one model with identity.**
WHERE Top level of the file; the object lives on the heap.
WHEN Compile time; the compiler records the class and applies both attributes above it.
WHY This is the classic case where a class beats a struct: shared, observed state.

11 `var forecast: Forecast?`

WHO The model, on the main actor.
WHAT **The loaded forecast, or nil before the first load.**
WHERE Inside the object, isolated to the main actor.
WHEN Changes when a load finishes.
WHY Views reading forecast redraw when it changes.

12 `var isLoading = false`

WHO The model, on the main actor.
WHAT **Tracks whether a refresh is in progress.**
WHERE Inside the model object, next to forecast; SwiftUI reads it to show a spinner.
WHEN Set to true at the start of refresh() and back to false when refresh() exits.
WHY A view can show a ProgressView while this is true.

13 `func refresh() async {`

WHO The ForecastModel, declaring the async method a view will call.
WHAT **An async method a view calls from .task or a button's Task.**
WHERE Isolated to the main actor.
WHEN Each refresh, typically when the user pulls down or the view appears.
WHY Because it is main-actor isolated, it can update UI state directly.

14 `isLoading = true`

WHO The model, flipping its own flag on the main actor.
WHAT **Turns on the loading indicator.**
WHERE On the main actor.
WHEN At the start of the refresh.
WHY Safe: this code is guaranteed to run on the main actor.

```
15  defer { isLoading = false }
```

WHO A defer statement inside refresh().
WHAT **Guarantees the indicator turns off when refresh ends.**
WHERE On the main actor.
WHEN When refresh exits, on every path.
WHY The same cleanup pattern as the spinner in the errors lesson.

```
16  forecast = await Self.download(city: "Lisbon")
```

WHO The model, awaiting background work.
WHAT **Suspends while download runs elsewhere, then stores the result back on the main actor.**
WHERE On the main actor, before and after the await.
WHEN Resumes when the download finishes.
WHY While suspended, the main thread keeps drawing and handling taps.

```
18  @concurrent
```

WHO The compiler, applying a Swift 6.2 attribute.
WHAT **Marks the function below to run on the concurrent thread pool, off the main actor.**
WHERE Inside ForecastModel, applied to the static method on the next line.
WHEN Compile time; affects where the body runs.
WHY With Approachable Concurrency, leaving the caller's actor is an explicit choice, and this is how you make it.

```
19  static func download(city: String) async -> Forecast {
```

WHO The ForecastModel type, declaring a static method that belongs to the type, not an instance.
WHAT **A static async function that produces a Forecast. @concurrent also makes it nonisolated.**
WHERE Runs on a background thread, not the main actor.
WHEN When awaited by refresh().
WHY It touches no model properties, so it is safe to run away from the main actor.

```
20  try? await Task.sleep(for: .seconds(1))
```

WHO The concurrency runtime, suspending download on the background thread pool.
WHAT **Stands in for a network request or heavy parsing.**
WHERE Inside download, off the main actor.
WHEN Each call to download, standing in for a one-second network request.
WHY Real code would use URLSession here, as the networking module shows.

```
21  return Forecast(city: city, high: 24)
```

WHO The download function, building the Sendable value that crosses back to the main actor.
WHAT **Returns a Sendable value to the main actor.**
WHERE The last line of download, still off the main actor.
WHEN After the simulated work, once the sleep has ended.
WHY Only Sendable values may cross back; the compiler checks it.

Try it

1. If download set isLoading itself, what would the compiler say, and why?
2. Remove Sendable from Forecast. Does it still conform implicitly?

CHECK YOURSELF

Check your understanding

1. In the Task example, which line prints second?
   A. Badge shows 4
   B. Tap handler finished
   C. Task returned 4
   D. Button tapped
2. Three independent downloads take 1, 2 and 1 seconds. With `async let`, about how long until all three are done?
   A. 4 seconds
   B. 1 second
   C. 2 seconds
   D. 3 seconds
3. Why does the `LikeCounter` actor always end at exactly 1000?
   A. Actor isolation runs one call at a time
   B. Actors always run on the main thread
   C. withTaskGroup runs the tasks one after another
   D. Integer addition in Swift cannot be interrupted
4. In a new Xcode 26 app project, where does a function you write run by default?
   A. On a random background thread
   B. On whichever thread called it last
   C. Nowhere until you mark it @concurrent
   D. On the main actor
5. Which value can be sent between tasks with no extra work?
   A. An instance of a class with a var property
   B. A closure that changes a captured var
   C. A struct whose stored properties are all Sendable
   D. A global var array

Answers: Concurrency: async/await and Actors

1. **B. Tap handler finished** `Task { }` returns immediately, so the tap handler keeps going and prints first. The task body runs only after the current code gives up the main actor at `await badgeTask.value`. That non-blocking start is why tasks keep buttons responsive.
2. **C. 2 seconds** `async let` starts all three at once, so the total is the slowest one: 2 seconds. Four seconds is the sequential `await` version, where the times add up.
3. **A. Actor isolation runs one call at a time** Actor isolation serializes access: each `like()` finishes before the next begins, so no read-add-write is interrupted. The child tasks really do run in parallel, and ordinary actors are not the main actor; the protection comes from the actor itself.
4. **D. On the main actor** New app targets set Default Actor Isolation to MainActor, so code is single-threaded on the main actor unless you opt out with `nonisolated` or `@concurrent`. That keeps beginner code free of data races by default.
5. **C. A struct whose stored properties are all Sendable** A non-public struct of Sendable parts is Sendable implicitly, because each receiver gets its own copy; a public one must declare the conformance. A class with mutable state, a closure that mutates captured state, and a global var can all be changed from two places at once, which is exactly a data race.

Programming Paradigms in Swift

CONCEPT

One problem, five styles

A **programming paradigm** is a style of organizing code: what you treat as the main building block, and how you express change. Swift is *multi-paradigm*, and good Swift code mixes styles on purpose.

This lesson solves one small problem five ways: given today's chores, how many minutes of work are already done? The data never changes: Email Sam (10 min, done), Gym (45 min, open), Read ch. 3 (30 min, done). The answer is always 40.

Imperative	Step-by-step commands that change variables.
Object-oriented	Objects bundle state with the methods that guard it.
Protocol-oriented	Capabilities described by protocols and shared through extensions.
Functional	Transform data with functions, without changing anything in place.
Declarative	Describe what the result should be; the framework works out how.

CODE · RUNS IN THE BROWSER

Imperative: tell the computer each step

Imperative code is a recipe: create a variable, loop, test, update. It maps directly onto what the processor does, which makes it easy to trace by hand. Here the data lives in three **parallel arrays**, where the same index in each array describes one chore.

```swift
let titles = ["Email Sam", "Gym", "Read ch. 3"]
let minutes = [10, 45, 30]
let done = [true, false, true]

var total = 0
for index in 0..<titles.count {
    if done[index] {
        total += minutes[index]
    }
}
print("Completed: \(total) min")
```

OUTPUT

```
Completed: 40 min
```

Line by line

1 `let titles = ["Email Sam", "Gym", "Read ch. 3"]`

WHO	Your code, creating the first of three parallel arrays.
WHAT	**An array of chore names.**
WHERE	A top-level constant holding an array of String.
WHEN	At run time, as the first statement.
WHY	Parallel arrays are the simplest structure, and also the most fragile.

```
2  let minutes = [10, 45, 30]
```

WHO Your code, creating a second array that lines up with titles by index.
WHAT **The minutes for each chore, in the same order.**
WHERE A top-level constant holding an array of Int.
WHEN At run time, right after titles.
WHY Index 1 here belongs to index 1 in titles, a rule nothing enforces.

```
3  let done = [true, false, true]
```

WHO Your code, creating the third parallel array.
WHAT **Whether each chore is finished.**
WHERE A top-level constant holding an array of Bool.
WHEN At run time, right after minutes.
WHY Sort one array without the others and the data silently goes wrong.

```
5  var total = 0
```

WHO Your code, creating the accumulator the loop will change.
WHAT **A mutable running total, starting at zero.**
WHERE A top-level var, declared before the loop so it survives after it.
WHEN At run time, before the loop starts.
WHY Imperative code builds results by changing variables step by step.

```
6  for index in 0..<titles.count {
```

WHO A for-in loop over a half-open range.
WHAT **Visits indices 0, 1 and 2.**
WHERE Top-level code; index is valid only inside the loop body.
WHEN Three passes, one per index of titles.
WHY Indices let one loop walk all three arrays together.

```
7  if done[index] {
```

WHO An if statement, reading the Bool at the current index.
WHAT **Checks whether the chore at this index is finished.**
WHERE Inside the loop body, guarding the addition.
WHEN Each pass of the loop, three times in total.
WHY Explicit control flow is the hallmark of imperative style.

```
8  total += minutes[index]
```

WHO The total variable, growing by the minutes of each finished chore.
WHAT **Adds this chore's minutes.**
WHERE Inside the if, so it runs only for done chores.
WHEN Only for finished chores.
WHY Mutation in place: the variable changes on each pass.

```
11 print("Completed: \(total) min")
```

WHO print, showing the final total.
WHAT **Prints Completed: 40 min.**
WHERE Top-level code; the console shows Completed: 40 min.
WHEN The last statement of the program, after the loop ends.
WHY Correct, but the reader had to simulate the loop to trust it.

> **Try it**
>
> 1. Add a fourth chore to only two of the three arrays. What happens when you run it?
> 2. Count how many chores are still open with a second variable in the same loop.

CODE · RUNS IN THE BROWSER

Object-oriented: an object guards its state

Object-oriented code bundles data with the methods allowed to change it. Outside code cannot touch the `private` properties; it can only send messages such as `add` and `complete`. This is **encapsulation**: the object keeps its own state consistent.

```
class TaskList {
    private var open: [String: Int] = [:]
    private var minutesDone = 0

    func add(_ title: String, minutes: Int) {
        open[title] = minutes
    }
    func complete(_ title: String) {
        if let minutes = open.removeValue(forKey: title) {
            minutesDone += minutes
        }
    }
    var report: String {
        "Completed: \(minutesDone) min, \(open.count) open"
    }
}

let today = TaskList()
today.add("Email Sam", minutes: 10)
today.add("Gym", minutes: 45)
today.add("Read ch. 3", minutes: 30)
today.complete("Email Sam")
today.complete("Read ch. 3")
print(today.report)
```

OUTPUT

```
Completed: 40 min, 1 open
```

Line by line

1 `class TaskList {`

- WHO — You, declaring a class.
- WHAT — **Starts an object type that owns today's chores.**
- WHERE — Top level of the file; instances live on the heap.
- WHEN — Compile time; the compiler records TaskList as a class with private state.
- WHY — A single shared list that several screens update is a natural object.

2 `private var open: [String: Int] = [:]`

- WHO — The TaskList object, keeping this hidden.
- WHAT — **A dictionary from chore title to minutes for unfinished chores.**
- WHERE — Inside the object; private hides it from all outside code.
- WHEN — Initialized to an empty dictionary when a TaskList is created.
- WHY — Hiding state means no other code can put the list into an invalid state.

```
3  private var minutesDone = 0
```

WHO Every TaskList object, which keeps its own running total.
WHAT **Running total of completed minutes, also hidden.**
WHERE Stored inside the object, hidden from outside code by private.
WHEN Initialized to zero when the object is created, then changed only by complete.
WHY The object, not its callers, is responsible for keeping this correct.

```
5  func add(_ title: String, minutes: Int) {
```

WHO The TaskList class, declaring the method that records a new chore.
WHAT **A method, or message, that adds an open chore.**
WHERE Inside the class, below the two private properties.
WHEN Each call to add, three times in this program.
WHY The only way in is through methods the object controls.

```
6  open[title] = minutes
```

WHO The object add was called on, writing into its private dictionary.
WHAT **Stores the chore's minutes under its title.**
WHERE Changes the object's private dictionary.
WHEN Each add call, immediately.
WHY Class methods can change properties without mutating.

```
8  func complete(_ title: String) {
```

WHO The TaskList class, declaring the method that marks a chore finished.
WHAT **A message that marks a chore finished.**
WHERE Inside the class, below add.
WHEN Each call to complete, twice in this program.
WHY Moving minutes from open to done happens in one place, so it is always done the same way.

```
9  if let minutes = open.removeValue(forKey: title) {
```

WHO The dictionary's removeValue method.
WHAT **Removes the chore and returns its minutes, or nil if it was not open.**
WHERE Inside complete, its only statement.
WHEN Each call; removeValue returns nil when the title is not open.
WHY Completing an unknown or already-finished chore does nothing, a rule the object enforces.

```
10 minutesDone += minutes
```

WHO The object complete was called on, updating its private total.
WHAT **Adds the finished chore's minutes to the total.**
WHERE Inside the if let.
WHEN Only when the chore was open.
WHY The total can never count a chore twice.

```
13 var report: String {
```

WHO The TaskList class, declaring a read-only computed property.
WHAT **A computed property describing progress.**
WHERE Inside the class, the only public way to see the state.
WHEN Each read of report, once in this program.
WHY Callers ask the object for an answer instead of reading its insides.

```
14  "Completed: \(minutesDone) min, \(open.count) open"
```

WHO The report getter, reading both private properties.
WHAT **Builds the report text.**
WHERE The single-expression body of report.
WHEN Each read of report, after all adds and completes.
WHY Only the object knows how its state turns into a summary.

```
18  let today = TaskList()
```

WHO The default initializer, called because every property has a default.
WHAT **Creates one list object.**
WHERE A top-level constant holding a reference.
WHEN At run time; the object is allocated on the heap here.
WHY let is fine: the reference stays fixed while the object changes.

```
19  today.add("Email Sam", minutes: 10)
```

WHO Your code, sending a message.
WHAT **Adds the first chore.**
WHERE Top-level code, calling a method that changes the object.
WHEN At run time, the first chore added.
WHY Reads like an instruction to the object.

```
20  today.add("Gym", minutes: 45)
```

WHO Your code, adding the second chore.
WHAT **Adds the second chore.**
WHERE Top-level code, calling add again on the same object.
WHEN At run time, right after the first add.
WHY Gym stays open in this example.

```
21  today.add("Read ch. 3", minutes: 30)
```

WHO Your code, adding the third chore.
WHAT **Adds the third chore.**
WHERE Top-level code, the last add call.
WHEN At run time; the dictionary now has three entries.
WHY All three chores are now open.

```
22  today.complete("Email Sam")
```

WHO Your code, completing the first chore.
WHAT **Finishes the first chore.**
WHERE Top-level code, calling complete on the same object.
WHEN At run time; Email Sam moves from open to minutesDone.
WHY The object moves 10 minutes to done.

```
23  today.complete("Read ch. 3")
```

WHO Your code, completing the third chore.
WHAT **Finishes the third chore.**
WHERE Top-level code, the second and last complete call.
WHEN At run time; Gym is left open.
WHY The object moves 30 more minutes to done.

```
24 print(today.report)
```

WHO	print, reading the object's report.
WHAT	**Prints Completed: 40 min, 1 open.**
WHERE	Top-level code; the console shows Completed: 40 min, 1 open.
WHEN	The last statement of the program.
WHY	The caller never saw the dictionary, only the answer.

> **Try it**
>
> 1. Call today.complete("Email Sam") a second time. Does the total change?
> 2. Try today.minutesDone = 999 at the end. Why does the compiler refuse?

CODE · RUNS IN THE BROWSER

Protocol-oriented: share a capability

Protocol-oriented code asks: what *capability* does the problem need? Anything with minutes and a done flag is `Trackable`. One extension then gives every sequence of Trackable values a `completedMinutes` property, whether it holds chores, workouts or study sessions.

```swift
protocol Trackable {
    var minutes: Int { get }
    var isDone: Bool { get }
}

extension Sequence where Element: Trackable {
    var completedMinutes: Int {
        var total = 0
        for item in self where item.isDone {
            total += item.minutes
        }
        return total
    }
}

struct Chore: Trackable {
    let title: String
    let minutes: Int
    let isDone: Bool
}
let today = [Chore(title: "Email Sam", minutes: 10, isDone: true),
             Chore(title: "Gym", minutes: 45, isDone: false),
             Chore(title: "Read ch. 3", minutes: 30, isDone: true)]
print("Completed: \(today.completedMinutes) min")
```

OUTPUT

```
Completed: 40 min
```

Line by line

1 `protocol Trackable {`

- WHO — You, declaring a protocol.
- WHAT — **Names the capability "has minutes and can be done".**
- WHERE — Top level of the file.
- WHEN — Compile time; the compiler records Trackable and its two requirements.
- WHY — The protocol describes what the calculation needs, nothing more.

2 `var minutes: Int { get }`

- WHO — The Trackable protocol, stating its first requirement.
- WHAT — **Requires a readable minutes property.**
- WHERE — Inside the protocol body, first requirement.
- WHEN — Compile time; each adopting type must supply a readable minutes.
- WHY — Any type with minutes can join in.

3 `var isDone: Bool { get }`

- WHO — The Trackable protocol, stating its second requirement.
- WHAT — **Requires a readable done flag.**
- WHERE — Inside the protocol body, the last requirement.
- WHEN — Compile time; each adopting type must supply a readable isDone.
- WHY — Two small requirements make the protocol easy to adopt.

6 `extension Sequence where Element: Trackable {`

- WHO — You, extending a standard library protocol with a constraint.
- WHAT — **Adds code to every sequence, such as an array, whose elements are Trackable.**
- WHERE — Top level of the file.
- WHEN — Compile time; the compiler attaches the extension to every sequence whose elements are Trackable.
- WHY — Constrained extensions share behavior with exactly the types that qualify.

7 `var completedMinutes: Int {`

- WHO — Every sequence whose elements are Trackable, which gains this property from the extension.
- WHAT — **A computed property available on arrays of chores, workouts and so on.**
- WHERE — Inside the constrained extension, so arrays of Chore get it for free.
- WHEN — Each read of completedMinutes, once in this program.
- WHY — Written once, reused by every Trackable type.

8 `var total = 0`

- WHO — The completedMinutes getter, creating a local accumulator.
- WHAT — **A local running total.**
- WHERE — Local to the getter.
- WHEN — Each read of the property, starting from zero every time.
- WHY — The body may be imperative; what makes this protocol-oriented is where it lives.

9 `for item in self where item.isDone {`

- WHO — A for-in loop over the sequence itself.
- WHAT — **Visits only finished items. self is the sequence the property was read on.**
- WHERE — Inside the getter, looping over self, the sequence the property was read on.
- WHEN — Once per finished item.
- WHY — item is known to be Trackable, so minutes and isDone are available.

```
10  total += item.minutes
```

WHO The completedMinutes getter, adding the current item's minutes.
WHAT **Adds each finished item's minutes.**
WHERE Inside the loop, reached only for items whose isDone is true.
WHEN Once per finished item.
WHY Works for any conforming type without knowing which one.

```
12  return total
```

WHO The completedMinutes getter, handing back the finished sum.
WHAT **Returns the accumulated minutes as the value of the property.**
WHERE End of the getter.
WHEN After the loop has visited every item in the sequence.
WHY Callers see only a clean property.

```
16  struct Chore: Trackable {
```

WHO You, adopting the capability.
WHAT **Declares Chore as Trackable.**
WHERE Top level of the file.
WHEN Compile time; the compiler checks that Chore supplies minutes and isDone.
WHY A plain value type, no superclass needed.

```
17  let title: String
```

WHO Every Chore value, which stores its own title.
WHAT **Stored title; extra data the protocol does not need.**
WHERE Stored inside each Chore value, first of three properties; the extension never reads it.
WHEN Assigned once by the memberwise initializer.
WHY Conforming types stay free to hold anything else.

```
18  let minutes: Int
```

WHO Every Chore value, which stores the minutes that completedMinutes adds up.
WHAT **Satisfies the minutes requirement.**
WHERE Stored inside each Chore value; this satisfies the first Trackable requirement.
WHEN Assigned once by the memberwise initializer, then read by the extension's loop.
WHY A stored let meets a { get } requirement.

```
19  let isDone: Bool
```

WHO Every Chore value, which stores the flag the extension filters on.
WHAT **Satisfies the isDone requirement.**
WHERE Stored inside each Chore value; this satisfies the second Trackable requirement.
WHEN Assigned once by the memberwise initializer, then read by the where clause in the loop.
WHY With both requirements met, Chore gets completedMinutes for free.

```
21  let today = [Chore(title: "Email Sam", minutes: 10, isDone: true),
```

WHO Your code, building the array that the extension will total.
WHAT **Starts an array of chores with the first one.**
WHERE A top-level constant of type [Chore].
WHEN At run time, as the array literal is evaluated.
WHY An array is a Sequence, so the extension applies.

```
22  Chore(title: "Gym", minutes: 45, isDone: false),
```

WHO The memberwise initializer, creating the one chore that is still open.
WHAT **Adds the open gym chore.**
WHERE The second element of the today array.
WHEN While the array is built.
WHY The open chore is what proves the extension filters on isDone instead of adding everything.

```
23  Chore(title: "Read ch. 3", minutes: 30, isDone: true)]
```

WHO The memberwise initializer, creating the last chore.
WHAT **Adds the reading chore and closes the array.**
WHERE The third element of the today array.
WHEN While the array is built.
WHY A second finished chore makes the total 40 rather than 10, so the sum visibly involves more than one item.

```
24  print("Completed: \(today.completedMinutes) min")
```

WHO The completedMinutes property that the extension added to the array.
WHAT **Prints Completed: 40 min.**
WHERE Top-level code; the console shows Completed: 40 min.
WHEN The last statement of the program; the getter runs during this line.
WHY Reads like a built-in feature of arrays, because the extension made it one.

Try it

1. Add struct Workout: Trackable with minutes and isDone, make an array of two workouts, and print its completedMinutes.
2. Remove isDone from Chore. Which line does the compiler complain about?

COMPARE

Loop or pipeline

Both programs use the same `Chore` struct. The left builds the answer by changing a variable inside a loop. The right is **functional**: it passes the data through `filter`, `map` and `reduce`, each returning a new value, and never changes anything in place.

A: Imperative loop

```swift
struct Chore {
    let title: String
    let minutes: Int
    let isDone: Bool
}

let today = [Chore(title: "Email Sam", minutes: 10, isDone: true),
             Chore(title: "Gym", minutes: 45, isDone: false),
             Chore(title: "Read ch. 3", minutes: 30, isDone: true)]

var total = 0
for chore in today {
    if chore.isDone {
        total += chore.minutes
    }
}
print("Completed: \(total) min")
```

OUTPUT

```
Completed: 40 min
```

1 `struct Chore {`

- WHO — You, declaring a value type.
- WHAT — **The same chore model as before.**
- WHERE — Top level of the file.
- WHEN — Compile time; the compiler records the Chore shape used by both sides.
- WHY — A struct keeps each chore's data together, unlike parallel arrays.

2 `let title: String`

- WHO — Every Chore value, which stores its own title.
- WHAT — **A stored constant holding the chore name, never read by the loop.**
- WHERE — Stored inside each Chore value, first of three properties.
- WHEN — Assigned once by the memberwise initializer.
- WHY — The title is here so each element reads like a real chore, even though only minutes and isDone affect the total.

3 `let minutes: Int`

- WHO — Every Chore value, which stores the minutes the loop adds up.
- WHAT — **A stored constant holding the chore's length in minutes.**
- WHERE — Stored inside each Chore value, second property.
- WHEN — Assigned once by the memberwise initializer, then read inside the if.
- WHY — Keeping minutes next to isDone in one struct is what lets the loop read both from a single chore value.

4 `let isDone: Bool`

- WHO — Every Chore value, which stores whether it is finished.
- WHAT — **A stored constant holding the done flag that the if statement tests.**
- WHERE — Stored inside each Chore value, the last property.
- WHEN — Assigned once by the memberwise initializer, then tested on every pass of the loop.
- WHY — The flag decides which chores count, so it belongs to the chore itself rather than to a separate array.

```
7  let today = [Chore(title: "Email Sam", minutes: 10, isDone: true),
```

WHO Your code, building the three chores the loop will visit.
WHAT **Starts an array literal of three Chore values, the first of them finished.**
WHERE A top-level constant holding an array of Chore.
WHEN At run time, as the array literal is evaluated.
WHY The left side uses exactly these values so that its loop and the pipeline on the right can be compared on equal terms.

```
8  Chore(title: "Gym", minutes: 45, isDone: false),
```

WHO The memberwise initializer, creating the second chore.
WHAT **The one chore that is not done, so the loop must skip it.**
WHERE The second element of the today array.
WHEN While the array is built.
WHY An open chore gives the if statement in the loop something to skip, which is the whole point of the check.

```
9  Chore(title: "Read ch. 3", minutes: 30, isDone: true)]
```

WHO The memberwise initializer, creating the third chore.
WHAT **The last chore, finished, worth 30 minutes.**
WHERE The third element of the today array.
WHEN While the array is built.
WHY A second finished chore makes the loop add twice, so the output shows accumulation rather than a single value.

```
11 var total = 0
```

WHO Your code, creating the accumulator the loop will change.
WHAT **Declares a mutable Int that starts at zero and gathers the minutes.**
WHERE A top-level var, declared before the loop so it still exists after it.
WHEN At run time, before the loop starts.
WHY The answer is built by mutation.

```
12 for chore in today {
```

WHO A for-in loop over the today array, binding each element to chore.
WHAT **Visits the three chores in order, binding each to chore for the body.**
WHERE Top-level code; chore is valid only inside the loop body.
WHEN Three passes, one per element of today.
WHY Explicit iteration: you say how to walk the data.

```
13 if chore.isDone {
```

WHO An if statement, reading the done flag of the current chore.
WHAT **Lets the body run only for finished chores and skips the Gym chore.**
WHERE Inside the loop body, guarding the addition.
WHEN Each pass of the loop, three times in total.
WHY Explicit branching: you say how to choose.

```
14 total += chore.minutes
```

WHO The total variable, growing by the minutes of the current chore.
WHAT **Adds the current chore's minutes to the running total.**
WHERE Inside the if, so it runs only for done chores.
WHEN On the first and third passes, the two finished chores.
WHY State changes on every pass.

```
17 print("Completed: \(total) min")
```

WHO print, showing the final total.
WHAT **Prints Completed: 40 min.**
WHERE Top-level code; the console shows Completed: 40 min.
WHEN The last statement of the left program, after the loop ends.
WHY Clear, but four lines describe how, not what.

B: Functional pipeline

```
1  struct Chore {
2      let title: String
3      let minutes: Int
4      let isDone: Bool
5  }
6
7  let today = [Chore(title: "Email Sam", minutes: 10, isDone: true),
8               Chore(title: "Gym", minutes: 45, isDone: false),
9               Chore(title: "Read ch. 3", minutes: 30, isDone: true)]
10
11 let total = today
12     .filter { $0.isDone }
13     .map { $0.minutes }
14     .reduce(0, +)
15 print("Completed: \(total) min")
```

OUTPUT

```
Completed: 40 min
```

```
1 struct Chore {
```

WHO You, declaring the same value type.
WHAT **The same chore model.**
WHERE Top level of the file.
WHEN Compile time; the compiler records the same Chore shape as on the left.
WHY Functional style works best with immutable values like these.

```
2 let title: String
```

WHO Every Chore value, which stores its own title.
WHAT **A stored constant holding the chore name, untouched by the pipeline.**
WHERE Stored inside each Chore value, first of three properties.
WHEN Assigned once by the memberwise initializer.
WHY Chore is unchanged from the left so that the only difference to study is how the total is computed.

```
3 let minutes: Int
```

WHO Every Chore value, which stores the minutes the map stage extracts.
WHAT **A stored constant holding the chore's length in minutes.**
WHERE Stored inside each Chore value, second property.
WHEN Assigned once by the memberwise initializer, then read by the map closure.
WHY The map stage can only pull out minutes because every chore carries its own copy of this property.

```
4  let isDone: Bool
```

WHO Every Chore value, which stores the flag the filter stage tests.
WHAT **A stored constant holding the done flag.**
WHERE Stored inside each Chore value, the last property.
WHEN Assigned once by the memberwise initializer, then read by the filter closure.
WHY The filter closure needs a Bool to test, and keeping it on the chore means no separate bookkeeping array.

```
7  let today = [Chore(title: "Email Sam", minutes: 10, isDone: true),
```

WHO Your code, building the same three chores as the left side.
WHAT **Starts an array literal of three Chore values, the first of them finished.**
WHERE A top-level constant holding an array of Chore.
WHEN At run time, as the array literal is evaluated.
WHY Using the same data on the right lets you compare the pipeline with the loop knowing the answer must be 40 again.

```
8  Chore(title: "Gym", minutes: 45, isDone: false),
```

WHO The memberwise initializer, creating the second chore.
WHAT **The one chore that is not done, so filter drops it.**
WHERE The second element of the today array.
WHEN While the array is built.
WHY The open chore is what the filter stage removes, which shows filter doing the job the if statement did on the left.

```
9  Chore(title: "Read ch. 3", minutes: 30, isDone: true)]
```

WHO The memberwise initializer, creating the third chore.
WHAT **The last chore, finished, worth 30 minutes.**
WHERE The third element of the today array.
WHEN While the array is built.
WHY With two finished chores, reduce has two numbers to add, so its result is visibly a sum.

```
11 let total = today
```

WHO Your code, starting a pipeline.
WHAT **Feeds the array into a chain of transformations. The result is a let.**
WHERE A top-level constant; the whole pipeline on the next three lines produces its value.
WHEN At run time, once, as the three chained calls run top to bottom.
WHY No var anywhere: nothing is changed in place.

```
12   .filter { $0.isDone }
```

WHO filter, a higher-order function (one that takes a function).
WHAT **Returns a new array with only the finished chores.**
WHERE First stage of the pipeline.
WHEN Runs once over the three chores.
WHY Says what to keep, not how to loop.

```
13   .map { $0.minutes }
```

WHO map, another higher-order function.
WHAT **Turns the finished chores into their minutes: [10, 30].**
WHERE The second stage of the pipeline, receiving only the finished chores from filter.
WHEN Runs over the filtered array.
WHY Each stage does one job and is easy to test alone.

```
14  .reduce(0, +)
```

WHO	reduce, the final higher-order function.
WHAT	**Combines the minutes into one number, starting from 0 and applying + to each.**
WHERE	The last stage of the pipeline, turning the array of minutes into one Int.
WHEN	Runs over the mapped array.
WHY	Passing the + operator as a function is a classic functional idiom.

```
15  print("Completed: \(total) min")
```

WHO	print, showing the final total.
WHAT	**Prints Completed: 40 min.**
WHERE	Top-level code; the console shows Completed: 40 min, the same as the left side.
WHEN	The last statement of the right program.
WHY	Same answer; the pipeline reads as a description of the result.

> **Takeaway**
>
> Both are correct. The pipeline has no mutable variable, so there is no running total to get wrong, and each stage reads as a sentence: keep the done ones, take their minutes, add them up. Loops still win for complex control flow, such as stopping early or updating several results at once.

CODE · LIVE SWIFTUI PREVIEW

Declarative: describe the screen

SwiftUI is **declarative** and **reactive**: `body` describes what the screen looks like for the current state, and SwiftUI redraws it when that state changes. Flip a toggle in the phone and the total updates on its own; no code ever tells the label to change.

```swift
import SwiftUI

struct Chore {
    let title: String
    let minutes: Int
    var isDone: Bool
}

struct TodayView: View {
    @State private var chores = [
        Chore(title: "Email Sam", minutes: 10, isDone: true),
        Chore(title: "Gym", minutes: 45, isDone: false),
        Chore(title: "Read ch. 3", minutes: 30, isDone: true),
    ]
    var body: some View {
        let sum = chores.reduce(0) { $1.isDone ? $0 + $1.minutes : $0 }
        List {
            ForEach(chores.indices, id: \.self) { i in
                Toggle(chores[i].title, isOn: $chores[i].isDone)
            }
            Text("Completed: \(sum) min")
        }
    }
}
#Preview { TodayView() }
```

Line by line

```
1  import SwiftUI
```

WHO The compiler, loading SwiftUI so View, List, ForEach, Toggle, Text and @State are available.
WHAT **Loads SwiftUI: views, @State and the preview.**
WHERE First line of the file.
WHEN Compile time, before any code in the file is checked.
WHY SwiftUI is Apple's declarative UI framework for every platform.

```
3  struct Chore {
```

WHO You, declaring the model.
WHAT **The same chore data, but isDone is now a var so a toggle can change it.**
WHERE Top level of the file.
WHEN Compile time; the compiler records Chore, this time with a mutable isDone.
WHY Declarative UI works best with simple value types as state.

```
4  let title: String
```

WHO Every Chore value, which stores the title the Toggle shows as its label.
WHAT **Stored title, shown as the toggle label.**
WHERE Stored inside each Chore value, first of three properties.
WHEN Assigned once when the @State array is first built.
WHY Never changes, so let.

```
5  let minutes: Int
```

WHO Every Chore value, which stores the minutes that the reduce in body adds up.
WHAT **A stored constant holding the chore's length in minutes.**
WHERE Stored inside each Chore value, second property.
WHEN Assigned once when the @State array is first built, then read on every recompute of body.
WHY The sum in body reads this property for every finished chore, so it has to live on the chore itself.

```
6  var isDone: Bool
```

WHO Every Chore value, whose isDone the Toggle binding writes to.
WHAT **The done flag, now mutable.**
WHERE Stored inside each Chore value, the last property, declared var so the binding can change it.
WHEN Changed when the user flips a toggle.
WHY Only what the user can change needs to be var.

```
9  struct TodayView: View {
```

WHO You, declaring a view.
WHAT **A struct that conforms to the View protocol.**
WHERE Top level of the file.
WHEN Compile time; the compiler checks that TodayView supplies a body.
WHY Views are lightweight struct values; SwiftUI creates and discards them freely.

```
10 @State private var chores = [
```

WHO SwiftUI's @State property wrapper.
WHAT **Declares the screen's source of truth: the chores array, owned by SwiftUI.**
WHERE Storage managed by SwiftUI, outside the struct value.
WHEN Created when the view first appears and kept across redraws.
WHY Changing @State is what triggers SwiftUI to recompute body.

```
11  Chore(title: "Email Sam", minutes: 10, isDone: true),
```

WHO The memberwise initializer, creating the first chore inside the @State array.
WHAT **The first chore, finished, so it counts toward the total from the first render.**
WHERE Inside the @State array.
WHEN Once, at first appearance.
WHY Same data as every version in this lesson.

```
12  Chore(title: "Gym", minutes: 45, isDone: false),
```

WHO The memberwise initializer, creating the second chore.
WHAT **The one chore that starts unfinished; flipping its toggle changes the total to 85.**
WHERE Inside the @State array.
WHEN Once, when the state is first created.
WHY Flip its toggle to see the total jump to 85.

```
13  Chore(title: "Read ch. 3", minutes: 30, isDone: true),
```

WHO The memberwise initializer, creating the last chore.
WHAT **The last chore, finished, worth 30 minutes.**
WHERE Inside the @State array.
WHEN Once, when the state is first created.
WHY The trailing comma keeps future additions to one line.

```
15  var body: some View {
```

WHO The View protocol's one requirement.
WHAT **Describes the screen as a value of some View type.**
WHERE Inside TodayView, the one property every View must supply.
WHEN Recomputed by SwiftUI whenever chores changes.
WHY body is a description, not a sequence of drawing commands.

```
16  let sum = chores.reduce(0) { $1.isDone ? $0 + $1.minutes : $0 }
```

WHO Your code, using a functional reduce.
WHAT **Adds the minutes of finished chores. $0 is the running total, $1 the current chore.**
WHERE A local constant inside body.
WHEN Every time body is recomputed.
WHY A functional calculation inside a declarative view: the paradigms mix naturally.

```
17  List {
```

WHO SwiftUI's List container, laying out the rows and the footer text.
WHAT **A scrolling, grouped list of rows.**
WHERE Inside body, as the outermost view.
WHEN Each recompute of body, after any toggle changes the @State array.
WHY You declare that there is a list; SwiftUI handles scrolling and layout.

```
18  ForEach(chores.indices, id: \.self) { i in
```

WHO SwiftUI's ForEach, producing one row per index of the chores array.
WHAT **Makes one row per chore index, identified by the index.**
WHERE Inside the List, above the total Text.
WHEN Each recompute of body; the id keeps each row tied to its index.
WHY Iterating indices lets each row bind to its own array element.

```
19 Toggle(chores[i].title, isOn: $chores[i].isDone)
```

WHO	A Toggle with a binding.
WHAT	**Shows a switch whose value reads and writes chores[i].isDone. The $ makes a binding.**
WHERE	One Toggle per row inside the ForEach, bound to that row's chore.
WHEN	Built each recompute; writes happen when the user flips it.
WHY	The binding writes straight into @State, which makes SwiftUI recompute body.

```
21 Text("Completed: \(sum) min")
```

WHO	SwiftUI's Text view, showing the sum computed at the top of body.
WHAT	**Shows the current total under the toggles.**
WHERE	Last row of the list.
WHEN	Each recompute, with the latest total.
WHY	Nothing ever updates this label directly; it is simply described from state.

```
25 #Preview { TodayView() }
```

WHO	Xcode's #Preview macro, which shows TodayView in the canvas.
WHAT	**Renders TodayView in the preview canvas.**
WHERE	At the end of the file.
WHEN	When the preview is displayed.
WHY	Previews let you check every state without running the whole app.

> **Try it**
>
> 1. Flip the Gym toggle on. What total do you expect before you look?
> 2. Add a fourth chore to the @State array. Notice that no other code needs to change.

CONCEPT

Choosing a paradigm

No paradigm wins everywhere. Experienced Swift developers mix them and pick the clearest tool for each part of the app. A typical SwiftUI app is declarative in its views, protocol-oriented and value-based in its models, functional when transforming data, object-oriented for a few shared models, and imperative inside small algorithms.

Imperative	Small algorithms with tricky control flow, early exits or tight performance needs.
Object-oriented	Shared objects with identity and a lifecycle, such as an @Observable model several screens watch.
Protocol-oriented	Behavior shared by many unrelated value types, like Trackable chores and workouts.
Functional	Transforming collections: filter, map, sort, total. Fewer variables, fewer bugs.
Declarative	User interfaces. Describe the screen for each state and let SwiftUI keep it current.

> **Clarity is the goal**
>
> Swift's API Design Guidelines put clarity at the point of use above brevity. Choose the style that makes the next reader, often you in six months, understand the code fastest.

CHECK YOURSELF

Check your understanding

1. Which paradigm does this line show?

```
let total = items.filter { $0.isDone }.map { $0.minutes }.reduce(0, +)
```

   A. Imperative
   B. Object-oriented
   C. Functional
   D. Declarative UI

2. In the SwiftUI version, what happens when the user turns on the Gym toggle?
   A. You must call a refresh method to update the total
   B. SwiftUI recomputes body, so the total updates
   C. The Text view edits its own string in place
   D. Nothing changes until the app restarts

3. Chores, workouts and study sessions are three unrelated structs, and all need a completed-minutes total. Which design fits best?
   A. A protocol plus one constrained extension
   B. A superclass that all three inherit from
   C. Copy the loop into each type
   D. Store everything in one array of Any

4. What is the main risk of the imperative version's three parallel arrays?
   A. They make the program slower
   B. Swift does not allow more than two arrays
   C. They cannot be printed
   D. They can silently fall out of step

Answers: Programming Paradigms in Swift

1. **C. Functional** It transforms data through functions and changes nothing in place, which is functional style. Imperative code would use a `var` and a loop. Declarative UI describes views, not calculations.
2. **B. SwiftUI recomputes body, so the total updates** The toggle's binding writes into `@State`; SwiftUI notices the change and recomputes `body`, which recalculates the total. That is reactive, declarative UI: you never update the label yourself.
3. **A. A protocol plus one constrained extension** A small protocol describes the shared capability, and one extension implements the total for all of them. Structs cannot inherit, so a superclass would force them to become classes, and `[Any]` loses type safety.
4. **D. They can silently fall out of step** Nothing ties index 1 in `titles` to index 1 in `minutes`, so one edit can silently mismatch the data. A `Chore` struct keeps each chore's fields together, which is why every later version used one.

Worked Examples

WORKED EXAMPLE

Struct copy versus class sharing, side by side

Problem.
Two assignments look identical but behave oppositely. Assign a settings struct to a second name and change the copy; then assign a profile class instance to a second name and change it. The prints and the diagram show which one shares.

Frame 1 of 6

A Settings struct is declared and its first value, a, is created with volume 5. The diagram shows one value box; the b box is empty because no copy exists yet.

```
struct Settings {
    var volume: Int
    var darkMode: Bool
}
var a = Settings(volume: 5, darkMode: false)
print("a volume: \(a.volume)")
```

OUTPUT SO FAR

```
a volume: 5
```

Frame 2 of 6

Assigning a to b copies the whole struct: two independent boxes now hold volume 5. The print from b matches a, so the change in the next frame will be meaningful.

```
struct Settings {
    var volume: Int
    var darkMode: Bool
}
var a = Settings(volume: 5, darkMode: false)
print("a volume: \(a.volume)")
+ var b = a
+ print("b volume: \(b.volume)")
```

OUTPUT SO FAR

```
a volume: 5
b volume: 5
```

Frame 3 of 6

Only the copy changes: b moves to volume 9 while a stays at 5, and the prints prove it. The diagram highlights the b box as the one that changed.

```
    struct Settings {
        var volume: Int
        var darkMode: Bool
    }
    var a = Settings(volume: 5, darkMode: false)
    print("a volume: \(a.volume)")
    var b = a
    print("b volume: \(b.volume)")
+   b.volume = 9
+   print("a volume: \(a.volume)")
+   print("b volume: \(b.volume)")
```

OUTPUT SO FAR

```
a volume: 5
b volume: 5
a volume: 5
b volume: 9
```

Frame 4 of 6

A Profile class joins the program and one object is created on the heap for Ana. The diagram switches shape: first is a reference, and the object lives in the heap box.

```
    struct Settings {
        var volume: Int
        var darkMode: Bool
    }
    var a = Settings(volume: 5, darkMode: false)
    print("a volume: \(a.volume)")
    var b = a
    print("b volume: \(b.volume)")
    b.volume = 9
    print("a volume: \(a.volume)")
    print("b volume: \(b.volume)")
+   class Profile {
+       var name: String
+       init(name: String) { self.name = name }
+   }
+   var first = Profile(name: "Ana")
+   print("first: \(first.name)")
```

OUTPUT SO FAR

```
a volume: 5
b volume: 5
a volume: 5
b volume: 9
first: Ana
```

Frame 5 of 6

Assigning first to second copies the reference, not the object: the diagram shows both names pointing at one heap box, and the print through second reads the same name.

```
  struct Settings {
      var volume: Int
      var darkMode: Bool
  }
  var a = Settings(volume: 5, darkMode: false)
  print("a volume: \(a.volume)")
  var b = a
  print("b volume: \(b.volume)")
  b.volume = 9
  print("a volume: \(a.volume)")
  print("b volume: \(b.volume)")
  class Profile {
      var name: String
      init(name: String) { self.name = name }
  }
  var first = Profile(name: "Ana")
  print("first: \(first.name)")
+ var second = first
+ print("second: \(second.name)")
```

OUTPUT SO FAR

```
a volume: 5
b volume: 5
a volume: 5
b volume: 9
first: Ana
second: Ana
```

Frame 6 of 6

Renaming through second changes the heap object, so reading through first shows Ana Maria too. One object, two names: the exact opposite of the struct copy above.

```
struct Settings {
    var volume: Int
    var darkMode: Bool
}
var a = Settings(volume: 5, darkMode: false)
print("a volume: \(a.volume)")
var b = a
print("b volume: \(b.volume)")
b.volume = 9
print("a volume: \(a.volume)")
print("b volume: \(b.volume)")
class Profile {
    var name: String
    init(name: String) { self.name = name }
}
var first = Profile(name: "Ana")
print("first: \(first.name)")
var second = first
print("second: \(second.name)")
+ second.name = "Ana Maria"
+ print("first: \(first.name)")
+ print("second: \(second.name)")
```

OUTPUT SO FAR

```
a volume: 5
b volume: 5
a volume: 5
b volume: 9
first: Ana
second: Ana
first: Ana Maria
second: Ana Maria
```

Line by line: the finished program

1 `struct Settings {`

WHO	You declare it; the compiler records the shape.
WHAT	**Starts Settings, a struct holding a volume and a dark-mode flag.**
WHERE	At the top of the program, before any value is created.
WHEN	Compile time; declaring the type runs no code.
WHY	Settings is plain data that gets copied around, which is what structs are for.

2 `var volume: Int`

WHO	Every Settings value, which stores its own volume.
WHAT	**Declares the stored volume property.**
WHERE	Inside each Settings value.
WHEN	Set by the memberwise initializer when a value is created.
WHY	A var, because the copy below changes its volume.

```
3  var darkMode: Bool
```

WHO Every Settings value, which stores its own flag.
WHAT **Declares the stored darkMode property.**
WHERE Inside each Settings value, beside volume.
WHEN Set by the memberwise initializer.
WHY Keeping both settings in one type keeps them together when copied.

```
5  var a = Settings(volume: 5, darkMode: false)
```

WHO The memberwise initializer Swift synthesized for Settings.
WHAT **Creates the first settings value, volume 5 and dark mode off, and stores it in a.**
WHERE A top-level variable holding the struct value itself.
WHEN At run time, when execution reaches this line.
WHY a holds the value directly, not a reference to an object.

```
6  print("a volume: \(a.volume)")
```

WHO The print function, reading a.
WHAT **Prints the volume stored in a: 5.**
WHERE Top-level code, writing to the console.
WHEN Right after a is created, and again after the copy changes.
WHY This is the baseline the copy experiment compares against.

```
7  var b = a
```

WHO Assignment, copying the struct value.
WHAT **Copies the whole Settings value from a into a new variable b.**
WHERE A second top-level variable; a and b now hold independent copies.
WHEN At run time, after a exists.
WHY Struct assignment copies, so b starts equal to a but shares nothing with it.

```
8  print("b volume: \(b.volume)")
```

WHO The print function, reading b.
WHAT **Prints the volume stored in b.**
WHERE Top-level code, writing to the console.
WHEN Right after the copy is made, and again after b changes.
WHY Equal values at first make the later difference meaningful.

```
9  b.volume = 9
```

WHO Your code, changing only the copy.
WHAT **Sets the volume of b to 9. The value in a is untouched.**
WHERE Changes the struct stored in b, in place.
WHEN After both prints of the starting value.
WHY This is the experiment: if assignment copied, a keeps 5; if it shared, a would change too.

```
10 print("a volume: \(a.volume)")
```

WHO The print function, reading a.
WHAT **Prints the volume stored in a: 5.**
WHERE Top-level code, writing to the console.
WHEN Right after a is created, and again after the copy changes.
WHY This is the baseline the copy experiment compares against.

```
11 print("b volume: \(b.volume)")
```

WHO The print function, reading b.
WHAT **Prints the volume stored in b.**
WHERE Top-level code, writing to the console.
WHEN Right after the copy is made, and again after b changes.
WHY Equal values at first make the later difference meaningful.

```
12 class Profile {
```

WHO You declare it; instances live on the heap.
WHAT **Starts Profile, a class holding a name.**
WHERE Under the struct part of the program.
WHEN Compile time; the class is declared before its first instance.
WHY A profile has identity that several names can share, which is what classes are for.

```
13 var name: String
```

WHO Every Profile object, which stores its own name on the heap.
WHAT **Declares the stored name property the initializer fills.**
WHERE Inside each Profile object.
WHEN Assigned once in init, changeable later through any reference.
WHY A var, because the sharing experiment renames through the second reference.

```
14 init(name: String) { self.name = name }
```

WHO You write it; classes get no memberwise initializer.
WHAT **Creates a Profile by copying the name argument into the object.**
WHERE Inside the class, on one line like the Account example in this module.
WHEN Each time Profile(name:) is called.
WHY self.name is the property; name alone is the parameter.

```
16 var first = Profile(name: "Ana")
```

WHO Your initializer, plus ARC counting the new reference.
WHAT **Creates one Profile object for Ana and stores a reference to it in first.**
WHERE A top-level constant holding a reference, not the object itself.
WHEN At run time, when execution reaches this line.
WHY let fixes the reference; the object it points at can still change.

```
17 print("first: \(first.name)")
```

WHO The print function, reading through first.
WHAT **Prints the name on the shared object: Ana at first, Ana Maria at the end.**
WHERE Top-level code, writing to the console.
WHEN Right after the object is created, and again at the very end.
WHY Reading through first is how the final frame proves the rename is shared.

```
18 var second = first
```

WHO Assignment, copying a reference.
WHAT **Makes second point at the very same Profile object as first. No new object is created.**
WHERE A second top-level constant; the heap still holds one object.
WHEN At run time, after first exists.
WHY This single line is the class counterpart of the struct copy above.

```
19 print("second: \(second.name)")
```

WHO The print function, reading through second.
WHAT **Prints the name seen through the second reference.**
WHERE Top-level code, writing to the console.
WHEN Right after second is created, and again at the very end.
WHY Both references read the same object, so both prints always agree.

```
20 second.name = "Ana Maria"
```

WHO Your code, changing the object through second.
WHAT **Sets the shared object name to Ana Maria through the second reference.**
WHERE Changes the one Profile object on the heap.
WHEN After both references exist and have printed.
WHY If references share the object, first must see this change too; the next prints test that.

```
21 print("first: \(first.name)")
```

WHO The print function, reading through first.
WHAT **Prints the name on the shared object: Ana at first, Ana Maria at the end.**
WHERE Top-level code, writing to the console.
WHEN Right after the object is created, and again at the very end.
WHY Reading through first is how the final frame proves the rename is shared.

```
22 print("second: \(second.name)")
```

WHO The print function, reading through second.
WHAT **Prints the name seen through the second reference.**
WHERE Top-level code, writing to the console.
WHEN Right after second is created, and again at the very end.
WHY Both references read the same object, so both prints always agree.

Try it

The starter copies a Point into p2 and changes the copy to x 7, then prints only the original. Add the missing print so the output proves both halves of the experiment: the original still at 1 and the copy at 7.

```
struct Point {
    var x: Int
}
var p1 = Point(x: 1)
var p2 = p1
p2.x = 7
print("p1: \(p1.x)")
```

EXPECTED OUTPUT

```
p1: 1
p2: 7
```

WORKED EXAMPLE

An order state machine that refuses bad moves

Problem.

An order may only move forward: placed to paid to shipped to delivered. Build a state machine that accepts each legal move, refuses the illegal one it is handed, and logs everything, so a full order history reads from the console.

Frame 1 of 6

The machine starts with two states and one new order sitting in placed. The print confirms the starting state; no moves have been requested yet, so nothing can be refused.

```
  enum OrderState {
      case placed, paid
  }
  var order = OrderState.placed
  print("State: \(order)")
```

OUTPUT SO FAR

```
State: placed
```

Frame 2 of 6

The enum gains shipped and delivered, and the request function arrives with its three legal moves. The order has not moved: this frame installs the rules, not the journey.

```
  enum OrderState {
+     case placed, paid, shipped, delivered
  }
+ func request(_ state: OrderState, to next: OrderState) -> OrderState {
+     let allowed = (state == .placed && next == .paid)
+         || (state == .paid && next == .shipped)
+         || (state == .shipped && next == .delivered)
+     if allowed {
+         print("Moved: \(state) -> \(next)")
+         return next
+     }
+     print("Refused: \(state) -> \(next)")
+     return state
+ }
+ var order = OrderState.placed
+ print("State: \(order)")
```

OUTPUT SO FAR

```
State: placed
```

Frame 3 of 6

The first request moves the order from placed to paid, and the log prints the accepted move. The state print after it confirms the variable advanced to paid.

```
    enum OrderState {
        case placed, paid, shipped, delivered
    }
    func request(_ state: OrderState, to next: OrderState) -> OrderState {
        let allowed = (state == .placed && next == .paid)
            || (state == .paid && next == .shipped)
            || (state == .shipped && next == .delivered)
        if allowed {
            print("Moved: \(state) -> \(next)")
            return next
        }
        print("Refused: \(state) -> \(next)")
        return state
    }
    var order = OrderState.placed
    print("State: \(order)")
+   order = request(order, to: .paid)
+   print("State: \(order)")
```

OUTPUT SO FAR

```
State: placed
Moved: placed -> paid
State: paid
```

Frame 4 of 6

The second request moves the order from paid to shipped, the only legal way forward. Two moves in, the machine is halfway through its one allowed path.

```
    enum OrderState {
        case placed, paid, shipped, delivered
    }
    func request(_ state: OrderState, to next: OrderState) -> OrderState {
        let allowed = (state == .placed && next == .paid)
            || (state == .paid && next == .shipped)
            || (state == .shipped && next == .delivered)
        if allowed {
            print("Moved: \(state) -> \(next)")
            return next
        }
        print("Refused: \(state) -> \(next)")
        return state
    }
    var order = OrderState.placed
    print("State: \(order)")
    order = request(order, to: .paid)
    print("State: \(order)")
+   order = request(order, to: .shipped)
+   print("State: \(order)")
```

OUTPUT SO FAR

```
State: placed
Moved: placed -> paid
State: paid
Moved: paid -> shipped
State: shipped
```

Frame 5 of 6

Now a bad move: the program asks to go from shipped back to paid. The request refuses, prints why in the log, and the order stays exactly where it was.

```
   enum OrderState {
       case placed, paid, shipped, delivered
   }
   func request(_ state: OrderState, to next: OrderState) -> OrderState {
       let allowed = (state == .placed && next == .paid)
           || (state == .paid && next == .shipped)
           || (state == .shipped && next == .delivered)
       if allowed {
           print("Moved: \(state) -> \(next)")
           return next
       }
       print("Refused: \(state) -> \(next)")
       return state
   }
   var order = OrderState.placed
   print("State: \(order)")
   order = request(order, to: .paid)
   print("State: \(order)")
   order = request(order, to: .shipped)
   print("State: \(order)")
+  order = request(order, to: .paid)
+  print("State: \(order)")
```

OUTPUT SO FAR

```
State: placed
Moved: placed -> paid
State: paid
Moved: paid -> shipped
State: shipped
Refused: shipped -> paid
State: shipped
```

Frame 6 of 6

The final request completes the path: shipped to delivered. The finished log shows three accepted moves and one refusal, which is the whole machine behaving as designed.

```
    enum OrderState {
        case placed, paid, shipped, delivered
    }
    func request(_ state: OrderState, to next: OrderState) -> OrderState {
        let allowed = (state == .placed && next == .paid)
            || (state == .paid && next == .shipped)
            || (state == .shipped && next == .delivered)
        if allowed {
            print("Moved: \(state) -> \(next)")
            return next
        }
        print("Refused: \(state) -> \(next)")
        return state
    }
    var order = OrderState.placed
    print("State: \(order)")
    order = request(order, to: .paid)
    print("State: \(order)")
    order = request(order, to: .shipped)
    print("State: \(order)")
    order = request(order, to: .paid)
    print("State: \(order)")
+   order = request(order, to: .delivered)
+   print("State: \(order)")
```

OUTPUT SO FAR

```
State: placed
Moved: placed -> paid
State: paid
Moved: paid -> shipped
State: shipped
Refused: shipped -> paid
State: shipped
Moved: shipped -> delivered
State: delivered
```

FINAL RESULT — CHECKED OUTPUT

```
State: placed
Moved: placed -> paid
State: paid
Moved: paid -> shipped
State: shipped
Refused: shipped -> paid
State: shipped
Moved: shipped -> delivered
State: delivered
```

Line by line: the finished program

```
1  enum OrderState {
```

WHO	You declare it; the compiler records the states.
WHAT	**Starts OrderState, an enum for the life of one order.**
WHERE	At the top of the program, before the function that uses it.
WHEN	Compile time; declaring it runs no code.
WHY	An enum makes an order exactly one of the listed states, never a typo.

```
2  case placed, paid, shipped, delivered
```

WHO	The OrderState type, completing its list.
WHAT	**Names all four states in life order: placed, paid, shipped, delivered.**
WHERE	The only line inside the enum.
WHEN	Compile time; the function below is written against this full list.
WHY	The full list is what makes illegal moves, like shipped back to paid, expressible and refusable.

```
4  func request(_ state: OrderState, to next: OrderState) -> OrderState {
```

WHO	You declare it; the call sites below run it.
WHAT	**Declares request, which tries to move an order from one state to a requested next state and returns the resulting state.**
WHERE	Under the enum, before any call.
WHEN	Compiled first; the body runs once per call.
WHY	One function guarding every move means no call site can skip the legality check.

```
5  let allowed = (state == .placed && next == .paid)
```

WHO	The request function, working out legality.
WHAT	**Starts the allowed test: placed may go to paid, or paid may go to shipped.**
WHERE	The first line inside request.
WHEN	At the start of every request call.
WHY	Writing the legal moves as conditions keeps the machine rules in one place.

```
6  || (state == .paid && next == .shipped)
```

WHO	The same allowed test, continued on the next line.
WHAT	**Adds another legal move pairing to the test: paid to shipped on this line, then shipped to delivered on the last one.**
WHERE	A continuation line of the allowed expression.
WHEN	Read as part of the same expression, at the start of each call.
WHY	Only three pairings make allowed true; every other pairing, including going backwards, is refused.

```
7  || (state == .shipped && next == .delivered)
```

WHO	The same allowed test, continued on the next line.
WHAT	**Adds another legal move pairing to the test: paid to shipped on this line, then shipped to delivered on the last one.**
WHERE	A continuation line of the allowed expression.
WHEN	Read as part of the same expression, at the start of each call.
WHY	Only three pairings make allowed true; every other pairing, including going backwards, is refused.

```
8  if allowed {
```

WHO	The request function, branching on the test.
WHAT	**Opens the branch taken when the requested move is legal.**
WHERE	Inside request, after the allowed test.
WHEN	Once per call, when allowed is true.
WHY	Legal moves share one path: print the move, then return the new state.

```
9  print("Moved: \(state) -> \(next)")
```

WHO	The print function, inside the legal branch.
WHAT	**Prints the move that just happened, naming the old and new states.**
WHERE	Inside the if in request.
WHEN	At the moment a legal move is accepted.
WHY	Logging accepted moves builds the order history the final frame shows.

```
10 return next
```

WHO	The request function, handing back the result.
WHAT	**Returns the requested state, so the caller variable advances.**
WHERE	The last line of the legal branch.
WHEN	Once per accepted move.
WHY	Returning the new state makes the call site a simple assignment.

```
12 print("Refused: \(state) -> \(next)")
```

WHO	The print function, on the refusal path.
WHAT	**Prints that the move was refused, naming both states so the reason is visible.**
WHERE	Inside request, after the if block.
WHEN	Once per call whose move is illegal.
WHY	A refusal is a normal outcome here, printed and handled, not an error that stops the program.

```
13 return state
```

WHO	The request function, handing back the unchanged result.
WHAT	**Returns the original state, so the caller variable stays where it was.**
WHERE	The last line of request.
WHEN	Once per refused move.
WHY	Returning the same state makes a refused move a no-op for the caller.

```
15 var order = OrderState.placed
```

WHO	Your top-level code; the calls below reassign it.
WHAT	**Creates an order in the placed state.**
WHERE	Top-level code, under the function.
WHEN	At run time, before any move is requested.
WHY	A var, because every request call reassigns it with the returned state.

```
16 print("State: \(order)")
```

WHO	The print function, reading the caller variable.
WHAT	**Prints the state the order is in after the last request.**
WHERE	Top-level code, after each request call.
WHEN	After the call above it, so it shows the result.
WHY	Printing after each request pairs every move or refusal with the state it left behind.

```
17 order = request(order, to: .paid)
```

WHO	Your top-level code, calling request.
WHAT	**Requests a move to paid and stores the returned state back in order.**
WHERE	Top-level code, at a request call site.
WHEN	At run time, each time this line runs.
WHY	The assignment writes back whatever request returned, so a refusal leaves the order unchanged.

```
18 print("State: \(order)")
```

WHO The print function, reading the caller variable.
WHAT **Prints the state the order is in after the last request.**
WHERE Top-level code, after each request call.
WHEN After the call above it, so it shows the result.
WHY Printing after each request pairs every move or refusal with the state it left behind.

```
19 order = request(order, to: .shipped)
```

WHO Your top-level code, calling request.
WHAT **Requests the move from paid to shipped and stores the result.**
WHERE Top-level code, the second request.
WHEN After the order has been paid.
WHY Paying first is what makes this move legal.

```
20 print("State: \(order)")
```

WHO The print function, reading the caller variable.
WHAT **Prints the state the order is in after the last request.**
WHERE Top-level code, after each request call.
WHEN After the call above it, so it shows the result.
WHY Printing after each request pairs every move or refusal with the state it left behind.

```
21 order = request(order, to: .paid)
```

WHO Your top-level code, calling request.
WHAT **Requests a move to paid and stores the returned state back in order.**
WHERE Top-level code, at a request call site.
WHEN At run time, each time this line runs.
WHY The assignment writes back whatever request returned, so a refusal leaves the order unchanged.

```
22 print("State: \(order)")
```

WHO The print function, reading the caller variable.
WHAT **Prints the state the order is in after the last request.**
WHERE Top-level code, after each request call.
WHEN After the call above it, so it shows the result.
WHY Printing after each request pairs every move or refusal with the state it left behind.

```
23 order = request(order, to: .delivered)
```

WHO Your top-level code, calling request.
WHAT **Requests the final move from shipped to delivered and stores the result.**
WHERE Top-level code, the last request.
WHEN After the refused attempt, in story order.
WHY Delivered is the end of the machine: no move out of it is legal.

```
24 print("State: \(order)")
```

WHO The print function, reading the caller variable.
WHAT **Prints the state the order is in after the last request.**
WHERE Top-level code, after each request call.
WHEN After the call above it, so it shows the result.
WHY Printing after each request pairs every move or refusal with the state it left behind.

Try it

The starter walks an order from placed to paid by direct assignment. Extend it: add the shipped case to the enum, move the order on to shipped, and let the final print show the new end state.

```
enum OrderState {
    case placed, paid
}
var order = OrderState.placed
order = .paid
print("State: \(order)")
```

EXPECTED OUTPUT

```
State: shipped
```

WORKED EXAMPLE

One helper for many types: protocol plus generics

Problem.

Books and podcasts are different types, but a shelf should announce either one. Define a protocol with a summary property, adopt it in both types, write one generic helper constrained to the protocol, and loop a mixed collection with it.

Frame 1 of 6

The Summarizable protocol asks for one summary property, and Book adopts it, computing its line from title and pages. The print shows the promise kept for the first type.

```
protocol Summarizable {
    var summary: String { get }
}
struct Book: Summarizable {
    let title: String
    let pages: Int
    var summary: String { "\(title), \(pages) pages" }
}
let book = Book(title: "Dune", pages: 412)
print(book.summary)
```

OUTPUT SO FAR

```
Dune, 412 pages
```

Frame 2 of 6

Podcast adopts the same protocol with different facts: title and minutes. Both values now exist, but nothing prints the podcast yet; the helper will be its first showcase.

```
  protocol Summarizable {
      var summary: String { get }
  }
  struct Book: Summarizable {
      let title: String
      let pages: Int
      var summary: String { "\(title), \(pages) pages" }
  }
+ struct Podcast: Summarizable {
+     let title: String
+     let minutes: Int
+     var summary: String { "\(title), \(minutes) minutes" }
+ }
+ let book = Book(title: "Dune", pages: 412)
+ let podcast = Podcast(title: "Swift Talk", minutes: 45)
+ print(book.summary)
```

OUTPUT SO FAR

```
Dune, 412 pages
```

Frame 3 of 6

The generic helper arrives: announce accepts any type constrained to Summarizable and prints its summary. Calling it with the book fixes its type parameter to Book for that call.

```
  protocol Summarizable {
      var summary: String { get }
  }
  struct Book: Summarizable {
      let title: String
      let pages: Int
      var summary: String { "\(title), \(pages) pages" }
  }
  struct Podcast: Summarizable {
      let title: String
      let minutes: Int
      var summary: String { "\(title), \(minutes) minutes" }
  }
  let book = Book(title: "Dune", pages: 412)
  let podcast = Podcast(title: "Swift Talk", minutes: 45)
  print(book.summary)
+ func announce<T: Summarizable>(_ item: T) {
+     print("Now showing: \(item.summary)")
+ }
+ announce(book)
```

OUTPUT SO FAR

```
Dune, 412 pages
Now showing: Dune, 412 pages
```

Frame 4 of 6

The same helper now takes the podcast, and its type parameter becomes Podcast. One function body serves both types, with the compiler checking each call separately.

```
  protocol Summarizable {
      var summary: String { get }
  }
  struct Book: Summarizable {
      let title: String
      let pages: Int
      var summary: String { "\(title), \(pages) pages" }
  }
  struct Podcast: Summarizable {
      let title: String
      let minutes: Int
      var summary: String { "\(title), \(minutes) minutes" }
  }
  let book = Book(title: "Dune", pages: 412)
  let podcast = Podcast(title: "Swift Talk", minutes: 45)
  print(book.summary)
  func announce<T: Summarizable>(_ item: T) {
      print("Now showing: \(item.summary)")
  }
  announce(book)
+ announce(podcast)
```

OUTPUT SO FAR

```
Dune, 412 pages
Now showing: Dune, 412 pages
Now showing: Swift Talk, 45 minutes
```

Frame 5 of 6

Both values join one array of any Summarizable, and a loop prints each summary in order. Unlike types now share one collection without losing the shared promise.

```
protocol Summarizable {
    var summary: String { get }
}
struct Book: Summarizable {
    let title: String
    let pages: Int
    var summary: String { "\(title), \(pages) pages" }
}
struct Podcast: Summarizable {
    let title: String
    let minutes: Int
    var summary: String { "\(title), \(minutes) minutes" }
}
let book = Book(title: "Dune", pages: 412)
let podcast = Podcast(title: "Swift Talk", minutes: 45)
print(book.summary)
func announce<T: Summarizable>(_ item: T) {
    print("Now showing: \(item.summary)")
}
announce(book)
announce(podcast)
+ let shelf: [any Summarizable] = [book, podcast]
+ for item in shelf {
+     print(item.summary)
+ }
```

OUTPUT SO FAR

```
Dune, 412 pages
Now showing: Dune, 412 pages
Now showing: Swift Talk, 45 minutes
Dune, 412 pages
Swift Talk, 45 minutes
```

Frame 6 of 6

A second book joins the shelf and the same loop prints three summaries. Nothing in the loop changed: adding a conformer is adding data, not code.

```
protocol Summarizable {
    var summary: String { get }
}
struct Book: Summarizable {
    let title: String
    let pages: Int
    var summary: String { "\(title), \(pages) pages" }
}
struct Podcast: Summarizable {
    let title: String
    let minutes: Int
    var summary: String { "\(title), \(minutes) minutes" }
}
let book = Book(title: "Dune", pages: 412)
let podcast = Podcast(title: "Swift Talk", minutes: 45)
print(book.summary)
func announce<T: Summarizable>(_ item: T) {
    print("Now showing: \(item.summary)")
}
announce(book)
announce(podcast)
+ let novel = Book(title: "Emma", pages: 320)
+ let shelf: [any Summarizable] = [book, podcast, novel]
+ for item in shelf {
+     print(item.summary)
+ }
```

OUTPUT SO FAR

```
Dune, 412 pages
Now showing: Dune, 412 pages
Now showing: Swift Talk, 45 minutes
Dune, 412 pages
Swift Talk, 45 minutes
Emma, 320 pages
```

FINAL RESULT — CHECKED OUTPUT

```
Dune, 412 pages
Now showing: Dune, 412 pages
Now showing: Swift Talk, 45 minutes
Dune, 412 pages
Swift Talk, 45 minutes
Emma, 320 pages
```

Line by line: the finished program

1 `protocol Summarizable {`

WHO	You declare it; conforming types implement it.
WHAT	**Starts the Summarizable protocol: anything describable in one line.**
WHERE	At the top of the program, before the types that adopt it.
WHEN	Compile time; a protocol has no code of its own to run.
WHY	A protocol names a capability, so the generic helper later can rely on it.

```
2  var summary: String { get }
```

WHO The Summarizable protocol, stating its one requirement.
WHAT **Requires a readable summary property of type String, with no body.**
WHERE The only line inside the protocol.
WHEN Compile time; conforming types are checked against it.
WHY One small requirement is easy to adopt and enough for the helper to do its job.

```
4  struct Book: Summarizable {
```

WHO You declare it; the values below are Books.
WHAT **Starts Book, a struct adopting Summarizable.**
WHERE Under the protocol.
WHEN Compile time; the compiler checks the summary property exists.
WHY A book is plain data, so a struct that also gains the shared capability.

```
5  let title: String
```

WHO Every Book value, which stores its own title.
WHAT **Declares the stored title property the summary reads.**
WHERE Inside each Book value.
WHEN Set by the memberwise initializer.
WHY The summary is built from the title, so the title is the stored fact.

```
6  let pages: Int
```

WHO Every Book value, which stores its own page count.
WHAT **Declares the stored pages property the summary reads.**
WHERE Inside each Book value, beside title.
WHEN Set by the memberwise initializer.
WHY Storing the count, not a formatted string, keeps the data reusable.

```
7  var summary: String { "\(title), \(pages) pages" }
```

WHO The Book type, fulfilling the protocol requirement.
WHAT **Computes the summary from title and pages in one expression: Dune, 412 pages.**
WHERE Inside Book, as a computed property.
WHEN Each time summary is read.
WHY A single-expression body returns its value without writing return.

```
9  struct Podcast: Summarizable {
```

WHO You declare it; the values below are Podcasts.
WHAT **Starts Podcast, a second struct adopting Summarizable.**
WHERE Under Book.
WHEN Compile time; checked against the same requirement.
WHY A second, unrelated type with the same capability is what makes the helper worthwhile.

```
10 let title: String
```

WHO Every Podcast value, which stores its own title.
WHAT **Declares the stored title property the summary reads.**
WHERE Inside each Podcast value.
WHEN Set by the memberwise initializer.
WHY The same property name as the Book title, by design: both types describe media.

```swift
let minutes: Int
```

WHO Every Podcast value, which stores its own length.
WHAT **Declares the stored minutes property the summary reads.**
WHERE Inside each Podcast value, beside title.
WHEN Set by the memberwise initializer.
WHY Minutes are the podcast counterpart of the book pages.

```swift
var summary: String { "\(title), \(minutes) minutes" }
```

WHO The Podcast type, fulfilling the protocol requirement.
WHAT **Computes the summary from title and minutes: Swift Talk, 45 minutes.**
WHERE Inside Podcast, as a computed property.
WHEN Each time summary is read.
WHY Each type formats its own facts; the protocol only demands the result.

```swift
let book = Book(title: "Dune", pages: 412)
```

WHO The memberwise initializer Swift synthesized for Book.
WHAT **Creates the Dune book with 412 pages.**
WHERE A top-level constant.
WHEN At run time, when execution reaches this line.
WHY A let, because this program never changes the book.

```swift
let podcast = Podcast(title: "Swift Talk", minutes: 45)
```

WHO The memberwise initializer Swift synthesized for Podcast.
WHAT **Creates the Swift Talk podcast with 45 minutes.**
WHERE A top-level constant, under the book.
WHEN At run time, after the book exists.
WHY Two values of two different types, ready for the shared helper.

```swift
print(book.summary)
```

WHO The print function, reading the computed property.
WHAT **Prints the book summary through the protocol requirement.**
WHERE Top-level code.
WHEN After the values exist; in the tryIt it follows the book creation.
WHY Printing directly first proves each conformer works before the helper exists.

```swift
func announce<T: Summarizable>(_ item: T) {
```

WHO You declare it; the call sites below run it.
WHAT **Declares announce, a generic function whose item may be any Summarizable type, marked by the constraint after the name.**
WHERE Under the values, before its call sites.
WHEN Compiled first; the body runs once per call.
WHY The constraint T: Summarizable is the whole trick: one function, every conforming type, fully type-checked.

```swift
print("Now showing: \(item.summary)")
```

WHO The announce function, reading through the protocol.
WHAT **Prints the item summary with a Now showing prefix.**
WHERE The only line inside announce.
WHEN Once per call.
WHY The body only touches summary, the one thing the constraint guarantees.

```
20 announce(book)
```

WHO Your top-level code, calling the generic helper.
WHAT **Calls announce with a book; the compiler fixes T to Book for this call.**
WHERE Top-level code, at an announce call site.
WHEN At run time, after the function is declared.
WHY No type name is written at the call: Swift infers it from the argument.

```
21 announce(podcast)
```

WHO Your top-level code, calling the generic helper again.
WHAT **Calls announce with the podcast; here T is Podcast.**
WHERE Top-level code, the second announce call.
WHEN Right after the book call.
WHY Same function, different concrete type: that reuse is the point of generics.

```
22 let novel = Book(title: "Emma", pages: 320)
```

WHO The memberwise initializer Swift synthesized for Book.
WHAT **Creates a second book, Emma with 320 pages.**
WHERE A top-level constant, just before the shelf.
WHEN At run time, after the podcast exists.
WHY A third value shows the shelf mixing not just types but counts of them.

```
23 let shelf: [any Summarizable] = [book, podcast, novel]
```

WHO Your top-level code, rebuilding the mixed collection.
WHAT **Builds the shelf again with all three values: two books around one podcast.**
WHERE Top-level code, replacing the two-item shelf line.
WHEN At run time, once.
WHY The loop below does not change at all, which is the payoff of the protocol.

```
24 for item in shelf {
```

WHO The for-in loop, visiting the shelf.
WHAT **Starts a loop over the mixed array, binding each boxed value to item.**
WHERE Top-level code, under the shelf line.
WHEN At run time; the body runs once per element.
WHY Looping the box works because every element promises a summary.

```
25 print(item.summary)
```

WHO The print function, reading through the protocol.
WHAT **Prints the summary of whichever conformer item holds this visit.**
WHERE Inside the loop.
WHEN Once per element, in shelf order.
WHY The same line prints book and podcast lines in turn, through the one protocol promise.

Try it

The starter defines Summarizable and a small Book that adopts it, then prints the book summary directly. Add the generic announce helper from the frames and call it with the book, so the output gains the Now showing line.

```
protocol Summarizable {
    var summary: String { get }
}
struct Book: Summarizable {
    let title: String
    var summary: String { "\(title), a book" }
}
let book = Book(title: "Dune")
print(book.summary)
```

EXPECTED OUTPUT

```
Dune, a book
Now showing: Dune, a book
```

MODULE 04

SwiftUI and Reactive Interfaces

Build reactive iPhone interfaces in SwiftUI: compose views and modifiers, lay them out with stacks and grids, drive them from @State and @Observable models, and add lists, navigation, forms, animation and Liquid Glass.

- Design user interfaces with reactive programming, UI elements and their properties
- Create various types of iOS applications in Swift

Describe the Screen, Not the Steps

CONCEPT

What SwiftUI is

SwiftUI is Apple's framework for building user interfaces on iPhone, iPad, Mac, Apple Watch, Apple TV, and Apple Vision Pro. A *framework* is a library of ready-made types that your code builds on.

SwiftUI is **declarative**: you describe what the screen should look like for the current data, and SwiftUI works out how to draw it and how to change it when the data changes. You never write "now move this label" or "now change that text".

View	A type that describes a piece of interface, from a single label to a whole screen.
body	The property where a view returns its description.
Modifier	A method such as .font() or .padding() that returns an adjusted view.
Preview	A live rendering of a view while you edit, shown in Xcode and in the phone on this page.

> **Before SwiftUI: UIKit**
>
> From the first iPhone SDK in 2008, screens were built with **UIKit**: view controller objects that create labels and buttons, then change them step by step, often laid out in Interface Builder storyboards. SwiftUI arrived in 2019. Many existing apps still contain UIKit, and the two can be mixed, but new screens are built in SwiftUI.

COMPARE

The same counter, two ways

Both snippets build one screen: a label that shows a count and a button that adds one. On the left, UIKit commands objects step by step. On the right, SwiftUI describes the screen in terms of the data. Read each side line by line, then tap the button in the phone.

A: Before: imperative UIKit

```swift
import UIKit
final class CounterViewController: UIViewController {
    private var count = 0
    private let label = UILabel()
    private let button = UIButton(type: .system)
    override func viewDidLoad() {
        super.viewDidLoad()
        label.text = "Count: 0"
        button.setTitle("Add one", for: .normal)
        button.addTarget(self, action: #selector(addOne),
                         for: .touchUpInside)
        for subview in [label, button] { view.addSubview(subview) }
    }
    @objc private func addOne() {
        count += 1
        label.text = "Count: \(count)"
    }
}
```

1 `import UIKit`

WHO The Swift compiler, which looks up UIViewController, UILabel and UIButton in UIKit while it builds this file.

WHAT **Loads UIKit, Apple's older object-based interface framework that provides UIViewController, UILabel and UIButton.**

WHERE Top of the file; the import covers this file only.

WHEN At compile time; the import does no work while the app runs, it only makes UIKit's names visible.

WHY Before 2019 every iPhone screen was built with UIKit, so you will meet it in older code bases.

2 `final class CounterViewController: UIViewController {`

WHO Your code subclasses it; UIKit creates and manages the object.

WHAT **Declares a class that controls one screen by inheriting the behavior of UIViewController. final means no other class may subclass it.**

WHERE An object on the heap that lives as long as the screen is on display.

WHEN Created when the app presents this screen.

WHY In UIKit a long-lived controller object owns the screen and changes its views over time.

3 `private var count = 0`

WHO The CounterViewController object owns this number; nothing outside the class can read it because it is private.

WHAT **Stores the current number of taps, starting at 0.**

WHERE Inside the controller object on the heap.

WHEN Set when the controller is created, then changed on every tap.

WHY This is the data. But the label will hold its own copy as text, so there are now two places that must agree.

4 `private let label = UILabel()`

WHO UIKit supplies the UILabel class; this controller creates one instance and keeps a reference to it.

WHAT **Creates a label object and keeps a reference to it so the code can change its text later.**

WHERE A stored property of the controller; the label object lives on the heap.

WHEN Created at the same moment as the controller object, before viewDidLoad runs.

WHY Imperative code must hold on to each view it plans to modify by hand.

```
5  private let button = UIButton(type: .system)
```

WHO — UIKit supplies the UIButton class; .system asks for the standard tinted button look.
WHAT — **Creates a standard system-styled button object.**
WHERE — Another stored property of the controller.
WHEN — Created alongside the label when the controller object is initialized.
WHY — Like the label, the button is an object you configure and wire up step by step.

```
6  override func viewDidLoad() {
```

WHO — UIKit calls this method; you override it.
WHAT — **Replaces the hook UIKit runs after the controller's root view has been created.**
WHERE — A method of CounterViewController that UIKit runs on the main thread, the only thread allowed to touch views.
WHEN — Once, after the root view loads and before the screen appears.
WHY — It is the traditional place to configure views one command at a time.

```
7      super.viewDidLoad()
```

WHO — UIViewController, the superclass, whose own viewDidLoad runs before your setup code.
WHAT — **Runs the inherited version of viewDidLoad first.**
WHERE — The first statement inside your viewDidLoad override, ahead of any of your own setup lines.
WHEN — As soon as UIKit calls viewDidLoad, before the label and button are configured below it.
WHY — Overrides of UIKit lifecycle methods call super so the framework's own setup still happens.

```
8      label.text = "Count: 0"
```

WHO — Your setup code, giving a direct command to the label object it stored earlier.
WHAT — **Writes the starting text "Count: 0" into the label by hand, separately from the count property.**
WHERE — Inside viewDidLoad, so it touches the label before the screen is first shown.
WHEN — Exactly once, while the controller sets up its views; nothing ever runs it again.
WHY — The 0 here is typed separately from count's starting value. Change one and forget the other, and the screen lies.

```
9      button.setTitle("Add one", for: .normal)
```

WHO — Your setup code, giving a direct command to the button object it stored earlier.
WHAT — **Sets the title shown in the button's normal state (not pressed, not disabled).**
WHERE — Inside viewDidLoad, right after the label receives its starting text.
WHEN — Once, during setup; the title then stays the same for the life of this screen.
WHY — UIKit buttons keep a separate title per control state, so you must say which state you mean.

```
10     button.addTarget(self, action: #selector(addOne),
```

WHO — UIKit's target-action system, which remembers which object and which method to call when the button fires.
WHAT — **Registers this controller (self) as the target and the method addOne as the action to call.**
WHERE — Inside viewDidLoad; the statement continues on the next line.
WHEN — Registered now; the method is called on each tap later.
WHY — Target-action comes from the Objective-C era: the button calls a method by name, which is why #selector and @objc appear.

```
11                    for: .touchUpInside)
```

WHO — UIKit, which watches the finger on the button.
WHAT — **Names the event: a finger lifted while still inside the button, which is a completed tap.**
WHERE — Second half of the addTarget call above.
WHEN — Matched every time a touch ends inside the button.
WHY — Lifting the finger outside the button cancels the tap, which is how people expect buttons to behave.

```
12  for subview in [label, button] { view.addSubview(subview) }
```

WHO — Your code commands the screen's root view.

WHAT — **Adds the label and the button to the screen. Layout constraints that position them are left out to keep the example short.**

WHERE — Inside viewDidLoad, on the controller's root view.

WHEN — Once, during viewDidLoad; after that both views stay in the hierarchy until the screen goes away.

WHY — In UIKit nothing appears until you insert it into the view hierarchy yourself; a real screen needs several more lines to place each view.

```
14  @objc private func addOne() {
```

WHO — UIKit calls it through the Objective-C runtime.

WHAT — **Declares the method the button triggers; @objc exposes it to the Objective-C runtime so #selector can find it.**

WHERE — A private method of CounterViewController, called on the main thread like all UIKit event handling.

WHEN — Each time the user lifts a finger inside the button, completing a tap.

WHY — Target-action predates Swift, so the method must be visible to Objective-C.

```
15  count += 1
```

WHO — The addOne action method, running because UIKit delivered the tap to it.

WHAT — **Adds one to the count property, so the controller's data is now one higher than the label shows.**

WHERE — The first statement inside addOne, before the label is touched.

WHEN — On every completed tap of the Add one button, never at setup.

WHY — Updating the data alone changes nothing on screen in UIKit; the label still shows the old text.

```
16  label.text = "Count: \(count)"
```

WHO — Your action method, issuing a second command to the label so it matches the new data.

WHAT — **Copies the new number into the label by hand.**

WHERE — Inside addOne, right after the data changes.

WHEN — On every tap, immediately after count changes, as the last statement of addOne.

WHY — Forget this line and the screen silently shows a stale number. This manual syncing is exactly what SwiftUI removes.

B: After: declarative SwiftUI

```swift
import SwiftUI

struct CounterView: View {
    @State private var count = 0

    var body: some View {
        VStack(spacing: 12) {
            Text("Count: \(count)")
            Button("Add one") {
                count += 1
            }
        }
    }
}

#Preview { CounterView() }
```

```
1  import SwiftUI
```

WHO The Swift compiler, which finds View, Text, Button and @State in SwiftUI while it builds CounterView.
WHAT **Makes View, VStack, Text, Button and the @State wrapper available in this file.**
WHERE Top of the file; it covers this file only.
WHEN At compile time only; nothing about the import runs when someone taps the button.
WHY SwiftUI is a framework, so each file that uses it imports it.

```
3  struct CounterView: View {
```

WHO You declare it; SwiftUI creates and displays its values.
WHAT **Declares a struct that describes the counter screen. There is no controller object.**
WHERE At file scope, so the preview at the bottom of the file, and any other view, can create it.
WHEN Compiled once; values are created whenever a parent needs one.
WHY A view is a cheap value that describes the screen, not an object you keep changing.

```
4  @State private var count = 0
```

WHO SwiftUI, which keeps the count in storage tied to this view on screen, outside the struct value itself.
WHAT **Declares the only copy of the count, starting at 0.**
WHERE Storage managed by SwiftUI, tied to this view on screen.
WHEN Created when the view first appears; kept across redraws.
WHY One source of truth: the text below is computed from it, so the two can never disagree.

```
6  var body: some View {
```

WHO SwiftUI calls this property whenever it needs the screen's description; your code never calls body itself.
WHAT **Returns the description of the screen for the current count.**
WHERE A computed property inside CounterView, evaluated on the main thread.
WHEN When the view appears and again after every change to count.
WHY The screen is a function of the data: same count, same screen.

```
7  VStack(spacing: 12) {
```

WHO SwiftUI's layout system, which measures the text and the button and stacks them in the center of the screen.
WHAT **Stacks the text and button vertically, 12 points apart.**
WHERE The root of the description returned by body.
WHEN Measured and placed again every time body produces a new description after count changes.
WHY Layout is described, not computed by hand with constraints.

```
8  Text("Count: \(count)")
```

WHO Swift builds the "Count: 3" style string through interpolation; SwiftUI then draws it as a Text view.
WHAT **Shows the count using string interpolation.**
WHERE The first child of the VStack, so it appears above the button.
WHEN Rebuilt with the new number every time body runs.
WHY The text is derived from count, so there is no line that copies the number into a label.

```
9  Button("Add one") {
```

WHO You declare the button; the person using the app triggers its action closure by tapping it.
WHAT **Creates a button whose action is the closure that follows.**
WHERE The second child of the VStack, directly under the count.
WHEN The closure runs on the main thread at the moment of each tap, never while body is being built.
WHY A closure replaces the target, selector and @objc machinery of UIKit.

```
10  count += 1
```

WHO	Your action closure; SwiftUI observes the write.
WHAT	**Adds one to the @State value; there is no second line that updates the text.**
WHERE	Inside the Button's action closure, the only code in this view that changes data.
WHEN	On each tap; SwiftUI then runs body again and updates the text.
WHY	You change data, and the screen follows automatically.

```
16  #Preview { CounterView() }
```

WHO	Xcode's preview canvas, and the simulated iPhone beside this code, which both run CounterView live.
WHAT	**Creates a CounterView and shows it live.**
WHERE	File scope; preview code is not shipped in the app.
WHEN	Every time you edit this file; the preview rebuilds and the count starts again at 0.
WHY	You can tap the button and test the screen without launching the whole app.

> **Takeaway**
>
> The UIKit version keeps the number in two places, `count` and `label.text`, and every change must update both by hand. The SwiftUI version keeps one source of truth and *describes* the text in terms of it, so the screen cannot fall out of sync. That shift, from commands to descriptions, is what **declarative** means.

CONCEPT

A view is a value, and body follows the data

A SwiftUI view is a small **struct**: a value, not a long-lived object on screen. Creating one costs almost nothing. SwiftUI calls its `body` to get a description, compares it with the previous one, and redraws only what differs.

This is **reactive programming**: the interface *reacts* to data. Tap the button in the figure and follow the loop.

Input	Data a view reads: its properties and its state.
Recompute	After an input changes, SwiftUI runs that view's body again.
Diff	Comparing the old and new descriptions to find the smallest change.

Interactive figure in the HTML version: swiftui data flow.

CODE · LIVE SWIFTUI PREVIEW

A body returns a view tree

Your first complete view to edit. Its body returns a **view tree**: container views holding child views, which may hold children of their own. Here a VStack holds an Image and two Text views, and a padding modifier wraps the whole stack. The indentation of the code mirrors the shape of the tree. Edit any line and the phone updates as you type.

```swift
import SwiftUI

struct ProfileCardView: View {
    var body: some View {
        VStack(spacing: 8) {
            Image(systemName: "person.circle")
                .font(.system(size: 64))
                .foregroundStyle(.tint)
            Text("Ada Lovelace")
                .font(.title2)
                .bold()
            Text("Mathematician and writer")
                .foregroundStyle(.secondary)
        }
        .padding()
    }
}

#Preview { ProfileCardView() }
```

Line by line

1 `import SwiftUI`

WHO	The Swift compiler, which resolves VStack, Image and Text from SwiftUI while it builds ProfileCardView.
WHAT	**Brings in VStack, Image and Text, plus the font and color styles used below.**
WHERE	The very first line of the file; it covers ProfileCardView and the preview below it.
WHEN	At compile time; once the app is built, the import costs nothing at run time.
WHY	Without the import none of these view types would be known to the compiler.

3 `struct ProfileCardView: View {`

WHO	You declare the type; SwiftUI creates and displays its values.
WHAT	**Declares a struct that conforms to the View protocol. A protocol is a list of requirements; View requires exactly one thing, a body.**
WHERE	At file scope, so any other view in the app can use it.
WHEN	Compiled once; a value is created whenever a parent (here, the preview) needs one.
WHY	Conforming to View lets SwiftUI treat your struct exactly like a built-in view. Naming it after what it shows keeps a growing app readable.

4 `var body: some View {`

WHO	SwiftUI reads this property; you never call it yourself.
WHAT	**A computed property that returns the root of this view's tree. some View means one concrete view type that the compiler works out for you.**
WHERE	Inside ProfileCardView, on the main thread.
WHEN	When the card appears, and again if anything it reads changes (nothing here can change yet).
WHY	The real type of this tree is long and nested; some View hides it, so you can rearrange the content without rewriting a type name.

```
5  VStack(spacing: 8) {
```

WHO SwiftUI's layout system, which treats the VStack as the parent node of the symbol, the name and the description.
WHAT **A container that places its three children top to bottom, 8 points apart.**
WHERE Inside body; the children between its braces are its branches.
WHEN Laid out whenever body runs; this card has no state, so that happens mainly when it first appears.
WHY Containers turn a flat list of views into a structured tree, and the braces in the code show that structure.

```
6  Image(systemName: "person.circle")
```

WHO SwiftUI, drawing an SF Symbol from Apple's icon library.
WHAT **First child: a person-in-a-circle symbol.**
WHERE The first child of the VStack, so it sits at the top of the card, above the name.
WHEN Created as a fresh value every time ProfileCardView's body is evaluated.
WHY SF Symbols are vector icons, so they stay sharp at any size and match the system font.

```
7  .font(.system(size: 64))
```

WHO SwiftUI, which sizes the SF Symbol as if it were a 64-point character of text.
WHAT **Sizes the symbol at 64 points, because symbols size themselves like text.**
WHERE Attached to the Image directly above it; the name and description are not affected.
WHEN Applied whenever body builds the Image, so the symbol is always drawn at 64 points.
WHY Symbols are designed to line up with text, so you size them with fonts rather than frames.

```
8  .foregroundStyle(.tint)
```

WHO SwiftUI, reading the app's accent color.
WHAT **Colors the symbol with the tint (accent) color, which is blue by default.**
WHERE Wraps the symbol after it was sized, so only the person icon takes the tint color.
WHEN Resolved each time the card is drawn, so a new accent color shows up immediately.
WHY Using .tint instead of a fixed color means changing the app's accent color updates every tinted element at once.

```
9  Text("Ada Lovelace")
```

WHO SwiftUI's Text view, which draws the string using the font set by the modifiers below it.
WHAT **The second child of the VStack: the person's name, shown in the middle of the card.**
WHERE Between the symbol and the description, the middle branch of the view tree.
WHEN A new Text value is created each time body runs, with the same string every time.
WHY Each child is a separate node in the tree, so each can be styled independently.

```
10 .font(.title2)
```

WHO SwiftUI, together with Dynamic Type, the system setting that scales text styles for each reader.
WHAT **Uses the second-level title text style for the name.**
WHERE Attached to the name Text only; the description keeps the default body style.
WHEN Applied as the name is built, and again at a new size if the reader changes their text size.
WHY Larger text makes the name the most important element on the card.

```
11 .bold()
```

WHO SwiftUI's text styling, which switches the name to the bold weight of the same font.
WHAT **Draws "Ada Lovelace" in a bold weight on top of the title 2 size set just above.**
WHERE Chained after .font(.title2) on the name Text, so both styles apply to the name.
WHEN Applied every time the name Text is created in body.
WHY Weight adds emphasis without changing the size.

```
12  Text("Mathematician and writer")
```

WHO	SwiftUI's Text view, drawing the one-line description of the person.
WHAT	**Third child: a short description under the name.**
WHERE	The last child of the VStack, so it appears at the bottom of the card under the name.
WHEN	Created alongside the other two children each time body is evaluated.
WHY	Supporting details live in their own node so they can be styled more quietly.

```
13  .foregroundStyle(.secondary)
```

WHO	SwiftUI, using the system's secondary label color.
WHAT	**Draws the description in a softer gray that adapts to light and dark mode.**
WHERE	Attached to the description Text only, so the name keeps the primary label color.
WHEN	Resolved as the card is drawn, and again automatically when the device switches to dark mode.
WHY	Visual hierarchy: less important text recedes so the name stands out.

```
15  .padding()
```

WHO	SwiftUI's layout system, which adds the platform's standard margin around the stack.
WHAT	**Adds the system default spacing around the whole stack.**
WHERE	Wraps the VStack, so a padding view becomes the new root of the tree.
WHEN	Measured during the same layout pass as the VStack it wraps, each time body runs.
WHY	Modifiers add nodes to the tree too: the tree is now padding, then VStack, then the three children.

```
19  #Preview { ProfileCardView() }
```

WHO	Xcode's preview canvas, and the iPhone on this page, which create a ProfileCardView to display.
WHAT	**Shows the finished profile card, with its symbol, name and description, in a live preview.**
WHERE	At file scope after ProfileCardView; preview code is stripped from the app you ship.
WHEN	Each time you change the card's code, so new children and modifiers appear right away.
WHY	You can watch the tree change shape as you add, remove and reorder children.

> **Try it**
>
> 1. Change "Ada Lovelace" to your own name, and .title2 to .largeTitle.
> 2. Add a fourth child below the last Text: Text("London").
> 3. Change VStack to HStack and watch the tree lay out sideways.

CODE · LIVE SWIFTUI PREVIEW

Views are configured by their inputs

Views take **inputs** through ordinary properties. HomeView creates two WelcomeCard values with different inputs, and each card draws itself from its own data. When an input changes, the parent’s body creates a fresh card value and SwiftUI runs the card’s body again; the card never edits itself.

```swift
import SwiftUI

struct WelcomeCard: View {
    let name: String
    let streak: Int
    var body: some View {
        Text("\(name): \(streak)-day streak")
            .padding()
            .background(.mint)
    }
}

struct HomeView: View {
    var body: some View {
        VStack(spacing: 24) {
            WelcomeCard(name: "Ada", streak: 3)
            WelcomeCard(name: "Grace", streak: 12)
        }
    }
}

#Preview { HomeView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which needs SwiftUI to understand both WelcomeCard and HomeView in this file.
- **WHAT** **Makes View, VStack, Text and the modifiers used by both structs in this file available.**
- **WHERE** Top of the file; it covers both views below.
- **WHEN** At compile time, once for the whole file, before either struct is checked.
- **WHY** One import serves every view declared in the same file.

3 `struct WelcomeCard: View {`

- **WHO** You declare the type; HomeView creates WelcomeCard values and SwiftUI displays them.
- **WHAT** **Declares a reusable child view that greets one person.**
- **WHERE** At file scope, so HomeView below, or any other screen in the app, can use it.
- **WHEN** Compiled once; a new value is created every time a parent's body runs.
- **WHY** Small reusable views keep screens short and consistent.

4 `let name: String`

- **WHO** HomeView, the parent, supplies the name each time it creates a card.
- **WHAT** **An input property: the person to greet. let means the card cannot change it.**
- **WHERE** A stored constant inside each WelcomeCard value, so Ada's card and Grace's card each hold their own.
- **WHEN** Fixed at the moment HomeView's body calls WelcomeCard(name:streak:); it never changes afterward.
- **WHY** Inputs flow down from parent to child; a let keeps the card a pure description of the data it was given.

5 `let streak: Int`

- **WHO** The parent view supplies this value too.
- **WHAT** **A second input: how many days in a row this person has studied.**
- **WHERE** A second stored constant next to name inside each card value.
- **WHEN** Set in the same initializer call as name, when HomeView's body creates the card.
- **WHY** A view can take as many inputs as it needs, just like a function takes parameters.

```
6  var body: some View {
```

WHO — SwiftUI calls WelcomeCard's body when it needs to draw a card; HomeView never calls it directly.
WHAT — **Describes the card using the two inputs.**
WHERE — Inside WelcomeCard, so it can read the card's own name and streak constants.
WHEN — When the card appears, and again whenever its parent passes different inputs.
WHY — Same inputs always produce the same card, which makes views easy to reason about.

```
7      Text("\(name): \(streak)-day streak")
```

WHO — Swift interpolates name and streak into one string; SwiftUI draws the result as a Text view.
WHAT — **Combines both inputs into one line of text, such as "Ada: 3-day streak".**
WHERE — The only content of the card; the two modifiers below wrap this single Text.
WHEN — Rebuilt whenever SwiftUI evaluates this card's body, with whatever inputs the card holds.
WHY — String interpolation reads the inputs directly, so the card always shows the data it was given.

```
8      .padding()
```

WHO — SwiftUI's layout system, which grows the card by the standard margin on every side.
WHAT — **Adds space around the text, inside the colored area.**
WHERE — Wraps the Text, so the space it adds sits inside the mint color applied next.
WHEN — Measured whenever the card is laid out, right after its Text is sized.
WHY — Breathing room makes the card readable instead of cramped.

```
9      .background(.mint)
```

WHO — SwiftUI's renderer, which paints a mint rectangle the size of the view it wraps.
WHAT — **Fills the padded card with mint, so the color reaches past the letters to the card edges.**
WHERE — Wraps the padding, so the color fills the padded area.
WHEN — Drawn each time the card renders, after the padding has set its size.
WHY — Because the background comes after the padding, it covers the padding too; the next lesson explores why order matters.

```
13 struct HomeView: View {
```

WHO — You declare the parent screen; the #Preview at the bottom of the file creates it.
WHAT — **Declares the parent screen that uses WelcomeCard twice.**
WHERE — At file scope, below WelcomeCard, which it uses as a building block.
WHEN — A HomeView value is created when the preview starts; it holds no inputs of its own.
WHY — Parents compose children, building bigger screens from small parts.

```
14     var body: some View {
```

WHO — SwiftUI calls HomeView's body to learn what the home screen contains.
WHAT — **Describes the home screen as a stack of two WelcomeCard values with different inputs.**
WHERE — Inside HomeView; it is the only place in the file where the two cards are created.
WHEN — When the home screen first appears, and again whenever SwiftUI needs a fresh description.
WHY — Each time this body runs it creates brand-new WelcomeCard values with the current inputs.

```
15         VStack(spacing: 24) {
```

WHO — SwiftUI's layout system, which stacks the two cards and centers them on the screen.
WHAT — **Stacks the two cards vertically, 24 points apart.**
WHERE — The root view returned by HomeView's body, the parent of both cards.
WHEN — Laid out whenever HomeView's body runs, after each card reports its size.
WHY — Generous spacing separates the cards visually.

```
16  WelcomeCard(name: "Ada", streak: 3)
```

WHO	HomeView creates the child; Swift supplies the initializer.
WHAT	**Creates a card for Ada with a 3-day streak, using the memberwise initializer Swift generated from the two let properties.**
WHERE	The first child of the VStack, so Ada's card appears above Grace's.
WHEN	A new card value is created every time HomeView's body runs, with the same inputs.
WHY	Passing inputs as labeled arguments makes it obvious what each value means.

```
17  WelcomeCard(name: "Grace", streak: 12)
```

WHO	HomeView again, creating a second card through the same memberwise initializer.
WHAT	**Creates a second card of the same type with different inputs.**
WHERE	The second child of the VStack, 24 points below Ada's card.
WHEN	Created in the same body evaluation as Ada's card, right after it.
WHY	One description, many results: reuse comes from inputs, not from copying code.

```
22  #Preview { HomeView() }
```

WHO	Xcode's preview canvas, and the phone on this page, which build HomeView and both of its cards.
WHAT	**Previews the parent, which in turn shows both cards.**
WHERE	File scope; not part of the shipped app.
WHEN	Every time you edit either struct, since a change to WelcomeCard also changes what HomeView shows.
WHY	Previewing the parent shows how the reusable child looks with real inputs.

> **Try it**
>
> 1. Add a third WelcomeCard with your own name and streak.
> 2. Change Grace's streak to 100. Only that card's text changes.
> 3. Change .mint to .yellow inside WelcomeCard: every card updates, because they share one description.

CHECK YOURSELF

Check your understanding

1. In the UIKit counter, someone deletes the line `label.text = "Count: \(count)"` from `addOne()`. After three taps, what does the screen show?
   A. Count: 3
   B. Count: 0
   C. The app crashes
   D. Count: 1
2. Which statement best describes a SwiftUI view?
   A. A long-lived object that stays on screen while you modify it step by step
   B. A UIKit view controller that happens to have a body property
   C. A value that describes part of the interface for the current data
   D. A storyboard file that Xcode draws and updates for you

3. When does SwiftUI run a view's `body`?

   A. When it first appears and whenever data it reads changes
   B. Exactly once, when the app launches and the view is created
   C. Every frame, 60 or 120 times a second, whether or not anything changed
   D. Only when your own code calls body on the view directly

4. Ada's streak grows, and the next time HomeView's body runs it passes `streak: 4` instead of `streak: 3`. What happens?

   A. The existing card object edits its own streak property in place and redraws itself
   B. Nothing changes on screen until the app restarts and HomeView is built again
   C. Every view in the app is rebuilt from scratch and the whole screen flickers briefly
   D. A new WelcomeCard value is created and only the changed text is redrawn

Answers: Describe the Screen, Not the Steps

1. **B. Count: 0** The stored `count` really is 3, but nothing copies it into the label, so the label keeps its starting text, "Count: 0". "Count: 3" looks right because the data is correct, but UIKit does not watch your data: you must update every view by hand. SwiftUI removes this whole class of bug.
2. **C. A value that describes part of the interface for the current data** A SwiftUI view is a small struct whose body describes the interface for the current data. SwiftUI creates and discards these values freely. The first choice describes UIKit objects, which you keep and mutate over time.
3. **A. When it first appears and whenever data it reads changes** SwiftUI runs body when the view appears and again after any input or state it reads changes. It does not run every frame; nothing would have changed. You never call body yourself; SwiftUI decides when a fresh description is needed.
4. **D. A new WelcomeCard value is created and only the changed text is redrawn** The card is a value with let inputs, so it never edits itself. HomeView produces a new card value, SwiftUI runs its body, compares the result with the old description, and redraws only the changed text. Rebuilding descriptions is cheap; SwiftUI does not redraw the whole screen.

Views and Modifiers

CONCEPT

The built-in views

Before you write custom views, learn the building blocks Apple ships. Each is a struct you create inside `body`, and each adapts automatically to light and dark mode, to the reader's text size, and to the platform it runs on.

Text	Shows read-only text, styled with fonts that scale with the reader's text-size setting.
Image(systemName:)	Draws an SF Symbol, a vector icon from Apple's symbol library.
Label	Pairs a title with a symbol; lists, menus and tabs decide how to show it.
Button	Runs an action closure when tapped; its look comes from a button style.
Shapes and Color	Circle, Capsule, RoundedRectangle and colors are views too. They fill the space they are offered; .fill paints a shape's inside and .stroke draws its outline.

> **SF Symbols**
>
> SF Symbols is Apple's library of thousands of icons designed to line up with San Francisco, the system font. Symbols scale with text, take their color from `foregroundStyle`, and often come in outline and `.fill` variants. Browse them in the free SF Symbols app from Apple's developer site.

CODE · LIVE SWIFTUI PREVIEW

Styling text

A quote-of-the-day card made only of Text views. Each one uses a different **text style**, a named size such as `.title2` or `.caption` that grows and shrinks with the reader's preferred text size. Edit the styles and watch the hierarchy change.

```swift
import SwiftUI

struct QuoteView: View {
    var body: some View {
        VStack(spacing: 12) {
            Text("Quote of the day")
                .font(.caption)
                .textCase(.uppercase)
            Text("Make it work, make it right, make it fast.")
                .font(.title2)
                .fontWeight(.semibold)
                .multilineTextAlignment(.center)
            Text("Kent Beck")
                .font(.subheadline)
                .italic()
                .foregroundStyle(.secondary)
        }
        .padding()
    }
}

#Preview { QuoteView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which looks up Text, VStack and the text-styling modifiers in SwiftUI.
- **WHAT** **Brings in Text, VStack and text modifiers such as font, fontWeight and textCase.**
- **WHERE** First line of the file that defines QuoteView; it applies to this file only.
- **WHEN** At compile time, before QuoteView and its modifiers are type-checked.
- **WHY** All of the styling vocabulary below is part of SwiftUI, not the Swift language.

3 `struct QuoteView: View {`

- **WHO** You declare QuoteView; SwiftUI displays it when the preview creates one.
- **WHAT** **Declares a view for a quote-of-the-day card.**
- **WHERE** At file scope, where the #Preview at the bottom of the file can reach it.
- **WHEN** A QuoteView value is created when the preview starts; the type itself is compiled once.
- **WHY** A focused view name tells readers what the screen shows.

4 `var body: some View {`

- **WHO** SwiftUI calls QuoteView's body to get the card's description; you never call it.
- **WHAT** **Returns one VStack holding the three styled lines of text: heading, quote and author.**
- **WHERE** A computed property inside QuoteView, run on the main thread.
- **WHEN** When the card first appears; with no state, SwiftUI has little reason to call it again.
- **WHY** Everything visible on the card is described here.

5 `VStack(spacing: 12) {`

- **WHO** SwiftUI's layout system, which centers each line of text under the one above it.
- **WHAT** **Stacks the heading, quote and author vertically, 12 points apart.**
- **WHERE** The root view returned by QuoteView's body, wrapped only by the padding at the end.
- **WHEN** Laid out after body runs, and again whenever the quote must rewrap for a new width.
- **WHY** Consistent spacing groups the three texts into one card.

6 `Text("Quote of the day")`

- WHO — SwiftUI's Text view, holding the plain string "Quote of the day".
- WHAT — **A small heading that labels the card, styled by the two modifiers below it.**
- WHERE — The first child of the VStack, so it sits above the quote.
- WHEN — Created as part of each evaluation of body, before its modifiers wrap it.
- WHY — A heading tells the reader what kind of content follows.

7 `.font(.caption)`

- WHO — SwiftUI, with Dynamic Type scaling the caption style to the reader's chosen text size.
- WHAT — **Sets the heading in the caption style, the smallest text on the card.**
- WHERE — Attached to the heading Text only; the quote and the author set their own fonts.
- WHEN — Applied each time the heading is built, and rescaled when the text size setting changes.
- WHY — A small label should not compete with the quote itself.

8 `.textCase(.uppercase)`

- WHO — SwiftUI's text rendering, which changes the letter case only at the moment of drawing.
- WHAT — **Displays the heading in capital letters without changing the string in the code.**
- WHERE — Wraps the heading after its caption font, so it affects only the heading's display.
- WHEN — Applied each time the heading is drawn, so the stored string stays in its original case.
- WHY — Keeping the original string lets translations and accessibility tools see normal text.

9 `Text("Make it work, make it right, make it fast.")`

- WHO — SwiftUI's Text view, which wraps the sentence onto as many lines as it needs.
- WHAT — **The quote itself; it wraps onto more lines when it does not fit the width.**
- WHERE — The second child of the VStack, the visual center of the card.
- WHEN — Created each time body runs; the line breaks are worked out later, during layout.
- WHY — Text wraps automatically, so you never measure strings by hand.

10 `.font(.title2)`

- WHO — SwiftUI and Dynamic Type, which scale the title 2 style for each reader.
- WHAT — **Uses the second-level title style, the largest text on the card.**
- WHERE — Attached to the quote Text only, so the heading above stays small.
- WHEN — Applied each time the quote is built, then again if the reader picks a new text size.
- WHY — The most important text gets the largest style.

11 `.fontWeight(.semibold)`

- WHO — SwiftUI's text styling, which chooses the semibold weight of the current font.
- WHAT — **Makes the quote a little heavier than regular text.**
- WHERE — Chained after .font(.title2) on the quote, so the quote is both large and semibold.
- WHEN — Applied whenever the quote Text is created in body.
- WHY — Weight is a second way to build hierarchy besides size.

12 `.multilineTextAlignment(.center)`

- WHO — SwiftUI's text layout engine, which decides where each wrapped line starts.
- WHAT — **Centers every wrapped line of the quote.**
- WHERE — Attached to the quote Text; single-line texts like the heading do not need it.
- WHEN — Used whenever the quote needs more than one line.
- WHY — Without it, wrapped lines start at the leading edge, which looks unbalanced on a centered card.

```
13  Text("Kent Beck")
```

WHO — SwiftUI's Text view, holding the author's name as a plain string.
WHAT — **The attribution line: the name of the person who said the quote.**
WHERE — The last child of the VStack, directly below the quote.
WHEN — Created in each evaluation of body, together with the heading and the quote.
WHY — Attribution belongs with the quote but should stay quiet.

```
14  .font(.subheadline)
```

WHO — SwiftUI and Dynamic Type, which size the subheadline style relative to the reader's setting.
WHAT — **Uses the subheadline style, slightly smaller than body text.**
WHERE — Attached to the author Text, the first of its three modifiers.
WHEN — Applied whenever the author line is built in body.
WHY — Secondary information gets a smaller style.

```
15  .italic()
```

WHO — SwiftUI's text styling, which switches to the italic version of the current font.
WHAT — **Slants the letters of "Kent Beck" while keeping the subheadline size.**
WHERE — The second modifier on the author Text, between its font and its color.
WHEN — Applied each time the author Text is created.
WHY — Italics are a familiar convention for attributions.

```
16  .foregroundStyle(.secondary)
```

WHO — SwiftUI, resolving the system secondary label color for the current appearance.
WHAT — **Colors the name a softer gray that adapts to dark mode.**
WHERE — The last modifier on the author Text, so only the attribution turns gray.
WHEN — Resolved as the card is drawn, and again when the device switches between light and dark mode.
WHY — Semantic colors such as .secondary stay readable in both light and dark mode.

```
18  .padding()
```

WHO — SwiftUI's layout system, which adds the standard margin on all four sides.
WHAT — **Keeps the card away from the screen edges.**
WHERE — Wraps the whole VStack, so it pads the card as one unit rather than each line.
WHEN — Measured during each layout pass, after the VStack knows its own size.
WHY — Wrapped text that touches the edge of the screen is hard to read.

```
22  #Preview { QuoteView() }
```

WHO — Xcode's preview canvas, and the phone on this page, which show QuoteView as you type.
WHAT — **Creates a QuoteView and renders the finished card with its three text styles.**
WHERE — At file scope after QuoteView; Xcode leaves preview code out of the shipped app.
WHEN — Each time you change a font, weight or color, so you can compare styles immediately.
WHY — Typography is easiest to judge by looking at it.

> **Try it**
>
> 1. Change .title2 to .largeTitle. The quote now wraps onto more lines, still centered.
> 2. Change .center to .leading and compare how the wrapped lines line up.
> 3. Change .semibold to .black, then to .light.

CODE · LIVE SWIFTUI PREVIEW

Symbols and labels

A weather summary built from SF Symbols. `Image(systemName:)` draws a symbol on its own, while `Label` pairs a symbol with a title so the system can choose how to show it. Symbols size like text, so you

style them with `.font`.

```swift
import SwiftUI

struct WeatherRowView: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 16) {
            HStack {
                Image(systemName: "sun.max.fill")
                    .font(.largeTitle)
                    .foregroundStyle(.orange)
                Text("Sunny, 24°")
                    .font(.title)
            }
            Label("Light wind", systemImage: "wind")
            Label("Humidity 40%", systemImage: "drop")
                .labelStyle(.titleOnly)
        }
        .padding()
    }
}

#Preview { WeatherRowView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which needs SwiftUI to know Image, Label and the stack types.
- **WHAT** **Brings in Image, Label, HStack, VStack and the label styles.**
- **WHERE** Top of the WeatherRowView file; each Swift file lists its own imports.
- **WHEN** At compile time; the symbol artwork itself comes from the system when the app runs.
- **WHY** SF Symbols are drawn through SwiftUI's Image and Label views.

3 `struct WeatherRowView: View {`

- **WHO** You declare WeatherRowView; the preview creates it and SwiftUI draws it.
- **WHAT** **Declares a view that summarizes today's weather.**
- **WHERE** At file scope, so a full weather screen elsewhere could embed this summary.
- **WHEN** A value is created when the preview starts; the type is compiled only once.
- **WHY** A weather summary is a natural mix of icons and short text.

4 `var body: some View {`

- **WHO** SwiftUI calls WeatherRowView's body to find out what the summary contains.
- **WHAT** **Returns a VStack of three rows: the temperature, the wind and the humidity.**
- **WHERE** A computed property inside WeatherRowView, evaluated on the main thread.
- **WHEN** When the summary appears; nothing in it changes, so it rarely runs again.
- **WHY** Describing the whole summary in one body keeps its structure visible at a glance.

5 `VStack(alignment: .leading, spacing: 16) {`

- **WHO** SwiftUI's layout system, which lines up the three rows down the screen.
- **WHAT** **Stacks the rows vertically, lined up on their leading (left, in English) edges, 16 points apart.**
- **WHERE** The root view returned by WeatherRowView's body, wrapped only by its final padding.
- **WHEN** Laid out after body runs, and again if the reader's text size changes.
- **WHY** Leading alignment makes a list of facts easy to scan.

```
6  HStack {
```

WHO SwiftUI's layout system, which places the sun and the temperature in one row.
WHAT **Places the big symbol and the temperature side by side.**
WHERE The first child of the VStack, above the wind and humidity labels.
WHEN Laid out together with its parent VStack, after body runs.
WHY An icon next to its value reads as one unit.

```
7  Image(systemName: "sun.max.fill")
```

WHO SwiftUI, which draws the sun.max.fill symbol from Apple's SF Symbols set.
WHAT **Draws a filled sun with rays, the usual icon for clear weather.**
WHERE The leading child of the HStack, before the temperature text.
WHEN Created each time body runs, then sized and colored by the two modifiers below.
WHY The .fill variant reads more strongly at large sizes.

```
8  .font(.largeTitle)
```

WHO SwiftUI and Dynamic Type, which treat the symbol like large-title text.
WHAT **Makes the sun as big as large-title text, much larger than the default symbol size.**
WHERE Attached directly to the sun Image, the first of its two modifiers.
WHEN Applied whenever the Image is built, and rescaled when the text size setting changes.
WHY Sizing symbols with fonts keeps them in proportion with nearby text at every text size.

```
9  .foregroundStyle(.orange)
```

WHO SwiftUI's renderer, which paints the symbol's shapes with the given color.
WHAT **Paints the sun orange instead of the default primary label color.**
WHERE Wraps the sun after it was sized; the temperature text keeps its normal color.
WHEN Applied each time the sun is drawn, in both light and dark mode.
WHY Symbols take their color from the foreground style, just like text.

```
10 Text("Sunny, 24°")
```

WHO SwiftUI's Text view, which draws the degree sign like any other character.
WHAT **Shows the conditions and the temperature, "Sunny, 24°", next to the sun.**
WHERE The trailing child of the HStack, right after the sun symbol.
WHEN Created in each evaluation of body, then styled by the font below it.
WHY Placing the value right beside its icon lets people read the pair as one fact.

```
11 .font(.title)
```

WHO SwiftUI and Dynamic Type, applying the title text style to the temperature.
WHAT **Sets the temperature in the title style, smaller than the sun but larger than the rows below.**
WHERE Attached only to the temperature Text inside the HStack.
WHEN Applied whenever the temperature Text is created in body.
WHY The headline fact deserves the largest text.

```
13 Label("Light wind", systemImage: "wind")
```

WHO SwiftUI's Label view, which pairs the wind symbol with its title automatically.
WHAT **Shows a wind symbol followed by a title.**
WHERE The second child of the VStack, under the temperature row.
WHEN Created each time body runs, with the icon and the title already aligned.
WHY Label keeps icon and text together as one meaningful item, aligned the way the system expects.

```
14  Label("Humidity 40%", systemImage: "drop")
```

WHO — SwiftUI's Label view again, this time pairing a drop symbol with its title.
WHAT — **Shows a drop symbol with a humidity title.**
WHERE — The third and last child of the VStack, under the wind row.
WHEN — Created in each evaluation of body, then restyled by .labelStyle below.
WHY — A Label still carries both parts even when only one is displayed.

```
15  .labelStyle(.titleOnly)
```

WHO — SwiftUI's label styling system, which decides how a Label shows its two parts.
WHAT — **Shows only the title of this Label and hides its icon.**
WHERE — Attached to the humidity Label only; the wind Label still shows its icon.
WHEN — Applied each time the humidity Label is built.
WHY — Containers and modifiers decide how a Label appears without you rewriting it.

```
17  .padding()
```

WHO — SwiftUI's layout system, which adds the standard margin on every side.
WHAT — **Adds standard space around the whole summary, between the rows and the screen edges.**
WHERE — Wraps the VStack, so the margin surrounds all three rows together.
WHEN — Measured during each layout pass, after the VStack has its size.
WHY — Content that touches the screen edges looks cramped and can sit under rounded corners.

```
21  #Preview { WeatherRowView() }
```

WHO — Xcode's preview canvas, and the phone on this page, which render WeatherRowView.
WHAT — **Creates a WeatherRowView and shows the sun, the temperature and both labels live.**
WHERE — At file scope after WeatherRowView; this code is not compiled into the shipped app.
WHEN — Each time you edit the file, for example right after swapping a symbol name.
WHY — You can swap symbols and styles and judge the result instantly.

> **Try it**
>
> 1. Change "sun.max.fill" to "cloud.rain" and .orange to .blue.
> 2. Change .titleOnly to .iconOnly on the humidity Label.
> 3. Add .labelStyle(.iconOnly) to the whole VStack, after .padding(), and see which rows change.

CODE · LIVE SWIFTUI PREVIEW

Buttons and button styles

A coffee brewing card with two buttons written in the two common forms: a title with an action, and a custom label. The `@State` line lets the view remember a number between taps; the State lesson later in this module explains it fully. Tap both buttons in the phone.

```swift
import SwiftUI

struct BrewCardView: View {
    @State private var cups = 0
    var body: some View {
        VStack(spacing: 16) {
            Text("Cups brewed: \(cups)")
            Button("Brew a cup") {
                cups += 1
            }
            .buttonStyle(.borderedProminent)
            Button {
                cups = 0
            } label: {
                Label("Start over", systemImage: "arrow.clockwise")
            }
            .buttonStyle(.bordered)
        }
    }
}

#Preview { BrewCardView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which resolves Button, Label, the button styles and @State from SwiftUI.
- **WHAT** **Brings in Button, Label, the button styles and the @State wrapper.**
- **WHERE** First line of the BrewCardView file; it applies to this file only.
- **WHEN** At compile time, before BrewCardView is type-checked.
- **WHY** Buttons and their styles are SwiftUI types.

3 `struct BrewCardView: View {`

- **WHO** You declare BrewCardView; the preview creates it and SwiftUI keeps its state alive.
- **WHAT** **Declares a card for counting cups of coffee.**
- **WHERE** At file scope, where the #Preview at the bottom of the file can create it.
- **WHEN** Compiled once; SwiftUI may create many BrewCardView values, but the cups state survives each one.
- **WHY** A tiny real task makes button behavior easy to see.

4 `@State private var cups = 0`

- **WHO** SwiftUI, which stores cups for this card outside the struct so the value survives rebuilds.
- **WHAT** **Remembers how many cups have been brewed, starting at 0.**
- **WHERE** Storage managed by SwiftUI, attached to this view on screen.
- **WHEN** Created when the view first appears and kept between taps.
- **WHY** A plain property would reset every time SwiftUI rebuilt the struct; @State keeps the value alive.

5 `var body: some View {`

- **WHO** SwiftUI calls BrewCardView's body after every change to cups; your code never calls it.
- **WHAT** **Describes the card for the current number of cups.**
- **WHERE** A computed property inside BrewCardView, run on the main thread.
- **WHEN** When the card appears and after every change to cups.
- **WHY** The buttons change cups; body turns cups into what you see.

```
6  VStack(spacing: 16) {
```

WHO	SwiftUI's layout system, which stacks the count above the two buttons.
WHAT	**Stacks the count and two buttons, 16 points apart.**
WHERE	The root view returned by body; this card has no outer padding.
WHEN	Laid out again after each tap, because body produces a new description.
WHY	Even spacing keeps the buttons easy to tell apart and to tap.

```
7  Text("Cups brewed: \(cups)")
```

WHO	Swift builds the "Cups brewed: 2" style string; SwiftUI draws it as a Text view.
WHAT	**Shows the current number of cups, for example "Cups brewed: 2".**
WHERE	The first child of the VStack, above both buttons.
WHEN	Rebuilt every time body runs, which happens after each tap on either button.
WHY	It reads the state directly, so it is always current.

```
8  Button("Brew a cup") {
```

WHO	You declare the button; the person using the app runs its action by tapping it.
WHAT	**The short form: a title string plus an action closure.**
WHERE	The second child of the VStack, between the cup count and the Start over button.
WHEN	Its closure runs on the main thread at each tap of Brew a cup.
WHY	Use this form when the button only needs text.

```
9  cups += 1
```

WHO	The action closure; SwiftUI notices the change.
WHAT	**Adds one to the cups state, which makes SwiftUI run body again.**
WHERE	Inside the action closure of the Brew a cup button.
WHEN	Once per tap on Brew a cup, never while body is being built.
WHY	The action changes data, and SwiftUI updates the text for you.

```
11 .buttonStyle(.borderedProminent)
```

WHO	SwiftUI, which draws the button with the system's filled, tinted appearance.
WHAT	**Gives the button a filled, tinted background.**
WHERE	Attached to the Brew a cup Button; the Start over button gets its own style below.
WHEN	Applied each time body creates the Brew a cup button.
WHY	One prominent button marks the main action on the screen.

```
12 Button {
```

WHO	You declare the second button; the user runs its action by tapping Start over.
WHAT	**The long form: the action closure comes first, then a custom label.**
WHERE	The third child of the VStack, below the Brew a cup button.
WHEN	Its action closure runs at each tap of Start over, on the main thread.
WHY	Use this form when the label needs more than plain text, such as an icon.

```
13 cups = 0
```

WHO	The Start over button's action closure, which overwrites the stored state.
WHAT	**Sets cups back to zero, so the text reads "Cups brewed: 0" after the redraw.**
WHERE	Inside the action closure of the second Button, the one with the custom label.
WHEN	Each time the user taps Start over; SwiftUI then runs body again.
WHY	Both buttons change the same state, and the one Text reflects either change.

14 `} label: {`

WHO Swift's multiple-trailing-closure syntax, which lets one call take an action closure and a label closure.
WHAT **Closes the action and opens the label closure that describes how the button looks.**
WHERE Between the two closures of this Button.
WHEN The label closure is evaluated whenever body runs and SwiftUI builds this button.
WHY Multiple trailing closures keep both parts readable without nested parentheses.

15 `Label("Start over", systemImage: "arrow.clockwise")`

WHO SwiftUI's Label view, which pairs the arrow symbol with the words Start over.
WHAT **Shows a circular arrow symbol beside the words Start over.**
WHERE Inside the Button's label closure, so it is what the button displays.
WHEN Built each time body runs and the second button is created.
WHY An icon helps people recognize a reset action at a glance.

17 `.buttonStyle(.bordered)`

WHO SwiftUI, which draws the button with the lighter, tinted bordered appearance.
WHAT **Gives the reset button a lighter, tinted background.**
WHERE Attached to the Start over Button, after its closing brace.
WHEN Applied each time body creates the Start over button.
WHY A secondary action looks quieter than the main one.

22 `#Preview { BrewCardView() }`

WHO Xcode's preview canvas, and the phone on this page, which run BrewCardView with working buttons.
WHAT **Creates a BrewCardView whose buttons really change the count when tapped.**
WHERE At file scope after BrewCardView; it is not compiled into the app you ship.
WHEN Each time the code changes; the preview restarts and cups goes back to 0.
WHY Previews are interactive, so you can test the buttons right here.

> **Try it**
>
> 1. Change .borderedProminent to .glassProminent to see the iOS 26 Liquid Glass style.
> 2. Change .bordered to .borderless.
> 3. Add .controlSize(.large) on the line below .buttonStyle(.borderedProminent).

CONCEPT

Modifiers wrap views

A **modifier** is a method you call on a view, such as `.padding()` or `.background(.yellow)`. It does not change the original view. It returns a *new* view that wraps the old one.

So `Text("Hi").padding().background(.yellow)` builds three layers: a Text, a padding view around the Text, and a background view around the padding. The yellow fills the padded area because the background wraps the padding.

Chain	Reading top to bottom, each modifier wraps everything above it.
Order matters	The same modifiers in a different order build a different tree and draw differently.
Inherited	Some modifiers, like .font on a VStack, flow down to every child inside.

Read a chain like a recipe

“Take the text, then pad it, then paint behind that.” When a view looks wrong, check the order of its modifiers first.

INTERACTIVE

Modifier stack lab

Each chip is one modifier applied to the same Text. Move chips up and down, switch them on and off, and watch the layers: every modifier wraps the view produced by the ones above it. Put `.background` above `.padding`, then below it, and notice which area the color fills. The generated code updates as you go.

This step is an interactive simulation in the HTML version.

CODE · LIVE SWIFTUI PREVIEW

Modifier order changes the result

Three Text views use the same few modifiers in different orders. The first pads and then paints, the second paints and then pads, and the third alternates to build two colored bands. Predict each result before you look at the phone, then try the challenges.

```swift
import SwiftUI

struct ModifierOrderView: View {
    var body: some View {
        VStack(spacing: 32) {
            Text("Padding first")
                .padding()
                .background(.yellow)
            Text("Background first")
                .background(.yellow)
                .padding()
            Text("Two layers")
                .padding(8)
                .background(.orange)
                .padding(8)
                .background(.blue)
        }
    }
}

#Preview { ModifierOrderView() }
```

Line by line

`1` `import SwiftUI`

- WHO — The Swift compiler, which resolves Text, VStack, padding and background from SwiftUI.
- WHAT — **Brings in Text, VStack and the padding and background modifiers compared below.**
- WHERE — First line of the ModifierOrderView file; it applies to this file only.
- WHEN — At compile time, before the three modifier chains are type-checked.
- WHY — Every modifier here is defined by SwiftUI.

```
3  struct ModifierOrderView: View {
```

WHO You declare ModifierOrderView; the preview creates it and SwiftUI draws it.
WHAT **Declares a view that puts three modifier orders side by side for comparison.**
WHERE At file scope, outside any other type, so the #Preview below can create one.
WHEN A value is created when the preview starts; the type is compiled just once.
WHY Comparing results in one screen makes the effect of order impossible to miss.

```
4  var body: some View {
```

WHO SwiftUI calls ModifierOrderView's body to get the three experiments.
WHAT **Returns a VStack holding three Text views, each wrapped by a different chain of modifiers.**
WHERE A computed property inside ModifierOrderView, run on the main thread.
WHEN When the view appears; it has no state, so it seldom runs again.
WHY Putting all three experiments in one body lets you compare them side by side on one screen.

```
5  VStack(spacing: 32) {
```

WHO SwiftUI's layout system, which centers the three experiments in a column.
WHAT **Stacks the examples with 32 points between them.**
WHERE The root view returned by body; each child is one complete modifier chain.
WHEN Laid out after body runs and whenever an experiment changes size.
WHY Wide spacing keeps the colored areas from touching, so each is easy to judge.

```
6  Text("Padding first")
```

WHO SwiftUI's Text view, the core of the first modifier chain.
WHAT **First experiment: the innermost layer of its chain.**
WHERE The first child of the VStack, at the top of the screen.
WHEN Created each time body runs, before the padding and background wrap it.
WHY The modifiers below wrap this Text one layer at a time.

```
7  .padding()
```

WHO SwiftUI's layout system, which reports a size larger than the Text by the default margin.
WHAT **Wraps the Text in a view that is larger by the default padding on every side.**
WHERE The first wrapper around "Padding first", directly on the Text.
WHEN Measured during each layout pass, before the background reads its size.
WHY Padding first means the next layer sees the bigger, padded view.

```
8  .background(.yellow)
```

WHO SwiftUI's renderer, which fills the full size of the padded view with yellow.
WHAT **Paints yellow behind the padded view, so the color extends past the letters.**
WHERE The outer wrapper of the first chain, around the padding.
WHEN Drawn each time the first experiment renders, after layout has sized the padding.
WHY A background fills the size of whatever it wraps, here the padded area.

```
9  Text("Background first")
```

WHO SwiftUI's Text view, the core of the second modifier chain.
WHAT **Second experiment, with the same two modifiers swapped.**
WHERE The second child of the VStack, between the other two experiments.
WHEN Created in the same body evaluation as the first experiment.
WHY Only the order changes, so any visible difference comes from order alone.

10 `.background(.yellow)`

WHO	SwiftUI's renderer, filling only the space that the bare Text occupies.
WHAT	**Paints yellow behind the bare Text, hugging the letters.**
WHERE	The first wrapper around "Background first", directly on the Text.
WHEN	Drawn each time the second experiment renders on screen.
WHY	At this point the view is only as big as its text, so the color is too.

11 `.padding()`

WHO	SwiftUI's layout system, adding transparent space around the yellow view.
WHAT	**Adds clear space outside the yellow area.**
WHERE	The outer wrapper of the second chain, around the background.
WHEN	Measured during each layout pass, after the background has taken the Text's size.
WHY	Padding after a background adds spacing between views, not color.

12 `Text("Two layers")`

WHO	SwiftUI's Text view, the core of the third and longest chain.
WHAT	**Third experiment: alternating padding and backgrounds.**
WHERE	The last child of the VStack, at the bottom of the screen.
WHEN	Created in each evaluation of body, then wrapped four times.
WHY	Alternating modifiers is a common way to build borders and rings.

13 `.padding(8)`

WHO	SwiftUI's layout system, which grows the Text by exactly 8 points per side.
WHAT	**Adds 8 points of space on every side of "Two layers", before any color.**
WHERE	The first of four wrappers around "Two layers".
WHEN	Measured during each layout pass, before the orange background reads its size.
WHY	This padding sets the width of the inner orange band.

14 `.background(.orange)`

WHO	SwiftUI's renderer, filling the text plus its first 8-point margin with orange.
WHAT	**Paints orange behind the text plus its 8 points.**
WHERE	The second of four wrappers, around the first padding.
WHEN	Drawn each time the third experiment renders.
WHY	It produces the inner orange band, whose width the first .padding(8) decided.

15 `.padding(8)`

WHO	SwiftUI's layout system, adding a second 8-point margin around the orange view.
WHAT	**Adds 8 more points around the orange area.**
WHERE	The third of four wrappers, around the orange background.
WHEN	Measured during each layout pass, after the orange view has its size.
WHY	This padding sets the width of the outer blue band.

16 `.background(.blue)`

WHO	SwiftUI's renderer, filling the whole outer size with blue behind the orange.
WHAT	**Paints blue behind everything so far, showing as a frame around the orange.**
WHERE	The fourth and outermost wrapper of the third chain.
WHEN	Drawn each time the third experiment renders, behind everything else in its chain.
WHY	Each background only shows where the views it wraps do not cover it.

```
21 #Preview { ModifierOrderView() }
```

WHO	Xcode's preview canvas, and the phone on this page, which render ModifierOrderView.
WHAT	**Creates a ModifierOrderView and shows all three modifier chains at once.**
WHERE	At file scope after ModifierOrderView; preview code never ships in the app.
WHEN	Each time you reorder a modifier, so the colored areas change as you type.
WHY	Instant feedback turns modifier order into something you can see.

Try it

1. In the second Text, move .padding() above .background(.yellow). It now matches the first.
2. Add .clipShape(Capsule()) right under .background(.yellow) in the first Text to make a pill. Then move it above .background and see what changes.
3. Change the first .padding(8) in "Two layers" to .padding(24). Which band grows, orange or blue?

CHECK YOURSELF

Check your understanding

1. Predict the result: `Text("Hi").background(.yellow).padding(20)`
   A. Yellow fills the text plus 20 points on every side
   B. Yellow hugs the text; the 20 points of padding around it stay clear
   C. Nothing yellow appears
   D. It fails to compile because padding must come first

2. You want a yellow pill with room around the word, but this pill hugs the letters: `Text("Sale").background(.yellow).clipShape(Capsule()).padding()`. Which fix works?
   A. Add a second .padding() at the very end
   B. Change Capsule() to RoundedRectangle(cornerRadius: 20)
   C. Move .padding() so it comes directly after Text("Sale")
   D. Replace .background(.yellow) with .foregroundStyle(.yellow)

3. A toolbar needs a trash symbol with the word Delete, and the toolbar should decide whether to show the icon, the title, or both. What should you write?
   A. Text("Delete") with .bold()
   B. An HStack holding an Image and a Text
   C. Image(systemName: "trash") on its own
   D. Label("Delete", systemImage: "trash")

4. What does calling `.padding()` on a Text give you?
   A. A new view wrapping the Text with space around it
   B. The same Text, with a padding property changed in place
   C. A UIKit label with insets applied to its text
   D. Nothing visible until you also call .background on it

Answers: Views and Modifiers

1. **B. Yellow hugs the text; the 20 points of padding around it stay clear** The background wraps the bare Text, so the yellow is exactly as big as the letters. The padding is applied afterward, outside the yellow, so it adds clear space. The first choice describes `.padding(20).background(.yellow)`, the reverse order.
2. **C. Move .padding() so it comes directly after Text("Sale")** The padding must be inside the color, so it has to wrap the Text before the background does. Adding more padding at the end only adds more clear space outside the pill. Changing the shape does not change how big the colored area is.
3. **D. Label("Delete", systemImage: "trash")** Label keeps the title and symbol together as one meaningful item, so containers and label styles can choose how to present it. An HStack looks similar but is just two unrelated views; nothing can restyle it as a single label, and an icon alone gives assistive technologies no title.
4. **A. A new view wrapping the Text with space around it** Modifiers never change the original view; they return a new view that wraps it. That is why order matters: each modifier wraps the result of the ones before it. Padding is visible on its own as extra space, even without a background.

Stacks, Frames, and Grids

CONCEPT

How SwiftUI lays out a screen

Layout in SwiftUI is a short conversation between a parent view and each child, repeated all the way down the view tree. Three steps happen every time.

1. Propose	The parent offers the child a size, such as 390 by 600 points.
2. Choose	The child picks its own size. Text takes just enough room for its words; Color and shapes take everything offered. The parent must respect the choice.
3. Place	The parent positions the child inside its own bounds, using its alignment.
Point	The unit of layout. One point may be two or three physical pixels, depending on the screen.

> **Let content decide size**
>
> Avoid fixed widths and heights where you can. Stacks, `Spacer`, padding and `.frame(maxWidth: .infinity)` let one layout fit every iPhone, iPad windows of any size, and larger text settings.

CODE · LIVE SWIFTUI PREVIEW

Stacks, alignment, and spacing

A contact header from a study-group chat. An **HStack** places the avatar beside a **VStack** of text, so stacks nest inside stacks. Each stack has two settings: *alignment*, which lines children up across the stack, and *spacing*, the gap between them.

```swift
import SwiftUI

struct ContactHeaderView: View {
    var body: some View {
        HStack(alignment: .top, spacing: 16) {
            Image(systemName: "person.circle")
                .font(.system(size: 56))
                .foregroundStyle(.tint)
            VStack(alignment: .leading, spacing: 4) {
                Text("Mia Chen")
                    .font(.title2)
                    .bold()
                Text("Study group: Mobile Apps")
                    .foregroundStyle(.secondary)
                Text("3 unread messages").font(.footnote)
            }
        }
        .padding()
    }
}

#Preview { ContactHeaderView() }
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, which resolves HStack, VStack, Image and Text from SwiftUI.
WHAT	**Brings in HStack, VStack, Image and Text.**
WHERE	First line of the ContactHeaderView file; it applies to this file only.
WHEN	At compile time, before the nested stacks are type-checked.
WHY	Stacks are SwiftUI's basic layout containers.

```
3  struct ContactHeaderView: View {
```

WHO	You declare ContactHeaderView; the preview creates it and SwiftUI lays it out.
WHAT	**Declares the header at the top of a chat conversation.**
WHERE	At file scope, so a chat screen could place this header above its messages.
WHEN	Created when the preview starts; a real app would create one per open conversation.
WHY	A header is a classic mix of horizontal and vertical layout.

```
4  var body: some View {
```

WHO	SwiftUI calls ContactHeaderView's body to learn the header's structure.
WHAT	**Returns an HStack holding the avatar and a nested VStack of three text lines.**
WHERE	A computed property inside ContactHeaderView, evaluated on the main thread.
WHEN	When the header first appears; without state it has little reason to run again.
WHY	The whole header is one tree of nested stacks, so its structure reads directly from the code.

```
5  HStack(alignment: .top, spacing: 16) {
```

WHO	SwiftUI's layout system, which arranges the avatar and the text block in one row.
WHAT	**Places the avatar and the text block side by side, top edges aligned, 16 points apart.**
WHERE	The root view returned by body, wrapped only by the padding at the end.
WHEN	Laid out after body runs, and again if the text size or the screen width changes.
WHY	Top alignment keeps the avatar level with the name even when the text block is taller.

```
6  Image(systemName: "person.circle")
```

WHO	SwiftUI, drawing the person.circle symbol from Apple's SF Symbols set.
WHAT	**Shows a person-in-a-circle symbol as the contact's avatar.**
WHERE	The leading child of the HStack, before the block of text.
WHEN	Created each time body runs, then sized and tinted by the two modifiers below.
WHY	A symbol stands in for a photo until one is available.

```
7  .font(.system(size: 56))
```

WHO	SwiftUI, which sizes the symbol as if it were a 56-point character of text.
WHAT	**Draws the avatar symbol at a fixed 56-point size.**
WHERE	Attached directly to the avatar Image, the first of its two modifiers.
WHEN	Applied whenever the avatar Image is built in body.
WHY	A large avatar anchors the header visually.

```
8  .foregroundStyle(.tint)
```

WHO	SwiftUI, reading the app's accent color, which is blue by default.
WHAT	**Colors the avatar symbol with the app's tint instead of the plain label color.**
WHERE	Wraps the avatar after it was sized; the text block is unaffected.
WHEN	Resolved each time the header is drawn, so a changed accent color shows immediately.
WHY	Tinted elements stay consistent with the rest of the app.

```
9  VStack(alignment: .leading, spacing: 4) {
```

WHO SwiftUI's layout system, laying out a column of text inside the row.
WHAT **A nested stack: three lines of text, leading edges aligned, 4 points apart.**
WHERE The trailing child of the HStack, to the right of the avatar.
WHEN Laid out as part of the HStack's layout pass, after body runs.
WHY Nesting stacks is how SwiftUI builds rows and columns; tight spacing groups the lines as one block.

```
10 Text("Mia Chen")
```

WHO SwiftUI's Text view, holding the contact's name as a plain string.
WHAT **Shows the contact's name, the first line of the text block.**
WHERE The first child of the inner VStack, at the top of the text block.
WHEN Created each time body runs, then styled by the two modifiers below.
WHY People scan a header for the name first, so it leads the block.

```
11 .font(.title2)
```

WHO SwiftUI and Dynamic Type, which scale the name's title 2 style to the reader's text size.
WHAT **Uses the title 2 text style for the name.**
WHERE Attached to the name Text only, the first of its two modifiers.
WHEN Applied whenever the name is built, and rescaled when the text size setting changes.
WHY A larger text style tells people the name matters more than the lines below it.

```
12 .bold()
```

WHO SwiftUI's text styling, which switches the name to its bold weight.
WHAT **Draws "Mia Chen" in bold on top of the title 2 size.**
WHERE Chained after .font(.title2) on the name Text.
WHEN Applied each time the name Text is created in body.
WHY Weight separates the name from the details below it.

```
13 Text("Study group: Mobile Apps")
```

WHO SwiftUI's Text view, holding the name of the contact's group.
WHAT **Shows which study group the contact belongs to, "Study group: Mobile Apps".**
WHERE The second child of the inner VStack, directly below the name.
WHEN Created in each evaluation of body, between the name and the status line.
WHY Supporting context sits right under the name.

```
14 .foregroundStyle(.secondary)
```

WHO SwiftUI, resolving the system secondary label color for light or dark mode.
WHAT **Draws the group line in a softer gray than the name above it.**
WHERE Attached only to the group Text, so the name keeps the primary color.
WHEN Resolved as the header is drawn, and again when the appearance changes.
WHY Quieter color makes this line read as a detail.

```
15 Text("3 unread messages").font(.footnote)
```

WHO SwiftUI's Text view, sized by Dynamic Type's footnote style.
WHAT **A small status line, with its modifier written on the same line.**
WHERE The last child of the inner VStack, at the bottom of the text block.
WHEN Created together with its font in each evaluation of body.
WHY Short single-modifier chains can stay on one line when that reads clearly.

`18` `.padding()`

WHO	SwiftUI's layout system, which adds the standard margin on every side of the row.
WHAT	**Adds standard space around the whole header, avatar and text together.**
WHERE	Wraps the outer HStack, so it pads the header as one unit.
WHEN	Measured during each layout pass, after the HStack knows its size.
WHY	A margin keeps the avatar and the text away from the screen edges and rounded corners.

`22` `#Preview { ContactHeaderView() }`

WHO	Xcode's preview canvas, and the phone on this page, which render ContactHeaderView.
WHAT	**Creates a ContactHeaderView and shows the avatar beside its three lines of text.**
WHERE	At file scope after ContactHeaderView; it is left out of the shipped app.
WHEN	Each time you change an alignment or spacing value, so you see the effect at once.
WHY	Alignment changes are easiest to understand by watching them.

> **Try it**
>
> 1. Change the VStack's alignment: .leading to .trailing.
> 2. Change the HStack's alignment from .top to .center, then .bottom, and watch the avatar move.
> 3. Change spacing: 16 to spacing: 48.

CODE · LIVE SWIFTUI PREVIEW

ZStack: views on top of views

An unread badge on a mail icon. A **ZStack** layers its children back to front, like sheets of paper; the last child is drawn on top. Its alignment decides where smaller children sit, here the top-trailing corner.

```swift
import SwiftUI

struct InboxIconView: View {
    var body: some View {
        ZStack(alignment: .topTrailing) {
            Image(systemName: "envelope")
                .font(.system(size: 64))
                .padding(8)
            Text("3")
                .font(.caption)
                .bold()
                .foregroundStyle(.white)
                .padding(.horizontal, 6)
                .frame(minWidth: 24, minHeight: 24)
                .background(.red)
                .clipShape(Capsule())
        }
    }
}

#Preview { InboxIconView() }
```

Line by line

```
1  import SwiftUI
```

WHO The Swift compiler, which resolves ZStack, Image, Text, frame and clipShape from SwiftUI.
WHAT **Brings in ZStack, Image, Text and the frame and clipShape modifiers.**
WHERE First line of the InboxIconView file; it applies to this file only.
WHEN At compile time, before the layered icon is type-checked.
WHY Layering views is part of SwiftUI's layout system.

```
3  struct InboxIconView: View {
```

WHO You declare InboxIconView; the preview creates it and SwiftUI draws the layers.
WHAT **Declares a mail icon with an unread-count badge.**
WHERE At file scope, so a toolbar or a tab bar could reuse this icon.
WHEN A value is created when the preview starts; the type is compiled once, ahead of time.
WHY Badges are one of the most common uses of layering.

```
4  var body: some View {
```

WHO SwiftUI calls InboxIconView's body to learn which layers to draw.
WHAT **Returns a ZStack with the envelope at the back and the red count badge in front.**
WHERE A computed property inside InboxIconView, run on the main thread.
WHEN When the icon appears; with a fixed count of 3 it rarely needs to run again.
WHY The icon and its badge are one small view tree, so one body describes both layers.

```
5  ZStack(alignment: .topTrailing) {
```

WHO SwiftUI's layout system, which stacks children on top of each other instead of side by side.
WHAT **Layers its children and pins smaller ones to the top-trailing corner.**
WHERE The root view returned by body; it has no outer modifiers.
WHEN Laid out after body runs, sized to fit its largest child, the padded envelope.
WHY Alignment places the badge exactly where people expect it, without measuring.

```
6  Image(systemName: "envelope")
```

WHO SwiftUI, drawing the envelope symbol from Apple's SF Symbols set.
WHAT **The back layer: a large outline envelope that represents the mailbox.**
WHERE First child, so it is drawn first, at the back.
WHEN Created each time body runs, then sized and padded by the modifiers below.
WHY In a ZStack, earlier children are drawn behind later ones, so the envelope goes first.

```
7  .font(.system(size: 64))
```

WHO SwiftUI, which sizes the envelope symbol as if it were a 64-point character.
WHAT **Draws the envelope at a fixed 64-point size.**
WHERE Attached directly to the envelope Image, before its padding.
WHEN Applied whenever the envelope Image is built in body.
WHY A large icon leaves room for the badge to overlap its corner.

```
8  .padding(8)
```

WHO SwiftUI's layout system, which grows the envelope's frame by 8 points per side.
WHAT **Adds 8 points of empty space around the envelope symbol.**
WHERE Wraps the sized envelope, so the ZStack's corner sits 8 points outside the icon.
WHEN Measured during each layout pass, before the ZStack aligns the badge to its corner.
WHY The extra room lets the badge sit at the corner without hiding the icon.

9 `Text("3")`

WHO SwiftUI's Text view, holding the unread count as a string.
WHAT **The front layer: the number 3 that becomes the red badge.**
WHERE The last child of the ZStack, so it is drawn in front of the envelope.
WHEN Created in each evaluation of body, then wrapped by seven modifiers.
WHY Putting the badge last guarantees it is never hidden by the icon.

10 `.font(.caption)`

WHO SwiftUI and Dynamic Type, applying the small caption style to the count.
WHAT **Uses the small caption style for the number.**
WHERE The first of the seven modifiers that turn the plain 3 into a badge.
WHEN Applied whenever the badge Text is built, and rescaled with the text size setting.
WHY Badges are small so they do not cover the icon.

11 `.bold()`

WHO SwiftUI's text styling, which switches the count to a bold weight.
WHAT **Draws the 3 in bold so the small digit stays legible.**
WHERE The second modifier on the badge Text, after its caption font.
WHEN Applied each time the badge Text is created in body.
WHY Small text needs extra weight to stay readable.

12 `.foregroundStyle(.white)`

WHO SwiftUI's renderer, which draws the digit in white.
WHAT **Colors the count white so it stands out against the red added below.**
WHERE The third modifier on the badge Text; it colors only the digit, not the background.
WHEN Applied each time the badge is drawn, in light and dark mode alike.
WHY White on red is the familiar look of iOS badges, and the bold weight above helps it read at this small size.

13 `.padding(.horizontal, 6)`

WHO SwiftUI's layout system, adding space only along the horizontal axis.
WHAT **Adds 6 points on the left and right only.**
WHERE Wraps the styled digit, before the frame sets its minimum size.
WHEN Measured during each layout pass of the badge.
WHY Wide numbers get room on the sides without making the badge taller.

14 `.frame(minWidth: 24, minHeight: 24)`

WHO SwiftUI's layout system, which enforces the minimum size but lets the badge grow beyond it.
WHAT **Makes the badge at least 24 by 24 points but lets it grow wider.**
WHERE Wraps the horizontally padded digit, before the red background.
WHEN Measured during each layout pass, after the padding reports its size.
WHY Minimum sizes keep a single digit round while longer counts can stretch.

15 `.background(.red)`

WHO SwiftUI's renderer, which fills the badge's frame with red.
WHAT **Paints a red rectangle behind the framed digit, at least 24 by 24 points.**
WHERE Wraps the frame, so the red fills the whole minimum-size area.
WHEN Drawn each time the badge renders, before the capsule clip trims it.
WHY Red is the familiar color for unread counts on iOS.

16 `.clipShape(Capsule())`

WHO — SwiftUI's renderer, which masks the badge so only the capsule-shaped part stays visible.
WHAT — **Trims the red area to a capsule: a circle for one digit, a pill for more.**
WHERE — The outermost modifier on the badge, applied after the red background.
WHEN — Applied each time the badge is drawn, cutting off the red corners.
WHY — A capsule adapts to any width, unlike a circle.

21 `#Preview { InboxIconView() }`

WHO — Xcode's preview canvas, and the phone on this page, which render InboxIconView.
WHAT — **Creates an InboxIconView and shows the envelope with its red badge.**
WHERE — At file scope after InboxIconView; preview code never ships in the app.
WHEN — Each time you edit the file, for example after changing the ZStack alignment.
WHY — Layering is easiest to understand by moving things and looking.

Try it

1. Change "3" to "128": the capsule stretches into a pill.
2. Change .topTrailing to .bottomLeading.
3. Move the whole Text block (all its modifier lines) above the Image. Where they overlap, the envelope is now drawn over the badge, because later children draw on top.

CODE · LIVE SWIFTUI PREVIEW

Spacer and full-width frames

A workout card. A **Spacer** is a flexible empty view that takes all the free space along its stack, pushing neighbors apart. `.frame(maxWidth: .infinity)` lets a view grow as wide as its parent allows. Tap the button to start and pause the workout.

```swift
import SwiftUI

struct WorkoutCardView: View {
    @State private var isRunning = false
    var body: some View {
        VStack(spacing: 20) {
            HStack {
                Text("Upper body")
                Spacer()
                Text(isRunning ? "Running" : "45 min")
            }
            Button { isRunning.toggle() } label: {
                Text(isRunning ? "Pause" : "Start workout")
                    .frame(maxWidth: .infinity)
            }
            .buttonStyle(.borderedProminent)
        }
        .padding()
    }
}

#Preview { WorkoutCardView() }
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, which resolves Spacer, Button, frame and @State from SwiftUI.
WHAT	**Brings in Spacer, Button, the frame modifier and @State.**
WHERE	First line of the WorkoutCardView file; it applies to this file only.
WHEN	At compile time, before WorkoutCardView is type-checked.
WHY	Flexible layout tools are part of SwiftUI.

```
3  struct WorkoutCardView: View {
```

WHO	You declare WorkoutCardView; the preview creates it and SwiftUI keeps its state.
WHAT	**Declares a card that starts and pauses a workout.**
WHERE	At file scope, so a workout list could show one card per session.
WHEN	Compiled once; SwiftUI keeps isRunning even when it rebuilds the struct value.
WHY	Real cards often need one row pushed apart and one button stretched wide.

```
4  @State private var isRunning = false
```

WHO	SwiftUI, which stores isRunning for this card outside the struct value.
WHAT	**Remembers whether the workout is running, starting as false.**
WHERE	Storage managed by SwiftUI for this view.
WHEN	Created when the card appears; changed by the button.
WHY	One Boolean drives both the status text and the button title.

```
5  var body: some View {
```

WHO	SwiftUI calls WorkoutCardView's body after each change to isRunning.
WHAT	**Describes the card for the current value of isRunning.**
WHERE	A computed property inside WorkoutCardView, evaluated on the main thread.
WHEN	When the card appears and after each tap.
WHY	Both labels are computed from the same state.

```
6  VStack(spacing: 20) {
```

WHO	SwiftUI's layout system, which stacks the info row above the full-width button.
WHAT	**Stacks the info row above the button, 20 points apart.**
WHERE	The root view returned by body, wrapped by the padding at the end.
WHEN	Laid out after each tap, because the status and the title may change width.
WHY	Twenty points of space separate the information from the action that changes it.

```
7  HStack {
```

WHO	SwiftUI's layout system, which arranges the name, the Spacer and the status in a row.
WHAT	**A row holding the workout name, a Spacer and the status.**
WHERE	The first child of the VStack, above the button.
WHEN	Laid out after body runs, with the Spacer sized last.
WHY	A name on the left and a value on the right is a standard row pattern.

```
8  Text("Upper body")
```

WHO	SwiftUI's Text view, holding the workout name as a fixed string.
WHAT	**Shows the name of the workout, "Upper body".**
WHERE	The leading child of the HStack, at the left edge in English.
WHEN	Created in each evaluation of body; its string never changes.
WHY	Fixed labels usually sit at the leading edge, where people start reading a row.

9 `Spacer()`

WHO SwiftUI's layout system, which gives a Spacer whatever width the other children leave over.
WHAT **An invisible view that takes all the leftover width in the row.**
WHERE The middle child of the HStack, between the name and the status.
WHEN Sized during each layout pass, after the two texts have taken their widths.
WHY It pushes the status to the trailing edge on every screen width, with no measuring.

10 `Text(isRunning ? "Running" : "45 min")`

WHO SwiftUI draws it; the ternary operator picks the string.
WHAT **Shows Running while the workout is on, otherwise the planned length.**
WHERE The trailing child of the HStack, pushed to the right edge by the Spacer.
WHEN Rebuilt every time body runs after isRunning changes.
WHY The status is derived from state, so it can never disagree with the button.

12 `Button { isRunning.toggle() } label: {`

WHO You declare the button; the person using the app flips the workout by tapping it.
WHAT **A button whose action flips isRunning between true and false, followed by a custom label.**
WHERE The second child of the VStack, below the info row.
WHEN Its action runs on the main thread at each tap, flipping isRunning.
WHY The label form is needed so the label itself can stretch to full width.

13 `Text(isRunning ? "Pause" : "Start workout")`

WHO SwiftUI's Text view, with the ternary operator choosing its string.
WHAT **The button title, which changes with the state.**
WHERE Inside the Button's label closure, so it is the button's visible title.
WHEN Rebuilt whenever body runs after isRunning changes.
WHY One button can offer the action that makes sense right now.

14 `.frame(maxWidth: .infinity)`

WHO SwiftUI's layout system, which lets the label accept all the width it is offered.
WHAT **Lets the label grow to the full width the button is offered.**
WHERE Wraps the title Text inside the Button's label closure, not the Button itself.
WHEN Measured during each layout pass, before the button style draws its background.
WHY The button style paints behind its label, so a wide label makes a wide button that is easy to hit.

16 `.buttonStyle(.borderedProminent)`

WHO SwiftUI, drawing the system's filled, tinted button appearance.
WHAT **Fills the whole wide button with the tint color and white text.**
WHERE Attached to the Button after its label closure closes.
WHEN Applied each time body creates the Start workout button.
WHY The main action on the card should stand out.

18 `.padding()`

WHO SwiftUI's layout system, adding the standard margin on every side.
WHAT **Adds standard space around the card, so the wide button stops short of the screen edges.**
WHERE Wraps the outer VStack, the last modifier in body.
WHEN Measured during each layout pass, after the VStack takes the full width.
WHY Full width still needs margins from the screen edges.

```
22  #Preview { WorkoutCardView() }
```

WHO	Xcode's preview canvas, and the phone on this page, which run WorkoutCardView.
WHAT	**Creates a WorkoutCardView whose Start workout button really flips the state.**
WHERE	At file scope after WorkoutCardView; this code is not part of the shipped app.
WHEN	Each time you edit the file; the preview restarts with isRunning set to false.
WHY	You can tap Start workout and watch both labels update.

> **Try it**
>
> 1. Delete Spacer(). The status slides over next to the name.
> 2. Delete .frame(maxWidth: .infinity). The button shrinks to fit its title.
> 3. Add a second Spacer() before Text("Upper body"). The two spacers split the free space.

INTERACTIVE

Layout lab

Choose a stack type, alignment, spacing and padding, add a Spacer, or stretch a child with `.frame(maxWidth: .infinity)`. The lab writes the matching SwiftUI code and renders the result. Watch which views hug their content and which take the leftover space, and predict each change before you make it.

This step is an interactive simulation in the HTML version.

CODE · LIVE SWIFTUI PREVIEW

Scrolling grids with LazyVGrid

A photo library grid. **ScrollView** lets content taller than the screen scroll. **LazyVGrid** arranges items in columns, and *lazy* means it creates each cell only when it is about to scroll into view, so long collections stay fast. For a small table that never scrolls, SwiftUI also offers `Grid` with `GridRow`.

```swift
import SwiftUI

struct PhotoGridView: View {
    let columns = [GridItem(.flexible()), GridItem(.flexible())]

    var body: some View {
        ScrollView {
            LazyVGrid(columns: columns, spacing: 12) {
                ForEach(1..<41) { number in
                    Text("Photo \(number)")
                        .frame(maxWidth: .infinity, minHeight: 100)
                        .background(.teal)
                        .clipShape(RoundedRectangle(cornerRadius: 10))
                }
            }
            .padding()
        }
    }
}

#Preview { PhotoGridView() }
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, which resolves ScrollView, LazyVGrid, GridItem and ForEach from SwiftUI.
WHAT	**Brings in ScrollView, LazyVGrid, GridItem and ForEach.**
WHERE	First line of the PhotoGridView file; it applies to this file only.
WHEN	At compile time, before the grid and its columns are type-checked.
WHY	Scrolling and grid layout are SwiftUI containers.

```
3  struct PhotoGridView: View {
```

WHO	You declare PhotoGridView; the preview creates it and SwiftUI draws the grid.
WHAT	**Declares a scrolling grid of photo placeholders.**
WHERE	At file scope, so a gallery screen could embed this grid.
WHEN	A value is created when the preview starts; the type is compiled once, at build time.
WHY	Photo grids are the classic case for lazy, scrolling layout.

```
4  let columns = [GridItem(.flexible()), GridItem(.flexible())]
```

WHO	Your view defines it; LazyVGrid reads it.
WHAT	**Describes two columns that share the available width equally.**
WHERE	A stored constant of PhotoGridView, outside body so the grid can read it by name.
WHEN	Set each time the view value is created.
WHY	Adding or removing a GridItem is all it takes to change the column count.

```
6  var body: some View {
```

WHO	SwiftUI calls PhotoGridView's body to get the scrolling grid.
WHAT	**Returns a ScrollView containing a two-column grid of 40 numbered cells.**
WHERE	A computed property inside PhotoGridView, run on the main thread.
WHEN	When the grid first appears; scrolling alone does not run body again.
WHY	The whole gallery is described in one place.

```
7  ScrollView {
```

WHO	SwiftUI, together with iOS touch handling.
WHAT	**Lets its content scroll vertically when it is taller than the screen.**
WHERE	The root view returned by body, filling the whole screen.
WHEN	Active whenever the user drags on the grid, moving the content up or down.
WHY	Without a ScrollView, content that does not fit is simply cut off.

```
8  LazyVGrid(columns: columns, spacing: 12) {
```

WHO	SwiftUI's layout system, which places cells left to right, then wraps to the next row.
WHAT	**Arranges its children into the two columns, with 12 points between rows.**
WHERE	Inside the ScrollView, wrapped by the padding below its closing brace.
WHEN	Creates cells only as they approach the visible area.
WHY	Lazy creation keeps memory and work low even with thousands of items.

```
9  ForEach(1..<41) { number in
```

WHO	SwiftUI's ForEach, which produces one view for each number in the range.
WHAT	**Produces one cell for each number from 1 to 40.**
WHERE	Inside the LazyVGrid, which asks it for cells as they scroll into view.
WHEN	Each cell's closure runs when the grid needs that cell.
WHY	ForEach turns a collection into views, so the grid has no hand-written cells.

```
10  Text("Photo \(number)")
```

WHO — SwiftUI's Text view, with Swift interpolating the cell's number into the string.
WHAT — **Labels each cell with its number, for example "Photo 7".**
WHERE — Inside the ForEach closure: the content of one grid cell.
WHEN — Created the first time the grid needs that cell, as it nears the visible area.
WHY — A placeholder stands in for a real image, which a later module loads.

```
11  .frame(maxWidth: .infinity, minHeight: 100)
```

WHO — SwiftUI's layout system, which stretches the cell to fill its column.
WHAT — **Stretches the cell to its column's full width and at least 100 points tall.**
WHERE — Wraps the cell's Text, before the teal background.
WHEN — Measured when the cell is laid out, using the column width the grid offers.
WHY — Cells fill their column so the grid looks even regardless of text length.

```
12  .background(.teal)
```

WHO — SwiftUI's renderer, which fills the stretched cell with teal.
WHAT — **Paints the whole cell teal, not just the area behind its letters.**
WHERE — Wraps the frame, so the color fills the whole cell.
WHEN — Drawn each time the cell is rendered on screen.
WHY — Background after frame colors the full cell, not just the text.

```
13  .clipShape(RoundedRectangle(cornerRadius: 10))
```

WHO — SwiftUI's renderer, which trims each cell to a rounded rectangle.
WHAT — **Rounds the teal cell's corners with a 10-point radius.**
WHERE — The outermost modifier on each cell, after the teal background.
WHEN — Applied each time a cell is drawn, including cells that scroll back into view.
WHY — Rounded corners match the look of the iOS Photos app.

```
16  .padding()
```

WHO — SwiftUI's layout system, adding the standard margin around the grid.
WHAT — **Adds margins around the grid inside the scroll area.**
WHERE — Wraps the LazyVGrid, inside the ScrollView.
WHEN — Measured during each layout pass of the scrolling content.
WHY — Padding inside the ScrollView scrolls with the content, so edges stay neat.

```
21  #Preview { PhotoGridView() }
```

WHO — Xcode's preview canvas, and the phone on this page, which render PhotoGridView.
WHAT — **Creates a PhotoGridView whose 40 cells you can scroll through in the preview.**
WHERE — At file scope after PhotoGridView; preview code does not ship in the app.
WHEN — Each time you edit the file, for example after adding a third GridItem.
WHY — Scrolling behavior is best checked by actually scrolling.

Try it

1. Add a third GridItem(.flexible()) to columns.
2. Change minHeight: 100 to minHeight: 180 and scroll through the taller cells.
3. Change 1..<41 to 1..<201. The lazy grid still scrolls smoothly because it builds only the cells near the screen.

CHECK YOURSELF

Check your understanding

1. A full-width row contains `HStack { Text("Total"); Spacer(); Text("$42") }`. Where do the two texts appear?
   A. Both texts centered together in the middle of the row
   B. Total at the leading edge, $42 at the trailing edge
   C. $42 first, then Total, because Spacer reverses the order
   D. Total, then a visible gray box, then $42

2. `Text("Save").background(.blue).frame(maxWidth: .infinity)` shows blue only behind the word, not across the screen. Why?
   A. The blue wraps only the text; the frame adds clear width outside it
   B. .infinity is not a valid maxWidth, so the frame is ignored and nothing grows
   C. Text views can never be wider than the words they contain, whatever you do
   D. A background is ignored whenever maxWidth is set on the same view

3. You need a scrolling gallery of 500 photos in three columns. What should you use?
   A. A VStack holding 167 HStacks with three Images in each
   B. A Grid with 167 GridRows, each holding three Image views
   C. An HStack with three Spacers and a ForEach over all 500 photos
   D. A ScrollView holding a LazyVGrid with three flexible columns

4. In `ZStack { Color.blue; Text("Hi") }`, which view is drawn on top?
   A. The blue color, because larger children are drawn on top
   B. Neither; a ZStack shows only its first child and hides the rest
   C. The text, because later children are drawn on top
   D. They sit side by side, because ZStack lays out horizontally

5. In SwiftUI's layout process, what happens right after a parent proposes a size to a child?
   A. The parent forces the child to take exactly that size
   B. The child chooses its own size, and the parent respects that choice
   C. The child is drawn on screen immediately
   D. The parent removes any child that does not fit

Answers: Stacks, Frames, and Grids

1. **B. Total at the leading edge, $42 at the trailing edge** Spacer takes all the free width between its neighbors, pushing Total to the leading edge and $42 to the trailing edge. Without the Spacer, the HStack would hug both texts and sit centered. A Spacer is invisible; it only occupies space.
2. **A. The blue wraps only the text; the frame adds clear width outside it** Modifier order again: the background colors only the view it wraps, which is still text-sized. The frame comes later and grows invisibly. Swap the two lines so the frame comes first and the blue spans the full width.
3. **D. A ScrollView holding a LazyVGrid with three flexible columns** LazyVGrid creates cells only as they approach the screen, and ScrollView makes the content scrollable. A Grid or nested stacks would build all 500 cells up front and cannot scroll by themselves. Grid suits small fixed tables, not long collections.
4. **C. The text, because later children are drawn on top** A ZStack draws its children back to front in the order you write them, so the last child is on top. Size does not decide layering, which is why the large blue color works as a background here instead of covering the text.
5. **B. The child chooses its own size, and the parent respects that choice** The proposal is only an offer. Each child decides its own size: Text hugs its words, while shapes and colors take everything offered. The parent then places the child using its alignment. A parent cannot force a size, which is why tools like frame exist.

State Drives the Screen

CONCEPT

One source of truth

State is any data that can change while the app runs and that the screen depends on: a count, a switch, the text in a search field. SwiftUI asks you to keep each piece of state in exactly one place, its **source of truth**, and to let every view read it from there.

The screen then follows a loop you never write by hand. Something happens, your code changes state, SwiftUI re-runs `body` for the views that read that state, compares the new description with the old one, and updates only what differs. This is **reactive programming**: the interface reacts to data instead of being told what to do.

Event	A tap, a keystroke, a timer firing, or data arriving from the network.
State change	Your code responds by writing a new value. That is its whole job.
Recompute	SwiftUI calls body again on every view that read the changed value.
Update	SwiftUI compares the old and new view trees and changes only what differs on screen.

> **A useful rule**
>
> If you catch yourself writing code whose job is to “update the label”, stop. Change the data instead, and let the label read the data.

CODE · LIVE SWIFTUI PREVIEW

Your first @State: a water tracker

A view that remembers how many glasses of water you drank today. Tap the button in the phone and watch the number change. Nothing in this code ever says “update the label”: changing `glasses` is enough, because SwiftUI re-runs `body` and redraws what changed.

```swift
import SwiftUI

struct WaterTrackerView: View {
    @State private var glasses = 0

    var body: some View {
        VStack(spacing: 16) {
            Text("\(glasses) glasses")
                .font(.largeTitle)
            Button("Drink a glass") {
                glasses += 1
            }
            .buttonStyle(.borderedProminent)
        }
    }
}

#Preview { WaterTrackerView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds your app.
- **WHAT** **Makes SwiftUI types such as View, Text, Button, VStack and the @State property wrapper available in this file.**
- **WHERE** Top of the file; an import applies to this one source file only.
- **WHEN** At compile time, before any of your code runs.
- **WHY** SwiftUI is a framework, not part of the Swift language, so every file that builds interface code imports it.

3 `struct WaterTrackerView: View {`

- **WHO** Your code declares the type; SwiftUI creates and uses its values.
- **WHAT** **Declares a structure named WaterTrackerView that promises to conform to the View protocol, so it must provide a body.**
- **WHERE** At file scope, so any other view in the app can create one.
- **WHEN** Compiled once; SwiftUI creates fresh WaterTrackerView values whenever a parent view recomputes its own body.
- **WHY** Views are lightweight structs (value types), so creating them over and over is cheap and they cannot be shared and mutated by accident.

4 `@State private var glasses = 0`

- **WHO** SwiftUI owns the storage; this view only reads and writes it.
- **WHAT** **Creates a piece of state named glasses that starts at 0 and survives even when SwiftUI throws away and rebuilds the struct.**
- **WHERE** Stored by SwiftUI outside the struct, attached to this view's position in the view tree, on the main thread.
- **WHEN** Initialized once, the first time this view appears on screen; later rebuilds reuse the stored value.
- **WHY** A plain var in a struct would reset to 0 every time the struct is recreated. private signals that only this view may change it, so it stays the single source of truth.

```
6  var body: some View {
```

WHO — SwiftUI calls this computed property; you never call it yourself.

WHAT — **Describes what the screen should look like for the current value of glasses. some View means "one specific view type the compiler works out".**

WHERE — Inside WaterTrackerView; it runs on the main thread.

WHEN — When the view first appears, and again every time a value it reads (here, glasses) changes.

WHY — body is a function of state: given the same state it always produces the same description, which makes the screen predictable.

```
7  VStack(spacing: 16) {
```

WHO — SwiftUI's layout system, which sizes and positions the count and the button.

WHAT — **Stacks its children vertically with 16 points of space between each one.**

WHERE — The root of the description that body returns.

WHEN — Laid out whenever body produces a new description.

WHY — Stacks are the basic building block of SwiftUI layout; spacing keeps the number and button from crowding each other.

```
8  Text("\(glasses) glasses")
```

WHO — SwiftUI draws the text; Swift builds the string.

WHAT — **Shows the current count using string interpolation, for example "3 glasses".**

WHERE — First child of the VStack, so it sits on top.

WHEN — Re-created with the new number every time body runs after glasses changes.

WHY — The label reads state directly, so it can never show a stale number: there is no separate copy to keep in sync.

```
9  .font(.largeTitle)
```

WHO — SwiftUI and Dynamic Type, the system text-size setting.

WHAT — **Uses the large title text style for the count.**

WHERE — Modifies only the Text directly above it.

WHEN — Applied every time body runs and re-creates this Text with the new count.

WHY — Text styles scale with the reader's preferred text size, unlike a fixed point size.

```
10 Button("Drink a glass") {
```

WHO — You declare it; the user triggers it by tapping.

WHAT — **Creates a button with a title and a closure (a block of code) to run when tapped.**

WHERE — Second child of the VStack, below the count.

WHEN — The closure runs on the main thread, only at the moment the user taps.

WHY — Buttons describe intent. The action changes data; it never touches the label directly.

```
11 glasses += 1
```

WHO — Your action closure writes; SwiftUI notices the write.

WHAT — **Adds one to glasses, so the count goes 0, 1, 2 and so on with each tap.**

WHERE — Inside the Button's action closure, the only place in this view that writes to glasses.

WHEN — On each tap. SwiftUI then schedules body to run again before the next frame is drawn.

WHY — Changing state is the only thing you do. SwiftUI handles the redraw, which removes a whole class of "forgot to update the screen" bugs.

```
13 .buttonStyle(.borderedProminent)
```

WHO — SwiftUI, using the system button appearance.

WHAT — **Gives the button a filled, tinted background so it reads as the main action.**

WHERE — Attached to the Drink a glass Button, so it styles that one button only.

WHEN — Applied each time body runs and SwiftUI rebuilds the button's description.

WHY — One prominent button per screen tells people what to do next.

```
18  #Preview { WaterTrackerView() }
```

WHO	Xcode's preview canvas (and this page's simulated iPhone).
WHAT	**Registers a preview that creates a WaterTrackerView and shows it live.**
WHERE	At file scope; preview code is not part of the shipped app.
WHEN	Each time you edit the file in Xcode, or edit the code on this page.
WHY	Previews let you see and use a view without building and launching the whole app.

> **Try it**
>
> 1. Change glasses += 1 to glasses += 2. How does the count move now?
> 2. Add a second Button below the first that sets glasses = 0 and name it Reset.

INTERACTIVE

Watch the loop run

On the left is a small screen with a counter and a switch. Tap them and follow the highlight through the diagram: the event, the state it changes, the `body` SwiftUI re-runs, the new view tree, and the small difference that reaches the screen. Then turn on **bug mode**, where the same number is stored twice. Watch the two copies drift apart, and notice that a screen can only be as correct as its single source of truth.

This step is an interactive simulation in the HTML version.

CODE · LIVE SWIFTUI PREVIEW

Derive, do not duplicate: a tip calculator

A tip calculator shows a bill, a tip percentage, a tip amount and a total. Only the first two are state. The tip and the total are **derived values**: computed properties that recalculate from state every time `body` runs, so they can never disagree with it. Drag both sliders in the phone.

```swift
import SwiftUI

struct TipCalculatorView: View {
    @State private var bill = 48.0
    @State private var tipPercent = 18.0

    var tip: Double { bill * tipPercent / 100 }
    var total: Double { bill + tip }

    var body: some View {
        VStack(spacing: 12) {
            Text("Bill: " + String(format: "$%.2f", bill))
            Slider(value: $bill, in: 0...200, step: 1)
            Text("Tip: \(Int(tipPercent))%")
            Slider(value: $tipPercent, in: 0...30, step: 1)
            Text("Total: " + String(format: "$%.2f", total))
                .font(.title)
        }
        .padding()
    }
}
#Preview { TipCalculatorView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which reads the import before it checks any of the calculator code below.
- **WHAT** **Brings in Slider, Text, VStack and @State, everything this calculator screen uses.**
- **WHERE** Top of the file, applying to the whole file.
- **WHEN** At compile time, once per build; an import costs nothing while the app is running.
- **WHY** The Swift language knows nothing about sliders or text; the SwiftUI framework supplies them.

3 `struct TipCalculatorView: View {`

- **WHO** Your code defines it; SwiftUI creates it and asks it for a body.
- **WHAT** **Declares the calculator screen as a struct that conforms to View.**
- **WHERE** File scope, so an app or a preview can create it.
- **WHEN** Compiled once; new values of it are created whenever a parent rebuilds.
- **WHY** A whole screen is just another view. There is no special screen type; small views compose into bigger ones.

4 `@State private var bill = 48.0`

- **WHO** SwiftUI stores the value; the first slider and the labels use it.
- **WHAT** **Holds the bill amount as a Double (a decimal number), starting at 48.0.**
- **WHERE** In storage SwiftUI keeps for this view, outside the struct itself.
- **WHEN** Created when the view first appears; changed every time the bill slider moves.
- **WHY** The bill is input the user controls, so it is state. Writing 48.0 rather than 48 makes Swift infer Double, which Slider requires.

```
5  @State private var tipPercent = 18.0
```

WHO — SwiftUI owns it; the second slider writes it.

WHAT — **Holds the tip percentage, starting at 18.**

WHERE — Next to bill in this view's state storage.

WHEN — Changes while the user drags the tip slider.

WHY — Also user input, so also state. The bill and the percentage are the only two facts this screen truly needs to remember.

```
7  var tip: Double { bill * tipPercent / 100 }
```

WHO — Swift evaluates it whenever body asks for it.

WHAT — **A computed property: works out the tip amount from bill and tipPercent each time it is read.**

WHERE — Inside the struct, but it stores nothing; it is a small function that looks like a property.

WHEN — Every time it is read, which happens during each body run.

WHY — Because the tip is calculated on demand, there is no stored tip that could fall out of date.

```
8  var total: Double { bill + tip }
```

WHO — Swift evaluates this getter each time body asks for total; nothing stores the result.

WHAT — **Adds the bill and the tip. A computed property can use another computed property.**

WHERE — Inside TipCalculatorView, right below tip. It is code, not storage, so it adds nothing to the view's state.

WHEN — Recalculated on every read, which happens each time body builds the Total label.

WHY — Derived values can build on each other: two pieces of state, any number of values computed from them.

```
10 var body: some View {
```

WHO — SwiftUI calls body; you never call it yourself, and you never ask it to refresh.

WHAT — **Describes the calculator for the current bill and tip percentage.**

WHERE — Inside TipCalculatorView, on the main thread.

WHEN — When the view first appears and after every slider movement that changes state.

WHY — The screen is a function of state, so dragging quickly can never leave it half updated.

```
11 VStack(spacing: 12) {
```

WHO — SwiftUI's layout system, which arranges the three labels and two sliders in a column.

WHAT — **Stacks the labels and sliders in one column, 12 points apart.**

WHERE — The root view that body returns; every label and slider on the screen is one of its children.

WHEN — Laid out again after each recompute, so a moved slider never leaves the column misaligned.

WHY — A single column keeps each label directly above the control it describes.

```
12 Text("Bill: " + String(format: "$%.2f", bill))
```

WHO — Foundation formats the number; SwiftUI draws the text.

WHAT — **Shows the bill as money with exactly two decimals, such as "Bill: $48.00".**

WHERE — First child of the VStack, directly above the bill slider it describes.

WHEN — Rebuilt each time body runs, which is every time the bill slider changes bill.

WHY — The format "%.2f" means two digits after the point, which avoids showing 48.0 or long decimals like 8.640000001.

```
13 Slider(value: $bill, in: 0...200, step: 1)
```

WHO — SwiftUI draws the slider; the user drags it.

WHAT — **Lets the user pick a bill from 0 to 200 in whole-dollar steps. $bill hands the slider a binding so it can write the value back.**

WHERE — Second child of the VStack, directly under the Bill label it controls.

WHEN — Writes to bill continuously while the finger moves.

WHY — The $ prefix gives the slider read and write access to the one stored value instead of a copy, so slider and label always agree.

```
14 Text("Tip: \(Int(tipPercent))%")
```

WHO Swift converts and interpolates the number; SwiftUI draws the resulting Tip label.
WHAT **Shows the percentage as a whole number, like "Tip: 18%".**
WHERE Third child of the VStack, between the bill slider and the tip slider.
WHEN Rebuilt each time body runs after the tip slider writes a new tipPercent.
WHY Converting to Int hides the meaningless ".0" that a Double would print.

```
15 Slider(value: $tipPercent, in: 0...30, step: 1)
```

WHO SwiftUI draws the slider; the user's finger drags it, and the binding writes the result.
WHAT **Lets the user choose a tip from 0 to 30 percent.**
WHERE Fourth child of the VStack, directly below the Tip label it controls.
WHEN Writes tipPercent continuously while the user drags, one whole percent per step.
WHY A bounded range keeps the input valid, so no other code has to guard against a negative tip.

```
16 Text("Total: " + String(format: "$%.2f", total))
```

WHO SwiftUI draws it; reading total runs the computed property, which reads both bill and tipPercent.
WHAT **Shows the total, for example "Total: $56.64".**
WHERE Last child of the VStack, under both sliders, where the eye lands last.
WHEN Rebuilt whenever either slider moves, because total depends on both pieces of state.
WHY The label reads the derived value, so it is right after every change without a single line of update code.

```
17 .font(.title)
```

WHO SwiftUI, applying the title text style, which Dynamic Type scales with the reader's text-size setting.
WHAT **Makes the total the largest text on the screen.**
WHERE Attached to the Total Text only; the other labels keep the default body style.
WHEN Applied each time body runs and rebuilds the Total label with a new amount.
WHY Visual hierarchy: the answer the user came for should be the easiest thing to find.

```
19 .padding()
```

WHO SwiftUI's layout system, which insets the whole VStack by the standard system margin.
WHAT **Adds the system's standard space around the whole stack.**
WHERE Applied to the VStack, outside all of its children.
WHEN Applied during every layout pass, after the VStack has worked out its own size.
WHY Without it the sliders run edge to edge, where they are hard to grab next to the screen border.

```
22 #Preview { TipCalculatorView() }
```

WHO The preview canvas and this page's simulated iPhone.
WHAT **Shows a live, draggable TipCalculatorView.**
WHERE File scope; previews are left out of the shipped app.
WHEN Re-rendered each time you edit the code, in Xcode or in this page's editor.
WHY You can test every slider position in seconds without building the app.

> **Try it**
>
> 1. Drag the tip slider to 25 and watch the total. No line of code ever assigns to total.
> 2. Add Text("Tip: " + String(format: "$%.2f", tip)) just above the total.
> 3. Add @State private var people = 1.0, a Slider for it from 1 to 8, and show the total divided by people.

COMPARE

The duplicated-state bug

Both screens show a total with an 18% tip. The left one stores the total as a second piece of state and refreshes it from a button. Drag its slider without tapping Recalculate and the total is wrong. The right one derives the total, so there is nothing to forget.

A: Bug: total stored as state

```swift
import SwiftUI

struct StaleTotalView: View {
    @State private var bill = 48.0
    @State private var total = 56.64
    var body: some View {
        VStack(spacing: 12) {
            Slider(value: $bill, in: 0...200, step: 1)
            Text("Bill: " + String(format: "$%.2f", bill))
            Text("Total: " + String(format: "$%.2f", total))
            Button("Recalculate") { total = bill * 1.18 }
        }
        .padding()
    }
}

#Preview { StaleTotalView() }
```

1 `import SwiftUI`

- WHO — The Swift compiler, which needs SwiftUI's types before it can check StaleTotalView.
- WHAT — **Brings in Slider, Button, Text and @State for this buggy version.**
- WHERE — First line of the buggy version's file, covering everything below it.
- WHEN — At compile time. The bug in this file compiles cleanly; it only shows up while the app runs.
- WHY — Both versions use the same building blocks; only the data design differs.

3 `struct StaleTotalView: View {`

- WHO — Your code declares the struct; the preview creates the value you see in the phone.
- WHAT — **Declares a screen that will show a stale (out of date) total.**
- WHERE — At file scope, as a separate screen so you can compare it with the fixed one.
- WHEN — Created when the preview starts, and created again whenever the code changes.
- WHY — Seeing the bug happen is the fastest way to recognize it in your own code later.

4 `@State private var bill = 48.0`

- WHO — SwiftUI stores it; the slider writes it.
- WHAT — **The bill amount, starting at 48 dollars.**
- WHERE — In SwiftUI-managed storage tied to StaleTotalView, outside the struct itself.
- WHEN — Starts at 48.0 when the view appears, then changes on every slider movement.
- WHY — This part is fine: the bill is real input and belongs in state.

5 `@State private var total = 56.64`

- WHO — SwiftUI stores total in its own slot, exactly as it stores bill, with no link between the two.
- WHAT — **Stores the total as its own piece of state, pre-filled with 48 plus 18 percent.**
- WHERE — A separate storage slot, independent of bill.
- WHEN — Changes only when some code assigns to it, which here is only the button.
- WHY — This is the bug: a second source of truth for a fact that depends on the first. Nothing ties it to bill.

```
6  var body: some View {
```

WHO — SwiftUI calls body and rebuilds the screen from whatever bill and total hold right now.
WHAT — **Describes the screen from both stored values.**
WHERE — Inside StaleTotalView, just below the two @State properties it reads.
WHEN — After every change to bill (each drag) or to total (each Recalculate tap).
WHY — body faithfully shows whatever is stored, correct or not.

```
7  VStack(spacing: 12) {
```

WHO — SwiftUI's layout system, stacking the slider, the two labels and the button.
WHAT — **Stacks the slider, labels and button in one column.**
WHERE — The root view returned by StaleTotalView's body.
WHEN — Laid out after each recompute, whether bill or total triggered it.
WHY — Same layout as the fixed version, so the only difference you see is the data bug.

```
8  Slider(value: $bill, in: 0...200, step: 1)
```

WHO — The user dragging; SwiftUI writing through the binding.
WHAT — **Writes the new bill through the $bill binding as the user drags; it never touches total.**
WHERE — First child of the VStack, at the top of the buggy screen.
WHEN — Continuously during the drag, once for every one-dollar step.
WHY — It updates bill only. It has no idea that total exists, so total stays where it was.

```
9  Text("Bill: " + String(format: "$%.2f", bill))
```

WHO — SwiftUI, reading bill straight from the source of truth.
WHAT — **Shows the current bill, which is always correct.**
WHERE — Second child of the VStack, directly below the slider.
WHEN — Rebuilt each time body runs, which happens on every bill change.
WHY — It reads the source of truth directly, so it can never be stale.

```
10 Text("Total: " + String(format: "$%.2f", total))
```

WHO — SwiftUI, reading the duplicated total rather than computing it from bill.
WHAT — **Shows the stored total, which is wrong whenever the bill moved since the last tap.**
WHERE — Third child of the VStack, right under the always-correct Bill label.
WHEN — Rebuilt whenever total changes, which is rarely.
WHY — The screen is only as correct as the data it reads; here that data can be out of date.

```
11 Button("Recalculate") { total = bill * 1.18 }
```

WHO — The user taps the button; your closure then copies a fresh result into total.
WHAT — **Overwrites total with bill times 1.18, the only moment the two values are guaranteed to agree.**
WHERE — Last child of the VStack, below the stale Total label.
WHEN — Only when tapped. Every other path that changes bill skips it.
WHY — Manual syncing like this is easy to forget as an app grows: a reset button or loaded data would each need the same line.

```
13 .padding()
```

WHO — SwiftUI's layout system, adding margin around the buggy screen's column.
WHAT — **Adds the standard system margin on all four sides of the VStack.**
WHERE — Attached to the VStack, so the space surrounds all four of its children.
WHEN — Applied during every layout pass of StaleTotalView.
WHY — Keeps the slider away from the screen edges.

```
17 #Preview { StaleTotalView() }
```

WHO Xcode's preview canvas, and the simulated iPhone on this page for the buggy version.
WHAT **Shows the buggy screen so you can reproduce the stale total.**
WHERE At file scope; preview code is stripped from the shipped app.
WHEN Each time the code is edited, so you can repeat the drag-then-read experiment.
WHY Reproducing a bug on purpose is how you learn to spot its shape.

B: Fix: total derived from bill

```
1  import SwiftUI
2
3  struct LiveTotalView: View {
4      @State private var bill = 48.0
5      var total: Double { bill * 1.18 }
6      var body: some View {
7          VStack(spacing: 12) {
8              Slider(value: $bill, in: 0...200, step: 1)
9              Text("Bill: " + String(format: "$%.2f", bill))
10             Text("Total: " + String(format: "$%.2f", total))
11         }
12         .padding()
13     }
14 }
15
16 #Preview { LiveTotalView() }
```

```
1 import SwiftUI
```

WHO The Swift compiler, loading the same SwiftUI framework the buggy version used.
WHAT **Brings in Slider, Text and @State for the fixed version.**
WHERE First line of the fixed version's file, covering LiveTotalView and its preview.
WHEN At compile time, before LiveTotalView is type-checked.
WHY The fix needs no new tools, only a different data design.

```
3 struct LiveTotalView: View {
```

WHO Your code declares the fixed screen; the preview creates and displays it.
WHAT **Declares LiveTotalView, the corrected screen that stores only the bill.**
WHERE At file scope, side by side with the buggy StaleTotalView.
WHEN Created when its preview starts and created again after each code edit.
WHY Same screen, one fewer stored value: removing the duplicate removes the bug along with it.

```
4 @State private var bill = 48.0
```

WHO SwiftUI stores bill; the slider is the only thing that writes it.
WHAT **The bill, now the only stored value on this screen.**
WHERE In SwiftUI-managed storage attached to LiveTotalView.
WHEN Starts at 48.0 and changes on every movement of the slider.
WHY One fact, one place: the single source of truth.

```
5 var total: Double { bill * 1.18 }
```

WHO Swift runs this getter whenever body reads total; nothing is cached between reads.
WHAT **Computes the total from bill, adding 18 percent.**
WHERE Inside LiveTotalView, as a computed property that adds no storage of its own.
WHEN On every read, so after every bill change.
WHY The total is now a view of the bill rather than a copy, so it cannot drift.

```
6  var body: some View {
```

WHO	SwiftUI calls body to describe the fixed screen from bill alone.
WHAT	**Describes the screen from bill and the derived total.**
WHERE	Inside LiveTotalView, directly below the derived total it reads.
WHEN	After every bill change; no other state exists that could trigger it.
WHY	One state change produces a fully consistent screen.

```
7  VStack(spacing: 12) {
```

WHO	SwiftUI's layout system, stacking the fixed screen's three views.
WHAT	**Stacks the slider and the Bill and Total labels in one column, 12 points apart.**
WHERE	The root view returned by LiveTotalView's body.
WHEN	Laid out again each time a bill change re-runs body.
WHY	Identical layout to the buggy version, minus the button.

```
8  Slider(value: $bill, in: 0...200, step: 1)
```

WHO	The user drags the slider; SwiftUI writes each new value through the binding.
WHAT	**Writes the new bill through $bill, the one and only stored value on this screen.**
WHERE	First child of the VStack, at the top of the fixed screen.
WHEN	Continuously during the drag, one dollar per step of the slider.
WHY	Writing bill is now enough to update everything that depends on it.

```
9  Text("Bill: " + String(format: "$%.2f", bill))
```

WHO	SwiftUI, reading bill directly from state on each body run.
WHAT	**Shows the current bill with two decimals, for example Bill: $63.00 after a drag.**
WHERE	Second child of the VStack, directly under the slider that changes it.
WHEN	Rebuilt on each body run, so with every one-dollar step of the slider.
WHY	It reads the source of truth directly, exactly as in the buggy version; this label was never the problem.

```
10 Text("Total: " + String(format: "$%.2f", total))
```

WHO	SwiftUI, reading total, which in turn reads bill through the computed property.
WHAT	**Shows a total that is correct at every slider position.**
WHERE	Third and last child of the VStack, where the Recalculate button used to follow.
WHEN	Rebuilt with every bill change, because total reads bill.
WHY	No Recalculate button is needed, so no user or programmer can forget to press it.

```
12 .padding()
```

WHO	SwiftUI's layout system, adding margin around LiveTotalView's stack.
WHAT	**Adds the standard system margin around the slider and both labels.**
WHERE	Attached to the VStack, outside its three children.
WHEN	Applied on each layout pass, including the ones caused by dragging.
WHY	Keeps the slider's thumb away from the screen edge, where a finger has trouble grabbing it.

```
16 #Preview { LiveTotalView() }
```

WHO	Xcode's preview canvas, and the simulated iPhone on this page for the fixed version.
WHAT	**Shows a live LiveTotalView whose total follows the slider at every position.**
WHERE	At file scope; like every #Preview, it is left out of the shipped app.
WHEN	On every code edit, so a change to the 1.18 rate shows up at once.
WHY	Drag both versions side by side and compare their totals.

Takeaway

Every extra copy of a fact is a promise to keep two values in sync by hand, and that promise breaks the first time some code path forgets. Store the minimum state and compute the rest: if a value can be calculated from other state, make it a computed property, not `@State`.

CONCEPT

Sharing state with a child: @Binding

As screens grow, you split them into small child views. Sometimes a child must *change* a value its parent owns, such as a reusable tip picker that sets the parent's percentage. Copying the value into the child would create a second source of truth, the bug you just saw.

A **binding** solves this. It is a two-way connection to state stored somewhere else: the child reads and writes through it, but the value still lives in the parent.

@State	Owns a value. Exactly one view should own each piece of state.
@Binding	Borrows read and write access to state owned elsewhere. It stores nothing itself.
$ prefix	Turns state into a binding: $tipPercent passes the connection, while tipPercent passes a copy of the number.
Controls	Slider, Toggle, Stepper and TextField all take bindings, which is why you have already been writing $.

Who owns it?

For every piece of state, ask which view owns it. Put `@State` on the highest view that needs the value, and pass bindings down to the children that change it.

CODE · LIVE SWIFTUI PREVIEW

A reusable child with @Binding

`TipPercentPicker` is a small reusable control. It does not own the percentage; it receives a binding to `TipScreen`'s state. Tap minus and plus in the phone: the stepper lives in the child, yet the big number in the parent changes, because both read the same single value.

```swift
import SwiftUI

struct TipPercentPicker: View {
    @Binding var percent: Int
    var body: some View {
        Stepper("Tip", value: $percent, in: 0...30)
    }
}

struct TipScreen: View {
    @State private var tipPercent = 18
    var body: some View {
        VStack(spacing: 20) {
            Text("\(tipPercent)%")
                .font(.system(size: 64, weight: .bold))
            TipPercentPicker(percent: $tipPercent)
        }
        .padding()
    }
}

#Preview { TipScreen() }
```

Line by line

1 `import SwiftUI`

WHO	The Swift compiler, which resolves both @State and @Binding from this one import.
WHAT	**Makes Stepper, Text, VStack and the @State and @Binding property wrappers available to both views in this file.**
WHERE	Top of the file; it covers both structs below.
WHEN	At compile time, before either struct in this file is checked.
WHY	One import serves every type in the file, however many views it declares.

3 `struct TipPercentPicker: View {`

WHO	Your code defines it; any parent can embed it.
WHAT	**Declares a small reusable control for choosing a tip percentage.**
WHERE	File scope, separate from the screen that uses it.
WHEN	A new value is created each time TipScreen's body runs.
WHY	Pulling a control into its own view makes it reusable on other screens and keeps each body short.

4 `@Binding var percent: Int`

WHO	The parent supplies the connection; SwiftUI routes reads and writes through it.
WHAT	**Declares percent as a binding: a two-way link to an Int stored somewhere else.**
WHERE	In the child, but the actual number lives in TipScreen's state storage.
WHEN	Connected when the parent creates the picker with TipPercentPicker(percent: $tipPercent).
WHY	The child can change the value without owning a copy, so there is still exactly one source of truth.

5 `var body: some View {`

WHO	SwiftUI calls the child's body whenever it needs the picker's description.
WHAT	**Describes the child, which is just one stepper.**
WHERE	Inside TipPercentPicker, the small reusable child view.
WHEN	Whenever the parent rebuilds it or percent changes.
WHY	A view can be as small as one control. Reuse, not size, decides whether something deserves its own view.

```
6  Stepper("Tip", value: $percent, in: 0...30)
```

WHO — SwiftUI draws the minus and plus buttons; the user taps them.

WHAT — **A stepper that changes percent by 1, between 0 and 30. $percent passes the binding along to the Stepper.**

WHERE — The whole content of TipPercentPicker's body; the child shows nothing else.

WHEN — Each tap writes through the binding, straight into TipScreen's state.

WHY — A binding can be handed on again; the Stepper never knows the value really lives one view up.

```
10 struct TipScreen: View {
```

WHO — Your code declares the parent screen; the preview creates it.

WHAT — **Declares the screen that owns the tip percentage.**

WHERE — At file scope, below the child view it uses.

WHEN — Created when the preview starts; SwiftUI keeps its tipPercent state across rebuilds.

WHY — The screen is the highest view that needs the value, so it is the natural owner.

```
11 @State private var tipPercent = 18
```

WHO — SwiftUI stores it on behalf of TipScreen.

WHAT — **The single source of truth for the percentage, starting at 18.**

WHERE — In TipScreen's state storage, not in the child.

WHEN — Created when TipScreen appears; changed whenever the child's stepper is tapped.

WHY — Ownership lives in one place; every other view borrows.

```
12 var body: some View {
```

WHO — SwiftUI calls TipScreen's body to describe the number and the picker.

WHAT — **Describes the screen: the big number and the picker.**

WHERE — Inside TipScreen, the view that owns tipPercent.

WHEN — After every change to tipPercent, including changes the child makes through the binding.

WHY — Both the number and the picker are rebuilt from the same value, so they always match.

```
13 VStack(spacing: 20) {
```

WHO — SwiftUI's layout system, stacking the big number above the child picker.

WHAT — **Stacks the number above the picker, 20 points apart.**

WHERE — The root view returned by TipScreen's body.

WHEN — Laid out again after each stepper tap changes tipPercent.

WHY — Generous spacing separates the display from the control.

```
14 Text("\(tipPercent)%")
```

WHO — SwiftUI draws it; the value comes from the parent's own tipPercent state.

WHAT — **Shows the current percentage, such as "18%".**

WHERE — First child of the parent's VStack, above the child picker.

WHEN — Rebuilt every time the child writes a new value through the binding.

WHY — This proves the binding works: the child changes the value and the parent displays it.

```
15 .font(.system(size: 64, weight: .bold))
```

WHO — SwiftUI's text rendering, using the system font at a fixed point size.

WHAT — **Draws the percentage in a 64-point bold system font, far larger than any built-in text style.**

WHERE — Attached to the percentage Text only, not to the stepper below it.

WHEN — Applied each time body rebuilds the percentage Text.

WHY — A huge number makes each tap obvious. In production code prefer text styles, which Dynamic Type can scale.

```
16  TipPercentPicker(percent: $tipPercent)
```

WHO	The parent, TipScreen, creating the child view and passing it a binding.
WHAT	**Embeds the picker and hands it a binding to tipPercent using the $ prefix.**
WHERE	Second child of the parent's VStack, below the big percentage.
WHEN	Each time TipScreen's body runs; the child value is new, but its binding still points at the same state.
WHY	Without the $ you would pass a plain Int copy, and the compiler would refuse because the child expects a Binding<Int>.

```
18  .padding()
```

WHO	SwiftUI's layout system, adding margin around TipScreen's stack.
WHAT	**Adds the standard margin around the percentage and the stepper.**
WHERE	Attached to TipScreen's VStack, outside both of its children.
WHEN	Applied on every layout pass of TipScreen.
WHY	Keeps the stepper away from the screen edges.

```
22  #Preview { TipScreen() }
```

WHO	The preview canvas and this page's phone.
WHAT	**Shows TipScreen, which in turn creates the picker.**
WHERE	At file scope; the preview is not compiled into the shipped app.
WHEN	Each time the code changes, in Xcode or in this page's editor.
WHY	Previewing the parent shows both views working together through the binding.

Try it

1. In TipScreen, add Button("Reset") { tipPercent = 15 } below the picker. The child follows the parent.
2. Break it on purpose: in the child change @Binding to @State, and at the call site pass tipPercent without the $. Tap the stepper: the big number no longer moves, because the child now has its own copy.

CODE · LIVE SWIFTUI PREVIEW

Two controls, one truth

A focus switch you can flip two ways: with the toggle or with the button. Both write the same `doNotDisturb` state, and the icon, its color, the toggle and the button title all read it. Use either control and every part of the screen agrees, because there is only one value to agree on.

```swift
import SwiftUI

struct FocusView: View {
    @State private var doNotDisturb = false

    var body: some View {
        VStack(spacing: 24) {
            Image(systemName: doNotDisturb ? "moon.fill" : "sun.max")
                .font(.system(size: 60))
                .foregroundStyle(doNotDisturb ? .indigo : .orange)
            Toggle("Do Not Disturb", isOn: $doNotDisturb)
            Button(doNotDisturb ? "Wake up" : "Go quiet") {
                doNotDisturb.toggle()
            }
            .buttonStyle(.borderedProminent)
        }
        .padding()
    }
}

#Preview { FocusView() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, which needs SwiftUI for every control on the focus screen.
- WHAT — **Brings in Image, Toggle, Button, VStack and @State.**
- WHERE — First line of the file; it applies to FocusView and its preview.
- WHEN — At compile time, before FocusView is type-checked.
- WHY — Every control on this screen comes from SwiftUI.

3 `struct FocusView: View {`

- WHO — Your code declares the focus screen; the preview creates it.
- WHAT — **Declares FocusView, a small screen that turns a quiet mode on and off.**
- WHERE — At file scope, so any other view could embed a FocusView.
- WHEN — Created when the preview starts; SwiftUI recreates the struct on each rebuild while the state persists.
- WHY — A single Boolean is enough state to drive four different visual changes.

4 `@State private var doNotDisturb = false`

- WHO — SwiftUI stores it; two controls write it.
- WHAT — **A Bool (true or false) that records whether quiet mode is on. It starts off.**
- WHERE — In SwiftUI-managed storage attached to FocusView, shared by both controls.
- WHEN — Changes when the toggle is flipped or the button is tapped.
- WHY — Two controls, one value: neither control keeps its own copy of the setting.

6 `var body: some View {`

- WHO — SwiftUI calls body to describe the icon, the toggle and the button.
- WHAT — **Describes the whole screen for the current doNotDisturb value.**
- WHERE — Inside FocusView, directly below the single piece of state it reads.
- WHEN — After every flip of doNotDisturb, whether the toggle or the button made it.
- WHY — Every visual detail is computed from one value, so none of them can disagree.

```
7  VStack(spacing: 24) {
```

WHO — SwiftUI's layout system, stacking the icon, the toggle and the button.
WHAT — **Stacks the icon, toggle and button with 24 points between them.**
WHERE — The root view returned by FocusView's body.
WHEN — Laid out again after each flip, because the icon and the button title change.
WHY — 24 points of spacing keep the icon, the switch and the button visually separate, so each reads as its own element.

```
8  Image(systemName: doNotDisturb ? "moon.fill" : "sun.max")
```

WHO — SwiftUI, drawing an SF Symbol (an icon from Apple's symbol set).
WHAT — **Shows a filled moon when quiet mode is on and a sun when it is off. The ?: operator picks one of two values.**
WHERE — First child of the VStack, at the top of the focus screen.
WHEN — Rebuilt each time body runs, which is after every change to doNotDisturb.
WHY — Choosing content inside body from state is the declarative way: describe both outcomes, let the value decide.

```
9  .font(.system(size: 60))
```

WHO — SwiftUI, sizing the SF Symbol as if it were a character of text.
WHAT — **Draws the symbol at 60 points; SF Symbols size like text.**
WHERE — Attached to the moon or sun Image directly above it.
WHEN — Applied each time body rebuilds the icon.
WHY — Symbols are drawn like a font, so a font size scales them crisply.

```
10 .foregroundStyle(doNotDisturb ? .indigo : .orange)
```

WHO — SwiftUI, choosing the icon's color from the current state.
WHAT — **Colors the icon indigo in quiet mode and orange otherwise.**
WHERE — Attached to the Image, after the font size modifier.
WHEN — Re-evaluated on each body run, so the color switches the moment the state flips.
WHY — Color reinforces state, but the icon also changes shape, so the meaning does not rely on color alone.

```
11 Toggle("Do Not Disturb", isOn: $doNotDisturb)
```

WHO — SwiftUI draws the switch; the user flips it.
WHAT — **A labeled on/off switch bound to doNotDisturb.**
WHERE — Second child of the VStack, between the icon and the button.
WHEN — Writes the new value the instant it is flipped, and redraws itself when the button changes the value.
WHY — Because it reads through a binding, the toggle also moves when some other code changes the value.

```
12 Button(doNotDisturb ? "Wake up" : "Go quiet") {
```

WHO — You declare the button; the user taps it to flip the mode from a second control.
WHAT — **A button whose title describes what tapping will do next.**
WHERE — Third child of the VStack, below the toggle it mirrors.
WHEN — Its title is rebuilt with every change; its action runs on tap.
WHY — Naming the action ("Go quiet"), not the current state, makes the button easy to understand.

```
13 doNotDisturb.toggle()
```

WHO — Your action closure, which writes to the same state the toggle is bound to.
WHAT — **Flips the Bool: false becomes true, true becomes false.**
WHERE — Inside the Button's trailing action closure, its only line of code.
WHEN — On each tap, after which SwiftUI re-runs body and the toggle slides to match.
WHY — The button changes the same state the toggle uses, so the toggle visibly flips too.

```
15  .buttonStyle(.borderedProminent)
```

WHO SwiftUI, applying the system's filled, tinted button appearance.
WHAT **Fills the Go quiet button with the accent color so it stands out from the toggle above it.**
WHERE Attached to the Go quiet / Wake up Button.
WHEN Applied each time body rebuilds the button with its current title.
WHY Marks the button as the screen's primary action.

```
17  .padding()
```

WHO SwiftUI's layout system, adding margin around the focus screen's stack.
WHAT **Adds the standard margin around the icon, the toggle and the button.**
WHERE Attached to FocusView's VStack, outside all three of its children.
WHEN Applied on every layout pass of FocusView.
WHY Keeps the toggle label away from the screen edge.

```
21  #Preview { FocusView() }
```

WHO Xcode's preview canvas and the phone beside this code, both running FocusView live.
WHAT **Shows a working FocusView whose toggle and button you can both use.**
WHERE At file scope; preview code never ships inside the app.
WHEN Each time you edit the code, so you can change the colors or symbols and try again.
WHY Lets you flip both controls and confirm they stay in step.

> **Try it**
>
> 1. Add a second Toggle("Silence calls", isOn: $doNotDisturb) below the first. Flip any of the three controls.
> 2. Add Text(doNotDisturb ? "Notifications silenced" : "Notifications on") under the icon.
> 3. Start in quiet mode by changing the initial value to true.

CHECK YOURSELF

Check your understanding

1. A view has `@State private var count = 0`, a computed property `var doubled: Int { count * 2 }`, a `Text("\(doubled)")`, and a button that runs `count += 1`. After three taps, what does the Text show?

   A. 0
   B. 3
   C. 6
   D. 2

2. A shopping screen stores `@State var items: [String]` and also `@State var itemCount = 0`. The Add button appends an item and adds 1 to itemCount. A new Delete button removes an item. What goes wrong?

   A. Nothing; both values are @State, so SwiftUI keeps them in sync for you
   B. itemCount stays too high, because the delete path never updates it
   C. The code does not compile, because two @State arrays cannot share a view
   D. SwiftUI merges the two values into one, so the count is always right

3. A child view must turn its parent's `isMuted` on and off. Which declaration belongs in the child?

   A. @Binding var isMuted: Bool
   B. @State private var isMuted = false
   C. let isMuted: Bool
   D. private var isMuted = false

4. In FocusView, the Toggle is bound with `isOn: $doNotDisturb`. You tap the button, which runs `doNotDisturb.toggle()`. What does the switch on screen do?

   A. It stays where it was until someone taps it
   B. It flips, then immediately flips back
   C. Nothing; the code does not compile because two controls cannot change one value
   D. It flips to match, because it reads the same state through its binding

Answers: State Drives the Screen

1. **C. 6** Each tap changes `count`, so SwiftUI re-runs `body`, which reads `doubled`, which computes 3 × 2 = 6. Answering 0 treats the computed property as if it were calculated once and stored, but computed properties run every time they are read.
2. **B. itemCount stays too high, because the delete path never updates it** itemCount is a second source of truth for a fact that `items.count` already knows. @State stores values; it does not know that one depends on the other. The fix is to delete itemCount and read `items.count`.
3. **A. @Binding var isMuted: Bool** A binding reads and writes state owned elsewhere, so the parent's value changes. @State is the tempting wrong answer: it would give the child its own private copy, and changes would never reach the parent. A let or plain var is a one-way copy.
4. **D. It flips to match, because it reads the same state through its binding** A binding is a two-way connection, not a copy. The button changes the state, SwiftUI re-runs body, and the Toggle reads the new value. Expecting it to stay put assumes the switch remembers its own position, which is exactly the second source of truth SwiftUI avoids. Any number of controls may share one piece of state.

Observable Models

CONCEPT

When state outgrows a view

`@State` is ideal for a few simple values that one view owns. Real apps also have **models**: types that hold the app's data and the rules about it, such as a coffee order with its shots, milk and price. Several screens may show the same order.

Write such a model as a class marked with the `@Observable` **macro** (a compiler feature that writes extra code for you). The **Observation** framework then records which properties each `body` reads, and when one of them changes, re-runs only the bodies that read it.

Model	Data plus the rules about it, kept separate from any screen.
@Observable	Makes a class report every read and write of its stored properties to observers such as SwiftUI.
Reference type	A class. Every view that holds one points at the same single object.
Per-property tracking	A view that reads only the price is not redrawn when the milk choice changes.

> **The older way: ObservableObject**
>
> Before iOS 17, models adopted `ObservableObject`, marked properties `@Published`, and views held them with `@StateObject` or `@ObservedObject`. Any change re-ran every observing view. You will meet this in older code; new code uses `@Observable`.

CODE · LIVE SWIFTUI PREVIEW

Your first @Observable model

The order now lives in a class marked `@Observable`. The view creates it once and keeps it alive with `@State`. Tap the button: the view changes the model, the model reports the change, and SwiftUI re-runs the body that read `shots` and `price`.

```swift
import SwiftUI

@Observable
final class CoffeeOrder {
    var shots = 2
    var price: Double { 3.25 + Double(shots) * 0.75 }
}

struct OrderView: View {
    @State private var order = CoffeeOrder()
    var body: some View {
        VStack(spacing: 16) {
            Text("Latte with \(order.shots) shots")
            Text(String(format: "$%.2f", order.price))
                .font(.largeTitle)
            Button("Add a shot") { order.shots += 1 }
                .buttonStyle(.borderedProminent)
        }
    }
}

#Preview { OrderView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which also receives the @Observable macro through this import.
- **WHAT** **Brings in SwiftUI, which also re-exports the Observation framework that provides @Observable.**
- **WHERE** First line of the file, covering both the model and the view.
- **WHEN** At compile time, before the @Observable macro below is expanded.
- **WHY** One import gives you both the views and the observation macro, so no separate import Observation is needed.

3 `@Observable`

- **WHO** The Swift compiler, expanding a macro from the Observation framework.
- **WHAT** **Rewrites the class below so every read and write of its stored properties is reported to observers such as SwiftUI.**
- **WHERE** Attached to the class declaration on the next line.
- **WHEN** Expanded at compile time; the generated tracking code then runs at every property access.
- **WHY** You get precise change tracking without writing any notification code or marking individual properties.

4 `final class CoffeeOrder {`

- **WHO** Your code; each order object lives on the heap (shared memory for objects).
- **WHAT** **Declares the order as a class, a reference type, that cannot be subclassed.**
- **WHERE** File scope. Every view that holds an order points at the same object in heap memory.
- **WHEN** An instance is created when OrderView first appears.
- **WHY** A model shared by several views must be one object, not copies. final tells readers and the compiler there are no subclasses.

```
5  var shots = 2
```

WHO The model owns it; the generated tracking code watches it.
WHAT **A stored property for the number of espresso shots, starting at 2.**
WHERE Inside each CoffeeOrder object on the heap.
WHEN Read during body; written when the button is tapped.
WHY Stored properties of an @Observable class are tracked automatically, with no @Published needed.

```
6  var price: Double { 3.25 + Double(shots) * 0.75 }
```

WHO Swift computes it; the model defines the pricing rule.
WHAT **A computed price: a 3.25 base plus 0.75 per shot.**
WHERE Inside CoffeeOrder, the model class, rather than in the view that shows it.
WHEN Recalculated on every read; with no stored copy, it can never be out of date.
WHY Reading price reads shots, so any view showing the price is automatically tracked on shots too.

```
9  struct OrderView: View {
```

WHO Your code declares the order screen; the preview creates it.
WHAT **Declares the screen that displays and changes the order.**
WHERE At file scope, below the CoffeeOrder model it displays.
WHEN Created when the preview starts; the order held in @State survives later rebuilds.
WHY The view describes the order; the rules stay in the model.

```
10 @State private var order = CoffeeOrder()
```

WHO SwiftUI keeps the reference alive for this view.
WHAT **Creates one CoffeeOrder and stores a reference to it in this view's state.**
WHERE The object is on the heap; the reference lives in SwiftUI's storage for this view.
WHEN Created once, when OrderView first appears, not on every body run.
WHY @State is how a view owns a model. Without it, a fresh order would be created each time the struct is rebuilt, losing the shots.

```
11 var body: some View {
```

WHO SwiftUI, with Observation recording what it reads.
WHAT **Describes the order for its current shots and price; every model property it reads becomes a dependency.**
WHERE Inside OrderView, the only view in this example.
WHEN At first appearance, and again when shots changes.
WHY body never subscribes to anything explicitly; reading a property is the subscription.

```
12 VStack(spacing: 16) {
```

WHO SwiftUI's layout system, stacking the order's description, price and button.
WHAT **Stacks the description, price and button.**
WHERE The root view returned by OrderView's body.
WHEN Laid out again each time a change to shots re-runs body.
WHY A plain column keeps attention on the data flow: tap, the model changes, the text updates.

```
13 Text("Latte with \(order.shots) shots")
```

WHO SwiftUI; Observation notes the read of shots.
WHAT **Describes the order, for example "Latte with 2 shots".**
WHERE First child of the VStack, above the price.
WHEN Rebuilt whenever order.shots changes, because Observation saw body read it.
WHY Reading order.shots inside body is exactly what makes this view update when shots changes.

```
14  Text(String(format: "$%.2f", order.price))
```

WHO	SwiftUI draws it; reading price makes Observation track shots behind the scenes.
WHAT	**Shows the price with two decimals, such as "$4.75".**
WHERE	Second child of the VStack, between the description and the button.
WHEN	Rebuilt whenever shots changes, because price depends on it.
WHY	The view formats the price but does not calculate it; the rule lives in one place, the model.

```
15  .font(.largeTitle)
```

WHO	SwiftUI with Dynamic Type, which scales the large title style to the reader's setting.
WHAT	**Draws the price in the large title style.**
WHERE	Attached to the price Text only, so the description keeps the body style.
WHEN	Applied each time body rebuilds the price Text.
WHY	The price is the number people check, so it gets the most weight.

```
16  Button("Add a shot") { order.shots += 1 }
```

WHO	The user tapping; your closure writing to the model.
WHAT	**Adds one shot to the shared order object.**
WHERE	Third child of the VStack, at the bottom of the screen.
WHEN	On each tap; Observation then marks OrderView's body as needing to run again.
WHY	order is a reference, so the closure changes the object itself. The model's generated code tells SwiftUI, and the view refreshes.

```
17  .buttonStyle(.borderedProminent)
```

WHO	SwiftUI, drawing the button with a filled, tinted background.
WHAT	**Styles the button as the primary action.**
WHERE	Attached to the Add a shot Button at the bottom of the stack.
WHEN	Applied each time body rebuilds the Add a shot button.
WHY	One prominent button tells people the main thing this screen lets them do: add a shot.

```
22  #Preview { OrderView() }
```

WHO	Xcode's preview canvas and this page's phone, running OrderView with a real model.
WHAT	**Shows a live OrderView, which creates its own order.**
WHERE	At file scope; the preview and its order exist only during development.
WHEN	Each time you edit the code; a fresh CoffeeOrder starts again at 2 shots.
WHY	Lets you exercise the model through the real interface.

Try it

1. Add a Button("Remove a shot") { order.shots -= 1 } below the first one.
2. Change the base price 3.25 to 3.50 inside the model. The view needs no change at all.
3. Add var drink = "Mocha" to the model and use it in the first Text instead of "Latte".

INTERACTIVE

See which views re-run

Three small views share one `@Observable` coffee order: a header reads the drink, a price tag reads the price, and a summary reads both. Change one property at a time and watch which bodies light up and how their run counters climb. Then switch to **ObservableObject** mode, the older approach, and compare how many views re-run for exactly the same change.

This step is an interactive simulation in the HTML version.

CODE · LIVE SWIFTUI PREVIEW

Editing a model with @Bindable

Controls need bindings, but a view that merely *receives* a model cannot use `$` on it. Marking the property `@Bindable` fixes that: `$order.shots` becomes a binding straight into the model. Here the preview plays the parent that creates the order; in an app, a parent holding it with `@State` would pass it in.

```swift
import SwiftUI

@Observable
final class CoffeeOrder {
    var shots = 2
    var oatMilk = false
}

struct OrderEditor: View {
    @Bindable var order: CoffeeOrder
    var body: some View {
        Form {
            Text(order.oatMilk ? "Oat latte" : "Latte")
                .font(.title2)
            Stepper("\(order.shots) shots",
                    value: $order.shots, in: 1...4)
            Toggle("Oat milk", isOn: $order.oatMilk)
        }
    }
}

#Preview { OrderEditor(order: CoffeeOrder()) }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which gets @Bindable and @Observable from this one import.
- **WHAT** **Brings in Form, Stepper, Toggle, @Observable and @Bindable.**
- **WHERE** First line of the file, covering the model, the editor and the preview.
- **WHEN** At compile time, before the macro on the next lines is expanded.
- **WHY** @Bindable is part of SwiftUI because it exists to connect observable models to controls.

3 `@Observable`

- **WHO** The compiler, expanding the Observation macro.
- **WHAT** **Makes the class below report reads and writes of shots and oatMilk.**
- **WHERE** Attached to the CoffeeOrder class declared on the next line.
- **WHEN** Expanded at compile time into tracking code for each stored property.
- **WHY** Only observable classes can be used with @Bindable.

4 `final class CoffeeOrder {`

- **WHO** Your code declares the model class; the preview creates the instance.
- **WHAT** **Declares the order model as a reference type.**
- **WHERE** At file scope; each CoffeeOrder instance lives on the heap and is shared by reference.
- **WHEN** An instance is created by the preview below.
- **WHY** The editor changes the very same object its creator holds, not a copy.

```
5  var shots = 2
```

WHO The CoffeeOrder model, which owns the shot count for this order.
WHAT **Number of espresso shots, starting at 2.**
WHERE A stored property inside each CoffeeOrder object.
WHEN Written each time the Stepper's minus or plus button is tapped.
WHY A tracked stored property, so both the label and the stepper refresh when it changes.

```
6  var oatMilk = false
```

WHO The CoffeeOrder model, which also owns the milk choice.
WHAT **Whether the drink uses oat milk, off by default.**
WHERE A stored property on the same object, right next to shots.
WHEN Written each time the user flips the Oat milk toggle.
WHY Each user choice is one tracked property of the model.

```
9  struct OrderEditor: View {
```

WHO Your code; whoever creates the order passes it in.
WHAT **Declares a form that edits an existing order.**
WHERE At file scope, so any screen that already has an order could present it.
WHEN Created by the preview, which hands it a fresh order.
WHY The editor never creates an order itself, so it can edit whichever order its caller chooses.

```
10  @Bindable var order: CoffeeOrder
```

WHO SwiftUI provides the wrapper; the caller provides the object.
WHAT **Stores a reference to the order and lets you write $order.property to get bindings to its properties.**
WHERE In the editor; the object itself lives wherever its creator keeps it.
WHEN Set when OrderEditor is created; the reference then stays fixed while the object's properties change.
WHY A plain var order could be read but not bound. @Bindable adds the $ syntax without taking ownership, which @State would imply.

```
11  var body: some View {
```

WHO SwiftUI, with Observation tracking reads.
WHAT **Describes the form for the current order.**
WHERE Inside OrderEditor, reading the order it was handed.
WHEN After any change to a property it reads.
WHY The same reactive loop as @State, now driven by the model.

```
12  Form {
```

WHO SwiftUI, which styles everything inside as grouped iOS settings rows.
WHAT **A container that styles its children as grouped settings rows.**
WHERE The root view returned by OrderEditor's body.
WHEN Built each time body runs, after any tracked property of the order changes.
WHY Form gives the familiar iOS settings look with no styling code.

```
13  Text(order.oatMilk ? "Oat latte" : "Latte")
```

WHO SwiftUI draws it; Observation records that this body read oatMilk.
WHAT **Names the drink based on the milk choice.**
WHERE First row of the Form, above the stepper.
WHEN Rebuilt whenever order.oatMilk changes, for example when the toggle flips.
WHY Shows instantly that the Toggle really changed the model.

```
14  .font(.title2)
```

WHO	SwiftUI with Dynamic Type, scaling the title 2 text style.
WHAT	**Uses the title 2 text style for the drink name.**
WHERE	Attached to the drink name Text in the first row.
WHEN	Applied each time body rebuilds the drink name.
WHY	Makes the summary row stand out from the controls.

```
15  Stepper("\(order.shots) shots",
```

WHO	SwiftUI draws it; the user taps minus and plus.
WHAT	**Starts a stepper whose label shows the current shot count.**
WHERE	Second row of the Form; the call continues on the next line.
WHEN	The label is rebuilt whenever shots changes.
WHY	The label reads the model, so it always matches the stored value.

```
16  value: $order.shots, in: 1...4)
```

WHO	@Bindable creates the binding; the Stepper writes through it.
WHAT	**Binds the stepper to the model's shots and limits it to 1 through 4.**
WHERE	The second line of the Stepper call that began on the line above.
WHEN	Writes through the binding on each minus or plus tap.
WHY	$order.shots is only possible because order is @Bindable; the range keeps the order valid.

```
17  Toggle("Oat milk", isOn: $order.oatMilk)
```

WHO	SwiftUI draws the switch; the user flips it, writing straight into the model.
WHAT	**An on/off switch bound to the model's oatMilk property.**
WHERE	Third and last row of the Form, below the shots stepper.
WHEN	Writes order.oatMilk the instant the switch is flipped.
WHY	Each control edits one model property directly, with no glue code in between.

```
22  #Preview { OrderEditor(order: CoffeeOrder()) }
```

WHO	The preview, standing in for the parent screen that would own the order.
WHAT	**Creates an order and passes it to the editor.**
WHERE	At file scope; this particular order exists only in the preview.
WHEN	Each time you edit the code, with a new order starting at 2 shots and no oat milk.
WHY	In an app, a parent would own the order with @State and pass it in exactly the same way.

> **Try it**
>
> 1. Add var extraHot = false to the model and a Toggle("Extra hot", isOn: $order.extraHot) to the form.
> 2. Delete the word @Bindable. The $order lines stop compiling, because a plain property cannot produce bindings.

CODE · LIVE SWIFTUI PREVIEW

Sharing a model through @Environment

Passing a model through every initializer gets tedious when the view that needs it sits several levels down. Instead, put it in the **environment**, a set of values SwiftUI hands down the view tree automatically. The screen injects the planner once with `.environment(planner)`, and any view inside reads it with `@Environment(StudyPlanner.self)`.

```swift
import SwiftUI

@Observable final class StudyPlanner { var sessionsDone = 0 }

struct SessionButton: View {
    @Environment(StudyPlanner.self) private var planner
    var body: some View {
        Button("Finish a session") { planner.sessionsDone += 1 }
    }
}

struct PlannerScreen: View {
    @State private var planner = StudyPlanner()
    var body: some View {
        VStack(spacing: 20) {
            Text("\(planner.sessionsDone) sessions done")
            SessionButton()
        }
        .environment(planner)
    }
}
#Preview { PlannerScreen() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which needs SwiftUI for the environment system and both views.
- **WHAT** **Brings in @Observable, @Environment, @State, Button and Text.**
- **WHERE** First line of the file, covering the model and both views.
- **WHEN** At compile time, before StudyPlanner's macro is expanded.
- **WHY** The environment system is part of SwiftUI.

3 `@Observable final class StudyPlanner { var sessionsDone = 0 }`

- **WHO** Your code, with the compiler expanding @Observable.
- **WHAT** **Declares an observable planner class with one tracked property, sessionsDone, that counts finished study sessions from 0. A model this small fits on one line.**
- **WHERE** File scope; the instance lives on the heap.
- **WHEN** An instance is created when PlannerScreen appears; sessionsDone is written by the button in the child view.
- **WHY** Only observable classes can be placed in the environment by type, and tracking keeps every view that reads sessionsDone up to date.

5 `struct SessionButton: View {`

- **WHO** Your code declares the child view; PlannerScreen creates it.
- **WHAT** **Declares a child view that finishes a session.**
- **WHERE** File scope; it is used inside PlannerScreen.
- **WHEN** Created each time the parent's body runs.
- **WHY** Its initializer takes no planner at all, which is the point of the environment.

```
6  @Environment(StudyPlanner.self) private var planner
```

WHO	SwiftUI looks up the planner that an ancestor view provided.
WHAT	**Reads the StudyPlanner object from the environment, found by its type.**
WHERE	In the child; the object came from PlannerScreen higher up the view tree.
WHEN	Resolved when the view is placed in the tree.
WHY	No intermediate view has to pass the planner along by hand. If no ancestor provided one, the app stops with an error.

```
7  var body: some View {
```

WHO	SwiftUI calls the child's body to describe its single button.
WHAT	**Describes SessionButton's content: one button that finishes a study session.**
WHERE	Inside SessionButton, below the @Environment property it uses.
WHEN	When the child first appears. Its body reads no planner property, so a new count does not force it to run again.
WHY	Child views have bodies of their own, just like screens.

```
8  Button("Finish a session") { planner.sessionsDone += 1 }
```

WHO	The user tapping; your closure writing to the shared model.
WHAT	**Adds one to the planner's session count.**
WHERE	The whole content of SessionButton's body.
WHEN	On each tap; the closure reads and writes sessionsDone only at that moment.
WHY	The child writes to the same object the parent reads, so the parent's text updates too.

```
12  struct PlannerScreen: View {
```

WHO	Your code declares the owning screen; the preview creates it.
WHAT	**Declares the screen that owns the planner.**
WHERE	At file scope, below the child view it contains.
WHEN	Created when the preview starts; its planner survives later rebuilds.
WHY	The top view that needs the model owns it; descendants borrow it from the environment.

```
13  @State private var planner = StudyPlanner()
```

WHO	SwiftUI, which keeps this reference alive for as long as PlannerScreen is on screen.
WHAT	**Creates the one StudyPlanner for this screen.**
WHERE	In SwiftUI-managed storage attached to PlannerScreen.
WHEN	Created once, when PlannerScreen first appears; rebuilds reuse the same object.
WHY	@State owns the object; .environment below only shares it.

```
14  var body: some View {
```

WHO	SwiftUI calls the parent's body; Observation records that it reads sessionsDone.
WHAT	**Describes the progress text and the child button.**
WHERE	Inside PlannerScreen, the owner of the planner.
WHEN	Whenever sessionsDone changes, because the Text reads it.
WHY	The parent displays what the child changes.

```
15  VStack(spacing: 20) {
```

WHO	SwiftUI's layout system, stacking the count above the child button.
WHAT	**Stacks the progress text above the button.**
WHERE	The root view of PlannerScreen's body, and the view that .environment is attached to.
WHEN	Laid out again each time a new count re-runs the parent's body.
WHY	Two views in a column make the relationship easy to see: the parent shows the count, the child changes it.

```
16  Text("\(planner.sessionsDone) sessions done")
```

WHO	SwiftUI draws it; Observation records the read of sessionsDone.
WHAT	**Shows the count, such as "2 sessions done".**
WHERE	First child of the parent's VStack, above SessionButton.
WHEN	Rebuilt whenever the child increments the count.
WHY	Proves the child and the parent share one object.

```
17  SessionButton()
```

WHO	PlannerScreen's body, creating a SessionButton value.
WHAT	**Places the child button with no arguments, not even the planner it needs.**
WHERE	Second child of the parent's VStack, directly under the session count.
WHEN	Created each time the parent's body runs; SwiftUI keeps the same child in the same spot of the tree.
WHY	Nothing is passed in; the child finds the planner in the environment.

```
19  .environment(planner)
```

WHO	SwiftUI's environment system, which passes values down the view tree.
WHAT	**Makes this planner available to every view inside the VStack, however deeply nested.**
WHERE	Applied to the VStack, so it covers that subtree only.
WHEN	Applied each time the parent's body runs, always with the same planner object.
WHY	Inject once near the top, read anywhere below. Apps often do this at the root so every screen can reach a shared model.

```
22  #Preview { PlannerScreen() }
```

WHO	Xcode's preview canvas, showing the owner screen so the environment is set up.
WHAT	**Shows PlannerScreen, which creates and injects the planner.**
WHERE	At file scope, previewing PlannerScreen, the view that injects the planner, rather than the child alone.
WHEN	Each time you edit the code, starting again from 0 sessions.
WHY	Previewing the owner ensures the environment is set up for the child.

> **Try it**
>
> 1. Add a second SessionButton() inside the VStack. Both write to the same planner.
> 2. Predict: what happens if you delete .environment(planner)? (In a real app it stops with an error saying no StudyPlanner was found in the environment.)

CONCEPT

Model, view, and view model

As an app grows, keep three jobs apart. The **model** holds data and rules. The **view** describes the screen and forwards what the user wants to do. **MVVM** (Model-View-ViewModel) is one common way to organise a SwiftUI app: the **view model** is a plain `@Observable` class that holds a screen's state and logic so the view stays simple. Small screens do not need one; screens with loading, errors, or several sources of data usually do.

Model	CoffeeOrder: shots, milk, the price rule and the four-shot limit.
View	OrderView: reads the order, shows it, and calls order.addShot() on a tap.
View model	A checkout screen's class that adds status and error messages for that one screen.
Payoff	Rules written in plain Swift run, and can be tested, with no screen at all.

Keep body thin

If body contains business rules, such as price math or limits, move them into the model. body should read values and describe layout.

CODE · RUNS IN THE BROWSER

The model runs without a screen

Here is the coffee order's logic as plain Swift: no `import SwiftUI`, no views. It enforces a four-shot limit and prices oat milk. Run it in the console. Because the rules live in the model, you can exercise them directly, which is exactly what automated tests do. In the app you would add `@Observable`, and every view would follow the same rules.

```swift
import Foundation

final class CoffeeOrder {
    var shots = 2
    var oatMilk = false

    var price: Double {
        let base = 3.25 + Double(shots) * 0.75
        return oatMilk ? base + 0.60 : base
    }

    func addShot() {
        if shots < 4 { shots += 1 }
    }
}

let order = CoffeeOrder()
print(String(format: "$%.2f", order.price))
for _ in 1...3 { order.addShot() }
order.oatMilk = true
print(order.shots, String(format: "$%.2f", order.price))
```

OUTPUT

```
$4.75
4 $6.85
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, which loads Foundation for String(format:).
- WHAT — **Loads Foundation, Apple's base library, which provides String(format:) for formatting prices.**
- WHERE — First line of the program; this file uses no SwiftUI at all.
- WHEN — At compile time, before the model below is checked.
- WHY — The model needs number formatting but nothing from SwiftUI. Importing only what you need keeps it usable in tests and tools.

```
3  final class CoffeeOrder {
```

WHO Your code declares the model; the top-level code below creates and uses it.
WHAT **Declares the order model as a class, the same shape as in the app.**
WHERE File scope; the object lives on the heap.
WHEN An instance is created by the let order line near the bottom of the program.
WHY Leaving @Observable off here keeps the focus on the rules; adding it later changes no logic.

```
4  var shots = 2
```

WHO The CoffeeOrder object, which stores its own shot count.
WHAT **Gives every new order two shots, the starting point for the price.**
WHERE A stored property inside each CoffeeOrder object on the heap.
WHEN Changed only when addShot() runs and the limit allows it.
WHY A sensible default means a new order is valid immediately.

```
5  var oatMilk = false
```

WHO The CoffeeOrder object, which also stores the milk choice.
WHAT **Records the milk choice, dairy by default.**
WHERE A stored property next to shots, inside the same object.
WHEN Set directly near the end of the program.
WHY Choices that affect price belong in the model, next to the price rule.

```
7  var price: Double {
```

WHO Swift runs this getter each time any code reads order.price.
WHAT **Begins a computed property that calculates the price.**
WHERE Inside CoffeeOrder, so the pricing rule travels with the data.
WHEN On every read; this program reads it twice, once per print.
WHY The price can never disagree with the shots and milk it depends on.

```
8  let base = 3.25 + Double(shots) * 0.75
```

WHO Swift, evaluating the getter's first line of arithmetic.
WHAT **Computes 3.25 plus 0.75 per shot. Double(shots) converts the Int so the math types match.**
WHERE A local constant that exists only while the getter runs.
WHEN Each time price is read, before the milk rule is applied.
WHY Swift never mixes Int and Double silently; the explicit conversion prevents accidental rounding bugs.

```
9  return oatMilk ? base + 0.60 : base
```

WHO Swift's ternary (?:) operator, choosing between two prices.
WHAT **Adds 60 cents for oat milk, otherwise returns the base price.**
WHERE The last line of the price getter, which returns its result.
WHEN Each time price is read, right after base is calculated.
WHY Keeping every pricing rule in one property means a price change touches one line.

```
12 func addShot() {
```

WHO Your code; any view or test may call it.
WHAT **Declares the one approved way to add a shot.**
WHERE A method declared inside CoffeeOrder, next to the data it protects.
WHEN Whenever a caller wants another shot; here, three times in the loop.
WHY Routing changes through a method lets the model enforce its rules, instead of trusting every caller.

```
13 if shots < 4 { shots += 1 }
```

WHO The model itself, guarding its own data against an invalid order.
WHAT **Adds a shot only while there are fewer than four.**
WHERE The only line inside addShot(), the model's guarded write path.
WHEN On each call; the third call in this program finds shots already at 4 and does nothing.
WHY The limit lives in one place, so no button anywhere can create a five-shot order.

```
17 let order = CoffeeOrder()
```

WHO The program, standing in for a view or a test.
WHAT **Creates one CoffeeOrder with the defaults: two shots and dairy milk.**
WHERE Top-level code, which runs from top to bottom.
WHEN As soon as the program starts running its top-level code.
WHY let is fine even though the order changes: the constant is the reference, not the object.

```
18 print(String(format: "$%.2f", order.price))
```

WHO Swift's print function, writing one line to standard output.
WHAT **Prints the starting price with two decimals: $4.75.**
WHERE The console below the code, where the first line, $4.75, appears.
WHEN Right after the order is created, before any changes.
WHY Checking the default first gives a known starting point.

```
19 for _ in 1...3 { order.addShot() }
```

WHO The top-level program, calling the model's own method three times.
WHAT **Asks for three more shots. The underscore means the loop counter is not needed.**
WHERE Top-level code, between the first print and the milk change.
WHEN After the first print; the third call runs into the 4-shot limit.
WHY Three requests from two shots would make five, so this exercises the limit.

```
20 order.oatMilk = true
```

WHO The top-level program, setting a property on the model directly.
WHAT **Switches this order to oat milk, which adds 60 cents to the price.**
WHERE Top-level code, between the loop and the final print.
WHEN After the loop has finished asking for shots.
WHY Tests the second pricing rule in the same run.

```
21 print(order.shots, String(format: "$%.2f", order.price))
```

WHO Swift's print, writing two values separated by a single space.
WHAT **Prints the shot count and the new price, separated by a space: 4 $6.85.**
WHERE The console, as the program's second and last line of output.
WHEN As the final statement of the program, after both changes are made.
WHY The output proves the limit held at 4 and oat milk added 60 cents.

Try it

1. Change the loop to run 10 times. Does shots ever pass 4?
2. Set order.oatMilk = false just before the last print. What price appears?
3. Add func removeShot() that never goes below 1, and call it before the last print.

CHECK YOURSELF

Check your understanding

1. An `@Observable` class `Order` has `drink` and `price`. View A reads only `drink`; view B reads only `price`. Your code changes `price`. Which bodies re-run?
   A. Only B
   B. A and B
   C. Only A
   D. Neither, until you call a refresh method
2. Why does a view that creates its own model write `@State private var order = Order()` instead of `let order = Order()`?
   A. @State makes the class safe to read and write from any thread
   B. Without @State a new Order is created on every rebuild, losing its data
   C. A plain property cannot hold a class inside a View struct
   D. @State turns the class into a struct so the view can copy it
3. A child view receives `var order: CoffeeOrder` (an @Observable class) and needs `Toggle("Oat", isOn: $order.oatMilk)`. It does not compile. What is the right fix?
   A. Mark the property @State
   B. Mark the property @Binding
   C. Change CoffeeOrder into a struct
   D. Mark the property @Bindable
4. Where should the rule “an order has at most four shots” live?
   A. In every view that changes shots, repeated each time
   B. In the #Preview block, where the order is first created
   C. In the model, inside a method such as addShot()
   D. In body, inside the action of the button that adds a shot

Answers: Observable Models

1. **A. Only B** Observation tracks reads per property, so only the view that read `price` re-runs. “A and B” is how the older `ObservableObject` approach behaved, where any published change re-ran every observing view.
2. **B. Without @State a new Order is created on every rebuild, losing its data** View structs are recreated often. @State gives the model a lifetime tied to the view’s place on screen, so it is created once and kept. A plain property can hold a class, but it would be replaced by a brand-new object on every rebuild.
3. **D. Mark the property @Bindable** @Bindable lets a view that receives an observable object create bindings to its properties without owning it. @State would claim ownership of something the parent created, and @Binding expects a Binding value, not an object.
4. **C. In the model, inside a method such as addShot()** Rules belong in the model, written once, where every caller must go through them and tests can check them without a screen. A check inside a button action looks adequate, but the next button or screen that changes shots would skip it.

Lists and Dynamic Data

CONCEPT

Rows from data: List, ForEach, Identifiable

Most apps show collections: recipes, messages, habits. `List` draws scrolling rows in the standard iOS style. `ForEach` creates one view per element of a collection, inside a List or any stack.

When data changes, SwiftUI must tell rows apart, so it knows which row was deleted, which moved and which only changed its text. Each element therefore needs a stable **identity**. Conforming to the `Identifiable` protocol means providing an `id` property that is unique and never changes.

List	A scrolling container with system row styling, separators, swipe gestures and sections.
ForEach	Builds views from data. It is not a loop that runs once; SwiftUI re-evaluates it whenever the data changes.
Identifiable	A protocol that requires an id. let id = UUID() gives each value a unique random identifier.
id: \.self	Uses the value itself as its identity. Fine for unique strings, wrong if duplicates can appear.

> **Positions are not identities**
>
> If rows are identified by their index, deleting the first row makes every other row look “changed”. Animations jump and selections land on the wrong item.

CODE · LIVE SWIFTUI PREVIEW

A list of recipes

An array of `Recipe` values becomes a scrolling list with a few lines of SwiftUI. Each recipe receives a unique random id when it is created, so SwiftUI can track every row. Look at the phone, then use the challenges to make the rows richer.

```swift
import SwiftUI

struct Recipe: Identifiable {
    let id = UUID()
    let name: String
    let minutes: Int
}

struct RecipeListView: View {
    let recipes = [
        Recipe(name: "Pancakes", minutes: 20),
        Recipe(name: "Tomato Soup", minutes: 35),
        Recipe(name: "Fried Rice", minutes: 15),
    ]
    var body: some View {
        List(recipes) { recipe in
            Text("\(recipe.name), \(recipe.minutes) min")
        }
    }
}

#Preview { RecipeListView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which makes UUID available through SwiftUI's re-export of Foundation.
- **WHAT** **Brings in List and Text, plus Foundation's UUID type, which SwiftUI re-exports. Identifiable itself comes from the Swift standard library.**
- **WHERE** First line of the file, covering the Recipe model and the list view.
- **WHEN** At compile time, before Recipe and RecipeListView are checked.
- **WHY** Everything this screen needs arrives with one import.

3 `struct Recipe: Identifiable {`

- **WHO** Your code defines it; List relies on its conformance.
- **WHAT** **Declares a recipe type and promises it has a unique id.**
- **WHERE** At file scope, so any view in the app can show recipes.
- **WHEN** Checked at compile time; values are created in the array below.
- **WHY** Identifiable is what lets List and ForEach use recipes directly, with no extra id argument.

4 `let id = UUID()`

- **WHO** Foundation generates the value; SwiftUI reads it.
- **WHAT** **Gives each recipe a universally unique identifier, a random 128-bit value, at creation.**
- **WHERE** A stored constant inside each Recipe value, next to its name and minutes.
- **WHEN** Once, when each Recipe is created; let means it never changes afterwards.
- **WHY** Two recipes with the same name still get different ids, so SwiftUI never confuses them.

5 `let name: String`

- **WHO** The Recipe struct, which stores the name as part of each value.
- **WHAT** **Stores the recipe's display name, such as Pancakes.**
- **WHERE** A constant stored property inside each Recipe value.
- **WHEN** Set once by the memberwise initializer when each recipe is created.
- **WHY** It is the text each row shows; let fits because a recipe's name does not change on this screen.

```
6   let minutes: Int
```

WHO The Recipe struct, which stores the cooking time alongside the name.
WHAT **How long the recipe takes, in whole minutes.**
WHERE A constant stored property inside each Recipe, declared after name.
WHEN Set once when each recipe is created in the array below.
WHY A second fact per row, so rows have more to show than a name.

```
9   struct RecipeListView: View {
```

WHO Your code declares the list screen; the preview below creates it.
WHAT **Declares RecipeListView, a screen that shows one row per recipe.**
WHERE At file scope, below the Recipe model it displays.
WHEN Created when the preview starts, together with its recipes array.
WHY The screen stays small because List does the heavy lifting.

```
10  let recipes = [
```

WHO Your code; in a real app the data might come from storage or a server.
WHAT **Starts a constant array of recipes. Swift infers its type, [Recipe].**
WHERE A stored constant of RecipeListView, created with each view value.
WHEN Created along with each RecipeListView value.
WHY let fits here because this screen only displays the list; nothing changes it.

```
11      Recipe(name: "Pancakes", minutes: 20),
```

WHO Swift's memberwise initializer, generated for structs.
WHAT **Creates the first recipe; its id is generated automatically.**
WHERE First element of the recipes array, so it becomes the top row.
WHEN When the recipes array is created, along with the view.
WHY Because id has a default value, the initializer does not ask for it.

```
12      Recipe(name: "Tomato Soup", minutes: 35),
```

WHO The memberwise initializer, called a second time with new values.
WHAT **Creates the Tomato Soup recipe with a 35-minute cooking time and its own new id.**
WHERE Second element of the recipes array, so it becomes the second row.
WHEN Right after the first recipe, as the array literal is evaluated.
WHY Order in the array is the order on screen.

```
13      Recipe(name: "Fried Rice", minutes: 15),
```

WHO The memberwise initializer, creating the last of the three recipes.
WHAT **Creates the third recipe. The trailing comma is allowed.**
WHERE Third and final element of the recipes array.
WHEN As the last step of building the array literal.
WHY A trailing comma makes adding a fourth line a one-line change.

```
15  var body: some View {
```

WHO SwiftUI calls body to describe the recipe screen.
WHAT **Describes the whole screen as one List built from the recipes array.**
WHERE Inside RecipeListView, below the data it turns into rows.
WHEN When the view first appears; this screen has no state, so nothing on it triggers another run.
WHY The body maps data to rows; it holds no row logic of its own.

```
16 List(recipes) { recipe in
```

WHO	SwiftUI's List, which reads each recipe's id to tell the rows apart.
WHAT	**Creates one row per recipe, using each recipe's id to track it. recipe names the current element.**
WHERE	Root of body; fills the screen and scrolls.
WHEN	Rows are built as they scroll into view.
WHY	List builds rows lazily, so a list of thousands stays fast.

```
17 Text("\(recipe.name), \(recipe.minutes) min")
```

WHO	SwiftUI draws the row; Swift string interpolation builds its text.
WHAT	**Shows one row, such as "Pancakes, 20 min".**
WHERE	Inside the row closure, once per recipe.
WHEN	Each time List builds the row for that recipe, for example as it scrolls into view.
WHY	A simple first row design; the challenges improve it.

```
22 #Preview { RecipeListView() }
```

WHO	Xcode's preview canvas, mirrored by the simulated iPhone on this page.
WHAT	**Shows RecipeListView with its three sample recipes.**
WHERE	At file scope; sample previews never ship in the app.
WHEN	Every time you edit the code, so row changes appear at once.
WHY	Lets you see the row design with real-looking data.

Try it

1. Add a fourth Recipe to the array. The list grows with no other change.
2. Replace the row with HStack { Text(recipe.name); Spacer(); Text("\(recipe.minutes) min").foregroundStyle(.secondary) }.
3. Add .listStyle(.plain) after the closing brace of the List.

CODE · LIVE SWIFTUI PREVIEW

Grouping rows with Section

A meal plan groups dishes under headers. `Section` adds a header and visual grouping; `ForEach` with `id: \.self` builds rows from plain strings, using each string as its own identity. The two ForEach calls show the long and the short way to write the same closure.

```swift
import SwiftUI

struct MealPlanView: View {
    let breakfast = ["Oatmeal", "Pancakes", "Yogurt"]
    let dinner = ["Tomato Soup", "Fried Rice"]

    var body: some View {
        List {
            Section("Breakfast") {
                ForEach(breakfast, id: \.self) { meal in
                    Label(meal, systemImage: "sun.max")
                }
            }
            Section("Dinner") {
                ForEach(dinner, id: \.self) { Text($0) }
            }
        }
        .listStyle(.insetGrouped)
    }
}

#Preview { MealPlanView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which needs SwiftUI for List, Section and Label.
- **WHAT** **Brings in List, Section, ForEach, Label and Text.**
- **WHERE** First line of the meal plan file, covering the view and its preview.
- **WHEN** At compile time, before MealPlanView is checked.
- **WHY** Sections are part of SwiftUI's list system.

3 `struct MealPlanView: View {`

- **WHO** Your code declares the meal plan screen; the preview creates it.
- **WHAT** **Declares MealPlanView, a screen that groups meals under headings.**
- **WHERE** At file scope, as its own view type that any screen could embed.
- **WHEN** Created when the preview starts; nothing on it changes afterwards.
- **WHY** Grouped data is common; this pattern applies to settings, contacts and inboxes too.

4 `let breakfast = ["Oatmeal", "Pancakes", "Yogurt"]`

- **WHO** Your code supplies the data; each string becomes one breakfast row.
- **WHAT** **Stores three breakfast dish names in a constant array of String.**
- **WHERE** A stored constant of MealPlanView, read by the first Section.
- **WHEN** Created along with each MealPlanView value.
- **WHY** Plain strings are enough when each name is unique.

5 `let dinner = ["Tomato Soup", "Fried Rice"]`

- **WHO** Your code supplies the second group's data.
- **WHAT** **Stores the two dinner dish names in their own constant array.**
- **WHERE** A stored constant of MealPlanView, read by the second Section.
- **WHEN** Created at the same moment as breakfast, with the view.
- **WHY** One array per section keeps each group's data separate.

```
7  var body: some View {
```

WHO SwiftUI calls body to describe the grouped list.
WHAT **Describes one List containing a Breakfast section and a Dinner section.**
WHERE Inside MealPlanView, below the two arrays it reads.
WHEN When MealPlanView first appears in the preview.
WHY Structure on screen mirrors structure in the data.

```
8      List {
```

WHO SwiftUI's List, building rows from the content you write inside it.
WHAT **Opens a List whose rows you write out yourself, including sections.**
WHERE The root view returned by MealPlanView's body.
WHEN Built when body runs, with rows created as they come on screen.
WHY This form of List lets you mix sections, ForEach and fixed rows.

```
9          Section("Breakfast") {
```

WHO SwiftUI's Section, which groups rows and draws the header above them.
WHAT **Starts a group of rows under the header "Breakfast".**
WHERE First child inside the List, so it appears at the top.
WHEN Built along with the List each time body runs.
WHY Headers let people scan a long list by category.

```
10             ForEach(breakfast, id: \.self) { meal in
```

WHO SwiftUI's ForEach, identifying each breakfast row by its string.
WHAT **Builds one row per breakfast string. id: \.self uses each string as its identity; meal names the current one.**
WHERE Inside the Breakfast section, producing all of that section's rows.
WHEN Evaluated when the section is built; breakfast is a let, so the rows never change afterwards.
WHY Strings are not Identifiable, so you tell ForEach how to identify them. Safe here because no two names repeat.

```
11                 Label(meal, systemImage: "sun.max")
```

WHO SwiftUI's Label view, combining an SF Symbol with text.
WHAT **Draws one breakfast row as a sun icon next to the dish name.**
WHERE Inside the ForEach closure, once for each breakfast string.
WHEN Built once per breakfast item as the section's rows are created.
WHY Label pairs an SF Symbol with text and aligns rows consistently.

```
14         Section("Dinner") {
```

WHO SwiftUI's Section, creating the second group and its header.
WHAT **Starts a second group of rows under the header Dinner.**
WHERE Second child inside the List, below the Breakfast section.
WHEN Built right after the Breakfast section, in the order written.
WHY Sections stack in the order you write them.

```
15             ForEach(dinner, id: \.self) { Text($0) }
```

WHO SwiftUI's ForEach, building the dinner rows.
WHAT **Builds one Text per dinner string, using the shorthand $0 for the current element.**
WHERE Inside the Dinner section, as its only content.
WHEN Evaluated when the Dinner section is built, once per dinner string.
WHY The short closure form suits one-line rows; use named parameters when the row grows.

```
18  .listStyle(.insetGrouped)
```

WHO — SwiftUI, applying the inset grouped list style.
WHAT — **Draws each section as a rounded card on a gray background.**
WHERE — Attached to the List, so it styles both sections at once.
WHEN — Applied each time body runs and the List is built.
WHY — The inset grouped style is the familiar iOS settings look, which makes sections stand out.

```
22  #Preview { MealPlanView() }
```

WHO — Xcode's preview canvas and the phone on this page, showing the grouped list.
WHAT — **Shows MealPlanView with both sections and their headers.**
WHERE — At file scope; the preview is excluded from the shipped app.
WHEN — Each time you edit the code, such as when you change the list style.
WHY — Switching list styles in the preview is the quickest way to compare how each one presents sections.

Try it

1. Add let lunch = ["Salad", "Wrap"] and a Section("Lunch") between the other two.
2. Change .insetGrouped to .plain and compare the look.
3. Give the Dinner rows a Label with the "moon" symbol, matching the Breakfast style.

CODE · LIVE SWIFTUI PREVIEW

Swipe to delete, and the empty state

Attach `onDelete` to a `ForEach` and every row gets the standard swipe-left Delete button; you remove the rows from the state array and the list animates them away. When the last row goes, an empty screen looks broken, so `ContentUnavailableView`, the system's standard empty state, appears in an `overlay`. Swipe all three items away in the phone.

```swift
import SwiftUI

struct ShoppingListView: View {
    @State private var items = ["Eggs", "Milk", "Bread"]
    var body: some View {
        List {
            ForEach(items, id: \.self) { Text($0) }
                .onDelete { items.remove(atOffsets: $0) }
        }
        .overlay {
            if items.isEmpty {
                ContentUnavailableView(
                    "List Complete",
                    systemImage: "cart",
                    description: Text("Everything is in the cart.")
                )
            }
        }
    }
}

#Preview { ShoppingListView() }
```

Line by line

```
1  import SwiftUI
```

WHO The Swift compiler, which needs SwiftUI for the list, the delete gesture and the empty state.
WHAT **Brings in List, ForEach, the onDelete and overlay modifiers, ContentUnavailableView and @State.**
WHERE First line of the shopping list file, covering the view and its preview.
WHEN At compile time, before ShoppingListView is checked.
WHY Swipe-to-delete and the standard empty state are both built into SwiftUI; ContentUnavailableView exists in iOS 17 and later.

```
3  struct ShoppingListView: View {
```

WHO Your code declares the shopping list screen; the preview creates it.
WHAT **Declares ShoppingListView, a list the user empties by deleting items.**
WHERE At file scope, as a standalone screen with its own state.
WHEN Created when the preview starts; its items survive later rebuilds.
WHY A list that the user empties on purpose needs a good empty state.

```
4  @State private var items = ["Eggs", "Milk", "Bread"]
```

WHO SwiftUI stores the array; the delete closure is the only code that changes it.
WHAT **The shopping items, stored as state because the user can delete them.**
WHERE In SwiftUI-managed storage attached to ShoppingListView.
WHEN Created once with three items; it loses one element on each delete.
WHY @State, not let: removing a row is a state change that re-runs body. The empty state is derived from this array, not stored separately.

```
5  var body: some View {
```

WHO SwiftUI calls body to describe the list and, when needed, its empty state.
WHAT **Describes the list and, when needed, the empty state.**
WHERE Inside ShoppingListView, directly below the items state.
WHEN When the view appears, and again after every delete changes items.
WHY Both normal and empty screens come from the same state.

```
6  List {
```

WHO SwiftUI's List, which hosts the rows and the delete gesture.
WHAT **Opens the scrolling list that holds one row per shopping item.**
WHERE The root view of body; the overlay below is attached to it.
WHEN Built each time body runs, including after every delete.
WHY The List stays in place even when empty; the overlay sits on top.

```
7  ForEach(items, id: \.self) { Text($0) }
```

WHO SwiftUI's ForEach, which also supplies the row positions that onDelete needs.
WHAT **Creates one row per item name, using each name as its identity.**
WHERE The only content inside the List, one row per element of items.
WHEN Evaluated on each body run, so the rows always match the current items.
WHY onDelete is a ForEach modifier, so even a simple list wraps its rows in a ForEach to support swipe-to-delete.

```
8  .onDelete { items.remove(atOffsets: $0) }
```

WHO SwiftUI calls your closure after the user swipes a row and taps Delete.
WHAT **Removes the rows at the given positions ($0, an IndexSet of row numbers) from the array.**
WHERE Attached to the ForEach, not the List, because it deletes the rows ForEach produced.
WHEN Only when the user deletes a row, by a full swipe or by tapping the Delete button the swipe reveals.
WHY You change the data; SwiftUI sees the smaller array and animates the matching row away. Deleting the last item is what makes the empty state appear.

```
10  .overlay {
```

WHO SwiftUI, layering one view on top of another.
WHAT **Layers the content in the closure on top of the List, at the same size.**
WHERE Attached to the List, so the overlay takes the List's size.
WHEN Its closure is evaluated each time body runs.
WHY An overlay can appear and disappear without changing how the List itself is laid out.

```
11  if items.isEmpty {
```

WHO Swift, inside a view builder (the closure syntax SwiftUI uses to build views).
WHAT **Includes the empty state only when there are no items.**
WHERE Inside the overlay closure, wrapping the empty-state view.
WHEN Checked on every body run, which means after each delete.
WHY An if inside a view builder adds or removes views based on state, which is how conditional UI works in SwiftUI.

```
12  ContentUnavailableView(
```

WHO SwiftUI, providing the standard empty-state view used across iOS.
WHAT **Starts the system empty-state view, centered in the available space.**
WHERE Inside the if, so it exists only in the overlay's empty case.
WHEN Created only while items is empty; otherwise the overlay holds nothing.
WHY It matches the empty states in Apple's own apps, so people instantly recognize it.

```
13  "List Complete",
```

WHO You choose the words; SwiftUI shows them as the title.
WHAT **Gives the empty state its bold title, List Complete.**
WHERE First argument of ContentUnavailableView, the title.
WHEN Shown from the moment the last item is deleted.
WHY The title explains why the screen is empty, in positive words when the user caused it.

```
14  systemImage: "cart",
```

WHO SwiftUI draws the SF Symbol named cart above the title.
WHAT **Picks the cart symbol as the empty state's large icon.**
WHERE Second argument of ContentUnavailableView, labelled systemImage.
WHEN Displayed together with the title while the list is empty.
WHY An icon ties the empty state to the screen's purpose.

```
15  description: Text("Everything is in the cart.")
```

WHO SwiftUI, drawing the description in a secondary text style.
WHAT **A short line of secondary text under the title.**
WHERE Third argument of ContentUnavailableView; it takes a Text view, not a String.
WHEN Displayed under the title while the list is empty.
WHY One sentence of context is enough; a good empty state never leaves people guessing.

```
22  #Preview { ShoppingListView() }
```

WHO Xcode's preview canvas and this page's iPhone, where you can delete the items.
WHAT **Shows ShoppingListView with its three starting items.**
WHERE At file scope; the preview does not ship with the app.
WHEN Each time you edit the code, starting again from three items.
WHY Swipe every row away to see the empty state appear.

Try it

1. Add "Coffee" to the starting array, then delete it from the middle of the list.
2. Change the icon to "checkmark.circle" and the title to "All Done".
3. Start empty: change the state to @State private var items: [String] = [] and the empty state shows at once.

CODE · LIVE SWIFTUI PREVIEW

Custom swipe actions

`swipeActions` puts your own buttons behind a row. Swipe a habit to the left and tap Done or Undo: each button inserts the habit into, or removes it from, the `done` set, and the row's icon follows. Custom actions replace the automatic Delete from `onDelete`, so add a delete button yourself when you need one.

```swift
import SwiftUI

struct TodayView: View {
    let habits = ["Read 10 pages", "Walk after lunch", "Stretch"]
    @State private var done: Set<String> = []

    var body: some View {
        List(habits, id: \.self) { habit in
            Label(habit, systemImage: done.contains(habit)
                  ? "checkmark.circle.fill" : "circle")
                .swipeActions {
                    Button("Done") { done.insert(habit) }
                        .tint(.green)
                    Button("Undo") { done.remove(habit) }
                        .tint(.gray)
                }
        }
    }
}

#Preview { TodayView() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, which needs SwiftUI for Label and swipeActions.
- WHAT — **Brings in List, Label, Button, @State and the swipeActions modifier.**
- WHERE — First line of the habits file, covering TodayView and its preview.
- WHEN — At compile time, before TodayView is checked.
- WHY — Swipe actions are a standard SwiftUI list feature.

```
3  struct TodayView: View {
```

WHO Your code declares the habits screen; the preview creates it.
WHAT **Declares TodayView, a list of habits you mark done with a swipe.**
WHERE At file scope, as a screen of its own with its own state.
WHEN Created when the preview starts; the done set survives rebuilds.
WHY Marking items done with a swipe is quicker than opening each one.

```
4  let habits = ["Read 10 pages", "Walk after lunch", "Stretch"]
```

WHO Your code supplies the habit names; nothing on this screen edits them.
WHAT **Stores three habit names in a constant array.**
WHERE A stored constant of TodayView, next to the done state.
WHEN Created along with each TodayView value.
WHY The list itself never changes on this screen, so it is a let.

```
5  @State private var done: Set<String> = []
```

WHO SwiftUI stores the set; the two swipe buttons change it.
WHAT **An empty set (a collection with no duplicates and no order) of finished habit names.**
WHERE In SwiftUI-managed storage attached to TodayView.
WHEN Grows and shrinks as swipe buttons are tapped.
WHY Only the changing part is state. A set makes "is this habit done?" a fast, simple check.

```
7  var body: some View {
```

WHO SwiftUI calls body to describe every habit row.
WHAT **Describes the list for the current done set.**
WHERE Inside TodayView, below the done set it reads.
WHEN After every Done or Undo tap changes the done set.
WHY Each row's icon is derived from the set, not stored separately.

```
8  List(habits, id: \.self) { habit in
```

WHO SwiftUI's List, building one row per habit string.
WHAT **Creates a row per habit name, using the name itself as the identity.**
WHERE The root view returned by TodayView's body.
WHEN Built when body runs; rows are created as they appear on screen.
WHY Habit names are unique, so using each string as its own id (id: \.self) is safe here.

```
9  Label(habit, systemImage: done.contains(habit)
```

WHO SwiftUI's Label view, drawing an icon beside the habit name.
WHAT **Starts a row label whose icon depends on whether the habit is in the done set.**
WHERE Inside the List's row closure, once per habit.
WHEN Rebuilt on each body run, which follows every change to done.
WHY The row reads the set, so its look always reflects the data.

```
10 ? "checkmark.circle.fill" : "circle")
```

WHO Swift's ternary (?:) operator, choosing the symbol name.
WHAT **Picks a filled checkmark for done habits and an empty circle otherwise.**
WHERE The second line of the Label call, finishing its systemImage argument.
WHEN Evaluated for every row each time body runs.
WHY A shape change, not just a color change, shows status to everyone, including people who cannot tell colors apart.

```
11  .swipeActions {
```

WHO — SwiftUI attaches the buttons; the user reveals them by swiping the row.
WHAT — **Attaches custom buttons that appear when the row is swiped from the trailing (right) edge.**
WHERE — Attached to each row's Label, inside the row closure.
WHEN — The buttons appear only while the row is swiped open.
WHY — Keeps secondary actions out of sight until needed, so rows stay clean.

```
12  Button("Done") { done.insert(habit) }
```

WHO — The user tapping; your closure changing state.
WHAT — **Inserts this row's habit name into the done set, marking it finished.**
WHERE — First button, placed at the far edge of the row.
WHEN — When the user taps Done; the row then slides closed and redraws with a checkmark.
WHY — Inserting into a set twice has no effect, so repeated taps are harmless.

```
13  .tint(.green)
```

WHO — SwiftUI, using the tint as the swipe button's background color.
WHAT — **Colors the Done button's background green.**
WHERE — Attached to the Done button inside swipeActions.
WHEN — Applied each time the row's swipe buttons are built.
WHY — Swipe buttons take their background from tint; green signals a positive action.

```
14  Button("Undo") { done.remove(habit) }
```

WHO — The user taps it; your closure removes the habit from the set.
WHAT — **Removes this habit's name from done, so it shows an empty circle again.**
WHERE — Second button in swipeActions, placed just inside Done.
WHEN — When the user swipes the row open and taps Undo.
WHY — Every action should be reversible when possible.

```
15  .tint(.gray)
```

WHO — SwiftUI, painting the Undo button with the given tint.
WHAT — **Colors the Undo button's background gray.**
WHERE — Attached to the Undo button, the second swipe action.
WHEN — Applied whenever the swipe buttons for a row are built.
WHY — A neutral color marks the less important action.

```
21  #Preview { TodayView() }
```

WHO — Xcode's preview canvas, plus the simulated iPhone here, showing TodayView.
WHAT — **Shows today's three habits, each one ready to swipe.**
WHERE — At file scope; this preview exists for development only.
WHEN — Each time you edit the code, starting with no habits done.
WHY — Gestures are much easier to judge by trying them than by reading code.

> **Try it**
>
> 1. Swap the order of the two buttons. Which one now sits at the far edge?
> 2. Change .tint(.green) to .tint(.orange).
> 3. Add "Meditate" to the habits array and mark it done.

CODE · LIVE SWIFTUI PREVIEW

Filtering with searchable

`searchable` adds the system search field. The system places it in the navigation container, so the list sits inside a `NavigationStack`. The query is ordinary state; the visible rows come from a computed property

that filters the full array. Type “pa” into the phone’s search field and watch the list narrow.

```
import SwiftUI

struct RecipeSearchView: View {
    let recipes = ["Pancakes", "Tomato Soup", "Fried Rice",
                   "Pad Thai", "Pasta Salad", "Tacos"]
    @State private var query = ""
    var results: [String] {
        if query.isEmpty { return recipes }
        return recipes.filter {
            $0.lowercased().contains(query.lowercased())
        }
    }
    var body: some View {
        NavigationStack {
            List(results, id: \.self) { Text($0) }
                .navigationTitle("Recipes")
                .searchable(text: $query)
        }
    }
}

#Preview { RecipeSearchView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, which needs SwiftUI for the list, the navigation stack and the search field.
- **WHAT** **Brings in NavigationStack, List, the searchable modifier and @State.**
- **WHERE** First line of the recipe search file, before any declaration.
- **WHEN** At compile time, before RecipeSearchView is checked.
- **WHY** The search field is a standard SwiftUI component; you do not draw it yourself.

3 `struct RecipeSearchView: View {`

- **WHO** Your code declares the search screen; the preview creates it.
- **WHAT** **Declares RecipeSearchView, a recipe list with a search field.**
- **WHERE** At file scope, as a screen of its own type.
- **WHEN** Created when the preview starts; the query survives rebuilds.
- **WHY** Search becomes essential once a list is longer than one screen.

4 `let recipes = ["Pancakes", "Tomato Soup", "Fried Rice",`

- **WHO** Your code supplies the full recipe catalog for this screen.
- **WHAT** **Starts the full, unfiltered array of recipe names.**
- **WHERE** A stored constant of RecipeSearchView; the literal continues on the next line.
- **WHEN** Created along with each RecipeSearchView value.
- **WHY** The full list never changes; filtering produces a new array instead of editing this one.

5 `"Pad Thai", "Pasta Salad", "Tacos"]`

- **WHO** Your code, finishing the recipes array literal.
- **WHAT** **Finishes the array literal with three more names.**
- **WHERE** The second line of the recipes array, aligned under its first element.
- **WHEN** Evaluated as part of the same array literal, with the view.
- **WHY** Several names starting with "Pa" make the filter easy to test.

```
6  @State private var query = ""
```

WHO — SwiftUI stores it; the search field writes it.
WHAT — **The current search text, empty at first.**
WHERE — In SwiftUI-managed storage attached to RecipeSearchView.
WHEN — Changes on every keystroke in the search field.
WHY — The query is the only state here; the results are derived from it.

```
7  var results: [String] {
```

WHO — Swift runs this getter each time body reads results.
WHAT — **Begins a computed property that returns the recipes matching the query.**
WHERE — Inside RecipeSearchView, between the state and body; it stores nothing.
WHEN — Recomputed each time body runs, so after every keystroke.
WHY — Deriving results avoids a second array that could fall out of step with the query.

```
8  if query.isEmpty { return recipes }
```

WHO — Swift, running the first check of the results getter.
WHAT — **Returns every recipe when the search field is empty.**
WHERE — First line of the results computed property.
WHEN — Each time results is read, before any filtering happens.
WHY — An empty search should show everything, not nothing.

```
9  return recipes.filter {
```

WHO — Swift's filter method on Array, called on the full recipes list.
WHAT — **Starts building a new array containing only the names for which the closure returns true.**
WHERE — Second path of the results getter, reached only when query has text.
WHEN — Each time results is read with a non-empty query.
WHY — filter never changes the original array, so the full list is always available.

```
10 $0.lowercased().contains(query.lowercased())
```

WHO — Swift's String methods lowercased() and contains(_:).
WHAT — **Keeps a name when its lowercase form contains the lowercase query.**
WHERE — The body of the filter closure; $0 is the current name.
WHEN — Once per recipe, each time results is read with a non-empty query.
WHY — Lowercasing both sides makes "pa" match "Pad Thai": people rarely type capitals in search.

```
13 var body: some View {
```

WHO — SwiftUI calls body to describe the searchable recipe screen.
WHAT — **Describes a navigation stack holding the filtered list and its search field.**
WHERE — Inside RecipeSearchView, below the results it reads.
WHEN — After every keystroke, because each one changes query.
WHY — Every keystroke changes state, re-runs body, and so re-filters the rows.

```
14 NavigationStack {
```

WHO — SwiftUI's NavigationStack, which provides the title bar and a place for the search field.
WHAT — **Wraps the list in a navigation container with a title bar.**
WHERE — The root view returned by body, wrapped around the List.
WHEN — Built when body runs; the stack stays in place as the query changes.
WHY — The system places the search field in the navigation container, so searchable needs a NavigationStack around the list to show it.

```
15  List(results, id: \.self) { Text($0) }
```

WHO — SwiftUI's List, showing the derived results.
WHAT — **Shows one row per name in results, using each string as its identity.**
WHERE — Inside the NavigationStack, as its root content.
WHEN — Rebuilt with the new results after each keystroke.
WHY — The list reads the derived array, so it always matches the query.

```
16  .navigationTitle("Recipes")
```

WHO — SwiftUI, passing the title up to the enclosing NavigationStack.
WHAT — **Sets the screen's title to Recipes, shown large at the top of the list.**
WHERE — Applied to the List, the stack's content.
WHEN — Applied when the List is built inside the stack.
WHY — A clear title tells people which screen they are on and what the search will look through.

```
17  .searchable(text: $query)
```

WHO — SwiftUI draws the search field; the user types into it.
WHAT — **Adds the system search field and binds its text to query.**
WHERE — Applied to the List; iOS decides where the field appears in the navigation container.
WHEN — Writes query on every keystroke, which re-runs body and re-filters the rows.
WHY — A binding means the field and the filter share one source of truth.

```
22  #Preview { RecipeSearchView() }
```

WHO — Xcode's preview canvas and this page's phone, where the search field accepts typing.
WHAT — **Shows RecipeSearchView with all six recipes before any search.**
WHERE — At file scope; no preview code ships in the app.
WHEN — Each time you edit the code, starting with an empty search.
WHY — Typing real queries is the fastest way to check the filter.

> **Try it**
>
> 1. Add three more recipes to the array and search for one of them.
> 2. Make the filter match only names that start with the query: use hasPrefix instead of contains.

CHECK YOURSELF

Check your understanding

1. Why does SwiftUI need list elements to have a stable identity?
   A. To sort the rows alphabetically before SwiftUI displays them on screen
   B. To match old rows to new ones, so deletes and moves hit the right row
   C. To save the rows to disk automatically whenever the data changes
   D. To make List scroll faster than a ScrollView holding the same rows

2. A list uses `ForEach(names, id: \.self)`, and people can add the same name twice, such as two contacts called “Sam”. What can go wrong?
   A. Nothing; SwiftUI numbers duplicate values automatically
   B. The code stops compiling, because \.self requires unique values
   C. Both rows share one identity, so a delete or edit can hit the wrong Sam
   D. The second Sam is hidden for good, because its id is already taken

3. Rows already use `.onDelete`. You add `.swipeActions { Button("Pin") { } }` to each row. What happens to the swipe-to-delete button?
   A. Your custom actions replace it, so add a Delete button of your own
   B. Delete and Pin both appear, because SwiftUI merges the two sets of actions
   C. The app crashes when a row is swiped, because two actions conflict
   D. Delete moves to the leading edge to make room for Pin on the trailing edge

4. Which design for a search screen keeps a single source of truth?
   A. Store recipes plus a second @State array, filteredRecipes, and update it on every keystroke
   B. Remove non-matching items from the recipes array as the user types
   C. Copy the recipes into the search field
   D. Store recipes and query, and compute the results with a computed property

Answers: Lists and Dynamic Data

1. **B. To match old rows to new ones, so deletes and moves hit the right row** Identity is how SwiftUI matches an old row to a new one after the data changes. Sorting and saving are your code's job; identity only answers "is this the same item as before?"
2. **C. Both rows share one identity, so a delete or edit can hit the wrong Sam** `id: \.self` uses the value as its identity, so two equal values have the same identity. It compiles fine, which is what makes the bug sneaky. Give such data a model type with `let id = UUID()`.
3. **A. Your custom actions replace it, so add a Delete button of your own** Once you define custom swipe actions, SwiftUI stops adding the automatic Delete. Expecting both to appear is the common surprise; the fix is a Delete button inside swipeActions, tinted red.
4. **D. Store recipes and query, and compute the results with a computed property** The query is the only thing that changes; results follow from it. A second filtered array is the duplicated-state bug in disguise, and removing items from the original array loses data the moment the query changes back.

Navigation, Tabs, and Sheets

CONCEPT

Three ways to move between screens

Most apps have more than one screen, and iOS gives people a few familiar ways to move between them. This lesson shows how to code each one. Choosing between them is a design decision, and Module 05 returns to that choice with real product scenarios.

Hierarchical (push)	Drill down from a list to a detail and back. NavigationStack slides each new screen in from the side and adds a back button.
Flat (tabs)	Switch between a few peer sections of equal importance, such as Today, Stats and Settings. TabView keeps each section in place while you visit another.
Modal (sheet)	Interrupt the current task for a short, self-contained job, like adding a habit. A sheet slides up and goes away when the job is done.
Alert	Ask a quick question that must be answered now, such as confirming a destructive action. Use alerts rarely so people keep reading them.

> **Match the pattern to the relationship**
>
> Ask how two screens relate. Parent and child? Push. Peers? Tabs. A side task that ends? Sheet. A yes-or-no that cannot wait? Alert. A whole feature trapped inside a sheet, or peers pushed on top of each other, makes an app feel like a maze.

CODE · LIVE SWIFTUI PREVIEW

NavigationStack and NavigationLink

A recipe book with two recipes. `NavigationStack` gives the screen a navigation bar and remembers which screens are pushed. Each `NavigationLink` is a row that pushes its destination when tapped. Tap a recipe in the phone, then use the back button.

```swift
import SwiftUI

struct RecipeBookView: View {
    var body: some View {
        NavigationStack {
            List {
                NavigationLink("Pancakes") {
                    Text("Whisk, pour, flip, serve.")
                        .navigationTitle("Pancakes")
                }
                NavigationLink("Tomato Soup") {
                    Text("Roast, simmer, blend.")
                        .navigationTitle("Tomato Soup")
                }
            }
            .navigationTitle("Recipes")
        }
    }
}

#Preview { RecipeBookView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the app.
- **WHAT** **Brings in NavigationStack, NavigationLink, List and the navigationTitle modifier used in this example.**
- **WHERE** Top of the file; the import covers this one source file only.
- **WHEN** At compile time, before the recipe list or any navigation code runs.
- **WHY** Navigation containers belong to the SwiftUI framework, not to the Swift language, so the file must import them.

3 `struct RecipeBookView: View {`

- **WHO** Your code declares the type; SwiftUI creates and displays its values.
- **WHAT** **Declares the root screen of a small recipe book app.**
- **WHERE** At file scope, so the preview (or an app's WindowGroup) can create it.
- **WHEN** Compiled once; SwiftUI creates a value whenever it needs to show this screen.
- **WHY** Keeping the whole navigation flow inside one root view makes it easy to see where navigation starts.

4 `var body: some View {`

- **WHO** SwiftUI calls it; you never call body yourself.
- **WHAT** **Returns the description of this screen: a navigation stack that contains a list.**
- **WHERE** Inside RecipeBookView, on the main thread.
- **WHEN** When the view first appears and whenever its inputs change. This view has no state, so that is rare.
- **WHY** Even navigation is declared, not commanded: you describe which links exist and SwiftUI handles the pushing.

```
5  NavigationStack {
```

WHO — SwiftUI's navigation system, which owns the stack of screens for this recipe book.

WHAT — **Creates a container that shows a navigation bar and keeps a stack of pushed screens, with the list as its root.**

WHERE — Wraps everything on this screen; only views inside it can push.

WHEN — Created once when the screen appears; the stack grows on each push and shrinks on each back tap.

WHY — Push navigation needs one owner that remembers the history. A NavigationLink with no NavigationStack around it has nowhere to push, so it shows as disabled and tapping it does nothing.

```
6  List {
```

WHO — SwiftUI, drawing a scrolling list in the iOS style.

WHAT — **Shows its children as rows in a scrollable list.**

WHERE — The root screen inside the NavigationStack.

WHEN — Laid out when the screen appears and again whenever body runs.

WHY — Lists are the natural home for links: each link row gets a chevron, which tells people that tapping goes deeper.

```
7  NavigationLink("Pancakes") {
```

WHO — You declare it; the user triggers it by tapping the row.

WHAT — **Creates a row titled Pancakes whose trailing closure describes the screen to push.**

WHERE — First row of the List, so Pancakes is the top recipe in the book.

WHEN — The row appears right away; SwiftUI displays the destination only after a tap.

WHY — The link pairs a label with its destination, so the relationship between list and detail is visible in one place in the code.

```
8  Text("Whisk, pour, flip, serve.")
```

WHO — SwiftUI, which builds this Text only for the pushed Pancakes screen.

WHAT — **The content of the Pancakes detail screen.**

WHERE — Inside the link's destination closure, so it appears on a new screen, not in the list.

WHEN — Shown after the user taps the Pancakes row.

WHY — A one-line destination keeps the example focused; in a real app this would be its own view, such as RecipeDetailView.

```
9  .navigationTitle("Pancakes")
```

WHO — The NavigationStack, which reads the title of whichever screen is on top.

WHAT — **Sets the title shown in the navigation bar while the detail screen is visible.**

WHERE — Attached to the destination's content, not to the stack itself.

WHEN — Applied when the detail screen is pushed; the back button then shows the previous screen's title.

WHY — Each screen names itself. Because titles travel with their screens, the bar is always correct after any push or pop.

```
11 NavigationLink("Tomato Soup") {
```

WHO — You declare the second link; the user triggers it by tapping the Tomato Soup row.

WHAT — **A second row that pushes a different destination.**

WHERE — Second row of the List, directly under the Pancakes link.

WHEN — Drawn with the list; its destination appears only after a tap.

WHY — Each link owns its own destination. That is simple for a few fixed rows but repetitive for many, which the next example fixes.

```
12  Text("Roast, simmer, blend.")
```

WHO — SwiftUI, which builds this Text only for the pushed Tomato Soup screen.
WHAT — **The body text of the soup detail screen.**
WHERE — Inside the second link's destination closure.
WHEN — Visible after the user taps Tomato Soup.
WHY — Notice the near-duplicate of the first link; repetition like this is the signal to switch to data-driven navigation.

```
13  .navigationTitle("Tomato Soup")
```

WHO — The NavigationStack, reading the top screen's title.
WHAT — **Names the soup screen in the navigation bar.**
WHERE — Attached to the soup destination's Text, inside the second link's closure.
WHEN — Applied each time the soup screen is pushed, and kept while it stays on top.
WHY — Titles tell people where they are; on this screen the back button reads Recipes, the title of the screen below.

```
16  .navigationTitle("Recipes")
```

WHO — The NavigationStack, which shows the title of its root screen.
WHAT — **Gives the list screen the large title Recipes.**
WHERE — Attached to the List, the root screen inside the stack, not to NavigationStack itself.
WHEN — Shown while the list is on top; after a push it becomes the back button's label.
WHY — The rule is "modifier on the content, not on the container": the stack asks each screen for its own title.

```
21  #Preview { RecipeBookView() }
```

WHO — Xcode's preview canvas, and the simulated iPhone beside this code, both running RecipeBookView.
WHAT — **Shows the recipe book live, so you can tap rows and go back.**
WHERE — At file scope, below RecipeBookView; the preview is a development tool and is left out of the shipped app.
WHEN — Every time you change the code; the preview restarts at the Recipes list with nothing pushed.
WHY — Navigation is easiest to understand by doing it: tap, push, go back.

> **Try it**
>
> 1. Add a third NavigationLink named "Green Salad" with its own Text and title.
> 2. Wrap the Pancakes Text in a VStack and add a second Text("Serves 4") below it.

CODE · LIVE SWIFTUI PREVIEW

Navigate with values

Instead of building each destination inside its link, a link can carry a *value*, and one `navigationDestination(for:)` turns any value of that type into a screen. The rows now come from data, so adding a recipe means adding one enum case, not copying a link. Tap a row in the phone.

```swift
import SwiftUI

enum Recipe: String, CaseIterable {
    case pancakes = "Pancakes", soup = "Tomato Soup"
}

struct RecipeListView: View {
    var body: some View {
        NavigationStack {
            List {
                ForEach(Recipe.allCases, id: \.self) { recipe in
                    NavigationLink(recipe.rawValue, value: recipe)
                }
            }
            .navigationDestination(for: Recipe.self) { recipe in
                Text("How to make \(recipe.rawValue)")
            }
        }
    }
}

#Preview { RecipeListView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the data-driven recipe list.
- **WHAT** **Makes NavigationStack, NavigationLink(value:) and the navigationDestination modifier available in this file.**
- **WHERE** First line of the file; it applies to the Recipe enum and the view below it.
- **WHEN** At compile time, before the enum or the view is type-checked.
- **WHY** Value-based navigation is part of SwiftUI on every iOS version this app supports, so no extra framework is needed.

3 `enum Recipe: String, CaseIterable {`

- **WHO** Your code defines the type; the compiler adds the conformances.
- **WHAT** **Declares the recipes as an enumeration with String raw values and an automatic list of all its cases.**
- **WHERE** At file scope, so both the list and the destination rule can use it.
- **WHEN** At compile time; Swift generates allCases and Hashable support.
- **WHY** A navigation value must be Hashable so the stack can store and compare it. Enums without associated values get Hashable for free; a struct would declare : Hashable.

4 `case pancakes = "Pancakes", soup = "Tomato Soup"`

- **WHO** Your code lists the cases; the compiler stores each raw value string.
- **WHAT** **Defines two cases, each with its display name as the raw value.**
- **WHERE** Inside the Recipe enum, as its complete list of cases.
- **WHEN** Fixed at compile time; no recipe can be added while the app runs.
- **WHY** Using the display text as the raw value gives every row a title without a separate lookup table.

```
7  struct RecipeListView: View {
```

WHO Your code declares it; SwiftUI creates it.
WHAT **The root screen: a list of recipes that pushes a detail screen.**
WHERE At file scope, beside the Recipe enum it depends on, so the preview can create it.
WHEN Compiled once; created whenever SwiftUI needs to show it.
WHY The view stores no recipe data of its own; it reads Recipe.allCases, which keeps data and screen separate.

```
8  var body: some View {
```

WHO SwiftUI calls body; your code only describes the stack, the rows and the destination rule.
WHAT **Describes the stack, its rows of links, and the rule that turns a Recipe into a screen.**
WHERE Inside RecipeListView, on the main thread.
WHEN On first appearance and whenever its inputs change.
WHY Navigation rules live in body, right next to the views they serve.

```
9  NavigationStack {
```

WHO SwiftUI's navigation system, which stores pushed Recipe values in order.
WHAT **Holds the list as its root and a stack of pushed values above it.**
WHERE The outermost view in body, so the List and the destination rule both live inside it.
WHEN Created on appearance; a value is added on each tap and removed on each back tap.
WHY The stack is essentially an array of values, which is why the values must be Hashable.

```
10 List {
```

WHO SwiftUI, drawing the recipe rows in the standard grouped list style.
WHAT **Displays the recipe rows in a scrolling list.**
WHERE The root screen inside the NavigationStack, at the bottom of the stack.
WHEN Laid out on appearance and on every body run.
WHY Rows that hold links get chevrons automatically, signaling that they lead somewhere.

```
11 ForEach(Recipe.allCases, id: \.self) { recipe in
```

WHO SwiftUI, which repeats the closure once per item.
WHAT **Creates one row for each case in Recipe.allCases, using the case itself as the row's identity.**
WHERE Inside the List, which turns each repeated NavigationLink into a row.
WHEN Each time body runs; enum cases never change while the app runs, so the rows stay stable.
WHY id: \.self says "each value identifies itself", which is safe here because every enum case is unique.

```
12 NavigationLink(recipe.rawValue, value: recipe)
```

WHO You declare it; the user taps it; the NavigationStack receives the value.
WHAT **Shows the recipe name and, when tapped, pushes this Recipe value onto the stack.**
WHERE Inside the ForEach closure, so one link exists for each Recipe case.
WHEN The push happens on tap; no destination view exists before that.
WHY The link only says WHAT was chosen, not HOW to show it. That split lets one destination rule serve every link of the same type.

```
15 .navigationDestination(for: Recipe.self) { recipe in
```

WHO The NavigationStack, which calls this closure whenever a Recipe is pushed.
WHAT **Registers how to turn any pushed Recipe value into a screen.**
WHERE Attached to the List, inside the NavigationStack; outside the stack it would never be found.
WHEN The closure runs only when a Recipe is pushed, once per pushed value.
WHY Destinations are built on demand and defined once, instead of once per row.

```
16  Text("How to make \(recipe.rawValue)")
```

WHO	SwiftUI, which builds this Text as the content of the pushed screen.
WHAT	**The detail screen, showing the name of the recipe that was tapped.**
WHERE	Inside the destination closure, so it appears as a new screen.
WHEN	Built each time a Recipe value is pushed, using that value's raw name.
WHY	The closure receives the exact value from the link, so one rule produces the right screen for every recipe.

```
22  #Preview { RecipeListView() }
```

WHO	Xcode's preview canvas, and the simulated iPhone beside this code, running RecipeListView.
WHAT	**Shows the data-driven recipe list so you can push and pop recipes.**
WHERE	At file scope; it is not part of the shipped app.
WHEN	Each time the code changes; the preview starts again from the list with an empty stack.
WHY	Seeing the same destination rule handle every row makes the value-based idea concrete.

Try it

1. Add .navigationTitle("Recipes") on the line after the List's closing brace.
2. Add a third case, salad = "Green Salad". The list and the navigation both pick it up with no other change.

INTERACTIVE

Inside a NavigationStack

A NavigationStack is driven by an array of values called `path`. The root screen sits at the bottom and is never in `path`; each pushed value adds a screen on top. Push a few screens and watch `path` grow; go back and watch it shrink. Open the sheet with + and notice that `path` does not change. Then try **Jump**, which replaces the whole array at once, the way a deep link does. Turn on **Predict first** to call each result before it happens.

This step is an interactive simulation in the HTML version.

CODE · LIVE SWIFTUI PREVIEW

Tabs for peer sections

A habit app with three peer sections. `TabView` draws a tab bar, and each `Tab` pairs a title and an SF Symbol with the content it shows. In iOS 26 and later the tab bar floats on Liquid Glass. Tap the tabs in the phone and notice the badge on Today.

```swift
import SwiftUI

struct HabitAppView: View {
    var body: some View {
        TabView {
            Tab("Today", systemImage: "checklist") {
                Label("3 habits left", systemImage: "flame")
            }
            .badge(3)
            Tab("Stats", systemImage: "chart.bar") {
                Label("Best streak: 12 days", systemImage: "star")
            }
            Tab("Settings", systemImage: "gear") {
                Label("Reminders on", systemImage: "bell")
            }
        }
        .tint(.green)
    }
}

#Preview { HabitAppView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the habit tracker.
- **WHAT** **Brings in TabView, Tab, Label and the badge and tint modifiers.**
- **WHERE** Top of the file; one import covers HabitAppView and all three tabs.
- **WHEN** At compile time, before HabitAppView is type-checked.
- **WHY** The Tab type is the current SwiftUI API for tabs; older tutorials use .tabItem, which you may still meet in existing code.

3 `struct HabitAppView: View {`

- **WHO** Your code declares the type; SwiftUI shows it as the app's top-level screen.
- **WHAT** **The top-level screen of a habit tracker.**
- **WHERE** At file scope; in a full app, WindowGroup would show this view first.
- **WHEN** Created when the app or preview launches.
- **WHY** Tabs belong at the very top of an app's structure, never pushed inside a navigation stack.

4 `var body: some View {`

- **WHO** SwiftUI calls body to learn which tabs exist and what each one shows.
- **WHAT** **Describes a tab bar with three tabs and their content.**
- **WHERE** Inside HabitAppView, on the main thread.
- **WHEN** On first appearance and whenever inputs change.
- **WHY** The whole tab structure is visible in one place, which makes the app's top-level map easy to review.

5 `TabView {`

- **WHO** SwiftUI and iOS, which draw the system tab bar.
- **WHAT** **Creates a container that shows one tab's content at a time plus a bar to switch tabs.**
- **WHERE** The root of this view, filling the screen.
- **WHEN** Created on appearance; switching tabs changes which content is visible while the others keep their state.
- **WHY** Peers deserve equal access: one tap reaches any section, and each section keeps its place while you are away.

```
6  Tab("Today", systemImage: "checklist") {
```

WHO You declare it; the user selects it by tapping its item.
WHAT **Defines the first tab: its title, its SF Symbol, and the content in the closure.**
WHERE First child of the TabView, so it is the leftmost item and selected at launch.
WHEN Its item appears right away; its content shows while the tab is selected.
WHY A short noun plus a recognizable symbol makes tabs easy to scan.

```
7  Label("3 habits left", systemImage: "flame")
```

WHO SwiftUI, drawing a Label with the flame symbol beside its text.
WHAT **The content of the Today tab: an icon and text side by side.**
WHERE Inside the Today tab's closure, as that tab's entire content.
WHEN Visible whenever the Today tab is selected, which is the case at launch.
WHY A real app would put a NavigationStack here; each tab usually owns its own stack, so drilling down in one tab never disturbs another.

```
9  .badge(3)
```

WHO iOS, which draws a small badge on the tab item.
WHAT **Shows the number 3 in a small red badge on the Today tab item.**
WHERE Applied to the Tab itself, not to its content.
WHEN Visible at all times, even while another tab is selected.
WHY Badges draw attention to a section that needs action. Reserve them for counts people care about, or they turn into noise.

```
10 Tab("Stats", systemImage: "chart.bar") {
```

WHO You declare the Stats tab; the user selects it by tapping its bar chart item.
WHAT **A second tab for streaks and statistics, with a bar chart symbol.**
WHERE Second child of the TabView, so it sits in the middle.
WHEN Its content is shown when the user taps Stats.
WHY Statistics are a peer of the daily list, not a child of it, so a tab fits better than a pushed screen.

```
11 Label("Best streak: 12 days", systemImage: "star")
```

WHO SwiftUI, drawing a Label with the star symbol beside its text.
WHAT **Placeholder content for the Stats tab: the best streak, shown with a star.**
WHERE Inside the Stats tab's closure, as that tab's only content.
WHEN Visible while the Stats tab is selected, after the user taps it.
WHY Simple content keeps the focus on the tab structure itself.

```
13 Tab("Settings", systemImage: "gear") {
```

WHO You declare the Settings tab; the user selects it by tapping the gear item.
WHAT **A third tab for settings, with a gear symbol.**
WHERE Last child of the TabView, so it appears at the trailing end.
WHEN Its content is shown when the user taps Settings.
WHY Settings conventionally goes last, where people expect it.

```
14 Label("Reminders on", systemImage: "bell")
```

WHO SwiftUI, drawing a Label with the bell symbol beside its text.
WHAT **Placeholder content for the Settings tab.**
WHERE Inside the Settings tab's closure, as that tab's only content.
WHEN Visible only while the Settings tab is selected.
WHY A real Settings tab would hold a Form of toggles and pickers, which the Forms lesson builds.

```
17  .tint(.green)
```

WHO — SwiftUI, passing the tint down to the tab bar.
WHAT — **Colors the selected tab item green instead of the default blue.**
WHERE — On the TabView, so it affects its tab bar and the controls inside it.
WHEN — Applied whenever the TabView is rebuilt.
WHY — One accent color, used consistently, becomes part of an app's identity.

```
21  #Preview { HabitAppView() }
```

WHO — Xcode's preview canvas, and the simulated iPhone beside this code, running HabitAppView.
WHAT — **Shows the tabbed app so you can switch sections.**
WHERE — At file scope, outside HabitAppView; preview code is left out of the shipped app.
WHEN — Every time the code changes; the preview relaunches with Today selected.
WHY — Tapping between tabs shows how flat navigation feels compared with pushing.

Try it

1. Change .badge(3) to .badge(12), then move it onto the Stats tab.
2. Add a fourth Tab("Profile", systemImage: "person") with a Text inside.
3. Change .green to .orange.

CODE · LIVE SWIFTUI PREVIEW

A sheet for a short task

An Add button in the navigation bar presents a *sheet*: a modal screen for a short, self-contained task. A Boolean in `@State` decides whether the sheet is showing. Inside the sheet, `@Environment(\.dismiss)` gives the sheet a way to close itself. Tap Add, then Save habit.

```swift
import SwiftUI

struct HabitListView: View {
    @State private var showingAdd = false
    var body: some View {
        NavigationStack {
            Text("Read 20 pages")
                .navigationTitle("Habits")
                .toolbar { Button("Add") { showingAdd = true } }
                .sheet(isPresented: $showingAdd) { AddHabitView() }
        }
    }
}

struct AddHabitView: View {
    @Environment(\.dismiss) private var dismiss
    var body: some View {
        Button("Save habit") { dismiss() }
    }
}

#Preview { HabitListView() }
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, while it builds both views in this file.
WHAT	**Brings in NavigationStack, the toolbar and sheet modifiers, and the dismiss environment value.**
WHERE	Top of the file, covering HabitListView and AddHabitView.
WHEN	At compile time, before either view is type-checked.
WHY	Presentation is part of SwiftUI, so this one import covers both screens in the file.

```
3  struct HabitListView: View {
```

WHO	Your code declares the parent screen; SwiftUI creates it and shows it first.
WHAT	**The parent screen, which owns the decision of whether the sheet is showing.**
WHERE	At file scope, so the preview can create it as the starting screen.
WHEN	Created when the preview launches; it stays on screen beneath the sheet.
WHY	The view that presents a sheet owns the state that controls it.

```
4  @State private var showingAdd = false
```

WHO	SwiftUI stores it; this view and the sheet modifier read it.
WHAT	**Remembers whether the add-habit sheet is on screen, starting hidden.**
WHERE	In SwiftUI-managed storage tied to HabitListView.
WHEN	Created when the view first appears; it becomes true on tap and false again when the sheet closes.
WHY	Presentation is state like any other: the sheet is visible because a value says so, not because code ran a "present" command.

```
5  var body: some View {
```

WHO	SwiftUI calls body each time showingAdd or another input changes.
WHAT	**Describes the screen, its toolbar button, and the sheet it can present.**
WHERE	Inside HabitListView, on the main thread.
WHEN	On first appearance and every time showingAdd changes.
WHY	Because the sheet is described in body, its existence always matches the state.

```
6  NavigationStack {
```

WHO	SwiftUI's navigation system, used here for its bar rather than for pushing.
WHAT	**Provides a navigation bar for the title and the toolbar button.**
WHERE	The outermost view in HabitListView's body.
WHEN	Created when the screen first appears and kept while the sheet comes and goes.
WHY	Toolbar items in the top bar need a navigation bar to live in, even when nothing is ever pushed.

```
7  Text("Read 20 pages")
```

WHO	SwiftUI, drawing a single line of placeholder text.
WHAT	**Placeholder content standing in for the list of habits.**
WHERE	The root content inside the NavigationStack, centered on the screen.
WHEN	Drawn on every body run, including when showingAdd changes.
WHY	A real screen would show a List here; the sheet works the same either way.

```
8  .navigationTitle("Habits")
```

WHO	The NavigationStack, which reads the title from its root content.
WHAT	**Shows Habits as the large title in the navigation bar.**
WHERE	Attached to the Text, the root content of the stack.
WHEN	Shown while this screen is visible, including behind the open sheet.
WHY	A title tells people where the new habit will end up.

```
9  .toolbar { Button("Add") { showingAdd = true } }
```

WHO — SwiftUI places the button; the user taps it.
WHAT — **Adds an Add button to the navigation bar; tapping it sets showingAdd to true.**
WHERE — In the navigation bar, on the trailing side by default on iPhone.
WHEN — The button appears with the screen; its action runs on tap.
WHY — The action only changes state. To choose a position explicitly, wrap the button in ToolbarItem(placement: .topBarTrailing).

```
10 .sheet(isPresented: $showingAdd) { AddHabitView() }
```

WHO — SwiftUI, which watches the showingAdd binding and presents the sheet.
WHAT — **Presents AddHabitView as a sheet whenever showingAdd is true.**
WHERE — Attached to the screen's content; the sheet slides up over everything.
WHEN — As soon as showingAdd becomes true. When the sheet closes, SwiftUI sets showingAdd back to false through the binding.
WHY — The $ creates a binding, a two-way connection, so the sheet can both read the flag and reset it when it closes. One value, no drift.

```
15 struct AddHabitView: View {
```

WHO — Your code declares it; SwiftUI creates it inside the sheet.
WHAT — **The content of the sheet: a small screen for adding a habit.**
WHERE — At file scope; it knows nothing about who presented it.
WHEN — Created each time the sheet is presented.
WHY — A separate view keeps the sheet reusable, and gives it its own environment, where dismiss lives.

```
16 @Environment(\.dismiss) private var dismiss
```

WHO — SwiftUI supplies the value from the environment.
WHAT — **Reads an action that closes whatever presented this view, here the sheet.**
WHERE — Inside the sheet's own view; it must be read here, not in the parent.
WHEN — Supplied when the view appears; calling it closes the sheet.
WHY — The sheet does not need a binding from its parent to close itself, so it stays independent of whoever presented it.

```
17 var body: some View {
```

WHO — SwiftUI calls AddHabitView's body when the sheet needs its content.
WHAT — **Describes the sheet's content: a single Save habit button.**
WHERE — Inside AddHabitView, the view that the sheet presents.
WHEN — When the sheet is presented, and again if its inputs change.
WHY — A sheet's content is an ordinary view, built with the same tools as any screen.

```
18 Button("Save habit") { dismiss() }
```

WHO — The user taps it; the dismiss action closes the sheet.
WHAT — **Calls dismiss, which closes the sheet and, through the binding, resets showingAdd to false.**
WHERE — The only view in the sheet's body, centered in the sheet.
WHEN — Runs when the user taps Save habit inside the open sheet.
WHY — A real form would save the new habit first, then dismiss. dismiss is a value you call like a function.

```
22 #Preview { HabitListView() }
```

WHO — Xcode's preview canvas, and the simulated iPhone beside this code, running HabitListView.
WHAT — **Shows the parent screen; the sheet appears when you tap Add.**
WHERE — At file scope, below both views; preview code is not part of the shipped app.
WHEN — Every time the code changes; the preview restarts with the sheet closed.
WHY — Opening and closing the sheet yourself shows the state round trip.

Try it

1. Swipe the sheet down instead of tapping Save habit, then tap Add again. It opens, which proves showingAdd was reset to false.
2. Wrap the Text in a VStack and add a Button("Add from here") { showingAdd = true }. Both buttons open the same sheet because they change the same state.

CODE · LIVE SWIFTUI PREVIEW

Confirm before destroying data

Resetting a streak destroys data, so the app asks first. `.alert` shows a system alert whenever its Boolean binding is true. The buttons inside describe the choices, and tapping any of them also closes the alert. Tap Reset streak, then try both buttons.

```swift
import SwiftUI

struct StreakView: View {
    @State private var streak = 12
    @State private var confirmingReset = false
    var body: some View {
        VStack(spacing: 20) {
            Text("\(streak)-day streak")
                .font(.largeTitle)
            Button("Reset streak") { confirmingReset = true }
                .tint(.red)
        }
        .alert("Reset your streak?", isPresented: $confirmingReset) {
            Button("Reset") { streak = 0 }
            Button("Cancel") { }
        } message: {
            Text("Your \(streak) days go back to zero.")
        }
    }
}

#Preview { StreakView() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds StreakView.
- WHAT — **Brings in the alert modifier along with VStack, Text and Button.**
- WHERE — Top of the file; it covers StreakView and its preview.
- WHEN — At compile time, before StreakView is type-checked.
- WHY — Alerts are standard system UI that SwiftUI exposes as a modifier.

3 `struct StreakView: View {`

- WHO — Your code declares the screen; SwiftUI creates and shows it.
- WHAT — **A screen that shows a habit streak and lets the person reset it.**
- WHERE — At file scope, so the preview can create StreakView directly.
- WHEN — Created when the preview launches, with a 12-day streak.
- WHY — Small views with one job are easy to reason about and to test.

```
4  @State private var streak = 12
```

WHO — SwiftUI stores it; the alert's button changes it.
WHAT — **The current streak length in days, the data at risk.**
WHERE — In SwiftUI-managed storage tied to StreakView, so it survives body re-runs.
WHEN — Created on first appearance; changed only if the person confirms.
WHY — It is the single source of truth for the number on screen and in the alert message.

```
5  @State private var confirmingReset = false
```

WHO — SwiftUI stores it; the alert modifier reads and resets it.
WHAT — **Whether the confirmation alert is visible.**
WHERE — Beside streak, in the same storage SwiftUI keeps for StreakView.
WHEN — Set to true by the button; set back to false by SwiftUI when the alert closes.
WHY — Naming state after the situation ("confirming reset") makes the code read like a sentence.

```
6  var body: some View {
```

WHO — SwiftUI calls body again whenever streak or confirmingReset changes.
WHAT — **Describes the streak, the reset button and the alert.**
WHERE — Inside StreakView, on the main thread, below its two state properties.
WHEN — On first appearance and whenever streak or confirmingReset changes.
WHY — The alert is part of the description, so it appears exactly when the state says it should.

```
7  VStack(spacing: 20) {
```

WHO — SwiftUI's layout system, stacking the streak above the reset button.
WHAT — **Stacks the streak and the button vertically with 20 points between them.**
WHERE — The root of StreakView's body; the alert is attached to it.
WHEN — Laid out each time body runs, such as right after the streak resets.
WHY — Space separates the information from the action that changes it.

```
8  Text("\(streak)-day streak")
```

WHO — SwiftUI, drawing the streak text from current state.
WHAT — **Shows the streak, for example "12-day streak".**
WHERE — Top child of the VStack, above the reset button.
WHEN — Rebuilt whenever streak changes, so it shows 0 right after a reset.
WHY — Reading state directly means the number can never be stale.

```
9  .font(.largeTitle)
```

WHO — SwiftUI, which maps the text style to a size using Dynamic Type.
WHAT — **Uses the large title text style, the biggest standard style.**
WHERE — Modifies only the streak Text directly above it.
WHEN — Each time the streak Text is rebuilt during a body run.
WHY — The streak is the most important fact on the screen, so it gets the largest style.

```
10 Button("Reset streak") { confirmingReset = true }
```

WHO — The user taps it; the action writes state.
WHAT — **Asks for confirmation by setting confirmingReset to true. It does not reset anything.**
WHERE — Second child of the VStack, below the streak text.
WHEN — Each time the user taps Reset streak; the alert then appears.
WHY — Destructive work moves into the alert, so a stray tap here costs nothing.

11 `.tint(.red)`

WHO SwiftUI, which passes the tint color to the button's label.
WHAT **Colors the Reset streak button's title red instead of the default blue.**
WHERE Attached to the Reset streak button only.
WHEN Applied whenever the button is rebuilt during a body run.
WHY Red signals danger before the tap. Buttons inside an alert usually declare role: .destructive instead, which tells iOS to style them as dangerous for you.

13 `.alert("Reset your streak?", isPresented: $confirmingReset) {`

WHO SwiftUI and iOS, which draw the standard alert.
WHAT **Shows an alert with this title while confirmingReset is true; the closure lists its buttons.**
WHERE Attached to the VStack; the alert appears centered over the whole screen.
WHEN Whenever confirmingReset becomes true; SwiftUI sets it back to false after any button is tapped.
WHY Phrase the title as a question that names the action, so people know exactly what they are agreeing to.

14 `Button("Reset") { streak = 0 }`

WHO The user chooses it; the action writes state.
WHAT **The confirming choice: sets the streak to 0, and the alert closes.**
WHERE First button in the alert's button closure.
WHEN Only after the person reads the alert and taps Reset.
WHY Label the button with the verb from the title (Reset), not a vague OK, so the choice is unmistakable.

15 `Button("Cancel") { }`

WHO The user chooses it when they decide to keep the streak.
WHAT **A button with an empty action: tapping it only closes the alert.**
WHERE Second button in the alert's button closure, after Reset.
WHEN When the person changes their mind and taps Cancel.
WHY Always offer a safe way out. Apps usually write Button("Cancel", role: .cancel) { }, which marks it as the safe choice so iOS styles and positions it accordingly; this example leaves roles out to stay short.

16 `} message: {`

WHO Swift's multiple trailing closure syntax, read by SwiftUI.
WHAT **Ends the buttons closure and starts a second closure that supplies the alert's message.**
WHERE Still part of the same .alert call, between its two closures.
WHEN The message is built each time the alert appears.
WHY The title asks the question; the message explains the consequence.

17 `Text("Your \(streak) days go back to zero.")`

WHO SwiftUI, which places this Text as the alert's message.
WHAT **The message, using the current streak so the consequence is concrete.**
WHERE Inside the message closure, shown below the alert's title.
WHEN Built each time the alert appears, using the current streak.
WHY Specific numbers make the risk real: "12 days" persuades more than "your progress".

22 `#Preview { StreakView() }`

WHO Xcode's preview canvas, and the simulated iPhone beside this code, running StreakView.
WHAT **Shows the streak screen so you can trigger the alert.**
WHERE At file scope, after StreakView; preview code is not part of the shipped app.
WHEN Every time the code changes; the preview restarts with a 12-day streak.
WHY Trying both alert buttons shows that only the confirming choice changes data.

Try it

1. Tap Reset streak, then Cancel. The number stays. Tap again and choose Reset.
2. Change the title to "Delete all progress?" and the Reset button's title to "Delete".
3. Add a third Button("Reset to 1") { streak = 1 } inside the alert.

CHECK YOURSELF

Check your understanding

1. A fitness app has three top-level sections people switch between all day: Today, History and Settings. Which structure fits best?
   A. A NavigationStack whose root list links to each section
   B. A TabView with one Tab per section
   C. A sheet for each section, opened from buttons on a home screen
   D. An alert that asks which section to open

2. A list uses `NavigationLink(recipe.name, value: recipe)` inside a `NavigationStack`, but tapping a row does nothing. What is the most likely cause?
   A. Recipe does not conform to CustomStringConvertible, so the link cannot show it
   B. The stack needs a navigationTitle before it will push any destination
   C. No navigationDestination(for: Recipe.self) is registered in the stack
   D. NavigationLink(value:) only accepts String values, not custom types

3. Why must a type used with `NavigationLink(value:)` conform to Hashable?
   A. So SwiftUI can print the value in the navigation bar title
   B. The stack keeps pushed values in a path and must compare them
   C. So the value is saved to disk automatically when the app quits
   D. Because only Hashable types can be captured inside a closure

4. A person taps Delete Account. What should the app show first?
   A. A sheet containing every account setting
   B. Nothing: delete right away so the app feels fast
   C. A pushed screen explaining what an account is
   D. An alert asking them to confirm, with the destructive choice clearly named

5. Where must `@Environment(\.dismiss) private var dismiss` be declared so that calling `dismiss()` closes a sheet?
   A. In the view shown inside the sheet
   B. In the parent view that has the .sheet modifier
   C. In the app struct, so every view can share it
   D. Anywhere; dismiss always closes the topmost sheet

Answers: Navigation, Tabs, and Sheets

1. **B. A TabView with one Tab per section** The sections are peers, so a `TabView` gives each one-tap access and keeps each in place while another is visible. Pushing them from a list, the usual first instinct, makes peers look like parent and child, and every switch costs a trip back through the list.
2. **C. No navigationDestination(for: Recipe.self) is registered in the stack** A value-based link only pushes a value; the stack then looks for a `navigationDestination` registered for that type. With none, there is no screen to show. A title is cosmetic and has nothing to do with pushing, and any Hashable type works as a value.
3. **B. The stack keeps pushed values in a path and must compare them** A NavigationStack keeps its pushed screens as a collection of values and must tell them apart to manage that path. Hashable provides exactly that. It does not print anything (the link's label does) and it does not save anything to disk.
4. **D. An alert asking them to confirm, with the destructive choice clearly named** Irreversible actions deserve a quick confirmation: an alert interrupts just long enough and names the consequence. Deleting immediately feels fast, but it turns one stray tap into lost data. A sheet or pushed screen is heavier than a yes-or-no question needs.
5. **A. In the view shown inside the sheet** dismiss acts on the presentation that contains the view reading it, so it belongs in the sheet's own view. Declared in the parent, where many people first put it, it would refer to the parent's own presentation, not to the sheet the parent is showing.

Forms and Input

CONCEPT

Controls are windows onto state

A form collects input with *controls*: text fields, switches, pickers, steppers and sliders. In SwiftUI every control is connected to a piece of state with a `$` binding, a two-way connection: the control displays the value and writes back whatever the person enters. The control never keeps its own copy.

`Form` arranges controls into the grouped rows familiar from the Settings app, and `Section` groups related rows under a header.

TextField and SecureField	Free text. SecureField hides what is typed, for passwords.
Toggle	One on-or-off choice that takes effect right away.
Picker	One choice from a fixed set: segmented for two to five short options, a menu for longer lists.
Stepper and Slider	Numbers. A Stepper for small exact counts, a Slider for an approximate value in a range.

> **Pick the control that fits the data**
>
> The right control prevents bad input before it happens. A Stepper cannot produce "two-ish", and a Picker cannot select a size that does not exist. Prefer a constrained control over free text whenever the valid answers are known.

CODE · LIVE SWIFTUI PREVIEW

A sign-up form

A sign-up form with two text fields and a switch. Below them, a section reads the same state back, so you can watch the bindings work: each keystroke writes into `@State`, and SwiftUI recomputes `body`. Type in the phone and watch the bottom section.

```swift
import SwiftUI

struct SignUpView: View {
    @State private var email = ""
    @State private var password = ""
    @State private var wantsTips = false

    var body: some View {
        Form {
            TextField("Email", text: $email)
            SecureField("Password", text: $password)
            Toggle("Weekly tips by email", isOn: $wantsTips)
            Section("What your state holds") {
                Text("Email: \(email)")
                Text("Password: \(password.count) characters")
                Text(wantsTips ? "Tips: on" : "Tips: off")
            }
        }
    }
}

#Preview { SignUpView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the sign-up screen.
- **WHAT** **Brings in Form, Section, TextField, SecureField and Toggle.**
- **WHERE** Top of the file; the import covers SignUpView and its preview.
- **WHEN** At compile time, before SignUpView is type-checked.
- **WHY** Every input control in this lesson comes from SwiftUI.

3 `struct SignUpView: View {`

- **WHO** Your code declares the screen; SwiftUI creates it and shows the form.
- **WHAT** **A screen where a new user enters an email, a password and a preference.**
- **WHERE** At file scope, so the preview or an app scene can create SignUpView.
- **WHEN** Created when the preview launches, with all three inputs empty or off.
- **WHY** Forms are ordinary views; nothing about them is special except the controls they contain.

4 `@State private var email = ""`

- **WHO** SwiftUI owns the storage; the email field writes it.
- **WHAT** **Holds the email text, starting as an empty string.**
- **WHERE** In SwiftUI-managed storage tied to SignUpView, outside the struct itself.
- **WHEN** Created on first appearance; updated on every keystroke in the email field.
- **WHY** The view, not the text field, owns the value, so any other part of the view (like the summary below) can read it.

5 `@State private var password = ""`

- **WHO** SwiftUI owns it; the SecureField writes it.
- **WHAT** **Holds the password text that the SecureField collects, starting empty.**
- **WHERE** In the same SwiftUI-managed storage as email, attached to SignUpView.
- **WHEN** Updated on every keystroke in the password field.
- **WHY** The field hides the characters on screen, but the stored value is plain text, so never print or log it.

```
6  @State private var wantsTips = false
```

WHO	SwiftUI owns the storage; the Toggle in the third row writes it.
WHAT	**Whether the person wants weekly tips by email, starting off.**
WHERE	Beside email and password, in SignUpView's SwiftUI-managed storage.
WHEN	Created as false on first appearance; changed each time the switch is flipped.
WHY	A default is a decision. Marketing email should be opt-in, so the switch starts off.

```
8  var body: some View {
```

WHO	SwiftUI calls body each time email, password or wantsTips changes.
WHAT	**Describes the form and the summary section.**
WHERE	Inside SignUpView, on the main thread, below the three state properties.
WHEN	On first appearance and after every change to any of the three state values.
WHY	Every keystroke changes state, so body runs often; it must stay cheap, which plain view descriptions are.

```
9  Form {
```

WHO	SwiftUI, which styles the Form's rows like the iOS Settings app.
WHAT	**Arranges the controls as grouped rows on a gray background, like the Settings app.**
WHERE	The root of SignUpView's body; every control below lives inside it.
WHEN	Laid out on appearance and after every state change.
WHY	Form applies platform styling for you, so you describe fields, not row decorations.

```
10 TextField("Email", text: $email)
```

WHO	SwiftUI shows the field; the user types; the binding writes.
WHAT	**A single-line text field with the placeholder Email, bound to email.**
WHERE	First row of the form, so email is the first thing people fill in.
WHEN	Each keystroke immediately updates email, which makes body run again.
WHY	The $ passes a binding, not a copy, so the field can write back. Without the $, the code does not compile, because the field needs write access.

```
11 SecureField("Password", text: $password)
```

WHO	SwiftUI shows the field and hides each character; the user types into it.
WHAT	**Works like TextField but displays dots instead of the characters typed.**
WHERE	Second row of the form, directly under the email field.
WHEN	Each keystroke writes the new text into password through the binding.
WHY	Use it for every password, so people nearby cannot read the screen.

```
12 Toggle("Weekly tips by email", isOn: $wantsTips)
```

WHO	SwiftUI draws the tips switch; the person signing up flips it.
WHAT	**A switch with a label, bound to wantsTips.**
WHERE	Third row of the form, after the two text fields.
WHEN	Each flip writes true or false into wantsTips.
WHY	A Toggle is for one yes-or-no choice; its label states what "on" means.

```
13 Section("What your state holds") {
```

WHO	SwiftUI, which draws the header above a new group of rows.
WHAT	**Starts a group of rows with a header, used here to display the stored state.**
WHERE	Inside the Form, below the three controls it summarizes.
WHEN	Rebuilt on every body run, so its rows always show current state.
WHY	Sections break long forms into chunks people can scan; the header names the chunk.

```
14  Text("Email: \(email)")
```

WHO	SwiftUI, which builds this row again on every body run.
WHAT	**Shows the current email text after the label, for example Email: ana@example.com.**
WHERE	First row of the summary section, under its header.
WHEN	Re-created after every keystroke in the email field.
WHY	This row proves the loop: the field writes state, state changes, body runs, and this Text shows the new value.

```
15  Text("Password: \(password.count) characters")
```

WHO	SwiftUI, reading password.count while it builds the row.
WHAT	**Shows only the length of the password, never the password itself.**
WHERE	Second row of the summary section, under the email row.
WHEN	Updated after every keystroke in the password field.
WHY	Even in a debug display, never reveal a secret; the count is enough to see the binding update.

```
16  Text(wantsTips ? "Tips: on" : "Tips: off")
```

WHO	SwiftUI, evaluating the ternary each time it builds the row.
WHAT	**Picks one of two strings with the ternary operator, based on the switch.**
WHERE	Third row of the summary section, mirroring the Toggle.
WHEN	Updated each time the Toggle is flipped.
WHY	A view can choose its content from state with an ordinary Swift expression.

```
22  #Preview { SignUpView() }
```

WHO	Xcode's preview canvas, and the simulated iPhone beside this code, running SignUpView.
WHAT	**Shows the form live; you can type into its fields.**
WHERE	At file scope, after SignUpView; preview code is not part of the shipped app.
WHEN	Every time the code changes; the preview restarts with empty fields.
WHY	Typing and watching the summary update is the fastest way to understand bindings.

Try it

1. Type an email in the phone and watch the first summary row change with every keystroke.
2. Change the email TextField to a SecureField. What changes on screen, and what stays the same in the summary?
3. Add a TextField("Name", text: $name) and the @State property it needs.

CODE · LIVE SWIFTUI PREVIEW

Choose one option with a Picker

Choosing a drink size: exactly one of three options. An enum lists the options, so an invalid size cannot exist. The `Picker` is bound to a `Size` value, and each option is tagged with the value it stands for. `.pickerStyle(.segmented)` draws the options side by side.

```swift
import SwiftUI

enum Size: String, CaseIterable {
    case small = "Small", medium = "Medium", large = "Large"
}

struct SizePickerView: View {
    @State private var size = Size.medium
    var body: some View {
        Form {
            Picker("Size", selection: $size) {
                ForEach(Size.allCases, id: \.self) { option in
                    Text(option.rawValue).tag(option)
                }
            }
            .pickerStyle(.segmented)
            Text("You picked \(size.rawValue).")
        }
    }
}

#Preview { SizePickerView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the size picker.
- **WHAT** **Brings in Picker, the pickerStyle modifier, Form and ForEach.**
- **WHERE** Top of the file, above the Size enum and the view that uses it.
- **WHEN** At compile time, before the enum and the view are type-checked.
- **WHY** All picker styles come from SwiftUI; switching style is a one-word change.

3 `enum Size: String, CaseIterable {`

- **WHO** Your code declares the enum; the compiler generates its allCases array.
- **WHAT** **Declares the only three sizes the shop sells, with display names as raw values and an allCases array.**
- **WHERE** At file scope, so any view can use Size.
- **WHEN** At compile time; Swift writes allCases and the raw value lookups for you.
- **WHY** An enum makes the set of valid choices part of the type: the picker cannot select a size that has no case.

4 `case small = "Small", medium = "Medium", large = "Large"`

- **WHO** Your code names the cases; the compiler stores each raw value string.
- **WHAT** **Three cases on one line, each with a String raw value.**
- **WHERE** Inside the Size enum, as its complete list of cases.
- **WHEN** Fixed at compile time; the shop's sizes cannot change while the app runs.
- **WHY** The raw value doubles as the label text, so the picker and the model can never disagree on names.

7 `struct SizePickerView: View {`

- **WHO** Your code declares the screen; SwiftUI creates it and shows the picker.
- **WHAT** **A screen that lets the customer choose a drink size.**
- **WHERE** At file scope, below the Size enum it uses.
- **WHEN** Created when the preview launches, with Medium selected.
- **WHY** Keeping one decision per view makes each control easy to study in isolation.

```swift
8  @State private var size = Size.medium
```

WHO — SwiftUI stores it; the Picker writes it.
WHAT — **The selected size, starting at Size.medium, the default choice.**
WHERE — In SwiftUI-managed storage tied to SizePickerView.
WHEN — Created on first appearance; changed whenever a segment is tapped.
WHY — The state's type is Size, not String, so the compiler checks every assignment. Defaulting to the most popular choice saves most people a tap.

```swift
9  var body: some View {
```

WHO — SwiftUI calls body on first appearance and after each new selection.
WHAT — **Describes the form with the picker and a confirmation line.**
WHERE — Inside SizePickerView, on the main thread.
WHEN — On first appearance and after every selection.
WHY — The confirmation line reads the same state the picker writes, so they always agree.

```swift
10 Form {
```

WHO — SwiftUI, styling the picker and the text as Form rows.
WHAT — **Puts the picker and the confirmation text into grouped rows.**
WHERE — The root of SizePickerView's body, filling the screen.
WHEN — Laid out on every body run, after each new selection.
WHY — Pickers look and behave like settings rows inside a Form.

```swift
11 Picker("Size", selection: $size) {
```

WHO — SwiftUI draws the picker; the user taps an option; the binding writes.
WHAT — **A control that selects one value and writes it into size through the binding.**
WHERE — First row of the form, above the confirmation text.
WHEN — The selection changes on tap, and body runs again.
WHY — The label "Size" is still read by VoiceOver even when the segmented style does not display it.

```swift
12 ForEach(Size.allCases, id: \.self) { option in
```

WHO — SwiftUI, repeating its closure for each case.
WHAT — **Builds one option for each case of Size, in declaration order.**
WHERE — Inside the Picker's content closure, where its options are listed.
WHEN — Each time the picker's content is evaluated.
WHY — Driving the options from allCases means adding a case to the enum automatically adds an option.

```swift
13 Text(option.rawValue).tag(option)
```

WHO — SwiftUI, pairing each label with a value.
WHAT — **Shows the option's name and tags it with the Size value it represents.**
WHERE — Inside the ForEach closure, so one tagged Text exists per Size case.
WHEN — Built with the picker; the tag is compared with size to decide which option is selected.
WHY — The tag's type must match the selection's type (Size). A tag of the wrong type can never be selected.

```swift
16 .pickerStyle(.segmented)
```

WHO — SwiftUI, which switches the Picker's appearance to segments.
WHAT — **Draws the options as a horizontal segmented control.**
WHERE — Attached to the Picker, after its content closure.
WHEN — Applied each time the Picker is rebuilt during a body run.
WHY — Segmented controls show every choice at once, which suits two to five short options. Longer lists belong in a menu.

```
17  Text("You picked \(size.rawValue).")
```

WHO	SwiftUI, building the confirmation row from current state.
WHAT	**Echoes the current selection, for example You picked Medium.**
WHERE	Second row of the form, below the Picker.
WHEN	Updated after each tap on a segment, in the same body run.
WHY	It shows that the binding wrote a real Size value that the rest of the view can use.

```
22  #Preview { SizePickerView() }
```

WHO	Xcode's preview canvas, and the simulated iPhone beside this code, running SizePickerView.
WHAT	**Shows the size picker live so you can tap each segment.**
WHERE	At file scope, after SizePickerView; preview code is not part of the shipped app.
WHEN	Every time the code changes; the preview restarts at Medium.
WHY	Switching styles in the preview shows how one Picker adapts its look without changing its logic.

Try it

1. Change .segmented to .menu. The same Picker becomes a pop-up menu row.
2. Add a fourth case, extraLarge = "Extra Large". The picker grows with no other change.
3. Change the default to Size.small.

CODE · LIVE SWIFTUI PREVIEW

Validate input and disable the button

A create-account form that refuses bad input politely. A computed `isValid` property decides whether the form is ready, `.disabled(!isValid)` grays out the button until it is, and a hint appears only while the password is too short. Type in the phone and watch the button come alive.

```
1  import SwiftUI
2
3  struct CreateAccountView: View {
4      @State private var email = ""
5      @State private var password = ""
6      var isValid: Bool { email.contains("@") && password.count >= 8 }
7
8      var body: some View {
9          Form {
10             TextField("Email", text: $email)
11             SecureField("Password (8+ characters)", text: $password)
12             if !password.isEmpty && password.count < 8 {
13                 Text("\(8 - password.count) more characters needed")
14                     .foregroundStyle(.red)
15             }
16             Button("Create account") { }
17                 .disabled(!isValid)
18         }
19     }
20 }
21
22 #Preview { CreateAccountView() }
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, while it builds the account form.
WHAT	**Brings in Form, TextField, SecureField, Button and the disabled modifier.**
WHERE	Top of the file; it covers CreateAccountView and its preview.
WHEN	At compile time, before CreateAccountView is type-checked.
WHY	Validation needs nothing beyond plain Swift logic plus SwiftUI's ability to recompute body.

```
3  struct CreateAccountView: View {
```

WHO	Your code declares the account screen; SwiftUI creates it and shows its fields.
WHAT	**A screen that collects an email and a password and only submits valid input.**
WHERE	At file scope, so the preview can create CreateAccountView directly.
WHEN	Created when the preview launches, with both fields empty and the button disabled.
WHY	Putting the rules next to the form keeps them easy to find and change.

```
4  @State private var email = ""
```

WHO	SwiftUI stores it; the email field writes it.
WHAT	**The email text, initially empty, which the validation rule checks for an @.**
WHERE	In SwiftUI-managed storage tied to CreateAccountView.
WHEN	Updated on every keystroke in the email field.
WHY	Each keystroke re-runs body, so validation is re-checked continuously.

```
5  @State private var password = ""
```

WHO	SwiftUI stores it; the SecureField writes it.
WHAT	**The password text, initially empty, which the rule and the hint both measure.**
WHERE	Beside email, in CreateAccountView's SwiftUI-managed storage.
WHEN	Updated on every keystroke, which re-checks the hint and the button.
WHY	The hint and the button both read this one value, so they always agree.

```
6  var isValid: Bool { email.contains("@") && password.count >= 8 }
```

WHO	Your code defines the rule; body and the disabled modifier read it.
WHAT	**A computed property that is true only when the email contains an @ and the password has at least 8 characters.**
WHERE	Inside the view, next to the state it depends on.
WHEN	Re-evaluated whenever body runs, which is after every keystroke.
WHY	One named rule means the button and any other view agree on what "valid" means. The @ check is deliberately simple; real services also validate on the server.

```
8  var body: some View {
```

WHO	SwiftUI calls body again after each keystroke changes email or password.
WHAT	**Describes the fields, the conditional hint and the button.**
WHERE	Inside CreateAccountView, below the state and the rule.
WHEN	On first appearance and after every keystroke.
WHY	Because body re-runs on each change, the form reacts instantly with no extra code.

```
9  Form {
```

WHO	SwiftUI, which styles the fields, hint and button as Form rows.
WHAT	**Arranges the fields, hint and button as grouped rows.**
WHERE	The root of CreateAccountView's body; every row lives inside it.
WHEN	Laid out on every body run, so the hint row can appear and vanish.
WHY	Form styling makes the disabled button look clearly inactive, matching system apps.

```
10  TextField("Email", text: $email)
```

WHO SwiftUI shows the email field; the user types into it.
WHAT **An email field with the placeholder Email, bound to the email state.**
WHERE First row of the form, above the password field.
WHEN Each keystroke writes email and makes body re-check isValid.
WHY The binding keeps the field and the validation rule looking at the same value.

```
11  SecureField("Password (8+ characters)", text: $password)
```

WHO SwiftUI shows the field and hides each character as the user types.
WHAT **A hidden-text field whose placeholder states the requirement.**
WHERE Second row of the form, under the email field.
WHEN Each keystroke writes password, which can show or hide the hint.
WHY Stating the rule up front lets people get it right the first time instead of learning from an error.

```
12  if !password.isEmpty && password.count < 8 {
```

WHO SwiftUI evaluates this condition inside body.
WHAT **Includes the hint below only when something has been typed but it is still too short.**
WHERE Inside the Form, between the fields and the button.
WHEN Checked on every body run; the hint appears and disappears as the length crosses 8.
WHY Do not scold an empty field before the person has started; show help at the moment it is useful.

```
13  Text("\(8 - password.count) more characters needed")
```

WHO SwiftUI, building the hint only while the if condition holds.
WHAT **Says exactly how many characters are missing.**
WHERE A row that exists only while the condition is true.
WHEN Updated with every keystroke while visible.
WHY A specific hint is easier to act on than "Invalid password". A constant such as let minLength = 8 would keep the three 8s in sync.

```
14  .foregroundStyle(.red)
```

WHO SwiftUI, which applies the color to the hint Text.
WHAT **Colors the hint text red, the system's warning color.**
WHERE Attached to the hint Text, inside the if block.
WHEN Applied whenever the hint is shown, on every body run while it exists.
WHY Red draws the eye; the words still carry the meaning, so color is not the only signal.

```
16  Button("Create account") { }
```

WHO The user taps it, but only while the button is enabled.
WHAT **The submit button; its action is empty in this example.**
WHERE Last row of the form, after the fields and the optional hint.
WHEN Its action would run on tap, but only while the button is enabled.
WHY A real app would call its sign-up service here; the empty closure keeps the focus on when tapping is allowed.

```
17  .disabled(!isValid)
```

WHO SwiftUI, which grays out the button and blocks its taps.
WHAT **Grays out the button and ignores taps while isValid is false.**
WHERE On the Button only; the fields stay editable.
WHEN Re-applied on each body run, so the button enables the moment the input becomes valid.
WHY Preventing an invalid submit is kinder than accepting it and showing an error afterward.

```
22 #Preview { CreateAccountView() }
```

WHO Xcode's preview canvas, and the simulated iPhone beside this code, running CreateAccountView.
WHAT **Shows the form so you can type and watch the rules apply.**
WHERE At file scope, after CreateAccountView; preview code is not part of the shipped app.
WHEN Every time the code changes; the preview restarts with the button disabled.
WHY Validation is about timing; you only feel it by typing.

> **Try it**
>
> 1. Type a 6-character password, then 8 characters. Watch the hint count down and vanish.
> 2. Change the rule to 12 characters in both isValid and the hint.
> 3. Enter " @" (a space and an at sign) as the email with a long password. It passes. How would you tighten isValid?

CODE · LIVE SWIFTUI PREVIEW

The complete order form

Everything together: sections, a stepper, a text field, a derived price, and a button that stays disabled until the cup has a name. The price on the button is computed from state, so it is always right. Change the shots and type a name in the phone.

```swift
import SwiftUI

struct CoffeeOrderView: View {
    @State private var shots = 2
    @State private var name = ""
    var total: Double { 3.0 + Double(shots) * 0.75 }

    var body: some View {
        Form {
            Section("Drink") {
                Stepper("Shots: \(shots)", value: $shots, in: 1...4)
            }
            Section("Pickup") {
                TextField("Name for the cup", text: $name)
                Button("Pay " + String(format: "$%.2f", total)) { }
                    .disabled(name.isEmpty)
            }
        }
    }
}

#Preview { CoffeeOrderView() }
```

Line by line

```
1 import SwiftUI
```

WHO The Swift compiler, while it builds the coffee order screen.
WHAT **Brings in Form, Section, Stepper, TextField and Button: everything a complete order screen needs.**
WHERE Top of the file; it covers CoffeeOrderView and its preview.
WHEN At compile time, before CoffeeOrderView is type-checked.
WHY A full form is just the same few controls combined.

```
3  struct CoffeeOrderView: View {
```

WHO Your code declares the screen; SwiftUI creates it and shows the order form.
WHAT **The full order screen for a latte: shots, a pickup name and a pay button.**
WHERE At file scope, so the preview can create CoffeeOrderView directly.
WHEN Created when the preview launches, with 2 shots and no name.
WHY One screen with one goal: get a correct, paid order.

```
4  @State private var shots = 2
```

WHO SwiftUI stores it; the Stepper writes it.
WHAT **The number of espresso shots, starting at 2.**
WHERE In SwiftUI-managed storage tied to CoffeeOrderView.
WHEN Changed on each tap of the stepper's minus or plus button.
WHY It is an input, so it is state; the price is not.

```
5  @State private var name = ""
```

WHO SwiftUI stores it; the TextField writes it.
WHAT **The name to write on the cup, initially empty.**
WHERE Beside shots, in CoffeeOrderView's SwiftUI-managed storage.
WHEN Updated on every keystroke in the name field.
WHY The button's enabled state depends on it, so it must be observable state.

```
6  var total: Double { 3.0 + Double(shots) * 0.75 }
```

WHO Your code defines the price rule; the Pay button's title reads it.
WHAT **Computes the price: a 3.00 base plus 0.75 per shot.**
WHERE Inside the view, next to the state it uses.
WHEN Recalculated each time body reads it, after every change to shots.
WHY The price is derived, never stored; a stored copy would be a second source of truth that could go stale.

```
8  var body: some View {
```

WHO SwiftUI calls body again whenever shots or name changes.
WHAT **Describes the two sections of the order form.**
WHERE Inside CoffeeOrderView, below the state and the total.
WHEN On first appearance and after every change to shots or name.
WHY Everything on screen is a function of two values, which keeps the screen easy to reason about.

```
9  Form {
```

WHO SwiftUI, which styles the order's sections as grouped rows.
WHAT **Arranges the Drink and Pickup sections as grouped rows on a gray background.**
WHERE The root of CoffeeOrderView's body, holding both sections.
WHEN Laid out on every body run, after each stepper tap or keystroke.
WHY Form gives the order screen the familiar look of a settings page.

```
10 Section("Drink") {
```

WHO SwiftUI, which draws the Drink header above its rows.
WHAT **Groups the drink options under a Drink header.**
WHERE First section of the form, at the top of the screen.
WHEN Rebuilt on every body run with the current shot count.
WHY Sections mirror how people think about the task: first the drink, then the pickup details.

```
11  Stepper("Shots: \(shots)", value: $shots, in: 1...4)
```

WHO	SwiftUI draws the stepper; the user taps it.
WHAT	**Shows the current shot count and changes it by one per tap, between 1 and 4.**
WHERE	Inside the Drink section, as its only row.
WHEN	Each tap writes shots, which updates the price on the button.
WHY	The range encodes a business rule (no more than 4 shots) directly in the control.

```
13  Section("Pickup") {
```

WHO	SwiftUI, which draws the Pickup header above its rows.
WHAT	**Groups the pickup name and the pay button under a Pickup header.**
WHERE	Second section of the form, below Drink.
WHEN	Rebuilt on every body run, so the Pay title stays current.
WHY	The commit action sits at the end of the flow, after everything it depends on.

```
14  TextField("Name for the cup", text: $name)
```

WHO	SwiftUI shows the field; the user types the name for the cup.
WHAT	**A text field with the placeholder Name for the cup, bound to name.**
WHERE	First row of the Pickup section, above the Pay button.
WHEN	Each keystroke writes name, which can enable the Pay button.
WHY	A descriptive placeholder explains why the name is needed.

```
15  Button("Pay " + String(format: "$%.2f", total)) { }
```

WHO	The user taps it, but only once the button is enabled.
WHAT	**A button whose title includes the live price, for example "Pay $4.50".**
WHERE	Last row of the Pickup section, at the end of the form.
WHEN	Its title is rebuilt on every body run, so it follows the stepper.
WHY	Showing the price on the action button means people see the cost at the moment they commit.

```
16  .disabled(name.isEmpty)
```

WHO	SwiftUI, which grays out the Pay button and blocks its taps.
WHAT	**Keeps the button inactive until a name has been typed.**
WHERE	Attached to the Pay button only; the fields stay editable.
WHEN	Re-applied on each body run, so the button enables with the first typed letter.
WHY	The barista needs a name, so the order cannot be sent without one.

```
22  #Preview { CoffeeOrderView() }
```

WHO	Xcode's preview canvas, and the simulated iPhone beside this code, running CoffeeOrderView.
WHAT	**Shows the order form live so you can change shots and type a name.**
WHERE	At file scope, after CoffeeOrderView; preview code is not part of the shipped app.
WHEN	Every time the code changes; the preview restarts at 2 shots.
WHY	Changing shots and watching the price follow is the clearest demonstration of derived state.

Try it

1. Add @State private var oatMilk = false, a Toggle("Oat milk", isOn: $oatMilk) in the Drink section, and + (oatMilk ? 0.6 : 0) at the end of total.
2. Add a Picker with .pickerStyle(.menu) to the Drink section offering Latte, Mocha and Cold Brew.
3. Change the Stepper range to 1…6 and check the price at 6 shots.

CODE · READ-ALONG

Dates, keyboards, and focus

Three things a real sign-up form needs that the in-page phone cannot show, so run this one in Xcode. A `DatePicker` binds to a `Date`. `.keyboardType` and `.textContentType` tell iOS to show the email keyboard and offer autofill. `@FocusState` lets a button move the cursor into the field. The preview on this page does not support DatePicker or focus, so this step is type-checked against the iOS 26 SDK and shown as a read-along.

```swift
import SwiftUI

struct ReservationView: View {
    enum Field { case email }
    @State private var date = Date()
    @State private var email = ""
    @FocusState private var focus: Field?

    var body: some View {
        Form {
            DatePicker("Arrival", selection: $date,
                       displayedComponents: .date)
            TextField("Email", text: $email)
                .keyboardType(.emailAddress)
                .textContentType(.emailAddress)
                .focused($focus, equals: .email)
            Button("Add email") { focus = .email }
        }
    }
}
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the reservation screen.
- WHAT — **Brings in Form, DatePicker, TextField, Button and the @FocusState wrapper.**
- WHERE — Top of the file; it covers ReservationView and everything in it.
- WHEN — At compile time, before ReservationView is type-checked.
- WHY — Dates, keyboards and focus are all SwiftUI features; nothing here needs another framework.

3 `struct ReservationView: View {`

- WHO — Your code declares the screen; SwiftUI creates it when the app or a preview shows it.
- WHAT — **A screen that collects an arrival date and an email for a reservation.**
- WHERE — At file scope, so a preview or an app scene can create ReservationView.
- WHEN — Created when the screen first appears, with today's date and an empty email.
- WHY — One screen, two inputs: enough to show a date control, a tuned keyboard and focus.

4 `enum Field { case email }`

- WHO — Your code declares the enum; the compiler makes it Hashable because it has no payloads.
- WHAT — **Names the fields that can hold focus; this form has only one, email.**
- WHERE — Nested inside ReservationView, so the name Field cannot clash with other screens.
- WHEN — Fixed at compile time; add a case here for every field you want to focus by code.
- WHY — Focus is tracked as a value, so the fields need a type whose values the compiler can compare.

```
5  @State private var date = Date()
```

WHO SwiftUI owns the storage; the DatePicker reads and writes it.
WHAT **Holds the chosen arrival date, starting at the current moment.**
WHERE In SwiftUI-managed storage tied to ReservationView, outside the struct value.
WHEN Created on first appearance; replaced each time a new date is picked.
WHY A DatePicker binds to a real Date value, not text, so the app never parses a typed date.

```
6  @State private var email = ""
```

WHO SwiftUI owns the storage; the TextField writes it on every keystroke.
WHAT **Holds the email text, starting empty.**
WHERE Beside date, in the same SwiftUI-managed storage.
WHEN Updated on every keystroke in the email field.
WHY The field needs a binding to write into, exactly as in the sign-up form.

```
7  @FocusState private var focus: Field?
```

WHO SwiftUI owns the storage and keeps it in step with the keyboard; your button writes it.
WHAT **Records which field has focus: .email while the cursor is in the field, nil when none has it.**
WHERE In SwiftUI-managed focus storage tied to ReservationView, like @State but for focus.
WHEN Set to .email when the person taps the field or the button; set to nil when the keyboard goes away.
WHY Focus is state like any other. Writing it from code lets a button put the cursor where it belongs.

```
9  var body: some View {
```

WHO SwiftUI calls body each time date, email or focus changes.
WHAT **Describes the form with its date row, email field and button.**
WHERE Inside ReservationView, below the three stored properties.
WHEN On first appearance and after every date change, keystroke or focus move.
WHY Every input lives in state, so the screen is always a direct picture of the three values.

```
10 Form {
```

WHO SwiftUI, which styles the three rows like the Settings app.
WHAT **Arranges the date picker, the email field and the button as grouped rows.**
WHERE The root of ReservationView's body; every control below lives inside it.
WHEN Laid out on appearance and after every state change.
WHY Inside a Form a DatePicker becomes a compact row that opens a calendar on tap.

```
11 DatePicker("Arrival", selection: $date,
```

WHO SwiftUI draws the picker; the person taps a date; the binding writes it.
WHAT **Starts a date control labeled Arrival, bound to date; the next line finishes the call.**
WHERE First row of the form, above the email field.
WHEN Each pick writes a new Date into date, which makes body run again.
WHY The system calendar handles months, locales and time zones, which a text field never could.

```
12 displayedComponents: .date)
```

WHO The DatePicker, reading which parts of a date it should offer.
WHAT **Limits the picker to day, month and year, hiding the time of day.**
WHERE The last argument of the DatePicker call started on the line above.
WHEN Fixed when the picker is built; use [.date, .hourAndMinute] to show both.
WHY An arrival date has no time, so offering one would only invite a wrong value.

```
13 TextField("Email", text: $email)
```

WHO SwiftUI shows the field; the person types; the binding writes.
WHAT **A single-line text field with the placeholder Email, bound to email.**
WHERE Second row of the form, under the date picker.
WHEN Each keystroke writes email and makes body run again.
WHY Plain text is right for an email address; the next three lines tune how it is typed.

```
14 .keyboardType(.emailAddress)
```

WHO iOS, when it raises the keyboard for this field.
WHAT **Asks for the email keyboard, which puts @ and . within easy reach.**
WHERE Attached to the email TextField only; other fields keep the default keyboard.
WHEN Each time this field gains focus and the keyboard appears.
WHY The right keyboard saves a mode switch on every address and reduces typos.

```
15 .textContentType(.emailAddress)
```

WHO iOS, which matches the field against saved contact and password data.
WHAT **Tells the system this field holds an email address, so it can offer autofill above the keyboard.**
WHERE On the email TextField, after the keyboard type.
WHEN When the field gains focus, before the person types anything.
WHY One tap on a suggestion beats typing an address, and it is correct every time.

```
16 .focused($focus, equals: .email)
```

WHO SwiftUI, which links this field to the focus state through a binding.
WHAT **Declares that this field has focus exactly when focus equals .email.**
WHERE On the email TextField, as its last modifier.
WHEN Checked whenever focus changes; also writes .email when the person taps the field.
WHY The link works both ways: tapping the field updates focus, and setting focus moves the cursor here.

```
17 Button("Add email") { focus = .email }
```

WHO The person taps; SwiftUI runs the closure and raises the keyboard.
WHAT **A button whose action moves focus into the email field.**
WHERE Third and last row of the form, under the email field.
WHEN Each tap sets focus to .email; the field then shows the cursor and keyboard.
WHY Moving focus from code lets a form guide people to the next thing to fill in.

> **Try it**
>
> 1. Predict: change .emailAddress on the keyboardType line to .numberPad. What keyboard appears for the email field?
> 2. In Xcode, change displayedComponents to [.date, .hourAndMinute] and watch the row change.

CHECK YOURSELF

Check your understanding

1. An app asks how many people will split a bill, from 1 to 12. Which control fits best?
   A. A Slider from 1 to 12
   B. A TextField that accepts any text
   C. A Stepper with the range 1…12
   D. A Toggle

2. Why does `TextField("Email", text: $email)` use `$email` instead of `email`?

   A. The $ marks email as optional, so an empty field is allowed
   B. The $ passes a binding, so the field can write changes back
   C. The $ makes the field hide its text, as SecureField does
   D. The $ formats the text as currency before showing it on screen

3. The pay button uses `.disabled(name.isEmpty)`. A customer types a single space as their name. What happens?

   A. The button stays disabled, because spaces do not count
   B. The app crashes, because a name cannot be blank
   C. TextField removes the space automatically before storing it
   D. The button enables, because " " is not an empty string

4. A Picker offers 14 coffee flavors. Which style should it use?

   A. .menu, which shows the current choice and lists all options on tap
   B. .segmented, so all 14 flavors stay visible at once
   C. 14 Toggles in a Section, one for each flavor
   D. Fourteen Buttons in an HStack, each setting the selected flavor

Answers: Forms and Input

1. **C. A Stepper with the range 1...12** A Stepper changes a small whole number one step at a time and cannot leave its range, so the answer is always exact and valid. A Slider seems reasonable but is built for approximate values; landing on exactly 7 with a thumb is fiddly. A TextField would accept "seven" or "-3".
2. **B. The $ passes a binding, so the field can write changes back** `$email` is a `Binding<String>`: a two-way connection to the state. The field reads it to display text and writes to it on every keystroke. Plain `email` would be a read-only copy, and the code would not compile. The $ has nothing to do with optionals, hiding text or currency.
3. **D. The button enables, because " " is not an empty string** `isEmpty` only checks for zero characters, and a space is a character, so the order can be sent with a blank-looking name. TextField does not trim input for you. A stronger rule trims first: `name.trimmingCharacters(in: .whitespaces).isEmpty`.
4. **A. .menu, which shows the current choice and lists all options on tap** Segmented controls appeal because everything is visible, but they only work for a handful of short options; 14 segments would be unreadably narrow. A menu keeps the form compact and still shows every option on tap. Toggles would allow several flavors at once, which the model does not allow.

Motion, Adaptivity, and Liquid Glass

CONCEPT

Animate changes of state

In SwiftUI you do not animate views; you animate *changes of state*. When a value changes inside an animation, SwiftUI compares the view tree before and after and smoothly interpolates every difference: position, size, color and opacity.

Explicit animation	Wrap a state change in withAnimation { }. Everything that changes because of it animates.
Implicit animation	Attach .animation(.easeInOut, value: x) to a view. That view animates whenever x changes, wherever the change came from.
Transition	How a view enters or leaves when an if adds or removes it: fade, scale or slide.
Reduce Motion	An accessibility setting for people who find movement on screen uncomfortable. Read it with @Environment(\.accessibilityReduceMotion) and skip large motion when it is true.

> **Motion should explain**
>
> Good animation shows cause and effect: the heart grows because you tapped it; the card appears where the toggle said it would. If an animation only decorates, cut it, and keep durations short so the app never feels slow.

CODE · LIVE SWIFTUI PREVIEW

Explicit animation with withAnimation

A favorite button. Tapping it flips one Boolean inside `withAnimation`, and three things change because of it: the symbol, the color and the size. SwiftUI animates all of them together with a spring. Tap Favorite in the phone a few times.

```swift
import SwiftUI

struct FavoriteButtonView: View {
    @State private var isFavorite = false

    var body: some View {
        VStack(spacing: 24) {
            Image(systemName: isFavorite ? "heart.fill" : "heart")
                .font(.system(size: 72))
                .foregroundStyle(isFavorite ? .pink : .gray)
                .scaleEffect(isFavorite ? 1.3 : 1.0)
            Button(isFavorite ? "Unfavorite" : "Favorite") {
                withAnimation(.spring) {
                    isFavorite.toggle()
                }
            }
            .buttonStyle(.borderedProminent)
        }
    }
}

#Preview { FavoriteButtonView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the favorite button.
- **WHAT** **Brings in withAnimation, the spring animation and scaleEffect, alongside the usual views.**
- **WHERE** Top of the file; it covers FavoriteButtonView and its preview.
- **WHEN** At compile time, before FavoriteButtonView is type-checked.
- **WHY** Animation is built into SwiftUI; there is no separate animation framework to learn.

3 `struct FavoriteButtonView: View {`

- **WHO** Your code declares the control; SwiftUI creates it and animates it.
- **WHAT** **A heart that can be favorited and unfavorited.**
- **WHERE** At file scope, so the preview or any list row could use it.
- **WHEN** Created when the preview launches, with the heart unfavorited.
- **WHY** A tiny, self-contained control is the clearest place to see animation at work.

4 `@State private var isFavorite = false`

- **WHO** SwiftUI stores it; the button changes it.
- **WHAT** **Whether the item is a favorite, starting false.**
- **WHERE** In SwiftUI-managed storage tied to FavoriteButtonView.
- **WHEN** Flipped on every tap of the button, inside withAnimation.
- **WHY** It is the only state; every visual difference below is derived from it.

6 `var body: some View {`

- **WHO** SwiftUI calls body before and after each tap to compare the results.
- **WHAT** **Describes the heart and the button for the current value of isFavorite.**
- **WHERE** Inside FavoriteButtonView, on the main thread.
- **WHEN** On first appearance and again after each tap changes isFavorite.
- **WHY** SwiftUI compares this description before and after the tap to work out what to animate.

```
7  VStack(spacing: 24) {
```

WHO SwiftUI's layout system, stacking the heart above its button.
WHAT **Stacks the heart above the button with 24 points between them.**
WHERE The root of FavoriteButtonView's body, centered on the screen.
WHEN Laid out on every body run; the heart's scale does not change its layout size.
WHY Generous space gives the growing heart room to move without crowding the button.

```
8  Image(systemName: isFavorite ? "heart.fill" : "heart")
```

WHO SwiftUI, drawing the heart symbol chosen by the ternary.
WHAT **Shows a filled heart when favorited and an outline otherwise.**
WHERE Top child of the VStack, above the button.
WHEN Rebuilt after each tap, switching between outline and filled.
WHY Filled versus outline is the standard iOS way to show an on-or-off state with a symbol.

```
9  .font(.system(size: 72))
```

WHO SwiftUI, which sizes SF Symbols the same way it sizes text.
WHAT **Draws the heart symbol at a 72-point font size.**
WHERE On the Image, before its color and scale modifiers.
WHEN Applied each time the Image is rebuilt during a body run.
WHY SF Symbols are sized like text, so font controls their size. A fixed size suits a decorative icon; for text, prefer text styles.

```
10 .foregroundStyle(isFavorite ? .pink : .gray)
```

WHO SwiftUI, which picks pink or gray from isFavorite.
WHAT **Colors the heart pink when favorited and gray otherwise.**
WHERE On the Image, after its size and before its scale.
WHEN Rebuilt after each tap; the color change is animated too.
WHY Color reinforces the change, but the symbol also switches from outline to filled, so the state stays clear to people who cannot tell pink from gray.

```
11 .scaleEffect(isFavorite ? 1.3 : 1.0)
```

WHO SwiftUI, applying a visual transform to the drawn heart.
WHAT **Draws the heart 30 percent larger when favorited.**
WHERE Wraps the sized, colored Image, after the other two modifiers.
WHEN On each body run; inside withAnimation, SwiftUI interpolates between 1.0 and 1.3.
WHY scaleEffect changes how the view is drawn, not the space it takes in layout, so the button below does not jump.

```
12 Button(isFavorite ? "Unfavorite" : "Favorite") {
```

WHO The user taps it to favorite or unfavorite the heart.
WHAT **A button whose title describes what the next tap will do.**
WHERE Second child of the VStack, below the heart.
WHEN Its title is rebuilt after each tap; its action runs on tap.
WHY A title that names the next action tells people exactly what will happen.

```
13 withAnimation(.spring) {
```

WHO SwiftUI's animation system, which plays a spring for this one change.
WHAT **Runs the closure and animates every view difference the state change causes, using a spring curve.**
WHERE Inside the button's action, wrapping only the state change.
WHEN On tap; the motion plays over the next fraction of a second.
WHY Explicit animation ties motion to one event: this tap animates, while a change to isFavorite from anywhere else would not.

```
14  isFavorite.toggle()
```

WHO	Your action closure writes the state; SwiftUI notices the write.
WHAT	**Flips isFavorite from false to true or from true to false.**
WHERE	Inside the withAnimation closure, as its only statement.
WHEN	Immediately on tap, before the spring animation starts.
WHY	toggle() reads like the action itself and is clearer than isFavorite = !isFavorite.

```
17  .buttonStyle(.borderedProminent)
```

WHO	SwiftUI, using the system's prominent button appearance.
WHAT	**Gives the Favorite button a filled, tinted capsule background.**
WHERE	Attached to the Button, after its action closure.
WHEN	Applied whenever the button is rebuilt, including when its title changes.
WHY	It is the only action on screen, so it may be prominent.

```
22  #Preview { FavoriteButtonView() }
```

WHO	Xcode's preview canvas, and the simulated iPhone beside this code, running FavoriteButtonView.
WHAT	**Shows the favorite button so you can tap it.**
WHERE	At file scope, after FavoriteButtonView; preview code is not part of the shipped app.
WHEN	Every time the code changes; the preview restarts with an outline heart.
WHY	Animation has to be seen; tapping repeatedly shows how springs settle.

> **Try it**
>
> 1. Remove withAnimation, keeping only isFavorite.toggle(). The heart still changes, but instantly.
> 2. Change .spring to .easeInOut and compare the feel.
> 3. Change 1.3 to 2.0.

CODE · LIVE SWIFTUI PREVIEW

Transitions for views that come and go

A recipe card that reveals nutrition facts. The facts exist only while `showNutrition` is true, so they are *inserted* and *removed*. `.transition(.scale)` says how they enter and leave, and `.animation(_:value:)` on the stack animates every change of `showNutrition`. Flip the switch in the phone.

```swift
import SwiftUI

struct RecipeCardView: View {
    @State private var showNutrition = false

    var body: some View {
        VStack(spacing: 16) {
            Text("Lentil Soup").font(.title)
            Toggle("Show nutrition", isOn: $showNutrition)
            if showNutrition {
                Text("320 calories, 18 g protein")
                    .padding()
                    .background(Color.mint)
                    .transition(.scale)
            }
        }
        .padding()
        .animation(.easeInOut, value: showNutrition)
    }
}

#Preview { RecipeCardView() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the recipe card.
- WHAT — **Brings in Toggle, the transition modifier and the animation(_:value:) modifier.**
- WHERE — Top of the file; it covers RecipeCardView and its preview.
- WHEN — At compile time, before RecipeCardView is type-checked.
- WHY — Transitions and implicit animations are both SwiftUI modifiers.

3 `struct RecipeCardView: View {`

- WHO — Your code declares the card; SwiftUI creates it and runs its transitions.
- WHAT — **A recipe card that can reveal extra details.**
- WHERE — At file scope, so the preview can create RecipeCardView directly.
- WHEN — Created when the preview launches, with the facts hidden.
- WHY — Progressive disclosure, showing details on request, keeps a screen calm.

4 `@State private var showNutrition = false`

- WHO — SwiftUI stores it; the Toggle writes it.
- WHAT — **Whether the nutrition facts are visible, starting hidden.**
- WHERE — In SwiftUI-managed storage tied to RecipeCardView.
- WHEN — Changed each time the Show nutrition switch is flipped.
- WHY — Visibility is state, so showing and hiding is just changing a Bool.

6 `var body: some View {`

- WHO — SwiftUI calls body again each time showNutrition changes.
- WHAT — **Describes the title, the switch and, conditionally, the facts.**
- WHERE — Inside RecipeCardView, on the main thread.
- WHEN — On first appearance and after every flip.
- WHY — Whether the facts exist is decided fresh on each run of body.

```
7  VStack(spacing: 16) {
```

WHO SwiftUI's layout system, stacking the card's title, switch and facts.
WHAT **Stacks the card's parts vertically, 16 points apart.**
WHERE The root of RecipeCardView's body, centered on the screen.
WHEN Laid out on every body run; its height changes when the facts come and go.
WHY The stack resizes as children are inserted, and the animation below smooths that too.

```
8  Text("Lentil Soup").font(.title)
```

WHO SwiftUI, drawing the card's heading in the title text style.
WHAT **The card's title, in the title text style.**
WHERE Top child of the VStack, above the switch.
WHEN Drawn on every body run; its text never changes.
WHY Chaining a short modifier on the same line is fine when it stays readable.

```
9  Toggle("Show nutrition", isOn: $showNutrition)
```

WHO The user flips it; the binding writes showNutrition.
WHAT **A switch that controls whether the facts are shown.**
WHERE Second child of the VStack, below the title.
WHEN Each flip writes the state; the stack's animation handles the motion.
WHY The Toggle knows nothing about animation; motion is attached where the change is displayed.

```
10 if showNutrition {
```

WHO SwiftUI evaluates this condition each time body runs.
WHAT **Includes the facts in the view tree only while showNutrition is true.**
WHERE Inside the VStack, below the Toggle, as its last child.
WHEN On every body run; flipping the state inserts or removes the Text.
WHY An if in body is how views appear and disappear in SwiftUI; there is no hide() method to call.

```
11 Text("320 calories, 18 g protein")
```

WHO SwiftUI, which creates this Text only while the condition is true.
WHAT **The nutrition facts: calories and protein for the soup.**
WHERE Inside the if, so it exists only when the condition is true.
WHEN Created on insertion and discarded on removal.
WHY Because it is created and destroyed, it needs a transition to describe its entrance and exit.

```
12 .padding()
```

WHO SwiftUI, which adds the default padding on every side.
WHAT **Adds standard spacing around the facts text before it gets a color.**
WHERE On the facts Text, before the background.
WHEN Applied each time the facts Text is built after an insertion.
WHY Padding comes before background so the colored area is larger than the text itself.

```
13 .background(Color.mint)
```

WHO SwiftUI, which fills the area behind the padded Text.
WHAT **Fills the padded area with mint green, making a colored block.**
WHERE Behind the padded Text, after the padding modifier.
WHEN Drawn whenever the facts are in the view tree.
WHY The colored block makes the scaling transition easy to see.

```
14  .transition(.scale)
```

WHO	SwiftUI, which runs the transition during insertion and removal.
WHAT	**Makes the facts grow from nothing when inserted and shrink away when removed.**
WHERE	On the view that is inserted and removed, inside the if.
WHEN	Only when the view enters or leaves while an animation is active.
WHY	A transition without an animation does nothing visible; here the .animation on the VStack supplies one.

```
17  .padding()
```

WHO	SwiftUI, which adds default spacing around the whole stack.
WHAT	**Adds standard spacing around the entire card, from title to facts.**
WHERE	On the VStack, after its closing brace, so it pads the whole card.
WHEN	Applied on every body run, as the stack grows and shrinks.
WHY	Keeps the switch and the facts away from the screen edges.

```
18  .animation(.easeInOut, value: showNutrition)
```

WHO	SwiftUI's animation system, watching showNutrition for changes.
WHAT	**Animates every change inside the VStack caused by showNutrition changing, with an ease-in-out curve.**
WHERE	On the VStack, so it covers the facts and the stack's own resizing.
WHEN	Each time showNutrition changes, whichever control changed it.
WHY	The value: parameter scopes the animation to one piece of state, so unrelated changes do not animate by surprise.

```
22  #Preview { RecipeCardView() }
```

WHO	Xcode's preview canvas, and the simulated iPhone beside this code, running RecipeCardView.
WHAT	**Shows the recipe card so you can flip the switch.**
WHERE	At file scope, after RecipeCardView; preview code is not part of the shipped app.
WHEN	Every time the code changes; the preview restarts with the facts hidden.
WHY	Transitions are easiest to compare by switching them back and forth.

> **Try it**
>
> 1. Change .scale to .opacity, then to .slide.
> 2. Delete the .animation line and flip the switch. The facts still appear, but with no motion.
> 3. Swap the .padding() and .background(Color.mint) lines on the facts. What happens to the mint area?

CONCEPT

One layout, many sizes

Your interface runs on a small iPhone, a large iPhone, an iPad in split view, and at whatever text size each reader chooses. *Dynamic Type* is the iOS setting that lets people pick their text size, from extra small up through five extra-large accessibility sizes. Text styles such as `.body` and `.headline` follow that setting automatically; fixed point sizes do not.

Flexible frames	Prefer stacks and maxWidth: .infinity over fixed widths and heights, so content can grow.
ViewThatFits	Offer several layouts in order of preference; SwiftUI shows the first one that fits.
Size classes	iOS reports a compact or regular width. An iPhone in portrait is compact; a full-screen iPad is regular. Use it to choose between one column and two.

Test the extremes Check the smallest device and the largest accessibility text size before calling a screen done.

> **Never truncate meaning**
>
> When text grows, let it wrap onto more lines, or switch a row of items to a column. Cutting words off with an ellipsis hides information from exactly the people who asked for larger text.

CODE · LIVE SWIFTUI PREVIEW

ViewThatFits picks the layout that fits

A delivery banner with three wordings, longest first. `ViewThatFits` measures each child at its natural size and shows the first one that fits. The slider changes the available width, standing in for a smaller phone, split view on iPad, or larger text. Drag it slowly to the left.

```swift
import SwiftUI

struct DeliveryBannerView: View {
    @State private var width = 340.0

    var body: some View {
        VStack(spacing: 30) {
            Slider(value: $width, in: 80...340)
            ViewThatFits {
                Text("Free delivery on orders over $25")
                Text("Free delivery over $25")
                Text("Free delivery")
            }
            .font(.headline)
            .frame(width: width)
            .border(Color.gray)
        }
        .padding()
    }
}

#Preview { DeliveryBannerView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the delivery banner.
- **WHAT** **Brings in ViewThatFits and Slider, along with Text and VStack.**
- **WHERE** Top of the file; it covers DeliveryBannerView and its preview.
- **WHEN** At compile time, before DeliveryBannerView is type-checked.
- **WHY** Adaptive layout tools are part of SwiftUI's layout system.

```
3  struct DeliveryBannerView: View {
```

WHO Your code declares the screen; SwiftUI creates it and lays it out.
WHAT **A banner that adapts its wording to the space available.**
WHERE At file scope, so the preview can create DeliveryBannerView directly.
WHEN Created when the preview launches, at the full 340-point width.
WHY Banners and button rows are the classic places where space runs out.

```
4  @State private var width = 340.0
```

WHO SwiftUI stores it; the Slider writes it.
WHAT **The width, in points, offered to the banner.**
WHERE In SwiftUI-managed storage tied to DeliveryBannerView.
WHEN Changed continuously while the slider is dragged.
WHY A live width lets you watch the decision happen; in a real app, the device and text size decide the width.

```
6  var body: some View {
```

WHO SwiftUI calls body again each time the slider changes width.
WHAT **Describes the slider and the adaptive banner.**
WHERE Inside DeliveryBannerView, on the main thread.
WHEN On first appearance and while the slider moves.
WHY Each new width triggers a fresh layout pass, where ViewThatFits decides again.

```
7  VStack(spacing: 30) {
```

WHO SwiftUI's layout system, stacking the slider above the banner.
WHAT **Stacks the slider above the banner, 30 points apart.**
WHERE The root of DeliveryBannerView's body, wrapping both children.
WHEN Laid out on every body run, which happens continuously while dragging.
WHY Keeps the control and the thing it controls visually separate.

```
8  Slider(value: $width, in: 80...340)
```

WHO The user drags it; the binding writes the new width.
WHAT **Sets the banner width anywhere from 80 to 340 points.**
WHERE Top child of the VStack, above the banner.
WHEN Writes width continuously while dragging.
WHY 80 points is narrower than every wording, so you can also see what happens when nothing fits.

```
9  ViewThatFits {
```

WHO SwiftUI's layout system, measuring each child against the offered width.
WHAT **Considers its children in order and displays only the first one that fits the space offered.**
WHERE Second child of the VStack, below the slider.
WHEN On every layout pass, including each time the width changes.
WHY You list your preferences once, and SwiftUI chooses on every device and at every text size.

```
10 Text("Free delivery on orders over $25")
```

WHO SwiftUI, which measures this wording first.
WHAT **The first and most informative wording, naming both the offer and the threshold.**
WHERE First child of ViewThatFits, its top preference.
WHEN Shown whenever the offered width is enough for the full sentence.
WHY Order matters: put the richest version first, so it wins whenever it can.

```
11  Text("Free delivery over $25")
```

WHO SwiftUI, which measures this wording only if the first does not fit.
WHAT **A shorter fallback that keeps the key condition.**
WHERE Second child of ViewThatFits, its first fallback.
WHEN Shown when the first wording does not fit but this one does.
WHY A good fallback keeps the essential fact, here the 25-dollar threshold.

```
12  Text("Free delivery")
```

WHO SwiftUI, which uses this wording when the others are too wide.
WHAT **The shortest wording, and the last child.**
WHERE Third and last child of ViewThatFits, its final fallback.
WHEN Shown when neither longer wording fits, or when nothing fits at all.
WHY If nothing fits, ViewThatFits uses the last child anyway, so make it the most compact.

```
14  .font(.headline)
```

WHO SwiftUI and Dynamic Type, which scale the headline style.
WHAT **Applies the headline text style to all three wordings.**
WHERE On ViewThatFits, so it flows down to every child.
WHEN Applied on every body run, before ViewThatFits measures its children.
WHY Because it is a text style, larger text sizes make each wording wider, which triggers the fallbacks too.

```
15  .frame(width: width)
```

WHO SwiftUI's layout system, which offers the banner a fixed width.
WHAT **Offers the banner exactly width points of horizontal space, as set by the slider.**
WHERE Wraps ViewThatFits, after the font modifier.
WHEN Recomputed as the slider moves, triggering a new fit decision.
WHY This is the space ViewThatFits measures each candidate against.

```
16  .border(Color.gray)
```

WHO SwiftUI, which draws the outline over the frame's edge.
WHAT **Draws a thin gray outline along the edge of the frame.**
WHERE On the frame, drawn just inside its bounds; a border never changes a view's size.
WHEN Drawn on every body run, tracking the frame as it narrows.
WHY It makes the invisible frame visible, so you can see the space each wording must fit into.

```
18  .padding()
```

WHO SwiftUI, which adds default spacing around the stack.
WHAT **Adds standard spacing around the slider and banner together.**
WHERE On the outer VStack, after its closing brace.
WHEN Applied on every body run, around the whole stack.
WHY Keeps the slider's ends away from the screen edges, where they are hard to grab.

```
22  #Preview { DeliveryBannerView() }
```

WHO Xcode's preview canvas, and the simulated iPhone beside this code, running DeliveryBannerView.
WHAT **Shows the banner and its slider so you can drag the width.**
WHERE At file scope, after DeliveryBannerView; preview code is not part of the shipped app.
WHEN Every time the code changes; the preview restarts at 340 points.
WHY Dragging the slider replaces the need for many test devices.

Try it

1. Drag the slider until the wording changes, then keep going past the shortest one.
2. Replace the three Texts with two candidates: an HStack holding Button("Keep shopping") { } and Button("Check out") { }, then a lone Button("Check out") { }. Drag again.
3. Change .headline to .largeTitle. Each wording now gives up at a wider width.

CODE · LIVE SWIFTUI PREVIEW

A first look at Liquid Glass

A preview of the Design module. iOS 26 introduced *Liquid Glass*, a translucent material for controls that float above content. `.glassEffect()` places any view on glass, `.buttonStyle(.glassProminent)` makes a glass button, and `GlassEffectContainer` lets nearby glass shapes render and blend together. Tap Play in the phone.

```swift
import SwiftUI

struct PlayerControlsView: View {
    @State private var isPlaying = false
    var title: String { isPlaying ? "Pause" : "Play" }
    var body: some View {
        ZStack {
            Color.teal.ignoresSafeArea()
            GlassEffectContainer {
                HStack(spacing: 16) {
                    Image(systemName: "music.note")
                        .padding()
                        .glassEffect()
                    Button(title) { isPlaying.toggle() }
                        .buttonStyle(.glassProminent)
                }
            }
        }
    }
}

#Preview { PlayerControlsView() }
```

Line by line

`1` `import SwiftUI`

WHO	The Swift compiler, while it builds the player controls.
WHAT	**Brings in the Liquid Glass APIs: glassEffect, GlassEffectContainer and the glass button styles.**
WHERE	Top of the file; it covers PlayerControlsView and its preview.
WHEN	At compile time; the glass APIs are checked against the iOS 26 SDK.
WHY	These APIs are available in iOS 26 and later, so an app that supports older versions must check availability.

```
3  struct PlayerControlsView: View {
```

WHO Your code declares the controls; SwiftUI renders them as glass.
WHAT **Floating playback controls for a music app.**
WHERE At file scope, so the preview can create PlayerControlsView directly.
WHEN Created when the preview launches, with playback paused.
WHY Floating controls over content are exactly the job Liquid Glass was designed for.

```
4  @State private var isPlaying = false
```

WHO SwiftUI stores it; the Play or Pause button flips it.
WHAT **Whether music is playing, starting paused.**
WHERE In SwiftUI-managed storage tied to PlayerControlsView.
WHEN Flipped on each tap of the Play or Pause button.
WHY The button title is derived from this one value.

```
5  var title: String { isPlaying ? "Pause" : "Play" }
```

WHO Your code defines it; body reads it for the button title.
WHAT **Names the action the next tap will perform.**
WHERE A computed property inside PlayerControlsView, next to isPlaying.
WHEN Evaluated each time body runs, after every tap.
WHY A named property keeps the Button line short.

```
6  var body: some View {
```

WHO SwiftUI calls body again each time isPlaying changes.
WHAT **Describes a colored background with glass controls on top.**
WHERE Inside PlayerControlsView, on the main thread.
WHEN On first appearance and after each tap of the button.
WHY Glass is a layer above content, so the code is layered the same way.

```
7  ZStack {
```

WHO SwiftUI's layout system, layering the backdrop and the controls.
WHAT **Layers its children on top of each other, the first at the back.**
WHERE The root of PlayerControlsView's body, filling the screen.
WHEN Laid out on every body run, with the controls centered on top.
WHY Glass needs something behind it to bend and reflect; layering controls over content is where it belongs.

```
8  Color.teal.ignoresSafeArea()
```

WHO SwiftUI, filling the screen with a solid color view.
WHAT **Fills the whole screen with teal, including behind the status bar.**
WHERE Bottom layer of the ZStack, behind the glass controls.
WHEN Drawn on every body run, under everything else.
WHY It stands in for content such as album art; glass picks up the color of whatever is underneath.

```
9  GlassEffectContainer {
```

WHO SwiftUI's glass renderer, which treats the shapes inside as one group.
WHAT **Groups the glass shapes inside it so they render together and can blend into each other when close.**
WHERE Around the row of controls, above the background.
WHEN Whenever the controls are drawn, on each frame they are visible.
WHY Glass cannot sample other glass. The container renders neighboring glass shapes together, which looks right and is more efficient.

```
10 HStack(spacing: 16) {
```

WHO SwiftUI's layout system, arranging the floating controls in a row.
WHAT **Places the symbol and the button side by side.**
WHERE Inside the GlassEffectContainer, as its only child.
WHEN Laid out on every body run, after the button title changes.
WHY Related controls sit together on one floating bar.

```
11 Image(systemName: "music.note")
```

WHO SwiftUI, drawing an SF Symbol from the system symbol set.
WHAT **Shows a music note symbol as the player's icon.**
WHERE Leading side of the HStack, before the button.
WHEN Drawn on every body run; the symbol never changes.
WHY A simple shape makes the glass around it easy to see.

```
12 .padding()
```

WHO SwiftUI, which adds default spacing around the symbol.
WHAT **Adds standard spacing around the music note before the glass is applied.**
WHERE On the music note Image, before the glass effect.
WHEN Applied on every body run, ahead of glassEffect.
WHY Glass is drawn in the view's bounds, so padding gives the glass shape room around the symbol.

```
13 .glassEffect()
```

WHO SwiftUI and the iOS renderer, which draw the glass material.
WHAT **Places the padded symbol on a capsule of regular Liquid Glass.**
WHERE Behind the padded image, filling its bounds.
WHEN Every frame the view is visible; the glass reacts to the content beneath it.
WHY The default is the regular variant in a capsule; .glassEffect(.clear, in: .rect(cornerRadius: 16)) changes both.

```
14 Button(title) { isPlaying.toggle() }
```

WHO The user taps it to start or pause the music.
WHAT **A button titled Play or Pause that flips isPlaying.**
WHERE Trailing side of the HStack, after the music note.
WHEN Its action runs on tap; its title updates right after.
WHY The title always names the next action, which is the convention for play and pause controls.

```
15 .buttonStyle(.glassProminent)
```

WHO SwiftUI, using the iOS 26 glass button appearance.
WHAT **Draws the Play or Pause button as tinted, prominent glass.**
WHERE Attached to the Play or Pause button, after its action closure.
WHEN Applied whenever the button is rebuilt, including after its title changes.
WHY Prominent glass marks the primary action; the quieter .glass style suits secondary actions.

```
22 #Preview { PlayerControlsView() }
```

WHO Xcode's preview canvas, and the simulated iPhone beside this code, running PlayerControlsView.
WHAT **Shows the glass controls over the teal background.**
WHERE At file scope, after PlayerControlsView; preview code is not part of the shipped app.
WHEN Every time the code changes; the preview restarts paused.
WHY Glass depends on what is behind it, so try it over different colors.

Try it

1. Change .glassProminent to .glass and compare the two styles.
2. Change .glassEffect() to .glassEffect(.clear, in: .rect(cornerRadius: 16)).
3. Change Color.teal to Color.orange. The glass picks up the new color.

CHECK YOURSELF

Check your understanding

1. You remove `withAnimation` from the favorite button, leaving only `isFavorite.toggle()`. What happens when you tap?
   A. Nothing changes on screen, because animation drives the update
   B. The heart changes symbol, color and size instantly
   C. The code no longer compiles, because toggle() needs an animation
   D. Only the color animates; the symbol and size change instantly
2. A view inside `if showDetails { }` has `.transition(.slide)`, but it simply pops in with no motion. What is the most likely cause?
   A. .slide only works on iPad, so iPhone shows no motion
   B. Transitions only play inside a ZStack, not a VStack
   C. The view needs an explicit id before a transition can run
   D. The change to showDetails happens outside any animation
3. A `ViewThatFits` has three Text children, and the available width is narrower than all of them. What does it show?
   A. Nothing; the space stays empty until something fits
   B. The first child, cut off at the edge
   C. The last child, even though it does not fit
   D. All three children, stacked vertically to make room
4. Which line respects a reader's Dynamic Type setting?
   A. .font(.system(size: 17))
   B. .font(.body)
   C. .frame(height: 20)
   D. .lineLimit(1)
5. Where does Liquid Glass belong in an iOS 26 interface?
   A. On every view, including body text, for a consistent look
   B. Only on images and photos, where its lensing is visible
   C. On controls that float above content, such as tab bars and toolbars
   D. Nowhere in SwiftUI; it is only available through UIKit

Answers: Motion, Adaptivity, and Liquid Glass

1. **B. The heart changes symbol, color and size instantly** The state still changes, so body still runs and the screen still updates; animation only controls how the change is shown. Expecting nothing to change confuses animating with updating. SwiftUI always redraws after a state change.
2. **D. The change to showDetails happens outside any animation** A transition describes how a view enters or leaves, but it only plays when the insertion happens during an animation. Wrap the change in `withAnimation`, or add `.animation(.easeInOut, value: showDetails)` to the container. Transitions work in any container and on every device.
3. **C. The last child, even though it does not fit** ViewThatFits picks the first child that fits; when none does, it falls back to the last one. That is why the most compact option goes last. The first child, a natural guess, is the one that gets skipped first when space runs out.
4. **B. .font(.body)** `.body` is a text style, so it grows and shrinks with the person's chosen text size. A fixed 17 points matches the default body size, which is why it misleads, but it never changes. A fixed height or a one-line limit would clip larger text.
5. **C. On controls that float above content, such as tab bars and toolbars** Liquid Glass forms a functional layer for controls and navigation that float above content. Putting it on everything in the name of consistency blurs that layering and makes content harder to read. SwiftUI supports it directly with `glassEffect` and the glass button styles.

Worked Examples

WORKED EXAMPLE

Build a product card, modifier by modifier

Problem.
Build a product card for a pair of trail shoes: name, price and a picture, styled as a card with a soft background and rounded corners, plus an Add to Bag button that counts how many pairs the shopper added. Success: the card looks finished and the count rises when the button is tapped.

Frame 1 of 8

The card starts as a single line of text: the product name. A Text view is the simplest thing SwiftUI can draw, and every later frame adds exactly one idea to this same view.

```
import SwiftUI

struct ProductCard: View {
    var body: some View {
        Text("Trail Shoes")
    }
}
#Preview {
    ProductCard()
}
```

Frame 2 of 8

A VStack stacks the name above the price, leading-aligned so both start at the same left edge. The body now describes two pieces of information instead of one, in the order a shopper reads them.

```
 import SwiftUI

 struct ProductCard: View {
     var body: some View {
+        VStack(alignment: .leading, spacing: 6) {
+            Text("Trail Shoes")
+            Text("$89.00")
+        }
+    }
 }
+#Preview {
+    ProductCard()
+}
```

Frame 3 of 8

An Image view joins the top of the stack, showing the running figure from SF Symbols. The stack grows to fit it; nothing else moves, because layout comes from the description, not from stored positions.

```
  import SwiftUI

  struct ProductCard: View {
      var body: some View {
          VStack(alignment: .leading, spacing: 6) {
+             Image(systemName: "figure.run")
+             Text("Trail Shoes")
+             Text("$89.00")
+         }
+     }
+ }
+ #Preview {
+     ProductCard()
+ }
```

Frame 4 of 8

Type styles arrive: the picture grows with largeTitle, the name becomes a headline, and the price turns secondary gray. Modifiers wrap the view they follow, so each one is written under its own view.

```
  import SwiftUI

  struct ProductCard: View {
      var body: some View {
          VStack(alignment: .leading, spacing: 6) {
              Image(systemName: "figure.run")
+                 .font(.largeTitle)
+             Text("Trail Shoes")
+                 .font(.headline)
+             Text("$89.00")
+                 .foregroundStyle(.secondary)
+         }
+     }
  }
+ #Preview {
+     ProductCard()
+ }
```

Frame 5 of 8

Padding first, then background: the padding becomes part of the view, so the yellow covers the text and the breathing room around it. This is the card shape we want; compare it with the next frame.

```
    import SwiftUI

    struct ProductCard: View {
        var body: some View {
            VStack(alignment: .leading, spacing: 6) {
                Image(systemName: "figure.run")
                    .font(.largeTitle)
                Text("Trail Shoes")
                    .font(.headline)
                Text("$89.00")
                    .foregroundStyle(.secondary)
            }
+           .padding()
+           .background(.yellow)
+       }
+   }
+   #Preview {
+       ProductCard()
+   }
```

Frame 6 of 8

The same two modifiers in the opposite order: background first paints only the tight stack, and the padding then adds clear space outside the color. Modifier order is not style; it changes the result.

```
    import SwiftUI

    struct ProductCard: View {
        var body: some View {
            VStack(alignment: .leading, spacing: 6) {
                Image(systemName: "figure.run")
                    .font(.largeTitle)
                Text("Trail Shoes")
                    .font(.headline)
                Text("$89.00")
                    .foregroundStyle(.secondary)
            }
+           .background(.yellow)
+           .padding()
        }
    }
    #Preview {
        ProductCard()
    }
```

Frame 7 of 8

Back to padding first, and a clip shape rounds the yellow block’s corners. The card now reads as one object instead of a colored rectangle, ready for its interactive part.

```
  import SwiftUI

  struct ProductCard: View {
      var body: some View {
          VStack(alignment: .leading, spacing: 6) {
              Image(systemName: "figure.run")
                  .font(.largeTitle)
              Text("Trail Shoes")
                  .font(.headline)
              Text("$89.00")
                  .foregroundStyle(.secondary)
          }
+         .padding()
+         .background(.yellow)
+         .clipShape(RoundedRectangle(cornerRadius: 12))
+     }
+ }
+ #Preview {
+     ProductCard()
+ }
```

Frame 8 of 8

The button writes to @State, and the bag count below it is computed from that same state. Run the check: tapping Add to Bag re-evaluates body and the count becomes In your bag: 1.

```
  import SwiftUI

  struct ProductCard: View {
+     @State private var bag = 0
+     var body: some View {
+         VStack(alignment: .leading, spacing: 6) {
+             Image(systemName: "figure.run")
+                 .font(.largeTitle)
+             Text("Trail Shoes")
+                 .font(.headline)
+             Text("$89.00")
+                 .foregroundStyle(.secondary)
+             Button("Add to Bag") {
+                 bag += 1
+             }
+             Text("In your bag: \(bag)")
+         }
+         .padding()
+         .background(.yellow)
+         .clipShape(RoundedRectangle(cornerRadius: 12))
+     }
+ }
+ #Preview {
+     ProductCard()
+ }
```

FINAL RESULT — CHECKED ON SCREEN

```
Tap “Add to Bag” and the screen shows “In your bag: 1”.
```

Line by line: the finished program

```
1  import SwiftUI
```

WHO — The Swift compiler, which loads the SwiftUI framework for this file.
WHAT — **Makes View, Text, VStack, Image, Button and @State available in this file.**
WHERE — The first line of the file; an import covers only its own file.
WHEN — At compile time, before any of the code below is checked.
WHY — SwiftUI is a separate framework, so every file that describes a screen imports it.

```
3  struct ProductCard: View {
```

WHO — You declare it; SwiftUI creates values of it whenever the card is shown.
WHAT — **Declares ProductCard, a struct that conforms to View and describes the product card.**
WHERE — At file scope, so the preview at the bottom (and later, other screens) can create it.
WHEN — Compiled once; a fresh value is created whenever a parent asks for the card.
WHY — A screen in SwiftUI is a value that describes an interface, not a controller that runs it.

```
4  @State private var bag = 0
```

WHO — SwiftUI, which stores the value outside the struct, tied to this card on screen.
WHAT — **Declares bag, the number of pairs added, starting at 0; @State makes SwiftUI keep and watch it.**
WHERE — Inside ProductCard, marked private because only this view changes it.
WHEN — Created when the card first appears and kept across every redraw.
WHY — The count text below is computed from bag, so the button and the text can never disagree.

```
5  var body: some View {
```

WHO — SwiftUI calls this property; your code never calls body directly.
WHAT — **Returns the description of the card for the current state.**
WHERE — A computed property inside ProductCard.
WHEN — When the card appears, and again after every change to bag.
WHY — Describing the screen as a function of the data is the whole SwiftUI model.

```
6  VStack(alignment: .leading, spacing: 6) {
```

WHO — SwiftUI’s layout system, which measures the children and stacks them.
WHAT — **Stacks the picture, name, price and button vertically, leading-aligned, with 6 points between children.**
WHERE — The root view returned by body.
WHEN — Measured and placed again every time body produces a new description.
WHY — A stack states the arrangement once instead of giving every view its own coordinates.

```
7  Image(systemName: "figure.run")
```

WHO — SwiftUI, which looks the name up in the SF Symbols library built into the system.
WHAT — **Shows the figure.run symbol, a runner, as the product picture.**
WHERE — The first child of the stack, so it sits at the top of the card.
WHEN — Drawn every time body runs; the symbol is a template image the system can recolor.
WHY — A symbol stands in for product photography while the card is designed, with no image files to manage.

```
8  .font(.largeTitle)
```

WHO — The font modifier, applied to the Image view directly above it.
WHAT — **Sets the image size with the largeTitle text style, so the runner renders big.**
WHERE — Chained under the Image; modifiers apply to the view written before them.
WHEN — Read while body builds the description, before layout measures the image.
WHY — SF Symbols are drawn like a font, so a text style sizes them, and the size follows the user’s text-size setting.

```
9  Text("Trail Shoes")
```

WHO Swift, which builds the string; SwiftUI, which draws it as a Text view.
WHAT **Shows the product name, Trail Shoes.**
WHERE A child of the stack, under the picture.
WHEN Rebuilt every time body runs, though the text itself never changes.
WHY The name is the first thing a shopper reads, so it leads the text part of the card.

```
10  .font(.headline)
```

WHO The font modifier on the Text view directly above it.
WHAT **Makes the name a headline: semibold and sized to stand out from the price below.**
WHERE Chained under the name Text.
WHEN Applied while the description is built, before the text is measured.
WHY One clear hierarchy level tells the shopper which line matters most.

```
11  Text("$89.00")
```

WHO Swift builds the string; SwiftUI draws it as a Text view.
WHAT **Shows the price, $89.00, exactly as written.**
WHERE A child of the stack, under the name.
WHEN Rebuilt every time body runs.
WHY Price is the second fact a shopper looks for; it sits with the name, not in the button.

```
12  .foregroundStyle(.secondary)
```

WHO The foregroundStyle modifier on the price Text directly above it.
WHAT **Colors the price with the secondary style, a gray that steps back from the name.**
WHERE Chained under the price Text.
WHEN Applied while the description is built.
WHY De-emphasizing the price keeps the name first in the reading order without shrinking the text.

```
13  Button("Add to Bag") {
```

WHO You declare it; the shopper triggers its action by tapping.
WHAT **Creates the Add to Bag button; the closure after the label is its action.**
WHERE A child of the stack, under the price.
WHEN The closure runs at the moment of a tap, never while body is being built.
WHY Buying is the card's job; the button is placed with the facts it acts on.

```
14  bag += 1
```

WHO The button's action closure, running on the main thread.
WHAT **Adds one to bag, the @State count of pairs.**
WHERE Inside the button's action, in ProductCard.
WHEN Once per tap, at the moment of the tap.
WHY Changing the state is the whole action; SwiftUI notices and re-evaluates body by itself.

```
16  Text("In your bag: \(bag)")
```

WHO Swift builds the string from bag; SwiftUI draws it.
WHAT **Shows how many pairs are in the bag, using interpolation to read the current count.**
WHERE The last child of the stack, under the button.
WHEN Rebuilt after every tap, because it reads the state the tap changed.
WHY The shopper gets confirmation that the tap worked, from the same single source of truth.

18 `.padding()`

WHO The padding modifier, wrapping the whole stack.
WHAT **Adds standard spacing inside all four edges of the stack.**
WHERE The first modifier on the stack itself, after its closing brace.
WHEN Applied before the modifiers that follow it in the chain.
WHY Content that touches its container's edges looks broken; padding gives the card its breathing room.

19 `.background(.yellow)`

WHO The background modifier, wrapping everything written before it.
WHAT **Paints yellow behind the view it is applied to.**
WHERE Second in the modifier chain on the stack.
WHEN Its size is decided by the view built so far, padding included or not.
WHY A colored block groups the card's contents into one object the eye can find on a busy screen.

20 `.clipShape(RoundedRectangle(cornerRadius: 12))`

WHO The clipShape modifier, the last one in the chain.
WHAT **Clips the card, background included, to a rounded rectangle with 12-point corners.**
WHERE After background, so the yellow corners are the ones being rounded.
WHEN Applied to the finished stack description on every redraw.
WHY Rounded corners are the visual cue that separates a card from a plain colored rectangle.

23 `#Preview {`

WHO Xcode and this page, which render the card without running an app.
WHAT **Starts a preview block that shows ProductCard while you work on it.**
WHERE At the bottom of the file, outside the struct.
WHEN Whenever the file is opened or edited; previews are not part of the shipped app.
WHY Seeing the card while you build it is how the frames in this example were checked.

24 `ProductCard()`

WHO The preview renderer, which creates the view value.
WHAT **Creates one ProductCard for the preview to display.**
WHERE The only line inside the preview block.
WHEN Each time the preview refreshes.
WHY The preview needs a value to render; this is the same creation a real screen would do.

Try it

The shop runs a 20% discount on the trail shoes. The starter prints the total for two pairs at full price. Change the program so it applies the discount to each pair first, still working in whole cents, and prints the discounted total for two pairs.

```
let priceCents = 8995
let pairs = 2
let totalCents = priceCents * pairs
print("Total: $\(totalCents / 100).\(totalCents % 100)")
```

EXPECTED OUTPUT

```
Total: $143.92
```

WORKED EXAMPLE

@State: the lamp that can finally change

Problem.
A desk lamp screen shows a lamp that is off and a status line. Build it so a Turn On button switches the lamp on: the circle lights up and the status line agrees. Success: tapping Turn On shows the lamp on.

Frame 1 of 6

The lamp is drawn from fixed words and a fixed gray fill: a circle and a status line that always says off. Nothing here can change, because nothing here is allowed to change.

```
import SwiftUI

struct LampView: View {
    var body: some View {
        VStack(spacing: 16) {
            Circle()
                .fill(.gray)
                .frame(width: 80, height: 80)
            Text("The lamp is off")
        }
        .padding()
    }
}

#Preview { LampView() }
```

Frame 2 of 6

Both the fill and the status now read one constant, isOn, so they can never disagree. But isOn is a let inside a struct value: once the value is created, nothing can assign to it.

```
 import SwiftUI

 struct LampView: View {
+    let isOn = false
+    var body: some View {
+        VStack(spacing: 16) {
+            Circle()
+                .fill(isOn ? .yellow : .gray)
+                .frame(width: 80, height: 80)
+            Text(isOn ? "The lamp is on" : "The lamp is off")
+        }
+        .padding()
+    }
+}
+
+#Preview { LampView() }
```

Frame 3 of 6

The let becomes @State. SwiftUI stores that value outside the struct and watches it, so this view is finally allowed to change its lamp, and body will run again when it does.

```
  import SwiftUI

  struct LampView: View {
+     @State private var isOn = false
      var body: some View {
          VStack(spacing: 16) {
              Circle()
                  .fill(isOn ? .yellow : .gray)
                  .frame(width: 80, height: 80)
              Text(isOn ? "The lamp is on" : "The lamp is off")
          }
          .padding()
      }
  }

  #Preview { LampView() }
```

Frame 4 of 6

A Button joins the stack under the status. Its action toggles isOn; SwiftUI notices the state change, runs body again, and the circle and status redraw together from the new value.

```
  import SwiftUI

  struct LampView: View {
      @State private var isOn = false
      var body: some View {
          VStack(spacing: 16) {
              Circle()
                  .fill(isOn ? .yellow : .gray)
                  .frame(width: 80, height: 80)
              Text(isOn ? "The lamp is on" : "The lamp is off")
+             Button("Turn On") {
+                 isOn.toggle()
+             }
+         }
+         .padding()
+     }
+ }
+
+ #Preview { LampView() }
```

Frame 5 of 6

The status line becomes a headline, because it carries the result of the tap. The lamp itself is unchanged: same state, same button, same fill rule; only the emphasis moves.

```
  import SwiftUI

  struct LampView: View {
      @State private var isOn = false
      var body: some View {
          VStack(spacing: 16) {
              Circle()
                  .fill(isOn ? .yellow : .gray)
                  .frame(width: 80, height: 80)
              Text(isOn ? "The lamp is on" : "The lamp is off")
+                 .font(.headline)
+             Button("Turn On") {
+                 isOn.toggle()
+             }
+         }
+         .padding()
+     }
+ }
+
+ #Preview { LampView() }
```

Frame 6 of 6

The circle grows to 100 points so the lit lamp dominates the screen. Nothing about the logic changes in this frame; run the check and Turn On still ends with The lamp is on.

```
  import SwiftUI

  struct LampView: View {
      @State private var isOn = false
      var body: some View {
          VStack(spacing: 16) {
              Circle()
                  .fill(isOn ? .yellow : .gray)
+                 .frame(width: 100, height: 100)
              Text(isOn ? "The lamp is on" : "The lamp is off")
                  .font(.headline)
              Button("Turn On") {
                  isOn.toggle()
              }
          }
          .padding()
      }
  }

  #Preview { LampView() }
```

FINAL RESULT — CHECKED ON SCREEN

```
Tap “Turn On” and the screen shows “The lamp is on”.
```

Line by line: the finished program

```
1  import SwiftUI
```

WHO The Swift compiler, which loads the SwiftUI framework for this file.
WHAT **Makes View, Text, VStack, Circle, Button and @State available in this file.**
WHERE The first line of the file; an import covers only its own file.
WHEN At compile time, before any of the code below is checked.
WHY SwiftUI is a separate framework, so every file that describes a screen imports it.

```
3  struct LampView: View {
```

WHO You declare it; SwiftUI creates values of it whenever the lamp is shown.
WHAT **Declares LampView, a struct that conforms to View and describes the lamp screen.**
WHERE At file scope, so the preview at the bottom can create it.
WHEN Compiled once; a fresh value is created whenever a parent asks for the lamp.
WHY A screen in SwiftUI is a value that describes an interface, not an object that runs it.

```
4  @State private var isOn = false
```

WHO SwiftUI, which stores the value outside the struct, tied to this lamp on screen.
WHAT **Declares isOn, whether the lamp is lit, starting false; @State makes SwiftUI keep and watch it.**
WHERE Inside LampView, marked private because only this view changes it.
WHEN Created when the lamp first appears and kept across every redraw.
WHY The circle and the status below are both computed from isOn, so they can never disagree.

```
5  var body: some View {
```

WHO SwiftUI calls this property; your code never calls body directly.
WHAT **Returns the description of the lamp screen for the current data.**
WHERE A computed property inside LampView.
WHEN When the lamp appears, and again after every change to isOn.
WHY Describing the screen as a function of the data is the whole SwiftUI model.

```
6  VStack(spacing: 16) {
```

WHO SwiftUI's layout system, which measures the children and stacks them.
WHAT **Stacks the circle, the status and any button vertically, 16 points apart.**
WHERE The root view returned by body.
WHEN Measured and placed again every time body produces a new description.
WHY A stack states the arrangement once instead of giving every view its own coordinates.

```
7  Circle()
```

WHO SwiftUI, which draws the shape at the size its modifiers give it.
WHAT **Creates a circle shape, the lamp bulb itself.**
WHERE The first child of the stack, so it sits at the top of the screen.
WHEN Drawn every time body runs, with the fill decided by the modifier below.
WHY A plain circle is the simplest honest picture of a round lamp.

```
8  .fill(isOn ? .yellow : .gray)
```

WHO The fill modifier, applied to the Circle directly above it.
WHAT **Paints the circle yellow when isOn is true and gray when it is false.**
WHERE Chained under the Circle; it colors only that shape.
WHEN Recalculated every time body runs, from the current value of isOn.
WHY One rule covers both states of the lamp, so the picture can never claim the lamp is on when it is off.

```
9   .frame(width: 100, height: 100)
```

WHO The frame modifier, applied to the filled Circle above it.
WHAT **Fixes the circle at 100 points wide and 100 points tall.**
WHERE Chained under the fill, so it sizes the painted circle.
WHEN Measured while layout runs, every time body produces the circle.
WHY A bigger lamp makes the result of a tap unmissable from across the room.

```
10  Text(isOn ? "The lamp is on" : "The lamp is off")
```

WHO Swift builds one of the two strings; SwiftUI draws it as a Text view.
WHAT **Shows The lamp is on when isOn is true and The lamp is off when it is false.**
WHERE A child of the stack, under the circle.
WHEN Rebuilt after every change to isOn, because it reads the same value the fill reads.
WHY Words and color tell the same story because both are computed from one source of truth.

```
11  .font(.headline)
```

WHO The font modifier on the status Text directly above it.
WHAT **Makes the status a headline: semibold and sized to stand out on the screen.**
WHERE Chained under the status Text, so only the status grows.
WHEN Applied while the description is built, before the text is measured.
WHY The status carries the result of the tap, so it earns the emphasis.

```
12  Button("Turn On") {
```

WHO You declare it; the person using the app triggers its action by tapping.
WHAT **Creates the Turn On button; the closure after the label is its action.**
WHERE A child of the stack, under the status line.
WHEN The closure runs at the moment of a tap, never while body is being built.
WHY The button sits with the lamp it controls, so cause and effect are on one screen.

```
13  isOn.toggle()
```

WHO The button's action closure, running on the main thread.
WHAT **Toggles isOn: true becomes false, false becomes true.**
WHERE Inside the button's action, in LampView.
WHEN Once per tap, at the moment of the tap.
WHY Changing the state is the whole action; SwiftUI notices and re-evaluates body by itself.

```
16  .padding()
```

WHO SwiftUI's layout system, which adds the platform's standard margin around the stack.
WHAT **Adds standard spacing inside all four edges of the stack.**
WHERE The first modifier on the stack itself, after its closing brace.
WHEN Measured during the same layout pass as the stack it wraps.
WHY Content that touches the edges of its container looks broken; padding gives the lamp room.

```
20  #Preview { LampView() }
```

WHO Xcode and this page, which render the lamp without running an app.
WHAT **Creates one LampView for the preview to display.**
WHERE At file scope, after LampView; preview code is not part of the shipped app.
WHEN Each time the preview refreshes.
WHY The preview needs a value to render; this is the same creation a real screen would do.

Try it

The starter models a lamp that is toggled twice, so it ends off again. Change the program so the lamp is toggled three times instead, and print the final status exactly as the lamp screen shows it.

```
var isOn = false
isOn.toggle()
isOn.toggle()
print(isOn ? "The lamp is on" : "The lamp is off")
```

EXPECTED OUTPUT

```
The lamp is on
```

WORKED EXAMPLE

Parent and child editing through a @Binding

Problem.

A focus timer shows its length in the parent view, and a child adjuster must change that same length. Build both views so stepping the child updates the parent display too. Success: tapping Add a minute shows Focus length: 26 min.

Frame 1 of 6

The parent owns the focus length as @State and shows it once. This is the whole screen so far: one view, one value, and a display that is computed straight from that value.

```
import SwiftUI

struct FocusTimerView: View {
    @State private var minutes = 25
    var body: some View {
        VStack(spacing: 16) {
            Text("Focus length: \(minutes) min")
        }
    }
}

#Preview { FocusTimerView() }
```

Frame 2 of 6

The adjuster wording is written a second time inside the same view, reading the same state. It works here, but the display code is now duplicated, and duplication is what extraction should remove.

```
  import SwiftUI

  struct FocusTimerView: View {
      @State private var minutes = 25
      var body: some View {
          VStack(spacing: 16) {
              Text("Focus length: \(minutes) min")
+             Text("Adjuster sees \(minutes) min")
+         }
+     }
+ }
+
+ #Preview { FocusTimerView() }
```

Frame 3 of 6

The adjuster is extracted into its own view, but it declares its own @State minutes. Tapping Add a minute raises only the child's copy; the parent still shows 25, so the two displays disagree.

```
  import SwiftUI

+ struct MinuteAdjuster: View {
      @State private var minutes = 25
      var body: some View {
+         VStack(spacing: 8) {
+             Text("Adjuster sees \(minutes) min")
+             Button("Add a minute") { minutes += 1 }
          }
      }
  }

+ struct FocusTimerView: View {
+     @State private var minutes = 25
+     var body: some View {
+         VStack(spacing: 16) {
+             Text("Focus length: \(minutes) min")
+             MinuteAdjuster()
+         }
+     }
+ }
+
+ #Preview { FocusTimerView() }
```

Frame 4 of 6

The child now declares @Binding var minutes, and the parent passes $minutes, a connection to its own state. The child stores nothing; its button writes straight into the parent's one value.

```
  import SwiftUI

  struct MinuteAdjuster: View {
+     @Binding var minutes: Int
      var body: some View {
          VStack(spacing: 8) {
              Text("Adjuster sees \(minutes) min")
              Button("Add a minute") { minutes += 1 }
          }
      }
  }

  struct FocusTimerView: View {
      @State private var minutes = 25
      var body: some View {
          VStack(spacing: 16) {
              Text("Focus length: \(minutes) min")
+             MinuteAdjuster(minutes: $minutes)
          }
      }
  }

  #Preview { FocusTimerView() }
```

Frame 5 of 6

The parent gains a Reset button that writes the same state from its side. Because the child reads through the binding, it follows the parent's write immediately: one value, two writers, no copies.

```
  import SwiftUI

  struct MinuteAdjuster: View {
      @Binding var minutes: Int
      var body: some View {
          VStack(spacing: 8) {
              Text("Adjuster sees \(minutes) min")
              Button("Add a minute") { minutes += 1 }
          }
      }
  }

  struct FocusTimerView: View {
      @State private var minutes = 25
      var body: some View {
          VStack(spacing: 16) {
              Text("Focus length: \(minutes) min")
              MinuteAdjuster(minutes: $minutes)
+             Button("Reset") { minutes = 25 }
+         }
+     }
+ }
+
+ #Preview { FocusTimerView() }
```

Frame 6 of 6

A headline names the screen, and the build is finished. Run the check: Add a minute writes through the binding, and the parent's own display rises to Focus length: 26 min.

```
  import SwiftUI

  struct MinuteAdjuster: View {
      @Binding var minutes: Int
      var body: some View {
          VStack(spacing: 8) {
              Text("Adjuster sees \(minutes) min")
              Button("Add a minute") { minutes += 1 }
          }
      }
  }

  struct FocusTimerView: View {
      @State private var minutes = 25
      var body: some View {
          VStack(spacing: 16) {
+             Text("Focus Timer").font(.headline)
+             Text("Focus length: \(minutes) min")
+             MinuteAdjuster(minutes: $minutes)
+             Button("Reset") { minutes = 25 }
+         }
+     }
+ }
+
+ #Preview { FocusTimerView() }
```

FINAL RESULT — CHECKED ON SCREEN

```
Tap “Add a minute” and the screen shows “Focus length: 26 min”.
```

Line by line: the finished program

1 `import SwiftUI`

- **WHO** The Swift compiler, which loads the SwiftUI framework for this file.
- **WHAT** **Makes View, Text, VStack, Button, @State and @Binding available in this file.**
- **WHERE** The first line of the file; an import covers only its own file.
- **WHEN** At compile time, before any of the code below is checked.
- **WHY** One import serves every view declared in the same file.

3 `struct MinuteAdjuster: View {`

- **WHO** You declare it; the parent view creates values of it.
- **WHAT** **Declares MinuteAdjuster, a reusable child view that shows and changes the focus length.**
- **WHERE** At file scope, above the parent that creates it.
- **WHEN** Compiled once; a fresh value is created whenever the parent body runs.
- **WHY** Extracting the adjuster into its own view is what raises the ownership question this example answers.

```
4  @Binding var minutes: Int
```

WHO	The parent supplies it; SwiftUI routes reads and writes through it to the parent storage.
WHAT	**Declares minutes as a binding: a two-way connection to an Int stored in the parent, holding nothing itself.**
WHERE	Inside MinuteAdjuster, where its own @State stood before.
WHEN	Connected at the moment the parent creates the adjuster with $minutes.
WHY	The child can finally change the one shared value instead of a copy, so both views always agree.

```
5  var body: some View {
```

WHO	SwiftUI calls this property; your code never calls body directly.
WHAT	**Returns the description of the adjuster for the current minutes.**
WHERE	A computed property inside MinuteAdjuster.
WHEN	When the adjuster appears, and again after every change to minutes.
WHY	The adjuster describes itself from the value it is given, wherever that value lives.

```
6  VStack(spacing: 8) {
```

WHO	SwiftUI's layout system, which measures the children and stacks them.
WHAT	**Stacks the readout above the button, 8 points apart.**
WHERE	The root view returned by the child's body.
WHEN	Measured and placed again every time body produces a new description.
WHY	The control and its readout stay together as one reusable piece.

```
7  Text("Adjuster sees \(minutes) min")
```

WHO	Swift builds the string from minutes; SwiftUI draws it as a Text view.
WHAT	**Shows the length as the adjuster sees it, such as Adjuster sees 25 min.**
WHERE	The first child of the child stack, above the button.
WHEN	Rebuilt every time body runs, with the minutes the child currently reads.
WHY	Seeing the child readout next to the parent display is how the disagreement, and later the agreement, becomes visible.

```
8  Button("Add a minute") { minutes += 1 }
```

WHO	You declare it; the person using the app triggers it by tapping.
WHAT	**Creates the Add a minute button, whose whole action adds one to minutes.**
WHERE	A child of the child stack, under the readout.
WHEN	The action runs at the moment of a tap, never while body is being built.
WHY	One line is the whole action: which minutes it changes is decided entirely by how minutes is declared above.

```
13  struct FocusTimerView: View {
```

WHO	You declare it; the preview creates values of it.
WHAT	**Declares FocusTimerView, the parent screen that owns the focus length.**
WHERE	At file scope, below the child it creates.
WHEN	Compiled once; created by the preview and, in an app, by whatever shows the timer.
WHY	The parent is the highest view that needs the length, so it is the natural owner of the state.

```
14  @State private var minutes = 25
```

WHO	SwiftUI, which stores the value outside the struct, tied to this screen on display.
WHAT	**Declares minutes, the focus length in minutes, starting at 25; this is the single source of truth.**
WHERE	Inside FocusTimerView, marked private because this view owns it.
WHEN	Created when the timer first appears and kept across every redraw.
WHY	Exactly one view owns each value; every other view borrows it or receives it.

```
15  var body: some View {
```

WHO SwiftUI calls this property; your code never calls body directly.
WHAT **Returns the description of the timer screen for the current minutes.**
WHERE A computed property inside FocusTimerView.
WHEN When the screen appears, and again after every change to minutes, whoever made it.
WHY Because the display is computed from state, a change made by the child shows up here without any copying code.

```
16      VStack(spacing: 16) {
```

WHO SwiftUI's layout system, which measures the children and stacks them.
WHAT **Stacks the displays and the adjuster vertically, 16 points apart.**
WHERE The root view returned by the parent's body.
WHEN Measured and placed again every time body produces a new description.
WHY One column keeps the owner display and the borrowed control in reading order.

```
17          Text("Focus Timer").font(.headline)
```

WHO Swift builds the string; SwiftUI draws it and applies the chained font.
WHAT **Shows the screen title, Focus Timer, as a headline.**
WHERE The first child of the parent stack, above the length display.
WHEN Rebuilt every time body runs, though the words never change.
WHY A named screen orients the reader before the numbers start changing.

```
18          Text("Focus length: \(minutes) min")
```

WHO Swift builds the string from minutes; SwiftUI draws it as a Text view.
WHAT **Shows the focus length from the parent state, such as Focus length: 25 min.**
WHERE A child of the parent stack.
WHEN Rebuilt after every change to minutes, including changes the child makes.
WHY This line is the test of the whole example: if the child truly shares the value, this number moves.

```
19          MinuteAdjuster(minutes: $minutes)
```

WHO The parent, FocusTimerView, creating the child view and handing it the connection.
WHAT **Creates one MinuteAdjuster and passes $minutes: a binding to the parent state, not a copy of the number.**
WHERE A child of the parent stack, under the length display.
WHEN A new child value is created every time the parent body runs, bound to the same storage.
WHY The dollar sign is the whole fix: it passes read-and-write access, so the child and parent share one value.

```
20          Button("Reset") { minutes = 25 }
```

WHO You declare it; the person using the app triggers it by tapping.
WHAT **Creates a Reset button whose action writes 25 back into the parent minutes.**
WHERE A child of the parent stack, under the adjuster.
WHEN The action runs at the moment of a tap.
WHY A write from the parent side proves the traffic is two-way: the child readout follows without being touched.

```
25  #Preview { FocusTimerView() }
```

WHO Xcode and this page, which render the timer without running an app.
WHAT **Creates one FocusTimerView for the preview, which in turn creates the adjuster.**
WHERE At file scope, after FocusTimerView; preview code is not part of the shipped app.
WHEN Each time the preview refreshes.
WHY Previewing the parent shows both views working together, which is the point of the example.

Try it

The starter models the broken frame: a parent copy and a child copy of the length, where the child step changes only its own copy. Rewrite it with one shared variable, apply the step to it, and print the parent display.

```
var parentMinutes = 25
var childMinutes = parentMinutes
childMinutes += 1
print("Focus length: \(parentMinutes) min")
```

EXPECTED OUTPUT

```
Focus length: 26 min
```

MODULE 05

Designing Great iOS Apps

Apply Apple's Human Interface Guidelines to design iPhone screens that everyone can read, reach and understand: Liquid Glass, Dynamic Type, color and contrast, layout and 44-point touch targets, SF Symbols, navigation, clear states and copy, and accessibility. Then prototype, test and critique a design the way professionals do.

- Design user interfaces with reactive programming, UI elements and their properties

Principles of iOS Design

CONCEPT

Design is how it works

Design is not a coat of paint you add at the end. It is every decision about what a screen shows, what people can do there, and how the app responds. As Steve Jobs put it, design is not just what it looks like and feels like: design is how it works.

Apple publishes its design advice as the **Human Interface Guidelines** (HIG), a free reference on Apple's developer website for how apps on iPhone, iPad, Mac, Apple Watch, Apple TV and Apple Vision Pro should look and behave. This module teaches the iPhone parts that matter most, with the real numbers behind them.

Human Interface Guidelines	Apple's design guidance, organized as foundations (color, typography, layout, accessibility), patterns (navigation, loading, feedback) and components (buttons, lists, tab bars).
Platform convention	A behavior people already know from other apps, such as swiping from the left edge to go back. Conventions let people use your app without learning it.
Content first	People open your app for their messages, photos or data. The interface frames that content instead of competing with it.

> **Look it up before you build it**
>
> Before you design a component, read its HIG page. Each one covers when to use it, best practices, and platform differences, often with the exact sizes you need.

CONCEPT

Hierarchy, harmony, consistency

With iOS 26 and Liquid Glass, Apple frames its guidance around three principles. Treat each one as a test you can run on any screen you design.

Hierarchy	The most important thing on screen is the most prominent. Controls and navigation float above the content and lift it up without fighting it. Test: squint at the screen. Does the right thing stand out first?
Harmony	Shapes, motion and materials agree with the hardware. Rounded corners nest inside the rounded corners of the display (Apple calls this concentricity), so the software and the device feel like one object.
Consistency	Follow platform conventions so your app behaves like the rest of iOS, and let it adapt across screen sizes, orientations, text sizes, and Light and Dark Mode.

Clarity, deference, depth

Starting with iOS 7 in 2013, the HIG described three themes: *clarity* (legible text, precise icons), *deference* (the interface steps back so content shines) and *depth* (layers and motion show hierarchy). Today's principles grow out of them: deference and depth live on as glass controls floating above your content.

CODE · LIVE SWIFTUI PREVIEW

System components do the heavy lifting

Platform conventions come built into SwiftUI's standard views. This settings screen has only a few lines of real content, yet it already supports Dynamic Type, Dark Mode, VoiceOver, right-to-left languages and the iOS 26 look, with no extra code. Each note below names something you get for free.

```swift
import SwiftUI

struct SettingsView: View {
    @State private var remindersOn = true
    @State private var units = "Miles"
    var body: some View {
        NavigationStack {
            Form {
                Section("Trips") {
                    Toggle("Reminders", isOn: $remindersOn)
                    Picker("Distance", selection: $units) {
                        Text("Miles").tag("Miles")
                        Text("Kilometers").tag("Kilometers")
                    }
                }
            }
            .navigationTitle("Settings")
        }
    }
}

#Preview { SettingsView() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the app.
- WHAT — **Makes SwiftUI's standard views (NavigationStack, Form, Toggle, Picker) available in this file.**
- WHERE — The first line of the file; an import applies only to the file it appears in.
- WHEN — At compile time, before any of your code runs.
- WHY — Standard views are where the platform conventions live. Importing SwiftUI is how you inherit them instead of rebuilding them by hand.

```
3  struct SettingsView: View {
```

WHO Your struct, which SwiftUI treats as a description of one screen.
WHAT **Declares a Settings screen that conforms to View, so it must supply a body.**
WHERE A value type in your app module; SwiftUI creates and discards instances freely.
WHEN Created whenever the preview, or a parent screen, builds it.
WHY Settings is the screen where people most expect iOS conventions. Matching the system Settings app makes yours instantly familiar.

```
4  @State private var remindersOn = true
```

WHO SwiftUI's state storage, which owns this value for as long as the screen exists.
WHAT **Holds whether trip reminders are switched on, starting as true.**
WHERE Stored by SwiftUI outside the struct, so it survives the struct being recreated.
WHEN Created the first time the screen appears and read on every body pass.
WHY A Toggle needs a single source of truth to bind to; private keeps that state owned by this screen alone.

```
5  @State private var units = "Miles"
```

WHO SwiftUI's state storage again, for a second value.
WHAT **Holds the chosen distance unit as a String, starting with "Miles".**
WHERE In the same SwiftUI-managed storage, separate from remindersOn.
WHEN Read by the Picker on every body pass and written when the person picks an option.
WHY Each control gets one clear source of truth, so the screen always shows exactly what is stored.

```
6  var body: some View {
```

WHO SwiftUI, which calls body to learn what to draw.
WHAT **Declares the computed property that returns the whole screen.**
WHERE Inside SettingsView; some View hides the long nested type the views below create.
WHEN On first display and again whenever remindersOn or units changes.
WHY Because body runs again on every change, the screen can never drift out of sync with its data.

```
7  NavigationStack {
```

WHO SwiftUI's NavigationStack container, which owns the bar above the Form.
WHAT **Gives the screen a navigation bar, a place for its title, and the ability to push detail screens.**
WHERE The outermost view of the screen.
WHEN Built when the screen appears and kept while it is shown.
WHY Convention: people expect a title at the top and a swipe from the left edge to go back. NavigationStack supplies both.

```
8  Form {
```

WHO SwiftUI's Form, a container for settings and data entry.
WHAT **Lays out its rows in the iOS grouped style: rounded sections on a gray background, separators, and standard row heights.**
WHERE The content of the navigation stack, filling the screen below the bar.
WHEN Built on every body pass; it scrolls on its own when rows overflow.
WHY Form gives you Apple's own settings layout, including rows at least 44 pt tall and standard margins, without measuring anything.

```
9  Section("Trips") {
```

WHO A Section inside the Form.
WHAT **Groups related rows under the header "Trips".**
WHERE The first, and here the only, group of rows in the Form.
WHEN Built on every body pass.
WHY Grouping by topic applies the proximity principle: related settings sit together under a short header, so people scan the headers first.

```
10  Toggle("Reminders", isOn: $remindersOn)
```

WHO — SwiftUI's Toggle, bound to the state through the $ binding.

WHAT — **Draws a labeled iOS switch that reads and writes remindersOn.**

WHERE — The first row of the Trips section.

WHEN — Redrawn whenever remindersOn changes; flips when the person taps it.

WHY — A system Toggle brings the standard switch, a label that scales with Dynamic Type, and VoiceOver support (it announces the label, that it is a switch, and whether it is on), none of which you write.

```
11  Picker("Distance", selection: $units) {
```

WHO — SwiftUI's Picker, bound to units.

WHAT — **Offers a choice between options; inside a Form it shows the current value and opens a menu when tapped.**

WHERE — The second row of the Trips section.

WHEN — Opens on tap and writes the chosen tag into units.

WHY — The system picks the right presentation for the context, a menu row inside a Form, so your app matches every other settings screen.

```
12  Text("Miles").tag("Miles")
```

WHO — A Text view carrying a tag.

WHAT — **Defines the option "Miles"; the tag is the value stored when someone chooses it.**

WHERE — Inside the Picker's content.

WHEN — Shown whenever the menu opens.

WHY — Keeping the label people see separate from the stored value lets you translate labels later without changing your data.

```
13  Text("Kilometers").tag("Kilometers")
```

WHO — A second Text option with its own tag.

WHAT — **Defines the option "Kilometers", stored as the string "Kilometers".**

WHERE — In the Picker's content, after Miles.

WHEN — Shown whenever the menu opens.

WHY — Options appear in the order you write them, so put the most common choice first.

```
17  .navigationTitle("Settings")
```

WHO — The navigation title modifier, read by the enclosing NavigationStack.

WHAT — **Shows "Settings" as a large title that shrinks into the bar when the content scrolls.**

WHERE — Attached to the Form, the root view inside the stack.

WHEN — Applied when the screen appears; it collapses as the person scrolls.

WHY — A title answers "where am I?", the first job of every screen, and shrinking on scroll hands the space back to the content.

```
22  #Preview { SettingsView() }
```

WHO — The #Preview macro, read by the Xcode canvas and by this page's live phone.

WHAT — **Registers SettingsView so it renders next to the code.**

WHERE — At file scope, outside the struct; it is not part of the app's screens.

WHEN — Every time the canvas refreshes after an edit.

WHY — Previews let you check the conventions you got for free: switch the canvas to Dark Mode or the largest text size and nothing breaks.

Try it

1. Add a third option, Text("Steps").tag("Steps"), then open the menu. How much styling did you have to write for the new row?
2. Add a second Section("Account") with a Toggle of your own. Notice the spacing and header style you never set.

COMPARE

Content first: custom chrome or system structure

Both screens show the same three trips. The left one spends its space on a custom banner, so the trips get one cramped line in a tiny fixed font. The right one lets the system draw the structure and gives the content the room.

A: Before: decoration first

```swift
import SwiftUI

struct PoorTrips: View {
    var body: some View {
        VStack(spacing: 0) {
            Text("MY TRIPS")
                .font(.system(size: 13, weight: .heavy))
                .foregroundColor(.white)
                .frame(maxWidth: .infinity, minHeight: 120)
                .background(Color.purple)
            Text("Lisbon  Kyoto  Oaxaca")
                .font(.system(size: 12))
            Spacer()
        }
    }
}

#Preview { PoorTrips() }
```

`1` `import SwiftUI`

- WHO — The Swift compiler, while it builds the app.
- WHAT — **Makes SwiftUI available, exactly as on the right-hand side.**
- WHERE — The first line of the file.
- WHEN — At compile time, before PoorTrips is type-checked; nothing runs here.
- WHY — Both versions use the same toolkit; the difference you are about to see is entirely in how it is used.

`3` `struct PoorTrips: View {`

- WHO — Your struct, describing the screen.
- WHAT — **Declares the first attempt at a trips screen.**
- WHERE — A value type in your app module.
- WHEN — Created when the preview builds it.
- WHY — Hand-built screens like this are common in first apps, because a banner feels like design. It is decoration.

```
4  var body: some View {
```

WHO SwiftUI, which calls body to learn what to draw.
WHAT **Returns the screen's content.**
WHERE Inside PoorTrips, the only member of the struct besides its body.
WHEN On first display and after any change.
WHY Everything below is placed by hand, so every convention must be recreated by hand, and here none are.

```
5  VStack(spacing: 0) {
```

WHO The VStack, which sizes and stacks the banner, the trips line and the spacer.
WHAT **Stacks the banner, the trips line and a spacer with no gap between them.**
WHERE The root view of the screen.
WHEN Built on every body pass.
WHY A raw stack has no title, no margins and no rows, so the screen gets none of the structure people expect.

```
6  Text("MY TRIPS")
```

WHO A Text view drawing the banner label in the first slot of the stack.
WHAT **Draws the screen's name as the banner label.**
WHERE The first child of the stack.
WHEN Built on every body pass.
WHY All capitals are slower to read than mixed case, and a navigation title is the conventional place for a screen's name.

```
7  .font(.system(size: 13, weight: .heavy))
```

WHO The font modifier with a fixed system size.
WHAT **Locks the label at 13 pt heavy.**
WHERE On the banner label only.
WHEN Applied at render time, and it never changes with the reader's text size.
WHY A fixed size ignores Dynamic Type, and 13 pt is smaller than body text even though this is meant to be the screen's main label.

```
8  .foregroundColor(.white)
```

WHO The foreground color modifier, the older spelling of foregroundStyle.
WHAT **Paints the label white.**
WHERE On the banner label.
WHEN Every render, in both Light and Dark Mode.
WHY Hard-coded white only works because the banner is hard-coded purple. Change one and the other breaks.

```
9  .frame(maxWidth: .infinity, minHeight: 120)
```

WHO The frame modifier, which proposes a size to the label.
WHAT **Stretches the banner across the full width and makes it at least 120 pt tall.**
WHERE Around the label, before the background is drawn.
WHEN During layout on every pass.
WHY On an 852 pt tall iPhone screen, 120 pt is about 14 percent of the height spent on a label, pushing the real content down.

```
10 .background(Color.purple)
```

WHO The background modifier with a fixed color.
WHAT **Fills the banner with purple.**
WHERE Behind the 120 pt frame.
WHEN Every time the banner is drawn, in Light and Dark Mode alike.
WHY A slab of color competes with the content for attention. Brand color works better as the accent on buttons and selection.

```
11  Text("Lisbon  Kyoto  Oaxaca")
```

WHO	A second Text view.
WHAT	**Crams all three trips into a single line of text.**
WHERE	The second child of the stack, right under the banner.
WHEN	Built on every body pass.
WHY	One line means no row to tap per trip and no room for details, and VoiceOver reads it as one undivided string.

```
12  .font(.system(size: 12))
```

WHO	The font modifier with a fixed size.
WHAT	**Sets the trips to 12 pt, which never scales.**
WHERE	On the trips line only.
WHEN	At render time, regardless of the text-size setting.
WHY	The content people came for is now the smallest text on the screen: the hierarchy is upside down.

```
13  Spacer()
```

WHO	A flexible spacer inside the stack.
WHAT	**Takes all leftover height, pushing the banner and the line to the top.**
WHERE	The last child of the stack.
WHEN	During layout, after the banner and trips line have taken their heights.
WHY	The empty space below proves the content was never short of room; it was simply given less of it.

```
18  #Preview { PoorTrips() }
```

WHO	The #Preview macro, read by the canvas and by this page's phone to show PoorTrips.
WHAT	**Renders PoorTrips in the canvas or the live phone.**
WHERE	At file scope, after the closing brace of PoorTrips.
WHEN	Whenever the canvas refreshes.
WHY	Look at the preview first as a whole: the banner wins your attention before any trip does.

B: After: content first

```swift
import SwiftUI

struct BetterTrips: View {
    let trips = ["Lisbon", "Kyoto", "Oaxaca"]

    var body: some View {
        NavigationStack {
            List {
                ForEach(trips, id: \.self) { trip in
                    Label(trip, systemImage: "airplane")
                }
            }
            .navigationTitle("My Trips")
        }
    }
}
#Preview { BetterTrips() }
```

```
1  import SwiftUI
```

WHO The Swift compiler, while it builds the app.
WHAT **Makes SwiftUI's NavigationStack, List and Label available.**
WHERE The first line of the file.
WHEN At compile time, before BetterTrips is type-checked; nothing runs here.
WHY The same framework as the left side; this time the screen leans on its standard structure.

```
3  struct BetterTrips: View {
```

WHO Your struct, describing the screen.
WHAT **Declares the improved trips screen.**
WHERE A value type in your app module.
WHEN Created when the preview builds it.
WHY Same data as the left side, so the only variable is the design.

```
4  let trips = ["Lisbon", "Kyoto", "Oaxaca"]
```

WHO A stored constant on the struct.
WHAT **Holds the three trip names as an array of strings.**
WHERE Inside BetterTrips, stored with the struct value.
WHEN Set when the view is created.
WHY Treating trips as data rather than one hard-coded sentence lets each trip become its own row.

```
6  var body: some View {
```

WHO SwiftUI, which calls body to learn what to draw.
WHAT **Returns the screen's content.**
WHERE Inside BetterTrips, directly under the trips constant.
WHEN On first display and after any change.
WHY The body below describes structure (a title and a list); the system handles the look.

```
7  NavigationStack {
```

WHO SwiftUI's NavigationStack, which supplies the bar that the title below fills.
WHAT **Provides the navigation bar and title area, and room to push a trip's detail screen later.**
WHERE The outermost view of the screen.
WHEN Built when the screen appears.
WHY It is the conventional frame for a list that people will drill into, so the next feature already has a home.

```
8  List {
```

WHO SwiftUI's List, which draws one system-styled row per trip from the ForEach.
WHAT **Draws a scrolling list with separators, standard margins and rows at least 44 pt tall.**
WHERE Inside the navigation stack, filling the screen.
WHEN Built on every body pass; rows scroll when they overflow.
WHY The content gets the full screen, and every trip gets a full-width row that is easy to tap.

```
9  ForEach(trips, id: \.self) { trip in
```

WHO ForEach, which turns data into views.
WHAT **Creates one row per trip, identifying each row by the trip name itself.**
WHERE Inside the List, so each trip becomes a separate row.
WHEN On every body pass; SwiftUI compares rows by their id to update only what changed.
WHY Stable ids let SwiftUI animate added or removed trips correctly; \.self works because each name is unique.

```
10  Label(trip, systemImage: "airplane")
```

WHO	A Label, which pairs a symbol with text.
WHAT	**Shows the trip name next to an airplane symbol, in the standard row layout.**
WHERE	The content of each row.
WHEN	Built once per trip on each body pass.
WHY	Label aligns and sizes the symbol with the text, and it uses body text (17 pt by default) that grows with Dynamic Type.

```
13  .navigationTitle("My Trips")
```

WHO	The navigation title modifier, read by the NavigationStack.
WHAT	**Shows "My Trips" as a large title that collapses into the bar on scroll.**
WHERE	Attached to the List.
WHEN	Applied when the screen appears.
WHY	It does the banner's job in the standard style, then gets out of the way so the list has more room.

```
17  #Preview { BetterTrips() }
```

WHO	The #Preview macro, read by the canvas and by this page's phone to show BetterTrips.
WHAT	**Renders BetterTrips next to the left-hand version.**
WHERE	At file scope, after the closing brace of BetterTrips.
WHEN	Whenever the canvas refreshes.
WHY	Compare the two phones: here your eye lands on the trips first, which is exactly what hierarchy means.

> **Takeaway**
>
> The left screen designs the frame; the right screen designs for the content. System structure (a navigation title, a List, Labels) gives the trips hierarchy, room, tappable rows and Dynamic Type for free, and it collapses out of the way when people scroll. Start from system components, and add custom styling only where it carries meaning.

CHECK YOURSELF

Check your understanding

1. A team wants a custom slide-out drawer (a "hamburger" menu) to switch between the four main sections of their app. Which recommendation follows iOS conventions?
   A. Keep the drawer, but make its button larger and easier to find
   B. Use a tab bar, so each section stays visible and one tap away
   C. Put all four sections into one long scrolling screen with headers
   D. Present the four sections in a sheet that slides up from the bottom
2. Which change does the most for *hierarchy* on a checkout screen?
   A. Make every button the same size and color so the screen looks calm
   B. Add a gradient background behind the order summary to lift it
   C. Give Place Order a prominent style and keep other actions plain
   D. Write every label in capital letters so each one stands out

3. Why does the Form and Toggle settings screen already work in Dark Mode, at large text sizes and with VoiceOver, without extra code?
   A. Standard components adapt to the settings and describe themselves to VoiceOver
   B. The #Preview macro switches those features on for the screen it shows
   C. iOS inverts the colors of every app automatically in Dark Mode
   D. Form shrinks long labels so that they always fit on one line

4. In Apple's iOS 26 principles, what does *harmony* ask of your design?
   A. Use exactly one font size throughout the app so nothing competes
   B. Match the app's colors to the person's current wallpaper
   C. Avoid animation entirely so the interface never distracts
   D. Nest rounded corners inside the display's own rounded corners

5. The older HIG theme of *deference* said the interface should defer to the content. Which photo app shows it best?
   A. Photos sit under a 120 pt decorative banner that carries the app name
   B. Photos fill the screen; small translucent controls float above
   C. Every editing tool is visible at all times beside the photo grid
   D. A bright patterned background surrounds the photo grid on every screen

Answers: Principles of iOS Design

1. **B. Use a tab bar, so each section stays visible and one tap away** A tab bar is the iOS convention for a few top-level sections: every section stays visible and one tap away, and people already know how it works. A drawer hides the sections behind an extra tap and is not an iOS pattern, so a bigger button does not fix it. A sheet is for a focused task, not for moving between main sections.
2. **C. Give Place Order a prominent style and keep other actions plain** Hierarchy means the most important thing is the most prominent. One prominent primary action, with quieter secondary actions, tells people what to do next. Identical buttons flatten the hierarchy, a gradient adds decoration without meaning, and capitals everywhere make everything shout equally, which is the same as nothing standing out.
3. **A. Standard components adapt to the settings and describe themselves to VoiceOver** Standard views such as Form, Toggle and Picker use adaptive colors and text styles and carry accessibility information (a label, a value, the fact that a toggle is a switch). That is the payoff of conventions: you inherit years of platform work. #Preview only renders the view. iOS does not invert your colors for Dark Mode; your views must use colors that adapt, which system components already do.
4. **D. Nest rounded corners inside the display's own rounded corners** Harmony is about software and hardware feeling like one object: shapes nest concentrically inside the rounded display, and motion and materials behave consistently. It does not ban animation or require one font size; hierarchy needs several sizes. Matching the wallpaper is not part of it.
5. **B. Photos fill the screen; small translucent controls float above** Deference means the content is the star and the interface steps back. Full-screen photos with small floating controls are exactly that, and it is how Liquid Glass controls work today. A banner, a busy background, or every tool on screen at once all compete with the photos for attention.

Liquid Glass

CONCEPT

What Liquid Glass is

Starting in iOS 26, Apple builds its interface from **Liquid Glass**: a translucent material that bends and focuses the light of whatever sits behind it, the way a lens does. Older iOS materials only blurred the background. Liquid Glass also picks up color from nearby content, shifts between a lighter and a darker look as content passes underneath, and responds to touch.

You get most of it for free. Build with Xcode 26 or later and the standard navigation bars, tab bars, toolbars, sheets, menus, toggles and sliders adopt Liquid Glass automatically.

Material	A reusable surface with consistent, physical-feeling rules that the system draws for you, like frosted glass in the real world.
Lensing	Bending the light from content behind the glass instead of just scattering it, so the edges of a control stay crisp.
Adaptive	Small glass elements flip between light and dark appearances as bright or dark content scrolls beneath them, to stay legible.
Interactive	Glass can brighten and flex under a finger, which confirms that a press landed.

> **One design language**
>
> Liquid Glass shipped across iOS, iPadOS, macOS, watchOS and tvOS 26 at the same time, building on the glass already used in visionOS, so the same SwiftUI code gets a consistent look across Apple's devices.

CONCEPT

Content below, controls above

Think of every iOS 26 screen as two layers. The **content layer** holds what people came for: photos, messages, lists, maps. The **navigation layer** floats above it and holds the controls that act on that content: tab bars, toolbars, a floating play button. Liquid Glass belongs only on the navigation layer. Content stays solid and readable, and the glass controls let it show through around them.

Glass is for controls	Toolbars, tab bars, floating buttons and sheets. Never put lists, cards or paragraphs of text on glass.
No glass on glass	A glass button inside a glass card looks muddy, and glass cannot sample other glass. Keep one glass layer.
Use it sparingly	When everything is glass, nothing stands out. A few floating controls create hierarchy.
Tint with purpose	Tint only the primary or meaningful action, such as a red record button, never every button.

Legibility comes first

Glass changes with whatever is behind it. Test your controls over your brightest and busiest content, in light and dark mode, and with *Reduce Transparency* and *Increase Contrast* turned on (Settings, Accessibility, Display & Text Size). The system glass makes itself frostier or higher-contrast for those settings automatically.

CODE · LIVE SWIFTUI PREVIEW

Float a control on glass

The simplest way to make your own glass: build a control, pad it, and add `.glassEffect()`. Here two icons float above a scrolling gallery. Scroll the colored tiles under the control in the preview and watch the capsule take on each color while the icons stay readable.

```swift
import SwiftUI

struct GalleryView: View {
    let tiles: [Color] = [.orange, .pink, .teal, .indigo]

    var body: some View {
        ScrollView {
            ForEach(tiles, id: \.self) { tile in
                tile.frame(height: 180)
            }
            .padding()
        }
        .overlay(alignment: .bottom) {
            HStack(spacing: 28) {
                Image(systemName: "heart")
                Image(systemName: "square.and.arrow.up")
            }
            .font(.title2)
            .padding(16)
            .glassEffect()
        }
    }
}

#Preview { GalleryView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds this file.
- **WHAT** **Makes SwiftUI available here, including ScrollView, ForEach, Image and the iOS 26 glassEffect modifier.**
- **WHERE** Top of the file; it covers every declaration below it in this file only.
- **WHEN** At compile time, before the app ever launches.
- **WHY** Liquid Glass is a UI framework feature, not part of the Swift language, so any file that draws glass must import SwiftUI (or UIKit).

```
3  struct GalleryView: View {
```

WHO Your code declares it; SwiftUI creates and redraws it.

WHAT **Declares a screen named GalleryView that conforms to View, so it must provide a body describing its interface.**

WHERE A value type (struct) at the top level of the file.

WHEN Defined at compile time; SwiftUI makes instances whenever the preview or app shows this screen.

WHY SwiftUI views are lightweight structs that are cheap to recreate, which lets the system rebuild them freely while content scrolls under the glass.

```
4  let tiles: [Color] = [.orange, .pink, .teal, .indigo]
```

WHO GalleryView owns this constant.

WHAT **Stores four saturated colors that stand in for photos in the gallery.**

WHERE A stored property inside the GalleryView struct.

WHEN Set once each time SwiftUI creates a GalleryView value.

WHY Glass only shows its behavior over varied content. Strong colors make the color pickup and the light or dark adaptation easy to see.

```
6  var body: some View {
```

WHO SwiftUI calls it; you write it.

WHAT **Returns the description of the whole screen: the scrolling gallery plus the floating control.**

WHERE A computed property on GalleryView, evaluated on the main thread.

WHEN When the view first appears and whenever SwiftUI needs a fresh description.

WHY some View lets the compiler infer the long nested type of this layout, so you describe the screen without naming that type.

```
7  ScrollView {
```

WHO SwiftUI, and later the user dragging with a finger.

WHAT **Creates the content layer: a vertically scrolling region that holds the tiles.**

WHERE The base of the screen, underneath the control that the overlay adds.

WHEN Built when body runs; it scrolls in response to touches.

WHY Content flowing under fixed controls is exactly the situation Liquid Glass was designed for: the control stays put while the content moves beneath it.

```
8  ForEach(tiles, id: \.self) { tile in
```

WHO SwiftUI's ForEach, which repeats its closure once per color in tiles.

WHAT **Makes one view per color in tiles, calls the current color tile, and uses the color itself as its identity.**

WHERE Inside the ScrollView, so the generated tiles stack vertically.

WHEN Each time body runs; SwiftUI compares identities to update only what changed.

WHY Generating views from data instead of copying them by hand keeps every tile consistent and makes it trivial to add more content.

```
9  tile.frame(height: 180)
```

WHO The SwiftUI layout system, when it sizes each tile inside the scroll view.

WHAT **Shows the color as a filled block as wide as the scroll view allows and exactly 180 points tall.**

WHERE Once per tile, inside the scroll view's column.

WHEN During layout, every time the view is drawn.

WHY Large blocks of color behave like photos: big areas moving under the glass make its lensing and tint pickup visible.

11 `.padding()`

WHO The padding modifier on the ForEach, inside the ScrollView.
WHAT **Adds the system default spacing, 16 points on iPhone, around the column of tiles.**
WHERE On the ForEach, inside the ScrollView.
WHEN During layout, after the four tiles report their 180-point heights.
WHY Content should not touch the screen edges. The default margin matches the rest of iOS, so the screen feels native.

13 `.overlay(alignment: .bottom) {`

WHO The overlay modifier on the ScrollView, which places the control on top.
WHAT **Layers the views in the braces on top of the ScrollView and pins them to its bottom edge.**
WHERE Attached to the ScrollView, so the control floats above the content but stays inside the safe area.
WHEN During layout; the overlay does not scroll with the content.
WHY This is the two-layer model in code: content underneath, a navigation-layer control above it, placed at the bottom where a thumb can reach it.

14 `HStack(spacing: 28) {`

WHO The HStack inside the overlay, which sizes and spaces the two icons.
WHAT **Lines the two icons up horizontally with 28 points between them.**
WHERE Inside the overlay, forming the contents of the floating control.
WHEN During layout of the overlay, each time the control is drawn.
WHY Generous spacing keeps each icon's tap area separate, so a fingertip does not trigger the wrong action.

15 `Image(systemName: "heart")`

WHO SwiftUI and the SF Symbols library built into iOS.
WHAT **Draws the heart symbol, the familiar like or favorite action.**
WHERE First item in the floating row.
WHEN Rendered every time the control is drawn.
WHY SF Symbols are vector shapes that stay crisp on glass. In a real app wrap each icon in a Button with a text label so VoiceOver can name it.

16 `Image(systemName: "square.and.arrow.up")`

WHO SwiftUI and SF Symbols.
WHAT **Draws the standard iOS share symbol.**
WHERE Second item in the floating row.
WHEN Rendered with the rest of the control.
WHY People already know this symbol means Share everywhere on iOS. Reusing the platform icon instead of inventing one is consistency in practice.

18 `.font(.title2)`

WHO The font modifier on the HStack, inherited by both symbols through the environment.
WHAT **Applies the title2 text style, 22 points at the default text size, which also sets the size of both symbols.**
WHERE On the HStack, so both images inherit it.
WHEN During layout, and again whenever the person changes their text size.
WHY Symbols size themselves from the font. Using a text style keeps the icons growing with Dynamic Type instead of staying a fixed size.

19 `.padding(16)`

WHO The padding modifier on the icon row, applied before the glass.
WHAT **Adds 16 points of space on every side of the icons, inside the glass that comes next.**
WHERE Between the icon row and the glass effect.
WHEN During layout, after the two symbols have measured themselves at title2 size.
WHY Modifier order matters: padding first makes the glass larger than the icons, giving a control taller than the 44-point minimum tap size and room for the edge to bend light.

```
20 .glassEffect()
```

WHO — SwiftUI, using the system Liquid Glass renderer (iOS 26 and later).

WHAT — **Draws the regular Liquid Glass material, in its default capsule shape, behind the padded icons.**

WHERE — On the navigation layer, above the ScrollView content.

WHEN — Rendered continuously; as tiles scroll beneath, the glass updates its refraction and color.

WHY — One modifier gives you the same material the system uses for toolbars and tab bars, including its automatic legibility adjustments and accessibility fallbacks.

```
25 #Preview { GalleryView() }
```

WHO — The Xcode preview canvas, and the simulated iPhone on this page.

WHAT — **Registers a preview that shows a GalleryView instance.**

WHERE — At file scope, outside the struct; it does not ship as part of the app's screens.

WHEN — Whenever the canvas refreshes after you edit the file.

WHY — Previews let you judge a design instantly. For glass, always scroll in the preview to check legibility over every kind of content.

> **Try it**
>
> 1. Move .padding(16) below .glassEffect(). Why does the capsule now hug the icons so tightly?
> 2. Change .title2 to .caption. Is the control still comfortable to tap?

CODE · LIVE SWIFTUI PREVIEW

Regular or clear, capsule or rectangle

Glass comes in two variants. **Regular** adapts to keep content readable almost anywhere; it is the default. **Clear** is far more transparent and does not adapt, so it is meant for controls over photos and video. The `in:` parameter sets the shape. Compare both over the same background.

```swift
import SwiftUI

struct VariantsView: View {
    var body: some View {
        ZStack {
            Color.indigo.ignoresSafeArea()
            VStack(spacing: 24) {
                Text("Regular glass")
                    .font(.headline)
                    .padding()
                    .glassEffect(.regular, in: .rect(cornerRadius: 16))
                Text("Clear glass")
                    .font(.headline)
                    .foregroundStyle(.white)
                    .padding()
                    .glassEffect(.clear, in: .capsule)
            }
        }
    }
}

#Preview { VariantsView() }
```

Line by line

```
1  import SwiftUI
```

WHO The Swift compiler, while it builds the file that declares VariantsView.
WHAT **Imports SwiftUI so this file can use ZStack, Text, Color and the Glass variants regular and clear.**
WHERE First line; it applies to this file only.
WHEN At compile time, before VariantsView is type-checked.
WHY The Glass type and its variants ship with SwiftUI in the iOS 26 SDK; the import is what makes .regular and .clear resolvable.

```
3  struct VariantsView: View {
```

WHO Your code; SwiftUI creates instances of it.
WHAT **Declares a demo screen that shows the two glass variants over one background.**
WHERE A top-level struct in this file.
WHEN Instantiated when the preview runs.
WHY Comparing variants over identical content isolates the one thing that differs: how much of the background each variant lets through.

```
4  var body: some View {
```

WHO SwiftUI calls body on VariantsView; your code returns the ZStack.
WHAT **Describes the screen: a colored background with two glass labels on top.**
WHERE A computed property on VariantsView, evaluated on the main thread.
WHEN On first display and whenever SwiftUI refreshes the view.
WHY Reading body top to bottom shows the layers in order: background first, glass above it.

```
5  ZStack {
```

WHO The ZStack, which centers the background and the labels on top of each other.
WHAT **Stacks its children on top of each other, back to front, centered.**
WHERE The root of body.
WHEN During layout, every time VariantsView is drawn.
WHY Glass needs something behind it. The ZStack makes the background the bottom layer and the glass the top layer, the same two-layer model as a real screen.

```
6  Color.indigo.ignoresSafeArea()
```

WHO The Color view, filled by SwiftUI as the bottom layer of the ZStack.
WHAT **Fills the entire screen with indigo, including the areas behind the status bar and home indicator.**
WHERE The bottom layer of the ZStack.
WHEN Drawn first, every time the view renders.
WHY Backgrounds may extend into the safe areas because they carry no information; only text and controls must stay inside. The rich color stands in for album art or a photo.

```
7  VStack(spacing: 24) {
```

WHO The VStack, which stacks the two glass labels and spaces them 24 points apart.
WHAT **Stacks the two glass labels vertically with 24 points between them.**
WHERE The top layer of the ZStack, centered on screen.
WHEN During layout of the ZStack's top layer.
WHY 24 points is a step on the common 8-point spacing rhythm and wide enough that the two glass shapes read as separate objects.

8 `Text("Regular glass")`

- **WHO** SwiftUI's text renderer, drawing the first label on the VStack.
- **WHAT** **Shows the label for the first sample.**
- **WHERE** First child of the VStack.
- **WHEN** Rendered with each evaluation of body.
- **WHY** Real text on glass is the honest test of a variant, because legibility is the reason to choose one over the other.

9 `.font(.headline)`

- **WHO** SwiftUI, reading the Dynamic Type setting.
- **WHAT** **Uses the headline text style: 17 points semibold at the default size.**
- **WHERE** On the first label.
- **WHEN** During layout and whenever the text size setting changes.
- **WHY** Semibold letters survive a moving, colorful background much better than regular weight, which is why labels on glass favor heavier text.

10 `.padding()`

- **WHO** The padding modifier on the first label, inside its glass.
- **WHAT** **Adds the default 16 points of space around the label, inside the glass that follows.**
- **WHERE** Between the first label and its glass shape.
- **WHEN** During layout, after the headline text has measured itself.
- **WHY** The glass shape is sized from whatever it wraps; padding gives the edge room to refract light without crowding the letters.

11 `.glassEffect(.regular, in: .rect(cornerRadius: 16))`

- **WHO** SwiftUI's Liquid Glass renderer.
- **WHAT** **Applies the regular variant in a rounded rectangle with 16-point corners instead of the default capsule.**
- **WHERE** Behind the first label, on top of the indigo background.
- **WHEN** Rendered continuously, adapting to what is behind it.
- **WHY** Regular is the versatile default: it adjusts its tint and contrast to the content underneath so text stays readable almost anywhere. Choose it unless you have a specific reason not to.

12 `Text("Clear glass")`

- **WHO** SwiftUI's text renderer, drawing the second label below the first.
- **WHAT** **Shows the label for the second sample.**
- **WHERE** Second child of the VStack.
- **WHEN** Rendered with each evaluation of body.
- **WHY** Placing it over exactly the same background as the first sample makes the difference in transparency obvious.

13 `.font(.headline)`

- **WHO** The font modifier on the second label, reading the Dynamic Type setting.
- **WHAT** **Gives the second label the same headline style as the first.**
- **WHERE** On the second label.
- **WHEN** During layout, and again whenever the text size setting changes.
- **WHY** Keeping everything identical except the variant makes the comparison fair, and matching text styles is how a real toolbar keeps its labels uniform.

```
14  .foregroundStyle(.white)
```

WHO The foregroundStyle modifier on the second label.
WHAT **Draws the second label in white.**
WHERE On the second label only.
WHEN During rendering of the second label, every time it is drawn.
WHY Clear glass does not adapt for you, so whatever sits on it must be bold and bright. White semibold text over a dark, rich background is the case clear glass is designed for.

```
15  .padding()
```

WHO The padding modifier on the second label, inside its clear glass.
WHAT **Adds the default 16 points around the white label.**
WHERE Between the second label and its clear capsule.
WHEN During layout, after the white headline text has measured itself.
WHY As with the first sample, padding sets the size of the visible glass shape and of the tap area.

```
16  .glassEffect(.clear, in: .capsule)
```

WHO SwiftUI's Liquid Glass renderer.
WHAT **Applies the clear variant in a capsule shape, much more transparent than regular.**
WHERE Behind the second label.
WHEN Rendered continuously while the second label is on screen, sampling the indigo behind it.
WHY Apple limits clear glass to controls over media-rich content such as photos or video, with bold, bright content on top, and a dimming layer behind it if legibility suffers.

```
22  #Preview { VariantsView() }
```

WHO The Xcode preview canvas, and the simulated iPhone on this page, running VariantsView.
WHAT **Shows VariantsView in the simulated iPhone.**
WHERE At file scope, outside the struct.
WHEN Every time you edit and the canvas refreshes.
WHY Swapping the background color in a preview is the fastest way to find out which variant stays legible for your content.

Try it

1. Change Color.indigo to Color.yellow. Which label becomes harder to read, and why does the regular one cope better?
2. Remove .foregroundStyle(.white). What does that tell you about putting ordinary text on clear glass?

CODE · LIVE SWIFTUI PREVIEW

Glass buttons in a shared container

For buttons, use the glass button styles: `.glass` for secondary actions and `.glassProminent` for the one primary action, tinted with the app's accent color. Wrap neighboring glass in a `GlassEffectContainer` so the pieces share one glass surface. Tap Save and Cancel in the preview.

```swift
import SwiftUI

struct EditorBar: View {
    @State private var saved = false

    var body: some View {
        VStack(spacing: 32) {
            Text(saved ? "Saved" : "Draft")
                .font(.title2)
            GlassEffectContainer(spacing: 16) {
                HStack(spacing: 16) {
                    Button("Cancel") { saved = false }
                        .buttonStyle(.glass)
                    Button("Save") { saved = true }
                        .buttonStyle(.glassProminent)
                }
            }
        }
    }
}

#Preview { EditorBar() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares EditorBar.
- **WHAT** **Imports SwiftUI, which provides GlassEffectContainer and the glass button styles used below.**
- **WHERE** Top of this file.
- **WHEN** At compile time, before EditorBar is type-checked.
- **WHY** Both .glass and .glassProminent are SwiftUI button styles added in iOS 26; without the import they do not exist.

3 `struct EditorBar: View {`

- **WHO** Your code declares EditorBar; SwiftUI creates it and redraws it after each tap.
- **WHAT** **Declares a small editor screen with a status label and two glass buttons.**
- **WHERE** A top-level struct in this file.
- **WHEN** Instantiated when the preview runs.
- **WHY** A Save and Cancel pair is the classic case of one primary action next to one secondary action.

4 `@State private var saved = false`

- **WHO** SwiftUI owns the storage; this view reads and writes it.
- **WHAT** **Creates a Boolean that remembers whether the draft has been saved, starting as false.**
- **WHERE** In SwiftUI-managed storage tied to this view's place on screen, not inside the struct value.
- **WHEN** Created the first time the view appears; it survives every redraw.
- **WHY** A tap must visibly change something. This state lets the label confirm each action, which is the feedback people expect.

```
6  var body: some View {
```

WHO — SwiftUI calls body on EditorBar whenever saved changes; your code returns the VStack.

WHAT — **Describes the status label and the button bar.**

WHERE — On EditorBar, evaluated on the main thread.

WHEN — At first display and again after saved changes.

WHY — Because the label reads saved, SwiftUI recomputes body automatically after every tap; you never update the label by hand.

```
7  VStack(spacing: 32) {
```

WHO — The VStack, which places the status text above the glass container.

WHAT — **Stacks the status text above the buttons with 32 points between them.**

WHERE — The root of body.

WHEN — During layout, every time EditorBar is drawn after a tap.

WHY — 32 points, four steps of 8, separates content from controls so they read as two different groups.

```
8  Text(saved ? "Saved" : "Draft")
```

WHO — SwiftUI's text renderer, reading saved to pick the string.

WHAT — **Shows Saved or Draft depending on the state value.**

WHERE — At the top of the VStack, in the content layer.

WHEN — Each time body runs, which happens after every tap that changes saved.

WHY — Visible status after an action is basic feedback: people should never wonder whether a tap worked.

```
9  .font(.title2)
```

WHO — SwiftUI with Dynamic Type.

WHAT — **Uses the title2 style, 22 points at the default text size.**

WHERE — On the status label.

WHEN — During layout and whenever the text size setting changes.

WHY — A larger style marks the status as the main content of this small screen, above the controls in the visual hierarchy.

```
10 GlassEffectContainer(spacing: 16) {
```

WHO — SwiftUI's Liquid Glass renderer.

WHAT — **Groups the glass inside it into one shared surface; shapes closer than 16 points can blend and morph into each other.**

WHERE — Around the button row, in the navigation layer.

WHEN — Rendered whenever the buttons are drawn or animate.

WHY — Glass cannot sample other glass, so neighboring pieces must share one container to look consistent. Grouping also lets the system render them together, which is faster.

```
11 HStack(spacing: 16) {
```

WHO — The HStack inside the GlassEffectContainer, which places the two buttons side by side.

WHAT — **Places the two buttons side by side, 16 points apart.**

WHERE — Inside the glass container.

WHEN — During layout of the container's contents.

WHY — Matching the container spacing keeps each button a separate capsule at rest while still letting them merge if they move closer in an animation.

```
12 Button("Cancel") { saved = false }
```

WHO — SwiftUI, then the user tapping.

WHAT — **Creates a Cancel button whose action sets saved back to false.**

WHERE — First, leading button in the row.

WHEN — The closure runs on the main thread each time the user taps.

WHY — The secondary action goes on the leading side and the primary action on the trailing side, the order people know from iOS dialogs.

```
13  .buttonStyle(.glass)
```

WHO SwiftUI's button styling, applied to the Cancel button only.
WHAT **Draws the button as a regular Liquid Glass capsule with standard padding and press feedback.**
WHERE On the Cancel button.
WHEN Rendered with the button and on every press.
WHY Plain glass gives a secondary action a quiet look that does not compete with the primary button.

```
14  Button("Save") { saved = true }
```

WHO SwiftUI, then the user.
WHAT **Creates a Save button whose action sets saved to true.**
WHERE The second, trailing button in the HStack, after Cancel.
WHEN The closure runs when the user taps Save.
WHY Button labels are verbs that say exactly what will happen. Save is clearer than OK or Submit.

```
15  .buttonStyle(.glassProminent)
```

WHO SwiftUI's button styling, applied to the Save button only.
WHAT **Draws the button as prominent glass, tinted with the app's accent color.**
WHERE On the Save button only.
WHEN Rendered with the button.
WHY One prominent action per group tells people what the main thing to do is. Making both buttons prominent would erase that hierarchy.

```
22  #Preview { EditorBar() }
```

WHO The Xcode preview canvas, and the simulated iPhone on this page, running EditorBar.
WHAT **Shows EditorBar in the simulated iPhone.**
WHERE At file scope, after the closing brace of EditorBar.
WHEN When the canvas refreshes.
WHY Tapping both buttons in the preview checks the feedback text and the visual weight of each style at once.

Try it

1. Give Cancel .glassProminent too. Which button would a hurried person press now?
2. Add .tint(.green) on the line after .buttonStyle(.glassProminent). What changes, and what does green now suggest?

COMPARE

Glass on glass, or one layer of glass

Both screens show the same workout summary and a Share button. The left one puts the content on glass and a glass button inside it. The right one keeps the content solid and moves the control into the system toolbar.

A: Poor: content on glass, glass on glass

```swift
import SwiftUI

struct GlassCard: View {
    var body: some View {
        VStack(spacing: 12) {
            Text("Morning Run")
                .font(.headline)
            Text("5.2 km in 28 minutes")
            Button("Share") { }
                .buttonStyle(.glass)
        }
        .padding()
        .glassEffect(.regular, in: .rect(cornerRadius: 24))
    }
}

#Preview { GlassCard() }
```

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares GlassCard.
- WHAT — **Imports SwiftUI for VStack, Text, Button and the glass APIs this card misuses.**
- WHERE — First line of this file.
- WHEN — At compile time, before GlassCard is type-checked.
- WHY — The problem in this example is design, not code: it compiles without a single warning, which is why the mistake is common.

3 `struct GlassCard: View {`

- WHO — Your code declares GlassCard; SwiftUI creates it for the preview.
- WHAT — **Declares a card showing a workout summary and a Share button.**
- WHERE — A top-level struct in this file, outside any other type.
- WHEN — Instantiated by the preview.
- WHY — A summary card is content, the information people came to see, which matters for the glass decision below.

4 `var body: some View {`

- WHO — SwiftUI calls body on GlassCard; your code returns the VStack.
- WHAT — **Describes the card's contents and its glass background.**
- WHERE — A computed property on GlassCard.
- WHEN — On first display and on every refresh.
- WHY — Everything wrong with this card lives in body, so reading it top to bottom shows how the layers got mixed.

5 `VStack(spacing: 12) {`

- WHO — The VStack, which stacks the title, detail and Share button inside the card.
- WHAT — **Stacks the title, detail and button vertically, centered, 12 points apart.**
- WHERE — The root of the card.
- WHEN — During layout, every time GlassCard is drawn.
- WHY — Centered lines are harder to scan than leading-aligned ones in a summary. A smaller problem than the glass, but a real one.

```
6  Text("Morning Run")
```

WHO SwiftUI's text renderer, drawing the workout title at the top of the card.
WHAT **Shows the workout title.**
WHERE First item in the card.
WHEN Rendered each time body runs.
WHY This is content, and it is about to be placed on glass, which is the first mistake.

```
7  .font(.headline)
```

WHO The font modifier on the title Text.
WHAT **Styles the title as headline, 17 points semibold.**
WHERE On the title Text, the first child of the VStack.
WHEN During layout of the title, and again when the text size setting changes.
WHY The type choice is fine. Good typography cannot rescue text that sits on a constantly shifting glass surface.

```
8  Text("5.2 km in 28 minutes")
```

WHO SwiftUI's text renderer, drawing the detail line under the title.
WHAT **Shows the workout detail in the default body style.**
WHERE Second item in the card.
WHEN Rendered each time body runs.
WHY Regular-weight body text on glass loses contrast over bright or busy backgrounds, exactly where people will try to read it.

```
9  Button("Share") { }
```

WHO SwiftUI and the user's tap.
WHAT **Creates a Share button with an empty action.**
WHERE Inside the card, below the text.
WHEN The empty closure would run on each tap.
WHY A button inside a content card is acceptable on its own; the trouble is the style on the next line.

```
10 .buttonStyle(.glass)
```

WHO SwiftUI's button styling, applied to the Share button inside the card.
WHAT **Gives the button its own Liquid Glass capsule.**
WHERE On the button, which already sits inside the glass card.
WHEN Rendered with the card.
WHY This is glass on glass. Glass cannot sample other glass, so the inner capsule looks flat and muddy, and the two layers compete for attention.

```
12 .padding()
```

WHO The padding modifier on the VStack, inside the card's glass.
WHAT **Adds 16 points of space around the card's contents.**
WHERE Between the VStack and the card's glass.
WHEN During layout, after the three children have measured themselves.
WHY The spacing itself is reasonable; here it only makes the oversized glass panel even larger.

```
13 .glassEffect(.regular, in: .rect(cornerRadius: 24))
```

WHO SwiftUI's Liquid Glass renderer.
WHAT **Turns the whole card, text and all, into a rounded glass panel.**
WHERE Behind the entire card.
WHEN Rendered continuously while the card is on screen, sampling whatever is behind it.
WHY Content does not belong on glass. Glass is reserved for the navigation layer floating above content; a glass card turns information into chrome and hurts legibility.

```
17 #Preview { GlassCard() }
```

WHO The Xcode preview canvas, and the simulated iPhone on this page, running GlassCard.
WHAT **Shows the card so you can judge it.**
WHERE At file scope, after the closing brace of GlassCard.
WHEN When the canvas refreshes.
WHY Look closely: the Share capsule barely separates from the glass around it, a sign of two competing layers.

B: Better: solid content, one glass layer

```
1  import SwiftUI
2
3  struct RunCard: View {
4      var body: some View {
5          NavigationStack {
6              VStack(alignment: .leading, spacing: 8) {
7                  Text("Morning Run")
8                      .font(.headline)
9                  Text("5.2 km in 28 minutes")
10                     .foregroundStyle(.secondary)
11             }
12             .navigationTitle("Activity")
13             .toolbar { Button("Share") { } }
14         }
15     }
16 }
17
18 #Preview { RunCard() }
```

```
1 import SwiftUI
```

WHO The Swift compiler, while it builds the file that declares RunCard.
WHAT **Imports SwiftUI, including NavigationStack and the toolbar API that receive Liquid Glass automatically.**
WHERE First line of this file.
WHEN At compile time, before RunCard is type-checked.
WHY With the iOS 26 SDK, standard bars and toolbars adopt glass on their own, so this fix needs no glass code at all.

```
3 struct RunCard: View {
```

WHO Your code declares RunCard; SwiftUI creates it for the preview.
WHAT **Declares the corrected version of the same workout screen.**
WHERE A top-level struct in this file, outside any other type.
WHEN Instantiated by the preview.
WHY Same information, rearranged into the right layers, so the only difference is the design.

```
4 var body: some View {
```

WHO SwiftUI calls body on RunCard; your code returns the NavigationStack.
WHAT **Describes a navigation container holding the summary and a toolbar.**
WHERE A computed property on RunCard.
WHEN On first display and on every refresh.
WHY Separating content from controls happens right here, in the structure of body.

5 `NavigationStack {`

WHO SwiftUI's NavigationStack, which draws the bar that the toolbar below fills.
WHAT **Provides a navigation bar with a title area and toolbar slots.**
WHERE The root of the screen.
WHEN Created on first display.
WHY The system container brings the bar, its glass and its behavior over scrolling content, with no custom drawing code.

6 `VStack(alignment: .leading, spacing: 8) {`

WHO The VStack inside the NavigationStack, which stacks the title and detail.
WHAT **Stacks the title and detail, aligned to the leading edge, 8 points apart.**
WHERE The content layer, inside the navigation stack.
WHEN During layout of the content layer, every time RunCard is drawn.
WHY Leading alignment gives the eye one straight edge to scan, and 8 points of spacing ties the two lines into one group.

7 `Text("Morning Run")`

WHO SwiftUI's text renderer, drawing the workout title in the content layer.
WHAT **Shows the workout title.**
WHERE First line of the content.
WHEN Rendered each time body runs.
WHY The same content as before, now on a solid background where it is always readable.

8 `.font(.headline)`

WHO The font modifier on the title Text.
WHAT **Styles the title as headline, 17 points semibold.**
WHERE On the title Text, the first child of the VStack.
WHEN During layout of the title, and again when the text size setting changes.
WHY Weight, not size, separates the title from the detail line below it.

9 `Text("5.2 km in 28 minutes")`

WHO SwiftUI's text renderer, drawing the detail line under the title.
WHAT **Shows the workout detail.**
WHERE Second line of the content.
WHEN Rendered each time body runs.
WHY On the solid content background, its contrast never depends on what happens to be behind it.

10 `.foregroundStyle(.secondary)`

WHO The foregroundStyle modifier on the detail Text.
WHAT **Uses the secondary label color, a softer gray that adapts to light and dark mode.**
WHERE On the detail line.
WHEN During rendering of the detail line, resolved again on an appearance change.
WHY Secondary color moves the detail one step down the hierarchy without changing its size.

12 `.navigationTitle("Activity")`

WHO The navigationTitle modifier on the VStack, read by the enclosing NavigationStack.
WHAT **Sets the screen title shown in the navigation bar.**
WHERE On the content, read by the enclosing NavigationStack.
WHEN When the screen appears.
WHY A title tells people where they are, and it belongs in the system bar rather than in a custom header.

```
13  .toolbar { Button("Share") { } }
```

WHO — SwiftUI and iOS 26.

WHAT — **Places a Share button in the navigation bar's toolbar.**

WHERE — The navigation layer, at the top trailing corner.

WHEN — Rendered with the navigation bar.

WHY — Toolbar items get Liquid Glass automatically in iOS 26. The control floats above content on a single glass layer, which is exactly where glass belongs.

```
18  #Preview { RunCard() }
```

WHO — The Xcode preview canvas, and the simulated iPhone on this page, running RunCard.

WHAT — **Shows RunCard in the simulated iPhone.**

WHERE — At file scope, after the closing brace of RunCard.

WHEN — When the canvas refreshes.

WHY — Compare both phones: same data, but now the content is solid and the only glass is the system toolbar.

> **Takeaway**
>
> Liquid Glass is for the controls that float above content, not for the content itself. The fix removed glass instead of adding more: the summary sits on a solid background, and the one control moved into the system toolbar, which iOS 26 renders as a single layer of glass with nothing nested inside it.

CODE · READ-ALONG

Tint, touch response, and morphing

Three more glass tools. `.tint(_:)` colors the glass to carry meaning, `.interactive()` makes it react to touch, and `glassEffectID(_:in:)` lets one glass shape grow out of another when it appears. The in-page phone does not simulate these, so this example is explained and type-checked against the iOS 26 SDK.

```swift
import SwiftUI

struct RecordControls: View {
    @Namespace private var glassSpace
    @State private var isRecording = false

    var body: some View {
        GlassEffectContainer(spacing: 24) {
            HStack(spacing: 24) {
                Button(isRecording ? "Stop" : "Record") {
                    withAnimation { isRecording.toggle() }
                }
                .padding()
                .glassEffect(.regular.tint(.red).interactive())
                .glassEffectID("record", in: glassSpace)
                if isRecording {
                    Button("Pause") { }
                        .padding()
                        .glassEffect(.regular.interactive())
                        .glassEffectID("pause", in: glassSpace)
                }
            }
        }
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares RecordControls.
- **WHAT** **Imports SwiftUI for Namespace, GlassEffectContainer, and the Glass type with its tint and interactive options.**
- **WHERE** First line of this file.
- **WHEN** At compile time, before RecordControls is type-checked.
- **WHY** These APIs exist only in the iOS 26 SDK and later, so the app must target iOS 26 to use them without availability checks.

3 `struct RecordControls: View {`

- **WHO** Your code declares RecordControls; a screen that shows it creates the value.
- **WHAT** **Declares a pair of floating recording controls.**
- **WHERE** A top-level struct in this file, outside any other type.
- **WHEN** Instantiated whenever a screen shows it.
- **WHY** Media controls floating over a camera view or an audio waveform are a textbook navigation-layer use of glass.

4 `@Namespace private var glassSpace`

- **WHO** SwiftUI owns the namespace; this view uses it.
- **WHAT** **Creates a namespace, a private naming scope used to match glass shapes across an animation.**
- **WHERE** Stored by SwiftUI for this view.
- **WHEN** Created when the view first appears and kept for its lifetime.
- **WHY** Morphing needs to know which shape before a change corresponds to which shape after it; IDs inside a shared namespace provide that link.

```
5  @State private var isRecording = false
```

WHO — SwiftUI-managed storage, changed by the Record button.
WHAT — **Tracks whether recording is in progress, starting as false.**
WHERE — SwiftUI storage tied to this view.
WHEN — Created on first appearance; toggled on each tap.
WHY — The interface is a function of state: whether the Pause control exists is decided by this one value.

```
7  var body: some View {
```

WHO — SwiftUI calls body on RecordControls whenever isRecording changes.
WHAT — **Describes the cluster of controls.**
WHERE — On RecordControls, on the main thread.
WHEN — On first display and after isRecording changes.
WHY — Reading isRecording inside body makes SwiftUI rebuild the cluster whenever it flips.

```
8  GlassEffectContainer(spacing: 24) {
```

WHO — SwiftUI's Liquid Glass renderer.
WHAT — **Groups the controls into one glass surface; shapes within 24 points of each other blend as they move.**
WHERE — The root of the cluster, in the navigation layer.
WHEN — Rendered continuously; most visible during the morph animation.
WHY — Without a shared container, Pause could not melt out of the Record shape, and the two would sample the background inconsistently.

```
9  HStack(spacing: 24) {
```

WHO — The HStack inside the GlassEffectContainer, which places Record and Pause side by side.
WHAT — **Lays the controls side by side, 24 points apart.**
WHERE — Inside the glass container.
WHEN — During layout of the container, before and after each morph.
WHY — Matching the container spacing means the shapes sit apart at rest but merge in the middle of the animation.

```
10 Button(isRecording ? "Stop" : "Record") {
```

WHO — SwiftUI, then the user tapping.
WHAT — **Creates the main button, labeled Record or Stop depending on the state.**
WHERE — The first control in the row.
WHEN — The label is recomputed with body; the action runs on each tap.
WHY — The label always names what the next tap will do, so nobody has to decode the button's current mode.

```
11 withAnimation { isRecording.toggle() }
```

WHO — SwiftUI's animation system, driven by withAnimation inside the button action.
WHAT — **Flips isRecording and animates every view change that results, including Pause appearing.**
WHERE — Inside the button's action, on the main thread.
WHEN — Each time the user taps the button.
WHY — Animated changes show where new controls come from. The glass morph explains that Pause belongs to recording.

```
13 .padding()
```

WHO — The padding modifier on the Record button, inside its glass.
WHAT — **Adds 16 points around the button label, inside the glass.**
WHERE — Between the label and its glass.
WHEN — During layout, after the Record or Stop label has measured itself.
WHY — A 17-point label plus 32 points of padding makes a target taller than the 44-point minimum for comfortable tapping.

```
14 .glassEffect(.regular.tint(.red).interactive())
```

WHO SwiftUI's Liquid Glass renderer.
WHAT **Draws regular glass tinted red that brightens and flexes when pressed.**
WHERE Behind the Record button.
WHEN Rendered continuously; the touch response plays on every press.
WHY The tint carries meaning: red signals recording. Apple asks you to tint only a primary or meaningful action, because tinting everything erases hierarchy.

```
15 .glassEffectID("record", in: glassSpace)
```

WHO SwiftUI's glass morphing, matching this shape by the id record inside glassSpace.
WHAT **Gives this glass shape the identity record inside glassSpace.**
WHERE On the Record button's glass.
WHEN Used during animations when glass shapes appear or disappear.
WHY Identified shapes can morph into each other instead of simply fading, which keeps the motion continuous and easy to follow.

```
16 if isRecording {
```

WHO SwiftUI, evaluating your condition.
WHAT **Includes the Pause button only while recording.**
WHERE Inside the HStack, after the main button.
WHEN Evaluated each time body runs.
WHY Showing a control only when it can do something keeps the floating cluster small and uncluttered.

```
17 Button("Pause") { }
```

WHO SwiftUI and the user.
WHAT **Creates a Pause button; its action is left empty in this sketch.**
WHERE The second control, present only while recording.
WHEN Appears after the first tap; its action would run on tap.
WHY Related actions appear next to each other so they read as one group.

```
18 .padding()
```

WHO The padding modifier on the Pause button, inside its glass.
WHAT **Pads the Pause label exactly like the Record label.**
WHERE On the Pause button.
WHEN During layout, after the Pause label has measured itself.
WHY Equal padding gives both glass shapes the same height, so the cluster looks like one family.

```
19 .glassEffect(.regular.interactive())
```

WHO SwiftUI's Liquid Glass renderer.
WHAT **Gives Pause untinted, touch-responsive regular glass.**
WHERE Behind the Pause button.
WHEN Rendered continuously while Pause is on screen, and animated as it appears.
WHY Leaving it untinted keeps red reserved for the one meaningful action.

```
20 .glassEffectID("pause", in: glassSpace)
```

WHO SwiftUI's glass morphing, matching this shape by the id pause inside glassSpace.
WHAT **Identifies the Pause glass inside the same namespace.**
WHERE On the Pause button's glass.
WHEN When Pause is inserted or removed.
WHY Because both IDs share one namespace and one container, SwiftUI morphs Pause out of the Record shape instead of popping it in.

Try it

1. Predict: if the HStack were not inside GlassEffectContainer, how would Pause appear when recording starts?

INTERACTIVE

Liquid Glass lab

Drag the glass toolbar over the photo, the text, the colored tiles and the bright and dark blocks (or focus it and use the arrow keys). Switch between Regular and Clear, change the blur and tint, then turn on the glass-on-glass mistake. The legibility meter computes the contrast between the icons and what is really behind them, using a Swift function compiled to WebAssembly.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. A recipe app has a scrolling list of recipes, a floating Add button and a tab bar. Which of these should use Liquid Glass?
   A. Each recipe row, so the list feels modern
   B. The floating Add button and the tab bar
   C. The recipe photos
   D. The recipe text, so the background shows through
2. A teammate wraps a card in `.glassEffect()` and gives the buttons inside it `.buttonStyle(.glass)`. What is the best fix?
   A. Switch the card to clear glass so the buttons stand out against it
   B. Tint every button a different color so each separates from the card
   C. Put a second GlassEffectContainer around the card and its buttons
   D. Remove the glass from the card, or move its buttons into the toolbar
3. When is `.glassEffect(.clear)` the right choice?
   A. For a settings list on a plain white background
   B. For any button that should look modern and new
   C. For controls over full-screen photos or video, with bold content on them
   D. Whenever regular glass looks too strong in Dark Mode
4. Why should neighboring glass controls share one `GlassEffectContainer`?
   A. Glass cannot sample other glass; one shared surface keeps them consistent
   B. Without it no glass appears at all, because the container draws the material
   C. It makes the buttons readable by VoiceOver, which otherwise skips glass
   D. It adds the 44-point minimum touch target to each button inside it

5. To match the brand, a designer tints every glass button in a toolbar bright orange. What does the screen lose?
   A. Dark Mode support, because tinted glass ignores the color scheme
   B. Hierarchy, because when every button is tinted nothing stands out
   C. Nothing, because tint is purely decorative and changes no behavior
   D. Performance, because tinted glass cannot be grouped in a container

Answers: Liquid Glass

1. **B. The floating Add button and the tab bar** Glass belongs to the navigation layer: controls that float above content, like a floating button and the tab bar (which gets glass automatically). Rows, photos and text are content. Putting them on glass hurts legibility and blurs the line between what people read and what they tap.
2. **D. Remove the glass from the card, or move its buttons into the toolbar** The card is content, and the buttons are glass on glass. Removing the card's glass fixes both problems. Clear glass or tints only change how the stack of layers looks, and a container groups controls; it does not make content on glass acceptable.
3. **C. For controls over full-screen photos or video, with bold content on them** Clear glass is very transparent and does not adapt, so Apple limits it to controls over media-rich content where the symbols or text on it are bold and bright (with a dimming layer if needed). Everywhere else, regular glass adapts for you and stays legible.
4. **A. Glass cannot sample other glass; one shared surface keeps them consistent** A single glass shape works without a container, so glass does appear. But neighbors in separate surfaces sample the background inconsistently and cannot blend. The container also has nothing to do with VoiceOver labels or tap sizes; those come from labels and padding.
5. **B. Hierarchy, because when every button is tinted nothing stands out** Tint is a signal. One tinted button says "this is the main action"; five say nothing. Tinted glass still adapts to light and dark mode and can be grouped in a container, so the loss is meaning, not a technical limit.

Typography and Dynamic Type

CONCEPT

San Francisco, points, and text styles

iOS uses one system typeface family, **San Francisco**. On iPhone you see **SF Pro**; the family also includes SF Compact (Apple Watch), SF Mono (code) and the serif New York. San Francisco tunes itself for each size: small text gets looser letter spacing and sturdier shapes, large text gets tighter spacing, so it reads well from an 11-point caption to a 34-point title.

Sizes on iOS are measured in **points** (pt), not pixels. The system draws each point with 2 or 3 physical pixels depending on the screen, so 17 points looks about the same size on every iPhone.

Text style	A named role such as body or headline. You pick the role; iOS picks the size, weight and spacing.
Dynamic Type	The setting that lets each person choose their text size, from xSmall to AX5. Text styles follow it automatically.
Default size	Large, where body text is 17 points. Apple treats 11 points as the smallest size for readable iOS text.

> **Why one typeface is enough**
>
> Hierarchy comes from size, weight and color, not from mixing fonts. Apple recommends keeping the number of typefaces small, and avoiding light weights (Ultralight, Thin, Light) for text people must read.

CONCEPT

The text style ladder

These are the eleven text styles at the default Large size. Each name describes a *role*, so the same code stays correct at every size: `.largeTitle` 34 pt, `.title` 28, `.title2` 22, `.title3` 20, `.headline` 17 semibold, `.body` 17, `.callout` 16, `.subheadline` 15, `.footnote` 13, `.caption` 12, `.caption2` 11.

Headline vs. body	Both are 17 points; headline is semibold. Weight alone separates a row's title from its detail.
Growing	At AX5, the largest accessibility size, body becomes 53 points and caption2 becomes 40, while largeTitle only reaches 60. Small styles grow fastest because they are the hardest to read.
Shrinking	At xSmall, body drops to 14 points, but caption and caption2 never go below 11.

> **Choose by role, not by number**
>
> If a design calls for 15-point gray text under a title, write `.font(.subheadline)` with `.foregroundStyle(.secondary)`, not `.font(.system(size: 15))`. The role scales with the person's setting; the number never does.

CODE · LIVE SWIFTUI PREVIEW

Text styles build hierarchy

A recipe card with four levels of text. Each line uses a text style and, where needed, weight or color, so the most important line is obvious at a glance. Nothing here sets a size in points, so the whole card scales with Dynamic Type.

```swift
import SwiftUI

struct RecipeCard: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 8) {
            Text("Weeknight dinner")
                .font(.caption)
                .textCase(.uppercase)
                .foregroundStyle(.secondary)
            Text("Lemon Garlic Pasta")
                .font(.title2)
                .bold()
            Text("Bright and fast, made from pantry staples.")
            Text("4 servings · 6 ingredients")
                .font(.footnote)
                .foregroundStyle(.secondary)
        }
        .padding()
    }
}

#Preview { RecipeCard() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares RecipeCard.
- WHAT — **Imports SwiftUI so this file can use Text, VStack and text styles such as .title2 and .footnote.**
- WHERE — First line of this file.
- WHEN — At compile time, before RecipeCard is type-checked.
- WHY — Text styles are part of SwiftUI's Font type, so using them takes nothing more than this import.

3 `struct RecipeCard: View {`

- WHO — Your code declares RecipeCard; SwiftUI creates it for the preview.
- WHAT — **Declares a card that previews a recipe with four levels of text.**
- WHERE — A top-level struct in this file, outside any other type.
- WHEN — Instantiated when the preview runs.
- WHY — A card is a compact place to practice hierarchy, because every line has a different job.

4 `var body: some View {`

- WHO — SwiftUI calls body on RecipeCard; your code returns the VStack.
- WHAT — **Describes the four lines of text and the spacing between them.**
- WHERE — A computed property on RecipeCard, on the main thread.
- WHEN — On first display and whenever the text size setting changes.
- WHY — Because body uses text styles, a Dynamic Type change simply re-runs layout with new sizes; you write no extra code.

```
5  VStack(alignment: .leading, spacing: 8) {
```

WHO The VStack, which stacks the four text lines and aligns their leading edges.
WHAT **Stacks the lines vertically, aligned to the leading edge, 8 points apart.**
WHERE The root of body.
WHEN During layout, every time RecipeCard is drawn.
WHY Leading alignment (left in English, right in Arabic or Hebrew) gives the eye one edge to return to, the most readable choice for multi-line text.

```
6  Text("Weeknight dinner")
```

WHO SwiftUI's text renderer, drawing the eyebrow label at the top of the card.
WHAT **Shows a category label above the title.**
WHERE The top line of the card.
WHEN Rendered each time body runs.
WHY A short label that sets context before a title is often called an eyebrow. It must be quieter than the title it introduces.

```
7  .font(.caption)
```

WHO SwiftUI with Dynamic Type.
WHAT **Uses the caption style: 12 points at the default size.**
WHERE On the eyebrow label.
WHEN During layout and whenever the text size setting changes.
WHY A small style suits a label people skim. It still grows to 22 points at AX1 and 43 at AX5 for people who need it.

```
8  .textCase(.uppercase)
```

WHO The textCase modifier on the eyebrow Text.
WHAT **Displays the label in capital letters without changing the underlying string.**
WHERE On the eyebrow label.
WHEN During rendering of the eyebrow, each time the card is drawn.
WHY Small capitals read as a label, not a sentence. Keeping the source text in normal case keeps translations and VoiceOver natural.

```
9  .foregroundStyle(.secondary)
```

WHO The foregroundStyle modifier on the eyebrow Text.
WHAT **Colors the eyebrow with the secondary label color.**
WHERE On the eyebrow label.
WHEN During rendering, adapting automatically in Dark Mode.
WHY Color is the third tool for hierarchy after size and weight: secondary gray says this line matters less than the title.

```
10 Text("Lemon Garlic Pasta")
```

WHO SwiftUI's text renderer, drawing the recipe name as the second line.
WHAT **Shows the recipe name.**
WHERE The second line of the card.
WHEN Rendered each time body runs.
WHY The title is the one line everybody reads, so it gets the most visual weight.

```
11 .font(.title2)
```

WHO SwiftUI with Dynamic Type.
WHAT **Uses title2: 22 points at the default size.**
WHERE On the title Text, the second child of the VStack.
WHEN During layout of the title, and again when the text size setting changes.
WHY Title2 anchors a card without shouting the way largeTitle (34 points) would; largeTitle is meant for the title of a whole screen.

```
12  .bold()
```

WHO The bold modifier on the title Text.
WHAT **Switches the title to the bold weight.**
WHERE On the title Text, after the title2 font.
WHEN During rendering of the title, and again when Bold Text is switched on.
WHY Size plus weight puts the title clearly at the top of the hierarchy, and it gets bolder still when someone turns on the Bold Text setting.

```
13  Text("Bright and fast, made from pantry staples.")
```

WHO SwiftUI's text renderer, drawing the description in the default body style.
WHAT **Shows the description in the body style, 17 points at the default size.**
WHERE The third line of the card.
WHEN Rendered each time body runs.
WHY Body is the default font for Text, so no modifier is needed. It is the style designed for reading sentences.

```
14  Text("4 servings · 6 ingredients")
```

WHO SwiftUI's text renderer, drawing the facts line at the bottom of the card.
WHAT **Shows supporting facts about the recipe.**
WHERE The last line of the card.
WHEN Rendered each time body runs.
WHY Supporting details go last and smallest, after the content people actually came for.

```
15  .font(.footnote)
```

WHO SwiftUI with Dynamic Type.
WHAT **Uses footnote: 13 points at the default size.**
WHERE On the facts line.
WHEN During layout of the facts line, and again when the text size setting changes.
WHY Footnote sits clearly below body, enough difference to read as secondary without becoming hard to read.

```
16  .foregroundStyle(.secondary)
```

WHO The foregroundStyle modifier on the facts Text.
WHAT **Colors the facts line secondary gray.**
WHERE On the facts line.
WHEN During rendering of the facts line, resolved again in Dark Mode.
WHY Repeating the eyebrow's treatment makes similar kinds of information look alike. That repetition is what consistency means.

```
18  .padding()
```

WHO The padding modifier on the VStack that holds the whole card.
WHAT **Adds the default 16 points of space around the whole card.**
WHERE Outside the VStack, as the last modifier before body returns.
WHEN During layout, after the four lines have measured themselves.
WHY 16 points matches the standard iPhone margin, so the card lines up with other iOS content.

```
22  #Preview { RecipeCard() }
```

WHO The Xcode preview canvas, and the simulated iPhone on this page, running RecipeCard.
WHAT **Shows RecipeCard in the simulated iPhone.**
WHERE At file scope, after the closing brace of RecipeCard.
WHEN When the canvas refreshes.
WHY Squint at the preview: if you can still tell which line is the title, the hierarchy works.

Try it

1. Change .title2 to .largeTitle. Does the card still feel balanced?
2. Delete both .foregroundStyle(.secondary) lines. Which line now competes with the title?

COMPARE

Fixed sizes freeze; text styles scale

Two flight rows that look identical at the default text size. The left one hard-codes point sizes and a gray; the right one uses text styles and a semantic color. Only the right one changes when a person picks a larger text size.

A: Poor: hard-coded sizes

```swift
import SwiftUI

struct FixedRow: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 4) {
            Text("Flight to Lisbon")
                .font(.system(size: 17, weight: .semibold))
            Text("Gate B12 · Boards at 9:40")
                .font(.system(size: 13))
                .foregroundStyle(.gray)
        }
    }
}

#Preview { FixedRow() }
```

`1` `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares FixedRow.
- WHAT — **Imports SwiftUI, whose Font.system(size:weight:) creates fixed-size fonts.**
- WHERE — First line of this file.
- WHEN — At compile time, before FixedRow is type-checked.
- WHY — The fixed-size API is real and occasionally right (for a decorative scoreboard, say), which is exactly why it is so easy to misuse for ordinary text.

`3` `struct FixedRow: View {`

- WHO — Your code declares FixedRow; SwiftUI creates it for the preview.
- WHAT — **Declares a flight row built with hard-coded font sizes.**
- WHERE — A top-level struct in this file, outside any other type.
- WHEN — Instantiated by the preview.
- WHY — It looks perfect at the default text size, which is why this bug ships: most people test only at Large.

`4` `var body: some View {`

- WHO — SwiftUI calls body on FixedRow; your code returns the VStack.
- WHAT — **Describes the title and detail lines.**
- WHERE — A computed property on FixedRow.
- WHEN — On display and refresh, but the sizes it produces never change.
- WHY — Nothing in this body reads the person's text size preference, so the row cannot adapt to it.

```
5  VStack(alignment: .leading, spacing: 4) {
```

WHO The VStack, which stacks the title and detail lines 4 points apart.
WHAT **Stacks the two lines, leading-aligned, 4 points apart.**
WHERE The root of body.
WHEN During layout, every time FixedRow is drawn.
WHY Four points keeps the lines tightly grouped as one item; the layout itself is fine.

```
6      Text("Flight to Lisbon")
```

WHO SwiftUI's text renderer, drawing the flight title at the top of the row.
WHAT **Shows the flight title.**
WHERE The first line of the row.
WHEN Rendered each time body runs.
WHY The most important line on the row, so it most needs to grow for people with low vision.

```
7      .font(.system(size: 17, weight: .semibold))
```

WHO The font modifier on the title Text, with a fixed system font.
WHAT **Sets San Francisco at exactly 17 points semibold, whatever text size the person chose.**
WHERE On the title Text, the first child of the VStack.
WHEN During layout of the title, and never again when the text size changes.
WHY It matches headline at the default size, but it is frozen: at AX5, where headline becomes 53 points, this stays at 17.

```
8      Text("Gate B12 · Boards at 9:40")
```

WHO SwiftUI's text renderer, drawing the gate and time under the title.
WHAT **Shows the gate and boarding time.**
WHERE The second line of the row.
WHEN Rendered each time body runs.
WHY Critical travel details in the smallest text on the row are exactly what people with low vision need to enlarge.

```
9      .font(.system(size: 13))
```

WHO The font modifier on the detail Text, with a fixed 13-point font.
WHAT **Sets the detail to a fixed 13 points.**
WHERE On the detail line.
WHEN During layout of the detail line, and never again when the text size changes.
WHY 13 points equals footnote at Large, but it never changes. Someone who chose xxxLarge expects 19 points here and still gets 13.

```
10     .foregroundStyle(.gray)
```

WHO The foregroundStyle modifier on the detail Text, with a fixed gray.
WHAT **Draws the detail in a fixed mid gray.**
WHERE On the detail line.
WHEN During rendering of the detail line, the same in Light and Dark Mode.
WHY Color.gray is the same in light and dark mode and ignores Increase Contrast, so this small text has weaker contrast than the system's secondary color would give it.

```
15 #Preview { FixedRow() }
```

WHO The Xcode preview canvas, and the simulated iPhone on this page, running FixedRow.
WHAT **Shows FixedRow in the simulated iPhone.**
WHERE At file scope, after the closing brace of FixedRow.
WHEN When the canvas refreshes.
WHY At the default size both previews look identical, which is the trap: the difference only appears when the text size changes.

B: Better: text styles and semantic color

```swift
import SwiftUI

struct ScalingRow: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 4) {
            Text("Flight to Lisbon")
                .font(.headline)
            Text("Gate B12 · Boards at 9:40")
                .font(.footnote)
                .foregroundStyle(.secondary)
        }
    }
}

#Preview { ScalingRow() }
```

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares ScalingRow.
- **WHAT** **Imports SwiftUI, whose text styles track Dynamic Type.**
- **WHERE** First line of this file.
- **WHEN** At compile time, before ScalingRow is type-checked.
- **WHY** Text styles are the standard way to set fonts in SwiftUI, so the scalable choice is also the shortest code.

3 `struct ScalingRow: View {`

- **WHO** Your code declares ScalingRow; SwiftUI creates it for the preview.
- **WHAT** **Declares the same flight row using text styles.**
- **WHERE** A top-level struct in this file, outside any other type.
- **WHEN** Instantiated by the preview.
- **WHY** Same layout and same look at the default size; only the font and color declarations change.

4 `var body: some View {`

- **WHO** SwiftUI calls body on ScalingRow, and again when the text size setting changes.
- **WHAT** **Describes the title and detail lines.**
- **WHERE** A computed property on ScalingRow.
- **WHEN** On display, and again whenever the person changes their text size.
- **WHY** Text styles subscribe to the Dynamic Type setting, so SwiftUI re-lays out the row automatically when it changes.

5 `VStack(alignment: .leading, spacing: 4) {`

- **WHO** The VStack, which stacks the title and detail lines 4 points apart.
- **WHAT** **Stacks the lines with the same 4-point spacing as before.**
- **WHERE** The root of body.
- **WHEN** During layout, every time ScalingRow is drawn.
- **WHY** Keeping the layout identical proves the fix is purely typographic.

6 `Text("Flight to Lisbon")`

- **WHO** SwiftUI's text renderer, drawing the flight title at the top of the row.
- **WHAT** **Shows the flight title.**
- **WHERE** The first line of the row.
- **WHEN** Rendered each time body runs.
- **WHY** Same content as the fixed row, so the comparison is fair.

```
7   .font(.headline)
```

WHO — SwiftUI with Dynamic Type.
WHAT — **Uses headline: 17 points semibold at Large, ranging from 14 at xSmall to 53 at AX5.**
WHERE — On the title Text, the first child of the VStack.
WHEN — During layout and every time the text size setting changes.
WHY — You choose the role and iOS supplies the size, weight and letter spacing for every setting, plus Bold Text support.

```
8   Text("Gate B12 · Boards at 9:40")
```

WHO — SwiftUI's text renderer, drawing the gate and time under the title.
WHAT — **Shows the gate and boarding time.**
WHERE — The second line of the row.
WHEN — Rendered each time body runs.
WHY — Now this critical detail can grow when someone needs it to.

```
9   .font(.footnote)
```

WHO — SwiftUI with Dynamic Type.
WHAT — **Uses footnote: 13 points at Large, 19 at xxxLarge and 44 at AX5.**
WHERE — On the detail line.
WHEN — During layout and whenever the setting changes.
WHY — At the default size it matches the fixed version exactly, but it respects the person's choice everywhere else.

```
10  .foregroundStyle(.secondary)
```

WHO — The foregroundStyle modifier on the detail Text, with the secondary role.
WHAT — **Uses the system secondary label color.**
WHERE — On the detail line.
WHEN — During rendering, adapting to light mode, dark mode and Increase Contrast.
WHY — Semantic colors adjust automatically for appearance and accessibility settings; a raw gray cannot.

```
15  #Preview { ScalingRow() }
```

WHO — The Xcode preview canvas, and the simulated iPhone on this page, running ScalingRow.
WHAT — **Shows ScalingRow in the simulated iPhone.**
WHERE — At file scope, after the closing brace of ScalingRow.
WHEN — When the canvas refreshes.
WHY — Try the Dynamic Type explorer later in this lesson to see what this row does that the fixed one cannot.

> **Takeaway**
>
> At the default size the two rows are identical, which is why fixed sizes slip through review. The difference is behavior: text styles follow the person's text size choice (footnote runs from 12 to 44 points) while `.system(size:)` stays frozen. Choose fonts by role, and the numbers take care of themselves.

CODE · LIVE SWIFTUI PREVIEW

Let the layout adapt to big text

Bigger text needs room. When a row's pieces no longer fit side by side, switch to a stacked layout instead of cutting words off. `ViewThatFits` tries each child in order and shows the first one that fits the available width.

```swift
import SwiftUI

struct ContactRow: View {
    var body: some View {
        ViewThatFits {
            HStack {
                Label("Ana Souza", systemImage: "person.circle")
                Spacer()
                Text("Mobile")
                    .foregroundStyle(.secondary)
            }
            VStack(alignment: .leading) {
                Label("Ana Souza", systemImage: "person.circle")
                Text("Mobile")
                    .foregroundStyle(.secondary)
            }
        }
        .padding()
    }
}

#Preview { ContactRow() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares ContactRow.
- **WHAT** **Imports SwiftUI for ViewThatFits, Label and the stacks.**
- **WHERE** First line of this file.
- **WHEN** At compile time, before ContactRow is type-checked.
- **WHY** ViewThatFits has been part of SwiftUI since iOS 16, so every iOS 26 device supports it.

3 `struct ContactRow: View {`

- **WHO** Your code declares ContactRow; SwiftUI creates it for the preview.
- **WHAT** **Declares a contact row that chooses its own layout.**
- **WHERE** A top-level struct in this file, outside any other type.
- **WHEN** Instantiated when the preview runs.
- **WHY** Rows with a name on one side and a detail on the other are where large text most often breaks.

4 `var body: some View {`

- **WHO** SwiftUI calls body on ContactRow; your code returns the ViewThatFits.
- **WHAT** **Describes both candidate layouts.**
- **WHERE** A computed property on ContactRow.
- **WHEN** On display and whenever the text size or available width changes.
- **WHY** body offers options; SwiftUI measures and picks, so you never compute text widths yourself.

5 `ViewThatFits {`

- **WHO** The SwiftUI layout system, measuring each candidate child in order.
- **WHAT** **Measures its children in order and displays the first whose ideal width fits.**
- **WHERE** The root of body.
- **WHEN** During every layout pass, including after a text size change or rotation.
- **WHY** It replaces fragile hand-tuned breakpoints: the decision comes from the real measured text, in any language and at any size.

```
6  HStack {
```

WHO The HStack, the first candidate that ViewThatFits measures.
WHAT **First choice: the name and the phone type on one line.**
WHERE The first child of ViewThatFits.
WHEN Measured first on every layout pass.
WHY Side by side is the most compact arrangement, so it is preferred whenever it fits.

```
7  Label("Ana Souza", systemImage: "person.circle")
```

WHO SwiftUI and SF Symbols.
WHAT **Shows a person symbol followed by the name.**
WHERE The leading side of the HStack.
WHEN Rendered each time body runs.
WHY Label pairs a symbol with text and keeps them aligned and scaled together as the font grows.

```
8  Spacer()
```

WHO The Spacer inside the HStack, which claims the free width.
WHAT **Pushes the phone type to the trailing edge.**
WHERE Between the name and the detail.
WHEN During layout of the HStack, after the Label and the Text report their widths.
WHY Pinning details to the trailing edge gives every row in a list the same two columns, which makes them easy to scan.

```
9  Text("Mobile")
```

WHO SwiftUI's text renderer, drawing the phone type at the trailing edge.
WHAT **Shows which kind of phone number this is.**
WHERE The trailing side of the HStack.
WHEN Rendered each time body runs.
WHY Supporting detail sits at the end of the row, where iOS lists usually show values.

```
10 .foregroundStyle(.secondary)
```

WHO The foregroundStyle modifier on the Mobile Text inside the HStack.
WHAT **Colors the detail secondary gray.**
WHERE On the detail text in the HStack.
WHEN During rendering of the HStack's detail text, whenever that layout wins.
WHY Lower emphasis than the name, matching how iOS styles values in lists.

```
12 VStack(alignment: .leading) {
```

WHO The VStack, the second candidate that ViewThatFits measures.
WHAT **Fallback: the same two pieces stacked vertically, leading-aligned.**
WHERE The second child of ViewThatFits.
WHEN Shown only when the HStack is too wide for the screen.
WHY Stacking keeps every word visible at accessibility sizes instead of cutting off the name with an ellipsis.

```
13 Label("Ana Souza", systemImage: "person.circle")
```

WHO SwiftUI and SF Symbols.
WHAT **The same name label, now on its own line.**
WHERE The top of the fallback VStack.
WHEN Rendered only when this layout wins.
WHY Repeating the content in both branches keeps them equivalent; only the arrangement changes.

```
14  Text("Mobile")
```

WHO	SwiftUI's text renderer, drawing the phone type on its own line.
WHAT	**Places the detail on its own line below the name.**
WHERE	The second line of the fallback VStack.
WHEN	Rendered only when this layout wins.
WHY	On its own line the detail can be as large as the person needs without crowding the name.

```
15  .foregroundStyle(.secondary)
```

WHO	The foregroundStyle modifier on the Mobile Text inside the VStack.
WHAT	**Keeps the detail secondary in the stacked layout.**
WHERE	On the detail text in the VStack.
WHEN	During rendering of the stacked detail text, whenever that layout wins.
WHY	The hierarchy should survive the layout switch, so both branches style the detail identically.

```
18  .padding()
```

WHO	The padding modifier on the ViewThatFits.
WHAT	**Adds the default 16 points around whichever layout wins.**
WHERE	On the ViewThatFits, outside both candidate layouts.
WHEN	During layout, after ViewThatFits has chosen a child.
WHY	Padding outside ViewThatFits applies equally to both options, so the switch never shifts the margins.

```
22  #Preview { ContactRow() }
```

WHO	The Xcode preview canvas, and the simulated iPhone on this page, running ContactRow.
WHAT	**Shows ContactRow in the simulated iPhone.**
WHERE	At file scope, after the closing brace of ContactRow.
WHEN	When the canvas refreshes.
WHY	Lengthen the name in the preview to watch the row switch layouts without any size checks in your code.

> **Try it**
>
> 1. Make the name much longer, with a middle name and a second surname. At what point does the row switch to the stacked layout?

CODE · READ-ALONG

Scaling custom sizes and fonts

Three tools for custom designs that still respect Dynamic Type. `@ScaledMetric` scales any number, such as an icon size. `Font.custom(_:size:relativeTo:)` scales a non-system font. The `dynamicTypeSize` environment value tells you when the person uses an accessibility size, so the layout can change.

```swift
import SwiftUI

struct BadgeRow: View {
    @ScaledMetric(relativeTo: .body) private var iconSize = 24.0
    @Environment(\.dynamicTypeSize) private var typeSize

    var body: some View {
        let layout = typeSize.isAccessibilitySize
            ? AnyLayout(VStackLayout(alignment: .leading))
            : AnyLayout(HStackLayout())
        layout {
            Image(systemName: "star.fill")
                .resizable()
                .scaledToFit()
                .frame(width: iconSize)
            Text("Top reviewer")
                .font(.custom("Georgia", size: 17, relativeTo: .body))
        }
        .dynamicTypeSize(...DynamicTypeSize.accessibility3)
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares BadgeRow.
- **WHAT** **Imports SwiftUI for ScaledMetric, AnyLayout, the stack layouts and custom fonts.**
- **WHERE** First line of this file.
- **WHEN** At compile time, before BadgeRow is type-checked.
- **WHY** Every Dynamic Type tool here is part of SwiftUI; no extra framework is needed.

3 `struct BadgeRow: View {`

- **WHO** Your code declares BadgeRow; a parent view creates it wherever the badge appears.
- **WHAT** **Declares a badge row with a star icon and a label in a custom font.**
- **WHERE** A top-level struct in this file, outside any other type.
- **WHEN** Instantiated wherever the badge appears.
- **WHY** Badges mix an icon, custom type and a layout, the three things that most often fail to scale.

4 `@ScaledMetric(relativeTo: .body) private var iconSize = 24.0`

- **WHO** SwiftUI, reading the Dynamic Type setting.
- **WHAT** **Declares a number that is 24 at the default text size and scales in proportion to the body style at other sizes.**
- **WHERE** Stored by SwiftUI for this view.
- **WHEN** Recomputed whenever the text size setting changes.
- **WHY** Icons and spacing set in fixed points look tiny next to enlarged text. ScaledMetric keeps them in proportion.

5 `@Environment(\.dynamicTypeSize) private var typeSize`

- **WHO** SwiftUI's environment, filled in by iOS.
- **WHAT** **Reads the person's current text size, from xSmall up to accessibility5.**
- **WHERE** An environment value passed down the view tree.
- **WHEN** Updated whenever the setting changes, which re-runs body.
- **WHY** Some layouts must change at accessibility sizes, and this value lets body make that decision.

```
7  var body: some View {
```

WHO SwiftUI calls body on BadgeRow, and again whenever typeSize changes.
WHAT **Describes the badge, choosing a layout first.**
WHERE A computed property on BadgeRow.
WHEN On display and after any text size change.
WHY Because it reads typeSize, body re-runs at exactly the moment the layout might need to switch.

```
8  let layout = typeSize.isAccessibilitySize
```

WHO Your code inside body.
WHAT **Starts choosing a layout based on whether the size is one of the five accessibility sizes, AX1 to AX5.**
WHERE A local constant inside body.
WHEN Every time body runs.
WHY At accessibility sizes body text is 28 points or more, so side-by-side layouts rarely fit and stacking reads better.

```
9  ? AnyLayout(VStackLayout(alignment: .leading))
```

WHO SwiftUI's AnyLayout, wrapping a VStackLayout for the accessibility sizes.
WHAT **When the size is an accessibility size, picks a vertical, leading-aligned layout.**
WHERE The first branch of the conditional expression.
WHEN Evaluated with the line above.
WHY AnyLayout wraps different layout types in one type, so SwiftUI can switch between them, animate the change, and keep each child's state.

```
10 : AnyLayout(HStackLayout())
```

WHO SwiftUI's AnyLayout, wrapping an HStackLayout for the standard sizes.
WHAT **Otherwise picks a horizontal layout.**
WHERE The second branch of the conditional expression.
WHEN Evaluated with the lines above.
WHY Side by side is the compact default for the seven standard sizes.

```
11 layout {
```

WHO The chosen layout, which arranges the star and the text.
WHAT **Arranges the children with whichever layout was chosen.**
WHERE The root view of body.
WHEN During layout, every time body runs with a new layout choice.
WHY The same children move between layouts instead of being rebuilt, unlike an if/else that swaps two entire stacks.

```
12 Image(systemName: "star.fill")
```

WHO SwiftUI and SF Symbols.
WHAT **Draws a filled star badge.**
WHERE The first child of the layout.
WHEN Rendered each time body runs.
WHY An icon beside text is the classic place where a fixed size looks wrong at large text sizes.

```
13 .resizable()
```

WHO The resizable modifier on the star image.
WHAT **Lets the symbol stretch to whatever frame it is given.**
WHERE On the star image.
WHEN During layout of the star, before the frame below sizes it.
WHY Without it, the frame below would only reserve space; the star itself would not change size.

```
14  .scaledToFit()
```

WHO The scaledToFit modifier on the star image.
WHAT **Keeps the star's proportions while it resizes.**
WHERE On the star image.
WHEN During layout of the star, as the frame below resizes it.
WHY A stretched or squashed icon looks broken; fitting preserves its aspect ratio.

```
15  .frame(width: iconSize)
```

WHO The frame modifier on the star image, reading iconSize.
WHAT **Sets the star's width to the scaled metric, 24 points at the default size.**
WHERE On the star image.
WHEN During layout, and again after every text size change.
WHY The icon now grows with the text, so the badge stays balanced at every size.

```
16  Text("Top reviewer")
```

WHO SwiftUI's text renderer, drawing the badge text next to the star.
WHAT **Shows the badge text.**
WHERE The second child of the layout.
WHEN Rendered each time body runs.
WHY The label is what makes the star meaningful, so it must stay readable at every size.

```
17  .font(.custom("Georgia", size: 17, relativeTo: .body))
```

WHO SwiftUI and the iOS font system.
WHAT **Uses the Georgia typeface at 17 points for the default size, scaled in step with the body style.**
WHERE On the badge text.
WHEN During layout and whenever the text size changes.
WHY Custom fonts are fine, but they must scale. relativeTo: ties the font to a text style so it grows exactly like body would.

```
19  .dynamicTypeSize(...DynamicTypeSize.accessibility3)
```

WHO The dynamicTypeSize modifier on the layout, capping its children.
WHAT **Caps this row at AX3: when the setting is AX4 or AX5, the row renders as if it were AX3.**
WHERE On the layout, affecting only its children.
WHEN During layout, only for the two largest settings.
WHY A last resort for a spot that truly cannot grow further. Use it rarely, and never cap below the accessibility sizes, because people chose that size on purpose.

> **Try it**
>
> 1. Predict: at AX5, what width does iconSize produce compared with the default 24 points, and why might that be a reason to keep the cap on this row?

INTERACTIVE

Dynamic Type explorer

Drag the slider through all twelve Dynamic Type sizes. The left column shows every text style at its true point size for that setting, with the change from the default. The phone shows one screen built two ways: with text styles it grows and switches to a stacked layout; with fixed sizes it stays small or gets cut off. Notice which styles grow fastest.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. A label uses `.font(.system(size: 13))`. A person sets their text size to AX3. How large is the label?
   A. 33 points, because it scales like footnote at AX3
   B. 13 points; system(size:) ignores Dynamic Type
   C. 26 points, because the setting doubles every font size
   D. It depends on the iPhone model and its screen size

2. A design shows a row title at 17 points semibold over a description at 17 points regular. Which text styles match?
   A. .title3 and .body
   B. .body with .bold() and .callout
   C. .headline and .body
   D. .subheadline and .footnote

3. At AX2, a row's name and value no longer fit side by side. What is the best response?
   A. Switch to a stacked layout with ViewThatFits
   B. Add .lineLimit(1) so the row stays one line
   C. Cap Dynamic Type at Large for the whole app
   D. Shrink the font with .system(size: 12)

4. Your brand uses a custom typeface. How should body text use it?
   A. .font(.custom("Brand", fixedSize: 17))
   B. Only in marketing screenshots; apps must ship San Francisco
   C. .font(.system(size: 17)) plus an Info.plist entry
   D. .font(.custom("Brand", size: 17, relativeTo: .body))

5. Between the default Large size and AX5, which text style grows the most in proportion?
   A. largeTitle, 34 to 60 points
   B. body, 17 to 53 points
   C. title, 28 to 58 points
   D. caption2, 11 to 40 points

Answers: Typography and Dynamic Type

1. **B. 13 points; system(size:) ignores Dynamic Type** A fixed size never changes. Footnote, the text style that is 13 points at the default size, would be 33 points at AX3, which is exactly the growth this label misses. Points are device-independent, so the model does not matter.
2. **C. .headline and .body** Headline and body are both 17 points at the default size; headline is semibold. Title3 is 20 points, callout is 16, and a bold body is a different weight from semibold, so those pairs would not match the design or its roles.
3. **A. Switch to a stacked layout with ViewThatFits** Stacking keeps every word visible at the size the person chose. A one-line limit truncates the name, and capping or shrinking undoes the setting that person relies on to read at all.
4. **D. .font(.custom("Brand", size: 17, relativeTo: .body))** relativeTo: .body makes the custom font scale exactly like the body style. fixedSize: never scales. Custom fonts are allowed in iOS apps, and while the Info.plist registers a bundled font, it does not make text scale.
5. **D. caption2, 11 to 40 points** caption2 grows about 3.6 times, body about 3.1 times, title about 2.1 times and largeTitle only about 1.8 times. Small text grows fastest because it is the hardest to read; titles are already large and would overflow the screen if they grew as much.

Color, Dark Mode, and Contrast

CONCEPT

Colors named for their job

iOS gives you colors named for what they *do*, not how they look. A **semantic color** such as the primary label color means “the main text on this screen.” The system picks the exact value for Light Mode, Dark Mode, Increase Contrast, and the slightly lighter background of a sheet. Your code states the purpose; iOS chooses the paint.

In SwiftUI you use them as the `.primary`, `.secondary` and `.tertiary` foreground styles, as the background every screen already has, and as system colors such as `.blue` and `.red`, which also shift per appearance: system blue is #007AFF in Light Mode and #0A84FF in Dark Mode.

Dark Mode	A system-wide dark appearance. People switch it in Control Center or schedule it for sunset, so every screen must work in both.
Accent color	The one color that marks what is interactive: buttons, switches, selected tabs. You set it with .tint or the AccentColor asset.
Hard-coded color	A fixed value such as Color(red: 0.8, green: 0.8, blue: 0.8). It never adapts, so it can be right in only one appearance.

> **Rule of thumb**
>
> Reach for a semantic color first. Hard-code only brand colors, and give each one a Dark and a High Contrast variant in the asset catalog.

CODE · LIVE SWIFTUI PREVIEW

Semantic colors adapt on their own

This card never names a shade. Every color is a *role*: primary for the title, secondary for supporting text, and a material for the card itself. The last line reads the current appearance from the environment so you can see which one the preview is showing while the colors above adapt.

```swift
import SwiftUI

struct TripCard: View {
    @Environment(\.colorScheme) private var scheme
    var body: some View {
        VStack(alignment: .leading, spacing: 4) {
            Text("Lisbon Weekend")
                .font(.headline)
                .foregroundStyle(.primary)
            Text("3 days, 2 nights")
                .foregroundStyle(.secondary)
            Text(scheme == .dark ? "Dark Mode" : "Light Mode")
                .font(.caption)
                .foregroundStyle(.secondary)
        }
        .padding()
        .background(.thinMaterial)
        .clipShape(RoundedRectangle(cornerRadius: 12))
    }
}

#Preview { TripCard() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the app.
- **WHAT** **Makes SwiftUI's views, modifiers and semantic styles such as .primary and .secondary available in this file.**
- **WHERE** The first line of the file; an import covers only the file it appears in.
- **WHEN** At compile time, before any view exists.
- **WHY** Semantic colors are part of the framework, so the file needs SwiftUI before it can name a role instead of a shade.

3 `struct TripCard: View {`

- **WHO** Your struct, which SwiftUI treats as a description of a piece of UI.
- **WHAT** **Declares a card view that conforms to View, so it must provide a body.**
- **WHERE** A value type in your app module; SwiftUI creates and discards instances freely.
- **WHEN** Created whenever a parent view, here the #Preview, builds its own content.
- **WHY** Small, focused views such as a card are easy to check in both appearances on their own before you place them in a full screen.

4 `@Environment(\.colorScheme) private var scheme`

- **WHO** SwiftUI's environment, the shared set of values (appearance, text size, locale) passed down the view tree.
- **WHAT** **Reads the current color scheme, .light or .dark, into the property scheme.**
- **WHERE** The value lives in SwiftUI's environment storage; this property is a read-only window onto it.
- **WHEN** Read every time body runs; SwiftUI re-runs body when the person switches appearance.
- **WHY** You rarely need this, because semantic colors adapt by themselves. Here it only labels the appearance; branching on it to pick shades by hand is the pattern to avoid.

```
5  var body: some View {
```

WHO SwiftUI, which calls body to learn what to draw.
WHAT **Declares the computed property that returns the card's content.**
WHERE Inside TripCard; some View hides the long nested type that the modifiers build.
WHEN On first display and again whenever scheme or any other input changes.
WHY Because body is recomputed on an appearance change, every color inside it is resolved again for Light or Dark Mode automatically.

```
6  VStack(alignment: .leading, spacing: 4) {
```

WHO SwiftUI's vertical stack layout.
WHAT **Stacks the three lines top to bottom, aligned on their leading edge (the left edge in English), 4 pt apart.**
WHERE The root of body, wrapped by the modifiers further down.
WHEN Built on each body pass.
WHY A tight 4 pt gap tells the eye these lines describe one item, and a shared leading edge is easy to scan.

```
7  Text("Lisbon Weekend")
```

WHO A Text view, which draws a string with the current font and foreground style.
WHAT **Shows the card's title.**
WHERE The first child of the stack.
WHEN Built with the stack on every body pass.
WHY The title is the most important line, so the next two lines give it the strongest treatment.

```
8  .font(.headline)
```

WHO SwiftUI's font modifier with a Dynamic Type text style.
WHAT **Sets the title to the headline style: 17 pt semibold at the default text size.**
WHERE Applies to this Text only.
WHEN Resolved at render time against the person's text-size setting.
WHY Weight, not just color, creates hierarchy, and a text style keeps scaling with Dynamic Type.

```
9  .foregroundStyle(.primary)
```

WHO The foreground style modifier with the primary semantic style.
WHAT **Colors the title with the primary label color: black in Light Mode, white in Dark Mode.**
WHERE On the title Text only.
WHEN Resolved again whenever the appearance or the Increase Contrast setting changes.
WHY Primary is reserved for the most important text and always carries the strongest contrast the system offers, in every appearance.

```
10 Text("3 days, 2 nights")
```

WHO A second Text view.
WHAT **Shows a supporting detail under the title.**
WHERE The second child of the stack, 4 pt below the title.
WHEN Built on every body pass.
WHY Supporting facts sit right under the thing they describe, so proximity does the grouping for you.

```
11 .foregroundStyle(.secondary)
```

WHO The same modifier with the secondary semantic style.
WHAT **Draws the detail in the secondary label color, a softer gray tuned separately for Light and Dark Mode.**
WHERE On the detail Text only.
WHEN Resolved again on every appearance change.
WHY Secondary steps the line down in importance without picking a gray by hand. It measures roughly 3.4:1 on white, so keep must-read information in .primary.

```
12  Text(scheme == .dark ? "Dark Mode" : "Light Mode")
```

WHO A Text view fed by the environment value read above.
WHAT **Shows which appearance is active, using the ternary operator to choose the string.**
WHERE The third child of the stack.
WHEN Recomputed whenever scheme changes, so it flips the moment the appearance does.
WHY It makes the invisible visible: you can watch the colors above adapt while this line confirms the mode.

```
13  .font(.caption)
```

WHO The font modifier with the caption text style.
WHAT **Sets the label to 12 pt at the default text size.**
WHERE On the mode label only.
WHEN Resolved at render time against the text-size setting.
WHY The least important line gets the smallest style, completing a three-step hierarchy: headline, body, caption.

```
14  .foregroundStyle(.secondary)
```

WHO The foreground style modifier again.
WHAT **Colors the mode label secondary, the same role as the detail line.**
WHERE On the mode label only.
WHEN Resolved again when the appearance changes.
WHY Reusing secondary instead of inventing a third gray keeps the palette small; the smaller size alone lowers its rank.

```
16  .padding()
```

WHO The padding modifier on the VStack, inside the material background.
WHAT **Adds the system default inset, 16 pt on iPhone, around the stack.**
WHERE Wraps the whole stack, inside the background that comes next.
WHEN Applied during layout on every pass.
WHY Padding before background puts the space inside the card, so the text never touches the card's edge.

```
17  .background(.thinMaterial)
```

WHO The background modifier with a material, a translucent blurred fill supplied by iOS.
WHAT **Fills the padded card with a thin material that lightens or darkens to match the appearance.**
WHERE Behind the padded stack.
WHEN Drawn every frame and adapted when the appearance changes.
WHY Materials are semantic too: they suit light and dark screens, while a hard-coded Color.white card would glare on a black Dark Mode screen.

```
18  .clipShape(RoundedRectangle(cornerRadius: 12))
```

WHO The clipShape modifier on the padded, backgrounded card.
WHAT **Clips the card, background included, to a rectangle with 12 pt rounded corners.**
WHERE The outermost modifier, so it clips everything above it.
WHEN Applied when the view is drawn.
WHY Rounded corners echo the rounded hardware and the system's own cards; clipping last makes the material follow the curve.

```
22  #Preview { TripCard() }
```

WHO Xcode's #Preview macro, and this course's live phone.
WHAT **Renders TripCard so you can see it without launching the app.**
WHERE At file scope; preview code is for the canvas, not for app behavior.
WHEN Whenever the file changes and the canvas refreshes.
WHY Previews make it cheap to check a design in Light and Dark Mode and at several text sizes before it reaches a device.

Try it

1. Change the second .foregroundStyle(.secondary) to .primary. Is it still obvious which line is the title?
2. Replace .thinMaterial with Color.white. Picture the card on a black Dark Mode screen: what would people see?

COMPARE

Hard-coded colors versus semantic colors

Both cards look similar at a glance in Light Mode. Read the left one closely: its status line is #CCCCCC on white, a contrast of 1.61:1, far below the 4.5:1 minimum. Now picture Dark Mode: the name is hard-coded black on a black screen, 1:1, invisible. The right card names roles instead, so it works in both.

A: Before: fixed shades

```swift
import SwiftUI

struct PoorProfile: View {
    let lightGray = Color(red: 0.8, green: 0.8, blue: 0.8)

    var body: some View {
        VStack(alignment: .leading, spacing: 4) {
            Text("Ada Lopez")
                .font(.headline)
                .foregroundColor(.black)
            Text("Last active 2 hours ago")
                .foregroundColor(lightGray)
        }
        .padding()
    }
}

#Preview { PoorProfile() }
```

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares PoorProfile.
- WHAT — **Brings in SwiftUI for Text, VStack and Color.**
- WHERE — The first line of this file only.
- WHEN — At compile time, before PoorProfile is type-checked.
- WHY — This flawed design compiles without a single warning; color mistakes are invisible to the compiler, which is why you test both appearances yourself.

3 `struct PoorProfile: View {`

- WHO — Your struct, a SwiftUI view.
- WHAT — **Declares the before version of a profile header.**
- WHERE — In the app module, as a value type.
- WHEN — Created each time its parent builds its content.
- WHY — Keeping the flawed and fixed versions as separate views lets you preview them side by side.

```
4  let lightGray = Color(red: 0.8, green: 0.8, blue: 0.8)
```

WHO A stored constant on the struct that holds one fixed Color.
WHAT **Defines a light gray with 80 percent in every channel, which is #CCCCCC.**
WHERE On each PoorProfile value; it never changes.
WHEN Set when the struct is created.
WHY This is the first mistake: a fixed shade cannot adapt, and on white it measures 1.61:1, far under the 4.5:1 that text needs.

```
6  var body: some View {
```

WHO SwiftUI, which reads body to get the content.
WHAT **Returns the header's view tree.**
WHERE Inside PoorProfile, below the lightGray constant.
WHEN On first display and whenever the appearance or other inputs change.
WHY Body does re-run on an appearance change, but nothing in it can respond, because every color is fixed.

```
7  VStack(alignment: .leading, spacing: 4) {
```

WHO The VStack, which stacks the name above the status line.
WHAT **Stacks the name above the status line, left-aligned, 4 pt apart.**
WHERE The root of body.
WHEN On each body pass.
WHY The layout is identical to the fixed version, which isolates the color problem.

```
8  Text("Ada Lopez")
```

WHO A Text view drawing the name as the first child of the VStack.
WHAT **Shows the person's name.**
WHERE The first child of the stack.
WHEN On each body pass.
WHY The name is the key information on this header, so losing it is the worst possible failure.

```
9  .font(.headline)
```

WHO The font modifier on the name Text.
WHAT **Uses the 17 pt semibold headline style.**
WHERE On the name only.
WHEN Resolved at render time.
WHY The type hierarchy is fine; the color on the next line is what breaks the design.

```
10 .foregroundColor(.black)
```

WHO The older foregroundColor modifier with a fixed color.
WHAT **Paints the name pure black no matter which appearance is active.**
WHERE On the name only.
WHEN Applied at render time and never resolved again for Dark Mode.
WHY In Dark Mode the screen turns black, so black on black is 1:1 and the name disappears. Apple has renamed foregroundColor to foregroundStyle, the modern spelling.

```
11 Text("Last active 2 hours ago")
```

WHO A Text view drawing the status line as the second child of the VStack.
WHAT **Shows a status line under the name.**
WHERE The second child of the stack.
WHEN On each body pass.
WHY Supporting information should be quieter than the name, but it still has to be readable.

12 `.foregroundColor(lightGray)`

WHO foregroundColor again, with the stored constant.
WHAT **Paints the status line #CCCCCC.**
WHERE On the status line only.
WHEN Applied at render time.
WHY Pale gray looks subtle on a bright monitor, but at 1.61:1 people with low vision, or anyone outdoors in sunlight, cannot read it.

14 `.padding()`

WHO The padding modifier on the VStack.
WHAT **Adds 16 pt of space around the stack.**
WHERE Wraps the VStack, as the last modifier in body.
WHEN During layout, after both lines have measured themselves.
WHY Spacing matches the fixed version exactly, so the only difference between the phones is color.

18 `#Preview { PoorProfile() }`

WHO The #Preview macro, read by the canvas and by this page's phone to show PoorProfile.
WHAT **Renders the before version.**
WHERE At file scope, after the closing brace of PoorProfile.
WHEN When the canvas refreshes.
WHY Switch the preview to Dark Mode and watch the name vanish; a test that takes seconds catches this whole class of bug.

B: After: semantic roles

```swift
import SwiftUI

struct BetterProfile: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 4) {
            Text("Ada Lopez")
                .font(.headline)
                .foregroundStyle(.primary)
            Text("Last active 2 hours ago")
                .font(.subheadline)
                .foregroundStyle(.secondary)
        }
        .padding()
    }
}

#Preview { BetterProfile() }
```

1 `import SwiftUI`

WHO The Swift compiler, while it builds the file that declares BetterProfile.
WHAT **Brings in SwiftUI, including the semantic styles .primary and .secondary.**
WHERE The first line of this file only.
WHEN At compile time, before BetterProfile is type-checked.
WHY Semantic styles come from the framework, so this file needs no color constants at all.

```
3  struct BetterProfile: View {
```

WHO Your struct, a SwiftUI view.
WHAT **Declares the fixed version of the same header.**
WHERE In the app module, as a value type.
WHEN Created each time its parent builds its content.
WHY Same content and layout as the left, so any difference you see comes from color alone.

```
4  var body: some View {
```

WHO SwiftUI, which reads body to get the content.
WHAT **Returns the header's view tree.**
WHERE Inside BetterProfile, its only member.
WHEN On first display and on every appearance change.
WHY Each re-run resolves the semantic styles below for the current appearance, so the colors follow the person's settings.

```
5  VStack(alignment: .leading, spacing: 4) {
```

WHO The VStack, which stacks the name above the status line.
WHAT **Stacks the name above the status, left-aligned, 4 pt apart.**
WHERE The root of body.
WHEN On each body pass.
WHY Identical layout keeps the comparison fair.

```
6  Text("Ada Lopez")
```

WHO A Text view drawing the name as the first child of the VStack.
WHAT **Shows the person's name.**
WHERE The first child of the stack.
WHEN On each body pass.
WHY The most important line gets the strongest style on the next two lines.

```
7  .font(.headline)
```

WHO The font modifier on the name Text.
WHAT **Uses the 17 pt semibold headline style.**
WHERE On the name only.
WHEN Resolved at render time against the text-size setting.
WHY Weight separates the name from the status even before color is considered.

```
8  .foregroundStyle(.primary)
```

WHO The foreground style modifier with the primary role.
WHAT **Colors the name with the primary label color: black in Light Mode, white in Dark Mode, stronger still with Increase Contrast.**
WHERE On the name only.
WHEN Resolved again on every appearance change.
WHY The role guarantees full contrast against the system background in every appearance, exactly what the key line needs.

```
9  Text("Last active 2 hours ago")
```

WHO A Text view drawing the status line as the second child of the VStack.
WHAT **Shows the status line.**
WHERE The second child of the stack.
WHEN On each body pass.
WHY The same supporting information as before, now styled by role.

```
10  .font(.subheadline)
```

WHO The font modifier on the status Text.
WHAT **Sets the status to the 15 pt subheadline style.**
WHERE On the status line only.
WHEN Resolved at render time.
WHY A smaller size is a second cue of lower importance, so color does not have to do all the work.

```
11  .foregroundStyle(.secondary)
```

WHO The foreground style modifier with the secondary role.
WHAT **Uses the secondary label color, a gray tuned separately for each appearance.**
WHERE On the status line only.
WHEN Resolved again on every appearance change.
WHY It is quieter than primary yet far more readable than #CCCCCC, and it adapts to Dark Mode and Increase Contrast without any code.

```
13  .padding()
```

WHO The padding modifier on the VStack.
WHAT **Adds 16 pt of space around the stack.**
WHERE Wraps the VStack, as the last modifier in body.
WHEN During layout, after both lines have measured themselves.
WHY Same spacing as the left, so color stays the only variable.

```
17  #Preview { BetterProfile() }
```

WHO The #Preview macro, read by the canvas and by this page's phone to show BetterProfile.
WHAT **Renders the fixed version.**
WHERE At file scope, after the closing brace of BetterProfile.
WHEN When the canvas refreshes.
WHY Preview it in both appearances: both lines stay readable in each.

> **Takeaway**
>
> A hard-coded color is a promise about one appearance. Semantic styles such as `.primary` and `.secondary`, plus the default background, let iOS keep text readable in Light Mode, Dark Mode and Increase Contrast. Check every screen in both appearances before you call it done.

CODE · LIVE SWIFTUI PREVIEW

One accent color marks what you can tap

The **accent color**, also called the tint, tells people what is interactive. Set it once near the top of a view tree and every standard control below picks it up: the switch, the filled part of the slider and the prominent button all turn indigo here. Keep the tint for interactive things only, so color keeps meaning “you can tap this.”

```swift
import SwiftUI

struct AlertsSettings: View {
    @State private var reminders = true
    @State private var volume = 60.0

    var body: some View {
        Form {
            Toggle("Daily reminders", isOn: $reminders)
            Slider(value: $volume, in: 0...100)
                .accessibilityLabel("Reminder volume")
            Button("Save Changes") { }
                .buttonStyle(.borderedProminent)
        }
        .tint(.indigo)
    }
}

#Preview { AlertsSettings() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares AlertsSettings.
- **WHAT** **Brings in SwiftUI's Form, Toggle, Slider, Button and the tint modifier.**
- **WHERE** The first line of this file.
- **WHEN** At compile time, before AlertsSettings is type-checked.
- **WHY** Standard controls already know how to use the accent color, so importing them is most of the work.

3 `struct AlertsSettings: View {`

- **WHO** Your struct, a SwiftUI view.
- **WHAT** **Declares a small settings screen for reminders.**
- **WHERE** In the app module.
- **WHEN** Created when the preview, or a parent screen, shows it.
- **WHY** Settings screens are full of controls, which makes them the clearest place to see the tint at work.

4 `@State private var reminders = true`

- **WHO** SwiftUI's @State storage, which owns reminders on behalf of AlertsSettings.
- **WHAT** **Holds whether daily reminders are on, starting at true.**
- **WHERE** Owned by SwiftUI on behalf of this view, outside the struct value.
- **WHEN** Created once when the view first appears and kept across body recomputations.
- **WHY** The switch needs real state to flip, and starting in the on position shows the tint immediately.

5 `@State private var volume = 60.0`

- **WHO** SwiftUI's @State storage again.
- **WHAT** **Holds the slider position as a Double, starting at 60.**
- **WHERE** Owned by SwiftUI for this view.
- **WHEN** Created once and updated while the slider is dragged.
- **WHY** A mid-range start shows both parts of the track: the tinted filled part and the gray empty part.

```
7  var body: some View {
```

WHO — SwiftUI, which reads body to build the screen.
WHAT — **Returns the form and its controls.**
WHERE — Inside AlertsSettings, below the two state properties.
WHEN — On first display and whenever reminders or volume change.
WHY — Every state change re-runs body, and each control redraws in the tint color that applies to it.

```
8  Form {
```

WHO — SwiftUI's Form container, which lays out the three rows.
WHAT — **Lays the controls out as a grouped settings list with standard margins and row heights.**
WHERE — The root of body.
WHEN — On each body pass.
WHY — Form gives you the iOS Settings look, inset grouped rows at least 44 pt tall with 16 pt margins, so you only decide the color.

```
9  Toggle("Daily reminders", isOn: $reminders)
```

WHO — A Toggle control bound to reminders.
WHAT — **Shows a labeled switch; each flip writes true or false back through the binding.**
WHERE — The first row of the form.
WHEN — Drawn on each pass; the binding updates on every tap.
WHY — The switch's on color comes from the tint, so the active state of an interactive control is drawn in the accent color.

```
10 Slider(value: $volume, in: 0...100)
```

WHO — A Slider control bound to volume.
WHAT — **Lets the person pick a value from 0 to 100; the filled part of the track uses the tint.**
WHERE — The second row of the form.
WHEN — Writes to volume continuously while it is dragged.
WHY — The same color on switch, slider and button teaches one rule: indigo means you can interact with it.

```
11 .accessibilityLabel("Reminder volume")
```

WHO — VoiceOver and the other assistive technologies.
WHAT — **Gives the slider, which has no visible label, a spoken name.**
WHERE — Attached to the Slider.
WHEN — Read aloud when VoiceOver focuses the slider.
WHY — Without a name VoiceOver can announce only the slider's value, and the person has to guess what it controls.

```
12 Button("Save Changes") { }
```

WHO — A Button with an empty action closure.
WHAT — **Shows a Save Changes button; the empty closure is where saving code would go.**
WHERE — The third row of the form.
WHEN — The closure runs on each tap.
WHY — The label is a verb phrase that says exactly what will happen when it is tapped.

```
13 .buttonStyle(.borderedProminent)
```

WHO — The button style modifier.
WHAT — **Draws the button as a filled shape in the tint color with a contrasting label.**
WHERE — On this button only.
WHEN — Applied at render time.
WHY — The prominent style marks the primary action. Use one prominent button per screen so the most important action is obvious.

```
15  .tint(.indigo)
```

WHO The tint modifier, which sets the accent color for everything inside it.
WHAT **Makes indigo the accent for the switch, the slider and the button in this form.**
WHERE On the Form, so it flows down to every child control.
WHEN Read by each control as it renders.
WHY Set the accent once, high in the tree, so it stays consistent. In a real app the AccentColor asset sets it for the whole app.

```
19  #Preview { AlertsSettings() }
```

WHO The #Preview macro, read by the canvas and by this page's phone to show AlertsSettings.
WHAT **Renders the settings screen in the phone.**
WHERE At file scope, after the closing brace of AlertsSettings.
WHEN When the canvas refreshes.
WHY Flip the switch and drag the slider in the preview to see the tint on both states.

> **Try it**
>
> 1. Change .indigo to .orange. Which three controls change color?
> 2. Move .tint(.indigo) from the Form onto the Button only. What goes back to the default blue?

CONCEPT

Contrast, measured

Contrast ratio compares how bright two colors are, using their *relative luminance*, a measure of perceived brightness from 0 (black) to 1 (white): (lighter + 0.05) ÷ (darker + 0.05). It runs from 1:1, identical colors, to 21:1, black on white. The Web Content Accessibility Guidelines (WCAG) set the minimums, and Apple's Human Interface Guidelines ask for at least 4.5:1 for small text and 3:1 for larger or bold text.

4.5:1	Normal text at level AA: body text, captions, labels.
3:1	Large text (18 pt and up, or 14 pt and up when bold), plus icons and control edges people need to see to use them.
7:1	The stricter AAA level for normal text (4.5:1 for large text). Worth aiming for on screens people read for a long time.
#767676	The lightest gray that passes 4.5:1 on white, at 4.54:1. #777777 measures 4.48:1 and fails.

> **Even system blue needs care**
>
> #007AFF on white is 4.02:1: fine for bold labels and icons, short of 4.5:1 for small regular text. When people turn on Increase Contrast, system colors switch to stronger variants automatically; your custom colors adapt only if you supply a High Contrast variant.

COMPARE

Color alone versus color plus shape and words

About 1 in 12 men and 1 in 200 women have a color vision deficiency, most often red-green. On the left, flight status lives only in green and red text, and green on white measures just 2.22:1. On the right, each status has a distinct symbol and a word, and color only reinforces them.

A: Before: color is the message

```swift
import SwiftUI

struct PoorStatus: View {
    var body: some View {
        List {
            Text("Flight 214")
                .foregroundStyle(.green)
            Text("Flight 377")
                .foregroundStyle(.red)
        }
    }
}

#Preview { PoorStatus() }
```

1 `import SwiftUI`

WHO	The Swift compiler, while it builds the file that declares PoorStatus.
WHAT	**Brings in SwiftUI for List and Text.**
WHERE	The first line of this file.
WHEN	At compile time, before PoorStatus is type-checked.
WHY	Nothing here is wrong as code; the problem is purely one of design.

3 `struct PoorStatus: View {`

WHO	Your struct, a SwiftUI view.
WHAT	**Declares the before version of a flight status list.**
WHERE	In the app module.
WHEN	Created when the preview shows it.
WHY	Status lists are the most common place where apps lean on color alone.

4 `var body: some View {`

WHO	SwiftUI, which reads body for the content.
WHAT	**Returns the List that holds the two flight rows.**
WHERE	Inside PoorStatus, its only member.
WHEN	On first display and whenever inputs change.
WHY	The whole message of this screen has to come out of these few lines, so every visual choice matters.

5 `List {`

WHO	SwiftUI's List, which draws the two Text views as system-styled rows.
WHAT	**Shows the flights as rows in a scrolling, system-styled list.**
WHERE	The root of body.
WHEN	On each body pass.
WHY	The list supplies standard row height and margins; the flaw is only in how status is shown.

6 `Text("Flight 214")`

WHO	A Text view drawing the first flight's name as its own row.
WHAT	**Shows the first flight's name.**
WHERE	The first row of the List.
WHEN	On each body pass.
WHY	Notice that the status is not written anywhere: it is supposed to come from color alone.

7 `.foregroundStyle(.green)`

WHO The foreground style modifier with system green.
WHAT **Paints the name green, meant to say on time.**
WHERE On the first row's text.
WHEN Resolved at render time.
WHY Green text on white is 2.22:1, too faint to read, and a person with red-green color blindness cannot tell it from the red row.

8 `Text("Flight 377")`

WHO A Text view drawing the second flight's name as its own row.
WHAT **Shows the second flight's name.**
WHERE The second row of the List, under Flight 214.
WHEN On each body pass.
WHY Again the status, the information people actually need, is missing from the words.

9 `.foregroundStyle(.red)`

WHO The foreground style modifier with system red.
WHAT **Paints the name red, meant to say canceled.**
WHERE On the second row's text.
WHEN Resolved at render time.
WHY Red alone carries the most urgent message on the screen; anyone who cannot perceive red misses a canceled flight.

14 `#Preview { PoorStatus() }`

WHO The #Preview macro, read by the canvas and by this page's phone to show PoorStatus.
WHAT **Renders the before version.**
WHERE At file scope, after the closing brace of PoorStatus.
WHEN When the canvas refreshes.
WHY Imagine the preview in grayscale: the two rows become identical, which is exactly what some people see.

B: After: symbol, word, then color

```swift
import SwiftUI

struct BetterStatus: View {
    var body: some View {
        List {
            Label("Flight 214: On time",
                  systemImage: "checkmark.circle")
            Label("Flight 377: Canceled",
                  systemImage: "xmark.circle.fill")
                .foregroundStyle(.red)
                .bold()
        }
    }
}

#Preview { BetterStatus() }
```

```
1  import SwiftUI
```

WHO The Swift compiler, while it builds the file that declares BetterStatus.
WHAT **Brings in SwiftUI for List and Label.**
WHERE The first line of this file.
WHEN At compile time, before BetterStatus is type-checked.
WHY Label, which pairs text with an SF Symbol, is the standard building block for status rows.

```
3  struct BetterStatus: View {
```

WHO Your struct, a SwiftUI view.
WHAT **Declares the fixed version of the status list.**
WHERE In the app module.
WHEN Created when the preview shows it.
WHY Same two flights, so you can compare the phones directly.

```
4  var body: some View {
```

WHO SwiftUI, which reads body for the content.
WHAT **Returns the List that holds the two labeled rows.**
WHERE Inside BetterStatus, its only member.
WHEN On first display and whenever inputs change.
WHY The content now carries the status in three independent ways.

```
5  List {
```

WHO SwiftUI's List, which draws the two Labels as system-styled rows.
WHAT **Shows the two rows in a system-styled list.**
WHERE The root of body.
WHEN On each body pass.
WHY Identical container to the left side.

```
6  Label("Flight 214: On time",
```

WHO A Label view, which pairs a title with an SF Symbol.
WHAT **Starts the first row; its title states the status in words.**
WHERE The first row; the argument list continues on the next line.
WHEN On each body pass.
WHY Words carry the meaning for everyone, including VoiceOver users, who hear the full sentence.

```
7  systemImage: "checkmark.circle")
```

WHO The Label's systemImage argument.
WHAT **Puts the checkmark circle symbol in front of the text.**
WHERE Completes the first Label.
WHEN Resolved when the row draws.
WHY A distinct shape is a second, color-free cue. The normal state stays neutral in color, so the exception stands out.

```
8  Label("Flight 377: Canceled",
```

WHO A second Label view.
WHAT **Starts the second row with its status in words.**
WHERE The second row; continues on the next line.
WHEN On each body pass.
WHY The most urgent fact on the screen is now written out, not implied.

```
9   systemImage: "xmark.circle.fill")
```

WHO The Label's systemImage argument.
WHAT **Uses a filled x-in-a-circle symbol.**
WHERE Completes the second Label.
WHEN Resolved when the row draws.
WHY An x shape differs from a checkmark at a glance, and the filled version adds visual weight to the problem row.

```
10  .foregroundStyle(.red)
```

WHO The foreground style modifier with system red.
WHAT **Colors this row's symbol and text red.**
WHERE On the second Label only.
WHEN Resolved at render time and adapted for Dark Mode.
WHY Red now reinforces a message already given by shape and words, which is the right job for color.

```
11  .bold()
```

WHO The bold modifier on the second Label.
WHAT **Makes the canceled row's text bold.**
WHERE On the second Label only.
WHEN Applied at render time.
WHY Red on white measures 3.55:1. Bold text at this size counts as large text, whose minimum is 3:1, so the row passes; in regular weight it would need 4.5:1.

```
16  #Preview { BetterStatus() }
```

WHO The #Preview macro, read by the canvas and by this page's phone to show BetterStatus.
WHAT **Renders the fixed version.**
WHERE At file scope, after the closing brace of BetterStatus.
WHEN When the canvas refreshes.
WHY In grayscale this version still reads correctly, which is the test it has to pass.

> **Takeaway**
>
> Color may reinforce meaning but must never carry it alone. Pair it with a symbol whose *shape* differs and with words, and keep colored text readable: make it bold or large, or keep the words in `.primary` and color only the symbol.

CODE · READ-ALONG

Custom colors that still adapt

A presence dot, the green circle on a contact photo, is a classic color-only signal. This version reads a brand color from the asset catalog, uses a UIKit semantic color, and responds to two accessibility settings: Increase Contrast and Differentiate Without Color. It uses APIs the live phone does not model, so it is shown and type-checked rather than run.

```swift
import SwiftUI

struct PresenceDot: View {
    let isOnline: Bool
    @Environment(\.colorSchemeContrast) private var contrast
    @Environment(\.accessibilityDifferentiateWithoutColor)
    private var avoidColorOnly
    var body: some View {
        ZStack {
            Circle().fill(isOnline ? Color("OnlineGreen") : .gray)
            if contrast == .increased {
                Circle().stroke(Color(.label), lineWidth: 2)
            }
            if avoidColorOnly {
                Image(systemName: isOnline ? "checkmark" : "minus")
                    .font(.system(size: 9, weight: .bold))
            }
        }
        .frame(width: 16, height: 16)
        .accessibilityLabel(isOnline ? "Online" : "Offline")
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares PresenceDot.
- **WHAT** **Brings in SwiftUI; on iOS that also exposes UIKit's semantic color names to Color(_:).**
- **WHERE** The first line of this file.
- **WHEN** At compile time, before PresenceDot is type-checked.
- **WHY** One import gives you both naming systems: SwiftUI roles like .primary and UIKit roles like .label.

3 `struct PresenceDot: View {`

- **WHO** Your struct, a SwiftUI view.
- **WHAT** **Declares a small online indicator, the dot you see on an avatar.**
- **WHERE** In the app module, reused wherever a contact appears.
- **WHEN** Created by each parent that shows a contact.
- **WHY** Building the accessible version once, as a reusable view, fixes the problem everywhere it is used.

4 `let isOnline: Bool`

- **WHO** A stored property supplied by the parent view.
- **WHAT** **Says whether the person is online.**
- **WHERE** On each PresenceDot value.
- **WHEN** Set when the parent creates the dot.
- **WHY** The dot only displays state; the parent owns the truth, which keeps this view simple and testable.

5 `@Environment(\.colorSchemeContrast) private var contrast`

- **WHO** SwiftUI's environment, reflecting the Increase Contrast setting.
- **WHAT** **Reads .standard or .increased into contrast.**
- **WHERE** Environment storage outside the struct.
- **WHEN** Read on every body pass; it changes when the person toggles Increase Contrast in Settings.
- **WHY** People who turn on Increase Contrast want stronger edges, and this value lets custom drawing respond to them.

```
6  @Environment(\.accessibilityDifferentiateWithoutColor)
```

WHO SwiftUI's environment, reflecting the Differentiate Without Color setting in Accessibility settings.
WHAT **Begins a property declaration that reads that setting.**
WHERE Environment storage outside the struct.
WHEN Read on each body pass.
WHY The setting exists for people who cannot rely on color. Honoring it is the minimum; a shape that is always visible is better still.

```
7  private var avoidColorOnly
```

WHO The property that the line above annotates.
WHAT **Names the value; true means the person asked for cues other than color.**
WHERE A private property of PresenceDot.
WHEN Read inside body, each time the second if is evaluated.
WHY A descriptive name makes the intent obvious in the body below.

```
8  var body: some View {
```

WHO SwiftUI, which reads body for the content.
WHAT **Returns the layered dot.**
WHERE Inside PresenceDot, below the three properties.
WHEN On first display and whenever isOnline or either setting changes.
WHY Because both settings live in the environment, toggling them in Settings re-runs this body immediately.

```
9  ZStack {
```

WHO The ZStack, which centers the fill, the ring and the symbol on each other.
WHAT **Layers the fill, an optional ring and an optional symbol on top of each other.**
WHERE The root of body.
WHEN On each body pass.
WHY Layering lets extra cues appear without changing the dot's size or position.

```
10 Circle().fill(isOnline ? Color("OnlineGreen") : .gray)
```

WHO A Circle shape filled with a color chosen by the ternary operator.
WHAT **Fills with the OnlineGreen color set when online and system gray otherwise.**
WHERE The bottom layer of the ZStack, under the optional ring and symbol.
WHEN Resolved on each pass and again on appearance changes.
WHY Color("OnlineGreen") loads a named color set from the asset catalog, where you can define Any, Dark and High Contrast variants, so even a brand color adapts.

```
11 if contrast == .increased {
```

WHO A conditional inside the view builder.
WHAT **Adds the next layer only when Increase Contrast is on.**
WHERE The middle layer of the ZStack.
WHEN Evaluated on each body pass.
WHY Stronger edges help the people who asked for them without cluttering the default look.

```
12 Circle().stroke(Color(.label), lineWidth: 2)
```

WHO A Circle outline drawn in UIKit's label color.
WHAT **Draws a 2 pt ring in the same color role as primary text.**
WHERE Over the fill, as the middle layer of the ZStack.
WHEN Only while Increase Contrast is on.
WHY UIKit semantic colors such as .label, .secondarySystemBackground and .systemGray3 reach SwiftUI through Color(_:) for roles SwiftUI has no short name for.

```
14 if avoidColorOnly {
```

WHO A second if inside the view builder, reading avoidColorOnly.
WHAT **Adds a symbol only when Differentiate Without Color is on.**
WHERE The top layer of the ZStack.
WHEN Evaluated on each body pass.
WHY This setting is a direct request for non-color cues, so the view answers it with a shape.

```
15 Image(systemName: isOnline ? "checkmark" : "minus")
```

WHO An SF Symbol chosen by the ternary operator.
WHAT **Draws a checkmark when online and a minus sign when offline.**
WHERE Centered on top of the dot.
WHEN Only while the setting is on.
WHY Shape now carries the status, so it reads correctly in grayscale and for every kind of color blindness.

```
16 .font(.system(size: 9, weight: .bold))
```

WHO The font modifier with a fixed system size.
WHAT **Sizes the symbol at 9 pt bold so it fits inside the 16 pt dot.**
WHERE On the symbol only.
WHEN Applied at render time.
WHY A fixed size is acceptable here because the dot is a fixed-size graphic; text that people read should always use text styles.

```
19 .frame(width: 16, height: 16)
```

WHO The frame modifier on the ZStack.
WHAT **Sizes the whole dot to 16 by 16 pt.**
WHERE On the ZStack, after all layers.
WHEN During layout, after all three layers have been built.
WHY One fixed frame keeps the dot identical in every state, so toggling settings never shifts the avatar layout.

```
20 .accessibilityLabel(isOnline ? "Online" : "Offline")
```

WHO VoiceOver and the other assistive technologies.
WHAT **Gives the dot a spoken name, Online or Offline.**
WHERE On the whole dot.
WHEN Read when VoiceOver focuses the dot.
WHY VoiceOver users get the status in words, which no color or shape alone can give them.

> **Try it**
>
> 1. Which line would you delete if the dot should always show its symbol, not only when the setting is on? Why might that be the better design?

INTERACTIVE

Contrast checker

Pick a text color and a background, or start from a preset of real iOS system colors. The ratio is computed by the Swift `contrastRatio` function from this course, compiled to WebAssembly and running in your browser. Watch the pass and fail badges, compare the Light and Dark Mode previews, then press *Suggest a passing color* to find the nearest shade of the same hue that passes.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. A screen hard-codes `Color.black` for its body text and leaves the background as the system default. What happens when the person turns on Dark Mode?
   A. SwiftUI inverts the text to white automatically
   B. The text stays black on a black background and becomes unreadable
   C. Nothing changes, because Dark Mode only affects Apple's own apps
   D. The app refuses to switch to Dark Mode

2. Which pair passes WCAG AA for 15 pt regular text?
   A. #777777 on white (4.48:1)
   B. #007AFF on white (4.02:1)
   C. #767676 on white (4.54:1)
   D. White on #34C759 (2.22:1)

3. A delivery app shows each order's status as a green, yellow or red dot. What is the best fix?
   A. Make the dots larger so the colors are easier to see
   B. Use brighter, more saturated shades of green, yellow and red
   C. Add a legend at the top of the screen that explains each color
   D. Give each status a distinct symbol and a word

4. Where should an app's accent (tint) color appear?
   A. On interactive elements: buttons, switches and selected tabs
   B. On every heading, so the brand color shows on every screen
   C. On body text, so long paragraphs feel livelier and more on-brand
   D. Only inside the app icon, never in the interface itself

5. A bold 17 pt label in system red (#FF3B30) on white measures 3.55:1. Is that acceptable?
   A. No, every piece of text needs at least 4.5:1 regardless of size or weight
   B. Yes, bold text at this size counts as large text, which needs 3:1
   C. Yes, because system colors are always accessible on white
   D. No, red text is never permitted for labels that people have to read

Answers: Color, Dark Mode, and Contrast

1. **B. The text stays black on a black background and becomes unreadable** The default background is semantic, so it turns black, but a hard-coded color never changes: black on black is 1:1. SwiftUI does not invert fixed colors for you; only semantic styles such as `.primary` adapt. (Smart Invert is a separate accessibility feature, not Dark Mode.)
2. **C. #767676 on white (4.54:1)** 15 pt regular is normal-size text, so it needs 4.5:1, and only #767676 clears it. #777777 looks almost identical but measures 4.48:1, which is why you measure instead of eyeballing. System blue at 4.02:1 would pass only the 3:1 bar for large or bold text.
3. **D. Give each status a distinct symbol and a word** Shape and words work for everyone, including people with color blindness and VoiceOver users. A legend is tempting, but it still asks people to match colors, which is exactly what some of them cannot do. Bigger or brighter dots are still color-only.
4. **A. On interactive elements: buttons, switches and selected tabs** The tint signals “you can interact with this.” Spending it on headings or body text teaches people to tap things that do nothing, and colored body text usually loses contrast as well.
5. **B. Yes, bold text at this size counts as large text, which needs 3:1** Large text means 18 pt and up, or 14 pt and up when bold, and its minimum is 3:1, so a bold 17 pt label passes. The same label in regular weight would need 4.5:1. Being a system color is no guarantee: system blue on white is only 4.02:1.

Layout, Spacing, and Touch Targets

CONCEPT

Safe areas and margins

A **safe area** is the part of the screen nothing covers: below the status bar and Dynamic Island, above the home indicator (34 pt on Face ID iPhones). The top inset changes from model to model, so never hard-code it; SwiftUI keeps content inside the safe area for you.

Layout margins are the standard gap between content and the screen edge: 16 pt on iPhone in portrait, 20 pt on iPad and some wider layouts. Lists, navigation titles and a bare `.padding()` all line up on that margin.

Point (pt) — Apple's layout unit. One point is 2 or 3 pixels depending on the screen, so a 44 pt button is about the same physical size on every iPhone.

Interactive figure in the HTML version: design layout grid.

CODE · LIVE SWIFTUI PREVIEW

Backgrounds bleed, content stays safe

Backgrounds may run edge to edge; content may not. Here the indigo color ignores the safe area and fills the whole screen, while the title and the button stay inside it, clear of the Dynamic Island and the home indicator. A spacer pushes the only action to the bottom, where a thumb reaches it easily.

```swift
import SwiftUI

struct WelcomeView: View {
    var body: some View {
        ZStack {
            Color.indigo.ignoresSafeArea()
            VStack(spacing: 16) {
                Text("Welcome")
                    .font(.largeTitle)
                    .foregroundStyle(.white)
                Spacer()
                Button("Get Started") { }
                    .buttonStyle(.borderedProminent)
                    .tint(.white)
                    .foregroundStyle(.indigo)
            }
            .padding()
        }
    }
}

#Preview { WelcomeView() }
```

Line by line

```
1  import SwiftUI
```

WHO The Swift compiler, while it builds the file that declares WelcomeView.
WHAT **Brings in SwiftUI's stacks, Color and the safe-area modifiers.**
WHERE The first line of this file.
WHEN At compile time, before WelcomeView is type-checked.
WHY Safe-area handling is built into SwiftUI's layout system, so you get it by importing the framework.

```
3  struct WelcomeView: View {
```

WHO Your struct, a SwiftUI view.
WHAT **Declares a full-screen welcome page, the kind shown on first launch.**
WHERE In the app module.
WHEN Created when the preview, or the app's first screen, shows it.
WHY Full-bleed screens are where safe-area mistakes are most visible, so they are the best place to learn the rule.

```
4  var body: some View {
```

WHO SwiftUI, which reads body for the content.
WHAT **Returns the layered screen.**
WHERE Inside WelcomeView, its only member.
WHEN On first display and on rotation or size changes.
WHY When the safe area changes, for example on rotation, SwiftUI re-runs layout and both layers adjust on their own.

```
5  ZStack {
```

WHO The ZStack, which layers the indigo color behind the VStack.
WHAT **Layers the background color behind the content.**
WHERE The root of body.
WHEN On each body pass.
WHY Separating background from content lets each follow a different safe-area rule.

```
6  Color.indigo.ignoresSafeArea()
```

WHO A Color view plus the ignoresSafeArea modifier.
WHAT **Fills the entire screen with indigo, including the areas under the status bar, the Dynamic Island and the home indicator.**
WHERE The bottom layer of the ZStack.
WHEN Laid out on each pass, using the insets iOS reports for the current device.
WHY Backgrounds should reach every edge or you get white strips at the top and bottom. Only decoration may ignore the safe area, never text or controls.

```
7  VStack(spacing: 16) {
```

WHO The VStack, which stacks the title, the spacer and the button inside the safe area.
WHAT **Stacks the title, a spacer and the button.**
WHERE The top layer; unlike the color, it stays inside the safe area.
WHEN On each body pass.
WHY Text and controls must never sit under the Dynamic Island or the home indicator, where they are hidden or awkward to tap.

```
8  Text("Welcome")
```

WHO A Text view drawing the title at the top of the VStack.
WHAT **Shows the screen's title.**
WHERE At the top of the stack, just below the top safe-area edge.
WHEN On each body pass.
WHY The safe area guarantees the title starts below the status bar on every model, without you knowing any model's numbers.

```
9  .font(.largeTitle)
```

WHO The font modifier with the largest text style.
WHAT **Sets the title to 34 pt at the default text size.**
WHERE On the title only.
WHEN Resolved at render time against the text-size setting.
WHY One large title states what the screen is about, and the text style keeps it scaling with Dynamic Type.

```
10 .foregroundStyle(.white)
```

WHO The foreground style modifier.
WHAT **Colors the title white.**
WHERE On the title only.
WHEN Applied at render time.
WHY White on this indigo measures 5.65:1, comfortably above the 3:1 that large text needs.

```
11 Spacer()
```

WHO A Spacer, which expands to take free space.
WHAT **Takes all leftover height, pushing the title up and the button down.**
WHERE Between the title and the button.
WHEN During layout on every pass.
WHY It puts the primary action in the bottom of the screen, the easiest area to reach with a thumb.

```
12 Button("Get Started") { }
```

WHO A Button with an empty action closure.
WHAT **Shows the screen's only action.**
WHERE At the bottom of the stack, just above the bottom safe-area edge.
WHEN The closure runs on each tap.
WHY One clear primary action, labeled with a verb, is easy to find and easy to hit.

```
13 .buttonStyle(.borderedProminent)
```

WHO The button style modifier.
WHAT **Gives the button a filled shape.**
WHERE On this button only.
WHEN Applied at render time.
WHY A filled shape shows exactly where to tap and gives a hit area larger than the text alone.

```
14 .tint(.white)
```

WHO The tint modifier, scoped to this button.
WHAT **Fills the prominent button with white instead of the default accent.**
WHERE On this button only.
WHEN Read when the button renders.
WHY A white button is the brightest thing on an indigo screen, so the eye lands on the primary action first.

```
15  .foregroundStyle(.indigo)
```

WHO The foreground style modifier, scoped to this button.
WHAT **Colors the button's label indigo.**
WHERE On this button only.
WHEN Applied at render time.
WHY Indigo on white is the same 5.65:1 pair as the title, inverted, so the label stays readable.

```
17  .padding()
```

WHO The padding modifier on the VStack.
WHAT **Insets the content 16 pt from the safe-area edges.**
WHERE Around the whole stack.
WHEN During layout, after the safe area has been applied to the VStack.
WHY The safe area keeps content off the hardware; the margin keeps it off the edges, so nothing touches the frame of the screen.

```
22  #Preview { WelcomeView() }
```

WHO The #Preview macro, read by the canvas and by this page's phone to show WelcomeView.
WHAT **Renders the welcome screen in the phone.**
WHERE At file scope, after the closing brace of WelcomeView.
WHEN When the canvas refreshes.
WHY Look at the top and bottom edges: color reaches them, text and button do not.

> **Try it**
>
> 1. Delete .ignoresSafeArea(). What appears at the top and bottom of the phone?
> 2. Delete Spacer(). Where does the button go, and is it still easy to reach with one thumb?

CONCEPT

Spacing rhythm, alignment and grouping

Calm layouts follow a **spacing rhythm**: a short scale of values used everywhere. Many iOS teams use multiples of 8 (8, 16, 24, 32) with 4 for fine steps. That is a team convention, not an Apple rule, but many of Apple's own metrics sit on the same 4-pt grid: 16 pt margins, 20 pt on iPad, 44 pt touch targets.

Proximity Things close together read as one group. Keep space inside a group clearly smaller than space between groups, for example 4 to 8 pt inside and 24 pt between.

Alignment Line content up on a few shared edges, usually the 16 pt leading margin. A clean left edge lets the eye scan straight down.

Consistency The same relationship always gets the same space. If a title sits 4 pt above its subtitle once, it does everywhere.

Breathing room Cramped padding of 2 to 4 pt around text makes a screen feel unfinished and pushes tap targets into each other.

> **Name your scale**
>
> Put the values in code once, for example an enum Spacing with tight = 4, item = 8 and group = 24, and use the names instead of raw numbers. Designers call such named values *design tokens*.

CODE · LIVE SWIFTUI PREVIEW

Three spacing values build a hierarchy

Three values do all the work: 4 pt binds the title to its time, 8 pt separates the detail lines, and 24 pt separates the two groups. Everything shares one leading edge, and the card keeps the standard 16 pt margin. Squint at the preview: you should see two clear clumps of content.

```swift
import SwiftUI

struct EventCard: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 24) {
            VStack(alignment: .leading, spacing: 4) {
                Text("Design Review")
                    .font(.title2)
                Text("Thursday, 3 PM")
                    .foregroundStyle(.secondary)
            }
            VStack(alignment: .leading, spacing: 8) {
                Label("Room 4B", systemImage: "mappin")
                Label("6 people", systemImage: "person.2")
            }
        }
        .padding(16)
        .frame(maxWidth: .infinity, alignment: .leading)
    }
}

#Preview { EventCard() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares EventCard.
- WHAT — **Brings in SwiftUI's stacks, Text and Label.**
- WHERE — The first line of this file.
- WHEN — At compile time, before EventCard is type-checked.
- WHY — Stacks with explicit spacing are SwiftUI's main tool for rhythm and grouping.

3 `struct EventCard: View {`

- WHO — Your struct, a SwiftUI view.
- WHAT — **Declares a card that summarizes a meeting.**
- WHERE — In the app module.
- WHEN — Created when the preview shows it.
- WHY — A card with a title, a time and details has natural groups, which makes it ideal for practicing proximity.

4 `var body: some View {`

- WHO — SwiftUI, which reads body for the content.
- WHAT — **Returns the card's nested stacks.**
- WHERE — Inside EventCard, its only member.
- WHEN — On first display and whenever inputs change.
- WHY — The nesting of the stacks below mirrors the grouping you want people to perceive.

```
5  VStack(alignment: .leading, spacing: 24) {
```

WHO The outer vertical stack.
WHAT **Places the two groups 24 pt apart, sharing a leading edge.**
WHERE The root of body.
WHEN On each body pass.
WHY 24 pt is three steps of the 8-pt scale and clearly larger than any gap inside a group, so the eye sees two separate clumps.

```
6  VStack(alignment: .leading, spacing: 4) {
```

WHO The first inner stack.
WHAT **Groups the title and its time 4 pt apart.**
WHERE The first child of the outer stack.
WHEN On each body pass.
WHY 4 pt is the tightest step, used for things that are really one unit, such as a title and its subtitle.

```
7  Text("Design Review")
```

WHO A Text view drawing the meeting name at the top of the first group.
WHAT **Shows the meeting name.**
WHERE The top of the first group.
WHEN On each body pass.
WHY The name is the thing people scan for, so it comes first.

```
8  .font(.title2)
```

WHO The font modifier with the title2 text style.
WHAT **Sets the name to 22 pt at the default text size.**
WHERE On the name only.
WHEN Resolved at render time against the text-size setting.
WHY Size establishes the top of the hierarchy so spacing only has to show grouping.

```
9  Text("Thursday, 3 PM")
```

WHO A Text view drawing the time 4 points under the name.
WHAT **Shows when the meeting happens.**
WHERE Directly under the name, 4 pt below.
WHEN On each body pass.
WHY The time belongs to the title, so it sits as close as possible to it.

```
10 .foregroundStyle(.secondary)
```

WHO The foreground style modifier with the secondary role.
WHAT **Draws the time in the secondary label color.**
WHERE On the time only.
WHEN Resolved at render time and on appearance changes.
WHY Color steps the time down one level of importance while proximity keeps it attached to the title.

```
12 VStack(alignment: .leading, spacing: 8) {
```

WHO The second inner stack.
WHAT **Groups the two detail lines 8 pt apart.**
WHERE The second child of the outer stack, 24 pt below the first group.
WHEN On each body pass.
WHY Details are related to each other but are separate facts, so they get the next step up, 8 pt.

```
13  Label("Room 4B", systemImage: "mappin")
```

WHO A Label, which pairs text with an SF Symbol.
WHAT **Shows the location with a map pin symbol.**
WHERE The first line of the details group.
WHEN On each body pass.
WHY Label aligns all symbols in one column and all text in another, so the details share a clean edge.

```
14  Label("6 people", systemImage: "person.2")
```

WHO A second Label, drawing the attendee count under the room.
WHAT **Shows the attendee count with a two-person symbol.**
WHERE The second line of the details group.
WHEN On each body pass.
WHY Using the same view type for every detail line keeps their spacing and alignment identical.

```
17  .padding(16)
```

WHO The padding modifier with an explicit value.
WHAT **Adds 16 pt around the card's content.**
WHERE Around the outer stack.
WHEN During layout, after the outer VStack has measured its two groups.
WHY 16 pt is the iPhone margin; writing it out makes the rhythm visible in the code.

```
18  .frame(maxWidth: .infinity, alignment: .leading)
```

WHO The frame modifier on the padded card.
WHAT **Stretches the card to the full available width and pins its content to the leading edge.**
WHERE Outside the padding, as the last modifier in body.
WHEN During layout, after the padding has added its 16 points.
WHY Without it the card would shrink to its content and center itself, and its left edge would no longer line up with the rest of the screen.

```
22  #Preview { EventCard() }
```

WHO The #Preview macro, read by the canvas and by this page's phone to show EventCard.
WHAT **Renders the card in the phone.**
WHERE At file scope, after the closing brace of EventCard.
WHEN When the canvas refreshes.
WHY Squint at the result: two blocks of content should stand out before you read a word.

> **Try it**
>
> 1. Change the outer spacing from 24 to 4. Can you still tell the two groups apart?
> 2. Change the outer stack's alignment to .center. What happens to the left edge?

COMPARE

Cramped and ragged versus grouped and aligned

Same four facts, two layouts. The left uses one tiny gap for everything, centers lines of different lengths, nudges one line by an arbitrary 11 pt, and leaves 3 pt of padding. The right groups by meaning, aligns everything to one edge, and uses only values from the 4, 8, 16, 24 scale.

A: Before: one gap, no edge

```swift
import SwiftUI

struct PoorSummary: View {
    var body: some View {
        VStack(spacing: 2) {
            Text("Order #1042")
            Text("Arrives Friday")
            Text("2 items").padding(.leading, 11)
            Text("Total $48.00")
        }
        .padding(3)
        .background(.thinMaterial)
    }
}

#Preview { PoorSummary() }
```

1 `import SwiftUI`

WHO	The Swift compiler, while it builds the file that declares PoorSummary.
WHAT	**Brings in SwiftUI for VStack and Text.**
WHERE	The first line of this file.
WHEN	At compile time, before PoorSummary is type-checked.
WHY	Layout mistakes compile cleanly; only your eye, or a design review, catches them.

3 `struct PoorSummary: View {`

WHO	Your struct, a SwiftUI view.
WHAT	**Declares the before version of an order summary card.**
WHERE	In the app module.
WHEN	Created when the preview shows it.
WHY	An order summary mixes an identifier, a date and money, which need clear grouping.

4 `var body: some View {`

WHO	SwiftUI, which reads body for the content.
WHAT	**Returns the card: a VStack of four lines with padding and a material behind it.**
WHERE	Inside PoorSummary, its only member.
WHEN	On first display, and again if any input or the appearance changes.
WHY	Everything this card communicates is decided by the few modifiers below.

5 `VStack(spacing: 2) {`

WHO	The VStack, which stacks the four Text lines 2 points apart.
WHAT	**Stacks the four lines 2 pt apart, centered, which is the default alignment.**
WHERE	The root of body.
WHEN	On each body pass.
WHY	One tiny gap for everything erases grouping, and centering lines of different lengths creates a ragged left edge.

```
6  Text("Order #1042")
```

WHO A Text view drawing the order number as the first line.
WHAT **Shows the order number.**
WHERE The first line of the VStack.
WHEN On each body pass.
WHY Same size, weight and color as every other line, so nothing tells you where to start reading.

```
7  Text("Arrives Friday")
```

WHO A Text view drawing the delivery day as the second line.
WHAT **Shows the delivery day.**
WHERE The second line, 2 pt below.
WHEN On each body pass.
WHY It belongs with the order number, but nothing in the layout says so.

```
8  Text("2 items").padding(.leading, 11)
```

WHO A Text view with a one-off padding.
WHAT **Pushes this single line 11 pt to the right.**
WHERE The third line of the VStack, pushed right by its own padding.
WHEN During layout of the third line, before the stack places it.
WHY An arbitrary nudge off the scale; misalignment like this looks like a bug, not a decision.

```
9  Text("Total $48.00")
```

WHO A Text view drawing the total as the last line.
WHAT **Shows the order total, styled exactly like the three lines above it.**
WHERE The last line of the VStack.
WHEN On each body pass.
WHY The number people care about most is styled exactly like everything else.

```
11 .padding(3)
```

WHO The padding modifier on the VStack, with a fixed 3-point value.
WHAT **Adds only 3 pt between the text and the card edge.**
WHERE Around the VStack, inside the material background.
WHEN During layout, after the four lines have measured themselves.
WHY 3 pt crowds the text against the edge; the platform norm is 16 pt.

```
12 .background(.thinMaterial)
```

WHO The background modifier with a material.
WHAT **Draws the card behind the padded text.**
WHERE Behind the stack and its padding.
WHEN Drawn each frame, behind the padded stack.
WHY The material itself is fine; it simply makes the cramped edges easy to see.

```
16 #Preview { PoorSummary() }
```

WHO The #Preview macro, read by the canvas and by this page's phone to show PoorSummary.
WHAT **Renders the before version.**
WHERE At file scope, after the closing brace of PoorSummary.
WHEN When the canvas refreshes.
WHY Try to find the total in under a second. That struggle is the cost of missing hierarchy.

B: After: grouped on one edge

```swift
import SwiftUI

struct BetterSummary: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 4) {
            Text("Order #1042").font(.headline)
            Text("Arrives Friday").foregroundStyle(.secondary)
            Text("2 items").padding(.top, 20)
            Text("Total $48.00").bold()
        }
        .padding(16)
        .frame(maxWidth: .infinity, alignment: .leading)
        .background(.thinMaterial)
    }
}

#Preview { BetterSummary() }
```

1 `import SwiftUI`

WHO	The Swift compiler, while it builds the file that declares BetterSummary.
WHAT	**Brings in SwiftUI for VStack and Text.**
WHERE	The first line of this file.
WHEN	At compile time, before BetterSummary is type-checked.
WHY	The fix needs no new APIs, only better values.

3 `struct BetterSummary: View {`

WHO	Your struct, a SwiftUI view.
WHAT	**Declares the fixed order summary.**
WHERE	In the app module.
WHEN	Created when the preview shows it.
WHY	The same four facts, so you can compare the phones directly.

4 `var body: some View {`

WHO	SwiftUI, which reads body for the content.
WHAT	**Returns the card: a leading-aligned VStack with padding, a full-width frame and a material.**
WHERE	Inside BetterSummary, its only member.
WHEN	On first display, and again if any input or the appearance changes.
WHY	Small, deliberate changes to spacing and style below turn a list of facts into a readable summary.

5 `VStack(alignment: .leading, spacing: 4) {`

WHO	The VStack, which stacks the four lines 4 points apart on a shared leading edge.
WHAT	**Stacks the lines 4 pt apart on a shared leading edge.**
WHERE	The root of body.
WHEN	On each body pass.
WHY	A shared left edge gives the eye one line to scan down; 4 pt binds lines inside a group.

```
6  Text("Order #1042").font(.headline)
```

WHO A Text view with the headline style.
WHAT **Shows the order number at 17 pt semibold.**
WHERE The first line of the VStack.
WHEN On each body pass.
WHY Weight marks where reading starts.

```
7  Text("Arrives Friday").foregroundStyle(.secondary)
```

WHO A Text view with the secondary role.
WHAT **Shows the delivery day in the secondary label color.**
WHERE 4 pt under the order number.
WHEN On each body pass.
WHY Proximity attaches it to the order, and the softer color ranks it below the order number.

```
8  Text("2 items").padding(.top, 20)
```

WHO A Text view with extra top padding.
WHAT **Adds 20 pt above this line.**
WHERE The start of the second group.
WHEN During layout of the third line, before the stack places it.
WHY 20 pt of padding plus the stack's 4 pt spacing makes a 24 pt gap, which says a new group starts here.

```
9  Text("Total $48.00").bold()
```

WHO A Text view made bold.
WHAT **Shows the total in bold.**
WHERE 4 pt under the item count.
WHEN On each body pass.
WHY The figure people look for gets emphasis, so it is found at a glance.

```
11 .padding(16)
```

WHO The padding modifier on the VStack, with an explicit 16-point value.
WHAT **Adds 16 pt between the text and the card edge.**
WHERE Around the VStack, inside the frame and the material.
WHEN During layout, after the four lines have measured themselves.
WHY 16 pt matches the iPhone margin and gives the text room to breathe.

```
12 .frame(maxWidth: .infinity, alignment: .leading)
```

WHO The frame modifier on the padded VStack.
WHAT **Stretches the card to full width with content on the leading edge.**
WHERE Outside the padding, inside the background.
WHEN During layout, after the padding has added its 16 points.
WHY A full-width card lines up with other cards and lists on the screen.

```
13 .background(.thinMaterial)
```

WHO The background modifier with a material.
WHAT **Draws the card behind the content.**
WHERE Behind the padded, full-width stack.
WHEN Drawn each frame, behind the padded, full-width stack.
WHY Same material as the left, so spacing and alignment are the only differences.

```
17  #Preview { BetterSummary() }
```

WHO	The #Preview macro, read by the canvas and by this page's phone to show BetterSummary.
WHAT	**Renders the fixed version.**
WHERE	At file scope, after the closing brace of BetterSummary.
WHEN	When the canvas refreshes.
WHY	Find the total again: the bold weight and the clear second group make it immediate.

> **Takeaway**
>
> Spacing is information. Small gaps say “these belong together,” large gaps say “new topic,” and a shared leading edge says “read down here.” Arbitrary values like 11 pt and 3 pt break the rhythm and make a screen feel unfinished even when nobody can say why.

CONCEPT

44 points and the thumb

Apple's Human Interface Guidelines set the minimum **hit target**, the area that responds to a touch, at **44 × 44 pt**, roughly 7 mm. A fingertip is wider than that, and a touch lands a few points away from where the person aims. Smaller targets get missed or, worse, hit their neighbor.

The visual can be smaller than the hit area: a 22 pt icon is fine if its tappable frame is 44 × 44 pt. Leave space between targets too, especially next to destructive ones.

Thumb zone	People often hold the phone in one hand and tap with a thumb. The lower middle of the screen is easiest to reach; the top corners are hardest.
Primary actions	Put frequent actions within easy reach, such as tab bars, bottom toolbars and a bottom button. Put rare or destructive ones farther away.
Whole rows	In a list row, or a checkbox with a label, the whole row should respond, not just the small box.

COMPARE

Tiny tap areas versus 44-point buttons

Two icon actions. On the left they are bare images with tap gestures: each hit area is only as big as its glyph, the trash icon is caption-sized, and they sit 6 pt apart. On the right each is a real Button whose label is framed to 44 × 44 pt, with 24 pt between them. The icons look almost the same; what you can hit does not.

A: Before: glyph-sized targets

```swift
import SwiftUI

struct PoorActions: View {
    var body: some View {
        HStack(spacing: 6) {
            Image(systemName: "heart")
                .onTapGesture { }
            Image(systemName: "trash")
                .font(.caption)
                .onTapGesture { }
        }
    }
}

#Preview { PoorActions() }
```

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares PoorActions.
- **WHAT** **Brings in SwiftUI for HStack, Image and gestures.**
- **WHERE** The first line of this file.
- **WHEN** At compile time, before PoorActions is type-checked.
- **WHY** Gestures compile and work on a simulator clicked with a precise mouse, which is exactly how this bug slips through.

3 `struct PoorActions: View {`

- **WHO** Your struct, a SwiftUI view.
- **WHAT** **Declares the before version of a pair of icon actions.**
- **WHERE** In the app module.
- **WHEN** Created when the preview shows it.
- **WHY** Icon rows in cards and toolbars are where undersized targets are most common.

4 `var body: some View {`

- **WHO** SwiftUI, which reads body for the content.
- **WHAT** **Returns the row of icons.**
- **WHERE** Inside PoorActions, its only member.
- **WHEN** On first display; nothing in this view changes afterwards.
- **WHY** Everything that decides the hit areas happens in the lines below.

5 `HStack(spacing: 6) {`

- **WHO** The HStack, which places the two images 6 points apart.
- **WHAT** **Places the two icons side by side, 6 pt apart.**
- **WHERE** The root of body.
- **WHEN** On each body pass.
- **WHY** With hit areas no bigger than the glyphs, a 6 pt gap means a slightly off tap meant for the heart can land on delete.

```
6   Image(systemName: "heart")
```

WHO An SF Symbol image.
WHAT **Draws the heart at the default body size, roughly 20 pt across.**
WHERE The first item in the row.
WHEN On each body pass.
WHY A 20 pt glyph is a fine visual, but as a target it is less than half the 44 pt minimum in each direction.

```
7   .onTapGesture { }
```

WHO A tap gesture recognizer attached to the image.
WHAT **Makes the image's own small frame respond to taps.**
WHERE On the heart image.
WHEN Fires when a tap lands inside the glyph's frame.
WHY A tap gesture is not a button: the hit area is the glyph, there is no pressed state, and VoiceOver does not announce it as a button.

```
8   Image(systemName: "trash")
```

WHO A second SF Symbol image.
WHAT **Draws the trash can.**
WHERE The second item, 6 pt to the right of the heart.
WHEN On each body pass.
WHY Delete is destructive, so it deserves the most protection from accidental taps.

```
9   .font(.caption)
```

WHO The font modifier, which also sizes symbols.
WHAT **Shrinks the trash symbol to caption size, about 12 pt.**
WHERE On the trash image only.
WHEN Resolved at render time.
WHY The most dangerous action is now the smallest target, the opposite of what you want.

```
10  .onTapGesture { }
```

WHO Another tap gesture recognizer.
WHAT **Makes the tiny trash glyph tappable.**
WHERE On the trash image.
WHEN Fires on a tap inside the glyph.
WHY Combined with the 6 pt gap, a miss on one icon often becomes a hit on the other.

```
15  #Preview { PoorActions() }
```

WHO The #Preview macro, read by the canvas and by this page's phone to show PoorActions.
WHAT **Renders the before version.**
WHERE At file scope, after the closing brace of PoorActions.
WHEN When the canvas refreshes.
WHY A mouse pointer is far more precise than a thumb; test tap targets on a real device.

B: After: 44 pt buttons

```swift
import SwiftUI

struct BetterActions: View {
    var body: some View {
        HStack(spacing: 24) {
            Button { } label: {
                Label("Favorite", systemImage: "heart")
                    .frame(width: 44, height: 44)
            }
            Button { } label: {
                Label("Delete", systemImage: "trash")
                    .frame(width: 44, height: 44)
            }
        }
        .labelStyle(.iconOnly)
    }
}
#Preview { BetterActions() }
```

```swift
import SwiftUI
```

WHO The Swift compiler, while it builds the file that declares BetterActions.
WHAT **Brings in SwiftUI for Button, Label and frame.**
WHERE The first line of this file.
WHEN At compile time, before BetterActions is type-checked.
WHY Real controls bring touch, keyboard and accessibility behavior that gestures lack.

```swift
struct BetterActions: View {
```

WHO Your struct, a SwiftUI view.
WHAT **Declares the fixed pair of icon actions.**
WHERE In the app module.
WHEN Created when the preview shows it.
WHY Same two actions, so the comparison is direct.

```swift
var body: some View {
```

WHO SwiftUI, which reads body for the content.
WHAT **Returns the row of buttons.**
WHERE Inside BetterActions, its only member.
WHEN On first display; nothing in this view changes afterwards.
WHY The hit areas are now decided explicitly instead of by accident.

```swift
HStack(spacing: 24) {
```

WHO The HStack, which places the two buttons 24 points apart.
WHAT **Places the two buttons side by side, 24 pt apart.**
WHERE The root of body.
WHEN On each body pass.
WHY 24 pt of space between hit areas keeps a sloppy tap from landing on the neighbor.

```
6  Button { } label: {
```

WHO A real SwiftUI Button with an empty action and a label closure.
WHAT **Creates a button whose appearance is built in the label closure.**
WHERE The first item in the row.
WHEN The action runs on tap.
WHY A Button gets a pressed highlight, keyboard and Switch Control support, and the button trait for VoiceOver, all for free.

```
7  Label("Favorite", systemImage: "heart")
```

WHO A Label, which pairs a title with an SF Symbol.
WHAT **Gives the button the word Favorite and the heart symbol.**
WHERE Inside the button's label closure.
WHEN On each body pass.
WHY Even when only the icon shows, the title becomes the spoken label, so VoiceOver says Favorite rather than a symbol name.

```
8  .frame(width: 44, height: 44)
```

WHO The frame modifier on the Favorite label, inside the first button.
WHAT **Gives the label a 44 by 44 pt frame with the symbol centered at its natural size.**
WHERE Inside the label closure, so the frame is part of the button.
WHEN During layout of the first button's label.
WHY Inside the label, the whole 44 pt square belongs to the button. With .plain style or onTapGesture, also add .contentShape(Rectangle()), because those respond only where something is drawn.

```
10 Button { } label: {
```

WHO A second Button, built the same way as the Favorite button.
WHAT **Creates the delete button.**
WHERE The second item, 24 pt to the right.
WHEN The action runs on tap.
WHY Using a real Button for every action keeps their behavior identical.

```
11 Label("Delete", systemImage: "trash")
```

WHO A Label inside the second button.
WHAT **Pairs the word Delete with the trash symbol.**
WHERE Inside the label closure.
WHEN On each body pass.
WHY The trash glyph is back at its normal size, and the word tells VoiceOver users exactly what the button does.

```
12 .frame(width: 44, height: 44)
```

WHO The frame modifier on the Delete label, inside the second button.
WHAT **Gives the delete label the same 44 by 44 pt frame.**
WHERE Inside the second label closure.
WHEN During layout of the second button's label.
WHY Equal hit areas make equal promises; the destructive action is no longer the hardest one to hit precisely.

```
15 .labelStyle(.iconOnly)
```

WHO The label style modifier.
WHAT **Shows only the icons of both labels and hides the words visually.**
WHERE On the HStack, so it applies to both buttons.
WHEN Applied at render time.
WHY You get a compact icon row while the words remain available to VoiceOver.

```
18  #Preview { BetterActions() }
```

WHO	The #Preview macro, read by the canvas and by this page's phone to show BetterActions.
WHAT	**Renders the fixed version.**
WHERE	At file scope, after the closing brace of BetterActions.
WHEN	When the canvas refreshes.
WHY	The icons look the same size as before; the difference is invisible but your thumb feels it.

> **Takeaway**
>
> Keep the visual small if you like, but make every hit area at least 44 × 44 pt and space neighbors apart. Use a real `Button`, not a tap gesture, so it gets a pressed state and VoiceOver announces it as a button, and keep destructive actions away from easy accidental taps.

INTERACTIVE

Touch target tester

Move the pointer over the phone: the circle is a 44 pt fingertip. Click to tap. Each tap lands where you aim plus a little random scatter, like a real finger, and the panel counts hits and misses per target. Turn on *Hit areas* to see what actually responds, then *Fix targets* to grow every hit area to 44 × 44 pt without changing how anything looks. The keyboard can run 50 simulated taps per target.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. A close button's icon is 20 × 20 pt. What is the right fix?
   A. Draw the icon 44 pt wide so people can see and hit it
   B. Keep the 20 pt icon but give its button a 44 × 44 pt hit area
   C. Add a long-press gesture as a backup for missed taps
   D. Move it to the top-left corner, where the thumb naturally rests
2. Your title is hidden under the Dynamic Island on one iPhone model but not on another. What went wrong?
   A. The top inset was hard-coded instead of respecting the safe area
   B. The view never called .padding(16), so the title sits too high on that model
   C. Dynamic Island iPhones need their own storyboard for the title area
   D. The title font was too large for the smaller of the two screens
3. In a card, the title and its date are 16 pt apart, and the date and the next section are also 16 pt apart. What does a reader perceive?
   A. Two clear groups
   B. One flat list in which nothing is grouped
   C. A strong hierarchy
   D. A correct layout, because 16 is on the 8-pt grid

4. Where should the most frequently used action go on a screen meant for one-handed use?
   A. Two clear groups, title with date and then the section
   B. One flat list in which nothing is grouped
   C. A strong hierarchy running from the title to the section
   D. A correct layout, because 16 is on the 8-pt grid

5. A 20 × 20 pt checkbox sits next to the label “Remember me,” and only the box responds. What should you change?
   A. Add a hint telling people to tap the box itself
   B. Enlarge the box to 44 pt and leave the label inert
   C. Use a larger font for the label so it is easier to read
   D. Make the box and its label one control at least 44 pt tall

Answers: Layout, Spacing, and Touch Targets

1. **B. Keep the 20 pt icon but give its button a 44 × 44 pt hit area** The minimum applies to the hit area, not the drawing, so a small icon in a 44 × 44 pt frame is correct. A 44 pt icon would look heavy for no gain, and the top-left corner is one of the hardest places to reach with a thumb.
2. **A. The top inset was hard-coded instead of respecting the safe area** Top insets differ between models, which is exactly why you never hard-code them. Content inside the safe area moves down automatically on every device. Padding adds a margin but cannot know the device's inset.
3. **B. One flat list in which nothing is grouped** Proximity only works when space inside a group is clearly smaller than space between groups. Equal gaps make everything look equally related. Being on the grid is necessary, not sufficient: 4 pt inside and 24 pt between would read as two groups.
4. **C. A strong hierarchy running from the title to the section** The lower middle of the screen is the easiest area for a thumb, which is why tab bars, toolbars and primary buttons live there. The top corners need a stretch or a second hand, and a menu adds a tap to the most common task.
5. **D. Make the box and its label one control at least 44 pt tall** People tap the words as often as the box, so the whole row should respond. In SwiftUI a `Toggle` with a label does this for you. A 44 pt box would look oversized and still ignore taps on the label.

SF Symbols and App Icons

CONCEPT

Symbols that speak San Francisco

SF Symbols is Apple's library of thousands of icons, called *symbols*, drawn to sit beside San Francisco, the system font. Every symbol comes in the same **nine weights** as the font, from ultralight to black, and in **three scales** (small, medium, large) relative to the text next to it.

Place a symbol beside text and it takes the text's size and weight, so the pair reads as one unit. In the figure, change the weight and text style and watch the symbol follow while the fixed image falls out of step.

Weight	Stroke thickness, set with .fontWeight and shared with the text.
Scale	Symbol size relative to its text, set with .imageScale.

Interactive figure in the HTML version: design symbol tuner.

CODE · LIVE SWIFTUI PREVIEW

Symbols size themselves like text

A `Label` pairs a symbol with a title. The symbol takes its size from the font and its stroke weight from the font weight, so you style the pair once. `.imageScale` enlarges or shrinks only the symbol, relative to the text beside it.

```swift
import SwiftUI

struct WeatherSummary: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            Label("Partly Cloudy", systemImage: "cloud.sun")
                .font(.title2)
                .fontWeight(.bold)
            Label("Rain at 4 PM", systemImage: "cloud.rain")
                .font(.body)
            Label("Wind 12 mph", systemImage: "wind")
                .font(.footnote)
                .foregroundStyle(.secondary)
            Label("UV Index 6", systemImage: "sun.max.fill")
                .imageScale(.large)
        }
        .padding()
    }
}

#Preview { WeatherSummary() }
```

Line by line

```
1  import SwiftUI
```

WHO The Swift compiler, while it builds the app.
WHAT **Brings in SwiftUI so this file can use Label, VStack, and the font modifiers. The symbols themselves need no import or asset: they ship inside iOS.**
WHERE First line of the file; it applies to this file only.
WHEN At compile time, before any code runs.
WHY Because symbols live in the system, using one by name adds nothing to your app size and always matches the current system style.

```
3  struct WeatherSummary: View {
```

WHO You declare the type; SwiftUI creates and draws it.
WHAT **Defines a small view that summarizes the weather in four labeled rows.**
WHERE A value type in your app module; SwiftUI rebuilds instances freely because they are cheap.
WHEN An instance is created whenever the preview or a parent view needs one.
WHY A focused view keeps every styling decision for this summary in one readable place.

```
4  var body: some View {
```

WHO SwiftUI reads this property.
WHAT **Returns the description of what the summary looks like right now.**
WHERE Inside WeatherSummary, evaluated on the main thread.
WHEN Whenever SwiftUI needs to draw it, including after the person changes the text size in Settings.
WHY Because body is recomputed when Dynamic Type changes, the symbols resize with their text with no extra code.

```
5  VStack(alignment: .leading, spacing: 12) {
```

WHO SwiftUI's vertical stack layout.
WHAT **Places the four labels top to bottom, lined up on their leading (left) edge, 12 points apart.**
WHERE The root view of body.
WHEN Laid out each time body is evaluated.
WHY A shared leading edge gives the eye one line to scan down; 12 pt keeps the rows visibly related without crowding them.

```
6  Label("Partly Cloudy", systemImage: "cloud.sun")
```

WHO SwiftUI's Label view, drawing a symbol supplied by iOS.
WHAT **Shows the cloud.sun symbol followed by the title "Partly Cloudy", spaced and aligned as one unit.**
WHERE The first row of the stack.
WHEN Drawn when the stack lays out its children.
WHY Label spaces icon and title correctly and hands the title to VoiceOver, which a hand-built HStack of Image and Text does not do for you.

```
7  .font(.title2)
```

WHO The Label and both of its parts, the symbol and the title.
WHAT **Applies the title2 text style, 22 pt at the default text size, to the title and the symbol together.**
WHERE This Label only; the other rows keep their own fonts.
WHEN At every layout; it also scales with the person's Dynamic Type setting.
WHY Symbols are sized by font, not by frame, so one modifier keeps icon and text in proportion at every text size.

```
8  .fontWeight(.bold)
```

WHO SwiftUI's font settings for this Label.
WHAT **Makes the title's letters and the symbol's strokes bold together.**
WHERE On the first Label only, after its title2 font.
WHEN At every layout of the first Label, and again when Bold Text is switched on.
WHY Matching weight is what makes a symbol look like part of its text; a regular-weight icon beside bold text looks thin and detached.

```
9  Label("Rain at 4 PM", systemImage: "cloud.rain")
```

WHO SwiftUI's Label, drawing the second row with the cloud.rain symbol.
WHAT **The second row: a rain cloud symbol and the time the rain starts.**
WHERE Second row of the stack.
WHEN Drawn at layout time.
WHY Each row gets a distinct symbol so people recognize it at a glance, while the words carry the exact detail.

```
10 .font(.body)
```

WHO The font modifier on the second Label.
WHAT **Uses the body text style, 17 pt at the default size, for the symbol and title.**
WHERE On the second Label only; the rows above and below keep their own fonts.
WHEN At every layout of the second Label, following the Dynamic Type setting.
WHY Body is the normal reading size, so this row reads as ordinary content one step below the bolder title2 headline.

```
11 Label("Wind 12 mph", systemImage: "wind")
```

WHO SwiftUI's Label, drawing the third row with the wind symbol.
WHAT **The third row: the wind symbol and the wind speed.**
WHERE Third row of the stack.
WHEN Drawn at layout time.
WHY Using Label for every row keeps spacing and alignment identical, which makes the group look deliberate.

```
12 .font(.footnote)
```

WHO The font modifier on the third Label.
WHAT **Shrinks the symbol and text to the footnote style, 13 pt at the default size.**
WHERE On the third Label only, before its secondary color.
WHEN At every layout of the third Label, following the Dynamic Type setting.
WHY Wind is a secondary detail here; a smaller style moves it down the visual hierarchy without hiding it.

```
13 .foregroundStyle(.secondary)
```

WHO SwiftUI, using the system secondary label color.
WHAT **Dims both the symbol and the text to the secondary gray.**
WHERE On the third Label only, after its footnote font.
WHEN At every layout; the gray adapts automatically in Dark Mode.
WHY Hierarchy can come from color as well as size, and the semantic .secondary style stays legible in both light and dark appearances.

```
14 Label("UV Index 6", systemImage: "sun.max.fill")
```

WHO SwiftUI's Label, drawing the fourth row with the filled sun symbol.
WHAT **The fourth row: a filled sun symbol and the UV index.**
WHERE Last row of the stack.
WHEN Drawn at layout time.
WHY The .fill variant is a solid shape that reads as a stronger signal, fitting a value people may need to act on.

```
15  .imageScale(.large)
```

WHO — SwiftUI, applying one of the three symbol scales.
WHAT — **Draws the sun at the large scale while the text stays at its normal size.**
WHERE — On the fourth Label only; it does not touch the rows above.
WHEN — At every layout of the fourth Label, scaling the symbol with the text beside it.
WHY — Scales emphasize an icon relative to its text while the symbol keeps its matching weight and stays aligned with the text.

```
17  .padding()
```

WHO — SwiftUI's padding modifier on the stack.
WHAT — **Adds the default system spacing, 16 pt on iPhone, on all four sides.**
WHERE — Around the whole VStack.
WHEN — At every layout, after the four Labels have measured themselves.
WHY — Content touching the screen edge feels cramped; the default padding matches the margins the rest of iOS uses.

```
21  #Preview { WeatherSummary() }
```

WHO — Xcode previews, and the live phone on this page.
WHAT — **Renders WeatherSummary so you can see and edit it without running the whole app.**
WHERE — At file scope, outside any type; it drives the preview canvas, not your app's screens.
WHEN — Whenever the file changes while the preview canvas is open.
WHY — Fast visual feedback is how you tune spacing, weight, and size before anyone uses the screen.

> **Try it**
>
> 1. Change .title2 to .largeTitle. Does the cloud.sun symbol grow with the words?
> 2. Change .fontWeight(.bold) to .fontWeight(.ultraLight) and compare the symbol strokes.
> 3. Change .imageScale(.large) to .imageScale(.small). What stays the same size?

CODE · READ-ALONG

Rendering modes, variants, and effects

Many symbols are built from layers. A *rendering mode* decides how the layers are colored: one color, one color at several opacities, colors you choose, or the colors designed into the symbol. Variants swap in a filled or circled version, and effects animate a symbol. These modifiers are not in the live preview, so this example is explained here; the figure shows each mode.

```swift
import SwiftUI

struct SymbolGallery: View {
    @State private var likes = 0
    var body: some View {
        VStack(spacing: 24) {
            Image(systemName: "cloud.sun.rain.fill")
                .symbolRenderingMode(.multicolor)
            Image(systemName: "person.crop.circle.badge.plus")
                .symbolRenderingMode(.hierarchical)
                .foregroundStyle(.blue)
            Image(systemName: "person.crop.circle.badge.plus")
                .symbolRenderingMode(.palette)
                .foregroundStyle(.blue, .green)
            Image(systemName: "wifi", variableValue: 0.5)
            Button("Like", systemImage: "heart") { likes += 1 }
                .symbolVariant(.fill)
                .symbolEffect(.bounce, value: likes)
        }
        .font(.largeTitle)
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the app.
- **WHAT** **Imports SwiftUI, which also exposes the symbol modifiers below: rendering mode, variant, and effect.**
- **WHERE** First line of the file.
- **WHEN** At compile time, before SymbolGallery is type-checked.
- **WHY** The symbol effect types come from a framework that SwiftUI re-exports, so this one import covers everything here.

3 `struct SymbolGallery: View {`

- **WHO** You declare it; SwiftUI draws it.
- **WHAT** **A gallery of five symbols that each demonstrate one symbol feature.**
- **WHERE** A view type in your app module.
- **WHEN** Created when a screen shows it.
- **WHY** Seeing the options side by side is the fastest way to pick the right one for your design.

4 `@State private var likes = 0`

- **WHO** SwiftUI, which stores this value outside the struct.
- **WHAT** **Counts how many times the Like button was tapped.**
- **WHERE** In storage SwiftUI keeps for this view; private because only this view uses it.
- **WHEN** Created once when the view first appears; it survives every redraw.
- **WHY** The bounce effect needs a value that changes so it knows when to play.

5 `var body: some View {`

- **WHO** SwiftUI reads body on SymbolGallery; your code returns the VStack.
- **WHAT** **Describes the gallery: five symbols stacked vertically, each with its own modifiers.**
- **WHERE** Inside SymbolGallery, on the main thread.
- **WHEN** At first display and every time likes changes.
- **WHY** Because likes is state, each tap recomputes body and triggers the effect.

```
6  VStack(spacing: 24) {
```

WHO The VStack, which stacks the five symbols 24 points apart.
WHAT **Stacks the five symbols 24 points apart.**
WHERE The root of body, holding the four Images and the Button.
WHEN At every layout, and again after each tap of the Like button.
WHY Generous spacing keeps each example visually separate so they can be compared.

```
7      Image(systemName: "cloud.sun.rain.fill")
```

WHO SwiftUI, loading a system symbol by name.
WHAT **Shows the filled cloud, sun, and rain symbol, which is made of several layers.**
WHERE First item in the stack.
WHEN Drawn at layout time.
WHY Weather symbols are layered, so they show clearly how each rendering mode colors the layers.

```
8          .symbolRenderingMode(.multicolor)
```

WHO SwiftUI and the symbol data from iOS.
WHAT **Draws the symbol in the colors designed into it, such as a yellow sun and blue raindrops.**
WHERE On the first Image only, the cloud.sun.rain.fill symbol.
WHEN At every draw of the first symbol.
WHY Multicolor suits symbols of real things with natural colors; its colors are fixed, so avoid it where an icon must match your accent color.

```
9      Image(systemName: "person.crop.circle.badge.plus")
```

WHO SwiftUI, loading a system symbol.
WHAT **A person in a circle with a plus badge, a common symbol for adding a person.**
WHERE Second item in the stack.
WHEN Drawn at layout time.
WHY Badge symbols have a clear primary shape and a secondary badge, ideal for comparing modes.

```
10         .symbolRenderingMode(.hierarchical)
```

WHO The symbolRenderingMode modifier on the second Image.
WHAT **Colors every layer with one color but at different opacities: the primary layer full strength, the others lighter.**
WHERE On the second Image only, the first person.crop.circle.badge.plus.
WHEN At every draw of the second symbol.
WHY Hierarchical adds depth while staying in one color, so it fits a single-accent design and still separates the badge from the person.

```
11         .foregroundStyle(.blue)
```

WHO The foregroundStyle modifier on the second Image, feeding hierarchical mode.
WHAT **Supplies the single color that hierarchical mode varies in opacity.**
WHERE On the second Image only, after its rendering mode.
WHEN At every draw of the second symbol, adapting in Dark Mode.
WHY A system color adapts to Dark Mode and Increase Contrast automatically.

```
12     Image(systemName: "person.crop.circle.badge.plus")
```

WHO SwiftUI, loading the same system symbol a second time.
WHAT **The same symbol again, so the next mode can be compared side by side.**
WHERE Third item in the stack.
WHEN Drawn at layout time.
WHY Holding the symbol constant isolates the one thing that changes: the rendering mode.

13 `.symbolRenderingMode(.palette)`

WHO The symbolRenderingMode modifier on the third Image.
WHAT **Lets you choose a separate color for each layer.**
WHERE On the third Image only, the second person.crop.circle.badge.plus.
WHEN At every draw of the third symbol.
WHY Palette is for colors you control, such as a status color on the badge and a neutral color on the main shape.

14 `.foregroundStyle(.blue, .green)`

WHO SwiftUI, assigning styles to layers in order.
WHAT **Paints the symbol's first layer blue and its second layer green.**
WHERE On the third Image only, after its palette rendering mode.
WHEN At every draw of the third symbol, one color per layer.
WHY Order matters: the first style paints the primary layer and the second the secondary, so swapping them swaps the emphasis.

15 `Image(systemName: "wifi", variableValue: 0.5)`

WHO SwiftUI and the symbol's variable-color layers.
WHAT **Shows the Wi-Fi symbol with its arcs filled partway, driven by a value from 0 to 1.**
WHERE Fourth item in the stack.
WHEN Redrawn whenever the value you pass changes.
WHY A variable value turns a symbol into a tiny gauge for signal, volume, or progress without drawing your own icons.

16 `Button("Like", systemImage: "heart") { likes += 1 }`

WHO SwiftUI's Button, and the person tapping it.
WHAT **A button with a heart symbol and the title Like; each tap adds 1 to likes.**
WHERE Last item in the stack.
WHEN The closure runs on every tap.
WHY This initializer builds a Label for you, so the title stays available to VoiceOver even where only the icon is shown.

17 `.symbolVariant(.fill)`

WHO The symbolVariant modifier on the Like button's label.
WHAT **Switches the heart to its filled variant without changing the name string.**
WHERE Symbols inside this button.
WHEN At every draw of the heart, before the bounce effect.
WHY iOS uses filled variants for selected or emphasized items; tab bars apply them automatically, and you can drive the variant from state.

18 `.symbolEffect(.bounce, value: likes)`

WHO SwiftUI and the symbol animation system.
WHAT **Plays a short bounce on the heart whenever likes changes.**
WHERE Symbols inside this button.
WHEN Each time likes changes, which is every tap.
WHY A brief animation tied to a real event confirms the tap; keep effects short and meaningful, because motion that only decorates distracts.

20 `.font(.largeTitle)`

WHO The font modifier on the VStack, inherited by every symbol inside it.
WHAT **Sizes every symbol in the stack to the largeTitle style, 34 pt at the default text size.**
WHERE Applies to all children of the VStack.
WHEN At every layout, and it grows with Dynamic Type.
WHY Symbols scale with font, so one modifier sizes the whole gallery consistently.

COMPARE

Icon-only buttons need words

An icon is faster than a word only when everyone already knows it. Share, trash, and search are universal; tray, tag, and bolt are not. Tap the buttons on both phones and predict what each will do before you read the result.

A: Before: icons alone

```swift
import SwiftUI
struct PoorActions: View {
    let symbols = ["tray", "tag", "bolt", "link"]
    @State private var tapped = "Nothing yet"
    var body: some View {
        VStack(spacing: 32) {
            HStack(spacing: 32) {
                ForEach(symbols, id: \.self) { symbol in
                    Button { tapped = symbol } label: {
                        Image(systemName: symbol)
                    }
                }
            }
            Text("Last tap: \(tapped)")
        }
    }
}
#Preview { PoorActions() }
```

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares PoorActions.
- WHAT — **Imports SwiftUI for Button, Image, and the stacks.**
- WHERE — Top of the file, above PoorActions.
- WHEN — At compile time, before PoorActions is type-checked.
- WHY — Every SwiftUI file imports the framework it uses.

2 `struct PoorActions: View {`

- WHO — You declare it; SwiftUI draws it.
- WHAT — **A row of four action buttons drawn with icons only.**
- WHERE — A view type in your app module.
- WHEN — Created when the preview shows it.
- WHY — This is the version to critique: it looks clean, but it asks people to guess.

3 `let symbols = ["tray", "tag", "bolt", "link"]`

- WHO — Your view, as a constant.
- WHAT — **Lists four symbol names, one per action.**
- WHERE — A stored property of PoorActions.
- WHEN — Set when the view is created.
- WHY — Only the drawing is stored; nothing says what each button does, which is exactly the problem.

```
4  @State private var tapped = "Nothing yet"
```

WHO SwiftUI state owned by this view.
WHAT **Remembers which button was tapped last.**
WHERE SwiftUI-managed storage for this view.
WHEN Updated on every tap.
WHY Showing the result lets you check whether your guess about each icon was right.

```
5  var body: some View {
```

WHO SwiftUI reads body on PoorActions after each tap; your code returns the VStack.
WHAT **Describes the buttons and the result line.**
WHERE Inside PoorActions, below the symbols constant and the tapped state.
WHEN At first display and after every tap.
WHY Changing tapped recomputes body, so the result line stays current.

```
6  VStack(spacing: 32) {
```

WHO The VStack, which places the icon row above the result line.
WHAT **Places the icon row above the result line, 32 points apart.**
WHERE The root of body, holding the HStack and the Text.
WHEN At every layout, and again after each tap changes tapped.
WHY Separating the controls from the feedback keeps the two easy to tell apart.

```
7  HStack(spacing: 32) {
```

WHO The HStack, which places the four icon buttons in one row.
WHAT **Lines the icon buttons up in one row, 32 points apart.**
WHERE Top of the VStack.
WHEN At every layout of the icon row.
WHY Tight icon rows are typical of toolbars, which is where ambiguous icons usually hide.

```
8  ForEach(symbols, id: \.self) { symbol in
```

WHO SwiftUI's ForEach, repeating its closure once per symbol name.
WHAT **Creates one button per symbol name, using the name itself as its identity.**
WHERE Inside the HStack, producing its four children.
WHEN Each time body runs.
WHY The names are unique, so they work as identities for SwiftUI to track each button.

```
9  Button { tapped = symbol } label: {
```

WHO SwiftUI, and the person tapping.
WHAT **A button that records its symbol name when tapped.**
WHERE One per symbol in the row.
WHEN The action runs on tap.
WHY Nothing in the label says what happens, so people learn only by trying, which is risky for actions such as archive or delete.

```
10 Image(systemName: symbol)
```

WHO SwiftUI, drawing a system symbol.
WHAT **Uses only the symbol as the button's visible label.**
WHERE Inside each button's label closure, as its only content.
WHEN At every draw of the four buttons.
WHY With no title, VoiceOver can describe only the drawing, not the action, and sighted people have to guess too.

```
14  Text("Last tap: \(tapped)")
```

WHO — SwiftUI's text renderer, reading tapped to build the result line.
WHAT — **Shows the name of the symbol tapped last.**
WHERE — Below the icon row.
WHEN — Redrawn after every tap.
WHY — It exposes the gap between picture and meaning: you tapped "tray", but was that Archive, Inbox, or Move?

```
18  #Preview { PoorActions() }
```

WHO — Xcode previews and the live phone on this page.
WHAT — **Renders PoorActions for testing.**
WHERE — At file scope, after the closing brace of PoorActions.
WHEN — Whenever the code changes.
WHY — Tapping through a preview is a quick, cheap usability test of your icons.

B: After: symbol plus title

```swift
import SwiftUI
struct BetterActions: View {
    let titles = ["Archive", "Add Tag", "Mark Urgent", "Copy Link"]
    let symbols = ["tray", "tag", "bolt", "link"]
    @State private var tapped = "Nothing yet"
    var body: some View {
        VStack(alignment: .leading, spacing: 20) {
            ForEach(titles.indices, id: \.self) { i in
                Button { tapped = titles[i] } label: {
                    Label(titles[i], systemImage: symbols[i])
                }
            }
            Text("Last tap: \(tapped)")
                .foregroundStyle(.secondary)
        }
    }
}
#Preview { BetterActions() }
```

```
1  import SwiftUI
```

WHO — The Swift compiler, while it builds the file that declares BetterActions.
WHAT — **Imports SwiftUI for Button, Label, and the stack.**
WHERE — Top of the file, above BetterActions.
WHEN — At compile time, before BetterActions is type-checked.
WHY — Every SwiftUI file imports the framework it uses.

```
2  struct BetterActions: View {
```

WHO — You declare it; SwiftUI draws it.
WHAT — **The same four actions, each with a visible title next to its symbol.**
WHERE — A view type in your app module.
WHEN — Created when the preview shows it.
WHY — Same features, no guessing: this is the version to ship when the symbols are not universal.

```
3  let titles = ["Archive", "Add Tag", "Mark Urgent", "Copy Link"]
```

WHO Your view, as a constant.
WHAT **Names each action with a short verb phrase.**
WHERE A stored property of BetterActions.
WHEN Set when the view is created.
WHY Button titles should say what happens; verbs such as Archive and Copy Link remove the guesswork.

```
4  let symbols = ["tray", "tag", "bolt", "link"]
```

WHO Your view, as a constant.
WHAT **Keeps the same symbols, in the same order as the titles.**
WHERE A stored property of BetterActions.
WHEN Set when the view is created.
WHY Symbols still speed up scanning once people know them; the title carries the meaning.

```
5  @State private var tapped = "Nothing yet"
```

WHO SwiftUI state owned by this view.
WHAT **Remembers which action was tapped last.**
WHERE SwiftUI-managed storage for this view.
WHEN Updated on every tap.
WHY It now records the action name, which is the information people actually care about.

```
6  var body: some View {
```

WHO SwiftUI reads body on BetterActions after each tap; your code returns the VStack.
WHAT **Describes the labeled buttons and the result line.**
WHERE Inside BetterActions, below the two constants and the tapped state.
WHEN At first display and after every tap.
WHY Changing tapped recomputes body, so the result line stays current.

```
7  VStack(alignment: .leading, spacing: 20) {
```

WHO The VStack, which stacks the four labeled buttons and the result line.
WHAT **Stacks the buttons on a shared leading edge, 20 points apart.**
WHERE The root of body, holding the ForEach and the Text.
WHEN At every layout, and again after each tap changes tapped.
WHY Titles need room; a vertical list with one leading edge is easy to scan and gives each button a comfortable tap area.

```
8  ForEach(titles.indices, id: \.self) { i in
```

WHO SwiftUI's ForEach, repeating its closure once per index in titles.
WHAT **Loops over positions 0 to 3 so each button can read the title and symbol at the same index.**
WHERE Inside the VStack, producing its four buttons.
WHEN Each time body runs.
WHY The index keeps the two arrays in step; in a real app a small Action struct holding both is cleaner.

```
9  Button { tapped = titles[i] } label: {
```

WHO SwiftUI, and the person tapping.
WHAT **A button that records its action title when tapped.**
WHERE One Button per index, inside the ForEach closure.
WHEN The action runs on tap.
WHY The visible label now promises exactly what the action does.

```
10  Label(titles[i], systemImage: symbols[i])
```

WHO — SwiftUI's Label, pairing the title and symbol at index i.
WHAT — **Shows the symbol followed by its title, as one label.**
WHERE — Inside each button's label closure, as its only content.
WHEN — At every draw of the four buttons.
WHY — Label gives VoiceOver the title as the button name, and a toolbar can later collapse it to icon-only without losing that name.

```
13  Text("Last tap: \(tapped)")
```

WHO — SwiftUI's text renderer, reading tapped to build the result line.
WHAT — **Shows the action tapped last.**
WHERE — Below the four buttons, as the last child of the VStack.
WHEN — Redrawn after every tap.
WHY — Now the feedback matches what people intended, which builds trust.

```
14  .foregroundStyle(.secondary)
```

WHO — The foregroundStyle modifier on the result Text.
WHAT — **Draws the result line in the secondary label color.**
WHERE — On the result Text only, not on the buttons above it.
WHEN — At every draw; it adapts to Dark Mode.
WHY — Feedback is supporting information, so it sits below the buttons in the visual hierarchy.

```
18  #Preview { BetterActions() }
```

WHO — Xcode previews and the live phone on this page.
WHAT — **Renders BetterActions for testing.**
WHERE — At file scope, after the closing brace of BetterActions.
WHEN — Whenever the code changes.
WHY — Compare both phones: the labeled version needs no explanation.

> **Takeaway**
>
> Use icon-only buttons only for symbols people already know (share, delete, search, close) or for crowded toolbars, and even then build them from a `Label` so VoiceOver can read the title. When the meaning is not universal, show the words. Keep one symbol per meaning across the whole app: if `tray` means Archive on one screen, it must not mean Inbox on another.

CODE · LIVE SWIFTUI PREVIEW

One Label, two looks

Sometimes space is tight and an icon alone is right, for example the familiar Share symbol. Write it as a `Label` anyway and hide the title with `.labelStyle(.iconOnly)`: the title leaves the screen, but VoiceOver still speaks it. Toolbars do this to labels automatically.

```swift
import SwiftUI

struct ShareControls: View {
    @State private var shareCount = 0
    var body: some View {
        VStack(spacing: 24) {
            Button { shareCount += 1 } label: {
                Label("Share", systemImage: "square.and.arrow.up")
            }
            .buttonStyle(.borderedProminent)
            Button { shareCount += 1 } label: {
                Label("Share", systemImage: "square.and.arrow.up")
            }
            .labelStyle(.iconOnly)
            .buttonStyle(.bordered)
            Text("Shared \(shareCount) times")
                .foregroundStyle(.secondary)
        }
        .controlSize(.large)
    }
}
#Preview { ShareControls() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares ShareControls.
- **WHAT** **Imports SwiftUI for Button, Label, Text, and the button and label styles.**
- **WHERE** Top of the file, above ShareControls.
- **WHEN** At compile time, before ShareControls is type-checked.
- **WHY** Every SwiftUI file imports the framework it uses.

3 `struct ShareControls: View {`

- **WHO** You declare it; SwiftUI draws it.
- **WHAT** **Two Share buttons built from the same Label: one shows the title, one shows only the icon.**
- **WHERE** A view type in your app module.
- **WHEN** Created when the preview shows it.
- **WHY** Putting them side by side shows that hiding the title changes the look, not the meaning.

4 `@State private var shareCount = 0`

- **WHO** SwiftUI state owned by this view.
- **WHAT** **Counts taps on either button.**
- **WHERE** SwiftUI-managed storage for this view.
- **WHEN** Updated on every tap of either button.
- **WHY** The count proves both buttons trigger the same action even though they look different.

5 `var body: some View {`

- **WHO** SwiftUI reads body on ShareControls after each tap; your code returns the VStack.
- **WHAT** **Describes the two buttons and the count.**
- **WHERE** Inside ShareControls, below the shareCount state.
- **WHEN** At first display and after every tap.
- **WHY** Because shareCount is state, the count updates as soon as either button is tapped.

```
6  VStack(spacing: 24) {
```

WHO The VStack, which stacks the two buttons and the count line.
WHAT **Stacks the two buttons and the count, 24 points apart.**
WHERE The root of body, holding both Buttons and the Text.
WHEN At every layout, and again after each tap changes shareCount.
WHY Clear space between buttons prevents accidental taps on the wrong one.

```
7  Button { shareCount += 1 } label: {
```

WHO SwiftUI, and the person tapping.
WHAT **The first Share button; tapping adds 1 to shareCount.**
WHERE Top of the stack.
WHEN The action runs on tap.
WHY The label closure lets the button use a full Label instead of plain text.

```
8  Label("Share", systemImage: "square.and.arrow.up")
```

WHO SwiftUI's Label with a system symbol.
WHAT **Shows the standard share symbol with the title Share.**
WHERE Inside the first button.
WHEN At every draw of the first button.
WHY square.and.arrow.up is the system share symbol people already know; reusing it keeps your app consistent with iOS.

```
10 .buttonStyle(.borderedProminent)
```

WHO SwiftUI's button styling, applied to the first button only.
WHAT **Gives the first button a filled, tinted background.**
WHERE The first button only.
WHEN At every draw of the first button, including its pressed state.
WHY A prominent style marks the primary action on a screen; use it for one action at a time.

```
11 Button { shareCount += 1 } label: {
```

WHO SwiftUI, and the person tapping.
WHAT **A second Share button with the identical action.**
WHERE Middle of the stack.
WHEN The action runs on tap.
WHY Same action, so the only difference you will see is presentation.

```
12 Label("Share", systemImage: "square.and.arrow.up")
```

WHO SwiftUI's Label, the same title and symbol as in the first button.
WHAT **The same Label value as in the first button.**
WHERE Inside the second button.
WHEN At every draw of the second button.
WHY Writing the full Label even when you plan to hide the title keeps the meaning attached to the control.

```
14 .labelStyle(.iconOnly)
```

WHO SwiftUI, choosing how the Label presents itself.
WHAT **Hides the title visually and draws only the symbol.**
WHERE The second button only.
WHEN At every draw of the second button, hiding its title.
WHY The title still becomes the accessibility label, so VoiceOver says "Share, button"; a bare Image would give it no name to read.

```
15  .buttonStyle(.bordered)
```

WHO SwiftUI's button styling, applied to the second button only.
WHAT **Gives the icon-only button a light tinted background shape.**
WHERE The second button only.
WHEN At every draw of the second button, including its pressed state.
WHY A visible shape shows where the tap area is, which matters more when there is no text.

```
16  Text("Shared \(shareCount) times")
```

WHO SwiftUI's text renderer, reading shareCount to build the count line.
WHAT **Shows how many times either button was tapped.**
WHERE Bottom of the stack.
WHEN Redrawn after every tap.
WHY Visible feedback confirms that the icon-only button really did something.

```
17  .foregroundStyle(.secondary)
```

WHO The foregroundStyle modifier on the count Text.
WHAT **Draws the count in the secondary label color.**
WHERE On the count Text only, not on the buttons above it.
WHEN At every draw; it adapts to Dark Mode.
WHY Supporting text stays quieter than the controls it describes.

```
19  .controlSize(.large)
```

WHO SwiftUI, passing a size to every control inside the stack.
WHAT **Makes both buttons use the large control size, with taller backgrounds and more padding.**
WHERE Applies to all buttons in the VStack.
WHEN At every layout of the VStack, before each button sizes itself.
WHY Icon-only buttons especially need a generous hit area, at least 44 by 44 points.

```
22  #Preview { ShareControls() }
```

WHO Xcode previews and the live phone on this page.
WHAT **Renders ShareControls for testing.**
WHERE At file scope, after the closing brace of ShareControls.
WHEN Whenever the code changes.
WHY Tap both buttons and watch the same count go up.

> **Try it**
>
> 1. Delete .labelStyle(.iconOnly). How do the two buttons differ now?
> 2. Change .controlSize(.large) to .controlSize(.small) and compare the tap targets.
> 3. Move .labelStyle(.iconOnly) onto the VStack. What happens to the first button?

CONCEPT

App icon basics

Your app icon is the first piece of your design anyone sees: on the Home Screen, in the App Store, in Settings, and in notifications. In iOS 26 and later you build it in **Icon Composer**, a tool that comes with Xcode 26, as a stack of flat layers on a 1024 x 1024 canvas. The system adds Liquid Glass depth and highlights and applies the rounded-rectangle mask, so you draw neither rounded corners nor drop shadows.

One idea A single, simple shape people recognize at a glance, even at the small sizes used in Settings and notifications.

No words or screenshots	Text is unreadable when small and cannot be translated; photos and pictures of your UI turn to mush.
Four appearances	People can show icons in default, dark, clear, or tinted styles. Icon Composer previews each, so check that your layers read in all of them.
Not an SF Symbol	Apple's license does not allow SF Symbols, or look-alikes, in app icons or logos. Draw your own mark.

> **The squint test**
>
> Shrink the icon until it is a few millimeters wide and place it among other icons. If you can still tell what it is, and it does not look like a neighbor, it works.

CHECK YOURSELF

Check your understanding

1. A row shows `Image(systemName: "bell")` next to a `Text` styled with `.font(.headline)`. The bell looks small and thin beside the semibold text. What is the best fix?
   A. Give the image .frame(width: 24, height: 24) so it matches the text height
   B. Put both in Label("Reminders", systemImage: "bell") and style the Label
   C. Replace the symbol with a larger PNG icon exported at 3x
   D. Add .bold() to the Text only, so the weights appear to match
2. A weather row should show a symbol in natural colors: a yellow sun and blue rain. Which rendering mode do you choose?
   A. .monochrome
   B. .hierarchical with .foregroundStyle(.yellow)
   C. .palette with a single color
   D. .multicolor
3. A toolbar has one icon-only button with the `bolt` symbol. It turns on Low Data Mode for the screen. What is the strongest design?
   A. Keep the icon; people will learn what it means after a few tries
   B. Color the icon red so it stands out from the other buttons
   C. Show a Label titled "Low Data Mode" so the words carry meaning
   D. Explain the icon in a tooltip that appears after a long press on it
4. Which of these belongs in an app icon?
   A. One simple shape built as layers in Icon Composer
   B. The app's name in bold letters on a brand-colored background
   C. A screenshot of the main screen, cropped to a square
   D. The SF Symbol that best represents the app

5. You write `Label("Share", systemImage: "square.and.arrow.up")` inside a button and apply `.labelStyle(.iconOnly)`. What does VoiceOver say when it reaches this button?

   A. Nothing, because hidden text is never read
   B. "Share, button"
   C. The symbol name, "square and arrow up"
   D. "Image"

Answers: SF Symbols and App Icons

1. **B. Put both in Label("Reminders", systemImage: "bell") and style the Label** Symbols take their size and weight from the font. Put both in a `Label` and style it once, and the bell matches the headline's 17 pt semibold, and keeps matching as Dynamic Type grows. A fixed frame stretches the drawing without changing its stroke weight and stops it scaling with the text; a PNG cannot change weight at all.
2. **D. .multicolor** Multicolor uses the colors designed into the symbol. Hierarchical uses one color at several opacities, so the rain would be yellow too. Palette lets you assign a color per layer, but with a single color every layer gets that color, which is no better than monochrome.
3. **C. Show a Label titled "Low Data Mode" so the words carry meaning** Icon-only buttons work only for symbols everyone already understands, and bolt has no shared meaning for this action. A title removes the guesswork, and a Label also gives VoiceOver a name. Color adds no meaning and fails for people who cannot distinguish it; a long-press hint is hidden, so most people never find it.
4. **A. One simple shape built as layers in Icon Composer** A single, simple shape survives every size and every appearance (default, dark, clear, tinted). Words are unreadable when small and cannot be translated, screenshots turn to mush, and Apple's license does not allow SF Symbols in app icons.
5. **B. "Share, button"** `.iconOnly` hides the title only visually; it remains the button's accessibility label, so VoiceOver announces the name and then the trait, button. That is why a Label beats a bare Image, which would leave VoiceOver with no meaningful name.

Choosing a Navigation Pattern

CONCEPT

Three ways to move through an app

Navigation is how people get from what they see to what they want. Module 04 showed how to code the three iOS patterns; this step is about choosing between them. Most apps combine all three. Choose by asking one question: how do these screens relate to each other?

Flat: tab bar	A few peer sections people switch between at any time, such as Library, Radio, and Search. SwiftUI: TabView with Tab. Use 3 to 5 tabs on iPhone; each tab is a place, never an action.
Hierarchical: navigation stack	Drilling down from general to specific: a list, then an item, then a detail. SwiftUI: NavigationStack with NavigationLink. The back button and a swipe from the left edge climb back up.
Modal: sheet	A focused, self-contained task that pauses the current context, such as writing a message or adding an event. SwiftUI: .sheet. It always offers a clear way out: Cancel, or Done when finished.

> **The usual combination**
>
> Tabs at the root, a `NavigationStack` inside each tab, and sheets for creating or editing. Many of the apps on your own phone, including Music and the App Store, follow this shape.

CODE · LIVE SWIFTUI PREVIEW

Flat sections with a tab bar

A `TabView` holds the top-level sections of an app. Each `Tab` gets a short noun and a symbol and keeps its screen alive while you visit the others. Tap the tabs in the phone. In iOS 26 and later the real tab bar floats above the content as Liquid Glass.

```swift
import SwiftUI

struct RecipesApp: View {
    var body: some View {
        TabView {
            Tab("Recipes", systemImage: "fork.knife") {
                Text("Browse recipes")
            }
            Tab("Groceries", systemImage: "cart") {
                Text("Your shopping list")
            }
            .badge(3)
            Tab("Saved", systemImage: "heart") {
                Text("Recipes you saved")
            }
        }
    }
}

#Preview { RecipesApp() }
```

Line by line

1 `import SwiftUI`

WHO	The Swift compiler, while it builds the file that declares RecipesApp.
WHAT	**Imports SwiftUI for TabView, Tab, and Text.**
WHERE	Top of the file, above RecipesApp.
WHEN	At compile time, before RecipesApp is type-checked.
WHY	Every SwiftUI file imports the framework it uses.

3 `struct RecipesApp: View {`

WHO	You declare it; in a full app, the WindowGroup in your App struct shows it first.
WHAT	**The root view of a recipe app: three top-level sections in a tab bar.**
WHERE	A view type at the top of your view hierarchy.
WHEN	Created at launch as the first screen.
WHY	The root is where top-level navigation belongs, so the app's whole structure is visible in one place.

4 `var body: some View {`

WHO	SwiftUI reads body on RecipesApp at launch; your code returns the TabView.
WHAT	**Describes the tab container and its tabs.**
WHERE	Inside RecipesApp, its only member.
WHEN	At launch and whenever its inputs change.
WHY	The tab structure is declared once; SwiftUI handles switching between tabs.

5 `TabView {`

WHO	SwiftUI's TabView, which draws the tab bar and swaps content when a tab is tapped.
WHAT	**Creates the tab bar and shows one tab's content at a time.**
WHERE	The root of the screen; the tab bar sits along the bottom edge, above the home indicator.
WHEN	Built at launch; it switches content whenever a tab is tapped.
WHY	A tab bar keeps every top-level section one tap away, which suits peers people move between often.

```
6  Tab("Recipes", systemImage: "fork.knife") {
```

WHO SwiftUI, building one tab item.
WHAT **Defines the first tab with the title Recipes and the fork.knife symbol.**
WHERE First item in the tab bar.
WHEN Drawn with the tab bar; its content shows when the tab is selected.
WHY Tab titles are short nouns for places; one or two words fit even on the narrowest iPhones.

```
7      Text("Browse recipes")
```

WHO SwiftUI's text renderer, drawing the Recipes tab's placeholder.
WHAT **Placeholder content for the Recipes section.**
WHERE Inside the Recipes tab.
WHEN Shown while Recipes is selected.
WHY In a real app this would be a NavigationStack with the recipe list, so the tab can drill down on its own.

```
9  Tab("Groceries", systemImage: "cart") {
```

WHO SwiftUI's TabView, building its second tab item from this Tab.
WHAT **The second tab: a shopping list section with the cart symbol.**
WHERE Second item in the tab bar.
WHEN Content shows when the tab is selected.
WHY The cart symbol matches what people expect for shopping, so the tab is recognizable before they read it.

```
10     Text("Your shopping list")
```

WHO SwiftUI's text renderer, drawing the Groceries tab's placeholder.
WHAT **Placeholder content for the Groceries section.**
WHERE Inside the Groceries tab.
WHEN Shown while Groceries is selected.
WHY Each tab owns its own content, so switching tabs never mixes sections together.

```
12     .badge(3)
```

WHO SwiftUI, drawing a badge on the tab item.
WHAT **Shows a badge with the number 3 on the Groceries tab.**
WHERE On the tab bar item, not inside the content.
WHEN Whenever the tab bar is drawn; a value of 0 hides it.
WHY A badge tells people something needs attention in a section without making them visit it; keep it for counts that matter, such as items still to buy.

```
13 Tab("Saved", systemImage: "heart") {
```

WHO SwiftUI's TabView, building its third tab item from this Tab.
WHAT **The third tab, for saved recipes, with the heart symbol.**
WHERE Third item in the tab bar.
WHEN Content shows when the tab is selected.
WHY Three to five tabs is the comfortable range on iPhone; each one is a place people return to.

```
14     Text("Recipes you saved")
```

WHO SwiftUI's text renderer, drawing the Saved tab's placeholder.
WHAT **Placeholder content for the Saved section.**
WHERE Inside the Saved tab.
WHEN Shown while Saved is selected.
WHY Short, specific placeholder text makes the prototype easy to test with real people.

```
20 #Preview { RecipesApp() }
```

WHO Xcode previews and the live phone on this page.
WHAT **Renders the tab app so you can tap between sections.**
WHERE At file scope, after the closing brace of RecipesApp.
WHEN Whenever the code changes.
WHY Checking titles and symbols in a preview catches labels that are too long or unclear.

> **Try it**
>
> 1. Add a fourth tab named Search with the magnifyingglass symbol.
> 2. Change .badge(3) to .badge(0). What happens to the badge?
> 3. Rename "Saved" to "Recipes You Have Saved". How does the tab bar cope with the long title?

CODE · LIVE SWIFTUI PREVIEW

Drill down with a navigation stack

A `NavigationStack` shows one screen at a time and remembers the path behind it. Each `NavigationLink` pushes a more specific screen. The back button in the top leading corner and a swipe from the left edge both go back. Tap a recipe in the phone, then return.

```
1  import SwiftUI
2
3  struct RecipeList: View {
4      let recipes = ["Pancakes", "Tomato Soup", "Green Curry"]
5
6      var body: some View {
7          NavigationStack {
8              List {
9                  ForEach(recipes, id: \.self) { recipe in
10                     NavigationLink(recipe) {
11                         Text("Steps for \(recipe)")
12                             .navigationTitle(recipe)
13                             .navigationBarTitleDisplayMode(.inline)
14                     }
15                 }
16             }
17             .navigationTitle("Recipes")
18         }
19     }
20 }
21
22 #Preview { RecipeList() }
```

Line by line

```
1 import SwiftUI
```

WHO The Swift compiler, while it builds the file that declares RecipeList.
WHAT **Imports SwiftUI for NavigationStack, List, and NavigationLink.**
WHERE Top of the file, above RecipeList.
WHEN At compile time, before RecipeList is type-checked.
WHY Every SwiftUI file imports the framework it uses.

```
3  struct RecipeList: View {
```

WHO You declare it; SwiftUI draws it.
WHAT **A list of recipes that drills down into each recipe.**
WHERE A view type in your app module, typically the content of one tab.
WHEN Created when its tab or screen appears.
WHY A list that leads to details is the classic hierarchical pattern.

```
4  let recipes = ["Pancakes", "Tomato Soup", "Green Curry"]
```

WHO Your view, as a constant.
WHAT **A fixed array of recipe names to list.**
WHERE A stored property of RecipeList.
WHEN Set when the view is created.
WHY Simple sample data keeps the focus on navigation; a real app would load recipes from a model.

```
6  var body: some View {
```

WHO SwiftUI reads body on RecipeList; your code returns the NavigationStack.
WHAT **Describes the stack and its root list.**
WHERE Inside RecipeList, below the recipes constant.
WHEN At first display and whenever inputs change.
WHY You declare the root once; SwiftUI manages the pushed screens.

```
7  NavigationStack {
```

WHO SwiftUI's NavigationStack, which owns the bar, the back button and the pushed screens.
WHAT **Hosts a stack of screens with a navigation bar, a back button, and the edge-swipe gesture.**
WHERE Wraps the list, at the root of this part of the app.
WHEN Created once; screens are pushed and popped as people tap links and go back.
WHY Owning the stack at the root means every pushed screen automatically gets a back button and the edge-swipe gesture.

```
8  List {
```

WHO SwiftUI's List, which draws one row per NavigationLink inside it.
WHAT **Shows the recipes as rows in a scrolling list.**
WHERE The root screen of the stack.
WHEN At every layout; it scrolls when rows exceed the screen.
WHY Lists are the standard way to show items people drill into, and link rows get the system chevron automatically.

```
9  ForEach(recipes, id: \.self) { recipe in
```

WHO SwiftUI's ForEach, repeating its closure once per recipe name.
WHAT **Creates one row per recipe name, using the name as its identity.**
WHERE Inside the List, producing its three rows.
WHEN Each time body runs.
WHY Stable identities let SwiftUI animate and track rows correctly.

```
10 NavigationLink(recipe) {
```

WHO SwiftUI, and the person tapping the row.
WHAT **Makes the row a link titled with the recipe name; tapping pushes the destination screen.**
WHERE One NavigationLink per row, inside the ForEach closure.
WHEN The destination's body runs only when the row is tapped and the screen is pushed.
WHY Rows that drill down show a chevron, so people can predict that tapping goes deeper.

```
11  Text("Steps for \(recipe)")
```

WHO — SwiftUI's text renderer, drawing the detail screen's content after a push.
WHAT — **The content of the detail screen for the tapped recipe.**
WHERE — The pushed screen, inside the NavigationLink's destination closure.
WHEN — Drawn after the push.
WHY — A detail screen shows one specific thing, the bottom of this small hierarchy.

```
12  .navigationTitle(recipe)
```

WHO — SwiftUI's navigation bar, reading this modifier on the detail Text.
WHAT — **Titles the pushed screen with the recipe name.**
WHERE — The detail screen only.
WHEN — When the detail screen appears.
WHY — The title tells people where they are, and it names this screen in the back button's long-press history menu.

```
13  .navigationBarTitleDisplayMode(.inline)
```

WHO — SwiftUI's navigation bar, reading this modifier on the detail Text.
WHAT — **Shows the title small and centered in the bar instead of large.**
WHERE — The detail screen only.
WHEN — When the detail screen appears.
WHY — Large titles suit the top of a section; detail screens usually use the inline style to give content more room.

```
17  .navigationTitle("Recipes")
```

WHO — SwiftUI's navigation bar, reading this modifier on the List.
WHAT — **Sets the large title of the root screen to Recipes.**
WHERE — On the List, the root of the stack.
WHEN — When the root screen appears.
WHY — It names the section at the top of the root screen, and it is how the back button history menu lists this screen.

```
22  #Preview { RecipeList() }
```

WHO — Xcode previews and the live phone on this page.
WHAT — **Renders the list so you can drill in and back out.**
WHERE — At file scope, after the closing brace of RecipeList.
WHEN — Whenever the code changes.
WHY — Walking the path in a preview shows whether every screen has a clear title and a clear way back.

> **Try it**
>
> 1. Add "Banana Bread" to the recipes array and open it.
> 2. Delete .navigationBarTitleDisplayMode(.inline). How does the detail title change?
> 3. Open a recipe, then go back. Which screen shows a large title, and which shows the inline one?

COMPARE

A tab is a place, not an action

A common beginner design puts "New" in the tab bar. Tapping it swaps the whole app for an editor with no Cancel, and a half-written draft sits there unexplained the next time someone opens the tab. Compare it with the standard pattern: a toolbar button that opens a sheet.

A: Before: an action in the tab bar

```swift
import SwiftUI

struct PoorTabs: View {
    @State private var draft = ""
    var body: some View {
        TabView {
            Tab("Inbox", systemImage: "tray") {
                Text("Your messages")
            }
            Tab("New", systemImage: "square.and.pencil") {
                TextField("Message", text: $draft)
                    .padding()
            }
        }
    }
}

#Preview { PoorTabs() }
```

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares PoorTabs.
- WHAT — **Imports SwiftUI for TabView, Tab, and TextField.**
- WHERE — Top of the file, above PoorTabs.
- WHEN — At compile time, before PoorTabs is type-checked.
- WHY — Every SwiftUI file imports the framework it uses.

3 `struct PoorTabs: View {`

- WHO — You declare it; SwiftUI draws it.
- WHAT — **A two-tab app whose second tab is the action of writing a message.**
- WHERE — The root view of the app.
- WHEN — Created at launch, when the App struct shows PoorTabs as its first screen.
- WHY — This is the version to critique: it mixes a place (Inbox) with an action (New).

4 `@State private var draft = ""`

- WHO — SwiftUI state owned by this view.
- WHAT — **Holds the text typed in the New tab.**
- WHERE — SwiftUI-managed storage that lives as long as the tab view.
- WHEN — Updated on every keystroke.
- WHY — The draft survives tab switches, so a half-written message silently waits in the tab, a sign that this is a task, not a place.

5 `var body: some View {`

- WHO — SwiftUI reads body on PoorTabs after each keystroke; your code returns the TabView.
- WHAT — **Describes the tab view.**
- WHERE — Inside PoorTabs, below the draft state.
- WHEN — At launch and after each change to draft.
- WHY — The structure is the problem here, not the code.

6 `TabView {`

WHO SwiftUI's TabView, which draws the two-item tab bar.
WHAT **Creates a tab bar with two tabs.**
WHERE Root of the screen.
WHEN Built at launch, and kept alive while the person switches tabs.
WHY Tabs promise peer places people can switch between freely.

7 `Tab("Inbox", systemImage: "tray") {`

WHO SwiftUI's TabView, building its first tab item from this Tab.
WHAT **A proper tab: the Inbox section.**
WHERE First item in the tab bar.
WHEN Shown when the Inbox tab is selected, which is the default at launch.
WHY Inbox is a place people return to, exactly what tabs are for.

8 `Text("Your messages")`

WHO SwiftUI's text renderer, drawing the Inbox tab's placeholder.
WHAT **Placeholder for the inbox list.**
WHERE Inside the Inbox tab.
WHEN Shown while Inbox is selected.
WHY Keeps the example short; the content is not the issue.

10 `Tab("New", systemImage: "square.and.pencil") {`

WHO SwiftUI's TabView, building its second tab item from this Tab.
WHAT **A tab whose only purpose is to start writing a message.**
WHERE Second item in the tab bar.
WHEN Shown when the New tab is selected, with the draft still in its field.
WHY This is the anti-pattern: an action in the tab bar has no Cancel, no Send, and no clear end.

11 `TextField("Message", text: $draft)`

WHO SwiftUI, and the person typing.
WHAT **The editor shown when the New tab is selected.**
WHERE Inside the New tab.
WHEN Shown while New is selected.
WHY Switching tabs neither cancels nor sends; it just hides the draft, which surprises people when they come back.

12 `.padding()`

WHO The padding modifier on the TextField.
WHAT **Adds default padding around the text field.**
WHERE Around the TextField, inside the New tab.
WHEN At every layout of the New tab, after the field measures itself.
WHY Keeps the field off the screen edges; even polished spacing cannot fix the wrong pattern.

18 `#Preview { PoorTabs() }`

WHO Xcode previews and the live phone on this page.
WHAT **Renders the two-tab version.**
WHERE At file scope, after the closing brace of PoorTabs.
WHEN Whenever the code changes.
WHY Type a draft, switch tabs, and come back: the leftover draft is the bug.

B: After: a toolbar button and a sheet

```swift
import SwiftUI

struct BetterInbox: View {
    @State private var isComposing = false
    var body: some View {
        NavigationStack {
            Text("Your messages")
                .navigationTitle("Inbox")
                .toolbar {
                    Button("Compose") { isComposing = true }
                }
                .sheet(isPresented: $isComposing) {
                    Text("New Message")
                }
        }
    }
}
#Preview { BetterInbox() }
```

1 `import SwiftUI`

WHO	The Swift compiler, while it builds the file that declares BetterInbox.
WHAT	**Imports SwiftUI for NavigationStack, toolbar, and sheet.**
WHERE	Top of the file, above BetterInbox.
WHEN	At compile time, before BetterInbox is type-checked.
WHY	Every SwiftUI file imports the framework it uses.

3 `struct BetterInbox: View {`

WHO	You declare it; SwiftUI draws it.
WHAT	**The inbox screen, with the compose action in its toolbar.**
WHERE	A screen that would sit inside the Inbox tab.
WHEN	Created when the Inbox appears.
WHY	The action now belongs to the screen it starts from.

4 `@State private var isComposing = false`

WHO	SwiftUI state owned by this view.
WHAT	**Tracks whether the compose sheet is showing.**
WHERE	SwiftUI-managed storage for this view.
WHEN	Set to true on tap; SwiftUI sets it back to false when the sheet is dismissed.
WHY	Presentation is driven by state: flip the Bool and SwiftUI shows or hides the sheet.

5 `var body: some View {`

WHO	SwiftUI reads body on BetterInbox whenever isComposing changes; your code returns the stack.
WHAT	**Describes the inbox, its toolbar, and the sheet.**
WHERE	Inside BetterInbox, below the isComposing state.
WHEN	At first display and whenever isComposing changes.
WHY	Changing isComposing recomputes body, which presents or dismisses the sheet.

```
6  NavigationStack {
```

WHO SwiftUI's NavigationStack, which draws the bar that the title and toolbar fill.
WHAT **Gives the inbox a navigation bar for its title and toolbar.**
WHERE Root of the screen.
WHEN Created once when the inbox appears, and kept while the sheet comes and goes.
WHY The toolbar lives in the navigation bar, so the screen needs a stack even before it pushes anything.

```
7      Text("Your messages")
```

WHO SwiftUI's text renderer, drawing the inbox placeholder as the root screen.
WHAT **Placeholder for the inbox list.**
WHERE Root screen of the stack.
WHEN At every layout of the root screen, including while the sheet is up.
WHY Keeps the example short and focused on the pattern.

```
8      .navigationTitle("Inbox")
```

WHO SwiftUI's navigation bar, reading this modifier on the placeholder Text.
WHAT **Titles the screen Inbox.**
WHERE The navigation bar of this screen.
WHEN When the screen appears.
WHY The title confirms where people are when the sheet closes and returns them here.

```
9      .toolbar {
```

WHO SwiftUI's toolbar system, reading this modifier on the placeholder Text.
WHAT **Adds items to the navigation bar of this screen.**
WHERE The top bar, on the trailing side by default.
WHEN When the screen appears.
WHY Actions that belong to a screen go in its toolbar, where people look for them.

```
10         Button("Compose") { isComposing = true }
```

WHO SwiftUI, and the person tapping.
WHAT **A toolbar button that starts the task by setting isComposing to true.**
WHERE In the navigation bar.
WHEN The action runs on tap.
WHY The label is a verb that names the action; many apps show the square.and.pencil symbol for this same job.

```
12     .sheet(isPresented: $isComposing) {
```

WHO SwiftUI's presentation system, reading isComposing through the binding.
WHAT **Presents its content as a sheet that slides up over the inbox while isComposing is true.**
WHERE Attached to the inbox screen, which stays visible at the edges behind the sheet.
WHEN As soon as isComposing becomes true; it closes when the person swipes down or code sets the value to false.
WHY A sheet says "a separate task is in progress", and dismissing it returns people exactly where they were.

```
13         Text("New Message")
```

WHO SwiftUI's text renderer, drawing the sheet's stand-in content.
WHAT **Stand-in content for the compose sheet.**
WHERE Inside the sheet's content closure, as its only view.
WHEN Drawn while the sheet is showing.
WHY The next step designs the real sheet with Cancel and Save; here, swipe the sheet down to dismiss it.

```
18 #Preview { BetterInbox() }
```

WHO Xcode previews and the live phone on this page.
WHAT **Renders the inbox with its compose button.**
WHERE At file scope, after the closing brace of BetterInbox.
WHEN Whenever the code changes.
WHY Tap Compose, dismiss the sheet, and notice you land back on the inbox.

> **Takeaway**
>
> Tabs are places people return to; actions start tasks. Put an action in the toolbar of the screen it belongs to and present the task as a sheet with a clear way out, so finishing or abandoning it returns people exactly where they were.

CODE · LIVE SWIFTUI PREVIEW

Inside the sheet: Cancel and Save

A sheet is a small, self-contained task, so it brings its own `NavigationStack` for a title and toolbar. Cancel sits on the leading side, the confirming action on the trailing side, and the confirm button stays disabled until the form is valid. Type a destination and watch Save come alive.

```swift
import SwiftUI
struct NewTripForm: View {
    @Environment(\.dismiss) private var dismiss
    @State private var destination = ""
    var body: some View {
        NavigationStack {
            Form { TextField("Destination", text: $destination) }
                .navigationTitle("New Trip")
                .navigationBarTitleDisplayMode(.inline)
                .toolbar {
                    ToolbarItem(placement: .cancellationAction) {
                        Button("Cancel") { dismiss() }
                    }
                    ToolbarItem(placement: .confirmationAction) {
                        Button("Save") { dismiss() }
                            .disabled(destination.isEmpty)
                    }
                }
        }
    }
}
#Preview { NewTripForm() }
```

Line by line

```
1 import SwiftUI
```

WHO The Swift compiler, while it builds the file that declares NewTripForm.
WHAT **Imports SwiftUI for Form, TextField, toolbar, and the dismiss action.**
WHERE Top of the file, above NewTripForm.
WHEN At compile time, before NewTripForm is type-checked.
WHY Every SwiftUI file imports the framework it uses.

```
2  struct NewTripForm: View {
```

WHO You declare it; a presenting screen shows it in .sheet.
WHAT **The content of a New Trip sheet: one field plus Cancel and Save.**
WHERE Its own view type, separate from the screen that presents it.
WHEN Created each time the sheet is presented.
WHY Designing the sheet as its own view lets any screen that adds a trip reuse it.

```
3  @Environment(\.dismiss) private var dismiss
```

WHO SwiftUI's environment, which supplies a dismiss action.
WHAT **Reads an action that closes whatever presented this view, here the sheet.**
WHERE Stored in this view; the value comes from the presentation that contains it.
WHEN Read when the view is created; called when the person taps Cancel or Save.
WHY The sheet can close itself without knowing who opened it or which Bool controls it.

```
4  @State private var destination = ""
```

WHO SwiftUI state owned by this view.
WHAT **Holds the destination typed so far.**
WHERE SwiftUI-managed storage for this sheet.
WHEN Updated on every keystroke; discarded when the sheet closes.
WHY Save's enabled state depends on it, so it must be state that SwiftUI watches.

```
5  var body: some View {
```

WHO SwiftUI reads body on NewTripForm after each keystroke; your code returns the stack.
WHAT **Describes the form and its toolbar.**
WHERE Inside NewTripForm, below the dismiss action and the destination state.
WHEN At first display and after every keystroke.
WHY Each keystroke recomputes body, which re-evaluates whether Save is enabled.

```
6  NavigationStack {
```

WHO SwiftUI's NavigationStack, which gives the sheet its own bar.
WHAT **Gives the sheet its own navigation bar for a title and buttons.**
WHERE Root of the sheet.
WHEN Created when the sheet appears.
WHY A sheet is a separate context; its own stack keeps its title and actions independent of the screen behind it.

```
7  Form { TextField("Destination", text: $destination) }
```

WHO SwiftUI's Form and TextField, and the person typing.
WHAT **A form with one text field bound to destination.**
WHERE The body of the sheet.
WHEN At every layout; the field updates destination as people type.
WHY Form gives the standard grouped style for data entry, so the sheet looks like every other iOS editor.

```
8  .navigationTitle("New Trip")
```

WHO SwiftUI's navigation bar, reading this modifier on the Form.
WHAT **Titles the sheet New Trip.**
WHERE Center of the sheet bar.
WHEN When the sheet appears.
WHY The title names the task, so people know exactly what they are creating.

```
9  .navigationBarTitleDisplayMode(.inline)
```

WHO SwiftUI's navigation bar, reading this modifier on the Form.
WHAT **Uses the small inline title instead of a large one.**
WHERE The sheet's own navigation bar, not the bar of the screen behind it.
WHEN When the sheet appears.
WHY Sheets use the compact inline title so the title and both buttons share one bar.

```
10 .toolbar {
```

WHO SwiftUI's toolbar system, reading this modifier on the Form.
WHAT **Adds the sheet's two actions to its bar.**
WHERE The top bar of the sheet.
WHEN When the sheet appears.
WHY A modal task needs its controls in a fixed, predictable spot.

```
11 ToolbarItem(placement: .cancellationAction) {
```

WHO SwiftUI, placing the item by its role.
WHAT **Puts its button in the standard spot for canceling: the leading side of the bar on iPhone.**
WHERE Leading edge of the sheet bar.
WHEN When the sheet appears.
WHY Semantic placements put buttons where iOS users expect them, and the system styles them correctly.

```
12 Button("Cancel") { dismiss() }
```

WHO The person tapping, then SwiftUI.
WHAT **Closes the sheet and discards the typed destination.**
WHERE Leading side of the bar.
WHEN When the person taps it.
WHY Every modal task needs an obvious way out; Cancel promises that nothing changes.

```
14 ToolbarItem(placement: .confirmationAction) {
```

WHO SwiftUI, placing the item by its role.
WHAT **Puts its button in the spot for completing the task: the trailing side of the bar.**
WHERE Trailing edge of the sheet bar.
WHEN When the sheet appears.
WHY iOS shows the confirming action on the trailing side, in a bolder style, so people find it without searching.

```
15 Button("Save") { dismiss() }
```

WHO The person tapping, then SwiftUI.
WHAT **Closes the sheet; a real app would store the trip first.**
WHERE Trailing side of the bar.
WHEN When the person taps it, if it is enabled.
WHY The label names the result with a verb, such as Save, Add, or Send, rather than a vague OK.

```
16 .disabled(destination.isEmpty)
```

WHO The disabled modifier on the Save button, reading destination.
WHAT **Grays out Save and ignores taps while the field is empty.**
WHERE The Save button only.
WHEN Re-evaluated every time body runs, so after every keystroke.
WHY Disabling an action until the input is valid prevents an error message later; the feedback is immediate and visible.

```
22  #Preview { NewTripForm() }
```

WHO	Xcode previews and the live phone on this page.
WHAT	**Renders the sheet content on its own, as if presented.**
WHERE	At file scope, after the closing brace of NewTripForm.
WHEN	Whenever the code changes.
WHY	Previewing the sheet alone lets you polish it without tapping through the app each time.

Try it

1. Type a destination and watch Save become enabled.
2. Swap .cancellationAction and .confirmationAction. Where do the buttons go, and why is that confusing?
3. Change "Save" to "OK". Which label tells people more about what will happen?

CONCEPT

Every screen needs a way back

People explore with confidence only when they know they can return. Getting back is part of the design, not an afterthought.

Back button and edge swipe	In a stack, the system back button in the top leading corner and a swipe from the left edge both return one level; pressing and holding the back button jumps further. Never hide it or replace it with a custom control that breaks the swipe.
Tabs remember	Each tab keeps its own path and scroll position. Switching tabs must never reset what people were doing, so give each tab its own NavigationStack.
Sheets end clearly	Cancel leaves without changes; Save or Done commits. Swiping a sheet down acts like Cancel, so when there are unsaved edits, block the swipe with .interactiveDismissDisabled and ask before discarding.
Avoid traps	Avoid opening a sheet from a sheet, and never hide top-level sections in a slide-out menu where nobody finds them.

A common bug

Wrapping the whole `TabView` in one `NavigationStack` gives every tab a single shared path: pushing a detail screen covers the tab bar, and the tabs fight over one title and toolbar. Put a stack inside each tab instead.

CODE · READ-ALONG

The iOS 26 tab bar

In iOS 26 and later the tab bar floats above the content as Liquid Glass. A search tab sits apart at the trailing end, and the bar can shrink while people scroll down to read. Each tab here owns its own `NavigationStack`. These modifiers are not in the live preview, so this example is explained rather than run.

```swift
import SwiftUI
struct RootTabs: View {
    @State private var query = ""
    var body: some View {
        TabView {
            Tab("Recipes", systemImage: "fork.knife") {
                NavigationStack {
                    List(1...30, id: \.self) { Text("Recipe \($0)") }
                        .navigationTitle("Recipes")
                }
            }
            Tab(role: .search) {
                NavigationStack {
                    Text("Results for \(query)")
                        .navigationTitle("Search")
                }
                .searchable(text: $query)
            }
        }
        .tabBarMinimizeBehavior(.onScrollDown)
    }
}
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares RootTabs.
- WHAT — **Imports SwiftUI, including the iOS 26 tab bar APIs used below.**
- WHERE — Top of the file, above RootTabs.
- WHEN — At compile time, before RootTabs is type-checked.
- WHY — Tab roles and the minimize behavior are part of SwiftUI in iOS 26 and later.

2 `struct RootTabs: View {`

- WHO — You declare it; your App shows it at launch.
- WHAT — **The root of an app in the iOS 26 style: a content tab and a search tab.**
- WHERE — Top of the view hierarchy.
- WHEN — Created at launch, when the App struct shows RootTabs as its first screen.
- WHY — Declaring the whole tab structure at the root makes the app map easy to read.

3 `@State private var query = ""`

- WHO — SwiftUI state owned by this view.
- WHAT — **Holds the text typed into the search field.**
- WHERE — SwiftUI-managed storage for the root view.
- WHEN — Updated on every keystroke in the search field.
- WHY — The search field needs a binding to read and write the query as people type.

4 `var body: some View {`

- WHO — SwiftUI reads body on RootTabs at launch and after each keystroke in the search field.
- WHAT — **Describes the tab view and both tabs.**
- WHERE — Inside RootTabs, below the query state.
- WHEN — At launch and when query changes.
- WHY — The structure is declared once; the system handles the glass effects and animation.

```
5  TabView {
```

WHO SwiftUI's TabView, which draws the floating glass tab bar on iOS 26 and later.
WHAT **Creates the floating Liquid Glass tab bar on iOS 26 and later.**
WHERE Root of the screen, with the bar floating near the bottom edge.
WHEN Built at launch, and kept alive while the person switches tabs.
WHY Using the standard container gives you the new look for free; a hand-made tab bar would miss it.

```
6  Tab("Recipes", systemImage: "fork.knife") {
```

WHO SwiftUI's TabView, building its first tab item from this Tab.
WHAT **The first, regular tab.**
WHERE Leading group of the tab bar.
WHEN Its content shows when it is selected.
WHY Regular tabs stay grouped together, separate from search.

```
7  NavigationStack {
```

WHO SwiftUI's NavigationStack, owned by the Recipes tab alone.
WHAT **Gives the Recipes tab its own navigation stack.**
WHERE Inside the Tab, so the stack belongs to this tab only.
WHEN Created with the tab; it keeps its path while other tabs are shown.
WHY Each tab keeps its own path, so switching to Search and back leaves people where they were in Recipes.

```
8  List(1...30, id: \.self) { Text("Recipe \($0)") }
```

WHO SwiftUI's List, drawing thirty rows from the range 1...30.
WHAT **Thirty placeholder rows, enough to scroll.**
WHERE Root screen of the Recipes stack.
WHEN At every layout of the Recipes tab, and as the list scrolls.
WHY Scrolling content is what lets the tab bar minimize, so the example needs a list taller than the screen.

```
9  .navigationTitle("Recipes")
```

WHO SwiftUI's navigation bar inside the Recipes stack, reading this modifier on the List.
WHAT **Titles the Recipes root screen.**
WHERE Inside the Recipes stack.
WHEN When the tab is shown.
WHY Each stack titles its own screens, which only works because each tab has its own stack.

```
12 Tab(role: .search) {
```

WHO SwiftUI and the iOS 26 tab bar.
WHAT **Declares a tab with the search role; the system supplies its Search title and magnifying glass symbol.**
WHERE Drawn apart from the other tabs, at the trailing end of the tab bar.
WHEN Selected like any other tab; its content shows when people tap it.
WHY Marking the role lets the system place and style search the same way in every app, instead of you imitating it.

```
13 NavigationStack {
```

WHO SwiftUI's NavigationStack, owned by the search tab alone.
WHAT **Gives the search tab its own stack.**
WHERE Inside the search Tab.
WHEN Created with the tab.
WHY Search results usually drill into details, so this tab needs its own back navigation too.

```
14  Text("Results for \(query)")
```

WHO SwiftUI's text renderer, reading query to build the results placeholder.
WHAT **Stand-in for real results built from the query.**
WHERE Root screen of the search stack.
WHEN Redrawn as query changes.
WHY Results that update while people type give immediate feedback.

```
15  .navigationTitle("Search")
```

WHO SwiftUI's navigation bar inside the search stack, reading this modifier on the Text.
WHAT **Titles the search screen.**
WHERE Inside the search stack.
WHEN When the tab is shown.
WHY A clear title confirms which tab people are in.

```
17  .searchable(text: $query)
```

WHO SwiftUI and the system search field.
WHAT **Adds a system search field bound to query.**
WHERE Attached to the search tab content.
WHEN Shown with the tab; it writes to query on every keystroke.
WHY The system field brings dictation, a clear button, and keyboard behavior people already know.

```
20  .tabBarMinimizeBehavior(.onScrollDown)
```

WHO SwiftUI and the iOS 26 tab bar.
WHAT **Lets the tab bar shrink to a compact form while people scroll down and expand again when they scroll up.**
WHERE On the TabView, so it applies to every tab.
WHEN While content in a tab scrolls.
WHY Content gets more of the screen while navigation stays one gesture away, the content-first idea behind the new design.

INTERACTIVE

Pick the pattern

Eight real product scenarios. For each one, pick the navigation pattern you would ship. You get instant feedback with the reasoning, the SwiftUI types to use, and a wireframe of your pick beside the recommended one. Your first pick counts toward the score; after it, open the other options to see why they fit worse.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. A podcast app has four areas people switch between all day: Listen Now, Library, Browse, and Search. Library drills into shows and episodes. Which structure fits?
   A. One NavigationStack whose root screen lists the four areas
   B. A TabView with one Tab per area, each with its own NavigationStack
   C. A sheet for each area, opened from a home screen
   D. A slide-out menu from the left edge listing the four areas

2. People tap + to create a calendar event and then either save it or give up. How should the editor appear?
   A. Pushed onto the navigation stack, with Back to leave
   B. As its own tab named Add, next to the calendar tab
   C. As a sheet, with Cancel on the leading side and Add trailing
   D. Replacing the calendar screen until the event has been saved

3. A student wraps the entire `TabView` in one `NavigationStack`. What will testers most likely notice?
   A. Pushed screens cover the tab bar, and the tabs share one title and toolbar
   B. Nothing; this is the structure Apple recommends for tabbed apps with detail screens
   C. The project no longer compiles, because TabView cannot sit inside a stack
   D. Tab badges stop appearing, because the stack takes over the navigation bar

4. In a New Trip sheet, where do Cancel and Save belong, and when should Save be enabled?
   A. Both at the bottom of the form, enabled from the moment it opens
   B. Save on the leading side so it is seen first, enabled from the start
   C. Cancel on the trailing side and Save on the leading side, enabled after one keystroke
   D. Cancel in .cancellationAction, Save in .confirmationAction, disabled until valid

5. In Settings, General leads to About, and each tap shows more specific detail about the item above it. Which pattern is this, and how do people go back?
   A. Sheets, each closed with a Done button in its toolbar
   B. A NavigationStack; Back or an edge swipe returns one level
   C. A TabView with General and About as peer tabs, switched by tapping
   D. One long scrolling screen, with no back navigation of any kind

Answers: Choosing a Navigation Pattern

1. **B. A TabView with one Tab per area, each with its own NavigationStack** Four peer areas used constantly call for tabs, and Library needs drill-down, so each tab gets its own stack. A single stack would bury the peers behind back buttons, sheets are for short tasks, and a slide-out menu hides the top-level sections where people rarely find them.
2. **C. As a sheet, with Cancel on the leading side and Add trailing** Creating an event is a self-contained task with a clear end, which is what sheets are for. Pushing it makes it look like a deeper level, so Back is ambiguous: saved or discarded? A tab turns an action into a place, and replacing the screen removes the way out.
3. **A. Pushed screens cover the tab bar, and the tabs share one title and toolbar** It compiles, which is why the mistake is common. With one stack outside the tabs, every push happens above the tab bar and all tabs share a single path and bar. The fix is a `NavigationStack` inside each `Tab`, so each section keeps its own history.
4. **D. Cancel in .cancellationAction, Save in .confirmationAction, disabled until valid** Semantic placements put Cancel on the leading side and the confirming action on the trailing side, where iOS users expect them. Disabling Save until the input is valid prevents errors instead of reporting them. Enabling it after one keystroke would still allow invalid input.
5. **B. A NavigationStack; Back or an edge swipe returns one level** General to specific is a hierarchy, so it is a stack. The system back button returns people one level at a time, and the edge swipe is the gesture they already know. Sheets would stack modal on modal, and tabs would treat a child as if it were a peer.

States, Feedback, and Clear Copy

CONCEPT

Every screen has five states

A screen is never just one picture. Before data arrives it is **loading**. With nothing saved yet it is **empty**. When a request fails it shows an **error**. With some data missing it is **partial**. Only when everything works is it **ideal**.

Teams often design only the ideal state, so the other four ship as a blank screen, a spinner that never stops, or a raw error code. Switch between the states on the right: each one still says where you are and offers a next step.

> **Design the empty state first**
>
> Every new user meets the empty state before anything else. Treat it as onboarding: say what will appear here and offer the action that fills it.

Interactive figure in the HTML version: design states switcher.

CODE · LIVE SWIFTUI PREVIEW

An empty state that invites action

An empty list should explain itself and offer the next step. `ContentUnavailableView` is the system layout for this: a symbol, a short title and a description, centered and sized with Dynamic Type. Tap **Plan a Trip** in the preview to leave the empty state.

```swift
import SwiftUI

struct TripsView: View {
    @State private var trips: [String] = []

    var body: some View {
        if trips.isEmpty {
            VStack(spacing: 16) {
                ContentUnavailableView("No Trips Yet",
                    systemImage: "airplane",
                    description: Text("Trips you plan appear here."))
                Button("Plan a Trip") { trips.append("Lisbon") }
                    .buttonStyle(.borderedProminent)
            }
        } else {
            List(trips, id: \.self) { Text($0) }
        }
    }
}

#Preview { TripsView() }
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, while it builds the app.
WHAT	**Makes SwiftUI types such as ContentUnavailableView, Button, List and @State available in this file.**
WHERE	The first line of the file; it applies to every declaration below it in this file only.
WHEN	At compile time, before any code runs.
WHY	SwiftUI is a framework, not part of the Swift language, so each file that draws UI imports it.

```
3  struct TripsView: View {
```

WHO	Your code declares the type; SwiftUI creates and redraws it.
WHAT	**Declares a screen named TripsView that conforms to View, so it must provide a body.**
WHERE	At the top level of the file, as a value type (struct) that SwiftUI can recreate cheaply.
WHEN	Declared at compile time; SwiftUI creates instances whenever the parent view needs one.
WHY	Naming the screen after its feature (Trips) keeps the design vocabulary and the code vocabulary the same.

```
4  @State private var trips: [String] = []
```

WHO	SwiftUI owns the storage behind @State; this view reads and writes it.
WHAT	**Holds the list of trip names and starts it empty, which is exactly the state a first-time user sees.**
WHERE	In storage SwiftUI keeps alive outside the struct, private to TripsView.
WHEN	Created the first time the view appears; every change triggers a new body.
WHY	Starting from an empty array forces you to design the empty state instead of assuming there is always data.

```
6  var body: some View {
```

WHO	SwiftUI calls this property; your code returns the description of the screen.
WHAT	**Describes what TripsView shows for the current value of trips.**
WHERE	Inside TripsView; it runs on the main actor, where all UI work happens.
WHEN	On first appearance and again every time trips changes.
WHY	Because body is recomputed from state, the screen can never show an empty list and a full list at once.

```
7  if trips.isEmpty {
```

WHO	Your view logic, evaluated by SwiftUI during body.
WHAT	**Chooses the empty-state layout when there are no trips.**
WHERE	Inside body; SwiftUI turns the if into two alternative branches of the view tree.
WHEN	Every time body runs.
WHY	An explicit branch makes the empty state a designed screen, not an accident of an empty List.

```
8  VStack(spacing: 16) {
```

WHO	The VStack inside the empty branch, which stacks the empty-state view and the button.
WHAT	**Stacks the explanation and the button vertically with 16 points between them.**
WHERE	Inside the empty branch only.
WHEN	Laid out whenever the empty branch is on screen.
WHY	16 pt is on the 8-pt spacing rhythm and keeps the action visually grouped with the message it belongs to.

```
9  ContentUnavailableView("No Trips Yet",
```

WHO	SwiftUI, which provides the standard empty-state layout (iOS 17 and later).
WHAT	**Starts a centered empty state whose title is “No Trips Yet”.**
WHERE	The first child of the VStack.
WHEN	Drawn while trips is empty.
WHY	The system view matches Apple’s own apps, scales with Dynamic Type, and is read sensibly by VoiceOver for free.

```
10  systemImage: "airplane",
```

WHO SF Symbols, the icon set built into iOS.
WHAT **Shows a large airplane symbol above the title.**
WHERE The image slot of ContentUnavailableView.
WHEN Drawn with the empty state.
WHY A symbol that names the feature (trips, so an airplane) tells people where they are faster than words.

```
11  description: Text("Trips you plan appear here."))
```

WHO SwiftUI renders the Text in a secondary style under the title.
WHAT **Explains in one sentence what this screen will contain.**
WHERE The description slot, below the title.
WHEN Drawn with the empty state.
WHY Good empty-state copy says what will appear, not “Nothing here”. It turns a dead end into a promise.

```
12  Button("Plan a Trip") { trips.append("Lisbon") }
```

WHO The user tapping; SwiftUI runs the closure.
WHAT **Adds a trip, which makes trips non-empty and swaps the screen to the list.**
WHERE Below the ContentUnavailableView, in the same VStack.
WHEN Each time the button is tapped.
WHY The label is a verb phrase naming the outcome. An empty state should always offer the action that fills it.

```
13  .buttonStyle(.borderedProminent)
```

WHO SwiftUI’s built-in button styles.
WHAT **Draws the button as a filled capsule in the app’s tint color.**
WHERE Applied to the Plan a Trip button only.
WHEN Whenever the button is drawn.
WHY The prominent style marks the single primary action on the screen; one prominent button per screen keeps the hierarchy clear.

```
15  } else {
```

WHO Your view logic, evaluated by SwiftUI as the other branch of the if.
WHAT **Starts the branch used once at least one trip exists.**
WHERE Inside body, as the alternative to the empty branch.
WHEN Whenever trips is not empty.
WHY The ideal state and the empty state are different screens, so they get different layouts.

```
16  List(trips, id: \.self) { Text($0) }
```

WHO SwiftUI List, backed by the system table on iOS.
WHAT **Shows one row per trip name, identifying rows by the string itself.**
WHERE The else branch, filling the screen.
WHEN Drawn whenever trips has items, and updated as trips changes.
WHY A standard List brings scrolling, separators, Dynamic Type and VoiceOver support without extra work.

```
21  #Preview { TripsView() }
```

WHO Xcode’s preview system (and the phone on this page).
WHAT **Renders TripsView so you can see and use it while editing.**
WHERE In the canvas, not in the shipped app.
WHEN Whenever the file changes.
WHY Previewing the empty state directly is how designers and developers catch blank screens before users do.

Try it

1. Change the description to “No trips.” Which version tells a new user more?
2. Start trips with ["Kyoto"] instead of []. Which state does the preview open in now?

CODE · LIVE SWIFTUI PREVIEW

One enum for loading, error and loaded

A screen that loads data moves through states. Modeling them as one `enum` makes impossible combinations, such as a spinner on top of an error, impossible to write. The preview starts in the failed state so you can try recovery: tap **Try Again**, watch the progress indicator, then see the result.

```swift
import SwiftUI

enum LoadState { case loading, failed, loaded }

struct TripsScreen: View {
    @State private var state = LoadState.failed
    var body: some View {
        switch state {
        case .failed:
            VStack(spacing: 12) {
                Text("Couldn’t load your trips.").font(.headline)
                Button("Try Again") { state = .loading }
            }
        case .loading:
            ProgressView("Loading your trips…")
                .task {
                    try? await Task.sleep(for: .seconds(1))
                    state = .loaded
                }
        case .loaded:
            Label("3 trips planned", systemImage: "airplane")
        }
    }
}

#Preview { TripsScreen() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares LoadState and TripsScreen.
- WHAT — **Brings in SwiftUI so the file can use ProgressView, Button, Label and @State.**
- WHERE — Top of the file, above the enum and the struct.
- WHEN — At compile time, before LoadState and TripsScreen are type-checked.
- WHY — Every SwiftUI file imports the framework it draws with.

```
3  enum LoadState { case loading, failed, loaded }
```

WHO Your code defines the type; the compiler checks every use of it.
WHAT **Lists the three states this screen can be in, and nothing else.**
WHERE At file scope, so any view in the file could use it.
WHEN Defined at compile time.
WHY With separate isLoading and hasError Booleans, both could be true at once. One enum value means exactly one state on screen.

```
5  struct TripsScreen: View {
```

WHO Your code; SwiftUI creates and redraws it.
WHAT **Declares the screen that shows whichever state is current.**
WHERE File scope, as a View value type.
WHEN Created when the preview or a parent shows it.
WHY Keeping all three states in one view guarantees they share the same place in the app.

```
6  @State private var state = LoadState.failed
```

WHO SwiftUI stores it; the view reads and changes it.
WHAT **Holds the current state and starts in failed so the preview shows the error first.**
WHERE SwiftUI-managed storage private to TripsScreen.
WHEN Created on first appearance; each change re-runs body.
WHY A real app would start at .loading. Starting at .failed here lets you design and test the error path on purpose.

```
7  var body: some View {
```

WHO SwiftUI calls body on TripsScreen whenever state changes; your code returns one branch.
WHAT **Returns the view for the current state.**
WHERE Inside TripsScreen, on the main actor.
WHEN Every time state changes.
WHY Because the screen is a function of state, changing one value is all it takes to move between screens.

```
8  switch state {
```

WHO Swift's switch, which SwiftUI supports inside body.
WHAT **Picks exactly one branch for the current enum value.**
WHERE The root of body.
WHEN Each time body runs.
WHY switch must cover every case, so if you add a new state later the compiler tells you to design its screen.

```
9  case .failed:
```

WHO The switch inside body, matching the current state value.
WHAT **Begins the branch for a failed load.**
WHERE Inside the switch, as its first case.
WHEN When state is .failed.
WHY An error is a state worth designing, not an afterthought printed to the console.

```
10 VStack(spacing: 12) {
```

WHO The VStack inside the failed branch, which stacks the message and the retry button.
WHAT **Groups the message and the retry button vertically.**
WHERE Inside the failed branch.
WHEN Drawn while the load has failed.
WHY Keeping the explanation and the recovery action together follows the proximity principle: related things sit close.

```
11  Text("Couldn't load your trips.").font(.headline)
```

WHO — SwiftUI Text, styled with the headline text style.
WHAT — **States what happened in plain words, in semibold 17 pt at the default text size.**
WHERE — Top of the error group.
WHEN — Drawn while state is .failed.
WHY — Name the user's goal (their trips), not the technical cause. The compare step next shows how to add the fix.

```
12  Button("Try Again") { state = .loading }
```

WHO — The user taps; SwiftUI runs the closure.
WHAT — **Moves the screen to the loading state, which starts a new load.**
WHERE — Below the message, inside the error group.
WHEN — Each time the user taps it.
WHY — Every error state needs a way forward. "Try Again" is a verb phrase that says exactly what will happen.

```
14  case .loading:
```

WHO — The switch inside body, matching the current state value.
WHAT — **Begins the branch shown while data is on its way.**
WHERE — Inside the switch, as its second case.
WHEN — When state is .loading.
WHY — Showing progress immediately reassures people that the tap worked.

```
15  ProgressView("Loading your trips…")
```

WHO — SwiftUI's system progress indicator.
WHAT — **Shows a spinning indicator with a label that says what is loading.**
WHERE — Centered on screen in the loading branch.
WHEN — While state is .loading.
WHY — A labeled spinner is clearer than a bare one, and VoiceOver reads the label. Use ProgressView(value:) when you know how much is left.

```
16  .task {
```

WHO — SwiftUI starts and cancels this async task.
WHAT — **Runs the load when the progress view appears.**
WHERE — Attached to the ProgressView, so it lives exactly as long as the loading screen.
WHEN — Each time the loading branch appears; cancelled automatically if it disappears first.
WHY — Tying the work to the loading view means leaving the screen cancels the request instead of leaking it.

```
17  try? await Task.sleep(for: .seconds(1))
```

WHO — Swift concurrency, suspending this task without blocking the UI.
WHAT — **Waits one second to stand in for a real network request.**
WHERE — Inside the task, off the critical path of drawing.
WHEN — Right after the loading view appears.
WHY — Simulating a delay lets you see and judge the loading state; a real app would await URLSession here.

```
18  state = .loaded
```

WHO — Your code, back on the main actor.
WHAT — **Marks the load as finished, which swaps in the loaded branch.**
WHERE — Inside the task closure, after the sleep returns.
WHEN — After the simulated request completes.
WHY — Changing one enum value replaces the whole screen, so the spinner can never linger after the data arrives.

```
20  case .loaded:
```

WHO The switch inside body, matching the current state value.
WHAT **Begins the branch for the ideal state.**
WHERE Inside the switch, as its last case.
WHEN When state is .loaded.
WHY The ideal state is only one of three branches, a reminder of how much of design lives outside it.

```
21  Label("3 trips planned", systemImage: "airplane")
```

WHO SwiftUI Label, pairing an SF Symbol with text.
WHAT **Stands in for the real trip list.**
WHERE The loaded branch, the only view shown once state is .loaded.
WHEN After a successful load.
WHY In a full app this would be the List from the previous step; a Label keeps the example short.

```
26  #Preview { TripsScreen() }
```

WHO Xcode previews and this page's phone.
WHAT **Renders TripsScreen so you can walk through the states.**
WHERE At file scope, after the closing brace of TripsScreen; the canvas reads it, the app does not.
WHEN On every edit, when the canvas refreshes with TripsScreen in its failed state.
WHY Previews let you step through failure and recovery without a real network outage.

> **Try it**
>
> 1. Change LoadState.failed to LoadState.loading on the @State line. What does a first-time user see now?
> 2. Change .seconds(1) to .seconds(4). How does the screen feel while you wait, and why does the label help?

CONCEPT

Words are interface too

Labels, titles and messages are designed just like layout and color. Apple's writing guidance comes down to being clear, concise and helpful, in the voice of a calm expert who is on the user's side.

Buttons are verbs	Name the action: "Save Trip", "Delete Photo", "Try Again". "OK", "Yes" and "Submit" force people to reread the question to know what will happen.
Errors answer three questions	What happened, why (when it helps), and what to do now: "Couldn't Load Trips. Your iPhone is offline. Check Wi-Fi, then try again."
No blame, no codes	Write "Couldn't save the card" rather than "You entered invalid data". Keep codes such as -1009 for your logs, not for people.
One word per idea	If the tab says Trips, never call them Journeys elsewhere. Consistent terms make an app predictable.
Front-load meaning	Put the important words first and cut filler such as "please", "successfully" and "an error has occurred".

> **Read it out loud**
>
> If a sentence would sound strange said to a friend, rewrite it. Reading aloud also previews what VoiceOver users will hear.

COMPARE

Vague error, clear error

Both screens appear when the trip list fails to load because the phone is offline. Read each one as a user who does not know what “-1009” means. Every line is the same kind of SwiftUI; only the words and the emphasis change.

A: Before: vague copy

```swift
import SwiftUI

struct VagueError: View {
    var body: some View {
        VStack(spacing: 12) {
            Image(systemName: "exclamationmark.triangle")
                .font(.largeTitle)
            Text("Error").font(.headline)
            Text("Operation failed (-1009).")
            Button("OK") { }
        }
        .padding()
    }
}

#Preview { VagueError() }
```

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares VagueError.
- WHAT — **Imports SwiftUI for VStack, Image, Text and Button.**
- WHERE — Top of the file, above VagueError.
- WHEN — At compile time, before VagueError is type-checked.
- WHY — Both versions use exactly the same framework; the difference is design, not technology.

3 `struct VagueError: View {`

- WHO — Your code; SwiftUI renders it.
- WHAT — **Declares the error screen as first written.**
- WHERE — At file scope, as a top-level struct.
- WHEN — Created when the preview shows it.
- WHY — Showing the flawed version on its own makes its problems easy to see.

4 `var body: some View {`

- WHO — SwiftUI calls body on VagueError; your code returns the VStack.
- WHAT — **Describes the error screen.**
- WHERE — Inside VagueError, its only member.
- WHEN — Whenever the view is drawn.
- WHY — Same structure as the fixed version, so you can compare line by line.

5 `VStack(spacing: 12) {`

- WHO — The VStack, which stacks the triangle, the two texts and the button.
- WHAT — **Stacks the icon, texts and button vertically.**
- WHERE — The root of body, holding all four children.
- WHEN — On every layout pass.
- WHY — The layout is fine; the problems in this version are all in the words.

```
6   Image(systemName: "exclamationmark.triangle")
```

WHO SF Symbols, supplying the exclamationmark.triangle glyph to the Image.
WHAT **Shows a generic warning triangle.**
WHERE Top of the stack.
WHEN Whenever the error shows.
WHY A generic alarm symbol adds anxiety without information; it says “bad” but not what is wrong.

```
7   .font(.largeTitle)
```

WHO SwiftUI text styles, which also size symbols.
WHAT **Draws the symbol at the large title size (34 pt by default).**
WHERE Applied to the triangle only.
WHEN At layout time of the triangle, scaling with the text size setting.
WHY Making the least useful element the biggest one puts the hierarchy upside down.

```
8   Text("Error").font(.headline)
```

WHO SwiftUI's text renderer, drawing the one-word title.
WHAT **Uses the single word “Error” as the title.**
WHERE Below the symbol, as the second child of the VStack.
WHEN Whenever the error shows.
WHY The title is the most-read line. “Error” tells people only that something went wrong, which they already guessed.

```
9   Text("Operation failed (-1009).")
```

WHO SwiftUI Text, showing a message copied from a system error.
WHAT **Displays developer wording and a raw error code.**
WHERE Below the title, as the third child of the VStack.
WHEN Whenever the error shows.
WHY -1009 means “not connected to the internet”, but only to developers. Users need the meaning and the fix, not the code.

```
10  Button("OK") { }
```

WHO The user taps it; the empty closure does nothing.
WHAT **Offers a button that only acknowledges the problem.**
WHERE Bottom of the stack.
WHEN When tapped, which changes nothing.
WHY “OK” is not an action and offers no way forward; people are left on a dead-end screen.

```
12  .padding()
```

WHO The padding modifier on the VStack.
WHAT **Adds the default 16 pt margin around the group.**
WHERE Around the whole VStack.
WHEN At layout time, after the four children have measured themselves.
WHY Same spacing as the fixed version, so only the copy differs between the two phones.

```
16  #Preview { VagueError() }
```

WHO Xcode previews, and the simulated iPhone on this page, running VagueError.
WHAT **Renders the vague version.**
WHERE At file scope, after the closing brace of VagueError; the canvas reads it, the app does not.
WHEN On every edit, when the canvas refreshes with VagueError.
WHY Side-by-side previews make copy reviews as concrete as visual reviews.

B: After: clear copy

```swift
import SwiftUI

struct ClearError: View {
    var body: some View {
        VStack(spacing: 12) {
            Image(systemName: "wifi")
                .font(.largeTitle)
            Text("Couldn’t Load Trips").font(.headline)
            Text("You’re offline. Check Wi-Fi, then try again.")
            Button("Try Again") { }
                .buttonStyle(.borderedProminent)
        }
        .multilineTextAlignment(.center)
        .padding()
    }
}

#Preview { ClearError() }
```

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares ClearError.
- **WHAT** **Imports SwiftUI for the same building blocks.**
- **WHERE** Top of the file, above ClearError.
- **WHEN** At compile time, before ClearError is type-checked.
- **WHY** Clear copy needs no new APIs, only better decisions.

3 `struct ClearError: View {`

- **WHO** Your code; SwiftUI renders it.
- **WHAT** **Declares the rewritten error screen.**
- **WHERE** At file scope, as a top-level struct.
- **WHEN** Created when the preview shows it.
- **WHY** A separate name keeps both versions visible side by side.

4 `var body: some View {`

- **WHO** SwiftUI calls body on ClearError; your code returns the VStack.
- **WHAT** **Describes the improved error screen.**
- **WHERE** Inside ClearError, its only member.
- **WHEN** Whenever the view is drawn.
- **WHY** Same skeleton as before, so every difference is a deliberate design change.

5 `VStack(spacing: 12) {`

- **WHO** The VStack, which stacks the Wi-Fi symbol, the two texts and the button.
- **WHAT** **Stacks symbol, title, explanation and action.**
- **WHERE** The root of body, holding all four children.
- **WHEN** On every layout pass.
- **WHY** Reading order matches the questions people ask: what happened, what now, how.

```
6  Image(systemName: "wifi")
```

WHO SF Symbols, supplying the wifi glyph to the Image.
WHAT **Shows a Wi-Fi symbol that points at the cause.**
WHERE Top of the stack.
WHEN Whenever the error shows.
WHY A specific symbol explains the problem before a single word is read.

```
7  .font(.largeTitle)
```

WHO The font modifier on the Wi-Fi Image, which sizes the symbol.
WHAT **Sizes the symbol at the large title size.**
WHERE Applied to the Wi-Fi symbol.
WHEN At layout time of the Wi-Fi symbol, scaling with the text size setting.
WHY Here the large symbol earns its size because it carries meaning.

```
8  Text("Couldn’t Load Trips").font(.headline)
```

WHO SwiftUI's text renderer, drawing the title in the headline style.
WHAT **Titles the screen with what failed, in the user’s terms.**
WHERE Below the symbol, as the second child of the VStack.
WHEN Whenever the error shows.
WHY The title answers “what happened?” using the feature name people know: Trips.

```
9  Text("You’re offline. Check Wi-Fi, then try again.")
```

WHO SwiftUI's text renderer, drawing the explanation under the title.
WHAT **Explains the cause and the fix in one short sentence.**
WHERE Below the title, as the third child of the VStack.
WHEN Whenever the error shows.
WHY It answers “why?” and “what do I do?” without blame or jargon, so people can recover on their own.

```
10 Button("Try Again") { }
```

WHO The user taps it; a real app would reload here.
WHAT **Offers the recovery action as a verb phrase.**
WHERE Bottom of the stack.
WHEN When tapped; it would start a new load.
WHY The button names the outcome, so it can be pressed without rereading the message.

```
11 .buttonStyle(.borderedProminent)
```

WHO SwiftUI's button styling, applied to the Try Again button.
WHAT **Fills the button with the tint color.**
WHERE Applied to Try Again.
WHEN At draw time of the Try Again button, including its pressed state.
WHY Prominence marks the single most useful next step on the screen.

```
13 .multilineTextAlignment(.center)
```

WHO The multilineTextAlignment modifier on the VStack, reaching every Text inside.
WHAT **Centers every line of wrapped text in the stack.**
WHERE Applied to the whole VStack, so it reaches each Text.
WHEN At layout time, including when larger text sizes wrap the message.
WHY Centered short messages match the centered symbol and read as one calm block.

```
14  .padding()
```

WHO The padding modifier on the VStack.
WHAT **Adds the default 16 pt margin.**
WHERE Around the whole VStack.
WHEN At layout time, after the four children have measured themselves.
WHY Keeps wrapped text away from the screen edges at every text size.

```
18  #Preview { ClearError() }
```

WHO Xcode previews, and the simulated iPhone on this page, running ClearError.
WHAT **Renders the clear version.**
WHERE At file scope, after the closing brace of ClearError; the canvas reads it, the app does not.
WHEN On every edit, when the canvas refreshes with ClearError.
WHY Comparing previews is a quick copy review before the screen ever ships.

> **Takeaway**
>
> The layouts are almost identical; the words are not. The clear version names what failed in the user's terms, gives the cause and the fix, and offers a verb button that moves forward. The vague version shows a code and a button that goes nowhere.

CONCEPT

Feedback: confirm, undo, and haptics

Feedback answers the question behind every tap: did it work? Respond right away, even when the work itself takes longer.

Progress	Show a ProgressView for any wait people can notice, and say what is happening. Use ProgressView(value:) when you know how much is left.
Confirmation	Ask first only before actions that are destructive and hard to reverse, such as deleting an account. The confirming button repeats the verb ("Delete Trip") and uses the destructive role, so it is drawn in red.
Undo	For frequent, reversible actions such as deleting a message, act immediately and offer Undo. Register changes with UndoManager and iOS supports shake to undo and the three-finger swipe.
Haptics	The .sensoryFeedback modifier plays a short vibration for success, warning or error. It reinforces what the screen shows; it is never the only signal.

> **Confirmation fatigue**
>
> If every action asks "Are you sure?", people learn to tap Yes without reading, and the one dialog that matters gets ignored. Save confirmations for rare, irreversible moments.

CODE · READ-ALONG

Confirming a destructive action

Deleting a trip removes plans from every device, so it deserves one clear question. This view uses a confirmation dialog, the destructive button role, and a success haptic. It is shown, not run, because the page's phone does not simulate dialogs or vibration.

```swift
import SwiftUI

struct DeleteTripButton: View {
    let onDelete: () -> Void
    @State private var isAsking = false
    @State private var deletions = 0

    var body: some View {
        Button("Delete Trip", role: .destructive) { isAsking = true }
            .confirmationDialog("Delete this trip?",
                isPresented: $isAsking, titleVisibility: .visible) {
                Button("Delete Trip", role: .destructive) {
                    onDelete()
                    deletions += 1
                }
                Button("Cancel", role: .cancel) { }
            } message: {
                Text("Its plans and notes will be removed everywhere.")
            }
            .sensoryFeedback(.success, trigger: deletions)
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares DeleteTripButton.
- **WHAT** **Imports SwiftUI for Button, confirmationDialog and sensoryFeedback.**
- **WHERE** Top of the file, above DeleteTripButton.
- **WHEN** At compile time, before DeleteTripButton is type-checked.
- **WHY** All three feedback tools are plain SwiftUI modifiers; no extra framework is needed.

3 `struct DeleteTripButton: View {`

- **WHO** Your code; a trip detail screen would include it.
- **WHAT** **Declares a reusable delete button with its confirmation built in.**
- **WHERE** At file scope, as a top-level struct.
- **WHEN** Created when a parent view shows it.
- **WHY** Packaging the confirmation with the button means no screen can delete a trip without asking.

4 `let onDelete: () -> Void`

- **WHO** The parent view supplies this closure.
- **WHAT** **Stores the work to do once the user confirms.**
- **WHERE** An immutable property of the struct.
- **WHEN** Set when the parent creates the button; called only after confirmation.
- **WHY** The button owns the question; the parent owns the data. Keeping them separate keeps both simple.

5 `@State private var isAsking = false`

- **WHO** SwiftUI stores it; the dialog reads it.
- **WHAT** **Tracks whether the confirmation dialog is showing.**
- **WHERE** Private SwiftUI-managed storage tied to this button's place on screen.
- **WHEN** Becomes true on the first tap and false when the dialog closes.
- **WHY** Presentation in SwiftUI is state: the dialog shows exactly while this value is true.

```
6  @State private var deletions = 0
```

WHO SwiftUI stores it; sensoryFeedback watches it.
WHAT **Counts completed deletions, starting at zero.**
WHERE Private SwiftUI-managed storage, beside isAsking.
WHEN Increases by one each time a deletion is confirmed.
WHY Haptics fire when a trigger value changes; a counter changes on every success and never on Cancel.

```
8  var body: some View {
```

WHO SwiftUI calls body on DeleteTripButton whenever isAsking or deletions changes.
WHAT **Describes the button and the feedback attached to it.**
WHERE Inside DeleteTripButton, below the closure and the two state properties.
WHEN Whenever isAsking or deletions changes, and on first display.
WHY Dialogs and haptics are modifiers on the view, so they live next to the control that triggers them.

```
9  Button("Delete Trip", role: .destructive) { isAsking = true }
```

WHO The user taps; SwiftUI runs the closure.
WHAT **Shows a red “Delete Trip” button that asks for confirmation instead of deleting.**
WHERE The root view of body, typically at the bottom of a detail screen.
WHEN Each time the user taps the red button.
WHY The destructive role colors the button red and tells VoiceOver and the system this action destroys data.

```
10 .confirmationDialog("Delete this trip?",
```

WHO SwiftUI presents the system dialog.
WHAT **Attaches a confirmation dialog whose title asks one specific question.**
WHERE On iPhone it slides up from the bottom, inside the thumb’s reach.
WHEN Whenever isAsking becomes true.
WHY A specific question (“this trip”) is answered faster than a generic “Are you sure?”.

```
11 isPresented: $isAsking, titleVisibility: .visible) {
```

WHO SwiftUI, through a binding to your state.
WHAT **Connects the dialog to isAsking and forces the title to show.**
WHERE The remaining arguments of confirmationDialog, continued from the line above.
WHEN Read on every update; SwiftUI sets it back to false on dismissal.
WHY The binding lets SwiftUI close the dialog for you; a visible title gives the question context.

```
12 Button("Delete Trip", role: .destructive) {
```

WHO The user confirming; SwiftUI runs the closure.
WHAT **Adds the red confirming action to the dialog.**
WHERE Inside the dialog’s action list.
WHEN When the user taps it.
WHY Repeating the exact verb (“Delete Trip”, not “Yes”) lets people confirm without rereading the title.

```
13 onDelete()
```

WHO The parent's closure, stored in onDelete, called by your confirming action.
WHAT **Performs the real deletion.**
WHERE Inside the confirming action.
WHEN Only after the user confirms.
WHY The destructive work runs in exactly one place, after exactly one deliberate tap.

```
14 deletions += 1
```

WHO Your code, inside the confirming action, right after onDelete returns.
WHAT **Changes the trigger so the success haptic plays.**
WHERE After the deletion in the same action.
WHEN Immediately after onDelete returns.
WHY Feedback follows the result, so the vibration means “it is done”, not “you tapped”.

```
16 Button("Cancel", role: .cancel) { }
```

WHO SwiftUI places it; the user taps it to back out.
WHAT **Adds the safe way out, styled and positioned by the system.**
WHERE Inside the dialog; iPhone shows it separated from the other actions.
WHEN When the user changes their mind.
WHY The cancel role puts Cancel where people expect it; dialogs always need an obvious exit.

```
17 } message: {
```

WHO SwiftUI's confirmationDialog, which draws the message closure under the title.
WHAT **Starts the dialog’s explanatory message.**
WHERE Below the dialog title.
WHEN Shown whenever the dialog is.
WHY The message explains the consequence so the decision is informed, not a reflex.

```
18 Text("Its plans and notes will be removed everywhere.")
```

WHO SwiftUI Text inside the dialog.
WHAT **States exactly what will be lost.**
WHERE The message slot of the dialog, below its title and above its buttons.
WHEN While the dialog is visible.
WHY Naming the consequence (plans, notes, every device) is what makes a confirmation worth its interruption.

```
20 .sensoryFeedback(.success, trigger: deletions)
```

WHO SwiftUI and the iPhone’s haptic engine (iOS 17 and later).
WHAT **Plays a short success vibration each time deletions changes.**
WHERE Attached to the button.
WHEN Right after a confirmed deletion; never on Cancel.
WHY A haptic confirms the result without demanding attention, and the system skips it on devices without haptics.

CHECK YOURSELF

Check your understanding

1. A new user opens the Saved tab of a recipe app and sees only a white screen under the title. What is the best fix?
   A. Show a spinner until they save their first recipe
   B. Show a ContentUnavailableView saying what appears here, plus a Browse button
   C. Hide the Saved tab until it has at least one recipe in it
   D. Show an alert explaining what the tab is for when it opens
2. Which error message follows the guidance from this lesson?
   A. Error -1009: The operation couldn’t be completed. Please try again later.
   B. Oops! Something went wrong. Please try again in a few minutes.
   C. You entered an invalid card. Please check your input and resubmit.
   D. Couldn’t Add Card. The card has expired. Check it and try again.

3. In the LoadState example, the screen starts in `.failed` and the user taps Try Again. What happens?

   A. The loaded label appears immediately, with no spinner in between
   B. Nothing, because the .task already ran when the screen first appeared
   C. A labeled spinner appears, then about a second later the loaded label
   D. The error view stays on screen and a spinner appears on top of it

4. People delete individual messages many times a day. Which design is best?

   A. Delete immediately and offer Undo
   B. Ask “Are you sure?” before every deletion
   C. Delete immediately with no way back
   D. Make people type DELETE to confirm

5. What should the confirming button in a “Delete this trip?” dialog say?

   A. OK
   B. Yes, I’m Sure
   C. Delete Trip
   D. Continue

Answers: States, Feedback, and Clear Copy

1. **B. Show a ContentUnavailableView saying what appears here, plus a Browse button** An empty state should explain what will appear and offer the action that fills it. Hiding the tab is tempting but makes the feature undiscoverable and changes the navigation under people’s fingers. A spinner falsely says something is loading.
2. **D. Couldn’t Add Card. The card has expired. Check it and try again.** It says what happened, why, and what to do next, without blame. “Oops! Something went wrong.” sounds friendly but gives people nothing to act on; the first option leaks a developer code; the third blames the user.
3. **C. A labeled spinner appears, then about a second later the loaded label** Setting `.loading` swaps in the ProgressView branch. That view appears fresh, so its `.task` starts, waits, and sets `.loaded`. A task runs each time its view appears, not once per screen. Because state is one enum, the error and the spinner can never show together.
4. **A. Delete immediately and offer Undo** Frequent, reversible actions should be fast and undoable. Asking every time is tempting because it feels safe, but it causes confirmation fatigue, so people stop reading the dialogs that really matter.
5. **C. Delete Trip** A verb phrase that repeats the action can be pressed without rereading the question, and with the destructive role it is drawn in red. “Yes, I’m Sure” and “OK” depend on remembering exactly what was asked, and “Continue” hides what happens next.

Designing for Accessibility

CONCEPT

Accessibility serves everyone

The World Health Organization estimates that about 1.3 billion people, roughly one in six, live with a significant disability. Many more have a *temporary* limit (a broken wrist) or a *situational* one (bright sun, one hand holding a stroller, a loud train). An accessible app works for all of them.

iOS ships the assistive technology. Your job is to give it accurate information.

VoiceOver	A screen reader. People swipe right and left to move between elements and double-tap to activate; it speaks what your code describes.
Voice Control and Switch Control	Operate the phone by saying control names or by using one or more switches, so every control needs a name people can say.
Display settings	Larger Text (Dynamic Type), Bold Text, Increase Contrast, Reduce Transparency and Smart Invert change how your UI is drawn.
Reduce Motion	Asks apps to replace sliding, zooming and bouncing with gentler effects such as fades.

> **Standard controls are accessible for free**
>
> Button, Toggle, Slider, Stepper and List already report their role, state and actions. Most of your work is adding the words only you know.

CONCEPT

What VoiceOver says, in order

When the VoiceOver cursor lands on an element, iOS builds a sentence from four properties, always in the same order. Knowing the order tells you which words belong where.

1. Label	What it is, in a word or two: “Favorite”, “Spice level”. Set with .accessibilityLabel. Never add the type (“Favorite button”); the trait adds that.
2. Value	Its current state, when it has one: “Saved”, “Medium”, “50 percent”. Set with .accessibilityValue.
3. Traits	What kind of thing it is and how it behaves: button, heading, image, selected, adjustable. Standard controls set these for you.
4. Hint	The result of activating it, read after a pause: “Adds it to Favorites.” Start with a verb; people can turn hints off, so never hide essential information there.

> **Putting it together**
>
> A slider labeled Spice level with the value Medium is announced as “Spice level, Medium, adjustable”, followed by VoiceOver’s own instruction to swipe up or down. Without a label or value, the same slider says only “50 percent, adjustable”.

CODE · LIVE SWIFTUI PREVIEW

Label every icon-only button

An icon-only button looks obvious to the designer who drew it and means nothing to VoiceOver. Give each one a **label** (what it is), a **value** when it has state, and a **hint** when the result is not obvious. Tap the heart in the preview: its visible state and its spoken value change together.

```swift
import SwiftUI

struct RecipeActions: View {
    @State private var isFavorite = false

    var body: some View {
        HStack(spacing: 32) {
            Button { isFavorite.toggle() } label: {
                Image(systemName: isFavorite ? "heart.fill" : "heart")
            }
            .accessibilityLabel("Favorite")
            .accessibilityValue(isFavorite ? "Saved" : "Not saved")
            .accessibilityHint("Adds or removes it from Favorites.")
            Button { } label: { Image(systemName: "paperplane") }
                .accessibilityLabel("Send to a Friend")
        }
        .font(.title2)
    }
}

#Preview { RecipeActions() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares RecipeActions.
- WHAT — **Imports SwiftUI, which includes the accessibility modifiers.**
- WHERE — Top of the file, above RecipeActions.
- WHEN — At compile time, before RecipeActions is type-checked.
- WHY — Accessibility in SwiftUI is not a separate framework; it is part of every view.

3 `struct RecipeActions: View {`

- WHO — Your code; SwiftUI renders it.
- WHAT — **Declares a row of actions for a recipe screen.**
- WHERE — At file scope, as a top-level struct.
- WHEN — Created when a parent or the preview shows it.
- WHY — Icon rows like this are the most common place labels go missing.

4 `@State private var isFavorite = false`

- WHO — SwiftUI stores it; the heart button reads and toggles it.
- WHAT — **Remembers whether this recipe is saved as a favorite.**
- WHERE — Private SwiftUI-managed storage tied to this view's place on screen.
- WHEN — Flips on every tap of the heart.
- WHY — One state value drives both the picture and the spoken value, so they can never disagree.

```
6  var body: some View {
```

WHO SwiftUI calls body on RecipeActions whenever isFavorite changes; your code returns the HStack.
WHAT **Describes the two buttons and their accessibility information.**
WHERE Inside RecipeActions, below the isFavorite state.
WHEN Whenever isFavorite changes, and on first display.
WHY Accessibility modifiers are recomputed with the view, so announcements stay current automatically.

```
7  HStack(spacing: 32) {
```

WHO The HStack, which places the two buttons 32 points apart.
WHAT **Places the two buttons side by side, 32 points apart.**
WHERE The root of body, holding both buttons.
WHEN On every layout pass.
WHY Generous spacing keeps the small icons from being tapped by mistake.

```
8  Button { isFavorite.toggle() } label: {
```

WHO The user taps; SwiftUI runs the action.
WHAT **Creates a button that toggles the favorite state.**
WHERE First item in the row.
WHEN On tap or on a VoiceOver double-tap.
WHY A real Button gets the button trait and activation for free; an Image with onTapGesture would not.

```
9  Image(systemName: isFavorite ? "heart.fill" : "heart")
```

WHO SF Symbols, supplying heart.fill or heart depending on isFavorite.
WHAT **Shows a filled heart when saved and an outline when not.**
WHERE Inside the heart button's label closure, as its only content.
WHEN Redrawn each time isFavorite changes.
WHY Fill versus outline is a shape change, not only a color change, so the state is visible to everyone.

```
11 .accessibilityLabel("Favorite")
```

WHO VoiceOver and Voice Control read it.
WHAT **Names the button “Favorite”.**
WHERE On the heart button.
WHEN Whenever assistive technology focuses or lists the button.
WHY Without it VoiceOver can only guess from the picture, and Voice Control users have no name to say.

```
12 .accessibilityValue(isFavorite ? "Saved" : "Not saved")
```

WHO VoiceOver reads it right after the label.
WHAT **Reports the button’s current state in words.**
WHERE On the heart button.
WHEN Re-read every time the state changes.
WHY The label stays stable while the value carries the state: “Favorite, Saved, button”.

```
13 .accessibilityHint("Adds or removes it from Favorites.")
```

WHO VoiceOver reads it after a short pause.
WHAT **Describes the result of activating the button.**
WHERE On the heart button.
WHEN After the label, value and traits, if hints are on.
WHY A hint starts with a verb and explains the outcome; it is optional extra help, never the only description.

```
14  Button { } label: { Image(systemName: "paperplane") }
```

WHO — The user taps; the empty closure stands in for sharing.
WHAT — **Creates a second icon-only button.**
WHERE — Second item in the row.
WHEN — When tapped; sharing is left out here.
WHY — A paper plane could mean send, share or travel. The picture alone is ambiguous even for sighted users.

```
15  .accessibilityLabel("Send to a Friend")
```

WHO — VoiceOver and Voice Control.
WHAT — **Names the button by the action it performs.**
WHERE — On the paper-plane button.
WHEN — Whenever it is focused.
WHY — The label removes the guesswork. No hint is needed because the label already says what happens.

```
17  .font(.title2)
```

WHO — SwiftUI text styles, which also size SF Symbols.
WHAT — **Draws both symbols at the title2 size (22 pt by default).**
WHERE — Applied to the whole row.
WHEN — At layout time, scaling with Dynamic Type.
WHY — Text styles let icons grow with the user’s text size. For real apps also give each button a 44 x 44 pt hit area.

```
21  #Preview { RecipeActions() }
```

WHO — Xcode previews, and the simulated iPhone on this page, running RecipeActions.
WHAT — **Renders the row so you can tap the heart.**
WHERE — At file scope, after the closing brace of RecipeActions; the canvas reads it, the app does not.
WHEN — On every edit, when the canvas refreshes with RecipeActions.
WHY — In Xcode, Accessibility Inspector can point at this preview and show exactly what you set.

> **Try it**
>
> 1. Remove the .accessibilityValue line. How would a VoiceOver user now know whether the recipe is saved?
> 2. Change the label to "Favorite button". Why would VoiceOver then say “button” twice?

CODE · LIVE SWIFTUI PREVIEW

Give controls meaningful values

Standard controls already announce their traits and support VoiceOver gestures, but they cannot guess what a number means. An unlabeled Slider with a value of 2 in 0 to 4 says “50 percent”. Here the slider speaks the same word the screen shows.

```swift
import SwiftUI

struct SpiceSettings: View {
    @State private var spice = 2.0
    @State private var servings = 4
    let levels = ["None", "Mild", "Medium", "Hot", "Fiery"]

    var body: some View {
        Form {
            Stepper("Serves \(servings)", value: $servings, in: 1...12)
            VStack(alignment: .leading) {
                Text("Spice level: \(levels[Int(spice)])")
                    .accessibilityHidden(true)
                Slider(value: $spice, in: 0...4, step: 1)
                    .accessibilityLabel("Spice level")
                    .accessibilityValue(levels[Int(spice)])
            }
        }
    }
}

#Preview { SpiceSettings() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares SpiceSettings.
- WHAT — **Imports SwiftUI for Form, Stepper and Slider.**
- WHERE — Top of the file, above SpiceSettings.
- WHEN — At compile time, before SpiceSettings is type-checked.
- WHY — These controls come with accessibility behavior built in.

3 `struct SpiceSettings: View {`

- WHO — Your code; SwiftUI renders it.
- WHAT — **Declares a small settings form for a recipe.**
- WHERE — At file scope, as a top-level struct.
- WHEN — Created by the preview or a parent.
- WHY — Forms full of sliders and steppers are where meaningless values hide.

4 `@State private var spice = 2.0`

- WHO — SwiftUI stores it; the slider reads and writes it.
- WHAT — **Holds the spice level as a number from 0 to 4.**
- WHERE — Private SwiftUI-managed storage tied to this view's place on screen.
- WHEN — Changes as the slider moves.
- WHY — Slider works with Double; the words people hear come from a separate lookup.

5 `@State private var servings = 4`

- WHO — SwiftUI stores it; the stepper changes it.
- WHAT — **Holds the number of servings.**
- WHERE — Private SwiftUI-managed storage, beside spice.
- WHEN — Changes on each stepper tap.
- WHY — A plain Int fits a stepper, which moves in whole steps.

```swift
let levels = ["None", "Mild", "Medium", "Hot", "Fiery"]
```

WHO Your code, as a constant that body and the slider's value both read.
WHAT **Names each of the five spice levels.**
WHERE A constant property of the view.
WHEN Read whenever body runs.
WHY One list feeds both the visible text and the spoken value, so sighted and VoiceOver users get the same words.

```swift
var body: some View {
```

WHO SwiftUI calls body on SpiceSettings whenever spice or servings changes.
WHAT **Describes the Form with its stepper row and its captioned slider row.**
WHERE Inside SpiceSettings, below the two state properties and the levels constant.
WHEN Whenever spice or servings changes.
WHY The spoken value is recomputed along with the screen.

```swift
Form {
```

WHO SwiftUI's grouped settings container.
WHAT **Lays out the controls as grouped rows.**
WHERE The root of body, holding the Stepper and the VStack.
WHEN On every layout pass.
WHY Form gives the familiar Settings look and correct VoiceOver row order.

```swift
Stepper("Serves \(servings)", value: $servings, in: 1...12)
```

WHO SwiftUI Stepper; the user taps plus or minus.
WHAT **Shows "Serves 4" with buttons that change the count from 1 to 12.**
WHERE First row of the form.
WHEN Updates on each tap.
WHY Its label already contains the number, so VoiceOver users hear the current value without extra code.

```swift
VStack(alignment: .leading) {
```

WHO The VStack inside the Form's second row, which stacks the caption over the slider.
WHAT **Puts the spice caption above its slider.**
WHERE Second row of the form.
WHEN On every layout pass.
WHY A visible caption above a slider tells sighted users what it controls.

```swift
Text("Spice level: \(levels[Int(spice)])")
```

WHO SwiftUI's text renderer, reading spice to pick the level name.
WHAT **Shows the current level in words, such as "Spice level: Medium".**
WHERE Above the slider, as the first child of the VStack.
WHEN Redrawn as the slider moves.
WHY Words are clearer than a bare track position for everyone.

```swift
.accessibilityHidden(true)
```

WHO VoiceOver skips this element.
WHAT **Removes the caption from the accessibility tree.**
WHERE On the caption Text only.
WHEN For as long as the caption is on screen.
WHY The slider will speak the same words, so hiding the caption saves a swipe and avoids hearing it twice.

```
14  Slider(value: $spice, in: 0...4, step: 1)
```

WHO — SwiftUI Slider; the user drags, or swipes up and down in VoiceOver.
WHAT — **Lets people pick a whole-number level from 0 to 4.**
WHERE — Below the caption, as the second child of the VStack.
WHEN — Updates spice as it moves.
WHY — Step 1 matches the five named levels, so there is never a value without a name.

```
15  .accessibilityLabel("Spice level")
```

WHO — VoiceOver, Voice Control and Switch Control, reading the name of the slider.
WHAT — **Names the slider Spice level, so VoiceOver has a label to speak before the value.**
WHERE — On the Slider only, not on the caption above it.
WHEN — Whenever it is focused.
WHY — Without a label VoiceOver says only the value and the trait, so people do not know what they are adjusting.

```
16  .accessibilityValue(levels[Int(spice)])
```

WHO — VoiceOver reads it after the label.
WHAT — **Replaces “50 percent” with the level name.**
WHERE — On the Slider only, after its label.
WHEN — Re-read after every adjustment.
WHY — People adjust until they hear the word they want: “Spice level, Hot, adjustable”.

```
22  #Preview { SpiceSettings() }
```

WHO — Xcode previews, and the simulated iPhone on this page, running SpiceSettings.
WHAT — **Renders the Form so you can drag the slider and watch the caption change.**
WHERE — At file scope, after the closing brace of SpiceSettings; the canvas reads it, the app does not.
WHEN — On every edit, when the canvas refreshes with SpiceSettings.
WHY — Drag the slider and watch the caption: the same word will be spoken.

> **Try it**
>
> 1. Change step: 1 to step: 0.5. What breaks in the levels lookup, and what would VoiceOver say between levels?

COMPARE

An unlabeled row, then a labeled one

Both rows look identical on screen. The difference is only what assistive technology gets. On the left, VoiceOver can only guess from the pictures. On the right, decoration is hidden and every button says what it does.

A: Before: icons only

```swift
import SwiftUI

struct RowBefore: View {
    var body: some View {
        HStack(spacing: 16) {
            Image(systemName: "fork.knife")
            Text("Lemon Pasta").font(.headline)
            Spacer()
            Button { } label: { Image(systemName: "heart") }
            Button { } label: { Image(systemName: "paperplane") }
        }
        .padding()
    }
}

#Preview { RowBefore() }
```

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the file that declares RowBefore.
- WHAT — **Imports SwiftUI for the HStack, Image, Text, Spacer and Button used below.**
- WHERE — Top of the file, above RowBefore.
- WHEN — At compile time, before RowBefore is type-checked.
- WHY — Both rows use exactly the same views.

3 `struct RowBefore: View {`

- WHO — Your code; SwiftUI renders it.
- WHAT — **Declares the row as first built.**
- WHERE — At file scope, as a top-level struct.
- WHEN — Created by the preview.
- WHY — This is how icon rows usually look before anyone tests with VoiceOver.

4 `var body: some View {`

- WHO — SwiftUI calls body on RowBefore; your code returns the HStack.
- WHAT — **Describes the row: icon, title, spacer and two icon-only buttons.**
- WHERE — Inside RowBefore, its only member.
- WHEN — Whenever it is drawn.
- WHY — Same structure as the fixed row.

5 `HStack(spacing: 16) {`

- WHO — The HStack, which places the icon, title, spacer and buttons in one row.
- WHAT — **Lines up the icon, title and buttons horizontally.**
- WHERE — The root of body, holding all five children.
- WHEN — On every layout pass.
- WHY — VoiceOver will visit the children in this order, left to right.

6 `Image(systemName: "fork.knife")`

- WHO — SF Symbols; VoiceOver treats it as an image.
- WHAT — **Draws a decorative fork and knife.**
- WHERE — Leading edge of the row.
- WHEN — Whenever the row shows.
- WHY — VoiceOver stops here and describes the picture, a swipe that tells the user nothing new.

```
7  Text("Lemon Pasta").font(.headline)
```

WHO SwiftUI's text renderer, drawing the recipe name in the headline style.
WHAT **Shows the recipe name.**
WHERE After the icon, as the second child of the HStack.
WHEN Whenever the row shows.
WHY Text is read as written, so this element is already fine.

```
8  Spacer()
```

WHO The Spacer inside the HStack, which claims the free width.
WHAT **Pushes the buttons to the trailing edge.**
WHERE Between the title and the buttons.
WHEN At layout time, after the title and the buttons report their widths.
WHY Spacers are invisible to VoiceOver, so they never cost a swipe.

```
9  Button { } label: { Image(systemName: "heart") }
```

WHO SwiftUI Button with only a symbol inside.
WHAT **Creates a favorite button with no words.**
WHERE Trailing side of the row.
WHEN When tapped or double-tapped in VoiceOver.
WHY VoiceOver can at best describe a heart. It cannot know the button adds the recipe to Favorites.

```
10 Button { } label: { Image(systemName: "paperplane") }
```

WHO SwiftUI Button with only a symbol inside.
WHAT **Creates a send button with no words.**
WHERE After the heart button, as the last child of the HStack.
WHEN When tapped or double-tapped in VoiceOver.
WHY A paper plane is ambiguous; Voice Control users have no reliable name to say to press it.

```
12 .padding()
```

WHO The padding modifier on the HStack.
WHAT **Adds the default 16 pt margins.**
WHERE Around the HStack, as the last modifier in body.
WHEN At layout time, after the row's children have measured themselves.
WHY Same spacing on both sides of the comparison.

```
16 #Preview { RowBefore() }
```

WHO Xcode previews, and the simulated iPhone on this page, running RowBefore.
WHAT **Renders the unlabeled row.**
WHERE At file scope, after the closing brace of RowBefore; the canvas reads it, the app does not.
WHEN On every edit, when the canvas refreshes with RowBefore.
WHY It looks finished, which is exactly why missing labels slip through.

B: After: labeled and decluttered

```swift
import SwiftUI

struct RowAfter: View {
    var body: some View {
        HStack(spacing: 16) {
            Image(systemName: "fork.knife").accessibilityHidden(true)
            Text("Lemon Pasta").font(.headline)
            Spacer()
            Button { } label: { Image(systemName: "heart") }
                .accessibilityLabel("Add to Favorites")
            Button { } label: { Image(systemName: "paperplane") }
                .accessibilityLabel("Send Recipe")
        }
        .padding()
    }
}

#Preview { RowAfter() }
```

1 `import SwiftUI`

- WHO: The Swift compiler, while it builds the file that declares RowAfter.
- WHAT: **Imports SwiftUI, including the accessibilityHidden and accessibilityLabel modifiers.**
- WHERE: Top of the file, above RowAfter.
- WHEN: At compile time, before RowAfter is type-checked.
- WHY: The fixes are ordinary modifiers.

3 `struct RowAfter: View {`

- WHO: Your code; SwiftUI renders it.
- WHAT: **Declares the fixed row.**
- WHERE: At file scope, as a top-level struct.
- WHEN: Created by the preview.
- WHY: Same visual design, better information.

4 `var body: some View {`

- WHO: SwiftUI calls body on RowAfter; your code returns the HStack.
- WHAT: **Describes the same row, with the icon hidden from VoiceOver and both buttons labeled.**
- WHERE: Inside RowAfter, its only member.
- WHEN: Whenever it is drawn.
- WHY: Same structure as before.

5 `HStack(spacing: 16) {`

- WHO: The HStack, which places the icon, title, spacer and buttons in one row.
- WHAT: **Lines up the same four elements.**
- WHERE: The root of body, holding all five children.
- WHEN: On every layout pass.
- WHY: Nothing moves on screen; only the accessibility tree changes.

```
6  Image(systemName: "fork.knife").accessibilityHidden(true)
```

WHO VoiceOver, which now skips the image.
WHAT **Keeps the decorative icon visible but removes it from VoiceOver.**
WHERE Leading edge of the row.
WHEN For as long as the row is on screen.
WHY Decoration that repeats nearby text is noise. Hiding it saves a swipe on every row of a long list.

```
7  Text("Lemon Pasta").font(.headline)
```

WHO SwiftUI's text renderer, drawing the recipe name in the headline style.
WHAT **Shows the recipe name.**
WHERE After the hidden icon, as the second child of the HStack.
WHEN Whenever the row shows.
WHY Unchanged: real text is already accessible.

```
8  Spacer()
```

WHO The Spacer inside the HStack, which claims the free width.
WHAT **Pushes the buttons to the trailing edge.**
WHERE Between title and buttons.
WHEN At layout time, after the title and the buttons report their widths.
WHY Unchanged and invisible to VoiceOver.

```
9  Button { } label: { Image(systemName: "heart") }
```

WHO SwiftUI's Button, with only the heart symbol inside its label.
WHAT **The same heart button.**
WHERE The trailing side of the row, as the fourth child of the HStack.
WHEN When tapped or double-tapped in VoiceOver.
WHY The visual design stays clean and icon-only.

```
10 .accessibilityLabel("Add to Favorites")
```

WHO VoiceOver and Voice Control.
WHAT **Names the button by its action.**
WHERE On the heart button.
WHEN Whenever it is focused or listed.
WHY Now VoiceOver says “Add to Favorites, button”, and Voice Control users can say “Tap Add to Favorites”.

```
11 Button { } label: { Image(systemName: "paperplane") }
```

WHO SwiftUI's Button, with only the paper plane symbol inside its label.
WHAT **The same send button.**
WHERE After the heart button, as the last child of the HStack.
WHEN When tapped or double-tapped in VoiceOver.
WHY The pixels do not change at all; the label on the next line is the whole fix.

```
12 .accessibilityLabel("Send Recipe")
```

WHO VoiceOver and Voice Control.
WHAT **Names the button by its action.**
WHERE On the paper-plane button.
WHEN Whenever it is focused.
WHY The label resolves the ambiguity of the picture for everyone who cannot see it.

14 `.padding()`

WHO The padding modifier on the HStack.
WHAT **Adds the default 16 pt margins.**
WHERE Around the HStack, as the last modifier in body.
WHEN At layout time, after the row's children have measured themselves.
WHY Same spacing as the left side.

18 `#Preview { RowAfter() }`

WHO Xcode previews, and the simulated iPhone on this page, running RowAfter.
WHAT **Renders the fixed row.**
WHERE At file scope, after the closing brace of RowAfter; the canvas reads it, the app does not.
WHEN On every edit, when the canvas refreshes with RowAfter.
WHY The two previews look the same; only VoiceOver or Accessibility Inspector reveals the difference.

> **Takeaway**
>
> Accessibility fixes rarely change the pixels. Hide what is decorative, label every icon-only control with its action, and VoiceOver goes from four guesses to three useful stops.

CODE · READ-ALONG

Headings, groups, and reading order

VoiceOver users navigate by swiping element to element and by jumping between headings with the rotor. This recipe header marks the title as a heading, reads it before the photo, merges two small facts into one stop, gives the photo a description, and adds a spoken alias for Voice Control.

```swift
import SwiftUI

struct RecipeHero: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 8) {
            ZStack(alignment: .bottomLeading) {
                Image("lemon-pasta", label: Text("Pasta with basil"))
                    .accessibilityIgnoresInvertColors()
                Text("Lemon Pasta").font(.title)
                    .accessibilityAddTraits(.isHeader)
                    .accessibilitySortPriority(1)
            }
            .accessibilityElement(children: .contain)
            HStack {
                Label("25 min", systemImage: "clock")
                Label("4 of 5 stars", systemImage: "star.fill")
            }
            .accessibilityElement(children: .combine)
            Button("Start Cooking") { }
                .accessibilityInputLabels(["Start Cooking", "Cook"])
        }
    }
}
```

Line by line

```
1  import SwiftUI
```

WHO The Swift compiler, while it builds the file that declares RecipeHero.
WHAT **Imports SwiftUI and its accessibility modifiers.**
WHERE Top of the file, above RecipeHero.
WHEN At compile time, before RecipeHero is type-checked.
WHY Every modifier below is standard SwiftUI.

```
3  struct RecipeHero: View {
```

WHO Your code; a recipe detail screen would show it.
WHAT **Declares the header of a recipe screen.**
WHERE At file scope, as a top-level struct.
WHEN Created by its parent.
WHY The top of a screen is where VoiceOver users start, so its order matters most.

```
4  var body: some View {
```

WHO SwiftUI calls body on RecipeHero; your code returns the VStack.
WHAT **Describes the header: a photo with a title over it, two facts, and the main button.**
WHERE Inside RecipeHero, its only member.
WHEN Whenever it is drawn.
WHY Structure for VoiceOver is declared right beside the visual structure.

```
5  VStack(alignment: .leading, spacing: 8) {
```

WHO The VStack, which stacks the photo block, the facts row and the button.
WHAT **Stacks photo, facts and button, aligned to the leading edge.**
WHERE The root of body, holding the ZStack, the HStack and the Button.
WHEN On every layout pass.
WHY VoiceOver follows this top-to-bottom order unless you change it.

```
6  ZStack(alignment: .bottomLeading) {
```

WHO The ZStack, which layers the title over the photo at its bottom-leading corner.
WHAT **Layers the title on top of the photo, in its bottom-leading corner.**
WHERE Top of the header.
WHEN On every layout pass.
WHY Layering means the photo is declared first, so by default it would be read before the title.

```
7  Image("lemon-pasta", label: Text("Pasta with basil"))
```

WHO SwiftUI Image with an explicit accessibility label.
WHAT **Shows the photo and gives VoiceOver a short description of it.**
WHERE The bottom layer of the ZStack.
WHEN Whenever the header shows.
WHY Without a label, an asset image is announced by its file name. Describe what matters in the photo, briefly.

```
8  .accessibilityIgnoresInvertColors()
```

WHO iOS Smart Invert, which reads this modifier on the photo and leaves it uninverted.
WHAT **Keeps the photo's real colors when Smart Invert is on.**
WHERE On the photo only.
WHEN Whenever the user has Smart Invert enabled.
WHY Smart Invert darkens interfaces for comfort; inverting a food photo would make it look spoiled.

```
9  Text("Lemon Pasta").font(.title)
```

WHO SwiftUI's text renderer, drawing the recipe name over the photo.
WHAT **Draws the recipe name over the photo at the title size.**
WHERE Top layer of the ZStack.
WHEN Whenever the header shows.
WHY The most important words on the screen, so they should be both seen and heard first.

```
10 .accessibilityAddTraits(.isHeader)
```

WHO VoiceOver's rotor, which collects elements carrying the header trait.
WHAT **Marks the title as a heading, announced as “Lemon Pasta, heading”.**
WHERE On the title Text.
WHEN Whenever VoiceOver is on.
WHY Headings let people jump between sections instead of swiping through every element.

```
11 .accessibilitySortPriority(1)
```

WHO VoiceOver's reading order, which sorts the title ahead of the photo.
WHAT **Moves the title ahead of the photo within their container.**
WHERE On the title Text.
WHEN Whenever VoiceOver builds the reading order.
WHY Higher priority is read first. People want the name of the dish before a description of its photo.

```
13 .accessibilityElement(children: .contain)
```

WHO SwiftUI's accessibility tree, grouping the ZStack's photo and title.
WHAT **Makes the ZStack a container that keeps its two children together.**
WHERE On the ZStack, after its two children.
WHEN Whenever VoiceOver builds the tree.
WHY Sort priority only compares siblings in the same container; this scopes it to the photo and title.

```
14 HStack {
```

WHO The HStack, which places the two fact Labels side by side.
WHAT **Puts two small facts side by side.**
WHERE Below the photo block, as the second child of the VStack.
WHEN On every layout pass.
WHY Small related facts are a classic source of extra swipes.

```
15 Label("25 min", systemImage: "clock")
```

WHO SwiftUI's Label, pairing the clock symbol with the cooking time.
WHAT **Shows the cooking time with a clock symbol.**
WHERE The first fact in the HStack.
WHEN Whenever the header shows.
WHY Text plus symbol, never symbol alone.

```
16 Label("4 of 5 stars", systemImage: "star.fill")
```

WHO SwiftUI's Label, pairing the star symbol with the rating in words.
WHAT **Shows the rating in words with a star symbol.**
WHERE The second fact in the HStack, after the time.
WHEN Whenever the header shows.
WHY Writing “4 of 5” instead of drawing five star images gives VoiceOver real words to read.

```
18  .accessibilityElement(children: .combine)
```

WHO	SwiftUI's accessibility tree, merging the HStack's two Labels.
WHAT	**Merges both labels into one element read as a single phrase.**
WHERE	On the HStack, after its two Labels.
WHEN	Whenever VoiceOver builds the tree.
WHY	One stop (“25 min, 4 of 5 stars”) instead of two keeps related facts together and halves the swipes.

```
19  Button("Start Cooking") { }
```

WHO	SwiftUI's Button with a plain text title, the last child of the VStack.
WHAT	**The screen’s main action.**
WHERE	Bottom of the header.
WHEN	When tapped or activated by voice.
WHY	A text button is accessible by default: “Start Cooking, button”.

```
20  .accessibilityInputLabels(["Start Cooking", "Cook"])
```

WHO	Voice Control, which listens for either phrase to activate the button.
WHAT	**Lets people activate the button by saying either phrase.**
WHERE	On the Start Cooking button only.
WHEN	Whenever Voice Control listens.
WHY	Short spoken aliases make long or wordy buttons easier to say; the first one is the primary name.

INTERACTIVE

Hear what VoiceOver hears

The same product card appears twice: as first built, and after the fixes from this lesson. Move the VoiceOver cursor with the Left and Right arrow keys or the Previous and Next buttons, and read the caption. Count the stops in each version, listen to what unlabeled images say, and find the information that is never announced at all. **Double-tap** activates the focused element; **Speak** reads the announcement aloud.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Respect motion and color settings

Some settings are requests your code must honor. This flight status indicator skips its bounce when Reduce Motion is on, and switches from a plain dot to distinct shapes when Differentiate Without Color is on. It also always has a spoken label.

```swift
import SwiftUI

struct FlightStatus: View {
    let isDelayed: Bool
    @Environment(\.accessibilityReduceMotion) private var reduceMotion
    @Environment(\.accessibilityDifferentiateWithoutColor)
    private var differentiate

    var body: some View {
        Image(systemName: symbol)
            .foregroundStyle(isDelayed ? .red : .green)
            .scaleEffect(isDelayed ? 1.25 : 1)
            .animation(reduceMotion ? nil : .bouncy, value: isDelayed)
            .accessibilityLabel(isDelayed ? "Delayed" : "On time")
    }

    var symbol: String {
        if !differentiate { return "circle.fill" }
        return isDelayed
            ? "exclamationmark.triangle.fill" : "checkmark.circle.fill"
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the file that declares FlightStatus.
- **WHAT** **Imports SwiftUI, including the accessibility environment values.**
- **WHERE** Top of the file, above FlightStatus.
- **WHEN** At compile time, before FlightStatus is type-checked.
- **WHY** User settings reach your views through the SwiftUI environment.

3 `struct FlightStatus: View {`

- **WHO** Your code; a flight list row would include it.
- **WHAT** **Declares a compact status indicator.**
- **WHERE** At file scope, as a top-level struct.
- **WHEN** Created for each row that shows a flight.
- **WHY** Compact dots in dense lists are where designers most often rely on color alone.

4 `let isDelayed: Bool`

- **WHO** The parent row passes it in.
- **WHAT** **Says whether this flight is delayed.**
- **WHERE** An immutable property of FlightStatus, set once per row.
- **WHEN** Set when the row is created; a change creates a new value.
- **WHY** One Boolean drives color, shape, motion and the spoken label, so they always agree.

5 `@Environment(\.accessibilityReduceMotion) private var reduceMotion`

- **WHO** iOS sets it from Settings > Accessibility > Motion; SwiftUI delivers it.
- **WHAT** **Reads whether the user asked for less motion.**
- **WHERE** The environment, shared by every view in the app.
- **WHEN** Read on each body pass; updates live if the setting changes.
- **WHY** Bouncing, zooming and sliding can cause dizziness or nausea for some people; this flag lets you tone it down.

```
6  @Environment(\.accessibilityDifferentiateWithoutColor)
```

WHO iOS sets it from Settings > Accessibility > Display & Text Size.
WHAT **Reads whether the user asked for cues that do not depend on color.**
WHERE SwiftUI's environment, read into the property on the next line.
WHEN Read on each body pass.
WHY About 1 in 12 men have some form of color vision deficiency; red versus green alone can be invisible to them.

```
7  private var differentiate
```

WHO Your view, naming the environment value the line above reads.
WHAT **Names the stored setting from the previous line.**
WHERE A property of FlightStatus.
WHEN Read in the symbol property below.
WHY A short, clear name keeps the logic below readable.

```
9  var body: some View {
```

WHO SwiftUI calls body on FlightStatus whenever isDelayed or a setting changes.
WHAT **Describes the indicator: one symbol with color, scale, animation and a spoken label.**
WHERE Inside FlightStatus, below the isDelayed property and the two environment values.
WHEN Whenever isDelayed or a setting changes.
WHY Settings are just more inputs to body, like state.

```
10 Image(systemName: symbol)
```

WHO SF Symbols, supplying whichever glyph the symbol property names.
WHAT **Draws the symbol chosen by the symbol property.**
WHERE The root of body; every modifier below attaches to this Image.
WHEN On every redraw of the indicator, whenever symbol returns a new name.
WHY Deciding the shape in one place keeps the visual and the setting in sync.

```
11 .foregroundStyle(isDelayed ? .red : .green)
```

WHO The foregroundStyle modifier on the symbol Image, reading isDelayed.
WHAT **Colors the symbol red when delayed and green when on time.**
WHERE On the symbol Image, before the scale effect.
WHEN On every redraw, resolved again for Dark Mode.
WHY Color is a fast extra cue for many people, but never the only one.

```
12 .scaleEffect(isDelayed ? 1.25 : 1)
```

WHO SwiftUI rendering, applying the scaleEffect to the symbol Image.
WHAT **Enlarges the symbol slightly when the flight is delayed.**
WHERE On the symbol Image, after its color.
WHEN On every redraw, growing the symbol by a quarter while isDelayed is true.
WHY A size change draws attention to a status that just got worse.

```
13 .animation(reduceMotion ? nil : .bouncy, value: isDelayed)
```

WHO SwiftUI's animation system, reading reduceMotion to pick the animation.
WHAT **Bounces the size change, or applies it instantly when Reduce Motion is on.**
WHERE On the symbol Image, covering the scale change above it.
WHEN Whenever isDelayed changes, which is when the scale effect changes.
WHY Reduce Motion does not mean no feedback; the status still changes, just without the bounce.

```
14  .accessibilityLabel(isDelayed ? "Delayed" : "On time")
```

WHO Voice Over, reading the label of the symbol Image.
WHAT **Speaks the status in words.**
WHERE On the symbol Image, as its last modifier.
WHEN Whenever it is focused.
WHY The dot's meaning must not live only in its color or shape; VoiceOver users need the word.

```
17  var symbol: String {
```

WHO Your code, as a computed property that body reads on the Image line.
WHAT **Computes the symbol name for the current status and settings.**
WHERE A computed property of FlightStatus.
WHEN Each time body reads it.
WHY Keeping the decision out of body keeps body short and readable.

```
18  if !differentiate { return "circle.fill" }
```

WHO Your code, inside the symbol getter, checking the setting first.
WHAT **Uses a plain dot when the setting is off.**
WHERE Inside symbol, as its first statement.
WHEN When Differentiate Without Color is off.
WHY The compact dot stays the default design for people who asked for nothing different.

```
19  return isDelayed
```

WHO Your code, inside the symbol getter, choosing between two shapes.
WHAT **Chooses between two distinct shapes when the setting is on.**
WHERE Inside symbol, as its second and last statement.
WHEN When Differentiate Without Color is on.
WHY Shapes stay distinguishable in grayscale and for every kind of color vision.

```
20  ? "exclamationmark.triangle.fill" : "checkmark.circle.fill"
```

WHO SF Symbols, supplying one of these two glyphs by name.
WHAT **A warning triangle for delayed, a check mark for on time.**
WHERE The two possible results.
WHEN Picked at runtime, each time symbol is read with the setting on.
WHY Both shapes carry the meaning on their own, which is the whole point of the setting.

CONCEPT

Test it the way people use it

Accessibility bugs are invisible until you use the app the way your users do. Check every feature as you build it, not the week before release.

VoiceOver on a device Turn it on in Settings > Accessibility > VoiceOver, or set the Accessibility Shortcut to a triple-click of the side button. Then complete your main task without looking.

Accessibility Inspector An Xcode developer tool (Xcode > Open Developer Tool) that shows each element's label, value and traits, and audits for missing labels, small hit areas and low contrast.

Extreme previews Use the canvas variants in Xcode to see a view at the largest accessibility text size and in Dark Mode side by side.

Automated audits In an XCTest UI test, `try app.performAccessibilityAudit()` fails the test when it finds issues such as clipped text, missing descriptions or low contrast.

> **Accessibility on the App Store**
>
> App Store product pages can list the accessibility features an app supports, such as VoiceOver and Larger Text. Test each feature on a real device before you declare it.

CHECK YOURSELF

Check your understanding

1. A Slider has no accessibility label or value set, and its position is 2 in the range 0...4. What does VoiceOver convey when it reaches the slider?
   A. Only the word Slider, with no position and no trait
   B. The raw number 2, followed by the range minimum and maximum
   C. Its position as a percentage, and that it is adjustable
   D. Its name followed by the word for the current level, such as Medium
2. Which is the best accessibility label for an icon-only trash button?
   A. Delete
   B. Delete button
   C. Trash can icon
   D. Tap to delete
3. A row shows a decorative leaf flourish, a title, and five separate star images for a rating. What is the best fix?
   A. Label each star “star” so VoiceOver reads all five
   B. Hide the flourish and merge the rating into one element
   C. Hide the whole row from VoiceOver so it costs no swipes at all
   D. Add a hint to each star that explains the rating scale
4. Reduce Motion is on. Your onboarding slides cards in from the side with a bounce. What should you do?
   A. Remove onboarding entirely for these users
   B. Nothing; Reduce Motion only affects system animations
   C. Slow the slide and the bounce down so they feel gentler
   D. Replace the slide and bounce with a simple crossfade
5. Your team wants missing labels and clipped text caught automatically on every build. What do you add?
   A. A UI test that calls app.performAccessibilityAudit()
   B. A unit test for the view model that checks every label string
   C. A screenshot of every screen attached to the App Store listing
   D. Turning on Smart Invert in the simulator before each build

Answers: Designing for Accessibility

1. **C. Its position as a percentage, and that it is adjustable** Without a label or value, VoiceOver can only report the control's position as a percentage and add the adjustable trait that every slider carries. The name and the level word appear only after you add `.accessibilityLabel` and `.accessibilityValue`, which is what the Spice level example did.
2. **A. Delete** A label names the action in a word or two. "Delete button" makes VoiceOver say "button" twice, because the trait already adds it. "Trash can icon" describes the picture, not the action, and "Tap to delete" is hint-style wording that also assumes a touch screen.
3. **B. Hide the flourish and merge the rating into one element** Decoration should be hidden and related parts combined into one meaningful stop. Labeling each star still costs five swipes and never says the score. Hiding the whole row removes the content entirely.
4. **D. Replace the slide and bounce with a simple crossfade** Reduce Motion asks apps to swap large movement for gentler effects such as fades; read `accessibilityReduceMotion` and choose a different animation. A slower slide is still motion, and the setting is a request to every app, not only the system.
5. **A. A UI test that calls app.performAccessibilityAudit()** The accessibility audit in XCTest UI tests checks the running app for problems such as missing descriptions, clipped text and low contrast, and fails the test. A view-model unit test never sees the UI, so it cannot find a missing label.

Process, Prototypes, and Critique

CONCEPT

Design is a loop, not a step

Good screens are rarely right the first time. Designers work in a loop: understand the problem, sketch, build something people can try, watch them use it, and go around again. It mirrors the software development life cycle: requirements feed the design, and what you learn in testing feeds the next lap.

Requirements	What the app must do, for whom, and how you will know it works, written so someone can check it.
Persona	A short, research-based portrait of a typical user: goals, context and frustrations. It keeps decisions about real people.
User flow	The path through screens to finish one task, including the error and empty branches.
Wireframe and prototype	A wireframe is a low-detail layout of boxes and labels; a prototype is something people can tap. Both are cheap to throw away.
Usability test	Watching representative people try real tasks with your prototype, then fixing whatever trips them up.

> **Cheap before expensive**
>
> You can change a sketch in seconds, a prototype in minutes, and shipped code in days. Find problems as early in the loop as you can.

CONCEPT

Requirements you can design from

A requirement helps only if a designer can act on it and a tester can check it. A common format is the **user story**: *As a* nurse, *I want* to swap a shift from my phone, *so that* I do not have to find a computer between patients. **Acceptance criteria** then say exactly when it is done: *given* an upcoming shift, *when* I request a swap, *then* my manager is notified and the shift shows Pending.

The persona behind the story shapes the screen. This nurse uses the app one-handed, often wearing gloves, in a hurry. That one sentence argues for large targets, a primary action within thumb reach, and status you can read at a glance.

Functional requirement	What the app does: swap a shift, notify a manager.
Design requirement	How well it must work, in numbers: controls at least 44 x 44 pt, text contrast at least 4.5:1, the whole flow usable with VoiceOver and at the largest text size.

> **Personas come from research**
>
> Base personas on interviews or observation of real users. A persona made up in a meeting only restates the team's assumptions with a name attached.

CODE · LIVE SWIFTUI PREVIEW

A wireframe you can run

SwiftUI previews make code a fast prototyping tool. This wireframe for the nurse's shift app uses gray placeholders instead of real data and styling, so feedback stays on structure: what is on the screen, in what order, and which action matters most. Rearranging a placeholder takes seconds.

```swift
import SwiftUI

struct ShiftWireframe: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 16) {
            Text("Shifts").font(.largeTitle).bold()
            RoundedRectangle(cornerRadius: 12)
                .fill(Color.gray.opacity(0.3))
                .frame(height: 120)
                .overlay { Text("Next shift") }
            ForEach(0..<3) { _ in
                RoundedRectangle(cornerRadius: 8)
                    .fill(Color.gray.opacity(0.15))
                    .frame(height: 44)
            }
            Spacer()
            Button("Swap a Shift") { }
                .buttonStyle(.borderedProminent)
        }
        .padding()
    }
}

#Preview { ShiftWireframe() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the prototype.
- **WHAT** **Makes SwiftUI's shapes, stacks, text and buttons available in this file.**
- **WHERE** The first line of the file.
- **WHEN** At compile time, before ShiftWireframe is type-checked.
- **WHY** A prototype in the real framework can grow into the real screen, and it already runs on a real iPhone for testing.

3 `struct ShiftWireframe: View {`

- **WHO** Your struct, describing a throwaway screen.
- **WHAT** **Declares the wireframe as an ordinary SwiftUI view.**
- **WHERE** Often in a scratch file or project kept apart from the shipping app.
- **WHEN** Built only for the preview and for test sessions.
- **WHY** Calling it a wireframe signals that it is disposable: the goal is to learn what works, not to ship this code.

```
4  var body: some View {
```

WHO SwiftUI, which calls body to learn what to draw.
WHAT **Returns the wireframe's layout.**
WHERE Inside ShiftWireframe, its only member.
WHEN On first display and after every edit in the preview.
WHY Because the preview re-renders on every edit, you can try a new arrangement in seconds during a design discussion.

```
5  VStack(alignment: .leading, spacing: 16) {
```

WHO The VStack, which stacks the title, the card, the three rows, the spacer and the button.
WHAT **Stacks the parts top to bottom, aligned on their leading edge, 16 pt apart.**
WHERE The root view of the wireframe.
WHEN Laid out on every body pass.
WHY One column with one shared left edge is the simplest layout to test, and 16 pt matches the system margin so the rhythm already feels right.

```
6  Text("Shifts").font(.largeTitle).bold()
```

WHO A Text view with the large title style and bold weight.
WHAT **Stands in for the navigation title at 34 pt, bold.**
WHERE The first child of the stack, at the top of the screen.
WHEN Built on every body pass.
WHY Even a wireframe answers "where am I?" first. Testers who cannot tell which screen they are on will struggle with every task.

```
7  RoundedRectangle(cornerRadius: 12)
```

WHO A RoundedRectangle shape, drawn as the second child of the VStack.
WHAT **Draws a placeholder box for the "next shift" card.**
WHERE The second child of the stack.
WHEN Built on every body pass.
WHY A box marks a region without committing to its content, so nobody debates fonts or colors yet.

```
8  .fill(Color.gray.opacity(0.3))
```

WHO The fill modifier on the shape.
WHAT **Fills the box with 30 percent gray.**
WHERE On the next-shift placeholder.
WHEN Every render of the card placeholder.
WHY Gray keeps attention on layout. Color invites feedback about taste instead of structure.

```
9  .frame(height: 120)
```

WHO The frame modifier on the card placeholder.
WHAT **Makes the card 120 pt tall; the stack gives it the full width.**
WHERE Around the filled box.
WHEN During layout, after the fill has been applied to the card shape.
WHY Rough sizes still test hierarchy: the most important item, the next shift, gets the largest block.

```
10 .overlay { Text("Next shift") }
```

WHO The overlay modifier, drawing a Text on top of the box.
WHAT **Labels the placeholder with what will go there.**
WHERE Centered on the 120 pt box.
WHEN Every render of the card placeholder, with the label centered on it.
WHY Labels let you ask testers useful questions, such as "Where would you look to see when you work next?"

```
11  ForEach(0..<3) { _ in
```

WHO ForEach over a range.
WHAT **Repeats the row placeholder three times, ignoring the index.**
WHERE Inside the stack, below the card.
WHEN On every body pass.
WHY Three rows suggest a list of upcoming shifts without inventing fake data; the underscore says the index is not needed.

```
12  RoundedRectangle(cornerRadius: 8)
```

WHO A shape view, once per repetition.
WHAT **Draws one placeholder row.**
WHERE Inside the ForEach closure, once per repetition.
WHEN Built three times on each body pass.
WHY A smaller corner radius than the card marks these as a different, lesser kind of element.

```
13  .fill(Color.gray.opacity(0.15))
```

WHO The fill modifier on each row placeholder.
WHAT **Fills each row with a lighter, 15 percent gray.**
WHERE On each row placeholder.
WHEN Every render of each of the three rows.
WHY Lighter gray than the card says "secondary", testing whether the hierarchy reads even without content.

```
14  .frame(height: 44)
```

WHO The frame modifier on each row placeholder.
WHAT **Makes each row 44 pt tall.**
WHERE Around each row placeholder.
WHEN During layout of each row, after its fill.
WHY The design requirement says every target is at least 44 pt, so the wireframe respects it from the first sketch.

```
16  Spacer()
```

WHO A Spacer inside the VStack, which claims the free height.
WHAT **Pushes the action button to the bottom of the screen.**
WHERE Between the rows and the button.
WHEN Sized during layout to fill leftover height.
WHY The persona uses the phone one-handed, so the primary action belongs low on the screen, within thumb reach.

```
17  Button("Swap a Shift") { }
```

WHO A Button with an empty action.
WHAT **Places the primary action, labeled with a verb, that does nothing yet.**
WHERE The last child of the stack, at the bottom.
WHEN Tappable in the preview; the empty closure runs on tap.
WHY The label names the task from the user story. For a first test, all you need to learn is whether people find it.

```
18  .buttonStyle(.borderedProminent)
```

WHO The button style modifier.
WHAT **Draws the button filled with the accent color, the most prominent standard style.**
WHERE On the Swap a Shift button.
WHEN Every render of the Swap a Shift button.
WHY Hierarchy in one line: exactly one prominent action per screen tells people what to do next.

```
20  .padding()
```

WHO	The padding modifier with no value.
WHAT	**Adds the system default inset, 16 pt on iPhone, around the whole stack.**
WHERE	Around the VStack, as the last modifier in body.
WHEN	During layout, after every child of the VStack has measured itself.
WHY	Content stays inside the standard margins even in a sketch, so testers react to realistic spacing.

```
24  #Preview { ShiftWireframe() }
```

WHO	The #Preview macro, read by the canvas and by this page's phone to show ShiftWireframe.
WHAT	**Renders the wireframe in the canvas and the live phone.**
WHERE	At file scope, after the closing brace of ShiftWireframe.
WHEN	After every edit, when the canvas refreshes with ShiftWireframe.
WHY	A preview you can hand to a tester on a real device is the fastest prototype you can build.

Try it

1. Move the Spacer() above the card. Which layout would a one-handed nurse prefer, and why?
2. Change the card's height to 60. Does the next shift still read as the most important thing on the screen?

CONCEPT

Test with a few people, then iterate

A **usability test** is simple: give a representative person a realistic task, ask them to think aloud, and watch. Do not explain the screen, do not help, and do not defend your design. Every hesitation is data.

Jakob Nielsen's widely cited rule of thumb is that about five users per round reveal most of the major problems. Several small rounds, with fixes in between, beat one large study.

A task, not an instruction	Say "Swap your Friday shift," not "Tap the Swap button." Naming the button gives the answer away.
What to measure	Task success, time on task, errors, and where people hesitate or backtrack.
Test the edges	Include a session at the largest text size, one with VoiceOver, and one on a real device in bright light.
TestFlight	Apple's beta-testing service, which delivers test builds of your app to testers' devices before release.

INTERACTIVE

Spot the problems

Time to critique like a designer. This invoices screen breaks eight guidelines from this module. Click, tap or Tab to anything on the phone to inspect it; some parts are fine, and the notes say so. Each problem explains what is wrong, why it matters, and the SwiftUI fix. Ask for a hint if you are stuck, then switch to After to see every fix applied to the same screen.

This step is an interactive simulation in the HTML version.

COMPARE

The critique, in code

Here is one invoice row from that screen, before and after. Four guidelines, four small changes: a text style instead of a fixed size, readable color instead of faint, a symbol and a word instead of a lone color, and a labeled 44 pt button instead of a tappable glyph.

A: Before: four violations

```swift
import SwiftUI

struct PoorInvoiceRow: View {
    var body: some View {
        HStack {
            Circle().fill(Color.red).frame(width: 9, height: 9)
            Text("Bluebird Bakery")
                .font(.system(size: 11))
                .foregroundStyle(.tertiary)
            Spacer()
            Image(systemName: "pencil")
                .onTapGesture { }
        }
        .padding(4)
    }
}

#Preview { PoorInvoiceRow() }
```

1 `import SwiftUI`

- WHO — The Swift compiler, while it builds the app.
- WHAT — **Makes SwiftUI available, as on the right-hand side.**
- WHERE — The first line of the file.
- WHEN — At compile time, before PoorInvoiceRow is type-checked.
- WHY — Same toolkit on both sides: every problem below is a design choice, not a framework limit.

3 `struct PoorInvoiceRow: View {`

- WHO — Your struct, describing one list row.
- WHAT — **Declares the flawed invoice row.**
- WHERE — A value type in your app module.
- WHEN — Created when the preview builds it.
- WHY — Rows repeat down the screen, so every flaw here is multiplied by the number of invoices.

4 `var body: some View {`

- WHO — SwiftUI, which calls body to learn what to draw.
- WHAT — **Returns the row's content.**
- WHERE — Inside PoorInvoiceRow, its only member.
- WHEN — On first display and after any change.
- WHY — All four problems live in the next few lines, which is why critique works best line by line.

```
5  HStack {
```

WHO The HStack, which places the dot, the name, the spacer and the pencil in one row.
WHAT **Lays the status dot, client name, spacer and edit icon side by side.**
WHERE The root view of the row.
WHEN Laid out on every body pass.
WHY The structure is fine; the problems are in what each child is and how it is styled.

```
6  Circle().fill(Color.red).frame(width: 9, height: 9)
```

WHO A Circle shape, filled and sized.
WHAT **Draws a 9 pt red dot meaning "overdue".**
WHERE The first child of the row.
WHEN Every render of the row, in red regardless of the person's settings.
WHY Color alone carries the status. About 1 in 12 men cannot reliably tell red from green, and VoiceOver has nothing to read.

```
7  Text("Bluebird Bakery")
```

WHO A Text view drawing the client name as the second child of the HStack.
WHAT **Shows the client's name.**
WHERE After the dot, as the second child of the HStack.
WHEN Built on every body pass.
WHY This is the most important text in the row, so the next two lines matter most.

```
8  .font(.system(size: 11))
```

WHO The font modifier with a fixed size.
WHAT **Locks the name at 11 pt.**
WHERE On the client name.
WHEN At render time; it never responds to the text-size setting.
WHY Body text is 17 pt at the default size and should grow with Dynamic Type; 11 pt fixed ignores the reader completely.

```
9  .foregroundStyle(.tertiary)
```

WHO The foreground style modifier with the tertiary level.
WHAT **Draws the name in the faintest label color, about 1.7:1 on white.**
WHERE On the client name.
WHEN Resolved at render time for the current appearance.
WHY Tertiary is meant for placeholder-level text. Must-read text needs at least 4.5:1, and this is far below it.

```
10 Spacer()
```

WHO A Spacer inside the HStack, which claims the free width.
WHAT **Pushes the edit icon to the trailing edge.**
WHERE Between the name and the icon.
WHEN Sized during layout, after the dot, the name and the pencil report their widths.
WHY Fine on its own; it just moves the next problem to the edge of the screen.

```
11 Image(systemName: "pencil")
```

WHO An SF Symbol image.
WHAT **Draws a small pencil glyph meaning "edit".**
WHERE The last child of the row.
WHEN Every render of the row, at the default symbol size.
WHY The visible glyph is under 20 pt, and nothing below enlarges the area you can touch.

```
12  .onTapGesture { }
```

WHO A tap gesture recognizer.
WHAT **Makes only the glyph's drawn shape respond to taps.**
WHERE Attached to the pencil image.
WHEN Runs its closure when a tap lands exactly on the glyph.
WHY A gesture is not a button: the target is far below 44 x 44 pt, there is no pressed state, and VoiceOver does not announce a button.

```
14  .padding(4)
```

WHO The padding modifier with a fixed value.
WHAT **Adds only 4 pt around the row.**
WHERE Around the HStack, as the last modifier in body.
WHEN During layout, after the four children have measured themselves.
WHY Content sits almost on the screen edge instead of inside the 16 pt margin.

```
18  #Preview { PoorInvoiceRow() }
```

WHO The #Preview macro, read by the canvas and by this page's phone to show PoorInvoiceRow.
WHAT **Renders the flawed row.**
WHERE At file scope, after the closing brace of PoorInvoiceRow.
WHEN Whenever the canvas refreshes.
WHY Try reading the name at arm's length: the preview makes the contrast problem obvious.

B: After: every fix applied

```swift
import SwiftUI

struct BetterInvoiceRow: View {
    var body: some View {
        HStack {
            Text("Bluebird Bakery").font(.body)
            Spacer()
            Label("Overdue", systemImage: "exclamationmark.triangle")
            Button { } label: {
                Image(systemName: "pencil")
                    .frame(width: 44, height: 44)
            }
            .accessibilityLabel("Edit Invoice")
        }
        .padding(.horizontal)
    }
}
#Preview { BetterInvoiceRow() }
```

```
1  import SwiftUI
```

WHO The Swift compiler, while it builds the app.
WHAT **Makes SwiftUI's Text, Label and Button available.**
WHERE The first line of the file.
WHEN At compile time, before BetterInvoiceRow is type-checked.
WHY The fixes use only standard views and modifiers; nothing custom is needed.

```
3  struct BetterInvoiceRow: View {
```

WHO Your struct, describing one list row.
WHAT **Declares the fixed invoice row.**
WHERE A value type in your app module.
WHEN Created when the preview builds it.
WHY Same content and the same actions, so the comparison isolates the design changes.

```
4  var body: some View {
```

WHO SwiftUI, which calls body to learn what to draw.
WHAT **Returns the row's content.**
WHERE Inside BetterInvoiceRow, its only member.
WHEN On first display and after any change.
WHY Each line below answers one of the four problems on the left.

```
5  HStack {
```

WHO The HStack, which places the name, the spacer, the status Label and the button in one row.
WHAT **Lays the name, spacer, status and edit button side by side.**
WHERE The root view of the row.
WHEN Laid out on every body pass.
WHY The name comes first now, because it is what people scan for; status follows it.

```
6  Text("Bluebird Bakery").font(.body)
```

WHO A Text view with the body text style.
WHAT **Shows the client's name at 17 pt by default, in the primary label color.**
WHERE The first child of the row.
WHEN Re-rendered whenever the text-size setting changes.
WHY A text style scales with Dynamic Type, and the default primary color clears 4.5:1 in both Light and Dark Mode.

```
7  Spacer()
```

WHO A Spacer inside the HStack, which claims the free width.
WHAT **Pushes the status and button to the trailing edge.**
WHERE Between the name and the status.
WHEN Sized during layout, after the name, the Label and the button report their widths.
WHY Status and actions line up in a column down the list, which makes rows easy to compare.

```
8  Label("Overdue", systemImage: "exclamationmark.triangle")
```

WHO A Label pairing an SF Symbol with a word.
WHAT **Shows a warning triangle and the word "Overdue".**
WHERE After the spacer, as the third child of the HStack.
WHEN Every render; it scales with Dynamic Type like the name.
WHY Shape plus word works for everyone, including VoiceOver users and anyone who cannot tell red from green. You may still tint the symbol, but color is no longer the message.

```
9  Button { } label: {
```

WHO A real Button with an empty action and a custom label.
WHAT **Turns the edit control into a button with a pressed state and the button trait.**
WHERE The last child of the row.
WHEN Its action runs when the person taps anywhere in its label.
WHY Buttons work with touch, keyboard, Switch Control and VoiceOver; a tap gesture works only with a precise touch.

```
10  Image(systemName: "pencil")
```

WHO The same SF Symbol as before.
WHAT **Draws the pencil glyph at its normal size.**
WHERE Inside the button's label.
WHEN Every render of the row, inside the button's label.
WHY The visual can stay small; only the touch area needs to grow.

```
11  .frame(width: 44, height: 44)
```

WHO The frame modifier, inside the label.
WHAT **Makes the label, and so the button's hit area, 44 x 44 pt.**
WHERE Around the glyph, inside the Button.
WHEN During layout of the button's label, after the pencil has measured itself.
WHY 44 x 44 pt is the HIG minimum. With .plain style or a tap gesture you would also add .contentShape(Rectangle()).

```
13  .accessibilityLabel("Edit Invoice")
```

WHO The accessibility label modifier, read by VoiceOver and Voice Control.
WHAT **Names the button "Edit Invoice".**
WHERE On the Button, after its label closure.
WHEN Read whenever an assistive technology focuses the button.
WHY An icon-only button needs words for people who cannot see it; Voice Control users can now say "Tap Edit Invoice".

```
15  .padding(.horizontal)
```

WHO The padding modifier for the horizontal edges.
WHAT **Adds the system default inset, 16 pt on iPhone, on the left and right.**
WHERE Around the HStack, as the last modifier in body.
WHEN During layout, after the four children have measured themselves.
WHY The row sits on the standard margin, aligned with the rest of the screen. Inside a List, the list supplies this margin for you.

```
18  #Preview { BetterInvoiceRow() }
```

WHO The #Preview macro, read by the canvas and by this page's phone to show BetterInvoiceRow.
WHAT **Renders the fixed row next to the flawed one.**
WHERE At file scope, after the closing brace of BetterInvoiceRow.
WHEN Whenever the canvas refreshes.
WHY The two rows hold the same information; only one of them works for everybody.

> **Takeaway**
>
> None of the four fixes changes what the row does; all of them change who can use it. That is the heart of design critique: name the guideline, show the evidence (11 pt fixed, about 1.7:1, color only, a glyph-sized target), and propose a specific, standard fix.

CHECK YOURSELF

Check your understanding

1. Which acceptance criterion could a tester actually check?
   A. The swap flow should feel fast and intuitive to every nurse on the team
   B. Given an upcoming shift, when the nurse requests a swap, then the shift shows Pending
   C. The app should look modern and clean on the newest iPhones
   D. Nurses will love swapping shifts from their phones

2. During a usability test, the participant stares at the screen and says, "I am not sure where to go." What should you do?
   A. Point to the Swap button so the session can continue on schedule
   B. Explain how the screen is meant to work before they go on
   C. Ask what they expected, note the hesitation, and let them keep trying
   D. End the session, because this participant is clearly not representative

3. Why start with a gray wireframe instead of a polished mockup?
   A. Gray keeps feedback on structure, and boxes are cheap to rearrange
   B. App Review requires a wireframe before it accepts a new app for sale
   C. Polished designs cannot be shown in SwiftUI previews, only in a device
   D. The wireframe is the final deliverable that the client signs off on

4. Your persona uses the app one-handed while walking between rooms. Which design decision follows most directly?
   A. Use a dense layout so everything fits on one screen without scrolling
   B. Use a thin custom font for an elegant feel that matches the brand
   C. Put the most common action in the top corner, next to the title
   D. Place the primary action within thumb reach, with 44 × 44 pt targets

5. In the critique, an invoice's status was shown only by a red dot. Which fix works for someone who cannot tell red from green?
   A. Make the dot much larger so the red is easier to see at a glance
   B. Add a symbol and the word "Overdue", keeping color as a bonus
   C. Switch to a brighter, more saturated red
   D. Show the status in a tooltip that appears when the dot is hovered

Answers: Process, Prototypes, and Critique

1. **B. Given an upcoming shift, when the nurse requests a swap, then the shift shows Pending** A good acceptance criterion describes an observable result for a specific situation, so anyone can say pass or fail. "Fast and intuitive", "modern and clean" and "will love" are opinions with no test attached. Turn them into numbers instead, for example "a first-time user swaps a shift in under a minute".
2. **C. Ask what they expected, note the hesitation, and let them keep trying** The hesitation is the finding. Asking what they expected tells you why the design failed them; helping or explaining hides the very problem you are there to find. Dismissing the participant throws away evidence that a real user will hit the same wall.
3. **A. Gray keeps feedback on structure, and boxes are cheap to rearrange** Low fidelity keeps the conversation on what matters first: what is on the screen, in what order, and what the primary action is. Polish invites comments about color and fonts, and it makes the team reluctant to throw work away. App Review never sees wireframes, and previews can show any level of polish.
4. **D. Place the primary action within thumb reach, with 44 × 44 pt targets** One-handed use while moving means a thumb, imperfect aim and divided attention. The bottom of the screen is the easiest region for a thumb to reach, and 44 pt targets tolerate imprecise taps. The top corner is the hardest place to reach one-handed, and dense layouts and thin fonts make glancing harder.
5. **B. Add a symbol and the word "Overdue", keeping color as a bonus** A distinct shape plus a word carries the meaning without relying on color, and VoiceOver can read the word. A bigger or brighter dot is still only a color. Hover does not exist on a touch screen, so a tooltip would never appear on iPhone.

Worked Examples

WORKED EXAMPLE

Assemble a now-playing row

Problem.
Build a now-playing row for a music app: artwork, the song title and artist, and a Play button, finished as a padded card. Success: tapping Play shows the status Playing in the row.

Frame 1 of 7

The row starts with two pieces: a music-note symbol standing in for album artwork, and the song title beside it in an HStack. Everything else in this example is added to this row.

```
import SwiftUI

struct NowPlayingRow: View {
    var body: some View {
        HStack(spacing: 12) {
            Image(systemName: "music.note")
                .font(.title)
            Text("Blue Sky")
        }
    }
}

#Preview { NowPlayingRow() }
```

Frame 2 of 7

The single title becomes a small VStack: the title on top and the artist underneath in secondary gray. Related words now travel together as one block inside the row.

```
 import SwiftUI

 struct NowPlayingRow: View {
     var body: some View {
         HStack(spacing: 12) {
             Image(systemName: "music.note")
                 .font(.title)
+            VStack(alignment: .leading, spacing: 2) {
+                Text("Blue Sky")
+                Text("The Wanderers")
+                    .foregroundStyle(.secondary)
+            }
         }
     }
 }
+
+#Preview { NowPlayingRow() }
```

Frame 3 of 7

A Spacer claims the free width and pushes a Play button to the trailing edge, where a row's action belongs. The button does nothing yet; this frame is about placement, not behavior.

```
  import SwiftUI

  struct NowPlayingRow: View {
      var body: some View {
          HStack(spacing: 12) {
              Image(systemName: "music.note")
                  .font(.title)
              VStack(alignment: .leading, spacing: 2) {
                  Text("Blue Sky")
                  Text("The Wanderers")
                      .foregroundStyle(.secondary)
              }
+             Spacer()
+             Button("Play") { }
+         }
+     }
+ }
+
+ #Preview { NowPlayingRow() }
```

Frame 4 of 7

Padding wraps the whole row, so the artwork and button no longer touch the edges of their container. The space sits inside whatever background comes next, which is why padding comes first.

```
  import SwiftUI

  struct NowPlayingRow: View {
      var body: some View {
          HStack(spacing: 12) {
              Image(systemName: "music.note")
                  .font(.title)
              VStack(alignment: .leading, spacing: 2) {
                  Text("Blue Sky")
                  Text("The Wanderers")
                      .foregroundStyle(.secondary)
              }
              Spacer()
              Button("Play") { }
          }
+         .padding()
+     }
+ }
+
+ #Preview { NowPlayingRow() }
```

Frame 5 of 7

A thin material background and rounded corners turn the padded row into a card. The clip shape comes last so the material itself is rounded, and the row can sit on any screen.

```
    import SwiftUI

    struct NowPlayingRow: View {
        var body: some View {
            HStack(spacing: 12) {
                Image(systemName: "music.note")
                    .font(.title)
                VStack(alignment: .leading, spacing: 2) {
                    Text("Blue Sky")
                    Text("The Wanderers")
                        .foregroundStyle(.secondary)
                }
                Spacer()
                Button("Play") { }
            }
            .padding()
+           .background(.thinMaterial)
+           .clipShape(RoundedRectangle(cornerRadius: 12))
+       }
+   }
+
+   #Preview { NowPlayingRow() }
```

Frame 6 of 7

The row gains @State for whether the song is playing, and a status word that reads it: Paused for now. Nothing changes the state yet, so the status cannot move; that wiring is next.

```
  import SwiftUI

  struct NowPlayingRow: View {
+     @State private var playing = false
+     var body: some View {
+         HStack(spacing: 12) {
+             Image(systemName: "music.note")
+                 .font(.title)
+             VStack(alignment: .leading, spacing: 2) {
+                 Text("Blue Sky")
+                 Text("The Wanderers")
+                     .foregroundStyle(.secondary)
+             }
+             Spacer()
+             Text(playing ? "Playing" : "Paused")
+             Button("Play") { }
+         }
+         .padding()
+         .background(.thinMaterial)
+         .clipShape(RoundedRectangle(cornerRadius: 12))
+     }
+ }
+
+ #Preview { NowPlayingRow() }
```

Frame 7 of 7

The Play button now sets playing to true. SwiftUI reruns body, and the status word becomes Playing. Run the check: before the tap the row says Paused, and Playing appears only after it.

```
1   import SwiftUI
2
3   struct NowPlayingRow: View {
4       @State private var playing = false
5       var body: some View {
6           HStack(spacing: 12) {
7               Image(systemName: "music.note")
8                   .font(.title)
9               VStack(alignment: .leading, spacing: 2) {
10                  Text("Blue Sky")
11                  Text("The Wanderers")
12                      .foregroundStyle(.secondary)
13              }
14              Spacer()
15              Text(playing ? "Playing" : "Paused")
16  +           Button("Play") { playing = true }
17          }
18          .padding()
19          .background(.thinMaterial)
20          .clipShape(RoundedRectangle(cornerRadius: 12))
21      }
22  }
23
24  #Preview { NowPlayingRow() }
```

FINAL RESULT — CHECKED ON SCREEN

```
Tap “Play” and the screen shows “Playing”.
```

Line by line: the finished program

1 `import SwiftUI`

WHO	The Swift compiler, which loads the SwiftUI framework for this file.
WHAT	**Makes View, Text, VStack, HStack, Image, Spacer, Button and @State available in this file.**
WHERE	The first line of the file; an import covers only its own file.
WHEN	At compile time, before any of the code below is checked.
WHY	SwiftUI is a separate framework, so every file that describes a screen imports it.

3 `struct NowPlayingRow: View {`

WHO	You declare it; SwiftUI creates values of it wherever a song row is shown.
WHAT	**Declares NowPlayingRow, a struct that conforms to View and describes one song row.**
WHERE	At file scope, so the preview at the bottom can create it.
WHEN	Compiled once; a fresh value is created whenever a parent asks for the row.
WHY	A row is a small reusable view: one description serves every song in a list.

4 `@State private var playing = false`

WHO	SwiftUI, which stores the value outside the struct, tied to this row on screen.
WHAT	**Declares playing, whether the song is playing, starting false; @State makes SwiftUI keep and watch it.**
WHERE	Inside NowPlayingRow, marked private because only this row changes it.
WHEN	Created when the row first appears and kept across every redraw.
WHY	The status word below is computed from playing, so the row can never claim to play a paused song.

```
5  var body: some View {
```

WHO SwiftUI calls this property; your code never calls body directly.
WHAT **Returns the description of the row for the current data.**
WHERE A computed property inside NowPlayingRow.
WHEN When the row appears, and again after every change to playing.
WHY Describing the row as a function of the data is the whole SwiftUI model.

```
6  HStack(spacing: 12) {
```

WHO SwiftUI's layout system, which measures the children and places them in a row.
WHAT **Places the artwork, the words, the spacer and the button side by side, 12 points apart.**
WHERE The root view returned by body.
WHEN Measured and placed again every time body produces a new description.
WHY A now-playing row is read left to right: picture, song, action.

```
7  Image(systemName: "music.note")
```

WHO SwiftUI, which looks the name up in the SF Symbols library built into the system.
WHAT **Shows the music.note symbol as the song's artwork.**
WHERE The first child of the row, so it sits at the leading edge.
WHEN Drawn every time body runs; the symbol is a template image the system can recolor.
WHY A symbol stands in for album artwork while the row is designed, with no image files to manage.

```
8  .font(.title)
```

WHO The font modifier, applied to the Image view directly above it.
WHAT **Sets the artwork size with the title text style, so the note renders large enough to read as a picture.**
WHERE Chained under the Image; modifiers apply to the view written before them.
WHEN Read while body builds the description, before layout measures the image.
WHY SF Symbols are drawn like a font, so a text style sizes them, and the size follows the user's text-size setting.

```
9  VStack(alignment: .leading, spacing: 2) {
```

WHO SwiftUI's layout system, which measures the two lines and stacks them.
WHAT **Stacks the title above the artist, leading-aligned, with 2 points between them.**
WHERE A child of the row, between the artwork and the spacer.
WHEN Measured and placed again every time body produces a new description.
WHY Grouping the two lines in a stack makes them one block that travels together inside the row.

```
10 Text("Blue Sky")
```

WHO Swift builds the string; SwiftUI draws it as a Text view.
WHAT **Shows the song title, Blue Sky.**
WHERE The first line of the small stack, above the artist.
WHEN Rebuilt every time body runs, though the words never change.
WHY The title outranks the artist, so it sits on top and keeps the default primary color.

```
11 Text("The Wanderers")
```

WHO Swift builds the string; SwiftUI draws it as a Text view.
WHAT **Shows the artist's name, The Wanderers.**
WHERE The second line of the small stack, under the title.
WHEN Rebuilt every time body runs, though the words never change.
WHY The artist is the second fact a listener checks; it belongs with the title, not at the end of the row.

```
12  .foregroundStyle(.secondary)
```

WHO	The foregroundStyle modifier on the artist Text directly above it.
WHAT	**Colors the artist with the secondary style, a gray that steps back from the title.**
WHERE	Chained under the artist Text.
WHEN	Applied while the description is built.
WHY	De-emphasizing the artist keeps the title first in the hierarchy without shrinking the text.

```
14  Spacer()
```

WHO	SwiftUI's layout system, which gives the spacer all the width nobody else claimed.
WHAT	**Pushes the status and the button to the trailing edge of the row.**
WHERE	Between the words and the status, inside the row.
WHEN	Sized during layout, after every other child has reported its width.
WHY	Actions belong at the trailing edge, where a listener's thumb and eye expect them.

```
15  Text(playing ? "Playing" : "Paused")
```

WHO	Swift builds one of the two strings from playing; SwiftUI draws it as a Text view.
WHAT	**Shows Playing when playing is true and Paused when it is false.**
WHERE	A child of the row, just before the button at the trailing edge.
WHEN	Rebuilt after every change to playing, because it reads the state the button changes.
WHY	The listener gets confirmation that the tap worked, from the same single source of truth as the button.

```
16  Button("Play") { playing = true }
```

WHO	You declare it; the listener triggers its action by tapping.
WHAT	**Creates the Play button; its whole action sets playing to true.**
WHERE	The last child of the row, at the trailing edge.
WHEN	The action runs at the moment of a tap, never while body is being built.
WHY	Changing the state is the whole action; SwiftUI notices and re-evaluates body by itself.

```
18  .padding()
```

WHO	The padding modifier, wrapping the whole row.
WHAT	**Adds standard spacing inside all four edges of the row.**
WHERE	The first modifier on the row itself, after its closing brace.
WHEN	Applied before the modifiers that follow it in the chain.
WHY	Content that touches its container's edges looks broken; padding gives the row its breathing room.

```
19  .background(.thinMaterial)
```

WHO	The background modifier, wrapping everything written before it.
WHAT	**Paints a thin material behind the row, a translucent fill that adapts to light and dark screens.**
WHERE	Second in the modifier chain on the row.
WHEN	Its size is decided by the view built so far, padding included.
WHY	A material background groups the row's contents into one card the eye can find in a list.

```
20  .clipShape(RoundedRectangle(cornerRadius: 12))
```

WHO	The clipShape modifier, the last one in the chain.
WHAT	**Clips the row, background included, to a rounded rectangle with 12-point corners.**
WHERE	After background, so the material's corners are the ones being rounded.
WHEN	Applied to the finished row description on every redraw.
WHY	Rounded corners are the visual cue that separates a card from a plain colored block.

```
24  #Preview { NowPlayingRow() }
```

WHO	Xcode and this page, which render the row without running an app.
WHAT	**Creates one NowPlayingRow for the preview to display.**
WHERE	At file scope, after NowPlayingRow; preview code is not part of the shipped app.
WHEN	Each time the preview refreshes.
WHY	Seeing the row while you build it is how the frames in this example were checked.

Try it

The starter models a player whose Play action was never wired: it prints the status while playing is still false. Add the line the button performs, setting playing to true before the status is printed.

```
var playing = false
print(playing ? "Playing" : "Paused")
```

EXPECTED OUTPUT

```
Playing
```

WORKED EXAMPLE

Repair a cramped row for large text and a small phone

Problem.

A lesson row looks fine at the default size but squeezes at accessibility text sizes on the smallest phone. Repair the same row so every word stays readable there. Success: on iPhone SE at accessibility size 1, the Captions toggle shows Captions on.

Frame 1 of 6

At the default size on a regular phone, the row looks fine: icon, title, length and button on one line. This is the version that passes review and ships, because nothing looks broken yet.

```
import SwiftUI

struct LessonRow: View {
    var body: some View {
        HStack(spacing: 12) {
            Image(systemName: "video")
            Text("Lesson 4: Closures")
            Text("12 minutes")
            Spacer()
            Button("Watch") { }
        }
        .padding()
    }
}

#Preview { LessonRow() }
```

Frame 2 of 6

Same code, harder conditions: an iPhone SE with Dynamic Type at accessibility 1. The title and length now fight the button for one line, and the words start to squeeze.

```
import SwiftUI

struct LessonRow: View {
    var body: some View {
        HStack(spacing: 12) {
            Image(systemName: "video")
            Text("Lesson 4: Closures")
            Text("12 minutes")
            Spacer()
            Button("Watch") { }
        }
        .padding()
    }
}

#Preview { LessonRow() }
```

Frame 3 of 6

The first repair is the cheap one: tighter spacing and no Spacer, so the pieces stop pushing each other. It gains a little room, but at this text size one line is still not enough.

```
import SwiftUI

struct LessonRow: View {
    var body: some View {
+       HStack(spacing: 4) {
            Image(systemName: "video")
            Text("Lesson 4: Closures")
            Text("12 minutes")
+           Button("Watch") { }
+       }
+       .padding()
+   }
+ }
+
+ #Preview { LessonRow() }
```

Frame 4 of 6

The real repair changes the arrangement: a VStack gives the title its own line, the length sits underneath in secondary gray, and the button drops below. Nothing can squeeze, because nothing shares a line.

```
  import SwiftUI

  struct LessonRow: View {
      var body: some View {
+         VStack(alignment: .leading, spacing: 8) {
+             HStack(spacing: 8) {
+                 Image(systemName: "video")
+                 Text("Lesson 4: Closures")
+             }
+             Text("12 minutes")
+                 .foregroundStyle(.secondary)
+             Button("Watch") { }
+         }
+         .padding()
+     }
+ }
+
+ #Preview { LessonRow() }
```

Frame 5 of 6

Spacing scales up to 12 points between groups, the title becomes a headline, and the button grows to a 44-point tall target. The layout now spends the small phone's height instead of its width.

```
  import SwiftUI

  struct LessonRow: View {
      var body: some View {
+         VStack(alignment: .leading, spacing: 12) {
+             HStack(spacing: 12) {
                  Image(systemName: "video")
                  Text("Lesson 4: Closures")
+                     .font(.headline)
+             }
+             Text("12 minutes")
+                 .foregroundStyle(.secondary)
+             Button("Watch") { }
+                 .frame(minHeight: 44)
+         }
+         .padding()
+     }
+ }
+
+ #Preview { LessonRow() }
```

Frame 6 of 6

The finished row adds a Captions toggle with a status line that reads the same state. Run the check at this size and device: tapping Captions turns the status to Captions on.

```
  import SwiftUI

  struct LessonRow: View {
+     @State private var captions = false
+     var body: some View {
+         VStack(alignment: .leading, spacing: 12) {
+             HStack(spacing: 12) {
+                 Image(systemName: "video")
+                 Text("Lesson 4: Closures")
+                     .font(.headline)
+             }
+             Text("12 minutes")
+                 .foregroundStyle(.secondary)
+             Button("Watch") { }
+                 .frame(minHeight: 44)
+             Toggle("Captions", isOn: $captions)
+             Text(captions ? "Captions on" : "Captions off")
+         }
+         .padding()
+     }
+ }
+
+ #Preview { LessonRow() }
```

FINAL RESULT — CHECKED ON SCREEN

```
Tap “Captions” and the screen shows “Captions on”.
```

Line by line: the finished program

1 `import SwiftUI`

- **WHO** The Swift compiler, which loads the SwiftUI framework for this file.
- **WHAT** **Makes View, Text, VStack, HStack, Image, Button and Toggle available in this file.**
- **WHERE** The first line of the file; an import covers only its own file.
- **WHEN** At compile time, before any of the code below is checked.
- **WHY** SwiftUI is a separate framework, so every file that describes a screen imports it.

3 `struct LessonRow: View {`

- **WHO** You declare it; SwiftUI creates values of it wherever a lesson is listed.
- **WHAT** **Declares LessonRow, a struct that conforms to View and describes one lesson row.**
- **WHERE** At file scope, so the preview at the bottom can create it.
- **WHEN** Compiled once; a fresh value is created whenever a parent asks for the row.
- **WHY** This row is the repair patient: the same code is judged at two sizes before it is changed.

4 `@State private var captions = false`

- **WHO** SwiftUI, which stores the value outside the struct, tied to this row on screen.
- **WHAT** **Declares captions, whether captions are on, starting false; @State makes SwiftUI keep and watch it.**
- **WHERE** Inside LessonRow, marked private because only this row changes it.
- **WHEN** Created when the row first appears and kept across every redraw.
- **WHY** The Toggle below binds to this one value, and the status line reads it, so they can never disagree.

```
5  var body: some View {
```

WHO — SwiftUI calls this property; your code never calls body directly.
WHAT — **Returns the description of the row for the current data and the current conditions.**
WHERE — A computed property inside LessonRow.
WHEN — When the row appears, and again whenever the device, text size or captions state changes.
WHY — The same body must serve every size, which is why the layout, not the content, is what gets repaired.

```
6  VStack(alignment: .leading, spacing: 12) {
```

WHO — SwiftUI's layout system, which measures the children and stacks them.
WHAT — **Stacks the icon-and-title line, the length, the button and the toggle vertically, leading-aligned, 12 points apart.**
WHERE — The root view returned by body.
WHEN — Measured and placed again every time body produces a new description.
WHY — The wider 12-point rhythm separates the groups: identity, detail, action, setting.

```
7  HStack(spacing: 12) {
```

WHO — SwiftUI's layout system, which measures the icon and title and places them in a row.
WHAT — **Places the icon beside the title on their own shared line, 12 points apart.**
WHERE — The first child of the stack.
WHEN — Measured and placed again every time body produces a new description.
WHY — The icon gets the same 12-point breathing room as the groups below it.

```
8  Image(systemName: "video")
```

WHO — SwiftUI, which looks the name up in the SF Symbols library built into the system.
WHAT — **Shows the video symbol, marking this row as a video lesson.**
WHERE — The first child of the small top line.
WHEN — Drawn every time body runs; the symbol scales with the text around it.
WHY — The icon keeps its place beside the title, so the row still reads as one lesson.

```
9  Text("Lesson 4: Closures")
```

WHO — Swift builds the string; SwiftUI draws it as a Text view.
WHAT — **Shows the lesson title, Lesson 4: Closures.**
WHERE — The second child of the small top line, beside the icon.
WHEN — Rebuilt every time body runs, and measured again at every text size.
WHY — On its own line the title can grow, and even wrap, without pushing anything off the screen.

```
10 .font(.headline)
```

WHO — The font modifier on the title Text directly above it.
WHAT — **Makes the title a headline: semibold and sized to lead the stack.**
WHERE — Chained under the title Text, so only the title grows in weight.
WHEN — Applied while the description is built, before the text is measured.
WHY — With the squeeze gone, hierarchy returns: title first, length second, actions last.

```
12 Text("12 minutes")
```

WHO — Swift builds the string; SwiftUI draws it as a Text view.
WHAT — **Shows the lesson length, 12 minutes.**
WHERE — A child of the stack, on its own line under the title line.
WHEN — Rebuilt every time body runs, and measured again at every text size.
WHY — On its own line the length can be as large as the reader needs without crowding the title.

```
13  .foregroundStyle(.secondary)
```

WHO The foregroundStyle modifier on the length Text directly above it.
WHAT **Colors the length with the secondary style, a gray that steps back from the title.**
WHERE Chained under the length Text.
WHEN Applied while the description is built.
WHY The length is supporting detail; secondary color keeps the title first in the hierarchy.

```
14  Button("Watch") { }
```

WHO You declare it; the student will trigger its action by tapping.
WHAT **Creates the Watch button with an empty action; watching the lesson is outside this example.**
WHERE The last child of the arrangement, at the trailing edge or below the words.
WHEN Its (empty) closure would run at the moment of a tap.
WHY The button is the row's one action, so the repair must keep it reachable and readable, not shrink it away.

```
15  .frame(minHeight: 44)
```

WHO The frame modifier, applied to the Watch button directly above it.
WHAT **Gives the button a minimum height of 44 points, the smallest comfortable touch target.**
WHERE Chained under the button, inside the stack.
WHEN Measured while layout runs, every time body produces the button.
WHY The repair is not finished when the words fit; the target must also be big enough to hit, which this module sets at 44 points.

```
16  Toggle("Captions", isOn: $captions)
```

WHO You declare it; SwiftUI draws the switch and binds it to the state.
WHAT **Creates the Captions toggle, a labeled switch that reads and writes the captions state.**
WHERE A child of the stack, under the button.
WHEN Redrawn whenever captions changes; flips when the student taps it.
WHY A system Toggle brings the standard switch, a label that scales with Dynamic Type, and VoiceOver support for free.

```
17  Text(captions ? "Captions on" : "Captions off")
```

WHO Swift builds one of the two strings from captions; SwiftUI draws it as a Text view.
WHAT **Shows Captions on when captions is true and Captions off when it is false.**
WHERE The last child of the stack, under the toggle.
WHEN Rebuilt after every change to captions, because it reads the state the toggle changes.
WHY The student gets confirmation that the tap worked, from the same single source of truth as the switch.

```
19  .padding()
```

WHO The padding modifier, wrapping the whole arrangement.
WHAT **Adds standard spacing inside all four edges of the arrangement.**
WHERE The first modifier on the root view, after its closing brace.
WHEN Measured during the same layout pass as the view it wraps.
WHY Padding keeps the enlarged text off the edges of the small screen, at every size.

```
23  #Preview { LessonRow() }
```

WHO Xcode and this page, which render the row without running an app.
WHAT **Creates one LessonRow for the preview to display.**
WHERE At file scope, after LessonRow; preview code is not part of the shipped app.
WHEN Each time the preview refreshes.
WHY The preview is where this row is tried at other devices and text sizes, frame by frame.

Try it

A row fits if its total width is at most the 320 points an iPhone SE offers. The starter totals the three pieces and declares the row fits. Find the missing cost, add it to the total, and print the honest verdict.

```
let iconWidth = 44
let titleWidth = 180
let buttonWidth = 88
let totalWidth = iconWidth + titleWidth + buttonWidth
print(totalWidth <= 320 ? "Row fits" : "Stack the row")
```

EXPECTED OUTPUT

```
Stack the row
```

WORKED EXAMPLE

Walk the VoiceOver reading order, then repair it

Problem.

VoiceOver reads a photo card in the order views appear. Walk that order, find the stops that say nothing useful, and repair them with the accessibility modifiers from this lesson. Success: every stop has a meaningful label and the decorative image is skipped.

The visual for this example in the HTML version is labelled Simulation: it models Apple frameworks or platform processes that cannot run in a web page.

Frame 1 of 8

VoiceOver visits the card in the order the views are written: image first, then title, then buttons. The first stop is the decorative photo, which VoiceOver reads aloud even though it adds nothing.

```swift
import SwiftUI

struct PhotoCard: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            Image(systemName: "photo")
            Text("Harbor at sunset")
            HStack(spacing: 24) {
                Button { } label: {
                    Image(systemName: "heart")
                }
                Button { } label: {
                    Image(systemName: "square.and.arrow.up")
                }
            }
        }
        .padding()
    }
}

#Preview { PhotoCard() }
```

Frame 2 of 8

The second stop is the title, Harbor at sunset, and it is the only stop so far that tells the listener something useful. A good reading order is built from stops like this one.

```swift
import SwiftUI

struct PhotoCard: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            Image(systemName: "photo")
            Text("Harbor at sunset")
            HStack(spacing: 24) {
                Button { } label: {
                    Image(systemName: "heart")
                }
                Button { } label: {
                    Image(systemName: "square.and.arrow.up")
                }
            }
        }
        .padding()
    }
}

#Preview { PhotoCard() }
```

Frame 3 of 8

The third stop is the heart button. It has no accessibility label, so VoiceOver can only announce Button: the listener hears that something can be tapped, but not what it does.

```swift
import SwiftUI

struct PhotoCard: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            Image(systemName: "photo")
            Text("Harbor at sunset")
            HStack(spacing: 24) {
                Button { } label: {
                    Image(systemName: "heart")
                }
                Button { } label: {
                    Image(systemName: "square.and.arrow.up")
                }
            }
        }
        .padding()
    }
}

#Preview { PhotoCard() }
```

Frame 4 of 8

The fourth stop is the share button, and it fails the same way: Button, with no label. Two of the card's four stops are dead ends, so the card needs repair, not polish.

```swift
import SwiftUI

struct PhotoCard: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            Image(systemName: "photo")
            Text("Harbor at sunset")
            HStack(spacing: 24) {
                Button { } label: {
                    Image(systemName: "heart")
                }
                Button { } label: {
                    Image(systemName: "square.and.arrow.up")
                }
            }
        }
        .padding()
    }
}

#Preview { PhotoCard() }
```

Frame 5 of 8

The first repair hides the decorative image with accessibilityHidden(true), the modifier this module used for the fork-and-knife picture. The image still shows on screen; VoiceOver now skips it and starts at the

title.

```
  import SwiftUI

  struct PhotoCard: View {
      var body: some View {
          VStack(alignment: .leading, spacing: 12) {
+             Image(systemName: "photo").accessibilityHidden(true)
              Text("Harbor at sunset")
              HStack(spacing: 24) {
                  Button { } label: {
                      Image(systemName: "heart")
                  }
                  Button { } label: {
                      Image(systemName: "square.and.arrow.up")
                  }
              }
          }
          .padding()
      }
  }

  #Preview { PhotoCard() }
```

Frame 6 of 8

The second repair names the heart button: accessibilityLabel("Favorite") goes on the button, after its label closure. VoiceOver now announces Favorite, button, and Voice Control users can say the name.

```
  import SwiftUI

  struct PhotoCard: View {
      var body: some View {
          VStack(alignment: .leading, spacing: 12) {
              Image(systemName: "photo").accessibilityHidden(true)
              Text("Harbor at sunset")
              HStack(spacing: 24) {
                  Button { } label: {
                      Image(systemName: "heart")
                  }
+                 .accessibilityLabel("Favorite")
+                 Button { } label: {
+                     Image(systemName: "square.and.arrow.up")
+                 }
+             }
+         }
+         .padding()
+     }
+ }
+
+ #Preview { PhotoCard() }
```

Frame 7 of 8

The third repair names the share button Share Photo, matching the words this module gave the same symbol in its Label lesson. Now every remaining stop in the diagram has a meaningful label.

```
  import SwiftUI

  struct PhotoCard: View {
      var body: some View {
          VStack(alignment: .leading, spacing: 12) {
              Image(systemName: "photo").accessibilityHidden(true)
              Text("Harbor at sunset")
              HStack(spacing: 24) {
                  Button { } label: {
                      Image(systemName: "heart")
                  }
                  .accessibilityLabel("Favorite")
                  Button { } label: {
                      Image(systemName: "square.and.arrow.up")
                  }
+                 .accessibilityLabel("Share Photo")
+             }
+         }
+         .padding()
+     }
+ }
+
+ #Preview { PhotoCard() }
```

Frame 8 of 8

Walk the repaired card once more: title, Favorite, Share Photo, three stops and every one of them meaningful. The code did not move; only the accessibility modifiers changed what VoiceOver meets.

```swift
import SwiftUI

struct PhotoCard: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            Image(systemName: "photo").accessibilityHidden(true)
            Text("Harbor at sunset")
            HStack(spacing: 24) {
                Button { } label: {
                    Image(systemName: "heart")
                }
                .accessibilityLabel("Favorite")
                Button { } label: {
                    Image(systemName: "square.and.arrow.up")
                }
                .accessibilityLabel("Share Photo")
            }
        }
        .padding()
    }
}

#Preview { PhotoCard() }
```

Line by line: the finished program

1 `import SwiftUI`

- **WHO** The Swift compiler, which loads the SwiftUI framework for this file.
- **WHAT** **Makes View, Text, VStack, HStack, Image, Button and the accessibility modifiers available in this file.**
- **WHERE** The first line of the file; an import covers only its own file.
- **WHEN** At compile time, before any of the code below is checked.
- **WHY** Accessibility in SwiftUI is not a separate framework; it is part of every view.

3 `struct PhotoCard: View {`

- **WHO** You declare it; SwiftUI creates values of it wherever a photo is shown.
- **WHAT** **Declares PhotoCard, a struct that conforms to View and describes one photo card.**
- **WHERE** At file scope, so the preview at the bottom can create it.
- **WHEN** Compiled once; a fresh value is created whenever a parent asks for the card.
- **WHY** A photo card is where unlabeled icon buttons are most common, which is why it is the patient here.

4 `var body: some View {`

- **WHO** SwiftUI calls this property; your code never calls body directly.
- **WHAT** **Returns the description of the card for the current data.**
- **WHERE** A computed property inside PhotoCard.
- **WHEN** When the card appears, and again if anything it reads changes.
- **WHY** VoiceOver reads the accessibility tree this body builds, in the order the views are written.

5 `VStack(alignment: .leading, spacing: 12) {`

- **WHO** SwiftUI's layout system, which measures the children and stacks them.
- **WHAT** **Stacks the photo, the title and the button row vertically, leading-aligned, 12 points apart.**
- **WHERE** The root view returned by body.
- **WHEN** Measured and placed again every time body produces a new description.
- **WHY** Top-to-bottom in the stack is the order VoiceOver will visit: the layout is the reading order.

```
6  Image(systemName: "photo").accessibilityHidden(true)
```

WHO — SwiftUI, which looks the name up in the SF Symbols library, and the accessibility system, which honors the chained modifier.

WHAT — **Shows the photo symbol and hides it from accessibility tools with accessibilityHidden(true).**

WHERE — The first child of the stack, at the top of the card; the modifier is chained on the same line.

WHEN — Drawn every time body runs; skipped by VoiceOver on every pass.

WHY — The picture is decorative, so hiding it removes a useless stop while the card looks exactly the same.

```
7  Text("Harbor at sunset")
```

WHO — Swift builds the string; SwiftUI draws it as a Text view.

WHAT — **Shows the photo's title, Harbor at sunset.**

WHERE — A child of the stack, under the photo.

WHEN — Rebuilt every time body runs, though the words never change.

WHY — A Text view is readable by VoiceOver exactly as written: this stop needs no repair.

```
8  HStack(spacing: 24) {
```

WHO — SwiftUI's layout system, which measures the two buttons and spaces them.

WHAT — **Places the two icon buttons side by side, 24 points apart.**

WHERE — A child of the stack, under the title.

WHEN — Measured and placed again every time body produces a new description.

WHY — Left-to-right in this row is the order VoiceOver visits the buttons.

```
9  Button { } label: {
```

WHO — You declare it; the listener triggers its action by tapping or double-tapping.

WHAT — **Opens the Favorite button; the closure before label is empty here, and the label closure follows.**

WHERE — The first child of the button row.

WHEN — Its action would run at the moment of activation.

WHY — A real Button gets the button trait and activation for free; only its name is missing.

```
10 Image(systemName: "heart")
```

WHO — SwiftUI, which looks the name up in the SF Symbols library built into the system.

WHAT — **Shows a heart symbol as the button's only content.**

WHERE — Inside the Favorite button's label closure, as its only content.

WHEN — Drawn every time body runs.

WHY — A picture is all this button shows, so a listener who cannot see it gets nothing from the label closure.

```
12 .accessibilityLabel("Favorite")
```

WHO — The accessibilityLabel modifier on the Favorite button directly above it; VoiceOver and Voice Control read it.

WHAT — **Names the heart button Favorite, the word VoiceOver announces and Voice Control users can say.**

WHERE — Chained under the Favorite button, after its label closure.

WHEN — Read whenever an assistive technology focuses or lists the button.

WHY — Without it VoiceOver can only guess from the picture, and Voice Control users have no name to say.

```
13 Button { } label: {
```

WHO — You declare it; the listener triggers its action by tapping or double-tapping.

WHAT — **Opens the Share button; the closure before label is empty here, and the label closure follows.**

WHERE — The second child of the button row, after the Favorite button.

WHEN — Its action would run at the moment of activation.

WHY — Same shape as the first button, same missing piece: a name.

14 `Image(systemName: "square.and.arrow.up")`

- **WHO** SwiftUI, which looks the name up in the SF Symbols library built into the system.
- **WHAT** **Shows the share symbol, a box with an arrow rising from it, as the button's only content.**
- **WHERE** Inside the Share button's label closure, as its only content.
- **WHEN** Drawn every time body runs.
- **WHY** Sighted users know this symbol means share; VoiceOver does not speak symbols, so the meaning must be supplied in words.

16 `.accessibilityLabel("Share Photo")`

- **WHO** The accessibilityLabel modifier on the Share button directly above it; VoiceOver and Voice Control read it.
- **WHAT** **Names the share button Share Photo, in the words this module's Label lesson gave the same symbol.**
- **WHERE** Chained under the Share button, after its label closure.
- **WHEN** Read whenever an assistive technology focuses or lists the button.
- **WHY** The label removes the guesswork: the button is named by the action it performs.

19 `.padding()`

- **WHO** The padding modifier, wrapping the whole stack.
- **WHAT** **Adds standard spacing inside all four edges of the card.**
- **WHERE** The first modifier on the stack itself, after its closing brace.
- **WHEN** Measured during the same layout pass as the stack it wraps.
- **WHY** Padding is part of the visual card; it has no effect on the reading order.

23 `#Preview { PhotoCard() }`

- **WHO** Xcode and this page, which render the card without running an app.
- **WHAT** **Creates one PhotoCard for the preview to display.**
- **WHERE** At file scope, after PhotoCard; preview code is not part of the shipped app.
- **WHEN** Each time the preview refreshes.
- **WHY** The preview shows the sighted card; the diagram beside each frame shows the card VoiceOver meets.

Try it

Elements carry a reading priority: lower numbers are read first. The starter prints the card's elements in the order they were stored, which is wrong. Sort them by priority and print one label per line, in reading order.

```
let elements = [("Share Photo", 3), ("Harbor at sunset", 1), ("Favorite", 2)]
for element in elements {
    print(element.0)
}
```

EXPECTED OUTPUT

```
Harbor at sunset
Favorite
Share Photo
```

MODULE 06

Storing Data: Local Storage and Databases

Choose where each piece of app data belongs, then store it: settings with @AppStorage, documents with Codable and FileManager, records with SwiftData, secrets in the Keychain, and shared data in iCloud with CloudKit.

- Store application data locally and in a database
- Integrate APIs such as location services and cloud storage

Where App Data Lives

CONCEPT

Six places your data can live

Every app juggles data: settings, the user's notes, downloaded images, a sign-in token. Where you keep each piece decides whether it survives a relaunch, a reinstall, or a new phone.

The simplest place is **memory**: the variables and objects your code creates while it runs. Memory is fast but temporary. iOS can end a background app at any moment to free resources, and anything held only in memory disappears with it. The other five places are *persistent*, meaning the data outlives the running app:

UserDefaults	A small key-value store for settings such as "dark mode on". In SwiftUI you use it through @AppStorage.
Files	Whole documents, JSON, images, or audio that you write into your app's folders with FileManager.
A database	Many structured records you can search, sort, and connect. Apple's modern framework is SwiftData.
The Keychain	An encrypted system vault for secrets such as passwords and access tokens.
The cloud	Copies on servers so data follows the user to every device: iCloud, CloudKit, or your own backend.

CONCEPT

The app sandbox

iOS gives every installed app its own private folder tree, called its **container**. Your code can read and write inside its container, but it cannot open another app's files, and other apps cannot open yours. This wall is the **app sandbox**, a core part of iOS security and privacy.

Folders inside the container have different jobs. Select a folder in the figure to see what belongs there, whether it is backed up, and the Swift property that finds it. Then clean up storage and delete the app to see what each folder loses.

Interactive figure in the HTML version: data sandbox files.

CONCEPT

What survives: backups, cleanup, deletion

Container folders differ in two ways that matter: whether iOS copies them into the user's iCloud or computer backup, and whether iOS may delete their contents on its own when the device runs low on space.

Documents	Backed up, never purged. Files the user created, such as notes and drawings.
Library/Application Support	Backed up, never purged. Your app's own data files, including the SwiftData database.

Library/Caches	Not backed up. iOS may empty it when storage is low (never while your app runs). Keep only data you can download or rebuild.
tmp	Not backed up. iOS may clear it whenever your app is not running. Short-lived scratch files only.

> **Deleting the app deletes the container**
>
> When the user deletes your app, iOS removes its whole container, and anything not synced to the cloud is gone. Large files you can download again should not bloat the user's backup: keep them in Caches, or set `isExcludedFromBackup` on them.

CODE · READ-ALONG

Save before iOS stops your app

An app moves between *scene phases*: `.active` (on screen, receiving touches), `.inactive`, and `.background`. In the background it can be suspended and later ended without warning. This view writes the draft to a file the moment the app moves to the background, so typing is never lost.

```swift
import SwiftUI

struct DraftView: View {
    @Environment(\.scenePhase) private var scenePhase
    @State private var draft = ""

    var body: some View {
        TextEditor(text: $draft)
            .onChange(of: scenePhase) { _, newPhase in
                if newPhase == .background {
                    saveDraft()
                }
            }
    }

    private func saveDraft() {
        let url = URL.documentsDirectory
            .appending(path: "draft.txt")
        try? Data(draft.utf8).write(to: url, options: .atomic)
    }
}
```

Line by line

`1` `import SwiftUI`

WHO	The Swift compiler, while it builds the app.
WHAT	**Makes SwiftUI available in this file, and with it Foundation types such as URL and Data that the save code uses.**
WHERE	At the top of the file; an import applies only to the file it appears in.
WHEN	At compile time, before any code runs.
WHY	SwiftUI re-exports Foundation, so a single import covers both the views and the file APIs here.

```
3  struct DraftView: View {
```

WHO Your code, defining a SwiftUI view type.
WHAT **Declares a view named DraftView that shows an editor for a text draft.**
WHERE A value type in your app module; SwiftUI creates and discards instances freely.
WHEN An instance is created whenever a parent view's body includes DraftView().
WHY Views are lightweight descriptions of the screen, so the data they show must be kept somewhere more durable, which is the point of this example.

```
4  @Environment(\.scenePhase) private var scenePhase
```

WHO SwiftUI, which tracks the state of every scene (window) of your app.
WHAT **Reads the current scene phase, which is .active, .inactive, or .background.**
WHERE A property of DraftView whose value SwiftUI supplies from the environment.
WHEN Updated whenever the user switches apps, locks the device, or comes back.
WHY Watching the scene phase is the recommended way to learn that your app is about to leave the screen.

```
5  @State private var draft = ""
```

WHO SwiftUI, which stores @State values outside the struct.
WHAT **Holds the text the user is typing, starting as an empty string.**
WHERE In memory owned by SwiftUI for this view, not on disk.
WHEN Created when the view first appears and kept while it stays on screen.
WHY Memory is the fast working copy, but it vanishes if iOS ends the app; the save below protects it.

```
7  var body: some View {
```

WHO SwiftUI, which reads body to learn what to draw.
WHAT **Describes the content of this view.**
WHERE A computed property of DraftView.
WHEN Every time a value it depends on, such as draft, changes.
WHY SwiftUI may compute body many times per second, so slow work such as writing files never belongs directly inside it.

```
8  TextEditor(text: $draft)
```

WHO SwiftUI's multiline text editor, and the user typing in it.
WHAT **Shows an editable text area bound to draft, so each keystroke updates the state.**
WHERE Fills the view on screen.
WHEN Drawn whenever body runs; it writes to draft as the user types.
WHY The $ creates a binding, a two-way connection, so the editor and the save function share one value.

```
9  .onChange(of: scenePhase) { _, newPhase in
```

WHO SwiftUI, which compares each new scenePhase with the previous one.
WHAT **Runs the closure whenever scenePhase changes, passing the old and the new phase.**
WHERE A modifier on the TextEditor; the closure runs on the main thread.
WHEN Each time the app moves between active, inactive, and background.
WHY The underscore ignores the old phase because only the new one matters; this two-parameter closure is the current form of onChange.

```
10 if newPhase == .background {
```

WHO Your code, inside the change handler.
WHAT **Checks whether the app has just moved to the background.**
WHERE Inside the onChange closure.
WHEN Immediately after the phase changes.
WHY Background is the last moment you can count on: afterwards iOS may suspend the app and end it without running any more of your code.

```
11  saveDraft()
```

WHO — Your code, calling the helper method below.
WHAT — **Writes the current draft to a file.**
WHERE — On the main thread, inside the handler.
WHEN — Once each time the app goes to the background.
WHY — Saving at this transition guards the text against the app being ended while the user is elsewhere.

```
16  private func saveDraft() {
```

WHO — DraftView, which owns this helper.
WHAT — **Declares a method that saves the draft to disk.**
WHERE — A private method of DraftView, invisible outside the struct.
WHEN — Only when called; here, when the app enters the background.
WHY — A named function makes the intent obvious and keeps side effects out of body.

```
17      let url = URL.documentsDirectory
```

WHO — Foundation, which knows where your container is on this device.
WHAT — **Gets the URL of the Documents folder inside your app's container.**
WHERE — A local constant in saveDraft; the folder itself is in the sandbox on the device.
WHEN — Each time saveDraft runs.
WHY — Never type container paths by hand: the location differs from device to device and from install to install.

```
18          .appending(path: "draft.txt")
```

WHO — Foundation's URL type, through its appending(path:) method, continuing the expression started on the line above.
WHAT — **Builds a new URL for a file named draft.txt inside Documents.**
WHERE — Continues the expression from the previous line.
WHEN — Each time saveDraft runs.
WHY — appending(path:) inserts the path separator correctly, which string concatenation easily gets wrong.

```
19      try? Data(draft.utf8).write(to: url, options: .atomic)
```

WHO — Foundation's Data type and the iOS file system.
WHAT — **Turns the text into UTF-8 bytes and writes them to draft.txt, replacing any older file.**
WHERE — On the device's storage, in the Documents folder.
WHEN — Each time the app moves to the background.
WHY — .atomic writes a temporary file first and then swaps it in, so a crash can never leave half a draft; try? ignores errors only to keep the example short, and a real app should report them.

> **Try it**
>
> 1. Change .background to .inactive. Which everyday actions, such as pulling down Control Center, would now trigger a save?
> 2. Where would you read draft.txt back into draft so the text reappears after a relaunch?

INTERACTIVE

Pick the right home for the data

Pick an example, or describe your own data by answering the questions about its size, shape, sensitivity, and who needs it. The tool recommends where to store it, explains why, and shows the Swift API you would use. Compare *Login token* with *10,000 recipes*, then read the "What would change the answer?" list: often a single answer decides everything.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. A fitness app lets the user pick a daily step goal, a single number such as 8,000. It must survive relaunches. Where should it live?
   A. The Keychain, through a small wrapper type
   B. UserDefaults, through @AppStorage
   C. A SwiftData database with one model
   D. Library/Caches, as a small text file

2. Your app downloads 300 MB of offline map tiles that can be downloaded again at any time. Which choice treats the user best?
   A. Documents, so the tiles are included in every backup
   B. UserDefaults, so the tiles are ready at launch
   C. The Keychain, so the tiles stay encrypted on the device
   D. Library/Caches, which iOS may clear and never backs up

3. Which statement about the app sandbox is true?
   A. Apps from the same developer can always read each other's Documents folders
   B. Files in tmp stay until your own code deletes them
   C. Your app reads and writes files only inside its own container
   D. Deleting the app keeps its Documents folder for the next install

4. A user writes a long journal entry, switches to another app, and overnight iOS ends your app to free memory. What protects the entry?
   A. Saving the text to a file when scenePhase turns .background
   B. Keeping the text in a @State property
   C. Writing it to tmp the next time the app launches, before body runs
   D. Writing it when iOS sends the final termination notice to the app

Answers: Where App Data Lives

1. **B. UserDefaults, through @AppStorage** A single, small, non-secret preference is exactly what UserDefaults is for, and @AppStorage reads and writes it from SwiftUI. The Keychain would work but is built for secrets and adds complexity for nothing. Caches is wrong because iOS may empty it, and a database is far more machinery than one number needs.
2. **D. Library/Caches, which iOS may clear and never backs up** Re-downloadable data belongs in Caches, or in Application Support marked with isExcludedFromBackup if it must not disappear. Putting 300 MB in Documents fills the user's iCloud backup with data that could simply be fetched again. UserDefaults and the Keychain are for tiny values and would be badly misused.
3. **C. Your app reads and writes files only inside its own container** Each app is confined to its own container; reaching anything else goes through system services, such as the photo picker. Apps from one developer share data only through an explicitly configured App Group. iOS can clear tmp whenever the app is not running, and deleting an app removes its entire container, Documents included.
4. **A. Saving the text to a file when scenePhase turns .background** A suspended app can be ended without running any more of your code, so there is no reliable last-chance notice. Save when the phase changes to .background. @State lives only in memory, and by the next launch the text is already gone.

Settings with @AppStorage

CONCEPT

UserDefaults: a small settings file

UserDefaults is a key-value store: you save a value under a string key, such as `"username"`, and read it back later with the same key. iOS keeps your defaults in a *property list* (a structured file format, often called a plist) in `Library/Preferences` inside your container, so values survive relaunches and are included in backups.

The first time your app reads a default, the whole file is loaded into memory and cached. That makes reads instant, and it is exactly why the file must stay small.

Good fits	Preferences and flags: theme, sort order, "has seen onboarding", the last selected tab.
Value types	Bool, Int, Double, String, Date, Data, URL, and arrays or dictionaries built from them.
Missing keys	Reading a key that was never saved gives a fallback: false, 0, 0.0, or nil.

CODE · READ-ALONG

The UserDefaults API

Every setting ultimately goes through `UserDefaults.standard`, the shared defaults object for your app. This snippet writes three values, reads them back with type-specific getters, reads a key that was never written, registers a fallback, and removes a key. It is shown rather than run, because the values belong to an app on a device.

```
import Foundation

let defaults = UserDefaults.standard

defaults.set(true, forKey: "hasFinishedOnboarding")
defaults.set(3, forKey: "launchCount")
defaults.set("Riley", forKey: "username")

let finished = defaults.bool(forKey: "hasFinishedOnboarding")
let launches = defaults.integer(forKey: "launchCount")
let name = defaults.string(forKey: "username") ?? "Guest"
let unknown = defaults.integer(forKey: "neverSaved")

defaults.register(defaults: ["fontSize": 17.0])
let size = defaults.double(forKey: "fontSize")

defaults.removeObject(forKey: "username")
```

Line by line

```
1  import Foundation
```

WHO	The Swift compiler, resolving the Foundation module so UserDefaults and its typed getters are known to this file.
WHAT	**Makes Foundation available; UserDefaults is a Foundation class.**
WHERE	At the top of this file.
WHEN	At compile time, before any line of this snippet runs; nothing is imported at run time.
WHY	Plain Swift files need this import for UserDefaults; SwiftUI files get Foundation automatically.

```
3  let defaults = UserDefaults.standard
```

WHO	Foundation, which provides one shared defaults object per app.
WHAT	**Gets the standard defaults object, which reads and writes your app's preferences file.**
WHERE	A constant referring to a shared object; the stored data lives in Library/Preferences in your container.
WHEN	The first access loads and caches the stored values.
WHY	Using the shared instance means every part of the app sees the same values; you rarely create your own.

```
5  defaults.set(true, forKey: "hasFinishedOnboarding")
```

WHO	UserDefaults.standard, through its set(_:forKey:) method, writing into the preferences cache for this app.
WHAT	**Stores the Bool value true under the key hasFinishedOnboarding.**
WHERE	In the in-memory cache immediately; iOS writes the plist file to disk on its own schedule.
WHEN	When this line runs, typically after the user taps a button.
WHY	Keys are plain strings, so a typo silently creates a different setting; choose clear names and reuse them.

```
6  defaults.set(3, forKey: "launchCount")
```

WHO	UserDefaults.standard again, the same set(_:forKey:) method accepting an Int this time.
WHAT	**Stores the Int value 3 under the key launchCount.**
WHERE	In the same preferences cache and file.
WHEN	When this line runs.
WHY	One method, set(_:forKey:), accepts every property-list type: numbers, strings, dates, and data.

```
7  defaults.set("Riley", forKey: "username")
```

WHO	UserDefaults.standard, with set(_:forKey:) storing a String value under its own key.
WHAT	**Stores the String Riley under the key username.**
WHERE	In the same preferences cache and file.
WHEN	When this line runs.
WHY	A display name is fine here because it is not secret; a password never belongs in UserDefaults.

```
9  let finished = defaults.bool(forKey: "hasFinishedOnboarding")
```

WHO	UserDefaults, answering from its in-memory cache.
WHAT	**Reads the Bool back, so finished is true.**
WHERE	A local constant; the read touches only memory, not the disk.
WHEN	When this line runs.
WHY	Typed getters such as bool(forKey:) return a plain value instead of an optional, which keeps reading code short.

```
10 let launches = defaults.integer(forKey: "launchCount")
```

WHO	UserDefaults.standard, through integer(forKey:), answering from the cache it filled on first access.
WHAT	**Reads the Int back, so launches is 3.**
WHERE	A local constant named launches in this snippet, holding a plain Int copied out of the defaults cache.
WHEN	When this line runs.
WHY	integer(forKey:) converts the stored number to an Int for you.

```swift
let name = defaults.string(forKey: "username") ?? "Guest"
```

WHO — UserDefaults, plus the nil-coalescing operator ??.
WHAT — **Reads the String; because string(forKey:) returns an optional, ?? supplies Guest when the key is missing.**
WHERE — A local constant named name, which holds a non-optional String because ?? already supplied the fallback.
WHEN — When this line runs.
WHY — Strings, data, and arrays have no natural zero value, so their getters return nil for a missing key.

```swift
let unknown = defaults.integer(forKey: "neverSaved")
```

WHO — UserDefaults.standard, through integer(forKey:), finding no entry for neverSaved and returning 0.
WHAT — **Reads a key that was never written; unknown is 0.**
WHERE — A local constant named unknown, holding the 0 that a missing Int key produces.
WHEN — When this line runs.
WHY — Missing number and Bool keys quietly return 0 or false, so you cannot tell never-set apart from set-to-zero; register defaults to make that explicit.

```swift
defaults.register(defaults: ["fontSize": 17.0])
```

WHO — The registration domain of UserDefaults, a layer of fallback values.
WHAT — **Registers 17.0 as the fallback for fontSize without saving anything to disk.**
WHERE — In memory only, for the current run of the app.
WHEN — Early at every launch, before any code reads these keys.
WHY — Registered defaults give missing keys sensible values while still letting a value the user saved win.

```swift
let size = defaults.double(forKey: "fontSize")
```

WHO — UserDefaults, checking saved values first and registered values second.
WHAT — **Reads fontSize; it is 17.0 because nothing was saved, so the registered fallback applies.**
WHERE — A local constant named size, a Double copied from the registration domain rather than from a saved value.
WHEN — When this line runs.
WHY — It shows the lookup order: a saved value, then a registered default, then zero.

```swift
defaults.removeObject(forKey: "username")
```

WHO — UserDefaults.standard, through removeObject(forKey:), dropping the username entry from the cache and the plist.
WHAT — **Deletes the username key entirely.**
WHERE — Removes it from the cache now and from the plist file soon after.
WHEN — When this line runs, for example when the user resets their profile.
WHY — After removal, reads return the fallback again, which is how you implement a reset of a preference.

> **Try it**
>
> 1. Predict the value of name if removeObject runs before the line that reads username.
> 2. Why might register(defaults:) be better than writing 17.0 the first time the app launches?

CODE · LIVE SWIFTUI PREVIEW

An onboarding flag that persists

`@AppStorage("key")` is a SwiftUI *property wrapper* (a type that adds behavior to a property) that reads and writes one UserDefaults key and refreshes the view whenever the value changes. Tap **Get Started**, then reload this page: the phone remembers your choice, because this preview really saves the value in your browser.

```swift
import SwiftUI

struct ContentView: View {
    @AppStorage("hasFinishedOnboarding") var isOnboarded = false

    var body: some View {
        VStack(spacing: 16) {
            if isOnboarded {
                Label("Home", systemImage: "house")
                    .font(.largeTitle)
                Button("Replay onboarding") {
                    isOnboarded = false
                }
            } else {
                Text("Welcome to Trailhead").font(.title)
                Button("Get Started") {
                    isOnboarded = true
                }
                .buttonStyle(.borderedProminent)
            }
        }
    }
}

#Preview { ContentView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so View, VStack, Label, Button and @AppStorage are known here.
- **WHAT** **Makes SwiftUI, including @AppStorage, available in this file.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before ContentView or its stored flag exists; nothing is imported at run time.
- **WHY** Each file imports the frameworks it uses.

3 `struct ContentView: View {`

- **WHO** Your ContentView struct, a SwiftUI view type you declare for the onboarding example.
- **WHAT** **Declares the screen that shows either a welcome page or the home page.**
- **WHERE** A SwiftUI view type in your app module.
- **WHEN** Created when the preview or the app shows ContentView().
- **WHY** One view with a flag keeps the example small; a real app might switch between two separate views.

4 `@AppStorage("hasFinishedOnboarding") var isOnboarded = false`

- **WHO** SwiftUI, connecting this property to a key in UserDefaults.standard.
- **WHAT** **Declares a Bool tied to the key hasFinishedOnboarding, with false used only when the key has never been saved.**
- **WHERE** The value is stored in UserDefaults, not in the struct; the struct holds just the wrapper.
- **WHEN** Read when the view is created; written whenever the property is assigned.
- **WHY** The initial value is a fallback, not a reset: it never overwrites a value the user already saved. The property name and the key do not have to match.

```
6  var body: some View {
```

WHO SwiftUI, which reads ContentView's body every time isOnboarded changes.
WHAT **Describes what the screen shows.**
WHERE A computed property of ContentView.
WHEN Again every time isOnboarded changes.
WHY Because @AppStorage triggers a refresh, the screen always matches the stored value.

```
7  VStack(spacing: 16) {
```

WHO SwiftUI's VStack layout container, laying out the Label, Text and Buttons inside it.
WHAT **Stacks its children vertically with 16 points between them.**
WHERE Centered on the screen.
WHEN Each time ContentView.body runs, which happens after every change to isOnboarded.
WHY One container for both states keeps the layout stable when the flag flips.

```
8  if isOnboarded {
```

WHO Your code, inside a SwiftUI view builder.
WHAT **Chooses which content to show based on the stored flag.**
WHERE Inside the VStack of ContentView.body, as the first statement of the view builder.
WHEN Each time body runs.
WHY SwiftUI view builders accept if and else, so the stored value directly selects the screen.

```
9  Label("Home", systemImage: "house")
```

WHO SwiftUI's Label view, drawing the Home text and the house SF Symbol side by side.
WHAT **Shows the word Home with a house symbol: the screen a returning user sees.**
WHERE Inside the first branch.
WHEN Only when the flag is true.
WHY Label pairs text and an icon and stays accessible to VoiceOver.

```
10 .font(.largeTitle)
```

WHO SwiftUI's font modifier, applied to the Home label on the line above.
WHAT **Makes the label use the large title text style.**
WHERE Applies to the Label above.
WHEN Whenever the label is drawn.
WHY Text styles scale with the user's Dynamic Type setting, unlike fixed point sizes.

```
11 Button("Replay onboarding") {
```

WHO SwiftUI, and the user tapping.
WHAT **Creates a button that resets the flag.**
WHERE Below the Home label, still inside the if isOnboarded branch of the VStack.
WHEN Its closure runs on each tap.
WHY A way back to onboarding makes the persistence easy to test, and many apps offer it in settings.

```
12 isOnboarded = false
```

WHO @AppStorage, writing to UserDefaults.
WHAT **Saves false under the key and refreshes the view, which shows the welcome screen again.**
WHERE Inside the button's action closure.
WHEN Each time the user taps the button.
WHY Assigning the property is all it takes; there is no separate save call.

```
14  } else {
```

WHO Your ContentView struct, using the else that SwiftUI's view builder allows inside body.
WHAT **Starts the branch shown when onboarding is not finished.**
WHERE Inside the VStack of ContentView.body, pairing with the if isOnboarded branch above.
WHEN When the flag is false.
WHY An else branch guarantees the screen always shows exactly one of the two states.

```
15      Text("Welcome to Trailhead").font(.title)
```

WHO SwiftUI's Text view, drawing the welcome line in the title text style.
WHAT **Shows the welcome message of a hiking app named Trailhead in the title text style.**
WHERE In the else branch.
WHEN While the flag is false.
WHY A first-run screen is the classic use of a persisted Bool flag; a modifier can sit on the same line when it is short.

```
16      Button("Get Started") {
```

WHO SwiftUI, and the user tapping.
WHAT **Creates the button that finishes onboarding.**
WHERE Below the welcome message.
WHEN Its closure runs on tap.
WHY The flag should change only when the user acts, not just because the screen appeared.

```
17          isOnboarded = true
```

WHO @AppStorage, writing to UserDefaults.
WHAT **Saves true under the key; the view refreshes and shows the home branch.**
WHERE Inside the action closure.
WHEN Each time the user taps the button.
WHY From now on every launch starts at Home, because the saved true beats the default false.

```
19      .buttonStyle(.borderedProminent)
```

WHO SwiftUI's buttonStyle modifier, applied to the Get Started button above it.
WHAT **Draws the button filled with the accent color.**
WHERE Applies to the Get Started button.
WHEN Whenever it is drawn.
WHY A prominent style marks the single main action on the screen.

```
25  #Preview { ContentView() }
```

WHO Xcode previews (here, the phone beside the code).
WHAT **Renders ContentView so you can try it without running a full app.**
WHERE At the bottom of the file; it is not part of the shipped app.
WHEN Whenever the preview refreshes.
WHY Previews make it quick to check both branches of a stored flag.

Try it

1. Tap Get Started, reload the page, and confirm Home appears. Then change the default to true. Does the phone change? Why not?
2. Rename the key to "onboardingDone". What does the phone show now, and why?

CODE · LIVE SWIFTUI PREVIEW

A theme picker

@AppStorage works with a Picker exactly like @State does: pass `$accentName` as the selection. The chosen tag is saved as a String, and the dictionary turns that saved name into a color. Pick a color, reload the page, and the choice is still there.

```swift
import SwiftUI

struct ThemeSettingsView: View {
    @AppStorage("accentName") private var accentName = "blue"
    let tints = ["blue": Color.blue, "green": .green, "pink": .pink]

    var body: some View {
        VStack(spacing: 24) {
            Picker("Accent", selection: $accentName) {
                Text("Blue").tag("blue")
                Text("Green").tag("green")
                Text("Pink").tag("pink")
            }
            .pickerStyle(.segmented)
            Image(systemName: "paintpalette")
                .font(.system(size: 72))
                .foregroundStyle(tints[accentName] ?? .blue)
        }
        .padding()
    }
}

#Preview { ThemeSettingsView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so Picker, Color, Image and @AppStorage are known here.
- **WHAT** **Makes SwiftUI available in this file.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before ThemeSettingsView or its stored accentName exists; nothing is imported at run time.
- **WHY** Every SwiftUI file starts with this import.

3 `struct ThemeSettingsView: View {`

- **WHO** Your ThemeSettingsView struct, a SwiftUI view type you declare for the color setting.
- **WHAT** **Declares a settings screen for choosing an accent color.**
- **WHERE** A SwiftUI view type.
- **WHEN** Created when the preview shows it.
- **WHY** A focused view per setting group keeps settings screens easy to read.

```
4  @AppStorage("accentName") private var accentName = "blue"
```

WHO The @AppStorage property wrapper from SwiftUI, reading and writing the accentName key in UserDefaults.standard.
WHAT **Stores the chosen color name under the key accentName, defaulting to blue.**
WHERE In UserDefaults; the view reads it through the wrapper.
WHEN Read on creation; written whenever the picker changes.
WHY Colors cannot be saved in UserDefaults directly, so the app saves a simple name and converts it when drawing.

```
5  let tints = ["blue": Color.blue, "green": .green, "pink": .pink]
```

WHO Your ThemeSettingsView struct, declaring a constant dictionary named tints.
WHAT **Maps each saved name to a SwiftUI Color.**
WHERE A constant property of the view.
WHEN Created with each view value.
WHY Writing Color.blue once tells Swift the dictionary type, so the other values can use the short .green form.

```
7  var body: some View {
```

WHO SwiftUI, which reads ThemeSettingsView's body every time accentName changes.
WHAT **Describes the screen.**
WHERE A computed property of the view.
WHEN Again whenever accentName changes.
WHY The stored value drives the screen, just like @State would.

```
8  VStack(spacing: 24) {
```

WHO SwiftUI's VStack layout container, holding the Picker and the palette Image.
WHAT **Stacks the picker and the preview icon vertically.**
WHERE The root of the screen.
WHEN Each time ThemeSettingsView.body runs, which happens after every change to accentName.
WHY Generous spacing separates the control from the result it controls.

```
9  Picker("Accent", selection: $accentName) {
```

WHO SwiftUI, and the user choosing.
WHAT **Shows a picker whose selection is bound to the stored accentName.**
WHERE Top of the stack.
WHEN Updates accentName as soon as the user picks a segment.
WHY The binding $accentName writes straight to UserDefaults, so no extra save code is needed.

```
10  Text("Blue").tag("blue")
```

WHO SwiftUI's Text view plus its tag modifier, which makes this option selectable in the Picker.
WHAT **Adds a Blue option whose tag value is the string blue.**
WHERE The first option inside the Picker labeled Accent, in ThemeSettingsView.body.
WHEN Whenever the picker is drawn.
WHY A tag must have the same type as the selection, here String, or the option can never be selected.

```
11  Text("Green").tag("green")
```

WHO SwiftUI's Text view plus tag, registering the second option with the Picker.
WHAT **Adds a Green option tagged green.**
WHERE The second option inside the Accent Picker, between the Blue and Pink options.
WHEN Whenever the picker is drawn.
WHY The tag string must match a key in tints for the color lookup to succeed.

```
12  Text("Pink").tag("pink")
```

WHO SwiftUI's Text view plus tag, registering the third and last option with the Picker.
WHAT **Adds a Pink option tagged pink.**
WHERE The last option inside the Accent Picker, just before the closing brace of the Picker.
WHEN Whenever the picker is drawn.
WHY Keeping the labels and tags side by side makes it easy to add another color later.

```
14  .pickerStyle(.segmented)
```

WHO SwiftUI's pickerStyle modifier, applied to the Accent Picker above.
WHAT **Draws the picker as a row of segments.**
WHERE Applies to the picker.
WHEN Whenever it is drawn.
WHY Segments suit two to five short, visible choices.

```
15  Image(systemName: "paintpalette")
```

WHO SwiftUI and SF Symbols, Apple's icon library.
WHAT **Shows a palette symbol that previews the chosen color.**
WHERE Below the segmented Picker, as the second child of the VStack in ThemeSettingsView.body.
WHEN Each time ThemeSettingsView.body runs; its color is read fresh from tints on the next line.
WHY Seeing the result immediately confirms what the setting does.

```
16  .font(.system(size: 72))
```

WHO SwiftUI's font modifier, sizing the paintpalette symbol on the line above.
WHAT **Sizes the symbol to 72 points.**
WHERE Applies to the image.
WHEN Whenever it is drawn.
WHY SF Symbols scale with the font, so a font size sets the icon size.

```
17  .foregroundStyle(tints[accentName] ?? .blue)
```

WHO Your dictionary lookup, then SwiftUI.
WHAT **Colors the symbol with the saved color, or blue if the saved name is unknown.**
WHERE Applies to the image.
WHEN Every time accentName changes.
WHY A stored value can be anything an older version of the app saved, so a fallback prevents a missing color.

```
19  .padding()
```

WHO SwiftUI's padding modifier, applied to the VStack that holds the picker and the icon.
WHAT **Adds standard space around the stack.**
WHERE Applies to the VStack.
WHEN Whenever it is drawn.
WHY Padding keeps the segmented control away from the screen edges.

```
23  #Preview { ThemeSettingsView() }
```

WHO Xcode's #Preview macro, which this page shows as the phone beside the code.
WHAT **Renders the settings screen beside the code.**
WHERE At the bottom of the file.
WHEN Whenever the preview refreshes.
WHY Previews let you test a stored setting instantly.

Try it

1. Add an Orange option. What two lines must change so the icon turns orange?
2. Choose Pink, then change the default "blue" to "green". Which color appears, and why?

CODE · LIVE SWIFTUI PREVIEW

A profile and settings form

Real settings screens mix types. Here a String, a Bool, and a Double each live under their own key, and a Form groups the controls the way the Settings app does. Type your name, toggle tips, change the text size, and reload the page: every value persists.

```swift
import SwiftUI

struct SettingsView: View {
    @AppStorage("username") private var username = ""
    @AppStorage("showTips") private var showTips = true
    @AppStorage("textSize") private var textSize = 17.0

    var body: some View {
        Form {
            TextField("Your name", text: $username)
            Toggle("Show tips", isOn: $showTips)
            Stepper("Text size: \(Int(textSize))",
                    value: $textSize, in: 12...28)
            Section("Preview") {
                Text("Hello, \(username)")
                    .font(.system(size: textSize))
                if showTips {
                    Text("Reload the page: nothing resets.")
                }
            }
        }
    }
}

#Preview { SettingsView() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so Form, TextField, Toggle, Stepper and @AppStorage are known here.
- **WHAT** **Makes SwiftUI available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before SettingsView or its three stored values exist; nothing is imported at run time.
- **WHY** Every SwiftUI file starts with this import.

3 `struct SettingsView: View {`

- **WHO** Your SettingsView struct, a SwiftUI view type you declare for the profile form.
- **WHAT** **Declares a settings form for the profile and reading preferences.**
- **WHERE** A SwiftUI view type.
- **WHEN** Created when the preview shows it.
- **WHY** Grouping related settings into one view mirrors the Settings app.

```
4  @AppStorage("username") private var username = ""
```

WHO The @AppStorage wrapper from SwiftUI, tied to the username key in UserDefaults.standard.
WHAT **Stores the user's display name as a String, empty until typed.**
WHERE In UserDefaults under the key username.
WHEN Written on every keystroke in the text field.
WHY A display name is not secret, so UserDefaults is an acceptable home.

```
5  @AppStorage("showTips") private var showTips = true
```

WHO The @AppStorage wrapper from SwiftUI, tied to the showTips key in UserDefaults.standard.
WHAT **Stores whether tips are shown, defaulting to true.**
WHERE In UserDefaults under the key showTips.
WHEN Written when the toggle flips.
WHY Bool flags are the most common kind of stored setting.

```
6  @AppStorage("textSize") private var textSize = 17.0
```

WHO The @AppStorage wrapper from SwiftUI, tied to the textSize key in UserDefaults.standard.
WHAT **Stores the preferred text size as a Double, defaulting to 17.**
WHERE In UserDefaults under the key textSize.
WHEN Written each time the stepper changes it.
WHY Writing 17.0 rather than 17 makes the type Double, which the font size below needs.

```
8  var body: some View {
```

WHO SwiftUI, which reads SettingsView's body whenever username, showTips or textSize changes.
WHAT **Describes the form.**
WHERE A computed property of the view.
WHEN Again when any of the three stored values changes.
WHY Every @AppStorage property triggers a refresh on change, just like @State.

```
9  Form {
```

WHO SwiftUI's Form container, giving the rows the grouped look of the Settings app.
WHAT **Creates a grouped, scrolling form styled like Settings.**
WHERE The root of the screen.
WHEN Each time SettingsView.body runs, which happens after any of the three stored values changes.
WHY Form gives settings the familiar iOS look with no extra styling.

```
10 TextField("Your name", text: $username)
```

WHO SwiftUI, and the user typing.
WHAT **Shows a text field bound to the stored username.**
WHERE First row of the form.
WHEN Writes username on every keystroke.
WHY The binding saves as the user types, so there is no Save button to forget.

```
11 Toggle("Show tips", isOn: $showTips)
```

WHO SwiftUI, and the user tapping.
WHAT **Shows a switch bound to the stored showTips flag.**
WHERE Second row of the form.
WHEN Writes showTips when flipped.
WHY A Toggle is the standard control for a stored Bool.

```
12 Stepper("Text size: \(Int(textSize))",
```

WHO SwiftUI, and the user tapping plus or minus.
WHAT **Shows a stepper whose label includes the current size as a whole number.**
WHERE Third row of the form.
WHEN The label updates each time textSize changes.
WHY Int(textSize) drops the .0 so the label reads 17, not 17.0.

```
13 value: $textSize, in: 12...28)
```

WHO SwiftUI's Stepper, finishing the initializer call that began on the line above.
WHAT **Binds the stepper to textSize and limits it to the range 12 through 28.**
WHERE Continues the Stepper call above.
WHEN Each tap changes textSize by 1.
WHY A range prevents unusable sizes from being saved in the first place.

```
14 Section("Preview") {
```

WHO SwiftUI's Section container, grouping the greeting and the tip under the Preview header.
WHAT **Starts a titled group that previews the settings.**
WHERE Below the controls in the form.
WHEN Each time SettingsView.body runs; the section itself never changes, only its contents do.
WHY Sections separate controls from their effect.

```
15 Text("Hello, \(username)")
```

WHO SwiftUI and string interpolation.
WHAT **Greets the user by the stored name.**
WHERE Inside the Preview section.
WHEN Again whenever username changes.
WHY Showing the stored value proves it round-trips through UserDefaults.

```
16 .font(.system(size: textSize))
```

WHO SwiftUI's font modifier, applied to the Hello greeting on the line above.
WHAT **Sets the greeting's font size from the stored Double.**
WHERE Applies to the greeting text.
WHEN Each time textSize changes, because the modifier reads the stored Double when body runs.
WHY Swift converts the Double to the CGFloat that font sizes use automatically.

```
17 if showTips {
```

WHO Your code, in the view builder.
WHAT **Shows the tip only while the stored flag is true.**
WHERE Inside the Preview section.
WHEN Each time body runs.
WHY Stored flags are often used to hide or show parts of the interface.

```
18 Text("Reload the page: nothing resets.")
```

WHO SwiftUI's Text view, drawing the one-line tip inside the Preview section.
WHAT **Shows a short tip.**
WHERE Inside the if block.
WHEN Only while showTips is true.
WHY It invites you to test persistence yourself.

```
25  #Preview { SettingsView() }
```

WHO	Xcode's #Preview macro, rendering SettingsView in the canvas, shown here as the phone beside the form.
WHAT	**Renders the form beside the code.**
WHERE	At the bottom of the file.
WHEN	Whenever the preview refreshes.
WHY	A preview is the quickest way to try every control.

> **Try it**
>
> 1. Add a Toggle for a new key "compactMode" with a default of false. How many lines did it take?
> 2. Change the range to 14...22 while the saved size is 28. What does the stepper show, and what does that teach you about validating stored values?

CONCEPT

What does not belong in UserDefaults

UserDefaults is so convenient that it gets misused. Its plist file is not encrypted by your app, it is copied into backups, and it can be read with ordinary tools from an unencrypted computer backup. The whole file is also loaded into memory. That rules out three kinds of data:

Secrets	Passwords, access tokens, API keys. Anyone who can read a backup can read them. Use the Keychain.
Large data	Images, long arrays, downloaded responses. They slow down launch and waste memory. Use files.
User content	Notes, messages, to-do items. UserDefaults has no queries, no relationships, and no partial saves. Use SwiftData or files.

> **A quick test**
>
> If you would happily show the value on a settings screen, and it is no more than a few kilobytes, UserDefaults is a good home. If either answer is no, choose another place.

CHECK YOURSELF

Check your understanding

1. A view declares `@AppStorage("launches") private var launches = 0`. The value 5 was saved under that key on an earlier run. What does the view see?
   A. 0, because the declaration says so
   B. 5, the saved value
   C. nil until the user changes it
   D. A crash, because 0 and 5 disagree
2. Which value is the riskiest to keep in @AppStorage?
   A. The selected tab index
   B. Whether the user has seen onboarding
   C. The user's session access token
   D. The preferred text size

3. Two different views each declare `@AppStorage("username")`. The user edits the name in one of them. What happens in the other?
   A. It shows the old name until the app restarts
   B. It shows an empty string until the app is relaunched
   C. The app crashes because one key is declared in two views
   D. It updates too, because both observe the same key

4. Why not keep a user's 2,000 recipes as an array in UserDefaults?
   A. The whole file loads into memory, with no way to query or update one recipe
   B. UserDefaults cannot store arrays of more than 100 items
   C. UserDefaults deletes values that have not been read for 24 hours
   D. Arrays of user content must be stored in the Keychain, never in a plist file

Answers: Settings with @AppStorage

1. **B. 5, the saved value** The value in the declaration is only a fallback for a key that has never been saved. Once a value exists, @AppStorage reads it. Thinking the declaration resets the value is the most common mistake with stored settings.
2. **C. The user's session access token** An access token lets anyone who holds it act as the user, and UserDefaults stores it as readable text that is also copied into backups. Tokens belong in the Keychain. The other three are harmless preferences.
3. **D. It updates too, because both observe the same key** @AppStorage observes the key in UserDefaults, so every view using that key refreshes when it changes. Declaring a key in several places is allowed. That said, defining key names once, for example as constants, prevents typos that would create two different settings.
4. **A. The whole file loads into memory, with no way to query or update one recipe** UserDefaults can store arrays, so that is not the problem. Its file is read in full and rewritten as a unit, with no queries or partial updates. A list of user content that grows belongs in SwiftData or a file you manage.

Codable and JSON

CONCEPT

Encoding and decoding

Memory holds Swift values; disks and networks hold *bytes*. Turning a value into bytes is **encoding** (also called serializing), and rebuilding the value from bytes is **decoding**. The most common format is **JSON** (JavaScript Object Notation): readable text made of objects in braces, arrays in brackets, strings, numbers, true, false, and null.

Swift's `Codable` protocol makes this almost free. When every stored property of your type is itself Codable (String, Int, Double, Bool, Date, arrays, optionals, and other Codable types), the compiler writes the encoding and decoding code for you.

Encodable	The type can turn itself into an external format such as JSON.
Decodable	The type can be created from an external format.
Codable	Shorthand for both: Encodable & Decodable.
Data	Foundation's type for a buffer of raw bytes. Encoders produce it; decoders consume it.

CODE · RUNS IN THE BROWSER

Encode a struct to JSON

`Book` adopts `Codable` and nothing more; the compiler generates the rest. `JSONEncoder` turns the value into `Data`, and the last line turns those bytes back into text so you can read them. Swift prints `"key" : value` with a space before each colon, and `.sortedKeys` puts the keys in alphabetical order.

```swift
import Foundation

struct Book: Codable {
    var title: String
    var author: String
    var pages: Int
    var isFavorite: Bool
}

let book = Book(title: "Swift Basics", author: "Ana Ruiz",
                pages: 240, isFavorite: true)

let encoder = JSONEncoder()
encoder.outputFormatting = [.prettyPrinted, .sortedKeys]

let data = try encoder.encode(book)
print("Bytes:", data.count)
print(String(decoding: data, as: UTF8.self))
```

OUTPUT

```
Bytes: 95
{
  "author" : "Ana Ruiz",
  "isFavorite" : true,
  "pages" : 240,
  "title" : "Swift Basics"
}
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, resolving the Foundation module so JSONEncoder and Data are known to this program.
- WHAT — **Makes Foundation available, which provides JSONEncoder and Data.**
- WHERE — At the top of the program.
- WHEN — At compile time, before the first line of this program runs; nothing is imported at run time.
- WHY — Codable is part of the Swift standard library, but the JSON encoder and decoder live in Foundation.

3 `struct Book: Codable {`

- WHO — Your code, with help from the compiler.
- WHAT — **Declares a Book type that can be encoded and decoded.**
- WHERE — A value type in this program.
- WHEN — The compiler generates the Codable code at compile time.
- WHY — Adopting Codable is enough when every stored property is Codable; you write no encoding code by hand.

4 `var title: String`

- WHO — Your Book struct, declaring its first stored property.
- WHAT — **Declares the title property; it becomes the JSON key "title".**
- WHERE — A stored property of Book.
- WHEN — Set when a Book is created.
- WHY — Synthesized Codable uses property names as JSON keys unless you say otherwise.

5 `var author: String`

- WHO — Your Book struct, declaring the second of its four stored properties.
- WHAT — **Declares the author property, encoded as a JSON string.**
- WHERE — A stored property of Book.
- WHEN — Set when a Book is created.
- WHY — Each Codable property type maps to a JSON type: String to string, Int to number, Bool to true or false.

6 `var pages: Int`

- WHO — Your Book struct, declaring the only Int property, which JSON will carry unquoted.
- WHAT — **Declares the page count, encoded as a JSON number.**
- WHERE — A stored property of Book.
- WHEN — Set when a Book is created.
- WHY — Numbers are written without quotes, so other programs read them as numbers.

7 `var isFavorite: Bool`

- WHO — Your Book struct, declaring the Bool property that ends its stored properties.
- WHAT — **Declares a flag, encoded as JSON true or false.**
- WHERE — A stored property of Book.
- WHEN — Set when a Book is created.
- WHY — JSON has real booleans, so no conversion to 0 or 1 is needed.

```
10 let book = Book(title: "Swift Basics", author: "Ana Ruiz",
```

WHO Swift's memberwise initializer for Book, generated by the compiler because you wrote no init.
WHAT **Creates a Book value in memory with all four properties.**
WHERE A constant in the program's top-level code.
WHEN When the program starts running.
WHY Structs get this initializer automatically, with labels matching the property names.

```
11 pages: 240, isFavorite: true)
```

WHO The same initializer call.
WHAT **Supplies the last two arguments.**
WHERE Continues the previous line.
WHEN When the program starts running.
WHY Lining up wrapped arguments keeps long calls easy to scan.

```
13 let encoder = JSONEncoder()
```

WHO Foundation's JSONEncoder class, created here with its default settings.
WHAT **Creates an encoder that converts Encodable values to JSON bytes.**
WHERE A local constant in top-level code.
WHEN Once, right after book exists and before the encode call two lines below.
WHY An encoder is a reusable object; configure it once and encode many values with it.

```
14 encoder.outputFormatting = [.prettyPrinted, .sortedKeys]
```

WHO The encoder's formatting options.
WHAT **Asks for line breaks and indentation, and for keys in alphabetical order.**
WHERE A property of this encoder only.
WHEN Before encoder.encode(book) runs; formatting options set afterward would not affect that output.
WHY The default output is one compact line in an unspecified key order; these options make it readable and predictable, which matters in tests and examples.

```
16 let data = try encoder.encode(book)
```

WHO JSONEncoder, running the code the compiler generated for Book.
WHAT **Encodes book into a Data value holding the JSON bytes.**
WHERE The bytes live in memory in data.
WHEN When this line runs.
WHY encode can throw, for example for a Double that is infinity, so it is marked with try.

```
17 print("Bytes:", data.count)
```

WHO Your top-level code, calling Swift's print with the count of data.
WHAT **Prints how many bytes the JSON takes: 95.**
WHERE Standard output: the console pane below this example, or Xcode's console in a real app.
WHEN Right after encode returns data, and before the JSON text itself is printed on the next line.
WHY Data is just bytes; counting them shows that the formatting spaces and line breaks cost space too.

```
18 print(String(decoding: data, as: UTF8.self))
```

WHO Swift's String(decoding:as:) initializer, then print writing the result.
WHAT **Interprets the bytes as UTF-8 text and prints the JSON.**
WHERE Standard output, as the second thing this program prints, after the Bytes line.
WHEN After the byte count has been printed, as the last line of the program.
WHY Printing the encoded text is the quickest way to check what your app would send or save.

Try it

1. Change outputFormatting to [.sortedKeys] only. How many bytes is the JSON now, and what disappeared?
2. Add a property var rating = 4.5 to Book. Where does "rating" appear in the output, and why there?

CODE · RUNS IN THE BROWSER

Decode JSON into Swift values

Decoding runs the other way: you tell `JSONDecoder` which type to build, here an array of books, and it reads the bytes. The first book has no `subtitle` key. Because the property is optional (`String?`), a missing key simply becomes `nil` instead of an error.

```swift
import Foundation

struct Book: Codable {
    var title: String
    var subtitle: String?
    var pages: Int
}

let json = """
[
  {"title": "Swift Basics", "pages": 240},
  {"title": "Saving Data", "subtitle": "Files to Cloud",
   "pages": 312}
]
"""

let data = Data(json.utf8)
let books = try JSONDecoder().decode([Book].self, from: data)
for book in books {
    print(book.title, "-", book.subtitle ?? "no subtitle")
}
print("Total pages:", books.reduce(0) { $0 + $1.pages })
```

OUTPUT

```
Swift Basics - no subtitle
Saving Data - Files to Cloud
Total pages: 552
```

Line by line

```
1  import Foundation
```

WHO The Swift compiler, resolving the Foundation module that supplies the JSONDecoder this program decodes with.
WHAT **Makes Foundation's JSONDecoder and Data available.**
WHERE At the top of the program.
WHEN At compile time, before the Book struct is compiled or the JSON literal exists; nothing is imported at run time.
WHY The decoder lives in Foundation, like the encoder.

```
3  struct Book: Codable {
```

WHO Your code, with compiler-generated Codable support.
WHAT **Declares the type the JSON will be decoded into.**
WHERE A value type in this program.
WHEN The decoding code is generated at compile time.
WHY The struct is the contract: the JSON must provide what the struct requires.

```
4      var title: String
```

WHO Your Book struct, declaring its first stored property, which the decoder must find in every element.
WHAT **Requires a "title" key holding a string.**
WHERE A stored property of Book.
WHEN Filled in during decoding.
WHY A non-optional property means the key must be present, or decoding throws.

```
5      var subtitle: String?
```

WHO Your Book struct, declaring the one property the JSON may leave out.
WHAT **Declares an optional subtitle.**
WHERE A stored property of Book.
WHEN Filled during decoding, or set to nil if the key is missing or null.
WHY Make a property optional when the data source may leave it out; that is more honest than inventing a fake default.

```
6      var pages: Int
```

WHO Your Book struct, declaring the Int property that the page counts fill.
WHAT **Requires a "pages" key holding a whole number.**
WHERE A stored property of Book.
WHEN During decode([Book].self, from:), once per element of the JSON array.
WHY If the JSON held "240" in quotes, decoding would fail with a type mismatch.

```
9  let json = """
```

WHO Swift's multi-line string literal.
WHAT **Starts a string that spans the following lines.**
WHERE A constant in top-level code.
WHEN Created when the program starts.
WHY Triple quotes let JSON contain double quotes without escaping each one.

```
10 [
```

WHO The JSON text itself.
WHAT **Opens a JSON array: the top level is a list of books.**
WHERE The first line of the string's content.
WHEN Read when decoding runs.
WHY The top-level JSON shape (array or object) must match the type you ask for, here [Book].

```
11  {"title": "Swift Basics", "pages": 240},
```

WHO The JSON text inside the multi-line string, which JSONDecoder reads as the first array element.
WHAT **Describes a book with no subtitle key at all.**
WHERE The first element of the array.
WHEN During decoding, as JSONDecoder builds the first Book in the array.
WHY This line tests the optional property: the key is simply absent.

```
12  {"title": "Saving Data", "subtitle": "Files to Cloud",
```

WHO The JSON text, opening the second object in the array.
WHAT **Starts a second book that does include a subtitle.**
WHERE The second element of the array.
WHEN During decoding, as JSONDecoder builds the second Book and finds its subtitle key.
WHY JSON allows line breaks between key-value pairs, just like Swift allows them between arguments.

```
13  "pages": 312}
```

WHO The JSON text, closing the second object with its last key.
WHAT **Finishes the second book with its page count.**
WHERE Continues the second element.
WHEN During decoding, when JSONDecoder reads the pages value for the second Book.
WHY Key order in JSON does not matter; keys are matched by name.

```
15  """
```

WHO Swift's multi-line string literal.
WHAT **Ends the string.**
WHERE The line after the last line of content.
WHEN At compile time the literal becomes an ordinary String.
WHY The closing quotes set the indentation that is stripped from every line.

```
17  let data = Data(json.utf8)
```

WHO Foundation's Data type, through its init that copies a string's UTF-8 view.
WHAT **Converts the text to UTF-8 bytes, the input a decoder expects.**
WHERE A constant in memory.
WHEN Right before the decode call on the next line, once the json string exists.
WHY In a real app these bytes come from a file or a network response rather than a string literal.

```
18  let books = try JSONDecoder().decode([Book].self, from: data)
```

WHO JSONDecoder, running the code generated for Book.
WHAT **Builds an array of Book values from the bytes.**
WHERE books is a constant array in memory.
WHEN When this line runs.
WHY [Book].self names the type to build; decode throws if anything does not fit, which is why try is required.

```
19  for book in books {
```

WHO Swift's for-in loop, iterating over the books array that decode returned.
WHAT **Visits each decoded book in order.**
WHERE Top-level code of this program, right after the decode call.
WHEN After decode([Book].self, from:) succeeds, once for each of the two books.
WHY Decoded values are ordinary Swift values; nothing about them is special anymore.

```
20 print(book.title, "-", book.subtitle ?? "no subtitle")
```

WHO Your code and the ?? operator.
WHAT **Prints each title and its subtitle, or a placeholder when it is nil.**
WHERE Standard output: the console pane below this example, one line per book.
WHEN On each pass of the for-in loop, twice in total for the two decoded books.
WHY Unwrapping with ?? is how you present an optional value that may be missing.

```
22 print("Total pages:", books.reduce(0) { $0 + $1.pages })
```

WHO Swift's reduce(_:_:) on the books array, then print writing the sum.
WHAT **Adds up the page counts: 240 + 312 = 552.**
WHERE Standard output, as the last line this program prints.
WHEN After the for-in loop has printed both titles.
WHY Once data is decoded, all of Swift's collection tools work on it.

Try it

1. Remove ? from subtitle and run it again. Which book causes the error, and which key does the message name?
2. Change 240 to "240" (with quotes). What kind of error do you get?

CODE · RUNS IN THE BROWSER

Nested types and CodingKeys

Real APIs rarely match Swift naming. This JSON uses `stargazers_count` (snake_case), but Swift style is `starCount` (camelCase). A nested `CodingKeys` enum maps each property to its JSON key. The `owner` object decodes into a nested `Owner` struct, and the extra `language` key is ignored.

```
import Foundation

struct Repo: Codable {
    struct Owner: Codable {
        var login: String
    }
    var name: String
    var starCount: Int
    var owner: Owner

    enum CodingKeys: String, CodingKey {
        case name, owner
        case starCount = "stargazers_count"
    }
}

let json = """
{"name": "swift", "stargazers_count": 69000,
 "owner": {"login": "swiftlang"}, "language": "C++"}
"""
let repo = try JSONDecoder().decode(Repo.self, from: Data(json.utf8))
print("\(repo.owner.login)/\(repo.name): \(repo.starCount) stars")
```

OUTPUT

```
swiftlang/swift: 69000 stars
```

Line by line

1 `import Foundation`

WHO The Swift compiler, resolving the Foundation module so JSONDecoder and Data are known to this program.
WHAT **Makes Foundation's JSONDecoder available.**
WHERE At the top of the program.
WHEN At compile time, before Repo and its CodingKeys are compiled; nothing is imported at run time.
WHY Every JSON example needs Foundation.

3 `struct Repo: Codable {`

WHO Your Repo struct, declared by you to mirror the parts of the API response you need.
WHAT **Declares a type describing a code repository returned by a web API.**
WHERE A value type in this program.
WHEN Codable support is generated at compile time.
WHY Model only the fields your app uses; the decoder ignores the rest.

4 `struct Owner: Codable {`

WHO Your Repo.Owner struct, nested inside Repo and declared by you for the owner object.
WHAT **Declares a type nested inside Repo for the "owner" JSON object.**
WHERE Inside Repo, so its full name is Repo.Owner.
WHEN Compiled together with Repo; the compiler generates its Codable code when it generates Repo's.
WHY Nesting keeps a helper type next to the only type that uses it; it must be Codable too.

5 `var login: String`

WHO Your Repo.Owner struct, declaring its single stored property.
WHAT **Holds the owner's login name from the nested object.**
WHERE A stored property of Repo.Owner.
WHEN Filled while the outer Repo is decoded.
WHY Nested JSON objects map naturally to nested Codable structs.

7 `var name: String`

WHO Your Repo struct, declaring the property whose name already matches its JSON key.
WHAT **Holds the repository name from the "name" key.**
WHERE A stored property of Repo.
WHEN During decode(Repo.self, from:), when JSONDecoder reads the name key.
WHY This name already matches its JSON key, so its mapping is trivial.

8 `var starCount: Int`

WHO Your Repo struct, declaring the property that CodingKeys maps to stargazers_count.
WHAT **Holds the number of stars, using Swift-style naming.**
WHERE A stored property of Repo.
WHEN Filled during decoding from a differently named key.
WHY Keep Swift names idiomatic and translate at the boundary, rather than letting API naming leak into your code.

```
9  var owner: Owner
```

WHO Your Repo struct, declaring the property whose type is the nested Owner struct.
WHAT **Holds the nested owner object.**
WHERE A stored property of Repo.
WHEN Decoded by running Owner's own decoding code.
WHY Codable composes: a Codable type made of Codable parts is automatically Codable.

```
11 enum CodingKeys: String, CodingKey {
```

WHO The compiler, which looks for an enum with exactly this name.
WHAT **Declares the list of keys used to encode and decode Repo.**
WHERE Nested inside Repo, where the compiler looks for an enum named CodingKeys.
WHEN Read at compile time while generating the Codable code.
WHY When CodingKeys exists, it replaces the default mapping, so it must list every property you want encoded or decoded.

```
12 case name, owner
```

WHO The CodingKeys enum you wrote, listing two cases without raw values.
WHAT **Maps name and owner to JSON keys with the same spelling.**
WHERE Cases of the enum.
WHEN Used during decoding and encoding.
WHY A case without a raw value uses its own name as the key.

```
13 case starCount = "stargazers_count"
```

WHO The CodingKeys enum you wrote, giving the starCount case an explicit raw value.
WHAT **Maps the starCount property to the JSON key stargazers_count.**
WHERE A case with a String raw value.
WHEN Used during decoding and encoding.
WHY The raw value is the exact JSON key; encoding back to JSON writes stargazers_count too.

```
17 let json = """
```

WHO Swift's multi-line string literal.
WHAT **Starts the sample API response.**
WHERE A constant in top-level code.
WHEN Created when the program starts.
WHY A literal stands in for bytes that would really arrive over the network.

```
18 {"name": "swift", "stargazers_count": 69000,
```

WHO The JSON text inside the multi-line string, standing in for the API response.
WHAT **Provides the name and the snake_case star count.**
WHERE The first line of the JSON object.
WHEN During decode(Repo.self, from:), when JSONDecoder looks up the name and stargazers_count keys.
WHY These are exactly the key names the CodingKeys enum promised.

```
19 "owner": {"login": "swiftlang"}, "language": "C++"}
```

WHO The JSON text, supplying the nested owner object and an extra key the Repo struct never asks for.
WHAT **Provides the nested owner object and an extra language key.**
WHERE The rest of the JSON object.
WHEN During decoding, when Repo.Owner is built from the nested object and the language key is skipped.
WHY The extra key shows that unknown keys are ignored rather than causing errors.

```
20 """
```

WHO Swift's multi-line string literal.
WHAT **Ends the string.**
WHERE After the last line of JSON.
WHEN At compile time, when the literal between the triple quotes becomes a String constant.
WHY Everything between the triple quotes becomes the string's content.

```
21 let repo = try JSONDecoder().decode(Repo.self, from: Data(json.utf8))
```

WHO JSONDecoder, using the CodingKeys you wrote.
WHAT **Decodes the bytes into a Repo with a nested Owner.**
WHERE A constant in memory.
WHEN When this line runs.
WHY One call decodes the entire nested structure.

```
22 print("\(repo.owner.login)/\(repo.name): \(repo.starCount) stars")
```

WHO Swift's string interpolation, reading three properties of repo, then print writing the result.
WHAT **Prints swiftlang/swift: 69000 stars.**
WHERE Standard output: the single line this program prints to the console pane below the code.
WHEN After decode(Repo.self, from:) has returned the repo value on the line above.
WHY Reading repo.owner.login shows the nested value arrived intact.

> **Try it**
>
> 1. Encode repo with a JSONEncoder and print the result. Which key appears: starCount or stargazers_count?
> 2. Delete the line case name, owner. What error does the compiler report, and why?

INTERACTIVE

The JSON decoding lab

The left panel is JSON arriving from a server; the right panel is the Swift struct it must fit. Edit the JSON, or load a broken sample, and watch the result update: either what `print(book)` shows, or the exact `DecodingError` Swift throws, with the path to the failing key. Then use the toggles to change the struct, such as making a property optional or adding CodingKeys, and see which mismatches each change fixes.

This step is an interactive simulation in the HTML version.

CODE · RUNS IN THE BROWSER

Reading a decoding error

Data from outside your app can be wrong in many ways, and `decode` reports each one as a different `DecodingError` case. Catching specific cases lets you log something useful, or show a better message. The four samples below are fine, missing a key, the wrong type, and not JSON at all. Module 08 (Networking) applies these same CodingKeys and DecodingError cases to real API responses.

```swift
import Foundation

struct Weather: Codable {
    var city: String
    var temperature: Double
}
let samples = [
    "{\"city\": \"Oslo\", \"temperature\": 4.5}",
    "{\"city\": \"Lima\"}",
    "{\"city\": \"Cairo\", \"temperature\": \"hot\"}",
    "{\"city\": \"Rome\", \"temp",
]
let decoder = JSONDecoder()
for text in samples {
    do {
        let w = try decoder.decode(Weather.self, from: Data(text.utf8))
        print("Decoded", w.city, w.temperature)
    } catch DecodingError.keyNotFound(let key, _) {
        print("Missing key:", key.stringValue)
    } catch DecodingError.typeMismatch(_, let context) {
        print("Wrong type at:", context.codingPath[0].stringValue)
    } catch {
        print("Not valid JSON:", text)
    }
}
```

OUTPUT

```
Decoded Oslo 4.5
Missing key: temperature
Wrong type at: temperature
Not valid JSON: {"city": "Rome", "temp
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, resolving the Foundation module so JSONDecoder, Data and the DecodingError cases are known here.
- **WHAT** **Makes JSONDecoder and DecodingError available.**
- **WHERE** At the top of the program.
- **WHEN** At compile time, before the samples array or the loop exists; nothing is imported at run time.
- **WHY** DecodingError is declared in the standard library; the JSON decoder that throws it is in Foundation.

3 `struct Weather: Codable {`

- **WHO** Your Weather struct, declared by you as the shape every sample must match.
- **WHAT** **Declares the shape every sample is expected to have.**
- **WHERE** A value type in this program.
- **WHEN** Compiled before the samples are decoded.
- **WHY** One strict type makes each kind of failure easy to see.

```
4  var city: String
```

WHO Your Weather struct, declaring the String property that every sample supplies.
WHAT **Requires a "city" key holding a string.**
WHERE A stored property of Weather.
WHEN During decode(Weather.self, from:), on each pass of the loop, when JSONDecoder reads the city key.
WHY Every sample provides a city, so failures come from the other key.

```
5  var temperature: Double
```

WHO Your Weather struct, declaring the Double property that the broken samples get wrong.
WHAT **Requires a "temperature" key holding a number.**
WHERE A stored property of Weather.
WHEN During decode(Weather.self, from:), when JSONDecoder reads the temperature key, or throws because it cannot.
WHY Non-optional means a missing value is an error, not nil.

```
7  let samples = [
```

WHO Your top-level code, declaring the samples array of four String literals.
WHAT **Starts an array of four JSON strings to decode.**
WHERE A constant in top-level code.
WHEN Created when the program starts.
WHY Testing several bad inputs side by side shows how the error cases differ.

```
8  "{\"city\": \"Oslo\", \"temperature\": 4.5}",
```

WHO Your code, with \" escaping each quote inside the Swift string.
WHAT **A valid sample that matches Weather exactly.**
WHERE First element of samples.
WHEN Decoded on the first pass of the loop.
WHY Including one valid input proves that the decoding code itself is correct.

```
9  "{\"city\": \"Lima\"}",
```

WHO Your top-level code, writing the Lima sample as a String literal with escaped quotes.
WHAT **A sample with no temperature key at all.**
WHERE The second element of the samples array, decoded on the second pass of the loop.
WHEN Decoded on the second pass.
WHY This produces DecodingError.keyNotFound.

```
10 "{\"city\": \"Cairo\", \"temperature\": \"hot\"}",
```

WHO Your top-level code, writing the Cairo sample whose temperature is the string hot.
WHAT **A sample whose temperature is a string instead of a number.**
WHERE The third element of the samples array, decoded on the third pass of the loop.
WHEN Decoded on the third pass.
WHY This produces DecodingError.typeMismatch.

```
11 "{\"city\": \"Rome\", \"temp",
```

WHO Your top-level code, writing the Rome sample as a deliberately truncated String literal.
WHAT **A sample cut off mid-way, so it is not valid JSON.**
WHERE The fourth and last element of the samples array, decoded on the final pass of the loop.
WHEN Decoded on the last pass.
WHY Truncated data, like a half-written file, produces DecodingError.dataCorrupted.

```
13 let decoder = JSONDecoder()
```

WHO Foundation's JSONDecoder class, created once here and reused by the decode call inside the loop.
WHAT **Creates one decoder to reuse for every sample.**
WHERE A constant in top-level code.
WHEN Once, before the loop.
WHY Reusing a configured decoder avoids repeating its setup.

```
14 for text in samples {
```

WHO Swift's for-in loop, visiting each String in the samples array.
WHAT **Tries each sample in turn.**
WHERE Top-level code of this program, wrapping the do-catch that follows.
WHEN Four times, once per sample: Oslo, Lima, Cairo and then Rome.
WHY Each iteration has its own do-catch, so one failure does not stop the rest.

```
15 do {
```

WHO Swift's do-catch error handling, opening the do block inside the loop body.
WHAT **Starts a block whose errors are handled by the catch clauses below.**
WHERE Inside the for-in loop over samples, so each sample gets a fresh do-catch.
WHEN On every pass of the loop, before the decode call for that sample.
WHY do-catch is how you turn a thrown error into a decision.

```
16 let w = try decoder.decode(Weather.self, from: Data(text.utf8))
```

WHO Foundation's JSONDecoder, inside decode(_:from:), running the code generated for Weather.
WHAT **Tries to build a Weather from the sample's bytes.**
WHERE Inside the do block.
WHEN Each iteration; it throws at the first problem it finds.
WHY try marks the exact call that can fail.

```
17 print("Decoded", w.city, w.temperature)
```

WHO Your top-level code, calling print with the two decoded properties of w.
WHAT **Prints the values when decoding succeeds.**
WHERE Standard output: the console pane below this example, the Decoded Oslo 4.5 line.
WHEN Only for valid samples.
WHY Code after a throwing call runs only if the call succeeded.

```
18 } catch DecodingError.keyNotFound(let key, _) {
```

WHO Swift pattern matching on the thrown error.
WHAT **Handles a missing key and binds which key it was.**
WHERE The first catch clause.
WHEN When decode throws keyNotFound.
WHY Matching the specific case gives you the key name to report, instead of a generic failure.

```
19 print("Missing key:", key.stringValue)
```

WHO Your top-level code, calling print with the stringValue of the bound key.
WHAT **Prints the name of the missing key.**
WHERE Standard output, inside the catch clause that handles DecodingError.keyNotFound.
WHEN Only on the second pass of the loop, when the Lima sample lacks a temperature key.
WHY stringValue turns the CodingKey into the plain key name.

```
20 } catch DecodingError.typeMismatch(_, let context) {
```

WHO Swift's catch pattern matching against the DecodingError.typeMismatch case.
WHAT **Handles a value of the wrong type and binds the error context.**
WHERE The second catch clause.
WHEN When decode throws typeMismatch.
WHY The context records where in the JSON the problem happened.

```
21 print("Wrong type at:", context.codingPath[0].stringValue)
```

WHO Your top-level code, calling print with the first key on context.codingPath.
WHAT **Prints the first key on the path to the bad value: temperature.**
WHERE Standard output, inside the catch clause that handles DecodingError.typeMismatch.
WHEN For the Cairo sample.
WHY codingPath lists the keys from the top of the JSON down to the failing value, which matters for nested data.

```
22 } catch {
```

WHO Swift's catch-all clause, which binds any remaining error to the implicit constant error.
WHAT **Catches every other error, including dataCorrupted.**
WHERE The last catch clause.
WHEN When no earlier clause matched.
WHY A final catch-all guarantees no error escapes unhandled.

```
23 print("Not valid JSON:", text)
```

WHO Your top-level code, calling print with the original text that failed to decode.
WHAT **Reports the broken sample.**
WHERE Standard output, inside the final catch clause, as the last line this program prints.
WHEN For the Rome sample.
WHY Showing the input that failed makes bad data much easier to track down.

> **Try it**
>
> 1. Change temperature to Double? in the struct. Which outputs change?
> 2. Add a fifth sample where city is the number 7. Which catch clause handles it?

CODE · READ-ALONG

Dates in JSON

JSON has no date type, so servers send dates as text, usually in the ISO 8601 format `2026-09-14T17:00:00Z`. A default `JSONDecoder` expects a number of seconds instead, and would throw a type mismatch. Setting a *date decoding strategy* tells it how to read dates. This one is shown rather than run.

```swift
import Foundation

struct Event: Codable {
    var name: String
    var startsAt: Date
}

let json = """
{"name": "Launch party", "startsAt": "2026-09-14T17:00:00Z"}
"""

let decoder = JSONDecoder()
decoder.dateDecodingStrategy = .iso8601
let event = try decoder.decode(Event.self, from: Data(json.utf8))
print(event.startsAt.formatted(date: .long, time: .shortened))

let encoder = JSONEncoder()
encoder.dateEncodingStrategy = .iso8601
let backToJSON = try encoder.encode(event)
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, resolving the Foundation module so Date, JSONDecoder and JSONEncoder are known here.
- **WHAT** **Makes Date, JSONDecoder, and JSONEncoder available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before the first line of this file runs; nothing is imported at run time.
- **WHY** Date is a Foundation type.

3 `struct Event: Codable {`

- **WHO** Your Event struct, declared by you with one String and one Date property.
- **WHAT** **Declares an event with a name and a start time.**
- **WHERE** A value type declared at the top level of this file, after the import.
- **WHEN** Compiled before the decode call below can name Event.self; the compiler generates its Codable code then.
- **WHY** Date is Codable, so the struct still gets synthesized Codable.

4 `var name: String`

- **WHO** Your Event struct, declaring the ordinary String property.
- **WHAT** **Holds the event name.**
- **WHERE** A stored property of Event.
- **WHEN** During decode(Event.self, from:), when JSONDecoder reads the name key.
- **WHY** An ordinary string, decoded the usual way.

5 `var startsAt: Date`

- **WHO** Your Event struct, declaring the property that the date strategy below exists for.
- **WHAT** **Holds the start time as a Date, a single moment in time.**
- **WHERE** A stored property of Event.
- **WHEN** Filled using the decoder's date strategy.
- **WHY** Store moments as Date, never as display strings; formatting belongs in the interface.

```
8  let json = """
```

WHO Swift's multi-line string literal.
WHAT **Starts a sample server response.**
WHERE A constant named json at the top level of this file, holding the sample response.
WHEN When this file runs, before the decoder is created and configured below.
WHY Triple quotes avoid escaping the JSON quotes.

```
9  {"name": "Launch party", "startsAt": "2026-09-14T17:00:00Z"}
```

WHO The JSON text inside the multi-line string, carrying the ISO 8601 date as a string.
WHAT **Gives the start time as an ISO 8601 string in UTC; the final Z means zero offset.**
WHERE The content of the string.
WHEN During decode(Event.self, from:), when the .iso8601 strategy parses the startsAt value.
WHY ISO 8601 is unambiguous across time zones, which is why most APIs use it.

```
10 """
```

WHO Swift's multi-line string literal.
WHAT **Ends the string.**
WHERE After the JSON line.
WHEN At compile time, when the literal between the triple quotes becomes the json constant.
WHY The literal becomes an ordinary String.

```
12 let decoder = JSONDecoder()
```

WHO Foundation's JSONDecoder class, created here so its date strategy can be changed on the next line.
WHAT **Creates a decoder.**
WHERE A constant named decoder at the top level of this file.
WHEN Before the dateDecodingStrategy is set and before the decode call two lines below.
WHY Strategies are settings on the decoder object, so you need your own instance to change them.

```
13 decoder.dateDecodingStrategy = .iso8601
```

WHO The decoder's date strategy.
WHAT **Tells the decoder to parse dates from ISO 8601 strings.**
WHERE A property of this decoder.
WHEN Before decoder.decode(Event.self, from:) runs; the strategy must be in place when the startsAt value is read.
WHY The default strategy, .deferredToDate, expects seconds since January 1, 2001, which almost no server sends.

```
14 let event = try decoder.decode(Event.self, from: Data(json.utf8))
```

WHO Foundation's JSONDecoder, inside decode(_:from:), using the .iso8601 strategy you set.
WHAT **Decodes the event, turning the string into a Date.**
WHERE A constant in memory.
WHEN When this line runs.
WHY A string the strategy cannot parse throws dataCorrupted, which you would catch like any other decoding error.

```
15 print(event.startsAt.formatted(date: .long, time: .shortened))
```

WHO Foundation's formatted(date:time:) on Date, then print writing the result.
WHAT **Prints the moment in the user's language and time zone, for example September 14, 2026 at 1:00 PM in New York.**
WHERE Xcode's console when this file runs in an app; this static example is shown, not executed.
WHEN After decode has produced event, so event.startsAt holds a real Date.
WHY The stored Date is time-zone neutral; converting to local time happens only for display.

```
17 let encoder = JSONEncoder()
```

WHO Foundation's JSONEncoder class, created here for the round trip back to JSON.
WHAT **Creates an encoder for sending or saving the event.**
WHERE A constant named encoder at the top level of this file, separate from decoder.
WHEN After the event has been decoded and printed, before dateEncodingStrategy is set.
WHY Encoding needs a matching strategy, or dates will not round-trip.

```
18 encoder.dateEncodingStrategy = .iso8601
```

WHO The encoder's date strategy.
WHAT **Writes dates as ISO 8601 strings.**
WHERE A property of this encoder.
WHEN Before encoder.encode(event) runs, so the startsAt Date is written as text instead of seconds.
WHY Using the same format in both directions keeps your files and API traffic consistent.

```
19 let backToJSON = try encoder.encode(event)
```

WHO Foundation's JSONEncoder, inside encode(_:), using the .iso8601 strategy you set.
WHAT **Encodes the event back into JSON bytes with the date as 2026-09-14T17:00:00Z.**
WHERE A constant holding Data.
WHEN When this line runs.
WHY Round-tripping (decode, then encode) is a simple test that your strategies match.

> **Try it**
>
> 1. Which strategy would you use if a server sent dates as seconds since 1970? Look for .secondsSince1970.

CHECK YOURSELF

Check your understanding

1. Given `struct Pet: Codable { var name: String; var age: Int }`, what happens when you decode `{"name": "Mochi"}`?
   A. You get Pet(name: "Mochi", age: 0)
   B. decode throws keyNotFound for age
   C. You get Pet(name: "Mochi", age: nil)
   D. The code does not compile, because age has no default
2. An API sends `"first_name"`, but your property is `firstName`. Which fix keeps idiomatic Swift naming and still decodes the value?
   A. Add CodingKeys with case firstName = "first_name"
   B. Make firstName optional so the decoder can skip it
   C. Rename the property to first_name so the names match
   D. Switch from JSONDecoder to JSONEncoder
3. Which stored property stops the compiler from synthesizing Codable for a struct?
   A. var scores: [String: Int]
   B. var birthday: Date? = nil
   C. var onTap: () -> Void
   D. var address: Address (a Codable struct)

4. A server sends `"price": "4.99"` (a string) and your property is `var price: Double`. What happens?
    A. Swift converts the string to 4.99 automatically
    B. price becomes 0.0
    C. decode throws keyNotFound for price
    D. decode throws typeMismatch for price

Answers: Codable and JSON

1. **B. decode throws keyNotFound for age** A non-optional property must be present in the JSON, so the decoder throws keyNotFound. Swift never invents a 0. To accept a missing age, declare it as Int? and it decodes as nil. Answer C cannot happen either, because a plain Int can never hold nil.
2. **A. Add CodingKeys with case firstName = "first_name"** CodingKeys maps a Swift name to a JSON key. (Setting decoder.keyDecodingStrategy = .convertFromSnakeCase is another valid fix.) Making the property optional is the trap: decoding would succeed, but firstName would always be nil, hiding the bug. Renaming works but breaks Swift naming conventions.
3. **C. var onTap: () -> Void** Synthesis works only when every stored property is Codable. Dictionaries with Codable keys and values, optionals, Date, and other Codable structs all qualify. A closure is code, not data, so it cannot be encoded, and the compiler reports that Codable cannot be synthesized.
4. **D. decode throws typeMismatch for price** JSONDecoder is strict about types: the key exists, so it is not keyNotFound, but its value has the wrong type. Fix it by asking the API for a number, or by writing a custom init(from:) that decodes a String and converts it.

Reading and Writing Files

CONCEPT

Files, URLs, and FileManager

A file is a named sequence of bytes on disk. In Swift you name files with `URL` values (a URL can point to a local file, not only a web page), move bytes with `Data`, and manage folders with `FileManager`, the Foundation object that creates, lists, moves, and deletes files.

URL.documentsDirectory Your container's Documents folder. Siblings: applicationSupportDirectory, cachesDirectory, temporaryDirectory.

appending(path:) Builds a child URL, such as Documents/notes.json, without gluing strings together.

write(to:options:) A Data method that writes bytes to a URL. The .atomic option makes the write all-or-nothing.

Data(contentsOf:) Reads a whole file into memory. It throws if the file is missing or unreadable.

> **Whole-file reads are fine**
>
> Reading an entire file into memory is the right approach for files up to a few megabytes, which covers most app data. Streaming APIs exist for huge files, but you will rarely need them.

CODE · READ-ALONG

Write a file, then read it back

The smallest complete round trip: build a URL inside Documents, write some bytes, read them back, and check that the file exists. Every file operation can fail (a full disk, a missing file, a locked device), so the calls are marked with `try` inside a `do` block.

```swift
import Foundation

let fileURL = URL.documentsDirectory
    .appending(path: "greeting.txt")

do {
    let text = "Hello from the sandbox"
    try Data(text.utf8).write(to: fileURL, options: .atomic)

    let bytes = try Data(contentsOf: fileURL)
    let readBack = String(decoding: bytes, as: UTF8.self)
    print(readBack)

    let exists = FileManager.default.fileExists(atPath: fileURL.path())
    print("Exists:", exists)
} catch {
    print("File error:", error.localizedDescription)
}
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, resolving the Foundation module while it builds this file.
- WHAT — **Makes URL, Data, and FileManager available.**
- WHERE — At the top of the file.
- WHEN — At compile time, before any line of this file runs; URL, Data, and FileManager become usable from here down.
- WHY — All of the file APIs live in Foundation.

3 `let fileURL = URL.documentsDirectory`

- WHO — Foundation, which knows where your container is.
- WHAT — **Gets the URL of your app's Documents folder.**
- WHERE — A constant; the folder itself is in your sandbox on the device.
- WHEN — When this line runs.
- WHY — The container path changes between installs, so always ask for it instead of storing it.

4 `.appending(path: "greeting.txt")`

- WHO — Foundation's URL type, through its appending(path:) method on the Documents URL.
- WHAT — **Extends the URL to a file named greeting.txt inside Documents.**
- WHERE — Continues the expression above.
- WHEN — When this line runs; no file is created yet.
- WHY — A URL is only an address; nothing touches the disk until you read or write.

6 `do {`

- WHO — Swift's do statement, which routes any error thrown inside it to the catch block below.
- WHAT — **Starts a block in which any thrown error jumps to the catch below.**
- WHERE — Top-level code of this file, wrapping the write, the read, and the existence check.
- WHEN — When execution reaches it.
- WHY — Grouping related file calls lets one catch handle every failure.

7 `let text = "Hello from the sandbox"`

- WHO — You, choosing the sample text this example will write to greeting.txt.
- WHAT — **Holds the text to save.**
- WHERE — A local constant in memory.
- WHEN — Inside the do block.
- WHY — Text is converted to bytes before it can be written.

8 `try Data(text.utf8).write(to: fileURL, options: .atomic)`

- WHO — Foundation and the iOS file system.
- WHAT — **Encodes the text as UTF-8 bytes and writes them to greeting.txt, creating or replacing it.**
- WHERE — On the device's storage, inside Documents.
- WHEN — When this line runs; it throws if the write fails.
- WHY — .atomic writes to a temporary file and then renames it into place, so readers never see a half-written file.

10 `let bytes = try Data(contentsOf: fileURL)`

- WHO — Foundation and the file system.
- WHAT — **Reads every byte of greeting.txt into memory.**
- WHERE — A local Data value in memory.
- WHEN — Immediately after write(to:options:) succeeded, so greeting.txt now exists in Documents.
- WHY — This throws if the file is missing, which is why it needs try.

```
11 let readBack = String(decoding: bytes, as: UTF8.self)
```

WHO Swift's String type, decoding the Data value with its String(decoding:as:) initializer.
WHAT **Turns the bytes back into text.**
WHERE A local constant inside the do block, alive only until the block ends.
WHEN After Data(contentsOf:) has read every byte of greeting.txt into bytes.
WHY Use the same encoding, UTF-8, in both directions.

```
12 print(readBack)
```

WHO Swift's print function, writing the text you just read back from the file.
WHAT **Prints Hello from the sandbox.**
WHERE The console: Xcode's console pane when this runs in an app, or the output area beside this example.
WHEN After decoding the text.
WHY Seeing the same text proves the round trip worked.

```
14 let exists = FileManager.default.fileExists(atPath: fileURL.path())
```

WHO FileManager, the shared file-system helper.
WHAT **Asks whether a file exists at that path; here it is true.**
WHERE A local Bool constant inside the do block, set from FileManager's answer.
WHEN After the write and the read have both succeeded, as a final check on greeting.txt.
WHY fileExists takes a path string, so path() converts the URL to one.

```
15 print("Exists:", exists)
```

WHO Swift's print function, showing the Bool that FileManager returned.
WHAT **Prints Exists: true.**
WHERE The console: Xcode's console pane in an app, or the output area beside this example.
WHEN Right after fileExists(atPath:) answers, as the last line of the do block.
WHY A quick existence check is useful before deciding whether to load or start fresh.

```
16 } catch {
```

WHO Swift's catch clause, receiving whichever error the write or the read threw.
WHAT **Receives any error thrown in the do block as the constant error.**
WHERE After the do block.
WHEN Only if a try failed.
WHY Without a catch, a failed write would go unnoticed or crash.

```
17 print("File error:", error.localizedDescription)
```

WHO Swift's print function, writing the error's localizedDescription for you to read.
WHAT **Prints a readable description of the failure.**
WHERE The console again, but only on the failure path: the successful prints above never run when this one does.
WHEN Only after an error.
WHY localizedDescription is suitable for logs; a real app would show the user a friendlier message.

> **Try it**
>
> 1. Which line would you change to store greeting.txt where it is not backed up and may be purged?

CODE · READ-ALONG

A store that saves Codable notes

Combine Codable and files and you have a complete persistence layer. `Note` is a Codable struct with an `id` and some `text`, defined elsewhere in the app. `save` encodes the whole array and writes it in one

atomic step; `load` treats a missing file as an empty list. Try the same steps in the simulated container beside the code.

```
import Foundation

struct NotesStore {
    let fileURL = URL.documentsDirectory
        .appending(path: "notes.json")

    func save(_ notes: [Note]) throws {
        let data = try JSONEncoder().encode(notes)
        try data.write(to: fileURL,
                       options: [.atomic, .completeFileProtection])
    }

    func load() throws -> [Note] {
        let manager = FileManager.default
        guard manager.fileExists(atPath: fileURL.path()) else {
            return []
        }
        let data = try Data(contentsOf: fileURL)
        return try JSONDecoder().decode([Note].self, from: data)
    }
}
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, resolving the Foundation module while it builds this store's file.
- **WHAT** **Makes JSONEncoder, JSONDecoder, Data, and FileManager available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; the JSON coders and FileManager become usable from here down.
- **WHY** Everything this store needs is in Foundation.

3 `struct NotesStore {`

- **WHO** You, declaring the NotesStore type that owns every file operation for notes.
- **WHAT** **Declares a small type responsible for saving and loading notes.**
- **WHERE** A value type in your app module.
- **WHEN** Created wherever the app needs to persist notes, often once.
- **WHY** Keeping file code in one type means views never deal with paths or JSON.

4 `let fileURL = URL.documentsDirectory`

- **WHO** Foundation's URL type, through its static documentsDirectory property.
- **WHAT** **Starts building the location of the notes file.**
- **WHERE** A stored constant of NotesStore.
- **WHEN** Computed when a NotesStore is created.
- **WHY** Documents is right because notes are content the user created and expects in backups.

```
5   .appending(path: "notes.json")
```

WHO Foundation's URL type, through appending(path:) on the Documents URL from the line above.
WHAT **Points at Documents/notes.json.**
WHERE Continues the property above.
WHEN When a NotesStore is created.
WHY One fixed file name means save and load always agree on where the data is.

```
7   func save(_ notes: [Note]) throws {
```

WHO Your NotesStore struct, declaring its save(_:) method.
WHAT **Declares a method that writes the whole notes array to disk.**
WHERE A method of NotesStore.
WHEN Whenever the caller decides to save, such as after an edit.
WHY throws passes failures to the caller, which knows how to tell the user.

```
8   let data = try JSONEncoder().encode(notes)
```

WHO Foundation's JSONEncoder, inside encode(_:), running the Codable code generated for Note.
WHAT **Encodes the array of notes as one JSON array in memory.**
WHERE A local Data value.
WHEN Each time save runs.
WHY Arrays of Codable values are Codable themselves, so one call encodes them all.

```
9   try data.write(to: fileURL,
```

WHO Foundation and the file system.
WHAT **Writes the JSON bytes to notes.json.**
WHERE In the Documents folder on the device.
WHEN Each time save runs.
WHY Writing the whole file each time is simple and fine for small to medium lists.

```
10  options: [.atomic, .completeFileProtection])
```

WHO Foundation's Data.WritingOptions, read by write(to:options:) on the line above.
WHAT **Writes all-or-nothing, and keeps the file encrypted and unreadable whenever the device is locked.**
WHERE Arguments of the write call above.
WHEN Applied during the write.
WHY .atomic prevents a crash from leaving half a file; .completeFileProtection protects personal notes on a lost phone.

```
13  func load() throws -> [Note] {
```

WHO Your NotesStore struct, declaring its load() method.
WHAT **Declares a method that reads notes back from disk.**
WHERE A method of NotesStore.
WHEN Typically once, when the app starts.
WHY Returning an array keeps the interface simple for callers.

```
14  let manager = FileManager.default
```

WHO Foundation's FileManager, through its shared default instance.
WHAT **Gets the shared file manager.**
WHERE A local constant inside load(), gone when the method returns.
WHEN Each time load runs.
WHY A short local name keeps the next line under control.

```
15 guard manager.fileExists(atPath: fileURL.path()) else {
```

WHO — FileManager and Swift's guard.
WHAT — **Checks whether notes.json exists before trying to read it.**
WHERE — At the start of load.
WHEN — Each time load runs.
WHY — On the very first launch there is no file yet, and that is normal, not an error.

```
16 return []
```

WHO — Your NotesStore struct, inside the else branch of the guard in load().
WHAT — **Returns an empty list when nothing has been saved yet.**
WHERE — Inside the guard's else branch.
WHEN — Only on first launch, or after the file was deleted.
WHY — An empty list lets the app start cleanly instead of showing an error.

```
18 let data = try Data(contentsOf: fileURL)
```

WHO — Foundation and the file system.
WHAT — **Reads the entire JSON file into memory.**
WHERE — A local Data value.
WHEN — After confirming the file exists.
WHY — This can still throw, for example if the device is locked and the file is protected.

```
19 return try JSONDecoder().decode([Note].self, from: data)
```

WHO — Foundation's JSONDecoder, inside decode(_:from:), running the Codable code generated for Note.
WHAT — **Decodes the bytes into an array of notes and returns it.**
WHERE — The return value of load.
WHEN — After Data(contentsOf:) has read notes.json in full, as the last line of load().
WHY — If the file is damaged, decode throws and the caller decides what to do.

> **Try it**
>
> 1. In the simulation, uncheck the atomic option, use Save, then crash, and Load. Which line of this code throws?

INTERACTIVE

Inside the app container

This is a simulated app container. Edit the notes, then press **Save** to encode them and write `Documents/notes.json`; select the file to see its bytes. **Load** decodes it back. Turn off the atomic option and use **Save, then crash** to see why a half-written file breaks decoding. Finally, **Delete app** shows what the user loses when the container is removed.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Handle file errors on purpose

Two failures are normal and need different responses. On first launch the file does not exist yet, which is not worth an alert. A file that exists but will not decode means damage, or an app update that changed the format, and the user should hear about it. Separate `catch` clauses let the app respond to each.

```swift
import Foundation

enum LoadResult {
    case loaded([Note])
    case firstLaunch
    case damaged(String)
}

func loadNotes(from url: URL) -> LoadResult {
    do {
        let data = try Data(contentsOf: url)
        let notes = try JSONDecoder().decode([Note].self, from: data)
        return .loaded(notes)
    } catch CocoaError.fileReadNoSuchFile {
        return .firstLaunch
    } catch let error as DecodingError {
        return .damaged("The notes file is damaged: \(error)")
    } catch {
        return .damaged(error.localizedDescription)
    }
}
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, resolving the Foundation module while it builds the file that holds loadNotes(from:).
- **WHAT** **Makes Data, JSONDecoder, CocoaError, and DecodingError available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; CocoaError and JSONDecoder become usable from here down.
- **WHY** File errors are reported as CocoaError values from Foundation.

3 `enum LoadResult {`

- **WHO** You, declaring the LoadResult enum that names each outcome of loading notes.
- **WHAT** **Declares the three outcomes of loading that the interface cares about.**
- **WHERE** A type in your app module.
- **WHEN** Compiled ahead of use.
- **WHY** An enum forces callers to handle every outcome, unlike a bare optional.

4 `case loaded([Note])`

- **WHO** Your LoadResult enum, declaring its success case.
- **WHAT** **The success case, carrying the decoded notes.**
- **WHERE** A case with an associated value.
- **WHEN** Produced when reading and decoding both succeed.
- **WHY** Associated values attach the data to the outcome.

5 `case firstLaunch`

- **WHO** Your LoadResult enum, declaring the case for a missing file.
- **WHAT** **Means no file exists yet.**
- **WHERE** A case of the enum.
- **WHEN** Produced on first launch.
- **WHY** Naming this case makes clear that a missing file is expected, not broken.

```
6  case damaged(String)
```

WHO	Your LoadResult enum, declaring the case for a file that exists but cannot be used.
WHAT	**Means the file exists but could not be used, with a message for the user.**
WHERE	A case with an associated String.
WHEN	Produced on read or decode failures.
WHY	The interface can show this message and offer to start fresh.

```
9  func loadNotes(from url: URL) -> LoadResult {
```

WHO	You, declaring loadNotes(from:), which converts every error into a LoadResult value.
WHAT	**Declares a function that never throws; it turns every error into a LoadResult.**
WHERE	A top-level function in this file, outside any type, so any screen can call it.
WHEN	When the app starts and wants its notes.
WHY	Converting errors into values at this boundary keeps view code simple.

```
10 do {
```

WHO	Swift's do statement, routing errors from the read and the decode to the catch clauses below.
WHAT	**Starts the block whose errors the catch clauses sort.**
WHERE	Inside loadNotes(from:), wrapping the read on the next line and the decode after it.
WHEN	Every time loadNotes(from:) runs, typically once at app launch.
WHY	One do block covers both the read and the decode.

```
11 let data = try Data(contentsOf: url)
```

WHO	Foundation and the file system.
WHAT	**Reads the whole file, or throws if it is missing or unreadable.**
WHERE	A local constant inside the do block of loadNotes(from:), holding the file's bytes.
WHEN	First, inside the do block.
WHY	Trying the read directly avoids a race between checking for the file and reading it.

```
12 let notes = try JSONDecoder().decode([Note].self, from: data)
```

WHO	Foundation's JSONDecoder, inside decode(_:from:), rebuilding each Note from the bytes.
WHAT	**Decodes the notes, or throws a DecodingError.**
WHERE	A local constant inside the do block, holding the decoded [Note] array.
WHEN	After a successful read.
WHY	Decode errors and file errors are different types, so they can be caught separately.

```
13 return .loaded(notes)
```

WHO	Your loadNotes(from:) function, returning its success case.
WHAT	**Returns the success case with the notes.**
WHERE	The last line of the do block, reached only when both throwing calls succeeded.
WHEN	Only when neither Data(contentsOf:) nor decode(_:from:) threw.
WHY	The caller receives the data with no error handling of its own.

```
14 } catch CocoaError.fileReadNoSuchFile {
```

WHO	Swift pattern matching on a Foundation error code.
WHAT	**Matches only the error that means the file does not exist.**
WHERE	The first catch clause.
WHEN	Only when Data(contentsOf:) threw fileReadNoSuchFile, which is first launch or after the file was deleted.
WHY	Matching a specific code keeps other read failures, such as permission problems, out of this branch.

```
15  return .firstLaunch
```

WHO Your loadNotes(from:) function, inside the catch for CocoaError.fileReadNoSuchFile.
WHAT **Reports that there is nothing to load yet.**
WHERE Inside the first catch.
WHEN When the file is missing.
WHY The interface can show an empty state, such as a "Write your first note" prompt.

```
16  } catch let error as DecodingError {
```

WHO Swift pattern matching with a type cast.
WHAT **Matches any decoding error and binds it as a DecodingError.**
WHERE The second catch clause.
WHEN When the file exists but its contents do not decode.
WHY The typed constant gives access to details such as the coding path for logging.

```
17  return .damaged("The notes file is damaged: \(error)")
```

WHO Your loadNotes(from:) function, inside the catch that bound error as a DecodingError.
WHAT **Reports damage along with the technical details.**
WHERE Inside the second catch.
WHEN After a decode failure.
WHY Keep the details for logs and support; show the user a calmer summary.

```
18  } catch {
```

WHO Swift's final catch clause, receiving any error the two earlier clauses did not match.
WHAT **Catches everything else.**
WHERE The final catch clause.
WHEN For any other failure, such as a locked device.
WHY An exhaustive catch is required because loadNotes itself does not throw.

```
19  return .damaged(error.localizedDescription)
```

WHO Your loadNotes(from:) function, inside the final catch clause.
WHAT **Reports the system's readable description of the failure.**
WHERE Inside the last catch.
WHEN Only for errors that are neither fileReadNoSuchFile nor a DecodingError, such as a file protected while the device is locked.
WHY localizedDescription is translated into the user's language by the system.

> **Try it**
>
> 1. What should the interface do differently for .firstLaunch and .damaged? Write one sentence for each.

CONCEPT

Files or a database?

Saving an array to one JSON file is simple, and it works well while the data is small and you always load and save all of it at once. A database earns its extra setup when the data grows or needs queries.

Choose files for	Documents the user opens whole, such as a drawing; media such as photos and audio; lists of a few hundred items; anything you export or share.
Choose a database for	Thousands of records; searching and sorting; changing one record without rewriting everything; relationships, such as folders that contain notes.

Or combine them	Keep large media in files and only the file name in a database record. SwiftData can do this for you with @Attribute(.externalStorage).

> **Hide the choice behind a type**
>
> Views should call `store.load()` and `store.save(notes)`, never FileManager directly. Then switching from a JSON file to a database later changes one type, not every screen.

CHECK YOURSELF

Check your understanding

1. Without `.atomic`, the app crashes halfway through writing notes.json. What is the likely result on the next launch?
   A. The previous notes.json is intact, so the old notes load
   B. notes.json holds half of the JSON, so decoding throws dataCorrupted
   C. iOS restores the file from the last iCloud backup
   D. The write finishes when the app relaunches
2. `load()` runs on first launch, before anything has ever been saved. What should a well-designed store do?
   A. Call fatalError, because the file is missing
   B. Show an alert that the file is damaged
   C. Return an empty array
   D. Create a sample note and save it
3. Which expression builds the URL for `Documents/photos/cat.jpg`?
   A. URL.documentsDirectory.appending(path: "photos/cat.jpg")
   B. URL.documentsDirectory + "/photos/cat.jpg"
   C. URL(string: "Documents/photos/cat.jpg")
   D. FileManager.default.documents("photos/cat.jpg")
4. An app records 50,000 sensor readings and must show only last week's readings, sorted by time. What is the best fit?
   A. One JSON file rewritten after every reading
   B. One UserDefaults array that grows with every reading
   C. One file per reading in the tmp folder
   D. A SwiftData database with one record per reading

Answers: Reading and Writing Files

1. **B. notes.json holds half of the JSON, so decoding throws dataCorrupted** A plain write replaces the file in place, so a crash can leave a truncated file that is no longer valid JSON. With .atomic, the bytes go to a temporary file first and are renamed over the old file only when complete, so you get either the old file or the new one, never half of one.
2. **C. Return an empty array** A missing file on first launch is the expected state, not an error. Returning an empty array lets the interface show a friendly empty state. Treating it as damage would alarm every new user, and crashing is never acceptable for an expected condition.
3. **A. URL.documentsDirectory.appending(path: "photos/cat.jpg")** appending(path:) builds a child URL from the real container location. URL has no + operator for strings, URL(string:) would describe a relative path that is not inside your container, and the FileManager method in the last choice does not exist.
4. **D. A SwiftData database with one record per reading** Many records, a filter, and a sort are exactly what a database does well: it can fetch just last week's rows without loading everything. Rewriting a growing JSON file after each reading gets slower every time, UserDefaults is for small settings, and tmp may be cleared.

SwiftData Fundamentals

CONCEPT

What SwiftData does

SwiftData is Apple's framework (iOS 17 and later) for saving model objects in an on-device database. You write ordinary Swift classes; SwiftData stores them in a SQLite file (SQLite is a small, fast database engine built into iOS) in Application Support, loads them back, and keeps your SwiftUI views up to date.

It is the modern replacement for *Core Data*, the older framework it is built on, and both can use the same store, which helps existing apps move over gradually.

@Model	A macro that turns a class into a stored model.
ModelContainer	The database: its schema (the list of model types) and the file it lives in. Usually one per app.
ModelContext	Your workspace for inserting, changing, and deleting objects before they are saved.
@Query	A property wrapper that fetches models for a view and refreshes the view when they change.

CONCEPT

Core Data, the framework underneath

Core Data is Apple's original framework for storing objects, first shipped in 2005 and still fully supported in iOS 26. You describe your types in a model editor, subclass `NSManagedObject`, and fetch with request objects. SwiftData is built on top of it: the same SQLite storage engine and the same migration machinery, driven by Swift macros and ordinary classes instead of an editor and Objective-C conventions.

You will still meet Core Data often: in apps written before SwiftData appeared, in most tutorials and books, whose sample apps usually predate it, and in large codebases where migrating is not worth the risk. When you do, map each idea to the one you already know:

NSManagedObject is @Model	A Core Data entity is a subclass of NSManagedObject with @NSManaged properties; a SwiftData model is a plain final class marked @Model.
NSPersistentContainer is ModelContainer	Core Data opens the store from a .xcdatamodeld file through NSPersistentContainer; SwiftData opens it from your types through ModelContainer.
NSFetchRequest is @Query	Core Data fetches with NSFetchRequest, or @FetchRequest in SwiftUI; SwiftData uses @Query in views and FetchDescriptor everywhere else.
NSManagedObjectContext is ModelContext	Both are the scratchpad where you insert, change, and delete objects before a save.

CODE · READ-ALONG

Define a model

A model is a class marked with `@Model`. The macro (code that writes code at compile time) adds everything needed to store, track, and observe the class. You declare ordinary stored properties and a normal initializer. The class is called `TaskItem` rather than `Task`, so it never clashes with Swift's concurrency type named Task.

```
1  import Foundation
2  import SwiftData
3
4  @Model
5  final class TaskItem {
6      var title: String
7      var isDone: Bool
8      var priority: Int
9      var createdAt: Date
10
11     init(title: String, priority: Int = 1) {
12         self.title = title
13         self.isDone = false
14         self.priority = priority
15         self.createdAt = .now
16     }
17 }
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, resolving the Foundation module while it builds this model file.
- **WHAT** **Makes Date available for the createdAt property.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; Date becomes usable for createdAt from here down.
- **WHY** Model files commonly use Foundation types such as Date, URL, and UUID.

2 `import SwiftData`

- **WHO** The Swift compiler, resolving the SwiftData module while it builds this model file.
- **WHAT** **Makes the @Model macro and the rest of SwiftData available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; the @Model macro becomes usable from here down.
- **WHY** Only files that use SwiftData need this import.

4 `@Model`

- **WHO** The SwiftData macro, expanded by the compiler.
- **WHAT** **Generates the code that lets SwiftData store, load, and track changes to this class.**
- **WHERE** Attached to the class declaration on the next line.
- **WHEN** At compile time, when the compiler expands the macro into TaskItem's persistence code; nothing runs on the device yet.
- **WHY** The macro writes the persistence code for you, so the class stays plain Swift.

```
5  final class TaskItem {
```

WHO	You, declaring the TaskItem class that SwiftData will store.
WHAT	**Declares the model class for one task.**
WHERE	A reference type in your app module.
WHEN	Instances are created when the user adds a task, or when SwiftData loads one from disk.
WHY	Models are classes because SwiftData must track each object's identity and changes; final allows no subclasses.

```
6      var title: String
```

WHO	Your TaskItem model, declaring the title property that SwiftData persists.
WHAT	**A stored property that becomes a column in the database.**
WHERE	A property of TaskItem, saved in the SQLite file.
WHEN	Saved whenever the context saves.
WHY	Plain var properties are persisted automatically; there is nothing extra to declare.

```
7      var isDone: Bool
```

WHO	Your TaskItem model, declaring the isDone flag that SwiftData persists.
WHAT	**Whether the task is complete.**
WHERE	A stored, persisted property.
WHEN	Changed when the user checks the task off.
WHY	Changing it on the object is enough; SwiftData notices and saves it.

```
8      var priority: Int
```

WHO	Your TaskItem model, declaring the priority property that SwiftData persists.
WHAT	**A priority from 1 (low) to 3 (high).**
WHERE	A stored, persisted property.
WHEN	Set when the task is created or edited.
WHY	A number is easy to sort by in a query.

```
9      var createdAt: Date
```

WHO	Your TaskItem model, declaring the createdAt date that SwiftData persists.
WHAT	**When the task was created.**
WHERE	A stored, persisted property.
WHEN	Set once in the initializer.
WHY	Storing a creation date gives you a stable default sort order.

```
11     init(title: String, priority: Int = 1) {
```

WHO	Your TaskItem model, declaring its init(title:priority:) initializer.
WHAT	**Declares how to create a new task; priority defaults to 1.**
WHERE	An initializer of TaskItem.
WHEN	Whenever your code creates a task.
WHY	Unlike structs, classes get no memberwise initializer, so models need one written out.

```
12         self.title = title
```

WHO	TaskItem's init(title:priority:), written by you, copying the title argument into the property.
WHAT	**Stores the given title.**
WHERE	In the new object.
WHEN	During initialization, as the first assignment, before the new TaskItem can be used or inserted.
WHY	self. separates the property from the parameter that has the same name.

13 `self.isDone = false`

WHO TaskItem's init(title:priority:), written by you, choosing a fixed starting value.
WHAT **Starts every task as not done.**
WHERE In the new object.
WHEN During initialization, right after title is set, so no caller ever passes isDone.
WHY Setting it here means callers never have to pass it.

14 `self.priority = priority`

WHO TaskItem's init(title:priority:), written by you, copying the priority argument into the property.
WHAT **Stores the given priority.**
WHERE In the new object.
WHEN During initialization, using the priority argument or its default of 1.
WHY Each stored property must have a value before init ends.

15 `self.createdAt = .now`

WHO TaskItem's init(title:priority:), written by you, calling Date.now.
WHAT **Records the current moment as the creation time.**
WHERE In the new object.
WHEN During initialization, as the last assignment, capturing the moment the task was created.
WHY .now is shorthand for Date.now; the type is known from the property.

> **Try it**
>
> 1. Add a property var notes: String = "". Why is giving it a default value a good habit for future versions of your app?

CODE · READ-ALONG

Create the container once, in the App

The `.modelContainer(for:)` modifier opens the database (creating it on the first launch) and puts its main `ModelContext` into the SwiftUI environment. Every view in the window can then read and write tasks. For a simple app, this one line is all the setup SwiftData needs.

```
import SwiftUI
import SwiftData

@main
struct TasksApp: App {
    var body: some Scene {
        WindowGroup {
            NavigationStack {
                TaskListView()
            }
        }
        .modelContainer(for: TaskItem.self)
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module while it builds the app's main file.
- **WHAT** **Makes App, Scene, and WindowGroup available.**
- **WHERE** At the top of the app's main file.
- **WHEN** At compile time, before any line of this file runs; App, Scene, and WindowGroup become usable from here down.
- **WHY** The app structure is defined with SwiftUI types.

2 `import SwiftData`

- **WHO** The Swift compiler, resolving the SwiftData module while it builds the app's main file.
- **WHAT** **Makes the modelContainer modifier available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; the modelContainer modifier becomes usable from here down.
- **WHY** SwiftData adds this modifier to SwiftUI scenes and views.

4 `@main`

- **WHO** The Swift compiler and iOS.
- **WHAT** **Marks this type as the app's entry point.**
- **WHERE** On the App struct below.
- **WHEN** When iOS launches the app.
- **WHY** Exactly one type in an app has @main.

5 `struct TasksApp: App {`

- **WHO** You, declaring the TasksApp struct that iOS launches because of @main.
- **WHAT** **Declares the app itself.**
- **WHERE** The root type of the app.
- **WHEN** Created once at launch.
- **WHY** An App describes the scenes, here one window, that the app shows.

6 `var body: some Scene {`

- **WHO** SwiftUI, which reads TasksApp's body to learn which scenes to create.
- **WHAT** **Describes the app's scenes.**
- **WHERE** A computed property of TasksApp.
- **WHEN** At launch, and whenever SwiftUI needs it.
- **WHY** Scenes are to the app what views are to a screen.

7 `WindowGroup {`

- **WHO** SwiftUI's WindowGroup scene type, inside TasksApp's body.
- **WHAT** **Creates the main window, or several windows on iPad.**
- **WHERE** The only scene inside TasksApp's body; its content becomes the main window.
- **WHEN** When iOS launches the app and SwiftUI evaluates body for the first time.
- **WHY** WindowGroup is the standard scene for a regular app.

8 `NavigationStack {`

- **WHO** SwiftUI's NavigationStack container, wrapping the task list.
- **WHAT** **Gives the list a navigation bar, where its toolbar button appears.**
- **WHERE** Inside the WindowGroup closure, as the root view of the main window.
- **WHEN** When the window's content is first built at launch, before TaskListView appears.
- **WHY** Toolbar items need a navigation container to show up in.

9 `TaskListView()`

WHO	Your TaskListView struct, created here as the stack's root screen.
WHAT	**Shows the task list as the first screen.**
WHERE	Inside the navigation stack.
WHEN	When the NavigationStack builds its content at launch, before any task is shown.
WHY	The list reads tasks through the environment set up below.

12 `.modelContainer(for: TaskItem.self)`

WHO	SwiftData, through the modelContainer(for:) modifier it adds to SwiftUI scenes.
WHAT	**Creates or opens the database for TaskItem and shares it with every view in the window.**
WHERE	A modifier on the WindowGroup.
WHEN	When the scene is created at launch.
WHY	Setting it on the scene, not on a single view, makes one shared store available everywhere.

CODE · READ-ALONG

Insert, list, and delete

Inside a view, `@Environment(\.modelContext)` gives you the context and `@Query` gives you the data. Insert a new object and it appears in the list immediately; delete one and it disappears. You never reload the list by hand, because `@Query` watches the context for you. The simulation beside the code shows the context, the database rows, and the query side by side.

```swift
import SwiftUI
import SwiftData

struct TaskListView: View {
    @Environment(\.modelContext) private var context
    @Query(sort: \TaskItem.createdAt, order: .reverse)
    private var tasks: [TaskItem]

    var body: some View {
        List {
            ForEach(tasks) { task in Text(task.title) }
                .onDelete { offsets in
                    for index in offsets {
                        context.delete(tasks[index])
                    }
                }
        }
        .toolbar {
            Button("Add", systemImage: "plus") {
                context.insert(TaskItem(title: "New task"))
            }
        }
    }
}
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, resolving the SwiftUI module while it builds the list screen's file.
WHAT	**Makes SwiftUI available.**
WHERE	At the top of the file.
WHEN	At compile time, before any line of this file runs; List, ForEach, and Button become usable from here down.
WHY	The list screen is a SwiftUI view.

```
2  import SwiftData
```

WHO	The Swift compiler, resolving the SwiftData module while it builds the list screen's file.
WHAT	**Makes @Query and ModelContext available.**
WHERE	At the top of the file.
WHEN	At compile time, before any line of this file runs; @Query and ModelContext become usable from here down.
WHY	Views that read or write models import SwiftData.

```
4  struct TaskListView: View {
```

WHO	You, declaring the TaskListView struct that shows, adds, and deletes tasks.
WHAT	**Declares the screen that lists tasks.**
WHERE	A SwiftUI view type.
WHEN	Created by the App at launch.
WHY	One view owns the list, the insert, and the delete for clarity.

```
5  @Environment(\.modelContext) private var context
```

WHO	SwiftUI's environment, filled in by .modelContainer.
WHAT	**Gives this view the main ModelContext.**
WHERE	A property of the view, provided from outside.
WHEN	Available as soon as the view is created.
WHY	The context is how you insert and delete; views never create their own.

```
6  @Query(sort: \TaskItem.createdAt, order: .reverse)
```

WHO	SwiftData's @Query macro, expanded by the compiler on the tasks property below.
WHAT	**Declares a fetch of all TaskItem objects, newest first.**
WHERE	Attached to the property on the next line.
WHEN	Runs when the view appears, then again whenever matching data changes.
WHY	The key path \TaskItem.createdAt names the property to sort by, checked by the compiler.

```
7  private var tasks: [TaskItem]
```

WHO	SwiftData, which fills this array.
WHAT	**Holds the current query results.**
WHERE	A read-only property of the view.
WHEN	Updated automatically after every insert, delete, or change.
WHY	You read it like a normal array, but only SwiftData writes it.

```
9  var body: some View {
```

WHO	SwiftUI, which reads TaskListView's body whenever the query results change.
WHAT	**Describes the screen.**
WHERE	A computed property of TaskListView, required by the View protocol.
WHEN	Again whenever tasks changes.
WHY	Because tasks is live, the list always matches the database.

```
10 List {
```

WHO SwiftUI's List container, holding the ForEach of tasks.
WHAT **Creates a scrolling list.**
WHERE The root of the screen.
WHEN Each time body is evaluated, which happens after every change to the query results.
WHY List renders only the rows on screen, so it handles thousands of tasks.

```
11 ForEach(tasks) { task in Text(task.title) }
```

WHO SwiftUI's ForEach, creating one Text row for each TaskItem in tasks.
WHAT **Creates one row per task, showing its title.**
WHERE Inside the List closure, producing its rows.
WHEN For every task in the results.
WHY Models are Identifiable automatically, so ForEach can track rows across changes.

```
12 .onDelete { offsets in
```

WHO SwiftUI, and the user swiping a row.
WHAT **Enables swipe-to-delete and receives the positions of the rows to remove.**
WHERE A modifier on the ForEach.
WHEN When the user deletes a row.
WHY onDelete gives positions, so you look up the objects in tasks.

```
13 for index in offsets {
```

WHO Swift's for-in loop, stepping through the IndexSet that onDelete passed in.
WHAT **Visits each position being deleted.**
WHERE Inside the delete handler.
WHEN Once per deleted row.
WHY The user can delete more than one row at a time in edit mode.

```
14 context.delete(tasks[index])
```

WHO SwiftData's ModelContext, through its delete(_:) method on the context from the environment.
WHAT **Marks that task as deleted; it disappears from the query at once.**
WHERE In the context, which then removes the row from the database when it saves.
WHEN Immediately, with the file updated at the next save.
WHY You delete through the context, never by changing the tasks array.

```
18 .toolbar {
```

WHO SwiftUI's toolbar modifier, adding items to the navigation bar that TasksApp provided.
WHAT **Adds items to the navigation bar.**
WHERE A modifier on the list.
WHEN Each time body is evaluated; the toolbar is described along with the list.
WHY Primary actions such as Add belong in the toolbar.

```
19 Button("Add", systemImage: "plus") {
```

WHO SwiftUI, and the user tapping.
WHAT **Shows a plus button in the toolbar.**
WHERE Inside the toolbar closure, so it appears in the navigation bar above the list.
WHEN Its action runs on each tap.
WHY The title stays available to VoiceOver even when only the icon is shown.

```
20  context.insert(TaskItem(title: "New task"))
```

WHO	SwiftData's ModelContext, through insert(_:), registering the TaskItem you just created.
WHAT	**Creates a task and registers it with the context, so @Query shows it at once.**
WHERE	In the context, then in the database after the next save.
WHEN	Each time the user taps the Add button, inside its action closure.
WHY	Creating a TaskItem alone stores nothing; insert is what makes it part of the database.

Try it

1. In the simulation, add a task and watch the database table. When does the new row appear there, compared with the list?

INTERACTIVE

Watch SwiftData work

Left: a small task app. Middle: the rows actually stored in the database file. Right: the `@Query` the view declares and its results. Add, complete, and delete tasks, then change the sort and type a filter. The Swift code updates to match. Notice which actions change the stored rows, which only change the query, and when autosave writes pending changes to disk.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Sort and filter with #Predicate

A `#Predicate` is a macro that turns a Swift closure into a query the database runs itself, so only matching rows are loaded. A query declared with `@Query` is fixed when the view is created, so a search text that changes must come in through the initializer, which builds a new `Query` each time.

```swift
import SwiftUI
import SwiftData

struct FilteredTaskList: View {
    @Query private var tasks: [TaskItem]

    init(search: String, hideDone: Bool) {
        let predicate = #Predicate<TaskItem> { task in
            (!hideDone || !task.isDone) &&
            (search.isEmpty ||
                task.title.localizedStandardContains(search))
        }
        _tasks = Query(
            filter: predicate,
            sort: [SortDescriptor(\.priority, order: .reverse),
                   SortDescriptor(\.title)]
        )
    }

    var body: some View {
        List(tasks) { task in Text(task.title) }
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module while it builds the filtered list's file.
- **WHAT** **Makes SwiftUI available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; View and List become usable from here down.
- **WHY** This is a SwiftUI view.

2 `import SwiftData`

- **WHO** The Swift compiler, resolving the SwiftData module while it builds the filtered list's file.
- **WHAT** **Makes Query, #Predicate, and SortDescriptor available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; Query, #Predicate, and SortDescriptor become usable from here down.
- **WHY** Queries and predicates come from SwiftData and Foundation.

4 `struct FilteredTaskList: View {`

- **WHO** You, declaring the FilteredTaskList struct that shows only matching tasks.
- **WHAT** **Declares a list that shows only the tasks matching a search.**
- **WHERE** A SwiftUI view type.
- **WHEN** Created by a parent view that owns the search text.
- **WHY** The parent holds the changing state; this view only displays results for it.

```
5  @Query private var tasks: [TaskItem]
```

WHO SwiftData's @Query property wrapper, left unconfigured here so init can configure it.
WHAT **Declares the query results without configuring the query yet.**
WHERE A property of the view.
WHEN Configured in init, then kept up to date automatically.
WHY Leaving the query unconfigured here lets init supply a filter built from parameters.

```
7  init(search: String, hideDone: Bool) {
```

WHO Your FilteredTaskList struct, declaring its init(search:hideDone:) initializer.
WHAT **Receives the search text and the hide-done switch from the parent.**
WHERE The initializer of FilteredTaskList, which the parent calls with the current search text and switch.
WHEN Every time the parent's body runs with new values.
WHY A new init with new values is how a query's filter changes.

```
8  let predicate = #Predicate<TaskItem> { task in
```

WHO The #Predicate macro, at compile time.
WHAT **Starts a condition that each TaskItem must satisfy.**
WHERE A local constant in init.
WHEN Built each time init runs; evaluated by the database.
WHY The macro checks the closure at compile time and translates it into a database query.

```
9  (!hideDone || !task.isDone) &&
```

WHO The #Predicate closure you wrote, evaluated by the database for each TaskItem row.
WHAT **Keeps a task if hiding is off, or if the task is not done.**
WHERE Inside the predicate closure.
WHEN For each candidate row, in the database.
WHY Folding the switch into the condition avoids writing two separate queries.

```
10 (search.isEmpty ||
```

WHO The #Predicate closure, reading the captured search string.
WHAT **Keeps every task when the search is empty.**
WHERE Inside the predicate closure.
WHEN For each candidate row.
WHY An empty search should show everything, not nothing.

```
11 task.title.localizedStandardContains(search))
```

WHO The #Predicate closure, calling localizedStandardContains on each task's title.
WHAT **Otherwise keeps tasks whose title contains the search text.**
WHERE Inside the predicate closure.
WHEN For each candidate row.
WHY localizedStandardContains ignores case and accents, the way the Finder's search does.

```
13 _tasks = Query(
```

WHO Your initializer, replacing the property wrapper itself.
WHAT **Builds a new Query for the tasks property.**
WHERE In init; the underscore names the wrapper behind tasks.
WHEN Each time init runs.
WHY Assigning to _tasks configures the query; assigning to tasks is not allowed.

```
14  filter: predicate,
```

WHO SwiftData's Query initializer, receiving the predicate through its filter parameter.
WHAT **Applies the predicate built above.**
WHERE An argument to Query.
WHEN When the query runs.
WHY Filtering in the database means unmatched rows are never loaded into memory.

```
15  sort: [SortDescriptor(\.priority, order: .reverse),
```

WHO SwiftData's Query initializer, receiving the first SortDescriptor in its sort array.
WHAT **Sorts by priority, highest first.**
WHERE An argument to Query.
WHEN When the query runs.
WHY A list of sort descriptors applies them in order.

```
16  SortDescriptor(\.title)]
```

WHO SwiftData's Query initializer, receiving the second SortDescriptor as a tie-breaker.
WHAT **Breaks ties by title, A to Z.**
WHERE The second sort descriptor.
WHEN When two tasks have the same priority.
WHY A tie-breaker keeps the order stable from one refresh to the next.

```
20  var body: some View {
```

WHO SwiftUI, which reads FilteredTaskList's body after the query results change.
WHAT **Describes the list.**
WHERE A computed property of FilteredTaskList, required by the View protocol.
WHEN Each time the query's results change, and again whenever the parent recreates this view with new init values.
WHY The body never filters or sorts; the query already did.

```
21  List(tasks) { task in Text(task.title) }
```

WHO SwiftUI's List, built directly from the tasks array the query filled.
WHAT **Shows one row per matching task.**
WHERE The root of this view.
WHEN Each time body is evaluated, after the database has already filtered and sorted the rows.
WHY List can take a collection directly when you do not need onDelete.

> **Try it**
>
> 1. How would you change the predicate to show only priority 3 tasks? Write the condition.

CODE · READ-ALONG

Fetch and save outside a view

Views use `@Query`, but other code, such as a Clear Completed button, can fetch directly with a `FetchDescriptor`. The main context autosaves pending changes at convenient moments, for example after user interactions and when the app moves to the background. Calling `save()` yourself guarantees the changes are on disk before you continue.

```swift
import SwiftData

@MainActor
func clearCompleted(in context: ModelContext) throws -> Int {
    let descriptor = FetchDescriptor<TaskItem>(
        predicate: #Predicate { $0.isDone }
    )
    let done = try context.fetch(descriptor)
    for task in done {
        context.delete(task)
    }
    try context.save()
    return done.count
}
```

Line by line

1 `import SwiftData`

- **WHO** The Swift compiler, resolving the SwiftData module while it builds this helper's file.
- **WHAT** **Makes ModelContext and FetchDescriptor available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; ModelContext and FetchDescriptor become usable from here down.
- **WHY** This code has no interface, so SwiftUI is not imported.

3 `@MainActor`

- **WHO** Swift concurrency's @MainActor attribute, checked by the compiler on clearCompleted(in:).
- **WHAT** **Requires this function to run on the main actor, the main thread.**
- **WHERE** Attached to the function below.
- **WHEN** Checked at compile time.
- **WHY** The main context belongs to the main actor, so code using it must run there.

4 `func clearCompleted(in context: ModelContext) throws -> Int {`

- **WHO** You, declaring clearCompleted(in:), a helper that deletes finished tasks.
- **WHAT** **Declares a function that deletes finished tasks and returns how many it removed.**
- **WHERE** A top-level function, often called from a button.
- **WHEN** When the user asks to clear completed tasks.
- **WHY** Passing the context in makes the function easy to test with an in-memory container.

5 `let descriptor = FetchDescriptor<TaskItem>(`

- **WHO** SwiftData's FetchDescriptor type, created for TaskItem by its initializer.
- **WHAT** **Starts describing what to fetch: TaskItem objects.**
- **WHERE** A local constant inside clearCompleted(in:), describing the fetch before it runs.
- **WHEN** Each time clearCompleted(in:) runs, before fetch(_:) is called.
- **WHY** A FetchDescriptor is the non-view equivalent of @Query.

6 `predicate: #Predicate { $0.isDone }`

- **WHO** The #Predicate macro, expanded by the compiler into a condition the database evaluates.
- **WHAT** **Limits the fetch to tasks that are done.**
- **WHERE** An argument of the descriptor.
- **WHEN** Evaluated by the database during the fetch.
- **WHY** The element type is inferred from FetchDescriptor<TaskItem>, so $0 is a TaskItem.

```
8  let done = try context.fetch(descriptor)
```

WHO — SwiftData's ModelContext, inside fetch(_:), running the descriptor against the store.
WHAT — **Runs the query and returns the matching tasks as an array.**
WHERE — A local constant inside clearCompleted(in:), holding the [TaskItem] array fetch returned.
WHEN — Immediately, when this line runs.
WHY — fetch can throw, for example if the store cannot be read.

```
9  for task in done {
```

WHO — Swift's for-in loop, stepping through the done array that fetch(_:) returned.
WHAT — **Visits each completed task.**
WHERE — Inside clearCompleted(in:), after the fetch and before the save.
WHEN — Once per completed task in done; the loop is skipped entirely when nothing is done.
WHY — Each object must be deleted through the context.

```
10 context.delete(task)
```

WHO — SwiftData's ModelContext, through delete(_:) on the context passed into the function.
WHAT — **Marks the task as deleted.**
WHERE — In the context; the file changes at the next save.
WHEN — Once per completed task, on each pass of the loop over done.
WHY — Any @Query on screen updates immediately.

```
12 try context.save()
```

WHO — SwiftData's ModelContext, inside save(), writing every pending deletion to the store.
WHAT — **Writes every pending change to the database file now.**
WHERE — From memory to the SQLite file in Application Support.
WHEN — When this line runs.
WHY — An explicit save makes the result durable before you report success.

```
13 return done.count
```

WHO — Your clearCompleted(in:) function, returning the number of deleted tasks.
WHAT — **Returns how many tasks were removed.**
WHERE — The return value of clearCompleted(in:), its last line.
WHEN — After save() has succeeded, so the count reflects deletions that are already on disk.
WHY — The caller can show "Removed 3 tasks" as feedback.

CHECK YOURSELF

Check your understanding

1. Which line adds a new task to the database?

   A. let t = TaskItem(title: "Run")
   B. context.insert(TaskItem(title: "Run"))
   C. tasks.append(TaskItem(title: "Run"))
   D. @Query var t = TaskItem(title: "Run")

2. Tasks A, B, and C were created at 9:00, 9:05, and 9:10. In what order does `@Query(sort: \TaskItem.createdAt, order: .reverse)` list them?

   A. A, B, C
   B. In random order
   C. C, B, A
   D. Alphabetically: A, B, C

3. Why does `FilteredTaskList` receive the search text in its initializer and build a new `Query`?

   A. @Query cannot sort results, so sorting must happen in init
   B. #Predicate accepts only constants, so the text must be copied in init
   C. SwiftData cannot search strings unless the view passes them to init
   D. The query is fixed at creation, so a new filter needs a new Query

4. Which statement about SwiftData storage is true?

   A. By default it stores models in a SQLite file
   B. It keeps data only in memory until you call save()
   C. It saves models to UserDefaults as plists
   D. It needs an internet connection to save

Answers: SwiftData Fundamentals

1. **B. context.insert(TaskItem(title: "Run"))** Creating a model object only makes it in memory; insert registers it with the context so it is shown and saved. The tasks array from @Query is read-only, so append does not compile, and @Query declares a fetch, not a new object.
2. **C. C, B, A** The sort is by createdAt, and .reverse puts the latest first, so C, B, A. Without order: .reverse it would be A, B, C. The titles happen to be alphabetical here, but the sort ignores them.
3. **D. The query is fixed at creation, so a new filter needs a new Query** The @Query attribute captures its configuration once. When the search text changes, the parent recreates the child view with the new text, and init builds a fresh query. Predicates can use captured values such as search, so the second choice is false.
4. **A. By default it stores models in a SQLite file** The default store is a SQLite file in Application Support, which is backed up. The main context also autosaves, so changes reach disk even if you never call save(), although calling it gives you a guarantee. SwiftData works fully offline.

Relationships and Schema Changes

CONCEPT

Models that point to each other

Real data is connected: a folder holds notes, a playlist holds songs, an order holds line items. In SwiftData a connection is just a property whose type is another model, or an array of them. That property is a **relationship**.

The most common shape is **one-to-many**: one Folder has many Notes, and each Note belongs to at most one Folder. The two properties that describe the same link from each end are *inverses* of each other. SwiftData keeps them in step, so setting `note.folder` also adds the note to `folder.notes`.

To-many end	An array, such as var notes: [Note].
To-one end	An optional, such as var folder: Folder?, because a note may not be in any folder yet.
@Relationship	An attribute where you name the inverse and choose what happens when an object is deleted.

CODE · READ-ALONG

Folder and Note, linked

Two models with a one-to-many relationship. The `@Relationship` attribute lives on the to-many end and names the inverse with a key path, `\Note.folder`. Its delete rule, `.cascade`, says that deleting a folder also deletes its notes.

```swift
import SwiftData

@Model
final class Folder {
    var name: String
    @Relationship(deleteRule: .cascade, inverse: \Note.folder)
    var notes: [Note] = []

    init(name: String) {
        self.name = name
    }
}

@Model
final class Note {
    var text: String
    var folder: Folder?

    init(text: String, folder: Folder? = nil) {
        self.text = text
        self.folder = folder
    }
}
```

Line by line

1 `import SwiftData`

WHO The Swift compiler, resolving the SwiftData module while it builds this models file.
WHAT **Makes @Model and @Relationship available.**
WHERE At the top of the file.
WHEN At compile time, before any line of this file runs; the @Model and @Relationship macros need it to expand.
WHY SwiftData also brings in the basic types these models use.

3 `@Model`

WHO SwiftData's @Model macro, expanded by the compiler on the Folder class.
WHAT **Makes the class below a stored model.**
WHERE On the Folder class.
WHEN At compile time, while the compiler expands the macro into Folder's persistence code.
WHY Every type that takes part in a relationship must be a model.

4 `final class Folder {`

WHO Your Folder class, the one end of the one-to-many link.
WHAT **Declares the model for a folder.**
WHERE A class in your app module.
WHEN Instances are created when the user makes a folder or SwiftData loads one.
WHY The one end of the one-to-many relationship.

5 `var name: String`

WHO Your Folder model, which owns this attribute.
WHAT **The folder's name.**
WHERE A stored property, saved in the database.
WHEN Set when the folder is created.
WHY An ordinary attribute next to the relationship.

6 `@Relationship(deleteRule: .cascade, inverse: \Note.folder)`

WHO The SwiftData @Relationship attribute.
WHAT **Sets the delete rule to cascade and names Note.folder as the inverse.**
WHERE Attached to the notes property on the next line.
WHEN Applied when objects are linked, and again when a folder is deleted.
WHY Naming the inverse explicitly removes any guesswork about which properties describe the same link.

7 `var notes: [Note] = []`

WHO Your Folder model, with SwiftData keeping the array in step with Note.folder.
WHAT **The to-many end: every note in this folder.**
WHERE A relationship stored as links in the database, not copies of the notes.
WHEN Changes whenever a note joins or leaves the folder.
WHY An empty array default means a new folder starts with no notes.

9 `init(name: String) {`

WHO Folder's init(name:), written by you.
WHAT **Creates a folder with a name.**
WHERE An initializer of Folder.
WHEN Whenever your code creates a folder.
WHY notes already has a default, so init only needs the name.

```
10  self.name = name
```

WHO Folder's init(name:), copying its only parameter.
WHAT **Stores the name.**
WHERE In the new object.
WHEN During initialization of a Folder, before SwiftData can track the new object.
WHY self. distinguishes the property from the parameter.

```
14  @Model
```

WHO SwiftData's @Model macro, this time expanded on the Note class.
WHAT **Makes Note a stored model too.**
WHERE On the Note class.
WHEN At compile time, when the compiler generates Note's persistence code.
WHY Both ends of a relationship are models.

```
15  final class Note {
```

WHO Your Note class, the many end of the relationship.
WHAT **Declares the model for a note.**
WHERE A class in your app module.
WHEN Created when the user writes a note.
WHY The many end of the relationship.

```
16      var text: String
```

WHO Your Note model, which owns this attribute.
WHAT **The note's content.**
WHERE A stored property of Note, saved as a column in the SQLite file.
WHEN Set on creation and changed by editing.
WHY An ordinary attribute.

```
17      var folder: Folder?
```

WHO Your Note model, with SwiftData mirroring this link into Folder.notes.
WHAT **The to-one end: the folder this note is in, or nil.**
WHERE A relationship property of Note.
WHEN Changes when the note is filed, moved, or unfiled.
WHY It is optional because a note can exist outside any folder, and because SwiftData sets it to nil when the link is removed.

```
19      init(text: String, folder: Folder? = nil) {
```

WHO Note's init(text:folder:), written by you.
WHAT **Creates a note, optionally already in a folder.**
WHERE An initializer of Note.
WHEN Whenever your code creates a note.
WHY A default of nil makes the folder argument optional at the call site.

```
20          self.text = text
```

WHO Note's init(text:folder:), storing the first parameter.
WHAT **Stores the text.**
WHERE In the new object.
WHEN During initialization of a Note, before the object joins any context.
WHY Every stored property needs a value before init ends.

```
21  self.folder = folder
```

WHO Note's init(text:folder:), storing the optional second parameter.
WHAT **Stores the folder link, if any.**
WHERE In the new object.
WHEN During initialization, whether the caller passed a folder or left the default nil.
WHY Once both objects are in a context, SwiftData mirrors this link into folder.notes.

CONCEPT

Delete rules

When you delete a folder, what should happen to its notes? The `deleteRule` on the relationship decides. Pick a rule in the figure, delete a folder, and watch the notes.

.nullify (default)	The notes stay, and their folder becomes nil.
.cascade	The notes are deleted too. Right when children cannot exist alone.
.deny	Saving fails while the folder still has notes, so nothing is deleted.
.noAction	Nothing is updated, leaving notes that point at a deleted folder. Rarely what you want.

Interactive figure in the HTML version: data cascade sim.

CODE · READ-ALONG

Connecting objects

You link objects by setting the to-one end or by appending to the to-many end; either way SwiftData updates the inverse. Insert each object into the context before linking it, so every object is managed by the same context. Functions that use the main context are marked `@MainActor`, because that context belongs to the main thread.

```
1  import SwiftData
2
3  @MainActor
4  func addSampleData(to context: ModelContext) {
5      let work = Folder(name: "Work")
6      context.insert(work)
7
8      let agenda = Note(text: "Draft the agenda")
9      context.insert(agenda)
10     agenda.folder = work
11
12     let budget = Note(text: "Check the budget")
13     context.insert(budget)
14     work.notes.append(budget)
15 }
```

Line by line

```
1  import SwiftData
```

WHO	The Swift compiler, resolving the SwiftData module for this helper file.
WHAT	**Makes ModelContext available.**
WHERE	At the top of the file.
WHEN	At compile time, before addSampleData(to:) can run; nothing is imported at run time.
WHY	This helper needs only SwiftData.

```
3  @MainActor
```

WHO	Swift concurrency's main actor, applied to addSampleData(to:).
WHAT	**Requires the function to run on the main actor.**
WHERE	Attached to the function below.
WHEN	Checked at compile time.
WHY	The context passed in is the main context, which must only be used on the main actor.

```
4  func addSampleData(to context: ModelContext) {
```

WHO	Your addSampleData(to:) helper, written by you.
WHAT	**Declares a helper that creates a folder with two notes.**
WHERE	A top-level function in your app module, outside any type.
WHEN	Called once, for example on first launch or in a preview.
WHY	Taking the context as a parameter makes the helper reusable in tests and previews.

```
5  let work = Folder(name: "Work")
```

WHO	Your code, calling Folder's init(name:).
WHAT	**Creates a Folder object in memory.**
WHERE	A local constant inside addSampleData(to:), holding a Folder that no context knows yet.
WHEN	When the function runs.
WHY	An object is not part of the database until it is inserted.

```
6  context.insert(work)
```

WHO	The ModelContext passed into addSampleData(to:), in insert(_:).
WHAT	**Registers the folder so it will be saved and shown by queries.**
WHERE	In the context, then in the database file on the next save.
WHEN	Immediately after the folder is created, and before anything is linked to it.
WHY	Insert first, then link: it keeps every object managed by the same context.

```
8  let agenda = Note(text: "Draft the agenda")
```

WHO	Your code, calling Note's init(text:folder:) with the folder left nil.
WHAT	**Creates the first note in memory.**
WHERE	A local constant inside addSampleData(to:), holding an unfiled Note.
WHEN	When the function runs.
WHY	It is not in any folder yet.

```
9  context.insert(agenda)
```

WHO	The ModelContext, in insert(_:), now registering the agenda note.
WHAT	**Registers the first note.**
WHERE	In the context's set of managed objects; the row reaches the database file at the next save.
WHEN	Immediately after the note is created, before the next line links it.
WHY	Inserted objects can now be linked safely.

```
10 agenda.folder = work
```

WHO	SwiftData's relationship tracking, triggered by assigning Note.folder.
WHAT	**Links the note to the folder from the to-one end; work.notes now includes agenda.**
WHERE	On the Note object; SwiftData updates the Folder side.
WHEN	Immediately in memory; saved with the next save.
WHY	You set one end, and SwiftData maintains the inverse for you.

```
12 let budget = Note(text: "Check the budget")
```

WHO	Your code, creating the second Note with init(text:folder:).
WHAT	**Creates a second note.**
WHERE	A local constant inside addSampleData(to:), holding a Note not yet in any folder.
WHEN	When the function runs.
WHY	This one is linked from the other end, to show both ways work.

```
13 context.insert(budget)
```

WHO	The ModelContext, in insert(_:), registering the budget note.
WHAT	**Registers the second note.**
WHERE	In the same context that already manages work and agenda.
WHEN	Immediately after the note is created, so it is managed before the append below.
WHY	Same habit: insert, then link.

```
14 work.notes.append(budget)
```

WHO	SwiftData's relationship tracking, triggered by appending to Folder.notes.
WHAT	**Links from the to-many end by appending to the array; budget.folder is now work.**
WHERE	On the Folder object; SwiftData updates the Note side.
WHEN	Immediately in memory; saved with the next save.
WHY	Appending reads naturally when you are working from the parent object.

CODE · READ-ALONG

Edit a model with @Bindable

Models are observable, so a view that reads a property refreshes when it changes. To edit one, a control needs a *binding*, a two-way connection written with `$`. `@Bindable` provides bindings to an observable object's properties. Each keystroke changes the stored note directly; there is no Save button, because autosave handles it.

```swift
import SwiftUI
import SwiftData

struct NoteEditor: View {
    @Bindable var note: Note

    var body: some View {
        Form {
            TextField("Text", text: $note.text, axis: .vertical)
            LabeledContent("Folder", value: note.folder?.name ?? "None")
            Button("Remove from folder") {
                note.folder = nil
            }
        }
        .navigationTitle("Edit Note")
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module for this editor view.
- **WHAT** **Makes SwiftUI and @Bindable available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; @Bindable needs it to resolve.
- **WHY** @Bindable is part of SwiftUI.

2 `import SwiftData`

- **WHO** The Swift compiler, resolving SwiftData so the Note model type is known here.
- **WHAT** **Makes the Note model's framework available here.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, when the compiler type-checks the Note property below.
- **WHY** The view works with SwiftData models.

4 `struct NoteEditor: View {`

- **WHO** Your NoteEditor struct, a SwiftUI view you define.
- **WHAT** **Declares a screen for editing one note.**
- **WHERE** A SwiftUI view type.
- **WHEN** Created when the user opens a note.
- **WHY** One editor view per model type is a common, simple structure.

5 `@Bindable var note: Note`

- **WHO** SwiftUI's @Bindable property wrapper.
- **WHAT** **Holds the note passed in and lets you create bindings to its properties.**
- **WHERE** A property of the view; the object itself is owned by SwiftData.
- **WHEN** Set when the view is created.
- **WHY** Use @Bindable, not @State, for an object someone else owns; @State is for values the view creates itself.

```
7  var body: some View {
```

WHO SwiftUI, reading NoteEditor's body to learn what to draw.
WHAT **Describes the editor.**
WHERE A computed property of NoteEditor, required by the View protocol.
WHEN Whenever a note property this view reads changes.
WHY Observation tracks exactly which properties body reads.

```
8  Form {
```

WHO SwiftUI's Form container, the root of NoteEditor's body.
WHAT **Groups the controls in a settings-style form.**
WHERE The root of the screen.
WHEN Whenever NoteEditor's body runs, which happens when a note property it reads changes.
WHY Forms suit editing screens.

```
9  TextField("Text", text: $note.text, axis: .vertical)
```

WHO SwiftUI, and the user typing.
WHAT **Edits note.text directly through a binding; the field grows to several lines.**
WHERE First row of the form.
WHEN Each keystroke changes the model object.
WHY $note.text exists only because note is @Bindable; the change is saved with the next autosave.

```
10 LabeledContent("Folder", value: note.folder?.name ?? "None")
```

WHO SwiftUI's LabeledContent view, reading through the note's folder relationship.
WHAT **Shows a label and a value: the note's folder name, or None.**
WHERE Second row of the form.
WHEN Refreshed when note.folder changes.
WHY Reading through the relationship works like any property access.

```
11 Button("Remove from folder") {
```

WHO SwiftUI, and the user tapping.
WHAT **Creates a button that unfiles the note.**
WHERE Third row of the form.
WHEN Its action runs on tap.
WHY Changing a relationship is just an assignment.

```
12 note.folder = nil
```

WHO SwiftData's relationship tracking, triggered by assigning nil to Note.folder.
WHAT **Removes the link; the note also disappears from the old folder's notes array.**
WHERE On the model object in the main context.
WHEN Each time the user taps the button.
WHY The inverse is updated automatically, and the row above refreshes to None.

```
15 .navigationTitle("Edit Note")
```

WHO SwiftUI's navigationTitle modifier on the Form.
WHAT **Sets the title in the navigation bar.**
WHERE A modifier on the form.
WHEN Whenever the view is shown inside a NavigationStack.
WHY A clear title tells the user what they are editing.

CODE · READ-ALONG

Previews with an in-memory container

A preview needs sample data, but it must never touch the real database. A container configured with `isStoredInMemoryOnly: true` lives only in memory and vanishes when the preview closes. Build it, insert sample objects, and pass it to the view with `.modelContainer`. The same pattern makes fast, isolated unit tests.

```swift
import SwiftUI
import SwiftData

#Preview {
    let config = ModelConfiguration(isStoredInMemoryOnly: true)
    let container = try! ModelContainer(
        for: Folder.self, Note.self,
        configurations: config
    )
    let folder = Folder(name: "Ideas")
    container.mainContext.insert(folder)
    let note = Note(text: "Try SwiftData")
    container.mainContext.insert(note)
    note.folder = folder

    return NavigationStack {
        NoteEditor(note: note)
    }
    .modelContainer(container)
}
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, resolving SwiftUI so #Preview and NavigationStack are available.
- WHAT — **Makes #Preview and the views available.**
- WHERE — At the top of the file.
- WHEN — At compile time, before Xcode builds the preview; nothing is imported at run time.
- WHY — Previews are written with SwiftUI.

2 `import SwiftData`

- WHO — The Swift compiler, resolving SwiftData for ModelContainer and ModelConfiguration.
- WHAT — **Makes ModelContainer and ModelConfiguration available.**
- WHERE — At the top of the file.
- WHEN — At compile time, when the preview target is built.
- WHY — The preview builds its own container.

4 `#Preview {`

- WHO — Xcode's #Preview macro, expanded by the compiler into a preview the canvas can run.
- WHAT — **Declares a preview that Xcode renders in the canvas.**
- WHERE — In the source file; it is not shipped in the app.
- WHEN — Whenever the canvas refreshes.
- WHY — Previews give instant feedback without launching the app.

```
5  let config = ModelConfiguration(isStoredInMemoryOnly: true)
```

WHO SwiftData's ModelConfiguration initializer, called inside the #Preview closure.
WHAT **Describes a store that lives only in memory.**
WHERE A local constant inside the preview.
WHEN Each time the preview builds.
WHY In memory means previews never read or damage real user data.

```
6  let container = try! ModelContainer(
```

WHO SwiftData's ModelContainer initializer, called with try! because this is only a preview.
WHAT **Creates a container with that configuration.**
WHERE A local constant inside the #Preview closure; the store it opens lives only in memory.
WHEN Each time the preview builds.
WHY try! is acceptable in a preview, where a failure should stop immediately; never use it for the real store.

```
7  for: Folder.self, Note.self,
```

WHO The ModelContainer initializer's for: parameter, listing the schema.
WHAT **Lists the model types in the schema.**
WHERE Arguments of the call.
WHEN When the container is created.
WHY Listing both types is clear, even though SwiftData would also find Note through the relationship.

```
8  configurations: config
```

WHO The ModelContainer initializer's configurations: parameter.
WHAT **Applies the in-memory configuration.**
WHERE The last argument of the ModelContainer call, after the model types.
WHEN When the container is created.
WHY Without it, the preview would open the default on-disk store.

```
10 let folder = Folder(name: "Ideas")
```

WHO Your code, calling Folder's init(name:) inside the preview.
WHAT **Creates a sample folder.**
WHERE A local constant inside the #Preview closure, before any insert.
WHEN Each time Xcode rebuilds the preview, so the sample data starts fresh every time.
WHY Realistic sample data shows the layout the way users will see it.

```
11 container.mainContext.insert(folder)
```

WHO The container's main context.
WHAT **Registers the sample folder.**
WHERE In the in-memory store.
WHEN Each time the preview rebuilds, right after the folder is created.
WHY mainContext is the same kind of context a running app's views use.

```
12 let note = Note(text: "Try SwiftData")
```

WHO Your code, calling Note's init(text:folder:) with no folder yet.
WHAT **Creates a sample note.**
WHERE A local constant inside the #Preview closure, holding the one note the editor will show.
WHEN Each time the preview rebuilds, after the folder has been inserted.
WHY The editor needs one note to show.

13 `container.mainContext.insert(note)`

WHO The container's mainContext, in insert(_:).
WHAT **Registers the sample note.**
WHERE In the in-memory store.
WHEN Each time the preview rebuilds, before the note is linked to the folder.
WHY Insert before linking, as in the app.

14 `note.folder = folder`

WHO SwiftData's relationship tracking, triggered by assigning Note.folder in the preview.
WHAT **Puts the note in the sample folder.**
WHERE In the in-memory store; the inverse Folder.notes is updated at the same time.
WHEN Each time the preview rebuilds, once both objects are inserted.
WHY Now the editor's Folder row has something to display.

16 `return NavigationStack {`

WHO Swift's return statement, handing a NavigationStack back to the #Preview macro.
WHAT **Returns the view to preview, wrapped in a navigation stack.**
WHERE The end of the preview closure.
WHEN Each time the preview rebuilds, after the sample data is in place.
WHY An explicit return is needed after statements; the stack shows the navigation title.

17 `NoteEditor(note: note)`

WHO Your NoteEditor view, created with the sample note.
WHAT **Shows the editor for the sample note.**
WHERE Inside the navigation stack.
WHEN Each time the preview rebuilds, inside the returned NavigationStack.
WHY Passing the model object matches how the app opens a note.

19 `.modelContainer(container)`

WHO SwiftData's modelContainer(_:) modifier on the NavigationStack.
WHAT **Puts the in-memory container into the preview's environment.**
WHERE A modifier on the navigation stack.
WHEN Each time the preview rebuilds, before NoteEditor reads its environment.
WHY Any @Query or modelContext inside now uses the sample data.

CODE · READ-ALONG

Change models safely with VersionedSchema

Once people use your app, their database matches your old models. SwiftData migrates simple changes automatically: adding a property with a default value, removing one, or renaming with `@Attribute(originalName:)`. For control, snapshot each version in a `VersionedSchema` and list the steps in a `SchemaMigrationPlan`. Version 1 here had `Note` with a `text` property.

```swift
import SwiftData

enum NotesSchemaV2: VersionedSchema {
    static let versionIdentifier = Schema.Version(2, 0, 0)
    static var models: [any PersistentModel.Type] { [Note.self] }

    @Model
    final class Note {
        @Attribute(originalName: "text") var body: String
        var isPinned: Bool = false
        init(body: String) { self.body = body }
    }
}

enum NotesMigrationPlan: SchemaMigrationPlan {
    static var schemas: [any VersionedSchema.Type] {
        [NotesSchemaV1.self, NotesSchemaV2.self]
    }
    static var stages: [MigrationStage] {
        [.lightweight(fromVersion: NotesSchemaV1.self,
                      toVersion: NotesSchemaV2.self)]
    }
}
```

Line by line

1 `import SwiftData`

- **WHO** The Swift compiler, resolving SwiftData for the schema and migration protocols.
- **WHAT** **Makes VersionedSchema and SchemaMigrationPlan available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before the schema enums and the migration plan are type-checked.
- **WHY** Migration types are part of SwiftData.

3 `enum NotesSchemaV2: VersionedSchema {`

- **WHO** Your NotesSchemaV2 enum, adopting SwiftData's VersionedSchema protocol.
- **WHAT** **Declares version 2 of the schema as a namespace.**
- **WHERE** A type in your app module.
- **WHEN** Read when the container opens a store.
- **WHY** An enum with no cases is a container for types that cannot be instantiated by mistake.

4 `static let versionIdentifier = Schema.Version(2, 0, 0)`

- **WHO** The VersionedSchema protocol's versionIdentifier requirement, which SwiftData reads.
- **WHAT** **Labels this snapshot as version 2.0.0.**
- **WHERE** A static property of the schema.
- **WHEN** Compared with the version recorded in the user's store at launch.
- **WHY** Each shipped schema needs a distinct, increasing version.

5 `static var models: [any PersistentModel.Type] { [Note.self] }`

- **WHO** The VersionedSchema protocol's models requirement, which SwiftData reads.
- **WHAT** **Lists the model types in this version.**
- **WHERE** A static computed property.
- **WHEN** Read when the container opens a store and during migration to version 2.
- **WHY** Note.self here means the Note nested in NotesSchemaV2.

```
7  @Model
```

WHO SwiftData's @Model macro, expanded on the Note class nested in NotesSchemaV2.
WHAT **Makes the nested class a model.**
WHERE Inside the schema enum.
WHEN At compile time, when the compiler generates persistence code for the version 2 Note.
WHY Nesting lets V1 and V2 each have a class named Note without clashing.

```
8  final class Note {
```

WHO Your version 2 Note class, nested inside NotesSchemaV2.
WHAT **Declares the version 2 shape of Note.**
WHERE Nested as NotesSchemaV2.Note, separate from NotesSchemaV1.Note.
WHEN Used by the app after migration.
WHY A frozen snapshot per version is what makes migrations reliable.

```
9  @Attribute(originalName: "text") var body: String
```

WHO The SwiftData @Attribute macro.
WHAT **Renames text to body while keeping the stored data.**
WHERE A property of version 2's Note.
WHEN Applied during migration from version 1.
WHY Without originalName, SwiftData would see one property deleted and another added, losing every note's text.

```
10  var isPinned: Bool = false
```

WHO Your version 2 Note model, which owns this new attribute.
WHAT **Adds a new property with a default value.**
WHERE A property of version 2's Note.
WHEN Existing notes receive false during migration.
WHY A new non-optional property needs a default, or old rows would have no value for it.

```
11  init(body: String) { self.body = body }
```

WHO NotesSchemaV2.Note's init(body:), written by you.
WHAT **Creates a version 2 note.**
WHERE An initializer of the nested class.
WHEN When the app creates notes after the update.
WHY isPinned already has a default, so init needs only body.

```
15  enum NotesMigrationPlan: SchemaMigrationPlan {
```

WHO Your NotesMigrationPlan enum, adopting SchemaMigrationPlan.
WHAT **Declares the plan that upgrades old stores.**
WHERE A type in your app module.
WHEN Consulted when a store with an older version is opened.
WHY One plan lists every version and every step between them.

```
16  static var schemas: [any VersionedSchema.Type] {
```

WHO The SchemaMigrationPlan protocol's schemas requirement, which SwiftData reads.
WHAT **Lists all schema versions, oldest first.**
WHERE A static computed property.
WHEN Read when the container opens.
WHY Order matters: SwiftData walks the list to find a path from the stored version.

```
17 [NotesSchemaV1.self, NotesSchemaV2.self]
```

WHO Your NotesMigrationPlan, returning the schema list.
WHAT **Names version 1 and version 2.**
WHERE The value returned by the schemas property, as an array of schema types.
WHEN Read when the container opens.
WHY Add each future version to the end of this list.

```
19 static var stages: [MigrationStage] {
```

WHO The SchemaMigrationPlan protocol's stages requirement, which SwiftData reads.
WHAT **Lists the migration steps.**
WHERE A static computed property.
WHEN Run in order when an upgrade is needed.
WHY One stage per pair of adjacent versions.

```
20 [.lightweight(fromVersion: NotesSchemaV1.self,
```

WHO SwiftData's MigrationStage.lightweight case, in the stages array.
WHAT **Declares an automatic, lightweight migration starting at version 1.**
WHERE The single stage in this plan.
WHEN On the first launch after the update.
WHY Lightweight stages handle renames and added defaults; a .custom stage lets you run your own code for harder changes.

```
21 toVersion: NotesSchemaV2.self)]
```

WHO SwiftData's MigrationStage.lightweight case, its toVersion: argument.
WHAT **Ends at version 2.**
WHERE Continues the stage above.
WHEN On the first launch after the update.
WHY The app then opens its container with ModelContainer(for: NotesSchemaV2.Note.self, migrationPlan: NotesMigrationPlan.self).

> **Try it**
>
> 1. Suppose version 3 adds var color: String with no default. What would you add to the plan, and why can lightweight migration not fill it in on its own?

CHECK YOURSELF

Check your understanding

1. Folder.notes uses `deleteRule: .cascade`. The user deletes a folder that contains three notes. What remains?
   A. The three notes, each with its folder set to nil
   B. Nothing: the folder and its notes are deleted
   C. Everything, because the delete is refused by SwiftData
   D. The folder alone, because its notes must be deleted first
2. Both objects are in the context, and you run `note.folder = work`. What is true about `work.notes`?
   A. It includes the note, because the inverse is maintained
   B. It does not change until the context saves
   C. It contains a copy of the note, not the original
   D. It is emptied and replaced by the single note

3. Why does NoteEditor declare `@Bindable var note: Note` rather than `@State var note: Note`?
   A. @State cannot hold a class, only structs and enums
   B. @Bindable writes changes to disk sooner than @State does
   C. The note is owned elsewhere; @Bindable only makes bindings to it
   D. They are interchangeable here; either one would compile and work the same

4. Version 2 of your app adds `var isPinned: Bool` with no default value, and existing users have data. What is the risk?
   A. None: SwiftData fills in false for every old row automatically
   B. All old notes are deleted, because the schema changed
   C. Bool properties cannot be stored, so the build fails
   D. Old rows have no value for it, so migration fails without a default

Answers: Relationships and Schema Changes

1. **B. Nothing: the folder and its notes are deleted** Cascade deletes the related objects along with the parent. Notes left behind with folder == nil is what .nullify does, SwiftData's default. A refused delete is .deny.
2. **A. It includes the note, because the inverse is maintained** The two properties describe one link, and SwiftData keeps both ends consistent in memory right away. Saving makes the change durable, but it is not what updates the inverse. Relationships hold references to the same objects, never copies.
3. **C. The note is owned elsewhere; @Bindable only makes bindings to it** @State is meant for state the view creates and owns for its lifetime; it ignores later values passed into it. The note already lives in the model context. @Bindable adds the $note.text bindings without taking ownership. Speed has nothing to do with it.
4. **D. Old rows have no value for it, so migration fails without a default** A migration must give every existing row a value for each new non-optional property. Declaring var isPinned: Bool = false, or making it optional, lets lightweight migration succeed. Otherwise the store cannot open after the update, which is a launch failure for every existing user.

Secrets in the Keychain

CONCEPT

Secrets need a vault

A **secret** is any value that lets someone act as the user: a password, an access token from signing in, a private key. The **Keychain** is an encrypted database that iOS manages outside your app's container. Items are encrypted with keys tied to the device and protected by the device passcode, and your app can read only its own items unless you deliberately share them between your own apps.

Item	One stored secret plus attributes that identify it, such as a service name and an account.
Accessibility	When the item can be read, for example only while the device is unlocked, and whether it may move to a new device.
Lifetime	Items live outside the container, so they can survive deleting and reinstalling the app. Never rely on that either way.

> **Never in UserDefaults**
>
> A token in UserDefaults or @AppStorage sits as readable text in a plist that is also copied into backups. The Keychain exists precisely to avoid that.

CODE · READ-ALONG

Describe the item you want

The Keychain API is a C-style interface from the Security framework. You describe items with dictionaries whose keys are constants such as `kSecClass`, and every call returns an `OSStatus` number, where 0 means success. Wrapping it in one small type keeps those details in a single file. Start with the dictionary that identifies your item.

```
import Foundation
import Security

enum KeychainError: Error {
    case unexpectedStatus(OSStatus)
}

struct TokenStore {
    let service = "com.example.trailhead"
    let account = "accessToken"

    var baseQuery: [String: Any] {
        [kSecClass as String: kSecClassGenericPassword,
         kSecAttrService as String: service,
         kSecAttrAccount as String: account]
    }
}
```

Line by line

```
1  import Foundation
```

WHO The Swift compiler, resolving the Foundation module while it builds TokenStore's file.
WHAT **Makes Data and other Foundation types available.**
WHERE At the top of the file.
WHEN At compile time, before TokenStore and KeychainError are type-checked; nothing is imported at run time.
WHY Tokens are stored as Data, a Foundation type.

```
2  import Security
```

WHO The Swift compiler, resolving the Security module so SecItem functions and kSec constants are known.
WHAT **Makes the Keychain functions and kSec constants available.**
WHERE At the top of the file.
WHEN At compile time, when the compiler type-checks the kSec constants used in baseQuery.
WHY The Keychain API lives in the Security framework.

```
4  enum KeychainError: Error {
```

WHO Your KeychainError enum, adopting Swift's Error protocol.
WHAT **Declares an error type for Keychain failures.**
WHERE A type in your app module.
WHEN Thrown when a Keychain call returns an unexpected status.
WHY Turning raw status numbers into a Swift error lets callers use ordinary try and catch.

```
5  case unexpectedStatus(OSStatus)
```

WHO Your KeychainError enum's only case.
WHAT **Carries the OSStatus code that the Keychain returned.**
WHERE A case with an associated value.
WHEN Created at the moment of failure.
WHY Keeping the code makes failures diagnosable, for example -25308 means the device was locked.

```
8  struct TokenStore {
```

WHO Your TokenStore struct, the one wrapper around the Keychain for this token.
WHAT **Declares the one type that talks to the Keychain for the access token.**
WHERE A value type in your app module.
WHEN Created wherever the token is needed.
WHY Everything Keychain-specific stays behind this type's simple methods.

```
9  let service = "com.example.trailhead"
```

WHO Your TokenStore struct, fixing the service name for every query.
WHAT **Names the service the item belongs to, usually your app's bundle identifier.**
WHERE A constant stored property of TokenStore, read by baseQuery.
WHEN Used in every query.
WHY Service and account together identify one item, like a folder and a file name.

```
10 let account = "accessToken"
```

WHO Your TokenStore struct, fixing the account name for every query.
WHAT **Names which secret this is within the service.**
WHERE A constant stored property of TokenStore, read by baseQuery alongside service.
WHEN Used in every query.
WHY A different account string would store a different, independent item.

```
12 var baseQuery: [String: Any] {
```

WHO Your TokenStore struct, in its baseQuery computed property.
WHAT **Builds the dictionary that identifies the item, shared by every operation.**
WHERE A computed property of TokenStore, rebuilt from service and account on every read.
WHEN Each time a method needs it.
WHY One definition guarantees that save, read, and delete all address the same item.

```
13 [kSecClass as String: kSecClassGenericPassword,
```

WHO The Security framework's constants.
WHAT **Says the item is a generic password, the class used for tokens and app secrets.**
WHERE The first entry of the dictionary.
WHEN Read by each Keychain call.
WHY The constants are Core Foundation strings, so as String converts them into Swift dictionary keys.

```
14 kSecAttrService as String: service,
```

WHO The Security framework's kSecAttrService key, in baseQuery.
WHAT **Adds the service attribute.**
WHERE The second entry of the baseQuery dictionary, after the class.
WHEN Read by each Keychain call.
WHY This narrows the search to your app's items.

```
15 kSecAttrAccount as String: account]
```

WHO The Security framework's kSecAttrAccount key, in baseQuery.
WHAT **Adds the account attribute and closes the dictionary.**
WHERE The third and last entry of the baseQuery dictionary.
WHEN Read by each Keychain call.
WHY With class, service, and account set, the query matches exactly one item.

CODE · READ-ALONG

Save, or update if it exists

`SecItemAdd` refuses to create a second item with the same class, service, and account, returning `errSecDuplicateItem`. The usual pattern is to try adding, and on that specific status update the existing item instead. Press save() twice in the simulation and watch the status codes in the log.

```swift
import Foundation
import Security

extension TokenStore {
    func save(_ token: String) throws {
        let data = Data(token.utf8)
        var item = baseQuery
        item[kSecValueData as String] = data
        item[kSecAttrAccessible as String] =
            kSecAttrAccessibleWhenUnlockedThisDeviceOnly

        var status = SecItemAdd(item as CFDictionary, nil)
        if status == errSecDuplicateItem {
            let change = [kSecValueData as String: data]
            status = SecItemUpdate(baseQuery as CFDictionary,
                                   change as CFDictionary)
        }
        guard status == errSecSuccess else {
            throw KeychainError.unexpectedStatus(status)
        }
    }
}
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, resolving Foundation so Data is available in save(_:).
- **WHAT** **Makes Data available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before save(_:) is type-checked; nothing is imported at run time.
- **WHY** The token is converted to Data before storing.

2 `import Security`

- **WHO** The Swift compiler, resolving Security so SecItemAdd and SecItemUpdate can be called.
- **WHAT** **Makes SecItemAdd and SecItemUpdate available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, when the compiler type-checks the two SecItem calls below.
- **WHY** These are Security framework functions.

4 `extension TokenStore {`

- **WHO** Your extension of TokenStore, adding save(_:).
- **WHAT** **Adds methods to the TokenStore type from the previous step.**
- **WHERE** An extension in another part of the same module.
- **WHEN** Compiled together with the type.
- **WHY** Extensions let you split one type across focused sections.

5 `func save(_ token: String) throws {`

- **WHO** Your TokenStore extension, in save(_:).
- **WHAT** **Declares a method that stores or replaces the token.**
- **WHERE** A method of TokenStore.
- **WHEN** After the user signs in, or when the server sends a new token.
- **WHY** throws reports failures to the caller instead of hiding them.

```
6  let data = Data(token.utf8)
```

WHO Foundation's Data initializer, reading the token's UTF-8 bytes.
WHAT **Converts the token text to bytes.**
WHERE A local constant in memory.
WHEN Each time save(_:) runs, before the Keychain is touched.
WHY The Keychain stores raw bytes as the item's value.

```
7  var item = baseQuery
```

WHO Your TokenStore extension, in save(_:), copying baseQuery.
WHAT **Copies the identifying dictionary so more keys can be added.**
WHERE A local variable inside save(_:), a copy of baseQuery that the next lines extend.
WHEN Each time save(_:) runs, before the value and accessibility keys are added.
WHY Dictionaries are values, so changing item leaves baseQuery untouched.

```
8  item[kSecValueData as String] = data
```

WHO The Security framework's value key.
WHAT **Adds the secret bytes to the new item.**
WHERE An entry in item.
WHEN Each time save(_:) runs, right after item is copied from baseQuery.
WHY kSecValueData is the encrypted payload; the other attributes are for finding it.

```
9  item[kSecAttrAccessible as String] =
```

WHO The Security framework's accessibility key.
WHAT **Begins choosing when the item may be read.**
WHERE An entry in item.
WHEN Each time save(_:) runs, after the secret bytes are added and before SecItemAdd.
WHY Always choose accessibility explicitly instead of relying on the default.

```
10 kSecAttrAccessibleWhenUnlockedThisDeviceOnly
```

WHO iOS data protection, which enforces this accessibility value on every later read.
WHAT **Allows reading only while the device is unlocked, and never moves the item to another device.**
WHERE The value of the accessibility entry.
WHEN Enforced on every later read.
WHY ThisDeviceOnly keeps a sign-in token out of backups restored onto other phones; use AfterFirstUnlock instead if background tasks must read it.

```
12 var status = SecItemAdd(item as CFDictionary, nil)
```

WHO The Security framework, which hands the request to the Keychain service.
WHAT **Tries to add the item and returns a status code.**
WHERE The item is stored in the system Keychain, outside your container.
WHEN When this line runs.
WHY The second argument could return the created item; nil means you do not need it.

```
13 if status == errSecDuplicateItem {
```

WHO Your TokenStore extension, in save(_:), inspecting the status from SecItemAdd.
WHAT **Checks for the specific code -25299: the item already exists.**
WHERE After the add attempt.
WHEN On every save after the first.
WHY Duplicate is expected when replacing a token, so it is handled rather than treated as failure.

```
14 let change = [kSecValueData as String: data]
```

WHO Your TokenStore extension, inside the duplicate-item branch of save(_:).
WHAT **Builds a dictionary holding only the attribute to change.**
WHERE A local constant inside the if block that handles errSecDuplicateItem.
WHEN Only when the item already exists.
WHY SecItemUpdate takes what to find and what to change as two separate dictionaries.

```
15 status = SecItemUpdate(baseQuery as CFDictionary,
```

WHO The Security framework's SecItemUpdate, talking to the Keychain service.
WHAT **Finds the existing item using baseQuery.**
WHERE Updates the item in the system Keychain.
WHEN Only after a duplicate.
WHY Reusing baseQuery guarantees the update targets the same item the add collided with.

```
16 change as CFDictionary)
```

WHO The Security framework's SecItemUpdate, its attributesToUpdate argument.
WHAT **Replaces the item's value with the new token; the status is stored in status.**
WHERE Continues the call above.
WHEN Only after a duplicate.
WHY Updating keeps the item's existing attributes, including its accessibility.

```
18 guard status == errSecSuccess else {
```

WHO Your TokenStore extension, in save(_:), checking the final status.
WHAT **Checks that the final status is 0, success.**
WHERE At the end of save.
WHEN Every time save(_:) runs, after either SecItemAdd or SecItemUpdate has returned.
WHY Any other code, such as -25308 for a locked device, must be reported.

```
19 throw KeychainError.unexpectedStatus(status)
```

WHO Your KeychainError.unexpectedStatus case, thrown from save(_:).
WHAT **Throws an error carrying the failing status code.**
WHERE Inside the guard's else branch.
WHEN Only when the final status is not errSecSuccess, for example -25308 on a locked device.
WHY The caller can log the code and ask the user to try again.

CODE · READ-ALONG

Read and delete

Reading uses `SecItemCopyMatching` with two extra keys: return the data, and return at most one match. The result comes back through an `inout` parameter of type `CFTypeRef?`. A missing item is not an error for this app; it simply means nobody has signed in, so `read()` returns `nil`.

```swift
import Foundation
import Security

extension TokenStore {
    func read() throws -> String? {
        var query = baseQuery
        query[kSecReturnData as String] = true
        query[kSecMatchLimit as String] = kSecMatchLimitOne
        var result: CFTypeRef?
        let status = SecItemCopyMatching(query as CFDictionary, &result)
        if status == errSecItemNotFound { return nil }
        guard status == errSecSuccess, let data = result as? Data else {
            throw KeychainError.unexpectedStatus(status)
        }
        return String(decoding: data, as: UTF8.self)
    }
    func delete() throws {
        let status = SecItemDelete(baseQuery as CFDictionary)
        guard status == errSecSuccess || status == errSecItemNotFound
        else { throw KeychainError.unexpectedStatus(status) }
    }
}
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, resolving Foundation so Data and String decoding are available in read().
- WHAT — **Makes Data and String decoding available.**
- WHERE — At the top of the file.
- WHEN — At compile time, before read() and delete() are type-checked; nothing is imported at run time.
- WHY — The stored bytes are turned back into a String.

2 `import Security`

- WHO — The Swift compiler, resolving Security so SecItemCopyMatching and SecItemDelete can be called.
- WHAT — **Makes SecItemCopyMatching and SecItemDelete available.**
- WHERE — At the top of the file.
- WHEN — At compile time, when the compiler type-checks the SecItem calls in read() and delete().
- WHY — Both are Security framework functions.

4 `extension TokenStore {`

- WHO — Your second extension of TokenStore, adding read() and delete().
- WHAT — **Adds reading and deleting to TokenStore.**
- WHERE — An extension of the type.
- WHEN — Compiled with the type.
- WHY — The three operations together make a complete wrapper.

5 `func read() throws -> String? {`

- WHO — Your TokenStore extension, in read().
- WHAT — **Declares a method that returns the token, or nil if none is saved.**
- WHERE — A method of TokenStore.
- WHEN — At launch, to decide whether to show the sign-in screen.
- WHY — An optional result separates no token yet from a real failure, which throws.

```
6  var query = baseQuery
```

WHO Your TokenStore extension, in read(), copying baseQuery.
WHAT **Copies the identifying dictionary.**
WHERE A local variable inside read(), a copy of baseQuery that gains two search options.
WHEN Each time read() runs, before the search options are added.
WHY Search options are added to the copy, not to baseQuery.

```
7  query[kSecReturnData as String] = true
```

WHO The Security framework's kSecReturnData key, set in read()'s query.
WHAT **Asks for the item's secret bytes to be returned.**
WHERE An entry in query.
WHEN Each time read() runs, right after query is copied from baseQuery.
WHY Without it, the call would only confirm that the item exists.

```
8  query[kSecMatchLimit as String] = kSecMatchLimitOne
```

WHO The Security framework's kSecMatchLimit key, set to kSecMatchLimitOne in read().
WHAT **Limits the result to a single item.**
WHERE An entry in query.
WHEN Each time read() runs, as the last addition to query before the search.
WHY With exactly one result, it comes back as Data rather than an array.

```
9  var result: CFTypeRef?
```

WHO Your TokenStore extension, in read(), declaring an optional for the Keychain to fill.
WHAT **Declares a slot for the Keychain to fill with the result.**
WHERE A local optional variable.
WHEN Before SecItemCopyMatching runs; it stays nil until that call writes into it.
WHY The C-style API returns its result through a pointer instead of a return value.

```
10 let status = SecItemCopyMatching(query as CFDictionary, &result)
```

WHO The Security framework and the Keychain service.
WHAT **Searches for the item and writes the data into result.**
WHERE The system Keychain, outside your container.
WHEN When this line runs.
WHY The & passes result as an inout pointer that the function can write to.

```
11 if status == errSecItemNotFound { return nil }
```

WHO Your TokenStore extension, in read(), checking the status from SecItemCopyMatching.
WHAT **Returns nil when no token exists (status -25300).**
WHERE Inside read(), right after the SecItemCopyMatching call.
WHEN Before sign-in, or after sign-out.
WHY Not found is a normal state, so it is not thrown as an error.

```
12 guard status == errSecSuccess, let data = result as? Data else {
```

WHO Your TokenStore extension, in read(), with Swift's guard unwrapping result.
WHAT **Requires success and that the result is Data.**
WHERE Inside read(), after the not-found check has been passed.
WHEN After SecItemCopyMatching returns with any status other than errSecItemNotFound.
WHY Checking both protects against unexpected statuses and result types.

```
13  throw KeychainError.unexpectedStatus(status)
```

WHO Your KeychainError.unexpectedStatus case, thrown from read().
WHAT **Throws for any other status, such as -25308 when the device is locked.**
WHERE Inside the guard's else branch.
WHEN Only when the status is neither success nor not-found, or the result is not Data.
WHY A locked device is a real, temporary failure the caller may retry later.

```
15  return String(decoding: data, as: UTF8.self)
```

WHO Swift's String(decoding:as:) initializer, reading the bytes the Keychain returned.
WHAT **Turns the bytes back into the token text.**
WHERE The return value of read(), an optional String holding the token.
WHEN After a successful read.
WHY UTF-8 matches the encoding used when saving.

```
17  func delete() throws {
```

WHO Your TokenStore extension, in delete().
WHAT **Declares a method that removes the token.**
WHERE A method of TokenStore.
WHEN When the user signs out.
WHY Signing out should always remove the secret from the device.

```
18  let status = SecItemDelete(baseQuery as CFDictionary)
```

WHO The Security framework's SecItemDelete, talking to the Keychain service.
WHAT **Deletes every item matching the base query.**
WHERE The system Keychain, outside your app's container; the status lands in a local constant.
WHEN When this line runs.
WHY The identifying dictionary alone is enough; no value is needed to delete.

```
19  guard status == errSecSuccess || status == errSecItemNotFound
```

WHO Your TokenStore extension, in delete(), checking the status from SecItemDelete.
WHAT **Accepts success, and also accepts that there was nothing to delete.**
WHERE Inside delete(), right after the SecItemDelete call.
WHEN After SecItemDelete returns, on every sign-out.
WHY Deleting something already gone should not count as an error.

```
20  else { throw KeychainError.unexpectedStatus(status) }
```

WHO Your KeychainError.unexpectedStatus case, thrown from delete().
WHAT **Throws for any other status.**
WHERE The guard's else branch.
WHEN Only on real failure.
WHY Callers learn about genuine problems, such as a locked device.

COMPARE

Never hard-code API keys

Both versions build a URL for a paid weather service. The left one embeds the service's secret key in the app; the right one talks only to your own server and authenticates with the signed-in user's token from the Keychain.

A: Risky: a secret compiled into the app

```swift
import Foundation

enum WeatherAPI {
    static let apiKey = "EXAMPLE-SECRET-1234"

    static func forecastURL(city: String) -> URL? {
        var parts = URLComponents(
            string: "https://api.example.com/forecast")
        parts?.queryItems = [
            URLQueryItem(name: "city", value: city),
            URLQueryItem(name: "key", value: apiKey),
        ]
        return parts?.url
    }
}
```

1 `import Foundation`

- **WHO** The Swift compiler, resolving Foundation for URLComponents in the risky version.
- **WHAT** **Makes URLComponents and URL available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before the risky WeatherAPI is type-checked; nothing is imported at run time.
- **WHY** Both versions use the same Foundation types.

3 `enum WeatherAPI {`

- **WHO** Your WeatherAPI enum, used only as a namespace, in the risky version.
- **WHAT** **Declares a namespace for weather requests.**
- **WHERE** A type in your app module.
- **WHEN** Compiled into the app binary.
- **WHY** Everything in this enum ships inside every copy of the app.

4 `static let apiKey = "EXAMPLE-SECRET-1234"`

- **WHO** Your code, and anyone who downloads the app.
- **WHAT** **Stores the paid service's key as a string literal.**
- **WHERE** In the compiled binary on every user's device.
- **WHEN** Present from the moment the app is installed.
- **WHY** This is the bug: string literals can be extracted from an app binary with free tools in minutes.

6 `static func forecastURL(city: String) -> URL? {`

- **WHO** Your WeatherAPI enum, in the risky forecastURL(city:).
- **WHAT** **Declares a function that builds the request URL.**
- **WHERE** A static method of WeatherAPI.
- **WHEN** Each time the app asks for a forecast.
- **WHY** The structure is fine; the problem is what goes into the URL.

7 `var parts = URLComponents(`

- **WHO** Foundation's URLComponents initializer, in the risky forecastURL(city:).
- **WHAT** **Starts building a URL from parts.**
- **WHERE** A local variable inside forecastURL(city:), holding the pieces of the URL.
- **WHEN** Each time forecastURL(city:) runs, which is every forecast request.
- **WHY** URLComponents escapes query values correctly.

```
8   string: "https://api.example.com/forecast")
```

WHO Foundation's URLComponents initializer, its string: argument naming the paid service.
WHAT **Points straight at the third-party service.**
WHERE The base address of the risky URL: the paid third-party service itself.
WHEN Each time forecastURL(city:) runs, before any query items are added.
WHY Calling the paid service directly is what forces the key onto the device.

```
9   parts?.queryItems = [
```

WHO Foundation's URLComponents, through its queryItems property.
WHAT **Starts the list of query parameters.**
WHERE On the parts variable inside forecastURL(city:); the optional chain skips it if parts is nil.
WHEN Each time forecastURL(city:) runs, after the base address is set.
WHY Each item becomes name=value in the URL.

```
10  URLQueryItem(name: "city", value: city),
```

WHO Foundation's URLQueryItem initializer, building the city parameter.
WHAT **Adds the city parameter.**
WHERE In the query list.
WHEN Each time forecastURL(city:) runs, as the first query item.
WHY Harmless, ordinary data.

```
11  URLQueryItem(name: "key", value: apiKey),
```

WHO Foundation, and anyone watching the traffic.
WHAT **Sends the secret key with every request.**
WHERE In the URL that leaves the device.
WHEN Each time forecastURL(city:) runs, so the key leaves the device with every request.
WHY Once extracted, the key lets anyone run up your bill or exhaust your quota.

```
13  return parts?.url
```

WHO Foundation's URLComponents, assembling the risky URL through its url property.
WHAT **Returns the finished URL.**
WHERE The result of forecastURL(city:), an optional URL that now contains the key.
WHEN Each time forecastURL(city:) runs, after both query items are in place.
WHY The URL is correct, which is why this mistake survives testing.

B: Safer: your server holds the key

```swift
import Foundation

enum WeatherAPI {
    static func forecastURL(city: String) -> URL? {
        var parts = URLComponents(
            string: "https://api.example.com/forecast")
        parts?.queryItems = [URLQueryItem(name: "city", value: city)]
        return parts?.url
    }

    static func authorize(_ request: inout URLRequest,
                          store: TokenStore) throws {
        guard let token = try store.read() else { return }
        request.setValue("Bearer \(token)",
                         forHTTPHeaderField: "Authorization")
    }
}
```

1 `import Foundation`

- **WHO** The Swift compiler, resolving Foundation for URLComponents and URLRequest in the safer version.
- **WHAT** **Makes URLComponents and URLRequest available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; the same import as the risky version.
- **WHY** Same imports, different design.

3 `enum WeatherAPI {`

- **WHO** Your WeatherAPI enum, in the safer version that ships no secret.
- **WHAT** **Declares the same namespace, now without any secret.**
- **WHERE** A type in your app module.
- **WHEN** Compiled into the app.
- **WHY** Nothing in this binary is worth stealing.

4 `static func forecastURL(city: String) -> URL? {`

- **WHO** Your WeatherAPI enum, in the safer forecastURL(city:).
- **WHAT** **Builds a URL for your own backend.**
- **WHERE** A static method of WeatherAPI, called without creating an instance.
- **WHEN** Each forecast request, right before the request is authorized and sent.
- **WHY** Your server calls the weather service with the key it keeps privately.

5 `var parts = URLComponents(`

- **WHO** Foundation's URLComponents initializer, in the safer forecastURL(city:).
- **WHAT** **Starts building the URL.**
- **WHERE** A local variable inside the safer forecastURL(city:), holding the URL pieces.
- **WHEN** Each time the safer forecastURL(city:) runs.
- **WHY** The same safe URL building as before.

```
6   string: "https://api.example.com/forecast")
```

WHO Foundation's URLComponents initializer, its string: argument naming your own server.
WHAT **Points at your own server's forecast endpoint.**
WHERE The base address of the safer URL: a server you control, not the paid service.
WHEN Each time the safer forecastURL(city:) runs, before the city is added.
WHY Here api.example.com stands for a server you control, which forwards the request.

```
7   parts?.queryItems = [URLQueryItem(name: "city", value: city)]
```

WHO Foundation's URLComponents and URLQueryItem, building a single city parameter.
WHAT **Adds only the city parameter.**
WHERE In the query part of the URL, as its only name=value pair.
WHEN Each time the safer forecastURL(city:) runs, after the base address is set.
WHY No key travels in the URL any more.

```
8   return parts?.url
```

WHO Foundation's URLComponents, assembling the safer URL through its url property.
WHAT **Returns the URL.**
WHERE The result of the safer forecastURL(city:), an optional URL with no secret in it.
WHEN Each time the safer forecastURL(city:) runs, after the city item is added.
WHY Authentication is added separately, below.

```
11  static func authorize(_ request: inout URLRequest,
```

WHO Your WeatherAPI enum, in authorize(_:store:).
WHAT **Declares a function that adds the user's credentials to a request.**
WHERE A static method; inout lets it modify the caller's request.
WHEN Before each request is sent.
WHY Keeping authorization in one place makes it easy to audit.

```
12  store: TokenStore) throws {
```

WHO Your WeatherAPI enum, declaring the second parameter of authorize(_:store:).
WHAT **Receives the TokenStore that reads from the Keychain.**
WHERE A parameter of authorize.
WHEN Each time authorize(_:store:) runs, once per outgoing request.
WHY Passing the store in makes the function easy to test with a fake.

```
13  guard let token = try store.read() else { return }
```

WHO Your TokenStore wrapper, in read(), called from authorize(_:store:).
WHAT **Reads the signed-in user's token, or returns if there is none.**
WHERE The system Keychain, read through store.read(); the token lands in a local constant.
WHEN Before each request is sent, as the first statement of authorize(_:store:).
WHY The token belongs to one user, expires, and can be revoked by your server.

```
14  request.setValue("Bearer \(token)",
```

WHO Foundation's URLRequest, in setValue(_:forHTTPHeaderField:), writing into the caller's request through inout.
WHAT **Sets the Authorization header value, using the standard Bearer scheme.**
WHERE In the outgoing request's headers.
WHEN Before the request is sent, once a token was found in the Keychain.
WHY Headers are not logged in URLs, and the server checks the token on every request.

```
15  forHTTPHeaderField: "Authorization")
```

WHO	Foundation's URLRequest, the forHTTPHeaderField: argument of setValue.
WHAT	**Names the header being set.**
WHERE	Continues the call above.
WHEN	Before the request is sent, together with the Bearer value on the line above.
WHY	Authorization is the standard header servers expect for tokens.

> **Takeaway**
>
> Anything compiled into an app ships to every user, and strings can be pulled out of a binary in minutes, so obfuscation only delays an attacker. Keep third-party keys on a server you control and let it call the service for you. The app carries only the signed-in user's own short-lived token, kept in the Keychain and sent in the Authorization header.

CHECK YOURSELF

Check your understanding

1. `SecItemAdd` returns `errSecDuplicateItem` (-25299). What does that mean, and what should the wrapper do?
   A. The Keychain is full; delete every item and retry
   B. The item already exists; update it with SecItemUpdate
   C. The device is locked; wait and retry the add later
   D. The token is malformed; ask the user to sign in once more
2. Where should an app keep the refresh token it receives at sign-in?
   A. In @AppStorage, under a private key name
   B. In a JSON file in Documents, marked isExcludedFromBackup
   C. In the Keychain, marked ThisDeviceOnly
   D. In a private constant inside the app
3. A teammate puts a paid API key in a Swift constant, arguing that the compiled app hides it. What is the best response?
   A. Agree, because compiled code cannot be read by anyone
   B. Agree, as long as the constant is marked private
   C. Agree, as long as the key is Base64-encoded first
   D. Disagree: strings can be pulled out of a binary
4. Using the wrapper from this lesson, `read()` returns `nil`. What does that tell you?
   A. No token is stored, so show the sign-in screen
   B. The device is locked, so retry after unlock
   C. The stored token is corrupted, so delete and re-save it
   D. The Keychain is turned off in Settings for this app

Answers: Secrets in the Keychain

1. **B. The item already exists; update it with SecItemUpdate** The Keychain refuses to create a second item with the same identifying attributes. Replacing a token is normal, so the wrapper updates the existing item. A locked device returns a different code, errSecInteractionNotAllowed (-25308).
2. **C. In the Keychain, marked ThisDeviceOnly** A refresh token is a long-lived secret, so it belongs in the Keychain, and ThisDeviceOnly keeps it from being restored onto another device. @AppStorage writes a readable plist that is copied into backups, a Documents file stays readable on the device even when it is excluded from backups, and a constant would be the same secret for every user.
3. **D. Disagree: strings can be pulled out of a binary** private only limits which Swift code can see the constant; the bytes are still in the binary. Base64 is an encoding, not encryption, and is reversed instantly. Only keeping the key off the device protects it.
4. **A. No token is stored, so show the sign-in screen** The wrapper returns nil only for errSecItemNotFound. A locked device produces errSecInteractionNotAllowed, which the wrapper throws as KeychainError.unexpectedStatus(-25308), so it never looks like a missing token.

iCloud and CloudKit Sync

CONCEPT

Data that follows the user

Local storage lives on one device. Cloud storage keeps a copy on servers, so the same data appears on the user's iPhone, iPad, and Mac, and survives a lost phone. iCloud offers several ways in, from tiny to large:

Key-value store	NSUbiquitousKeyValueStore syncs a few settings, up to 1 MB in total, like UserDefaults across devices.
iCloud Documents	Files in an iCloud folder, for document-based apps.
CloudKit	Databases of records on Apple's servers. A user's private data counts against their own iCloud storage, not your account.
SwiftData sync	Your SwiftData models mirrored to CloudKit automatically, with very little code.

> **What you need**
>
> iCloud features require a paid Apple Developer Program membership, the iCloud capability in Xcode with a CloudKit container, and a user signed in to iCloud. Your app must keep working when they are not signed in.

CONCEPT

CloudKit: containers, databases, records

A CloudKit **container** is your app's space on Apple's servers, with an identifier such as `iCloud.com.example.tasks`. Each container holds three databases:

Public	Shared by everyone who uses the app, such as a catalog of trails. Anyone can read it; writing requires an iCloud account.
Private	Visible only to the signed-in user: their notes, their tasks. Even you, the developer, cannot read it.
Shared	Records that other users chose to share with this user, such as a family shopping list, through a CKShare.
CKRecord	One stored item: a record type such as "Task", a unique record ID, and fields holding strings, numbers, dates, files, and references to other records.

> **Local first**
>
> Cloud data is a copy, not the only copy. Apps keep a local store and sync it, so they launch instantly and work offline.

CODE · READ-ALONG

Save and query records

Working with CloudKit directly means creating `CKRecord` values and sending them to a database with async calls. This small type saves a task to the user's private database and fetches the titles of open tasks. Every call goes over the network, so each one is `async` and can throw.

```
import CloudKit

struct TaskCloud {
    let database = CKContainer(identifier: "iCloud.com.example.tasks")
        .privateCloudDatabase

    func add(title: String) async throws -> CKRecord.ID {
        let record = CKRecord(recordType: "Task")
        record["title"] = title
        record["isDone"] = false
        let saved = try await database.save(record)
        return saved.recordID
    }

    func openTitles() async throws -> [String] {
        let predicate = NSPredicate(format: "isDone == 0")
        let query = CKQuery(recordType: "Task", predicate: predicate)
        let (results, _) = try await database.records(matching: query)
        return results.compactMap { _, result in
            try? result.get()["title"] as? String
        }
    }
}
```

Line by line

1 `import CloudKit`

- **WHO** The Swift compiler, resolving the CloudKit module while it builds this TaskCloud file.
- **WHAT** **Makes CKContainer, CKRecord, CKQuery, and NSPredicate available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; CKContainer and CKRecord must be known before TaskCloud is declared.
- **WHY** CloudKit re-exports Foundation, so one import covers both.

3 `struct TaskCloud {`

- **WHO** Your TaskCloud struct, the one type in the app that talks to CloudKit.
- **WHAT** **Declares a small type that talks to CloudKit for tasks.**
- **WHERE** A value type in your app module.
- **WHEN** Created wherever the app needs cloud access.
- **WHY** Isolating CloudKit code behind one type keeps views simple and testable.

4 `let database = CKContainer(identifier: "iCloud.com.example.tasks")`

- **WHO** CloudKit's CKContainer initializer, looking up the container named iCloud.com.example.tasks.
- **WHAT** **Opens your app's container by its identifier.**
- **WHERE** A stored property; the container itself is on Apple's servers.
- **WHEN** When a TaskCloud is created.
- **WHY** The identifier must match the container enabled in the app's iCloud capability.

```
5   .privateCloudDatabase
```

WHO	CloudKit's CKContainer, handing back its privateCloudDatabase property for the signed-in user.
WHAT	**Chooses the signed-in user's private database.**
WHERE	Continues the property above.
WHEN	When a TaskCloud is created.
WHY	Personal tasks belong in the private database, where only their owner can see them.

```
7   func add(title: String) async throws -> CKRecord.ID {
```

WHO	Your TaskCloud struct, declaring the add(title:) method that a view calls from inside a Task.
WHAT	**Declares an async method that saves a task and returns its record ID.**
WHERE	A method of TaskCloud.
WHEN	When the user adds a task.
WHY	async lets the app keep responding while the network request is in flight.

```
8   let record = CKRecord(recordType: "Task")
```

WHO	CloudKit's CKRecord initializer, creating a record of type Task on the device.
WHAT	**Creates a new record of type Task with a fresh unique ID.**
WHERE	In memory, not yet on the server.
WHEN	At the start of every add(title:) call, before anything is sent to Apple's servers.
WHY	Record types are created in the development environment the first time you save one.

```
9   record["title"] = title
```

WHO	CKRecord's subscript setter, storing the title string under the title key.
WHAT	**Sets the title field.**
WHERE	On the in-memory record.
WHEN	Right after the record is created, before add(title:) reaches the save call.
WHY	Fields are set by name, like a dictionary of supported value types.

```
10  record["isDone"] = false
```

WHO	CKRecord's subscript setter, storing false under the isDone key as the number 0.
WHAT	**Sets the isDone field; CloudKit stores a Bool as a number, 0 or 1.**
WHERE	On the in-memory record.
WHEN	Immediately after the title is set, still in memory inside add(title:).
WHY	Knowing it is stored as a number explains the query below.

```
11  let saved = try await database.save(record)
```

WHO	CloudKit, over the network.
WHAT	**Uploads the record and waits for the server to confirm.**
WHERE	On Apple's servers, in the user's private database.
WHEN	Suspends here until the server replies.
WHY	try await marks a call that both waits and can fail, for example when the user is offline.

```
12  return saved.recordID
```

WHO	Your add(title:) method, returning the recordID of the record the server confirmed.
WHAT	**Returns the server-confirmed record ID.**
WHERE	The last line of add(title:), after the await on database.save(record) has resumed.
WHEN	After the save succeeds.
WHY	Keep the ID to update or delete this record later.

```
15 func openTitles() async throws -> [String] {
```

WHO Your TaskCloud struct, declaring openTitles(), the method that fetches unfinished tasks.
WHAT **Declares an async method that fetches the titles of unfinished tasks.**
WHERE A method of TaskCloud.
WHEN When the app refreshes its list.
WHY Returning plain titles keeps CloudKit types out of the interface.

```
16 let predicate = NSPredicate(format: "isDone == 0")
```

WHO Foundation's NSPredicate initializer, building the filter that CloudKit will run on the server.
WHAT **Describes the condition: isDone equals 0, meaning false.**
WHERE A local constant inside openTitles(), the first of the two values the query needs.
WHEN At the start of every openTitles() call, before the query is built or sent.
WHY CloudKit queries use NSPredicate, and the field must be marked queryable in the CloudKit schema.

```
17 let query = CKQuery(recordType: "Task", predicate: predicate)
```

WHO CloudKit's CKQuery initializer, pairing the Task record type with the isDone predicate.
WHAT **Combines the record type and the condition into a query.**
WHERE A local constant inside openTitles(), built on the device right before the network call.
WHEN Each time openTitles() runs, right after the predicate exists and before records(matching:) is awaited.
WHY The query is evaluated on the server, so only matching records are downloaded.

```
18 let (results, _) = try await database.records(matching: query)
```

WHO CloudKit, over the network.
WHAT **Runs the query and returns a list of results plus a cursor for more pages.**
WHERE The work happens on Apple's servers.
WHEN Suspends until the server replies.
WHY The underscore ignores the cursor; a large result set would need it to fetch the next page.

```
19 return results.compactMap { _, result in
```

WHO Swift's compactMap on the results array that CloudKit's records(matching:) returned.
WHAT **Transforms each result into a title, dropping any that fail.**
WHERE The method's return value.
WHEN After the query returns.
WHY Each record has its own Result, because single records can fail even when the query succeeds.

```
20 try? result.get()["title"] as? String
```

WHO Swift's Result type and casting.
WHAT **Gets the record if it loaded, then reads its title as a String.**
WHERE Inside the compactMap closure in openTitles(), where result is one record's Result value.
WHEN Once per matched record, after the query reply has arrived from Apple's servers.
WHY try? and as? both produce nil on failure, and compactMap drops the nils.

> **Try it**
>
> 1. Which database would you use instead for a list of hiking trails that every user of the app should see?

COMPARE

Making a model sync-ready

SwiftData can mirror a model to CloudKit only if the model follows CloudKit's rules. Both versions compile; the left one fails when the app tries to open a synced store. Compare them line by line.

A: Cannot sync

```swift
import Foundation
import SwiftData

@Model
final class Trip {
    @Attribute(.unique) var code: String
    var name: String
    var startDate: Date
    @Relationship(deleteRule: .deny)
    var stops: [Stop]

    init(code: String, name: String) {
        self.code = code
        self.name = name
        self.startDate = .now
        self.stops = []
    }
}
```

1 `import Foundation`

WHO The Swift compiler, resolving the Foundation module so this local-only Trip model can use Date.
WHAT **Makes Date available.**
WHERE At the top of the file.
WHEN At compile time, before any line of this model file runs; the Date type must be known for startDate.
WHY Both versions start the same way.

2 `import SwiftData`

WHO The Swift compiler, resolving the SwiftData module that supplies @Model, @Attribute and @Relationship for this file.
WHAT **Makes @Model, @Attribute, and @Relationship available.**
WHERE At the top of the file.
WHEN At compile time, before any line of this file runs; the CloudKit rules it breaks are checked only later, when the container opens.
WHY Sync rules are checked when the container opens, not by the compiler.

4 `@Model`

WHO The SwiftData @Model macro, expanded by the compiler on the Trip class that cannot sync.
WHAT **Makes Trip a stored model.**
WHERE On the class below.
WHEN At compile time, when the macro generates the persistence code for this Trip; the sync rules are not checked here.
WHY A fine model for a local-only store.

5 `final class Trip {`

WHO Your Trip class, the version that follows local-only habits and breaks four CloudKit rules.
WHAT **Declares a trip model.**
WHERE A class in your app module.
WHEN Created when the user plans a trip.
WHY Every problem below is legal Swift, which is why it slips through.

```
6  @Attribute(.unique) var code: String
```

WHO The SwiftData @Attribute macro.
WHAT **Requires every trip to have a different code.**
WHERE A property of Trip.
WHEN Enforced on every save.
WHY Not allowed with CloudKit: two offline devices could each create the same code, and the server cannot enforce uniqueness.

```
7  var name: String
```

WHO Your Trip model, declaring name as a required String with no default.
WHAT **A required name with no default value.**
WHERE A property of Trip.
WHEN Assigned in init(code:name:) and only there; a record arriving from CloudKit without this field has nothing to fill it.
WHY Not allowed: a record can arrive from CloudKit without this field, so every property needs a default or must be optional.

```
8  var startDate: Date
```

WHO Your Trip model, declaring startDate as a required Date with no default.
WHAT **A required date with no default.**
WHERE A property of Trip.
WHEN Assigned to .now in init(code:name:), so a synced record missing the field cannot be loaded.
WHY Same problem: no default value.

```
9  @Relationship(deleteRule: .deny)
```

WHO The SwiftData @Relationship attribute.
WHAT **Refuses to delete a trip that still has stops.**
WHERE On the stops property.
WHEN Checked by SwiftData each time the context saves a deleted Trip; with CloudKit sync the container refuses to open at all.
WHY The .deny rule is not supported with CloudKit, and no inverse is named.

```
10 var stops: [Stop]
```

WHO Your Trip model, declaring the to-many relationship to Stop as a non-optional array.
WHAT **A non-optional to-many relationship.**
WHERE A property of Trip.
WHEN Changes as stops are added.
WHY CloudKit requires relationships to be optional, because related records can arrive at different times.

```
12 init(code: String, name: String) {
```

WHO Your Trip class, declaring the initializer that must fill every property by hand.
WHAT **Creates a trip.**
WHERE The initializer of the local-only Trip class, inside the class body after the stops property.
WHEN When the user plans a trip.
WHY It has to set every property, because none has a default.

```
13 self.code = code
```

WHO Trip's init(code:name:), written by you, storing the code parameter.
WHAT **Stores the code.**
WHERE In the new object.
WHEN During initialization, as the first assignment, because code has no default to fall back on.
WHY Required because code has no default.

```
14 self.name = name
```

WHO Trip's init(code:name:), storing the name parameter into the required name property.
WHAT **Stores the name.**
WHERE In the new object.
WHEN During initialization, right after code is set and before the object can be inserted into a context.
WHY Required because name has no default.

```
15 self.startDate = .now
```

WHO Trip's init(code:name:), capturing the current moment from Date.now.
WHAT **Sets the start date to now.**
WHERE In the new object.
WHEN During initialization, at the instant the trip is created, because startDate declares no default of its own.
WHY Required because startDate has no default.

```
16 self.stops = []
```

WHO Trip's init(code:name:), giving the non-optional stops array its starting value.
WHAT **Starts with no stops.**
WHERE In the new object.
WHEN During initialization, as the final assignment; without it the compiler would reject the initializer.
WHY Required because stops has no default.

B: Sync-ready

```swift
import Foundation
import SwiftData

@Model
final class Trip {
    var code: String = ""
    var name: String = ""
    var startDate: Date = Date.now
    @Relationship(deleteRule: .cascade, inverse: \Stop.trip)
    var stops: [Stop]? = []

    init(code: String, name: String) {
        self.code = code
        self.name = name
    }
}
```

```
1 import Foundation
```

WHO The Swift compiler, resolving the Foundation module so the sync-ready Trip can use Date and Date.now.
WHAT **Makes Date available.**
WHERE At the top of the file.
WHEN At compile time, before any line of this file runs; the Date.now default needs the module in scope.
WHY The same imports as before.

```
2  import SwiftData
```

WHO The Swift compiler, resolving the SwiftData module whose @Model and @Relationship macros this sync-ready file uses.
WHAT **Makes the SwiftData macros available.**
WHERE At the top of the file.
WHEN At compile time, before any line of this file runs; no CloudKit import is needed because SwiftData handles the mirroring.
WHY Nothing CloudKit-specific is imported; SwiftData handles the sync.

```
4  @Model
```

WHO The SwiftData @Model macro, expanded by the compiler on the sync-ready Trip class.
WHAT **Makes Trip a stored model.**
WHERE On the class below.
WHEN At compile time, generating the same persistence code as before; the defaults and optionals are what make the result sync-safe.
WHY The same macro; only the property declarations change.

```
5  final class Trip {
```

WHO Your Trip class, redesigned so every property has a default and every relationship is optional.
WHAT **Declares the sync-ready trip model.**
WHERE A class in your app module.
WHEN Created locally or received from another device.
WHY A model designed for sync from the start avoids a migration later.

```
6      var code: String = ""
```

WHO Your Trip model, declaring code as a plain String with an empty-string default and no @Attribute(.unique).
WHAT **A code with a default, and no uniqueness constraint.**
WHERE A property of Trip.
WHEN Has a value even if a record arrives without one.
WHY If codes must not repeat, detect duplicates in your own code after sync instead.

```
7      var name: String = ""
```

WHO Your Trip model, declaring name with an empty-string default that CloudKit can rely on.
WHAT **A name with a default value.**
WHERE A property of Trip.
WHEN Always has a value.
WHY Defaults let partially synced records load safely.

```
8      var startDate: Date = Date.now
```

WHO Your Trip model, giving startDate a Date.now default that is evaluated when the object is created.
WHAT **A start date that defaults to now.**
WHERE A property of Trip.
WHEN Always has a value.
WHY Every attribute now has a default.

```
9      @Relationship(deleteRule: .cascade, inverse: \Stop.trip)
```

WHO The SwiftData @Relationship attribute.
WHAT **Uses a supported delete rule and names the inverse.**
WHERE On the stops property.
WHEN Applied on delete and when linking.
WHY CloudKit sync requires relationships to have inverses.

```
10 var stops: [Stop]? = []
```

WHO Your Trip model, declaring stops as an optional array of Stop, the shape CloudKit sync requires.
WHAT **An optional to-many relationship with an empty default.**
WHERE A property of Trip.
WHEN May briefly be nil while related records are still downloading.
WHY Code reads it with trip.stops ?? [] to treat missing as empty.

```
12 init(code: String, name: String) {
```

WHO Your sync-ready Trip class, declaring an initializer that only needs the two values with no useful default.
WHAT **Creates a trip.**
WHERE The initializer of the sync-ready Trip class, after the stops property inside the class body.
WHEN When the user plans a trip.
WHY Defaults cover the other properties, so init is shorter.

```
13 self.code = code
```

WHO Trip's init(code:name:), replacing the empty-string default with the real code.
WHAT **Stores the code.**
WHERE In the new object.
WHEN During initialization, after the default values are already in place; this assignment overrides one of them.
WHY Overrides the default with the real value.

```
14 self.name = name
```

WHO Trip's init(code:name:), replacing the empty-string default with the name the user typed.
WHAT **Stores the name.**
WHERE In the new object.
WHEN During initialization, as the last assignment; startDate and stops keep the defaults declared on their properties.
WHY Overrides the default with the real value.

> **Takeaway**
>
> CloudKit can deliver records in any order, and from devices running older versions of your app. So with sync, every property needs a default value or must be optional, every relationship must be optional and have an inverse, unique constraints are not allowed, and the .deny delete rule is unsupported. Design models this way from the start if sync might come later.

CODE · READ-ALONG

Turn on SwiftData sync

With the iCloud capability (CloudKit plus a container) and the Remote notifications background mode enabled in Xcode, a SwiftData store syncs automatically, using the first CloudKit container in the app's entitlements. Naming the container, as here, makes the choice explicit. Sync runs in the background, and @Query results update when changes arrive from other devices.

```swift
import SwiftUI
import SwiftData

@main
struct TripsApp: App {
    let container: ModelContainer

    init() {
        let config = ModelConfiguration(
            cloudKitDatabase: .private("iCloud.com.example.trips")
        )
        do {
            container = try ModelContainer(for: Trip.self,
                                           configurations: config)
        } catch {
            fatalError("Could not open the store: \(error)")
        }
    }

    var body: some Scene {
        WindowGroup { TripListView() }
            .modelContainer(container)
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so this file can declare TripsApp with App, Scene and WindowGroup.
- **WHAT** **Makes App and WindowGroup available.**
- **WHERE** At the top of the app's main file.
- **WHEN** At compile time, before any line of the app's main file runs; nothing is imported at run time.
- **WHY** The app structure is SwiftUI.

2 `import SwiftData`

- **WHO** The Swift compiler, resolving the SwiftData module that supplies ModelContainer, ModelConfiguration and the modelContainer modifier.
- **WHAT** **Makes ModelContainer and ModelConfiguration available.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before TripsApp.init() can run; the container types must be known before launch.
- **WHY** Sync is configured through SwiftData, not CloudKit calls.

4 `@main`

- **WHO** The Swift compiler and iOS.
- **WHAT** **Marks the app's entry point.**
- **WHERE** On the App struct below.
- **WHEN** When iOS launches the app: the runtime finds the @main type, creates a TripsApp value, and runs its init() before any scene appears.
- **WHY** Exactly one type in the app has @main.

```
5  struct TripsApp: App {
```

WHO Your TripsApp struct, the one type that conforms to App and owns the synced container.
WHAT **Declares the app.**
WHERE At file scope in the app's main file, as the root type that iOS instantiates at launch.
WHEN Created once at launch.
WHY The container is created here, before any view needs it.

```
6  let container: ModelContainer
```

WHO Your TripsApp struct, declaring a stored constant that init() must assign exactly once.
WHAT **Holds the one container the whole app shares.**
WHERE A stored property of the App.
WHEN Set once in init.
WHY Creating the container yourself lets you pass a configuration and handle errors.

```
8  init() {
```

WHO Your TripsApp struct, declaring the init() that iOS runs right after creating the app value.
WHAT **Sets up the container at launch.**
WHERE TripsApp's initializer, inside the struct body between the container property and body.
WHEN Once, when iOS starts the app.
WHY Doing it in init guarantees the store exists before the first view appears.

```
9  let config = ModelConfiguration(
```

WHO SwiftData's ModelConfiguration initializer, called by your TripsApp.init().
WHAT **Starts describing how the store behaves.**
WHERE A local constant inside TripsApp.init(), used only to build the container a few lines later.
WHEN Once per launch, as the first statement of init(), before the container exists or any view is created.
WHY A configuration controls the store's location, memory-only mode, and sync.

```
10 cloudKitDatabase: .private("iCloud.com.example.trips")
```

WHO SwiftData's CloudKit integration, told through the cloudKitDatabase argument to mirror this store into the private database of iCloud.com.example.trips.
WHAT **Mirrors the store to the private database of this container.**
WHERE An argument of the configuration; data lives locally and in the user's iCloud.
WHEN Sync runs in the background whenever the device is online.
WHY Use .automatic to pick the first container, or .none to turn sync off, for example in previews and tests.

```
12 do {
```

WHO Swift's do-catch error handling, wrapping the throwing ModelContainer initializer.
WHAT **Starts the block that may fail to open the store.**
WHERE Inside TripsApp.init(), directly after the config constant and before body is ever read.
WHEN Once per launch, when init() reaches the container creation that might throw.
WHY Opening a store can fail, for example when a model breaks a CloudKit rule.

```
13 container = try ModelContainer(for: Trip.self,
```

WHO SwiftData's ModelContainer initializer, opening the store for Trip and, through the relationship, Stop.
WHAT **Creates the container for Trip; Stop is included through the relationship.**
WHERE Assigns the stored property.
WHEN Once per launch, inside the do block; this is the moment a model that breaks a CloudKit rule makes the initializer throw.
WHY This is where a non-sync-ready model would throw.

```
14  configurations: config)
```

WHO — SwiftData's ModelContainer initializer, reading the config built above as its configurations argument.
WHAT — **Applies the sync configuration.**
WHERE — Continues the call above.
WHEN — In the same call as the line above, during launch, before the container is assigned.
WHY — Without it, the store would be local only.

```
15  } catch {
```

WHO — Swift's do-catch error handling, receiving the error that ModelContainer(for:configurations:) threw.
WHAT — **Handles a store that cannot be opened.**
WHERE — The catch block inside TripsApp.init(), the only other path that can leave init().
WHEN — Only if creation failed.
WHY — Every code path must assign container or stop the app.

```
16  fatalError("Could not open the store: \(error)")
```

WHO — The Swift runtime, through the fatalError function, stopping the app with the error text in the console.
WHAT — **Stops the app with a message that includes the error.**
WHERE — Inside the catch block.
WHEN — Only when the store cannot open.
WHY — During development, a loud crash reveals a broken model immediately; a shipping app might fall back to a local-only store.

```
20  var body: some Scene {
```

WHO — SwiftUI, reading the body property of TripsApp to learn which scenes to show.
WHAT — **Describes the app's window.**
WHERE — A computed property of TripsApp, below init(), returning the app's single WindowGroup.
WHEN — After init() has assigned container, when SwiftUI builds the app's scenes at launch and whenever it re-evaluates them.
WHY — The scene receives the container from init.

```
21  WindowGroup { TripListView() }
```

WHO — SwiftUI's WindowGroup scene, creating the main window with TripListView as its root view.
WHAT — **Shows the trip list in the main window.**
WHERE — Inside TripsApp's body, as the one scene the app declares.
WHEN — When the window is created at launch, after the container from init() is already available.
WHY — TripListView uses @Query exactly as in a local-only app.

```
22  .modelContainer(container)
```

WHO — SwiftData's modelContainer(_:) scene modifier, receiving the container that init() built.
WHAT — **Shares the synced container with every view in the window.**
WHERE — A modifier on the WindowGroup.
WHEN — When the WindowGroup scene is created at launch, before TripListView's @Query runs its first fetch.
WHY — Views do not know or care that the data syncs.

INTERACTIVE

Two devices, one list

An iPhone and an iPad share one grocery list through iCloud. Edit on either device and watch the change travel up to iCloud and down to the other device. Turn on airplane mode, make edits, and watch them queue. Then press **Run the conflict scenario**: both devices edit the same item offline. Compare what each conflict policy keeps and what it loses.

This step is an interactive simulation in the HTML version.

CONCEPT

Conflicts and offline edits

Sync is never instant. A phone in a tunnel keeps working offline and uploads later, so two devices can change the same record before either sees the other's change. That is a **conflict**, and something must decide which version survives.

Last writer wins	A policy for sync layers: the version that reaches the server last replaces the other. Simple, but one edit silently disappears, and arrival order, not typing order, decides.
Field-level merge	Keep both changes when they touched different fields, such as a title on one device and a checkmark on the other.
Keep both	Save the late version as a copy, or show both and let the user choose.
In CloudKit code	With the default save policy, saving an outdated record directly fails with CKError.serverRecordChanged, which carries the server's version so you can merge and save again. CKSyncEngine helps manage this.

> **Automatic sync decides for you**
>
> SwiftData's automatic sync resolves conflicts without asking you or the user. If losing a concurrent edit is unacceptable, use CloudKit directly so you control the merge.

CHECK YOURSELF

Check your understanding

1. Which CloudKit database should hold a user's private journal entries?
   A. The public database
   B. The private database
   C. The shared database
   D. The iCloud key-value store

2. A model has `@Attribute(.unique) var email: String`. You turn on SwiftData CloudKit sync. What happens?
   A. Nothing; uniqueness is enforced on the server
   B. Only the first device to sync keeps its values
   C. Opening the synced container fails
   D. The email field is encrypted automatically

3. Both devices are offline. The iPhone renames Milk to "Oat milk" at 9:01, and the iPad renames it to "Whole milk" at 9:02. The iPad reconnects first, then the iPhone. With last writer wins, what does the list say at the end?
   A. Whole milk, because it was typed last
   B. Milk, because the two conflicting edits cancel out
   C. Both names, shown side by side as two items
   D. Oat milk, because the iPhone uploaded last

4. Why do synced apps still keep a full local store?
    A. So the app launches instantly and works offline, syncing later
    B. Because CloudKit cannot store strings longer than 255 bytes
    C. Because iCloud deletes unread records every night
    D. Because the public database requires one

Answers: iCloud and CloudKit Sync

1. **B. The private database** Personal data that only its owner should see goes in the private database, which counts against the user's own iCloud storage. The public database is readable by every user of the app, and the key-value store is limited to 1 MB of small settings.
2. **C. Opening the synced container fails** CloudKit cannot guarantee uniqueness across devices that create records offline, so SwiftData rejects unique constraints when sync is enabled and the container fails to load. Remove the constraint and handle duplicates in your own code after sync.
3. **D. Oat milk, because the iPhone uploaded last** Last writer wins means the last write to reach the server, not the last one typed. The iPhone uploaded after the iPad, so Oat milk overwrote Whole milk, and the iPad user's newer edit was lost without warning. That surprise is why some apps merge fields or keep both versions.
4. **A. So the app launches instantly and works offline, syncing later** Local-first design means the interface never waits on the network: reads and writes hit the local store, and sync catches up in the background. The other statements are false.

Worked Examples

WORKED EXAMPLE

Decode a weather reading, field by field

Problem.
A weather service sends one reading as JSON text. Decode it into a `Codable` struct, growing the struct field by field, survive a reading whose temperature arrives as text, and print one summary line per reading.

Frame 1 of 7

The smallest useful reading: a struct with one String field decodes a one-key JSON object, and the print proves the city arrived. Every later frame grows this same program.

```
1   import Foundation
2
3   struct WeatherReading: Codable {
4       var city: String
5   }
6   let json = "{\"city\": \"Venice\"}"
7   let reading = try JSONDecoder().decode(WeatherReading.self, from: Data(json.utf8))
8   print("City: \(reading.city)")
```

OUTPUT SO FAR

```
City: Venice
```

Frame 2 of 7

A Double field joins the struct and the JSON gains a matching number. The decoder fills both fields in one call, and two prints show each value landed in its own property.

```
1   import Foundation
2
3   struct WeatherReading: Codable {
4       var city: String
5   +   var temperature: Double
6   + }
7   + let json = "{\"city\": \"Venice\", \"temperature\": 28.5}"
8   + let reading = try JSONDecoder().decode(WeatherReading.self, from: Data(json.utf8))
9   + print("City: \(reading.city)")
10  + print("Temperature: \(reading.temperature)")
```

OUTPUT SO FAR

```
City: Venice
Temperature: 28.5
```

Frame 3 of 7

A third String field, condition, completes the reading the service sends. The struct now matches the JSON key for key, so one decode call fills all three properties at once.

```
1    import Foundation
2
3    struct WeatherReading: Codable {
4        var city: String
5        var temperature: Double
6  +     var condition: String
7  + }
8  + let json = "{\"city\": \"Venice\", \"temperature\": 28.5, \"condition\": \"Sunny\"}"
9  + let reading = try JSONDecoder().decode(WeatherReading.self, from: Data(json.utf8))
10 + print("City: \(reading.city)")
11 + print("Temperature: \(reading.temperature)")
12 + print("Condition: \(reading.condition)")
```

OUTPUT SO FAR

```
City: Venice
Temperature: 28.5
Condition: Sunny
```

Frame 4 of 7

The three separate prints become one summary line built by interpolation. This exact line is the goal output; the next frame keeps the code and breaks only the incoming JSON.

```
1    import Foundation
2
3    struct WeatherReading: Codable {
4        var city: String
5        var temperature: Double
6        var condition: String
7    }
8    let json = "{\"city\": \"Venice\", \"temperature\": 28.5, \"condition\": \"Sunny\"}"
9    let reading = try JSONDecoder().decode(WeatherReading.self, from: Data(json.utf8))
10 + print("\(reading.city): \(reading.temperature) degrees, \(reading.condition)")
```

OUTPUT SO FAR

```
Venice: 28.5 degrees, Sunny
```

Frame 5 of 7

The same program, one changed input: temperature now arrives in quotes, as text. A Double cannot be built from a string, so decode throws and nothing prints at all.

This frame is unfinished on purpose: The JSON sends temperature as a string, so decoding a Double throws a type mismatch before any print runs.

```
  import Foundation

  struct WeatherReading: Codable {
      var city: String
      var temperature: Double
      var condition: String
  }
+ let json = "{\"city\": \"Venice\", \"temperature\": \"28.5\", \"condition\": \"Sunny\"}"
  let reading = try JSONDecoder().decode(WeatherReading.self, from: Data(json.utf8))
  print("\(reading.city): \(reading.temperature) degrees, \(reading.condition)")
```

ERROR (ON PURPOSE)

```
Expected to decode Double but found a string instead
```

Frame 6 of 7

The repair decodes temperature as the String it really is, mapped by CodingKeys onto temperatureText, then converts that text to a Double. The summary line prints exactly as before.

```
  import Foundation

+ struct FlexibleReading: Codable {
      var city: String
+     var temperatureText: String
      var condition: String
+     enum CodingKeys: String, CodingKey {
+         case city, condition
+         case temperatureText = "temperature"
+     }
+ }
+ let json = "{\"city\": \"Venice\", \"temperature\": \"28.5\", \"condition\": \"Sunny\"}"
+ let flex = try JSONDecoder().decode(FlexibleReading.self, from: Data(json.utf8))
+ let temperature = Double(flex.temperatureText) ?? 0
+ print("\(flex.city): \(temperature) degrees, \(flex.condition)")
```

OUTPUT SO FAR

```
Venice: 28.5 degrees, Sunny
```

Frame 7 of 7

The repaired decoder is now the program: one shared decoder reads two readings in the service's text format, converts each temperature, and prints the same summary shape for both.

```
  import Foundation

  struct FlexibleReading: Codable {
      var city: String
      var temperatureText: String
      var condition: String
      enum CodingKeys: String, CodingKey {
          case city, condition
          case temperatureText = "temperature"
      }
  }
+ let first = "{\"city\": \"Venice\", \"temperature\": \"28.5\", \"condition\": \"Sunny\"}"
+ let second = "{\"city\": \"Oslo\", \"temperature\": \"4.5\", \"condition\": \"Clear\"}"
+ let decoder = JSONDecoder()
+ let flexA = try decoder.decode(FlexibleReading.self, from: Data(first.utf8))
+ print("\(flexA.city): \(Double(flexA.temperatureText) ?? 0) degrees, \(flexA.condition)")
+ let flexB = try decoder.decode(FlexibleReading.self, from: Data(second.utf8))
+ print("\(flexB.city): \(Double(flexB.temperatureText) ?? 0) degrees, \(flexB.condition)")
```

OUTPUT SO FAR

```
Venice: 28.5 degrees, Sunny
Oslo: 4.5 degrees, Clear
```

FINAL RESULT — CHECKED OUTPUT

```
Venice: 28.5 degrees, Sunny
Oslo: 4.5 degrees, Clear
```

Line by line: the finished program

1 `import Foundation`

- **WHO** The Swift compiler, resolving the Foundation module for this program.
- **WHAT** **Makes JSONDecoder, JSONEncoder and Data available.**
- **WHERE** At the top of the program.
- **WHEN** At compile time, before any decoding happens.
- **WHY** The JSON coders live in Foundation, not in the core language.

3 `struct FlexibleReading: Codable {`

- **WHO** Your code, declaring the tolerant replacement type.
- **WHAT** **Declares FlexibleReading, a Codable struct that expects temperature as text.**
- **WHERE** A value type in this program.
- **WHEN** Compiled once; used by the decode call below it.
- **WHY** When a service sends the wrong type, decode what it actually sends, then convert.

4 `var city: String`

- **WHO** Your struct, declaring the property the decoder fills.
- **WHAT** **Declares a city property of type String, filled from the city key.**
- **WHERE** A stored property of the struct being defined.
- **WHEN** Filled during decoding, when JSONDecoder reads the city key.
- **WHY** Every reading names the place it was taken, so city is the first field.

```
5  var temperatureText: String
```

WHO Your FlexibleReading struct, declaring the property the decoder fills.
WHAT **Declares temperatureText, a String filled from the temperature key via CodingKeys below.**
WHERE A stored property of FlexibleReading.
WHEN Filled during decoding, when JSONDecoder reads the temperature key.
WHY The text form is kept only long enough to convert it into a real Double.

```
6  var condition: String
```

WHO Your struct, declaring the property the decoder fills.
WHAT **Declares a condition property of type String, filled from the condition key.**
WHERE A stored property of the struct being defined.
WHEN Filled during decoding, when JSONDecoder reads the condition key.
WHY Conditions arrive as words such as Sunny, so a String holds them as sent.

```
7  enum CodingKeys: String, CodingKey {
```

WHO The compiler, which looks for an enum with exactly this name.
WHAT **Declares the list of keys used to decode FlexibleReading.**
WHERE Nested inside FlexibleReading, where the compiler looks for it.
WHEN Read at compile time while the Codable code is generated.
WHY When CodingKeys exists it replaces the default mapping, so it must list every property.

```
8  case city, condition
```

WHO The CodingKeys enum you wrote, listing two cases without raw values.
WHAT **Maps city and condition to JSON keys with the same spelling.**
WHERE Cases of the CodingKeys enum.
WHEN Used during decoding of FlexibleReading.
WHY A case without a raw value uses its own name as the key.

```
9  case temperatureText = "temperature"
```

WHO The CodingKeys enum you wrote, giving one case an explicit raw value.
WHAT **Maps the temperatureText property to the JSON key temperature.**
WHERE A case of the CodingKeys enum, with a String raw value.
WHEN Used during decoding of FlexibleReading.
WHY The raw value is the exact JSON key; the Swift name stays honest about the type.

```
12 let first = "{\"city\": \"Venice\", \"temperature\": \"28.5\", \"condition\": \"Sunny\"}"
```

WHO Your top-level code, writing the first faulty reading as a String literal.
WHAT **Holds the Venice reading, with its temperature sent as text.**
WHERE A constant in top-level code, standing in for bytes from the weather service.
WHEN Created when the program starts.
WHY The repaired decoder now faces the same faulty shape that broke the strict one.

```
13 let second = "{\"city\": \"Oslo\", \"temperature\": \"4.5\", \"condition\": \"Clear\"}"
```

WHO Your top-level code, writing the second faulty reading as a String literal.
WHAT **Holds the Oslo reading, also with its temperature sent as text.**
WHERE A constant in top-level code, standing in for bytes from the weather service.
WHEN Created when the program starts, next to the first reading.
WHY A repair that works once could be luck; a second reading proves it is a path.

```
14 let decoder = JSONDecoder()
```

WHO Foundation's JSONDecoder class, created once here and reused below.
WHAT **Creates one decoder for both readings in this program.**
WHERE A constant in top-level code.
WHEN Once, before either decode call.
WHY Reusing a configured decoder avoids repeating its setup.

```
15 let flexA = try decoder.decode(FlexibleReading.self, from: Data(first.utf8))
```

WHO JSONDecoder, running the Codable code generated for FlexibleReading.
WHAT **Decodes the first reading into a FlexibleReading value.**
WHERE A constant in top-level code, in memory.
WHEN When this line runs, using the shared decoder.
WHY The first of two decodes through the repaired path.

```
16 print("\(flexA.city): \(Double(flexA.temperatureText) ?? 0) degrees, \(flexA.condition)")
```

WHO Swift string interpolation and the Double conversion, then print.
WHAT **Prints the Venice summary, converting the text temperature inline.**
WHERE Standard output, the first of two lines.
WHEN After the first decode succeeds.
WHY The summary shape matches the strict program's, so callers cannot tell which path ran.

```
17 let flexB = try decoder.decode(FlexibleReading.self, from: Data(second.utf8))
```

WHO JSONDecoder, running the Codable code generated for FlexibleReading.
WHAT **Decodes the second reading into a FlexibleReading value.**
WHERE A constant in top-level code, in memory.
WHEN When this line runs, after the first reading has printed.
WHY The same decoder and struct handle a different city with no new code.

```
18 print("\(flexB.city): \(Double(flexB.temperatureText) ?? 0) degrees, \(flexB.condition)")
```

WHO Swift string interpolation and the Double conversion, then print.
WHAT **Prints the Oslo summary, converting the text temperature inline.**
WHERE Standard output, the second of two lines.
WHEN After the second decode succeeds.
WHY Two readings, one repaired path: the fix is the program now, not a workaround.

Try it

A second service sends readings with a city and a numeric temperature, but no condition. The starter requires a condition, so decoding throws. Make the condition optional and print the summary with `Unknown` when it is missing.

```
import Foundation

struct Reading: Codable {
    var city: String
    var temperature: Double
    var condition: String
}
let json = "{\"city\": \"Miami\", \"temperature\": 30.0}"
let reading = try JSONDecoder().decode(Reading.self, from: Data(json.utf8))
print("\(reading.city): \(reading.temperature) degrees, \(reading.condition)")
```

EXPECTED OUTPUT

```
Miami: 30.0 degrees, Unknown
```

WORKED EXAMPLE

Create and save related models: authors and books

Problem.

An author writes many books, and each book belongs to one author. Model that relationship, create one author with two books, save them, and read the relationship back from the author side.

The visual for this example in the HTML version is labelled Simulation: it models Apple frameworks or platform processes that cannot run in a web page.

Frame 1 of 7

The running code in this example models SwiftData in plain Swift; this frame is the real thing, read along: an @Model class for Author with one stored property, its name.

```
import SwiftData

@Model
final class Author {
    var name: String
    init(name: String) {
        self.name = name
    }
}
```

Frame 2 of 7

Book joins as a second @Model class. Its author property is the to-one end of the link: optional, because a book can exist before it is assigned to an author.

```
    import SwiftData

    @Model
    final class Author {
        var name: String
        init(name: String) {
            self.name = name
        }
    }
+
+ @Model
+ final class Book {
+     var title: String
+     var author: Author?
+     init(title: String, author: Author? = nil) {
+         self.title = title
+         self.author = author
+     }
+ }
```

Frame 3 of 7

Author gains the to-many end: a books array with @Relationship naming Book.author as its inverse and a cascade delete rule. SwiftData will keep both ends in step automatically.

```
    import SwiftData

    @Model
    final class Author {
        var name: String
+     @Relationship(deleteRule: .cascade, inverse: \Book.author)
+     var books: [Book] = []
+     init(name: String) {
+         self.name = name
+     }
+ }
+
+ @Model
+ final class Book {
+     var title: String
+     var author: Author?
+     init(title: String, author: Author? = nil) {
+         self.title = title
+         self.author = author
+     }
+ }
```

Frame 4 of 7

The mirror begins: plain Swift classes with the same properties, no @Model macro, and an array standing in for the store. One author is created and listed as a record, unsaved.

```
+ final class Author {
+     var name: String
+     var books: [Book] = []
+     init(name: String) { self.name = name }
+ }
+ final class Book {
+     var title: String
+     var author: Author?
+     init(title: String, author: Author?) { self.title = title; self.author = author }
+ }
+ var records: [String] = []
+ let author = Author(name: "Octavia Butler")
+ records.append("Author: \(author.name)")
+ for record in records { print(record) }
```

Frame 5 of 7

Two books are created already pointing at the author, then appended to the author's books array, mirroring how SwiftData maintains the inverse. All three records exist only in memory.

```
  final class Author {
      var name: String
      var books: [Book] = []
      init(name: String) { self.name = name }
  }
  final class Book {
      var title: String
      var author: Author?
      init(title: String, author: Author?) { self.title = title; self.author = author }
  }
  var records: [String] = []
  let author = Author(name: "Octavia Butler")
  records.append("Author: \(author.name)")
+ let kindred = Book(title: "Kindred", author: author)
+ author.books.append(kindred)
+ records.append("Book: \(kindred.title)")
+ let fledgling = Book(title: "Fledgling", author: author)
+ author.books.append(fledgling)
+ records.append("Book: \(fledgling.title)")
+ for record in records { print(record) }
```

Frame 6 of 7

A save step flips the mirror's saved flag, standing in for the ModelContext save that writes SwiftData records to disk. The diagram now reports the three records as saved.

```
  final class Author {
      var name: String
      var books: [Book] = []
      init(name: String) { self.name = name }
  }
  final class Book {
      var title: String
      var author: Author?
      init(title: String, author: Author?) { self.title = title; self.author = author }
  }
  var records: [String] = []
  let author = Author(name: "Octavia Butler")
  records.append("Author: \(author.name)")
  let kindred = Book(title: "Kindred", author: author)
  author.books.append(kindred)
  records.append("Book: \(kindred.title)")
  let fledgling = Book(title: "Fledgling", author: author)
  author.books.append(fledgling)
  records.append("Book: \(fledgling.title)")
+ let saved = true
+ print("Store saved: \(records.count) records")
+ for record in records { print(record) }
```

Frame 7 of 7

Reading the relationship from the other side: looping over the author's books prints both titles, proving the links survived the save. In SwiftData this same loop needs no store code at all.

```
  final class Author {
      var name: String
      var books: [Book] = []
      init(name: String) { self.name = name }
  }
  final class Book {
      var title: String
      var author: Author?
      init(title: String, author: Author?) { self.title = title; self.author = author }
  }
  var records: [String] = []
  let author = Author(name: "Octavia Butler")
  records.append("Author: \(author.name)")
  let kindred = Book(title: "Kindred", author: author)
  author.books.append(kindred)
  records.append("Book: \(kindred.title)")
  let fledgling = Book(title: "Fledgling", author: author)
  author.books.append(fledgling)
  records.append("Book: \(fledgling.title)")
  let saved = true
  print("Store saved: \(records.count) records")
  for record in records { print(record) }
+ for book in author.books { print("\(author.name) wrote \(book.title)") }
```

Line by line: the finished program

```
1  final class Author {
```

WHO	You, declaring a plain Swift class for the mirror program.
WHAT	**Declares Author with the same properties as the @Model version, but no macro.**
WHERE	A class in this program, standing in for the SwiftData model.
WHEN	Instances are created when the mirror code runs.
WHY	The mirror keeps the shape and drops the framework, so the engine can run it.

```
2  var name: String
```

WHO	Your model, declaring the property SwiftData persists.
WHAT	**Declares a name property of type String.**
WHERE	A stored property of the model, saved in the database.
WHEN	Set when the object is created and saved with the context.
WHY	An ordinary attribute: plain var properties are persisted automatically.

```
3  var books: [Book] = []
```

WHO	Your model, declaring the to-many end of the relationship.
WHAT	**Declares a books array holding every book linked to this author, starting empty.**
WHERE	A relationship property of the author model.
WHEN	Changes whenever a book joins or leaves the author.
WHY	An array is the natural shape for the many end of a one-to-many link.

```
4  init(name: String) { self.name = name }
```

WHO	Author's initializer, written by you.
WHAT	**Creates a new author and stores the given name, all on one line.**
WHERE	An initializer of the mirror Author class.
WHEN	Whenever the mirror code creates an author.
WHY	A one-line initializer keeps the mirror short enough to grow across frames.

```
6  final class Book {
```

WHO	You, declaring a plain Swift class for the mirror program.
WHAT	**Declares the mirror Book class, with no @Model macro.**
WHERE	A class in this program, standing in for the SwiftData model.
WHEN	Instances are created when the mirror code runs.
WHY	Same idea as the real model: a class whose instances can be linked and stored.

```
7  var title: String
```

WHO	Your model, declaring the property SwiftData persists.
WHAT	**Declares a title property of type String.**
WHERE	A stored property of the book model, saved in the database.
WHEN	Set when the book is created.
WHY	An ordinary attribute, stored next to the relationship below it.

```
8  var author: Author?
```

WHO	Your model, declaring the to-one end of the relationship.
WHAT	**Declares an optional author: the author this book belongs to, or nil.**
WHERE	A relationship property of the book model.
WHEN	Changes when the book is assigned, moved or unassigned.
WHY	Optional, because a book can exist before anyone assigns it to an author.

```
9  init(title: String, author: Author?) { self.title = title; self.author = author }
```

WHO Book's initializer, written by you.
WHAT **Creates a new book, storing both the title and the author on one line.**
WHERE An initializer of the mirror Book class.
WHEN Whenever the mirror code creates a book.
WHY Passing the author at creation links the book from the to-one end immediately.

```
11 var records: [String] = []
```

WHO Your mirror program, declaring the array that plays the store.
WHAT **Creates an empty records array: the in-memory stand-in for the SwiftData store.**
WHERE A variable in top-level code, in memory only.
WHEN Created when the program starts, before any object exists.
WHY An array cannot persist anything, which is exactly what the saved flag will dramatize.

```
12 let author = Author(name: "Octavia Butler")
```

WHO Your mirror program, calling Author's initializer.
WHAT **Creates the author Octavia Butler in memory; nothing has stored her yet.**
WHERE A constant in top-level code.
WHEN When this line runs, after the classes exist.
WHY Creating an object and storing it are separate steps, in the mirror and in SwiftData alike.

```
13 records.append("Author: \(author.name)")
```

WHO Your mirror program, appending a description to the records array.
WHAT **Records the author as the store's first entry, as a readable string.**
WHERE In the records array, in memory.
WHEN Immediately after the author is created.
WHY This append mirrors context.insert: the moment the object joins the store's world.

```
14 let kindred = Book(title: "Kindred", author: author)
```

WHO Your mirror program, calling Book's initializer with the author.
WHAT **Creates the book Kindred, already pointing at Octavia Butler.**
WHERE A constant in top-level code, in memory.
WHEN When this line runs, after the author exists.
WHY The book knows its author from birth; the author's list is updated on the next line.

```
15 author.books.append(kindred)
```

WHO Your mirror program, appending to the author's books array.
WHAT **Links Kindred from the to-many end, mirroring how SwiftData maintains the inverse.**
WHERE On the author object, in memory.
WHEN Immediately after Kindred is created.
WHY In SwiftData, setting either end updates the other; the mirror does that second step by hand.

```
16 records.append("Book: \(kindred.title)")
```

WHO Your mirror program, appending a description to the records array.
WHAT **Records Kindred as the store's second entry.**
WHERE In the records array, in memory.
WHEN Immediately after the book is linked.
WHY Each stored object gets one record line, so the printout counts the store.

```
17 let fledgling = Book(title: "Fledgling", author: author)
```

WHO Your mirror program, calling Book's initializer with the author.
WHAT **Creates the second book, Fledgling, also pointing at Octavia Butler.**
WHERE A constant in top-level code, in memory.
WHEN When this line runs, after Kindred exists.
WHY A one-to-many link earns its name only when the many side has more than one member.

```
18 author.books.append(fledgling)
```

WHO Your mirror program, appending to the author's books array.
WHAT **Links Fledgling from the to-many end, next to Kindred.**
WHERE On the author object, in memory.
WHEN Immediately after Fledgling is created.
WHY The author's array now holds both books, in the order they were linked.

```
19 records.append("Book: \(fledgling.title)")
```

WHO Your mirror program, appending a description to the records array.
WHAT **Records Fledgling as the store's third entry.**
WHERE In the records array, in memory.
WHEN Immediately after the book is linked.
WHY Three records now exist, and every one of them is still only in memory.

```
20 let saved = true
```

WHO Your mirror program, declaring the flag that plays the save state.
WHAT **Sets saved to true, marking the moment the mirror store is written out.**
WHERE A constant in top-level code.
WHEN After all three records exist, before the saved line prints.
WHY A Bool cannot hold records; it only stands in for the save having happened.

```
21 print("Store saved: \(records.count) records")
```

WHO Your mirror program, calling print with the record count interpolated.
WHAT **Prints how many records the save covered: all three.**
WHERE Standard output, one line.
WHEN Right after the saved flag is set.
WHY The count in the output must match the records the diagram lists as saved.

```
22 for record in records { print(record) }
```

WHO Your mirror program, looping over the records array with print inside.
WHAT **Prints every record currently in the mirror store, one per line.**
WHERE Standard output, one line per record.
WHEN At the end of the program, after all appends.
WHY Printing the store makes its contents provable instead of assumed.

```
23 for book in author.books { print("\(author.name) wrote \(book.title)") }
```

WHO Your mirror program, looping over the author's books and printing inside.
WHAT **Prints one line per book, read back through the author's side of the relationship.**
WHERE Standard output, one line per book.
WHEN At the end of the program, after the save.
WHY Reading the link from the other side proves both ends describe the same relationship.

Try it

The starter creates an author and a book but never files the book, so the author's count prints 0. Add the missing link so the count prints 1.

```
final class Author {
    var name: String
    var books: [Book] = []
    init(name: String) { self.name = name }
}
final class Book {
    var title: String
    init(title: String) { self.title = title }
}
let author = Author(name: "Ursula Le Guin")
let book = Book(title: "A Wizard of Earthsea")
print("Books by \(author.name): \(author.books.count)")
```

EXPECTED OUTPUT

```
Books by Ursula Le Guin: 1
```

WORKED EXAMPLE

Load, edit, save, reload — and the save somebody forgot

Problem.

A settings screen keeps one number, the volume. Load it from its saved JSON, change it, and prove which changes survive a reload: one edit is lost, one is kept.

Frame 1 of 6

A JSON string plays the role of the disk. Decoding it loads the saved settings, and the print proves the stored volume, 5, is what the program starts with.

```
import Foundation

struct Settings: Codable {
    var volume: Int
}
var disk = "{\"volume\": 5}"
let settings = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
print("Loaded volume: \(settings.volume)")
```

OUTPUT SO FAR

```
Loaded volume: 5
```

Frame 2 of 6

The loaded value becomes a var and is edited to 8 in memory. Nothing has touched the disk string yet, so this new value exists only while the program runs.

```
  import Foundation

  struct Settings: Codable {
      var volume: Int
  }
  var disk = "{\"volume\": 5}"
+ var settings = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
  print("Loaded volume: \(settings.volume)")
+ settings.volume = 8
+ print("In memory volume: \(settings.volume)")
```

OUTPUT SO FAR

```
Loaded volume: 5
In memory volume: 8
```

Frame 3 of 6

Reloading decodes the disk string again, and the old value, 5, comes back. The edit was never saved: this is the classic forgotten-save bug, proven by the output alone.

```
  import Foundation

  struct Settings: Codable {
      var volume: Int
  }
  var disk = "{\"volume\": 5}"
  var settings = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
  print("Loaded volume: \(settings.volume)")
  settings.volume = 8
  print("In memory volume: \(settings.volume)")
+ let reloaded = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
+ print("After reload volume: \(reloaded.volume)")
```

OUTPUT SO FAR

```
Loaded volume: 5
In memory volume: 8
After reload volume: 5
```

Frame 4 of 6

This time the edit is encoded back into the disk string before anything reloads. Printing the disk shows the new JSON, with volume 8, so the save really happened.

```
   import Foundation

   struct Settings: Codable {
       var volume: Int
   }
   var disk = "{\"volume\": 5}"
   var settings = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
   print("Loaded volume: \(settings.volume)")
   settings.volume = 8
+ let data = try JSONEncoder().encode(settings)
+ disk = String(data: data, encoding: .utf8) ?? ""
+ print("On disk: \(disk)")
```

OUTPUT SO FAR

```
Loaded volume: 5
On disk: {"volume":8}
```

Frame 5 of 6

Reloading now decodes the updated disk string, and 8 survives. Same reload code as the buggy frame; the only difference is that the save happened first this time.

```
   import Foundation

   struct Settings: Codable {
       var volume: Int
   }
   var disk = "{\"volume\": 5}"
   var settings = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
   print("Loaded volume: \(settings.volume)")
   settings.volume = 8
   let data = try JSONEncoder().encode(settings)
   disk = String(data: data, encoding: .utf8) ?? ""
   print("On disk: \(disk)")
+ let reloaded = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
+ print("After reload volume: \(reloaded.volume)")
```

OUTPUT SO FAR

```
Loaded volume: 5
On disk: {"volume":8}
After reload volume: 8
```

Frame 6 of 6

One more full cycle at volume 10: copy, edit, encode, save, reload. The second reload also returns the new value, so the load-edit-save pattern is now the habit.

```
  import Foundation

  struct Settings: Codable {
      var volume: Int
  }
  var disk = "{\"volume\": 5}"
  var settings = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
  print("Loaded volume: \(settings.volume)")
  settings.volume = 8
  let data = try JSONEncoder().encode(settings)
  disk = String(data: data, encoding: .utf8) ?? ""
  print("On disk: \(disk)")
  let reloaded = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
  print("After reload volume: \(reloaded.volume)")
+ var again = reloaded
+ again.volume = 10
+ let data2 = try JSONEncoder().encode(again)
+ disk = String(data: data2, encoding: .utf8) ?? ""
+ let check = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
+ print("Second reload volume: \(check.volume)")
```

OUTPUT SO FAR

```
Loaded volume: 5
On disk: {"volume":8}
After reload volume: 8
Second reload volume: 10
```

FINAL RESULT — CHECKED OUTPUT

```
Loaded volume: 5
On disk: {"volume":8}
After reload volume: 8
Second reload volume: 10
```

Line by line: the finished program

1 `import Foundation`

- **WHO** The Swift compiler, resolving the Foundation module for this program.
- **WHAT** **Makes JSONDecoder, JSONEncoder and Data available.**
- **WHERE** At the top of the program.
- **WHEN** At compile time, before any loading or saving happens.
- **WHY** Loading and saving settings as JSON is Foundation work.

3 `struct Settings: Codable {`

- **WHO** Your code, declaring the type the saved JSON must fit.
- **WHAT** **Declares Settings, a Codable struct holding the whole settings screen's state.**
- **WHERE** A value type in this program.
- **WHEN** Compiled once; used by every decode and encode call below.
- **WHY** One struct for the whole file keeps save and load in perfect agreement.

```
4  var volume: Int
```

WHO Your Settings struct, declaring the property the coders fill.
WHAT **Declares a volume property of type Int, the one setting this screen keeps.**
WHERE A stored property of Settings, saved in the JSON.
WHEN Filled during decoding and read during encoding.
WHY A single number keeps the example focused on the save, not the settings.

```
6  var disk = "{\"volume\": 5}"
```

WHO Your top-level code, writing the saved JSON as a String literal.
WHAT **Creates the disk: a string holding the settings exactly as a file would.**
WHERE A variable in top-level code, standing in for a file on the device.
WHEN Created when the program starts, holding volume 5.
WHY A var string can be rewritten by the save step, just like a real file can.

```
7  var settings = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
```

WHO JSONDecoder, running the Codable code generated for Settings.
WHAT **Decodes the disk string into a Settings value the program may edit.**
WHERE A variable in top-level code, in memory.
WHEN When this line runs, at the start of the program.
WHY var, not let: the next frames change this copy, which is where the trouble starts.

```
8  print("Loaded volume: \(settings.volume)")
```

WHO Swift string interpolation, reading the loaded value, then print.
WHAT **Prints the volume that loading produced.**
WHERE Standard output, one line.
WHEN Right after the load.
WHY This line is the baseline every later print is compared against.

```
9  settings.volume = 8
```

WHO Your top-level code, assigning to the in-memory copy.
WHAT **Changes the volume to 8 in memory only; the disk string is untouched.**
WHERE On the settings variable, in memory.
WHEN After the load, before any save exists in this program.
WHY Editing a value and saving it are two different events; this frame has only the first.

```
10 let data = try JSONEncoder().encode(settings)
```

WHO Foundation's JSONEncoder, running the Codable code generated for Settings.
WHAT **Encodes the edited settings into JSON bytes.**
WHERE A constant in top-level code, in memory.
WHEN After the edit, as the first half of the save.
WHY Encoding is the save's first half; nothing is stored until the bytes reach the disk.

```
11 disk = String(data: data, encoding: .utf8) ?? ""
```

WHO Your top-level code, converting bytes to text and assigning to disk.
WHAT **Replaces the disk string with the freshly encoded JSON, completing the save.**
WHERE The disk variable, standing in for the settings file.
WHEN Immediately after encoding.
WHY From this line on, the disk and memory agree; ?? "" only guards a conversion that cannot fail for JSON bytes.

```swift
print("On disk: \(disk)")
```

WHO Your top-level code, calling print with the disk string itself.
WHAT **Prints the exact contents of the disk after the save.**
WHERE Standard output, one line.
WHEN Right after the save.
WHY Showing the stored bytes proves the save wrote the new value, not the old one.

```swift
let reloaded = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
```

WHO JSONDecoder, running the Codable code generated for Settings.
WHAT **Decodes the disk string again into a fresh Settings value.**
WHERE A constant in top-level code, in memory.
WHEN After the edit, reading the same disk string as the load did.
WHY Reloading always answers from the disk, never from the edited copy.

```swift
print("After reload volume: \(reloaded.volume)")
```

WHO Swift string interpolation, reading the reloaded value, then print.
WHAT **Prints the volume the reload produced.**
WHERE Standard output, one line.
WHEN Right after the reload.
WHY Compare this number with the in-memory print: they agree only if a save happened between them.

```swift
var again = reloaded
```

WHO Your top-level code, copying the reloaded struct value.
WHAT **Copies the reloaded settings into a new variable for the second edit.**
WHERE A variable in top-level code, in memory.
WHEN After the first reload has proved the save worked.
WHY Structs are values, so this copy can be edited without touching the reloaded original.

```swift
again.volume = 10
```

WHO Your top-level code, assigning to the copied settings.
WHAT **Changes the copied volume to 10, again in memory only for now.**
WHERE On the again variable, in memory.
WHEN After the copy, before the second encode.
WHY The second cycle repeats the pattern deliberately: edit first, then save, then reload.

```swift
let data2 = try JSONEncoder().encode(again)
```

WHO Foundation's JSONEncoder, running the Codable code generated for Settings.
WHAT **Encodes the second edit into JSON bytes.**
WHERE A constant in top-level code, in memory.
WHEN After the second edit, as the first half of the second save.
WHY Same encoder call as the first save; the pattern, not the code, is what changed.

```swift
disk = String(data: data2, encoding: .utf8) ?? ""
```

WHO Your top-level code, converting bytes to text and assigning to disk.
WHAT **Replaces the disk string with the second edit's JSON, completing the second save.**
WHERE The disk variable, standing in for the settings file.
WHEN Immediately after the second encode.
WHY The disk now holds volume 10; the final reload has something new to find.

```
19 let check = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
```

WHO — JSONDecoder, running the Codable code generated for Settings.
WHAT — **Decodes the disk string one last time into a fresh Settings value.**
WHERE — A constant in top-level code, in memory.
WHEN — After the second save, as the program's final read of the disk.
WHY — A reload is the only honest test of a save, so the program ends with one.

```
20 print("Second reload volume: \(check.volume)")
```

WHO — Swift string interpolation, reading the final reloaded value, then print.
WHAT — **Prints the volume the second reload produced: 10.**
WHERE — Standard output, the last line this program prints.
WHEN — Right after the final reload.
WHY — The second cycle persisting proves the first save was a habit forming, not luck.

Try it

The starter loads volume 3, edits it to 7, and reloads — but never saves, so the reload prints the old value. Add the save between the edit and the reload so the final print shows 7.

```
1  import Foundation
2
3  struct Settings: Codable {
4      var volume: Int
5  }
6  var disk = "{\"volume\": 3}"
7  var settings = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
8  settings.volume = 7
9  let reloaded = try JSONDecoder().decode(Settings.self, from: Data(disk.utf8))
10 print("Volume after reload: \(reloaded.volume)")
```

EXPECTED OUTPUT

```
Volume after reload: 7
```

MODULE 07

Device APIs: Location, Maps, Cloud and More

Ask for permission in a way people trust, then put location, maps, iCloud storage, photos, on-device intelligence, widgets, App Intents and notifications to work in modern Swift.

- Integrate APIs such as location services and cloud storage
- Create various types of iOS applications in Swift

Permission and Privacy

CONCEPT

Your app borrows the device's senses

An iPhone knows where it is, sees through its cameras, hears through its microphones and holds years of photos and contacts. Apple calls these **protected resources**: data or hardware your app may use only with the person's permission.

Your app never draws the permission question itself. It calls an API, and **iOS** (the operating system) shows a standard alert that your code cannot restyle or answer. The person decides, and can change that decision at any time in the Settings app.

Protected resource	Location, camera, microphone, photos, contacts, calendars, Bluetooth, health data, and the right to send notifications.
Authorization status	The current answer, which your code reads before using the resource: not asked yet, allowed, denied or restricted.
Restricted	A parent or an employer has blocked the resource with Screen Time or device management. The person cannot change it either.
Least privilege	Ask for the smallest access that works: while the app is in use instead of always, one picked photo instead of the whole library.

> **Permission is a relationship**
>
> People grant access to apps they trust. Every request spends a little of that trust, so ask only when the benefit is obvious to them.

CONCEPT

Usage descriptions live in Info.plist

Info.plist (information property list) is your app's settings file: keys and values that iOS reads before your code runs. In Xcode 26 you edit it on the target's **Info** tab. For each protected resource you add a **usage description**, a sentence iOS prints inside its alert. Be specific: "Trailhead shows trails within walking distance of you" beats "We need your location."

NSLocationWhenInUseUsageDescription	Using location while the app is in use.
NSLocationAlwaysAndWhenInUseUsageDescription	Location even when the app is closed. Add it only if a feature truly needs that.
NSCameraUsageDescription, **NSMicrophoneUsageDescription**	Taking photos or video, and recording sound.
NSPhotoLibraryUsageDescription	Reading the whole photo library. The system photo picker needs no key at all.

> **Missing key, broken feature**
>
> Without the key, iOS refuses, quietly or loudly. A location request is silently ignored, so no alert ever appears. Touching the camera crashes the app on the spot. Notifications are the exception: they have no key, and iOS writes the alert text itself.

CODE · READ-ALONG

Check the status before you ask

Before using a protected resource, read its authorization status and handle every case. This class does that for Trailhead, a hiking app. `CLLocationManager` is Core Location's object for permission and status. Tap the feature in the simulation beside the code and watch which branch runs.

```
1  import CoreLocation
2
3  @MainActor
4  final class TrailFinder {
5      let manager = CLLocationManager()
6
7      func findNearbyTrails() {
8          switch manager.authorizationStatus {
9          case .notDetermined:
10             manager.requestWhenInUseAuthorization()
11         case .authorizedWhenInUse, .authorizedAlways:
12             showNearbyTrails()
13         case .denied:
14             showOpenSettingsPrompt()
15         case .restricted:
16             showRestrictedMessage()
17         @unknown default:
18             break
19         }
20     }
21 }
```

Line by line

1 `import CoreLocation`

- **WHO** The Swift compiler, while it builds the app.
- **WHAT** **Makes Core Location types such as CLLocationManager and CLAuthorizationStatus visible in this file.**
- **WHERE** At the top of the file; an import covers only this one source file.
- **WHEN** At compile time, before any code runs.
- **WHY** Core Location is a separate Apple framework, not part of the Swift language, so every file that touches location imports it.

3 `@MainActor`

- **WHO** Swift concurrency's main actor, which owns the main thread.
- **WHAT** **Requires every property and method of the class below to run on the main thread.**
- **WHERE** On the TrailFinder declaration, so it covers the whole class.
- **WHEN** Checked at compile time; at run time, calls from background code hop to the main thread first.
- **WHY** Permission requests and screen updates belong on the main thread, and the attribute lets Swift 6 data-race checking prove it.

```
4  final class TrailFinder {
```

WHO Your code: a type you define for the trail feature.
WHAT **Declares a class named TrailFinder that no other class may subclass.**
WHERE At file scope; each instance lives on the heap and is shared by reference.
WHEN The type exists at compile time; an instance appears when a screen creates one.
WHY A class suits an object that keeps a long-lived manager, and final tells the compiler that no subclass will override anything.

```
5  let manager = CLLocationManager()
```

WHO Core Location, which creates the manager object.
WHAT **Creates one location manager and keeps it in a constant property of this TrailFinder.**
WHERE A stored property inside each TrailFinder instance, on the heap.
WHEN When a TrailFinder instance is initialized.
WHY The manager must outlive any single method; a local variable would be released when the method returns, and its pending request with it.

```
7  func findNearbyTrails() {
```

WHO Your code, called by the button the person taps.
WHAT **Declares the method that decides what tapping "Show trails near me" does.**
WHERE An instance method of TrailFinder, running on the main actor.
WHEN Each time the person taps the feature.
WHY Checking at the moment of use, not at launch, ties any request to an action the person just took.

```
8  switch manager.authorizationStatus {
```

WHO Core Location, which reports the answer stored for this app.
WHAT **Reads the current status and runs exactly one of the cases below.**
WHERE Inside findNearbyTrails, on the main thread.
WHEN On every tap, so the code reacts to the latest answer, including changes made in Settings.
WHY The person can change the answer while the app runs; reading it fresh avoids acting on a stale value.

```
9  case .notDetermined:
```

WHO The switch over manager.authorizationStatus, matching the .notDetermined case.
WHAT **Matches when this app has never asked, or an Allow Once grant has expired.**
WHERE The first branch of the switch.
WHEN Usually on the first tap after installing the app.
WHY This is the only state in which a request shows an alert, so it is the only place the code asks.

```
10 manager.requestWhenInUseAuthorization()
```

WHO iOS, which shows its standard alert on your behalf.
WHAT **Asks iOS to display the location alert containing your Info.plist purpose string.**
WHERE Called on the main thread; the alert is drawn by the system, outside your app's views.
WHEN Right after the tap; the person's answer arrives later through the manager's delegate, not as a return value.
WHY Asking only for "when in use" follows least privilege, because most features never need location in the background.

```
11 case .authorizedWhenInUse, .authorizedAlways:
```

WHO The switch over manager.authorizationStatus, matching either of the two granted cases.
WHAT **Matches either granted state by listing two patterns in one case.**
WHERE The second branch of the switch.
WHEN After Allow Once or Allow While Using App, or after Always was granted in Settings.
WHY Both states let the feature work right now, so one branch avoids duplicate code.

12 `showNearbyTrails()`

WHO Your code: a helper that fetches the position and sorts the list.
WHAT **Runs the real feature, because permission is already in place.**
WHERE A TrailFinder method whose body is hidden in this example.
WHEN Only when the status says access is granted.
WHY Keeping the feature in a helper makes the switch read like a decision table.

13 `case .denied:`

WHO The switch over manager.authorizationStatus, matching the .denied case after a refusal.
WHAT **Matches when the person chose Don't Allow or switched location off for this app in Settings.**
WHERE The third branch of the switch.
WHEN Any time after a refusal; iOS never shows the alert again for this install.
WHY Asking again would do nothing, so the right response is to explain and point to Settings.

14 `showOpenSettingsPrompt()`

WHO Your code, drawing your own message.
WHAT **Shows a friendly card that says what is off and offers an Open Settings button.**
WHERE A TrailFinder helper that updates the screen on the main thread.
WHEN When the person taps the feature while access is denied.
WHY Only the person can reverse a denial, and only in Settings, so you give them a direct path there.

15 `case .restricted:`

WHO The switch over manager.authorizationStatus, matching the .restricted case set by Screen Time or device management.
WHAT **Matches when Screen Time or device management blocks location for this person.**
WHERE The fourth branch of the switch.
WHEN On managed or parent-controlled devices, whatever the person wants.
WHY The person cannot fix this in Settings, so it needs a different message than a denial.

16 `showRestrictedMessage()`

WHO Your TrailFinder class, calling its own showRestrictedMessage() helper.
WHAT **Shows a short note that location is unavailable on this device, without a Settings button.**
WHERE A TrailFinder helper method.
WHEN Only in the restricted state.
WHY A Settings link would frustrate people here, because someone else holds the switch.

17 `@unknown default:`

WHO The Swift compiler, which checks the switch for completeness.
WHAT **Catches any status that a future iOS version might add.**
WHERE The last branch of the switch.
WHEN Never with today's statuses; it exists for ones that do not exist yet.
WHY The status type comes from a framework that can grow, and @unknown makes the compiler warn you when Apple adds a case you have not handled.

18 `break`

WHO The @unknown default branch of the switch in findNearbyTrails(), ending with a bare break.
WHAT **Does nothing and leaves the switch, because a Swift case cannot be empty.**
WHERE Inside the @unknown default branch.
WHEN Only if an unknown future status ever appears.
WHY Doing nothing is a safe fallback; the rest of the app keeps working.

Try it

1. In the simulation, choose Don't Allow, then tap the feature again. Which case runs now, and why does no alert appear?
2. Turn on the Screen Time restriction in the simulation and tap the feature. Which line is highlighted?
3. In Xcode, delete the @unknown default case. What does the compiler warn you about?

INTERACTIVE

Walk the whole permission journey

Trailhead runs on a simulated iPhone, with its Info.plist, authorization status, Swift switch and a log beside it. Try five journeys. Choose Allow Once, then send the app to the background. Choose Don't Allow, tap the feature again, and follow the recovery card into Settings. Empty the purpose string and ask. Turn on the Screen Time restriction. Turn on priming. Notice that the alert appears only from `notDetermined`, and only iOS draws it.

This step is an interactive simulation in the HTML version.

CONCEPT

Ask in context, prime honestly, recover gracefully

Timing decides whether people say yes. Ask right after the person taps something that clearly needs the resource, such as *Show trails near me*. A request at launch, before they have seen anything, looks suspicious and is often refused.

If the screen does not already make the reason clear, show a **priming screen** first: your own short explanation of the benefit. Give it one button, such as Continue, that leads straight to the system alert.

One shot	iOS shows its alert once per install. After Don't Allow, asking again does nothing; only Settings can change the answer.
Degrade, do not block	When access is denied, keep the rest of the app working: list trails alphabetically instead of by distance.
Show the way back	Offer a button that opens your app's page in Settings, and say what will improve.

Apple's rule for priming screens

The Human Interface Guidelines ask that a pre-permission screen include only one button, and that it open the system alert. A Not Now or Skip button there can feel manipulative, and App Review's guidelines forbid pressuring people into granting access.

CODE · LIVE SWIFTUI PREVIEW

A priming screen in SwiftUI

A priming screen is an ordinary SwiftUI view. This one names the benefit, then offers a single Continue button. In a real app the button would call `requestWhenInUseAuthorization()`; here a `@State` flag swaps

in a message, so you can see the moment the system alert would take over. Edit the text and tap Continue in the preview.

```swift
import SwiftUI

struct LocationPrimer: View {
    @State private var continued = false

    var body: some View {
        VStack(spacing: 16) {
            Image(systemName: "location.fill")
                .font(.largeTitle)
            Text("Trailhead sorts trails by distance.")
                .font(.headline)
            if continued {
                Text("The system alert appears now.")
            } else {
                Button("Continue") { continued = true }
                    .buttonStyle(.borderedProminent)
            }
        }
    }
}

#Preview { LocationPrimer() }
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so this file can declare LocationPrimer with VStack, Image, Text, Button and @State.
- **WHAT** **Loads SwiftUI, which supplies View, VStack, Image, Text, Button and @State.**
- **WHERE** The first line of the file; it applies to this file only.
- **WHEN** At compile time, before any line of this file runs; the @State flag and the Button exist only once SwiftUI is in scope.
- **WHY** SwiftUI is a framework, not part of the language, so the file must import it before using its views.

3 `struct LocationPrimer: View {`

- **WHO** Your code, describing one screen for SwiftUI.
- **WHAT** **Declares a view type named LocationPrimer that promises to provide a body.**
- **WHERE** At file scope; SwiftUI creates and discards these small structs freely.
- **WHEN** Defined at compile time; SwiftUI builds a value whenever a parent shows this screen.
- **WHY** Views are lightweight value types, so SwiftUI can recreate them cheaply whenever state changes.

4 `@State private var continued = false`

- **WHO** SwiftUI, which stores the value outside the struct and watches it.
- **WHAT** **Creates a true-or-false flag that records whether the person tapped Continue.**
- **WHERE** In storage that SwiftUI keeps for this view while it is on screen.
- **WHEN** Created when the view first appears; read every time the body is computed.
- **WHY** Changing a @State value makes SwiftUI recompute the body, which is how the screen swaps its content.

```
6  var body: some View {
```

WHO SwiftUI, which calls body to learn what to draw.
WHAT **Declares the computed property that returns this screen's content.**
WHERE Inside LocationPrimer; the result is a description, not pixels.
WHEN When the view appears and again after continued changes.
WHY some View lets the compiler keep the exact nested view type without you writing it out.

```
7  VStack(spacing: 16) {
```

WHO SwiftUI's vertical stack layout.
WHAT **Arranges the icon, the headline and the button top to bottom with 16 points between them.**
WHERE The root of the body, centered on the screen.
WHEN Laid out each time the body is computed.
WHY A simple stack keeps the explanation short and scannable, which is the whole point of a priming screen.

```
8  Image(systemName: "location.fill")
```

WHO SwiftUI, drawing an SF Symbol (Apple's built-in icon set).
WHAT **Shows the filled location arrow, the icon people already link with location.**
WHERE At the top of the stack.
WHEN Every time the body is drawn.
WHY A familiar symbol tells people what the request is about before they read a word.

```
9  .font(.largeTitle)
```

WHO SwiftUI's font(_:) modifier, applied to the location.fill symbol image.
WHAT **Sizes the symbol to match the largest text style.**
WHERE Applied to the Image on the line above.
WHEN When SwiftUI renders the image.
WHY SF Symbols scale with fonts, so a text style keeps the icon in step with Dynamic Type.

```
10 Text("Trailhead sorts trails by distance.")
```

WHO SwiftUI, drawing a label.
WHAT **States the benefit the person gets in exchange for sharing location.**
WHERE Below the icon in the stack.
WHEN Every time the body is drawn.
WHY Leading with the benefit, not the request, is what makes priming honest and effective.

```
11 .font(.headline)
```

WHO SwiftUI's font(_:) modifier, applied to the benefit sentence above it.
WHAT **Sets the label to the bold headline text style.**
WHERE Applied to the Text on the line above.
WHEN Each time SwiftUI lays out the headline Text, and again when the person changes their Dynamic Type size.
WHY Built-in text styles adapt to the person's preferred text size automatically.

```
12 if continued {
```

WHO SwiftUI's view builder, evaluating a condition inside body.
WHAT **Chooses which content to show based on the continued flag.**
WHERE Inside the VStack, below the headline.
WHEN Each time body is recomputed.
WHY Describing both states in body lets SwiftUI swap them automatically when the flag changes.

```
13  Text("The system alert appears now.")
```

WHO — SwiftUI's Text view, standing in for the alert that iOS would draw in a real app.
WHAT — **Shows a placeholder message where iOS would present its permission alert.**
WHERE — In the true branch of the if.
WHEN — After the person taps Continue.
WHY — In the preview there is no real alert, so this message marks the hand-off to iOS.

```
14  } else {
```

WHO — SwiftUI's view builder inside LocationPrimer's body, switching to the not-yet-continued branch.
WHAT — **Starts the content shown while the flag is still false.**
WHERE — The second branch of the if.
WHEN — Before the person has tapped Continue.
WHY — Exactly one of the two branches appears at a time, so the button disappears once it has done its job.

```
15  Button("Continue") { continued = true }
```

WHO — SwiftUI, and the person tapping.
WHAT — **Shows the screen's only button; its action sets the flag to true.**
WHERE — In the else branch, at the bottom of the stack.
WHEN — The closure runs on the main thread when the person taps.
WHY — Apple asks that a priming screen offer one button that leads to the system alert; a real app would request permission here.

```
16  .buttonStyle(.borderedProminent)
```

WHO — SwiftUI's buttonStyle(_:) modifier, applied to the Continue button.
WHAT — **Draws the button filled with the accent color.**
WHERE — Applied to the Button above.
WHEN — Each time the Continue button is drawn, which is only while continued is still false.
WHY — A prominent style marks Continue as the expected next step.

```
22  #Preview { LocationPrimer() }
```

WHO — Xcode's canvas, using the #Preview macro.
WHAT — **Registers a live preview that shows a LocationPrimer.**
WHERE — At file scope; preview code is not part of the shipped screen.
WHEN — While you edit in Xcode, and here in the simulated iPhone.
WHY — Previews let you check the wording and layout without running the whole app.

> **Try it**
>
> 1. Change the headline to describe a different benefit, such as a warning before sunset.
> 2. Replace .borderedProminent with .glassProminent to see the Liquid Glass style of iOS 26.
> 3. Add a second button labeled Not Now. Then explain why Apple's guidelines ask you to remove it again.

CODE · READ-ALONG

Send people to Settings

When access is denied, show what still works and offer one tap back to Settings. `UIApplication.openSettingsURLString` is a special link that opens your app's own page in the Settings app. `openURL` is a SwiftUI action that opens any link the system understands. `ContentUnavailableView` is SwiftUI's standard layout for an empty or unavailable screen.

```swift
import SwiftUI

struct LocationOffView: View {
    @Environment(\.openURL) private var openURL

    var body: some View {
        ContentUnavailableView {
            Label("Location Is Off", systemImage: "location")
        } description: {
            Text("You can still browse every trail.")
        } actions: {
            Button("Open Settings", action: openSettings)
                .buttonStyle(.borderedProminent)
        }
    }

    private func openSettings() {
        let link = UIApplication.openSettingsURLString
        guard let url = URL(string: link) else { return }
        openURL(url)
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module for LocationOffView and, on iOS, UIKit's UIApplication with it.
- **WHAT** **Loads SwiftUI, which also brings UIKit's UIApplication into this file on iOS.**
- **WHERE** The first line of the file.
- **WHEN** At compile time, before any line of this recovery screen runs; the openSettingsURLString constant must be known before the file builds.
- **WHY** The view, the environment action and the Settings link constant all come from these frameworks.

3 `struct LocationOffView: View {`

- **WHO** Your LocationOffView struct, the screen Trailhead shows when location is denied.
- **WHAT** **Declares the screen Trailhead shows in place of the nearby list when location is denied.**
- **WHERE** At file scope, as a SwiftUI view type.
- **WHEN** Shown when the status check lands in the denied case.
- **WHY** A dedicated view keeps the recovery design in one place, reusable wherever location is missing.

4 `@Environment(\.openURL) private var openURL`

- **WHO** SwiftUI's environment, the set of values it hands every view.
- **WHAT** **Reads the system's open-a-link action and stores it in a property named openURL.**
- **WHERE** A property of this view, filled in by SwiftUI.
- **WHEN** Each time SwiftUI creates the view.
- **WHY** Using the environment action keeps the view testable and works the same in every SwiftUI container.

```
6  var body: some View {
```

WHO SwiftUI, which calls body to learn what to draw.
WHAT **Declares the content of the recovery screen.**
WHERE The required body property inside LocationOffView, between the openURL environment property and the openSettings() helper.
WHEN When the view appears and whenever its inputs change.
WHY All visible content of a SwiftUI view comes from body.

```
7  ContentUnavailableView {
```

WHO SwiftUI's standard empty-state view.
WHAT **Starts a centered layout with a label, a description and actions, styled like Apple's own apps.**
WHERE The root of the body; it fills the available space.
WHEN Every time body runs.
WHY People recognize this layout from built-in apps, so they understand the message instantly.

```
8  Label("Location Is Off", systemImage: "location")
```

WHO SwiftUI, drawing a title with an icon.
WHAT **Names the problem in a few words next to the location symbol.**
WHERE In the label slot at the top of the empty-state view.
WHEN Every time body runs.
WHY A short, plain title tells people what changed before they read the details.

```
9  } description: {
```

WHO Swift's multiple trailing closure syntax.
WHAT **Opens the second closure, which supplies the description text.**
WHERE Between the label and actions closures of ContentUnavailableView.
WHEN Evaluated with the rest of body.
WHY Labeled trailing closures keep each part of the view readable without nested parentheses.

```
10 Text("You can still browse every trail.")
```

WHO SwiftUI's Text view, filling the description slot of ContentUnavailableView.
WHAT **Reassures people that the app still works, just without distance sorting.**
WHERE In the description slot, under the title.
WHEN Every time body runs.
WHY Degrading gracefully, instead of blocking the app, keeps people around long enough to change their mind.

```
11 } actions: {
```

WHO Swift's multiple trailing closure syntax.
WHAT **Opens the third closure, which holds the buttons.**
WHERE The last part of ContentUnavailableView.
WHEN Evaluated with the rest of body.
WHY Actions sit below the explanation, so people read first and act second.

```
12 Button("Open Settings", action: openSettings)
```

WHO SwiftUI, and the person tapping.
WHAT **Shows a button whose action is the openSettings method defined below.**
WHERE In the actions slot of the empty-state view.
WHEN The method runs on the main thread when tapped.
WHY Passing a method by name keeps body short and puts the link logic in one testable place.

```
13  .buttonStyle(.borderedProminent)
```

WHO — SwiftUI's buttonStyle(_:) modifier, applied to the Open Settings button in the actions slot.
WHAT — **Fills the button with the accent color.**
WHERE — Applied to the Open Settings button.
WHEN — Each time the recovery screen is drawn, so the button is prominent from the first frame.
WHY — This is the one action that fixes the problem, so it deserves emphasis.

```
17  private func openSettings() {
```

WHO — Your LocationOffView struct, declaring the private openSettings() helper that the button calls.
WHAT — **Declares the helper that builds the Settings link and opens it.**
WHERE — A private method of LocationOffView, visible only inside this type.
WHEN — Each time the person taps Open Settings.
WHY — Keeping it private shows that nothing outside the view should trigger it.

```
18      let link = UIApplication.openSettingsURLString
```

WHO — UIKit, which provides the constant.
WHAT — **Gets the special string that points at this app's page in Settings.**
WHERE — A local constant inside openSettings.
WHEN — On each tap, just before opening.
WHY — Apple supplies this constant so you never hard-code a private Settings path that could break.

```
19      guard let url = URL(string: link) else { return }
```

WHO — Foundation's URL(string:) initializer, combined with Swift's guard statement in openSettings().
WHAT — **Turns the string into a URL value, or exits quietly if that fails.**
WHERE — Inside openSettings, after the line above.
WHEN — On each tap of Open Settings, right after the link constant is read and before openURL is called.
WHY — URL(string:) returns an optional, and guard handles the failure without crashing.

```
20      openURL(url)
```

WHO — iOS, through SwiftUI's open-URL action.
WHAT — **Leaves your app and opens its page in the Settings app.**
WHERE — Runs in your app; Settings then opens as a separate app.
WHEN — Immediately after the tap.
WHY — Only Settings can reverse a denial, so the shortest path there is the kindest design.

> **Try it**
>
> 1. In Xcode, run the app in the iOS Simulator, deny location, then tap Open Settings. Which Settings page opens?
> 2. Rewrite the description so it says what the person gains by turning location back on.

CONCEPT

Privacy manifests and App Store privacy labels

Permission alerts cover the device's senses; Apple also expects openness about *data*. A **privacy manifest** is a file named `PrivacyInfo.xcprivacy` inside your app. It lists the data your code collects, any domains used for tracking, and why you call certain **required-reason APIs**: APIs that could fingerprint a device, such as `UserDefaults`, file timestamps and free disk space. Third-party SDKs ship their own manifests, and Xcode combines them into a privacy report when you archive the app.

App Privacy details — The "nutrition label" on your App Store page, filled in through App Store Connect: what data you collect and whether it is linked to the person.

Tracking	Linking data from your app with data from other companies' apps or websites, for advertising or data brokers.
App Tracking Transparency	Before tracking, add NSUserTrackingUsageDescription and call ATTrackingManager to ask. Most apps never need it.

> **Labels must match reality**
>
> You write the labels yourself, and Apple expects them to match what the app and every SDK inside it actually do. Review them whenever you add an analytics or advertising library.

CHECK YOURSELF

Check your understanding

1. Trailhead calls `requestWhenInUseAuthorization()`, but no alert ever appears and nothing crashes. The status still reads `.notDetermined`. What is the most likely cause?
   A. The person chose Don't Allow during an earlier launch of the app.
   B. Info.plist has no NSLocationWhenInUseUsageDescription key.
   C. The request must be made from a background thread, not the main one.
   D. Location alerts only appear after the phone restarts.
2. A person chose Don't Allow last week. Today they tap *Show trails near me*. What should the app do?
   A. Call requestWhenInUseAuthorization() again so the alert reappears.
   B. Close the screen, because the feature cannot work.
   C. Keep listing trails alphabetically and offer an Open Settings button.
   D. Show a custom alert that looks like the system one, with an Allow button.
3. A parent uses Screen Time to block Location Services on a child's iPhone. The child taps the feature. Which case of the `switch` runs?
   A. .restricted
   B. .denied
   C. .notDetermined
   D. @unknown default
4. Which priming screen follows Apple's guidelines?
   A. A screen at first launch that lists every permission the app might ever need.
   B. A screen with Continue and Not Now buttons, so people who decline never see the system alert.
   C. No screen at all, even when the reason is unclear, because the system alert always says enough.
   D. A screen after the tap that explains the benefit, with one Continue button.

Answers: Permission and Privacy

1. **B. Info.plist has no NSLocationWhenInUseUsageDescription key.** Without the usage description, iOS silently ignores a location request, so the status never leaves `.notDetermined`. A previous Don't Allow would also explain the silence, but then the status would read `.denied`, not `.notDetermined`.
2. **C. Keep listing trails alphabetically and offer an Open Settings button.** A denied app keeps working in a reduced way and offers the only real fix: Settings. Asking again feels natural, but iOS shows its alert once per install, so the call does nothing. A fake system alert is deceptive and against Apple's rules.
3. **A. .restricted** Screen Time and device management produce `.restricted`: access is blocked by someone other than the person using the phone. `.denied` looks close, but it means the person said no themselves and could change it in Settings.
4. **D. A screen after the tap that explains the benefit, with one Continue button.** Priming belongs right after the action that needs the resource, explains the benefit, and offers a single button that leads to the system alert. The Not Now version seems considerate because it saves the one-time alert, but Apple asks for one button only, since extra choices can feel manipulative.

Core Location, the Modern Way

CONCEPT

How a phone knows where it is

Core Location is Apple's framework for position, altitude and direction. It blends several sources: GPS satellites outdoors, nearby Wi-Fi networks and cell towers in cities, and motion sensors that fill the gaps between fixes. You never pick a source. You describe what you need, and Core Location chooses.

Each answer is a `CLLocation`: a value describing one position at one moment.

coordinate	A CLLocationCoordinate2D holding latitude (degrees north or south of the equator) and longitude (degrees east or west of Greenwich, England).
horizontalAccuracy	The radius, in meters, of the circle the true position is likely inside. Smaller is better; a negative value means the position is invalid.
altitude, speed, course	Height in meters, speed in meters per second, and direction of travel in degrees. A negative speed or course means unknown.
timestamp	When the fix was measured. An old timestamp means you received a cached position.

> **Degrees are not meters**
>
> A degree of latitude is about 111 km everywhere, but a degree of longitude shrinks toward the poles. That is why you measure distance with an API instead of subtracting coordinates.

CONCEPT

Updates arrive over time

A position is not one answer; it changes as people walk. Swift models this with an **AsyncSequence**: a sequence whose next value arrives later. You read it with `for try await`, and the loop body runs once per value, pausing in between without freezing the screen.

`CLLocationUpdate.liveUpdates()` is that sequence for location. Each `CLLocationUpdate` carries an optional `location` plus flags that explain any gap. A `CLServiceSession` states your goal, such as `.whenInUse`. While you hold the session, iOS shows the permission alert when needed and keeps your interest in location alive.

authorizationDenied	The person said no, so location is nil.
locationUnavailable	No position can be found right now.
accuracyLimited	The person chose approximate location.
stationary	The device stopped moving, so updates pause.

CODE · READ-ALONG

Stream locations into an @Observable model

This model owns Trailhead's location stream. `@Observable` makes SwiftUI redraw any view that reads `location` or `status`. A view starts it with `.task { try? await model.start() }`; SwiftUI cancels that

task when the view disappears, which ends the loop. Pick a state in the simulation beside the code to see which lines run.

```swift
import CoreLocation
import Observation

@Observable @MainActor
final class LocationModel {
    var location: CLLocation?
    var status = "Starting"
    private var session: CLServiceSession?

    func start() async throws {
        session = CLServiceSession(authorization: .whenInUse)
        for try await update in CLLocationUpdate.liveUpdates() {
            status = "Live"
            if let fix = update.location { location = fix }
            if update.accuracyLimited { status = "Approximate" }
            if update.stationary { status = "Holding still" }
            if update.locationUnavailable { status = "No signal" }
            if update.authorizationDenied { status = "Location off" }
        }
    }
    func stop() { session?.invalidate(); session = nil }
}
```

Line by line

1 `import CoreLocation`

- **WHO** The Swift compiler, resolving the CoreLocation module so LocationModel can use CLLocation, CLServiceSession and CLLocationUpdate.
- **WHAT** **Makes CLLocation, CLServiceSession and CLLocationUpdate available in this file.**
- **WHERE** At the top of the file; it covers this file only.
- **WHEN** At compile time, before any line of this model file runs; nothing is imported at run time.
- **WHY** Location types live in the Core Location framework, which every file must import to use them.

2 `import Observation`

- **WHO** The Swift compiler, resolving the Observation module that defines the @Observable macro used on LocationModel.
- **WHAT** **Loads the Observation framework, home of the @Observable macro.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before the @Observable macro on the class below can expand.
- **WHY** This file does not import SwiftUI, which would bring Observation along, so it imports it directly.

4 `@Observable @MainActor`

- **WHO** @Observable is a macro that rewrites the class; @MainActor is Swift concurrency's main-thread actor.
- **WHAT** **Makes every stored property trackable by SwiftUI, and confines the class to the main thread.**
- **WHERE** On the LocationModel declaration below, covering the whole class.
- **WHEN** The macro expands at compile time, and the compiler checks actor isolation at the same time; at run time nothing is re-checked, the runtime only hops to the main actor when a call arrives from elsewhere.
- **WHY** Views read these properties to draw, so changes must happen on the main thread and must trigger redraws.

```
5  final class LocationModel {
```

WHO — Your LocationModel class, the @Observable object that owns Trailhead's location stream.
WHAT — **Declares a reference type that one screen, or several, can share.**
WHERE — At file scope; the instance lives on the heap.
WHEN — A view usually creates it once with @State and keeps it while on screen.
WHY — A class gives every view the same model instance, and final rules out subclass surprises.

```
6  var location: CLLocation?
```

WHO — Your model, written by the loop below and read by views.
WHAT — **Stores the most recent position, or nil before the first fix arrives.**
WHERE — A stored property of LocationModel, observed by SwiftUI.
WHEN — Updated each time a new location arrives.
WHY — Optional is honest: at launch, or after a denial, there may be no position at all.

```
7  var status = "Starting"
```

WHO — Your LocationModel class, declaring the status property that the loop below rewrites and the screen reads.
WHAT — **Holds a short, human-readable state for the screen to show.**
WHERE — A stored property of LocationModel; Swift infers the type String.
WHEN — Rewritten on every loop iteration.
WHY — Turning flags into a message in the model keeps views simple and the logic testable.

```
8  private var session: CLServiceSession?
```

WHO — Your model holds it; Core Location reads it to know what you need.
WHAT — **Keeps the service session alive between start and stop.**
WHERE — A private stored property, invisible outside the class.
WHEN — Set in start(), cleared in stop().
WHY — A session works only while something holds a reference to it; storing it here controls exactly how long that is.

```
10 func start() async throws {
```

WHO — Your code, called from a SwiftUI .task.
WHAT — **Declares a method that can pause while waiting for updates and can throw an error.**
WHERE — On the main actor, inside the task that SwiftUI created for the view.
WHEN — When the view appears; it keeps running until the loop ends or the task is cancelled.
WHY — async lets it wait without blocking the screen; throws passes stream errors up to the caller.

```
11 session = CLServiceSession(authorization: .whenInUse)
```

WHO — Core Location, which also shows the permission alert if needed.
WHAT — **Declares that this feature needs when-in-use authorization and starts the session.**
WHERE — Stored in the model's session property.
WHEN — Once, right before updates begin.
WHY — The session replaces manual request calls: iOS asks at the right moment and keeps your interest alive while you hold it.

```
12 for try await update in CLLocationUpdate.liveUpdates() {
```

WHO — Core Location produces the updates; Swift concurrency drives the loop.
WHAT — **Waits for each CLLocationUpdate and runs the body once per update.**
WHERE — On the main actor; between updates the method is suspended, not blocking anything.
WHEN — Every time Core Location has something to report, for as long as the task runs.
WHY — An async loop reads like ordinary code, with no delegate callbacks to wire up.

13 `status = "Live"`

- WHO — Your LocationModel, inside the for try await loop in start(), writing to its own status property.
- WHAT — **Resets the message to the normal state before checking the flags.**
- WHERE — First line of the loop body.
- WHEN — At the start of each iteration.
- WHY — Without the reset, an old message such as "Location off" would stick after the problem is fixed.

14 `if let fix = update.location { location = fix }`

- WHO — Your model, unwrapping a value from Core Location.
- WHAT — **Saves the new position, but only if this update actually carries one.**
- WHERE — Inside the loop body.
- WHEN — On updates with a position; diagnostic-only updates skip it.
- WHY — update.location is optional, and keeping the last good fix is better than blanking the screen.

15 `if update.accuracyLimited { status = "Approximate" }`

- WHO — Core Location sets the flag; your model reacts.
- WHAT — **Marks the position as approximate when the person turned off Precise Location.**
- WHERE — Inside the loop body.
- WHEN — On every update while approximate location is in effect.
- WHY — The position can be kilometers off, so the screen should not promise exact distances.

16 `if update.stationary { status = "Holding still" }`

- WHO — Core Location, which detects that the device stopped moving.
- WHAT — **Tells the screen that updates are paused because nothing is changing.**
- WHERE — Inside the loop body.
- WHEN — When the device rests; updates resume automatically once it moves.
- WHY — Pausing saves battery, and explaining the pause stops people wondering why nothing changes.

17 `if update.locationUnavailable { status = "No signal" }`

- WHO — Core Location, reporting a temporary failure.
- WHAT — **Shows that no position can be determined right now.**
- WHERE — Inside the loop body.
- WHEN — In deep canyons, tunnels or buildings, until a signal returns.
- WHY — It is not an error to throw; the loop keeps waiting, so a message is the right response.

18 `if update.authorizationDenied { status = "Location off" }`

- WHO — Core Location, reporting the person's permission choice.
- WHAT — **Shows that location access is off for this app.**
- WHERE — Last check in the loop body, so it wins over the others.
- WHEN — After Don't Allow, or when location is switched off in Settings.
- WHY — The sequence keeps running, so if the person turns access back on, real positions flow again.

21 `func stop() { session?.invalidate(); session = nil }`

- WHO — Your code, called when the feature closes.
- WHAT — **Ends the session and drops the model's reference to it.**
- WHERE — An instance method of LocationModel, on the main actor.
- WHEN — When the person leaves the map, or the app no longer needs location.
- WHY — Releasing the session tells iOS you are done, which ends the battery cost and the location indicator.

Try it

1. In the simulation, choose accuracyLimited. Which lines run, and what does the person see?
2. Imagine deleting the line status = "Live". After the person turns location back on in Settings, what would the screen still say?
3. In Xcode, add a property that stores update.location?.horizontalAccuracy and show it on screen.

CODE · READ-ALONG

Distance between two places

`distance(from:)` returns the distance in meters between two `CLLocation` values, measured along the curved surface of the Earth rather than through it. `Measurement` then turns raw meters into text in the units people use where they live. The simulation beside the code computes the same kind of distance with real Swift compiled to WebAssembly; drag the You pin and watch the numbers change.

```swift
import CoreLocation

let empireState = CLLocation(latitude: 40.7484,
                             longitude: -73.9857)
let timesSquare = CLLocation(latitude: 40.7580,
                             longitude: -73.9855)

let meters = empireState.distance(from: timesSquare)
print(meters) // about 1066

let walk = Measurement(value: meters, unit: UnitLength.meters)
let text = walk.formatted(.measurement(width: .abbreviated,
                                       usage: .road))
print(text) // "0.66 mi" in the US
```

Line by line

1 `import CoreLocation`

- WHO — The Swift compiler, resolving the CoreLocation module so this file can build CLLocation values and call distance(from:).
- WHAT — **Makes CLLocation and its distance method available, along with Foundation's Measurement.**
- WHERE — At the top of the file.
- WHEN — At compile time, before the empireState constant is created; Foundation's Measurement arrives through the same import.
- WHY — Core Location also re-exports Foundation, so one import covers both.

3 `let empireState = CLLocation(latitude: 40.7484,`

- WHO — Your code, creating a value with Core Location's initializer.
- WHAT — **Starts building a location for the Empire State Building from its latitude in degrees.**
- WHERE — A constant at file scope.
- WHEN — Once, when this code runs.
- WHY — Building a CLLocation from known coordinates lets you measure against landmarks without any sensor.

```
4   longitude: -73.9857)
```

WHO The same initializer call.
WHAT **Supplies the longitude; negative means west of Greenwich.**
WHERE The continuation of the line above, aligned for readability.
WHEN Together with the line above.
WHY Wrapping long calls keeps each line short enough to read without scrolling.

```
5   let timesSquare = CLLocation(latitude: 40.7580,
```

WHO Your file-scope constant timesSquare, built with CLLocation's init(latitude:longitude:).
WHAT **Starts a second location, for Times Square, about ten blocks north.**
WHERE A constant at file scope.
WHEN Once, when this code runs.
WHY Two values are all distance(from:) needs; no permission is involved when you supply the coordinates yourself.

```
6   longitude: -73.9855)
```

WHO The same initializer call.
WHAT **Supplies the longitude, almost the same as the Empire State Building's.**
WHERE The continuation of the line above.
WHEN Together with the line above.
WHY Nearly equal longitudes mean the two places lie almost due north and south of each other.

```
8   let meters = empireState.distance(from: timesSquare)
```

WHO Core Location, doing the math.
WHAT **Computes the surface distance between the two places as a Double in meters.**
WHERE Runs on the device, with no network or sensor involved.
WHEN Immediately; the call is fast and synchronous.
WHY The Earth is curved, so subtracting degrees would be wrong; the API does the geometry for you.

```
9   print(meters) // about 1066
```

WHO Swift's print function, writing the Double that distance(from:) returned.
WHAT **Writes the raw number to the console, roughly one kilometer.**
WHERE The Xcode console when you run this.
WHEN Right after the distance is computed.
WHY Printing the raw value first shows what the API returns before any formatting.

```
11  let walk = Measurement(value: meters, unit: UnitLength.meters)
```

WHO Foundation's Measurement initializer, pairing the meters Double with UnitLength.meters.
WHAT **Wraps the number together with its unit, meters.**
WHERE A constant at file scope.
WHEN After the distance is known.
WHY A number with a unit can be converted and formatted safely; a bare Double cannot.

```
12  let text = walk.formatted(.measurement(width: .abbreviated,
```

WHO Foundation's formatted(_:) method on Measurement, using a measurement format style.
WHAT **Starts formatting the measurement with short unit names such as km or mi.**
WHERE A constant at file scope; the call continues on the next line.
WHEN Right after walk is created, as the second-to-last statement; the call finishes on the next line.
WHY Formatting follows the person's region settings, so you never hard-code units or decimal marks.

```
13  usage: .road))
```

WHO — The same formatting call.
WHAT — **Asks for units people use for road distances in their region.**
WHERE — The continuation of the line above.
WHEN — Together with the line above.
WHY — With .road, people in the US see miles and people in most other countries see kilometers.

```
14  print(text) // "0.66 mi" in the US
```

WHO — Swift's print function, writing the localized string that formatted(_:) produced.
WHAT — **Shows the friendly, localized result.**
WHERE — The Xcode console, as the second and last line of output after the raw meters value.
WHEN — As the final statement of the example, once text has been formatted for the person's region.
WHY — This is the string you would put on screen, ready for any language and region.

Try it

1. In the simulation, drag the You pin close to Times Square. Why does the distance get small but rarely reach exactly zero?
2. In Xcode, swap the two locations in distance(from:). Does the result change?
3. Change .road to .person and run it again. What changes in the output?

INTERACTIVE

Measure the world with real Swift

Every distance and bearing in this lab is computed by real Swift functions compiled to WebAssembly, using the same kind of great-circle math as `distance(from:)`. Drag the You pin or use the arrow keys, and pick a landmark to draw its route. Try Use my location: your browser asks first, just as iOS does, and the position never leaves this page. Then turn on live updates and watch each value arrive.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Turn coordinates into a place name

Reverse geocoding turns coordinates into a readable address. In iOS 26 and later you use MapKit's `MKReverseGeocodingRequest`; the older `CLGeocoder` is deprecated. The request goes to Apple's servers, so it needs a network connection, and you should make it once per place, not on every location update.

```swift
import MapKit

func placeName(for point: CLLocation) async -> String {
    let request = MKReverseGeocodingRequest(location: point)
    guard let request else { return "Unknown place" }
    do {
        let items = try await request.mapItems
        let address = items.first?.address
        return address?.shortAddress ?? "Unknown place"
    } catch {
        return "Unknown place"
    }
}

let spot = CLLocation(latitude: 40.7484, longitude: -73.9857)
print(await placeName(for: spot))
```

Line by line

1 `import MapKit`

- **WHO** The Swift compiler, resolving the MapKit module so placeName(for:) can use MKReverseGeocodingRequest.
- **WHAT** **Loads MapKit, which provides geocoding requests and also brings in Core Location.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before any line of this file runs; MapKit also brings CLLocation into scope.
- **WHY** Since iOS 26, address lookups live in MapKit next to the rest of Apple's map data.

3 `func placeName(for point: CLLocation) async -> String {`

- **WHO** Your file-scope placeName(for:) function, declared by you and awaited by the call at the bottom.
- **WHAT** **Declares a function that takes a location and eventually returns a readable place name.**
- **WHERE** At file scope, callable from any async context.
- **WHEN** Whenever the app needs a name, such as when the person saves a trail start.
- **WHY** async lets it wait for the server without blocking, and returning a String keeps callers simple.

4 `let request = MKReverseGeocodingRequest(location: point)`

- **WHO** MapKit, Apple's map framework.
- **WHAT** **Creates a lookup request for the coordinate, or nil if the coordinate is invalid.**
- **WHERE** A local constant inside placeName.
- **WHEN** At the start of each call.
- **WHY** The initializer is failable, so invalid input never reaches the server.

5 `guard let request else { return "Unknown place" }`

- **WHO** Swift's guard statement in placeName(for:), unwrapping the optional MKReverseGeocodingRequest.
- **WHAT** **Unwraps the optional request, or returns a fallback name if it is nil.**
- **WHERE** Inside placeName, right after the request is made.
- **WHEN** Immediately; the rest of the function runs only with a real request.
- **WHY** Exiting early keeps the happy path flat and readable.

6 `do {`

- **WHO** Swift's do-catch error handling, wrapping the throwing mapItems lookup.
- **WHAT** **Starts a block whose throwing calls are caught below.**
- **WHERE** Inside placeName(for:), after the guard that confirmed the request exists.
- **WHEN** After the request is ready.
- **WHY** Network lookups can fail, and a name lookup should never crash the app.

```
7  let items = try await request.mapItems
```

WHO MapKit, which asks Apple's servers.
WHAT **Waits for the matching map items, which hold names and addresses.**
WHERE The request runs on the network; your function is suspended meanwhile.
WHEN When the server replies, usually within a moment.
WHY try marks a possible failure; await marks the pause while the server works.

```
8  let address = items.first?.address
```

WHO Your code, reading MapKit's result.
WHAT **Takes the address of the best match, or nil if there were no results.**
WHERE Inside the do block.
WHEN After the server replies.
WHY Optional chaining with ?. handles an empty list without crashing.

```
9  return address?.shortAddress ?? "Unknown place"
```

WHO Your placeName(for:) function, reading shortAddress from MapKit's address value with optional chaining.
WHAT **Returns a compact address, or the fallback when none exists.**
WHERE Inside the do block.
WHEN As soon as the address is read.
WHY shortAddress is sized for a label on a phone; ?? supplies a default in one step.

```
10 } catch {
```

WHO Swift's do-catch error handling in placeName(for:), receiving any error that request.mapItems threw.
WHAT **Catches any error thrown by the lookup, such as no network.**
WHERE Right after the do block.
WHEN Only if mapItems threw.
WHY Catching here means callers always receive a String and never need their own error code.

```
11 return "Unknown place"
```

WHO Your placeName(for:) function, returning its fallback from inside the catch block.
WHAT **Returns the same fallback text used for invalid coordinates.**
WHERE Inside the catch block.
WHEN After a failed lookup.
WHY People see a calm placeholder instead of an error message about servers.

```
15 let spot = CLLocation(latitude: 40.7484, longitude: -73.9857)
```

WHO Your file-scope constant spot, built with CLLocation's init(latitude:longitude:) as test input.
WHAT **Creates the Empire State Building's location as test input.**
WHERE A constant at file scope.
WHEN When this code runs.
WHY A famous landmark makes it easy to check that the returned address is right.

```
16 print(await placeName(for: spot))
```

WHO Swift concurrency runs the call; print shows the result.
WHAT **Looks up the place name and prints it.**
WHERE Top-level code, which may use await directly.
WHEN After the server replies.
WHY Calling the function once shows the full round trip from coordinates to text.

> **Try it**
>
> 1. In Xcode, also print items.first?.name. What does it hold for a famous landmark?
> 2. Pass a coordinate in the middle of an ocean. Which return statement produces the result?

CONCEPT

Accuracy, battery and the background

Precision costs power, and GPS is the hungriest source. Ask only for what your feature needs. Pass a **live configuration** to `liveUpdates(_:)` to describe the activity, and Core Location tunes accuracy and frequency to match.

Configurations	.default, .automotiveNavigation, .otherNavigation, .fitness and .airborne.
Approximate location	People can turn off Precise Location. Updates then set accuracyLimited, and positions can be kilometers off. Design for it.
Background updates	Add the Location updates background mode and hold a CLBackgroundActivitySession, started while the app is on screen. iOS shows an indicator while it runs.
CLMonitor	Watches conditions, such as entering a circular area, without streaming every position. Relaunching a closed app this way needs Always authorization.

> **Stop when you are done**
>
> Cancel the task that loops over updates and invalidate your session as soon as the feature closes. The location indicator disappears, and so does the battery drain.

CHECK YOURSELF

Check your understanding

1. An update arrives with `horizontalAccuracy` equal to 65. What does that mean?
   A. The phone moved 65 meters since the previous update arrived.
   B. The true position is probably within 65 meters of the coordinate.
   C. The coordinate is accurate to 65 decimal places in each axis.
   D. The person allowed 65 percent precision in the alert.

2. Using the `LocationModel` loop from this lesson, an update arrives with `authorizationDenied` set and no location. Predict the state after that iteration.
   A. status is "Location off"; location keeps its old value.
   B. status is "Live"; location keeps its old value.
   C. status is "Location off"; location is set back to nil.
   D. The loop throws an error, and the method ends there.

3. A teammate calls the reverse geocoding function inside the `liveUpdates()` loop, so the address label is always fresh. What is the problem?
   A. MKReverseGeocodingRequest only works with Always authorization, so it fails.
   B. Nothing; this is the recommended pattern for a fresh address label.
   C. It sends a network request on nearly every update and may be throttled.
   D. Reverse geocoding can only run in the background, so it never completes.

4. A running app must keep recording a route while the phone is locked in a pocket. What does it need?
   A. Nothing extra; liveUpdates() keeps running after the screen locks.
   B. Only the NSLocationAlwaysAndWhenInUseUsageDescription key and an Always request.
   C. A timer that wakes the app every second to read the location.
   D. The Location updates background mode plus a CLBackgroundActivitySession.

Answers: Core Location, the Modern Way

1. **B. The true position is probably within 65 meters of the coordinate.** `horizontalAccuracy` is the radius, in meters, of the circle around the coordinate where the true position likely lies. Distance moved is a common misreading; that number comes from comparing two locations with `distance(from:)`.
2. **A. status is "Location off"; location keeps its old value.** The body sets "Live", skips the `if let` because location is nil, then the last check sets "Location off". Nothing ever assigns nil, so the old position stays. The loop does not throw on denial; it keeps running so updates can resume.
3. **C. It sends a network request on nearly every update and may be throttled.** Each lookup is a trip to Apple's servers. Look up a place once, or only after the person has moved a meaningful distance. "Nothing" seems right because it works in a quick test, but it fails under real use and drains the battery.
4. **D. The Location updates background mode plus a CLBackgroundActivitySession.** The background mode allows location work after the app leaves the screen, and the background activity session keeps it running with a visible indicator. The Always key and request change only what the app may ask for; a usage description supplies alert text, and neither keeps your app running once it leaves the screen.

MapKit for SwiftUI

CONCEPT

A map is just another view

MapKit is Apple's mapping framework: the same maps, search and directions that power the Maps app. In SwiftUI, `Map` is an ordinary view. You place it in a layout like a `Text`, and you list what appears on it inside its trailing closure, called the *map content builder*.

Which part of the world is visible is a separate value, a `MapCameraPosition`. It can be `.automatic` (fit all the content), `.region` (a center plus a span), `.camera` (a 3D viewpoint with distance, heading and pitch), `.userLocation(fallback:)` or `.item` (one place).

Marker	A balloon pin with a title, an optional symbol and a tint color.
Annotation	Any SwiftUI view you pin to a coordinate, such as a photo or a badge.
UserAnnotation	The blue dot that shows where the person holding the phone is.
MapCircle and MapPolyline	Shapes drawn on the map: a circle with a radius in meters, or a line through points or along a route.

> **The blue dot still needs permission**
>
> Map styles such as `.standard`, `.imagery` and `.hybrid`, and the buttons you add with `.mapControls`, need nothing special. `UserAnnotation` only appears after the app has location authorization, exactly as in the earlier lessons.

CODE · READ-ALONG

Markers and annotations

A trail map with one standard pin, one custom pin and the user's blue dot. The coordinates `.pier39` and `.coitTower` are constants defined in a small extension that is not shown. Use the map simulation beside the code to drop markers and watch the same calls appear.

```swift
import SwiftUI
import MapKit

struct TrailMapView: View {
    @State private var position: MapCameraPosition = .automatic
    var body: some View {
        Map(position: $position) {
            Marker("Pier 39", systemImage: "figure.walk",
                   coordinate: .pier39)
            Annotation("Coit Tower", coordinate: .coitTower) {
                Image(systemName: "binoculars.fill").padding(6)
                    .background(.white, in: .circle)
            }
            UserAnnotation()
        }
        .mapStyle(.standard(elevation: .realistic))
        .mapControls {
            MapUserLocationButton()
            MapCompass()
        }
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the app.
- **WHAT** **Makes SwiftUI available in this file: the View protocol, the @State property wrapper, Image and every view modifier used below.**
- **WHERE** The top of the file; an import only affects the file it appears in.
- **WHEN** At compile time, before any of this code runs.
- **WHY** Map is a SwiftUI view, so the file needs SwiftUI as well as MapKit.

2 `import MapKit`

- **WHO** The Swift compiler, resolving the MapKit module so TrailMapView can use Map, Marker, Annotation and UserAnnotation.
- **WHAT** **Loads Apple's mapping framework, which provides Map, Marker, Annotation, map styles and map controls.**
- **WHERE** File scope, right after the SwiftUI import.
- **WHEN** At compile time, before any line of this file runs; with SwiftUI already imported, the SwiftUI flavor of MapKit loads too.
- **WHY** When a file imports both SwiftUI and MapKit, Swift also loads the SwiftUI flavor of MapKit, which is what makes Map usable as a view.

4 `struct TrailMapView: View {`

- **WHO** Your code declares it; SwiftUI creates and draws it.
- **WHAT** **Declares a view type for the trail map screen that conforms to the View protocol.**
- **WHERE** A value type at file scope; SwiftUI places its instances in the view tree.
- **WHEN** Compiled once; an instance is made whenever a parent view's body includes TrailMapView().
- **WHY** Giving the map its own view keeps each screen small, reusable and easy to preview on its own.

```
5  @State private var position: MapCameraPosition = .automatic
```

WHO — SwiftUI, which owns the storage behind every @State property.

WHAT — **Remembers which part of the world the map shows, starting with automatic framing of all the content.**

WHERE — Inside TrailMapView; SwiftUI keeps the value outside the struct so it survives each body recomputation.

WHEN — Created when the view first appears, then updated whenever your code or the user's gestures move the map.

WHY — The map reads and writes this value through a binding, so pans and zooms flow back into your state.

```
6  var body: some View {
```

WHO — SwiftUI reads this property.

WHAT — **Describes the screen: one Map with its content and two modifiers.**

WHERE — The computed property that the View protocol requires, inside TrailMapView.

WHEN — When the view first appears and again whenever state it reads changes.

WHY — SwiftUI rebuilds this description cheaply and then updates only what changed on the real map.

```
7  Map(position: $position) {
```

WHO — MapKit's SwiftUI Map view.

WHAT — **Creates the interactive map, ties its camera to the position state and opens the map content builder.**

WHERE — The root of body, so it fills all the space its parent offers.

WHEN — Described every time body runs; the underlying map stays alive between runs.

WHY — The $ prefix passes a binding, a two-way connection, so the map can read your camera and also report the user's gestures.

```
8  Marker("Pier 39", systemImage: "figure.walk",
```

WHO — MapKit draws the balloon pin.

WHAT — **Adds a standard pin titled Pier 39 with a walking-figure symbol inside its balloon.**

WHERE — Inside the map content builder, so it lives on the map surface rather than in the normal layout.

WHEN — Drawn whenever the map is on screen and the coordinate is in view.

WHY — Marker gives the familiar Maps look with one line of code and handles selection and VoiceOver labels for you.

```
9  coordinate: .pier39)
```

WHO — Your code supplies the location; MapKit positions the pin.

WHAT — **Places the marker at the latitude and longitude stored in the pier39 constant.**

WHERE — The second line of the Marker initializer; .pier39 is a static CLLocationCoordinate2D in a hidden extension.

WHEN — Read whenever the map lays out its content.

WHY — Named coordinate constants keep raw numbers out of view code and are easy to reuse in previews and tests.

```
10 Annotation("Coit Tower", coordinate: .coitTower) {
```

WHO — MapKit hosts your SwiftUI view on the map.

WHAT — **Pins a custom view to the tower's coordinate and shows the title Coit Tower beneath it.**

WHERE — In the map content builder; the closure that follows is an ordinary SwiftUI view builder.

WHEN — Rendered while that coordinate is visible; it moves with the map as the user pans.

WHY — Choose Annotation when a plain balloon is not enough, for example a photo, a rating badge or a price tag.

```
11 Image(systemName: "binoculars.fill").padding(6)
```

WHO — SwiftUI renders the symbol.

WHAT — **Draws the binoculars SF Symbol, Apple's built-in icon set, with 6 points of space around it.**

WHERE — Inside the Annotation closure, so it appears on the map at the tower.

WHEN — Each time the annotation is drawn.

WHY — The padding leaves room for the circle behind it, so the icon does not touch the edge.

```
12  .background(.white, in: .circle)
```

WHO SwiftUI's background(_:in:) modifier, applied to the padded binoculars Image inside the Annotation.
WHAT **Paints a white circle behind the padded symbol.**
WHERE A modifier on the Image inside the annotation.
WHEN Applied whenever the annotation renders.
WHY A solid backdrop keeps the icon readable on top of busy streets or satellite imagery.

```
14  UserAnnotation()
```

WHO MapKit, working with Core Location.
WHAT **Shows the blue dot at the device's current location.**
WHERE In the map content builder, alongside the pins.
WHEN Only after the app has location authorization and the device has a location fix.
WHY Without permission the dot simply stays hidden, so the rest of the map keeps working.

```
16  .mapStyle(.standard(elevation: .realistic))
```

WHO MapKit, which renders the map tiles and 3D buildings.
WHAT **Uses the standard street map and draws terrain and buildings in 3D when the camera tilts.**
WHERE A modifier on Map, outside the content builder.
WHEN Applied on the first render and again whenever the style value changes.
WHY Style belongs to the whole map, so it is a modifier rather than content; .imagery and .hybrid swap in satellite views.

```
17  .mapControls {
```

WHO MapKit, which owns the floating map controls.
WHAT **Opens a builder that lists which system controls float over the map.**
WHERE A modifier on Map; the controls sit above the map surface.
WHEN Built along with the rest of body.
WHY Listing controls replaces the defaults, so you decide exactly which buttons appear.

```
18  MapUserLocationButton()
```

WHO MapKit, reacting when the user taps it.
WHAT **Adds the arrow button that recenters the map on the user's location.**
WHERE Inside mapControls; the system chooses its corner.
WHEN Visible while the map is on screen; it only finds the user once location is authorized.
WHY The system button already shows the right tracking state and moves the camera for you.

```
19  MapCompass()
```

WHO MapKit, which watches the map heading.
WHAT **Adds a compass that appears once the map is rotated and turns it back to north when tapped.**
WHERE Inside mapControls, next to the location button.
WHEN Shown only while the heading is not north.
WHY People who rotate the map with two fingers need a quick way back to north-up.

Try it

1. In the simulation beside the code, switch the tool to Annotation and tap the map. Which line appears in the code panel?
2. In Xcode, change .realistic to .flat, or .standard to .imagery, and compare the preview canvas.
3. Add .tint(.green) on a new line under the Marker. What changes on the pin?

CODE · READ-ALONG

Move the camera

The camera is just state. This map starts on a region around the pier, and a button flies to a tilted 3D `MapCamera`. Assigning a new `MapCameraPosition` inside `withAnimation` makes the map glide instead of jump. `onMapCameraChange` tells you where the user ended up after they pan or zoom.

```swift
import SwiftUI
import MapKit

struct CameraDemo: View {
    @State private var position = MapCameraPosition.region(
        MKCoordinateRegion(center: .pier39,
            latitudinalMeters: 800, longitudinalMeters: 800))
    var body: some View {
        Map(position: $position)
            .onMapCameraChange(frequency: .onEnd) { context in
                print(context.region.span.latitudeDelta)
            }
            .safeAreaInset(edge: .bottom) {
                Button("Tilt over the pier", action: tilt)
            }
    }
    func tilt() {
        let camera = MapCamera(centerCoordinate: .pier39,
            distance: 900, heading: 45, pitch: 60)
        withAnimation { position = .camera(camera) }
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so CameraDemo can use View, @State, Button and withAnimation.
- **WHAT** **Brings in SwiftUI views, state and the withAnimation function.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before any line of CameraDemo runs; the withAnimation call below is resolved during the build, never at run time.
- **WHY** The screen is a SwiftUI view, and the animated camera change uses SwiftUI's animation system.

2 `import MapKit`

- **WHO** The Swift compiler, resolving the MapKit module so CameraDemo can use Map, MapCameraPosition, MapCamera and MKCoordinateRegion.
- **WHAT** **Loads Map, MapCameraPosition, MapCamera and MKCoordinateRegion.**
- **WHERE** File scope, below the SwiftUI import.
- **WHEN** At compile time, before the position state can be declared with a MapCameraPosition value.
- **WHY** Every camera type on this screen comes from MapKit.

```
4  struct CameraDemo: View {
```

WHO Your code; SwiftUI creates and renders it.
WHAT **Declares a small screen that shows a map and one camera button.**
WHERE A struct at file scope.
WHEN Instantiated whenever a parent view includes CameraDemo().
WHY Keeping the camera experiment in its own view makes it easy to preview and reason about.

```
5  @State private var position = MapCameraPosition.region(
```

WHO SwiftUI stores the value; your code and the map both change it.
WHAT **Creates the camera state and starts it as a region rather than automatic framing.**
WHERE A @State property inside CameraDemo; Swift infers its type, MapCameraPosition, from the right side.
WHEN Initialized the first time the view appears; later assignments replace it.
WHY Starting from a known region means the first frame already shows the pier, with no jump after loading.

```
6  MKCoordinateRegion(center: .pier39,
```

WHO MapKit's MKCoordinateRegion initializer, called while the initial value of position is built.
WHAT **Describes a rectangle of the Earth centered on the pier coordinate.**
WHERE The argument to .region, still part of the property's initial value.
WHEN Evaluated once, when the state is first created.
WHY A region is the simplest way to say "show this area": a center point plus how much to show around it.

```
7  latitudinalMeters: 800, longitudinalMeters: 800))
```

WHO MapKit converts these distances into a span of degrees.
WHAT **Makes the region 800 meters tall and 800 meters wide.**
WHERE The remaining arguments of the MKCoordinateRegion initializer.
WHEN Computed once when the region is built.
WHY Meters are easier to reason about than degrees, which shrink in width as you move away from the equator.

```
8  var body: some View {
```

WHO SwiftUI, reading CameraDemo's body property to learn what to draw.
WHAT **Describes the map, a camera-change handler and a floating button.**
WHERE The required body property of CameraDemo.
WHEN On first appearance and whenever position changes.
WHY Because the map is bound to position, SwiftUI re-renders body and the map follows the new camera.

```
9  Map(position: $position)
```

WHO MapKit's SwiftUI Map view, created by CameraDemo's body with a binding to position.
WHAT **Shows a map with no pins whose camera is tied to the position state.**
WHERE The root view of body.
WHEN Described each time body runs.
WHY Map content is optional, so a bare map is fine when the camera is the whole lesson.

```
10 .onMapCameraChange(frequency: .onEnd) { context in
```

WHO MapKit calls this closure; you write what it does.
WHAT **Registers a handler that receives the camera details after the map moves.**
WHERE A modifier on Map; the context parameter carries the region, the camera and the visible rectangle.
WHEN Once each time a pan, zoom or animation finishes; .continuous would fire on every frame instead.
WHY .onEnd keeps the work small, which matters if you fetch data for the visible area.

```
11 print(context.region.span.latitudeDelta)
```

WHO Your closure, running on the main thread.
WHAT **Prints how many degrees of latitude the visible area now covers, a measure of the zoom level.**
WHERE Inside the camera-change handler; the output goes to the Xcode console.
WHEN After every finished camera move.
WHY Reading context.region is how you learn what the user is looking at, for example to search that area.

```
13 .safeAreaInset(edge: .bottom) {
```

WHO SwiftUI's safeAreaInset(edge:) modifier, applied to the Map in CameraDemo.
WHAT **Reserves space along the bottom edge and places the content of this closure there.**
WHERE A modifier on Map, so the map's own controls stay clear of the button.
WHEN Laid out with the rest of body.
WHY An inset keeps the button from covering map features, unlike a plain overlay.

```
14 Button("Tilt over the pier", action: tilt)
```

WHO SwiftUI draws the button; the user taps it.
WHAT **Shows a button that calls the tilt method below when tapped.**
WHERE Inside the bottom safe-area inset.
WHEN The action runs on the main thread each time the user taps.
WHY Passing a method name keeps the view short and gives the camera logic a readable name.

```
17 func tilt() {
```

WHO Your code, called by the button.
WHAT **Declares a method that moves the camera to a tilted 3D view.**
WHERE A method on CameraDemo; it can change position because @State storage is shared with the view.
WHEN Each time the button is tapped.
WHY Camera changes are just state changes, so a plain method is all you need.

```
18 let camera = MapCamera(centerCoordinate: .pier39,
```

WHO MapKit's MapCamera initializer, called inside tilt() when the button is tapped.
WHAT **Creates a viewpoint looking at the pier.**
WHERE A local constant inside tilt().
WHEN Built fresh on every tap.
WHY A camera, unlike a region, can tilt and rotate, which is what makes the 3D view possible.

```
19 distance: 900, heading: 45, pitch: 60)
```

WHO MapKit reads these values when it positions the camera.
WHAT **Places the camera 900 meters from the pier, facing northeast (45 degrees) and tilted 60 degrees from straight down.**
WHERE The remaining arguments of the MapCamera initializer.
WHEN Evaluated on each tap.
WHY Pitch shows buildings and terrain from the side, which works best with the .realistic elevation style.

```
20 withAnimation { position = .camera(camera) }
```

WHO SwiftUI animates; MapKit flies the map.
WHAT **Replaces the camera position with the new 3D camera inside an animation transaction.**
WHERE Inside tilt(); the change reaches Map through the binding.
WHEN Immediately on tap; the flight then plays over a fraction of a second.
WHY Without withAnimation the map would jump, which disorients people who lose track of where they are.

Try it

1. Add .buttonStyle(.glassProminent) under the Button so it floats on Liquid Glass.
2. Change pitch to 0 and heading to 0. Is that view different from the starting region?
3. Replace .camera(camera) with .automatic. What does the map frame when there is no content?

INTERACTIVE

Build a map by hand

This map runs inside a simulated iPhone, and the SwiftUI on the right always matches it. Pick a tool and tap the map to drop a `Marker` or an `Annotation`. Switch between `.standard`, `.imagery` and `.hybrid`, tilt to `.realistic` elevation, zoom and pan, then run the coffee search. Notice that each change is one small edit to the map content or one modifier. Finally press Reset camera and see how `.automatic` frames everything.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Search for places

`MKLocalSearch` asks Apple's servers for places that match plain words, such as "coffee", near a region. The results are `MKMapItem` values: a place with a name, an address and a location. Each one becomes a `Marker`, and `Map(selection:)` tells you which one the user tapped.

```swift
import SwiftUI
import MapKit

struct CoffeeSearchView: View {
    @State private var results: [MKMapItem] = []
    @State private var selected: MKMapItem?
    var body: some View {
        Map(selection: $selected) {
            ForEach(results, id: \.self) { Marker(item: $0) }
        }
        .task { await search(for: "coffee") }
    }
    func search(for text: String) async {
        let request = MKLocalSearch.Request()
        request.naturalLanguageQuery = text
        request.region = MKCoordinateRegion(center: .pier39,
            latitudinalMeters: 2000, longitudinalMeters: 2000)
        let search = MKLocalSearch(request: request)
        let response = try? await search.start()
        results = response?.mapItems ?? []
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so CoffeeSearchView can use View, @State, ForEach and the task modifier.
- **WHAT** **Brings in SwiftUI views, state and the task modifier.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before any line of CoffeeSearchView runs; the task modifier and ForEach are checked during the build.
- **WHY** The search results are displayed by a SwiftUI view.

2 `import MapKit`

- **WHO** The Swift compiler, resolving the MapKit module so CoffeeSearchView can use MKLocalSearch, MKMapItem, Map and Marker.
- **WHAT** **Loads MKLocalSearch, MKMapItem and the SwiftUI Map types.**
- **WHERE** The second line of the file, at file scope below the SwiftUI import, covering this file only.
- **WHEN** At compile time, before the results array of MKMapItem can be declared.
- **WHY** Place search is a MapKit service, not part of SwiftUI.

4 `struct CoffeeSearchView: View {`

- **WHO** Your code; SwiftUI renders it.
- **WHAT** **Declares a screen that searches for coffee and pins every result.**
- **WHERE** A struct at file scope.
- **WHEN** Created whenever a parent shows CoffeeSearchView().
- **WHY** One view that owns its results keeps the search and its display together.

5 `@State private var results: [MKMapItem] = []`

- **WHO** SwiftUI stores it; the search method fills it.
- **WHAT** **Holds the places the search found, starting empty.**
- **WHERE** View state inside CoffeeSearchView.
- **WHEN** Empty on first appearance, replaced when the search returns.
- **WHY** Because it is @State, assigning new results re-renders the map with new markers.

6 `@State private var selected: MKMapItem?`

- **WHO** SwiftUI and the map, which writes the tapped item into it.
- **WHAT** **Remembers which result the user tapped, or nil when nothing is selected.**
- **WHERE** View state next to results.
- **WHEN** Changes every time the user taps a marker or taps away.
- **WHY** Your code can show details or directions for whatever this holds.

7 `var body: some View {`

- **WHO** SwiftUI, reading CoffeeSearchView's body each time results or selected changes.
- **WHAT** **Describes the map and starts the search.**
- **WHERE** The required body property.
- **WHEN** On first appearance and whenever results or selected change.
- **WHY** The view is a function of its state: new results produce new markers automatically.

8 `Map(selection: $selected) {`

- **WHO** MapKit's SwiftUI Map view, created by CoffeeSearchView's body with a binding to selected.
- **WHAT** **Creates a map whose tapped marker is written into the selected state.**
- **WHERE** The root of body; without a position binding the camera starts in automatic mode.
- **WHEN** Described each time body runs.
- **WHY** Selection support turns pins into buttons with no gesture code of your own.

```
9  ForEach(results, id: \.self) { Marker(item: $0) }
```

WHO SwiftUI repeats the content; MapKit draws each pin.
WHAT **Adds one marker per result, using the place's own name and category styling.**
WHERE Inside the map content builder; id: \.self identifies each map item by the item itself.
WHEN Re-evaluated whenever results changes.
WHY Marker(item:) reads the name and icon from the MKMapItem, and it tags the pin so selection returns that item.

```
11 .task { await search(for: "coffee") }
```

WHO SwiftUI starts and owns the task.
WHAT **Runs the asynchronous search when the view appears.**
WHERE A modifier on Map; the work runs off the drawing path.
WHEN Once when the view appears; SwiftUI cancels it if the view goes away first.
WHY The task modifier ties async work to the view's lifetime, so nothing keeps running for a closed screen.

```
13 func search(for text: String) async {
```

WHO Your code, called from the task.
WHAT **Declares an asynchronous method that searches for any text.**
WHERE A method on CoffeeSearchView.
WHEN Runs whenever the task calls it.
WHY async lets it wait for the network without freezing the screen.

```
14 let request = MKLocalSearch.Request()
```

WHO MapKit's MKLocalSearch.Request initializer, called inside search(for:).
WHAT **Creates an empty search request that you configure on the next lines.**
WHERE A local constant inside search(for:).
WHEN At the start of each search(for:) call, which the task modifier makes once when the view appears.
WHY Separating the request from the search lets you set the query, region and result types first.

```
15 request.naturalLanguageQuery = text
```

WHO Your code fills it in; Apple's servers interpret it.
WHAT **Sets the plain-language words to look for, here "coffee".**
WHERE A property on the request object.
WHEN Before the search starts.
WHY Natural-language queries match names, categories and addresses, just like typing in the Maps app.

```
16 request.region = MKCoordinateRegion(center: .pier39,
```

WHO MapKit uses it to rank results.
WHAT **Tells the search to prefer places around the pier.**
WHERE The region property of the request.
WHEN Before the search starts.
WHY Without a region, results can come from far away; a region keeps them relevant to what the user sees.

```
17 latitudinalMeters: 2000, longitudinalMeters: 2000)
```

WHO MapKit, converting meters to degrees.
WHAT **Makes the search area about 2 kilometers on each side.**
WHERE The size arguments of the region.
WHEN Evaluated on each call.
WHY A walkable area suits a coffee search; a city-wide search would need a larger span.

```
18 let search = MKLocalSearch(request: request)
```

WHO	MapKit's MKLocalSearch initializer, wrapping the configured request inside search(for:).
WHAT	**Creates the search object that will talk to Apple's servers.**
WHERE	A local constant in the method.
WHEN	Created per call; it is used once.
WHY	Keeping it in a constant also lets you cancel it if the user types a new query.

```
19 let response = try? await search.start()
```

WHO	MapKit sends the request; Apple's servers reply over the network.
WHAT	**Starts the search, waits for the reply, and turns any failure into nil.**
WHERE	The await suspends this method; the main thread stays free to draw.
WHEN	Resumes when the network replies, usually within a second.
WHY	try? keeps the example short; a real app would catch the error and tell the user the search failed.

```
20 results = response?.mapItems ?? []
```

WHO	Your code, back on the main actor.
WHAT	**Stores the found places, or an empty list if the search failed.**
WHERE	Writes the results state of the view.
WHEN	Right after the search returns.
WHY	Updating state is all it takes: SwiftUI redraws the map with the new markers.

Try it

1. Change "coffee" to "bookstore" or "playground" and run the app in the iOS Simulator. How do the marker icons change?
2. Use the Search coffee button in the map simulation earlier in this lesson and compare its Marker lines with this code.
3. Add .onChange(of: selected) { print(selected?.name ?? "none") } to the Map and tap a few markers.

CODE · READ-ALONG

Directions and Look Around

`MKDirections` asks Apple's servers for a route between two `MKMapItem` places. Each `MKRoute` carries its path, `expectedTravelTime` (seconds) and `distance` (meters), and `MapPolyline` draws it. For a street-level photo preview, `LookAroundPreview(initialScene:)` shows a scene you fetch with `MKLookAroundSceneRequest`.

```swift
import SwiftUI
import MapKit
struct WalkView: View {
    let destination: MKMapItem
    @State private var route: MKRoute?
    var body: some View {
        Map {
            if let route {
                MapPolyline(route).stroke(.blue, lineWidth: 5)
            }
        }
        .task { await findRoute() }
    }
    func findRoute() async {
        let request = MKDirections.Request()
        request.source = .forCurrentLocation()
        request.destination = destination
        request.transportType = .walking
        let directions = MKDirections(request: request)
        route = try? await directions.calculate().routes.first
    }
}
```

Line by line

1 `import SwiftUI`

WHO — The Swift compiler, resolving the SwiftUI module so WalkView can use View, @State and the task modifier.

WHAT — **Brings in SwiftUI views, state and the task modifier.**

WHERE — Top of the file.

WHEN — At compile time, before any line of WalkView runs; the optional MKRoute state is type-checked against SwiftUI during the build.

WHY — The route is shown by a SwiftUI view.

2 `import MapKit`

WHO — The Swift compiler, resolving the MapKit module so WalkView can use MKDirections, MKRoute, MKMapItem and MapPolyline.

WHAT — **Loads MKDirections, MKRoute, MKMapItem and MapPolyline.**

WHERE — The second line of the file, at file scope directly above the WalkView declaration.

WHEN — At compile time, before the destination and route properties can name their MapKit types.

WHY — Routing is a MapKit service.

3 `struct WalkView: View {`

WHO — Your code; SwiftUI renders it.

WHAT — **Declares a screen that draws a walking route to one destination.**

WHERE — A struct at file scope.

WHEN — Created when a parent passes in a destination.

WHY — Taking the destination as a parameter makes the view reusable for any place.

```
4  let destination: MKMapItem
```

WHO The parent view supplies it.
WHAT **Stores the place to walk to, for example a search result.**
WHERE A constant property of WalkView, set by the memberwise initializer.
WHEN Fixed for the life of this view value.
WHY MKMapItem is the currency of MapKit: search returns it and directions accept it.

```
5  @State private var route: MKRoute?
```

WHO SwiftUI stores it; findRoute fills it.
WHAT **Holds the route once it arrives, or nil before then or after a failure.**
WHERE View state inside WalkView.
WHEN nil at first, set when directions return.
WHY An optional honestly models "no route yet", and the map draws nothing until it exists.

```
6  var body: some View {
```

WHO SwiftUI, reading WalkView's body when the view appears and again after route is set.
WHAT **Describes the map with the route line, and starts the request.**
WHERE The required body property.
WHEN On first appearance and again when route changes.
WHY When the route arrives, the state change redraws the map with the line.

```
7  Map {
```

WHO MapKit's SwiftUI Map view, created by WalkView's body with only a content builder.
WHAT **Creates a map whose content builder follows.**
WHERE The root of body; with no position binding the camera frames the content automatically.
WHEN Described each time body runs.
WHY Automatic framing zooms to fit the route as soon as it appears.

```
8  if let route {
```

WHO The map content builder.
WHAT **Adds the following content only when a route exists, unwrapping the optional.**
WHERE Inside the map content builder, which supports if statements like a view builder.
WHEN Evaluated on every body run.
WHY MapPolyline needs a real route, so the check prevents drawing anything before the reply.

```
9  MapPolyline(route).stroke(.blue, lineWidth: 5)
```

WHO MapKit draws the line on the map.
WHAT **Draws the route's path as a blue line 5 points wide.**
WHERE Map content, layered on top of the streets.
WHEN Whenever a route is present.
WHY A thick, high-contrast line is easy to follow at a glance while walking.

```
12 .task { await findRoute() }
```

WHO SwiftUI owns the task.
WHAT **Requests the route when the view appears.**
WHERE A modifier on Map.
WHEN Once on appearance; cancelled automatically if the view disappears.
WHY Directions need the network, so they run asynchronously instead of blocking the screen.

```
14 func findRoute() async {
```

WHO Your code, called by the task.
WHAT **Declares the method that builds and sends the directions request.**
WHERE A method on WalkView.
WHEN Each time the task runs.
WHY Keeping the network call out of body keeps body fast and side-effect free.

```
15 let request = MKDirections.Request()
```

WHO MapKit's MKDirections.Request initializer, called inside findRoute().
WHAT **Creates an empty directions request.**
WHERE A local constant in findRoute().
WHEN At the start of each findRoute() call, which the task modifier triggers once when WalkView appears.
WHY You set the start, the end and the travel mode before asking for a route.

```
16 request.source = .forCurrentLocation()
```

WHO MapKit and Core Location.
WHAT **Sets the starting point to wherever the device is now.**
WHERE The source property of the request; .forCurrentLocation() is a class method of MKMapItem.
WHEN Before the request is sent.
WHY It saves you from fetching coordinates yourself, but it still requires location permission.

```
17 request.destination = destination
```

WHO Your findRoute() method, copying WalkView's destination property onto the request.
WHAT **Sets the end of the route to the place this view was given.**
WHERE The destination property of the request.
WHEN Before the request is sent.
WHY Both ends must be MKMapItem values so MapKit knows exact locations and entrances.

```
18 request.transportType = .walking
```

WHO Apple's routing servers read it.
WHAT **Asks for a walking route instead of driving.**
WHERE The transportType property of the request.
WHEN Before the request is sent.
WHY Walking routes use paths and crossings and give realistic walking times; .automobile and .transit exist too.

```
19 let directions = MKDirections(request: request)
```

WHO MapKit's MKDirections initializer, wrapping the configured request inside findRoute().
WHAT **Creates the object that sends the request to Apple's servers.**
WHERE A local constant in findRoute().
WHEN Created once per call.
WHY Holding it lets you call cancel() if the user leaves before the reply.

```
20 route = try? await directions.calculate().routes.first
```

WHO MapKit and the network; then your code on the main actor.
WHAT **Waits for the routes, keeps the first one, and stores nil if routing fails.**
WHERE Writes the route state; the await suspends without blocking the main thread.
WHEN Resumes when the server replies.
WHY The first route is the recommended one; try? keeps the demo short, but a real app would explain a failure.

Try it

1. After the route is stored, print Int((route?.expectedTravelTime ?? 0) / 60) to see the walking minutes.
2. Change .walking to .automobile and compare route.distance for the same destination.
3. Show LookAroundPreview(initialScene:) under the map, loading the scene with try await MKLookAroundSceneRequest(mapItem: destination).scene.

CHECK YOURSELF

Check your understanding

1. A `Map(position: $position)` shows three markers. Your code sets `position = .automatic`. What does the map show?
   A. The user's current location
   B. A region that fits all three markers
   C. The whole world, zoomed all the way out
   D. Whatever region it showed before the assignment

2. Each trail pin must show a small round photo of the trail with a colored ring. Which map content fits best?
   A. A Marker with a systemImage and a tint color
   B. A MapCircle at each trail, filled with the photo
   C. An Annotation with a photo view in its closure
   D. A UserAnnotation placed at each trail

3. A map contains `UserAnnotation()`, but the blue dot never appears on a real phone. Info.plist has no location usage description and the app never asks for authorization. What is the fix?
   A. Add .mapStyle(.hybrid) so the dot stands out from the map
   B. Replace UserAnnotation with a Marker at (0, 0)
   C. Set the camera to .userLocation(fallback: .automatic)
   D. Add the usage key and ask for authorization

4. A coffee search near San Francisco returns cafes in other cities. The request sets `naturalLanguageQuery` only. What should you add?
   A. request.region set to the area the user is viewing
   B. A longer query, such as "coffee shop cafe espresso"
   C. Map(selection:) so the user can pick the right one
   D. transportType = .walking on the request

Answers: MapKit for SwiftUI

1. **B. A region that fits all three markers** `.automatic` frames the map content, so all three markers come into view. It does not follow the user: that is `.userLocation(fallback:)`, which you must ask for explicitly and which needs location permission.
2. **C. An Annotation with a photo view in its closure** `Annotation` pins any SwiftUI view to a coordinate, so it can show a clipped photo with a ring. A `Marker` is the simpler choice, but it only draws the standard balloon with a symbol or text inside.
3. **D. Add the usage key and ask for authorization** The blue dot needs the device location, which needs the user's permission: a usage-description string plus a request, for example through `CLServiceSession`. The style and camera look like fixes, but no style can show a location the app is not allowed to read, and `.userLocation(fallback:)` simply falls back when that permission is missing.
4. **A. request.region set to the area the user is viewing** The region tells `MKLocalSearch` where to look, and `onMapCameraChange` can give you the visible region. More query words do not narrow the area, and `transportType` belongs to directions requests, not to search.

Cloud Storage with CloudKit

CONCEPT

Why apps keep data in the cloud

Data saved with SwiftData or in files lives on one device. People expect more: the grocery list from their iPhone should appear on their iPad, survive a lost phone, and sometimes be shared with family. A *cloud-based storage API* keeps a copy on servers and syncs it between devices.

Apple gives you three built-in options. **iCloud key-value storage** syncs a few small settings. **CloudKit** stores structured records in Apple's databases. **SwiftData with CloudKit sync** mirrors your local models for you. A **third-party backend** is the fourth choice when you need more than Apple devices.

Container	Your app's own space in iCloud, named like iCloud.com.example.groceries.
Database	Each container has three: private (one user's data), public (readable by every user) and shared (records a user shares with others).
Record	One entry, a CKRecord: a set of named fields plus a record type, such as GroceryItem.
Record zone	A group of records inside a database; custom zones add change tracking and sharing.

CONCEPT

Turn on CloudKit

CloudKit needs a one-time setup in Xcode. Select your app target, open **Signing & Capabilities**, click **+ Capability** and add **iCloud**. Tick **CloudKit** and create a container, for example `iCloud.com.example.groceries`. Xcode adds an *entitlement*, a signed permission inside your app, that lets it reach that container. This step requires a paid Apple Developer Program membership.

The **CloudKit Console** is a web dashboard where you browse records, edit the *schema* (record types and their fields) and add indexes. Every container has two environments: *development*, used while you build and test, and *production*, used by people who install your app from the App Store. You deploy the schema to production before you release.

> **Who pays for the storage**
>
> Records in a user's private database count against *that user's* iCloud storage, not yours. To test, sign in to an Apple Account in the iOS Simulator's Settings app first; without one, private saves fail.

CODE · READ-ALONG

Check the iCloud account first

Before showing sync features, ask CloudKit whether this device has a usable iCloud account. `accountStatus()` returns one of five states, and each deserves a different response. Pick a state in the simulation beside the code to see which branch runs and what the user sees.

```swift
import CloudKit

func checkAccount() async throws {
    let container = CKContainer.default()
    let status = try await container.accountStatus()
    switch status {
    case .available:
        enableCloudFeatures()
    case .noAccount:
        showSignInHint()
    case .restricted:
        showRestrictedMessage()
    case .temporarilyUnavailable:
        showVerifyAccountHint()
    case .couldNotDetermine:
        showTryAgainLater()
    @unknown default:
        showTryAgainLater()
    }
}
```

Line by line

1 `import CloudKit`

- **WHO** The Swift compiler, while it builds the app.
- **WHAT** **Loads Apple's CloudKit framework: containers, databases, records and the account-status API.**
- **WHERE** The top of the file; the import applies to this file only.
- **WHEN** At compile time, when the compiler resolves the CloudKit module for this file; nothing is imported while the app runs.
- **WHY** CloudKit is a separate framework, so any file that talks to iCloud imports it.

3 `func checkAccount() async throws {`

- **WHO** Your code; the app calls it at launch and when it returns to the foreground.
- **WHAT** **Declares a function that asks iCloud about the account and reacts to the answer.**
- **WHERE** A free function here; in an app it usually lives on the type that owns syncing.
- **WHEN** Runs before the app shows any sync features.
- **WHY** It is async because the answer may need the system to check, and throws because that check can fail; the caller catches errors and shows a try-again message.

4 `let container = CKContainer.default()`

- **WHO** CloudKit, using the container named in your app's entitlements.
- **WHAT** **Gets the app's default container, its private space in iCloud.**
- **WHERE** A local constant; the container object is shared and cheap to fetch.
- **WHEN** Each time checkAccount runs.
- **WHY** default() picks the container Xcode created for you, so the identifier is not repeated in code.

5 `let status = try await container.accountStatus()`

- **WHO** CloudKit asks iOS about the Apple Account signed in on this device.
- **WHAT** **Waits for the account status and stores the answer, a CKAccountStatus value.**
- **WHERE** The await suspends this function without blocking the main thread.
- **WHEN** At launch, and again whenever the user might have signed in or out.
- **WHY** Asking first lets you explain a problem instead of failing on the first save.

6 `switch status {`

WHO Your checkAccount() function, branching on the CKAccountStatus value that accountStatus() returned.
WHAT **Chooses one branch for each possible account state.**
WHERE Inside checkAccount, right after the status arrives.
WHEN Once per call of checkAccount(), right after the await on accountStatus() resumes with an answer.
WHY A switch on an enum makes the compiler point out any state you forgot to handle.

7 `case .available:`

WHO iOS reports this when a user is signed in and iCloud is usable.
WHAT **Matches the healthy state.**
WHERE The first case of the switch.
WHEN Most launches on a typical device.
WHY Only this state means your saves and private queries can succeed.

8 `enableCloudFeatures()`

WHO Your enableCloudFeatures() helper, called from the .available case of this switch.
WHAT **Turns on syncing and any screens that depend on iCloud.**
WHERE A helper of yours, not shown, that updates your model.
WHEN Right after the status reads .available.
WHY Sync features stay hidden until they can actually work.

9 `case .noAccount:`

WHO iOS reports this when nobody is signed in to iCloud on the device.
WHAT **Matches the signed-out state.**
WHERE A case of the switch.
WHEN On devices without an Apple Account, or after the user signs out.
WHY Private-database saves would fail with .notAuthenticated, so you handle it up front.

10 `showSignInHint()`

WHO Your showSignInHint() helper, called from the .noAccount case to update the screen.
WHAT **Explains that the list stays on this device until the user signs in to iCloud in Settings.**
WHERE A helper of yours that updates the screen.
WHEN When the status is .noAccount.
WHY The app keeps working locally; people simply learn how to turn sync on.

11 `case .restricted:`

WHO iOS reports this when parental controls or a device management profile block iCloud.
WHAT **Matches the restricted state.**
WHERE A case of the switch.
WHEN On supervised devices, such as some school or company phones.
WHY The user cannot fix it in your app, so pestering them to sign in would be wrong.

12 `showRestrictedMessage()`

WHO Your showRestrictedMessage() helper, called from the .restricted case to update the screen.
WHAT **Tells the user that iCloud is turned off by a restriction on this device.**
WHERE A helper of yours that updates the screen.
WHEN When the status is .restricted.
WHY A clear, calm message avoids confusion about why sync never starts.

13 `case .temporarilyUnavailable:`

WHO iOS reports this when an account is signed in but not ready.
WHAT **Matches the state where the user must act, for example re-enter a password in Settings.**
WHERE A case of the switch.
WHEN After a password change or when Apple asks the user to verify the account.
WHY This is different from no account: the user is signed in, so the advice is different too.

14 `showVerifyAccountHint()`

WHO Your showVerifyAccountHint() helper, called from the .temporarilyUnavailable case.
WHAT **Asks the user to check their Apple Account in Settings.**
WHERE A helper of yours that updates the screen.
WHEN When the status is .temporarilyUnavailable.
WHY Pointing at the exact fix gets sync running again sooner.

15 `case .couldNotDetermine:`

WHO iOS reports this when it could not find out the account status.
WHAT **Matches the unknown state.**
WHERE A case of the switch.
WHEN Rarely, for example when the system is still starting up.
WHY Unknown is not the same as signed out, so you retry instead of blaming the user.

16 `showTryAgainLater()`

WHO Your showTryAgainLater() helper, called from the .couldNotDetermine case to show a message and plan a retry.
WHAT **Shows a gentle message and schedules another check.**
WHERE A helper of yours.
WHEN When the status could not be determined.
WHY The data stays safe on the device while the app waits.

17 `@unknown default:`

WHO The Swift compiler checks it; iOS might add new states in the future.
WHAT **Catches any account status that did not exist when this code was written.**
WHERE The last case of the switch.
WHEN Only if a future iOS adds a new case.
WHY @unknown default compiles today but warns you when Apple adds a case, so you remember to handle it.

18 `showTryAgainLater()`

WHO The same showTryAgainLater() helper, reused here inside the @unknown default branch.
WHAT **Falls back to the safe, retrying behavior for an unexpected state.**
WHERE Inside the @unknown default branch.
WHEN Only for a status this code does not know.
WHY When in doubt, keep data local and retry rather than assume iCloud works.

Try it

1. In the simulation beside the code, choose .noAccount. Which line runs, and does the app still work?
2. In Xcode, delete the @unknown default case and build. What does the compiler say?
3. Sign out of iCloud in the iOS Simulator's Settings app and print status at launch.

CODE · READ-ALONG

Save a record

A `CKRecord` is like a dictionary with a type name. You set its fields, pick a database and call `save`. The same code writes to the private database (only this user) or the public one (every user). Save items in the console beside the code and watch where each record lands.

```
import CloudKit
import Observation

@MainActor @Observable
final class GroceryStore {
    let container = CKContainer.default()
    var items: [CKRecord] = []
    func add(_ name: String, isShared: Bool) async {
        let record = CKRecord(recordType: "GroceryItem")
        record["name"] = name
        record["quantity"] = 1
        let database = isShared
            ? container.publicCloudDatabase
            : container.privateCloudDatabase
        do {
            try await database.save(record)
            items.append(record)
        } catch let error as CKError {
            handle(error.code)
        } catch { print(error) }
    }
}
```

Line by line

1 `import CloudKit`

- **WHO** The Swift compiler, resolving the CloudKit module so GroceryStore can name CKContainer, CKRecord and CKError.
- **WHAT** **Loads CKContainer, CKRecord, CKDatabase and CKError.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before GroceryStore is built; the CloudKit types are resolved then, not while the app runs.
- **WHY** Every type in this store comes from CloudKit.

2 `import Observation`

- **WHO** The Swift compiler, resolving the Observation module so the @Observable macro below can expand.
- **WHAT** **Loads the Observation framework that provides the @Observable macro.**
- **WHERE** Top of the file, after CloudKit.
- **WHEN** At compile time, when the compiler expands the @Observable macro on GroceryStore; no code runs for this import.
- **WHY** This file has no SwiftUI import, and @Observable lives in Observation.

```
4  @MainActor @Observable
```

WHO The compiler applies both attributes to the class below.
WHAT **@MainActor keeps all of the store's work on the main thread; @Observable lets SwiftUI views watch its properties.**
WHERE Directly above the class declaration.
WHEN At compile time; their effects apply every time the store is used.
WHY Views read items on the main thread, so keeping the store there prevents data races.

```
5  final class GroceryStore {
```

WHO Your code; one instance is shared by the app's views.
WHAT **Declares the object that owns the grocery list and talks to CloudKit.**
WHERE A reference type on the heap, usually created once when the app launches.
WHEN Created at launch and kept for the app's lifetime.
WHY A class gives every screen the same list; final tells the compiler nobody subclasses it.

```
6      let container = CKContainer.default()
```

WHO CloudKit, using the container from your entitlements.
WHAT **Stores a reference to the app's default iCloud container.**
WHERE A constant property of the store.
WHEN Set once when the store is created.
WHY The container is the doorway to both the private and the public database.

```
7      var items: [CKRecord] = []
```

WHO The store owns it; SwiftUI views observe it.
WHAT **Holds the records shown on screen, starting empty.**
WHERE A stored property of the observable class.
WHEN Grows after each successful save and is replaced after each query.
WHY Because the class is @Observable, any view that reads items redraws when it changes.

```
8      func add(_ name: String, isShared: Bool) async {
```

WHO Your code, called when the user taps Save.
WHAT **Declares an asynchronous method that saves one grocery item.**
WHERE A method on GroceryStore, running on the main actor.
WHEN Each time the user adds an item.
WHY async lets the method wait for iCloud without freezing the screen.

```
9          let record = CKRecord(recordType: "GroceryItem")
```

WHO CloudKit creates the record object on the device.
WHAT **Makes a new, empty record of type GroceryItem with a fresh unique ID.**
WHERE In memory on the device; nothing is uploaded yet.
WHEN At the start of each add call.
WHY The record type works like a table name: queries later ask for every GroceryItem.

```
10         record["name"] = name
```

WHO Your code fills the field; CloudKit stores it.
WHAT **Sets the record's name field to the text the user typed.**
WHERE A field inside the in-memory record.
WHEN Inside add(_:isShared:), right after the record is created and before database.save(record) sends it to iCloud.
WHY In development, CloudKit adds a new field to the schema the first time you save it.

```
11  record["quantity"] = 1
```

WHO Your code; CloudKit stores the value as a number.
WHAT **Sets the quantity field to 1.**
WHERE Another field of the same record.
WHEN Immediately after the name field is set, still in memory, before the save call uploads the record.
WHY Fields hold simple types such as strings, numbers, dates, locations and files.

```
12  let database = isShared
```

WHO Your code decides where the record goes.
WHAT **Starts choosing a database based on the isShared flag.**
WHERE A local constant inside add.
WHEN Each time add(_:isShared:) runs, after both fields are set and before the do block starts the upload.
WHY The same record type can live in either database, so the choice is a single value.

```
13  ? container.publicCloudDatabase
```

WHO CloudKit's public database, shared by every user of the app.
WHAT **Picks the public database when the item should be visible to everyone.**
WHERE The true branch of the conditional expression.
WHEN When isShared is true.
WHY Everyone can read public data, but only signed-in users can write it.

```
14  : container.privateCloudDatabase
```

WHO CloudKit's private database, one per user.
WHAT **Picks the signed-in user's private database otherwise.**
WHERE The false branch of the conditional expression.
WHEN When isShared is false.
WHY Private data is visible only to its owner; even you, the developer, cannot read another user's records.

```
15  do {
```

WHO Swift's do-catch statement, wrapping the throwing database.save(record) call inside add(_:isShared:).
WHAT **Starts a block whose errors the catch clauses below will handle.**
WHERE Inside add, around the network call.
WHEN Every time add(_:isShared:) reaches the upload, so each save gets its own chance to fail cleanly.
WHY Saving can fail for many reasons: no account, no network, a full iCloud.

```
16  try await database.save(record)
```

WHO CloudKit uploads the record to Apple's servers.
WHAT **Sends the record and waits until the server confirms it was stored.**
WHERE The await suspends add; the main thread keeps drawing.
WHEN Resumes after the server replies, or throws on failure.
WHY Waiting for the confirmation means items only shows records that really exist in iCloud.

```
17  items.append(record)
```

WHO Your code, back on the main actor.
WHAT **Adds the saved record to the on-screen list.**
WHERE Writes the observed items property.
WHEN Only after a successful save.
WHY Observing views redraw automatically with the new row.

```
18 } catch let error as CKError {
```

WHO Swift error handling, matching CloudKit errors.
WHAT **Catches errors that are CKError values and names the error.**
WHERE The first catch clause of the do block.
WHEN Only when the save throws a CloudKit error.
WHY CKError carries a code that tells you exactly what went wrong.

```
19 handle(error.code)
```

WHO Your helper, not shown, decides what to tell the user.
WHAT **Passes the specific error code, such as .notAuthenticated, to your handling code.**
WHERE Inside the CKError catch clause.
WHEN After a CloudKit failure.
WHY One central handler keeps messages and retry rules consistent across the app.

```
20 } catch { print(error) }
```

WHO Swift's catch-all clause, the final catch of the do block in add(_:isShared:).
WHAT **Catches any other error and prints it to the console.**
WHERE The final catch clause, which makes the handling complete.
WHEN Only for errors that are not CKError.
WHY Swift requires every possible error to be handled in a non-throwing method.

Try it

1. In the console beside the code, save an item as Ana with Only me, then switch to Ben. Where did the item go?
2. Sign out in the console and save with Everyone. Which error code appears in the log?
3. In Xcode, add record["addedAt"] = Date.now and save. Look for the new field in the CloudKit Console.

CODE · READ-ALONG

Query records back

A `CKQuery` asks a database for every record of a type that matches a predicate, a filter condition. The results come back as records, and you convert each one into your own `GroceryItem` struct (an `id`, a `name` and a `quantity`, defined beside this function and not shown). Run queries in the console beside the code.

```swift
import CloudKit

func loadItems(from database: CKDatabase) async -> [GroceryItem] {
    let query = CKQuery(recordType: "GroceryItem",
        predicate: NSPredicate(value: true))
    do {
        let (results, _) = try await database
            .records(matching: query)
        let records = results.compactMap { try? $0.1.get() }
        return records.map { record in
            GroceryItem(id: record.recordID,
                name: record["name"] as? String ?? "",
                quantity: record["quantity"] as? Int ?? 1)
        }
    } catch {
        print("Query failed:", error)
        return []
    }
}
```

Line by line

1 `import CloudKit`

- **WHO** The Swift compiler, resolving the CloudKit module so loadItems(from:) can use CKQuery and CKDatabase.
- **WHAT** **Loads CKQuery, CKDatabase and CKRecord.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before loadItems(from:) exists as runnable code; no work happens at run time.
- **WHY** Queries are part of CloudKit.

3 `func loadItems(from database: CKDatabase) async -> [GroceryItem] {`

- **WHO** Your code, called at launch and on pull to refresh.
- **WHAT** **Declares an asynchronous function that returns the grocery items in one database.**
- **WHERE** A free function here; in an app it would be a method on GroceryStore.
- **WHEN** Whenever the list needs fresh data.
- **WHY** Taking the database as a parameter lets the same code read the private or the public list.

4 `let query = CKQuery(recordType: "GroceryItem",`

- **WHO** CloudKit builds the query object.
- **WHAT** **Starts a query for records of type GroceryItem.**
- **WHERE** A local constant on the device; nothing is sent yet.
- **WHEN** Every time loadItems(from:) runs, which is at launch and on each pull to refresh.
- **WHY** The record type must match the one you saved with, character for character.

5 `predicate: NSPredicate(value: true))`

- **WHO** Foundation's NSPredicate, a filter that CloudKit understands.
- **WHAT** **Uses a filter that matches every record, so the query returns them all.**
- **WHERE** The second argument of CKQuery.
- **WHEN** Evaluated on Apple's servers when the query runs.
- **WHY** Matching everything requires the recordName field to be marked Queryable in the CloudKit Console, a common first surprise.

```
6  do {
```

WHO	Swift's do-catch statement, wrapping the throwing records(matching:) call inside loadItems(from:).
WHAT	**Starts a block whose errors the catch below handles.**
WHERE	Around the network call.
WHEN	Every time loadItems(from:) runs, just before the query is sent to Apple's servers.
WHY	Queries can fail when the device is offline or iCloud is busy.

```
7  let (results, _) = try await database
```

WHO	CloudKit sends the query to Apple's servers.
WHAT	**Waits for the matches and keeps the results, ignoring the second value, a cursor for fetching more.**
WHERE	The await suspends this function without blocking the main thread.
WHEN	Resumes when the server replies.
WHY	The cursor matters only for long lists; a small grocery list arrives in one batch.

```
8  .records(matching: query)
```

WHO	CloudKit's CKDatabase, in its async records(matching:) method, which contacts Apple's servers.
WHAT	**Runs the query against this database.**
WHERE	A method on CKDatabase, continued from the line above.
WHEN	Executed on the server.
WHY	Splitting the call across lines keeps each line short and readable.

```
9  let records = results.compactMap { try? $0.1.get() }
```

WHO	Your code, processing the reply on the device.
WHAT	**Keeps every record that loaded successfully and drops the ones that failed.**
WHERE	Each result is a pair of record ID and Result; $0.1 is the Result.
WHEN	Right after the reply arrives.
WHY	Each record can succeed or fail on its own, so one bad record does not lose the whole list.

```
10 return records.map { record in
```

WHO	Your loadItems(from:) function, mapping each CKRecord into a GroceryItem with Swift's map.
WHAT	**Converts every CKRecord into a GroceryItem and returns the array.**
WHERE	Inside the do block, after a successful query.
WHEN	Once per successful query, after compactMap has dropped any records that failed to load.
WHY	Views work with a simple struct, so CloudKit types do not spread through your interface code.

```
11 GroceryItem(id: record.recordID,
```

WHO	Your struct's memberwise initializer.
WHAT	**Builds an item whose id is the record's unique CloudKit ID.**
WHERE	Inside the map closure.
WHEN	Once for each CKRecord the map closure visits, after the query returned successfully.
WHY	Keeping the record ID lets you update or delete the right record later.

```
12 name: record["name"] as? String ?? "",
```

WHO	Your code reading a CloudKit field.
WHAT	**Reads the name field as a String, using an empty string if it is missing or a different type.**
WHERE	The name argument of GroceryItem.
WHEN	Once per record, as the map closure reads the name field while building that GroceryItem.
WHY	Fields come back untyped, so as? checks the type safely instead of crashing.

```
13  quantity: record["quantity"] as? Int ?? 1)
```

WHO Your code reading a CloudKit field.
WHAT **Reads quantity as an Int, defaulting to 1.**
WHERE The quantity argument of GroceryItem.
WHEN Once per record, right after the name is read and before GroceryItem's initializer finishes.
WHY Old records saved before the field existed still load with a sensible value.

```
15  } catch {
```

WHO Swift's catch clause, paired with the do block in loadItems(from:).
WHAT **Catches any error from the query.**
WHERE The catch clause of the do block.
WHEN Only when the query fails.
WHY The function promises to return an array, so failure must still produce one.

```
16  print("Query failed:", error)
```

WHO Your loadItems(from:) function, calling print inside its catch clause.
WHAT **Writes the error to the Xcode console.**
WHERE Inside the catch clause.
WHEN After a failed query.
WHY Seeing the exact error, such as a missing index, speeds up debugging.

```
17  return []
```

WHO Your loadItems(from:) function, returning from inside the catch clause.
WHAT **Returns an empty list after a failure.**
WHERE The last statement of the catch clause.
WHEN Only after a failed query.
WHY Callers keep any items they already show instead of crashing; a real app would also show a message.

Try it

1. In the console beside the code, switch between Ana and Ben and press Refresh. Which records does each query return?
2. Change the predicate to NSPredicate(format: "quantity > %d", 1). Which items would come back?
3. Go offline in the console and refresh. What does the app show while the query fails?

INTERACTIVE

Private, public, signed out, offline

The phone runs a small Groceries app, and the panel shows what CloudKit stores. Save an item as Ana with Only me, then one with Everyone. Switch to Ben: Ana's private item disappears while the public one stays. Sign out and try to save, then read the public list. Finally turn the network off. After each move, read the request log: every result matches a `CKError` code your app must handle.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Handle CloudKit errors

Every CloudKit call can fail, and `CKError.code` says why. Good apps turn each code into a clear message and a plan: sign in, wait for the network, free up space, or merge a conflicting change. Some errors also carry `retryAfterSeconds`, the server's request to wait before trying again.

```swift
import CloudKit

func describe(_ error: CKError) -> String {
    switch error.code {
    case .notAuthenticated:
        return "Sign in to iCloud in Settings to sync."
    case .networkUnavailable, .networkFailure:
        return "You are offline. We will sync later."
    case .quotaExceeded:
        return "Your iCloud storage is full."
    case .serverRecordChanged:
        return "Another device changed this item."
    default:
        if let wait = error.retryAfterSeconds {
            return "iCloud is busy. Retry in \(Int(wait)) s."
        }
        return "Something went wrong. Try again."
    }
}
```

Line by line

1 `import CloudKit`

- **WHO** The Swift compiler, resolving the CloudKit module so describe(_:) can match CKError.Code cases.
- **WHAT** **Loads CKError and its Code enumeration.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before describe(_:) is built; nothing is loaded while the app runs.
- **WHY** The error codes are defined by CloudKit.

3 `func describe(_ error: CKError) -> String {`

- **WHO** Your code, called from every catch clause that handles CloudKit errors.
- **WHAT** **Declares a function that turns an error into a message for the user.**
- **WHERE** A free function, so any part of the app can use it.
- **WHEN** Each time a CloudKit call fails.
- **WHY** One shared function keeps wording consistent across every screen.

4 `switch error.code {`

- **WHO** Your describe(_:) function, branching on the CKError.Code stored in error.code.
- **WHAT** **Chooses a branch based on the specific CloudKit error code.**
- **WHERE** The first statement inside describe(_:), covering the whole function body.
- **WHEN** Once per call of describe(_:), each time a catch clause hands it a CKError.
- **WHY** Different failures need different advice, so the code, not the text of the error, drives the decision.

5 `case .notAuthenticated:`

- **WHO** CloudKit reports this when no iCloud account is signed in.
- **WHAT** **Matches saves and private reads attempted while signed out.**
- **WHERE** A case of the switch.
- **WHEN** After a save or private query on a signed-out device.
- **WHY** The fix is in Settings, so the message points there.

```
6   return "Sign in to iCloud in Settings to sync."
```

WHO Your describe(_:) function, inside the .notAuthenticated case.
WHAT **Returns advice that names the exact place to fix the problem.**
WHERE Inside the .notAuthenticated case.
WHEN For that error only.
WHY Specific instructions get people unstuck; a generic failure message does not.

```
7   case .networkUnavailable, .networkFailure:
```

WHO CloudKit reports these when the device is offline or the connection drops.
WHAT **Matches both network problems in one case.**
WHERE A case listing two codes separated by a comma.
WHEN In airplane mode, in a tunnel, or on flaky Wi-Fi.
WHY Both have the same fix, so one branch avoids duplicate code.

```
8   return "You are offline. We will sync later."
```

WHO Your describe(_:) function, inside the case for .networkUnavailable and .networkFailure.
WHAT **Promises to retry and reassures the user.**
WHERE Inside the network case.
WHEN Only when error.code is .networkUnavailable or .networkFailure, such as in airplane mode.
WHY Keeping the change on the device and uploading later is better than losing it.

```
9   case .quotaExceeded:
```

WHO CloudKit reports this when the user's iCloud storage is full.
WHAT **Matches saves that do not fit in the user's storage.**
WHERE A case of the switch.
WHEN When a private-database save exceeds the user's quota.
WHY Only the user can free space or upgrade, so the app explains instead of retrying.

```
10  return "Your iCloud storage is full."
```

WHO Your describe(_:) function, inside the .quotaExceeded case.
WHAT **Returns an honest explanation of the full storage.**
WHERE Inside the .quotaExceeded case.
WHEN For that error only.
WHY Retrying would fail again, so the message must say what is wrong.

```
11  case .serverRecordChanged:
```

WHO CloudKit reports this when another device saved the same record first.
WHAT **Matches a conflict between your copy and the server's newer copy.**
WHERE A case of the switch.
WHEN When two devices edit the same item before syncing.
WHY The error includes the server's version, so your app can merge the changes and save again.

```
12  return "Another device changed this item."
```

WHO Your describe(_:) function, inside the .serverRecordChanged case.
WHAT **Explains the conflict to the user.**
WHERE Inside the .serverRecordChanged case.
WHEN Only when error.code is .serverRecordChanged, meaning another device saved this record first.
WHY A real app often merges silently and only asks the user when the changes truly clash.

```
13 default:
```

WHO Your describe(_:) function, in the default branch of its switch over error.code.
WHAT **Handles every other error code.**
WHERE The last case of the switch.
WHEN For codes not listed above, such as .serviceUnavailable.
WHY CKError.Code has many cases; the rare ones share a generic path.

```
14 if let wait = error.retryAfterSeconds {
```

WHO CloudKit's servers set this value on some errors.
WHAT **Checks whether the server asked you to wait, and unwraps the number of seconds.**
WHERE Inside the default case.
WHEN When the server is busy or your app sent too many requests.
WHY Retrying sooner than asked can get your app throttled for longer.

```
15 return "iCloud is busy. Retry in \(Int(wait)) s."
```

WHO Your describe(_:) function, inside the if let block of the default branch, using string interpolation.
WHAT **Builds a message that includes the wait, rounded to whole seconds.**
WHERE Inside the if let block.
WHEN Only when the server sent a wait time.
WHY Telling people when it will work again is kinder than an open-ended failure.

```
17 return "Something went wrong. Try again."
```

WHO Your describe(_:) function, at the end of the default branch after the retryAfterSeconds check.
WHAT **Returns a generic fallback message.**
WHERE The end of the default case.
WHEN For unexpected errors without a wait time.
WHY Every path must return a String, and a simple retry is the safest advice.

> **Try it**
>
> 1. Call describe on CKError(.quotaExceeded) in a test and check the message.
> 2. Add a case for .unknownItem, which means the record was deleted on another device.
> 3. Write a retry helper that waits error.retryAfterSeconds with Task.sleep before trying again.

CODE · READ-ALONG

Sync small settings with iCloud key-value storage

`NSUbiquitousKeyValueStore` works like `UserDefaults`, but its values follow the user to every device. It holds at most 1 MB per user and at most 1,024 keys, so use it for preferences, not documents. Turn it on with the **Key-value storage** checkbox under the iCloud capability.

```swift
import Foundation

final class SettingsSync {
    private let store = NSUbiquitousKeyValueStore.default

    var sortOrder: String {
        get { store.string(forKey: "sortOrder") ?? "aisle" }
        set { store.set(newValue, forKey: "sortOrder") }
    }

    init() {
        NotificationCenter.default.addObserver(
            forName: NSUbiquitousKeyValueStore
                .didChangeExternallyNotification,
            object: store, queue: .main
        ) { _ in
            print("Another device changed a setting.")
        }
        store.synchronize()
    }
}
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, resolving the Foundation module so SettingsSync can use NSUbiquitousKeyValueStore.
- **WHAT** **Loads Foundation, which contains NSUbiquitousKeyValueStore and NotificationCenter.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before SettingsSync is built; nothing is imported while the app runs.
- **WHY** The key-value store is a Foundation class, so CloudKit is not needed here.

3 `final class SettingsSync {`

- **WHO** Your code; the app creates one instance at launch.
- **WHAT** **Declares a small class that reads, writes and watches synced settings.**
- **WHERE** A reference type on the heap.
- **WHEN** Created once and kept for the app's lifetime.
- **WHY** One owner for synced settings keeps the rest of the app simple.

4 `private let store = NSUbiquitousKeyValueStore.default`

- **WHO** iOS, which syncs the store with iCloud in the background.
- **WHAT** **Keeps a reference to the app's shared iCloud key-value store.**
- **WHERE** A private constant property of SettingsSync.
- **WHEN** Set when the object is created.
- **WHY** private hides the store, so every read and write goes through this class.

6 `var sortOrder: String {`

- **WHO** Your views read and set it.
- **WHAT** **Declares a computed property for how the grocery list is sorted.**
- **WHERE** A property of SettingsSync with no storage of its own.
- **WHEN** Evaluated on every read and write.
- **WHY** A computed property makes the iCloud store look like an ordinary variable.

```
7  get { store.string(forKey: "sortOrder") ?? "aisle" }
```

WHO The key-value store answers from its local copy.
WHAT **Reads the saved sort order, or returns "aisle" if none exists.**
WHERE The getter of sortOrder.
WHEN Whenever code reads sortOrder.
WHY Reads are instant because the store keeps a local copy; the default covers a brand-new user.

```
8  set { store.set(newValue, forKey: "sortOrder") }
```

WHO The key-value store saves locally, then iOS uploads it.
WHAT **Writes the new sort order under the sortOrder key.**
WHERE The setter of sortOrder; newValue is the assigned value.
WHEN Whenever code assigns sortOrder.
WHY iOS decides when to upload, often within seconds, so the write never blocks the app.

```
11 init() {
```

WHO Swift calls it when the app creates SettingsSync.
WHAT **Starts listening for changes from other devices and refreshes the store.**
WHERE The initializer of the class.
WHEN Once, when the app creates its SettingsSync instance at launch, before any setting is read.
WHY Listening early means no change from another device is missed.

```
12 NotificationCenter.default.addObserver(
```

WHO Foundation's notification center, which delivers system announcements.
WHAT **Registers a closure to run when a named notification is posted.**
WHERE Inside init; the registration lasts as long as the app runs.
WHEN Registered once at creation.
WHY iOS announces iCloud changes as notifications rather than calling your code directly.

```
13 forName: NSUbiquitousKeyValueStore
```

WHO iOS posts this notification.
WHAT **Starts naming the notification to listen for.**
WHERE The first argument of addObserver.
WHEN Read once inside init(), when addObserver registers the closure; the name never changes afterward.
WHY The name is long, so it is split across two lines.

```
14 .didChangeExternallyNotification,
```

WHO iOS posts it when values arrive from iCloud.
WHAT **Completes the name: the notification for changes made on another device.**
WHERE The second half of the forName argument.
WHEN Posted after another device's change is downloaded.
WHY Local writes do not post it, so you react only to outside changes.

```
15 object: store, queue: .main
```

WHO NotificationCenter uses these to filter and schedule.
WHAT **Listens only to this store and runs the closure on the main queue.**
WHERE The remaining addObserver arguments.
WHEN Applied to every delivery.
WHY The main queue is where interface updates must happen.

```
16  ) { _ in
```

WHO — NotificationCenter will call the closure that starts here.
WHAT — **Closes the argument list and opens the trailing closure, ignoring the notification parameter.**
WHERE — The end of the addObserver call.
WHEN — The closure runs on every external change.
WHY — _ in says the notification details are not needed; the store already holds the new values.

```
17  print("Another device changed a setting.")
```

WHO — Your closure, on the main thread.
WHAT — **Logs the change; a real app would refresh its interface here.**
WHERE — Inside the notification closure.
WHEN — Each time another device's change arrives.
WHY — Reacting keeps every device showing the same sort order.

```
19  store.synchronize()
```

WHO — The key-value store and iOS.
WHAT — **Asks the store to bring its local copy up to date with what is already on disk and in iCloud.**
WHERE — The last line of init.
WHEN — Once, as the last step of init(), right after the observer is registered.
WHY — Apple recommends one call at launch; it does not force an upload, so you never need to call it after each write.

> **Try it**
>
> 1. Run the app on two iOS Simulator devices signed in to the same Apple Account and change sortOrder on one.
> 2. Add a Bool setting, showPrices, using store.bool(forKey:) and store.set(_:forKey:).
> 3. Try saving a large image Data value. Why is the key-value store the wrong place for it?

CONCEPT

CloudKit or your own backend?

CloudKit fits apps whose users live on Apple devices. There are no servers for you to run, people sign in with the Apple Account already on their phone, and private data stays tied to that account.

Choose a **third-party or custom backend**, for example Firebase, Supabase or your own REST server, when the same people also use Android or the web, when the server must run your own logic (payments, moderation, scheduled jobs), or when people sign in with something other than an Apple Account. Your app then talks to it with `URLSession`.

Apple-only, personal data	CloudKit, or SwiftData with CloudKit sync.
Cross-platform or server logic	A third-party or custom backend.
A handful of preferences	iCloud key-value storage.

SwiftData can sync for you

Create the model container with `ModelConfiguration(cloudKitDatabase: .automatic)` and SwiftData mirrors your models to the private database. To sync, every property must be optional or have a default value, every relationship must be optional, and no property may be marked unique.

CHECK YOURSELF

Check your understanding

1. Ana saves a GroceryItem to her **private** database. Ben opens the same app on his phone, signed in to his own Apple Account, and runs the same query on his private database. What does Ben see?
   A. Ana's item, because both phones share the same container
   B. Ana's item, but only after the next CloudKit sync finishes
   C. None of Ana's items: each user has a separate private database
   D. An error, because two users cannot share one container

2. A user is signed out of iCloud. Which outcome matches CloudKit's behavior?
   A. Private saves fail with .notAuthenticated, but public reads work
   B. Every call works, but nothing syncs until the user signs in
   C. The app crashes the first time it calls CKContainer.default()
   D. Public saves succeed, but private reads fail until sign-in

3. Your app must remember whether the user prefers metric units, on all of their devices. Which storage fits best?
   A. A public CloudKit database
   B. A custom REST backend
   C. A CKRecord in the private database for every change
   D. NSUbiquitousKeyValueStore

4. A fitness startup ships on iPhone and Android. Coaches on a website must see their clients' workouts. Which storage design fits?
   A. Only CloudKit's private database for all users
   B. A custom backend reached over HTTPS
   C. SwiftData with CloudKit sync on every device
   D. iCloud key-value storage on every device

Answers: Cloud Storage with CloudKit

1. **C. None of Ana's items: each user has a separate private database** The container is shared by the app, but every user gets their own private database inside it. Ben's query only searches Ben's data. The shared container suggests Ben should see it, yet sharing requires the public database or a `CKShare`.
2. **A. Private saves fail with .notAuthenticated, but public reads work** Reading public data needs no account, so it still works. Writing anywhere, public or private, needs a signed-in user and fails with `.notAuthenticated`. Nothing crashes, which is why checking `accountStatus()` first lets you explain the situation.
3. **D. NSUbiquitousKeyValueStore** One small preference is exactly what iCloud key-value storage is for: a few keys that follow the user. A private CloudKit record would also work, but it is heavier than needed. The public database would expose the setting to every user.
4. **B. A custom backend reached over HTTPS** Android users and a coaches' website need a server every platform can reach, so a third-party or custom backend fits. SwiftData with CloudKit sync appeals because it needs so little code, but it only syncs between one person's Apple devices.

Photos, Camera and Sharing

CONCEPT

The photo picker keeps the library private

A photo library holds years of private moments, so iOS guards it closely. The friendliest way to get one picture is to never touch the library at all. **PhotosPicker**, from the PhotosUI framework, shows the system photo picker. The picker runs *outside your app*, in a separate system process, so your code never sees the grid of photos.

Because your app sees nothing until the user chooses, iOS shows **no permission alert** and needs no usage-description key. When the user taps a photo, your app receives a `PhotosPickerItem`: a lightweight handle to the chosen item, not the image itself. You then ask the item to load its contents.

PhotosPickerItem	A handle to one picked photo or video. Loading its data is a separate, asynchronous step.
Transferable	A Swift protocol for types that convert themselves to and from data, such as Data, Image and String. Pickers, drag and drop, and sharing all rely on it.
Full library access	Reading the whole library through PHPhotoLibrary needs the NSPhotoLibraryUsageDescription key and a permission alert. Apps that only let people pick photos rarely need it.

CODE · READ-ALONG

Pick a photo and load it

This view shows a **Choose Photo** button. When the user picks an image, a task loads its bytes, turns them into an image and displays it. Run the simulation beside the code to watch each phase: the picker sheet, the selection, the loading, and the result.

```swift
import SwiftUI
import PhotosUI

struct PostcardView: View {
    @State private var item: PhotosPickerItem?
    @State private var photo: Image?

    var body: some View {
        VStack(spacing: 16) {
            photo?.resizable().scaledToFit()
            PhotosPicker("Choose Photo", selection: $item,
                         matching: .images)
        }
        .task(id: item) {
            let data = try? await item?.loadTransferable(
                type: Data.self)
            if let data, let image = UIImage(data: data) {
                photo = Image(uiImage: image)
            }
        }
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds the app.
- **WHAT** **Makes SwiftUI views and modifiers available in this file, and on iOS also the UIKit image type UIImage that the loading code uses.**
- **WHERE** At the top of the file; an import applies only to the file it appears in.
- **WHEN** At compile time, before any code runs.
- **WHY** Views like VStack and Image, and modifiers like .task, are defined in SwiftUI, not in the Swift language itself.

2 `import PhotosUI`

- **WHO** The Swift compiler, resolving the PhotosUI module so PostcardView can use PhotosPicker and PhotosPickerItem.
- **WHAT** **Brings in the PhotosUI framework, home of the PhotosPicker view and the PhotosPickerItem type.**
- **WHERE** At the top of the file, next to the other import.
- **WHEN** At compile time, before PostcardView is built; the picker types are resolved then, never at run time.
- **WHY** The photo picker is a separate framework, so only files that show a picker pay for importing it.

4 `struct PostcardView: View {`

- **WHO** Your code, defining a SwiftUI view type.
- **WHAT** **Declares a screen named PostcardView that promises to describe its interface through a body property.**
- **WHERE** At file scope, so any other view in the app can create a PostcardView.
- **WHEN** The type exists at compile time; SwiftUI creates values of it whenever a parent view includes it.
- **WHY** Small structs that conform to View are the building blocks of every SwiftUI screen.

```
5  @State private var item: PhotosPickerItem?
```

WHO	SwiftUI, which owns the storage behind @State.
WHAT	**Holds the item the user picked, or nil before anyone picks; the picker writes into it through a binding.**
WHERE	In storage SwiftUI keeps for this view, outside the struct itself, so the value survives each redraw.
WHEN	Starts as nil when the view first appears, then changes each time the user picks a photo.
WHY	It is optional because "nothing picked yet" is a real state, and private because only this view should change it.

```
6  @State private var photo: Image?
```

WHO	SwiftUI, which redraws the view when this state changes.
WHAT	**Stores the finished, displayable image once loading succeeds.**
WHERE	In SwiftUI-managed storage for this view, next to item.
WHEN	Stays nil until the load task assigns an image, which triggers a fresh body.
WHY	Keeping the picked item and the loaded image separate lets the screen show the old photo while a new one loads.

```
8  var body: some View {
```

WHO	SwiftUI, which reads body to learn what to draw.
WHAT	**Declares the computed property that describes this screen; some View means "one specific view type the compiler works out".**
WHERE	A computed property of PostcardView, declared after its two @State properties.
WHEN	Every time SwiftUI needs the interface, for example after item or photo changes.
WHY	Describing the screen as a function of state means you never update the display by hand.

```
9  VStack(spacing: 16) {
```

WHO	SwiftUI's VStack container, laying out PostcardView's content.
WHAT	**Stacks the photo and the picker button vertically with 16 points of space between them.**
WHERE	At the top of body, as the root view of the screen.
WHEN	Each time body runs.
WHY	A vertical stack is the simplest layout for "image above button".

```
10 photo?.resizable().scaledToFit()
```

WHO	Swift optional chaining, then SwiftUI image modifiers.
WHAT	**When photo holds an image, makes it resizable and scales it to fit the space without cropping; when photo is nil, the whole expression is nil and SwiftUI draws nothing.**
WHERE	First child of the VStack, above the button.
WHEN	Each time body runs; it shows an image only after the load task has set photo.
WHY	The question mark handles the empty state in one line, so the screen needs no placeholder before the first pick.

```
11 PhotosPicker("Choose Photo", selection: $item,
```

WHO	PhotosUI, which shows the system photo picker when the button is tapped.
WHAT	**Draws a button titled Choose Photo and passes it a binding to item, so the picker can write the chosen item back into your state.**
WHERE	Second child of the VStack.
WHEN	The button appears with the view; the picker sheet opens only when the user taps it.
WHY	The $ prefix creates a binding, a two-way connection that lets the system picker set your property without seeing anything else in your app.

```
12  matching: .images)
```

WHO — PhotosUI, filtering what the picker offers.
WHAT — **Limits the picker to still images, so videos never appear as choices.**
WHERE — The last argument of the PhotosPicker initializer.
WHEN — Applied when the picker sheet opens.
WHY — Filtering at the source means the loading code below can assume it receives image data.

```
14  .task(id: item) {
```

WHO — SwiftUI, which starts and cancels this asynchronous task.
WHAT — **Attaches async work to the VStack and restarts it whenever item changes value.**
WHERE — A modifier on the VStack, so the task lives as long as the view is on screen.
WHEN — Once when the view appears, then again after every new pick; a running task is cancelled first.
WHY — Loading can take a while for large or iCloud-stored photos, and tying the work to item means a second pick never races a stale first load.

```
15  let data = try? await item?.loadTransferable(
```

WHO — The PhotosPickerItem, which asks the system to deliver the picked file.
WHAT — **Starts loading the chosen photo, waits without blocking the screen, and turns any failure or missing item into nil.**
WHERE — Inside the task, off the visible work of drawing the screen.
WHEN — After each pick; it may wait while iOS downloads a full-size original from iCloud.
WHY — try? keeps the code short: a failed load simply leaves the old photo in place instead of crashing.

```
16  type: Data.self)
```

WHO — The Transferable system behind loadTransferable(type:), delivering the picked item as Data.
WHAT — **Asks for the item as raw bytes, using Data, which already conforms to Transferable.**
WHERE — The argument that finishes the loadTransferable call.
WHEN — Decided at compile time; used when the load runs.
WHY — Data works for any image format, and the next line decides how to decode it.

```
17  if let data, let image = UIImage(data: data) {
```

WHO — Your code, with UIKit decoding the bytes.
WHAT — **Continues only if bytes arrived and UIImage can decode them as a picture.**
WHERE — Inside the task, after loading finishes.
WHEN — Once per completed load.
WHY — Two checks in one if let keep the happy path flat and skip quietly when the data is not a valid image.

```
18  photo = Image(uiImage: image)
```

WHO — Your code, then SwiftUI.
WHAT — **Wraps the decoded UIImage in a SwiftUI Image and stores it in the photo state.**
WHERE — Inside the if let, on the main actor where SwiftUI state lives.
WHEN — Right after a successful decode.
WHY — Assigning to @State is all it takes: SwiftUI notices the change and redraws body with the new picture.

Try it

1. Change matching: .images to matching: .screenshots. Which photos would the picker offer now?
2. Remove the ? after photo. What does the compiler report, and why?
3. Pick a photo in the simulation, then read the "What your app can see" panel. Why is it so short?

CODE · READ-ALONG

Load straight into your own type

Loading `Data` and decoding it yourself works, but you can teach your own type to be `Transferable`. Then the picker hands you a finished `TrailPhoto` in one call, and every place that loads photos shares the same decoding and the same error handling.

```swift
import SwiftUI
import PhotosUI

struct TrailPhoto: Transferable {
    let image: UIImage

    static var transferRepresentation: some TransferRepresentation {
        DataRepresentation(importedContentType: .image) { data in
            guard let image = UIImage(data: data) else {
                throw CocoaError(.fileReadCorruptFile)
            }
            return TrailPhoto(image: image)
        }
    }
}

func load(_ item: PhotosPickerItem) async -> TrailPhoto? {
    try? await item.loadTransferable(type: TrailPhoto.self)
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so this file can use UIImage and the Transferable protocol.
- **WHAT** **Makes SwiftUI and, on iOS, UIImage available; SwiftUI also brings in the Transferable protocol.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before TrailPhoto or load(_:) exists as runnable code.
- **WHY** Each file imports what it uses; this one needs UIImage and the transfer types.

2 `import PhotosUI`

- **WHO** The Swift compiler, resolving the PhotosUI module so load(_:) can accept a PhotosPickerItem.
- **WHAT** **Brings in PhotosPickerItem for the loading function at the bottom.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, when the compiler checks the parameter type of load(_:); nothing happens at run time.
- **WHY** Without it, the compiler would not know the PhotosPickerItem type.

4 `struct TrailPhoto: Transferable {`

- **WHO** Your code, declaring a model type.
- **WHAT** **Defines a TrailPhoto value and promises that it knows how to be created from transferred data.**
- **WHERE** At file scope, usable anywhere in the app.
- **WHEN** Checked at compile time; values are created when a load succeeds.
- **WHY** Conforming your own type to Transferable keeps decoding in one place instead of repeating it in every view.

```
5  let image: UIImage
```

WHO Your TrailPhoto struct, holding the one value its import closure produces.
WHAT **Stores the decoded picture as a UIImage, which can report its pixel size and be drawn by SwiftUI.**
WHERE A stored property inside each TrailPhoto value.
WHEN Set once when the value is created and never changed afterward.
WHY A let property makes TrailPhoto immutable, so a loaded photo cannot change behind your back.

```
7  static var transferRepresentation: some TransferRepresentation {
```

WHO The Transferable protocol, which requires this property.
WHAT **Describes the formats TrailPhoto can be created from or exported to.**
WHERE A static property, so it belongs to the type, not to any single photo.
WHEN Read by the system whenever something asks to load or share a TrailPhoto.
WHY A type-level description lets the picker, drag and drop, and the share sheet all understand your type the same way.

```
8  DataRepresentation(importedContentType: .image) { data in
```

WHO The Transferable system, using a uniform type identifier.
WHAT **Says that any image file can become a TrailPhoto, and hands the closure the file as Data.**
WHERE The single value returned by TrailPhoto's static transferRepresentation property.
WHEN The closure runs during loadTransferable, once the bytes have arrived.
WHY .image is the general "image" content type, so JPEG, HEIC and PNG photos all match.

```
9  guard let image = UIImage(data: data) else {
```

WHO Your import closure, with UIKit decoding.
WHAT **Tries to decode the bytes and jumps to the else block if they are not a valid picture.**
WHERE At the start of the import closure.
WHEN Once per load, after the data arrives.
WHY guard puts the failure case first and keeps the success path at the left margin.

```
10 throw CocoaError(.fileReadCorruptFile)
```

WHO Your import closure, raising a standard Foundation error.
WHAT **Reports that the file could not be read as an image.**
WHERE Inside the guard's else block.
WHEN Only when decoding fails.
WHY Throwing lets the caller decide how to react, instead of silently producing an empty photo.

```
12 return TrailPhoto(image: image)
```

WHO Your import closure inside DataRepresentation, returning the value the picker will hand back.
WHAT **Builds the finished TrailPhoto from the decoded image.**
WHERE The last line of the import closure.
WHEN After a successful decode.
WHY The caller receives a ready-to-use model value, not raw bytes.

```
17 func load(_ item: PhotosPickerItem) async -> TrailPhoto? {
```

WHO Your code, offering one helper for every screen.
WHAT **Declares an async function that turns a picked item into a TrailPhoto, or nil if that fails.**
WHERE At file scope, callable from any view's task.
WHEN Runs whenever a view calls it, typically from .task(id:) after a pick.
WHY One loading function keeps behavior consistent across the app.

```
18  try? await item.loadTransferable(type: TrailPhoto.self)
```

WHO	The PhotosPickerItem and your TrailPhoto conformance working together.
WHAT	**Loads the item directly as a TrailPhoto, running the import closure above, and turns any thrown error into nil.**
WHERE	The single expression of the function, so its value is returned automatically.
WHEN	When load is called and awaited.
WHY	Asking for your own type instead of Data moves decoding out of the view and makes the view code shorter.

Try it

1. Add a second guard that throws when image.size.width is under 600. Which error would you throw?
2. Rewrite the .task in the previous step to call load(_:) and store the TrailPhoto.

CONCEPT

The camera is a protected sensor

The camera is different from the photo picker: your app sees every frame live, so iOS always asks the user first. You add the `NSCameraUsageDescription` key to Info.plist with a sentence that explains why, and iOS shows that sentence in its alert. Without the key, the app crashes as soon as it asks for camera access.

Camera code lives in **AVFoundation**, Apple's framework for audio and video. Its `AVCaptureDevice` type reports and requests camera permission.

AVCaptureSession	Connects a camera input to outputs such as still photos or video frames. Use it for a custom camera screen, like a barcode scanner.
UIImagePickerController	A ready-made UIKit camera screen from before SwiftUI. Apps wrap it in a UIViewControllerRepresentable, a SwiftUI adapter for UIKit screens, to take one quick photo.
Microphone	Recording video with sound also needs NSMicrophoneUsageDescription and a second alert.

The Simulator has no camera

The iOS Simulator on your Mac cannot provide camera frames, so test camera features on a real iPhone. The photo picker does work in the Simulator, which includes a few sample photos.

CODE · READ-ALONG

Check camera permission

Before showing a camera screen, ask AVFoundation where things stand. The answer is one of four states, and only one of them, `.notDetermined`, lets you show the system alert. Try each path in the simulation, including what happens when the Info.plist key is missing.

```swift
import AVFoundation

func canUseCamera() async -> Bool {
    switch AVCaptureDevice.authorizationStatus(for: .video) {
    case .authorized:
        return true
    case .notDetermined:
        return await AVCaptureDevice.requestAccess(for: .video)
    case .denied, .restricted:
        return false
    @unknown default:
        return false
    }
}
```

Line by line

1 `import AVFoundation`

- **WHO** The Swift compiler, resolving the AVFoundation module so canUseCamera() can call AVCaptureDevice.
- **WHAT** **Brings in AVFoundation, which defines AVCaptureDevice and its authorization API.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before canUseCamera() is built; AVFoundation is resolved then, not when the user taps.
- **WHY** Camera permission belongs to the capture framework, not to SwiftUI.

3 `func canUseCamera() async -> Bool {`

- **WHO** Your code, offering one question the rest of the app can ask.
- **WHAT** **Declares an async function that answers "may this app use the camera right now?", asking the user if needed.**
- **WHERE** At file scope, callable from a button action inside a Task.
- **WHEN** Each time the user starts a camera feature.
- **WHY** async lets the function wait for the user to answer the alert without freezing the screen.

4 `switch AVCaptureDevice.authorizationStatus(for: .video) {`

- **WHO** AVFoundation, which reads the decision iOS has stored for this app.
- **WHAT** **Gets the current camera permission and branches on it; .video means the camera, as opposed to .audio for the microphone.**
- **WHERE** At the start of the function.
- **WHEN** Instantly, every call; reading the status never shows an alert.
- **WHY** Checking first avoids asking again after the user has already decided, which iOS would ignore anyway.

5 `case .authorized:`

- **WHO** The switch over AVCaptureDevice.authorizationStatus(for: .video) inside canUseCamera().
- **WHAT** **Matches when the user already allowed camera access.**
- **WHERE** First case of the switch.
- **WHEN** Whenever permission was granted earlier.
- **WHY** The common path after the first launch comes first and returns immediately.

6 `return true`

- **WHO** Your canUseCamera() function, returning from the .authorized case.
- **WHAT** **Answers yes, so the caller can open the camera screen.**
- **WHERE** Inside the .authorized case.
- **WHEN** Right away, with no alert.
- **WHY** No waiting is needed when the answer is already known.

```
7  case .notDetermined:
```

WHO The same switch over the camera authorization status, matching its second pattern.
WHAT **Matches when the app has never asked, which is the only state in which iOS will show its alert.**
WHERE The second case of the switch in canUseCamera(), after .authorized.
WHEN Typically the first time the user taps a camera feature after installing.
WHY Asking here, at the moment of need, gives the request clear context.

```
8  return await AVCaptureDevice.requestAccess(for: .video)
```

WHO iOS, which shows the system alert with your NSCameraUsageDescription text.
WHAT **Shows the alert, waits for the user to tap Allow or Don't Allow, and returns their choice as a Bool.**
WHERE Inside the .notDetermined case.
WHEN Once per install; afterward the stored answer is used.
WHY The system, not your app, draws the alert, so users can trust it, and await keeps the interface responsive meanwhile.

```
9  case .denied, .restricted:
```

WHO The switch in canUseCamera(), matching two statuses with one case.
WHAT **Matches when the user said no, or when a parent or company policy blocks the camera.**
WHERE Third case, handling two states together.
WHEN Whenever access is off.
WHY Both mean "not now", and your app cannot change either one with code.

```
10 return false
```

WHO Your canUseCamera() function, returning from the .denied and .restricted case.
WHAT **Answers no, so the caller can explain the situation and offer a link to Settings.**
WHERE Inside the denied or restricted case.
WHEN Right away, with no alert, whenever the stored status is .denied or .restricted.
WHY A clear false lets the screen degrade gracefully instead of opening a black camera view.

```
11 @unknown default:
```

WHO The Swift compiler, which requires a plan for future cases.
WHAT **Catches any status Apple might add in a later iOS version.**
WHERE Last case of the switch.
WHEN Only if a new, unknown status appears.
WHY @unknown makes the compiler warn you when Apple adds a case, while the app stays safe meanwhile.

```
12 return false
```

WHO Your canUseCamera() function, returning from its @unknown default case.
WHAT **Treats any unknown status as "no access".**
WHERE Inside the @unknown default case.
WHEN Only for statuses this code does not know about.
WHY Failing closed is the safe choice for anything privacy related.

Try it

1. In the simulation, empty the purpose string and tap the camera feature. What happens, and why is iOS so strict?
2. After a user taps Don't Allow, canUseCamera() returns false. What should the screen offer next?

CODE · READ-ALONG

Share with ShareLink

ShareLink is a SwiftUI button that opens the system share sheet: the panel listing Messages, Mail, AirDrop and other apps. Your app hands over one Transferable item and never learns where it went. The user picks the destination, and the system does the rest.

```swift
import SwiftUI

struct TrailShareView: View {
    let trailURL = URL(string: "https://example.com/trails/42")!
    let photo = Image(systemName: "photo")

    var body: some View {
        VStack(spacing: 20) {
            ShareLink(item: trailURL,
                      subject: Text("Ridge Loop"),
                      message: Text("Hike this with me"))
            ShareLink(item: photo,
                      preview: SharePreview("Trail photo",
                                            image: photo)) {
                Label("Share Photo", systemImage: "photo")
            }
        }
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so TrailShareView can use ShareLink and SharePreview.
- **WHAT** **Makes SwiftUI available, including ShareLink and SharePreview.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before TrailShareView is built; ShareLink is resolved then, not when the sheet opens.
- **WHY** Sharing is built into SwiftUI, so no extra framework is needed.

3 `struct TrailShareView: View {`

- **WHO** Your TrailShareView struct, which SwiftUI creates and draws.
- **WHAT** **Declares a small screen with two share buttons.**
- **WHERE** At file scope, so any trail detail screen in the app can include a TrailShareView.
- **WHEN** Created whenever a parent view includes it.
- **WHY** Keeping sharing in its own view makes it easy to reuse on a trail detail screen.

4 `let trailURL = URL(string: "https://example.com/trails/42")!`

- **WHO** Foundation's URL(string:) initializer, building TrailShareView's trailURL constant.
- **WHAT** **Builds the web address of a trail page; the ! asserts the string is a valid URL.**
- **WHERE** A constant stored in each TrailShareView value.
- **WHEN** Created when the view value is created.
- **WHY** The force unwrap is safe only because the string is a fixed literal you can check by eye; never force unwrap user input.

```
5  let photo = Image(systemName: "photo")
```

WHO SwiftUI, creating an image from an SF Symbol.
WHAT **Stands in for a trail photo; in a real app this would be the image the user loaded.**
WHERE A constant in each TrailShareView value.
WHEN Created with the view value.
WHY SwiftUI's Image conforms to Transferable, so it can be shared directly.

```
7  var body: some View {
```

WHO SwiftUI, which reads TrailShareView's body to learn what to draw.
WHAT **Describes the screen.**
WHERE The computed body property of TrailShareView, below the two stored constants.
WHEN Whenever SwiftUI needs to draw it.
WHY Every view supplies its interface through body.

```
8  VStack(spacing: 20) {
```

WHO SwiftUI's VStack container, arranging the two ShareLink buttons.
WHAT **Stacks the two share buttons vertically, 20 points apart.**
WHERE The root view of TrailShareView's body, holding both ShareLinks.
WHEN Each time body runs.
WHY Simple vertical layout for two actions.

```
9  ShareLink(item: trailURL,
```

WHO SwiftUI, which draws a Share button with the standard icon.
WHAT **Creates a button that shares the trail URL when tapped.**
WHERE First child of the VStack.
WHEN The button appears with the view; the share sheet opens on tap.
WHY URL is Transferable, and apps like Messages show a rich link preview for it.

```
10 subject: Text("Ridge Loop"),
```

WHO The share sheet, passing hints to destinations.
WHAT **Suggests a subject line, which apps such as Mail use for the email subject.**
WHERE An argument of the first ShareLink.
WHEN Used only if the chosen destination supports a subject.
WHY A subject makes the shared item meaningful in an inbox.

```
11 message: Text("Hike this with me"))
```

WHO The system share sheet, passing the message to the destination the user picks.
WHAT **Suggests a message to accompany the link, which the user can still edit.**
WHERE The last argument of the first ShareLink.
WHEN Used when the user picks a destination that accepts text.
WHY Prefilled text saves the user typing without forcing words on them.

```
12 ShareLink(item: photo,
```

WHO SwiftUI's ShareLink view, this time initialized with the photo Image.
WHAT **Creates a second share button for the photo.**
WHERE Second child of the VStack.
WHEN Appears with the view; opens the share sheet on tap.
WHY Any Transferable value, including your own types, can be shared the same way.

```
13  preview: SharePreview("Trail photo",
```

WHO The share sheet header.
WHAT **Supplies the title shown at the top of the share sheet.**
WHERE Inside the second ShareLink's arguments.
WHEN Shown when the sheet opens.
WHY Images have no title of their own, so the preview tells the user what they are about to send.

```
14  image: photo)) {
```

WHO The share sheet header.
WHAT **Uses the photo itself as the thumbnail in the sheet header, then opens the custom label closure.**
WHERE Finishes the SharePreview and the ShareLink arguments.
WHEN Rendered when the sheet opens.
WHY A thumbnail lets the user confirm the right picture before sending it.

```
15  Label("Share Photo", systemImage: "photo")
```

WHO SwiftUI's Label view, used as the second ShareLink's custom label.
WHAT **Replaces the default Share label with your own text and icon.**
WHERE Inside the ShareLink's label closure.
WHEN Each time body runs.
WHY A specific label tells users exactly what will be shared.

> **Try it**
>
> 1. Replace trailURL with a String such as "Ridge Loop, 6.4 km". Does ShareLink still compile? Why?
> 2. Add a subject and a message to the photo ShareLink.

CHECK YOURSELF

Check your understanding

1. An app lets people choose one photo as a profile picture, using `PhotosPicker`. What permission setup does it need?

   A. The NSPhotoLibraryUsageDescription key and a permission alert before the picker opens
   B. None: the picker runs outside the app and hands back only the chosen item
   C. The NSCameraUsageDescription key, because photos come from the camera
   D. Full access through PHPhotoLibrary, requested at launch

2. In the PostcardView example, the user picks a large photo and then, while it is still loading, picks a second one. What does `.task(id: item)` do?

   A. Runs both loads in parallel; whichever finishes last wins
   B. Ignores the second pick until the first load finishes
   C. Cancels the running task and starts a new one
   D. Crashes, because item changed while a task was using it

3. A custom barcode scanner crashes on a real iPhone the moment it calls `AVCaptureDevice.requestAccess(for: .video)`. What is the most likely cause?

   A. Info.plist has no NSCameraUsageDescription key
   B. The user denied camera access earlier
   C. requestAccess must be called on a background thread
   D. The Simulator has no camera

4. You want users to send a trail photo to a friend through Messages or AirDrop. Which approach fits iOS best?

   A. Ask for full library access, then save the photo where Messages can find it
   B. Build your own list of messaging apps and open each one with a URL
   C. Copy the photo to the clipboard for pasting
   D. Use ShareLink with the image and a SharePreview

Answers: Photos, Camera and Sharing

1. **B. None: the picker runs outside the app and hands back only the chosen item** The system picker runs in its own process, so the app sees nothing until the user chooses, and iOS shows no alert. The library usage key is a reasonable guess, but it is only needed when your code reads the whole library through PHPhotoLibrary, which a simple picker never does.
2. **C. Cancels the running task and starts a new one** When the id value changes, SwiftUI cancels the old task and starts a fresh one, so the screen always reflects the latest pick. Running both in parallel is the bug this modifier prevents: a slow first load could finish last and overwrite the newer photo.
3. **A. Info.plist has no NSCameraUsageDescription key** Touching the camera without a usage description makes iOS stop the app immediately. A previous denial does not crash anything; it just returns false. The Simulator answer does not fit, because the crash happens on a real device, so the missing key is the culprit.
4. **D. Use ShareLink with the image and a SharePreview** ShareLink opens the system share sheet: the user chooses the destination and your app only hands over the item. A custom list of apps sounds flexible but is brittle, cannot reach AirDrop, and needs permissions and code that the share sheet already provides.

On-Device Intelligence

CONCEPT

A language model that lives on the phone

A **language model** is a program trained on huge amounts of text to predict likely next words, which lets it write, summarize and answer in plain language. In iOS 26 and later, the **Foundation Models** framework gives your app direct access to the on-device model behind Apple Intelligence.

Because the model runs on the phone, the user's text never leaves the device, features work offline, and each request is free: no server, no account, no API key.

Good at	Summarizing, pulling details out of text, classifying and tagging, and writing short, friendly text.
Not a fact source	A compact model has limited knowledge of the world. It can sound confident and still be wrong, so never present its output as truth.
Requirements	A device that supports Apple Intelligence, with Apple Intelligence turned on in Settings and the model downloaded.

> **Treat it as an extra**
>
> Many devices cannot run the model. The app must still work without it, and the feature should hide or explain itself when the model is missing.

CODE · READ-ALONG

Check availability first

Before offering a model-powered feature, ask `SystemLanguageModel` whether the model can run right now. The answer is `.available` or `.unavailable` with a reason, and each reason calls for a different message. Pick each state in the simulation to see which branch runs and what the user sees.

```swift
import FoundationModels

func availabilityMessage() -> String {
    let model = SystemLanguageModel.default
    switch model.availability {
    case .available:
        return "Ready to suggest trips."
    case .unavailable(.deviceNotEligible):
        return "This device cannot run the model."
    case .unavailable(.appleIntelligenceNotEnabled):
        return "Turn on Apple Intelligence in Settings."
    case .unavailable(.modelNotReady):
        return "The model is still downloading."
    case .unavailable:
        return "Trip ideas are unavailable right now."
    }
}
```

Line by line

```
1  import FoundationModels
```

WHO The Swift compiler, resolving the FoundationModels module so this file can name SystemLanguageModel.
WHAT **Brings in the Foundation Models framework, which defines SystemLanguageModel and its availability states.**
WHERE At the top of the file.
WHEN At compile time, before availabilityMessage() is ever called; nothing is imported while the app runs.
WHY The language model is its own framework, available in iOS 26 and later.

```
3  func availabilityMessage() -> String {
```

WHO Your code, offering one answer the interface can display.
WHAT **Declares a function that turns the model's current state into a short message for the user.**
WHERE At file scope, callable from any view.
WHEN Each time a screen with the trip feature appears.
WHY Turning technical states into human words in one place keeps every screen consistent.

```
4  let model = SystemLanguageModel.default
```

WHO The Foundation Models framework.
WHAT **Gets the shared handle to the general-purpose on-device model.**
WHERE A local constant inside the function.
WHEN Each call; creating the handle is cheap and does not load the model.
WHY default is the model built into the system, so there is nothing to download or bundle with your app.

```
5  switch model.availability {
```

WHO The system, which knows the device, its settings and the model's download state.
WHAT **Reads the current availability and branches on it.**
WHERE The first statement of availabilityMessage(), right after the model handle is fetched.
WHEN Every time a screen calls availabilityMessage(); the check returns at once because it reads system state, not the model.
WHY Availability can change while the app runs, for example when a download finishes, so check it rather than caching the answer.

```
6  case .available:
```

WHO The switch over model.availability in availabilityMessage(), comparing against the .available case.
WHAT **Matches when the model is ready to use.**
WHERE The first case of the switch, the only branch that means the feature can run.
WHEN On supported devices with Apple Intelligence on and the model downloaded.
WHY The success path comes first and is the only one that should show the feature.

```
7  return "Ready to suggest trips."
```

WHO Your availabilityMessage() function, handing a String back to the view.
WHAT **Returns the ready message, which the view shows beside a button that starts the feature.**
WHERE Inside the .available case.
WHEN Whenever the model is ready.
WHY A positive, specific message tells the user what the feature does.

```
8  case .unavailable(.deviceNotEligible):
```

WHO Your switch, matching an associated value.
WHAT **Matches when the device hardware cannot run the model at all.**
WHERE Second case; .unavailable carries a reason, and this pattern checks for one specific reason.
WHEN On older or unsupported devices, every time.
WHY Nothing the user does will change this, so the app should quietly hide the feature.

```
9   return "This device cannot run the model."
```

WHO Your availabilityMessage() function, inside the .deviceNotEligible branch.
WHAT **Returns a message saying the device cannot run the model.**
WHERE Inside the deviceNotEligible case.
WHEN Whenever the device is not supported.
WHY An honest message beats a button that silently fails.

```
10  case .unavailable(.appleIntelligenceNotEnabled):
```

WHO The switch in availabilityMessage(), matching the .unavailable case whose reason is .appleIntelligenceNotEnabled.
WHAT **Matches when the device could run the model but the user has turned Apple Intelligence off.**
WHERE The third case of the switch, between the device check and the download check.
WHEN Until the user turns it on in Settings.
WHY This state is the user's choice, so the app can explain the benefit, but must respect the decision.

```
11  return "Turn on Apple Intelligence in Settings."
```

WHO Your availabilityMessage() function, inside the .appleIntelligenceNotEnabled branch.
WHAT **Returns a message pointing the user to Settings.**
WHERE Inside the appleIntelligenceNotEnabled case.
WHEN Whenever Apple Intelligence is off.
WHY Telling users exactly where to go saves them a search.

```
12  case .unavailable(.modelNotReady):
```

WHO The switch in availabilityMessage(), matching .unavailable with the reason .modelNotReady.
WHAT **Matches when the model is still downloading or the system is preparing it.**
WHERE The fourth case of the switch, the last one that names a specific reason.
WHEN Often shortly after Apple Intelligence is turned on.
WHY This state is temporary, so the app should check again later instead of hiding the feature for good.

```
13  return "The model is still downloading."
```

WHO Your availabilityMessage() function, inside the .modelNotReady branch.
WHAT **Returns a message saying the model is not ready yet.**
WHERE Inside the modelNotReady case.
WHEN While the download or preparation is in progress.
WHY Setting the expectation that it will work soon keeps users from giving up on the feature.

```
14  case .unavailable:
```

WHO Your switch, matching any remaining reason.
WHAT **Catches every other unavailable reason, including any Apple adds in future versions.**
WHERE Last case, after the specific reasons.
WHEN Only for reasons the cases above did not name.
WHY Listing specific cases first and a general one last makes the switch complete, so the compiler accepts it.

```
15  return "Trip ideas are unavailable right now."
```

WHO Your availabilityMessage() function, inside the catch-all .unavailable branch.
WHAT **Returns a general fallback message.**
WHERE Inside the catch-all case.
WHEN Only when the system reports a reason the earlier cases did not list.
WHY A safe default keeps the interface sensible even for states you did not foresee.

Try it

1. Add a Bool property isTripFeatureVisible that is true only for .available and .modelNotReady. Why include the second one?
2. In the simulation, select each state. Which ones show a button, and why?

CODE · READ-ALONG

Ask with a session

A **session** is one conversation with the model. You give it *instructions*, standing rules you write that shape every answer, then send *prompts*, the individual requests. The session keeps a transcript, so a later prompt can build on earlier ones.

```
1  import FoundationModels
2
3  @MainActor
4  final class TripPlanner {
5      private let session = LanguageModelSession(
6          instructions: "You suggest short, friendly day trips.")
7
8      func suggest(near place: String) async -> String {
9          do {
10             let response = try await session.respond(
11                 to: "Suggest a day trip near \(place).")
12             return response.content
13         } catch let error as LanguageModelSession.GenerationError {
14             return "No suggestion: \(error.localizedDescription)"
15         } catch {
16             return "Something went wrong. Try again."
17         }
18     }
19 }
```

Line by line

1 `import FoundationModels`

- WHO — The Swift compiler, resolving the FoundationModels module so TripPlanner can use LanguageModelSession.
- WHAT — **Brings in LanguageModelSession and its error type.**
- WHERE — At the top of the file.
- WHEN — At compile time, before any TripPlanner is created; the import costs nothing at run time.
- WHY — Everything for talking to the on-device model lives in this framework.

3 `@MainActor`

- WHO — Swift's main actor, the global actor that owns the main thread, applied to the whole TripPlanner class.
- WHAT — **Ties every use of this class to the main actor, the context that runs interface code.**
- WHERE — An attribute on the class declaration below.
- WHEN — Enforced at compile time.
- WHY — The planner feeds a view, so keeping it on the main actor avoids data races when the view reads its results.

```
4  final class TripPlanner {
```

WHO Your TripPlanner class, which a view keeps alive for one conversation.
WHAT **Declares a class that owns one conversation with the model.**
WHERE At file scope; a view would keep one instance for as long as the conversation lasts.
WHEN Created when the trip screen creates it.
WHY A class gives the session one shared identity, so every prompt goes to the same conversation; final means no subclasses.

```
5  private let session = LanguageModelSession(
```

WHO The Foundation Models framework.
WHAT **Creates the session that holds the transcript of this conversation.**
WHERE A private stored property of TripPlanner, hidden from other code.
WHEN When the TripPlanner is created.
WHY Reusing one session keeps context between prompts; a new session would forget everything said before.

```
6      instructions: "You suggest short, friendly day trips.")
```

WHO You, the developer, writing standing rules for the model.
WHAT **Sets instructions that apply to every prompt in this session.**
WHERE The argument that finishes the session initializer.
WHEN Sent to the model with each request in this session.
WHY Instructions come from your code, never from the user, so they are the right place for tone, length and safety rules.

```
8  func suggest(near place: String) async -> String {
```

WHO Your TripPlanner class, declaring the one method views call to get a suggestion.
WHAT **Declares an async method that returns a trip suggestion for a place.**
WHERE An instance method of TripPlanner, so it can reach the private session property.
WHEN Each time the user asks for a suggestion, from a Task in a button action.
WHY async lets the method wait for the model without freezing the screen while text is generated.

```
9      do {
```

WHO Swift's do-catch, wrapping the throwing call to session.respond(to:).
WHAT **Opens a block whose errors flow to the catch clauses below.**
WHERE At the start of the method body.
WHEN Every time suggest(near:) runs, before the prompt is sent to the model.
WHY Generation can fail for several reasons, and the user deserves a readable message for each.

```
10         let response = try await session.respond(
```

WHO The on-device model, through the session.
WHAT **Sends a prompt, waits for the complete answer, and stores it in response.**
WHERE Inside the do block.
WHEN Each call; the model generates on the device, so no network is needed.
WHY try marks that generation can throw, and await marks the wait; a session handles one request at a time.

```
11             to: "Suggest a day trip near \(place).")
```

WHO Your code, building the prompt.
WHAT **Inserts the user's place into the prompt with string interpolation.**
WHERE The argument of respond(to:).
WHEN Built just before the request is sent.
WHY Instructions hold the rules and the prompt holds the specific request, which keeps both easy to change.

```
12  return response.content
```

WHO Your suggest(near:) method, reading the content property of the model's response.
WHAT **Returns the generated text, which the response exposes as content.**
WHERE Inside the do block, after the answer arrives.
WHEN Only when session.respond(to:) returned without throwing, so a full answer exists.
WHY The response also carries the transcript entries, but the view only needs the text.

```
13  } catch let error as LanguageModelSession.GenerationError {
```

WHO Swift error handling, matching the framework's own error type.
WHAT **Catches generation errors, such as a guardrail violation (the safety system flagged the prompt or answer) or a transcript too long for the context window.**
WHERE After the do block.
WHEN Only when the model reports a generation problem.
WHY Matching the specific type first lets you react to model problems differently from unexpected ones.

```
14  return "No suggestion: \(error.localizedDescription)"
```

WHO Your suggest(near:) method, inside the catch clause for LanguageModelSession.GenerationError.
WHAT **Returns a message that includes the error's readable description.**
WHERE Inside the GenerationError catch clause.
WHEN Only when the model threw a GenerationError, such as a guardrail violation or a transcript that outgrew the context window.
WHY Users see why nothing appeared instead of a blank screen.

```
15  } catch {
```

WHO Swift's final catch clause, which receives whatever the GenerationError clause did not match.
WHAT **Catches any other error that respond might throw.**
WHERE The last catch clause.
WHEN Only for errors not already matched.
WHY A do block in a non-throwing method must handle every error, so a general catch completes it.

```
16  return "Something went wrong. Try again."
```

WHO Your suggest(near:) method, inside the last catch clause.
WHAT **Returns a general fallback message.**
WHERE Inside the general catch.
WHEN Only after an unexpected failure, which should be uncommon.
WHY A calm, actionable message keeps the user's trust when something unexpected happens.

Try it

1. Change the instructions to ask for answers under 20 words. Does the prompt line need to change too?
2. Call suggest(near: "Lisbon"), then suggest(near: "somewhere warmer"). Why can the second prompt skip the city name?

CODE · READ-ALONG

Get Swift values back with @Generable

Free text is hard for code to use. With **guided generation**, you describe a Swift type and the framework makes the model fill it in. `@Generable` is a *macro*, code that writes code at compile time, which teaches the framework your type's shape. `@Guide` adds a description or a rule to one property.

```swift
import FoundationModels

@Generable
struct TripIdea {
    @Guide(description: "A short, catchy title")
    var title: String
    @Guide(description: "Things to do", .count(3))
    var activities: [String]
    @Guide(.range(1...5))
    var days: Int
}

func planTrip(to place: String) async throws -> TripIdea {
    let session = LanguageModelSession()
    let response = try await session.respond(
        to: "Plan a trip to \(place).",
        generating: TripIdea.self)
    return response.content
}
```

Line by line

1 `import FoundationModels`

WHO	The Swift compiler, resolving the FoundationModels module so the @Generable and @Guide macros can expand in this file.
WHAT	**Brings in the @Generable and @Guide macros and the session type.**
WHERE	At the top of the file.
WHEN	At compile time, before the macros below are expanded; nothing is imported while the app runs.
WHY	The macros live in the same framework as the model.

3 `@Generable`

WHO	A Swift macro from Foundation Models, run by the compiler.
WHAT	**Generates code that describes TripIdea's properties as a schema the model must follow.**
WHERE	Attached to the struct declaration below.
WHEN	At compile time; the schema is used at every request that generates a TripIdea.
WHY	You write a normal Swift type and the macro produces the description, so the schema can never drift from the type.

4 `struct TripIdea {`

WHO	Your TripIdea struct, the shape the @Generable macro turns into a schema.
WHAT	**Declares the value the model will produce: a title, a list of activities and a number of days.**
WHERE	At file scope, directly under the @Generable attribute, so planTrip(to:) and any view can use it.
WHEN	Values are created by the framework when a response arrives.
WHY	A struct is a plain value, easy to show in a view or save.

5 `@Guide(description: "A short, catchy title")`

WHO	The @Guide macro from Foundation Models, attached to the title property.
WHAT	**Attaches a natural-language hint to the next property.**
WHERE	Directly above the title property.
WHEN	Included in the schema at compile time and read by the model during generation.
WHY	Clear descriptions steer the model the same way a good prompt does, but per field.

```
6  var title: String
```

WHO Your TripIdea struct, declaring its first stored property.
WHAT **Holds the trip's title.**
WHERE A stored property of TripIdea.
WHEN Filled first, because the model generates properties in the order you declare them.
WHY Declaration order matters: put the fields that help the model reason, such as a title, before the details.

```
7  @Guide(description: "Things to do", .count(3))
```

WHO The @Guide macro, attached to activities with both a description and a .count(3) rule.
WHAT **Describes the next property and adds a rule: the array must contain exactly three items.**
WHERE Directly above the activities property, between the title and the days declarations.
WHEN Enforced while the model generates.
WHY The framework constrains decoding, so the model cannot produce two or four items; your code never has to check.

```
8  var activities: [String]
```

WHO Your TripIdea struct, declaring the array that .count(3) applies to.
WHAT **Holds the list of activities.**
WHERE A stored property of TripIdea.
WHEN Filled after the title.
WHY An array of strings maps directly onto a List or ForEach in SwiftUI.

```
9  @Guide(.range(1...5))
```

WHO The @Guide macro, attached to days with only a .range(1...5) rule and no description.
WHAT **Adds a rule that the next number must be between 1 and 5.**
WHERE Directly above the days property, the last stored property of TripIdea.
WHEN Enforced while the model generates the days value, after title and activities are filled.
WHY A range guarantees a sensible value, so the interface never shows a 40-day day trip.

```
10 var days: Int
```

WHO Your TripIdea struct, declaring the Int that .range(1...5) constrains.
WHAT **Holds the suggested trip length as a real Int.**
WHERE A stored property of TripIdea.
WHEN Filled last, after title and activities, because the model generates properties in declaration order.
WHY A typed number can drive a Stepper or a calculation directly, with no text parsing.

```
13 func planTrip(to place: String) async throws -> TripIdea {
```

WHO Your planTrip(to:) function, the one place that asks the model for a TripIdea.
WHAT **Declares an async function that asks the model for a TripIdea and passes any error to the caller.**
WHERE At file scope, below the TripIdea struct it returns, callable from any view's Task.
WHEN Each time the user requests a plan.
WHY throws lets the calling view show its own error message.

```
14 let session = LanguageModelSession()
```

WHO The Foundation Models framework.
WHAT **Creates a fresh session with no instructions.**
WHERE A local constant inside the function.
WHEN Each call, so every plan starts with no earlier context.
WHY A one-off request does not need a long-lived conversation.

```
15 let response = try await session.respond(
```

WHO The on-device language model, reached through session.respond(to:generating:).
WHAT **Sends the request and waits for a complete, typed answer.**
WHERE Inside planTrip(to:), right after the fresh session is created.
WHEN Every time planTrip(to:) runs; the call suspends until the model has produced the whole TripIdea.
WHY The same respond method works for text and for typed values; the generating argument decides.

```
16 to: "Plan a trip to \(place).",
```

WHO Your planTrip(to:) function, interpolating the place parameter into the prompt.
WHAT **Builds the prompt with the user's place.**
WHERE The first argument of respond.
WHEN Just before the request.
WHY The prompt can stay short because the type and its guides carry the structure.

```
17 generating: TripIdea.self)
```

WHO The framework's guided generation.
WHAT **Tells the session to produce a TripIdea instead of free text.**
WHERE The second argument of respond.
WHEN Applied while the model generates, so every token it produces must fit TripIdea's schema.
WHY Passing the type turns the model's output into a Swift value you can trust to match its shape.

```
18 return response.content
```

WHO Your planTrip(to:) function, reading the typed content of the response.
WHAT **Returns the finished TripIdea.**
WHERE The last line of planTrip.
WHEN After session.respond(to:generating:) returns, once the whole TripIdea has been generated.
WHY content is already a TripIdea, so there is no JSON decoding step.

> **Try it**
>
> 1. Change .count(3) to .count(5). What will every TripIdea now contain?
> 2. Add @Guide(description: "Best season to visit") var season: String. Does planTrip need to change?

INTERACTIVE

Plain text versus guided generation

Pick a prompt and generate in **Plain text** mode, then switch to **Guided** mode and generate again. Change the `.count` and `.range` rules and watch both the code and the output follow them. Notice that guided output arrives as Swift values filling in field by field and always obeys the rules, while plain text leaves your code parsing prose. The responses here are scripted so you can see the mechanics; the real model's words vary.

This step is an interactive simulation in the HTML version.

CONCEPT

Vision, Core ML and other machine-learning tools

Language is one kind of on-device intelligence. Apple ships several frameworks that run trained **machine-learning models**, programs that learned patterns from examples, directly on the phone. They use the Neural Engine, a part of the iPhone's chip built for this kind of math.

Vision	Analyzes images: reads text with RecognizeTextRequest and finds faces, barcodes and document edges.
Core ML	Runs models you trained or converted yourself, such as a plant classifier. Create ML, an app that comes with Xcode, trains simple models from labeled examples without code.
Natural Language	Detects a text's language, splits it into sentences and words, and tags names of people and places.
Speech and Translation	Turn speech into text and translate between languages, on device for supported languages.

> **Prefer on-device first**
>
> On-device processing keeps data private, works offline and costs nothing per request. Reach for a server model only when a task truly needs more knowledge or power, and tell users when their data leaves the phone.

CHECK YOURSELF

Check your understanding

1. Your trip feature checks availability and gets `.unavailable(.appleIntelligenceNotEnabled)`. What should the app do?
   A. Call respond(to:) anyway; the system will turn Apple Intelligence on
   B. Stop the app with fatalError, because a required framework is missing
   C. Explain the benefit, point to Settings, and keep the app working
   D. Hide the feature permanently, because the device cannot run the model
2. Why use `@Generable` with `respond(to:generating:)` instead of asking for JSON in a plain prompt?
   A. Generation follows the type's shape, so the value needs no parsing
   B. It gives the model more knowledge about the world
   C. It sends the request to a larger model on a server
   D. It skips the model and fills the struct with default values
3. A cooking app sends *"What is the population of Peru?"* to the on-device model. How should the app treat the answer?
   A. As reliable, because it was computed on the device
   B. As possibly wrong, because the model has limited world knowledge
   C. As current, because the model looks the number up online
   D. It will never answer, because the model refuses every factual question
4. An app needs to read the words printed on a photographed trail sign. Which framework fits best?
   A. Foundation Models, with a prompt asking what the sign says
   B. Natural Language, which splits text into words
   C. Core Location, which knows where the sign is
   D. Vision, with RecognizeTextRequest

Answers: On-Device Intelligence

1. **C. Explain the benefit, point to Settings, and keep the app working** This state means the device could run the model but the user turned Apple Intelligence off, so a gentle pointer to Settings respects their choice. Hiding the feature for good is the right response to .deviceNotEligible, not to this state, which the user can change.
2. **A. Generation follows the type's shape, so the value needs no parsing** Guided generation constrains decoding to your schema, so the result always matches the type and its guides. Asking for JSON in a prompt looks easier, but the model may produce invalid or incomplete JSON that your code must then detect and repair.
3. **B. As possibly wrong, because the model has limited world knowledge** The on-device model is compact and is not a fact source; it can state wrong numbers confidently. Running on device is about privacy and cost, not accuracy, and the model does not browse the web.
4. **D. Vision, with RecognizeTextRequest** Vision's text recognition finds and reads text in images. Natural Language sounds right because it works with words, but it needs text that has already been extracted; it cannot look at a photo.

Beyond the App: Widgets, Intents and Notifications

CONCEPT

Your app can appear outside its own window

People meet your app in many places besides its own screen: a widget on the Home Screen, a Live Activity on the Lock Screen, a request to Siri, a notification. Most of these run as **app extensions**: separate small programs bundled inside your app that iOS launches on its own schedule, with tight limits on memory and running time.

Widgets	Glanceable snapshots of your app's data on the Home Screen, Lock Screen and StandBy, built with WidgetKit.
Live Activities	A view of an event in progress, such as a delivery or a hike, that updates on the Lock Screen. Built with ActivityKit.
App Intents	Actions your app offers to Siri, Shortcuts, Spotlight, the Action button and Control Center controls.
Notifications	Alerts your app schedules on the device or your server sends, built with UserNotifications.

> **Sharing data with an extension**
>
> An extension has its own sandbox, so it cannot read your app's files. Add the *App Groups* capability to both targets: it gives them one shared container and a shared UserDefaults suite.

CODE · READ-ALONG

A timeline of snapshots

A widget never runs live code on the Home Screen. Instead, iOS asks your **timeline provider** for a *timeline*: a list of entries, each a snapshot of data with the date it should appear, plus a policy that says when to ask again. iOS then shows each entry at its date.

```swift
import WidgetKit

struct WaterEntry: TimelineEntry {
    let date: Date
    let glasses: Int
}

struct WaterProvider: TimelineProvider {
    func getTimeline(in context: Context,
                     completion: @escaping
                     (Timeline<WaterEntry>) -> Void) {
        let glasses = WaterStore.shared.glassesToday
        let step: TimeInterval = 3600 // 1 hour
        let start = Date.now
        let entries = (0..<4).map { i in
            WaterEntry(date: start + Double(i) * step,
                       glasses: glasses)
        }
        completion(Timeline(entries: entries,
                            policy: .atEnd))
    }
}
```

Line by line

1 `import WidgetKit`

- **WHO** The Swift compiler, resolving the WidgetKit module for the widget extension target.
- **WHAT** **Brings in WidgetKit, which defines TimelineEntry, TimelineProvider and Timeline.**
- **WHERE** At the top of a file in the widget extension target.
- **WHEN** At compile time, when Xcode builds the widget extension; nothing is imported while the widget runs.
- **WHY** Widgets are their own framework and their own target, separate from the main app.

3 `struct WaterEntry: TimelineEntry {`

- **WHO** Your code, conforming to a WidgetKit protocol.
- **WHAT** **Declares one snapshot of the widget's data.**
- **WHERE** At file scope in the widget extension.
- **WHEN** Values are created each time the provider builds a timeline.
- **WHY** TimelineEntry requires only a date, so you add whatever data your widget displays.

4 `let date: Date`

- **WHO** WidgetKit, which reads this property.
- **WHAT** **Stores the moment at which iOS should start showing this entry.**
- **WHERE** A required stored property of each entry.
- **WHEN** Read by the system to decide which entry is on screen now.
- **WHY** Dates let iOS switch entries on schedule without waking your code.

5 `let glasses: Int`

- **WHO** Your widget view, which reads this property.
- **WHAT** **Stores how many glasses of water the user had logged when the snapshot was made.**
- **WHERE** A stored property of each entry.
- **WHEN** Set when the provider builds the timeline; it never changes afterward.
- **WHY** Entries are frozen snapshots, which is exactly why a widget can show stale data until the next reload.

```
8  struct WaterProvider: TimelineProvider {
```

WHO Your code, conforming to TimelineProvider.
WHAT **Declares the type that supplies entries; its placeholder and getSnapshot methods (needed for previews and the widget gallery) are left out here to focus on the timeline.**
WHERE At file scope in the widget extension.
WHEN Created by the widget configuration; its methods are called by the system, never by you.
WHY Separating "which data, when" from "how it looks" keeps the widget view simple.

```
9  func getTimeline(in context: Context,
```

WHO WidgetKit, which calls this method.
WHAT **Begins the method that builds a whole timeline; context describes the widget, such as its size family.**
WHERE An instance method of WaterProvider, the one TimelineProvider requirement shown in full here.
WHEN When the widget is first added, when the policy says it is time, and when the app asks for a reload.
WHY The system decides when to call it, which protects battery life across every widget on the device.

```
10 completion: @escaping
```

WHO WidgetKit, which creates the completion closure and passes it into getTimeline(in:completion:).
WHAT **Receives a callback that you must call with the finished timeline; @escaping means it may be called after the method returns.**
WHERE The second parameter of getTimeline.
WHEN Passed in each time the system asks for a timeline.
WHY A callback lets you finish asynchronous work, such as a network fetch, before handing the timeline back.

```
11 (Timeline<WaterEntry>) -> Void) {
```

WHO The Swift type system.
WHAT **Spells out the callback's type: it takes a timeline of WaterEntry values and returns nothing.**
WHERE Finishes the parameter list and opens the method body.
WHEN Checked at compile time.
WHY The generic Timeline type ensures you can only deliver entries of the widget's own type.

```
12 let glasses = WaterStore.shared.glassesToday
```

WHO Your shared data store, read from the extension.
WHAT **Reads today's glass count, which the app saved in a container both targets can reach.**
WHERE The first line of getTimeline(in:completion:), before any entry is built.
WHEN Each time a new timeline is built.
WHY The extension cannot see the app's memory, so the app must save data where the extension can read it, such as an App Group.

```
13 let step: TimeInterval = 3600 // 1 hour
```

WHO Your getTimeline(in:completion:) method, choosing the entry spacing.
WHAT **Sets the spacing between entries to 3600 seconds, one hour.**
WHERE A local constant in getTimeline.
WHEN Each time a timeline is built.
WHY TimeInterval is a number of seconds; spacing entries lets the widget change on schedule with no new call.

```
14 let start = Date.now
```

WHO Foundation's Date type, through its static now property.
WHAT **Captures the current moment as the date of the first entry.**
WHERE A local constant in getTimeline.
WHEN At the moment the system calls the method.
WHY Starting at now means the first entry appears immediately.

```
15 let entries = (0..<4).map { i in
```

WHO Swift's map on a range.
WHAT **Builds four entries, one for each number from 0 to 3.**
WHERE Inside getTimeline(in:completion:), after step and start are known.
WHEN Each time a timeline is built.
WHY Planning several entries ahead lets the widget update for hours with a single call.

```
16 WaterEntry(date: start + Double(i) * step,
```

WHO Your code, inside the map closure.
WHAT **Creates an entry dated 0, 1, 2 and 3 hours from now.**
WHERE The body of the closure.
WHEN Once per number in the range.
WHY Double(i) converts the Int so it can be multiplied by the TimeInterval step.

```
17 glasses: glasses)
```

WHO Your map closure, finishing the WaterEntry initializer call.
WHAT **Gives every entry the same glass count; a richer widget could store a different goal or message in each one.**
WHERE The second argument of the WaterEntry initializer.
WHEN Once for each of the four numbers in 0..<4, as map builds that entry.
WHY Only the date differs here, so the entry shows when the snapshot was planned for.

```
19 completion(Timeline(entries: entries,
```

WHO Your code, handing the result back to WidgetKit.
WHAT **Delivers the timeline made of the four entries.**
WHERE The last statement of getTimeline.
WHEN Once per call, after the entries are ready.
WHY Calling completion is mandatory: until you do, the widget keeps showing its old timeline.

```
20 policy: .atEnd))
```

WHO WidgetKit's TimelineReloadPolicy, read from the policy argument of Timeline.
WHAT **Asks the system for a new timeline after the last entry's date passes.**
WHERE The second argument of Timeline.
WHEN Read by the system when it plans the next reload.
WHY .atEnd keeps the widget fed automatically; .after(date) picks a specific time and .never waits for the app to ask.

> **Try it**
>
> 1. Change 0..<4 to 0..<6 and the step to 1800. How far ahead does one timeline now reach?
> 2. In the simulation, log a glass in the app. Why does the widget keep showing the old number?
> 3. Switch the policy to .never. What must the app now do to refresh the widget?

CODE · READ-ALONG

Declare the widget

The provider supplies data; a **Widget** joins it to a view and describes the widget for the gallery, the screen where people choose widgets to add. In Xcode you add the target with File, New, Target, Widget Extension, and Xcode creates files like this one.

```swift
import SwiftUI
import WidgetKit

struct WaterWidgetView: View {
    let entry: WaterEntry
    var body: some View {
        Text("\(entry.glasses) of 8 glasses")
            .containerBackground(.fill.tertiary, for: .widget)
    }
}

struct WaterWidget: Widget {
    var body: some WidgetConfiguration {
        StaticConfiguration(kind: "Water",
                            provider: WaterProvider()) {
            WaterWidgetView(entry: $0)
        }
        .configurationDisplayName("Water")
        .description("Glasses of water you drank today.")
        .supportedFamilies([.systemSmall, .systemMedium])
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, resolving the SwiftUI module so WaterWidgetView can use View and Text.
- **WHAT** **Brings in SwiftUI, because widget views are ordinary SwiftUI views.**
- **WHERE** At the top of a file in the widget extension.
- **WHEN** At compile time, when the widget extension is built; nothing is imported while the widget is on screen.
- **WHY** You design widgets with the same views and modifiers as your app.

2 `import WidgetKit`

- **WHO** The Swift compiler, resolving the WidgetKit module so WaterWidget can adopt the Widget protocol.
- **WHAT** **Brings in the Widget protocol and StaticConfiguration.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, right after the SwiftUI import is resolved for this file.
- **WHY** WidgetKit adds the widget-specific types on top of SwiftUI.

4 `struct WaterWidgetView: View {`

- **WHO** Your WaterWidgetView struct, the view WidgetKit renders for each entry.
- **WHAT** **Declares the view that draws one entry.**
- **WHERE** At file scope in the widget extension.
- **WHEN** Rendered by iOS for each entry, ahead of time, into a snapshot.
- **WHY** A widget view is drawn once per entry and archived, so it cannot hold live state or run timers.

5 `let entry: WaterEntry`

- **WHO** The widget configuration, which passes the entry in.
- **WHAT** **Stores the snapshot this view displays.**
- **WHERE** A stored property of the view.
- **WHEN** Set when iOS renders this entry.
- **WHY** The view depends only on its entry, which makes widget rendering predictable.

```
6  var body: some View {
```

WHO SwiftUI, reading body when WidgetKit asks it to render an entry.
WHAT **Describes what the widget shows.**
WHERE The required body property of WaterWidgetView, below the entry property it reads.
WHEN When iOS renders an entry.
WHY Same body pattern as any SwiftUI view.

```
7  Text("\(entry.glasses) of 8 glasses")
```

WHO SwiftUI's Text view, interpolating entry.glasses into its string.
WHAT **Shows the entry's glass count against a goal of eight.**
WHERE The root view of the widget.
WHEN Each time WidgetKit renders an entry into a snapshot, ahead of the moment it will be shown.
WHY Short, glanceable text suits a small widget.

```
8  .containerBackground(.fill.tertiary, for: .widget)
```

WHO WidgetKit, which owns the widget's background.
WHAT **Declares the widget's background as a subtle system fill.**
WHERE A modifier on the root view.
WHEN Applied at render time.
WHY Declaring the background this way lets iOS remove or tint it where the widget appears, such as StandBy or a tinted Home Screen.

```
12 struct WaterWidget: Widget {
```

WHO Your code, conforming to WidgetKit's Widget protocol.
WHAT **Declares the widget itself: its identity, data source, view and gallery description.**
WHERE At file scope in the widget extension, listed in the extension's entry point.
WHEN Read by iOS when it loads the extension.
WHY One Widget type per widget kind keeps each widget's configuration in one place.

```
13 var body: some WidgetConfiguration {
```

WHO WidgetKit, reading body when it loads the widget extension.
WHAT **Describes the widget's configuration, much as a view's body describes its interface.**
WHERE The required body property of WaterWidget, which returns a WidgetConfiguration instead of a View.
WHEN When iOS loads the widget.
WHY Widget configurations use the same declarative style as views.

```
14 StaticConfiguration(kind: "Water",
```

WHO WidgetKit's StaticConfiguration initializer, called inside WaterWidget's body.
WHAT **Creates a widget the user does not configure, identified by the kind string "Water".**
WHERE The root of the configuration body.
WHEN When iOS loads the widget.
WHY The kind string is how the app later asks for a reload with reloadTimelines(ofKind: "Water").

```
15 provider: WaterProvider()) {
```

WHO WidgetKit, receiving the WaterProvider instance it will call for timelines.
WHAT **Connects the provider that builds timelines, then opens the closure that turns an entry into a view.**
WHERE The second argument of StaticConfiguration.
WHEN The provider is called whenever iOS needs a timeline.
WHY Keeping data and view separate means either can change without touching the other.

```
16 WaterWidgetView(entry: $0)
```

WHO Your closure, called by WidgetKit.
WHAT **Builds the view for each entry; $0 is the closure's first parameter, the entry.**
WHERE Inside the StaticConfiguration closure.
WHEN Once per entry rendered.
WHY Shorthand keeps a one-line closure readable.

```
18 .configurationDisplayName("Water")
```

WHO The widget gallery, which reads this modifier on WaterWidget's configuration.
WHAT **Sets the name shown when people browse widgets to add.**
WHERE A modifier on the configuration.
WHEN Read when the gallery opens.
WHY A clear name helps people find your widget among many.

```
19 .description("Glasses of water you drank today.")
```

WHO The widget gallery, reading the description modifier below the display name.
WHAT **Sets the one-line description under the name.**
WHERE A modifier on the configuration.
WHEN Read when the gallery opens.
WHY The description tells people what the widget is for before they add it.

```
20 .supportedFamilies([.systemSmall, .systemMedium])
```

WHO WidgetKit, reading the supportedFamilies modifier when it offers sizes.
WHAT **Limits the widget to the small and medium Home Screen sizes.**
WHERE The last modifier on the configuration.
WHEN Read when the gallery offers sizes.
WHY Only offer sizes your view was designed for, so it never appears stretched or cramped.

> **Try it**
>
> 1. Add .accessoryCircular to supportedFamilies. Where else could the widget now appear?
> 2. Below the count, show the entry time with Text(entry.date, style: .time).

INTERACTIVE

Watch a widget across a day

Scrub the clock or press Play to move through a day. Log a glass in the app and notice that the widget keeps its old snapshot until a reload happens. Try each reload policy, change the entry spacing, then press the button that calls `WidgetCenter.shared.reloadTimelines(ofKind:)`. The system chooses exact reload times: a frequently viewed widget typically gets about 40 to 70 refreshes a day, so plan entries ahead instead of reloading constantly.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

App Intents: actions Siri and Shortcuts can run

An **App Intent** is a Swift type that describes one action your app can perform, such as logging a glass of water. Once declared, the system can offer it in Siri, the Shortcuts app, Spotlight, the Action button and interactive widgets, often without opening your app.

```swift
import AppIntents

struct LogWaterIntent: AppIntent {
    static let title: LocalizedStringResource = "Log Water"
    @Parameter(title: "Glasses", default: 1)
    var glasses: Int

    func perform() async throws
        -> some IntentResult & ProvidesDialog {
        WaterStore.shared.add(glasses)
        return .result(dialog: "Logged \(glasses) glasses.")
    }
}

struct WaterShortcuts: AppShortcutsProvider {
    static var appShortcuts: [AppShortcut] {
        AppShortcut(intent: LogWaterIntent(),
                    phrases: ["Log water in \(.applicationName)"],
                    shortTitle: "Log Water",
                    systemImageName: "drop")
    }
}
```

Line by line

1 `import AppIntents`

- **WHO** The Swift compiler, resolving the AppIntents module so LogWaterIntent can adopt AppIntent.
- **WHAT** **Brings in the App Intents framework: AppIntent, @Parameter and AppShortcutsProvider.**
- **WHERE** At the top of a file in the app target.
- **WHEN** At compile time, when the app target is built; Xcode also extracts intent metadata during that build.
- **WHY** App Intents is the modern way to expose actions to the system; it replaces older SiriKit intent definition files.

3 `struct LogWaterIntent: AppIntent {`

- **WHO** Your code, conforming to AppIntent.
- **WHAT** **Declares one action the system can run on the user's behalf.**
- **WHERE** At file scope in the app.
- **WHEN** Discovered at build time; Xcode extracts its metadata so Siri and Shortcuts know about it after install.
- **WHY** Describing actions in Swift lets the system list them before your app ever runs.

4 `static let title: LocalizedStringResource = "Log Water"`

- **WHO** The Shortcuts app and Siri, which display this title.
- **WHAT** **Names the action in a form that can be translated into other languages.**
- **WHERE** A static property of the intent type.
- **WHEN** Read at build time and whenever the system shows the action.
- **WHY** A short verb phrase reads well in the Shortcuts editor and in spoken responses.

5 `@Parameter(title: "Glasses", default: 1)`

- **WHO** The App Intents framework, through a property wrapper.
- **WHAT** **Marks the next property as an input the user can set, with the label Glasses and a default of 1.**
- **WHERE** Directly above the glasses property.
- **WHEN** The value is filled in just before perform runs.
- **WHY** The default lets the action run with no questions, while Shortcuts users can still change it.

```
6  var glasses: Int
```

WHO Your LogWaterIntent struct, declaring the property that @Parameter wraps.
WHAT **Holds how many glasses to log.**
WHERE A stored property of the intent.
WHEN Set by the system before perform runs.
WHY A typed Int means Shortcuts shows a number field, not free text.

```
8  func perform() async throws
```

WHO The system, which calls this method when the action runs.
WHAT **Begins the method that does the actual work.**
WHERE An instance method of LogWaterIntent, the one method AppIntent requires.
WHEN Each time the user runs the action from Siri, Shortcuts or a widget.
WHY async throws lets the action wait for work and report failures to the system.

```
9  -> some IntentResult & ProvidesDialog {
```

WHO The Swift type system.
WHAT **Promises a result that also includes dialog, the text Siri speaks or Shortcuts shows.**
WHERE The return type of perform.
WHEN Checked at compile time.
WHY Declaring the result shape lets the system know a confirmation will follow.

```
10 WaterStore.shared.add(glasses)
```

WHO Your shared data store.
WHAT **Adds the glasses to today's total.**
WHERE The first statement of perform(), before the result is returned.
WHEN Each time the action runs.
WHY The intent reuses the app's own model code instead of duplicating the logic; a real app would then call WidgetCenter.shared.reloadTimelines(ofKind: "Water").

```
11 return .result(dialog: "Logged \(glasses) glasses.")
```

WHO The App Intents framework.
WHAT **Finishes the action and gives the system a confirmation to speak or display.**
WHERE The last line of perform.
WHEN After the work succeeds.
WHY A confirmation reassures the user that the action worked without opening the app.

```
15 struct WaterShortcuts: AppShortcutsProvider {
```

WHO Your code, conforming to AppShortcutsProvider.
WHAT **Declares ready-made shortcuts that exist as soon as the app is installed.**
WHERE At file scope in the app; an app has one provider.
WHEN Read at build time and by the system after install.
WHY Users can run these by voice without building anything in the Shortcuts app.

```
16 static var appShortcuts: [AppShortcut] {
```

WHO The App Intents framework.
WHAT **Lists the app's shortcuts.**
WHERE A static property of the provider.
WHEN Read by the system, not by your code.
WHY A static list lets Siri learn the phrases before the app runs.

```
17 AppShortcut(intent: LogWaterIntent(),
```

WHO The App Intents framework.
WHAT **Creates a shortcut that runs LogWaterIntent.**
WHERE The single element of the list.
WHEN Read when the system registers shortcuts.
WHY Each shortcut wraps one intent with phrases and a title.

```
18 phrases: ["Log water in \(.applicationName)"],
```

WHO Siri, which listens for these phrases.
WHAT **Defines what the user can say; .applicationName is replaced with the app's actual name.**
WHERE An argument of AppShortcut.
WHEN Used whenever the user speaks to Siri.
WHY Every phrase must include the app name, so Siri knows which app should handle it.

```
19 shortTitle: "Log Water",
```

WHO The Shortcuts app and Spotlight.
WHAT **Sets the short label shown on the shortcut tile.**
WHERE An argument of AppShortcut.
WHEN Displayed wherever the shortcut appears.
WHY A short title fits small tiles and search results.

```
20 systemImageName: "drop")
```

WHO The Shortcuts app and Spotlight.
WHAT **Picks the SF Symbol named drop as the shortcut's icon.**
WHERE The last argument of AppShortcut.
WHEN Displayed with the title.
WHY An icon makes the shortcut recognizable at a glance.

> **Try it**
>
> 1. Change default: 1 to default: 2. What happens the next time the user says the phrase?
> 2. Add a second phrase, "Drink water with \(.applicationName)". Why must it mention the app name?

CODE · READ-ALONG

Live Activities in brief

A **Live Activity** shows an event in progress on the Lock Screen and in the Dynamic Island, the pill-shaped area at the top of newer iPhones. The app sets `NSSupportsLiveActivities` to YES in Info.plist, defines the activity's data, then starts, updates and ends it. A server can also update it through push notifications while the app is closed.

```swift
import ActivityKit

struct HikeAttributes: ActivityAttributes {
    struct ContentState: Codable, Hashable {
        var kilometersLeft: Double
    }
    var trailName: String
}

func trackHike() async throws {
    let start = HikeAttributes.ContentState(kilometersLeft: 6.4)
    let activity = try Activity.request(
        attributes: HikeAttributes(trailName: "Ridge Loop"),
        content: ActivityContent(state: start, staleDate: nil))
    let halfway = HikeAttributes.ContentState(kilometersLeft: 3.2)
    await activity.update(
        ActivityContent(state: halfway, staleDate: nil))
    let done = HikeAttributes.ContentState(kilometersLeft: 0)
    await activity.end(
        ActivityContent(state: done, staleDate: nil),
        dismissalPolicy: .default)
}
```

Line by line

1 `import ActivityKit`

- WHO — The Swift compiler, resolving the ActivityKit module so HikeAttributes can adopt ActivityAttributes.
- WHAT — **Brings in ActivityKit, which defines ActivityAttributes, Activity and ActivityContent.**
- WHERE — At the top of a file in the app target.
- WHEN — At compile time, when the app target is built; nothing is imported while the hike runs.
- WHY — The app starts and updates Live Activities; a widget extension draws them.

3 `struct HikeAttributes: ActivityAttributes {`

- WHO — Your code, conforming to ActivityAttributes.
- WHAT — **Describes the data for a hike Live Activity.**
- WHERE — At file scope, shared by the app and the widget extension that draws it.
- WHEN — Used when the activity starts and every time it updates.
- WHY — One type describes both the fixed details and the changing state.

4 `struct ContentState: Codable, Hashable {`

- WHO — ActivityKit, which requires this nested type.
- WHAT — **Declares the part of the activity that changes over time.**
- WHERE — Nested inside HikeAttributes, where ActivityKit expects a type named ContentState.
- WHEN — A new value is sent with every update.
- WHY — Codable lets the system store the state and receive it from push updates; Hashable lets it detect changes.

5 `var kilometersLeft: Double`

- WHO — Your HikeAttributes.ContentState struct, declaring its only property.
- WHAT — **Holds the distance remaining on the trail.**
- WHERE — A property of each state value.
- WHEN — Changes with each update.
- WHY — Keep changing data small, because every update must carry all of it.

```
7  var trailName: String
```

WHO Your HikeAttributes struct, declaring a property outside ContentState.
WHAT **Holds the trail's name, which stays the same for the whole activity.**
WHERE A stored property of HikeAttributes, outside ContentState.
WHEN Set once, when the activity starts.
WHY Separating fixed data from changing data keeps updates small.

```
10 func trackHike() async throws {
```

WHO Your trackHike() function, which owns the activity from start to end.
WHAT **Declares a function that walks through an activity's whole life.**
WHERE At file scope in the app.
WHEN Here all at once for illustration; a real app spreads these steps over the hike, for example after each location update.
WHY async because update and end wait for the system; throws because starting can fail.

```
11 let start = HikeAttributes.ContentState(kilometersLeft: 6.4)
```

WHO Your trackHike() function, calling ContentState's memberwise initializer.
WHAT **Creates the first state: 6.4 kilometers to go.**
WHERE A local constant in trackHike.
WHEN Before the activity starts.
WHY An activity always begins with a complete state.

```
12 let activity = try Activity.request(
```

WHO ActivityKit's static Activity.request(attributes:content:), which asks iOS to show the activity.
WHAT **Asks the system to start the Live Activity and returns a handle to it.**
WHERE Inside trackHike(), right after the start state is built.
WHEN When the hike begins; it must start while the app is in the foreground.
WHY try because the request fails if the user turned Live Activities off for your app.

```
13 attributes: HikeAttributes(trailName: "Ridge Loop"),
```

WHO Your trackHike() function, building the HikeAttributes value for the request.
WHAT **Supplies the fixed details: the trail name.**
WHERE An argument of request.
WHEN Once, when Activity.request runs; the attributes cannot change for the life of the activity.
WHY Fixed details never need to be sent again.

```
14 content: ActivityContent(state: start, staleDate: nil))
```

WHO ActivityKit's ActivityContent initializer, wrapping the start state.
WHAT **Wraps the starting state; staleDate nil means the system never marks it as outdated.**
WHERE The last argument of request.
WHEN When Activity.request runs, as the content the Lock Screen shows first.
WHY A stale date lets the system flag old data if updates stop arriving.

```
15 let halfway = HikeAttributes.ContentState(kilometersLeft: 3.2)
```

WHO Your trackHike() function, building a second ContentState value.
WHAT **Creates a new state halfway along the trail.**
WHERE A local constant in trackHike.
WHEN Here right after the start; in a real app, whenever a location update shows the hiker has moved.
WHY Each update replaces the whole changing state.

16 `await activity.update(`

WHO ActivityKit's Activity.update(_:), called on the handle returned by request.
WHAT **Sends the new state to the Lock Screen and Dynamic Island.**
WHERE Inside trackHike, using the handle from request.
WHEN Each time progress changes.
WHY await waits for the system to accept the update.

17 `ActivityContent(state: halfway, staleDate: nil))`

WHO ActivityKit's ActivityContent initializer, wrapping the halfway state.
WHAT **Wraps the halfway state for delivery.**
WHERE The argument of update.
WHEN Each time activity.update(_:) is called with new progress.
WHY The same wrapper type is used for start, update and end.

18 `let done = HikeAttributes.ContentState(kilometersLeft: 0)`

WHO Your trackHike() function, building the ContentState for the finish.
WHAT **Creates the final state with no distance left.**
WHERE A local constant in trackHike.
WHEN When the hike finishes.
WHY Ending with a final state shows the user a clear result.

19 `await activity.end(`

WHO ActivityKit's Activity.end(_:dismissalPolicy:), called on the same handle.
WHAT **Ends the Live Activity.**
WHERE The last statement of trackHike.
WHEN When the event is over.
WHY Activities should end promptly; leaving them running wastes the user's attention.

20 `ActivityContent(state: done, staleDate: nil),`

WHO ActivityKit's ActivityContent initializer, wrapping the done state.
WHAT **Supplies the final state to show.**
WHERE The first argument of end.
WHEN When activity.end runs, as the last content the Lock Screen shows.
WHY The final state stays visible briefly so the user sees the outcome.

21 `dismissalPolicy: .default)`

WHO ActivityKit's ActivityUIDismissalPolicy, read from the dismissalPolicy argument of end.
WHAT **Lets the system decide how long the ended activity remains on the Lock Screen.**
WHERE The last argument of end.
WHEN After the activity ends.
WHY The default respects the user's attention; .immediate removes it at once.

Try it

1. Add a startedAt: Date property to HikeAttributes. Should it go inside ContentState? Why or why not?
2. Give the final update a staleDate 15 minutes from now. What would the system do if no further update arrived?

CODE · READ-ALONG

Schedule a local notification

A **local notification** is one your app schedules on the device itself, with no server. You ask permission once, describe the content, choose a *trigger* (after a delay, at a calendar time, or on arriving somewhere), and hand the request to the notification center. iOS delivers it even if your app is not running.

```swift
import UserNotifications

func scheduleSunsetReminder() async throws {
    let center = UNUserNotificationCenter.current()
    let granted = try await center.requestAuthorization(
        options: [.alert, .sound, .badge])
    guard granted else { return }

    let content = UNMutableNotificationContent()
    content.title = "Sunset soon"
    content.body = "Head back to the trailhead."
    content.sound = .default

    let trigger = UNTimeIntervalNotificationTrigger(
        timeInterval: 30 * 60, repeats: false)
    let request = UNNotificationRequest(
        identifier: "sunset", content: content, trigger: trigger)
    try await center.add(request)
}
```

Line by line

1 `import UserNotifications`

- **WHO** The Swift compiler, resolving the UserNotifications module so this file can use UNUserNotificationCenter.
- **WHAT** **Brings in the UserNotifications framework.**
- **WHERE** At the top of the file.
- **WHEN** At compile time, before scheduleSunsetReminder() exists as runnable code.
- **WHY** Notifications have their own framework, used by both local and push notifications.

3 `func scheduleSunsetReminder() async throws {`

- **WHO** Your scheduleSunsetReminder() function, called when the user turns the reminder on.
- **WHAT** **Declares an async function that asks permission and schedules one reminder.**
- **WHERE** At file scope in the app.
- **WHEN** When the user turns on sunset reminders, a moment that explains why permission is needed.
- **WHY** Asking in context, right after the user chose the feature, earns more Allow answers than asking at launch.

4 `let center = UNUserNotificationCenter.current()`

- **WHO** The system notification center.
- **WHAT** **Gets the object that manages this app's notifications.**
- **WHERE** A local constant inside scheduleSunsetReminder(), referring to the one shared center.
- **WHEN** Each time scheduleSunsetReminder() runs, before the permission request.
- **WHY** One shared center per app handles permission, scheduling and removal.

```
5  let granted = try await center.requestAuthorization(
```

WHO iOS, which shows the system alert the first time.
WHAT **Asks for permission and waits for the answer; later calls return the stored answer with no alert.**
WHERE Inside scheduleSunsetReminder(), the first await in the function.
WHEN The alert appears once per install; after that the call returns immediately.
WHY Notifications need no Info.plist usage key: the system writes the alert text itself.

```
6  options: [.alert, .sound, .badge])
```

WHO Your scheduleSunsetReminder() function, choosing the UNAuthorizationOptions to request.
WHAT **Requests banners, sounds and app-icon badges.**
WHERE The argument of requestAuthorization.
WHEN When requestAuthorization runs; the options shape the alert the first time and the stored answer afterward.
WHY Ask only for the kinds of interruption your feature actually uses.

```
7  guard granted else { return }
```

WHO Your scheduleSunsetReminder() function, checking the Bool from requestAuthorization.
WHAT **Stops quietly if the user said no.**
WHERE Right after the permission request.
WHEN Whenever permission is denied.
WHY Scheduling without permission would do nothing, so leave early and let the interface explain.

```
9  let content = UNMutableNotificationContent()
```

WHO The UserNotifications framework, through the UNMutableNotificationContent initializer that builds an editable notification.
WHAT **Creates an editable description of what the notification shows.**
WHERE A local constant inside scheduleSunsetReminder(), after the permission guard.
WHEN Each time scheduleSunsetReminder() gets past the guard with permission granted.
WHY Content is separate from timing, so the same content could be reused with another trigger.

```
10 content.title = "Sunset soon"
```

WHO Your scheduleSunsetReminder() function, writing the content's title property.
WHAT **Sets the bold first line of the banner.**
WHERE On the content object.
WHEN Before center.add(request) runs; the title is frozen into the request when it is scheduled.
WHY A short title states the point at a glance.

```
11 content.body = "Head back to the trailhead."
```

WHO Your scheduleSunsetReminder() function, writing the content's body property.
WHAT **Sets the message below the title.**
WHERE On the content object.
WHEN Before the request is built, right after the title is set.
WHY The body tells the user what to do next.

```
12 content.sound = .default
```

WHO Your scheduleSunsetReminder() function, assigning UNNotificationSound.default to the content.
WHAT **Uses the standard notification sound.**
WHERE On the content object.
WHEN Played when the notification arrives, if the user allows sounds.
WHY The default sound respects the user's settings and Focus modes.

```
14 let trigger = UNTimeIntervalNotificationTrigger(
```

WHO The UserNotifications framework, through the UNTimeIntervalNotificationTrigger initializer that describes a delayed firing.
WHAT **Creates a trigger that fires after a delay.**
WHERE A local constant inside scheduleSunsetReminder(), separate from the content it will be paired with.
WHEN Created now; it fires later.
WHY Time-interval triggers suit countdowns, while calendar triggers suit fixed times of day.

```
15 timeInterval: 30 * 60, repeats: false)
```

WHO Your scheduleSunsetReminder() function, filling in the trigger's two arguments.
WHAT **Sets the delay to 1800 seconds and fires only once.**
WHERE The arguments of the trigger.
WHEN Counted from the moment the request is added.
WHY Writing 30 * 60 makes "30 minutes" readable in seconds.

```
16 let request = UNNotificationRequest(
```

WHO The UserNotifications framework, through the UNNotificationRequest initializer that bundles identifier, content and trigger.
WHAT **Bundles an identifier, the content and the trigger into one request.**
WHERE A local constant inside scheduleSunsetReminder(), the last value built before center.add.
WHEN Each time scheduleSunsetReminder() reaches this point, once content and trigger exist.
WHY The request is the unit the system stores and later delivers.

```
17 identifier: "sunset", content: content, trigger: trigger)
```

WHO Your scheduleSunsetReminder() function, passing the identifier, content and trigger.
WHAT **Names the request "sunset" and attaches content and trigger.**
WHERE The arguments of UNNotificationRequest.
WHEN When the UNNotificationRequest is built, just before it is handed to center.add.
WHY Adding another request with the same identifier replaces the pending one, which prevents duplicate reminders.

```
18 try await center.add(request)
```

WHO The system notification center.
WHAT **Schedules the request; iOS now delivers it even if your app is closed.**
WHERE The last line of the function.
WHEN Now; the banner appears when the trigger fires.
WHY try because scheduling can fail, for example with an invalid trigger.

Try it

1. Replace the trigger with a UNCalendarNotificationTrigger that fires every day at 18:00. Which DateComponents do you set?
2. In the simulation, tap Don't Allow, then trigger the feature again. Does a second alert appear?

CHECK YOURSELF

Check your understanding

1. The app now shows 3 glasses, but the widget still shows 2. The provider uses policy `.never`. What makes the widget catch up right after the user logs a glass?
   A. Nothing; iOS reloads every widget about once a minute on its own
   B. The app calls WidgetCenter.shared.reloadTimelines(ofKind: "Water")
   C. Add a Timer to the widget view that refreshes the count every second
   D. Add .systemLarge to supportedFamilies so the widget redraws
2. Why does the widget extension need an App Group to read the glass count the app saved?
   A. The extension has its own sandbox; an App Group gives both a shared container
   B. Widgets are not allowed to use Foundation types such as Date without one
   C. App Groups make the widget update in real time whenever the app saves
   D. A widget only appears in the gallery if it belongs to an App Group
3. You want users to say *"Log water in Water"* to Siri without opening the app. What do you build?
   A. A Live Activity that listens for the spoken phrase
   B. A repeating local notification with that phrase
   C. An App Intent plus an AppShortcutsProvider
   D. A widget with a large button labeled Log Water
4. A delivery app wants the minutes remaining visible on the Lock Screen and in the Dynamic Island while the order is on its way. What fits best?
   A. A widget with an .after policy that reloads every minute
   B. A local notification that fires again every minute
   C. An App Intent that Siri reads aloud
   D. A Live Activity, updated by the app or by push

Answers: Beyond the App: Widgets, Intents and Notifications

1. **B. The app calls WidgetCenter.shared.reloadTimelines(ofKind: "Water")** With .never, the system will not ask for a new timeline on its own, so the app must request a reload when its data changes. A Timer in the view is the instinctive fix, but widget views are rendered ahead of time as snapshots and never run live code.
2. **A. The extension has its own sandbox; an App Group gives both a shared container** An extension is a separate program with its own sandbox, so the app's files are invisible to it until both join an App Group. Real-time updates are easy to assume, but App Groups only share storage; reloads still follow the timeline policy.
3. **C. An App Intent plus an AppShortcutsProvider** App Intents describe actions the system can run, and AppShortcutsProvider registers spoken phrases for them. A widget button gets close, but widgets only respond to taps; Siri needs a registered intent and phrase.
4. **D. A Live Activity, updated by the app or by push** Live Activities exist for events in progress and appear on the Lock Screen and in the Dynamic Island. A minute-by-minute widget reload might seem equivalent, but the system budgets widget refreshes, so it would fall behind and cannot use the Dynamic Island.

Worked Examples

WORKED EXAMPLE

Ask for location: granted, denied, and the recovery

Problem.
Trailhead needs location to show nearby trails. Ask at the moment of use, survive a refusal with a manual fallback, and recover when the person later changes their mind in Settings.

The visual for this example in the HTML version is labelled Simulation: it models Apple frameworks or platform processes that cannot run in a web page.

Frame 1 of 6

Nothing has been asked yet, so the status is notDetermined and the app waits for the person to tap the trails feature before requesting anything at all.

```
enum AuthStatus {
    case notDetermined, denied, granted
}
var status = AuthStatus.notDetermined
switch status {
case .notDetermined: print("Status: not determined - not asked yet")
case .denied: print("Fallback: manual city entry mode")
case .granted: print("Showing trails near you")
}
```

Frame 2 of 6

The person taps the feature, the status check runs, and the system prompt appears. This plain-Swift model prints the prompt line; on a device, iOS draws the real alert itself.

```
enum AuthStatus {
    case notDetermined, denied, granted
}
var status = AuthStatus.notDetermined
switch status {
case .notDetermined: print("Status: not determined - not asked yet")
case .denied: print("Fallback: manual city entry mode")
case .granted: print("Showing trails near you")
}
+ print("System prompt: Trailhead wants to use your location")
+ print("Waiting for the person's answer")
```

Frame 3 of 6

The canned answer is Don't Allow, so ask returns denied and the switch takes its denied branch. Instead of a dead end, the app falls back to manual city entry.

```
  enum AuthStatus {
      case notDetermined, denied, granted
  }
+ func ask(userAllows: Bool) -> AuthStatus {
+     print("System prompt: Trailhead wants to use your location")
+     return userAllows ? AuthStatus.granted : AuthStatus.denied
+ }
+ var status = ask(userAllows: false)
+ switch status {
+ case .notDetermined: print("Status: not determined - not asked yet")
+ case .denied: print("Fallback: manual city entry mode")
+ case .granted: print("Showing trails near you")
+ }
```

Frame 4 of 6

Still denied, the app keeps working: the person types a city and the trails list answers from that instead of from GPS. A refusal degraded the feature, not the app.

```
  enum AuthStatus {
      case notDetermined, denied, granted
  }
  func ask(userAllows: Bool) -> AuthStatus {
      print("System prompt: Trailhead wants to use your location")
      return userAllows ? AuthStatus.granted : AuthStatus.denied
  }
  var status = ask(userAllows: false)
  switch status {
  case .notDetermined: print("Status: not determined - not asked yet")
  case .denied: print("Fallback: manual city entry mode")
  case .granted: print("Showing trails near you")
  }
+ let city = "Venice"
+ print("Manual mode: showing trails near \(city)")
```

Frame 5 of 6

The person later flips the switch in Settings, and a fresh check reads granted, because the code reads the status anew instead of remembering the old denial forever.

```
  enum AuthStatus {
      case notDetermined, denied, granted
  }
  func ask(userAllows: Bool) -> AuthStatus {
      print("System prompt: Trailhead wants to use your location")
      return userAllows ? AuthStatus.granted : AuthStatus.denied
  }
  var status = ask(userAllows: false)
  switch status {
  case .notDetermined: print("Status: not determined - not asked yet")
  case .denied: print("Fallback: manual city entry mode")
  case .granted: print("Showing trails near you")
  }
+ status = AuthStatus.granted
+ switch status {
+ case .notDetermined: print("Status: not determined - not asked yet")
+ case .denied: print("Fallback: manual city entry mode")
+ case .granted: print("Showing trails near you")
+ }
```

Frame 6 of 6

The whole journey in one function: ask, fall back on denial, then re-check and run the real feature once Settings says granted. Every branch ends somewhere useful.

```
  enum AuthStatus {
      case notDetermined, denied, granted
  }
  func ask(userAllows: Bool) -> AuthStatus {
      print("System prompt: Trailhead wants to use your location")
      return userAllows ? AuthStatus.granted : AuthStatus.denied
  }
+ func runJourney() {
+     var status = ask(userAllows: false)
+     switch status {
+     case .notDetermined: print("Status: not determined - not asked yet")
+     case .denied: print("Fallback: manual city entry mode")
+     case .granted: print("Showing trails near you")
+     }
+     status = AuthStatus.granted
+     switch status {
+     case .notDetermined: print("Status: not determined - not asked yet")
+     case .denied: print("Fallback: manual city entry mode")
+     case .granted: print("Showing trails near you")
+     }
+ }
+ runJourney()
```

Line by line: the finished program

```
1  enum AuthStatus {
```

WHO Your code, declaring the model's status type.
WHAT **Declares AuthStatus, an enum with the three answers this model handles.**
WHERE A type in this program, standing in for CLAuthorizationStatus.
WHEN Compiled once; every status value below is one of its cases.
WHY An enum makes the states a closed list the switch must cover completely.

```
2  case notDetermined, denied, granted
```

WHO Your AuthStatus enum, listing its cases on one line.
WHAT **Names the three states: not yet asked, refused, and allowed.**
WHERE The cases of AuthStatus.
WHEN Read at compile time; matched by the switches below.
WHY These mirror the real statuses an app checks before using location.

```
4  func ask(userAllows: Bool) -> AuthStatus {
```

WHO Your program, declaring the function that models the system request.
WHAT **Declares ask, which shows the prompt and returns the status the canned answer implies.**
WHERE A function in this program.
WHEN Called whenever the model needs to ask for permission.
WHY One function for the ask keeps every frame's prompt wording identical.

```
5  print("System prompt: Trailhead wants to use your location")
```

WHO Your program, printing the line that models the system prompt.
WHAT **Prints the prompt the system would show for Trailhead's location request.**
WHERE Standard output, standing in for the iOS alert on a device.
WHEN At the moment the app asks, after the feature is tapped.
WHY On a device iOS draws this alert itself; the model prints it so the moment is visible.

```
6  return userAllows ? AuthStatus.granted : AuthStatus.denied
```

WHO Your ask function, choosing its return value with a ternary.
WHAT **Returns granted when the canned answer allows, and denied when it refuses.**
WHERE The return value of ask, becoming the caller's new status.
WHEN At the end of each ask call.
WHY The person's tap is the only input; the model keeps it as a plain Bool so frames stay deterministic.

```
8  func runJourney() {
```

WHO Your program, declaring the function that runs the whole story.
WHAT **Declares runJourney, which asks, falls back, re-checks and runs the feature in order.**
WHERE A function in this program.
WHEN Called once, by the last line of the program.
WHY Gathering the journey in one function makes the frame a complete, runnable story.

```
9  var status = ask(userAllows: false)
```

WHO runJourney, calling ask with the canned answer false.
WHAT **Asks inside the journey and stores the refusal as the local status.**
WHERE A local variable of runJourney.
WHEN At the start of the journey, when the feature is first tapped.
WHY The journey starts with the same refusal the earlier frames survived.

```
10 switch status {
```

WHO runJourney, switching over its local status.
WHAT **Reads the local status and runs exactly one branch.**
WHERE Inside runJourney, on its local status variable.
WHEN Twice in this program: after the ask, and again after the re-check.
WHY The same decision table serves both checks, which is why it was worth extracting.

```
11 case .notDetermined: print("Status: not determined - not asked yet")
```

WHO The switch inside runJourney, matching the notDetermined case.
WHAT **Prints that nothing has been asked yet.**
WHERE The first branch of the journey's switch.
WHEN Only while the local status is notDetermined.
WHY Neither journey check lands here, but the switch must still cover the case.

```
12 case .denied: print("Fallback: manual city entry mode")
```

WHO The switch inside runJourney, matching the denied case.
WHAT **Prints the fallback: manual city entry mode.**
WHERE The second branch of the journey's switch.
WHEN On the first journey check, right after the refusal.
WHY The fallback appears in the log, so the denial is visibly survived, not hidden.

```
13 case .granted: print("Showing trails near you")
```

WHO The switch inside runJourney, matching the granted case.
WHAT **Prints the real feature running: trails near you.**
WHERE The third branch of the journey's switch.
WHEN On the second journey check, after the re-check reads granted.
WHY The journey ends on the granted branch, exactly where a recovered permission should end.

```
15 status = AuthStatus.granted
```

WHO runJourney, assigning the status the Settings change produced.
WHAT **Sets the local status to granted for the second check.**
WHERE The local status variable of runJourney.
WHEN Between the two switches, standing in for the Settings visit.
WHY Between the checks, the only thing that changed is the reported status.

```
16 switch status {
```

WHO runJourney, switching over its local status.
WHAT **Reads the local status and runs exactly one branch.**
WHERE Inside runJourney, on its local status variable.
WHEN Twice in this program: after the ask, and again after the re-check.
WHY The same decision table serves both checks, which is why it was worth extracting.

```
17 case .notDetermined: print("Status: not determined - not asked yet")
```

WHO The switch inside runJourney, matching the notDetermined case.
WHAT **Prints that nothing has been asked yet.**
WHERE The first branch of the journey's switch.
WHEN Only while the local status is notDetermined.
WHY Neither journey check lands here, but the switch must still cover the case.

```
18  case .denied: print("Fallback: manual city entry mode")
```

WHO The switch inside runJourney, matching the denied case.
WHAT **Prints the fallback: manual city entry mode.**
WHERE The second branch of the journey's switch.
WHEN On the first journey check, right after the refusal.
WHY The fallback appears in the log, so the denial is visibly survived, not hidden.

```
19  case .granted: print("Showing trails near you")
```

WHO The switch inside runJourney, matching the granted case.
WHAT **Prints the real feature running: trails near you.**
WHERE The third branch of the journey's switch.
WHEN On the second journey check, after the re-check reads granted.
WHY The journey ends on the granted branch, exactly where a recovered permission should end.

```
22  runJourney()
```

WHO Your program, calling the journey function.
WHAT **Runs the whole permission journey from the first tap to the granted feature.**
WHERE The last line of the program.
WHEN Once, after every declaration exists.
WHY A function does nothing until it is called; this call is the frame's starting gun.

Try it

The starter's denied branch is a dead end: it prints that location is off and offers nothing. Change that one branch to offer the manual fallback instead, printing the fallback line.

```
enum AuthStatus {
    case notDetermined, denied, granted
}
let status = AuthStatus.denied
switch status {
case .notDetermined: print("Ask first, then show trails")
case .denied: print("Location is off - nothing to show")
case .granted: print("Showing trails near you")
}
```

EXPECTED OUTPUT

```
Fallback: manual city entry mode
```

WORKED EXAMPLE

Build a widget timeline, entry by entry

Problem.

A trail widget shows conditions that change through the morning. Build its timeline entry by entry, thirty minutes apart from a fixed start, and finish with the reload policy that keeps it fed.

The visual for this example in the HTML version is labelled Simulation: it models Apple frameworks or platform processes that cannot run in a web page.

Frame 1 of 6

The builder starts at minute 540, which is 9:00 by integer arithmetic, and makes exactly one entry. The diagram shows that single entry as the current one.

```
  let start = 540
  let policy = "atEnd"
  func clockLabel(_ total: Int) -> String {
      let hour = total / 60
      let minute = total % 60
      if minute < 10 { return "\(hour):0\(minute)" }
      return "\(hour):\(minute)"
  }
  var entries: [String] = []
  entries.append("\(clockLabel(start + 0)) Trail conditions: Sunny")
  for entry in entries { print(entry) }
```

Frame 2 of 6

A second entry joins, thirty minutes later at 9:30, built by the same clockLabel function. The timeline now covers an hour of the morning, and the newest entry is current.

```
  let start = 540
  let policy = "atEnd"
  func clockLabel(_ total: Int) -> String {
      let hour = total / 60
      let minute = total % 60
      if minute < 10 { return "\(hour):0\(minute)" }
      return "\(hour):\(minute)"
  }
  var entries: [String] = []
  entries.append("\(clockLabel(start + 0)) Trail conditions: Sunny")
+ entries.append("\(clockLabel(start + 30)) Trail conditions: Sunny")
+ for entry in entries { print(entry) }
```

Frame 3 of 6

The third entry lands at 10:00 and its title changes to Breezy, because each entry is a frozen snapshot of what the widget should show at that moment.

```
  let start = 540
  let policy = "atEnd"
  func clockLabel(_ total: Int) -> String {
      let hour = total / 60
      let minute = total % 60
      if minute < 10 { return "\(hour):0\(minute)" }
      return "\(hour):\(minute)"
  }
  var entries: [String] = []
  entries.append("\(clockLabel(start + 0)) Trail conditions: Sunny")
  entries.append("\(clockLabel(start + 30)) Trail conditions: Sunny")
+ entries.append("\(clockLabel(start + 60)) Trail conditions: Breezy")
+ for entry in entries { print(entry) }
```

Frame 4 of 6

The fourth entry completes the planned timeline at 10:30. Four snapshots now cover two hours, so the widget can change on schedule without any code running.

```
  let start = 540
  let policy = "atEnd"
  func clockLabel(_ total: Int) -> String {
      let hour = total / 60
      let minute = total % 60
      if minute < 10 { return "\(hour):0\(minute)" }
      return "\(hour):\(minute)"
  }
  var entries: [String] = []
  entries.append("\(clockLabel(start + 0)) Trail conditions: Sunny")
  entries.append("\(clockLabel(start + 30)) Trail conditions: Sunny")
  entries.append("\(clockLabel(start + 60)) Trail conditions: Breezy")
+ entries.append("\(clockLabel(start + 90)) Trail conditions: Breezy")
+ for entry in entries { print(entry) }
```

Frame 5 of 6

The program names the last entry as the current one. In WidgetKit the system, not your code, picks whichever entry’s time has arrived; this model simply reports the newest.

```
  let start = 540
  let policy = "atEnd"
  func clockLabel(_ total: Int) -> String {
      let hour = total / 60
      let minute = total % 60
      if minute < 10 { return "\(hour):0\(minute)" }
      return "\(hour):\(minute)"
  }
  var entries: [String] = []
  entries.append("\(clockLabel(start + 0)) Trail conditions: Sunny")
  entries.append("\(clockLabel(start + 30)) Trail conditions: Sunny")
  entries.append("\(clockLabel(start + 60)) Trail conditions: Breezy")
  entries.append("\(clockLabel(start + 90)) Trail conditions: Breezy")
  for entry in entries { print(entry) }
+ print("Current entry: \(entries[3])")
```

Frame 6 of 6

The final line prints the reload policy, atEnd: when the last entry passes, the system asks the provider for a fresh timeline, keeping the widget fed automatically.

```
    let start = 540
    let policy = "atEnd"
    func clockLabel(_ total: Int) -> String {
        let hour = total / 60
        let minute = total % 60
        if minute < 10 { return "\(hour):0\(minute)" }
        return "\(hour):\(minute)"
    }
    var entries: [String] = []
    entries.append("\(clockLabel(start + 0)) Trail conditions: Sunny")
    entries.append("\(clockLabel(start + 30)) Trail conditions: Sunny")
    entries.append("\(clockLabel(start + 60)) Trail conditions: Breezy")
    entries.append("\(clockLabel(start + 90)) Trail conditions: Breezy")
    for entry in entries { print(entry) }
    print("Current entry: \(entries[3])")
+   print("Reload policy: \(policy)")
```

Line by line: the finished program

```
1  let start = 540
```

WHO — Your program, fixing the timeline's start as a number of minutes.
WHAT — **Sets start to 540 minutes, which is 9:00, counted from midnight.**
WHERE — A constant in top-level code.
WHEN — When the program starts, before any entry is built.
WHY — Minutes as integers keep the model deterministic; no clock is read.

```
2  let policy = "atEnd"
```

WHO — Your program, naming the reload policy for this timeline.
WHAT — **Sets the policy to atEnd, the value the final frame will print.**
WHERE — A constant in top-level code.
WHEN — When the program starts; read only in the final frame.
WHY — In WidgetKit this is Timeline's .atEnd policy: ask again when the last entry passes.

```
3  func clockLabel(_ total: Int) -> String {
```

WHO — Your program, declaring the label builder.
WHAT — **Declares clockLabel, which turns a minute count into a clock label.**
WHERE — A function in this program.
WHEN — Called once per entry, as each entry is appended.
WHY — One formatter means every entry's label is built the same way.

```
4  let hour = total / 60
```

WHO — clockLabel, dividing the minute count by sixty.
WHAT — **Computes the hour by integer division, discarding any leftover minutes.**
WHERE — A local constant inside clockLabel.
WHEN — On every clockLabel call.
WHY — Integer division is the whole trick: 570 minutes is hour 9, remainder 30.

```
5  let minute = total % 60
```

WHO — clockLabel, taking the remainder after the hour.
WHAT — **Computes the leftover minutes with the remainder operator.**
WHERE — A local constant inside clockLabel.
WHEN — On every clockLabel call, next to the hour.
WHY — Hour and minute together are the label; each is one arithmetic step.

```
6  if minute < 10 { return "\(hour):0\(minute)" }
```

WHO clockLabel, checking whether the minute needs a leading zero.
WHAT **Returns the label with a zero padded before a one-digit minute.**
WHERE Inside clockLabel, as its first return.
WHEN When the minute is under ten, such as exactly on the hour.
WHY Without this, 9:00 would print as 9:0, and the diagram's labels would disagree.

```
7  return "\(hour):\(minute)"
```

WHO clockLabel, building its label by interpolation.
WHAT **Returns the hour and the two-digit minute joined by a colon.**
WHERE Inside clockLabel, as its last line.
WHEN When the minute is ten or more.
WHY The common path needs no padding, so it is the plain return.

```
9  var entries: [String] = []
```

WHO Your program, declaring the timeline under construction.
WHAT **Creates the empty entries array the appends below will fill.**
WHERE A variable in top-level code.
WHEN When the program starts, before the first append.
WHY An array is the timeline: ordered snapshots, each with its moment.

```
10 entries.append("\(clockLabel(start + 0)) Trail conditions: Sunny")
```

WHO Your program, appending the first entry to the timeline.
WHAT **Builds the 9:00 entry, labelled by clockLabel from start plus zero minutes.**
WHERE The first element of the entries array.
WHEN First, before the later entries are appended.
WHY Offset zero means this entry is the one showing when the timeline is delivered.

```
11 entries.append("\(clockLabel(start + 30)) Trail conditions: Sunny")
```

WHO Your program, appending the second entry to the timeline.
WHAT **Builds the 9:30 entry, thirty minutes after the start.**
WHERE The second element of the entries array.
WHEN After the 9:00 entry, in timeline order.
WHY Entries are planned ahead, so the widget can change without code running.

```
12 entries.append("\(clockLabel(start + 60)) Trail conditions: Breezy")
```

WHO Your program, appending the third entry to the timeline.
WHAT **Builds the 10:00 entry, whose title changes to Breezy.**
WHERE The third element of the entries array.
WHEN After the 9:30 entry, in timeline order.
WHY A new title in this snapshot is what makes the widget appear to update.

```
13 entries.append("\(clockLabel(start + 90)) Trail conditions: Breezy")
```

WHO Your program, appending the fourth entry to the timeline.
WHAT **Builds the 10:30 entry, the last one this timeline plans.**
WHERE The fourth element of the entries array.
WHEN After the 10:00 entry, completing the timeline.
WHY Four entries cover two hours; the policy decides what happens after that.

```
14 for entry in entries { print(entry) }
```

WHO Your program, looping over the entries array with print inside.
WHAT **Prints every entry built so far, one per line, in timeline order.**
WHERE Standard output, one line per entry.
WHEN At the end of the program, after all appends.
WHY The printout is the timeline in text form; compare it with the diagram, entry for entry.

```
15 print("Current entry: \(entries[3])")
```

WHO Your program, calling print with the last entry.
WHAT **Prints which entry is current: the fourth, at 10:30.**
WHERE Standard output, one line.
WHEN After the entry loop, at the end of the program.
WHY In this model the newest entry is current; on a device the system picks by time.

```
16 print("Reload policy: \(policy)")
```

WHO Your program, calling print with the policy constant.
WHAT **Prints the reload policy line: atEnd.**
WHERE Standard output, the last line this program prints.
WHEN At the very end of the program.
WHY This line names what WidgetKit should do when the timeline runs out.

Try it

A ferry widget starts at minute 600 and should list two departures, at 0 and 45 minutes. The starter builds only the first. Add the second entry and print the whole timeline.

```
let start = 600
func clockLabel(_ total: Int) -> String {
    let hour = total / 60
    let minute = total % 60
    if minute < 10 { return "\(hour):0\(minute)" }
    return "\(hour):\(minute)"
}
var entries: [String] = []
entries.append("\(clockLabel(start + 0)) Ferry departs")
print(entries[0])
```

EXPECTED OUTPUT

```
10:00 Ferry departs
10:45 Ferry departs
```

WORKED EXAMPLE

One delivery, four endings: delivered, cancelled, failed

Problem.

A photo-print service takes an order from request to delivery. Follow one order all the way, then two more that end early: one cancelled by the customer, one failed by the network, with a retry.

The visual for this example in the HTML version is labelled Simulation: it models Apple frameworks or platform processes that cannot run in a web page.

Frame 1 of 7

The operation starts Idle: the report function is defined, the phase variable holds Idle, and the first log line shows an order that has not been requested yet.

```
    func report(_ label: String, _ phase: String, _ detail: String) {
        print("\(label) \(phase): \(detail)")
    }
    var phase = "Idle"
    print("Order 1 \(phase): waiting for a request")
```

Frame 2 of 7

The customer places the order and the phase moves to Requested. The log line is the program's proof: same report function, new phase, new detail.

```
    func report(_ label: String, _ phase: String, _ detail: String) {
        print("\(label) \(phase): \(detail)")
    }
    var phase = "Idle"
+   phase = "Requested"
+   report("Order 1", phase, "photo print ordered")
```

Frame 3 of 7

Work begins and the phase moves to In progress. Both log lines print, so the history shows the order was requested before printing started, in that order.

```
    func report(_ label: String, _ phase: String, _ detail: String) {
        print("\(label) \(phase): \(detail)")
    }
    var phase = "Idle"
    phase = "Requested"
    report("Order 1", phase, "photo print ordered")
+   phase = "In progress"
+   report("Order 1", phase, "printing your photo")
```

Frame 4 of 7

The print finishes and the phase reaches Delivered, the successful ending. One order has now walked the full path the diagram shows across the top.

```
    func report(_ label: String, _ phase: String, _ detail: String) {
        print("\(label) \(phase): \(detail)")
    }
    var phase = "Idle"
    phase = "Requested"
    report("Order 1", phase, "photo print ordered")
    phase = "In progress"
    report("Order 1", phase, "printing your photo")
+   phase = "Delivered"
+   report("Order 1", phase, "ready for pickup")
```

Frame 5 of 7

A second order starts the same way, but the customer cancels before printing. The log says plainly that nothing was charged, so a cancelled order is an ending, not an error.

```
    func report(_ label: String, _ phase: String, _ detail: String) {
        print("\(label) \(phase): \(detail)")
    }
    var phase = "Idle"
    phase = "Requested"
+ report("Order 2", phase, "photo print ordered")
+ phase = "Cancelled"
+ report("Order 2", phase, "cancelled by you - nothing was charged")
```

Frame 6 of 7

A third order hits a canned network failure and lands in Failed. The recovery is printed, not wished for: a retry is scheduled, so the story continues in the next frame.

```
    func report(_ label: String, _ phase: String, _ detail: String) {
        print("\(label) \(phase): \(detail)")
    }
    var phase = "Idle"
    phase = "Requested"
+ report("Order 3", phase, "photo print ordered")
+ phase = "Failed"
+ report("Order 3", phase, "network unreachable")
+ print("Retry scheduled: Order 3 will try again in 5 minutes")
```

Frame 7 of 7

The scheduled retry runs the same path as the first order and reaches Delivered. Failure was a detour with a plan, which is what a recovery is supposed to be.

```
    func report(_ label: String, _ phase: String, _ detail: String) {
        print("\(label) \(phase): \(detail)")
    }
    var phase = "Idle"
    phase = "Requested"
+ report("Order 3 retry", phase, "photo print ordered")
+ phase = "In progress"
+ report("Order 3 retry", phase, "printing your photo")
+ phase = "Delivered"
+ report("Order 3 retry", phase, "ready for pickup")
```

Line by line: the finished program

```
1  func report(_ label: String, _ phase: String, _ detail: String) {
```

WHO	Your program, declaring the log function.
WHAT	**Declares report, which prints one log line from a label, a phase and a detail.**
WHERE	A function in this program.
WHEN	Called at every phase change in every frame.
WHY	One log format means the eye can compare orders line by line.

```
2  print("\(label) \(phase): \(detail)")
```

WHO report, building its line by interpolation and printing it.
WHAT **Prints the label, the phase in words, and the detail, joined as one log line.**
WHERE Standard output, one line per report call.
WHEN Each time report is called.
WHY The phase word in the line must match the diagram's phase for that frame.

```
4  var phase = "Idle"
```

WHO Your program, declaring the phase variable.
WHAT **Creates phase at Idle, the state every order starts in.**
WHERE A variable in top-level code.
WHEN When the program starts, before any transition.
WHY The variable is the state machine: transitions are assignments to it.

```
5  phase = "Requested"
```

WHO Your program, assigning the next phase.
WHAT **Moves the phase to Requested, the moment the customer places the order.**
WHERE The phase variable, in top-level code.
WHEN At the start of the order's story.
WHY Requested is always the first transition; nothing can be printed or shipped before it.

```
6  report("Order 3 retry", phase, "photo print ordered")
```

WHO Your program, reporting through the shared log function.
WHAT **Logs the retry's request for Order 3: a photo print was ordered.**
WHERE Standard output, one line.
WHEN Right after the phase becomes Requested on the retry.
WHY The retry label keeps the second attempt's log separate from the first attempt's.

```
7  phase = "In progress"
```

WHO Your program, assigning the next phase.
WHAT **Moves the phase to In progress, when the service starts the work.**
WHERE The phase variable, in top-level code.
WHEN After the request, before delivery can be reported.
WHY In progress is the only phase where work is happening; the others wait or are done.

```
8  report("Order 3 retry", phase, "printing your photo")
```

WHO Your program, reporting through the shared log function.
WHAT **Logs the retry in progress: the photo is being printed.**
WHERE Standard output, one line.
WHEN Right after the phase becomes In progress on the retry.
WHY Same work, second attempt: the phase sequence repeats exactly.

```
9  phase = "Delivered"
```

WHO Your program, assigning the final phase.
WHAT **Moves the phase to Delivered, the successful ending.**
WHERE The phase variable, in top-level code.
WHEN After the work completes, as the order's last transition.
WHY Delivered is terminal: no transition in this model leaves it.

```
10  report("Order 3 retry", phase, "ready for pickup")
```

WHO Your program, reporting through the shared log function.
WHAT **Logs the retry delivered: ready for pickup.**
WHERE Standard output, one line.
WHEN Right after the phase becomes Delivered on the retry.
WHY Delivery on the retry is the proof that scheduling it was worthwhile.

Try it

A repair-visit order in the starter jumps straight from Requested to Delivered, skipping the work. Add the In progress step between them so the log shows all three phases.

```
func report(_ phase: String, _ detail: String) {
    print("\(phase): \(detail)")
}
var phase = "Idle"
phase = "Requested"
report(phase, "book a repair visit")
phase = "Delivered"
report(phase, "engineer visited")
```

EXPECTED OUTPUT

```
Requested: book a repair visit
In progress: engineer is on the way
Delivered: engineer visited
```

MODULE 08

Networking and Backend Communication

Explain how a request travels from an iPhone to a server and back, speak HTTP and REST, decode JSON with Codable, call APIs with URLSession and async/await, handle failures and secrets safely, choose between polling, WebSockets and push, and structure a testable API layer.

- Use network protocols for backend communication
- Integrate APIs such as location services and cloud storage

How an App Talks to a Server

CONCEPT

Your app is a client

Almost every app you use talks to a computer it does not control. A weather app asks a forecast service for today's temperature; a messaging app hands your message to a server that delivers it. The app on the phone is the **client**: it starts the conversation. The **server** is a program on another machine, usually in a data center, that waits for requests and answers them.

Everything that lives on servers (databases, accounts, business rules) is called the **backend**. Your Swift code never reaches into the backend's database. It sends a message across the network and waits for a reply.

Request	A message from the client asking the server to do something: send data, save data, delete data.
Response	The server's answer: a status that says how it went, plus any data.
Network	Everything in between: Wi-Fi or cellular radio, routers, cables and other companies' networks.
URLSession	The Foundation class that iOS apps use to send requests and receive responses.

> **Design for the gap**
>
> Reading memory takes nanoseconds. A network reply takes tens to hundreds of milliseconds, and sometimes never arrives. Every screen that loads data needs a plan for "still waiting" and "it failed", not only for "it worked".

CONCEPT

The layers a request travels through

When your code asks for `https://api.open-meteo.com/v1/forecast`, several **protocols** (agreed rules for exchanging messages) cooperate. Each solves one problem and relies on the layer beneath it. You write code only for the top layer; URLSession and iOS handle the rest.

DNS	Domain Name System. Turns a name such as api.open-meteo.com into an IP address, the way a contacts app turns a name into a phone number.
IP	Internet Protocol. Gives every machine an address and moves small packets toward it, hop by hop. Packets can be lost or arrive out of order.
TCP	Transmission Control Protocol. Builds a reliable, ordered stream of bytes on top of IP by numbering packets and resending lost ones.
TLS	Transport Layer Security. Encrypts the stream and proves, with a certificate, that the server is who it claims to be. It is the S in HTTPS.
HTTP	Hypertext Transfer Protocol. The request and response format your code works with: method, URL, headers, body and status code.

CONCEPT

Latency, bandwidth and round trips

Bandwidth is how much data a connection carries per second. **Latency** is how long one message takes to reach the other side. For the small JSON replies most apps fetch, latency matters far more than bandwidth.

A **round trip** is one message out and its reply back; its duration is the **RTT** (round-trip time). A brand-new HTTPS connection spends several round trips on setup (look up the name, open TCP, negotiate TLS) before your request even leaves. On good Wi-Fi an RTT might be 20 ms; on a busy cellular network, 100 ms or more. Multiply by the number of round trips and the delay becomes visible.

Reuse connections	URLSession keeps connections open, so the next request to the same server skips the setup.
Make fewer requests	One request that returns 20 items beats 20 requests that return one item each.

> **Never block the main thread**
>
> The main thread draws your interface. If it sits waiting for the network, the app freezes. Swift's `await` lets your code wait for a reply while the main thread keeps drawing.

INTERACTIVE

Follow one request

Step through a single HTTPS request, one message at a time. Change the round-trip time to model fast Wi-Fi or a weak cellular signal, compare HTTP/2 over TCP and TLS with HTTP/3 over QUIC, and switch on connection reuse. Watch the round-trip counter: on a fresh connection, setup costs more than the request itself.

This step is an interactive simulation in the HTML version.

CONCEPT

HTTP/1.1, HTTP/2 and HTTP/3

The request format has stayed familiar since HTTP/1.1, but the way it travels has changed twice.

HTTP/1.1 (1997)	Text messages over TCP. One request at a time per connection, so a slow reply blocks the ones queued behind it (head-of-line blocking).
HTTP/2 (2015)	Binary frames and multiplexing: many requests share one TCP connection as interleaved streams, with compressed headers. One lost TCP packet still stalls every stream until it is resent.
HTTP/3 (2022)	Runs on QUIC, a transport built on UDP. QUIC includes TLS 1.3, so connecting and encrypting happen together, each stream recovers from loss independently, and a connection can survive a move from Wi-Fi to cellular.

URLSession chooses for you

You never pick a version in code. URLSession agrees on HTTP/2 during the TLS handshake, and on iOS 15 and later it uses HTTP/3 once a server advertises support. Your `URLRequest` looks the same either way.

CODE · READ-ALONG

Measuring the journey from Swift

URLSession records when each phase of a request happened. A **delegate** (an object that receives callbacks) can read those metrics. This logger prints the negotiated protocol (h2 means HTTP/2, h3 means HTTP/3), whether an open connection was reused, how long the TLS handshake took and the total time.

```swift
import Foundation

final class JourneyLogger: NSObject, URLSessionTaskDelegate {
    func urlSession(_ session: URLSession, task: URLSessionTask,
        didFinishCollecting metrics: URLSessionTaskMetrics) {
        guard let t = metrics.transactionMetrics.last else { return }
        print("Protocol:", t.networkProtocolName ?? "unknown")
        print("Reused connection:", t.isReusedConnection)
        if let start = t.secureConnectionStartDate,
           let end = t.secureConnectionEndDate {
            print("TLS handshake:", end.timeIntervalSince(start))
        }
        print("Total seconds:", metrics.taskInterval.duration)
    }
}

func measureJourney() async throws {
    let url = URL(string: "https://httpbin.org/get")!
    let logger = JourneyLogger()
    _ = try await URLSession.shared.data(from: url, delegate: logger)
}
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, while it builds the app.
- **WHAT** **Makes URLSession, URL and the task-metrics types available in this file.**
- **WHERE** The first line of the file; an import applies to this file only.
- **WHEN** At compile time, before any of your code runs.
- **WHY** URLSession belongs to the Foundation framework, not to the Swift language, so every file that does networking imports it.

```
3  final class JourneyLogger: NSObject, URLSessionTaskDelegate {
```

WHO Your code declares it; URLSession will call into it.
WHAT **Declares a class that can receive callbacks about network tasks.**
WHERE A reference type: instances live on the heap, and URLSession keeps a reference while the task runs.
WHEN Compiled once; an instance is created each time measureJourney() runs.
WHY Delegate protocols come from Objective-C, so the class inherits from NSObject; final says nothing subclasses it.

```
4  func urlSession(_ session: URLSession, task: URLSessionTask,
```

WHO URLSession, which looks for a method with exactly this signature.
WHAT **Begins the callback that delivers metrics, naming the session and the task (one request) they describe.**
WHERE A method inside JourneyLogger; its signature continues on the next line.
WHEN Called once per task, after the response has completely arrived.
WHY The name and argument labels must match the protocol requirement exactly, or URLSession never calls it.

```
5  didFinishCollecting metrics: URLSessionTaskMetrics) {
```

WHO URLSession supplies the metrics object.
WHAT **Finishes the signature: metrics holds every timestamp URLSession recorded for the task.**
WHERE The last parameter of the delegate method; the body starts at the brace.
WHEN Filled in at call time, when collection is complete.
WHY The external label didFinishCollecting reads like a sentence at the call site, while metrics is the short name used inside.

```
6  guard let t = metrics.transactionMetrics.last else { return }
```

WHO Your delegate, reading data URLSession gathered.
WHAT **Takes the last transaction (one request and response exchange), or leaves the method if there is none.**
WHERE The first statement inside the callback.
WHEN Every time the callback runs.
WHY A task can contain several transactions, for example a redirect followed by the real request; the last one produced the response your code saw.

```
7  print("Protocol:", t.networkProtocolName ?? "unknown")
```

WHO Your code, using a value negotiated by URLSession and the server.
WHAT **Prints the protocol that was used: "http/1.1", "h2" or "h3".**
WHERE Inside the callback; the output goes to the Xcode console.
WHEN After the task finished.
WHY The property is optional because some transactions, such as a response loaded from the local cache, used no network protocol; ?? substitutes a readable default.

```
8  print("Reused connection:", t.isReusedConnection)
```

WHO URLSession, which tracks its pool of open connections.
WHAT **Prints true when the request rode on a connection that was already open.**
WHERE Inside the metrics callback of JourneyLogger, right after the protocol line is printed.
WHEN After the task finished.
WHY A reused connection skips DNS, TCP and TLS entirely, which is why a second request to the same server is usually much faster.

```
9  if let start = t.secureConnectionStartDate,
```

WHO Your code, unwrapping an optional timestamp.
WHAT **Binds the moment the TLS handshake began, if there was one.**
WHERE Inside the callback; the condition continues on the next line.
WHEN After the task finished.
WHY A reused connection has no handshake, so these dates are nil; if let skips the print instead of crashing.

```
10 let end = t.secureConnectionEndDate {
```

WHO Your code, unwrapping a second optional.
WHAT **Binds the moment the TLS handshake completed.**
WHERE The second clause of the same if statement; both must succeed.
WHEN Evaluated right after the first clause.
WHY Combining clauses with a comma keeps the happy path flat instead of nesting two ifs.

```
11 print("TLS handshake:", end.timeIntervalSince(start))
```

WHO Your code, using Date's timeIntervalSince to subtract the two timestamps.
WHAT **Prints the handshake duration in seconds, for example 0.031.**
WHERE Inside the if body, so only when both dates exist.
WHEN After the task finished, on a fresh connection.
WHY Measuring real phases on real networks shows where time goes, instead of guessing.

```
13 print("Total seconds:", metrics.taskInterval.duration)
```

WHO URLSession, which recorded when the task started and ended.
WHAT **Prints the whole task duration, from start to the last byte of the response.**
WHERE Inside the callback, after the per-phase details.
WHEN After the task finished.
WHY Comparing the total with the TLS time tells you whether setup or the server dominates.

```
17 func measureJourney() async throws {
```

WHO Your code; a caller awaits it.
WHAT **Declares a function that performs one measured request.**
WHERE A top-level function in this file.
WHEN Runs whenever a caller awaits it, for example from a button action.
WHY async lets it wait for the network without blocking a thread; throws passes failures such as no connection to the caller.

```
18 let url = URL(string: "https://httpbin.org/get")!
```

WHO Foundation parses the string.
WHAT **Creates a URL value for a public test endpoint that echoes requests.**
WHERE A local constant inside measureJourney.
WHEN Each time the function runs.
WHY The ! is acceptable only because this literal is known to be valid; never force-unwrap URLs built from user input.

```
19 let logger = JourneyLogger()
```

WHO Your code creates the delegate.
WHAT **Makes one logger instance.**
WHERE A local constant; the object itself lives on the heap.
WHEN Just before the request starts.
WHY URLSession holds the delegate while the task runs, so it stays alive until the metrics arrive.

```
20 _ = try await URLSession.shared.data(from: url, delegate: logger)
```

WHO URLSession performs DNS, TCP, TLS and HTTP for you.
WHAT **Sends a GET request, waits for the full response and tells the logger about this one task.**
WHERE Called from measureJourney; the network work happens on URLSession's own threads.
WHEN The function suspends at await until the response arrives or an error is thrown.
WHY The _ = discards the data because only the metrics matter here; the delegate parameter attaches the logger to this single request.

Try it

1. In the panel, turn on connection reuse. Which three timestamps would be nil for that request, and why?
2. Predict what networkProtocolName prints for a server that supports HTTP/3 on your very first request to it.

CHECK YOURSELF

Check your understanding

1. Your app makes its first request to a new server over HTTPS with HTTP/2. What has to happen before the request itself is sent?
   A. Only a DNS lookup
   B. A DNS lookup, a TCP handshake and a TLS handshake
   C. A TLS handshake only, because HTTPS does not use TCP
   D. Nothing; HTTP/2 sends the request immediately
2. A list screen makes 30 separate requests, one per row, each returning about 200 bytes. On cellular the screen takes three seconds to fill. Which change helps most?
   A. Compress each JSON reply
   B. Use a faster JSON decoder
   C. Switch the phone to a higher-bandwidth plan
   D. Fetch all 30 rows in one request
3. Which statement about HTTP/3 is true?
   A. It runs over QUIC, a UDP transport with TLS 1.3 built in
   B. It keeps TCP but skips the TLS handshake to save a round trip
   C. You enable it by setting a property on each URLRequest you send
   D. It needs a WebSocket connection to be opened before requests
4. A teammate calls a function that blocks until the network replies, directly inside a button action on the main thread. What does the user experience?
   A. Nothing unusual, because the reply usually arrives in milliseconds
   B. iOS detects the blocking call and moves it to a background thread
   C. The app freezes until the reply arrives or the request times out
   D. The button dims and ignores taps until the reply arrives

Answers: How an App Talks to a Server

1. **B. A DNS lookup, a TCP handshake and a TLS handshake** A fresh HTTP/2 connection needs the server's address (DNS), a reliable byte stream (TCP) and encryption plus server authentication (TLS), each costing at least one round trip. HTTPS absolutely runs on TCP for HTTP/1.1 and HTTP/2; only HTTP/3 replaces TCP with QUIC.
2. **D. Fetch all 30 rows in one request** Tiny replies are dominated by latency: every request pays at least one round trip. One request pays it once. Compression and bandwidth help with large bodies, not with 200-byte replies, and decoding 200 bytes is already instant.
3. **A. It runs over QUIC, a UDP transport with TLS 1.3 built in** QUIC combines the transport and TLS 1.3 handshakes, which is where the saved round trip comes from, and encryption is mandatory. URLSession uses HTTP/3 automatically when the server supports it; there is no per-request switch.
4. **C. The app freezes until the reply arrives or the request times out** The main thread draws the interface and handles touches. While it waits, nothing redraws and taps go unanswered, possibly for the full timeout. iOS does not rescue blocking code; using `await` with URLSession's async methods is what keeps the main thread free.

Speaking HTTP

CONCEPT

Anatomy of a request and a response

An HTTP message is structured text (HTTP/2 and HTTP/3 carry the same parts in binary frames). A request says what you want; a response says what happened. Here is a complete request for one to-do item:

```
GET /todos/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
Accept: application/json
```

Method	The verb: what to do with the resource, such as GET, POST, PUT, PATCH or DELETE.
URL	Which resource: scheme, host, path and optional query items.
Headers	Name and value pairs that describe the message: its format, the formats you accept, who is calling.
Body	Optional data, usually JSON. Sent with POST, PUT and PATCH, and returned by most responses.
Status code	A three-digit number at the start of every response, such as 200 OK or 404 Not Found.

CONCEPT

Methods and idempotency

The method tells the server your intent. Two properties matter when you write client code. A **safe** method only reads; it changes nothing on the server. An **idempotent** method has the same effect whether you send it once or five times. That matters because networks fail mid-request: if the reply never came, can you simply send it again?

GET	Read a resource. Safe and idempotent, so retrying is always fine. No body.
POST	Create something or trigger an action. Neither safe nor idempotent: sending it twice may place two orders.
PUT	Replace a resource with the body you send. Idempotent: the end state is the same however often it runs.
PATCH	Change some fields of a resource. Not guaranteed to be idempotent ("add 1 to the count" is a patch).
DELETE	Remove a resource. Idempotent: once it is gone, deleting again leaves it gone, even if the reply is 404.

A retry rule of thumb

Retry GET, PUT and DELETE automatically after a timeout. Never blindly retry a POST; the Errors, Retries and Security lesson builds a POST that carries an Idempotency-Key header so a retry cannot create a second order.

CODE · READ-ALONG

Build URLs with URLComponents

A URL has parts: the **scheme** (https), the **host** (the server's name), the **path** (which resource) and **query items** (name=value options after the ?). Spaces, accents and symbols such as & must be *percent-encoded* (written as %20, %26). `URLComponents` assembles the parts and encodes them for you, so user input cannot break the URL.

```swift
import Foundation

func searchURL(for city: String) -> URL? {
    var parts = URLComponents()
    parts.scheme = "https"
    parts.host = "geocoding-api.open-meteo.com"
    parts.path = "/v1/search"
    parts.queryItems = [
        URLQueryItem(name: "name", value: city),
        URLQueryItem(name: "count", value: "1")
    ]
    return parts.url
}
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, while it builds the file that holds searchURL(for:).
- WHAT — **Brings URLComponents, URLQueryItem and URL into this file.**
- WHERE — Top of the file.
- WHEN — At compile time, before searchURL(for:) can be called.
- WHY — These URL types are Foundation types, available on every Apple platform.

3 `func searchURL(for city: String) -> URL? {`

- WHO — Your code; a view or model calls it with whatever the user typed.
- WHAT — **Declares a function that builds a city-search URL for the Open-Meteo geocoding API.**
- WHERE — A top-level function; it could equally live inside an API client type.
- WHEN — Each time the app needs a search URL, for example on every search.
- WHY — It returns URL? because building a URL can fail, and the caller should decide what to do then.

4 `var parts = URLComponents()`

- WHO — Foundation's URLComponents struct, created empty by your code.
- WHAT — **Creates an empty set of URL parts to fill in.**
- WHERE — A local variable (a struct value) inside the function.
- WHEN — At the start of every call.
- WHY — It is var because each following line mutates one part.

```
5  parts.scheme = "https"
```

WHO Your code sets the protocol.
WHAT **Chooses HTTPS, the encrypted form of HTTP.**
WHERE Stored in the parts value.
WHEN While the URL is being built.
WHY App Transport Security blocks plain http by default, so https is the normal choice.

```
6  parts.host = "geocoding-api.open-meteo.com"
```

WHO Your code names the server.
WHAT **Sets the host name that DNS will turn into an IP address.**
WHERE Stored in the parts value.
WHEN While the URL is being built.
WHY Keeping the host separate from the path makes it easy to point at a test server later.

```
7  parts.path = "/v1/search"
```

WHO Your code names the resource.
WHAT **Sets the path: version 1 of the search endpoint.**
WHERE Stored in the parts value; a path must start with /.
WHEN While the URL is being built.
WHY APIs often put a version in the path so old apps keep working when a new version ships.

```
8  parts.queryItems = [
```

WHO Your code supplies options for the request.
WHAT **Starts the array of name and value pairs that become the text after the ?.**
WHERE A property of the parts value; the array spans the next lines.
WHEN While the URL is being built.
WHY Query items are passed as separate values, so URLComponents can encode each one correctly.

```
9  URLQueryItem(name: "name", value: city),
```

WHO URLComponents encodes the user's text.
WHAT **Adds name=<the city>, percent-encoding spaces, accents, & and = inside the value.**
WHERE The first element of the queryItems array.
WHEN Encoding happens when parts.url is read.
WHY Pasting raw user text into a URL string lets an & split it into two items; a query item keeps it one value.

```
10 URLQueryItem(name: "count", value: "1")
```

WHO Your code adds a second option.
WHAT **Asks the API for only the best match.**
WHERE The second element; items are joined with & in this order.
WHEN Encoded when parts.url is read.
WHY Requesting only what you need keeps replies small and fast.

```
12 return parts.url
```

WHO URLComponents assembles the final URL.
WHAT **Returns the encoded URL, or nil if the parts cannot form a valid URL.**
WHERE The last line of the function.
WHEN When the caller reads the result.
WHY For "São Paulo" it produces …search?name=S%C3%A3o%20Paulo&count=1, exactly what the server expects.

Try it

1. In the panel, type a city with a space or an accent. Which characters change in the final URL?
2. Type rock+roll. URLComponents leaves the + alone. Why might a server read it as a space?

CONCEPT

Headers that matter

Headers travel with every request and response. Their names are case-insensitive. You set request headers on a `URLRequest`, and you read response headers from an `HTTPURLResponse`.

Content-Type	The format of the body you are sending, such as application/json. Set it whenever you send a body so the server knows how to parse it.
Accept	The formats the client can read. application/json asks for JSON back.
Authorization	Proves who is calling, usually the word Bearer followed by a token the user received at sign-in.
Cache-Control and ETag	Response headers that say how long a reply may be reused and label its version. They return in the last lesson of this module.
User-Agent	Names the client software. URLSession fills it in, along with Host, Content-Length and Accept-Encoding; leave those to URLSession.

CONCEPT

Status codes: how did it go?

Every response starts with a three-digit status code. The first digit names the family, so an app can react sensibly even to a code it has never seen.

2xx Success	200 OK, 201 Created (after a POST), 204 No Content (success with an empty body).
3xx Redirection	301 and 302 point to a new URL, and URLSession follows them for you. 304 Not Modified means your cached copy is still good.
4xx Client error	The request was wrong: 400 Bad Request, 401 Unauthorized (sign in again), 403 Forbidden, 404 Not Found, 409 Conflict, 422 invalid data, 429 Too Many Requests.
5xx Server error	The server failed: 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable. Often temporary, so a later retry can succeed.

> **A 404 is not an error to URLSession**
>
> URLSession throws only when no response arrives at all: offline, timed out, untrusted certificate. A 404 or a 500 is a successfully delivered piece of bad news, so your code must check `statusCode` itself.

CODE · READ-ALONG

Building a POST request

To create a resource you send a POST with a JSON body. A `URLRequest` bundles everything the request needs: URL, method, headers, body and a timeout. This function only *builds* the request; sending it comes in the URLSession lesson. `JSONEncoder` turns a Swift value into JSON bytes.

```swift
import Foundation

struct NewTodo: Encodable {
    let title: String
    let completed: Bool
    let userId: Int
}

func makeCreateRequest(title: String) throws -> URLRequest {
    let url = URL(string: "https://jsonplaceholder.typicode.com")!
        .appending(path: "todos")
    var request = URLRequest(url: url)
    request.httpMethod = "POST"
    request.setValue("application/json",
                     forHTTPHeaderField: "Content-Type")
    request.setValue("application/json",
                     forHTTPHeaderField: "Accept")
    let todo = NewTodo(title: title, completed: false, userId: 1)
    request.httpBody = try JSONEncoder().encode(todo)
    request.timeoutInterval = 15
    return request
}
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, while it builds the file that declares NewTodo and makeCreateRequest.
- **WHAT** **Imports URLRequest, JSONEncoder and the other Foundation types used below.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before any request can be built.
- **WHY** Networking and JSON encoding both live in Foundation.

3 `struct NewTodo: Encodable {`

- **WHO** Your code defines the shape of the body.
- **WHAT** **Declares a value type that can be written out as JSON.**
- **WHERE** A top-level struct; instances live on the stack or inline in other values.
- **WHEN** Compiled once; a value is created for each request.
- **WHY** Encodable is all a request body needs; the compiler writes the encoding code for you.

4 `let title: String`

- **WHO** The NewTodo struct; JSONEncoder reads it when the body is encoded.
- **WHAT** **A stored property that becomes the JSON key "title".**
- **WHERE** The first stored property inside NewTodo.
- **WHEN** Set when a NewTodo is created.
- **WHY** Property names match the API's keys, so no extra mapping code is needed.

5 `let completed: Bool`

- **WHO** The NewTodo struct; JSONEncoder writes it as a JSON boolean.
- **WHAT** **Becomes "completed": true or false in the JSON.**
- **WHERE** The second stored property inside NewTodo, after title.
- **WHEN** Set when a NewTodo is created.
- **WHY** Swift Bool maps directly to JSON true and false.

```
6   let userId: Int
```

WHO The NewTodo struct; JSONEncoder writes it as a JSON number.
WHAT **Becomes "userId": a JSON number.**
WHERE The last stored property inside NewTodo.
WHEN Set when a NewTodo is created.
WHY The API spells it userId, which is also fine Swift style, so the names line up.

```
9   func makeCreateRequest(title: String) throws -> URLRequest {
```

WHO Your code; the caller later passes the result to URLSession.
WHAT **Declares a function that builds, but does not send, a POST request.**
WHERE A top-level function; often a method of an API client type.
WHEN Whenever the user creates a new to-do.
WHY It throws because encoding can fail; separating building from sending makes the request easy to inspect in a test.

```
10  let url = URL(string: "https://jsonplaceholder.typicode.com")!
```

WHO Foundation parses the base address.
WHAT **Creates the server's base URL from a fixed string.**
WHERE A local constant inside the function.
WHEN Each time makeCreateRequest(title:) runs, before the request exists.
WHY Force-unwrapping is tolerable for a literal you control and have tested; never for user input.

```
11      .appending(path: "todos")
```

WHO Foundation's URL type, through its appending(path:) method.
WHAT **Adds the path component /todos, producing the collection URL.**
WHERE Chained onto the URL from the line above.
WHEN Each call, right after the base URL is created.
WHY appending(path:) inserts the slash and encodes the component correctly, unlike string concatenation.

```
12  var request = URLRequest(url: url)
```

WHO Foundation's URLRequest initializer, called by your code.
WHAT **Creates a request with defaults: GET, no headers, no body, a 60-second timeout.**
WHERE A local variable; URLRequest is a struct.
WHEN Each time makeCreateRequest(title:) runs, right after the URL is built.
WHY It is var because the following lines change the method, headers and body.

```
13  request.httpMethod = "POST"
```

WHO Your code sets the verb.
WHAT **Changes the method from GET to POST.**
WHERE Stored in the request value.
WHEN While the request is being built.
WHY POST asks the collection to create a new member from the body.

```
14  request.setValue("application/json",
```

WHO Your code adds a header.
WHAT **Starts setting a header whose value is application/json.**
WHERE On the request; the header name follows on the next line.
WHEN While the request is being built.
WHY The server must know the body is JSON before it can parse it.

```
15  forHTTPHeaderField: "Content-Type")
```

WHO — URLRequest stores the header.
WHAT — **Names the header: Content-Type, the format of the body being sent.**
WHERE — Second half of the setValue call.
WHEN — While the request is being built.
WHY — setValue replaces any earlier value for that header, so calling it twice cannot create duplicates.

```
16  request.setValue("application/json",
```

WHO — Your code adds another header.
WHAT — **Starts a second header with the same value.**
WHERE — On the same request value, after the Content-Type header is set.
WHEN — While the request is being built.
WHY — Same value, different meaning: this one describes the reply you want.

```
17  forHTTPHeaderField: "Accept")
```

WHO — URLRequest stores the header.
WHAT — **Names it Accept: the client can read JSON responses.**
WHERE — Second half of the call.
WHEN — While the request is being built.
WHY — Some servers can answer in several formats; Accept picks one.

```
18  let todo = NewTodo(title: title, completed: false, userId: 1)
```

WHO — Your code creates the body value.
WHAT — **Builds a NewTodo with the title the user typed.**
WHERE — A local constant on the stack.
WHEN — Each time makeCreateRequest(title:) runs, once both headers are set.
WHY — Working with a typed struct instead of a hand-written JSON string means the compiler catches misspelled keys.

```
19  request.httpBody = try JSONEncoder().encode(todo)
```

WHO — JSONEncoder converts the value; the request stores the bytes.
WHAT — **Encodes the struct as JSON Data and sets it as the request body.**
WHERE — The httpBody property of the request, inside makeCreateRequest(title:).
WHEN — Each call; try passes any encoding error to the caller.
WHY — The body must be Data (bytes), and JSONEncoder produces exactly that.

```
20  request.timeoutInterval = 15
```

WHO — URLSession will enforce it when the request is sent.
WHAT — **Gives up if no data arrives for 15 seconds, instead of the 60-second default.**
WHERE — The timeoutInterval property of the same request, set after the body.
WHEN — Applied later, while the request is in flight.
WHY — A user waiting on a Save button should hear about a problem sooner than one minute.

```
21  return request
```

WHO — Your makeCreateRequest(title:) function, handing the value to its caller.
WHAT — **Hands the finished request to the caller.**
WHERE — The last line of the function.
WHEN — After every part has been set.
WHY — URLRequest is a value type, so the caller gets an independent copy it can still modify.

Try it

1. Which line would you change to make this request update to-do 5 completely instead of creating one?
2. Remove the Content-Type line in your head. What might the server do with the body then?

INTERACTIVE

HTTP request builder

Send real HTTP requests from this page. Pick a method and a preset endpoint, add headers or a JSON body, then send. You will see the raw request, the status code, the response headers the browser lets this page read, the timing and the body, plus the equivalent Swift `URLRequest` code. Try a POST, a 404, and the endpoint that requires an Authorization header.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. A user edits only their display name and taps Save. The API supports partial updates of a profile. Which method fits best?
   A. GET /profile
   B. DELETE /profile
   C. PATCH /profile with the new name
   D. POST /profiles with the new name
2. A request times out and you never see a reply. Which one can your app safely resend automatically?
   A. POST /orders with the full order in the body
   B. PUT /cart/items/42 with the full item
   C. POST /payments with the card token in the body
   D. PATCH /counter with "add 1" in the body
3. The server answers `data(from:)` with 404 Not Found. What does URLSession do?
   A. Throws URLError(.fileDoesNotExist) and returns no data
   B. Retries the request three times, then throws
   C. Throws URLError(.badServerResponse) before returning
   D. Returns normally; the response has statusCode 404
4. Code builds a URL as `"https://api.example.com/search?q=" + text`, and a user searches for *salt & pepper*. What reaches the server?
   A. q is "salt " and a stray second item " pepper" appears
   B. q is "salt & pepper", exactly as the user typed it in the field
   C. The compiler rejects the concatenation because & is not URL-safe
   D. URLSession percent-encodes the & before sending, so q is "salt & pepper"

5. An API answers 503 Service Unavailable with a Retry-After header. What should the app do?

   A. Sign the user out and show the sign-in screen
   B. Wait for the Retry-After interval, then send the request again
   C. Show "Page not found" and stop retrying
   D. Delete its cached data and reload from scratch

Answers: Speaking HTTP

1. **C. PATCH /profile with the new name** PATCH changes some fields of an existing resource. POST to the collection would ask the server to create a new profile, and GET only reads.
2. **B. PUT /cart/items/42 with the full item** PUT replaces the item with the same body every time, so sending it twice leaves the same end state. The POSTs could create a second order or payment, and "add 1" would add 2.
3. **D. Returns normally; the response has statusCode 404** A response arrived, so from URLSession's point of view the transfer succeeded. Your code must read `statusCode` and treat 404 as a failure. URLSession throws only when no usable response arrives.
4. **A. q is "salt " and a stray second item " pepper" appears** An & in a query string separates items, and neither the compiler nor URLSession repairs a string you assembled yourself. Building the URL with `URLComponents` and a `URLQueryItem` encodes it as `%26`, so the server receives the whole phrase as one value.
5. **B. Wait for the Retry-After interval, then send the request again** 5xx codes mean the server failed, and 503 specifically signals a temporary condition. Retry-After tells you how long to wait. Signing out is the reaction to 401, and "not found" is 404.

REST Resources and JSON

CONCEPT

REST: resources and verbs

REST (Representational State Transfer) is the most common style of web API. Instead of calling remote functions by name, you act on **resources**, things the server knows about, each with its own URL. The HTTP method says what to do. The server replies with a **representation** of the resource, almost always JSON. JSONPlaceholder, a free practice API, follows the pattern:

GET /todos	The collection: a JSON array of every to-do.
GET /todos/1	One member of the collection, identified by its id.
POST /todos	Create a to-do from the JSON body; the reply is 201 Created with the new id.
PATCH /todos/1	Change some fields of to-do 1, such as completed.
DELETE /todos/1	Remove to-do 1.

> **Stateless by design**
>
> Each request carries everything the server needs, including who is calling. The server does not remember earlier requests, which lets any one of many machines answer the next one.

CONCEPT

From JSON to Swift types

JSON (JavaScript Object Notation) is text built from a handful of value kinds, and each maps to a Swift type. Swift's `Codable` protocol converts between them automatically when names and types line up. `Decodable` reads JSON into Swift values, `Encodable` writes them out, and `Codable` means both.

Object { }	A struct whose stored properties match the keys.
Array [ ]	A Swift Array, such as [Todo].
String, number, true/false	String; Int or Double; Bool.
null or a missing key	An Optional property, such as let dueDate: String?. A non-optional property with no value is a decoding error.

> **Model only what you use**
>
> Your struct may skip keys it does not need. Extra keys in the JSON are ignored, so the server can add fields without breaking your app.

CODE · RUNS IN THE BROWSER

Decode your first response

This is the JSON that `GET https://jsonplaceholder.typicode.com/todos/1` really returns, embedded as a string so the program runs without a network. The struct mirrors its four keys. `JSONDecoder` reads bytes (`Data`), not strings, so the text is converted first.

```swift
import Foundation

struct Todo: Codable {
    let userId: Int
    let id: Int
    let title: String
    let completed: Bool
}

let json = """
{"userId": 1, "id": 1,
 "title": "delectus aut autem", "completed": false}
"""

let data = Data(json.utf8)
let todo = try JSONDecoder().decode(Todo.self, from: data)
print(todo.title)
print(todo.completed ? "Done" : "Still to do")
```

OUTPUT

```
delectus aut autem
Still to do
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, while it builds this script.
- WHAT — **Makes JSONDecoder and Data available.**
- WHERE — Top of the file.
- WHEN — At compile time, before the Todo struct or the decoder exist.
- WHY — JSON support is part of Foundation, not the core language.

3 `struct Todo: Codable {`

- WHO — Your code; the compiler generates the decoding logic.
- WHAT — **Declares a value type that can be read from and written to JSON.**
- WHERE — A top-level struct in this script, above the JSON it will decode.
- WHEN — Compiled once; values are created by the decoder at run time.
- WHY — Conforming to Codable is all it takes: the compiler matches each stored property to a JSON key of the same name.

4 `let userId: Int`

- WHO — The Todo struct; the generated decoder assigns it from the "userId" key.
- WHAT — **Holds the "userId" number.**
- WHERE — The first stored property inside Todo.
- WHEN — Filled in during decoding.
- WHY — The JSON number 1 has no fractional part, so Int is the right type.

5 `let id: Int`

- WHO — The Todo struct; the generated decoder assigns it from the "id" key.
- WHAT — **Holds the to-do's own identifier.**
- WHERE — The second stored property inside Todo.
- WHEN — Filled in during decoding.
- WHY — let makes decoded values read-only, which is what data from a server usually should be.

```
6  let title: String
```

WHO The Todo struct; the generated decoder assigns it from the "title" key.
WHAT **Holds the "title" text.**
WHERE The third stored property inside Todo.
WHEN Filled in during decoding.
WHY A JSON string maps to a Swift String.

```
7  let completed: Bool
```

WHO The Todo struct; the generated decoder assigns it from the "completed" key.
WHAT **Holds the true or false flag.**
WHERE The last stored property inside Todo.
WHEN Filled in during decoding.
WHY A JSON boolean maps to Bool; the string "false" would not.

```
10 let json = """
```

WHO Your code, standing in for a server reply.
WHAT **Opens a multi-line string literal holding the JSON text.**
WHERE A global constant in this script.
WHEN At program start, before the decoder runs.
WHY Triple quotes let the JSON keep its own double quotes without escaping each one.

```
11 {"userId": 1, "id": 1,
```

WHO The JSON text itself.
WHAT **Starts the object and gives the first two keys their numbers.**
WHERE Inside the string literal; Swift does not interpret it.
WHEN Only read when the decoder parses it.
WHY This is the exact response JSONPlaceholder sends, so the model is designed against real data.

```
12 "title": "delectus aut autem", "completed": false}
```

WHO The JSON text itself, the second line of the literal.
WHAT **Supplies the remaining two keys and closes the object.**
WHERE Inside the string literal.
WHEN Only read when JSONDecoder parses the bytes.
WHY The placeholder title is Latin filler text, which is what this practice API returns.

```
13 """
```

WHO The Swift compiler, which recognizes the closing delimiter.
WHAT **Closes the multi-line string.**
WHERE Its indentation sets how much leading space is removed from each line.
WHEN At compile time, when the string literal is parsed.
WHY Placing it at column 0 keeps the JSON text exactly as written.

```
15 let data = Data(json.utf8)
```

WHO Foundation's Data type, built from the string's UTF-8 view.
WHAT **Converts the string to raw UTF-8 bytes.**
WHERE A global constant in this script, directly below the JSON.
WHEN At program start, after json exists.
WHY URLSession hands you Data, never a String, so decoding from Data matches what a real request gives you.

```swift
let todo = try JSONDecoder().decode(Todo.self, from: data)
```

WHO	JSONDecoder parses the bytes and calls Todo's generated initializer.
WHAT	**Turns the JSON into a fully typed Todo value, or throws if anything does not match.**
WHERE	Top-level script code, right after data is created.
WHEN	Immediately after data is created.
WHY	Todo.self passes the type itself, telling the decoder what shape to expect.

```swift
print(todo.title)
```

WHO	Your code, reading a property of the decoded Todo.
WHAT	**Prints the decoded title: delectus aut autem.**
WHERE	Top-level script code; output goes to the console.
WHEN	After decoding succeeded on the line above.
WHY	From here on you work with properties and types, not string keys.

```swift
print(todo.completed ? "Done" : "Still to do")
```

WHO	Your code, testing the decoded Bool.
WHAT	**Prints a message based on the Bool, using the ternary operator.**
WHERE	Top-level script code, the last line of the program.
WHEN	After the title is printed.
WHY	Typed values make logic like this safe; there is no chance completed is accidentally a string.

> **Try it**
>
> 1. Add a fifth key, "priority": 2, to the JSON. Does decoding still succeed? Why?
> 2. Change "completed": false to "completed": "no". Predict the error before you run it.

CODE · RUNS IN THE BROWSER

Rename keys with CodingKeys

Open-Meteo's weather API names its fields `temperature_2m` and `wind_speed_10m`, which are poor Swift property names. A nested enum named `CodingKeys` maps each Swift property to the JSON key it comes from. Once you write it, list every property that should be decoded. The JSON's `interval` key has no property, so it is simply ignored.

```swift
import Foundation

struct Conditions: Decodable {
    let time: String
    let temperature: Double
    let windSpeed: Double

    enum CodingKeys: String, CodingKey {
        case time
        case temperature = "temperature_2m"
        case windSpeed = "wind_speed_10m"
    }
}

let json = """
{"time": "2026-10-20T14:15", "interval": 900,
 "temperature_2m": 25.7, "wind_speed_10m": 18.8}
"""
let data = Data(json.utf8)
let now = try JSONDecoder().decode(Conditions.self, from: data)
print("\(now.temperature) °C, wind \(now.windSpeed) km/h")
print("Measured at \(now.time)")
```

OUTPUT

```
25.7 °C, wind 18.8 km/h
Measured at 2026-10-20T14:15
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, building the script that declares Conditions.
- WHAT — **Imports JSONDecoder and Data.**
- WHERE — Top of the file.
- WHEN — At compile time, before Conditions or the decoder exist.
- WHY — Every decoding example needs Foundation.

3 `struct Conditions: Decodable {`

- WHO — Your code declares it; the compiler generates its decoder.
- WHAT — **Declares a read-only model for the current weather.**
- WHERE — A top-level struct in this script, with CodingKeys nested inside.
- WHEN — Values are created during decoding.
- WHY — Decodable is enough because the app only reads weather; it never sends it.

4 `let time: String`

- WHO — The Conditions struct; the generated decoder assigns it from the "time" key.
- WHAT — **Holds the measurement time as text.**
- WHERE — The first stored property inside Conditions.
- WHEN — Filled while JSONDecoder decodes a Conditions value.
- WHY — Its JSON key is already a good Swift name, so it maps unchanged.

```
5  let temperature: Double
```

WHO The Conditions struct; the decoder assigns it from the key CodingKeys names.
WHAT **Holds degrees Celsius, such as 25.7.**
WHERE The second stored property inside Conditions.
WHEN Filled during decoding, after time.
WHY Double because the value has a fractional part; Int would fail to decode 25.7.

```
6  let windSpeed: Double
```

WHO The Conditions struct; the decoder assigns it from wind_speed_10m.
WHAT **Holds wind speed in kilometres per hour.**
WHERE The last stored property inside Conditions.
WHEN Filled during decoding, after temperature.
WHY Swift style is lowerCamelCase, so the property is windSpeed even though the JSON says otherwise.

```
8  enum CodingKeys: String, CodingKey {
```

WHO The compiler's generated decoder looks for this exact name.
WHAT **Declares the list of keys, each backed by the JSON key string.**
WHERE Nested inside Conditions, so it belongs only to this type.
WHEN Used at run time whenever a Conditions value is decoded or encoded.
WHY Without it, keys must equal property names; with it, you choose the mapping.

```
9  case time
```

WHO The CodingKeys enum, consulted by the generated decoder.
WHAT **Maps time to the JSON key "time".**
WHERE The first case inside CodingKeys.
WHEN Used when the decoder looks up the "time" key.
WHY A case without a raw value uses its own name as the string.

```
10  case temperature = "temperature_2m"
```

WHO The CodingKeys enum, mapping the property to its raw value.
WHAT **Maps the temperature property to the JSON key temperature_2m.**
WHERE The second case inside CodingKeys.
WHEN Used when the decoder looks up temperature_2m in the JSON.
WHY The raw value is the key exactly as the server spells it, including the height suffix 2m.

```
11  case windSpeed = "wind_speed_10m"
```

WHO The CodingKeys enum, mapping windSpeed to its raw value.
WHAT **Maps windSpeed to wind_speed_10m.**
WHERE The last case inside CodingKeys.
WHEN Used when the decoder looks up wind_speed_10m in the JSON.
WHY Explicit mappings document the API right in the model, which helps the next reader.

```
15  let json = """
```

WHO Your code, standing in for the server.
WHAT **Opens the string holding the "current" object from a real Open-Meteo reply.**
WHERE A global constant in this script, below the Conditions type.
WHEN At program start, before decoding.
WHY Testing a model against a captured response is quick and works offline.

```
16 {"time": "2026-10-20T14:15", "interval": 900,
```

WHO The JSON text, copied from a real Open-Meteo reply.
WHAT **Provides the time and an interval key that the model does not use.**
WHERE Inside the string literal.
WHEN Read when JSONDecoder parses the bytes.
WHY interval is there on purpose: extra keys are skipped without error.

```
17 "temperature_2m": 25.7, "wind_speed_10m": 18.8}
```

WHO The JSON text, continuing the same object.
WHAT **Provides the two measurements under their snake_case names.**
WHERE Inside the string literal.
WHEN Read during parsing, right after the time key.
WHY These are the names CodingKeys points to.

```
18 """
```

WHO The Swift compiler, closing the multi-line literal.
WHAT **Ends the multi-line string.**
WHERE Column 0, so no indentation is stripped.
WHEN At compile time, when the literal is parsed.
WHY The closing delimiter must be on its own line.

```
19 let data = Data(json.utf8)
```

WHO Foundation's Data type, built from the string's bytes.
WHAT **Converts the text to bytes.**
WHERE A global constant in this script, right after the JSON.
WHEN At program start, once json exists.
WHY The decoder works on Data, the same type URLSession returns.

```
20 let now = try JSONDecoder().decode(Conditions.self, from: data)
```

WHO JSONDecoder, guided by CodingKeys.
WHAT **Decodes the bytes into a Conditions value.**
WHERE Top-level script code, directly after data.
WHEN Right after data exists.
WHY If a mapped key were misspelled, this line would throw keyNotFound and name the missing key.

```
21 print("\(now.temperature) °C, wind \(now.windSpeed) km/h")
```

WHO Your code, reading two decoded properties.
WHAT **Prints 25.7 °C, wind 18.8 km/h using string interpolation.**
WHERE Top-level script code, after decoding.
WHEN After decoding succeeds; otherwise the thrown error ends the script first.
WHY The rest of the app uses the clean Swift names; snake_case never leaks past the model.

```
22 print("Measured at \(now.time)")
```

WHO Your code, reading the decoded time string.
WHAT **Prints the timestamp string.**
WHERE Top-level script code, the last line of the program.
WHEN After the first print.
WHY A real app would convert this ISO 8601 text to a Date for display.

Try it

1. Misspell the raw value "wind_speed_10m". What does the error name?
2. Delete the line case time. Why does the program no longer compile?

CODE · RUNS IN THE BROWSER

Nested JSON, nested structs

Real responses nest objects inside objects. JSONPlaceholder's `/users/1` keeps the street and city inside an `address` object. The simplest model mirrors that nesting: a second struct for the inner object, used as the type of a property. Codable decodes the whole tree in one call.

```swift
import Foundation

struct User: Decodable {
    let name: String
    let address: Address
}

struct Address: Decodable {
    let street: String
    let city: String
}

let json = """
{"id": 1, "name": "Leanne Graham",
 "address": {"street": "Kulas Light", "city": "Gwenborough"}}
"""

let data = Data(json.utf8)
let user = try JSONDecoder().decode(User.self, from: data)
print("\(user.name) lives on \(user.address.street)")
print("in \(user.address.city)")
```

OUTPUT

```
Leanne Graham lives on Kulas Light
in Gwenborough
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, building the script that declares User and Address.
- WHAT — **Imports JSONDecoder and Data.**
- WHERE — Top of the file.
- WHEN — At compile time, before any decoding.
- WHY — Required for any JSON work.

3 `struct User: Decodable {`

- WHO — Your code declares it; the compiler generates a decoder that calls Address's.
- WHAT — **Declares the outer model, matching the top-level JSON object.**
- WHERE — A top-level struct in this script, declared before Address.
- WHEN — Values are created during decoding.
- WHY — One struct per JSON object is the most readable mapping.

```
4   let name: String
```

WHO The User struct; the generated decoder assigns it from "name".
WHAT **Holds the user's full name.**
WHERE The first stored property inside User.
WHEN Filled while JSONDecoder decodes the outer object.
WHY A plain string key, mapped by name.

```
5   let address: Address
```

WHO Your struct, delegating to another Decodable type.
WHAT **Holds the nested address object as an Address value.**
WHERE Inside User; the Address value is stored inline.
WHEN Decoded right after name, by Address's own generated code.
WHY Because Address is Decodable, User needs no extra code to handle the nesting.

```
8   struct Address: Decodable {
```

WHO Your code declares it; the compiler generates its decoder too.
WHAT **Declares the model for the inner "address" object.**
WHERE A top-level struct, reusable wherever an address appears.
WHEN Values are created while a User is decoded.
WHY Splitting models by JSON object keeps each one small and testable.

```
9   let street: String
```

WHO The Address struct; the decoder assigns it from "street".
WHAT **Holds the street name.**
WHERE The first stored property inside Address.
WHEN Filled while the nested address object is decoded.
WHY Only the keys you need are modelled; suite and zipcode are ignored.

```
10  let city: String
```

WHO The Address struct; the decoder assigns it from "city".
WHAT **Holds the city name.**
WHERE The last stored property inside Address.
WHEN Filled during decoding, right after street.
WHY Matches the "city" key exactly.

```
13  let json = """
```

WHO Your code, standing in for the server.
WHAT **Opens a trimmed copy of the /users/1 response.**
WHERE A global constant in this script, below both structs.
WHEN At program start, before the decoder runs.
WHY The real reply has more keys; leaving them out changes nothing because extras are ignored.

```
14  {"id": 1, "name": "Leanne Graham",
```

WHO The JSON text, trimmed from the real /users/1 reply.
WHAT **Starts the user object with an id the model skips and the name it keeps.**
WHERE Inside the string literal.
WHEN Read when JSONDecoder parses the outer object.
WHY id is ignored because User has no id property.

```
15  "address": {"street": "Kulas Light", "city": "Gwenborough"}}
```

WHO The JSON text, holding the inner object.
WHAT **Holds the nested address object, then closes the outer object.**
WHERE Inside the string literal.
WHEN Read when the decoder reaches the address key.
WHY The inner braces are what the Address struct mirrors.

```
16  """
```

WHO The Swift compiler, closing the multi-line literal.
WHAT **Ends the multi-line string that holds the JSON.**
WHERE Column 0 of its own line, below the JSON text.
WHEN At compile time, when the literal is parsed.
WHY Keeps the JSON free of extra indentation.

```
18  let data = Data(json.utf8)
```

WHO Foundation's Data type, built from the string's UTF-8 bytes.
WHAT **Converts the text to bytes.**
WHERE A global constant in this script, after the JSON.
WHEN At program start, once json exists.
WHY JSONDecoder reads Data, not String, so the text is converted once here.

```
19  let user = try JSONDecoder().decode(User.self, from: data)
```

WHO JSONDecoder walks the tree: User first, then Address inside it.
WHAT **Decodes the entire nested structure in one call.**
WHERE Top-level script code, right after data.
WHEN After data exists; the script stops here if decoding throws.
WHY You never write loops or key lookups; the types describe the shape.

```
20  print("\(user.name) lives on \(user.address.street)")
```

WHO Your code, reading into the nested value.
WHAT **Prints the name and street, reaching into the nested value with dot syntax.**
WHERE Top-level script code, after decoding.
WHEN After decoding succeeds on the line above.
WHY The nesting in Swift reads exactly like the nesting in the JSON.

```
21  print("in \(user.address.city)")
```

WHO Your code, reading the nested city.
WHAT **Prints the city on a second line.**
WHERE Top-level script code, the last line.
WHEN After the first print.
WHY user.address.city is fully type-checked; a typo would not compile.

> **Try it**
>
> 1. Remove "city" from the JSON. Which key and which path will the error report?

CODE · RUNS IN THE BROWSER

Flatten with a nested container

Sometimes you want a flat Swift type even though the JSON nests. Then you write `init(from:)` yourself. A **container** is a view of one JSON object, read through a CodingKey enum. `nestedContainer` steps into an inner object so you can read its keys directly. Use this sparingly: nested structs are less code.

```swift
import Foundation

struct User: Decodable {
    let name: String
    let city: String

    enum CodingKeys: String, CodingKey { case name, address }
    enum AddressKeys: String, CodingKey { case city }
    init(from decoder: Decoder) throws {
        let root = try decoder.container(keyedBy: CodingKeys.self)
        name = try root.decode(String.self, forKey: .name)
        let address = try root.nestedContainer(
            keyedBy: AddressKeys.self, forKey: .address)
        city = try address.decode(String.self, forKey: .city)
    }
}

let json = """
{"name": "Leanne Graham", "address": {"city": "Gwenborough"}}
"""
let user = try JSONDecoder().decode(User.self, from: Data(json.utf8))
print("\(user.name), \(user.city)")
```

OUTPUT

```
Leanne Graham, Gwenborough
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, building the script with the custom init(from:).
- **WHAT** **Imports the decoding types.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before the decoding types are used.
- **WHY** Decoder and the container types are Foundation-level Codable machinery.

3 `struct User: Decodable {`

- **WHO** Your code declares it and writes its decoder by hand.
- **WHAT** **Declares a flat model: no Address type at all.**
- **WHERE** A top-level struct in this script; its initializer follows below.
- **WHEN** Values are created by the custom initializer below.
- **WHY** The rest of the app only ever needs the city, so a flat type is simpler to use.

4 `let name: String`

- **WHO** The User struct; your init(from:) assigns it.
- **WHAT** **Holds the top-level name.**
- **WHERE** The first stored property inside User.
- **WHEN** Assigned in init(from:), from the outer container.
- **WHY** A let property must be assigned exactly once in the initializer.

```
5  let city: String
```

WHO The User struct; your init(from:) assigns it from the nested container.
WHAT **Holds the city, which in the JSON sits inside address.**
WHERE The second stored property inside User.
WHEN Assigned in init(from:), after name.
WHY This is the flattening: one property for a value that was two levels deep.

```
7  enum CodingKeys: String, CodingKey { case name, address }
```

WHO Your code, for the outer object.
WHAT **Lists the two outer keys this initializer reads.**
WHERE Nested inside User, above the initializer that uses it.
WHEN Used when init(from:) asks for the outer container.
WHY Each JSON object level gets its own key enum.

```
8  enum AddressKeys: String, CodingKey { case city }
```

WHO Your code, for the inner object.
WHAT **Lists the only key read from inside address.**
WHERE Nested inside User, next to CodingKeys.
WHEN Used when init(from:) steps into the address object.
WHY A separate enum makes it a compile error to ask the address container for a key that belongs elsewhere.

```
9  init(from decoder: Decoder) throws {
```

WHO JSONDecoder calls it in place of the generated initializer.
WHAT **Starts a custom decoding initializer.**
WHERE Inside User; Decodable requires exactly this signature.
WHEN Each time a User is decoded.
WHY Writing it yourself replaces the compiler's automatic version for this type.

```
10  let root = try decoder.container(keyedBy: CodingKeys.self)
```

WHO The Decoder that JSONDecoder passes in, asked for a keyed container.
WHAT **Gets a keyed container for the outer JSON object.**
WHERE A local constant inside the initializer.
WHEN First thing during decoding.
WHY It throws if the JSON at this point is not an object.

```
11  name = try root.decode(String.self, forKey: .name)
```

WHO The outer container, reading one key and handing you a String.
WHAT **Reads "name" as a String and assigns it.**
WHERE Inside init(from:), the first assignment to a property.
WHEN Right after the container is created.
WHY Each read names the type and the key, so a mismatch is caught with a precise error.

```
12  let address = try root.nestedContainer(
```

WHO The outer container, asked to open a nested object.
WHAT **Begins stepping into the object stored under a key.**
WHERE Inside the initializer; the arguments follow on the next line.
WHEN After name is read.
WHY nestedContainer gives you the inner object without a separate struct.

```
13  keyedBy: AddressKeys.self, forKey: .address)
```

WHO The outer container, finishing the nestedContainer call.
WHAT **Chooses the inner key enum and the outer key that holds the object.**
WHERE The second line of the same call.
WHEN During decoding, right after name is read.
WHY The keyedBy type defines which keys are legal inside the address object.

```
14  city = try address.decode(String.self, forKey: .city)
```

WHO The nested container for the address object.
WHAT **Reads "city" from inside address and assigns it.**
WHERE Inside init(from:), the last assignment.
WHEN After the nested container is created.
WHY After this line every let property has a value, so initialization is complete.

```
18  let json = """
```

WHO Your code, standing in for the server.
WHAT **Opens the JSON text.**
WHERE A global constant in this script, below the User type.
WHEN At program start, before the decoder runs.
WHY The same shape as before: city is nested inside address.

```
19  {"name": "Leanne Graham", "address": {"city": "Gwenborough"}}
```

WHO The JSON text, nesting city inside address.
WHAT **A user object with a nested address object.**
WHERE Inside the string literal.
WHEN Read when JSONDecoder parses the object.
WHY The Swift type is flat, but the JSON is not; the initializer bridges the two.

```
20  """
```

WHO The Swift compiler, closing the literal.
WHAT **Ends the multi-line string holding the JSON.**
WHERE Column 0 of its own line, below the JSON text.
WHEN At compile time, when the literal is parsed.
WHY Keeps the text unindented.

```
21  let user = try JSONDecoder().decode(User.self, from: Data(json.utf8))
```

WHO JSONDecoder, which calls your init(from:).
WHAT **Converts the text to bytes and decodes a flat User.**
WHERE Top-level script code, right after the JSON.
WHEN After json exists; conversion to Data and decoding happen in one line.
WHY Callers cannot tell the type has a hand-written initializer; the call looks the same.

```
22  print("\(user.name), \(user.city)")
```

WHO Your code, reading both flat properties.
WHAT **Prints the name and city.**
WHERE Top-level script code, the last line.
WHEN After the custom initializer has run without throwing.
WHY user.city reads flat even though the data was nested.

CODE · RUNS IN THE BROWSER

When decoding fails

Module 06 (Data) showed each `DecodingError` case on local JSON; here the same errors come from a live API, whose replies the short strings below stand in for. Catching specific cases turns a crash into a useful message. Each call passes different JSON; `\"` writes a quote inside a Swift string.

```
import Foundation

struct Todo: Decodable {
    let id: Int
    let title: String
}
func check(_ json: String) {
    do {
        let data = Data(json.utf8)
        let todo = try JSONDecoder().decode(Todo.self, from: data)
        print("Decoded:", todo.title)
    } catch DecodingError.keyNotFound(let key, _) {
        print("Missing key:", key.stringValue)
    } catch DecodingError.typeMismatch(let type, let context) {
        let path = context.codingPath.map { $0.stringValue }
        print("Wrong type at \(path): expected \(type)")
    } catch {
        print("Not valid JSON")
    }
}

check("{\"id\": 1, \"title\": \"Walk the dog\"}")
check("{\"id\": 2}")
check("{\"id\": \"3\", \"title\": \"Read\"}")
check("{\"id\": 4, \"title\": ")
```

OUTPUT

```
Decoded: Walk the dog
Missing key: title
Wrong type at ["id"]: expected Int
Not valid JSON
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, building the script that declares check(_:).
- **WHAT** **Imports JSONDecoder and DecodingError.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before check(_:) can run.
- **WHY** DecodingError is defined alongside the Codable types.

```
3  struct Todo: Decodable {
```

WHO Your code declares it; the compiler generates its decoder.
WHAT **Declares a small model with two required properties.**
WHERE A top-level struct in this script, above check(_:).
WHEN Decoded once per check call.
WHY Required (non-optional) properties make the failure cases easy to trigger.

```
4      let id: Int
```

WHO The Todo struct; the generated decoder assigns it from "id".
WHAT **Must be a JSON number.**
WHERE The first stored property inside Todo.
WHEN Filled during decoding, if the key holds a number.
WHY A quoted "3" is a string, not a number, and will not decode as Int.

```
5      let title: String
```

WHO The Todo struct; the decoder assigns it from "title".
WHAT **Must be present and a string.**
WHERE The last stored property inside Todo.
WHEN Filled during decoding, if the key is present.
WHY Non-optional, so a missing title is an error rather than nil.

```
7  func check(_ json: String) {
```

WHO Your code; the four calls at the bottom invoke it.
WHAT **Declares a helper that tries to decode one JSON string and reports the result.**
WHERE A top-level function in this script, below Todo.
WHEN Once for each call at the bottom.
WHY It does not throw: every error is handled inside, which is what a screen would do.

```
8      do {
```

WHO Swift's error-handling syntax, in your check function.
WHAT **Starts a block whose thrown errors flow to the catch clauses below.**
WHERE Inside check(_:), wrapping the decode call.
WHEN Each time check(_:) is called.
WHY do/catch is where you decide what each failure means to the user.

```
9          let data = Data(json.utf8)
```

WHO Foundation's Data type, built from the argument string.
WHAT **Converts the text to bytes.**
WHERE A local constant inside do.
WHEN Each call, before the decoder runs.
WHY Same first step as with a real network response.

```
10         let todo = try JSONDecoder().decode(Todo.self, from: data)
```

WHO JSONDecoder, which either returns a Todo or throws.
WHAT **Attempts the decode; on failure, jumps to the first matching catch.**
WHERE Inside the do block of check(_:).
WHEN Each call, immediately after data exists.
WHY try marks the exact line that can fail.

```
11 print("Decoded:", todo.title)
```

WHO Your code, reading the decoded title.
WHAT **Reports success with the decoded title.**
WHERE Inside do; runs only if decoding succeeded.
WHEN After a successful decode.
WHY Lines after a throwing call run only on the success path.

```
12 } catch DecodingError.keyNotFound(let key, _) {
```

WHO Swift pattern matching on the thrown error.
WHAT **Catches a missing key and binds which key it was.**
WHERE The first catch clause.
WHEN When a required key is absent.
WHY The underscore ignores the context value, which this message does not need.

```
13 print("Missing key:", key.stringValue)
```

WHO Your code, reading the bound key.
WHAT **Prints the name of the missing key.**
WHERE Inside the first catch.
WHEN For the second call below.
WHY stringValue turns the CodingKey back into the JSON key text.

```
14 } catch DecodingError.typeMismatch(let type, let context) {
```

WHO Swift pattern matching on the thrown DecodingError.
WHAT **Catches a value of the wrong JSON type, binding the expected type and the context.**
WHERE The second catch clause.
WHEN When, say, a string arrives where a number was expected.
WHY The context carries the codingPath, the route to the bad value.

```
15 let path = context.codingPath.map { $0.stringValue }
```

WHO Your code, using map with a shorthand closure.
WHAT **Turns the path of CodingKeys into an array of strings.**
WHERE Inside the second catch.
WHEN For the third call below.
WHY A readable path such as ["address", "city"] points straight at the problem in big JSON.

```
16 print("Wrong type at \(path): expected \(type)")
```

WHO Your code, interpolating the path and the expected type.
WHAT **Prints the path and the type the model wanted.**
WHERE Inside the second catch.
WHEN For the third call.
WHY Interpolating a type prints its name, Int here.

```
17 } catch {
```

WHO Swift's catch-all clause, with the error bound as error.
WHAT **Catches every other error, bound automatically as error.**
WHERE The final catch clause.
WHEN For the fourth call, whose text is not valid JSON.
WHY A non-throwing function must handle every possible error, so a catch-all is required.

```
18  print("Not valid JSON")
```

WHO Your code, reporting the remaining case.
WHAT **Reports the remaining case: the text could not be parsed at all.**
WHERE Inside the last catch.
WHEN For the fourth call.
WHY That case is DecodingError.dataCorrupted; this example keeps the message general.

```
22  check("{\"id\": 1, \"title\": \"Walk the dog\"}")
```

WHO Your code, calling check with valid JSON.
WHAT **Decodes valid JSON and prints the title.**
WHERE Top-level script code, the first of four calls.
WHEN First of the four calls, when the script runs.
WHY A baseline: the happy path.

```
23  check("{\"id\": 2}")
```

WHO Your code, calling check with incomplete JSON.
WHAT **Passes JSON with no title.**
WHERE Top-level script code, the second call.
WHEN Second, after the first call returns.
WHY The missing title triggers keyNotFound, so the first catch clause runs.

```
24  check("{\"id\": \"3\", \"title\": \"Read\"}")
```

WHO Your code, calling check with a quoted id.
WHAT **Passes an id written as a string.**
WHERE Top-level script code, the third call.
WHEN Third, after the missing-key case.
WHY Triggers typeMismatch at ["id"].

```
25  check("{\"id\": 4, \"title\": ")
```

WHO Your code, calling check with truncated text.
WHAT **Passes text that stops in the middle.**
WHERE Top-level script code, the last call.
WHEN Last, after the type-mismatch case.
WHY Truncated replies happen on flaky networks, and they must not crash the app.

> **Try it**
>
> 1. Add check("{\"id\": 5, \"title\": null}"). Which catch clause runs, and why?

INTERACTIVE

JSON to Swift decoder lab

Load a real response (or the offline sample), then edit the JSON or the Swift model. The lab decodes it the way `JSONDecoder` does and reports either the decoded value or the exact `DecodingError` case, coding path and message. Try deleting a key, putting a number in quotes, making a property optional, setting a value to null, or decoding an array as a single object.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. The JSON is `{"id": 7, "title": "Run", "done": true}`. The struct has `let id: Int`, `let title: String` and `let completed: Bool`, with no CodingKeys. What happens?
   A. It decodes, and completed defaults to false
   B. It throws keyNotFound for completed
   C. It throws typeMismatch for done
   D. It decodes, and completed becomes nil
2. The server sometimes leaves out `"dueDate"`. How should you declare the property?
   A. let dueDate: String
   B. var dueDate = "" so the default covers a missing key
   C. let dueDate: String?
   D. let dueDate: Any
3. The JSON key is `temp_c` and you want a Swift property called `temperature`. What do you add?
   A. Declare it as var temperature: Double? so the key is optional
   B. A string replace of temp_c with temperature before decoding
   C. Decode into [String: Double] and read the temp_c key
   D. CodingKeys with case temperature = "temp_c"
4. Decoding `{"id": "42"}` into a struct with `let id: Int` throws which error?
   A. typeMismatch at ["id"], expecting Int
   B. keyNotFound at ["id"], because "42" is not a number
   C. No error; JSONDecoder converts "42" to 42
   D. valueNotFound at ["id"]

Answers: REST Resources and JSON

1. **B. It throws keyNotFound for completed** Without CodingKeys, the property completed looks for a key named "completed", which is missing. Non-optional properties have no default during decoding, and done is simply ignored as an extra key.
2. **C. let dueDate: String?** An optional property decodes to nil when the key is missing or null. The tempting `var dueDate = ""` does not help: the generated decoder still requires the key and throws keyNotFound. Any is not Decodable at all.
3. **D. CodingKeys with case temperature = "temp_c"** CodingKeys maps property names to JSON keys. Making the property optional would only hide the problem: it would always be nil because no key named temperature exists. Editing the text or decoding a dictionary throws away the typed model you want.
4. **A. typeMismatch at ["id"], expecting Int** The key exists, so it is not keyNotFound, and the value is not null, so it is not valueNotFound. JSONDecoder never converts between strings and numbers; a quoted number is a string.

URLSession with async/await

CONCEPT

Meet URLSession

`URLSession` is the networking engine built into every Apple platform. It manages connections, follows redirects, stores cookies, caches responses, negotiates HTTP versions and does all of the waiting off the main thread. You describe *what* you want; it handles *how*.

Its main methods are `async`: you `await` them and receive the result as a return value, or a thrown error.

URLSession.shared	A ready-made session with default settings, fine for most simple requests.
data(from:) and data(for:)	Fetch a URL or send a URLRequest; returns the body as Data plus the URLResponse.
upload and download	upload(for:from:) sends a body; download(from:) saves a large file to disk.
bytes(for:)	Delivers the body as it arrives instead of waiting for all of it.
URLSessionConfiguration	Builds custom sessions: timeouts, caching, cellular rules, extra headers.

> **Before async/await**
>
> Older code passes a *completion handler* closure to `dataTask(with:)` and then calls `resume()`. You will meet it in existing projects; the async methods replace it.

CODE · READ-ALONG

Your first real fetch

Four phases appear in nearly every network call: build the URL, await the response, check the status code, decode the body. This function fetches one to-do from JSONPlaceholder using the `Todo` struct from the JSON lesson. It is `async throws`: callers must `await` it, and every failure (offline, a bad status, unexpected JSON) arrives as a thrown error.

```swift
import Foundation

enum APIError: Error {
    case badStatus(Int)
}

func fetchTodo(id: Int) async throws -> Todo {
    let base = URL(string: "https://jsonplaceholder.typicode.com")!
    let url = base.appending(path: "todos/\(id)")
    let (data, response) =
        try await URLSession.shared.data(from: url)
    guard let http = response as? HTTPURLResponse else {
        throw URLError(.badServerResponse)
    }
    guard (200..<300).contains(http.statusCode) else {
        throw APIError.badStatus(http.statusCode)
    }
    return try JSONDecoder().decode(Todo.self, from: data)
}
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, building the file that holds fetchTodo(id:).
- WHAT — **Imports URLSession, URL, HTTPURLResponse and JSONDecoder.**
- WHERE — Top of the file.
- WHEN — At compile time, before fetchTodo(id:) can be called.
- WHY — All four phases use Foundation types.

3 `enum APIError: Error {`

- WHO — Your code declares it; fetchTodo throws it.
- WHAT — **Declares the app's own error type for problems URLSession does not report.**
- WHERE — A top-level enum in the networking file, above fetchTodo(id:).
- WHEN — A value is created only when a bad status arrives.
- WHY — Conforming to Error lets you throw it; an enum lists every failure your API layer can produce.

4 `case badStatus(Int)`

- WHO — The APIError enum, carrying the status code as a payload.
- WHAT — **One case that carries the status code that was received.**
- WHERE — The only case inside APIError.
- WHEN — Created when the status check fails.
- WHY — Keeping the code lets callers react differently to 401 and 503.

7 `func fetchTodo(id: Int) async throws -> Todo {`

- WHO — Your code; a model or view awaits it.
- WHAT — **Declares an asynchronous, throwing function that returns a decoded Todo.**
- WHERE — A top-level function in the networking file, below APIError.
- WHEN — Whenever a caller awaits it.
- WHY — async allows suspension while waiting; throws lets every failure reach the caller with full detail.

```swift
8  let base = URL(string: "https://jsonplaceholder.typicode.com")!
```

WHO Foundation's URL initializer, parsing the literal.
WHAT **Phase one: creates the API's base URL.**
WHERE A local constant inside fetchTodo(id:), the first statement.
WHEN Each time fetchTodo(id:) runs, before anything touches the network.
WHY A literal you control can be force-unwrapped safely; it would fail on the very first test run if it were wrong.

```swift
9  let url = base.appending(path: "todos/\(id)")
```

WHO Foundation's URL type, through appending(path:).
WHAT **Adds the resource path, for example /todos/1.**
WHERE A local constant inside fetchTodo(id:), right after base.
WHEN Each call, immediately after the base URL exists.
WHY The id is an Int, so it cannot inject unexpected characters into the path.

```swift
10 let (data, response) =
```

WHO Your code, declaring the two results of the fetch.
WHAT **Declares two constants that receive the tuple returned on the next line.**
WHERE Local constants inside fetchTodo(id:), filled by the next line.
WHEN Assigned when the network replies.
WHY Destructuring a tuple names both results at once.

```swift
11 try await URLSession.shared.data(from: url)
```

WHO URLSession performs the whole request on its own threads.
WHAT **Phase two: sends a GET request and suspends until the full body arrives.**
WHERE Called from fetchTodo; the caller's thread is freed while waiting.
WHEN Execution resumes here after the reply, or jumps out with a thrown URLError.
WHY try handles transport failures; await marks the point where this function pauses without blocking anything.

```swift
12 guard let http = response as? HTTPURLResponse else {
```

WHO Swift's as? cast, checking the response's real type.
WHAT **Checks that the response really is an HTTP response and unwraps it.**
WHERE Inside fetchTodo(id:), immediately after the await returns.
WHEN Right after the reply arrives.
WHY data(from:) returns the general URLResponse type; statusCode exists only on its HTTP subclass.

```swift
13 throw URLError(.badServerResponse)
```

WHO Your code, reusing a Foundation error.
WHAT **Fails with a standard error if the cast did not succeed.**
WHERE Inside the guard's else branch, which must leave the function.
WHEN Practically never for https URLs, but the compiler requires a plan.
WHY Reusing URLError keeps error handling uniform for callers.

```swift
15 guard (200..<300).contains(http.statusCode) else {
```

WHO Your code, checking the status code URLSession delivered.
WHAT **Phase three: accepts only 2xx status codes.**
WHERE Inside fetchTodo(id:), after the cast succeeds and before decoding.
WHEN After the cast succeeds.
WHY URLSession does not throw for 404 or 500; without this line an error page would be passed to the decoder.

16 `throw APIError.badStatus(http.statusCode)`

WHO	Your code, creating and throwing the APIError value.
WHAT	**Throws your own error carrying the real status code.**
WHERE	The else branch of the status guard.
WHEN	Whenever the server answers with 4xx or 5xx.
WHY	The caller learns what actually happened instead of a confusing decoding error.

18 `return try JSONDecoder().decode(Todo.self, from: data)`

WHO	JSONDecoder, which either returns a Todo or throws a DecodingError.
WHAT	**Phase four: decodes the body into a Todo and returns it.**
WHERE	The last line of fetchTodo.
WHEN	Only after the status check passed.
WHY	A decoding failure is thrown too, so the caller handles every problem in one catch.

> **Try it**
>
> 1. Which line would you change to accept only 200 and nothing else in the 2xx family?
> 2. List the three kinds of errors a caller of fetchTodo could receive.

CODE · READ-ALONG

A small API client

Instead of scattering URLs across your views, gather one API's calls in one type. `WeatherClient` knows Open-Meteo's address and query items and returns a typed `CurrentWeather` (the CodingKeys model from the JSON lesson, wrapped in a `Forecast`). Views never see URLs or JSON. The session is a stored property, so a test can pass in a differently configured one.

```
import Foundation

struct WeatherClient: Sendable {
    var session = URLSession.shared
    func current(latitude: Double,
                 longitude: Double) async throws -> CurrentWeather {
        var parts = URLComponents(
            string: "https://api.open-meteo.com/v1/forecast")!
        parts.queryItems = [
            URLQueryItem(name: "latitude", value: "\(latitude)"),
            URLQueryItem(name: "longitude", value: "\(longitude)"),
            URLQueryItem(name: "current",
                         value: "temperature_2m,wind_speed_10m")
        ]
        let (data, response) = try await session.data(from: parts.url!)
        guard let http = response as? HTTPURLResponse,
              http.statusCode == 200 else {
            throw URLError(.badServerResponse)
        }
        return try JSONDecoder()
            .decode(Forecast.self, from: data).current
    }
}
```

Line by line

```
1  import Foundation
```

WHO	The Swift compiler, building the file that declares WeatherClient.
WHAT	**Imports the networking and JSON types.**
WHERE	Top of the file.
WHEN	At compile time, before WeatherClient can be used.
WHY	The client is pure Foundation; it does not need SwiftUI.

```
3  struct WeatherClient: Sendable {
```

WHO	Your code declares it; the compiler checks the Sendable promise.
WHAT	**Declares the client as a value type that is safe to share between concurrent tasks.**
WHERE	A top-level struct in the networking layer, below the models.
WHEN	Created by whichever model needs weather.
WHY	Sendable lets Swift 6 prove it can be used from the main actor and from background tasks without data races.

```
4  var session = URLSession.shared
```

WHO	Your struct stores a URLSession.
WHAT	**Uses the shared session unless someone passes another.**
WHERE	A stored property with a default value.
WHEN	Set when the client is created.
WHY	Making the session injectable lets you supply a session with caching or short timeouts later.

```
5  func current(latitude: Double,
```

WHO	Your code declares it; WeatherModel awaits it.
WHAT	**Begins a method that returns the weather at a coordinate.**
WHERE	Inside WeatherClient; the signature continues on the next line.
WHEN	Whenever a caller awaits it.
WHY	Labels make the call site read clearly: current(latitude:longitude:).

```
6  longitude: Double) async throws -> CurrentWeather {
```

WHO	Your code, completing the method declaration.
WHAT	**Finishes the signature: asynchronous, throwing, returning a CurrentWeather.**
WHERE	Part of the method declaration.
WHEN	Checked at compile time; the body runs on each call.
WHY	Returning a domain type instead of Data keeps JSON details inside the client.

```
7  var parts = URLComponents(
```

WHO	Foundation's URLComponents initializer, called by your code.
WHAT	**Starts building the request URL from a base string.**
WHERE	A local variable inside current.
WHEN	Each time current(latitude:longitude:) runs, before the request.
WHY	URLComponents encodes query values correctly.

```
8  string: "https://api.open-meteo.com/v1/forecast")!
```

WHO	URLComponents parses the base address.
WHAT	**Supplies the forecast endpoint and force-unwraps the result.**
WHERE	The argument of the call on the previous line.
WHEN	Each call, as the first step of building the URL.
WHY	The string is a constant you control, so a failure here would be a programming error found immediately.

```
9  parts.queryItems = [
```

WHO Your code, assigning to the queryItems property.
WHAT **Starts the list of query items.**
WHERE On the URLComponents value.
WHEN Each call, right after parts is created.
WHY Each option is a separate item the API documents.

```
10 URLQueryItem(name: "latitude", value: "\(latitude)"),
```

WHO Your code, creating the first query item.
WHAT **Adds the latitude, converted to text with interpolation.**
WHERE The first query item.
WHEN Each call, while the array literal is built.
WHY Query values are always strings in a URL.

```
11 URLQueryItem(name: "longitude", value: "\(longitude)"),
```

WHO Your code, creating the second query item.
WHAT **Adds the longitude, converted to text with interpolation.**
WHERE The second query item.
WHEN Each call, right after the latitude item.
WHY Together the two values identify the place to forecast.

```
12 URLQueryItem(name: "current",
```

WHO Your code, starting the third query item.
WHAT **Starts the item that lists which current measurements to return.**
WHERE The third query item; its value is on the next line.
WHEN Each call, after the two coordinate items.
WHY Open-Meteo returns only the variables you name, keeping the response small.

```
13 value: "temperature_2m,wind_speed_10m")
```

WHO Your code, supplying the item's value.
WHAT **Asks for temperature and wind speed, separated by a comma.**
WHERE Value of the third item.
WHEN Each call, as the array literal is completed.
WHY The comma is legal in a query value, so URLComponents leaves it as is.

```
15 let (data, response) = try await session.data(from: parts.url!)
```

WHO The injected URLSession, on its own background threads.
WHAT **Sends the request and suspends until the reply arrives.**
WHERE Inside current; the network work runs on URLSession's threads.
WHEN Resumes when the body has fully arrived or an error is thrown.
WHY Using the stored session rather than URLSession.shared respects whatever session the creator injected.

```
16 guard let http = response as? HTTPURLResponse,
```

WHO Swift's as? cast, checking the response type.
WHAT **Unwraps the response as an HTTP response.**
WHERE Inside current; the guard continues on the next line.
WHEN After the await resumes with a reply.
WHY You need HTTPURLResponse to read the status code.

```
17  http.statusCode == 200 else {
```

WHO Your code, comparing the status code to 200.
WHAT **Requires exactly 200 OK, the only success this endpoint returns.**
WHERE Second condition of the same guard.
WHEN After the cast succeeds.
WHY Open-Meteo answers 400 with an error message for impossible coordinates; this rejects it.

```
18  throw URLError(.badServerResponse)
```

WHO Your code, throwing a Foundation error value.
WHAT **Fails with a standard error when either condition is false.**
WHERE The guard's else branch.
WHEN Only on a bad reply.
WHY Callers handle it exactly like a transport error.

```
20  return try JSONDecoder()
```

WHO Foundation's JSONDecoder, created by your code.
WHAT **Creates a decoder and begins the return statement; the call continues on the next line.**
WHERE The last statement of current.
WHEN After the status check passed.
WHY A fresh JSONDecoder is cheap, and its default settings suit this API.

```
21  .decode(Forecast.self, from: data).current
```

WHO JSONDecoder decodes Forecast; your code then reads current.
WHAT **Decodes the outer Forecast object and returns only its current part.**
WHERE Chained onto the decoder from the line above.
WHEN After the status check passed.
WHY The nested wrapper is an implementation detail callers never see.

> **Try it**
>
> 1. Add a third measurement, relative_humidity_2m, to the request. Which two places must change?

INTERACTIVE

Watch a request happen

Pick a city and fetch its current weather from Open-Meteo, for real. The Swift code beside it is the client and model from this lesson. Each line lights up as the matching phase actually happens in this page: building the URL, waiting for the network, checking the status, decoding, updating the screen state. The timings are measured, not invented. Fetch the same city twice and compare, then try an invalid coordinate.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Loading state as an enum

A screen that loads data is always in one of a few states. An enum with associated values makes them explicit, so the view can never show a spinner and stale data at the same time by mistake. `@Observable` lets SwiftUI watch `state`; `@MainActor` keeps every change on the main thread, where interface updates belong.

```swift
import Observation

enum LoadState {
    case idle
    case loading
    case loaded(CurrentWeather)
    case failed(String)
}

@MainActor
@Observable
final class WeatherModel {
    private(set) var state = LoadState.idle
    private let client = WeatherClient()
    func load(latitude: Double, longitude: Double) async {
        state = .loading
        do {
            state = .loaded(try await client.current(
                latitude: latitude, longitude: longitude))
        } catch {
            state = .failed("Could not load the weather.")
        }
    }
}
```

Line by line

1 `import Observation`

WHO	The Swift compiler, building the file that holds WeatherModel.
WHAT	**Imports the @Observable macro.**
WHERE	Top of the file.
WHEN	At compile time, before the @Observable macro can expand.
WHY	This file holds no views, so it imports Observation directly instead of all of SwiftUI.

3 `enum LoadState {`

WHO	Your code declares it; WeatherModel stores one value of it.
WHAT	**Declares every state the weather screen can be in.**
WHERE	A top-level enum in the model file, above WeatherModel.
WHEN	A value exists for as long as the model does.
WHY	An enum makes impossible combinations, such as loading and failed at once, unrepresentable.

4 `case idle`

WHO	The LoadState enum; the model starts in this case.
WHAT	**Nothing has been requested yet.**
WHERE	The first case inside LoadState.
WHEN	Before the first load.
WHY	Distinguishing idle from loading lets a screen show a prompt instead of a spinner.

```
5  case loading
```

WHO	The LoadState enum; load sets this case first.
WHAT	**A request is in flight.**
WHERE	The second case inside LoadState.
WHEN	From the start of a load until it finishes.
WHY	The view shows progress during this state.

```
6  case loaded(CurrentWeather)
```

WHO	The LoadState enum, carrying a CurrentWeather payload.
WHAT	**Success, carrying the weather as an associated value.**
WHERE	The third case inside LoadState.
WHEN	After a successful load.
WHY	The data travels with the state, so there is no separate optional property that might be stale.

```
7  case failed(String)
```

WHO	The LoadState enum, carrying a message payload.
WHAT	**Failure, carrying a message for the user.**
WHERE	The last case inside LoadState.
WHEN	After any thrown error.
WHY	The view can show the message and a retry button.

```
10 @MainActor
```

WHO	The Swift compiler enforces it.
WHAT	**Isolates the whole class to the main actor, meaning the main thread.**
WHERE	Applies to every property and method of WeatherModel.
WHEN	Checked at compile time; enforced at run time by switching to the main thread.
WHY	SwiftUI reads state on the main thread, so all writes must happen there too; Swift 6 turns violations into compile errors.

```
11 @Observable
```

WHO	The Observation macro, expanded by the compiler.
WHAT	**Makes SwiftUI track reads of this class's properties.**
WHERE	Applies to the class below.
WHEN	Expanded at compile time; tracking happens whenever a view's body runs.
WHY	When state changes, only the views that read it are redrawn.

```
12 final class WeatherModel {
```

WHO	Your code declares it; WeatherView creates and observes it.
WHAT	**Declares the model that owns the screen's state and loading logic.**
WHERE	A reference type on the heap, shared by the view that owns it.
WHEN	Created once when the view first appears.
WHY	A class gives the view one stable object whose state can change over time.

```
13 private(set) var state = LoadState.idle
```

WHO	WeatherModel owns it; SwiftUI observes it.
WHAT	**Stores the current state, starting at idle.**
WHERE	A stored property, observed by SwiftUI.
WHEN	Read every time the view's body runs; written during load.
WHY	private(set) lets views read state but only the model change it.

```
14  private let client = WeatherClient()
```

WHO WeatherModel owns it; only load uses it.
WHAT **Holds the API client from the previous step.**
WHERE A private stored property.
WHEN Created with the model.
WHY The model talks to the client; the view never does.

```
15  func load(latitude: Double, longitude: Double) async {
```

WHO Your model; the view calls it.
WHAT **Declares the loading method. It does not throw: it turns errors into state.**
WHERE A method of WeatherModel, isolated to the main actor like the class.
WHEN When the view appears, or when the user taps Try Again.
WHY Moving error handling into the model keeps the view free of do/catch.

```
16      state = .loading
```

WHO WeatherModel, writing its own state property.
WHAT **Switches to the loading state.**
WHERE On the main actor.
WHEN Immediately, before any network work.
WHY The view redraws with a spinner right away, so the tap feels responsive.

```
17      do {
```

WHO Swift's error-handling syntax, inside load.
WHAT **Starts the block that may throw.**
WHERE Inside load, wrapping the one call that can throw.
WHEN Each time load runs, after state becomes loading.
WHY Every failure path ends in the catch below.

```
18          state = .loaded(try await client.current(
```

WHO Your model awaits the client.
WHAT **Suspends until the weather arrives, then stores it inside the loaded case.**
WHERE On the main actor; the network work itself runs elsewhere.
WHEN Resumes on the main actor after the reply.
WHY While suspended, the main thread is free to draw the spinner.

```
19              latitude: latitude, longitude: longitude))
```

WHO WeatherModel, passing its parameters to the client.
WHAT **Passes the coordinates through to the client.**
WHERE Arguments of the call on the previous line.
WHEN Each load, as the call on the line above is evaluated.
WHY The model decides what to load; the client decides how.

```
20      } catch {
```

WHO Swift's catch clause, paired with the do block.
WHAT **Catches any error: offline, timeout, bad status or bad JSON.**
WHERE Inside load, directly after the do block.
WHEN Only when something failed.
WHY One catch covers every failure the client can throw.

```
21  state = .failed("Could not load the weather.")
```

WHO — WeatherModel, writing its state on the main actor.
WHAT — **Stores a friendly failure message.**
WHERE — On the main actor.
WHEN — After any error thrown by the client.
WHY — Users need a plain sentence and a way to retry, not an error code.

CODE · READ-ALONG

Calling it from SwiftUI with .task

The view owns the model with `@State` and switches over its state. `.task` starts the load when the view appears and cancels it automatically if the view disappears first, so you never fetch for a screen nobody sees. The panel runs the same flow against the live API.

```
1  import SwiftUI
2  struct WeatherView: View {
3      @State private var model = WeatherModel()
4
5      var body: some View {
6          VStack(spacing: 12) {
7              switch model.state {
8              case .idle, .loading:
9                  ProgressView("Loading weather")
10             case .loaded(let now):
11                 Text("\(now.temperature, specifier: "%.1f") °C")
12             case .failed(let message):
13                 Text(message)
14                 Button("Try Again") { Task { await load() } }
15             }
16         }
17         .task { await load() }
18     }
19
20     private func load() async {
21         await model.load(latitude: 35.68, longitude: 139.69)
22     }
23 }
```

Line by line

```
1  import SwiftUI
```

WHO — The Swift compiler, building the file that holds WeatherView.
WHAT — **Imports SwiftUI, which also re-exports Observation.**
WHERE — Top of the file.
WHEN — At compile time, before the view can be built.
WHY — View, ProgressView, Button and .task all come from SwiftUI; the model file needed only Observation.

```
2  struct WeatherView: View {
```

WHO — Your code; SwiftUI renders it.
WHAT — **Declares the weather screen.**
WHERE — A struct describing interface, recreated often.
WHEN — Whenever SwiftUI needs a fresh description.
WHY — Views are cheap values; the model is where long-lived state lives.

```
3  @State private var model = WeatherModel()
```

WHO SwiftUI stores the model for the view's lifetime.
WHAT **Creates the model once and keeps it across body recomputations.**
WHERE Stored by SwiftUI outside the struct.
WHEN Created when the view first appears.
WHY @State with an @Observable class is the modern way for a view to own its model.

```
5  var body: some View {
```

WHO SwiftUI calls it; your code describes the screen inside.
WHAT **Describes what to show for the current state.**
WHERE A computed property of the view.
WHEN Every time model.state changes.
WHY body must be fast and free of side effects; loading happens in .task, not here.

```
6  VStack(spacing: 12) {
```

WHO SwiftUI's layout system, arranging the children.
WHAT **Stacks the content vertically.**
WHERE The root of body.
WHEN Each time body runs, which happens after model.state changes.
WHY The stack is the view the .task modifier attaches to.

```
7  switch model.state {
```

WHO Your code; Observation records this read.
WHAT **Chooses content based on the current state.**
WHERE Inside the VStack builder.
WHEN Each time body runs; Observation re-runs body when state changes.
WHY switch must cover every case, so adding a state later forces you to design its screen.

```
8  case .idle, .loading:
```

WHO Swift pattern matching inside the view builder.
WHAT **Handles both the not-started and in-progress states.**
WHERE The first case of the switch over model.state.
WHEN Before the first load finishes.
WHY Both look the same to the user here, so one case lists both.

```
9  ProgressView("Loading weather")
```

WHO SwiftUI, drawing the spinner and its label.
WHAT **Shows a spinner with a label.**
WHERE Inside the first case.
WHEN While state is idle or loading.
WHY A label tells VoiceOver users what is happening.

```
10 case .loaded(let now):
```

WHO Swift pattern matching, binding the payload.
WHAT **Matches success and binds the associated weather value.**
WHERE The second case of the switch.
WHEN After a successful load.
WHY The data is available only in this branch, so it can never be shown in the wrong state.

```
11 Text("\(now.temperature, specifier: "%.1f") °C")
```

WHO SwiftUI, formatting and drawing the text.
WHAT **Shows the temperature with one decimal place.**
WHERE Inside the loaded case.
WHEN After a successful load, each time body runs.
WHY The specifier formats the Double inside Text without extra code.

```
12 case .failed(let message):
```

WHO Swift pattern matching, binding the message.
WHAT **Matches failure and binds the message.**
WHERE The last case of the switch.
WHEN After load stored a failed state.
WHY Failure is a first-class screen state, not an afterthought.

```
13 Text(message)
```

WHO SwiftUI, drawing the message text.
WHAT **Shows the friendly error message.**
WHERE Inside the failed case.
WHEN After a failure, each time body runs.
WHY The model wrote the message; the view only displays it.

```
14 Button("Try Again") { Task { await load() } }
```

WHO SwiftUI runs the action when the user taps.
WHAT **Starts a new task that reloads.**
WHERE Inside the failed case.
WHEN When the user taps the button.
WHY A Button action is synchronous, so Task { } is needed to call async code.

```
17 .task { await load() }
```

WHO SwiftUI manages this task's lifetime.
WHAT **Starts loading when the view appears.**
WHERE A modifier on the VStack.
WHEN Once when the view appears; cancelled automatically if it disappears first.
WHY Tying work to the view's lifetime prevents updates for screens that are gone.

```
20 private func load() async {
```

WHO WeatherView; .task and the button call it.
WHAT **Wraps the model call with this screen's coordinates.**
WHERE A private helper method of the view.
WHEN Called from .task and from the Try Again button.
WHY Both callers share one place that knows the coordinates.

```
21 await model.load(latitude: 35.68, longitude: 139.69)
```

WHO Your model does the work.
WHAT **Loads the weather for Tokyo.**
WHERE On the main actor.
WHEN Each time load runs.
WHY Real apps take the coordinate from user input or from location services.

CHECK YOURSELF

Check your understanding

1. In what order does a typical fetch function work?
   A. Decode, await the response, check the status, build the URL
   B. Check the status, build the URL, decode, await the response
   C. Build the URL, await the response, check the status, decode
   D. Await the response, build the URL, decode, check the status
2. The user leaves a screen before its `.task` load finishes. What happens to the work?
   A. It keeps running and later updates a view that no longer exists
   B. SwiftUI cancels the task, and the pending await throws a cancellation error
   C. The app crashes when the task tries to update the missing view
   D. SwiftUI pauses the task and resumes it when the user returns
3. Why is `WeatherModel` marked `@MainActor`?
   A. So every write to state happens on the main thread
   B. So the network request itself runs on the main thread
   C. Because @Observable tracks only main-actor isolated classes
   D. So URLSession can finish the request faster on a busy phone
4. What is wrong with this code? `let (data, _) = try await URLSession.shared.data(from: url)` followed by `return try JSONDecoder().decode(User.self, from: data)`
   A. data(from:) can return before the whole body arrives, so data may be incomplete
   B. JSONDecoder cannot decode Data directly; it needs a String first
   C. The tuple is in the wrong order: the response comes before the data
   D. It never checks the status code, so a 500 error page reaches the decoder

Answers: URLSession with async/await

1. **C. Build the URL, await the response, check the status, decode** You need a URL before you can send anything, a response before you can check it, and a successful status before decoding makes sense. Decoding before checking the status feeds error pages to the decoder.
2. **B. SwiftUI cancels the task, and the pending await throws a cancellation error** .task is tied to the view's lifetime. Cancellation makes URLSession stop the request and throw, which is why your code should treat cancellation as quiet, not as a failure to show.
3. **A. So every write to state happens on the main thread** The request still runs on URLSession's own threads; await simply suspends the model without blocking. The attribute guarantees that every write to state happens on the main thread. @Observable works without it.
4. **D. It never checks the status code, so a 500 error page reaches the decoder** Discarding the response skips the status check. When the server fails, the decoder receives an error body and throws a misleading DecodingError instead of a clear "server error 500". data(from:) does wait for the whole body, the decoder does take Data, and the tuple order is data first.

Errors, Retries and Security

CONCEPT

Everything that can go wrong

On a phone, failure is normal: elevators, tunnels, crowded stadiums, airplane mode. Each kind of failure deserves a different response, so your code has to tell them apart.

No connection	URLError .notConnectedToInternet or .networkConnectionLost. Show saved data if you have it, plus a calm offline message.
Timeout or unreachable	.timedOut (no data within the timeout, 60 seconds by default), .cannotFindHost (the DNS lookup failed) or .cannotConnectToHost. Often temporary.
Bad status	A reply arrived with 4xx or 5xx. URLSession does not throw; your status check does.
Bad data	DecodingError: the JSON does not match your model. Retrying will not help, so report it.
Cancelled	URLError .cancelled or CancellationError: your own code stopped the work. Show nothing.

> **Write messages for people**
>
> "The operation couldn't be completed (NSURLErrorDomain error -1009)" means nothing to a user. "You appear to be offline. Showing saved results." tells them what happened and what they still have.

CODE · READ-ALONG

Turn errors into messages

One function translates any error into a sentence a person can act on, or `nil` when nothing should be shown at all. Casting with `as?` checks whether the error is a `URLError`, whose `code` says what failed. Keeping this in one place means every screen speaks the same language.

```swift
import Foundation

func userMessage(for error: any Error) -> String? {
    if error is CancellationError { return nil }
    if let urlError = error as? URLError {
        switch urlError.code {
        case .cancelled:
            return nil
        case .notConnectedToInternet, .networkConnectionLost:
            return "You appear to be offline."
        case .timedOut:
            return "The server took too long to answer."
        case .cannotFindHost, .cannotConnectToHost:
            return "The server cannot be reached right now."
        default:
            return "A network problem stopped the request."
        }
    }
    if error is DecodingError {
        return "The server sent data this app cannot read."
    }
    return "Something went wrong. Please try again."
}
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, building the file that holds userMessage(for:).
- WHAT — **Imports URLError and DecodingError.**
- WHERE — Top of the file.
- WHEN — At compile time, before userMessage(for:) can be called.
- WHY — Both error types are Foundation types.

3 `func userMessage(for error: any Error) -> String? {`

- WHO — Your code; every screen's catch block can call it.
- WHAT — **Declares a function that maps any error to an optional message.**
- WHERE — A top-level function, often placed in the networking layer.
- WHEN — After a request fails.
- WHY — Returning nil gives callers a clear signal to show nothing.

4 `if error is CancellationError { return nil }`

- WHO — Swift concurrency produces this error.
- WHAT — **Returns nil when a Swift task was cancelled.**
- WHERE — First check inside the function.
- WHEN — When .task or Task.cancel() stopped the work.
- WHY — Cancellation is the app's own decision, so there is nothing to tell the user.

5 `if let urlError = error as? URLError {`

- WHO — Swift's as? cast, checking whether the error is a URLError.
- WHAT — **Checks whether the error came from URLSession and unwraps it.**
- WHERE — Inside userMessage(for:), after the cancellation check.
- WHEN — For every error that reaches this line; the body runs only for URLError values.
- WHY — URLError carries a precise code for each network problem.

```
6  switch urlError.code {
```

WHO Your code, reading the unwrapped error's code.
WHAT **Branches on the specific network failure.**
WHERE Inside the if let.
WHEN For every URLError, right after the cast succeeds.
WHY A switch keeps all network cases visible in one place.

```
7      case .cancelled:
```

WHO Swift pattern matching on URLError.Code.
WHAT **Matches URLSession's own cancellation code.**
WHERE The first case of the switch inside userMessage(for:).
WHEN When a URLSession task was cancelled, for example because its Swift task was.
WHY URLSession reports cancellation as URLError, not CancellationError, so both checks are needed.

```
8          return nil
```

WHO Your code, leaving the function early.
WHAT **Shows nothing for cancellation.**
WHERE Inside the cancelled case.
WHEN After a cancelled URLSession task reaches this function.
WHY Consistent with the CancellationError check above.

```
9      case .notConnectedToInternet, .networkConnectionLost:
```

WHO Swift pattern matching, listing two codes.
WHAT **Matches both flavors of "no connection".**
WHERE The second case of the switch.
WHEN In airplane mode, or when the signal drops mid-request.
WHY Users experience both the same way, so they share one message.

```
10         return "You appear to be offline."
```

WHO Your code, returning the offline sentence.
WHAT **Returns the offline message.**
WHERE Inside the no-connection case.
WHEN When there is no connection.
WHY It says what happened without blame or jargon.

```
11     case .timedOut:
```

WHO Swift pattern matching on the timedOut code.
WHAT **Matches a request that waited too long for data.**
WHERE The third case of the switch.
WHEN After the request's timeout interval passed.
WHY A timeout suggests a slow network or server, which a retry may fix.

```
12         return "The server took too long to answer."
```

WHO Your code, returning the timeout sentence.
WHAT **Returns the timeout message.**
WHERE Inside the timedOut case.
WHEN After a request exceeded its timeoutInterval.
WHY It hints that trying again later is reasonable.

```
13  case .cannotFindHost, .cannotConnectToHost:
```

WHO Swift pattern matching, listing two host-related codes.
WHAT **Matches DNS failure and refused connections.**
WHERE The fourth case of the switch.
WHEN When the server's name or address does not answer.
WHY These usually mean the server side is down or unreachable.

```
14  return "The server cannot be reached right now."
```

WHO Your code, returning the unreachable sentence.
WHAT **Returns the unreachable message.**
WHERE Inside the cannot-reach-host case.
WHEN When the host is unreachable.
WHY "Right now" signals the problem is probably temporary.

```
15  default:
```

WHO Swift pattern matching, catching every remaining code.
WHAT **Catches every other URLError code.**
WHERE The last case of the switch.
WHEN For rarer problems such as an untrusted certificate.
WHY URLError has dozens of codes; default keeps the function complete.

```
16  return "A network problem stopped the request."
```

WHO Your code, returning the general network sentence.
WHAT **Returns a general network message.**
WHERE Inside the default case, the last branch of the switch.
WHEN For any other network failure.
WHY Still more helpful than a raw error code.

```
19  if error is DecodingError {
```

WHO Swift's is check, testing the error's type.
WHAT **Checks for a JSON mismatch.**
WHERE After the URLError branch.
WHEN When the reply did not match the model.
WHY This is usually a bug or an API change, not a user problem.

```
20  return "The server sent data this app cannot read."
```

WHO Your code, returning the bad-data sentence.
WHAT **Returns the bad-data message.**
WHERE Inside the DecodingError branch.
WHEN After JSONDecoder threw while decoding a reply.
WHY Honest wording; a real app would also log the error for its developers.

```
22  return "Something went wrong. Please try again."
```

WHO Your code, returning the fallback sentence.
WHAT **Returns a fallback for any other error.**
WHERE The last line of the function.
WHEN For errors not matched above, such as your own APIError.
WHY Every error gets some message; none falls through silently.

CODE · RUNS IN THE BROWSER

Retry with exponential backoff

Temporary failures such as timeouts and 503 often succeed a moment later. **Exponential backoff** waits longer after each failure (200, 400, 800 ms), giving a struggling server room to recover. Production code also adds random **jitter** so thousands of phones do not retry in lockstep. Retry only idempotent requests; the next step makes a POST one. Here an actor plays a server that fails twice.

```
enum FetchError: Error { case timedOut }
actor FlakyServer {
    private var calls = 0
    func fetch() throws -> String {
        calls += 1
        if calls < 3 { throw FetchError.timedOut }
        return "200 OK on call \(calls)"
    }
}

func withRetry<T>(attempts: Int,
                  _ operation: () async throws -> T) async throws -> T {
    var delay = 200
    for attempt in 1..<attempts {
        do { return try await operation() } catch {
            print("Attempt \(attempt) failed; retry in \(delay) ms")
            try await Task.sleep(for: .milliseconds(delay))
            delay *= 2
        }
    }
    return try await operation()
}
let server = FlakyServer()
print(try await withRetry(attempts: 4) { try await server.fetch() })
```

OUTPUT

```
Attempt 1 failed; retry in 200 ms
Attempt 2 failed; retry in 400 ms
200 OK on call 3
```

Line by line

1 `enum FetchError: Error { case timedOut }`

- WHO — Your code declares it; FlakyServer throws it.
- WHAT — **Declares a stand-in for a network timeout.**
- WHERE — A top-level enum in this script, the first line.
- WHEN — Thrown by the fake server below.
- WHY — A tiny custom error keeps the example runnable without a network.

2 `actor FlakyServer {`

- WHO — Swift's actor keyword; the runtime serializes access to it.
- WHAT — **Declares an actor: a type whose state only one task can touch at a time.**
- WHERE — A reference type on the heap.
- WHEN — One instance is created near the bottom.
- WHY — It counts calls safely even if many tasks call it concurrently.

```
3   private var calls = 0
```

WHO The FlakyServer actor, which alone can touch it.
WHAT **Counts how many times fetch has been called.**
WHERE Isolated inside the actor.
WHEN Incremented on each call.
WHY Mutable state in an actor is protected from data races automatically.

```
4   func fetch() throws -> String {
```

WHO The FlakyServer actor; withRetry's closure awaits it.
WHAT **Declares the fake request.**
WHERE A method of FlakyServer.
WHEN Each attempt the retry helper makes.
WHY Callers outside the actor must await it, which the retry closure does.

```
5       calls += 1
```

WHO The FlakyServer actor, updating its own counter.
WHAT **Records one more attempt.**
WHERE Inside fetch, the first statement.
WHEN At the start of each call.
WHY The count decides when the server recovers.

```
6       if calls < 3 { throw FetchError.timedOut }
```

WHO The FlakyServer actor, deciding whether to fail.
WHAT **Fails on the first two calls.**
WHERE Inside fetch, after the counter is updated.
WHEN Calls 1 and 2.
WHY This simulates a server that is briefly overloaded.

```
7       return "200 OK on call \(calls)"
```

WHO The FlakyServer actor, returning its success string.
WHAT **Succeeds from the third call on.**
WHERE Inside fetch, the last line.
WHEN On the third call and every call after it.
WHY The message shows how many attempts it took.

```
11  func withRetry<T>(attempts: Int,
```

WHO Your code declares it; the last line of the script calls it.
WHAT **Declares a generic retry helper that works for any result type T.**
WHERE A top-level generic function.
WHEN Whenever an operation should be retried.
WHY Generics let one helper wrap any request, whatever it returns.

```
12      _ operation: () async throws -> T) async throws -> T {
```

WHO Your code, finishing the generic signature.
WHAT **Takes the work as an async throwing closure and returns its result.**
WHERE The rest of the signature.
WHEN Checked at compile time; the body runs on each withRetry call.
WHY Passing the work as a closure means the helper can run it again.

```
13  var delay = 200
```

WHO The withRetry helper, initializing its backoff.
WHAT **Sets the first wait, in milliseconds.**
WHERE A local variable inside withRetry, before the loop.
WHEN Once per withRetry call.
WHY Starting small keeps the first retry quick.

```
14  for attempt in 1..<attempts {
```

WHO The withRetry helper, counting attempts.
WHAT **Loops over every attempt except the last.**
WHERE Inside withRetry, wrapping every attempt but the last.
WHEN Up to attempts minus one times.
WHY The final attempt happens after the loop, so its error can escape to the caller.

```
15  do { return try await operation() } catch {
```

WHO The withRetry helper, invoking the closure it was given.
WHAT **Tries the operation; on success returns at once, on failure falls into catch.**
WHERE Inside the for loop of withRetry.
WHEN Once per loop iteration, up to attempts minus one times.
WHY An early return ends the loop the moment anything succeeds.

```
16  print("Attempt \(attempt) failed; retry in \(delay) ms")
```

WHO The withRetry helper, logging to the console.
WHAT **Logs the failure and the upcoming wait.**
WHERE Inside the catch block of the loop.
WHEN After each failed attempt except the last.
WHY Logging retries makes flaky networks visible during testing.

```
17  try await Task.sleep(for: .milliseconds(delay))
```

WHO Swift concurrency's clock, suspending the current task.
WHAT **Suspends this task without blocking any thread.**
WHERE Inside the catch block, after the log line.
WHEN Between one failed attempt and the next.
WHY Task.sleep throws if the task is cancelled, so a retry loop never outlives its screen.

```
18  delay *= 2
```

WHO The withRetry helper, updating its backoff.
WHAT **Doubles the wait for next time: 200, 400, 800.**
WHERE Inside the catch block, the last statement.
WHEN After each sleep, before the next iteration.
WHY Doubling is the exponential part of exponential backoff.

```
21  return try await operation()
```

WHO The withRetry helper, making the final attempt.
WHAT **Makes the final attempt; if it throws, the error goes to the caller.**
WHERE After the for loop, the last line of withRetry.
WHEN Only if every earlier attempt failed.
WHY Giving up eventually is essential; retrying forever drains battery and hammers the server.

```
23 let server = FlakyServer()
```

WHO Your code, creating the actor instance.
WHAT **Creates the fake server.**
WHERE A global constant in this script, below withRetry.
WHEN At program start, before the retry begins.
WHY One shared instance, so the call count persists across attempts.

```
24 print(try await withRetry(attempts: 4) { try await server.fetch() })
```

WHO Your code, calling withRetry and printing its result.
WHAT **Retries the fetch up to four times and prints the result.**
WHERE Top-level script code, the last line.
WHEN At program start, right after server exists.
WHY The trailing closure is the operation; await is needed because fetch is isolated to the actor.

> **Try it**
>
> 1. Change calls < 3 to calls < 5. What happens with attempts: 4?
> 2. How would you add jitter so each phone waits a random time between 0 and delay?

CODE · READ-ALONG

Make a POST safe to retry

Retrying a POST is dangerous only because the server cannot tell a retry from a new order. An **idempotency key** fixes that: the app creates a unique id once, stores it with the pending order, and sends it in an `Idempotency-Key` header on every attempt. A server that supports the key answers a repeat with the first result instead of creating a second order.

```swift
import Foundation

struct PendingOrder: Codable {
    let items: [String]
    var idempotencyKey = UUID().uuidString
}

func createOrderRequest(_ order: PendingOrder) throws -> URLRequest {
    let url = URL(string: "https://api.example.com/orders")!
    var request = URLRequest(url: url)
    request.httpMethod = "POST"
    request.setValue("application/json",
                     forHTTPHeaderField: "Content-Type")
    request.setValue(order.idempotencyKey,
                     forHTTPHeaderField: "Idempotency-Key")
    request.httpBody = try JSONEncoder().encode(order)
    return request
}
```

Line by line

```
1  import Foundation
```

WHO The Swift compiler, building the file that holds PendingOrder and createOrderRequest.
WHAT **Imports URLRequest, JSONEncoder and UUID.**
WHERE Top of the orders file.
WHEN At compile time, before an order can be built.
WHY UUID, like the request and encoder types, comes from Foundation.

```
3  struct PendingOrder: Codable {
```

WHO Your code declares it; the checkout screen creates one per order.
WHAT **Declares the order that is waiting to be sent, including its retry key.**
WHERE A top-level struct in the orders file.
WHEN A value is created when the user taps Place Order, before any request.
WHY Codable both ways lets the app save the pending order to disk and reload it after a crash with the same key.

```
4      let items: [String]
```

WHO The PendingOrder struct; JSONEncoder writes it into the body.
WHAT **The product ids the user is buying.**
WHERE The first stored property inside PendingOrder.
WHEN Set when the order is created.
WHY Real orders carry more, but one array is enough to show the pattern.

```
5      var idempotencyKey = UUID().uuidString
```

WHO Foundation's UUID generator, run once by the memberwise initializer.
WHAT **Creates a random 36-character id and stores it with the order.**
WHERE The second stored property inside PendingOrder, with a default value.
WHEN Once, when the order value is created; never again for that order.
WHY The key must be created once and reused for every retry; generating it inside the request function would give each retry a new key and defeat the purpose.

```
8  func createOrderRequest(_ order: PendingOrder) throws -> URLRequest {
```

WHO Your code declares it; the retry helper calls it on every attempt.
WHAT **Builds the POST request for a pending order without sending it.**
WHERE A top-level function in the orders file, below the struct.
WHEN On every attempt, including retries after a timeout.
WHY Building from the stored order means each attempt carries the identical key and body.

```
9      let url = URL(string: "https://api.example.com/orders")!
```

WHO Foundation's URL initializer, parsing the literal.
WHAT **Creates the orders collection URL.**
WHERE A local constant inside createOrderRequest, the first statement.
WHEN Each attempt, before the request exists.
WHY A literal you control can be force-unwrapped; it would fail on the first test run if it were wrong.

```
10     var request = URLRequest(url: url)
```

WHO Foundation's URLRequest initializer, called by your code.
WHAT **Creates a request with default settings: GET, no headers, no body.**
WHERE A local variable inside createOrderRequest, after url.
WHEN Each attempt, right after the URL is built.
WHY It is var because the next lines change the method, headers and body.

```
11  request.httpMethod = "POST"
```

WHO Your code, setting the request's method.
WHAT **Changes the method from GET to POST.**
WHERE The httpMethod property of the request.
WHEN Each attempt, as the request is configured.
WHY POST is the method that is unsafe to retry, which is exactly why the key is needed.

```
12  request.setValue("application/json",
```

WHO The URLRequest value, storing a header through setValue.
WHAT **Starts setting a header whose value is application/json.**
WHERE On the request, inside createOrderRequest.
WHEN Each attempt, after the method is set.
WHY The server must know the body is JSON before it can parse the order.

```
13  forHTTPHeaderField: "Content-Type")
```

WHO The URLRequest value, keyed by the Content-Type name.
WHAT **Names the header: Content-Type.**
WHERE The second line of the same setValue call.
WHEN Each attempt, completing the first header.
WHY setValue replaces any earlier value, so repeated calls never create duplicate headers.

```
14  request.setValue(order.idempotencyKey,
```

WHO Your code, reading the stored key from the order.
WHAT **Starts setting a header whose value is the order's unique key.**
WHERE On the request, after the Content-Type header.
WHEN Each attempt, with the same key every time.
WHY This header is what lets the server recognise a retry of an order it already created.

```
15  forHTTPHeaderField: "Idempotency-Key")
```

WHO The URLRequest value, keyed by the Idempotency-Key name.
WHAT **Names the header: Idempotency-Key.**
WHERE The second line of the setValue call.
WHEN Each attempt, completing the second header.
WHY The name is a widely used convention, so many payment and order APIs recognise it unchanged.

```
16  request.httpBody = try JSONEncoder().encode(order)
```

WHO JSONEncoder converts the order; the request stores the bytes.
WHAT **Encodes the pending order as JSON and sets it as the body.**
WHERE The httpBody property of the request.
WHEN Each attempt; try passes an encoding failure to the caller.
WHY The key travels in the body too, so the server can store it with the order it creates.

```
17  return request
```

WHO Your createOrderRequest function, handing the value to the caller.
WHAT **Returns the finished request for URLSession to send.**
WHERE The last line of createOrderRequest.
WHEN Each attempt, after every part is set.
WHY URLRequest is a value type, so the retry helper can send it as many times as needed.

Try it

1. Move the UUID call into createOrderRequest. Why would retries stop being safe?
2. Which line would you change so a pending order survives the app being killed before its first attempt?

INTERACTIVE

Retry lab

Two hundred simulated phones hit a server the moment it recovers from an outage. Compare no retries, immediate retries, exponential backoff and backoff with jitter. Watch the load against the server's capacity, how many users succeed and how many requests were wasted. Then trigger real failures from this page: a request that times out and a server that answers 503.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Cancel work nobody needs

Search-as-you-type would send a request per keystroke unless stale ones are cancelled. `.task(id:)` restarts its task whenever the id changes and cancels the previous one. A short sleep at the start acts as a **debounce**: if another key arrives within 300 ms, the sleeping task is cancelled before it ever touches the network.

```swift
import SwiftUI

struct SearchView: View {
    @State private var query = ""
    @State private var results: [String] = []
    let client = SearchClient()

    var body: some View {
        List(results, id: \.self) { Text($0) }
            .searchable(text: $query)
            .task(id: query) {
                do {
                    try await Task.sleep(for: .milliseconds(300))
                    results = try await client.search(query)
                } catch {
                    if Task.isCancelled { return }
                    results = []
                }
            }
    }
}
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, building the file that holds SearchView.
WHAT	**Imports SwiftUI, which provides View, List, .searchable and .task.**
WHERE	Top of the file.
WHEN	At compile time, before SearchView can be built.
WHY	The view, List and .task all come from SwiftUI.

```
3  struct SearchView: View {
```

WHO	Your code declares it; SwiftUI renders it.
WHAT	**Declares a searchable list screen.**
WHERE	A top-level view struct in the search feature.
WHEN	Rendered while the screen is visible.
WHY	All the cancellation logic fits inside one modifier.

```
4  @State private var query = ""
```

WHO	SwiftUI stores it outside the struct; the search field writes it.
WHAT	**Holds the text in the search field.**
WHERE	View state owned by SwiftUI.
WHEN	Updated on every keystroke.
WHY	Its value doubles as the task id below.

```
5  @State private var results: [String] = []
```

WHO	SwiftUI stores it; the task closure writes it.
WHAT	**Holds the latest search results.**
WHERE	View state owned by SearchView, next to query.
WHEN	Replaced after each completed search.
WHY	An empty array is a sensible starting value.

```
6  let client = SearchClient()
```

WHO	SearchView owns it; the task closure calls it.
WHAT	**Holds a client that performs the network search.**
WHERE	A stored constant of the view.
WHEN	Created with the view.
WHY	The client is a Sendable struct, so it is cheap to keep in a view.

```
8  var body: some View {
```

WHO	SwiftUI calls it; your code describes the list inside.
WHAT	**Describes the screen: a list, a search field and the task that fills the list.**
WHERE	The computed body property of SearchView.
WHEN	Each time query or results change.
WHY	No network calls here; they live in .task.

```
9  List(results, id: \.self) { Text($0) }
```

WHO	SwiftUI, building one row per result.
WHAT	**Shows one row per result string.**
WHERE	The root view of body.
WHEN	Each time body runs, after query or results change.
WHY	id: \.self identifies rows by their text.

```
10  .searchable(text: $query)
```

WHO SwiftUI, adding the system search field.
WHAT **Adds a system search field bound to query.**
WHERE A modifier on the List.
WHEN Writes query as the user types.
WHY The binding is how keystrokes reach state.

```
11  .task(id: query) {
```

WHO SwiftUI manages the task.
WHAT **Runs the closure now, and again whenever query changes, cancelling the previous run.**
WHERE A modifier on the List.
WHEN On appear and after every keystroke.
WHY Automatic cancellation is exactly what search-as-you-type needs.

```
12  do {
```

WHO Swift's error-handling syntax, inside the task closure.
WHAT **Starts the work that may throw or be cancelled.**
WHERE Inside the task closure.
WHEN Each time the task runs, on appear and after every keystroke.
WHY Both sleeping and searching can throw.

```
13  try await Task.sleep(for: .milliseconds(300))
```

WHO Swift concurrency's clock, suspending this task.
WHAT **Waits 300 ms before searching.**
WHERE Inside the do block, the first statement of each run.
WHEN At the start of each run.
WHY If another key arrives, this sleep throws CancellationError and the old search never starts.

```
14  results = try await client.search(query)
```

WHO Your client, then SwiftUI state.
WHAT **Searches with the current text and stores the results.**
WHERE Inside do; the update happens on the main actor.
WHEN Only if the user paused typing for 300 ms.
WHY A cancelled task's URLSession request is cancelled too, saving data and battery.

```
15  } catch {
```

WHO Swift's catch clause, paired with the do block.
WHAT **Catches cancellation and real failures alike.**
WHERE Inside the task closure.
WHEN When the sleep or the search throws.
WHY Both paths need different handling, decided on the next line.

```
16  if Task.isCancelled { return }
```

WHO Swift concurrency, reporting the task's cancellation flag.
WHAT **Leaves quietly when this run was cancelled.**
WHERE Inside the catch block, before results is touched.
WHEN After the user typed again or left the screen.
WHY A newer run is already in charge; clearing results now would flash an empty list.

```
17  results = []
```

WHO	SearchView, writing its own state.
WHAT	**Clears the list after a genuine failure.**
WHERE	Inside the catch block, after the cancellation check.
WHEN	Only for real errors.
WHY	A real app would also show a message from userMessage(for:).

CONCEPT

HTTPS, certificates and App Transport Security

TLS gives every HTTPS connection two guarantees. **Encryption**: nobody on the coffee-shop Wi-Fi can read or alter the traffic. **Authentication**: the server proves its identity with a **certificate**, a signed document that binds its domain name to a public key. The signature comes from a **certificate authority** (CA) that iOS already trusts. If the name does not match, the certificate has expired or the chain does not lead to a trusted CA, URLSession refuses to connect.

App Transport Security (ATS)	Since iOS 9, apps load only HTTPS URLs with modern TLS by default. A plain http:// request fails.
Exceptions	Info.plist keys under NSAppTransportSecurity can relax ATS for specific domains. Some require a justification during App Review; avoid them.
Certificate pinning	Accepting only a specific certificate or key for your own server, used by high-risk apps such as banking. It must be updated whenever the certificate changes.

CONCEPT

Never ship a secret in the app

Everything inside your app is on every user's phone, and an app binary can be copied and inspected. A string constant such as an API key survives compilation as readable bytes. Hiding it only slows an attacker down, because the app must eventually send the key over the network.

Keep keys on a server	Your app calls your own backend, which adds the secret key and calls the third-party API.
Per-user tokens	At sign-in your server issues a token for that user only, which can expire and be revoked.
Tokens go in the Keychain	The Keychain is the encrypted store iOS provides for small secrets. UserDefaults is a settings file that was never designed for secrets; never put tokens there.

Interactive figure in the HTML version: networking secret leak.

CODE · READ-ALONG

Read a token and authorize a request

After sign-in, your app saves the user's token with `SecItemAdd`. Later, each request that needs it reads the token back and adds an `Authorization: Bearer` header. The Keychain has a C-style API: you describe the item you want with a dictionary of `kSec` keys and pass it to a function.

```swift
import Foundation
import Security

func savedToken() -> String? {
    let query: [String: Any] = [
        kSecClass as String: kSecClassGenericPassword,
        kSecAttrAccount as String: "session-token",
        kSecReturnData as String: true
    ]
    var result: AnyObject?
    let status = SecItemCopyMatching(query as CFDictionary, &result)
    guard status == errSecSuccess, let data = result as? Data else {
        return nil
    }
    return String(decoding: data, as: UTF8.self)
}
func authorized(_ request: URLRequest) -> URLRequest {
    var request = request
    if let token = savedToken() {
        request.setValue("Bearer \(token)",
                         forHTTPHeaderField: "Authorization")
    }
    return request
}
```

Line by line

1 `import Foundation`

WHO	The Swift compiler, building the file that holds savedToken and authorized.
WHAT	**Imports URLRequest and Data.**
WHERE	Top of the file.
WHEN	At compile time, before either function can run.
WHY	The second function builds on URLRequest.

2 `import Security`

WHO	The Swift compiler, loading the Security framework's interface.
WHAT	**Imports the Keychain functions and kSec constants.**
WHERE	Top of the file.
WHEN	At compile time, before SecItemCopyMatching can be referenced.
WHY	The Keychain lives in the Security framework.

4 `func savedToken() -> String? {`

WHO	Your code declares it; authorized(_:) calls it.
WHAT	**Declares a function that returns the stored token, or nil if there is none.**
WHERE	A top-level function, typically inside a TokenStore type.
WHEN	Before every authorized request.
WHY	nil means "not signed in", which the caller can handle.

```
5  let query: [String: Any] = [
```

WHO Your code, building the Keychain query.
WHAT **Starts the dictionary that describes the item to find.**
WHERE A local constant inside savedToken, the first statement.
WHEN Each time savedToken runs, before the Keychain is searched.
WHY The Keychain API takes a dictionary of attributes rather than parameters.

```
6      kSecClass as String: kSecClassGenericPassword,
```

WHO The Security framework defines both constants; your code pairs them.
WHAT **Says the item is a generic password, the class used for tokens.**
WHERE First entry of the query.
WHEN Each call, as the query dictionary is built.
WHY Items are grouped by class; this one fits any app-defined secret.

```
7      kSecAttrAccount as String: "session-token",
```

WHO Your code, naming the item it wants.
WHAT **Names the item by account, the same name used when it was saved.**
WHERE The second entry of the query dictionary.
WHEN Each call, while the dictionary literal is built.
WHY The account string is how you find your item again.

```
8      kSecReturnData as String: true
```

WHO The Security framework defines the key; your code sets it to true.
WHAT **Asks for the stored bytes to be returned.**
WHERE The third and last entry of the query dictionary.
WHEN Each call, as the dictionary literal is completed.
WHY Without it, the call only reports whether the item exists.

```
10 var result: AnyObject?
```

WHO Your code, reserving space for the Keychain's answer.
WHAT **Prepares a slot for the Keychain to write the result into.**
WHERE A local variable inside savedToken, after the query.
WHEN Each call, before SecItemCopyMatching runs.
WHY The C-style API returns data through a pointer argument.

```
11 let status = SecItemCopyMatching(query as CFDictionary, &result)
```

WHO The Keychain service in iOS, through the Security framework.
WHAT **Searches the Keychain and fills result if it finds the item.**
WHERE Runs in the Security framework on the device.
WHEN Each call, when the search runs on the device.
WHY The & passes result by reference; the returned status says whether it worked.

```
12 guard status == errSecSuccess, let data = result as? Data else {
```

WHO Your code, checking the status and casting the result.
WHAT **Continues only if the item was found and came back as Data.**
WHERE Inside savedToken, right after the Keychain search.
WHEN After SecItemCopyMatching returns, on every call.
WHY errSecItemNotFound simply means nobody has signed in yet.

```
13  return nil
```

WHO Your code, leaving savedToken without a token.
WHAT **Reports that no token is available.**
WHERE The guard's else branch.
WHEN When there is no item.
WHY The caller can then show the sign-in screen.

```
15  return String(decoding: data, as: UTF8.self)
```

WHO Swift's String initializer, decoding the UTF-8 bytes.
WHAT **Converts the stored bytes back to text.**
WHERE The last line of savedToken.
WHEN When a token was found.
WHY Tokens are saved as UTF-8 Data, so they decode the same way.

```
17  func authorized(_ request: URLRequest) -> URLRequest {
```

WHO Your code; the API client calls it before sending.
WHAT **Returns a copy of a request with the token attached.**
WHERE A top-level function below savedToken, in the same networking file.
WHEN Before each request to your own server.
WHY Centralizing this means no request accidentally goes out without the header.

```
18  var request = request
```

WHO Your code, copying the value parameter.
WHAT **Makes a mutable copy of the parameter.**
WHERE A local variable that shadows the parameter.
WHEN Each time authorized(_:) runs, before the header is added.
WHY Parameters are constants; URLRequest is a value type, so the copy is independent.

```
19  if let token = savedToken() {
```

WHO Your code, calling savedToken and unwrapping its result.
WHAT **Reads the token and continues only if there is one.**
WHERE Inside authorized(_:), after the copy.
WHEN Each call; the body runs only when a token exists.
WHY Requests to public endpoints still work when nobody is signed in.

```
20  request.setValue("Bearer \(token)",
```

WHO The URLRequest value, storing the header through setValue.
WHAT **Starts setting the header value: Bearer followed by the token.**
WHERE Inside the if let.
WHEN When a token exists.
WHY Bearer is the standard scheme: whoever holds the token is let in.

```
21  forHTTPHeaderField: "Authorization")
```

WHO The URLRequest value, keyed by this header name.
WHAT **Names the header: Authorization.**
WHERE Second line of the call.
WHEN When a token exists.
WHY The token travels only over HTTPS, so it is encrypted in transit.

```
23  return request
```

WHO	Your authorized(_:) function, handing back the copy.
WHAT	**Hands back the possibly authorized request.**
WHERE	The last line of authorized(_:).
WHEN	Each call, whether or not a token was added.
WHY	Returning a new value keeps the function free of side effects and easy to test.

CHECK YOURSELF

Check your understanding

1. Which of these errors should produce no message at all?
   A. URLError(.timedOut) on a slow network
   B. URLError(.notConnectedToInternet) in airplane mode
   C. URLError(.cancelled) after the user left
   D. DecodingError.keyNotFound from a changed API

2. A retry helper retries every failed request three times. Which request is dangerous to retry blindly?
   A. GET /forecast
   B. POST /payments
   C. PUT /settings with the complete settings object
   D. DELETE /drafts/9

3. Why add random jitter to backoff delays?
   A. To make the average retry arrive sooner
   B. Because Task.sleep cannot wait an exact number of ms
   C. To hide the retry pattern from anyone watching traffic
   D. So clients that failed together do not all retry at once

4. A teammate stores a paid API key as a constant in the app. What is the right fix?
   A. Keep the key on your own server and call the API from there
   B. Move it from Swift source into Info.plist, which is not compiled
   C. Base64-encode it in the source so it is not a readable string
   D. Write it to UserDefaults on first launch and delete it from source

5. With default settings, an iOS 26 app requests `http://api.example.com/data` (no s). What happens?
   A. It works, and Xcode logs a warning at build time
   B. App Transport Security blocks it and the request fails
   C. iOS quietly rewrites the URL to https before sending
   D. It works on Wi-Fi but App Transport Security blocks it on cellular

Answers: Errors, Retries and Security

1. **C. URLError(.cancelled) after the user left** Cancellation is your own app stopping work nobody needs anymore. Telling the user about it would be noise. The others are real problems that deserve a clear message.
2. **B. POST /payments** If the first POST reached the server but the reply was lost, retrying charges the customer twice. GET, PUT and DELETE are idempotent, so repeating them leaves the same end state.
3. **D. So clients that failed together do not all retry at once** Without jitter, every phone that failed at the same moment waits exactly 200 ms, then 400 ms, and they all return together, overloading the server again. Randomness spreads them out. In the retry lab, compare the spikes.
4. **A. Keep the key on your own server and call the API from there** Anything shipped in the app, encoded or not, can be extracted, and anything the app sends can be read with a proxy. Info.plist is plain text inside the bundle, Base64 is an encoding rather than encryption, and UserDefaults is an unencrypted file on the device.
5. **B. App Transport Security blocks it and the request fails** ATS rejects plain HTTP loads unless Info.plist contains an exception. It does not rewrite the URL; the request fails with an error, which is why every API you call should use https.

Live Data Beyond Request and Response

CONCEPT

When the server has news

HTTP is client-initiated: the server can only answer questions. But scores change, messages arrive and delivery drivers move. There are several ways to get news from a server to an app, each with different costs.

Polling	Ask every few seconds: "anything new?" Simple, but most answers are "no", and every request wakes the radio.
Long polling	Ask, and the server holds the request open until there is news or a timeout; then ask again.
Streaming (Server-Sent Events)	One long HTTP response that the server keeps writing lines into. News flows one way: server to client.
WebSocket	An HTTP connection upgraded into a two-way channel. Either side can send a message at any time, with little overhead per message.
Push notifications	Apple's push service delivers a message to the device even when your app is not running.

> **Battery is part of the design**
>
> A cellular radio stays in a high-power state for several seconds after each transfer. Many small requests spread over a minute can cost more energy than one larger request.

INTERACTIVE

Live score lab

A match produces goals at unpredictable moments. Deliver them to a phone by polling (pick the interval), over a WebSocket or with push notifications. Compare how late each update arrives, how many requests carry no news, how long the radio stays awake and how much work the server does for a thousand fans. Then send the app to the background and see which approach still delivers.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

WebSockets with URLSessionWebSocketTask

A WebSocket starts as an HTTP request and is then upgraded into a two-way channel that stays open. `URLSessionWebSocketTask` exposes it with async methods: `send` a message, and `await receive()` for the next one. A loop keeps receiving until the surrounding task is cancelled. Each message is either text or binary data.

```swift
import Foundation

func followGame(id: Int) async throws {
    let url = URL(string: "wss://scores.example.com/live")!
    let socket = URLSession.shared.webSocketTask(with: url)
    socket.resume()
    defer { socket.cancel(with: .goingAway, reason: nil) }

    try await socket.send(.string("subscribe \(id)"))

    while !Task.isCancelled {
        let message = try await socket.receive()
        switch message {
        case .string(let text):
            print("Score update:", text)
        case .data(let bytes):
            print("Binary frame of \(bytes.count) bytes")
        @unknown default:
            break
        }
    }
}
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, building the file that holds followGame(id:).
- **WHAT** **Imports URLSession and its WebSocket task type.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before followGame(id:) can run.
- **WHY** WebSocket support is built into URLSession; no extra library is needed.

3 `func followGame(id: Int) async throws {`

- **WHO** Your code; a model awaits it inside a task.
- **WHAT** **Declares a function that listens for live score updates.**
- **WHERE** A top-level function in the live-scores feature.
- **WHEN** For as long as the game screen is open.
- **WHY** It runs until cancelled or until the connection fails, so it is async throws.

4 `let url = URL(string: "wss://scores.example.com/live")!`

- **WHO** Foundation's URL initializer, parsing the wss literal.
- **WHAT** **Creates the WebSocket address. wss means WebSocket over TLS.**
- **WHERE** A local constant inside followGame(id:), the first statement.
- **WHEN** Once per call, before the socket exists.
- **WHY** wss is to ws what https is to http; App Transport Security expects the secure form.

5 `let socket = URLSession.shared.webSocketTask(with: url)`

- **WHO** URLSession's shared instance, creating a suspended task.
- **WHAT** **Creates a WebSocket task for that address, not yet connected.**
- **WHERE** A local constant; the task object lives on the heap.
- **WHEN** Once per call, right after the URL is built.
- **WHY** Like other URLSession tasks, it starts suspended so you can configure it first.

```
6  socket.resume()
```

WHO — URLSession, which begins the upgrade handshake on its own threads.
WHAT — **Starts the connection: an HTTP request asking to upgrade to WebSocket.**
WHERE — Inside followGame(id:), directly after the task is created.
WHEN — Immediately after creation, before any message is sent.
WHY — Tasks do nothing until resumed.

```
7  defer { socket.cancel(with: .goingAway, reason: nil) }
```

WHO — Swift's defer statement, run by the function's exit path.
WHAT — **Schedules a clean close of the socket for whenever the function exits.**
WHERE — Inside followGame; runs on every exit path.
WHEN — At exit: normal return, thrown error or cancellation.
WHY — Leaving sockets open wastes server resources and battery.

```
9  try await socket.send(.string("subscribe \(id)"))
```

WHO — URLSession sends one WebSocket message.
WHAT — **Tells the server which game to follow.**
WHERE — Inside followGame(id:), after the socket is resumed and before the loop.
WHEN — Once, right after connecting.
WHY — The message format is whatever your server defines; text is easiest to debug.

```
11  while !Task.isCancelled {
```

WHO — Swift concurrency, reporting the task's cancellation flag.
WHAT — **Loops until the task running this function is cancelled.**
WHERE — Inside followGame(id:), wrapping the receive loop.
WHEN — For the life of the screen.
WHY — When the view disappears, .task cancels, the loop ends and defer closes the socket.

```
12  let message = try await socket.receive()
```

WHO — URLSession, delivering the next WebSocket frame.
WHAT — **Suspends until the server sends the next message.**
WHERE — Inside the while loop, the first statement of each iteration.
WHEN — Each time a message arrives.
WHY — No polling: the function sleeps at no cost until there is news.

```
13  switch message {
```

WHO — Your code, inspecting the received message.
WHAT — **Branches on the kind of message.**
WHERE — Inside the while loop, after receive returns.
WHEN — Once per message, as soon as it arrives.
WHY — A WebSocket message is either text or binary.

```
14  case .string(let text):
```

WHO — Swift pattern matching on URLSessionWebSocketTask.Message.
WHAT — **Matches a text message and binds its content.**
WHERE — The first case of the switch over message.
WHEN — For each text frame the server sends.
WHY — JSON updates usually arrive as text.

```
15 print("Score update:", text)
```

WHO Your code, printing the text payload.
WHAT **Handles the update; a real app would decode JSON and update the model.**
WHERE Inside the text case.
WHEN For each text message.
WHY Printing keeps the example focused on the socket itself.

```
16 case .data(let bytes):
```

WHO Swift pattern matching, binding the binary payload.
WHAT **Matches a binary message.**
WHERE The second case of the switch.
WHEN For each binary frame the server sends.
WHY Binary suits compact formats or images.

```
17 print("Binary frame of \(bytes.count) bytes")
```

WHO Your code, reading the Data's count.
WHAT **Reports the binary message's size.**
WHERE Inside the data case.
WHEN For each binary message.
WHY Shows the payload arrives as Data.

```
18 @unknown default:
```

WHO The Swift compiler, which asks for this case on a non-frozen enum.
WHAT **Handles message kinds Apple might add in the future.**
WHERE The last case of the switch over message.
WHEN Compile time, plus at run time if a new kind appears.
WHY The enum may grow, so the compiler asks for a future-proof case.

```
19 break
```

WHO Your code, choosing to do nothing.
WHAT **Leaves the switch without handling an unknown message kind.**
WHERE Inside the @unknown default case.
WHEN Only if a future SDK adds a new message kind.
WHY Ignoring is safer than crashing on something new.

CODE · READ-ALONG

Streaming a response line by line

Some servers keep one HTTP response open and write events into it as they happen. Server-Sent Events use lines that start with `data:`. `bytes(for:)` hands you the body as it arrives, and `.lines` turns it into an asynchronous sequence of lines that you read with `for try await`. Live logs and progress feeds work this way.

```swift
import Foundation

func streamEvents() async throws {
    let url = URL(string: "https://api.example.com/v1/events")!
    var request = URLRequest(url: url)
    request.setValue("text/event-stream",
                     forHTTPHeaderField: "Accept")
    let (bytes, response) =
        try await URLSession.shared.bytes(for: request)
    guard (response as? HTTPURLResponse)?.statusCode == 200 else {
        throw URLError(.badServerResponse)
    }
    for try await line in bytes.lines {
        guard line.hasPrefix("data: ") else { continue }
        let payload = line.dropFirst(6)
        print("Event:", payload)
    }
}
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, building the file that holds streamEvents.
- WHAT — **Imports URLSession and URLRequest.**
- WHERE — Top of the file.
- WHEN — At compile time, before streamEvents can run.
- WHY — Streaming uses the same session as ordinary requests.

3 `func streamEvents() async throws {`

- WHO — Your code declares it; a .task modifier would await it.
- WHAT — **Declares a function that prints events for as long as the server sends them.**
- WHERE — A top-level function in the events feature.
- WHEN — Runs until the stream ends, fails or is cancelled.
- WHY — A long-running async function fits naturally into a .task modifier.

4 `let url = URL(string: "https://api.example.com/v1/events")!`

- WHO — Foundation's URL initializer, parsing the literal.
- WHAT — **Creates the stream's address.**
- WHERE — A local constant inside streamEvents, the first statement.
- WHEN — Once per call, before the request is built.
- WHY — It is an ordinary https URL; no special scheme is needed.

5 `var request = URLRequest(url: url)`

- WHO — Foundation's URLRequest initializer, called by your code.
- WHAT — **Creates a request so a header can be added.**
- WHERE — A local variable inside streamEvents, after url.
- WHEN — Once per call, before the Accept header is set.
- WHY — data(from:) cannot set headers; a URLRequest can.

```
6  request.setValue("text/event-stream",
```

WHO The URLRequest value, storing the header through setValue.
WHAT **Starts setting the Accept header value for Server-Sent Events.**
WHERE On the request inside streamEvents, before it is sent.
WHEN Before sending, as part of building the request.
WHY Servers often serve both normal JSON and a stream from one URL; Accept chooses the stream.

```
7  forHTTPHeaderField: "Accept")
```

WHO The URLRequest value, keyed by the Accept header name.
WHAT **Names the header being set: Accept.**
WHERE Second line of the call.
WHEN Before sending, completing the setValue call.
WHY Same setValue call as for any other header.

```
8  let (bytes, response) =
```

WHO Your code, declaring the stream and the response.
WHAT **Declares constants for the byte stream and the response.**
WHERE Local constants inside streamEvents, filled by the next line.
WHEN Assigned as soon as the response headers arrive.
WHY Unlike data(for:), this returns before the body is complete.

```
9  try await URLSession.shared.bytes(for: request)
```

WHO URLSession's shared instance, on its background threads.
WHAT **Sends the request and returns once headers arrive, with the body still flowing.**
WHERE Called inside streamEvents; the body keeps flowing on URLSession's threads.
WHEN Resumes after the headers, not after the whole body.
WHY Waiting for an endless body would never finish; bytes lets you start reading immediately.

```
10 guard (response as? HTTPURLResponse)?.statusCode == 200 else {
```

WHO Your code, casting the response and comparing the status.
WHAT **Checks the status before reading any events.**
WHERE Inside streamEvents, right after the headers arrive.
WHEN Right after the headers arrive.
WHY Optional chaining makes the comparison false if the cast fails.

```
11 throw URLError(.badServerResponse)
```

WHO Your code, throwing a Foundation error value.
WHAT **Fails when the stream could not be opened.**
WHERE The guard's else branch.
WHEN Only on a bad status.
WHY Same convention as the ordinary API client.

```
13 for try await line in bytes.lines {
```

WHO Swift concurrency, iterating an asynchronous sequence.
WHAT **Waits for each complete line of text as it arrives.**
WHERE Inside streamEvents, after the status check.
WHEN Each time the server finishes writing a line.
WHY try covers network errors in the middle of the stream; the loop ends when the server closes it.

```
14 guard line.hasPrefix("data: ") else { continue }
```

WHO Your code, testing the line's prefix.
WHAT **Skips lines that are not event data, such as comments or event names.**
WHERE Inside the for loop, the first statement.
WHEN For every line the server sends, as it arrives.
WHY Server-Sent Events put the payload after data: and use other prefixes for metadata.

```
15 let payload = line.dropFirst(6)
```

WHO Swift's String slicing, through dropFirst.
WHAT **Removes the six characters of "data: ".**
WHERE Inside the for loop, after the prefix check.
WHEN For each data line.
WHY dropFirst returns a Substring, which avoids copying the text.

```
16 print("Event:", payload)
```

WHO Your code, printing the Substring.
WHAT **Handles the event; real code would decode JSON and update the model.**
WHERE Inside the for loop, the last statement.
WHEN For each event, the moment it arrives.
WHY The interface can update event by event instead of after a long wait.

CONCEPT

Push notifications at a glance

Your app cannot keep a connection open in the background; iOS suspends it to save battery. For news that must reach someone who is not using your app, use the **Apple Push Notification service** (APNs).

Permission	The app asks the user through UNUserNotificationCenter before it shows alerts. Many people decline, so the app must still work without push.
Device token	The app registers with APNs and receives a token that identifies this app on this device, then sends the token to your server.
Your server sends	Your backend sends the message and the token to APNs, authenticated with a key from your Apple developer account. APNs delivers it.
Not guaranteed	Delivery is best effort, and silent background pushes are throttled. Treat a push as a hint to fetch fresh data, not as the data itself.

> **Live Activities**
>
> A Live Activity on the Lock Screen, built with ActivityKit, can also be updated by push. It is the system's answer for live scores, deliveries and rides.

CODE · READ-ALONG

Register for push and receive the token

Three pieces make push work on the device. The app asks permission through `UNUserNotificationCenter`, registers with APNs, and receives a **device token** in an app delegate method, which it sends to your server. A SwiftUI app attaches the delegate with `@UIApplicationDelegateAdaptor(AppDelegate.self)` in its App struct. Your server later hands that token to APNs with each message.

```swift
import SwiftUI
import UserNotifications

final class AppDelegate: NSObject, UIApplicationDelegate {
    func application(
        _ application: UIApplication,
        didRegisterForRemoteNotificationsWithDeviceToken token: Data
    ) {
        let hex = token.map { String(format: "%02x", $0) }.joined()
        Task { try? await sendTokenToServer(hex) }
    }
}

@MainActor
func requestPushPermission() async throws {
    let center = UNUserNotificationCenter.current()
    let granted = try await center.requestAuthorization(
        options: [.alert, .sound, .badge])
    guard granted else { return }
    UIApplication.shared.registerForRemoteNotifications()
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, building the file that holds AppDelegate.
- **WHAT** **Imports SwiftUI, which on iOS also brings in UIKit's UIApplication and UIApplicationDelegate.**
- **WHERE** Top of the app delegate file.
- **WHEN** At compile time, before the delegate type can be declared.
- **WHY** UIApplication and its delegate protocol are UIKit types; SwiftUI re-exports UIKit on iOS, so no separate import is needed.

2 `import UserNotifications`

- **WHO** The Swift compiler, loading the UserNotifications framework.
- **WHAT** **Makes UNUserNotificationCenter and its authorization options available.**
- **WHERE** Top of the same file, after SwiftUI.
- **WHEN** At compile time, before requestPushPermission can reference the center.
- **WHY** Permission for alerts lives in its own framework, shared by local and remote notifications.

4 `final class AppDelegate: NSObject, UIApplicationDelegate {`

- **WHO** Your code declares it; iOS calls its methods through the adaptor.
- **WHAT** **Declares the delegate object that receives app-level callbacks, including the device token.**
- **WHERE** A top-level class in the app target; the App struct names it in @UIApplicationDelegateAdaptor.
- **WHEN** Created once at launch, before the first scene appears.
- **WHY** The token arrives only through a UIApplicationDelegate method, so even a SwiftUI app needs this one class; NSObject is required by the Objective-C protocol.

5 `func application(`

- **WHO** iOS calls it; the compiler matches the name against the protocol.
- **WHAT** **Begins the delegate method that delivers the device token.**
- **WHERE** Inside AppDelegate; the parameters continue on the next two lines.
- **WHEN** Called once per registration, after APNs has answered.
- **WHY** The full name is long, so splitting it across lines keeps each under the width limit.

6 `_ application: UIApplication,`

- **WHO** iOS supplies the application object.
- **WHAT** **The first parameter: the running app, unlabeled at the call site.**
- **WHERE** The first parameter of the delegate method.
- **WHEN** Filled in when iOS calls the method.
- **WHY** The protocol fixes this signature; the parameter is unused here but must be present.

7 `didRegisterForRemoteNotificationsWithDeviceToken token: Data`

- **WHO** iOS supplies the token it received from APNs.
- **WHAT** **The second parameter: the device token as raw bytes.**
- **WHERE** The last parameter of the delegate method.
- **WHEN** Filled in once APNs has issued a token for this app on this device.
- **WHY** The external label is the protocol's; the short internal name token keeps the body readable.

8 `) {`

- **WHO** The Swift compiler, closing the parameter list.
- **WHAT** **Ends the signature and opens the method body.**
- **WHERE** Between the parameters and the body of the delegate method.
- **WHEN** Parsed at compile time; nothing runs for punctuation.
- **WHY** Putting the parenthesis on its own line is the standard style for a wrapped signature.

9 `let hex = token.map { String(format: "%02x", $0) }.joined()`

- **WHO** Your code, iterating the token's bytes.
- **WHAT** **Turns each byte into two hexadecimal characters and joins them into one string.**
- **WHERE** A local constant inside the delegate method, the first statement.
- **WHEN** Immediately after iOS delivers the token.
- **WHY** Servers expect the token as a 64-character hex string, not raw Data; printing Data directly gives a useless description.

10 `Task { try? await sendTokenToServer(hex) }`

- **WHO** Swift concurrency starts the task; your server code receives the string.
- **WHAT** **Uploads the token to your backend without blocking the delegate method.**
- **WHERE** Inside the delegate method, after hex is built.
- **WHEN** Right after the token is formatted; the upload finishes later.
- **WHY** The delegate method is synchronous, so a Task is needed to call async code; try? tolerates a failed upload because apps re-register at every launch and receive the token again.

14 `@MainActor`

- **WHO** The Swift compiler, enforcing the isolation.
- **WHAT** **Runs the function below on the main actor.**
- **WHERE** An attribute on requestPushPermission, the function declared on the next line.
- **WHEN** Checked at compile time; callers hop to the main actor at run time.
- **WHY** UIApplication.shared is main-actor isolated, and Swift 6 refuses to call it from anywhere else.

15 `func requestPushPermission() async throws {`

- **WHO** Your code declares it; a settings screen or onboarding flow awaits it.
- **WHAT** **Declares the function that asks for permission and then registers with APNs.**
- **WHERE** A top-level function in the app target, below AppDelegate.
- **WHEN** Called when the user turns on notifications, not at first launch.
- **WHY** Asking in context, after explaining the benefit, is why fewer people decline.

```
16 let center = UNUserNotificationCenter.current()
```

WHO The UserNotifications framework, returning the app's notification center.
WHAT **Gets the object that manages notification permission for this app.**
WHERE A local constant inside requestPushPermission.
WHEN Each time the function runs, before the prompt.
WHY There is exactly one center per app; current() hands you that shared instance.

```
17 let granted = try await center.requestAuthorization(
```

WHO iOS shows the permission alert; the user answers it.
WHAT **Shows the system prompt and suspends until the user responds, returning true or false.**
WHERE Inside requestPushPermission; the options follow on the next line.
WHEN The first call shows the alert; later calls return the saved answer at once.
WHY iOS asks only once; after that the user changes the answer in Settings, so the app must handle false gracefully.

```
18 options: [.alert, .sound, .badge])
```

WHO Your code, choosing what the app may do.
WHAT **Requests permission to show alerts, play sounds and update the badge count.**
WHERE The argument of requestAuthorization.
WHEN Passed when the prompt is shown.
WHY Request only what the app uses; each option appears in the user's notification settings.

```
19 guard granted else { return }
```

WHO Your code, reading the user's answer.
WHAT **Leaves the function if the user declined.**
WHERE Inside requestPushPermission, after the prompt.
WHEN After the user answered, or immediately if they had already decided.
WHY Registering without permission would still produce a token, but alerts would never appear; this app registers only when it can show them.

```
20 UIApplication.shared.registerForRemoteNotifications()
```

WHO UIApplication, which contacts APNs on the app's behalf.
WHAT **Asks APNs for a device token for this app on this device.**
WHERE The last line of requestPushPermission, on the main actor.
WHEN After permission is granted; the token arrives later in the delegate.
WHY The call returns immediately; the result comes through the delegate method above because the round trip to APNs takes time.

Try it

1. The user declines the prompt. Which lines never run, and what should the app still be able to do?
2. Where would you add application(_:didFailToRegisterForRemoteNotificationsWithError:), and what would you log there?

CHECK YOURSELF

Check your understanding

1. A chat screen must show new messages within a second while it is open, and users send messages too. What fits best?
   A. Polling every 30 seconds
   B. A WebSocket
   C. Push notifications only
   D. Downloading the history once at launch
2. A news app must alert people to breaking stories even when it is closed. What is required?
   A. A WebSocket kept open in the background
   B. Polling every minute in the background
   C. Push notifications through APNs
   D. Server-Sent Events
3. An app polls every 5 seconds, and the data changes about once every 10 minutes. Roughly what fraction of requests carry news?
   A. About half
   B. All of them
   C. About 1 in 10
   D. About 1 in 120
4. Why does the WebSocket receive loop check `Task.isCancelled`?
   A. So the loop ends when its task is cancelled
   B. Because receive() refuses to run until the flag is checked
   C. So the socket reconnects faster after a dropped connection
   D. So a decoding error inside the loop cannot crash the app

Answers: Live Data Beyond Request and Response

1. **B. A WebSocket** A WebSocket carries messages in both directions the moment they happen, over one open connection. Polling every 30 seconds is far too slow, and push alone is best effort and meant for when the app is not in front.
2. **C. Push notifications through APNs** iOS suspends apps in the background, so their sockets, polls and streams stop. Only the system's push service can reach a device on the app's behalf.
3. **D. About 1 in 120** Ten minutes is 600 seconds, which is 120 polls, and only one of them finds a change. The other 119 wake the radio and load the server for nothing.
4. **A. So the loop ends when its task is cancelled** The loop would otherwise keep listening forever. Ending it lets defer close the socket, freeing the server connection and the radio.

Designing the API Layer

CONCEPT

Layers with clear jobs

As an app grows, networking code spreads everywhere unless you give it a home. A common structure has four layers, and each talks only to the one below it.

View	SwiftUI. Shows state and forwards user actions. Knows nothing about URLs or JSON.
Model	An @Observable class that holds screen state and decides when to load. Testable without any interface.
Service protocol	Describes what data the app needs, such as todos(page:), not how it is fetched.
Live service	The implementation that uses URLSession. Tests and previews swap in a stub that returns fixed data.

> **Dependency injection**
>
> The model receives its service through its initializer instead of creating one. That single decision lets you test the model without a network, preview screens offline and point the app at a different server without touching any view.

CODE · READ-ALONG

A protocol and a live implementation

The protocol is the contract: one async method that returns a page of to-dos. `Sendable` promises values are safe to share across concurrent tasks. `LiveTodoService` fulfils the contract with URLSession. `validate(_:)` is the status check from the URLSession lesson, moved into a helper. JSONPlaceholder pages with `_page` and `_limit`.

```swift
import Foundation

protocol TodoService: Sendable {
    func todos(page: Int) async throws -> [Todo]
}

struct LiveTodoService: TodoService {
    var session = URLSession.shared
    var pageSize = 20

    func todos(page: Int) async throws -> [Todo] {
        var parts = URLComponents(
            string: "https://jsonplaceholder.typicode.com/todos")!
        parts.queryItems = [
            URLQueryItem(name: "_page", value: String(page)),
            URLQueryItem(name: "_limit", value: String(pageSize))
        ]
        let (data, response) = try await session.data(from: parts.url!)
        try validate(response)
        return try JSONDecoder().decode([Todo].self, from: data)
    }
}
```

Line by line

1 `import Foundation`

- **WHO** The Swift compiler, building the file that holds TodoService and LiveTodoService.
- **WHAT** **Imports URLSession, URLComponents and JSONDecoder.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before either type is used.
- **WHY** Only the live implementation needs networking types; the protocol does not.

3 `protocol TodoService: Sendable {`

- **WHO** Your code declares it; the compiler enforces it on conforming types.
- **WHAT** **Declares the contract every to-do source must meet.**
- **WHERE** A top-level protocol in the services file, above the live implementation.
- **WHEN** Checked at compile time wherever a type claims to conform.
- **WHY** Depending on a protocol instead of a concrete type is what makes swapping in a stub possible.

4 `func todos(page: Int) async throws -> [Todo]`

- **WHO** The TodoService protocol; every conforming type must implement it.
- **WHAT** **Requires one method that returns a page of to-dos.**
- **WHERE** The only requirement inside TodoService.
- **WHEN** Called by the model when it needs more data.
- **WHY** The signature says nothing about URLs or JSON, so implementations are free to use anything.

7 `struct LiveTodoService: TodoService {`

- **WHO** Your code declares it; the compiler checks it meets TodoService.
- **WHAT** **Declares the real implementation, backed by the network.**
- **WHERE** A top-level struct in the services file, below the protocol.
- **WHEN** Created once at app launch and handed to the model.
- **WHY** A struct with only Sendable properties satisfies Sendable automatically.

```
8  var session = URLSession.shared
```

WHO The LiveTodoService struct; todos(page:) uses it.
WHAT **Stores the session to use.**
WHERE A stored property with a default.
WHEN Set when the service is created.
WHY An injectable session allows custom caching or timeouts.

```
9  var pageSize = 20
```

WHO The LiveTodoService struct; the _limit query item reads it.
WHAT **Sets how many items to request per page.**
WHERE A stored property with a default.
WHEN Set when the service is created.
WHY Large enough to fill a screen, small enough to arrive quickly.

```
11 func todos(page: Int) async throws -> [Todo] {
```

WHO Your struct, satisfying the protocol.
WHAT **Implements the required method.**
WHERE Inside LiveTodoService, fulfilling the protocol requirement.
WHEN Each time the model asks for a page.
WHY The exact signature from the protocol; the compiler checks it matches.

```
12 var parts = URLComponents(
```

WHO Foundation's URLComponents initializer, called by your code.
WHAT **Starts building the collection URL.**
WHERE A local variable inside todos(page:), the first statement.
WHEN Each time the model asks for a page.
WHY Query items will be added next.

```
13 string: "https://jsonplaceholder.typicode.com/todos")!
```

WHO URLComponents, parsing the collection address.
WHAT **Parses the collection address.**
WHERE Argument of the call above.
WHEN Each call, as parts is created.
WHY A constant string, safe to force-unwrap.

```
14 parts.queryItems = [
```

WHO Your code, assigning the paging parameters.
WHAT **Starts the paging parameters.**
WHERE On the parts value inside todos(page:).
WHEN Each call, right after parts exists.
WHY Paging belongs in the query, not the path.

```
15 URLQueryItem(name: "_page", value: String(page)),
```

WHO Your code, converting the page number to text.
WHAT **Asks for a specific page, starting at 1.**
WHERE The first query item in the array.
WHEN Each call, with the page the model requested.
WHY This API names the parameter _page; other APIs use page or a cursor.

```
16 URLQueryItem(name: "_limit", value: String(pageSize))
```

WHO	Your code, converting pageSize to text.
WHAT	**Asks for pageSize items per page.**
WHERE	The second query item in the array.
WHEN	Each call, with the same pageSize every time.
WHY	Without a limit, this API returns all 200 to-dos at once.

```
18 let (data, response) = try await session.data(from: parts.url!)
```

WHO	The stored URLSession, on its own background threads.
WHAT	**Fetches the page and suspends until it arrives.**
WHERE	Inside todos(page:), after the URL is complete.
WHEN	Each call; the method suspends here until the page arrives.
WHY	Same pattern as every earlier fetch.

```
19 try validate(response)
```

WHO	The shared validate helper, called by your code.
WHAT	**Throws unless the status is 2xx.**
WHERE	Inside todos(page:), between the fetch and the decode.
WHEN	After the reply arrives and before any decoding.
WHY	One shared helper means every service checks status the same way.

```
20 return try JSONDecoder().decode([Todo].self, from: data)
```

WHO	JSONDecoder, decoding a JSON array into [Todo].
WHAT	**Decodes a JSON array into an array of Todo values.**
WHERE	The last line of the method.
WHEN	After validation passed, as the last step of each call.
WHY	[Todo].self asks for an array; decoding Todo.self here would throw typeMismatch.

CODE · READ-ALONG

Pagination in the model

Long lists arrive in **pages**. Page-number APIs take `page` and `limit`; cursor APIs return an opaque `next` token that you send back, which stays correct when new items are inserted. This model loads the next page when the list's last row appears, ignores duplicate triggers while a load is running and stops when a page comes back empty.

```swift
import Observation

@MainActor
@Observable
final class TodoListModel {
    private(set) var todos: [Todo] = []
    private(set) var isLoading = false
    private(set) var hasMore = true
    private var nextPage = 1
    private let service: any TodoService
    init(service: any TodoService) { self.service = service }
    func loadNextPage() async {
        guard !isLoading, hasMore else { return }
        isLoading = true
        defer { isLoading = false }
        guard let page = try? await service.todos(page: nextPage)
        else { return }
        todos += page
        hasMore = !page.isEmpty
        nextPage += 1
    }
}
```

Line by line

1 `import Observation`

- **WHO** The Swift compiler, building the file that holds TodoListModel.
- **WHAT** **Imports the @Observable macro from the Observation framework.**
- **WHERE** Top of the file.
- **WHEN** At compile time, before the macro can expand.
- **WHY** The model needs observation, not networking types.

3 `@MainActor`

- **WHO** The Swift compiler, which enforces the isolation.
- **WHAT** **Keeps all of the model's state on the main actor.**
- **WHERE** Applies to the class below.
- **WHEN** Enforced at compile time.
- **WHY** The list view reads todos on the main thread, so writes must happen there too.

4 `@Observable`

- **WHO** The Observation macro, expanded by the compiler.
- **WHAT** **Lets SwiftUI redraw when todos or isLoading change.**
- **WHERE** Applies to the class below.
- **WHEN** Expanded at compile time.
- **WHY** Views showing a spinner at the bottom of the list react to isLoading automatically.

5 `final class TodoListModel {`

- **WHO** Your code declares it; the list screen creates one.
- **WHAT** **Declares the list screen's model.**
- **WHERE** A reference type on the heap.
- **WHEN** Created once per list screen.
- **WHY** One object owns the pages loaded so far.

```
6  private(set) var todos: [Todo] = []
```

WHO	TodoListModel owns it; the list view reads it.
WHAT	**Holds every item loaded so far.**
WHERE	An observed stored property of TodoListModel.
WHEN	Grows by one page at a time.
WHY	Views read it; only the model appends to it.

```
7  private(set) var isLoading = false
```

WHO	TodoListModel owns it; a footer spinner reads it.
WHAT	**Records whether a page request is in flight.**
WHERE	An observed stored property, after todos.
WHEN	True during each load.
WHY	Prevents duplicate requests and drives a loading indicator.

```
8  private(set) var hasMore = true
```

WHO	TodoListModel owns it; the guard in loadNextPage reads it.
WHAT	**Records whether more pages may exist.**
WHERE	An observed stored property, after isLoading.
WHEN	Becomes false at the end of the data.
WHY	Stops the app from asking for pages that do not exist.

```
9  private var nextPage = 1
```

WHO	TodoListModel alone reads and writes it.
WHAT	**Remembers which page to request next.**
WHERE	A private stored property of the model.
WHEN	Incremented after each successful load.
WHY	Page numbers in this API start at 1.

```
10 private let service: any TodoService
```

WHO	TodoListModel owns it; loadNextPage calls through it.
WHAT	**Holds whatever service it was given: live or stub.**
WHERE	A private constant of the model, set only in init.
WHEN	Set once in the initializer.
WHY	any TodoService means "some type that meets the contract"; the model never knows which.

```
11 init(service: any TodoService) { self.service = service }
```

WHO	Your code; the screen or a test calls it.
WHAT	**Receives the service from outside.**
WHERE	The only initializer of TodoListModel.
WHEN	When the screen creates its model.
WHY	This is dependency injection: the caller decides which service to use.

```
12 func loadNextPage() async {
```

WHO	Your model; the view calls it from the last row's .task.
WHAT	**Declares the method that fetches one more page.**
WHERE	A method of TodoListModel, isolated to the main actor like the class.
WHEN	When the user scrolls to the end of the list.
WHY	It does not throw; errors leave the list as it was so the user can try again.

```
13 guard !isLoading, hasMore else { return }
```

WHO TodoListModel, checking its own flags.
WHAT **Exits if a load is already running or nothing is left.**
WHERE First line of the method.
WHEN On every call, before any state changes.
WHY The last row can appear several times quickly; this guard prevents duplicate pages.

```
14 isLoading = true
```

WHO TodoListModel, writing its own flag.
WHAT **Marks the load as started.**
WHERE On the main actor.
WHEN Before the await, so the next caller sees it.
WHY Set before the await, so a second call during the wait is rejected by the guard.

```
15 defer { isLoading = false }
```

WHO Swift's defer statement, run on every exit from the method.
WHAT **Guarantees isLoading is reset however the method exits.**
WHERE Inside loadNextPage, directly after isLoading is set.
WHEN At exit, success or failure.
WHY Forgetting to reset it on an early return would freeze pagination forever.

```
16 guard let page = try? await service.todos(page: nextPage)
```

WHO Your service, live or stub.
WHAT **Fetches the next page; try? turns any error into nil.**
WHERE On the main actor while suspended.
WHEN Each call that passes the first guard; the method suspends here.
WHY For a list, failing quietly and letting the user scroll again is often the right behavior.

```
17 else { return }
```

WHO TodoListModel, leaving the method early.
WHAT **Leaves without changes if the fetch failed.**
WHERE The guard's else branch.
WHEN Only when the service threw and try? produced nil.
WHY nextPage is untouched, so the same page is requested next time.

```
18 todos += page
```

WHO TodoListModel, appending to its observed array.
WHAT **Appends the new items to the list.**
WHERE On the main actor.
WHEN After a successful fetch.
WHY Observation redraws the list with the new rows.

```
19 hasMore = !page.isEmpty
```

WHO TodoListModel, updating its own flag.
WHAT **Stops paging when a page comes back empty.**
WHERE On the main actor.
WHEN After each successful fetch, before nextPage moves.
WHY An empty page is this API's signal that the end was reached.

20 `nextPage += 1`

WHO TodoListModel, advancing its private counter.
WHAT **Moves on to the following page.**
WHERE On the main actor.
WHEN After each successful fetch.
WHY Only advances on success, so a failed page is retried rather than skipped.

CODE · SWIFT TESTING

Test the model with a stub

Because the model depends on a protocol, a test can hand it a **stub**: a fake service that returns fixed data instantly. No network, no flakiness, no waiting. This Swift Testing test checks two behaviors of the model from the previous step: a page appends its items, and an empty page ends pagination.

```swift
import Testing

struct StubTodoService: TodoService {
    let pages: [[Todo]]

    func todos(page: Int) async throws -> [Todo] {
        page <= pages.count ? pages[page - 1] : []
    }
}

@MainActor
@Test func appendsPagesUntilEmpty() async {
    let first = Todo(userId: 1, id: 1, title: "Plan", completed: false)
    let stub = StubTodoService(pages: [[first]])
    let model = TodoListModel(service: stub)

    await model.loadNextPage()
    #expect(model.todos.count == 1)
    #expect(model.hasMore)

    await model.loadNextPage()
    #expect(model.hasMore == false)
}
```

Line by line

1 `import Testing`

WHO The Swift compiler, building the test target.
WHAT **Imports Swift Testing: @Test and #expect.**
WHERE Top of the test file, in the test target.
WHEN At compile time, when the test target is built.
WHY Swift Testing is the modern testing framework that ships with Xcode.

3 `struct StubTodoService: TodoService {`

WHO Your test code declares it; the compiler checks the conformance.
WHAT **Declares a fake service that meets the same contract as the live one.**
WHERE A struct in the test target only.
WHEN Created inside each test.
WHY The model cannot tell the difference, which is the point.

```
4  let pages: [[Todo]]
```

WHO The StubTodoService struct; todos(page:) indexes into it.
WHAT **Holds the pages to return, in order.**
WHERE The only stored property inside StubTodoService.
WHEN Set when the test creates the stub.
WHY Each test decides exactly what the "server" contains.

```
6  func todos(page: Int) async throws -> [Todo] {
```

WHO Your stub, satisfying the protocol.
WHAT **Returns a page from memory instead of the network.**
WHERE Inside StubTodoService, fulfilling the TodoService requirement.
WHEN Whenever the model asks for a page.
WHY Marked async throws to match the protocol, even though it never waits or fails.

```
7  page <= pages.count ? pages[page - 1] : []
```

WHO The StubTodoService struct, choosing a page from memory.
WHAT **Returns the requested page, or an empty array past the end.**
WHERE The method body; a single expression returns implicitly.
WHEN Each time the model calls todos(page:).
WHY An empty array after the last page mimics how the real API signals the end.

```
11 @MainActor
```

WHO The Swift compiler, which checks the isolation.
WHAT **Runs the test on the main actor.**
WHERE Applies to the test function below.
WHEN When the test runs.
WHY TodoListModel is main-actor isolated, so the test must be too.

```
12 @Test func appendsPagesUntilEmpty() async {
```

WHO Swift Testing discovers it through the @Test macro.
WHAT **Declares an async test with a descriptive name.**
WHERE A top-level function in the test target.
WHEN When you run tests in Xcode or from the command line.
WHY async lets the test await the model's methods directly.

```
13 let first = Todo(userId: 1, id: 1, title: "Plan", completed: false)
```

WHO The test function, creating a fixture value.
WHAT **Creates one known to-do.**
WHERE A local constant inside appendsPagesUntilEmpty, the first statement.
WHEN At the start of the test.
WHY Known input makes the expected output obvious.

```
14 let stub = StubTodoService(pages: [[first]])
```

WHO The test function, building the fake service.
WHAT **Creates a stub with one page that holds one item.**
WHERE A local constant inside the test, after first.
WHEN During the arrange phase, before the model exists.
WHY The second page is deliberately missing, so it comes back empty.

```
15 let model = TodoListModel(service: stub)
```

WHO	The test function, acting as the composition root.
WHAT	**Creates the real model with the fake service injected.**
WHERE	A local constant inside the test, after stub.
WHEN	During the arrange phase, right before the first load.
WHY	This is the payoff of dependency injection: the real logic under test, with no network.

```
17 await model.loadNextPage()
```

WHO	TodoListModel, running its real pagination logic.
WHAT	**Loads page 1 from the stub.**
WHERE	On the main actor.
WHEN	During the act phase, the first time the test calls it.
WHY	It completes instantly because the stub never waits.

```
18 #expect(model.todos.count == 1)
```

WHO	Swift Testing's #expect macro, evaluating the comparison.
WHAT	**Checks that exactly one item was appended.**
WHERE	Inside appendsPagesUntilEmpty, after the first load.
WHEN	During the first assert phase, after loadNextPage returns.
WHY	On failure, #expect prints both sides of the comparison.

```
19 #expect(model.hasMore)
```

WHO	Swift Testing's #expect macro, checking a Bool.
WHAT	**Checks that the model still expects more pages.**
WHERE	Inside the test, right after the count check.
WHEN	During the first assert phase, before the second load.
WHY	A non-empty page must not end pagination.

```
21 await model.loadNextPage()
```

WHO	TodoListModel, asking the stub for page 2.
WHAT	**Requests page 2, which the stub returns empty.**
WHERE	On the main actor.
WHEN	During the second act phase, after the first assertions.
WHY	Exercises the end-of-data path.

```
22 #expect(model.hasMore == false)
```

WHO	Swift Testing's #expect macro, checking the flag flipped.
WHAT	**Checks that pagination has stopped.**
WHERE	Inside the test, the last line.
WHEN	During the final assert phase, after the second load.
WHY	Without this behavior the app would request empty pages forever.

CONCEPT

HTTP caching: do not download it twice

Many responses do not change between requests. HTTP has caching built in, and URLSession's `URLCache` follows its rules automatically.

Cache-Control: max-age=60	The server says the response stays fresh for 60 seconds. Within that window URLSession answers from the cache without using the network.
ETag	A version label for the response, such as "v42". URLSession stores it with the cached copy.

If-None-Match and 304	When the copy is stale, URLSession asks "still v42?". If nothing changed, the server answers 304 Not Modified with no body, and your code receives the cached body as a normal 200 response.
Cache policy	Each request can override the default: .returnCacheDataElseLoad for offline-first screens, .reloadIgnoringLocalCacheData for pull to refresh.

INTERACTIVE

Cache lab

Request the same resource again and again while you control the clock, the server's data, the cache policy and the network. Watch which requests are answered from the cache, which revalidate with a 304 and which download the whole body, and count the bytes and time saved. The last button sends a real conditional request, so you can see a genuine 304.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Configure the cache

The shared session already has a small cache. A custom `URLSessionConfiguration` lets you size it, choose a default policy and wait for connectivity instead of failing at once. Individual requests can still pick their own policy.

```
1  import Foundation
2
3  func makeCachingSession() -> URLSession {
4      let config = URLSessionConfiguration.default
5      config.urlCache = URLCache(memoryCapacity: 10_000_000,
6                                 diskCapacity: 100_000_000)
7      config.requestCachePolicy = .useProtocolCachePolicy
8      config.timeoutIntervalForRequest = 20
9      config.waitsForConnectivity = true
10     return URLSession(configuration: config)
11 }
12
13 func offlineFirstRequest(for url: URL) -> URLRequest {
14     var request = URLRequest(url: url)
15     request.cachePolicy = .returnCacheDataElseLoad
16     return request
17 }
```

Line by line

1 `import Foundation`

WHO	The Swift compiler, building the file that holds makeCachingSession.
WHAT	**Imports URLSession, URLCache and URLRequest.**
WHERE	Top of the file.
WHEN	At compile time, before either function can run.
WHY	Caching is part of the URL Loading System in Foundation.

```
3  func makeCachingSession() -> URLSession {
```

WHO Your code; typically called once at app launch.
WHAT **Declares a factory for a session with a larger cache.**
WHERE A top-level function in the networking setup file.
WHEN Once; the session is then reused.
WHY Sessions are meant to be long-lived and shared, not created per request.

```
4      let config = URLSessionConfiguration.default
```

WHO Foundation's URLSessionConfiguration, providing a fresh default copy.
WHAT **Starts from the default configuration.**
WHERE A local constant; configuration is a class, so its properties can still change.
WHEN Once, when the session is created.
WHY default stores cookies and cache on disk; ephemeral would keep everything in memory.

```
5      config.urlCache = URLCache(memoryCapacity: 10_000_000,
```

WHO Foundation's URLCache initializer, called by your code.
WHAT **Creates a cache with a 10 MB memory budget.**
WHERE The urlCache property of config, inside makeCachingSession.
WHEN Once, when the session is created.
WHY Underscores make large numbers readable; memory hits are the fastest.

```
6                             diskCapacity: 100_000_000)
```

WHO Foundation's URLCache initializer, receiving its second argument.
WHAT **Gives the cache a 100 MB disk budget.**
WHERE Second argument of the URLCache initializer.
WHEN Once, when the session is created.
WHY Disk entries survive app restarts, so a cold start can show cached data.

```
7      config.requestCachePolicy = .useProtocolCachePolicy
```

WHO Your code, setting a URLSessionConfiguration property.
WHAT **Follows the server's Cache-Control and ETag rules by default.**
WHERE The requestCachePolicy property of config.
WHEN Applied to every request from this session.
WHY This is already the default; writing it documents the intent.

```
8      config.timeoutIntervalForRequest = 20
```

WHO Your code, setting the request timeout on config.
WHAT **Fails a request if no data arrives for 20 seconds.**
WHERE The timeoutIntervalForRequest property of config.
WHEN Enforced during each request.
WHY Shorter than the 60-second default, so users hear about problems sooner.

```
9      config.waitsForConnectivity = true
```

WHO Your code, setting a connectivity option on config.
WHAT **Waits for a network to appear instead of failing immediately when offline.**
WHERE The waitsForConnectivity property of config.
WHEN When a request starts without connectivity.
WHY Useful for background-friendly work; the request timeout still applies once it starts.

```
10 return URLSession(configuration: config)
```

WHO Foundation's URLSession initializer, called by your code.
WHAT **Creates the session from the configuration.**
WHERE The last line of the factory.
WHEN Once, when the session is created.
WHY The configuration is copied, so later changes to config do not affect the session.

```
13 func offlineFirstRequest(for url: URL) -> URLRequest {
```

WHO Your code declares it; offline-first screens call it.
WHAT **Declares a helper for screens that prefer cached data.**
WHERE A top-level function below makeCachingSession.
WHEN Whenever such a screen loads.
WHY The policy is per request, so other screens keep normal behavior.

```
14 var request = URLRequest(url: url)
```

WHO Foundation's URLRequest initializer, called by your code.
WHAT **Creates a request for the given URL with default settings.**
WHERE A local variable inside offlineFirstRequest(for:).
WHEN Each time offlineFirstRequest(for:) runs.
WHY It must be var to change its policy.

```
15 request.cachePolicy = .returnCacheDataElseLoad
```

WHO Your code, setting the request's cachePolicy property.
WHAT **Uses any cached copy, however old; loads from the network only if none exists.**
WHERE The cachePolicy property of the request, inside offlineFirstRequest(for:).
WHEN Applied when the request is sent.
WHY Ideal for content that rarely changes, or when showing something beats showing nothing.

```
16 return request
```

WHO Your offlineFirstRequest(for:) function, handing back the value.
WHAT **Returns the configured request.**
WHERE The last line of offlineFirstRequest(for:).
WHEN Each call, after the policy is set.
WHY The caller sends it with any session.

CONCEPT

GraphQL

GraphQL is a query language for APIs, created at Facebook. Instead of many REST endpoints, a GraphQL server exposes one endpoint, and the client sends a query describing exactly the fields it wants, nested as deeply as needed. The response mirrors the query's shape.

For example, `{ user(id: 1) { name todos(first: 3) { title } } }` returns a user's name and three to-do titles in one round trip, with nothing extra.

Good fit	Screens that combine several related resources, or many clients (phone, watch, web) that need different fields.
Costs	More server complexity, harder HTTP caching because queries usually travel as POST requests, and protection needed against very expensive queries.
On iOS	It is still JSON over HTTP, so URLSession and Codable work. Libraries such as Apollo iOS generate Swift types from the schema.

CONCEPT

gRPC

gRPC is a remote procedure call framework, created at Google: the client calls methods on a server as if they were local functions. You describe services and messages in a `.proto` file (Protocol Buffers), and a code generator produces the Swift client. Messages travel as compact binary over HTTP/2, and calls can stream in either direction.

Good fit	Traffic between services inside a backend, high-volume or streaming APIs, and teams that want a strict, generated contract.
Costs	Binary payloads are hard to read while debugging, browsers need a proxy, and both sides must share the .proto files.
On iOS	The open-source grpc-swift package generates async/await clients. Most public APIs your apps call will still be REST with JSON.

CONCEPT

REST API design you will meet

Sooner or later you will design the backend side too, or review one. These conventions make an API predictable for every client that uses it.

Plural nouns, methods as verbs	/users/7/todos rather than /getTodosForUser?id=7. The HTTP method already says what to do.
Honest statuses and error bodies	201 with the new resource after a POST; 422 with a JSON body naming the invalid field. RFC 9457 "problem details" standardizes that body.
Versioning	A version in the path (/v1/) or a header, so old app versions keep working; not every user updates.
Idempotency keys	Many APIs accept a unique Idempotency-Key header on POST, the header the Errors, Retries and Security lesson added to an order request. If the server sees the same key twice, it returns the first result instead of charging twice.
Rate limits	Answer 429 Too Many Requests with a Retry-After header, so clients know when to back off.

CHECK YOURSELF

Check your understanding

1. Why does `TodoListModel` receive `any TodoService` in its initializer instead of creating a `LiveTodoService` itself?

   A. Swift requires a protocol for any async method
   B. So tests and previews can inject a stub with fixed data
   C. Calls through a protocol run faster than direct calls
   D. URLSession cannot be stored inside an @Observable class

2. A response carried `ETag: "v7"` and `max-age=0`. The data has not changed. With `.useProtocolCachePolicy`, what does the next request do?
   A. Downloads the full body again, because max-age=0 disables caching
   B. Fails with URLError, because the cached copy is stale
   C. Sends If-None-Match: "v7", gets 304, and returns the cached body
   D. Never contacts the server, because the ETag proves the copy is good

3. An inbox where new messages constantly arrive at the top needs paging. Page numbers or cursors?
   A. Cursors, because inserts shift page boundaries and repeat rows
   B. Page numbers, because the inbox is sorted and numbers are simpler
   C. Neither; download the whole inbox once and filter it locally
   D. Page numbers with a larger limit, so fewer pages shift

4. A watch app and a phone app need very different fields from the same related data, ideally in one request each. Which API style fits best?
   A. gRPC
   B. Polling
   C. Server-Sent Events
   D. GraphQL

5. The last row of a list appears twice in quick succession, calling `loadNextPage()` twice. What stops two identical page requests?
   A. The hasMore check, since the first call sets hasMore to false
   B. The !isLoading check, since isLoading is set before the await
   C. defer { isLoading = false }, which blocks the second call
   D. nextPage += 1, so the second call asks for a different page

Answers: Designing the API Layer

1. **B. So tests and previews can inject a stub with fixed data** Injection decouples the model from the network. The live app passes LiveTodoService; tests pass a stub. Speed and language rules have nothing to do with it.
2. **C. Sends If-None-Match: "v7", gets 304, and returns the cached body** max-age=0 means the copy must be revalidated before use, but the ETag makes revalidation cheap: only headers travel, and URLSession turns the 304 into the cached 200 response for you.
3. **A. Cursors, because inserts shift page boundaries and repeat rows** If two messages arrive while you read page 1, page 2 now starts two items earlier and repeats them. A cursor marks a position in the data itself, so inserts do not shift it.
4. **D. GraphQL** GraphQL lets each client ask for exactly the fields it needs across related resources in one query. gRPC suits service-to-service contracts, and the other two are delivery patterns, not API shapes.
5. **B. The !isLoading check, since isLoading is set before the await** The first call sets isLoading to true before awaiting, so the second call hits the guard and returns. hasMore is still true at that moment, and defer only runs when the first call finishes.

Worked Examples

WORKED EXAMPLE

One weather request, station by station

Problem.
Follow one weather request from its URL to the line on screen. A misspelled city returns a 404 and a friendly fallback; fixing the spelling returns JSON that decodes into the line the screen shows.

Frame 1 of 7

The journey starts at the URL station: the city the user typed, Lisbn, is inserted into the query string, and the printed URL already carries the spelling mistake.

```
  let city = "Lisbn"
  let url = "https://api.example.com/weather?city=\(city)"
  print("URL: \(url)")
```

Frame 2 of 7

At the Request station the program describes a GET request for that URL. Nothing has travelled yet; the method and address are simply written down and printed.

```
  let city = "Lisbn"
  let url = "https://api.example.com/weather?city=\(city)"
+ let method = "GET"
+ print("URL: \(url)")
+ print("Request: \(method) \(url)")
```

Frame 3 of 7

Transport delivers the canned reply in this model: status 404 with an error body. The printed status and body are exactly what a real response would hand the code.

```
  let city = "Lisbn"
  let url = "https://api.example.com/weather?city=\(city)"
  let method = "GET"
+ let status = 404
+ let body = "{\"error\":\"city not found\"}"
+ print("URL: \(url)")
+ print("Request: \(method) \(url)")
+ print("Status: \(status)")
+ print("Body: \(body)")
```

Frame 4 of 7

At the Response station the code checks the status, sees 404, and prints a friendly fallback instead of decoding. The diagram marks this station failed for this request.

```
  let city = "Lisbn"
  let url = "https://api.example.com/weather?city=\(city)"
  let method = "GET"
  let status = 404
  let body = "{\"error\":\"city not found\"}"
  print("URL: \(url)")
  print("Request: \(method) \(url)")
  print("Status: \(status)")
+ if status == 404 {
+     print("Fallback: weather unavailable for \(city)")
+ }
```

Frame 5 of 7

The repair is one letter: the city is now Lisbon. The same journey now answers 200, and the body holds the weather JSON the next station will decode.

```
+ let city = "Lisbon"
  let url = "https://api.example.com/weather?city=\(city)"
  let method = "GET"
+ let status = 200
+ let body = "{\"city\":\"Lisbon\",\"temp\":21,\"summary\":\"sunny\"}"
  print("URL: \(url)")
  print("Request: \(method) \(url)")
  print("Status: \(status)")
+ print("Body: \(body)")
```

Frame 6 of 7

At the Decode station a Decodable struct and JSONDecoder turn the canned body into typed values. The prints read the decoded fields, not the raw JSON text.

```
+ import Foundation
+
+ struct Weather: Decodable {
+     let city: String
+     let temp: Int
+     let summary: String
+ }
+
+ let city = "Lisbon"
+ let url = "https://api.example.com/weather?city=\(city)"
+ let method = "GET"
+ let status = 200
+ let body = "{\"city\":\"Lisbon\",\"temp\":21,\"summary\":\"sunny\"}"
+ let weather = try JSONDecoder()
+     .decode(Weather.self, from: Data(body.utf8))
+ print("Status: \(status)")
+ print("City: \(weather.city)")
+ print("Temp: \(weather.temp)")
+ print("Summary: \(weather.summary)")
```

Frame 7 of 7

The final station is the UI update: one line built from the decoded fields. Lisbon, 21 degrees and sunny is exactly what the screen would show the user.

```
1   import Foundation
2
3   struct Weather: Decodable {
4       let city: String
5       let temp: Int
6       let summary: String
7   }
8
9   let city = "Lisbon"
10  let url = "https://api.example.com/weather?city=\(city)"
11  let method = "GET"
12  let status = 200
13  let body = "{\"city\":\"Lisbon\",\"temp\":21,\"summary\":\"sunny\"}"
14  let weather = try JSONDecoder()
15      .decode(Weather.self, from: Data(body.utf8))
16  print("Status: \(status)")
17  print("City: \(weather.city)")
18  print("Temp: \(weather.temp)")
19  print("Summary: \(weather.summary)")
20 + print("\(weather.city): \(weather.temp)° and \(weather.summary)")
```

Line by line: the finished program

1 `import Foundation`

- **WHO** The Swift compiler, which loads the Foundation framework.
- **WHAT** **Makes JSONDecoder and Data available in this program.**
- **WHERE** The first line of the file, above the model struct.
- **WHEN** At compile time, before any decoding code is checked.
- **WHY** JSON decoding lives in Foundation, as every decoding step in this module showed.

3 `struct Weather: Decodable {`

- **WHO** You declare it; JSONDecoder creates values of it.
- **WHAT** **Declares the weather model that the JSON body will decode into.**
- **WHERE** At file scope, above the program that uses it.
- **WHEN** Compiled once; values are created when decoding runs.
- **WHY** A typed model replaces string keys with checked properties.

4 `let city: String`

- **WHO** The Weather struct; the decoder assigns it from the city key.
- **WHAT** **Stores the city name decoded from the JSON string.**
- **WHERE** The first stored property inside the Weather struct.
- **WHEN** Filled in during decoding, from the body text.
- **WHY** Property names match the JSON keys, so no mapping code is needed.

```
5  let temp: Int
```

WHO The Weather struct; the decoder assigns it from the temp key.
WHAT **Stores the temperature as a whole number of degrees.**
WHERE The second stored property inside the Weather struct.
WHEN Filled in during decoding, from the body text.
WHY The canned body sends 21 with no fraction, so Int is the right type.

```
6  let summary: String
```

WHO The Weather struct; the decoder assigns it from the summary key.
WHAT **Stores the short weather description decoded from the JSON.**
WHERE The last stored property inside the Weather struct.
WHEN Filled in during decoding, from the body text.
WHY The summary is the word the final screen line shows to the user.

```
9  let city = "Lisbon"
```

WHO Your code, naming the place to look up.
WHAT **Stores the correctly spelled city name, Lisbon.**
WHERE The first line of the program, before the URL is built.
WHEN First, before any request work happens in this program.
WHY The repair in this example is this one corrected spelling.

```
10 let url = "https://api.example.com/weather?city=\(city)"
```

WHO Your code, building the address as a string.
WHAT **Builds the full request URL by inserting the city into the query part.**
WHERE The second line, using the city value declared above it.
WHEN After the city exists and before the request is described.
WHY A weather request names its place in the URL, as the module lessons showed.

```
11 let method = "GET"
```

WHO Your code, choosing the HTTP method.
WHAT **Stores the method GET, because this request only reads data.**
WHERE With the other request facts, above the prints that use it.
WHEN While the request is being described, before transport runs.
WHY GET is the safe, idempotent read method from the HTTP lesson.

```
12 let status = 200
```

WHO Your model of the server reply, standing in for transport.
WHAT **Stores the canned status code 200, the request succeeded.**
WHERE With the reply facts, after the request description lines.
WHEN When the canned response arrives in this program.
WHY Checking for a 2xx status is phase three of every fetch in this module.

```
13 let body = "{\"city\":\"Lisbon\",\"temp\":21,\"summary\":\"sunny\"}"
```

WHO Your model of the server, supplying a canned body.
WHAT **Stores the success JSON body with city, temperature and summary fields.**
WHERE Beside the status value that describes the same reply.
WHEN When the canned response arrives in this program.
WHY Canned JSON lets the decode step run in the engine with no network.

```
14 let weather = try JSONDecoder()
```

WHO JSONDecoder, created by your code for this body.
WHAT **Creates a decoder and begins the decoding statement.**
WHERE In the output section, after the body value exists.
WHEN After the status check passed with 200 in this journey.
WHY Decoding is phase four: it only runs on a successful response.

```
15 .decode(Weather.self, from: Data(body.utf8))
```

WHO JSONDecoder, parsing the body bytes into a Weather value.
WHAT **Decodes the body text, converted to Data, into a typed Weather value.**
WHERE The continuation of the decoding statement on the line above.
WHEN Immediately after the decoder is created, in the same statement.
WHY URLSession hands apps Data, so decoding from Data matches real code.

```
16 print("Status: \(status)")
```

WHO The print function from the Swift standard library.
WHAT **Prints the status code that the canned transport delivered.**
WHERE In the output section, after the reply values exist.
WHEN After the response arrives, before any decoding happens.
WHY The status decides whether decoding is even worth trying.

```
17 print("City: \(weather.city)")
```

WHO The print function, reading the decoded model.
WHAT **Prints the city field of the decoded Weather value.**
WHERE In the output section, after decoding has succeeded.
WHEN After decoding, as the first decoded field line.
WHY Reading a property proves the JSON key landed in the model.

```
18 print("Temp: \(weather.temp)")
```

WHO The print function, reading the decoded model.
WHAT **Prints the temperature field of the decoded Weather value.**
WHERE In the output section, between the city and summary lines.
WHEN After decoding, as the second decoded field line.
WHY A number here proves the JSON number decoded as a number.

```
19 print("Summary: \(weather.summary)")
```

WHO The print function, reading the decoded model.
WHAT **Prints the summary field of the decoded Weather value.**
WHERE In the output section, after the temperature line.
WHEN After decoding, as the last decoded field line.
WHY All three fields are now typed values the screen can use directly.

```
20 print("\(weather.city): \(weather.temp)° and \(weather.summary)")
```

WHO Your code, formatting the line the screen would show.
WHAT **Prints the final user-facing line built from the decoded fields.**
WHERE The last line of the program, after every field was printed.
WHEN Last, once decoding has succeeded and the fields are known.
WHY This is the UI update station: data becomes one readable sentence.

Try it

Build the same opening journey for Porto: store the city, build its weather URL, then print the URL line and the request line for a GET request to that URL.

```
let city = "Porto"
let url = "https://api.example.com/weather?city=\(city)"
print("URL: \(url)")
```

EXPECTED OUTPUT

```
URL: https://api.example.com/weather?city=Porto
Request: GET https://api.example.com/weather?city=Porto
```

WORKED EXAMPLE

Retry a flaky service with bounded backoff

Problem.

A service fails twice, then succeeds. Build a retry loop around a scripted fake transport that tries at most three times, records the backoff delays it would wait, and prints the successful payload.

Frame 1 of 6

The script comes first: three outcomes written as data, failure, failure, success. The print confirms the plan holds three results before any transport exists.

```
  let results = ["failure", "failure", "success"]
  print("Scripted results: \(results.count)")
```

OUTPUT SO FAR

```
Scripted results: 3
```

Frame 2 of 6

A fake transport type replays the script by counting its calls. One fetch returns the first scripted outcome, and the print shows attempt one failing.

```
  let results = ["failure", "failure", "success"]
+ struct FakeTransport {
+     var results: [String]
+     var calls = 0
+     mutating func fetch() -> String {
+         calls += 1
+         return results[calls - 1]
+     }
+ }
+ var transport = FakeTransport(results: results)
+ let first = transport.fetch()
+ print("Attempt 1: \(first)")
```

OUTPUT SO FAR

```
Attempt 1: failure
```

Frame 3 of 6

A bounded loop replaces the single attempt: it fetches, logs each attempt, and breaks the moment a result is success. Three attempts appear, the third succeeding.

```
   let results = ["failure", "failure", "success"]
   struct FakeTransport {
       var results: [String]
       var calls = 0
       mutating func fetch() -> String {
           calls += 1
           return results[calls - 1]
       }
   }
   var transport = FakeTransport(results: results)
+ for attempt in 1...3 {
+     let result = transport.fetch()
+     print("Attempt \(attempt): \(result)")
+     if result == "success" {
+         break
+     }
+ }
```

OUTPUT SO FAR

```
Attempt 1: failure
Attempt 2: failure
Attempt 3: success
```

Frame 4 of 6

Backoff joins the loop, but as data: each failed attempt appends the current delay and doubles it. The printed list records 200 then 400, with no actual waiting.

```
    let results = ["failure", "failure", "success"]
    struct FakeTransport {
        var results: [String]
        var calls = 0
        mutating func fetch() -> String {
            calls += 1
            return results[calls - 1]
        }
    }
    var transport = FakeTransport(results: results)
+ var delays: [Int] = []
+ var delay = 200
+ for attempt in 1...3 {
+     let result = transport.fetch()
+     print("Attempt \(attempt): \(result)")
+     if result == "success" {
+         break
+     }
+     delays.append(delay)
+     delay *= 2
+ }
+ print("Delays: \(delays)")
```

OUTPUT SO FAR

```
Attempt 1: failure
Attempt 2: failure
Attempt 3: success
Delays: [200, 400]
```

Frame 5 of 6

The successful attempt now stores a payload before breaking, and the program prints it after the delays. The payload line proves the loop kept the winning result.

```
  let results = ["failure", "failure", "success"]
  struct FakeTransport {
      var results: [String]
      var calls = 0
      mutating func fetch() -> String {
          calls += 1
          return results[calls - 1]
      }
  }
  var transport = FakeTransport(results: results)
  var delays: [Int] = []
  var delay = 200
+ var payload = ""
+ for attempt in 1...3 {
+     let result = transport.fetch()
+     print("Attempt \(attempt): \(result)")
+     if result == "success" {
+         payload = "Weather for Lisbon"
+         break
+     }
+     delays.append(delay)
+     delay *= 2
+ }
+ print("Delays: \(delays)")
+ print("Payload: \(payload)")
```

OUTPUT SO FAR

```
Attempt 1: failure
Attempt 2: failure
Attempt 3: success
Delays: [200, 400]
Payload: Weather for Lisbon
```

Frame 6 of 6

A final print reads the transport call count: three calls. The loop stopped because an attempt succeeded, not because it ran out of tries, and the log shows it.

```
  let results = ["failure", "failure", "success"]
  struct FakeTransport {
      var results: [String]
      var calls = 0
      mutating func fetch() -> String {
          calls += 1
          return results[calls - 1]
      }
  }
  var transport = FakeTransport(results: results)
  var delays: [Int] = []
  var delay = 200
  var payload = ""
  for attempt in 1...3 {
      let result = transport.fetch()
      print("Attempt \(attempt): \(result)")
      if result == "success" {
          payload = "Weather for Lisbon"
          break
      }
      delays.append(delay)
      delay *= 2
  }
  print("Delays: \(delays)")
  print("Payload: \(payload)")
+ print("Calls: \(transport.calls)")
```

OUTPUT SO FAR

```
Attempt 1: failure
Attempt 2: failure
Attempt 3: success
Delays: [200, 400]
Payload: Weather for Lisbon
Calls: 3
```

FINAL RESULT — CHECKED OUTPUT

```
Attempt 1: failure
Attempt 2: failure
Attempt 3: success
Delays: [200, 400]
Payload: Weather for Lisbon
Calls: 3
```

Line by line: the finished program

```
let results = ["failure", "failure", "success"]
```

WHO	Your code, scripting the fake transport in advance.
WHAT	**Stores the three scripted outcomes in the order attempts will meet them.**
WHERE	The first line of the program, above the transport type.
WHEN	At program start, before any attempt is made.
WHY	Scripted data keeps the example deterministic: no randomness, no network.

```
2  struct FakeTransport {
```

WHO You declare it; the program creates one value of it.
WHAT **Declares a fake transport that replays scripted results instead of networking.**
WHERE At file scope, above the code that creates it.
WHEN Compiled once; one value is created below.
WHY A stand-in transport is the testing seam this module teaches.

```
3  var results: [String]
```

WHO Each FakeTransport value, which stores its own script.
WHAT **Stores the scripted outcomes this transport will replay in order.**
WHERE The first stored property inside the FakeTransport struct.
WHEN Set when the transport is created, from the script above.
WHY The script is data the caller supplies, so scenarios are easy to change.

```
4  var calls = 0
```

WHO Each FakeTransport value, which counts its own attempts.
WHAT **Counts how many times fetch has been called on this transport.**
WHERE The second stored property inside the FakeTransport struct.
WHEN Starts at zero and grows by one on every fetch call.
WHY The call count is the index into the script, so order is guaranteed.

```
5  mutating func fetch() -> String {
```

WHO Callers of the transport, one attempt at a time.
WHAT **Declares the one fake request: it returns the next scripted outcome.**
WHERE A method inside FakeTransport, below its stored properties.
WHEN Once per attempt, whenever retry code asks for a result.
WHY mutating lets the method advance the call count stored in the struct.

```
6  calls += 1
```

WHO The fetch method, updating its own transport value.
WHAT **Adds one to the call count before the result is read.**
WHERE The first statement inside fetch, before the return line.
WHEN At the start of every fetch call, once per attempt.
WHY Counting first means calls minus one indexes the outcome just used.

```
7  return results[calls - 1]
```

WHO The fetch method, handing back the scripted outcome.
WHAT **Returns the scripted result for this call number.**
WHERE The last statement inside fetch, after the count grows.
WHEN Once per attempt, after the counter has advanced.
WHY Indexing with calls minus one replays the script in its written order.

```
10 var transport = FakeTransport(results: results)
```

WHO Your code, creating the transport for this run.
WHAT **Creates the fake transport and hands it the scripted results.**
WHERE Top-level code below the transport type declaration.
WHEN At program start, before the first attempt is made.
WHY It is var because fetch mutates the value as calls are counted.

```
11 var delays: [Int] = []
```

WHO Your code, preparing the record of planned waits.
WHAT **Creates an empty list that will hold each delay the loop would wait.**
WHERE Top-level code, before the retry loop starts.
WHEN Before the loop, while it is still empty.
WHY Delays are recorded as data and printed, never actually slept through.

```
12 var delay = 200
```

WHO Your code, setting the first backoff wait.
WHAT **Stores the first planned wait: 200 milliseconds.**
WHERE Top-level code, beside the delays list it feeds.
WHEN Before the loop, and doubled after every failed attempt.
WHY Starting at 200 matches the backoff lesson: 200, 400, 800.

```
13 var payload = ""
```

WHO Your code, preparing storage for the successful result.
WHAT **Creates the payload text, empty until an attempt succeeds.**
WHERE Top-level code, before the retry loop starts.
WHEN Before the loop; filled only on the successful attempt.
WHY An empty starting value makes a total failure visible in the output.

```
14 for attempt in 1...3 {
```

WHO The retry loop, written by your code.
WHAT **Opens a loop that tries at most three attempts, numbered from one.**
WHERE Top-level code, wrapping the attempt statements below.
WHEN Runs until an attempt succeeds or three attempts are used.
WHY A bounded loop is the promise that retrying always gives up eventually.

```
15 let result = transport.fetch()
```

WHO The transport, replaying the next scripted outcome.
WHAT **Fetches one outcome and stores it for the checks below.**
WHERE The first statement inside the retry loop body.
WHEN Once per attempt, at the top of each iteration.
WHY Every attempt goes through the same fetch, scripted or real.

```
16 print("Attempt \(attempt): \(result)")
```

WHO The print function, logging inside the loop.
WHAT **Prints the attempt number and its outcome as a log line.**
WHERE Inside the loop, right after the fetch returns.
WHEN Once per attempt, before the success check runs.
WHY An attempt log is how retry behavior stays visible while testing.

```
17 if result == "success" {
```

WHO Your code, checking the outcome of this attempt.
WHAT **Opens the success branch that runs only when the result is success.**
WHERE Inside the loop, after the attempt log line.
WHEN Once per attempt, after its outcome is known.
WHY Success ends the work; failure falls through to the backoff lines.

```
18 payload = "Weather for Lisbon"
```

WHO Your code, keeping the successful result.
WHAT **Stores the payload text that the successful attempt delivered.**
WHERE Inside the success branch, before the break line.
WHEN Only on the attempt whose result is success.
WHY Later code reads the payload, not the raw per-attempt result.

```
19 break
```

WHO The break statement, controlled by your success check.
WHAT **Leaves the retry loop immediately, skipping any further attempts.**
WHERE Inside the success branch, as its last statement.
WHEN Only when an attempt succeeds, never after a failure.
WHY Breaking on success is what makes the attempt count stop at three here.

```
21 delays.append(delay)
```

WHO Your code, recording the planned wait.
WHAT **Appends the current delay to the record of waits.**
WHERE Inside the loop, after the success branch did not run.
WHEN After each failed attempt, before the delay doubles.
WHY Recording instead of sleeping keeps the run instant and deterministic.

```
22 delay *= 2
```

WHO Your code, growing the backoff wait.
WHAT **Doubles the delay for the next failure: 200 becomes 400.**
WHERE Inside the loop, as the last statement of the body.
WHEN After each failed attempt, once its delay was recorded.
WHY Doubling is the exponential part of exponential backoff.

```
24 print("Delays: \(delays)")
```

WHO The print function from the Swift standard library.
WHAT **Prints the recorded delays as one list, in the order planned.**
WHERE Top-level code, right after the retry loop ends.
WHEN After the loop, once a success or the bound stopped it.
WHY The list proves the waits grew: 200 then 400 milliseconds.

```
25 print("Payload: \(payload)")
```

WHO The print function from the Swift standard library.
WHAT **Prints the payload that the successful attempt delivered.**
WHERE Top-level code, after the delays line is printed.
WHEN After the loop, when the payload holds its final value.
WHY A filled payload line is the visible sign the retries worked.

```
26 print("Calls: \(transport.calls)")
```

WHO The print function, reading the transport counters.
WHAT **Prints how many fetch calls the transport actually received.**
WHERE The last line of the program, after the payload line.
WHEN Last, after the loop has finished all its attempts.
WHY Three calls confirms the loop stopped at success, not at its bound.

Try it

The starter makes one attempt against the scripted transport and stops at its failure. Change it to loop up to three times like the example, recording delays, and print the delays list after the loop.

```
let results = ["failure", "failure", "success"]
struct FakeTransport {
    var results: [String]
    var calls = 0
    mutating func fetch() -> String {
        calls += 1
        return results[calls - 1]
    }
}
var transport = FakeTransport(results: results)
let first = transport.fetch()
print("Attempt 1: \(first)")
```

EXPECTED OUTPUT

```
Attempt 1: failure
Attempt 2: failure
Attempt 3: success
Delays: [200, 400]
```

WORKED EXAMPLE

Cache: miss, fresh hit, stale

Problem.

Build a tiny weather cache keyed by city, with a thirty minute freshness window and a logical clock the program advances by assignment. Show a miss that fetches, a fresh hit that does not, and a stale entry that refetches.

Frame 1 of 6

The cache starts as three pieces of state: a values dictionary, a times dictionary, and a fetch counter. The print proves it starts completely empty.

```
  var values: [String: String] = [:]
  var times: [String: Int] = [:]
  var fetches = 0
  print("Cached cities: \(values.count)")
```

OUTPUT SO FAR

```
Cached cities: 0
```

Frame 2 of 6

A lookup function appears in its simplest form: a cached city returns HIT, anything else fetches, stores, and returns MISS. At minute zero Lisbon misses and fetches.

```
  var values: [String: String] = [:]
  var times: [String: Int] = [:]
  var fetches = 0
+ func lookup(city: String, now: Int) -> String {
+     if let value = values[city] {
+         return "HIT \(city): \(value)"
+     }
+     fetches += 1
+     values[city] = "sunny 21"
+     times[city] = now
+     return "MISS \(city): fetched sunny 21"
+ }
+ var now = 0
+ print(lookup(city: "Lisbon", now: now))
```

OUTPUT SO FAR

```
MISS Lisbon: fetched sunny 21
```

Frame 3 of 6

The clock advances to minute ten and the same lookup runs again. This time Lisbon is cached, so the report is HIT and no fetch lines run at all.

```
  var values: [String: String] = [:]
  var times: [String: Int] = [:]
  var fetches = 0
  func lookup(city: String, now: Int) -> String {
      if let value = values[city] {
          return "HIT \(city): \(value)"
      }
      fetches += 1
      values[city] = "sunny 21"
      times[city] = now
      return "MISS \(city): fetched sunny 21"
  }
  var now = 0
  print(lookup(city: "Lisbon", now: now))
+ now = 10
+ print(lookup(city: "Lisbon", now: now))
```

OUTPUT SO FAR

```
MISS Lisbon: fetched sunny 21
HIT Lisbon: sunny 21
```

Frame 4 of 6

The lookup gains its freshness rule: a cached entry returns HIT only within thirty minutes. At minute forty-five the entry is stale, so the program refetches and stores sunny 22.

```
  var values: [String: String] = [:]
  var times: [String: Int] = [:]
  var fetches = 0
  func lookup(city: String, now: Int) -> String {
+     if let value = values[city], let stored = times[city] {
+         if now - stored <= 30 { return "HIT \(city): \(value)" }
+         fetches += 1
+         values[city] = "sunny 22"
+         times[city] = now
+         return "STALE \(city): refetched sunny 22"
+ }
+     fetches += 1
+     values[city] = "sunny 21"
+     times[city] = now
+     return "MISS \(city): fetched sunny 21"
+ }
+ var now = 0
+ print(lookup(city: "Lisbon", now: now))
+ now = 10
+ print(lookup(city: "Lisbon", now: now))
+ now = 45
+ print(lookup(city: "Lisbon", now: now))
```

OUTPUT SO FAR

```
MISS Lisbon: fetched sunny 21
HIT Lisbon: sunny 21
STALE Lisbon: refetched sunny 22
```

Frame 5 of 6

The fetch counter prints after the timeline: two fetches. The miss fetched, the fresh hit did not, and the stale lookup did, which is exactly the promised behavior.

```
var values: [String: String] = [:]
var times: [String: Int] = [:]
var fetches = 0
func lookup(city: String, now: Int) -> String {
    if let value = values[city], let stored = times[city] {
        if now - stored <= 30 { return "HIT \(city): \(value)" }
        fetches += 1
        values[city] = "sunny 22"
        times[city] = now
        return "STALE \(city): refetched sunny 22"
}
    fetches += 1
    values[city] = "sunny 21"
    times[city] = now
    return "MISS \(city): fetched sunny 21"
}
var now = 0
print(lookup(city: "Lisbon", now: now))
now = 10
print(lookup(city: "Lisbon", now: now))
now = 45
print(lookup(city: "Lisbon", now: now))
+ print("Fetches: \(fetches)")
```

OUTPUT SO FAR

```
MISS Lisbon: fetched sunny 21
HIT Lisbon: sunny 21
STALE Lisbon: refetched sunny 22
Fetches: 2
```

Frame 6 of 6

One more lookup, for Porto at minute fifty, misses on its own key and raises the count to three. Cache entries are per city; Lisbon being fresh never helps Porto.

```
    var values: [String: String] = [:]
    var times: [String: Int] = [:]
    var fetches = 0
    func lookup(city: String, now: Int) -> String {
        if let value = values[city], let stored = times[city] {
            if now - stored <= 30 { return "HIT \(city): \(value)" }
            fetches += 1
            values[city] = "sunny 22"
            times[city] = now
            return "STALE \(city): refetched sunny 22"
    }
        fetches += 1
        values[city] = "sunny 21"
        times[city] = now
        return "MISS \(city): fetched sunny 21"
    }
    var now = 0
    print(lookup(city: "Lisbon", now: now))
    now = 10
    print(lookup(city: "Lisbon", now: now))
    now = 45
    print(lookup(city: "Lisbon", now: now))
+ now = 50
+ print(lookup(city: "Porto", now: now))
+ print("Fetches: \(fetches)")
```

OUTPUT SO FAR

```
MISS Lisbon: fetched sunny 21
HIT Lisbon: sunny 21
STALE Lisbon: refetched sunny 22
MISS Porto: fetched sunny 21
Fetches: 3
```

FINAL RESULT — CHECKED OUTPUT

```
MISS Lisbon: fetched sunny 21
HIT Lisbon: sunny 21
STALE Lisbon: refetched sunny 22
MISS Porto: fetched sunny 21
Fetches: 3
```

Line by line: the finished program

```
1   var values: [String: String] = [:]
```

WHO	Your code, creating the cache storage.
WHAT	**Creates the dictionary that maps each city to its cached weather text.**
WHERE	The first line of the program, at file scope.
WHEN	At program start, empty, before any lookup runs.
WHY	A dictionary keyed by city is the whole storage idea of a small cache.

```
2  var times: [String: Int] = [:]
```

WHO Your code, creating the timestamp storage.
WHAT **Creates the dictionary that maps each city to the minute it was stored.**
WHERE Beside the values dictionary, at file scope.
WHEN At program start, empty, before any lookup runs.
WHY Freshness needs a stored time beside every stored value.

```
3  var fetches = 0
```

WHO Your code, counting real fetches for proof.
WHAT **Counts how many times a lookup actually fetched new data.**
WHERE Beside the two cache dictionaries, at file scope.
WHEN Starts at zero and grows only on a miss or a stale refetch.
WHY The counter is the evidence that a fresh hit did no fetching.

```
4  func lookup(city: String, now: Int) -> String {
```

WHO Screens that need weather, calling through this function.
WHAT **Declares the one lookup entry point: a city and a logical time go in, a report line comes out.**
WHERE At file scope, below the cache storage it uses.
WHEN Once per lookup, whenever weather is asked for.
WHY One entry point means the freshness rule lives in exactly one place.

```
5  if let value = values[city], let stored = times[city] {
```

WHO The lookup function, checking value and time together.
WHAT **Opens the cached branch only when both a value and its stored time exist.**
WHERE The first statement inside lookup, before any fetching.
WHEN On every lookup, before the miss path can run.
WHY Freshness can only be judged when the value and its age are both known.

```
6  if now - stored <= 30 { return "HIT \(city): \(value)" }
```

WHO The lookup function, applying the freshness window.
WHAT **Returns the fresh hit at once when the entry is at most thirty minutes old.**
WHERE Inside the cached branch, as a single-line freshness check.
WHEN On every cached lookup, after the age is compared.
WHY Thirty minutes is the freshness window this cache promises.

```
7  fetches += 1
```

WHO The lookup function, counting a real fetch.
WHAT **Adds one to the fetch counter for a stale refetch.**
WHERE Inside the cached branch, after the freshness check failed.
WHEN Only on a stale lookup, before the new value is stored.
WHY Counting refetches separately from hits keeps the proof honest.

```
8  values[city] = "sunny 22"
```

WHO The lookup function, storing the refetched value.
WHAT **Stores the new weather text fetched for the stale city.**
WHERE Inside the cached branch, beside its timestamp update.
WHEN Only on a stale lookup, after the fetch counter grows.
WHY The refetched value replaces the stale one in the same slot.

```
9   times[city] = now
```

WHO The lookup function, stamping the refetched value.
WHAT **Records the current logical minute as the new stored time.**
WHERE Inside the cached branch, right after the new value.
WHEN Only on a stale lookup, together with the value store.
WHY Value and time must move together or the age math lies.

```
10  return "STALE \(city): refetched sunny 22"
```

WHO The lookup function, reporting the stale path.
WHAT **Returns the stale report naming the refetched value.**
WHERE Inside the cached branch, as its last statement.
WHEN Only on a stale lookup, after the refetch is stored.
WHY The word STALE in the output makes the third cache state visible.

```
12  fetches += 1
```

WHO The lookup function, counting a real fetch.
WHAT **Adds one to the fetch counter for a first-time fetch.**
WHERE In the miss path of lookup, after the cached branch closes.
WHEN Only when the city has no cached entry at all.
WHY A miss always fetches; the counter records that it happened.

```
13  values[city] = "sunny 21"
```

WHO The lookup function, storing a fetched value.
WHAT **Stores the weather text fetched on a miss.**
WHERE In the miss path, beside its timestamp store.
WHEN Only on a miss, after the fetch counter grows.
WHY Storing on a miss is what turns the next lookup into a hit.

```
14  times[city] = now
```

WHO The lookup function, stamping the fetched value.
WHAT **Records the current logical minute as the stored time.**
WHERE In the miss path, right after the value store.
WHEN Only on a miss, together with the value store.
WHY Without a stored time, no later lookup could judge freshness.

```
15  return "MISS \(city): fetched sunny 21"
```

WHO The lookup function, reporting the miss path.
WHAT **Returns the miss report naming the value that was fetched.**
WHERE The last statement inside lookup, in the miss path.
WHEN Only on a miss, after the fetch has been stored.
WHY The word MISS in the output makes the first cache state visible.

```
17  var now = 0
```

WHO Your code, playing the clock by hand.
WHAT **Creates the logical clock variable and sets it to minute zero.**
WHERE Top-level code below the lookup function.
WHEN Before the first lookup, at the start of the timeline.
WHY Time is plain integer data here, advanced by assignment, never by waiting.

```
18 print(lookup(city: "Lisbon", now: now))
```

WHO The print function, showing one lookup report.
WHAT **Runs one lookup for Lisbon at the current minute and prints its report.**
WHERE Top-level code, in the timeline sequence of lookups.
WHEN Once per timeline step, at whatever minute now holds.
WHY Each printed report is one cache state: miss, hit or stale.

```
19 now = 10
```

WHO Your code, advancing the logical clock.
WHAT **Moves the logical time to minute ten by assignment.**
WHERE Top-level code, between the first and second lookups.
WHEN After the first lookup and before the second one.
WHY Ten minutes is inside the thirty minute freshness window.

```
20 print(lookup(city: "Lisbon", now: now))
```

WHO The print function, showing one lookup report.
WHAT **Runs one lookup for Lisbon at the current minute and prints its report.**
WHERE Top-level code, in the timeline sequence of lookups.
WHEN Once per timeline step, at whatever minute now holds.
WHY Each printed report is one cache state: miss, hit or stale.

```
21 now = 45
```

WHO Your code, advancing the logical clock.
WHAT **Moves the logical time to minute forty-five by assignment.**
WHERE Top-level code, between the second and third lookups.
WHEN After the fresh hit and before the stale check.
WHY Forty-five minutes is past the thirty minute freshness window.

```
22 print(lookup(city: "Lisbon", now: now))
```

WHO The print function, showing one lookup report.
WHAT **Runs one lookup for Lisbon at the current minute and prints its report.**
WHERE Top-level code, in the timeline sequence of lookups.
WHEN Once per timeline step, at whatever minute now holds.
WHY Each printed report is one cache state: miss, hit or stale.

```
23 now = 50
```

WHO Your code, advancing the logical clock.
WHAT **Moves the logical time to minute fifty by assignment.**
WHERE Top-level code, before the last lookup in the program.
WHEN After the stale refetch stored its new value at forty-five.
WHY Five minutes after a refetch, the new entry is fresh again.

```
24 print(lookup(city: "Porto", now: now))
```

WHO The print function, showing one lookup report.
WHAT **Runs one lookup for Porto at the current minute and prints its report.**
WHERE Top-level code, after the Lisbon timeline finishes.
WHEN Last lookup, at minute fifty, for a city never cached.
WHY A different key misses on its own; cache entries are per city.

```
25 print("Fetches: \(fetches)")
```

WHO The print function from the Swift standard library.

WHAT **Prints the total number of real fetches the lookups caused.**

WHERE The last line of the program, after every lookup.

WHEN Last, once the whole timeline has run.

WHY This count is the proof: fresh hits added nothing to it.

Try it

The starter computes the age of an entry stored at minute zero and checked at minute forty-five, and prints only the age. Add the freshness check and print the verdict word, HIT or STALE, after the age line.

```
let storedAt = 0
let now = 45
let age = now - storedAt
print("Age: \(age)")
```

EXPECTED OUTPUT

```
Age: 45
STALE
```

MODULE 09

Architecture, Testing and Shipping

Structure an app with MVVM and dependency injection, prove it works with Swift Testing and UI tests, find bugs with the debugger and Instruments, and take it through TestFlight and App Review to the App Store.

- Create various types of iOS applications in Swift
- Understand the essentials of Apple development and its ecosystem

Architecture That Survives Change

CONCEPT

Give every piece one job

An app that works today still has to survive new features, a redesign and a second developer. **Architecture** is how you divide code so each piece has one job and one reason to change: **separation of concerns**.

MVVM (Model, View, View Model) is one common way to organise a SwiftUI app: the view model is a plain `@Observable` class that holds a screen's state and logic so the view stays simple. Small screens do not need one; screens with loading, errors, or several sources of data usually do. Each layer talks only to its neighbor, so you can redesign a screen without touching networking.

View	A SwiftUI struct that draws the screen from current data and forwards taps. No rules, no networking.
View model	An @Observable class that holds screen state, applies the rules (limits, validation, formatting) and calls services.
Model	Plain data such as a StudySession struct. It knows nothing about screens.
Service	Code that talks to the outside world (network, location, storage) behind a protocol, so tests can replace it.

> **Where this came from**
>
> UIKit apps used MVC, and view controllers often grew into thousand-line files. Before iOS 17, SwiftUI view models used `ObservableObject` and `@Published`. The `@Observable` macro replaces both with less code and more precise updates.

CODE · RUNS IN THE BROWSER

A view model is plain Swift

The rules of a focus timer live in a class that does not import SwiftUI. It ignores bad input, caps the total and turns numbers into a status message. Nothing here draws pixels, so it runs anywhere: in the console below, in a test, or behind any screen.

```swift
final class FocusModel {
    let goal = 60
    private(set) var logged = 0

    var status: String {
        logged >= goal ? "Goal reached" : "\(goal - logged) min left"
    }

    func log(_ minutes: Int) {
        guard minutes > 0 else { return }
        logged = min(logged + minutes, goal)
    }
}

let model = FocusModel()
model.log(25)
print(model.status)
model.log(-10)
print(model.status)
model.log(50)
print(model.status, model.logged)
```

OUTPUT

```
35 min left
35 min left
Goal reached 60
```

Line by line

1 `final class FocusModel {`

- **WHO** You declare it; the Swift compiler checks it and the runtime allocates each instance on the heap.
- **WHAT** **Declares FocusModel, the view model that owns the focus timer's state and rules.**
- **WHERE** At file scope, so any view or test in the app module can create one.
- **WHEN** Compiled once; an instance appears later, when a screen or a test calls FocusModel().
- **WHY** A class gives one shared instance that a view can observe. final says no subclass will override it, which documents intent and lets the compiler call methods directly.

2 `let goal = 60`

- **WHO** Each FocusModel instance stores its own copy.
- **WHAT** **A constant goal of 60 minutes for the session.**
- **WHERE** A stored property inside the object, on the heap with the rest of the instance.
- **WHEN** Set when the instance is created and never changed afterwards.
- **WHY** let documents that the goal is fixed for this screen. A later version could pass it in through an initializer.

3 `private(set) var logged = 0`

- **WHO** FocusModel alone may write it; any other code may read it.
- **WHAT** **Minutes logged so far, starting at zero, with a private setter.**
- **WHERE** A stored property of the instance; private(set) restricts writes to code inside this class.
- **WHEN** Read whenever something asks for status; changed only inside log(_:).
- **WHY** Forcing every change through one method means a view can never bypass the rules by writing logged = 999.

```
5  var status: String {
```

WHO Whoever reads model.status, usually a SwiftUI view's body.
WHAT **A computed property that builds the message from the current numbers.**
WHERE Inside FocusModel. It stores nothing; its body runs on every read.
WHEN Each time status is read, so it can never be stale.
WHY Formatting is presentation logic. Keeping it here means the view only displays a string and the rule can be tested without a screen.

```
6  logged >= goal ? "Goal reached" : "\(goal - logged) min left"
```

WHO The ternary operator, evaluated by the running program.
WHAT **Returns "Goal reached" once the goal is met, otherwise the remaining minutes inserted into text with interpolation.**
WHERE The single expression of the computed property, so Swift returns it without a return keyword.
WHEN Each time something reads status, so the message always matches the current numbers.
WHY One side-effect-free expression is easy to read and easy to cover with one test per branch.

```
9  func log(_ minutes: Int) {
```

WHO Called by the view when the user taps a button, or by a test.
WHAT **Declares the only way to add minutes. The underscore lets callers write log(25) without a label.**
WHERE An instance method of FocusModel. Classes are reference types, so it can change logged without the mutating keyword.
WHEN Once per call, synchronously, before the caller's next line runs.
WHY A single entry point is where validation lives, so every caller gets the same rules.

```
10  guard minutes > 0 else { return }
```

WHO The guard statement, checked at run time.
WHAT **Leaves the method immediately when the input is zero or negative.**
WHERE First line of log(_:), before any state changes.
WHEN On every call, before logged is touched.
WHY Rejecting bad input first keeps the main path unindented and makes the invalid case explicit, and therefore testable.

```
11  logged = min(logged + minutes, goal)
```

WHO FocusModel, updating its own stored property.
WHAT **Adds the minutes but caps the total at the goal with min.**
WHERE Inside log(_:), after validation passed.
WHEN Only for positive input.
WHY Capping is a business rule. Enforcing it here means no screen can ever show "-15 min left".

```
15  let model = FocusModel()
```

WHO Top-level code, standing in for a screen or a test.
WHAT **Creates one FocusModel and keeps a reference to it in model.**
WHERE At the top level of this program; the object itself lives on the heap.
WHEN When the program reaches this line.
WHY let fixes the reference, not the object, so the object's var properties can still change through its methods.

```
16  model.log(25)
```

WHO The caller, simulating a tap on a "Log 25 minutes" button.
WHAT **Logs 25 valid minutes, so logged becomes 25.**
WHERE A method call on the single shared instance.
WHEN Right after the model is created.
WHY Calling the method directly is exactly what a unit test does in the next lesson.

```
17 print(model.status)
```

WHO — Swift's print function, writing to standard output.
WHAT — **Prints "35 min left", because 60 minus 25 is 35.**
WHERE — The console panel below the code.
WHEN — After the first log call.
WHY — Reading the computed property checks the formatting rule with no user interface at all.

```
18 model.log(-10)
```

WHO — The caller, simulating bad input such as a typo.
WHAT — **Asks to log negative minutes; the guard rejects it, so nothing changes.**
WHERE — The same instance, so a change here would be visible to every reader.
WHEN — Third statement of the program.
WHY — Trying invalid input on purpose is how you confirm a rule actually protects the data.

```
19 print(model.status)
```

WHO — print again, reading the same computed property.
WHAT — **Prints "35 min left" a second time, proving the negative value was ignored.**
WHERE — Standard output: the first line in the console panel below the code.
WHEN — After the rejected call.
WHY — Printing the same value twice is a tiny regression check: if someone deletes the guard, this line changes.

```
20 model.log(50)
```

WHO — The caller, logging more minutes than remain.
WHAT — **Asks for 50 more minutes; 25 plus 50 is 75, which min caps to 60.**
WHERE — The same shared instance that already holds 25 logged minutes.
WHEN — Fifth statement, after the rejected negative input.
WHY — Testing just past a limit (a boundary value) is where off-by-one bugs usually hide.

```
21 print(model.status, model.logged)
```

WHO — print with two items, which it separates with a space.
WHAT — **Prints "Goal reached 60": the message and the capped total.**
WHERE — Standard output: the third and last line in the console panel.
WHEN — Last line of the program.
WHY — Showing both the message and the raw number confirms the cap and the message agree.

> **Try it**
>
> 1. Change goal to 90 and predict all three lines before you run it.
> 2. Delete the guard line. What does the second line print now, and which rule broke?

CODE · LIVE SWIFTUI PREVIEW

Connect the view model to a view

The same model now carries `@Observable`, and a view shows it. The view reads `status` and forwards the tap; it contains no rules. Tap the button in the phone: SwiftUI sees that `logged` changed and redraws only what reads it.

```swift
import SwiftUI

@Observable final class FocusModel {
    let goal = 60
    private(set) var logged = 0
    var status: String {
        logged >= goal ? "Goal reached" : "\(goal - logged) min left"
    }
    func log(_ minutes: Int) { logged = min(logged + minutes, goal) }
}

struct FocusView: View {
    @State private var model = FocusModel()
    var body: some View {
        VStack(spacing: 16) {
            Text(model.status).font(.title2)
            Button("Log 25 minutes") { model.log(25) }
                .buttonStyle(.borderedProminent)
        }
    }
}

#Preview { FocusView() }
```

Line by line

1 `import SwiftUI`

WHO	The Swift compiler, while it builds the app.
WHAT	**Loads SwiftUI, which also re-exports the Observation framework that provides @Observable.**
WHERE	First line of the file; it applies only to this file.
WHEN	At compile time, before any of your code runs.
WHY	Views, @State and the @Observable macro are framework features, not part of the core language.

3 `@Observable final class FocusModel {`

WHO	The @Observable macro, which rewrites the class at compile time.
WHAT	**Turns every stored property into a tracked property so SwiftUI knows exactly which views read it.**
WHERE	Attached to the class declaration at file scope.
WHEN	Expanded during compilation; tracking happens at run time on each property read.
WHY	Fine-grained tracking means a change to logged redraws only views that read logged or status, not the whole screen.

4 `let goal = 60`

WHO	Each FocusModel instance, which stores its own constant goal.
WHAT	**The fixed 60-minute goal.**
WHERE	Stored property; let properties are not tracked because they cannot change.
WHEN	Set once when the instance is created; @Observable has nothing to track.
WHY	Constants need no observation, so the macro leaves them alone.

```
5  private(set) var logged = 0
```

WHO The model owns writes; the view may only read.
WHAT **Minutes logged, now observable.**
WHERE Tracked stored property of the class.
WHEN Read during body; written by log(_:).
WHY private(set) keeps the rule from the previous step: views cannot change the total directly.

```
6  var status: String {
```

WHO The view's body, which reads it.
WHAT **The computed message, unchanged from the console version.**
WHERE A computed property; SwiftUI records that it reads logged underneath.
WHEN Every time body runs, because the Text below reads status.
WHY Because status reads logged, any change to logged invalidates every view that showed status.

```
7  logged >= goal ? "Goal reached" : "\(goal - logged) min left"
```

WHO The ternary operator, evaluated by the running app.
WHAT **Chooses the finished message or the minutes remaining.**
WHERE Body of the computed property.
WHEN On each read of status, which happens during each body call.
WHY The rule is identical to the tested console version; only the observation wrapper is new.

```
9  func log(_ minutes: Int) { logged = min(logged + minutes, goal) }
```

WHO Called from the button's action closure.
WHAT **Adds minutes with the cap; the guard from the previous step is left out here to save space.**
WHERE An instance method of the model.
WHEN On every tap of the button, after SwiftUI runs its action closure.
WHY Assigning logged triggers observation, so SwiftUI schedules a redraw automatically. No manual refresh call exists or is needed.

```
12 struct FocusView: View {
```

WHO You declare it; SwiftUI calls it.
WHAT **A view type that describes the focus screen.**
WHERE File scope; a lightweight struct that SwiftUI may create many times.
WHEN Created whenever a parent view's body runs.
WHY Views are cheap value descriptions. Long-lived state belongs in the model, not in the struct itself.

```
13 @State private var model = FocusModel()
```

WHO SwiftUI, which stores the value outside the struct.
WHAT **Creates the model once and keeps it alive for as long as this view is on screen.**
WHERE Inside the view; @State storage lives in SwiftUI's memory, not in the struct.
WHEN Initialized the first time the view appears; reused on every later body call.
WHY @State is the right owner for an @Observable model the view creates itself. Without it, each redraw could create a fresh model and lose progress.

```
14 var body: some View {
```

WHO SwiftUI calls it whenever tracked data it read last time changes.
WHAT **Describes what the screen should look like right now.**
WHERE The View protocol's only requirement.
WHEN At first appearance and after each relevant change.
WHY body must be fast and free of side effects because SwiftUI may call it often.

```
15 VStack(spacing: 16) {
```

WHO SwiftUI's layout system, arranging the text and the button.
WHAT **Stacks the text and the button vertically, 16 points apart.**
WHERE The root of this view's content.
WHEN Rebuilt on each body call (cheaply).
WHY Layout is the view's real job; it is the only kind of decision left in this struct.

```
16 Text(model.status).font(.title2)
```

WHO SwiftUI renders the string; Observation records the read.
WHAT **Shows the model's message in a large font.**
WHERE First child of the VStack, above the button.
WHEN On every body call; Observation re-runs body after logged changes.
WHY The view displays a ready-made string instead of computing it, so there is nothing here to test.

```
17 Button("Log 25 minutes") { model.log(25) }
```

WHO The user taps; SwiftUI runs the closure.
WHAT **A button whose action forwards the intent to the model.**
WHERE Inside the stack, below the text.
WHEN Every time the user taps it, which then triggers one redraw.
WHY The view says what happened (a tap); the model decides what it means. That split is what a view model is for.

```
18 .buttonStyle(.borderedProminent)
```

WHO SwiftUI's styling system, applied to this one button.
WHAT **Draws the button as a filled, prominent button.**
WHERE A modifier applied to the Button.
WHEN Applied on every render; it has no effect on the model or its rules.
WHY Styling is a view concern, so it sits here and never in the model.

```
23 #Preview { FocusView() }
```

WHO Xcode's preview canvas (here, the phone beside the code).
WHAT **Registers a preview that shows FocusView.**
WHERE File scope; previews are compiled only for development tools, never shipped as a screen.
WHEN Whenever you open the canvas or edit the file.
WHY A preview is a fast visual check of one view in isolation, which is only easy when the view does not create its own network or database.

> **Try it**
>
> 1. Tap the button three times. Why does the text stop at "Goal reached" instead of going negative?
> 2. Add a second button that logs 5 minutes. Which type did you change, and which did you not need to touch?

COMPARE

Rules in the view vs. rules in the model

Both versions draw the same screen. On the left, the 60-minute rule and the message format live inside `body`. On the right, the view only displays and forwards. Ask yourself what you would edit to change the goal, and how you would test the rule.

A: Before: rules buried in the view

```swift
import SwiftUI

struct FocusView: View {
    @State private var logged = 0

    var body: some View {
        VStack {
            Text(logged >= 60 ? "Goal reached"
                 : "\(60 - logged) min left")
            Button("Log 25 minutes") {
                logged = min(logged + 25, 60)
            }
        }
    }
}
```

1 `import SwiftUI`

- **WHO** The Swift compiler, while it builds this file.
- **WHAT** **Imports SwiftUI for View, Text, Button and @State.**
- **WHERE** First line of the view's file; it applies to this file only.
- **WHEN** At compile time, before the app ever runs.
- **WHY** Needed for any SwiftUI view.

3 `struct FocusView: View {`

- **WHO** You declare it; SwiftUI renders it.
- **WHAT** **A focus screen that will also hold the rules.**
- **WHERE** At file scope, alongside nothing else: the rules live here too.
- **WHEN** Created by its parent view each time the parent's body runs.
- **WHY** It looks harmless, but this one struct is about to take on three jobs.

4 `@State private var logged = 0`

- **WHO** SwiftUI owns the storage.
- **WHAT** **Keeps the minute total as view state.**
- **WHERE** Inside the view, in storage that SwiftUI manages for this screen only.
- **WHEN** Lives while the view is on screen.
- **WHY** The data now belongs to one screen, so a second screen or a widget cannot share it.

6 `var body: some View {`

- **WHO** SwiftUI calls it on each change.
- **WHAT** **Describes the screen, and here also decides what the message says.**
- **WHERE** The View protocol's only requirement, inside FocusView.
- **WHEN** On every redraw, which happens after each tap changes logged.
- **WHY** Everything below runs inside rendering code, the hardest place to test.

7 `VStack {`

- **WHO** SwiftUI's layout system, arranging the children.
- **WHAT** **Stacks the text and the button.**
- **WHERE** The root of body's content, holding both the text and the button.
- **WHEN** Rebuilt on each redraw; cheap, because views are value descriptions.
- **WHY** Layout is the legitimate part of this view.

8 `Text(logged >= 60 ? "Goal reached"`

WHO The view, computing a business message.
WHAT **Starts a ternary that decides the message, with the goal hard-coded as 60.**
WHERE Inside Text's argument, in the middle of layout code.
WHEN On every redraw, so the rule is re-decided inside rendering code.
WHY A rule hidden in an argument cannot be unit tested; you would have to render the view to check it.

9 `: "\(60 - logged) min left")`

WHO The same expression, continued.
WHAT **Formats the remaining minutes, repeating the magic number 60.**
WHERE Second half of the ternary.
WHEN On every redraw, together with the first half of the expression.
WHY The goal now appears twice. Change one and forget the other and the screen lies.

10 `Button("Log 25 minutes") {`

WHO The user taps; SwiftUI runs the closure.
WHAT **A button whose action holds the update rule itself.**
WHERE Second child of the VStack, below the message.
WHEN On each tap; the closure body runs on the main thread.
WHY Rules in action closures get copied into every screen that needs them.

11 `logged = min(logged + 25, 60)`

WHO The view, changing state directly.
WHAT **Adds 25 and caps at 60, the third copy of the goal.**
WHERE Inside the button's action closure.
WHEN On each tap, before SwiftUI schedules the next redraw.
WHY Validation, limits and formatting are now scattered across body, invisible to tests.

B: After: the view only displays

```swift
import SwiftUI

struct FocusView: View {
    @State private var model = FocusModel()

    var body: some View {
        VStack {
            Text(model.status)
            Button("Log 25 minutes") { model.log(25) }
        }
    }
}
```

1 `import SwiftUI`

WHO The Swift compiler, while it builds this file.
WHAT **Imports SwiftUI for View, Text, Button and @State.**
WHERE First line of the view's file.
WHEN At compile time, before the app runs.
WHY The model file imports only Observation; only the view needs SwiftUI.

```
3  struct FocusView: View {
```

WHO You declare it; SwiftUI renders it.
WHAT **The same screen with a single job: layout.**
WHERE At file scope; the model lives in its own file next to it.
WHEN Created by its parent view each time the parent's body runs.
WHY A view with one job stays small as the app grows.

```
4  @State private var model = FocusModel()
```

WHO SwiftUI keeps the model alive.
WHAT **Owns one FocusModel for the life of the screen.**
WHERE View state storage managed by SwiftUI.
WHEN Created on first appearance.
WHY The state lives in a type that other screens or tests can also create.

```
6  var body: some View {
```

WHO SwiftUI calls it when status or logged changes.
WHAT **Describes the screen using only values the model provides.**
WHERE The View protocol's only requirement.
WHEN On every relevant change.
WHY No decisions remain here, only arrangement.

```
7  VStack {
```

WHO SwiftUI's layout system, arranging the two children.
WHAT **Stacks the two controls.**
WHERE The root of body's content.
WHEN Rebuilt on each redraw; cheap, because it is only a description.
WHY Arranging views is the one job left for this struct, and it does nothing else.

```
8  Text(model.status)
```

WHO SwiftUI renders; Observation tracks the read.
WHAT **Shows whatever message the model computed.**
WHERE First child of the VStack.
WHEN Each redraw; Observation records that this view read status.
WHY The format rule is tested once, in the model, and every screen reuses it.

```
9  Button("Log 25 minutes") { model.log(25) }
```

WHO The user taps; the model decides.
WHAT **Forwards the intent to log 25 minutes.**
WHERE Second child of the VStack.
WHEN On each tap; the model changes logged and SwiftUI redraws.
WHY Changing the goal or the cap is now a one-line edit in one file.

> **Takeaway**
>
> The left view does three jobs: storing state, enforcing rules and laying out controls. The goal appears three times, and nothing can test the rule without rendering a screen. The right view does one job. Its rules live in one tested place, so changing the goal is a one-line edit.

CONCEPT

Dependency injection with protocols

Module 08 (Networking) introduced protocol-backed services with a stub; here the same idea becomes the testing seam for a whole screen. A view model that builds its own `URLSession` is welded to the

network: previews need Wi-Fi, and tests become slow and flaky. **Dependency injection** means an object receives what it depends on from outside, usually as an initializer argument.

Describe the dependency with a **protocol**, then inject a live, mock or failing version. Try each one in the figure. The view and the view model never change.

Protocol	The contract: what the service can do, not how it does it.
Mock	A stand-in that returns fixed data instantly, for previews and tests.

Interactive figure in the HTML version: shipping di swap.

CODE · READ-ALONG

The contract and the view model

The protocol names one ability: report the current temperature. The view model stores *any* type that has that ability and never learns whether it talks to a server, a file or a test.

```swift
import Observation

protocol WeatherService: Sendable {
    func currentTemperature() async throws -> Double
}

@MainActor @Observable
final class WeatherViewModel {
    private let service: any WeatherService
    private(set) var summary = "Not loaded"
    init(service: any WeatherService) { self.service = service }

    func load() async {
        summary = "Loading…"
        do {
            let t = try await service.currentTemperature()
            summary = "Now: \(t) °C"
        } catch {
            summary = "Could not load weather"
        }
    }
}
```

Line by line

1 `import Observation`

WHO	The Swift compiler, while it builds the view model's file.
WHAT	**Imports the framework that provides the @Observable macro, without importing SwiftUI.**
WHERE	Top of the view model's file.
WHEN	At compile time, before the app runs.
WHY	A view model that does not import SwiftUI cannot accidentally depend on views, which keeps it testable on any platform.

```
3  protocol WeatherService: Sendable {
```

WHO You define the contract; the compiler enforces it on every conforming type.
WHAT **Declares a protocol, a list of requirements any weather source must meet.**
WHERE File scope, usually in a Services folder.
WHEN Checked at compile time; no code runs here.
WHY Sendable promises conforming types are safe to share across concurrent tasks, which Swift 6 data-race checking requires.

```
4  func currentTemperature() async throws -> Double
```

WHO Every conforming type must implement it.
WHAT **Requires an asynchronous method that may fail and returns degrees Celsius.**
WHERE Inside the protocol; there is no body here, only a signature.
WHEN Called later by the view model when the screen loads.
WHY async allows a network wait without freezing the screen; throws lets a failure travel to a do/catch instead of crashing.

```
7  @MainActor @Observable
```

WHO The compiler (for @MainActor) and the Observation macro.
WHAT **Pins every property and method to the main thread and makes properties observable.**
WHERE Attributes on the class below.
WHEN Enforced at compile time; observation works at run time.
WHY SwiftUI reads the model on the main thread. @MainActor makes Swift prove that every write happens there too.

```
8  final class WeatherViewModel {
```

WHO You declare it; a screen creates it.
WHAT **The view model for a weather card.**
WHERE File scope in the feature's folder.
WHEN Instantiated by the app or by a test.
WHY final plus a class: one shared, observable instance that nobody subclasses.

```
9  private let service: any WeatherService
```

WHO The view model owns this reference privately.
WHAT **Stores whichever WeatherService was handed in.**
WHERE A stored property; any marks an existential box that can hold any conforming type.
WHEN Set once in init; used in load().
WHY The view model depends on the contract, not a concrete type. That single word is what makes swapping possible.

```
10 private(set) var summary = "Not loaded"
```

WHO The view reads it; only the view model writes it.
WHAT **The text the card shows, starting with a neutral message.**
WHERE An observable stored property.
WHEN Read on every redraw; written during load().
WHY Exposing ready-to-show text keeps the view free of formatting decisions.

```
11 init(service: any WeatherService) { self.service = service }
```

WHO Whoever creates the view model: the app, a preview or a test.
WHAT **Receives the dependency as an argument and stores it.**
WHERE The class's only initializer.
WHEN Once, when the app, a preview or a test creates the view model.
WHY Initializer injection makes the dependency impossible to forget: you cannot create the view model without choosing a service.

```
13 func load() async {
```

WHO — Called by the view, typically from .task when the card appears.
WHAT — **Fetches the temperature and updates summary.**
WHERE — An async method on the main actor.
WHEN — Each time the screen asks to load.
WHY — async lets the method pause at await while the main thread keeps drawing frames.

```
14 summary = "Loading…"
```

WHO — The view model, on the main actor.
WHAT — **Shows a loading message right away.**
WHERE — First statement of load().
WHEN — Before the request starts.
WHY — Immediate feedback tells the user something is happening; the view updates automatically.

```
15 do {
```

WHO — Swift's error-handling syntax, inside load().
WHAT — **Starts a block whose thrown errors jump to catch.**
WHERE — Inside load(), wrapping the one call that can throw.
WHEN — Each time load() runs.
WHY — The protocol method throws, so the call must be handled here or passed upward.

```
16 let t = try await service.currentTemperature()
```

WHO — Whatever service was injected; the view model does not know which.
WHAT — **Suspends until the service answers, then stores the reading in t.**
WHERE — Inside the do block; the main thread is free while suspended.
WHEN — After the loading message is set.
WHY — try marks the call that may throw; await marks the place the function may pause.

```
17 summary = "Now: \(t) °C"
```

WHO — The view model, back on the main actor.
WHAT — **Formats the reading, for example "Now: 21.5 °C".**
WHERE — Inside do, after a successful call.
WHEN — When the service returns.
WHY — Assigning summary triggers observation, so the card redraws with no extra code.

```
18 } catch {
```

WHO — Swift's catch clause, paired with the do block in load().
WHAT — **Catches any error the service threw.**
WHERE — Paired with the do block.
WHEN — Only when currentTemperature() throws.
WHY — Every failure becomes a visible, friendly state instead of a crash.

```
19 summary = "Could not load weather"
```

WHO — The view model, back on the main actor.
WHAT — **Shows a failure message.**
WHERE — Inside the catch block, the failure path of load().
WHEN — After an error, such as no network.
WHY — A failure path you can trigger on purpose, with a failing service, is a failure path you can test.

CODE · RUNS IN THE BROWSER

Prove it with a stand-in

A mock conforms to the same protocol and returns a fixed value. The function at the bottom plays the view model's role: it accepts *any* `WeatherService`. Run it and watch one function produce two results without a network.

```
protocol WeatherService {
    func currentTemperature() async throws -> Double
}

struct MockWeatherService: WeatherService {
    var reading = 21.5
    func currentTemperature() async throws -> Double { reading }
}

func summary(from service: any WeatherService) async -> String {
    do {
        let t = try await service.currentTemperature()
        return "Now: \(t) °C"
    } catch {
        return "Could not load weather"
    }
}

print(await summary(from: MockWeatherService()))
print(await summary(from: MockWeatherService(reading: 30)))
```

OUTPUT

```
Now: 21.5 °C
Now: 30.0 °C
```

Line by line

1 `protocol WeatherService {`

- WHO — You define the contract.
- WHAT — **The same requirement as before, minus Sendable to keep this console example short.**
- WHERE — At file scope; in a real project, Core/Services/WeatherService.swift.
- WHEN — Checked at compile time.
- WHY — Code that depends on this protocol works with every type that conforms to it.

2 `func currentTemperature() async throws -> Double`

- WHO — Each conforming type implements it.
- WHAT — **Requires an async, throwing method that returns a Double.**
- WHERE — Inside the protocol; a requirement has a signature but no body.
- WHEN — Called by summary(from:) each time it runs, twice in this program.
- WHY — Matching the app's real contract means a mock that passes here would also plug into the real view model.

```
5  struct MockWeatherService: WeatherService {
```

WHO You write it for previews and tests; the compiler checks the conformance.
WHAT **A mock type that promises to meet the WeatherService contract.**
WHERE File scope; in a real project it often lives in the test target or a Preview Content folder.
WHEN Compiled with the program.
WHY A struct is a simple value with no network, no disk and no timing surprises.

```
6      var reading = 21.5
```

WHO Each mock instance, which stores its own reading.
WHAT **The temperature the mock will report, with a default.**
WHERE A stored property; the memberwise initializer accepts reading: as an optional argument.
WHEN Set when the mock is created.
WHY A configurable value lets one mock type produce many scenarios: hot, cold or freezing.

```
7      func currentTemperature() async throws -> Double { reading }
```

WHO The mock, satisfying the protocol.
WHAT **Returns the stored reading immediately.**
WHERE Inside the mock, fulfilling the protocol requirement.
WHEN Whenever a caller awaits it.
WHY It is still marked async throws to match the contract, even though it never waits or fails.

```
10 func summary(from service: any WeatherService) async -> String {
```

WHO Code that consumes the dependency, playing the view model's part.
WHAT **Takes any conforming service and returns the text to display.**
WHERE A top-level function, standing in for the view model's load().
WHEN Each time it is called below.
WHY The parameter type is the protocol, so the caller chooses the implementation. That is dependency injection in one line.

```
11     do {
```

WHO Swift's error-handling syntax, inside summary(from:).
WHAT **Begins a block that can throw.**
WHERE Inside summary(from:), around the call that can throw.
WHEN Each time summary(from:) is called, twice in this program.
WHY Same shape as the real view model, so the logic you see here is the logic you ship.

```
12         let t = try await service.currentTemperature()
```

WHO Whichever service was passed in.
WHAT **Awaits the temperature; here the mock answers instantly.**
WHERE Inside the do block, the success path.
WHEN At each call; execution pauses here until the service answers.
WHY This line cannot tell a mock from a live server, which is the point.

```
13         return "Now: \(t) °C"
```

WHO summary(from:), producing its result.
WHAT **Returns formatted text such as "Now: 21.5 °C".**
WHERE Inside the do block, reached only on success.
WHEN After the service returns.
WHY Swift prints whole Doubles with .0, so 30 shows as 30.0. A test would catch that if you wanted "30".

```
14 } catch {
```

WHO Swift's catch clause, paired with the do block in summary(from:).
WHAT **Handles any thrown error.**
WHERE Right after the do block, pairing with it.
WHEN Only if currentTemperature() throws.
WHY Keeps the function total: it always returns a string.

```
15 return "Could not load weather"
```

WHO summary(from:), producing its failure result.
WHAT **Returns the failure message.**
WHERE Inside the catch block.
WHEN Only when the service throws, which this mock never does.
WHY A failing stand-in, built in the next lesson, exercises this branch on purpose.

```
19 print(await summary(from: MockWeatherService()))
```

WHO Top-level code, acting as a preview or a test.
WHAT **Injects a default mock and prints "Now: 21.5 °C".**
WHERE Program entry; top-level code may use await directly.
WHEN Runs first, producing the first output line.
WHY Choosing the dependency at the call site is what lets tests and previews avoid the network.

```
20 print(await summary(from: MockWeatherService(reading: 30)))
```

WHO Top-level code, acting as a second preview or test.
WHAT **Injects a mock configured to 30 and prints "Now: 30.0 °C".**
WHERE The program's top level, where await is allowed directly.
WHEN Runs second, producing the second output line.
WHY Same consumer, different scenario, no code changes inside summary(from:).

> **Try it**
>
> 1. Add a third line that injects MockWeatherService(reading: -4). Predict the output first.
> 2. Write a FailingWeatherService whose method throws, inject it, and make the catch branch print.

CODE · READ-ALONG

The live implementation

The real service calls the free Open-Meteo forecast API with `URLSession`. It conforms to the same protocol, so the view model accepts it without a single change. Only this file knows that HTTP and JSON exist.

```swift
import Foundation

struct LiveWeatherService: WeatherService {
    let latitude, longitude: Double
    var session: URLSession = .shared

    private struct Reply: Decodable { let current: Current }
    private struct Current: Decodable { let temperature_2m: Double }

    func currentTemperature() async throws -> Double {
        let url = URL(string: "https://api.open-meteo.com/v1/forecast"
            + "?latitude=\(latitude)&longitude=\(longitude)"
            + "&current=temperature_2m")!
        let (data, response) = try await session.data(from: url)
        guard (response as? HTTPURLResponse)?.statusCode == 200 else {
            throw URLError(.badServerResponse)
        }
        let reply = try JSONDecoder().decode(Reply.self, from: data)
        return reply.current.temperature_2m
    }
}
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, while it builds the service's file.
- WHAT — **Imports Foundation for URL, URLSession and JSONDecoder.**
- WHERE — Top of the service's file.
- WHEN — At compile time, before the app runs.
- WHY — Networking types live in Foundation; the view model never imports them.

3 `struct LiveWeatherService: WeatherService {`

- WHO — You write it; the compiler checks it meets the contract.
- WHAT — **The production weather source.**
- WHERE — File scope, in a Services folder shared by features.
- WHEN — Created once, when the app starts.
- WHY — A struct with no mutable state is automatically Sendable, which the protocol demands.

4 `let latitude, longitude: Double`

- WHO — Each service instance, which carries its own coordinates.
- WHAT — **The location to ask about, declared as two constants of the same type.**
- WHERE — Stored properties; the memberwise initializer takes them as arguments.
- WHEN — Set once, when the composition root creates the service.
- WHY — Configuration is injected too, so tests could point the service at any place.

5 `var session: URLSession = .shared`

- WHO — The service holds a URLSession, Foundation's HTTP client.
- WHAT — **Uses the shared session unless a different one is passed in.**
- WHERE — A stored property with a default value.
- WHEN — Set once at creation; the default applies unless a test passes another.
- WHY — A test can inject a session configured with a fake URLProtocol, so even this live code is testable offline.

```
7  private struct Reply: Decodable { let current: Current }
```

WHO JSONDecoder fills it in.
WHAT **A private type that mirrors the top level of the API's JSON.**
WHERE Nested inside the service, invisible to the rest of the app.
WHEN Used only while JSONDecoder converts the downloaded bytes.
WHY The server's JSON shape stays an internal detail; the app only ever sees a Double.

```
8  private struct Current: Decodable { let temperature_2m: Double }
```

WHO JSONDecoder, which matches property names to JSON keys.
WHAT **Mirrors the "current" object and its temperature_2m field.**
WHERE Nested inside the service and private, like Reply.
WHEN Used only during decoding.
WHY The property name matches the JSON key exactly. Setting keyDecodingStrategy to convertFromSnakeCase would allow a Swift-style name instead.

```
10 func currentTemperature() async throws -> Double {
```

WHO The view model calls it through the protocol.
WHAT **Implements the requirement with a real network request.**
WHERE Inside LiveWeatherService, fulfilling the protocol.
WHEN Each time the view model's load() runs.
WHY Same signature as the mock, which is why the two are interchangeable.

```
11 let url = URL(string: "https://api.open-meteo.com/v1/forecast"
```

WHO Foundation's URL parser, through URL(string:).
WHAT **Starts building the request address from the API's base URL.**
WHERE Inside currentTemperature(), the first step of the request.
WHEN On each call, before any network traffic.
WHY Splitting the string over three lines keeps each line short and readable.

```
12 + "?latitude=\(latitude)&longitude=\(longitude)"
```

WHO String concatenation and interpolation.
WHAT **Adds the coordinates as query parameters.**
WHERE Continues the URL expression.
WHEN On each call, while the URL string is assembled.
WHY Values come from injected configuration, not hard-coded numbers.

```
13 + "&current=temperature_2m")!
```

WHO URL(string:), which returns an optional.
WHAT **Asks only for the current temperature and force-unwraps the URL.**
WHERE Ends the URL expression.
WHEN On each call; a malformed constant would crash at the first run in development.
WHY The force unwrap is acceptable only because this string is a constant you control; never force-unwrap user input.

```
14 let (data, response) = try await session.data(from: url)
```

WHO URLSession performs the HTTPS request on a background system queue.
WHAT **Downloads the response body (data) and metadata (response).**
WHERE Inside the method; the caller's thread is free while it waits.
WHEN When the server replies, typically within a second.
WHY The async data(from:) API replaces completion handlers and throws on transport errors such as no connection.

```
15 guard (response as? HTTPURLResponse)?.statusCode == 200 else {
```

WHO Your code, inspecting the reply.
WHAT **Checks that the server answered with HTTP status 200 OK.**
WHERE Right after the download.
WHEN On every reply, before any decoding.
WHY A server error page is still "data"; checking the status stops you from decoding an error message as weather.

```
16 throw URLError(.badServerResponse)
```

WHO Your code raising an error.
WHAT **Throws a standard URLError when the status was not 200.**
WHERE Inside the guard's else branch.
WHEN Only when the status code was not 200.
WHY The view model's catch turns this into "Could not load weather".

```
18 let reply = try JSONDecoder().decode(Reply.self, from: data)
```

WHO JSONDecoder, Foundation's JSON parser.
WHAT **Parses the JSON bytes into the private Reply type.**
WHERE Inside currentTemperature(), after the status check.
WHEN After a successful status check.
WHY Decoding throws if the shape is wrong, so malformed data becomes an error, not garbage on screen.

```
19 return reply.current.temperature_2m
```

WHO The service, handing its answer back through the protocol.
WHAT **Returns just the number the protocol promised.**
WHERE Last line of currentTemperature().
WHEN After decoding succeeds, as the last step of each call.
WHY The rest of the app sees a Double and never learns what JSON looked like.

CODE · READ-ALONG

Choose the implementation at the edges

The **composition root** is the one place where concrete types are chosen. The app injects the live service; the preview injects the mock. `WeatherView` (a `Text` that calls `load()` in `.task`) is the same in both.

```swift
import SwiftUI

@main
struct StudyBuddyApp: App {
    @State private var weather = WeatherViewModel(
        service: LiveWeatherService(latitude: 25.77, longitude: -80.19)
    )

    var body: some Scene {
        WindowGroup {
            WeatherView(model: weather)
        }
    }
}

#Preview("Mock data") {
    let model = WeatherViewModel(service: MockWeatherService())
    WeatherView(model: model)
}
```

Line by line

1 `import SwiftUI`

WHO	The Swift compiler, while it builds the app file.
WHAT	**Imports SwiftUI for App, Scene, WindowGroup and #Preview.**
WHERE	Top of StudyBuddyApp.swift, the app's entry file.
WHEN	At compile time, before the app runs.
WHY	The app entry point is a SwiftUI type.

3 `@main`

WHO	The Swift compiler, which generates the entry point, and iOS, which calls it.
WHAT	**Marks the type below as the program's entry point.**
WHERE	On the app struct; exactly one per app.
WHEN	At launch, iOS starts the app here.
WHY	There is no main.swift or storyboard; @main tells Swift where execution begins.

4 `struct StudyBuddyApp: App {`

WHO	You declare it; SwiftUI runs it.
WHAT	**The app itself, conforming to the App protocol.**
WHERE	File scope, in the app target.
WHEN	Created once, when iOS launches the app.
WHY	The app struct is the natural composition root because everything else is created beneath it.

5 `@State private var weather = WeatherViewModel(`

WHO	SwiftUI keeps this model alive for the app's lifetime.
WHAT	**Creates the single production view model.**
WHERE	App-level state storage inside StudyBuddyApp, managed by SwiftUI.
WHEN	Once, at launch; SwiftUI then keeps the instance alive.
WHY	@State at the app level means one shared model, not a new one every time a scene is rebuilt.

```
6   service: LiveWeatherService(latitude: 25.77, longitude: -80.19)
```

WHO	The composition root, choosing the concrete type.
WHAT	**Injects the live service configured for Miami.**
WHERE	The initializer argument of WeatherViewModel, inside the App struct.
WHEN	Once, at launch, before the first screen appears.
WHY	This is the only line in the app that names LiveWeatherService. Replacing the weather provider means editing one line.

```
7   )
```

WHO	The Swift compiler, matching the opening parenthesis.
WHAT	**Closes the initializer call.**
WHERE	End of the property declaration.
WHEN	At compile time; no code runs for punctuation.
WHY	Splitting the call keeps the line under the width limit.

```
9   var body: some Scene {
```

WHO	SwiftUI, which builds the app's scenes from it.
WHAT	**Describes the app's scenes (windows).**
WHERE	The App protocol's requirement.
WHEN	At launch and when scenes change.
WHY	Scenes are to apps what views are to screens.

```
10  WindowGroup {
```

WHO	iOS and SwiftUI, which manage the app's windows.
WHAT	**Creates the main window, or several on iPad.**
WHERE	Inside the app's body.
WHEN	At launch, and again if iPadOS opens another window.
WHY	WindowGroup is the standard scene for a phone app.

```
11  WeatherView(model: weather)
```

WHO	SwiftUI, which renders the root view.
WHAT	**Shows the weather screen and passes it the live model.**
WHERE	Root view of the window.
WHEN	At launch, as the window's first content.
WHY	The view receives its model too; it never creates services itself.

```
16  #Preview("Mock data") {
```

WHO	Xcode's preview canvas, which compiles and renders it.
WHAT	**Declares a named preview for the same view.**
WHERE	File scope; compiled only for development.
WHEN	Each time the canvas refreshes.
WHY	A named preview lets you keep several scenarios side by side, such as mock, failing and empty.

```
17  let model = WeatherViewModel(service: MockWeatherService())
```

WHO	The preview, acting as a second composition root.
WHAT	**Builds a view model around the mock.**
WHERE	Inside the preview body.
WHEN	Each time the preview refreshes after an edit.
WHY	The preview renders instantly, offline and with predictable data.

```
18  WeatherView(model: model)
```

WHO	SwiftUI, rendering the view inside the canvas.
WHAT	**Renders the unchanged view with mock data.**
WHERE	The preview's body, the content the canvas shows.
WHEN	Each time the preview refreshes.
WHY	Same view, different dependency: the payoff of injection.

CONCEPT

Organize by feature

Group files by **feature** (what the user does), not by kind (all views in one folder). When you change "Sessions", everything you need sits together. Shared services and models live in one core area, and tests mirror the app's structure.

Features/Sessions/	SessionListView.swift, SessionListViewModel.swift and any views only this feature uses.
Features/Spots/	The map, list and view model for nearby study spots.
Core/	Models (StudySession), Services (WeatherService and its live version) and shared helpers.
Resources/	Assets, the String Catalog (Localizable.xcstrings) and the privacy manifest (PrivacyInfo.xcprivacy).
Tests	A unit-test target (Swift Testing) and a UI-test target (XCTest), each mirroring the feature folders.

> **Rule of thumb**
>
> If deleting a feature means hunting through five folders, organize by feature. Xcode 16 and later use folders that stay in sync with disk, so moving a file in Finder or Xcode keeps both views identical.

CHECK YOURSELF

Check your understanding

1. Which line belongs in the view model rather than the view?
   A. `Text(model.status).font(.title2)`
   B. `logged = min(logged + minutes, goal)`
   C. `.buttonStyle(.borderedProminent).padding()`
   D. `VStack(spacing: 16) { Text(model.status) }`
2. A `FocusModel` has a goal of 60. You call `log(40)`, `log(-5)`, then `log(30)`. What does `status` return?
   A. "Goal reached"
   B. "20 min left"
   C. "-10 min left"
   D. "25 min left"

3. Why does `WeatherViewModel` take `any WeatherService` in its initializer instead of creating `LiveWeatherService` itself?
   A. Calls through a protocol are faster than calls on a concrete type
   B. Swift forbids a class from creating a struct in its own initializer
   C. So tests and previews can inject a mock or failing service
   D. So the view can swap the service for another one while running
4. Your team adds a "Flashcards" feature. Where should `FlashcardDeckViewModel.swift` go?
   A. ViewModels/, next to every other view model in the app
   B. Core/Services/
   C. Resources/
   D. Features/Flashcards/, beside the views it serves

Answers: Architecture That Survives Change

1. **B. `logged = min(logged + minutes, goal)`** Capping the total is a business rule, so it belongs in the model where one test covers it. Displaying text, styling a button and stacking views are layout jobs that stay in the view.
2. **A. "Goal reached"** 40 is logged, -5 is rejected by the guard, and 40 + 30 = 70 is capped at 60, so the goal is reached. "-10 min left" is what you would see without the cap.
3. **C. So tests and previews can inject a mock or failing service** Injection lets the caller choose the implementation: live in the app, mock in previews, failing in tests. The view never chooses services; the composition root does.
4. **D. Features/Flashcards/, beside the views it serves** Organizing by feature keeps everything a change touches in one folder. A folder of all view models scatters each feature across the project.

Testing with Swift Testing

CONCEPT

Verification and validation

Testing answers two different questions. **Verification** asks "did we build it right?": does the code do what the requirement says? **Validation** asks "did we build the right thing?": does it actually solve the user's problem?

Automated tests are verification. They compare behavior with requirements you wrote down, every time you build. Validation needs people: usability sessions, TestFlight feedback and analytics. A perfectly verified feature still fails validation if students never use it.

Requirement	When a bill is split, each share includes the tip and is rounded to the cent.
Verification	A unit test checks that a $100 bill with a 20% tip split four ways gives exactly 30.
Validation	Five students split a real dinner bill with the app while you watch where they hesitate.

> **Find bugs early**
>
> A bug caught by a test costs seconds. The same bug found in TestFlight costs a new build, and in the App Store it costs a review cycle and one-star reviews. That is why tests run on every build.

CONCEPT

The testing pyramid

Tests differ in speed, cost and what they can catch. The **testing pyramid** recommends many fast unit tests, fewer integration tests and a handful of UI tests for the journeys that matter most.

Select each layer in the figure, then switch to the ice-cream cone, a suite that leans on UI tests, and compare how long you wait for an answer.

Unit test	Checks one function or type in isolation, in milliseconds.
Integration test	Checks real pieces working together, such as a view model with an in-memory store.
UI test	Drives the built app like a person would, through the accessibility layer.

Interactive figure in the HTML version: shipping test pyramid.

CODE · SWIFT TESTING

Your first tests

Swift Testing marks tests with `@Test` and checks results with `#expect`. In a real project the function lives in the app target and tests sit in a separate test target that starts with `@testable import StudyBuddy`. Here both share one file. Press Command-U in Xcode to run every test.

```swift
import Testing

func share(of bill: Double, tip: Int, people: Int) -> Double? {
    guard people > 0, bill >= 0 else { return nil }
    let total = bill * (1 + Double(tip) / 100)
    let each = total / Double(people)
    return (each * 100).rounded() / 100
}

@Test func splitsTotalWithTip() {
    #expect(share(of: 100, tip: 20, people: 4) == 30)
}

@Test func rejectsZeroPeople() {
    #expect(share(of: 50, tip: 10, people: 0) == nil)
}
```

Line by line

1 `import Testing`

- **WHO** The compiler, building the test target.
- **WHAT** **Imports Swift Testing, the framework that provides @Test, #expect and #require.**
- **WHERE** Top of the test file; test targets link the framework automatically in Xcode 16 and later.
- **WHEN** At compile time, when Xcode builds the test target.
- **WHY** Swift Testing is the modern framework for unit tests. XCTest remains for UI tests and older code.

3 `func share(of bill: Double, tip: Int, people: Int) -> Double? {`

- **WHO** App code, the code under test.
- **WHAT** **Declares a function that returns one person's share, or nil for invalid input.**
- **WHERE** Normally in the app target; shown here so the example is complete.
- **WHEN** Called once by each test that exercises it.
- **WHY** Returning an optional makes "invalid input" a value the caller must handle and a test can check.

4 `guard people > 0, bill >= 0 else { return nil }`

- **WHO** The function, validating its input.
- **WHAT** **Returns nil for zero or negative people, or a negative bill.**
- **WHERE** First line of the function.
- **WHEN** On every call, before any arithmetic happens.
- **WHY** Two separate conditions means two behaviors worth a test each.

5 `let total = bill * (1 + Double(tip) / 100)`

- **WHO** share(of:tip:people:), doing the arithmetic.
- **WHAT** **Adds the tip, converting the whole-number percent into a fraction.**
- **WHERE** Inside the function, after the guard.
- **WHEN** Only for input that passed the guard.
- **WHY** Double(tip) is required because Swift never mixes Int and Double silently.

6 `let each = total / Double(people)`

- **WHO** share(of:tip:people:), still on valid input.
- **WHAT** **Divides the total among the people.**
- **WHERE** Inside the function, on the line after the tip.
- **WHEN** After the tip has been added to the bill.
- **WHY** Splitting after tipping is the requirement; doing it the other way round is a classic bug.

```
7   return (each * 100).rounded() / 100
```

WHO The function, using Double's rounding.
WHAT **Rounds the share to the nearest cent, so 6.666 becomes 6.67.**
WHERE Last line of the function.
WHEN Last, for valid input only; invalid input already returned nil.
WHY Rounding is a requirement too; without it equality checks on money fail on tiny binary fractions.

```
10  @Test func splitsTotalWithTip() {
```

WHO The @Test macro, which registers the function with the test runner.
WHAT **Declares a test; its name describes the behavior it checks.**
WHERE Global function in the test file; tests can also live inside a type.
WHEN Discovered at build time and run when you press Command-U or click the diamond in the gutter.
WHY A name that reads like a sentence tells you what broke when it turns red, without opening the file.

```
11      #expect(share(of: 100, tip: 20, people: 4) == 30)
```

WHO The #expect macro, evaluated by the test runner.
WHAT **Checks that $100 plus 20%, split four ways, is exactly 30.**
WHERE Inside splitsTotalWithTip(), its only statement.
WHEN When the runner executes this test.
WHY If it fails, #expect records the actual value of each side, such as "→ 525.0", so you see why, not just that it failed.

```
14  @Test func rejectsZeroPeople() {
```

WHO The @Test macro, registering a second test.
WHAT **Declares a second test for the invalid-input branch.**
WHERE A global function in the test file.
WHEN Runs in parallel with the other tests by default.
WHY Every branch of the guard deserves its own test, because each can break independently.

```
15      #expect(share(of: 50, tip: 10, people: 0) == nil)
```

WHO The #expect macro, comparing the result with nil.
WHAT **Checks that splitting among zero people returns nil instead of a number.**
WHERE Inside rejectsZeroPeople(), its only statement.
WHEN When the runner executes this test.
WHY Without the guard, 55 divided by 0 is infinity, a value that would appear on screen as "inf".

> **Try it**
>
> 1. Change 30 to 25 in the first test. Read the failure message: which value does it show for the left side?
> 2. Add a test that a negative bill returns nil.

CODE · SWIFT TESTING

Suites, readable names and #require

A `@Suite` groups related tests under one heading in the Test navigator. Display names describe behavior in plain words. `#require` unwraps an optional and stops the test immediately if it is `nil`, so later lines never run on bad data. The `share` function from the previous step is compiled with this file.

```swift
import Testing

@Suite("Splitting a bill")
struct BillSplitTests {
    @Test("Zero tip splits only the bill")
    func zeroTip() throws {
        let each = try #require(share(of: 90, tip: 0, people: 3))
        #expect(each == 30)
        #expect(each * 3 == 90)
    }

    @Test("Shares round to the nearest cent")
    func roundsToCents() {
        #expect(share(of: 20, tip: 0, people: 3) == 6.67)
    }
}
```

Line by line

1 `import Testing`

- **WHO** The Swift compiler, building the test target.
- **WHAT** **Imports Swift Testing for @Suite, @Test, #expect and #require.**
- **WHERE** Top of the test file.
- **WHEN** At compile time, when the test target builds.
- **WHY** Every test file imports it.

3 `@Suite("Splitting a bill")`

- **WHO** The @Suite macro, read by Xcode's test discovery.
- **WHAT** **Gives the type below a readable name in Xcode's Test navigator and in reports.**
- **WHERE** Attached to the struct on the next line.
- **WHEN** Read when tests are discovered.
- **WHY** Grouping by behavior makes a long list of tests scannable, and you can run one suite on its own.

4 `struct BillSplitTests {`

- **WHO** You declare it; the runner creates a fresh instance for every test.
- **WHAT** **A struct that holds related tests.**
- **WHERE** Test target, file scope.
- **WHEN** A new instance is made before each test runs.
- **WHY** A fresh instance per test means tests cannot leak state into each other, which keeps them isolated and repeatable.

5 `@Test("Zero tip splits only the bill")`

- **WHO** The @Test macro with a display name.
- **WHAT** **Registers the method below and shows this sentence instead of the method name.**
- **WHERE** Inside the suite, attached to the method below.
- **WHEN** Read at discovery, when Xcode builds the list of tests.
- **WHY** A display name can include spaces and punctuation, so the report reads like a specification.

```
6  func zeroTip() throws {
```

WHO The test runner calls it.
WHAT **A test method that is allowed to throw.**
WHERE An instance method of the suite.
WHEN When the runner executes this test, on a fresh suite instance.
WHY throws is required because #require throws to stop the test early.

```
7  let each = try #require(share(of: 90, tip: 0, people: 3))
```

WHO The #require macro, which throws on failure.
WHAT **Unwraps the optional result; if it is nil, records a failure and ends the test right here.**
WHERE First line of zeroTip().
WHEN As soon as the test starts.
WHY The next lines need a real number. Stopping early avoids confusing follow-on failures and force-unwrap crashes.

```
8  #expect(each == 30)
```

WHO The #expect macro, comparing the share with 30.
WHAT **Checks the unwrapped share is 30.**
WHERE Inside the test, after the unwrap.
WHEN Only if #require succeeded.
WHY Unlike #require, a failing #expect records the problem and keeps going, so you see every failure at once.

```
9  #expect(each * 3 == 90)
```

WHO The #expect macro, checking a property of the result.
WHAT **Checks the shares add back up to the bill.**
WHERE Inside zeroTip(), the last check.
WHEN After the first expectation, pass or fail.
WHY Checking a property (shares sum to the total) catches bugs that a single example might miss.

```
12 @Test("Shares round to the nearest cent")
```

WHO The @Test macro, registering the rounding test.
WHAT **Names the rounding behavior.**
WHERE Inside the suite, attached to the method below.
WHEN Read at discovery, when Xcode lists the tests.
WHY Each requirement gets its own test, so a failure points at exactly one rule.

```
13 func roundsToCents() {
```

WHO The test runner, which calls it on a fresh suite instance.
WHAT **A test that cannot throw, because it uses only #expect.**
WHERE An instance method of the suite.
WHEN When the runner executes this test, possibly in parallel with zeroTip().
WHY Only mark a test throws when it really calls something that throws.

```
14 #expect(share(of: 20, tip: 0, people: 3) == 6.67)
```

WHO The #expect macro, comparing the share with 6.67.
WHAT **Checks that $20 split three ways rounds 6.666... up to 6.67.**
WHERE Inside roundsToCents(), its only statement.
WHEN When this test runs.
WHY Choosing an input that needs rounding up catches a function that truncates instead of rounding.

> **Try it**
>
> 1. Change people to 0 in the #require line. How many failures are reported, and why not three?

CODE · SWIFT TESTING

One test, many inputs

A **parameterized test** runs the same check for every element of `arguments`. Each element becomes its own test case, reported separately with its inputs, so one bad row never hides the others. Adding a case is one line of data instead of a new function.

```
1  import Testing
2
3  @Test("Each share includes the tip", arguments: [
4      (bill: 100.0, tip: 20, people: 4, expected: 30.0),
5      (bill: 60.0, tip: 15, people: 2, expected: 34.5),
6      (bill: 10.0, tip: 0, people: 3, expected: 3.33),
7      (bill: 0.0, tip: 18, people: 5, expected: 0.0),
8  ])
9  func shareIncludesTip(bill: Double, tip: Int, people: Int,
10                        expected: Double) {
11     #expect(share(of: bill, tip: tip, people: people) == expected)
12 }
13
14 @Test("Invalid party sizes are rejected", arguments: [0, -1, -12])
15 func rejectsBadPeople(people: Int) {
16     #expect(share(of: 50, tip: 10, people: people) == nil)
17 }
```

Line by line

1 `import Testing`

- WHO — The Swift compiler, building the test target.
- WHAT — **Imports Swift Testing for @Test and #expect.**
- WHERE — Top of the test file.
- WHEN — At compile time, before the parameterized tests are discovered.
- WHY — Parameterized tests use the same @Test macro, with an arguments label added.

3 `@Test("Each share includes the tip", arguments: [`

- WHO — The @Test macro, given a collection of inputs.
- WHAT — **Declares a parameterized test and starts the list of cases.**
- WHERE — Global test function in the test target.
- WHEN — At discovery, the runner creates one test case per element.
- WHY — A table of cases states the requirement as data, which is easier to review than four near-identical functions.

4 `(bill: 100.0, tip: 20, people: 4, expected: 30.0),`

- WHO — The test runner passes this tuple to the function.
- WHAT — **The standard case: 100 plus 20%, split four ways, is 30.**
- WHERE — First element of the arguments array.
- WHEN — Run as its own test case.
- WHY — Labels make each row self-explanatory; they line up with the parameter names below.

```
5   (bill: 60.0, tip: 15, people: 2, expected: 34.5),
```

WHO	The test runner, which passes this tuple to the function.
WHAT	**A case with an odd percentage: 69 split two ways is 34.5.**
WHERE	Second element of the arguments array.
WHEN	Run as its own case, in parallel with the others.
WHY	Different numbers exercise the percent-to-fraction conversion, where a bug would show up.

```
6   (bill: 10.0, tip: 0, people: 3, expected: 3.33),
```

WHO	The test runner, which passes this tuple to the function.
WHAT	**A case that needs rounding down: 3.333... becomes 3.33.**
WHERE	Third element of the arguments array.
WHEN	Runs as its own separately reported test case.
WHY	Pairs with the earlier rounding-up test, so both directions are covered.

```
7   (bill: 0.0, tip: 18, people: 5, expected: 0.0),
```

WHO	The test runner, which passes this tuple to the function.
WHAT	**A boundary case: a zero bill is valid and gives zero.**
WHERE	Fourth and last element of the arguments array.
WHEN	Runs as its own separately reported test case.
WHY	Boundary values (zero, one, the maximum) are where off-by-one and validation bugs hide.

```
8   ])
```

WHO	The Swift compiler, matching the brackets.
WHAT	**Closes the arguments array and the @Test attribute.**
WHERE	End of the @Test attribute, just above the function it marks.
WHEN	At compile time; the macro reads the whole list then.
WHY	The trailing comma on the last row is legal Swift and makes adding a row a one-line change.

```
9   func shareIncludesTip(bill: Double, tip: Int, people: Int,
```

WHO	The test runner calls it once per case.
WHAT	**Declares parameters that receive each tuple's values in order.**
WHERE	A global function in the test file, marked by the attribute above.
WHEN	Four times, once per row.
WHY	Parameter types must match the tuple elements or the macro reports a compile error.

```
10  expected: Double) {
```

WHO	The Swift compiler, reading the rest of the signature.
WHAT	**Finishes the parameter list with the expected result.**
WHERE	Continuation of the signature, aligned for readability.
WHEN	At compile time; the macro checks it against the tuple type.
WHY	Passing the expected value as data keeps the assertion line identical for every case.

```
11  #expect(share(of: bill, tip: tip, people: people) == expected)
```

WHO	The #expect macro, run once for every row.
WHAT	**Compares each computed share with that row's expected value.**
WHERE	Body of the parameterized test.
WHEN	Once per case: four times in total.
WHY	A failure report names the exact arguments, such as bill → 60.0, so you know which row broke.

```
14 @Test("Invalid party sizes are rejected", arguments: [0, -1, -12])
```

WHO The @Test macro with a simple array.
WHAT **Runs the test three times, once for each bad party size.**
WHERE A global test function, below the first one.
WHEN Three cases at discovery.
WHY A single parameter needs no tuple; any collection of Sendable values works.

```
15 func rejectsBadPeople(people: Int) {
```

WHO The test runner, which calls it once per argument.
WHAT **Receives one party size per case.**
WHERE A global function marked by the attribute above.
WHEN Three times: with 0, then -1, then -12.
WHY Testing several invalid values guards against a check like people != 0 that lets -1 through.

```
16 #expect(share(of: 50, tip: 10, people: people) == nil)
```

WHO The #expect macro, run once for every argument.
WHAT **Expects nil for every invalid party size.**
WHERE The body of rejectsBadPeople(people:), its only statement.
WHEN Once per case: three times in total.
WHY The same expectation over many inputs documents the whole invalid range, not one example.

Try it

1. Add a row (bill: 45.0, tip: 10, people: 3, expected: 16.5) and predict whether it passes.
2. Change -1 to 1 in the second test. Which case fails, and what does its failure message name?

INTERACTIVE

Bug lab: what do your tests catch?

Here is the same `share` function with its test suite. Switch on one bug at a time and watch which tests turn red and what each failure message reports. Then turn on bug D by itself: every test stays green. Finish by choosing the expectation that would have caught it.

This step is an interactive simulation in the HTML version.

CODE · SWIFT TESTING

Test a view model with stand-ins

The `WeatherViewModel`, its protocol and `MockWeatherService` from the architecture lesson are compiled with this file. A failing stand-in drives the error branch on purpose. These tests run in milliseconds, offline, with the same result every time.

```swift
import Testing

struct FailingWeatherService: WeatherService {
    struct Offline: Error {}
    func currentTemperature() async throws -> Double {
        throw Offline()
    }
}

@MainActor
struct WeatherViewModelTests {
    @Test func showsReadingFromService() async {
        let model = WeatherViewModel(service: MockWeatherService())
        await model.load()
        #expect(model.summary == "Now: 21.5 °C")
    }

    @Test func showsMessageWhenServiceFails() async {
        let model = WeatherViewModel(service: FailingWeatherService())
        await model.load()
        #expect(model.summary == "Could not load weather")
    }
}
```

Line by line

1 `import Testing`

WHO	The Swift compiler, building the test target.
WHAT	**Imports Swift Testing for @Test and #expect.**
WHERE	Top of the test file.
WHEN	At compile time, before the suite is discovered.
WHY	The view-model tests use @Test and #expect like the pure-function tests; only the setup differs.

3 `struct FailingWeatherService: WeatherService {`

WHO	You write it in the test target.
WHAT	**A stand-in that always fails, conforming to the same protocol as the live service.**
WHERE	File scope in the tests; the app never ships it.
WHEN	Created inside the failure test, then thrown away when it ends.
WHY	Real outages are rare and random. A stand-in makes the failure path happen on demand.

4 `struct Offline: Error {}`

WHO	Swift's error-handling system, which accepts any Error type.
WHAT	**Declares a tiny error type to throw.**
WHERE	Nested inside the stand-in.
WHEN	Defined at compile time; an instance is created on each throw.
WHY	Any type that conforms to Error can be thrown; an empty struct is enough for a test.

5 `func currentTemperature() async throws -> Double {`

WHO	The view model calls it through the protocol.
WHAT	**Implements the protocol requirement for the failing stand-in.**
WHERE	Inside the failing stand-in, fulfilling the protocol.
WHEN	When load() runs in the test.
WHY	Matching the signature is what makes this type injectable.

```
6  throw Offline()
```

WHO The failing stand-in, raising the error.
WHAT **Throws immediately instead of returning a temperature.**
WHERE The method's only statement.
WHEN On every call, with no delay.
WHY Simulates no connection without touching Wi-Fi settings.

```
10 @MainActor
```

WHO The Swift compiler, which enforces actor isolation.
WHAT **Runs every test in this suite on the main actor.**
WHERE Attribute on the suite, so it applies to every test inside.
WHEN Enforced at compile time; applied when tests run.
WHY WeatherViewModel is main-actor isolated, so tests that touch it must be too.

```
11 struct WeatherViewModelTests {
```

WHO The test runner, which discovers the @Test methods inside.
WHAT **A suite for the view model; without @Suite its name is the type name.**
WHERE The test target, at file scope.
WHEN A new instance per test.
WHY @Suite is optional; any type containing @Test methods is a suite.

```
12 @Test func showsReadingFromService() async {
```

WHO The test runner, which awaits the async method.
WHAT **An async test for the success path.**
WHERE Inside WeatherViewModelTests, the first test method.
WHEN When the runner executes it, on the main actor.
WHY Tests can be async, so you await the real load() rather than sleeping and hoping.

```
13 let model = WeatherViewModel(service: MockWeatherService())
```

WHO The test, acting as composition root.
WHAT **Builds the view model around the mock that reports 21.5.**
WHERE First line of the success test.
WHEN At the start of the test.
WHY Arrange: set up exactly the world this test needs and nothing more.

```
14 await model.load()
```

WHO The view model, running its real load() logic.
WHAT **Runs the real load logic against the mock.**
WHERE Second line of the success test.
WHEN After the view model is created; the test waits here until load() finishes.
WHY Act: perform the one behavior under test. await finishes only when load() has completed.

```
15 #expect(model.summary == "Now: 21.5 °C")
```

WHO The #expect macro, reading the observable property.
WHAT **Checks the exact text the screen will show.**
WHERE Last line of the success test.
WHEN After load() has returned.
WHY Assert: verify the outcome. Testing the summary string verifies the formatting rule too.

```
18 @Test func showsMessageWhenServiceFails() async {
```

WHO The test runner, which awaits the async method.
WHAT **An async test for the failure path.**
WHERE Inside the suite, below the success test.
WHEN When run, possibly in parallel with the first test.
WHY Error paths are where real users get stuck, so they deserve their own tests.

```
19 let model = WeatherViewModel(service: FailingWeatherService())
```

WHO The test, acting as its own composition root.
WHAT **Injects the failing stand-in instead of the mock.**
WHERE First line of the failure test.
WHEN At the start of the failure test.
WHY Only the injected dependency differs between the two tests.

```
20 await model.load()
```

WHO The view model, whose catch block now runs.
WHAT **Calls the service, which throws, so the catch branch runs.**
WHERE Second line of the failure test.
WHEN After setup; the error never escapes load().
WHY The view model handles the error itself; the test only observes the result.

```
21 #expect(model.summary == "Could not load weather")
```

WHO The #expect macro, reading the failure message.
WHAT **Checks the friendly failure message appears.**
WHERE Last line of the failure test.
WHEN After load() has returned.
WHY If someone later deletes the catch block, this test fails before users see a blank card.

CONCEPT

The test-driven loop

Test-driven development (TDD) writes the test before the code. Each loop is small: one behavior at a time. The failing test proves the test can fail, the passing test proves the code works, and the suite becomes a safety net for every later change.

Good unit tests are **F.I.R.S.T.**: fast, isolated (no shared state), repeatable (same result every run), self-validating (pass or fail with no reading of logs) and timely (written with the code, not months later).

Red	Write a test for the next small behavior. Run it and watch it fail.
Green	Write the simplest code that makes it pass. No extra features.
Refactor	Clean up names and duplication while the tests stay green.

> **In Xcode**
>
> Command-U runs every test. The diamond beside a test runs just that one. Command-6 opens the Test navigator, where red marks failures. A breakpoint in a test pauses it like app code.

CHECK YOURSELF

Check your understanding

1. Every unit test passes, but in a hallway session three of five students cannot find how to start a study session. What kind of problem is this?
   A. A verification failure: the tests check the wrong thing
   B. A validation finding: the app misses a real need
   C. An integration failure between the view and the view model
   D. A flaky test that passes only on some machines

2. A test runs `let each = try #require(share(of: 90, tip: 0, people: 0))` followed by two `#expect` lines. How many issues are recorded?
   A. Three: #require and both #expect lines fail
   B. Two: #require passes and both #expect lines fail
   C. Zero: #require only warns, and the #expect lines pass
   D. One: #require fails and ends the test

3. In the bug lab, bug D (negative bills allowed) left all four tests green. What does a green suite prove?
   A. The function has no bugs, because every test exercised it
   B. The tests are poorly written and should be replaced
   C. The function is correct for the cases the tests check, and nothing more
   D. Bug D cannot affect real users, because the tests cover the real inputs

4. `@Test(arguments: [0, -1, -12])` marks a test that receives one `Int`. How does Xcode report it?
   A. As three separate test cases, each showing its argument
   B. As one test that stops at the first failing value
   C. As three copies of the test function you must write
   D. It does not compile, because arguments must be tuples

5. A view-model test calls the live weather API and fails some mornings when the network is slow. What is the best fix?
   A. Retry the test up to three times before reporting a failure
   B. Inject a mock service so the test never touches the network
   C. Add a two-second sleep before the expectation
   D. Move the test to the UI-test target, which allows slow tests

Answers: Testing with Swift Testing

1. **B. A validation finding: the app misses a real need** Tests verified that the code matches its specification. Watching real users is validation, and it found that the specification itself misses a need. Neither the tests nor the integration are broken.
2. **D. One: #require fails and ends the test** `share` returns nil for zero people. `#require` records one issue and throws, so the expectations below never run. That is its job: stop before later lines work on bad data.
3. **C. The function is correct for the cases the tests check, and nothing more** Tests only check what they ask about. No test tried a negative bill, so the bug was invisible. Adding the missing edge case turns it red.
4. **A. As three separate test cases, each showing its argument** Each argument becomes its own test case with its value in the report, and the cases can run in parallel. A failing value never hides the others.
5. **B. Inject a mock service so the test never touches the network** A unit test must be fast and repeatable. Injecting a stand-in removes the network, so the result depends only on your code. Retries and sleeps hide the flakiness without curing it.

UI Tests, Previews and Accessibility Checks

CONCEPT

How a UI test sees your app

A **UI test** launches your real app in the simulator and drives it from a separate process, the test runner. It cannot call your functions or read your variables. It sees only what VoiceOver sees: the **accessibility tree**, a list of elements with a type (button, text field), a label and an identifier.

A UI test finds elements with **queries** such as `app.buttons["addSessionButton"]`, then taps, types and swipes like a person. Each test takes seconds, so save UI tests for journeys that must never break.

Good at	Proving that critical journeys work end to end: launch, add a session, see it in the list.
Poor at	Checking business rules, every edge case or exact pixels. Unit tests and previews do those better.

> **Two frameworks, one project**
>
> Swift Testing does not drive user interfaces. UI tests use XCTest's `XCUIApplication` in a separate UI-testing target. Both frameworks live happily in one project.

CODE · READ-ALONG

Give elements stable identifiers

An **accessibility identifier** is a name only tests can see. Unlike the visible label, it does not change when you translate the app or reword a button. This sheet tags its text field and Save button; the list screen's plus button gets `addSessionButton` the same way.

```swift
import SwiftUI

struct NewSessionView: View {
    let store: SessionStore
    @Environment(\.dismiss) private var dismiss
    @State private var title = ""

    var body: some View {
        Form {
            TextField("Title", text: $title)
                .accessibilityIdentifier("sessionTitleField")
            Button("Save") {
                store.add(title)
                dismiss()
            }
            .disabled(title.isEmpty)
            .accessibilityIdentifier("saveSessionButton")
        }
    }
}
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, building the app target.
WHAT	**Imports SwiftUI for Form, TextField, Button and modifiers.**
WHERE	Top of the view's file.
WHEN	At compile time, before NewSessionView can be built.
WHY	Identifiers are ordinary SwiftUI modifiers; no test framework is imported into the app.

```
3  struct NewSessionView: View {
```

WHO	You declare it; SwiftUI presents it as a sheet.
WHAT	**The form for adding a study session.**
WHERE	In the app target, under Features/Sessions/.
WHEN	Created when the user taps the plus button.
WHY	A small, focused view is easy to tag and easy to test.

```
4  let store: SessionStore
```

WHO	The list screen passes it in.
WHAT	**The shared store that receives the new session.**
WHERE	A stored property of the view.
WHEN	Set when the sheet is created.
WHY	Injecting the store means a test or preview can supply a seeded or empty one.

```
5  @Environment(\.dismiss) private var dismiss
```

WHO	SwiftUI provides it from the environment.
WHAT	**An action that closes the sheet.**
WHERE	Inside the view, read from the environment SwiftUI passes down.
WHEN	Read when the view is built; called after saving.
WHY	The sheet closes itself without knowing who presented it.

```
6  @State private var title = ""
```

WHO	SwiftUI, which keeps the value alive between redraws.
WHAT	**The text the user is typing, starting empty.**
WHERE	View state owned by this sheet only.
WHEN	Changes on each keystroke.
WHY	Local, temporary input belongs in @State, not in the shared store.

```
8  var body: some View {
```

WHO	SwiftUI, which calls it to describe the sheet.
WHAT	**Describes the sheet's form.**
WHERE	The View protocol's only requirement.
WHEN	On appearance and each keystroke.
WHY	Each redraw also refreshes the accessibility tree that UI tests read.

```
9  Form {
```

WHO	SwiftUI, which styles the container as a grouped list.
WHAT	**Groups the controls in the standard grouped style.**
WHERE	Root of the sheet's content.
WHEN	On each redraw, including after every keystroke.
WHY	Standard containers give correct accessibility behavior for free.

```
10  TextField("Title", text: $title)
```

WHO — SwiftUI, and the user typing.
WHAT — **A text field whose placeholder (and accessibility label) is "Title".**
WHERE — First row of the form.
WHEN — Updates title on every keystroke through the binding.
WHY — The label "Title" will become "Título" in Spanish, which is why tests should not search for it.

```
11      .accessibilityIdentifier("sessionTitleField")
```

WHO — SwiftUI, writing into the accessibility tree.
WHAT — **Attaches an identifier that tests can query: app.textFields["sessionTitleField"].**
WHERE — Modifier on the text field.
WHEN — Applied on every render; invisible to users and VoiceOver.
WHY — An identifier is a contract with your tests that survives translation and copy edits.

```
12  Button("Save") {
```

WHO — The user taps; SwiftUI runs the closure.
WHAT — **A button labeled Save; the label is also its accessibility label.**
WHERE — Second row of the form.
WHEN — Its action runs when the user (or a UI test) taps it.
WHY — A clear verb as the label helps VoiceOver users; the identifier below helps tests.

```
13      store.add(title)
```

WHO — The shared SessionStore, through its add(_:) method.
WHAT — **Adds the new title to the top of the list.**
WHERE — Inside the button's action.
WHEN — On tap, before the sheet closes.
WHY — The view forwards the intent; the store owns the data.

```
14      dismiss()
```

WHO — SwiftUI, through the dismiss action from the environment.
WHAT — **Closes the sheet and returns to the list.**
WHERE — Inside the action, after saving.
WHEN — On tap, right after the store is updated.
WHY — The UI test will look for the new row on the list after this.

```
16  .disabled(title.isEmpty)
```

WHO — SwiftUI, which grays out the button and ignores taps.
WHAT — **Dims the button and blocks taps until something is typed.**
WHERE — Modifier on the Save button.
WHEN — Re-evaluated on each keystroke.
WHY — Prevents empty sessions. A unit test on the store could check this rule too.

```
17  .accessibilityIdentifier("saveSessionButton")
```

WHO — SwiftUI, writing into the accessibility tree.
WHAT — **Tags the button for app.buttons["saveSessionButton"].**
WHERE — Modifier on the Save button.
WHEN — Applied on every render.
WHY — Identifiers use lowerCamelCase names that describe purpose, not appearance.

CODE · READ-ALONG

Known data through launch arguments

A UI test must start from data it knows, not whatever was left in the simulator. The test passes a **launch argument**, a string handed to the app at launch, and the app checks for it before choosing its data.

```
import Foundation
import Observation

@Observable final class SessionStore {
    private(set) var titles: [String]

    init(titles: [String]) { self.titles = titles }

    func add(_ title: String) {
        titles.insert(title, at: 0)
    }

    static func forCurrentLaunch() -> SessionStore {
        let arguments = ProcessInfo.processInfo.arguments
        if arguments.contains("-ui-testing") {
            return SessionStore(titles: ["Algebra review",
                                         "Biology flashcards"])
        }
        return SessionStore(titles: [])
    }
}
```

Line by line

1 `import Foundation`

- WHO — The Swift compiler, building the store's file.
- WHAT — **Imports Foundation for ProcessInfo.**
- WHERE — Top of the store's file.
- WHEN — At compile time, before the store's file can use ProcessInfo.
- WHY — Launch arguments are read through Foundation.

2 `import Observation`

- WHO — The Swift compiler, building the store's file.
- WHAT — **Imports the @Observable macro.**
- WHERE — Top of the store's file, after Foundation.
- WHEN — At compile time, before the @Observable macro can expand.
- WHY — The store is observable so the list redraws when a session is added.

4 `@Observable final class SessionStore {`

- WHO — You declare it; the app creates one.
- WHAT — **The shared, observable list of session titles.**
- WHERE — In Core/Models/, shared by every feature.
- WHEN — Created once at launch by forCurrentLaunch().
- WHY — One store shared by the list and the sheet keeps both screens consistent.

```
5  private(set) var titles: [String]
```

WHO The store, which owns every write.
WHAT **The session titles, newest first.**
WHERE A tracked stored property of the store.
WHEN Read by the list on each redraw.
WHY A real app would store full StudySession models in SwiftData; strings keep this example short.

```
7  init(titles: [String]) { self.titles = titles }
```

WHO Callers that create a store.
WHAT **Accepts the starting titles.**
WHERE The store's only initializer.
WHEN Once, whenever a store is created.
WHY Seeding through init makes every scenario (empty, seeded, huge) one line in a test or preview.

```
9  func add(_ title: String) {
```

WHO The sheet's Save button.
WHAT **Declares the method that adds a session title.**
WHERE An instance method of the store.
WHEN Each time the user taps Save on the sheet.
WHY All writes go through one method.

```
10 titles.insert(title, at: 0)
```

WHO The store, changing its own array.
WHAT **Puts the new title first.**
WHERE Inside add(_:), its only statement.
WHEN On each save; the list redraws right after.
WHY Newest first is a product decision that a unit test can pin down.

```
13 static func forCurrentLaunch() -> SessionStore {
```

WHO The app's entry point calls it once.
WHAT **Chooses which store this launch should use.**
WHERE A static factory on the type.
WHEN Once, at app launch, before the first screen appears.
WHY Keeping the decision in one function makes the test hook easy to find and audit.

```
14 let arguments = ProcessInfo.processInfo.arguments
```

WHO iOS, which passes the launch arguments to the process.
WHAT **Reads the array of strings the app was launched with.**
WHERE First line of the factory.
WHEN Once, at launch, when forCurrentLaunch() runs.
WHY XCUIApplication.launchArguments ends up here, so the test controls the app's starting state.

```
15 if arguments.contains("-ui-testing") {
```

WHO The app, deciding which data to start with.
WHAT **Checks whether a UI test launched it.**
WHERE Inside the factory, after reading the arguments.
WHEN Once, at launch, right after the arguments are read.
WHY Users never pass this flag, so normal launches are unaffected.

```
16  return SessionStore(titles: ["Algebra review",
```

WHO The factory, building the test store.
WHAT **Starts building a store seeded with known sessions.**
WHERE The testing branch of the if statement.
WHEN Only when the flag is present.
WHY Tests can now assert on exact rows such as "Algebra review".

```
17  "Biology flashcards"])
```

WHO The Swift compiler, reading the continued argument.
WHAT **Finishes the two-item seed list.**
WHERE Continuation of the call.
WHEN At launch, testing branch only.
WHY Seed data lives in memory only, so tests never touch or delete a real user's data.

```
19  return SessionStore(titles: [])
```

WHO The factory, building the normal store.
WHAT **Returns an empty store for a normal launch.**
WHERE The normal branch, after the if statement.
WHEN Every launch without the flag.
WHY A real app would load saved data here instead.

CODE · READ-ALONG

Write the UI test

This XCTest UI test performs one complete journey: launch with known data, open the sheet, type a title, save, and wait for the new row. Test methods start with `test`. Every query uses an identifier, so the test survives translation and redesign.

```swift
import XCTest

final class SessionFlowUITests: XCTestCase {
    @MainActor
    func testAddingASessionShowsItInTheList() throws {
        let app = XCUIApplication()
        app.launchArguments = ["-ui-testing"]
        app.launch()

        app.buttons["addSessionButton"].tap()
        let title = app.textFields["sessionTitleField"]
        title.tap()
        title.typeText("Chemistry quiz prep")
        app.buttons["saveSessionButton"].tap()

        let row = app.staticTexts["Chemistry quiz prep"]
        XCTAssertTrue(row.waitForExistence(timeout: 2))
    }
}
```

Line by line

```
1  import XCTest
```

WHO	The compiler, building the UI-testing target.
WHAT	**Imports XCTest, which provides XCUIApplication and assertions.**
WHERE	Top of a file in the UI-test target, not the app.
WHEN	At compile time, when the UI-test target builds.
WHY	UI tests are an XCTest feature; Swift Testing covers unit and integration tests.

```
3  final class SessionFlowUITests: XCTestCase {
```

WHO	XCTest discovers subclasses of XCTestCase.
WHAT	**Declares a class that groups UI tests for the sessions journey.**
WHERE	In the UI-test target, a separate bundle from the app.
WHEN	Instantiated by the runner for each test method.
WHY	Naming the class after the journey makes failures easy to locate.

```
4  @MainActor
```

WHO	The Swift compiler, which enforces actor isolation.
WHAT	**Runs the test on the main actor.**
WHERE	Attribute on the test method.
WHEN	Checked at compile time.
WHY	XCUIApplication APIs are main-actor isolated in current SDKs.

```
5  func testAddingASessionShowsItInTheList() throws {
```

WHO	XCTest finds methods whose names begin with test.
WHAT	**Declares one UI test with a sentence-like name.**
WHERE	Inside the test class.
WHEN	When you run the UI-test target or click its diamond.
WHY	The name states the expected behavior, so a failure reads like a broken requirement.

```
6  let app = XCUIApplication()
```

WHO	XCTest, the testing framework.
WHAT	**Creates a proxy for the app under test; it does not launch yet.**
WHERE	Inside the test, in the runner process.
WHEN	At the start of the test.
WHY	The test talks to the app through this proxy, never through your Swift types.

```
7  app.launchArguments = ["-ui-testing"]
```

WHO	The test, configuring the launch.
WHAT	**Passes the flag the store checks for.**
WHERE	On the proxy, before launch.
WHEN	Once per test, before every launch.
WHY	Known starting data makes the test repeatable on any simulator.

```
8  app.launch()
```

WHO	XCTest and the simulator.
WHAT	**Installs and starts the app, then waits until it is idle.**
WHERE	The simulator, in the app's own process.
WHEN	Takes a second or more; the test waits until the app is idle.
WHY	A fresh launch per test isolates tests from each other.

```
10  app.buttons["addSessionButton"].tap()
```

WHO XCTest, querying the accessibility tree.
WHAT **Finds the button with that identifier and taps it.**
WHERE Runs against the app's current screen.
WHEN Right after the app finishes launching.
WHY If no element matches, the test fails with a "No matches found" error at this line.

```
11  let title = app.textFields["sessionTitleField"]
```

WHO XCTest, building a query.
WHAT **Creates a query for the sheet's text field.**
WHERE Inside the test, in the runner process.
WHEN Resolved lazily, when it is used.
WHY Queries are descriptions, re-evaluated each time, so they stay valid after the screen changes.

```
12  title.tap()
```

WHO XCTest, acting like a finger on the screen.
WHAT **Taps the field so the keyboard appears.**
WHERE On the New Session sheet.
WHEN After the sheet has opened.
WHY Typing requires keyboard focus, exactly as for a person.

```
13  title.typeText("Chemistry quiz prep")
```

WHO XCTest, via the simulated keyboard.
WHAT **Types the title one character at a time.**
WHERE Into the field that has keyboard focus.
WHEN Right after the field is tapped.
WHY Real typing exercises the binding and the disabled-until-typed rule on Save.

```
14  app.buttons["saveSessionButton"].tap()
```

WHO XCTest, tapping on the user's behalf.
WHAT **Taps Save, which adds the session and closes the sheet.**
WHERE On the New Session sheet.
WHEN After the title has been typed.
WHY If Save were still disabled, the tap would do nothing and the next check would catch it.

```
16  let row = app.staticTexts["Chemistry quiz prep"]
```

WHO XCTest, building a query for the new row.
WHAT **Queries for a text element whose label is the new title.**
WHERE On the list screen, after the sheet is gone.
WHEN After the sheet's closing animation starts.
WHY The row's text is user data, not interface copy, so querying by label is safe here.

```
17  XCTAssertTrue(row.waitForExistence(timeout: 2))
```

WHO XCTest's assertion and waiting APIs.
WHAT **Waits up to two seconds for the row, then asserts it exists.**
WHERE The last line of the test.
WHEN After saving; it polls until the row appears or two seconds pass.
WHY Waiting for a condition instead of sleeping a fixed time keeps tests fast and stops flaky failures during animations.

INTERACTIVE

Query explorer

Pick a query and see which elements it matches in the accessibility tree. Then switch the app to Spanish: label-based queries stop matching, identifier queries keep working. Turn the `-ui-testing` argument off and watch the seeded rows disappear. Finally, run the UI test under each setting.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Audit accessibility and launch time

UI tests can check quality as well as behavior. An **accessibility audit** scans the current screen for problems such as low contrast, missing labels, small tap targets and text clipped at large sizes; Module 05 (Design) has the full audit checklist, and this test automates it. A **performance test** measures launch time and flags regressions against a recorded baseline.

```
1  import XCTest
2
3  final class QualityUITests: XCTestCase {
4      @MainActor
5      func testSessionListPassesAccessibilityAudit() throws {
6          let app = XCUIApplication()
7          app.launchArguments = ["-ui-testing"]
8          app.launch()
9          try app.performAccessibilityAudit()
10     }
11
12     @MainActor
13     func testLaunchPerformance() throws {
14         measure(metrics: [XCTApplicationLaunchMetric()]) {
15             XCUIApplication().launch()
16         }
17     }
18 }
```

Line by line

1 `import XCTest`

- WHO — The Swift compiler, building the UI-test target.
- WHAT — **Imports XCTest for UI tests and metrics.**
- WHERE — Top of a file in the UI-test target.
- WHEN — At compile time, when the UI-test target builds.
- WHY — Audits and launch metrics are XCTest features.

3 `final class QualityUITests: XCTestCase {`

- WHO — XCTest, which discovers XCTestCase subclasses.
- WHAT — **Groups quality checks that are not tied to one journey.**
- WHERE — In the UI-test target.
- WHEN — A new instance is created for each test method.
- WHY — Separate quality tests can run less often, for example only before a release.

```
4  @MainActor
```

WHO The Swift compiler, which enforces actor isolation.
WHAT **Main-actor isolation for the test.**
WHERE On the audit test method below.
WHEN Checked at compile time.
WHY Required to use XCUIApplication.

```
5  func testSessionListPassesAccessibilityAudit() throws {
```

WHO XCTest, which runs methods that start with test.
WHAT **A test that audits the first screen.**
WHERE Inside the quality test class.
WHEN When the UI-test target runs.
WHY throws is needed because the audit reports problems by throwing.

```
6      let app = XCUIApplication()
```

WHO XCTest, the testing framework.
WHAT **Creates the proxy for the app under test.**
WHERE First line of the audit test.
WHEN At the start of the test, before launch.
WHY Same setup as any UI test.

```
7      app.launchArguments = ["-ui-testing"]
```

WHO The test, configuring the launch.
WHAT **Launches with seeded rows.**
WHERE On the proxy, before launch.
WHEN Once, before the only launch in this test.
WHY An empty list would hide problems in the rows themselves.

```
8      app.launch()
```

WHO XCTest and the simulator.
WHAT **Starts the app and waits until it is idle.**
WHERE In the simulator, in the app's own process.
WHEN Before the audit, so there is a screen to inspect.
WHY The audit inspects whatever is on screen at this moment.

```
9      try app.performAccessibilityAudit()
```

WHO XCTest's audit engine, the same checks as Accessibility Inspector.
WHAT **Scans the screen for contrast, element descriptions, hit-region size, Dynamic Type and clipped text, recording a failure for each issue.**
WHERE Against the current screen.
WHEN After launch; call it again after navigating to audit other screens.
WHY It catches problems VoiceOver and low-vision users hit, automatically, on every run.

```
12 @MainActor
```

WHO The Swift compiler, which enforces actor isolation.
WHAT **Puts the performance test on the main actor.**
WHERE On the performance test method below.
WHEN Checked at compile time.
WHY XCUIApplication is main-actor isolated, so this test needs the same isolation as the audit test above.

13 `func testLaunchPerformance() throws {`

- **WHO** XCTest, which runs methods that start with test.
- **WHAT** **A performance test for cold launch.**
- **WHERE** Inside the quality test class, below the audit.
- **WHEN** When run, usually on a real device for meaningful numbers.
- **WHY** Launch time is the first impression; regressions sneak in one framework at a time.

14 `measure(metrics: [XCTApplicationLaunchMetric()]) {`

- **WHO** XCTest's measurement system, inherited from XCTestCase.
- **WHAT** **Runs the block several times and records launch duration.**
- **WHERE** The only statement of the performance test.
- **WHEN** Runs the block five times by default, then compares with the baseline.
- **WHY** Set a baseline once in Xcode; later runs fail if launch gets noticeably slower.

15 `XCUIApplication().launch()`

- **WHO** XCTest, starting a fresh copy of the app.
- **WHAT** **Launches the app from scratch.**
- **WHERE** Inside the measured block.
- **WHEN** Once per iteration of the measured block.
- **WHY** Measuring the real launch includes everything your app does at startup.

CODE · READ-ALONG

Previews as visual checks

Previews are the fastest visual test you have. Write one per interesting state: sample data, empty, the largest text size, dark mode. You see every state at a glance after each edit, without launching the app or tapping your way there.

```swift
import SwiftUI

extension SessionStore {
    static var sample: SessionStore {
        SessionStore(titles: ["Algebra review", "Biology flashcards"])
    }
}

#Preview("Sample data") {
    SessionListView(store: .sample)
}

#Preview("Empty state") {
    SessionListView(store: SessionStore(titles: []))
}

#Preview("Largest text") {
    SessionListView(store: .sample)
        .environment(\.dynamicTypeSize, .accessibility3)
}
```

Line by line

```
1  import SwiftUI
```

WHO	The Swift compiler, building the app target in a debug build.
WHAT	**Imports SwiftUI for #Preview and environment values.**
WHERE	Top of the previews file.
WHEN	At compile time, in debug builds where previews are compiled.
WHY	Previews are a SwiftUI and Xcode feature.

```
3  extension SessionStore {
```

WHO	You, adding members to an existing type without editing its file.
WHAT	**Adds sample data for previews.**
WHERE	Often in a Preview Content folder, so it is excluded from release builds.
WHEN	Compiled with the app; used only by previews.
WHY	One shared sample keeps every preview consistent.

```
4  static var sample: SessionStore {
```

WHO	The previews below, each of which reads it.
WHAT	**A computed property that builds a fresh seeded store on each use.**
WHERE	Static member of SessionStore.
WHEN	Each time a preview reads it.
WHY	A fresh instance per preview means one preview cannot change another's data.

```
5  SessionStore(titles: ["Algebra review", "Biology flashcards"])
```

WHO	The store's initializer, called by the sample property.
WHAT	**Creates a store with two known rows.**
WHERE	Body of the computed property.
WHEN	On each read, so every preview gets its own copy.
WHY	The same rows the UI test uses, so what you check by eye matches what tests check by code.

```
9  #Preview("Sample data") {
```

WHO	Xcode's preview canvas, showing the first of three tabs.
WHAT	**A named preview of the normal state.**
WHERE	At file scope, one of several previews.
WHEN	Whenever the canvas refreshes.
WHY	Names appear as tabs in the canvas, so each state is one click away.

```
10 SessionListView(store: .sample)
```

WHO	SwiftUI, rendering inside the canvas.
WHAT	**Renders the list with sample rows.**
WHERE	The body of the "Sample data" preview.
WHEN	Each time the preview refreshes.
WHY	The view takes its store as input, which is what makes previews this easy.

```
13 #Preview("Empty state") {
```

WHO	Xcode's preview canvas, showing the second tab.
WHAT	**A preview of the first-launch state.**
WHERE	At file scope, the second preview.
WHEN	Each time the canvas refreshes.
WHY	Empty states are easy to forget and are the first thing a new user sees.

```
14 SessionListView(store: SessionStore(titles: []))
```

WHO SwiftUI, rendering inside the canvas.
WHAT **Renders the list with no rows.**
WHERE The body of the "Empty state" preview.
WHEN Each time the preview refreshes.
WHY Seeing a blank screen here tells you the view needs a ContentUnavailableView.

```
17 #Preview("Largest text") {
```

WHO Xcode's preview canvas, showing the third tab.
WHAT **A preview at an accessibility text size.**
WHERE At file scope, the third preview.
WHEN Each time the canvas refreshes.
WHY Large Dynamic Type sizes reveal clipped and overlapping text instantly.

```
18 SessionListView(store: .sample)
```

WHO SwiftUI, rendering inside the canvas.
WHAT **The same list, about to be enlarged.**
WHERE The body of the "Largest text" preview, before its modifier.
WHEN Each time the preview refreshes.
WHY Same data, different environment: one variable at a time.

```
19 .environment(\.dynamicTypeSize, .accessibility3)
```

WHO SwiftUI's environment, which passes values down to every child view.
WHAT **Forces a very large accessibility text size for this preview only.**
WHERE Modifier on the preview's view.
WHEN Each time the preview refreshes.
WHY The canvas's Variants button can also show every text size at once without code.

> **Try it**
>
> 1. Add a "Dark mode" preview that applies .preferredColorScheme(.dark).

CHECK YOURSELF

Check your understanding

1. A UI test uses `app.buttons["Save"]`. It passes in English and fails after the app is translated into Spanish. What is the best fix?
   A. Launch the app in English for UI tests, whatever the simulator language
   B. Query by identifier: `app.buttons["saveSessionButton"]`
   C. Add a sleep before the tap so the label can load
   D. Use `app.buttons.element(boundBy: 2)` instead
2. Why does the test set `app.launchArguments = ["-ui-testing"]` before `app.launch()`?
   A. So the app launches faster in the simulator during tests
   B. Because XCTest requires at least one launch argument
   C. So the app starts from known in-memory data
   D. So the app turns on VoiceOver for the accessibility checks

3. Which check belongs in a unit test rather than a UI test?
   A. `share(of:tip:people:)` returns nil for zero people
   B. Tapping Save on the sheet shows the new row in the list
   C. The first screen passes an accessibility audit
   D. Cold launch stays under its recorded baseline

4. A UI test sometimes fails because the new row appears after the sheet's closing animation. What should it do?
   A. Retry the whole test up to three times
   B. Sleep for five seconds before querying for the row
   C. Turn off the animation with `UIView.setAnimationsEnabled(false)`
   D. Wait with `waitForExistence(timeout:)`

Answers: UI Tests, Previews and Accessibility Checks

1. **B. Query by identifier: `app.buttons["saveSessionButton"]`** A subscript string matches an identifier or a label. Labels change with language; identifiers do not. Forcing English hides the bug from every other user, a sleep changes nothing about the label, and index-based queries break as soon as the layout changes.
2. **C. So the app starts from known in-memory data** The app checks for the flag and seeds a known store. Without it the test depends on whatever data the simulator holds, which differs between machines and runs.
3. **A. `share(of:tip:people:)` returns nil for zero people** The split rule is pure logic: a unit test checks it in milliseconds and points straight at the failure. The other three need the running app.
4. **D. Wait with `waitForExistence(timeout:)`** Waiting for a condition returns as soon as the row appears and fails only after the timeout. A fixed sleep is either too short (flaky) or too long (slow), retries hide the problem, and the test runner cannot reach into the app to switch off its animations.

Debugging and Instruments

CONCEPT

Read the crash before you change code

When a Swift program hits a programmer error it **traps**: it stops immediately instead of continuing with corrupt data. In Xcode, the debugger pauses on the failing line, the console prints the reason, and the Debug navigator (Command-7) shows the **stack trace**, the chain of function calls that led there. Read all three before editing anything.

Index out of range	An array was read at a position it does not have, such as sessions[3] in a three-item array.
Unexpectedly found nil	A force unwrap (!) met nil. Replace it with if let, guard let or ?? and handle the empty case.
EXC_BAD_ACCESS	Code touched memory that was already freed, often through unowned references or unsafe pointers.
Stack trace	Read from the top down until you reach the first frame from your own code; that is usually where to look.

> **Crashes after release**
>
> Crashes from testers and customers appear in Xcode's Organizer. They are **symbolicated**, meaning raw memory addresses are turned back into function names and line numbers, using the dSYM file saved with your archive.

COMPARE

Print debugging, then the fix

The average of 25, 30 and 40 minutes should be 31.67, but the screen shows 31. A temporary `print` reveals the values involved. The fix converts before dividing and guards the empty list, which would otherwise crash with a division by zero.

A: Before: a debug print exposes the bug

```
1  func averageMinutes(_ sessions: [Int]) -> Double {
2      let total = sessions.reduce(0, +)
3      print("debug: total=\(total) count=\(sessions.count)")
4      return Double(total / sessions.count)
5  }
6
7  print(averageMinutes([25, 30, 40]))
```

OUTPUT

```
debug: total=95 count=3
31.0
```

```
1  func averageMinutes(_ sessions: [Int]) -> Double {
```

WHO Your code, called by the stats screen.
WHAT **Declares a function that should return the average session length.**
WHERE At file scope, in the stats feature.
WHEN Each time the weekly summary is shown.
WHY Returning Double promises fractional minutes, a promise the buggy body breaks.

```
2  let total = sessions.reduce(0, +)
```

WHO reduce, combining every element with +.
WHAT **Adds up all the minutes into an Int: 95 here.**
WHERE First line of the function.
WHEN On each call, before the debug print.
WHY Summing with reduce(0, +) is clear and correct; the bug is on another line.

```
3  print("debug: total=\(total) count=\(sessions.count)")
```

WHO print, added temporarily while hunting the bug.
WHAT **Shows the two values that feed the division.**
WHERE Just before the suspicious line.
WHEN On each call, until you delete it.
WHY Printing the inputs to a calculation tells you whether the inputs or the calculation are wrong. Remove such prints before committing.

```
4  return Double(total / sessions.count)
```

WHO Integer division, then a conversion.
WHAT **Divides 95 by 3 as integers, which gives 31, then converts 31 to 31.0.**
WHERE Last line of the function.
WHEN On each call, after the debug print.
WHY The conversion happens too late: the fraction is already gone. An empty array would also crash here with a division by zero.

```
7  print(averageMinutes([25, 30, 40]))
```

WHO Top-level code, playing the stats screen.
WHAT **Calls the function and prints the result, 31.0.**
WHERE The program's top level, its entry point.
WHEN After the debug line prints.
WHY Seeing total=95 and count=3 but 31.0 points straight at the division.

B: After: convert first, guard the empty case

```
1  func averageMinutes(_ sessions: [Int]) -> Double {
2      guard !sessions.isEmpty else { return 0 }
3      let total = sessions.reduce(0, +)
4      return Double(total) / Double(sessions.count)
5  }
6
7  print(averageMinutes([25, 30, 40]))
8  print(averageMinutes([]))
```

OUTPUT

```
31.666666666666668
0.0
```

```swift
1  func averageMinutes(_ sessions: [Int]) -> Double {
```

WHO Your code, the same signature as before.
WHAT **Declares the corrected average function.**
WHERE At file scope, replacing the buggy version.
WHEN Each time the summary is shown.
WHY Keeping the signature means no caller has to change.

```swift
2  guard !sessions.isEmpty else { return 0 }
```

WHO guard, checked at run time.
WHAT **Returns 0 for a week with no sessions.**
WHERE First line, before any arithmetic.
WHEN On each call, before any arithmetic.
WHY Integer division by zero traps. Handling the empty case is a product decision written down in code.

```swift
3  let total = sessions.reduce(0, +)
```

WHO reduce, adding every element with +.
WHAT **Sums the minutes, as before.**
WHERE After the guard, so the array is known to be non-empty.
WHEN Only for a non-empty array.
WHY This line was never the problem, so it stays.

```swift
4  return Double(total) / Double(sessions.count)
```

WHO Floating-point division on two converted Doubles.
WHAT **Converts both numbers first, so 95 / 3 keeps its fraction.**
WHERE Last line of the function.
WHEN On each call with data.
WHY Convert before dividing whenever the result must keep a fraction.

```swift
7  print(averageMinutes([25, 30, 40]))
```

WHO Top-level code, playing the stats screen again.
WHAT **Prints 31.666666666666668, the full Double value.**
WHERE The program's top level.
WHEN Runs first, producing the first output line.
WHY The view should still round this for display, for example with a format style.

```swift
8  print(averageMinutes([]))
```

WHO Top-level code, trying the edge case.
WHAT **Prints 0.0 instead of crashing.**
WHERE The program's top level.
WHEN Runs second, producing the second output line.
WHY Each bug you fix deserves a regression test so it can never quietly return.

> **Takeaway**
>
> A print is a quick way to see values, but it changes the code and is easy to leave behind. The real fix is a one-line change, and the guard handles a crash nobody had seen yet. Turn both cases into unit tests so the bug stays fixed.

CONCEPT

Breakpoints and LLDB

A **breakpoint** pauses the app just before a line runs. Click a line number in Xcode's gutter to add one. While paused, the Variables view lists every local value, and the console accepts **LLDB** commands. LLDB

is the debugger built into Xcode.

v total	Reads a variable directly from memory without running code. It is the fastest and safest command.
p total	Evaluates an expression and prints it with type-aware formatting.
po model	Prints an object's description, the text from CustomStringConvertible or debugDescription.
Stepping	Step Over (F6) runs the line, Step Into (F7) enters the call, Step Out (F8) finishes the function, Continue resumes.
Condition	Edit a breakpoint and add a condition such as minutes < 0 so it stops only when the bug happens.

> **Debug without editing code**
>
> Give a breakpoint a Log Message action and tick "Automatically continue". It prints like `print` but never touches your source, so nothing slips into a commit.

CODE · READ-ALONG

Log with Logger, not print

`print` output vanishes once the app leaves Xcode. The unified logging system keeps messages with a level, a subsystem and a category, so you can filter them in Xcode's console or the Console app. It also hides private values from logs collected off the device.

```
import OSLog

extension Logger {
    static let sync = Logger(subsystem: "com.example.StudyBuddy",
                             category: "sync")
}

struct SyncService {
    func upload(_ count: Int, for email: String) async {
        Logger.sync.info("Uploading \(count) sessions")
        Logger.sync.debug("Account: \(email, privacy: .private)")
        do {
            try await Task.sleep(for: .milliseconds(200))
            Logger.sync.notice("Upload finished")
        } catch {
            Logger.sync.error("Upload failed: \(error)")
        }
    }
}
```

Line by line

```
1  import OSLog
```

WHO The Swift compiler, building the service's file.
WHAT **Imports Apple's unified logging API, which provides Logger.**
WHERE Top of the service's file.
WHEN At compile time, before Logger can be referenced.
WHY Logger is part of the operating system, so it adds no dependency and very little overhead.

```
3  extension Logger {
```

WHO You, adding members to the system Logger type.
WHAT **Starts an extension that defines the app's named loggers.**
WHERE At file scope, often in Core/Logging.swift.
WHEN Compiled once; used everywhere.
WHY One place for every logger keeps subsystem and category names consistent across the app.

```
4  static let sync = Logger(subsystem: "com.example.StudyBuddy",
```

WHO The logging system, which tags each message.
WHAT **Creates a logger whose subsystem is the app's bundle identifier.**
WHERE A static constant on Logger.
WHEN Created the first time anything uses Logger.sync.
WHY Using the bundle ID as the subsystem lets you filter the console to just your app.

```
5  category: "sync")
```

WHO The logging system, which stores the category with every message.
WHAT **Names the category, the part of the app this logger covers, and closes the call.**
WHERE Second argument of the initializer, aligned under the first.
WHEN Set once, when the logger is created.
WHY Categories let you show only sync messages while chasing a sync bug.

```
8  struct SyncService {
```

WHO You declare it; the app calls it when sessions change.
WHAT **A service that uploads study sessions.**
WHERE In Core/Services/, next to the other services.
WHEN Created once by the composition root.
WHY Services are where logging pays off most, because they talk to the outside world.

```
9  func upload(_ count: Int, for email: String) async {
```

WHO The view model, after the user saves.
WHAT **An async method that uploads some number of sessions for an account.**
WHERE An instance method of SyncService.
WHEN Each time a sync starts.
WHY The parameters are what we want in the logs, one public and one private.

```
10 Logger.sync.info("Uploading \(count) sessions")
```

WHO Logger, at the info level.
WHAT **Records the start of an upload with the number of sessions.**
WHERE First line of upload, before any work starts.
WHEN At the start of each sync.
WHY Numbers are public by default, so they appear in the log even when collected from a tester's device.

```
11 Logger.sync.debug("Account: \(email, privacy: .private)")
```

WHO Logger, at the debug level.
WHAT **Logs the email but marks it private.**
WHERE Second line of upload.
WHEN At the start of each sync.
WHY Private values appear while you debug in Xcode but are redacted in logs collected from a device, which protects users. Debug-level messages are not stored by default.

```
12 do {
```

WHO Swift's error-handling syntax, inside upload.
WHAT **Begins the block that can fail.**
WHERE Inside upload, around the call that can throw.
WHEN On each sync attempt.
WHY The catch below is where a failure is logged.

```
13 try await Task.sleep(for: .milliseconds(200))
```

WHO The Swift concurrency runtime.
WHAT **Waits 200 milliseconds, standing in for the real network call.**
WHERE Inside do; the thread is free while waiting.
WHEN During each sync, between the start and finish messages.
WHY Task.sleep throws if the task is cancelled, which gives the catch block something real to log.

```
14 Logger.sync.notice("Upload finished")
```

WHO Logger, at the notice level.
WHAT **Records a successful finish.**
WHERE Inside do, after the wait.
WHEN When the upload completes.
WHY notice is the default level and is kept on disk, so it is visible after the fact.

```
15 } catch {
```

WHO Swift's catch clause, paired with the do block in upload.
WHAT **Catches any error from the block.**
WHERE Directly after the do block.
WHEN Only when something throws.
WHY Every failure path deserves a log line; silent failures are the hardest bugs to find.

```
16 Logger.sync.error("Upload failed: \(error)")
```

WHO Logger, at the error level.
WHAT **Records the failure and the error's description.**
WHERE Inside the catch block.
WHEN Only when the upload fails or is cancelled.
WHY Error-level messages stand out in the console and are always stored.

CONCEPT

Visual debuggers and Instruments

Some bugs are easier to see than to read. Xcode can pause the app and draw its views or its objects. **Instruments** records the app over time. Profile with Command-I: it builds an optimized Release version and opens Instruments. Measure on a real device whenever you can.

View hierarchy Debug > View Debugging shows every view in an exploded 3D stack: find hidden, zero-size or clipped views.

Memory graph	The Debug Memory Graph button draws live objects and their references. Leaked objects get a purple warning.
Time Profiler	Samples call stacks many times a second to show where CPU time goes, especially on the main thread.
Allocations	Tracks every allocation. Repeat an action and look for memory that keeps growing.
Leaks and Hangs	Leaks finds unreachable memory; Hangs flags moments when the main thread stopped responding.

COMPARE

A retain cycle, and [weak self]

ARC frees an object when nothing holds it strongly. On the left, the model owns a timer whose closure captures `self` strongly: model to timer to closure to model, a **retain cycle**. Releasing the model frees nothing. On the right, `[weak self]` breaks the loop and both objects are freed.

A: Before: strong capture leaks

```swift
final class StudyTimer {
    var onTick: (() -> Void)?
    deinit { print("StudyTimer freed") }
}

final class SessionModel {
    var seconds = 0
    let timer = StudyTimer()
    init() {
        timer.onTick = { self.seconds += 1 }
    }
    deinit { print("SessionModel freed") }
}

var model: SessionModel? = SessionModel()
model = nil
print("screen closed")
```

OUTPUT

```
screen closed
```

1 `final class StudyTimer {`

WHO	You declare it; ARC (Automatic Reference Counting) manages its memory.
WHAT	**A class standing in for a repeating timer that calls back on every tick.**
WHERE	At file scope; each instance lives on the heap.
WHEN	An instance is created when a SessionModel is created.
WHY	Classes are reference types, so ARC counts how many strong references point at each instance; on this side that count never reaches zero.

```
2  var onTick: (() -> Void)?
```

WHO The timer stores the closure it will call.
WHAT **An optional closure property, called on each tick.**
WHERE A stored property of the timer; the closure itself is also a heap object.
WHEN Assigned once in SessionModel's init.
WHY Storing a closure keeps it, and everything it captured, alive for as long as the timer lives; here that includes the model.

```
3  deinit { print("StudyTimer freed") }
```

WHO ARC calls deinit automatically.
WHAT **Prints a message at the moment the timer is destroyed.**
WHERE Inside StudyTimer; a class can have at most one deinit.
WHEN Exactly when the last strong reference to the timer goes away.
WHY A print in deinit is the simplest leak detector; on this side the line never appears, because the timer never dies.

```
6  final class SessionModel {
```

WHO You declare it; a study screen would own one.
WHAT **The view model for a running study session.**
WHERE At file scope; instances live on the heap.
WHEN Created when the screen opens.
WHY It owns the timer, which is normal; the problem is that the timer's closure owns it back.

```
7  var seconds = 0
```

WHO The model owns this counter.
WHAT **Seconds studied so far.**
WHERE A stored property of the model.
WHEN Incremented by the timer's closure on each tick.
WHY The closure needs to reach this property, which is why it refers to self at all; a strong reference is the default when it does.

```
8  let timer = StudyTimer()
```

WHO The model holds a strong reference to its timer.
WHAT **Creates the timer together with the model.**
WHERE A stored property: model to timer is a strong arrow.
WHEN Created together with each model, before init's body runs.
WHY An owner holding what it uses strongly is correct; the cycle appears because the arrow also points back.

```
9  init() {
```

WHO Swift calls it when SessionModel() runs.
WHAT **The initializer that wires the timer to the model.**
WHERE Inside SessionModel, its only initializer.
WHEN Once per model, after all stored properties have values.
WHY self is fully usable here because every stored property already has a default value, which is also what lets the closure capture it.

```
10 timer.onTick = { self.seconds += 1 }
```

WHO The closure, which captures self strongly by default.
WHAT **Stores a closure in the timer that increments the model's seconds.**
WHERE Inside init; the closure lives on the heap, owned by the timer.
WHEN Assigned once, during initialization.
WHY Now model holds timer, timer holds closure, closure holds model. The loop keeps every count above zero forever.

12 `deinit { print("SessionModel freed") }`

- **WHO** ARC, the moment the model's count reaches zero.
- **WHAT** **Prints when the model is destroyed.**
- **WHERE** Inside SessionModel, after init.
- **WHEN** When the model's strong reference count reaches zero.
- **WHY** The model's deinit should run before the timer's; on this side neither runs, which is the symptom to look for.

15 `var model: SessionModel? = SessionModel()`

- **WHO** Top-level code, playing the screen that owns the model.
- **WHAT** **Creates a model and holds it in an optional variable.**
- **WHERE** A global variable in this program.
- **WHEN** When the program starts.
- **WHY** An optional lets the program drop its reference by assigning nil, like a screen closing; the test is whether that frees anything.

16 `model = nil`

- **WHO** Top-level code, releasing its reference.
- **WHAT** **Drops the only reference the "screen" held.**
- **WHERE** The global variable that played the screen's reference.
- **WHEN** Immediately after creation, before any tick could happen.
- **WHY** Nothing else should hold the model strongly, but the closure does, so ARC frees nothing on this line.

17 `print("screen closed")`

- **WHO** Swift's print function, writing the only line this side produces.
- **WHAT** **Marks the point after the screen let go.**
- **WHERE** Standard output, the console below the code.
- **WHEN** After the release, as the last line of the program.
- **WHY** Any deinit message that appears before this line proves memory was freed in time; on this side none does.

B: After: [weak self] breaks the cycle

```swift
final class StudyTimer {
    var onTick: (() -> Void)?
    deinit { print("StudyTimer freed") }
}

final class SessionModel {
    var seconds = 0
    let timer = StudyTimer()
    init() {
        timer.onTick = { [weak self] in self?.seconds += 1 }
    }
    deinit { print("SessionModel freed") }
}

var model: SessionModel? = SessionModel()
model = nil
print("screen closed")
```

OUTPUT

```
SessionModel freed
StudyTimer freed
screen closed
```

```
1  final class StudyTimer {
```

WHO You declare it; ARC (Automatic Reference Counting) manages its memory.
WHAT **A class standing in for a repeating timer that calls back on every tick.**
WHERE At file scope; each instance lives on the heap.
WHEN An instance is created when a SessionModel is created.
WHY Classes are reference types, so ARC counts strong references; on this side the count does reach zero, and the deinit below proves it.

```
2  var onTick: (() -> Void)?
```

WHO The timer stores the closure it will call.
WHAT **An optional closure property, called on each tick.**
WHERE A stored property of the timer; the closure itself is also a heap object.
WHEN Assigned once in SessionModel's init.
WHY Storing a closure keeps it alive for as long as the timer lives, but with a weak capture the closure no longer keeps the model alive.

```
3  deinit { print("StudyTimer freed") }
```

WHO ARC calls deinit automatically.
WHAT **Prints a message at the moment the timer is destroyed.**
WHERE Inside StudyTimer; a class can have at most one deinit.
WHEN Exactly when the last strong reference to the timer goes away.
WHY A print in deinit is the simplest leak detector; on this side it prints second, right after the model is freed.

```
6  final class SessionModel {
```

WHO You declare it; a study screen would own one.
WHAT **The view model for a running study session.**
WHERE At file scope; instances live on the heap.
WHEN Created when the screen opens.
WHY It owns the timer, which is normal; with [weak self] the timer no longer owns it back.

```
7  var seconds = 0
```

WHO The model owns this counter.
WHAT **Seconds studied so far.**
WHERE A stored property of the model.
WHEN Incremented by the timer's closure on each tick.
WHY The closure still needs to reach this property, so it refers to self, but through a weak reference it does not own.

```
8  let timer = StudyTimer()
```

WHO The model holds a strong reference to its timer.
WHAT **Creates the timer together with the model.**
WHERE A stored property: model to timer is a strong arrow.
WHEN Created together with each model, before init's body runs.
WHY An owner holding what it uses strongly is correct; this arrow stays strong, and only the arrow pointing back is weakened.

```
9  init() {
```

WHO Swift calls it when SessionModel() runs.
WHAT **The initializer that wires the timer to the model.**
WHERE Inside SessionModel, its only initializer.
WHEN Once per model, after all stored properties have values.
WHY self is fully usable here because every stored property already has a value, so the capture list can refer to it.

```
10  timer.onTick = { [weak self] in self?.seconds += 1 }
```

WHO The closure, with a capture list.
WHAT **Captures self weakly, so the closure refers to the model without keeping it alive.**
WHERE Inside init; the weak reference becomes nil automatically when the model dies.
WHEN Assigned once; checked on every tick.
WHY A weak reference does not add to the count. self? safely does nothing if the model is already gone.

```
12  deinit { print("SessionModel freed") }
```

WHO ARC, the moment the model's count reaches zero.
WHAT **Prints when the model is destroyed.**
WHERE Inside SessionModel, after init.
WHEN When the model's strong reference count reaches zero.
WHY The model's deinit runs before ARC releases the timer it owns, so this line prints first.

```
15  var model: SessionModel? = SessionModel()
```

WHO Top-level code, playing the screen that owns the model.
WHAT **Creates a model and holds it in an optional variable.**
WHERE A global variable in this program.
WHEN When the program starts.
WHY An optional lets the program drop its reference by assigning nil, like a screen closing; here that is enough to free both objects.

```
16  model = nil
```

WHO Top-level code, releasing its reference.
WHAT **Drops the only reference the "screen" held.**
WHERE The global variable that played the screen's reference.
WHEN Immediately after creation, at the same moment as on the left.
WHY The closure holds the model only weakly, so this release drops the count to zero and ARC frees it here.

```
17  print("screen closed")
```

WHO Swift's print function, writing the last of three lines.
WHAT **Marks the point after the screen let go.**
WHERE Standard output, the console below the code.
WHEN After the release, as the last line of the program.
WHY Both deinit messages appear before this line, proving memory was freed in time.

> **Takeaway**
>
> A cycle needs every arrow in the loop to be strong. Break one with weak (it becomes nil when the object dies) or unowned (it promises the object outlives the reference, and crashes if wrong). Prefer weak for closures stored by objects you own and for delegates.

INTERACTIVE

Memory graph lab

Each box is a heap object, each arrow a reference, and each badge the strong-reference count ARC tracks. Pop the screen and watch ARC free what it can. With a strong capture, the remaining objects form a cycle and are flagged as leaked. Turn on `[weak self]`, reset and pop again. Then try the delegate scenario.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. The app stops with "Fatal error: Unexpectedly found nil while unwrapping an Optional value". What is the most likely cause?
   A. A retain cycle kept an object alive too long
   B. A force unwrap (`!`) met nil
   C. An array was indexed past its last element
   D. The main thread was blocked for too long
2. In the left program, `model = nil` runs but neither deinit message prints. Why?
   A. deinit only runs when the app quits, not when a variable becomes nil
   B. Top-level variables keep their objects alive until the program ends
   C. The closure captures the model strongly, so the cycle keeps it alive
   D. print does not work inside deinit, so the messages were lost
3. You want to pause only when `log(_:)` receives a negative number, without editing code. What do you use?
   A. A conditional breakpoint with the condition `minutes < 0`
   B. A `print` inside the guard, removed before committing
   C. The memory graph debugger, filtered to FocusModel
   D. Instruments Allocations, recording while you tap
4. Scrolling feels jerky and you suspect slow work on the main thread. Which tool shows where that time goes?
   A. The view hierarchy debugger
   B. Leaks
   C. Logger with the debug level
   D. Instruments Time Profiler

Answers: Debugging and Instruments

1. **B. A force unwrap (`!`) met nil** That message comes from `!` meeting nil. An index past the end reports "Index out of range", and retain cycles never crash; they quietly leak memory.
2. **C. The closure captures the model strongly, so the cycle keeps it alive** The model owns the timer, the timer owns the closure, and the closure owns the model. Releasing the outside reference leaves each object with one strong reference from the loop. `[weak self]` removes one arrow and the whole chain is freed.
3. **A. A conditional breakpoint with the condition `minutes < 0`** A condition makes the breakpoint stop only when the expression is true. A print requires editing source, and the memory tools answer different questions.
4. **D. Instruments Time Profiler** Time Profiler samples call stacks and shows which functions use the CPU, and on which thread. The view debugger shows layout and Leaks shows memory, not time.

Performance, Accessibility and Localization

CONCEPT

The main thread and the frame budget

iOS redraws the screen 60 times a second, or up to 120 on ProMotion displays, so each frame has about 16.7 ms (or 8.3 ms) to be ready. Every SwiftUI `body`, every tap handler and all layout run on the **main thread**. When main-thread work overruns the budget, a frame arrives late: a **hitch**, felt as a stutter. If the main thread stays blocked for 250 ms or more, Xcode reports a **hang**, and taps stop responding.

Keep body cheap	No decoding, sorting or formatter creation inside body. Compute once and store the result.
Go lazy	LazyVStack and List build rows only as they scroll into view; VStack builds every row up front.
Size images	Decode a thumbnail at display size, off the main actor, instead of a full camera photo.

> **Energy is performance too**
>
> Work that causes hitches also drains the battery: a busy CPU, timers that poll, best-accuracy location when "nearby" is enough, and many small network requests. Xcode's Energy gauge shows the cost while the app runs.

INTERACTIVE

Frame budget lab

Scroll a simulated list of study spots and watch each frame's time against the budget line. Decode full-size photos on the main thread at 120 Hz and count the hitches. Then move the work off the main actor, remove the formatter from `body`, and switch the eager `VStack` to `LazyVStack`. Notice which change fixes which symptom.

This step is an interactive simulation in the HTML version.

CODE · READ-ALONG

Move heavy work off the main actor

Decoding a 12-megapixel photo can take tens of milliseconds, several frames' worth. Here a function marked `@concurrent` decodes a small thumbnail on a background thread, and the row shows it when ready. The row's `body` stays fast because it only displays a finished image.

```swift
import SwiftUI

@concurrent
func makeThumbnail(from data: Data) async -> UIImage? {
    guard let image = UIImage(data: data) else { return nil }
    let size = CGSize(width: 160, height: 160)
    return await image.byPreparingThumbnail(ofSize: size)
}

struct SpotRow: View {
    let spot: StudySpot
    @State private var thumbnail: UIImage?

    var body: some View {
        HStack {
            if let thumbnail { Image(uiImage: thumbnail) }
            Text(spot.name)
        }
        .task {
            thumbnail = await makeThumbnail(from: spot.photoData)
        }
    }
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, building the spots feature.
- **WHAT** **Imports SwiftUI, which also brings in UIImage from UIKit on iOS.**
- **WHERE** Top of the spots feature's file.
- **WHEN** At compile time, before the app runs.
- **WHY** UIImage is still the standard way to decode image data on iOS.

3 `@concurrent`

- **WHO** The Swift 6.2 compiler and the concurrency runtime.
- **WHAT** **Marks the function below to always run on the shared background thread pool, never on the caller's actor.**
- **WHERE** An attribute on a global async function.
- **WHEN** Applied every time the function is called.
- **WHY** Even when called from main-actor code, the slow decoding now happens elsewhere, so frames keep flowing.

4 `func makeThumbnail(from data: Data) async -> UIImage? {`

- **WHO** Any caller that awaits it, here a row's .task.
- **WHAT** **Declares an async function that turns raw image bytes into a small image, or nil.**
- **WHERE** At file scope in the spots feature.
- **WHEN** Once per row, when the row first appears.
- **WHY** A free function with no shared state is safe to run concurrently for many rows at once.

```
5  guard let image = UIImage(data: data) else { return nil }
```

WHO — UIImage, which parses the compressed bytes.
WHAT — **Creates an image from the data, or returns nil if the bytes are not a valid image.**
WHERE — First line of the function, on a background thread.
WHEN — On each call, on a background thread.
WHY — Corrupt data becomes a missing thumbnail instead of a crash.

```
6  let size = CGSize(width: 160, height: 160)
```

WHO — Your code, choosing the target size.
WHAT — **The thumbnail size in pixels: 80 points on a 2x screen.**
WHERE — Inside the function, after the image is known to be valid.
WHEN — On each call, just before resizing.
WHY — Decoding at display size uses a fraction of the memory of a full photo, which also helps scrolling.

```
7  return await image.byPreparingThumbnail(ofSize: size)
```

WHO — UIKit's image pipeline, through byPreparingThumbnail(ofSize:).
WHAT — **Decodes and downsizes the image, then returns it ready to draw.**
WHERE — Last line of the function.
WHEN — Suspends until the thumbnail is ready, typically a few milliseconds.
WHY — A prepared image draws instantly; an undecoded one would be decoded on the main thread at first display.

```
10 struct SpotRow: View {
```

WHO — SwiftUI, which creates one per visible spot.
WHAT — **A row in the study-spot list.**
WHERE — In Features/Spots/, next to the list that uses it.
WHEN — Created as the row scrolls into a lazy list.
WHY — Rows are created often, so they must be cheap to build.

```
11 let spot: StudySpot
```

WHO — The list, which passes each spot in.
WHAT — **The model for this row: a name and photo bytes.**
WHERE — A stored property of the view.
WHEN — Set when the row is created.
WHY — The row receives data; it never fetches it.

```
12 @State private var thumbnail: UIImage?
```

WHO — SwiftUI, which stores the image between redraws.
WHAT — **The decoded thumbnail, nil until ready.**
WHERE — View state for this row only.
WHEN — Set once, when the task finishes.
WHY — Storing the result means decoding happens once, not on every redraw.

```
14 var body: some View {
```

WHO — SwiftUI, on the main thread.
WHAT — **Describes the row using only ready-made values.**
WHERE — The View protocol's only requirement.
WHEN — On appearance and when thumbnail changes.
WHY — Nothing slow happens here, so this body fits easily inside a frame.

```
15 HStack {
```

WHO SwiftUI's layout system, placing two children side by side.
WHAT **Places the image and the name side by side.**
WHERE Root of the row's content.
WHEN On each redraw of the row.
WHY Standard stacks are fast to lay out.

```
16 if let thumbnail { Image(uiImage: thumbnail) }
```

WHO SwiftUI, evaluating the if statement inside the view builder.
WHAT **Shows the thumbnail once it exists, and nothing before.**
WHERE First child of the HStack.
WHEN On each redraw; true after the task finishes.
WHY A placeholder or ProgressView could fill the gap; either way, body never waits for decoding.

```
17 Text(spot.name)
```

WHO SwiftUI, rendering a text label.
WHAT **Shows the spot's name next to the thumbnail.**
WHERE Second child of the HStack.
WHEN On each redraw, including the very first one.
WHY The text appears immediately, before the image is ready.

```
19 .task {
```

WHO SwiftUI, which starts an async task tied to the view's lifetime.
WHAT **Runs the closure when the row appears.**
WHERE Modifier on the HStack, the row's root view.
WHEN When the row appears; cancelled automatically if it scrolls away first.
WHY Automatic cancellation avoids wasted work for rows the user flicked past.

```
20 thumbnail = await makeThumbnail(from: spot.photoData)
```

WHO The row, on the main actor, awaiting background work.
WHAT **Waits for the thumbnail, then stores it, which triggers one redraw.**
WHERE Inside the task closure.
WHEN After decoding completes in the background.
WHY The main thread is free during the await and only does the cheap assignment afterwards.

CONCEPT

An accessibility pass before every release

The World Health Organization estimates that one in six people lives with a significant disability, and everyone has moments of bright sunlight, tired eyes or one free hand. iOS supplies the tools; your job is to not break them. Check each screen against this short list, use Accessibility Inspector (Xcode > Open Developer Tool) to audit it, and automate the audit with `performAccessibilityAudit()`. Module 05 (Design) walks through the full audit checklist.

VoiceOver	Every control has a spoken label; icon-only buttons need accessibilityLabel. Decorative images are hidden.
Dynamic Type	Use text styles such as .body and test at the largest accessibility sizes. Nothing is clipped.
Contrast	Body text needs at least 4.5:1 against its background, in light and dark mode.
Targets	Tap targets are at least 44 by 44 points.

Motion and color Respect Reduce Motion, and never use color alone to show state.

CODE · LIVE SWIFTUI PREVIEW

Make a custom control speak

An icon-only button is just "plus circle fill" to VoiceOver unless you describe it. Here the button gets a label (what it is), a value (its current state) and a hint (what happens). The decorative circle is hidden. Turn on VoiceOver on a device, or use Accessibility Inspector, to hear the difference.

```swift
import SwiftUI

struct GoalRow: View {
    @State private var minutes = 45
    var body: some View {
        HStack(spacing: 16) {
            Circle().stroke(Color.green, lineWidth: 6)
                .frame(width: 44, height: 44)
                .accessibilityHidden(true)
            Text("Algebra")
            Spacer()
            Button {
                minutes = min(minutes + 15, 60)
            } label: {
                Image(systemName: "plus.circle.fill").font(.title)
            }
            .accessibilityLabel("Add 15 minutes")
            .accessibilityValue("\(minutes) of 60 minutes")
            .accessibilityHint("Logs more study time for Algebra")
        }
        .padding()
    }
}

#Preview { GoalRow() }
```

Line by line

1 `import SwiftUI`

- WHO — The Swift compiler, building this file.
- WHAT — **Imports SwiftUI, including its accessibility modifiers.**
- WHERE — Top of the row's file.
- WHEN — At compile time, before the app runs.
- WHY — Accessibility in SwiftUI is a set of ordinary modifiers, no extra framework.

3 `struct GoalRow: View {`

- WHO — You declare it; SwiftUI renders it.
- WHAT — **A row showing one subject and a button to log time.**
- WHERE — At file scope, in the goals feature.
- WHEN — Created by the parent list.
- WHY — Custom rows are exactly where accessibility gets forgotten.

```
4  @State private var minutes = 45
```

WHO SwiftUI, which stores the value.
WHAT **Minutes logged for this subject, starting at 45.**
WHERE View state owned by this row.
WHEN Starts at 45 and changes on each tap of the button.
WHY The value read aloud must follow this state, so it is computed from it below.

```
5  var body: some View {
```

WHO SwiftUI, which calls it on the main thread.
WHAT **Describes the row and its accessibility information together.**
WHERE The View protocol's only requirement.
WHEN On appearance and after each tap.
WHY Because labels live in body, they update automatically with the state.

```
6  HStack(spacing: 16) {
```

WHO SwiftUI's layout system, arranging the ring, name and button.
WHAT **Places the ring, the name and the button in a row.**
WHERE Root of the row's content.
WHEN On each redraw, after every tap.
WHY VoiceOver visits the children in reading order: left to right.

```
7  Circle().stroke(Color.green, lineWidth: 6)
```

WHO SwiftUI's shape drawing, through Circle and stroke.
WHAT **Draws a green ring as decoration.**
WHERE First child of the row, at the leading edge.
WHEN On each redraw; its look never changes.
WHY Color alone carries no meaning here, so nothing is lost for people who cannot see it.

```
8  .frame(width: 44, height: 44)
```

WHO SwiftUI's layout system, sizing the circle.
WHAT **Sizes the ring to 44 by 44 points.**
WHERE Modifier on the decorative circle.
WHEN On each layout pass.
WHY 44 points is Apple's minimum comfortable size for anything a finger aims at.

```
9  .accessibilityHidden(true)
```

WHO SwiftUI, editing the accessibility tree.
WHAT **Removes the decorative ring from VoiceOver.**
WHERE Modifier on the decorative circle.
WHEN Applied on every render.
WHY Hiding decoration means VoiceOver users do not hear "image" on every row.

```
10 Text("Algebra")
```

WHO SwiftUI, rendering text that VoiceOver can read.
WHAT **Shows the subject name, which VoiceOver reads as is.**
WHERE Middle of the row, after the ring.
WHEN On each redraw of GoalRow, before and after taps.
WHY Text is accessible by default and scales with Dynamic Type.

11 `Spacer()`

WHO SwiftUI's layout system, filling the leftover width.
WHAT **Pushes the button to the trailing edge.**
WHERE Between the name and the button.
WHEN On each layout pass, taking whatever width is left.
WHY Spacers have no accessibility element, so they cost nothing to VoiceOver.

12 `Button {`

WHO The user, or VoiceOver's double-tap, activates it.
WHAT **Starts a button with a custom label view.**
WHERE Last child of the row, at the trailing edge.
WHEN Its action runs on each activation.
WHY A real Button gets the button trait and keyboard support automatically; a tap gesture would not.

13 `minutes = min(minutes + 15, 60)`

WHO The button's action closure, on the main thread.
WHAT **Adds 15 minutes, capped at the 60-minute goal.**
WHERE Inside the action closure.
WHEN On each activation by a tap or a VoiceOver double-tap.
WHY The change updates the spoken value below, too.

14 `} label: {`

WHO Swift's multiple-trailing-closure syntax, read by the compiler.
WHAT **Switches from the action to the button's visible label.**
WHERE Between the two closures.
WHEN At compile time; it only separates the two closures.
WHY A custom label can be any view, which is why it needs an explicit spoken label.

15 `Image(systemName: "plus.circle.fill").font(.title)`

WHO SwiftUI, drawing an SF Symbol.
WHAT **Shows a large plus icon as the only visible label.**
WHERE Inside the label closure.
WHEN On each redraw of the button's label.
WHY Without help, VoiceOver would read the symbol's name, which means nothing to a student.

17 `.accessibilityLabel("Add 15 minutes")`

WHO VoiceOver, which speaks it first.
WHAT **Names what the button does.**
WHERE Modifier on the button, after its closing brace.
WHEN Read whenever VoiceOver focuses the button.
WHY A label is short and says what the control is, not how it looks.

18 `.accessibilityValue("\(minutes) of 60 minutes")`

WHO VoiceOver, which speaks it after the label.
WHAT **Reports the current progress, such as "45 of 60 minutes".**
WHERE Modifier on the button, after the label.
WHEN Re-read after each change.
WHY Values carry state that changes; labels stay constant.

```
19  .accessibilityHint("Logs more study time for Algebra")
```

WHO	VoiceOver, after a short pause.
WHAT	**Explains the result of activating the button.**
WHERE	Modifier on the button, after the value.
WHEN	Read only if the user waits, and can be turned off in settings.
WHY	Hints help newcomers without slowing down experienced users.

```
21  .padding()
```

WHO	SwiftUI's layout system, adding default insets.
WHAT	**Adds standard spacing around the row.**
WHERE	Modifier on the HStack.
WHEN	Each layout pass of GoalRow.
WHY	Comfortable spacing also makes targets easier to hit.

```
25  #Preview { GoalRow() }
```

WHO	Xcode's canvas, here the phone beside the code.
WHAT	**Shows the row in the canvas so you can tap and inspect it.**
WHERE	At file scope; compiled only for development tools.
WHEN	Whenever the file changes.
WHY	Pair the preview with Accessibility Inspector to hear exactly what VoiceOver will say.

> **Try it**
>
> 1. Write out what VoiceOver says for the button after two taps.
> 2. Delete .accessibilityHidden(true). What extra element would VoiceOver now stop on?

CODE · READ-ALONG

Localize with a String Catalog

Add a **String Catalog** (Localizable.xcstrings) and Xcode collects every user-facing string from SwiftUI at build time. Translators fill in each language there. Plurals and formatted numbers, durations and dates adapt to the user's language and region automatically.

```swift
import SwiftUI

struct WeekSummaryView: View {
    let sessionCount: Int
    let minutes: Int
    let weekStart: Date

    var body: some View {
        VStack(alignment: .leading) {
            Text("This week")
                .font(.headline)
            Text("\(sessionCount) sessions")
            Text(Duration.seconds(minutes * 60),
                 format: .units(allowed: [.hours, .minutes]))
            Text(weekStart, format: .dateTime.month().day())
        }
    }
}

func goalBanner() -> String {
    String(localized: "Weekly goal reached",
           comment: "Banner shown when the weekly goal is met")
}
```

Line by line

1 `import SwiftUI`

- **WHO** The Swift compiler, building this file.
- **WHAT** **Imports SwiftUI; Text initializers accept localizable keys.**
- **WHERE** Top of the stats feature's file.
- **WHEN** At compile time, before the app runs.
- **WHY** Localization in SwiftUI mostly happens by using Text with literals, which you already do.

3 `struct WeekSummaryView: View {`

- **WHO** SwiftUI, which renders it inside the stats screen.
- **WHAT** **A small summary card for the week.**
- **WHERE** In Features/Stats/, next to the stats screen.
- **WHEN** Created by the stats screen.
- **WHY** Every string in this view will be extracted for translation.

4 `let sessionCount: Int`

- **WHO** The parent screen, which passes it in.
- **WHAT** **How many sessions were logged this week.**
- **WHERE** A stored property of the view.
- **WHEN** Set when the parent creates the card.
- **WHY** A number that drives a plural phrase needs special handling in other languages.

5 `let minutes: Int`

- **WHO** The parent stats screen, which passes it in.
- **WHAT** **Total minutes studied this week.**
- **WHERE** A stored property of the view.
- **WHEN** Set when the parent creates the card.
- **WHY** Will be shown as a localized duration, not as raw minutes.

```
6  let weekStart: Date
```

WHO The parent stats screen, which passes it in.
WHAT **The first day of the week being summarized.**
WHERE A stored property of the view.
WHEN Set when the parent creates the card.
WHY Dates must be formatted for the user's region; month and day order differ worldwide.

```
8  var body: some View {
```

WHO SwiftUI, which calls it to draw the card.
WHAT **Describes the card's four lines.**
WHERE The View protocol's only requirement.
WHEN On appearance and when inputs change.
WHY Each Text below is looked up in the String Catalog at run time.

```
9  VStack(alignment: .leading) {
```

WHO SwiftUI's layout system, stacking the four lines.
WHAT **Stacks the lines, aligned to the leading edge.**
WHERE Root of the card's content.
WHEN On each redraw of WeekSummaryView.
WHY leading, not left: in right-to-left languages such as Arabic it flips automatically.

```
10 Text("This week")
```

WHO SwiftUI and the String Catalog.
WHAT **A string literal that Xcode extracts as a translatable key.**
WHERE First line of the card.
WHEN Looked up when the view renders, in the user's language.
WHY Text with a literal is localizable by default; Text(verbatim:) opts out for things like codes.

```
11 .font(.headline)
```

WHO SwiftUI's typography system, through the font modifier.
WHAT **Uses the headline text style.**
WHERE Modifier on the title text.
WHEN On each redraw, with the user's current text size.
WHY Text styles also scale with Dynamic Type, which matters more in languages with longer words.

```
12 Text("\(sessionCount) sessions")
```

WHO SwiftUI and the String Catalog.
WHAT **An interpolated string; the catalog key becomes "%lld sessions".**
WHERE Second line of the card.
WHEN Looked up on each render with the current count.
WHY In the catalog, choose Vary by Plural to provide "1 session" and "5 sessions", and the forms other languages need.

```
13 Text(Duration.seconds(minutes * 60),
```

WHO SwiftUI and Foundation's formatting.
WHAT **Starts a Text that displays a Duration value.**
WHERE Third line of the card.
WHEN On each render, using the user's current language.
WHY Formatting a typed value beats building "1 hr 35 min" by hand, which can only ever be English.

```
14  format: .units(allowed: [.hours, .minutes]))
```

WHO Foundation's Duration format style.
WHAT **Shows hours and minutes, such as "1 hr, 35 min" in English.**
WHERE Second argument of the Text.
WHEN Each render, using the user's locale.
WHY The same value reads "1 h y 35 min" for a Spanish-speaking user, with no extra code.

```
15  Text(weekStart, format: .dateTime.month().day())
```

WHO SwiftUI and Foundation's date format style.
WHAT **Shows the month and day in the user's regional order, such as "Oct 20" or "20 oct".**
WHERE Fourth and last line of the card.
WHEN On each render, using the user's region settings.
WHY Never hard-code date patterns like MM/dd; regions disagree about order and separators.

```
20  func goalBanner() -> String {
```

WHO Code that builds text outside a Text view, such as a view model preparing an alert message.
WHAT **Returns a localized String for use where Text is not available.**
WHERE At file scope, beside the view.
WHEN Each time a banner is built.
WHY Plain String values are not localized automatically; String(localized:) marks them.

```
21  String(localized: "Weekly goal reached",
```

WHO Foundation's localization lookup, through String(localized:comment:).
WHAT **Looks up the key in the String Catalog for the user's language.**
WHERE The function's only expression.
WHEN At run time, each time the function is called.
WHY Xcode also extracts this key into the catalog at build time.

```
22  comment: "Banner shown when the weekly goal is met")
```

WHO Translators, who read it in the catalog.
WHAT **Explains where and how the string is used.**
WHERE Second argument of String(localized:comment:).
WHEN Read by humans, never shown to users.
WHY Context prevents bad translations of short, ambiguous phrases.

CHECK YOURSELF

Check your understanding

1. A list stutters while scrolling on a 120 Hz iPhone. Time Profiler shows `UIImage(data:)` inside a row's `body`. What is the best fix?

   A. Lower the display to 60 Hz so each frame gets 16.7 ms
   B. Decode a thumbnail off the main actor and store it in state
   C. Wrap the row in a `VStack` so rows are built up front
   D. Call `UIImage(data:)` twice so the second call hits a cache

2. Which element should get `.accessibilityHidden(true)`?

   A. A decorative ring beside text that states the progress
   B. An icon-only delete button at the trailing end of each row
   C. The only text on the screen that explains why saving failed
   D. A toggle with no visible label at the end of a settings row

3. `Text("\(count) sessions")` reads "1 sessions" when count is 1. Where do you fix it?
   A. Add an if statement in body for count == 1
   B. Use `Text(verbatim:)` so the count is inserted as is
   C. In the String Catalog, set the key to Vary by Plural
   D. Store "session" and append "s" in code

4. What does a hang mean in Xcode's diagnostics?
   A. The app used more than 1 GB of memory at once
   B. One frame took longer than its 16.7 ms budget
   C. A network request waited more than 60 s
   D. The main thread was blocked for 250 ms or more

Answers: Performance, Accessibility and Localization

1. **B. Decode a thumbnail off the main actor and store it in state** Decoding on the main thread blows the 8.3 ms frame budget. Doing it once, in the background, at display size removes the work from every frame. You cannot control the refresh rate, and a VStack makes things worse by building every row.
2. **A. A decorative ring beside text that states the progress** Hide decoration that repeats information found elsewhere. The icon-only button and the unlabeled toggle need labels, not hiding, and hiding an error message would silence it for VoiceOver users.
3. **C. In the String Catalog, set the key to Vary by Plural** Plural rules differ by language: some have three or more forms. The catalog handles them per language. An if statement or adding "s" only works for English.
4. **D. The main thread was blocked for 250 ms or more** A late frame is a hitch; a long block of the main thread is a hang, where taps and scrolling freeze. Memory and network problems have their own reports.

From Archive to App Store

CONCEPT

Identity and signing

Shipping follows a fixed road: identity and signing, version and privacy, archive and upload, TestFlight, App Review, release and monitoring. It starts with identity. iOS runs only code that Apple can trace to a known developer, and **code signing** attaches that proof to every build. With **Automatically manage signing** checked in the target's Signing & Capabilities tab, Xcode creates and renews the pieces for you; you mainly choose a team and a bundle ID.

Bundle ID	Reverse-DNS name such as com.example.StudyBuddy. Unique worldwide, and fixed once a build is uploaded.
Team	An Apple Developer Program membership. A free personal team runs apps on your own device but cannot use TestFlight or the App Store.
Certificate	Proves who signed the build: Apple Development for testing, Apple Distribution for release.
Provisioning profile	Ties a bundle ID, a certificate, allowed devices and capabilities together.
Capabilities	Features such as App Groups (sharing data with a widget) or push notifications. Adding one updates the App ID and profile.

> **Choose the bundle ID carefully**
>
> After the first upload, the bundle ID can never change for that app record. Avoid version numbers or temporary names in it.

CODE · READ-ALONG

Version, build and configuration

Every upload needs a **version** users see (1.2.1) and a **build number** that is unique for that version (42). Xcode stores them as MARKETING_VERSION and CURRENT_PROJECT_VERSION. Showing both in an About row lets testers report exactly which build they used. `#if DEBUG` picks settings per **build configuration**: Debug while developing, Release when archiving.

```swift
import SwiftUI

enum AppInfo {
    static var version: String {
        let info = Bundle.main.infoDictionary
        let short = info?["CFBundleShortVersionString"] as? String
        let build = info?["CFBundleVersion"] as? String
        return "Version \(short ?? "?") (\(build ?? "?"))"
    }

    static var apiBase: URL {
        #if DEBUG
        URL(string: "https://staging.example.com")!
        #else
        URL(string: "https://api.example.com")!
        #endif
    }
}

struct AboutRow: View {
    var body: some View {
        LabeledContent("App", value: AppInfo.version)
    }
}
```

Line by line

1 `import SwiftUI`

WHO	The Swift compiler, building the settings feature.
WHAT	**Imports SwiftUI for the About row.**
WHERE	Top of the file that holds app-wide helpers.
WHEN	At compile time, before the app runs.
WHY	Bundle and URL come along with SwiftUI through Foundation.

3 `enum AppInfo {`

WHO	You declare it; nothing ever creates an instance.
WHAT	**A namespace for app-wide build information.**
WHERE	In Core/, shared by every feature.
WHEN	Its static members are evaluated when first read.
WHY	An enum with no cases cannot be instantiated, which makes it a tidy container for static helpers.

4 `static var version: String {`

WHO	The About row in Settings, which reads it.
WHAT	**A computed property that returns a display string such as "Version 1.2.1 (42)".**
WHERE	A static member of AppInfo.
WHEN	Each time something reads it; nothing is cached.
WHY	Computing from the bundle means the text can never disagree with the real build.

```
5  let info = Bundle.main.infoDictionary
```

WHO Foundation's Bundle, which reads the app's compiled Info.plist.
WHAT **Gets the dictionary of build settings stored inside the app.**
WHERE First line of the getter.
WHEN At run time, on each read.
WHY Xcode copies MARKETING_VERSION and CURRENT_PROJECT_VERSION into Info.plist when it builds the app.

```
6  let short = info?["CFBundleShortVersionString"] as? String
```

WHO Your code, reading one key.
WHAT **Gets the marketing version, the number users see in the App Store.**
WHERE Inside the getter, after the dictionary is read.
WHEN On each read of version.
WHY Each new App Store release needs a higher version than the one already live.

```
7  let build = info?["CFBundleVersion"] as? String
```

WHO Your code, reading another key.
WHAT **Gets the build number, which identifies one specific upload.**
WHERE Inside the getter, next to the version lookup.
WHEN On each read of version.
WHY Uploading the same version and build twice is rejected, so the number goes up with every upload.

```
8  return "Version \(short ?? "?") (\(build ?? "?"))"
```

WHO The getter, producing its result.
WHAT **Combines both values, falling back to "?" if a key is missing.**
WHERE Last line of the getter.
WHEN On each read, after both lookups.
WHY The nil-coalescing operator avoids force unwraps; a missing key shows "?" instead of crashing.

```
11 static var apiBase: URL {
```

WHO Services that build request URLs.
WHAT **Returns the server address for the current build configuration.**
WHERE A static member of AppInfo.
WHEN Each time a service builds a request URL.
WHY One switch point keeps test builds from writing into production data.

```
12 #if DEBUG
```

WHO The Swift compiler, at build time.
WHAT **Includes the next line only when the DEBUG flag is set, which Xcode does for the Debug configuration.**
WHERE A compile-time condition inside the getter.
WHEN Decided while building; the other branch is not compiled at all.
WHY Release builds cannot accidentally point at staging, because that code is not in them.

```
13 URL(string: "https://staging.example.com")!
```

WHO Foundation's URL initializer, parsing the staging literal.
WHAT **The staging server for development builds.**
WHERE Inside the DEBUG branch.
WHEN Returned in Debug builds only.
WHY A staging server lets you test against fake data without touching real accounts.

```
14 #else
```

WHO The Swift compiler's conditional compilation.
WHAT **Starts the branch used when DEBUG is not set.**
WHERE Between the two compile-time branches.
WHEN Decided at build time, not at run time.
WHY Archive builds use the Release configuration, so they always take this branch.

```
15 URL(string: "https://api.example.com")!
```

WHO Foundation's URL initializer, parsing the production literal.
WHAT **The production server address.**
WHERE Inside the release branch.
WHEN Returned in Release builds, including TestFlight and App Store builds.
WHY TestFlight runs the same Release build that ships, so testers see real behavior.

```
16 #endif
```

WHO The Swift compiler's conditional compilation.
WHAT **Ends the conditional compilation block.**
WHERE Closes the #if block inside the getter.
WHEN Decided at build time.
WHY Keep conditional blocks small so both configurations stay easy to read.

```
20 struct AboutRow: View {
```

WHO SwiftUI, inside the Settings screen.
WHAT **A row that displays the version and build.**
WHERE In Features/Settings/, beside the Settings screen.
WHEN Created when Settings opens.
WHY Testers can quote this text in TestFlight feedback.

```
21 var body: some View {
```

WHO SwiftUI, which calls it to draw the row.
WHAT **Describes the row's content.**
WHERE The View protocol's only requirement.
WHEN When the Settings screen shows the row.
WHY The row only displays; AppInfo does the work.

```
22 LabeledContent("App", value: AppInfo.version)
```

WHO SwiftUI's standard label-and-value control.
WHAT **Shows "App" on the leading side and the version string on the trailing side, the standard settings layout.**
WHERE The only content of the row's body.
WHEN On each render of the row.
WHY LabeledContent matches the look of Settings rows and reads well with VoiceOver.

CODE · READ-ALONG

The privacy manifest

Apps must explain their data use. **Purpose strings** in Info.plist explain each permission prompt. The **privacy manifest**, a file named PrivacyInfo.xcprivacy, declares tracking, collected data and any **required-reason APIs**, APIs that could fingerprint a device. Uploads that use them without a declared reason are rejected with ITMS-91053. This is a property list, not Swift.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
    "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>NSPrivacyTracking</key>
    <false/>
    <key>NSPrivacyCollectedDataTypes</key>
    <array/>
    <key>NSPrivacyAccessedAPITypes</key>
    <array>
        <dict>
            <key>NSPrivacyAccessedAPIType</key>
            <string>NSPrivacyAccessedAPICategoryUserDefaults</string>
            <key>NSPrivacyAccessedAPITypeReasons</key>
            <array>
                <string>CA92.1</string>
            </array>
        </dict>
    </array>
</dict>
</plist>
```

Line by line

1 `<?xml version="1.0" encoding="UTF-8"?>`

- **WHO** Xcode, which writes this header when you add a new App Privacy file.
- **WHAT** **Declares that the file is XML encoded as UTF-8.**
- **WHERE** First line of PrivacyInfo.xcprivacy, a resource in the app target.
- **WHEN** Read at build time and by App Store Connect at upload.
- **WHY** Property lists are XML; the header lets every tool parse the file the same way.

2 `<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"`

- **WHO** XML parsers, including Xcode's property-list reader.
- **WHAT** **Names the property-list document type.**
- **WHERE** Second line, continued on the next.
- **WHEN** Read whenever a tool parses the file.
- **WHY** It identifies the format; you never edit it by hand.

3 `"http://www.apple.com/DTDs/PropertyList-1.0.dtd">`

- **WHO** XML parsers, including Xcode's property-list reader.
- **WHAT** **Points to the official definition of the format.**
- **WHERE** Continuation of the DOCTYPE line.
- **WHEN** Read during parsing, never at app run time.
- **WHY** Split over two lines here only to fit the page.

4 `<plist version="1.0">`

- **WHO** Property-list parsers in Xcode and App Store Connect.
- **WHAT** **Opens the property list.**
- **WHERE** Root element of the file.
- **WHEN** Read at build and upload time.
- **WHY** Everything inside is plain data: keys, values, arrays and dictionaries.

```
5  <dict>
```

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Opens the top-level dictionary of privacy keys.**
WHERE Directly inside the plist element.
WHEN Read when the archive is uploaded.
WHY Each key below answers one privacy question about the app.

```
6  <key>NSPrivacyTracking</key>
```

WHO App Store Connect, which checks it against your answers.
WHAT **Asks whether the app tracks users across other companies' apps and websites.**
WHERE First key in the dictionary.
WHEN Read at upload and when the privacy report is generated.
WHY Tracking requires App Tracking Transparency permission; Study Buddy does not track.

```
7  <false/>
```

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Answers no to the tracking question.**
WHERE The value for NSPrivacyTracking.
WHEN Read when the archive is uploaded.
WHY An honest false means no tracking prompt and no tracking domains to list.

```
8  <key>NSPrivacyCollectedDataTypes</key>
```

WHO App Store Connect, comparing it with your App Privacy answers.
WHAT **Lists the kinds of data the app sends off the device.**
WHERE Second key in the top-level dictionary.
WHEN Read at upload, alongside the App Privacy answers you enter on the website.
WHY These declarations should match the privacy "nutrition label" on the App Store page.

```
9  <array/>
```

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **An empty list: Study Buddy collects nothing.**
WHERE The value for collected data types.
WHEN Read when the archive is uploaded.
WHY Location used only on the device to sort spots is not "collected". Collecting means sending data off the device where you or a partner can keep it.

```
10  <key>NSPrivacyAccessedAPITypes</key>
```

WHO App Store Connect's upload checks.
WHAT **Lists required-reason APIs the app calls.**
WHERE Third key in the top-level dictionary.
WHEN Read at upload; missing entries trigger ITMS-91053.
WHY Required-reason categories include user defaults, file timestamps, system boot time, disk space and active keyboards.

```
11  <array>
```

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Opens the list of API declarations.**
WHERE The value for accessed API types.
WHEN Read when the archive is uploaded.
WHY One dictionary per API category the app uses.

12 `<dict>`

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Opens one API declaration.**
WHERE First element of the array.
WHEN Read when the archive is uploaded.
WHY Each declaration pairs a category with the reasons you use it.

13 `<key>NSPrivacyAccessedAPIType</key>`

WHO App Store Connect's upload checks.
WHAT **Names which category this declaration covers.**
WHERE Inside the declaration dictionary.
WHEN Read when the archive is uploaded.
WHY Study Buddy saves settings with UserDefaults and @AppStorage, which counts.

14 `<string>NSPrivacyAccessedAPICategoryUserDefaults</string>`

WHO App Store Connect's upload checks.
WHAT **Names the UserDefaults category, which also covers @AppStorage.**
WHERE The value for the API type key.
WHEN Read when the archive is uploaded.
WHY Exact spelling matters; Xcode's property-list editor offers the valid names.

15 `<key>NSPrivacyAccessedAPITypeReasons</key>`

WHO App Store Connect's upload checks.
WHAT **Introduces the list of approved reasons.**
WHERE Inside the declaration, after the type.
WHEN Read when the archive is uploaded.
WHY You may only choose reason codes that Apple publishes for that category.

16 `<array>`

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Opens the list of reason codes.**
WHERE Value of the reasons key.
WHEN Read when the archive is uploaded.
WHY Several reasons are allowed if several apply.

17 `<string>CA92.1</string>`

WHO App Store Connect and App Review.
WHAT **Reason CA92.1: the app reads and writes defaults that only this app uses.**
WHERE The only reason in the list.
WHEN Checked at upload and may be checked in review.
WHY Declaring a reason is a promise; using the API for anything else breaks the guidelines.

18 `</array>`

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Closes the reasons list.**
WHERE End of the reasons value.
WHEN Read when the archive is uploaded.
WHY Tags must nest correctly or the upload fails to parse the file.

```
19 </dict>
```

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Closes the UserDefaults declaration.**
WHERE End of the first array element.
WHEN Read when the archive is uploaded.
WHY Add another dictionary here if the app later reads, say, disk space.

```
20 </array>
```

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Closes the list of API declarations.**
WHERE End of the accessed API types value.
WHEN Read when the archive is uploaded.
WHY Third-party SDKs ship their own manifests, which Xcode merges into the app's privacy report.

```
21 </dict>
```

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Closes the top-level dictionary.**
WHERE End of the privacy keys.
WHEN Read when the archive is uploaded.
WHY Xcode can generate a privacy report from the archive (Organizer, Generate Privacy Report).

```
22 </plist>
```

WHO Property-list parsers in Xcode and App Store Connect.
WHAT **Closes the property list.**
WHERE Last line of PrivacyInfo.xcprivacy.
WHEN Read when the archive is uploaded.
WHY The file must be a member of the app target or it is not copied into the build.

CONCEPT

TestFlight and App Review

TestFlight delivers beta builds to real people before release. Testers install the TestFlight app, run your build and send feedback with screenshots. **App Review** then checks the release build against Apple's App Review Guidelines. Most rejections are avoidable.

Internal testers	Up to 100 members of your App Store Connect team. Builds are available as soon as they finish processing.
External testers	Up to 10,000 people by email or public link. The first build of a version needs Beta App Review. Builds expire after 90 days.
2.1 Completeness	Crashes, placeholder text and broken links. Test the exact build you submit.
2.3 Metadata	Screenshots and descriptions must match what the app really does.
5.1.1 Data	Explain every permission. If users can create an account, they must be able to delete it in the app.

> **Help the reviewer**
> If the app needs a login, put a working demo account in App Review Information. Reviewers who cannot get past the first screen reject the build.

CONCEPT

After approval: release, watch, maintain

Once approved, you release manually, automatically or on a date. A **phased release** rolls an update out over seven days to people with automatic updates (1%, 2%, 5%, 10%, 20%, 50%, then 100%). If crash reports spike, pause it; anyone can still download the update manually. Shipping starts the maintenance phase of the life cycle, not the end of it.

Crashes	Xcode's Organizer collects crash reports from users who share analytics, symbolicated with your archive's dSYMs.
Hangs and energy	The Organizer also reports hangs, launch time and battery use from real devices.
Analytics and ratings	App Store Connect shows downloads, retention, ratings and reviews.
Maintenance release	Fix, test, bump to 1.2.1 with a new build number, TestFlight, submit. Request an expedited review only for critical bugs.

INTERACTIVE

Release pipeline lab

Ship Study Buddy 1.2.1. Each stage asks one release decision. A wrong choice shows what really happens (a rejected upload, a crash at launch, a reviewer's message), so you can try again. Stages unlock in order; select a locked stage to see what breaks if you skip it.

This step is an interactive simulation in the HTML version.

CHECK YOURSELF

Check your understanding

1. Version 1.2 (build 41) is live. You fix a crash. Which version and build should the upload use?
   A. 1.2.0 (42)
   B. 1.1 (42)
   C. 1.2.1 (42)
   D. 1.2 (41)
2. Why does an archived TestFlight build talk to `api.example.com` even though you were using staging all week?
   A. TestFlight rewrites every URL in the build to its production form
   B. Archive builds Release, which leaves out the DEBUG branch
   C. Apple blocks staging servers from TestFlight builds at upload time
   D. The privacy manifest lists only the production domain, so staging fails
3. An upload succeeds, but App Store Connect emails ITMS-91053: Missing API declaration. What is missing?
   A. A required-reason entry in PrivacyInfo.xcprivacy
   B. A purpose string for location in Info.plist
   C. A demo account entered under App Review Information
   D. A provisioning profile that includes the push capability

4. Day 3 of a phased release shows a spike in crashes on the new version. What is the best first move?
   A. Remove the app from sale until the crash is fixed
   B. Let the rollout reach day 7 so the fixed build can replace it sooner
   C. Ask affected users to delete and reinstall the app
   D. Pause the rollout, fix the crash and ship a maintenance release

Answers: From Archive to App Store

1. **C. 1.2.1 (42)** A new release needs a version higher than the live one, and every upload needs a build number not used before for that version. Reusing 1.2 (41) is rejected at upload, 1.1 is lower than the live version, and 1.2.0 (42) is a fresh build of the version that is already live, not a new release.
2. **B. Archive builds Release, which leaves out the DEBUG branch** Archive builds the Release configuration, where DEBUG is not set. The Debug branch is removed at compile time, so the build contains only the production address.
3. **A. A required-reason entry in PrivacyInfo.xcprivacy** ITMS-91053 is about required-reason APIs declared in the privacy manifest, such as UserDefaults with reason CA92.1. Missing purpose strings produce ITMS-90683, demo accounts matter only during review, and a profile problem stops the upload itself.
4. **D. Pause the rollout, fix the crash and ship a maintenance release** Pausing stops automatic updates to more users while you fix and test the bug, then ship 1.2.2. The rollout exists exactly to limit the damage of a bad build.

Capstone: Study Buddy from Idea to Release

CONCEPT

The life cycle, applied

The **software development life cycle** (SDLC) is the path every product follows: requirements, design, implementation, testing, deployment and maintenance. Modern teams run it in short **iterations**. Each loop delivers a small working slice, gathers feedback and shapes the next loop.

Your capstone, **Study Buddy**, begins with requirements. *Functional* requirements say what the app does; *non-functional* requirements say how well it must do it.

Plan and log	Plan study sessions, log minutes and see a weekly total, stored on the device with SwiftData.
Study spots	Show quiet places to study near the student, using location and MapKit.
Resources	Download study resources from a REST API with URLSession and keep them for offline use.
Widget	A home-screen widget shows the next scheduled session.
Qualities	Works offline, supports VoiceOver and Dynamic Type, keeps location on the device, launches in under a second.

CONCEPT

User stories and acceptance criteria

A **user story** states one requirement from the user's side: *As a student, I want to log a study session so that I can see my weekly total.* A good story is small enough for one iteration and valuable on its own.

Each story gets **acceptance criteria**, concrete conditions written as *Given* (the starting situation), *When* (the action) and *Then* (the observable result). If you cannot imagine the test, the criterion is too vague.

Given	a planner with 90 minutes of Algebra logged this week
When	the student logs a 30-minute Biology session
Then	the weekly total is 120 minutes, and the newest session is listed first
Too vague	"Logging feels quick" becomes "logging takes at most two taps from the Today screen".

INTERACTIVE

Story to test mapper

Pick a Study Buddy story. Sort each clause into Given, When or Then, or flag it as not testable. The Swift Testing skeleton on the right is generated as you go: misplaced clauses become TODO comments, and vague clauses get a measurable rewrite. Complete all three stories.

This step is an interactive simulation in the HTML version.

CONCEPT

Wireframes and architecture

A **wireframe** is a quick, low-detail sketch of each screen: boxes, labels and arrows between screens, no colors. Sketch Today, Spots, Resources and Settings, then walk every story through the sketches before writing code. Changing a sketch costs seconds; changing shipped code costs a release.

The **architecture diagram** gives every requirement a home, using the MVVM layers and feature folders from the first lesson.

Views	A TabView with Today, Spots and Resources tabs, one feature folder each.
View models	One @Observable view model for each screen that loads data or applies rules; a static screen stays a plain view.
Services	Location (CLLocationUpdate), spot search (MapKit) and resources (URLSession), each behind a protocol.
Storage	SwiftData models in an App Group container, so the widget extension reads the same store.
Widget	A WidgetKit extension whose timeline provider reads the next session.

CODE · RUNS IN THE BROWSER

Build the model behind the first story

Iteration 1 starts with the model for "log a study session". It has no user interface and no storage yet, so it is quick to write and to check. Run it: the output is exactly what the story's acceptance criteria promise.

```swift
struct StudySession: Equatable {
    let subject: String
    let minutes: Int
}

struct StudyPlanner {
    private(set) var sessions: [StudySession] = []

    var weeklyMinutes: Int {
        sessions.reduce(0) { $0 + $1.minutes }
    }

    mutating func log(_ session: StudySession) {
        guard session.minutes > 0 else { return }
        sessions.insert(session, at: 0)
    }
}

var planner = StudyPlanner()
planner.log(StudySession(subject: "Algebra", minutes: 90))
planner.log(StudySession(subject: "Biology", minutes: 30))
print(planner.weeklyMinutes, planner.sessions.first?.subject ?? "none")
```

OUTPUT

```
120 Biology
```

Line by line

1 `struct StudySession: Equatable {`

WHO	You declare it; Swift copies it like any value.
WHAT	**A model type for one logged session.**
WHERE	Core/Models/, shared by every feature.
WHEN	Created each time the student logs time.
WHY	Equatable lets tests compare whole sessions with ==. In the real app this becomes a SwiftData @Model class.

2 `let subject: String`

WHO	Each session value stores its own subject.
WHAT	**The subject studied, such as "Biology".**
WHERE	A stored constant of the struct.
WHEN	Set once, when the session is created.
WHY	let keeps a logged session from being edited by accident; editing would be its own story.

3 `let minutes: Int`

WHO	Each session value stores its own length.
WHAT	**How long the student studied, in whole minutes.**
WHERE	A stored constant of the struct.
WHEN	Set once, when the session is created.
WHY	Whole minutes match how students think about study time and avoid rounding questions.

6 `struct StudyPlanner {`

WHO	The Today screen's view model will own one.
WHAT	**A value type that holds the week's sessions and their rules.**
WHERE	Core/Models/, next to StudySession.
WHEN	Created when the app starts, or fresh in every test.
WHY	Keeping the rules in a plain struct makes each acceptance criterion a fast unit test.

7 `private(set) var sessions: [StudySession] = []`

WHO	The planner owns writes; screens and tests read.
WHAT	**The logged sessions, newest first, starting empty.**
WHERE	A stored property with a private setter.
WHEN	Read by the list; changed only by log(_:).
WHY	Forcing writes through log(_:) keeps the zero-minute rule impossible to bypass.

9 `var weeklyMinutes: Int {`

WHO	The weekly-total label reads it.
WHAT	**A computed property for the total minutes.**
WHERE	Inside StudyPlanner, below the stored list.
WHEN	Every time something reads it.
WHY	Computing the total instead of storing it means it can never disagree with the list.

10 `sessions.reduce(0) { $0 + $1.minutes }`

WHO	reduce, walking the array once.
WHAT	**Starts at 0 and adds each session's minutes; $0 is the running total, $1 the session.**
WHERE	Body of the computed property.
WHEN	On each read of weeklyMinutes.
WHY	One expression states the rule exactly: the total is the sum of all session lengths.

```swift
13 mutating func log(_ session: StudySession) {
```

WHO The Log button's view model, or a test.
WHAT **The only way to add a session.**
WHERE A mutating method, because it changes a struct's stored property.
WHEN Each time the student logs time.
WHY mutating makes the change visible at the call site: you can only call it on a var.

```swift
14 guard session.minutes > 0 else { return }
```

WHO The guard statement, checked at run time.
WHAT **Ignores sessions of zero or negative length.**
WHERE First line of log(_:), before any change.
WHEN On each call, before the array changes.
WHY This line exists because of an acceptance criterion: "a zero-minute session is not logged".

```swift
15 sessions.insert(session, at: 0)
```

WHO The planner, changing its own array.
WHAT **Adds the session at the front of the list.**
WHERE Inside log(_:), after validation.
WHEN For valid sessions only.
WHY Inserting at index 0 implements "the newest session is listed first" directly.

```swift
19 var planner = StudyPlanner()
```

WHO Top-level code, playing the Today screen.
WHAT **Creates an empty planner in a variable.**
WHERE The program's top level.
WHEN When the program starts.
WHY var is required because log(_:) mutates the struct.

```swift
20 planner.log(StudySession(subject: "Algebra", minutes: 90))
```

WHO Top-level code, setting up the Given.
WHAT **Logs 90 minutes of Algebra.**
WHERE The program's top level.
WHEN First change to the planner.
WHY This mirrors "Given a planner with 90 minutes of Algebra logged this week".

```swift
21 planner.log(StudySession(subject: "Biology", minutes: 30))
```

WHO Top-level code, performing the When.
WHAT **Logs a 30-minute Biology session.**
WHERE The program's top level.
WHEN Second change to the planner.
WHY This mirrors "When the student logs a 30-minute Biology session".

```swift
22 print(planner.weeklyMinutes, planner.sessions.first?.subject ?? "none")
```

WHO print, checking the Then by eye.
WHAT **Prints the total and the newest subject: "120 Biology".**
WHERE Standard output, the console below the code.
WHEN After both sessions are logged.
WHY Next you will turn this eyeball check into automated tests that run on every build.

> **Try it**
>
> 1. Log a third session of 0 minutes before the print. Predict the output, then run it.

CODE · SWIFT TESTING

Turn acceptance criteria into tests

Each acceptance criterion becomes a test, and the Given, When and Then become comments above the lines that perform them. When these tests pass, the story is verified. The `StudyPlanner` from the previous step is compiled with this file.

```swift
import Testing

@Suite("Story: log a study session")
struct LogSessionStoryTests {
    @Test("Logging a session updates the weekly total")
    func loggingUpdatesWeeklyTotal() {
        // Given a planner with 90 minutes of Algebra logged this week
        var planner = StudyPlanner()
        planner.log(StudySession(subject: "Algebra", minutes: 90))
        // When the student logs a 30-minute Biology session
        planner.log(StudySession(subject: "Biology", minutes: 30))
        // Then the weekly total is 120 minutes
        #expect(planner.weeklyMinutes == 120)
        // And the newest session is listed first
        #expect(planner.sessions.first?.subject == "Biology")
    }

    @Test("A zero-minute session is not logged")
    func zeroMinutesIgnored() {
        var planner = StudyPlanner()
        planner.log(StudySession(subject: "Biology", minutes: 0))
        #expect(planner.sessions.isEmpty)
    }
}
```

Line by line

1 `import Testing`

- WHO — The Swift compiler, building the unit-test target.
- WHAT — **Imports Swift Testing for @Suite, @Test and #expect.**
- WHERE — Top of the story's test file.
- WHEN — At compile time, when the test target builds.
- WHY — Acceptance tests for model logic are ordinary unit tests: fast and precise.

3 `@Suite("Story: log a study session")`

- WHO — The @Suite macro, read by Xcode's test discovery.
- WHAT — **Names the suite after the user story it verifies.**
- WHERE — Attached to the struct below.
- WHEN — Read at discovery, when Xcode lists the tests.
- WHY — Naming suites after stories gives a living checklist: a green suite means a verified story.

4 `struct LogSessionStoryTests {`

- WHO — The test runner, which creates a fresh instance per test.
- WHAT — **Holds every test for this story.**
- WHERE — In the unit-test target, under a Sessions folder.
- WHEN — A new instance per test.
- WHY — Fresh instances keep each criterion's test independent.

```
5  @Test("Logging a session updates the weekly total")
```

WHO The @Test macro, registering the main criterion's test.
WHAT **Registers the test with a display name taken from the criterion.**
WHERE Inside the suite, attached to the method below.
WHEN Read at discovery, when Xcode builds the list of tests.
WHY The report reads like the specification, so anyone can see which promise broke.

```
6  func loggingUpdatesWeeklyTotal() {
```

WHO The test runner, which calls it on a fresh suite instance.
WHAT **The test for the main acceptance criterion.**
WHERE An instance method of the suite.
WHEN When the suite runs, on Command-U or in continuous integration.
WHY One behavior per test keeps failures easy to diagnose.

```
7  // Given a planner with 90 minutes of Algebra logged this week
```

WHO Readers of the test; the compiler ignores comments.
WHAT **Labels the setup lines with the criterion's Given clause.**
WHERE Directly above the setup lines it describes.
WHEN Never runs; it is documentation for people.
WHY Copying the clause word for word links the code to the story it proves.

```
8  var planner = StudyPlanner()
```

WHO The test, setting up the Given.
WHAT **Creates an empty planner.**
WHERE First statement of the test.
WHEN At the start of the test.
WHY Each test builds its own planner, so no state leaks between tests.

```
9  planner.log(StudySession(subject: "Algebra", minutes: 90))
```

WHO The test, finishing the Given.
WHAT **Logs the 90 minutes the Given describes.**
WHERE The setup section, under the Given comment.
WHEN Before the action under test.
WHY Setup uses the same public method as the app, so it cannot create impossible states.

```
10 // When the student logs a 30-minute Biology session
```

WHO Readers; the compiler ignores comments.
WHAT **Labels the single action under test.**
WHERE Above the single action line.
WHEN Never runs; it is documentation for people.
WHY A test should have exactly one When; two actions usually mean two tests.

```
11 planner.log(StudySession(subject: "Biology", minutes: 30))
```

WHO The test, performing the When.
WHAT **Logs the 30-minute Biology session.**
WHERE The action section, under the When comment.
WHEN After the setup is complete.
WHY This is the behavior under test.

```
12  // Then the weekly total is 120 minutes
```

WHO Readers; the compiler ignores comments.
WHAT **Labels the first outcome.**
WHERE Directly above its expectation.
WHEN Never runs; it is documentation for people.
WHY Each Then clause gets its own expectation, so a report shows exactly which one failed.

```
13  #expect(planner.weeklyMinutes == 120)
```

WHO The #expect macro, checking the total.
WHAT **Checks the total is 90 plus 30.**
WHERE The assertion section, under the Then comment.
WHEN After the action, as the first check of the Then section.
WHY An exact number makes the criterion measurable.

```
14  // And the newest session is listed first
```

WHO Readers; the compiler ignores comments.
WHAT **Labels the second outcome, joined with And.**
WHERE Directly above the second expectation.
WHEN Never runs; it is documentation for people.
WHY And keeps several outcomes of one action in one readable scenario.

```
15  #expect(planner.sessions.first?.subject == "Biology")
```

WHO The #expect macro, checking the order.
WHAT **Checks the first session is the Biology one just logged.**
WHERE The assertion section, last line of the test.
WHEN After the action, even if the previous expectation failed.
WHY Optional chaining with == is safe: an empty list gives nil, which is not "Biology", so the test fails instead of crashing.

```
18  @Test("A zero-minute session is not logged")
```

WHO The @Test macro, registering the edge-case test.
WHAT **Registers a second criterion: invalid sessions are ignored.**
WHERE Inside the suite, below the first test.
WHEN Read at discovery, alongside the first test.
WHY Edge cases are acceptance criteria too; write them down with the story.

```
19  func zeroMinutesIgnored() {
```

WHO The test runner, which calls it on a fresh suite instance.
WHAT **The test for the zero-minute rule.**
WHERE An instance method of the suite.
WHEN When the suite runs, possibly in parallel with the first test.
WHY A short name for a short test.

```
20      var planner = StudyPlanner()
```

WHO The test, setting up an empty planner.
WHAT **Creates an empty planner: the Given is no sessions yet.**
WHERE First statement of the edge-case test.
WHEN At the start of the test.
WHY An empty start makes the expected result obvious.

```
21 planner.log(StudySession(subject: "Biology", minutes: 0))
```

WHO	The test, performing the When.
WHAT	**Tries to log a zero-minute session.**
WHERE	Second statement of the test.
WHEN	After the empty planner exists.
WHY	Zero is the boundary between valid and invalid, the most likely spot for a bug.

```
22 #expect(planner.sessions.isEmpty)
```

WHO	The #expect macro, checking the list.
WHAT	**Then: the list is still empty.**
WHERE	Last line of the edge-case test.
WHEN	After the attempted log.
WHY	If someone deletes the guard, this test fails before the bug reaches TestFlight.

CONCEPT

Plan the build, the tests and the release

Plans turn the design into a schedule. Build in thin vertical slices: each iteration delivers one story end to end, from view to storage, on a real device.

Iteration 1	A walking skeleton: tabs, the SwiftData model and "log a session" end to end, on TestFlight for internal testers.
Iteration 2	Study spots: the location permission flow, MapKit search and distance sorting.
Iteration 3	Resources from the REST API with offline caching, then the widget.
Test plan	A unit test per acceptance criterion, integration tests with an in-memory store, UI tests for the three core journeys and an accessibility audit per screen.
Release plan	Privacy manifest and purpose strings, external TestFlight with five students (validation), App Review, phased release, crash monitoring.

> **Definition of done**
>
> A story is done when its acceptance tests pass, its screens pass the accessibility audit and it runs on a real device. Compiling is not done.

CHECK YOURSELF

Check your understanding

1. Which acceptance criterion is testable as written?
   A. The spots list is pleasant to use
   B. Spots within 2 km are listed nearest first
   C. The widget always shows current data
   D. Logging a session feels quick

2. In "Given 90 minutes logged, When the student logs 30 more, Then the total is 120", which part becomes the `#expect` line?
   A. Then
   B. Given
   C. When
   D. The story title
3. All acceptance tests pass, but in a TestFlight session students ignore the weekly total entirely. What has happened?
   A. The tests are wrong and must be rewritten before release
   B. The app failed verification, so the tests were incomplete
   C. The app passed verification but not validation
   D. The phased release should be paused
4. Why does the widget extension read SwiftData from an App Group container?
   A. Widgets cannot use SwiftData directly, so the App Group holds a JSON copy
   B. An App Group container is faster to read than the app's own sandbox
   C. App Review rejects widgets whose data is not in an App Group container
   D. The extension is a separate process, so it needs a shared container

Answers: Capstone: Study Buddy from Idea to Release

1. **B. Spots within 2 km are listed nearest first** A distance and an order can be checked by code. "Pleasant", "always" and "feels quick" need rewriting into something measurable, such as "at most two taps".
2. **A. Then** Then describes the observable result, which is what an expectation checks. Given becomes setup code and When becomes the single action under test.
3. **C. The app passed verification but not validation** The tests prove the feature works as specified (verification). Real use shows whether it meets a real need (validation). That feedback goes into the next iteration's requirements.
4. **D. The extension is a separate process, so it needs a shared container** Each process has its own sandbox. An App Group capability creates a container both can open, so the widget sees the sessions the app saved. Widgets can use SwiftData directly, and the group is about access, not speed or review rules.

Worked Examples

WORKED EXAMPLE

Build a retain cycle, watch it leak, then break it

Problem.
A Pilot and a Ship hold each other. First build the strong two-way link and prove with deinit prints that releasing both variables frees nothing; then make one reference weak and watch both objects free.

Frame 1 of 6

Two classes, each with an optional reference to the other and a deinit print. Creating one of each is harmless; nothing links them yet, and the creation marker prints.

```
final class Pilot {
    var ship: Ship?
    deinit { print("Pilot freed") }
}
final class Ship {
    var pilot: Pilot?
    deinit { print("Ship freed") }
}
var pilot: Pilot? = Pilot()
var ship: Ship? = Ship()
print("Created pilot and ship")
```

OUTPUT SO FAR

```
Created pilot and ship
```

Frame 2 of 6

One direction of reference is added: the pilot holds the ship. That arrow alone is ordinary ownership; each object still also has its outside variable holding it.

```
final class Pilot {
    var ship: Ship?
    deinit { print("Pilot freed") }
}
final class Ship {
    var pilot: Pilot?
    deinit { print("Ship freed") }
}
var pilot: Pilot? = Pilot()
var ship: Ship? = Ship()
print("Created pilot and ship")
+ pilot?.ship = ship
+ print("Pilot holds the ship")
```

OUTPUT SO FAR

```
Created pilot and ship
Pilot holds the ship
```

Frame 3 of 6

The second direction closes the loop: the ship now holds the pilot too. Every object in the loop has a strong reference from inside the loop itself.

```
   final class Pilot {
       var ship: Ship?
       deinit { print("Pilot freed") }
   }
   final class Ship {
       var pilot: Pilot?
       deinit { print("Ship freed") }
   }
   var pilot: Pilot? = Pilot()
   var ship: Ship? = Ship()
   print("Created pilot and ship")
   pilot?.ship = ship
   print("Pilot holds the ship")
+  ship?.pilot = pilot
+  print("Cycle closed")
```

OUTPUT SO FAR

```
Created pilot and ship
Pilot holds the ship
Cycle closed
```

Frame 4 of 6

Both outside variables are set to nil, and the after release marker prints, but no freed lines appear before it. The cycle keeps both objects alive: this is the leak, proven.

```
   final class Pilot {
       var ship: Ship?
       deinit { print("Pilot freed") }
   }
   final class Ship {
       var pilot: Pilot?
       deinit { print("Ship freed") }
   }
   var pilot: Pilot? = Pilot()
   var ship: Ship? = Ship()
   print("Created pilot and ship")
   pilot?.ship = ship
   print("Pilot holds the ship")
   ship?.pilot = pilot
   print("Cycle closed")
+  pilot = nil
+  ship = nil
+  print("after release")
```

OUTPUT SO FAR

```
Created pilot and ship
Pilot holds the ship
Cycle closed
after release
```

Frame 5 of 6

The repair changes exactly one word: the ship reference back to the pilot becomes weak. The same creation and linking runs, and the printed markers are unchanged so far.

```
1    final class Pilot {
2        var ship: Ship?
3        deinit { print("Pilot freed") }
4    }
5    final class Ship {
6  +     weak var pilot: Pilot?
7        deinit { print("Ship freed") }
8    }
9    var pilot: Pilot? = Pilot()
10   var ship: Ship? = Ship()
11   print("Created pilot and ship")
12   pilot?.ship = ship
13   print("Pilot holds the ship")
14   ship?.pilot = pilot
15   print("Cycle closed")
```

OUTPUT SO FAR

```
Created pilot and ship
Pilot holds the ship
Cycle closed
```

Frame 6 of 6

The same release now frees both objects: Pilot freed prints when its variable goes nil, Ship freed follows, and only then does the after release marker print.

```
    final class Pilot {
        var ship: Ship?
        deinit { print("Pilot freed") }
    }
    final class Ship {
        weak var pilot: Pilot?
        deinit { print("Ship freed") }
    }
    var pilot: Pilot? = Pilot()
    var ship: Ship? = Ship()
    print("Created pilot and ship")
    pilot?.ship = ship
    print("Pilot holds the ship")
    ship?.pilot = pilot
    print("Cycle closed")
+ pilot = nil
+ ship = nil
+ print("after release")
```

OUTPUT SO FAR

```
Created pilot and ship
Pilot holds the ship
Cycle closed
Pilot freed
Ship freed
after release
```

FINAL RESULT — CHECKED OUTPUT

```
Created pilot and ship
Pilot holds the ship
Cycle closed
Pilot freed
Ship freed
after release
```

Line by line: the finished program

1 `final class Pilot {`

WHO	You declare it; ARC manages every instance.
WHAT	**Declares the Pilot class, one half of the pair in this example.**
WHERE	At file scope, above the Ship class it refers to.
WHEN	Compiled once; an instance is created near the bottom.
WHY	Classes are reference types, so ARC counts the references to each instance.

2 `var ship: Ship?`

WHO	Each Pilot instance, holding its ship strongly.
WHAT	**Stores an optional strong reference from the pilot to its ship.**
WHERE	A stored property inside Pilot, on the heap with the instance.
WHEN	Set after both objects exist, when the link is made.
WHY	A plain var property is strong: it keeps the ship alive.

```
3  deinit { print("Pilot freed") }
```

WHO ARC, at the moment the last strong reference goes.
WHAT **Prints a line when a Pilot instance is destroyed.**
WHERE Inside Pilot; a class can have at most one deinit.
WHEN Exactly when the pilot strong reference count reaches zero.
WHY A print in deinit is the leak detector used in the debugging lesson.

```
5  final class Ship {
```

WHO You declare it; ARC manages every instance.
WHAT **Declares the Ship class, the other half of the pair.**
WHERE At file scope, below the Pilot class it refers to.
WHEN Compiled once; an instance is created near the bottom.
WHY Two classes that can point at each other are all a cycle needs.

```
6  weak var pilot: Pilot?
```

WHO Each Ship instance, referring without owning.
WHAT **Stores an optional weak reference from the ship back to its pilot.**
WHERE A stored property inside Ship, replacing the strong version.
WHEN Set after both objects exist, when the link is made.
WHY weak does not add to the count, so this arrow cannot keep the pilot alive.

```
7  deinit { print("Ship freed") }
```

WHO ARC, at the moment the last strong reference goes.
WHAT **Prints a line when a Ship instance is destroyed.**
WHERE Inside Ship; a class can have at most one deinit.
WHEN Exactly when the ship strong reference count reaches zero.
WHY Watching for this line, or its absence, is the whole experiment.

```
9  var pilot: Pilot? = Pilot()
```

WHO Top-level code, playing the screen that owns the pilot.
WHAT **Creates a Pilot and holds it in an optional variable.**
WHERE Top-level code below both class declarations.
WHEN When the program reaches this line, before any linking.
WHY Optional variables can later be set to nil to release the object.

```
10 var ship: Ship? = Ship()
```

WHO Top-level code, playing the screen that owns the ship.
WHAT **Creates a Ship and holds it in an optional variable.**
WHERE Top-level code, right after the pilot variable.
WHEN When the program reaches this line, before any linking.
WHY Each object starts with exactly one strong reference: its variable.

```
11 print("Created pilot and ship")
```

WHO The print function from the Swift standard library.
WHAT **Prints the marker that both objects now exist.**
WHERE Top-level code, after both variables are created.
WHEN After creation, before any links are made.
WHY A marker line makes the output order easy to follow.

```
12 pilot?.ship = ship
```

WHO Top-level code, wiring the first direction.
WHAT **Stores the ship in the pilot property, creating the first strong arrow.**
WHERE Top-level code, after both objects exist.
WHEN After creation, as the first link between the pair.
WHY One direction of reference is normal ownership and leaks nothing.

```
13 print("Pilot holds the ship")
```

WHO The print function from the Swift standard library.
WHAT **Prints the marker that the first link is in place.**
WHERE Top-level code, right after the first link line.
WHEN After the first link, before the second direction exists.
WHY The marker separates the one-way stage from the cycle stage.

```
14 ship?.pilot = pilot
```

WHO Top-level code, wiring the second direction.
WHAT **Stores the pilot in the ship property, closing the reference loop.**
WHERE Top-level code, after the first link is in place.
WHEN After the first link, as the moment the cycle forms.
WHY Now each object holds the other strongly: the cycle is complete.

```
15 print("Cycle closed")
```

WHO The print function from the Swift standard library.
WHAT **Prints the marker that both directions of reference now exist.**
WHERE Top-level code, right after the second link line.
WHEN After the second link, before any release is tried.
WHY From here, releasing the variables is the test that matters.

```
16 pilot = nil
```

WHO Top-level code, releasing its pilot reference.
WHAT **Drops the outside strong reference to the pilot object.**
WHERE Top-level code, in the release section of the program.
WHEN After the links are made, as the first release step.
WHY Whether anything is freed now depends entirely on the links.

```
17 ship = nil
```

WHO Top-level code, releasing its ship reference.
WHAT **Drops the outside strong reference to the ship object.**
WHERE Top-level code, right after the pilot release.
WHEN After the pilot release, as the second release step.
WHY After this line no outside reference to either object remains.

```
18 print("after release")
```

WHO The print function from the Swift standard library.
WHAT **Prints the marker that both releases have happened.**
WHERE The last line of the program, after both nil assignments.
WHEN Last, after every release step has run.
WHY Any freed lines before this marker prove the release worked.

Try it

A Driver and a Car hold each other strongly and leak the same way. Change the starter so releasing both variables frees both objects, keeping the driver to car link strong.

```
final class Driver {
    var car: Car?
    deinit { print("Driver freed") }
}
final class Car {
    var driver: Driver?
    deinit { print("Car freed") }
}
var driver: Driver? = Driver()
var car: Car? = Car()
driver?.car = car
car?.driver = driver
driver = nil
car = nil
print("after release")
```

EXPECTED OUTPUT

```
Driver freed
Car freed
after release
```

WORKED EXAMPLE

Find the work that busts a 16.7 ms frame

Problem.

Model one frame of a scrolling list in tenths of a millisecond: input, update, layout and render for twenty items. Find the segment that pushes the total over the 16.7 ms budget, then cache it and fit.

Frame 1 of 6

The model starts with the item count, a tenths formatter, and the input segment alone: twenty items at one tenth each. The first total is two milliseconds, well inside budget.

```
  let items = 20
  func ms(_ tenths: Int) -> String {
      "\(tenths / 10).\(tenths % 10)"
  }
  let inputTenths = items * 1
  var totalTenths = inputTenths
  print("Input: \(ms(inputTenths)) ms")
  print("Total: \(ms(totalTenths)) ms")
```

Frame 2 of 6

Update joins at two tenths per item, four milliseconds. The running total is six milliseconds; the frame still fits with plenty of the budget left.

```
  let items = 20
  func ms(_ tenths: Int) -> String {
      "\(tenths / 10).\(tenths % 10)"
  }
  let inputTenths = items * 1
+ let updateTenths = items * 2
+ var totalTenths = inputTenths + updateTenths
+ print("Input: \(ms(inputTenths)) ms")
+ print("Update: \(ms(updateTenths)) ms")
+ print("Total: \(ms(totalTenths)) ms")
```

Frame 3 of 6

Naive layout arrives and it redoes work per item: items times items in the formula. Layout alone is twelve milliseconds and the total is already over budget at eighteen.

```
  let items = 20
  func ms(_ tenths: Int) -> String {
      "\(tenths / 10).\(tenths % 10)"
  }
  let inputTenths = items * 1
  let updateTenths = items * 2
+ let layoutTenths = items * items * 3 / 10
+ var totalTenths = inputTenths + updateTenths
+ totalTenths += layoutTenths
+ print("Input: \(ms(inputTenths)) ms")
+ print("Update: \(ms(updateTenths)) ms")
+ print("Layout: \(ms(layoutTenths)) ms")
+ print("Total: \(ms(totalTenths)) ms")
```

Frame 4 of 6

Render adds three more milliseconds, for a total of twenty-one. The diagram and the printed total agree: this frame drops, and layout is most of the cost.

```
  let items = 20
  func ms(_ tenths: Int) -> String {
      "\(tenths / 10).\(tenths % 10)"
  }
  let inputTenths = items * 1
  let updateTenths = items * 2
  let layoutTenths = items * items * 3 / 10
+ let renderTenths = items + 10
+ var totalTenths = inputTenths + updateTenths
+ totalTenths += layoutTenths
+ totalTenths += renderTenths
+ print("Input: \(ms(inputTenths)) ms")
+ print("Update: \(ms(updateTenths)) ms")
+ print("Layout: \(ms(layoutTenths)) ms")
+ print("Render: \(ms(renderTenths)) ms")
+ print("Total: \(ms(totalTenths)) ms")
```

Frame 5 of 6

The repair caches layout: the result is computed once, stored, and reused, so layout costs three milliseconds instead of twelve. The total drops to twelve and the frame fits.

```
  let items = 20
  func ms(_ tenths: Int) -> String {
      "\(tenths / 10).\(tenths % 10)"
  }
  let inputTenths = items * 1
  let updateTenths = items * 2
+ let cachedLayoutTenths = 30
+ let layoutTenths = cachedLayoutTenths
+ let renderTenths = items + 10
+ var totalTenths = inputTenths + updateTenths
+ totalTenths += layoutTenths
+ totalTenths += renderTenths
+ print("Input: \(ms(inputTenths)) ms")
+ print("Update: \(ms(updateTenths)) ms")
+ print("Layout: \(ms(layoutTenths)) ms")
+ print("Render: \(ms(renderTenths)) ms")
+ print("Total: \(ms(totalTenths)) ms")
```

Frame 6 of 6

A verdict line compares the total in tenths against one hundred sixty-seven, the budget in tenths. Twelve milliseconds is on time, and the diagram shows the same four segments.

```
  let items = 20
  func ms(_ tenths: Int) -> String {
      "\(tenths / 10).\(tenths % 10)"
  }
  let inputTenths = items * 1
  let updateTenths = items * 2
  let cachedLayoutTenths = 30
  let layoutTenths = cachedLayoutTenths
  let renderTenths = items + 10
  var totalTenths = inputTenths + updateTenths
  totalTenths += layoutTenths
  totalTenths += renderTenths
  print("Input: \(ms(inputTenths)) ms")
  print("Update: \(ms(updateTenths)) ms")
  print("Layout: \(ms(layoutTenths)) ms")
  print("Render: \(ms(renderTenths)) ms")
  print("Total: \(ms(totalTenths)) ms")
+ if totalTenths <= 167 { print("Verdict: on time") }
```

Line by line: the finished program

```
1  let items = 20
```

WHO Your code, fixing the workload size.
WHAT **Stores the number of list items one frame must handle: twenty.**
WHERE The first line of the program, above every cost line.
WHEN First, before any segment cost is computed.
WHY Every segment cost below is computed from this one count.

```
2  func ms(_ tenths: Int) -> String {
```

WHO Your code, formatting durations for display.
WHAT **Declares a helper that turns tenths of a millisecond into text.**
WHERE At file scope, above the cost lines that use it.
WHEN Compiled once; called by every print below.
WHY Working in integer tenths keeps every printed total exact.

```
3  "\(tenths / 10).\(tenths % 10)"
```

WHO The helper function, building its return text.
WHAT **Returns whole milliseconds, a dot, then the tenths digit.**
WHERE The only statement inside the ms helper function.
WHEN Each time a print formats a segment or a total.
WHY Dividing and taking the remainder formats tenths with no fractions.

```
5  let inputTenths = items * 1
```

WHO Your model of one frame, costing the input stage.
WHAT **Computes input cost: one tenth of a millisecond per item, twenty total.**
WHERE With the segment cost lines, after the item count.
WHEN When the frame model is built, before totals are summed.
WHY Input is cheap per item, which the printed two milliseconds shows.

```
6  let updateTenths = items * 2
```

WHO Your model of one frame, costing the update stage.
WHAT **Computes update cost: two tenths per item, forty total.**
WHERE With the segment cost lines, below the input cost.
WHEN When the frame model is built, before totals are summed.
WHY Update costs more than input because every item changes state.

```
7  let cachedLayoutTenths = 30
```

WHO Your code, keeping the layout result for reuse.
WHAT **Stores the layout cost computed once: thirty tenths, three milliseconds.**
WHERE With the segment cost lines, above the layout cost line.
WHEN Computed once, before the frame costs are summed.
WHY Caching means paying the layout cost a single time, not per item.

```
8  let layoutTenths = cachedLayoutTenths
```

WHO Your model of one frame, costing layout the cached way.
WHAT **Takes the layout cost from the cached value instead of recomputing it.**
WHERE With the segment cost lines, right after the cached value.
WHEN When the frame model is built, before totals are summed.
WHY Reuse is the repair: same layout result, a quarter of the cost.

```
9  let renderTenths = items + 10
```

WHO Your model of one frame, costing the render stage.
WHAT **Computes render cost: the item count plus ten, thirty tenths total.**
WHERE With the segment cost lines, below the layout cost.
WHEN When the frame model is built, before totals are summed.
WHY Render is a fixed cost plus a small per item cost here.

```
10 var totalTenths = inputTenths + updateTenths
```

WHO Your code, starting the running frame total.
WHAT **Starts the total with the input and update costs added.**
WHERE After the input and update lines, before the prints.
WHEN After both costs exist, before layout joins the model.
WHY Starting from the segments present keeps each frame total honest.

```
11 totalTenths += layoutTenths
```

WHO Your code, extending the running frame total.
WHAT **Adds the layout cost into the total for this frame.**
WHERE After the total starts, before the print lines.
WHEN After the layout cost exists, in segment order.
WHY Adding segments one by one mirrors how a frame spends its budget.

```
12 totalTenths += renderTenths
```

WHO Your code, extending the running frame total.
WHAT **Adds the render cost into the total for this frame.**
WHERE After the layout addition, before the print lines.
WHEN After the render cost exists, as the final segment.
WHY Render is the last segment, so this completes the frames cost.

```
13 print("Input: \(ms(inputTenths)) ms")
```

WHO The print function, formatting through the ms helper.
WHAT **Prints the input segment cost with one decimal place.**
WHERE In the output section, after the totals are summed.
WHEN After the model is built, as the first printed segment.
WHY Each segment prints so the diagram can be checked against code.

```
14 print("Update: \(ms(updateTenths)) ms")
```

WHO The print function, formatting through the ms helper.
WHAT **Prints the update segment cost with one decimal place.**
WHERE In the output section, after the input line.
WHEN After the model is built, as the second printed segment.
WHY Update at four milliseconds is the second largest honest cost.

```
15 print("Layout: \(ms(layoutTenths)) ms")
```

WHO The print function, formatting through the ms helper.
WHAT **Prints the layout segment cost with one decimal place.**
WHERE In the output section, after the update line.
WHEN After the model is built, as the third printed segment.
WHY This is the line that changes when caching replaces recomputation.

```
16 print("Render: \(ms(renderTenths)) ms")
```

WHO The print function, formatting through the ms helper.
WHAT **Prints the render segment cost with one decimal place.**
WHERE In the output section, after the layout line.
WHEN After the model is built, as the last printed segment.
WHY Render stays the same in both versions; layout was the problem.

```
17 print("Total: \(ms(totalTenths)) ms")
```

WHO The print function, formatting through the ms helper.
WHAT **Prints the total frame cost with one decimal place.**
WHERE In the output section, after every segment line.
WHEN After the totals are summed, as the verdict input.
WHY Sixteen point seven is the budget this total is judged against.

```
18 if totalTenths <= 167 { print("Verdict: on time") }
```

WHO Your code, judging the total against the budget.
WHAT **Prints the on-time verdict when the total is at most one hundred sixty-seven tenths.**
WHERE The last line of the program, after the total print.
WHEN Last, once the final total is known.
WHY Comparing in tenths keeps the budget check exact, with no rounding.

Try it

The starter costs layout as items times items for ten items and totals 15.0 ms. Change layout to a single cached cost of thirty tenths and print the new total.

```
let items = 10
let inputTenths = 20
let layoutTenths = items * items
let renderTenths = 30
let totalTenths = inputTenths + layoutTenths + renderTenths
print("Total: \(totalTenths / 10).\(totalTenths % 10) ms")
```

EXPECTED OUTPUT

```
Total: 8.0 ms
```

WORKED EXAMPLE

Walk a release through its gates, including the one that fails

Problem.

Model a release as a checklist: build, signing, archive, validation and distribution gates, each printed from a simple fact. Validation fails on a missing privacy entry; add the entry and walk the release through.

The visual for this example in the HTML version is labelled Simulation: it models Apple frameworks or platform processes that cannot run in a web page.

Frame 1 of 6

The checklist helper formats any gate line, and the first fact says the build compiled. The Build gate prints PASS and the pipeline stands at its first stage.

```
    func gate(_ name: String, _ passed: Bool) -> String {
        if passed { return "\(name): PASS" }
        return "\(name): FAIL"
    }
    let buildOK = true
    print(gate("Build", buildOK))
```

Frame 2 of 6

The signing fact joins the checklist and its gate prints PASS. A signed build is the only kind that can be archived for distribution later.

```
    func gate(_ name: String, _ passed: Bool) -> String {
        if passed { return "\(name): PASS" }
        return "\(name): FAIL"
    }
    let buildOK = true
+   let signed = true
+   print(gate("Build", buildOK))
+   print(gate("Signing", signed))
```

Frame 3 of 6

The archive fact joins and the Archive gate prints PASS. Three stages are cleared; the archive is the exact build validation will now inspect.

```
    func gate(_ name: String, _ passed: Bool) -> String {
        if passed { return "\(name): PASS" }
        return "\(name): FAIL"
    }
    let buildOK = true
    let signed = true
+   let archived = true
+   print(gate("Build", buildOK))
+   print(gate("Signing", signed))
+   print(gate("Archive", archived))
```

Frame 4 of 6

Validation runs against a missing privacy entry and prints FAIL, with the reason and a blocked distribution line. The diagram marks the Validation stage as failed.

```
  func gate(_ name: String, _ passed: Bool) -> String {
      if passed { return "\(name): PASS" }
      return "\(name): FAIL"
  }
  let buildOK = true
  let signed = true
  let archived = true
+ var hasPrivacy = false
+ print(gate("Build", buildOK))
+ print(gate("Signing", signed))
+ print(gate("Archive", archived))
+ print(gate("Validation", hasPrivacy))
+ print("Validation failed: missing privacy entry")
+ print("Distribution: BLOCKED")
```

Frame 5 of 6

The fix is the missing entry itself: the privacy fact becomes true and nothing else changes. Validation now prints PASS at the same stage that failed before.

```
  func gate(_ name: String, _ passed: Bool) -> String {
      if passed { return "\(name): PASS" }
      return "\(name): FAIL"
  }
  let buildOK = true
  let signed = true
  let archived = true
+ var hasPrivacy = true
  print(gate("Build", buildOK))
  print(gate("Signing", signed))
  print(gate("Archive", archived))
  print(gate("Validation", hasPrivacy))
```

Frame 6 of 6

With every earlier gate green, the distribution line prints READY and the pipeline reaches its last stage. The release walked through all five gates in order.

```
  func gate(_ name: String, _ passed: Bool) -> String {
      if passed { return "\(name): PASS" }
      return "\(name): FAIL"
  }
  let buildOK = true
  let signed = true
  let archived = true
  var hasPrivacy = true
  print(gate("Build", buildOK))
  print(gate("Signing", signed))
  print(gate("Archive", archived))
  print(gate("Validation", hasPrivacy))
+ if hasPrivacy { print("Distribution: READY") }
```

Line by line: the finished program

```
1  func gate(_ name: String, _ passed: Bool) -> String {
```

WHO Your code, formatting one release gate result.
WHAT **Declares a helper that turns a gate name and its result into a report line.**
WHERE At file scope, above the release facts it reports on.
WHEN Compiled once; called once per gate below.
WHY One formatter means every gate line has the same shape.

```
2  if passed { return "\(name): PASS" }
```

WHO The gate helper, handling a passed gate.
WHAT **Returns the PASS report for the named gate when its fact is true.**
WHERE The first statement inside the gate helper function.
WHEN Each time the helper is called with a true result.
WHY PASS lines are how the checklist shows a gate was cleared.

```
3  return "\(name): FAIL"
```

WHO The gate helper, handling a failed gate.
WHAT **Returns the FAIL report for the named gate when its fact is false.**
WHERE The last statement inside the gate helper function.
WHEN Each time the helper is called with a false result.
WHY A FAIL line must be just as visible as a PASS line.

```
5  let buildOK = true
```

WHO Your checklist model, recording the build fact.
WHAT **Stores that the release build compiled successfully.**
WHERE With the release facts, after the gate helper.
WHEN Set before the gate lines are printed below.
WHY Each gate in this model is one fact the release depends on.

```
6  let signed = true
```

WHO Your checklist model, recording the signing fact.
WHAT **Stores that the archive was signed with the distribution certificate.**
WHERE With the release facts, below the build fact.
WHEN Set before the gate lines are printed below.
WHY Unsigned builds cannot be archived for distribution at all.

```
7  let archived = true
```

WHO Your checklist model, recording the archive fact.
WHAT **Stores that the archive was created from the release build.**
WHERE With the release facts, below the signing fact.
WHEN Set before the gate lines are printed below.
WHY Archiving is the stage the release lesson builds toward.

```
8  var hasPrivacy = true
```

WHO Your checklist model, recording the added entry.
WHAT **Stores that the required privacy entry is now present: true.**
WHERE With the release facts, replacing the false version.
WHEN Set before validation is reported below.
WHY Adding the entry is the whole fix; no other fact changes.

9 `print(gate("Build", buildOK))`

- **WHO** The print function, reporting through the gate helper.
- **WHAT** **Prints the Build gate line using the build fact.**
- **WHERE** In the output section, after the facts written so far.
- **WHEN** As the first gate line, in release order.
- **WHY** Gates are reported in the order the release pipeline runs them.

10 `print(gate("Signing", signed))`

- **WHO** The print function, reporting through the gate helper.
- **WHAT** **Prints the Signing gate line using the signing fact.**
- **WHERE** In the output section, after the build line.
- **WHEN** As the second gate line, in release order.
- **WHY** Signing follows building in the pipeline and in this output.

11 `print(gate("Archive", archived))`

- **WHO** The print function, reporting through the gate helper.
- **WHAT** **Prints the Archive gate line using the archive fact.**
- **WHERE** In the output section, after the signing line.
- **WHEN** As the third gate line, in release order.
- **WHY** An archive exists only when the first two gates passed.

12 `print(gate("Validation", hasPrivacy))`

- **WHO** The print function, reporting through the gate helper.
- **WHAT** **Prints the Validation gate line using the privacy fact.**
- **WHERE** In the output section, after the archive line.
- **WHEN** As the fourth gate line, in release order.
- **WHY** Validation is the gate this example watches fail, then pass.

13 `if hasPrivacy { print("Distribution: READY") }`

- **WHO** Your code, opening the final stage after validation.
- **WHAT** **Prints that distribution is ready, but only when validation passed.**
- **WHERE** The last line of the program, after the validation print.
- **WHEN** Last, after every earlier gate has been reported.
- **WHY** Distribution is the reward at the end of a clean pipeline.

Try it

The starter release is signed but has no app icon, so its verdict is blocked. Supply the missing icon fact and make the same checklist print the ready verdict.

```
let signed = true
let hasIcon = false
let ready = signed && hasIcon
print(ready ? "Release: READY" : "Release: BLOCKED")
```

EXPECTED OUTPUT

```
Release: READY
```